PHP — Funções essenciais

Manual oficial do PHP (php.net) — English

Fonte oficial: https://www.php.net/manual/
Gerado para o projeto doc-frog

Strings

Introduction

These functions all manipulate strings in various ways. Some more specialized sections can be found in the regular expression and URL handling sections.

For information on how strings behave, especially with regard to usage of single quotes, double quotes, and escape sequences, see the Strings entry in the Types section of the manual.

Installing/Configuring

Table of Contents

Installation

There is no installation needed to use these functions; they are part of the PHP core.

As of PHP 8.1.0, crypt() can be built against a system crypt library by specifying the **--with-external-libcrypt** configure option.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

CRYPT_SALT_LENGTH (int)

CRYPT_STD_DES (int)

Indicates whether standard DES-based hashes are supported in crypt(). Always 1.

CRYPT_EXT_DES (int)

Indicates whether extended DES-based hashes are supported in crypt(). Always 1.

CRYPT_MD5 (int)

Indicates whether MD5 hashes are supported in crypt(). Always 1.

CRYPT_BLOWFISH (int)

Indicates whether Blowfish hashes are supported in crypt(). Always 1.

CRYPT_SHA256 (int)

Indicates whether SHA-256 hashes are supported in crypt(). Always 1.

CRYPT_SHA512 (int)

Indicates whether SHA-512 hashes are supported in crypt(). Always 1.

HTML_SPECIALCHARS (int)

HTML_ENTITIES (int)

ENT_COMPAT (int)

ENT_QUOTES (int)

ENT_NOQUOTES (int)

ENT_IGNORE (int)

ENT_SUBSTITUTE (int)

ENT_DISALLOWED (int)

`ENT_HTML401` (int)

`ENT_XML1` (int)

`ENT_XHTML` (int)

`ENT_HTML5` (int)

`CHAR_MAX` (int)

`LC_CTYPE` (int)

Character classification and conversion affected by locale setting.

`LC_NUMERIC` (int)

Decimal separator affected by locale setting.

`LC_TIME` (int)

Date and time formatting affected by locale setting.

`LC_COLLATE` (int)

String comparison affected by locale setting.

`LC_MONETARY` (int)

Monetary formatting affected by locale setting.

`LC_ALL` (int)

Affects all functions that any of the other `LC_*` constants affect.

`LC_MESSAGES` (int)

System responses affected by locale setting. Available if PHP was compiled with `libintl`.

`STR_PAD_LEFT` (int)

`STR_PAD_RIGHT` (int)

`STR_PAD_BOTH` (int)

nl_langinfo() `LC_TIME` Category Constants

Constants	Description
`ABDAY_1` (int)	Abbreviated name of first day of the week.
`ABDAY_2` (int)	Abbreviated name of second day of the week.
`ABDAY_3` (int)	Abbreviated name of third day of the week.
`ABDAY_4` (int)	Abbreviated name of fourth day of the week.
`ABDAY_5` (int)	Abbreviated name of fifth day of the week.
`ABDAY_6` (int)	Abbreviated name of sixth day of the week.
`ABDAY_7` (int)	Abbreviated name of seventh day of the week.
`DAY_1` (int)	Name of the first day of the week.

DAY_2 (int)	Name of the second day of the week.
DAY_3 (int)	Name of the third day of the week.
DAY_4 (int)	Name of the fourth day of the week.
DAY_5 (int)	Name of the fifth day of the week.
DAY_6 (int)	Name of the sixth day of the week.
DAY_7 (int)	Name of the seventh day of the week.
ABMON_1 (int)	Abbreviated name of the first month of the year.
ABMON_2 (int)	Abbreviated name of the second month of the year.
ABMON_3 (int)	Abbreviated name of the third month of the year.
ABMON_4 (int)	Abbreviated name of the fourth month of the year.
ABMON_5 (int)	Abbreviated name of the fifth month of the year.
ABMON_6 (int)	Abbreviated name of the sixth month of the year.
ABMON_7 (int)	Abbreviated name of the seventh month of the year.
ABMON_8 (int)	Abbreviated name of the eighth month of the year.
ABMON_9 (int)	Abbreviated name of the ninth month of the year.
ABMON_10 (int)	Abbreviated name of the tenth month of the year.
ABMON_11 (int)	Abbreviated name of the eleventh month of the year.
ABMON_12 (int)	Abbreviated name of the twelfth month of the year.
MON_1 (int)	Name of the first month of the year.
MON_2 (int)	Name of the second month of the year.
MON_3 (int)	Name of the third month of the year.
MON_4 (int)	Name of the fourth month of the year.

`MON_5` (int)	Name of the fifth month of the year.
`MON_6` (int)	Name of the sixth month of the year.
`MON_7` (int)	Name of the seventh month of the year.
`MON_8` (int)	Name of the eighth month of the year.
`MON_9` (int)	Name of the ninth month of the year.
`MON_10` (int)	Name of the tenth month of the year.
`MON_11` (int)	Name of the eleventh month of the year.
`MON_12` (int)	Name of the twelfth month of the year.
`AM_STR` (int)	String for Ante meridian.
`PM_STR` (int)	String for Post meridian.
`D_T_FMT` (int)	String that can be used as the format string for strftime() to represent time and date.
`D_FMT` (int)	String that can be used as the format string for strftime() to represent date.
`T_FMT` (int)	String that can be used as the format string for strftime() to represent time.
`T_FMT_AMPM` (int)	String that can be used as the format string for strftime() to represent time in 12-hour format with ante/post meridian.
`ERA` (int)	Alternate era.
`ERA_YEAR` (int)	Year in alternate era format.
`ERA_D_T_FMT` (int)	Date and time in alternate era format (string can be used in strftime()).
`ERA_D_FMT` (int)	Date in alternate era format (string can be used in strftime()).
`ERA_T_FMT` (int)	Time in alternate era format (string can be used in strftime()).
`ALT_DIGITS` (int)	Alternative symbols for digits.

nl_langinfo() `LC_MONETARY` Category Constants

Constants	Description
`INT_CURR_SYMBOL` (int)	International currency symbol.

CURRENCY_SYMBOL (int)	Local currency symbol.
CRNCYSTR (int)	Same value as CURRENCY_SYMBOL.
MON_DECIMAL_POINT (int)	Decimal point character.
MON_THOUSANDS_SEP (int)	Thousands separator (groups of three digits).
MON_GROUPING (int)	Like "grouping" element.
POSITIVE_SIGN (int)	Sign for positive values.
NEGATIVE_SIGN (int)	Sign for negative values.
INT_FRAC_DIGITS (int)	International fractional digits.
FRAC_DIGITS (int)	Local fractional digits.
P_CS_PRECEDES (int)	Returns 1 if CURRENCY_SYMBOL precedes a positive value.
P_SEP_BY_SPACE (int)	Returns 1 if a space separates CURRENCY_SYMBOL from a positive value.
N_CS_PRECEDES (int)	Returns 1 if CURRENCY_SYMBOL precedes a negative value.
N_SEP_BY_SPACE (int)	Returns 1 if a space separates CURRENCY_SYMBOL from a negative value.
P_SIGN_POSN (int)	• Returns 0 if parentheses surround the quantity and CURRENCY_SYMBOL. • Returns 1 if the sign string precedes the quantity and CURRENCY_SYMBOL. • Returns 2 if the sign string follows the quantity and CURRENCY_SYMBOL. • Returns 3 if the sign string immediately precedes the CURRENCY_SYMBOL. • Returns 4 if the sign string immediately follows the CURRENCY_SYMBOL.
N_SIGN_POSN (int)	

nl_langinfo() LC_NUMERIC Category Constants

Constants	Description
DECIMAL_POINT (int)	Decimal point character.
RADIXCHAR (int)	Same value as DECIMAL_POINT.
THOUSANDS_SEP (int)	Separator character for thousands (groups of three digits).
THOUSEP (int)	Same value as THOUSANDS_SEP.
GROUPING (int)	

nl_langinfo() LC_MESSAGES Category Constants

Constants	Description
YESEXPR (int)	Regex string for matching "yes" input.
NOEXPR (int)	Regex string for matching "no" input.
YESSTR (int)	Output string for "yes".
NOSTR (int)	Output string for "no".

nl_langinfo() LC_CTYPE Category Constants

Constants	Description
CODESET (int)	Return a string with the name of the character encoding.

String Functions

See Also

For even more powerful string handling and manipulating functions take a look at the Perl compatible regular expression functions. For working with multibyte character encodings, take a look at the Multibyte String functions.

Table of Contents

addcslashes

addcslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

addcslashes — Quote string with slashes in a C style

Description

function **addcslashes**(string `$string`, string `$characters`): string

Returns a string with backslashes before characters that are listed in `characters` parameter.

Parameters

`string`

The string to be escaped.

`characters`

A list of characters to be escaped. If `characters` contains characters `\n`, `\r` etc., they are converted in C-like style, while other non-alphanumeric characters with ASCII codes lower than 32 and higher than 126 converted to octal representation.

When you define a sequence of characters in the `characters` argument make sure that you know what characters come between the characters that you set as the start and end of the range.

Example #1 addcslashes() with Ranges

```
<?php
echo addcslashes('foo[ ]', 'A..z');
// output:  \f\o\o\[ \]
// All upper and lower-case letters will be escaped
// ... but so will the [\]^_`
?>
```

Also, if the first character in a range has a higher ASCII value than the second character in the range, no range will be constructed. Only the start, end and period characters will be escaped. Use the ord() function to find the ASCII value for a character.

Example #2 addcslashes() with Characters in Wrong Order

```
<?php
echo addcslashes("zoo['.']", 'z..A');
// output:  \zoo['\.']
?>
```

Be careful if you choose to escape characters 0, a, b, f, n, r, t and v. They will be converted to \0, \a, \b, \f, \n, \r, \t and \v, all of which are predefined escape sequences in C. Many of these sequences are also defined in other C-derived languages, including PHP, meaning that you may not get the desired result if you use the output of **addcslashes()** to generate code in those languages with these characters defined in `characters`.

Return Values

Returns the escaped string.

Examples

`characters` like "\0..\37", which would escape all characters with ASCII code between 0 and 31.

Example #3 addcslashes() example

```
<?php
$not_escaped = "PHP isThirty\nYears Old!\tYay to the Elephant!\n";
$escaped = addcslashes($not_escaped, "\0..\37!@\177..\377");
echo $escaped;
?>
```

See Also

- stripcslashes() - Un-quote string quoted with addcslashes
- stripslashes() - Un-quotes a quoted string
- addslashes() - Quote string with slashes
- htmlspecialchars() - Convert special characters to HTML entities
- quotemeta() - Quote meta characters

addslashes

addslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

addslashes — Quote string with slashes

Description

function **addslashes**(string `$string`): string

Returns a string with backslashes added before characters that need to be escaped. These characters are:

- single quote (')
- double quote (")
- backslash (\)
- NUL (the NUL byte)

A use case of **addslashes()** is escaping the aforementioned characters in a string that is to be evaluated by PHP:

Example #1 Escaping Characters

```
<?php
$str = "O'Reilly?";
eval("echo '" . addslashes($str) . "';");
?>
```

The **addslashes()** is sometimes incorrectly used to try to prevent SQL Injection. Instead, database-specific escaping functions and/or prepared statements should be used.

Parameters

`string`

The string to be escaped.

Return Values

Returns the escaped string.

Examples

Example #2 An addslashes() example

```
<?php
$str = "Is your name O'Reilly?";

// Outputs: Is your name O\'Reilly?
echo addslashes($str);
?>
```

See Also

- stripcslashes() - Un-quote string quoted with addcslashes
- stripslashes() - Un-quotes a quoted string
- addcslashes() - Quote string with slashes in a C style
- htmlspecialchars() - Convert special characters to HTML entities
- quotemeta() - Quote meta characters
- get_magic_quotes_gpc() - Gets the current configuration setting of magic_quotes_gpc

bin2hex

bin2hex

(PHP 4, PHP 5, PHP 7, PHP 8)

bin2hex — Convert binary data into hexadecimal representation

Description

function **bin2hex**(string `$string`): string

Returns an ASCII string containing the hexadecimal representation of `string`. The conversion is done byte-wise with the high-nibble first.

Parameters

`string`

A string.

Return Values

Returns the hexadecimal representation of the given string.

Examples

Example #1 bin2hex() example

```
<?php

$hex = bin2hex('Hello world!');

var_dump($hex);
var_dump(hex2bin($hex));
?>
```

The above example will output:

```
string(24) "48656c6c6f20776f726c6421"
string(12) "Hello world!"
```

See Also

- hex2bin() - Decodes a hexadecimally encoded binary string
- pack() - Pack data into binary string

chop

chop

(PHP 4, PHP 5, PHP 7, PHP 8)

chop — Alias of rtrim()

Description

This function is an alias of: rtrim().

Notes

> **Note** :
>
> **chop()** is different than the Perl `chop()` function, which removes the last character in the string.

chr

chr

(PHP 4, PHP 5, PHP 7, PHP 8)

chr — Generate a single-byte string from a number

Description

function **chr**(int `$codepoint`): string

Returns a one-character string containing the character specified by interpreting `codepoint` as an unsigned integer.

This can be used to create a one-character string in a single-byte encoding such as ASCII, ISO-8859, or Windows 1252, by passing the position of a desired character in the encoding's mapping table. However, note that this function is not aware of any string encoding, and in particular cannot be passed a Unicode code point value to generate a string in a multibyte encoding like UTF-8 or UTF-16.

This function complements ord().

Parameters

`codepoint`

An integer between 0 and 255.

Values outside the valid range (0..255) will be bitwise and'ed with 255, which is equivalent to the following algorithm:

```
while ($bytevalue < 0) {
    $bytevalue += 256;
}
$bytevalue %= 256;
```

Return Values

A single-character string containing the specified byte.

Changelog

Version	Description
8.5.0	Passing integers outside the interval `[0, 255]` is now deprecated.
7.4.0	The function no longer silently accepts unsupported `codepoints`, and casts these to `0`.

Examples

Example #1 chr() example

```
<?php
// Assumes the string will be used as ASCII or an ASCII-compatible encoding

$str = "The string ends in escape: ";
$str .= chr(27); /* add an escape character at the end of $str */
echo $str, PHP_EOL;

/* Often this is more useful */
$str = sprintf("The string ends in escape: %c", 27);
echo $str, PHP_EOL;
?>
```

Example #2 Overflow behavior

```
<?php
echo chr(-159), chr(833), PHP_EOL;
?>
```

The above example will output:

```
aA
```

Example #3 Building a UTF-8 string from individual bytes

```
<?php
$str = chr(240) . chr(159) . chr(144) . chr(152);
echo $str, PHP_EOL;
?>
```

The above example will output:

```
🐘
```

See Also

- sprintf() - Return a formatted string with a format string of %c
- ord() - Convert the first byte of a string to a value between 0 and 255
- An » ASCII-table
- mb_chr() - Return character by Unicode code point value
- IntlChar::chr() - Return Unicode character by code point value

chunk_split

chunk_split

(PHP 4, PHP 5, PHP 7, PHP 8)

chunk_split — Split a string into smaller chunks

Description

function **chunk_split**(string `$string`, int `$length` = 76, string `$separator` = "\r\n"): string

Can be used to split a string into smaller chunks which is useful for e.g. converting base64_encode() output to match RFC 2045 semantics. It inserts `separator` every `length` characters.

Parameters

`string`

The string to be chunked.

`length`

The chunk length.

`separator`

The line ending sequence.

Return Values

Returns the chunked string.

Examples

Example #1 chunk_split() example

```
<?php
$data = 'This is quite a long string, which will get broken up because the line is going to be too long after base64 encoding
it.';

// format $data using RFC 2045 semantics
$new_string = chunk_split(base64_encode($data));
echo $new_string, PHP_EOL;
?>
```

See Also

- str_split() - Convert a string to an array
- explode() - Split a string by a string
- wordwrap() - Wraps a string to a given number of characters
- » RFC 2045

convert_cyr_string

convert_cyr_string

(PHP 4, PHP 5, PHP 7)

convert_cyr_string — Convert from one Cyrillic character set to another

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.4.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **convert_cyr_string**(string `$str`, string `$from`, string `$to`): string

Converts from one Cyrillic character set to another.

Parameters

`str`

The string to be converted.

`from`

The source Cyrillic character set, as a single character.

`to`

The target Cyrillic character set, as a single character.

Supported characters are:

- k - koi8-r
- w - windows-1251
- i - iso8859-5
- a - x-cp866
- d - x-cp866
- m - x-mac-cyrillic

Return Values

Returns the converted string.

Changelog

Version	Description
8.0.0	Removed this function.
7.4.0	Deprecated this function.

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- mb_convert_encoding() - Convert a string from one character encoding to another
- iconv() - Convert a string from one character encoding to another

convert_uudecode

convert_uudecode

(PHP 5, PHP 7, PHP 8)

convert_uudecode — Decode a uuencoded string

Description

function **convert_uudecode**(string `$string`): string|false

convert_uudecode() decodes a uuencoded string.

> **Note** :
>
> **convert_uudecode()** neither accepts the `begin` nor the `end` line, which are part of uuencoded *files*.

Parameters

`string`

The uuencoded data.

Return Values

Returns the decoded data as a string or `false` on failure.

Examples

Example #1 convert_uudecode() example

```php
<?php
echo convert_uudecode("+22!L;W9E(%!(4\"$`\n`");
?>
```

The above example will output:

```
I love PHP!
```

See Also

- convert_uuencode() - Uuencode a string

convert_uuencode

convert_uuencode

(PHP 5, PHP 7, PHP 8)

convert_uuencode — Uuencode a string

Description

function **convert_uuencode**(string `$string`): string

convert_uuencode() encodes a string using the uuencode algorithm.

Uuencode translates all strings (including binary data) into printable characters, making them safe for network transmissions. Uuencoded data is about 35% larger than the original.

> **Note** :
>
> **convert_uuencode()** neither produces the `begin` nor the `end` line, which are part of uuencoded *files*.

Parameters

`string`

The data to be encoded.

Return Values

Returns the uuencoded data.

Changelog

Version	Description
8.0.0	Prior to this version, trying to convert an empty string returned `false` for no particular reason.

Examples

Example #1 convert_uuencode() example

```php
<?php
$some_string = "test\ntext text\r\n";
```

```
echo convert_uuencode($some_string);
?>
```

The above example will output:

```
0=&5S=`IT97AT('1E>'0-"@``
`
```

See Also

- convert_uudecode() - Decode a uuencoded string
- base64_encode() - Encodes data with MIME base64

count_chars

count_chars

(PHP 4, PHP 5, PHP 7, PHP 8)

count_chars — Return information about characters used in a string

Description

function **count_chars**(string `$string`, int `$mode` = 0): array|string

Counts the number of occurrences of every byte-value (0..255) in `string` and returns it in various ways.

Parameters

`string`

The examined string.

`mode`

See return values.

Return Values

Depending on `mode` **count_chars()** returns one of the following:

- 0 - an array with the byte-value as key and the frequency of every byte as value.
- 1 - same as 0 but only byte-values with a frequency greater than zero are listed.
- 2 - same as 0 but only byte-values with a frequency equal to zero are listed.
- 3 - a string containing all unique characters is returned.
- 4 - a string containing all unused characters is returned.

Changelog

Version	Description
8.0.0	Prior to this version, the function returned `false` on failure.

Examples

Example #1 **count_chars()** example

```
<?php
$data = "Two Ts and one F.";

foreach (count_chars($data, 1) as $i => $val) {
```

```
    echo "There were $val instance(s) of \"" , chr($i) , "\" in the string.\n";
}
?>
```

The above example will output:

```
There were 4 instance(s) of " " in the string.
There were 1 instance(s) of "." in the string.
There were 1 instance(s) of "F" in the string.
There were 2 instance(s) of "T" in the string.
There were 1 instance(s) of "a" in the string.
There were 1 instance(s) of "d" in the string.
There were 1 instance(s) of "e" in the string.
There were 2 instance(s) of "n" in the string.
There were 2 instance(s) of "o" in the string.
There were 1 instance(s) of "s" in the string.
There were 1 instance(s) of "w" in the string.
```

See Also

- strpos() - Find the position of the first occurrence of a substring in a string
- substr_count() - Count the number of substring occurrences

crc32

crc32

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

crc32 — Calculates the crc32 polynomial of a string

Description

function **crc32**(string `$string`): int

Generates the cyclic redundancy checksum polynomial of 32-bit lengths of the `string`. This is usually used to validate the integrity of data being transmitted.

> **Warning**
>
> Because PHP's integer type is signed many crc32 checksums will result in negative integers on 32bit platforms. On 64bit installations all **crc32()** results will be positive integers though.
>
> So you need to use the "%u" formatter of sprintf() or printf() to get the string representation of the unsigned **crc32()** checksum in decimal format.
>
> For a hexadecimal representation of the checksum you can either use the "%x" formatter of sprintf() or printf() or the dechex() conversion functions, both of these also take care of converting the **crc32()** result to an unsigned integer.
>
> Having 64bit installations also return negative integers for higher result values was considered but would break the hexadecimal conversion as negatives would get an extra 0xFFFFFFFF######## offset then. As hexadecimal representation seems to be the most common use case we decided to not break this even if it breaks direct decimal comparisons in about 50% of the cases when moving from 32 to 64bits.
>
> In retrospect having the function return an integer maybe wasn't the best idea and returning a hex string representation right away (as e.g. md5() does) might have been a better plan to begin with.
>
> For a more portable solution you may also consider the generic hash(). `hash("crc32b", $str)` will return the same string as `str_pad(dechex(crc32($str)), 8, '0', STR_PAD_LEFT)`.

Parameters

`string`

The data.

Return Values

Returns the crc32 checksum of `string` as an integer.

Examples

Example #1 Displaying a crc32 checksum

This example shows how to print a converted checksum with the printf() function:

```php
<?php
$checksum = crc32("The quick brown fox jumped over the lazy dog.");
printf("%u\n", $checksum);
?>
```

See Also

- hash() - Generate a hash value (message digest)
- md5() - Calculate the md5 hash of a string
- sha1() - Calculate the sha1 hash of a string

crypt

crypt

(PHP 4, PHP 5, PHP 7, PHP 8)

crypt — One-way string hashing

> **Warning**
>
> This function is not (yet) binary safe!

Description

function **crypt**(#[\SensitiveParameter] string `$string`, string `$salt`): string

crypt() will return a hashed string using the standard Unix DES-based algorithm or alternative algorithms. password_verify() is compatible with **crypt()**. Therefore, password hashes created by **crypt()** can be used with password_verify().

Prior to PHP 8.0.0, the `salt` parameter was optional. However, **crypt()** creates a weak hash without the `salt`, and raises an `E_NOTICE` error without it. Make sure to specify a strong enough salt for better security.

password_hash() uses a strong hash, generates a strong salt, and applies proper rounds automatically. password_hash() is a simple **crypt()** wrapper and compatible with existing password hashes. Use of password_hash() is encouraged.

The hash type is triggered by the salt argument. If no salt is provided, PHP will auto-generate either a standard two character (DES) or a twelve character (MD5) salt, depending on the availability of MD5 crypt(). PHP sets a constant named `CRYPT_SALT_LENGTH` which indicates the longest valid salt allowed by the available hashes.

The standard DES-based **crypt()** returns the salt as the first two characters of the output. It also only uses the first eight characters of `string`, so longer strings that start with the same eight characters will generate the same result (when the same salt is used).

> **Warning**
>
> None of the algorithms listed below are suitable for hashing new passwords. `CRYPT_STD_DES`, `CRYPT_EXT_DES` and `CRYPT_MD5` are far too fast to resist offline cracking; md5crypt was declared end of life for that reason (CVE-2012-3287). The runtime of `CRYPT_SHA256` and `CRYPT_SHA512` grows with the square of the length of `string`, so unbounded input can be used to exhaust CPU (CVE-2016-20013). Use password_hash() instead, which selects a suitable algorithm and cost automatically.

The following hash types are supported:

- `CRYPT_STD_DES` - Standard DES-based hash with a two character salt from the alphabet "./0-9A-Za-z". Using invalid characters in the salt will cause crypt() to fail.

- `CRYPT_EXT_DES` - Extended DES-based hash. The "salt" is a 9-character string consisting of an underscore followed by 4 characters of iteration count and 4 characters of salt. Each of these 4-character strings encode 24 bits, least significant character first. The values `0` to `63` are encoded as `./0-9A-Za-z`. Using invalid characters in the salt will cause crypt() to fail.
- `CRYPT_MD5` - MD5 hashing with a twelve character salt starting with 1
- `CRYPT_BLOWFISH` - Blowfish hashing with a salt as follows: "$2a$", "$2x$" or "$2y$", a two digit cost parameter, "$", and 22 characters from the alphabet "./0-9A-Za-z". Using characters outside of this range in the salt will cause crypt() to return a zero-length string. The two digit cost parameter is the base-2 logarithm of the iteration count for the underlying Blowfish-based hashing algorithm and must be in range 04-31, values outside this range will cause crypt() to fail. "$2x$" hashes are potentially weak; "$2a$" hashes are compatible and mitigate this weakness. For new hashes, "$2y$" should be used.
- `CRYPT_SHA256` - SHA-256 hash with a sixteen character salt prefixed with 5. If the salt string starts with 'rounds=<N>$', the numeric value of N is used to indicate how many times the hashing loop should be executed, much like the cost parameter on Blowfish. The default number of rounds is 5000, there is a minimum of 1000 and a maximum of 999,999,999. Any selection of N outside this range will be truncated to the nearest limit.
- `CRYPT_SHA512` - SHA-512 hash with a sixteen character salt prefixed with 6. If the salt string starts with 'rounds=<N>$', the numeric value of N is used to indicate how many times the hashing loop should be executed, much like the cost parameter on Blowfish. The default number of rounds is 5000, there is a minimum of 1000 and a maximum of 999,999,999. Any selection of N outside this range will be truncated to the nearest limit.

Parameters

`string`

The string to be hashed.

> **Caution**
>
> Using the `CRYPT_BLOWFISH` algorithm, will result in the `string` parameter being truncated to a maximum length of 72 bytes.

`salt`

A salt string to base the hashing on. If not provided, the behaviour is defined by the algorithm implementation and can lead to unexpected results.

Return Values

Returns the hashed string or a string that is shorter than 13 characters and is guaranteed to differ from the salt on failure.

> **Warning**
>
> When validating passwords, a string comparison function that isn't vulnerable to timing attacks should be used to compare the output of **crypt()** to the previously known hash. PHP provides hash_equals() for this purpose.

Changelog

Version	Description
8.0.0	The `salt` is no longer optional.

Examples

Example #1 **crypt()** examples

```php
<?php
$user_input = 'rasmuslerdorf';
$hashed_password =
'$6$rounds=1000000$NJy4rIPjpOaU$0ACEYGg/aKCY3v8O8AfyiO7CTfZQ8/W231Qfh2tRLmfdvFD6XfHk12u6hMr9cYIA4hnpjLNSTRtUwYr9km9Ij/';

// Validate an existing crypt() hash in a way that is compatible with non-PHP software.
if (hash_equals($hashed_password, crypt($user_input, $hashed_password))) {
   echo "Password verified!";
}
?>
```

Notes

> **Note** :
>
> There is no decrypt function, since **crypt()** uses a one-way algorithm.

See Also

- hash_equals() - Timing attack safe string comparison
- password_hash() - Creates a password hash
- The Unix man page for your crypt function for more information

echo

echo

(PHP 4, PHP 5, PHP 7, PHP 8)

echo — Output one or more strings

Description

function **echo**(string `...$expressions`): void

Outputs one or more expressions, with no additional newlines or spaces.

`echo` is not a function but a language construct. Its arguments are a list of expressions following the `echo` keyword, separated by commas, and not delimited by parentheses. Unlike some other language constructs, `echo` does not have any return value, so it cannot be used in the context of an expression.

`echo` also has a shortcut syntax, where you can immediately follow the opening tag with an equals sign. This syntax is available even with the short_open_tag configuration setting disabled.

```
I have <?=$foo?> foo.
```

The major differences to print are that `echo` accepts multiple arguments and doesn't have a return value.

Parameters

`expressions`

> One or more string expressions to output, separated by commas. Non-string values will be coerced to strings, even when the `strict_types` directive is enabled.

Return Values

No value is returned.

Examples

Example #1 `echo` examples

```
<?php
echo "echo does not require parentheses.";

// Strings can either be passed individually as multiple arguments or
// concatenated together and passed as a single argument
echo 'This ', 'string ', 'was ', 'made ', 'with multiple parameters.', "\n";
echo 'This ' . 'string ' . 'was ' . 'made ' . 'with concatenation.' . "\n";

// No newline or space is added; the below outputs "helloworld" all on one line
echo "hello";
echo "world";

// Same as above
echo "hello", "world";

echo "This string spans
```

```
multiple lines. The newlines will be
output as well";

echo "This string spans\nmultiple lines. The newlines will be\noutput as well.";

// The argument can be any expression which produces a string
$foo = "example";
echo "foo is $foo"; // foo is example

$fruits = ["lemon", "orange", "banana"];
echo implode(" and ", $fruits); // lemon and orange and banana

// Non-string expressions are coerced to string, even if declare(strict_types=1) is used
echo 6 * 7; // 42

// However, the following examples will work:
($some_var) ? print 'true' : print 'false'; // print is also a construct, but
                                            // it is a valid expression, returning 1,
                                            // so it may be used in this context.

echo $some_var ? 'true': 'false'; // evaluating the expression first and passing it to echo
?>
```

Example #2 `echo` is not an expression

```
<?php
// Because echo does not behave as an expression, the following code is invalid.
($some_var) ? echo 'true' : echo 'false';
?>
```

Notes

> **Note** :
>
> Because this is a language construct and not a function, it cannot be called using variable functions, or named arguments.

> **Note** : **Using with parentheses**
>
> Surrounding a single argument to `echo` with parentheses will not raise a syntax error, and produces syntax which looks like a normal function call. However, this can be misleading, because the parentheses are actually part of the expression being output, not part of the `echo` syntax itself.
>
> **Example #3 Using Parentheses**
>
> ```
> <?php
> echo "hello", PHP_EOL;
> // outputs "hello"
>
> echo("hello"), PHP_EOL;
> // also outputs "hello", because ("hello") is a valid expression
>
> echo(1 + 2) * 3, PHP_EOL;
> // outputs "9"; the parentheses cause 1+2 to be evaluated first, then 3*3
> // the echo statement sees the whole expression as one argument
>
> echo "hello", " world", PHP_EOL;
> // outputs "hello world"
>
> echo("hello"), (" world"), PHP_EOL;
> // outputs "hello world"; the parentheses are part of each expression
> ?>
> ```
>
> **Example #4 Invalid Expression**
>
> ```
> <?php
> echo("hello", " world"), PHP_EOL;
> ```

```
// Throws a Parse Error because ("hello", " world") is not a valid expression
?>
```

Tip

Passing multiple arguments to `echo` can avoid complications arising from the precedence of the concatenation operator in PHP. For instance, the concatenation operator has higher precedence than the ternary operator, and prior to PHP 8.0.0 had the same precedence as addition and subtraction:

```
<?php
// Below, the expression 'Hello ' . isset($name) is evaluated first,
// and is always true, so the argument to echo is always $name
echo 'Hello ' . isset($name) ? $name : 'John Doe' . '!';

// The intended behaviour requires additional parentheses
echo 'Hello ' . (isset($name) ? $name : 'John Doe') . '!';

// In PHP prior to 8.0.0, the below outputs "2", rather than "Sum: 3"
echo 'Sum: ' . 1 + 2;

// Again, adding parentheses ensures the intended order of evaluation
echo 'Sum: ' . (1 + 2);
```

If multiple arguments are passed in, then parentheses will not be required to enforce precedence, because each expression is separate:

```
<?php
echo "Hello ", isset($name) ? $name : "John Doe", "!";

echo "Sum: ", 1 + 2;
```

See Also

- print - Output a string
- printf() - Output a formatted string
- flush() - Flush system output buffer
- Ways to specify literal strings

explode

explode

(PHP 4, PHP 5, PHP 7, PHP 8)

explode — Split a string by a string

Description

function **explode**(string `$separator`, string `$string`, int `$limit` = `PHP_INT_MAX`): array

Returns an array of strings, each of which is a substring of `string` formed by splitting it on boundaries formed by the string `separator`.

Parameters

`separator`

The boundary string.

`string`

The input string.

`limit`

If `limit` is set and positive, the returned array will contain a maximum of `limit` elements with the last element containing the rest of `string`.

If the `limit` parameter is negative, all components except the last `-limit` are returned.

If the `limit` parameter is zero, then this is treated as 1.

> **Note** :
>
> Prior to PHP 8.0, implode() accepted its parameters in either order. **explode()** has never supported this: you must ensure that the `separator` argument comes before the `string` argument.

Return Values

Returns an array of strings created by splitting the `string` parameter on boundaries formed by the `separator`.

If `separator` is an empty string (""), **explode()** throws a ValueError. If `separator` contains a value that is not contained in `string` and a negative `limit` is used, then an empty array will be returned, otherwise an array containing `string` will be returned. If `separator` values appear at the start or end of `string`, said values will be added as an empty array value either in the first or last position of the returned array respectively.

Changelog

Version	Description
8.0.0	**explode()** will now throw ValueError when `separator` parameter is given an empty string (""). Previously, **explode()** returned `false` instead.

Examples

Example #1 explode() examples

```php
<?php
// Example 1
$pizza  = "piece1 piece2 piece3 piece4 piece5 piece6";
$pieces = explode(" ", $pizza);
echo $pieces[0], PHP_EOL; // piece1
echo $pieces[1], PHP_EOL; // piece2

// Example 2
$data = "foo:*:1023:1000::/home/foo:/bin/sh";
list($user, $pass, $uid, $gid, $gecos, $home, $shell) = explode(":", $data);
echo $user, PHP_EOL; // foo
echo $pass, PHP_EOL; // *
?>
```

Example #2 explode() return examples

```php
<?php
/*
   A string that doesn't contain the delimiter will simply
   return a one-length array of the original string.
*/
$input1 = "hello";
$input2 = "hello,there";
$input3 = ',';
var_dump( explode( ',', $input1 ) );
var_dump( explode( ',', $input2 ) );
var_dump( explode( ',', $input3 ) );

?>
```

The above example will output:

```
array(1) {
  [0]=>
  string(5) "hello"
}
array(2) {
  [0]=>
  string(5) "hello"
  [1]=>
  string(5) "there"
```

```
}
array(2) {
  [0]=>
  string(0) ""
  [1]=>
  string(0) ""
}
```

Example #3 `limit` parameter examples

```php
<?php
$str = 'one|two|three|four';

// positive limit
print_r(explode('|', $str, 2));

// negative limit
print_r(explode('|', $str, -1));
?>
```

The above example will output:

```
Array
(
    [0] => one
    [1] => two|three|four
)
Array
(
    [0] => one
    [1] => two
    [2] => three
)
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- preg_split() - Split string by a regular expression
- str_split() - Convert a string to an array
- mb_split() - Split multibyte string using regular expression
- str_word_count() - Return information about words used in a string
- strtok() - Tokenize string
- implode() - Join array elements with a string

fprintf

fprintf

(PHP 5, PHP 7, PHP 8)

fprintf — Write a formatted string to a stream

Description

function **fprintf**(resource `$stream`, string `$format`, mixed `...$values`): int

Write a string produced according to `format` to the stream resource specified by `stream`.

Parameters

`stream`

A file system pointer resource that is typically created using fopen().

`format`

The format string is composed of zero or more directives: ordinary characters (excluding `%`) that are copied directly to the result and *conversion specifications*, each of which results in fetching its own parameter.

A conversion specification follows this prototype: `%[argnum$][flags][width][.precision]specifier`.

Argnum

An integer followed by a dollar sign `$`, to specify which number argument to treat in the conversion.

Flags

Flag	Description
`-`	Left-justify within the given field width; Right justification is the default
`+`	Prefix positive numbers with a plus sign `+`; Default only negative are prefixed with a negative sign.
(space)	Pads the result with spaces. This is the default.
`0`	Only left-pads numbers with zeros. With `s` specifiers this can also right-pad with zeros.
`'`(char)	Pads the result with the character (char).

Width

Either an integer that says how many characters (minimum) this conversion should result in, or `*`. If `*` is used, then the width is supplied as an additional integer value preceding the one formatted by the specifier.

Precision

A period `.` optionally followed by either an integer or `*`, whose meaning depends on the specifier:

- For `e`, `E`, `f` and `F` specifiers: this is the number of digits to be printed after the decimal point (by default, this is 6).
- For `g`, `G`, `h` and `H` specifiers: this is the maximum number of significant digits to be printed.
- For `s` specifier: it acts as a cutoff point, setting a maximum character limit to the string.

> **Note** :
>
> If the period is specified without an explicit value for precision, 0 is assumed. If `*` is used, the precision is supplied as an additional integer value preceding the one formatted by the specifier.

Specifiers

Specifier	Description
`%`	A literal percent character. No argument is required.
`b`	The argument is treated as an integer and presented as a binary number.
`c`	The argument is treated as an integer and presented as the character with that ASCII.
`d`	The argument is treated as an integer and presented as a (signed) decimal number.
`e`	The argument is treated as scientific notation (e.g. 1.2e+2).
`E`	Like the `e` specifier but uses uppercase letter (e.g. 1.2E+2).
`f`	The argument is treated as a float and presented as a floating-point number (locale aware).
`F`	The argument is treated as a float and presented as a floating-point number (non-locale aware).

Specifier	Description
g	General format. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of X: If P > X ≥ −4, the conversion is with style f and precision P − (X + 1). Otherwise, the conversion is with style e and precision P − 1.
G	Like the g specifier but uses E and f.
h	Like the g specifier but uses F. Available as of PHP 8.0.0.
H	Like the g specifier but uses E and F. Available as of PHP 8.0.0.
o	The argument is treated as an integer and presented as an octal number.
s	The argument is treated and presented as a string.
u	The argument is treated as an integer and presented as an unsigned decimal number.
x	The argument is treated as an integer and presented as a hexadecimal number (with lowercase letters).
X	The argument is treated as an integer and presented as a hexadecimal number (with uppercase letters).

> **Warning**
>
> The c type specifier ignores padding and width.

> **Warning**
>
> Attempting to use a combination of the string and width specifiers with character sets that require more than one byte per character may result in unexpected results.

Variables will be co-erced to a suitable type for the specifier:

Type Handling

Type	Specifiers
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Return Values

Returns the length of the string written.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the number of arguments is zero. Prior to PHP 8.0.0, a **`E_WARNING`** was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[width]` is less than zero or bigger than **`PHP_INT_MAX`**. Prior to PHP 8.0.0, a **`E_WARNING`** was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[precision]` is less than zero or bigger than **`PHP_INT_MAX`**. Prior to PHP 8.0.0, a **`E_WARNING`** was emitted instead.

As of PHP 8.0.0, a ArgumentCountError is thrown when less arguments are given than required. Prior to PHP 8.0.0, **`false`** was returned and a **`E_WARNING`** emitted instead.

Changelog

Version	Description
8.0.0	This function no longer returns **`false`** on failure.
8.0.0	Throw a ValueError if the number of arguments is zero; previously this function emitted a **`E_WARNING`** instead.
8.0.0	Throw a ValueError if `[width]` is less than zero or bigger than **`PHP_INT_MAX`**; previously this function emitted a **`E_WARNING`** instead.
8.0.0	Throw a ValueError if `[precision]` is less than zero or bigger than **`PHP_INT_MAX`**; previously this function emitted a **`E_WARNING`** instead.

Version	Description
8.0.0	Throw a ArgumentCountError when less arguments are given than required; previously this function emitted a `E_WARNING` instead.

Examples

Example #1 fprintf(): zero-padded integers

```
<?php
if (!($fp = fopen('date.txt', 'w'))) {
    return;
}

$year = 2005;
$month = 5;
$day = 6;

fprintf($fp, "%04d-%02d-%02d", $year, $month, $day);
// will write the formatted ISO date to date.txt
?>
```

Example #2 fprintf(): formatting currency

```
<?php
if (!($fp = fopen('currency.txt', 'w'))) {
    return;
}

$money1 = 68.75;
$money2 = 54.35;
$money = $money1 + $money2;
// echo $money will output "123.1";
$len = fprintf($fp, '%01.2f', $money);
// will write "123.10" to currency.txt

echo "wrote $len bytes to currency.txt";
// use the return value of fprintf to determine how many bytes we wrote
?>
```

See Also

- printf() - Output a formatted string
- sprintf() - Return a formatted string
- vprintf() - Output a formatted string
- vsprintf() - Return a formatted string
- vfprintf() - Write a formatted string to a stream
- sscanf() - Parses input from a string according to a format
- fscanf() - Parses input from a file according to a format
- number_format() - Format a number with grouped thousands
- date() - Format a Unix timestamp

get_html_translation_table

get_html_translation_table

(PHP 4, PHP 5, PHP 7, PHP 8)

get_html_translation_table — Returns the translation table used by htmlspecialchars() and htmlentities()

Description

function **get_html_translation_table**(int `$table` = `HTML_SPECIALCHARS`, int `$flags` = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401, string `$encoding` = "UTF-8"): array

get_html_translation_table() will return the translation table that is used internally for htmlspecialchars() and htmlentities().

> **Note** :

> Special characters can be encoded in several ways. E.g. " can be encoded as `"`, `"` or `"`. **get_html_translation_table()** returns only the form used by htmlspecialchars() and htmlentities().

Parameters

`table`

Which table to return. Either **`HTML_ENTITIES`** or **`HTML_SPECIALCHARS`**.

`flags`

A bitmask of one or more of the following flags, which specify which quotes the table will contain as well as which document type the table is for. The default is `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Available `flags` constants

Constant Name	Description
`ENT_COMPAT`	Table will contain entities for double-quotes, but not for single-quotes.
`ENT_QUOTES`	Table will contain entities for both double and single quotes.
`ENT_NOQUOTES`	Table will neither contain entities for single quotes nor for double quotes.
`ENT_SUBSTITUTE`	Replace invalid code unit sequences with a Unicode Replacement Character U+FFFD (UTF-8) or &#xFFFD; (otherwise) instead of returning an empty string.
`ENT_HTML401`	Table for HTML 4.01.
`ENT_XML1`	Table for XML 1.
`ENT_XHTML`	Table for XHTML.
`ENT_HTML5`	Table for HTML 5.

`encoding`

Encoding to use. If omitted, the default value for this argument is UTF-8.

The following character sets are supported:

Supported charsets

Charset	Aliases	Description
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Little used cyrillic charset (Latin/Cyrillic).
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adds the Euro sign, French and Finnish letters missing in Latin-1 (ISO-8859-1).
UTF-8		ASCII compatible multi-byte 8-bit Unicode.
cp866	ibm866, 866	DOS-specific Cyrillic charset.
cp1251	Windows-1251, win-1251, 1251	Windows-specific Cyrillic charset.
cp1252	Windows-1252, 1252	Windows specific charset for Western European.
KOI8-R	koi8-ru, koi8r	Russian.
BIG5	950	Traditional Chinese, mainly used in Taiwan.
GB2312	936	Simplified Chinese, national standard character set.
BIG5-HKSCS		Big5 with Hong Kong extensions, Traditional Chinese.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japanese
EUC-JP	EUCJP, eucJP-win	Japanese
MacRoman		Charset that was used by Mac OS.
''		An empty string activates detection from script encoding (Zend multibyte), default_charset and current locale (see nl_langinfo() and setlocale()), in this order. Not recommended.

> **Note** :
>
> Any other character sets are not recognized. The default encoding will be used instead and a warning will be emitted.

Return Values

Returns the translation table as an array, with the original characters as keys and entities as values.

Changelog

Version	Description
8.1.0	`flags` changed from `ENT_COMPAT` to `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.

Examples

Example #1 Translation Table Example

```php
<?php
var_dump(get_html_translation_table(HTML_ENTITIES, ENT_QUOTES | ENT_HTML5));
?>
```

The above example will output something similar to:

```
array(1510) {
  ["
"]=>
  string(9) "&NewLine;"
  ["!"]=>
  string(6) "&excl;"
  [""]=>
  string(6) "&quot;"
  ["#"]=>
  string(5) "&num;"
  ["$"]=>
  string(8) "&dollar;"
  ["%"]=>
  string(8) "&percnt;"
  ["&"]=>
  string(5) "&amp;"
  ["'"]=>
  string(6) "&apos;"
  // ...
}
```

See Also

- htmlspecialchars() - Convert special characters to HTML entities
- htmlentities() - Convert all applicable characters to HTML entities
- html_entity_decode() - Convert HTML entities to their corresponding characters

hebrev

hebrev

(PHP 4, PHP 5, PHP 7, PHP 8)

hebrev — Convert logical Hebrew text to visual text

Description

function **hebrev**(string `$string`, int `$max_chars_per_line` = 0): string

Converts logical Hebrew text to visual text.

The function tries to avoid breaking words.

Parameters

`string`

A Hebrew input string.

`max_chars_per_line`

This optional parameter indicates maximum number of characters per line that will be returned.

Return Values

Returns the visual string.

See Also

- hebrevc() - Convert logical Hebrew text to visual text with newline conversion

hebrevc

hebrevc

(PHP 4, PHP 5, PHP 7)

hebrevc — Convert logical Hebrew text to visual text with newline conversion

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.4.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **hebrevc**(string `$hebrew_text`, int `$max_chars_per_line` = 0): string

This function is similar to hebrev() with the difference that it converts newlines (\n) to "
\n".

The function tries to avoid breaking words.

Parameters

`hebrew_text`

A Hebrew input string.

`max_chars_per_line`

This optional parameter indicates maximum number of characters per line that will be returned.

Return Values

Returns the visual string.

Changelog

Version	Description
8.0.0	Removed this function.
7.4.0	Deprecated this function.

See Also

- hebrev() - Convert logical Hebrew text to visual text

hex2bin

hex2bin

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

hex2bin — Decodes a hexadecimally encoded binary string

Description

function **hex2bin**(string `$string`): string|false

Decodes a hexadecimally encoded binary string.

> **Caution**
>
> This function does *NOT* convert a hexadecimal number to a binary number. This can be done using the base_convert() function.

Parameters

`string`

Hexadecimal representation of data.

Return Values

Returns the binary representation of the given data or `false` on failure.

Errors/Exceptions

If the hexadecimal input string is of odd length or invalid hexadecimal string an `E_WARNING` level error is thrown.

Examples

Example #1 hex2bin() example

```
<?php
$hex = hex2bin("6578616d706c65206865782064617461");
var_dump($hex);
?>
```

The above example will output something similar to:

```
string(16) "example hex data"
```

See Also

- bin2hex() - Convert binary data into hexadecimal representation
- unpack() - Unpack data from binary string

html_entity_decode

html_entity_decode

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

html_entity_decode — Convert HTML entities to their corresponding characters

Description

function **html_entity_decode**(string `$string`, int `$flags` = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401, ?string `$encoding` = `null`): string

html_entity_decode() is the opposite of htmlentities() in that it converts HTML entities in the `string` to their corresponding characters.

More precisely, this function decodes all the entities (including all numeric entities) that a) are necessarily valid for the chosen document type — i.e., for XML, this function does not decode named entities that might be defined in some DTD — and b) whose character or

characters are in the coded character set associated with the chosen encoding and are permitted in the chosen document type. All other entities are left as is.

Parameters

`string`

The input string.

`flags`

A bitmask of one or more of the following flags, which specify how to handle quotes and which document type to use. The default is `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Available `flags` constants

Constant Name	Description
`ENT_COMPAT`	Will convert double-quotes and leave single-quotes alone.
`ENT_QUOTES`	Will convert both double and single quotes.
`ENT_NOQUOTES`	Will leave both double and single quotes unconverted.
`ENT_SUBSTITUTE`	Replace invalid code unit sequences with a Unicode Replacement Character U+FFFD (UTF-8) or &#xFFFD; (otherwise) instead of returning an empty string.
`ENT_HTML401`	Handle code as HTML 4.01.
`ENT_XML1`	Handle code as XML 1.
`ENT_XHTML`	Handle code as XHTML.
`ENT_HTML5`	Handle code as HTML 5.

`encoding`

An optional argument defining the encoding used when converting characters.

If omitted, `encoding` defaults to the value of the default_charset configuration option.

Although this argument is technically optional, you are highly encouraged to specify the correct value for your code if the default_charset configuration option may be set incorrectly for the given input.

The following character sets are supported:

Supported charsets

Charset	Aliases	Description
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Little used cyrillic charset (Latin/Cyrillic).
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adds the Euro sign, French and Finnish letters missing in Latin-1 (ISO-8859-1).
UTF-8		ASCII compatible multi-byte 8-bit Unicode.
cp866	ibm866, 866	DOS-specific Cyrillic charset.
cp1251	Windows-1251, win-1251, 1251	Windows-specific Cyrillic charset.
cp1252	Windows-1252, 1252	Windows specific charset for Western European.
KOI8-R	koi8-ru, koi8r	Russian.
BIG5	950	Traditional Chinese, mainly used in Taiwan.
GB2312	936	Simplified Chinese, national standard character set.
BIG5-HKSCS		Big5 with Hong Kong extensions, Traditional Chinese.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japanese
EUC-JP	EUCJP, eucJP-win	Japanese

Charset	Aliases	Description
MacRoman		Charset that was used by Mac OS.
''		An empty string activates detection from script encoding (Zend multibyte), default_charset and current locale (see nl_langinfo() and setlocale()), in this order. Not recommended.

> **Note** :
>
> Any other character sets are not recognized. The default encoding will be used instead and a warning will be emitted.

Return Values

Returns the decoded string.

Changelog

Version	Description
8.1.0	`flags` changed from `ENT_COMPAT` to `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.
8.0.0	`encoding` is nullable now.

Examples

Example #1 Decoding HTML entities

```
<?php
$orig = "I'll \"walk\" the <b>dog</b> now";

$a = htmlentities($orig);

$b = html_entity_decode($a);

echo $a, PHP_EOL; // I'll &quot;walk&quot; the &lt;b&gt;dog&lt;/b&gt; now

echo $b, PHP_EOL; // I'll "walk" the <b>dog</b> now
?>
```

Notes

> **Note** :
>
> You might wonder why trim(html_entity_decode(' ')); doesn't reduce the string to an empty string, that's because the ' ' entity is not ASCII code 32 (which is stripped by trim()) but ASCII code 160 (0xa0) in the default ISO 8859-1 encoding.

See Also

- htmlentities() - Convert all applicable characters to HTML entities
- htmlspecialchars() - Convert special characters to HTML entities
- get_html_translation_table() - Returns the translation table used by htmlspecialchars and htmlentities
- urldecode() - Decodes URL-encoded string

htmlentities

htmlentities

(PHP 4, PHP 5, PHP 7, PHP 8)

htmlentities — Convert all applicable characters to HTML entities

Description

```
function htmlentities(
    string $string,
    int $flags = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401,
    ?string $encoding = null,
```

```
    bool $double_encode = true
): string
```

This function is identical to htmlspecialchars() in all ways, except with **htmlentities()**, all characters which have HTML character entity equivalents are translated into these entities. The get_html_translation_table() function can be used to return the translation table used dependent upon the provided `flags` constants.

If you want to decode instead (the reverse) you can use html_entity_decode().

Parameters

`string`

The input string.

`flags`

A bitmask of one or more of the following flags, which specify how to handle quotes, invalid code unit sequences and the used document type. The default is `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Available `flags` constants

Constant Name	Description
`ENT_COMPAT`	Will convert double-quotes and leave single-quotes alone.
`ENT_QUOTES`	Will convert both double and single quotes.
`ENT_NOQUOTES`	Will leave both double and single quotes unconverted.
`ENT_IGNORE`	Silently discard invalid code unit sequences instead of returning an empty string. Using this flag is discouraged as it » may have security implications.
`ENT_SUBSTITUTE`	Replace invalid code unit sequences with a Unicode Replacement Character U+FFFD (UTF-8) or &#FFFD; (otherwise) instead of returning an empty string.
`ENT_DISALLOWED`	Replace invalid code points for the given document type with a Unicode Replacement Character U+FFFD (UTF-8) or &#FFFD; (otherwise) instead of leaving them as is. This may be useful, for instance, to ensure the well-formedness of XML documents with embedded external content.
`ENT_HTML401`	Handle code as HTML 4.01.
`ENT_XML1`	Handle code as XML 1.
`ENT_XHTML`	Handle code as XHTML.
`ENT_HTML5`	Handle code as HTML 5.

`encoding`

An optional argument defining the encoding used when converting characters.

If omitted, `encoding` defaults to the value of the default_charset configuration option.

Although this argument is technically optional, you are highly encouraged to specify the correct value for your code if the default_charset configuration option may be set incorrectly for the given input.

The following character sets are supported:

Supported charsets

Charset	Aliases	Description
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Little used cyrillic charset (Latin/Cyrillic).
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adds the Euro sign, French and Finnish letters missing in Latin-1 (ISO-8859-1).
UTF-8		ASCII compatible multi-byte 8-bit Unicode.
cp866	ibm866, 866	DOS-specific Cyrillic charset.
cp1251	Windows-1251, win-1251, 1251	Windows-specific Cyrillic charset.

Charset	Aliases	Description
cp1252	Windows-1252, 1252	Windows specific charset for Western European.
KOI8-R	koi8-ru, koi8r	Russian.
BIG5	950	Traditional Chinese, mainly used in Taiwan.
GB2312	936	Simplified Chinese, national standard character set.
BIG5-HKSCS		Big5 with Hong Kong extensions, Traditional Chinese.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japanese
EUC-JP	EUCJP, eucJP-win	Japanese
MacRoman		Charset that was used by Mac OS.
''		An empty string activates detection from script encoding (Zend multibyte), default_charset and current locale (see nl_langinfo() and setlocale()), in this order. Not recommended.

> **Note** :
>
> Any other character sets are not recognized. The default encoding will be used instead and a warning will be emitted.

`double_encode`

When `double_encode` is turned off PHP will not encode existing html entities. The default is to convert everything.

Return Values

Returns the encoded string.

If the input `string` contains an invalid code unit sequence within the given `encoding` an empty string will be returned, unless either the `ENT_IGNORE` or `ENT_SUBSTITUTE` flags are set.

Changelog

Version	Description
8.1.0	`flags` changed from `ENT_COMPAT` to `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.
8.0.0	`encoding` is nullable now.

Examples

Example #1 A htmlentities() example

```php
<?php
$str = "A 'quote' is <b>bold</b>";

echo htmlentities($str);
echo "\n\n";
echo htmlentities($str, ENT_COMPAT);
?>
```

The above example will output:

```
A &#039;quote&#039; is &lt;b&gt;bold&lt;/b&gt;

A 'quote' is &lt;b&gt;bold&lt;/b&gt;
```

Example #2 Usage of `ENT_IGNORE`

```php
<?php
$str = "\x8F!!!";

// Outputs an empty string
echo htmlentities($str, ENT_QUOTES, "UTF-8");

// Outputs "!!!"
echo htmlentities($str, ENT_QUOTES | ENT_IGNORE, "UTF-8");
?>
```

See Also

- html_entity_decode() - Convert HTML entities to their corresponding characters
- get_html_translation_table() - Returns the translation table used by htmlspecialchars and htmlentities
- htmlspecialchars() - Convert special characters to HTML entities
- nl2br() - Inserts HTML line breaks before all newlines in a string
- urlencode() - URL-encodes string

htmlspecialchars

htmlspecialchars

(PHP 4, PHP 5, PHP 7, PHP 8)

htmlspecialchars — Convert special characters to HTML entities

Description

function **htmlspecialchars**(
  string $string,
  int $flags = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401,
  ?string $encoding = null,
  bool $double_encode = true
): string

Certain characters have special significance in HTML, and should be represented by HTML entities if they are to preserve their meanings. This function returns a string with these conversions made. If you require all input substrings that have associated named entities to be translated, use htmlentities() instead.

If the input string passed to this function and the final document share the same character set, this function is sufficient to prepare input for inclusion in most contexts of an HTML document. If, however, the input can represent characters that are not coded in the final document character set and you wish to retain those characters (as numeric or named entities), both this function and htmlentities() (which only encodes substrings that have named entity equivalents) may be insufficient. You may have to use mb_encode_numericentity() instead.

Performed translations

Character	Replacement
& (ampersand)	`&`
" (double quote)	`"`, unless `ENT_NOQUOTES` is set
' (single quote)	`'` (for `ENT_HTML401`) or `'` (for `ENT_XML1`, `ENT_XHTML` or `ENT_HTML5`), but only when `ENT_QUOTES` is set
< (less than)	`<`
> (greater than)	`>`

Parameters

`string`

The string being converted.

`flags`

A bitmask of one or more of the following flags, which specify how to handle quotes, invalid code unit sequences and the used document type. The default is `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Available `flags` constants

Constant Name	Description
ENT_COMPAT	Will convert double-quotes and leave single-quotes alone.
ENT_QUOTES	Will convert both double and single quotes.
ENT_NOQUOTES	Will leave both double and single quotes unconverted.
ENT_IGNORE	Silently discard invalid code unit sequences instead of returning an empty string. Using this flag is discouraged as it » may have security implications.
ENT_SUBSTITUTE	Replace invalid code unit sequences with a Unicode Replacement Character U+FFFD (UTF-8) or &#xFFFD; (otherwise) instead of returning an empty string.
ENT_DISALLOWED	Replace invalid code points for the given document type with a Unicode Replacement Character U+FFFD (UTF-8) or &#xFFFD; (otherwise) instead of leaving them as is. This may be useful, for instance, to ensure the well-formedness of XML documents with embedded external content.
ENT_HTML401	Handle code as HTML 4.01.
ENT_XML1	Handle code as XML 1.
ENT_XHTML	Handle code as XHTML.
ENT_HTML5	Handle code as HTML 5.

`encoding`

An optional argument defining the encoding used when converting characters.

If omitted, `encoding` defaults to the value of the default_charset configuration option.

Although this argument is technically optional, you are highly encouraged to specify the correct value for your code if the default_charset configuration option may be set incorrectly for the given input.

For the purposes of this function, the encodings `ISO-8859-1`, `ISO-8859-15`, `UTF-8`, `cp866`, `cp1251`, `cp1252`, and `KOI8-R` are effectively equivalent, provided the `string` itself is valid for the encoding, as the characters affected by **htmlspecialchars()** occupy the same positions in all of these encodings.

The following character sets are supported:

Supported charsets

Charset	Aliases	Description
ISO-8859-1	ISO8859-1	Western European, Latin-1.
ISO-8859-5	ISO8859-5	Little used cyrillic charset (Latin/Cyrillic).
ISO-8859-15	ISO8859-15	Western European, Latin-9. Adds the Euro sign, French and Finnish letters missing in Latin-1 (ISO-8859-1).
UTF-8		ASCII compatible multi-byte 8-bit Unicode.
cp866	ibm866, 866	DOS-specific Cyrillic charset.
cp1251	Windows-1251, win-1251, 1251	Windows-specific Cyrillic charset.
cp1252	Windows-1252, 1252	Windows specific charset for Western European.
KOI8-R	koi8-ru, koi8r	Russian.
BIG5	950	Traditional Chinese, mainly used in Taiwan.
GB2312	936	Simplified Chinese, national standard character set.
BIG5-HKSCS		Big5 with Hong Kong extensions, Traditional Chinese.
Shift_JIS	SJIS, SJIS-win, cp932, 932	Japanese
EUC-JP	EUCJP, eucJP-win	Japanese
MacRoman		Charset that was used by Mac OS.
''		An empty string activates detection from script encoding (Zend multibyte), default_charset and current locale (see nl_langinfo() and setlocale()), in this order. Not recommended.

> **Note** :
>
> Any other character sets are not recognized. The default encoding will be used instead and a warning will be emitted.

`double_encode`

When `double_encode` is turned off PHP will not encode existing html entities, the default is to convert everything.

Return Values

The converted string.

If the input `string` contains an invalid code unit sequence within the given `encoding` an empty string will be returned, unless either the `ENT_IGNORE` or `ENT_SUBSTITUTE` flags are set.

Changelog

Version	Description
8.1.0	`flags` changed from `ENT_COMPAT` to `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.

Examples

Example #1 htmlspecialchars() example

```php
<?php
$new = htmlspecialchars("<a href='test'>Test</a>", ENT_QUOTES);
echo $new; // &lt;a href=&#039;test&#039;&gt;Test&lt;/a&gt;
?>
```

Example #2 `ENT_IGNORE` and `ENT_SUBSTITUTE` with invalid UTF-8 sequences

```php
<?php
// "\x80" is not a valid UTF-8 byte sequence.
$input = "atta\x80ck";

// ENT_IGNORE silently drops the invalid byte - output: "attack"
echo bin2hex(htmlspecialchars($input, ENT_QUOTES | ENT_IGNORE, 'UTF-8')), "\n";
// 61747461636b

// ENT_SUBSTITUTE replaces the invalid byte with U+FFFD - output: "atta" + replacement char + "ck"
echo bin2hex(htmlspecialchars($input, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')), "\n";
// 61747461efbfbd636b
?>
```

Notes

> **Note** :
>
> Note that this function does not translate anything beyond what is listed above. For full entity translation, see htmlentities().

> **Note** :
>
> In case of an ambiguous `flags` value, the following rules apply:
>
> - When neither of `ENT_COMPAT`, `ENT_QUOTES`, `ENT_NOQUOTES` is present, the default is `ENT_NOQUOTES`.
> - When more than one of `ENT_COMPAT`, `ENT_QUOTES`, `ENT_NOQUOTES` is present, `ENT_QUOTES` takes the highest precedence, followed by `ENT_COMPAT`.
> - When neither of `ENT_HTML401`, `ENT_HTML5`, `ENT_XHTML`, `ENT_XML1` is present, the default is `ENT_HTML401`.
> - When more than one of `ENT_HTML401`, `ENT_HTML5`, `ENT_XHTML`, `ENT_XML1` is present, `ENT_HTML5` takes the highest precedence, followed by `ENT_XHTML`, `ENT_XML1` and `ENT_HTML401`.

> - When more than one of `ENT_DISALLOWED`, `ENT_IGNORE`, `ENT_SUBSTITUTE` are present, `ENT_IGNORE` takes the highest precedence, followed by `ENT_SUBSTITUTE`.

See Also

- get_html_translation_table() - Returns the translation table used by htmlspecialchars and htmlentities
- htmlspecialchars_decode() - Convert special HTML entities back to characters
- strip_tags() - Strip HTML and PHP tags from a string
- htmlentities() - Convert all applicable characters to HTML entities
- nl2br() - Inserts HTML line breaks before all newlines in a string

htmlspecialchars_decode

htmlspecialchars_decode

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

htmlspecialchars_decode — Convert special HTML entities back to characters

Description

function **htmlspecialchars_decode**(string `$string`, int `$flags` = ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401): string

This function is the opposite of htmlspecialchars(). It converts special HTML entities back to characters.

The converted entities are: `&`, `"` (when `ENT_NOQUOTES` is not set), `'` (when `ENT_QUOTES` is set), `<` and `>`.

Parameters

`string`

The string to decode.

`flags`

A bitmask of one or more of the following flags, which specify how to handle quotes and which document type to use. The default is `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`.

Available `flags` constants

Constant Name	Description
`ENT_COMPAT`	Will convert double-quotes and leave single-quotes alone.
`ENT_QUOTES`	Will convert both double and single quotes.
`ENT_NOQUOTES`	Will leave both double and single quotes unconverted.
`ENT_SUBSTITUTE`	Replace invalid code unit sequences with a Unicode Replacement Character U+FFFD (UTF-8) or &#xFFFD; (otherwise) instead of returning an empty string.
`ENT_HTML401`	Handle code as HTML 4.01.
`ENT_XML1`	Handle code as XML 1.
`ENT_XHTML`	Handle code as XHTML.
`ENT_HTML5`	Handle code as HTML 5.

Return Values

Returns the decoded string.

Changelog

Version	Description
8.1.0	`flags` changed from `ENT_COMPAT` to `ENT_QUOTES` | `ENT_SUBSTITUTE` | `ENT_HTML401`.

Examples

Example #1 A htmlspecialchars_decode() example

```php
<?php
$str = "<p>this -&gt; &quot;</p>\n";

echo htmlspecialchars_decode($str);

// note that here the quotes aren't converted
echo htmlspecialchars_decode($str, ENT_NOQUOTES);
?>
```

The above example will output:

```
<p>this -> "</p>
<p>this -> &quot;</p>
```

See Also

- htmlspecialchars() - Convert special characters to HTML entities
- html_entity_decode() - Convert HTML entities to their corresponding characters
- get_html_translation_table() - Returns the translation table used by htmlspecialchars and htmlentities

implode

implode

(PHP 4, PHP 5, PHP 7, PHP 8)

implode — Join array elements with a string

Description

function **implode**(string `$separator`, array `$array`): string

Alternative signature (not supported with named arguments):

function **implode**(array `$array`): string

Legacy signature (deprecated as of PHP 7.4.0, removed as of PHP 8.0.0):

function **implode**(array `$array`, string `$separator`): string

Join array elements with a `separator` string.

Parameters

`separator`

Optional. Defaults to an empty string.

`array`

The array of strings to implode.

Return Values

Returns a string containing a string representation of all the array elements in the same order, with the separator string between each element.

Changelog

Version	Description
8.0.0	Passing the `separator` after the `array` is no longer supported.
7.4.0	Passing the `separator` after the `array` (i.e. using the legacy signature) has been deprecated.

Examples

Example #1 implode() example

```php
<?php

$array = ['lastname', 'email', 'phone'];
var_dump(implode(",", $array)); // string(20) "lastname,email,phone"

// Empty string when using an empty array:
var_dump(implode('hello', [])); // string(0) ""

// The separator is optional:
var_dump(implode(['a', 'b', 'c'])); // string(3) "abc"

?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- explode() - Split a string by a string
- preg_split() - Split string by a regular expression
- http_build_query() - Generate URL-encoded query string

join

join

(PHP 4, PHP 5, PHP 7, PHP 8)

join — Alias of implode()

Description

This function is an alias of: implode().

lcfirst

lcfirst

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

lcfirst — Make a string's first character lowercase

Description

function **lcfirst**(string `$string`): string

Returns a string with the first character of `string` lowercased if that character is an ASCII character in the range `"A"` (0x41) to `"Z"` (0x5a).

Parameters

`string`

The input string.

Return Values

Returns the resulting string.

Changelog

Version	Description
8.2.0	Case conversion no longer depends on the locale set with setlocale(). Only ASCII characters will be converted.

Examples

Example #1 lcfirst() example

```
<?php
$foo = 'HelloWorld';
echo lcfirst($foo), PHP_EOL;             // helloWorld

$bar = 'HELLO WORLD!';
echo lcfirst($bar), PHP_EOL;             // hELLO WORLD!
echo lcfirst(strtoupper($bar)), PHP_EOL; // hELLO WORLD!
?>
```

See Also

- ucfirst() - Make a string's first character uppercase
- strtolower() - Make a string lowercase
- strtoupper() - Make a string uppercase
- ucwords() - Uppercase the first character of each word in a string

levenshtein

levenshtein

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

levenshtein — Calculate Levenshtein distance between two strings

Description

function **levenshtein**(
  string `$string1`,
  string `$string2`,
  int `$insertion_cost` = 1,
  int `$replacement_cost` = 1,
  int `$deletion_cost` = 1
): int

The Levenshtein distance is defined as the minimal number of characters you have to replace, insert or delete to transform `string1` into `string2`. The complexity of the algorithm is `O(m*n)`, where `n` and `m` are the length of `string1` and `string2` (rather good when compared to similar_text(), which is `O(max(n,m)**3)`, but still expensive).

If `insertion_cost`, `replacement_cost` and/or `deletion_cost` are unequal to `1`, the algorithm adapts to choose the cheapest transforms. E.g. if `$insertion_cost + $deletion_cost < $replacement_cost`, no replacements will be done, but rather inserts and deletions instead.

Parameters

`string1`

One of the strings being evaluated for Levenshtein distance.

`string2`

One of the strings being evaluated for Levenshtein distance.

`insertion_cost`

Defines the cost of insertion.

`replacement_cost`

Defines the cost of replacement.

`deletion_cost`

Defines the cost of deletion.

Return Values

This function returns the Levenshtein-Distance between the two argument strings.

Changelog

Version	Description
8.0.0	Prior to this version, **levenshtein()** had to be called with either two or five arguments.
8.0.0	Prior to this version, **levenshtein()** would return `-1` if one of the argument strings is longer than 255 characters.

Examples

Example #1 **levenshtein()** example

```php
<?php
// input misspelled word
$input = 'carrrot';

// array of words to check against
$words  = array('apple','pineapple','banana','orange',
                'radish','carrot','pea','bean','potato');

// no shortest distance found, yet
$shortest = -1;

// loop through words to find the closest
foreach ($words as $word) {

    // calculate the distance between the input word,
    // and the current word
    $lev = levenshtein($input, $word);

    // check for an exact match
    if ($lev == 0) {

        // closest word is this one (exact match)
        $closest = $word;
        $shortest = 0;

        // break out of the loop; we've found an exact match
        break;
    }

    // if this distance is less than the next found shortest
    // distance, OR if a next shortest word has not yet been found
    if ($lev <= $shortest || $shortest < 0) {
        // set the closest match, and shortest distance
        $closest  = $word;
        $shortest = $lev;
    }
}

echo "Input word: $input\n";
if ($shortest == 0) {
    echo "Exact match found: $closest\n";
} else {
    echo "Did you mean: $closest?\n";
}

?>
```

The above example will output:

```
Input word: carrrot
Did you mean: carrot?
```

See Also

- soundex() - Calculate the soundex key of a string
- similar_text() - Calculate the similarity between two strings
- metaphone() - Calculate the metaphone key of a string

localeconv

localeconv

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

localeconv — Get numeric formatting information

Description

function **localeconv**(): array

Returns an associative array containing localized numeric and monetary formatting information.

Parameters

This function has no parameters.

Return Values

localeconv() returns data based upon the current locale as set by setlocale(). The associative array that is returned contains the following fields:

Array element	Description
decimal_point	Decimal point character
thousands_sep	Thousands separator
grouping	Array containing numeric groupings
int_curr_symbol	International currency symbol (i.e. USD)
currency_symbol	Local currency symbol (i.e. $)
mon_decimal_point	Monetary decimal point character
mon_thousands_sep	Monetary thousands separator
mon_grouping	Array containing monetary groupings
positive_sign	Sign for positive values
negative_sign	Sign for negative values
int_frac_digits	International fractional digits
frac_digits	Local fractional digits
p_cs_precedes	1 if currency_symbol precedes a positive value, 0 if it succeeds one
p_sep_by_space	1 if a space separates currency_symbol from a positive value, 0 otherwise
n_cs_precedes	1 if currency_symbol precedes a negative value, 0 if it succeeds one
n_sep_by_space	1 if a space separates currency_symbol from a negative value, 0 otherwise
p_sign_posn	• 0 - Parentheses surround the quantity and currency_symbol • 1 - The sign string precedes the quantity and currency_symbol • 2 - The sign string succeeds the quantity and currency_symbol • 3 - The sign string immediately precedes the currency_symbol • 4 - The sign string immediately succeeds the currency_symbol
n_sign_posn	• 0 - Parentheses surround the quantity and currency_symbol • 1 - The sign string precedes the quantity and currency_symbol • 2 - The sign string succeeds the quantity and currency_symbol • 3 - The sign string immediately precedes the currency_symbol

Array element	Description
	• 4 - The sign string immediately succeeds the currency_symbol

The `p_sign_posn` and `n_sign_posn` contain a string of formatting options. Each number represents one of the above listed conditions.

The grouping fields contain arrays that define the way numbers should be grouped. For example, the monetary grouping field for the nl_NL locale (in UTF-8 mode with the euro sign), would contain a 2 item array with the values 3 and 3. The higher the index in the array, the farther left the grouping is. If an array element is equal to `CHAR_MAX`, no further grouping is done. If an array element is equal to 0, the previous element should be used.

Examples

Example #1 localeconv() example

```
<?php
if (false !== setlocale(LC_ALL, 'nl_NL.UTF-8@euro')) {
    $locale_info = localeconv();
    print_r($locale_info);
}
?>
```

The above example will output:

```
Array
(
    [decimal_point] => .
    [thousands_sep] =>
    [int_curr_symbol] => EUR
    [currency_symbol] => €
    [mon_decimal_point] => ,
    [mon_thousands_sep] =>
    [positive_sign] =>
    [negative_sign] => -
    [int_frac_digits] => 2
    [frac_digits] => 2
    [p_cs_precedes] => 1
    [p_sep_by_space] => 1
    [n_cs_precedes] => 1
    [n_sep_by_space] => 1
    [p_sign_posn] => 1
    [n_sign_posn] => 2
    [grouping] => Array
        (
        )

    [mon_grouping] => Array
        (
            [0] => 3
            [1] => 3
        )

)
```

See Also

- setlocale() - Set locale information

ltrim

ltrim

(PHP 4, PHP 5, PHP 7, PHP 8)

ltrim — Strip whitespace (or other characters) from the beginning of a string

Description

function **ltrim**(string `$string`, string `$characters` = " \n\r\t\v\x00"): string

Strip whitespace (or other characters) from the beginning of a string.

Without the second parameter, **ltrim()** will strip these characters:

- `" "`: ASCII SP character `0x20`, an ordinary space.
- `"\t"`: ASCII HT character `0x09`, a tab.
- `"\n"`: ASCII LF character `0x0A`, a new line (line feed).
- `"\r"`: ASCII CR character `0x0D`, a carriage return.
- `"\0"`: ASCII NUL character `0x00`, the NUL-byte.
- `"\v"`: ASCII VT character `0x0B`, a vertical tab.

Parameters

`string`

The input string.

`characters`

Optionally, the stripped characters can also be specified using the `characters` parameter. Simply list all characters that need to be stripped. With `..` it is possible to specify an incrementing range of characters.

Return Values

This function returns a string with whitespace stripped from the beginning of `string`.

Examples

Example #1 Usage example of ltrim()

```php
<?php

$text = "\t\tThese are a few words :) ...  ";
$binary = "\x09Example string\x0A";
$hello  = "Hello World";
var_dump($text, $binary, $hello);

print "\n";


$trimmed = ltrim($text);
var_dump($trimmed);

$trimmed = ltrim($text, " \t.");
var_dump($trimmed);

$trimmed = ltrim($hello, "Hdle");
var_dump($trimmed);

// trim the ASCII control characters at the beginning of $binary
// (from 0 to 31 inclusive)
$clean = ltrim($binary, "\x00..\x1F");
var_dump($clean);

?>
```

The above example will output:

```
```

```
string(32) "            These are a few words :) ...  "
string(16) "    Example string
"
string(11) "Hello World"

string(30) "These are a few words :) ...  "
string(30) "These are a few words :) ...  "
string(7) "o World"
string(15) "Example string
"
```

See Also

- trim() - Strip whitespace (or other characters) from the beginning and end of a string
- rtrim() - Strip whitespace (or other characters) from the end of a string

md5

md5

(PHP 4, PHP 5, PHP 7, PHP 8)

md5 — Calculate the md5 hash of a string

> **Warning**
>
> It is not recommended to use this function to secure passwords, due to the fast nature of this hashing algorithm. See the Password Hashing FAQ for details and best practices.

Description

function **md5**(string `$string`, bool `$binary` = `false`): string

Calculates the MD5 hash of `string` using the » RSA Data Security, Inc. MD5 Message-Digest Algorithm, and returns that hash.

Parameters

`string`

The string.

`binary`

If the optional `binary` is set to `true`, then the md5 digest is instead returned in raw binary format with a length of 16.

Return Values

Returns the hash as a 32-character hexadecimal number.

Examples

Example #1 A md5() example

```php
<?php
$str = 'apple';

if (md5($str) === '1f3870be274f6c49b3e31a0c6728957f') {
    echo "Would you like a green or red apple?";
}
?>
```

See Also

- hash() - Generate a hash value (message digest)
- password_hash() - Creates a password hash

md5_file

md5_file

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

md5_file — Calculates the md5 hash of a given file

Description

function **md5_file**(string `$filename`, bool `$binary = false`): string|false

Calculates the MD5 hash of the file specified by the `filename` parameter using the » RSA Data Security, Inc. MD5 Message-Digest Algorithm, and returns that hash. The hash is a 32-character hexadecimal number.

Parameters

`filename`

The filename

`binary`

When `true`, returns the digest in raw binary format with a length of 16.

Return Values

Returns a string on success, `false` otherwise.

Examples

Example #1 Usage example of md5_file()

```
<?php
$file = '/examples/book.xml';

echo 'MD5 file hash of ' . $file . ': ' . md5_file($file);
?>
```

See Also

- hash_file() - Generate a hash value using the contents of a given file
- hash_init() - Initialize an incremental hashing context
- md5() - Calculate the md5 hash of a string

metaphone

metaphone

(PHP 4, PHP 5, PHP 7, PHP 8)

metaphone — Calculate the metaphone key of a string

Description

function **metaphone**(string `$string`, int `$max_phonemes = 0`): string

Calculates the metaphone key of `string`.

Similar to soundex() metaphone creates the same key for similar sounding words. It's more accurate than soundex() as it knows the basic rules of English pronunciation. The metaphone generated keys are of variable length.

Metaphone was developed by Lawrence Philips <lphilips at verity dot com>. It is described in ["Practical Algorithms for Programmers", Binstock & Rex, Addison Wesley, 1995].

Parameters

`string`

The input string.

`max_phonemes`

This parameter restricts the returned metaphone key to `max_phonemes` *characters* in length. However, the resulting phonemes are always transcribed completely, so the resulting string length may be slightly longer than `max_phonemes`. The default value of `0` means no restriction.

Return Values

Returns the metaphone key as a string.

Changelog

Version	Description
8.0.0	The function returned `false` on failure.

Examples

Example #1 metaphone() basic example

```php
<?php
var_dump(metaphone('programming'));
var_dump(metaphone('programmer'));
?>
```

The above example will output:

```
string(7) "PRKRMNK"
string(6) "PRKRMR"
```

Example #2 Using the `max_phonemes` parameter

```php
<?php
var_dump(metaphone('programming', 5));
var_dump(metaphone('programmer', 5));
?>
```

The above example will output:

```
string(5) "PRKRM"
string(5) "PRKRM"
```

Example #3 Using the `max_phonemes` parameter

In this example, **metaphone()** is advised to produce a string of five characters, but that would require to split the final phoneme (`'x'` is supposed to be transcribed to `'KS'`), so the function returns a string with six characters.

```php
<?php
var_dump(metaphone('Asterix', 5));
?>
```

The above example will output:

```
string(6) "ASTRKS"
```

See Also

- levenshtein() - Calculate Levenshtein distance between two strings
- similar_text() - Calculate the similarity between two strings
- soundex() - Calculate the soundex key of a string

money_format

money_format

(PHP 4 >= 4.3.0, PHP 5, PHP 7)

money_format — Formats a number as a currency string

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.4.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **money_format**(string `$format`, float `$number`): string

money_format() returns a formatted version of `number`. This function wraps the C library function **strfmon()**, with the difference that this implementation converts only one number at a time.

Parameters

`format`

The format specification consists of the following sequence:

- a `%` character
- optional flags
- optional field width
- optional left precision
- optional right precision
- a required conversion character

Flags

One or more of the optional flags below can be used:

=f

The character = followed by a (single byte) character f to be used as the numeric fill character. The default fill character is space.

^

Disable the use of grouping characters (as defined by the current locale).

+ or (

Specify the formatting style for positive and negative numbers. If + is used, the locale's equivalent for + and - will be used. If (is used, negative amounts are enclosed in parenthesis. If no specification is given, the default is +.

!

Suppress the currency symbol from the output string.

-

If present, it will make all fields left-justified (padded to the right), as opposed to the default which is for the fields to be right-justified (padded to the left).

Field width

w

A decimal digit string specifying a minimum field width. Field will be right-justified unless the flag `-` is used. Default value is 0 (zero).

Left precision

#n

The maximum number of digits (n) expected to the left of the decimal character (e.g. the decimal point). It is used usually to keep formatted output aligned in the same columns, using the fill character if the number of digits is less than n. If the number of actual digits is bigger than n, then this specification is ignored.

If grouping has not been suppressed using the `^` flag, grouping separators will be inserted before the fill characters (if any) are added. Grouping separators will not be applied to fill characters, even if the fill character is a digit.

To ensure alignment, any characters appearing before or after the number in the formatted output such as currency or sign symbols are padded as necessary with space characters to make their positive and negative formats an equal length.

Right precision

.p

A period followed by the number of digits (p) after the decimal character. If the value of p is 0 (zero), the decimal character and the digits to its right will be omitted. If no right precision is included, the default will dictated by the current locale in use. The amount being formatted is rounded to the specified number of digits prior to formatting.

Conversion characters

`i`

The number is formatted according to the locale's international currency format (e.g. for the USA locale: USD 1,234.56).

`n`

The number is formatted according to the locale's national currency format (e.g. for the de_DE locale: EU1.234,56).

`%`

Returns the `%` character.

`number`

The number to be formatted.

Return Values

Returns the formatted string. Characters before and after the formatting string will be returned unchanged. Non-numeric `number` causes returning `null` and emitting `E_WARNING`.

Changelog

Version	Description
8.0.0	Removed this function.
7.4.0	Deprecated this function. Instead, use NumberFormatter::formatCurrency().

Examples

Example #1 **money_format()** Example

We will use different locales and format specifications to illustrate the use of this function.

```
<?php
```

```php
$number = 1234.56;

// let's print the international format for the en_US locale
setlocale(LC_MONETARY, 'en_US');
echo money_format('%i', $number) . "\n";
// USD 1,234.56

// Italian national format with 2 decimals`
setlocale(LC_MONETARY, 'it_IT');
echo money_format('%.2n', $number) . "\n";
// Eu 1.234,56

// Using a negative number
$number = -1234.5672;

// US national format, using () for negative numbers
// and 10 digits for left precision
setlocale(LC_MONETARY, 'en_US');
echo money_format('%(#10n', $number) . "\n";
// ($        1,234.57)

// Similar format as above, adding the use of 2 digits of right
// precision and '*' as a fill character
echo money_format('%=*(#10.2n', $number) . "\n";
// ($********1,234.57)

// Let's justify to the left, with 14 positions of width, 8 digits of
// left precision, 2 of right precision, without the grouping character
// and using the international format for the de_DE locale.
setlocale(LC_MONETARY, 'de_DE');
echo money_format('%=*^-14#8.2i', 1234.56) . "\n";
// Eu 1234,56****

// Let's add some blurb before and after the conversion specification
setlocale(LC_MONETARY, 'en_GB');
$fmt = 'The final value is %i (after a 10%% discount)';
echo money_format($fmt, 1234.56) . "\n";
// The final value is  GBP 1,234.56 (after a 10% discount)

?>
```

Notes

> **Note** :
>
> The function **money_format()** is only defined if the system has strfmon capabilities. For example, Windows does not, so **money_format()** is undefined in Windows.

> **Note** :
>
> The `LC_MONETARY` category of the locale settings, affects the behavior of this function. Use setlocale() to set to the appropriate default locale before using this function.

See Also

- setlocale() - Set locale information
- sscanf() - Parses input from a string according to a format
- sprintf() - Return a formatted string
- printf() - Output a formatted string
- number_format() - Format a number with grouped thousands

nl_langinfo

nl_langinfo

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

nl_langinfo — Query language and locale information

Description

function **nl_langinfo**(int `$item`): string|false

nl_langinfo() is used to access individual elements of the locale categories. Unlike localeconv(), which returns all of the elements, **nl_langinfo()** allows you to select any specific element.

Parameters

`item`

`item` may be an integer value of the element or the constant name of the element. The following is a list of constant names for `item` that may be used and their description. Some of these constants may not be defined or hold no value for certain locales.

nl_langinfo Constants

Constant	Description
LC_TIME Category Constants	
ABDAY_(1-7)	Abbreviated name of n-th day of the week.
DAY_(1-7)	Name of the n-th day of the week (DAY_1 = Sunday).
ABMON_(1-12)	Abbreviated name of the n-th month of the year.
MON_(1-12)	Name of the n-th month of the year.
AM_STR	String for Ante meridian.
PM_STR	String for Post meridian.
D_T_FMT	String that can be used as the format string for strftime() to represent time and date.
D_FMT	String that can be used as the format string for strftime() to represent date.
T_FMT	String that can be used as the format string for strftime() to represent time.
T_FMT_AMPM	String that can be used as the format string for strftime() to represent time in 12-hour format with ante/post meridian.
ERA	Alternate era.
ERA_YEAR	Year in alternate era format.
ERA_D_T_FMT	Date and time in alternate era format (string can be used in strftime()).
ERA_D_FMT	Date in alternate era format (string can be used in strftime()).
ERA_T_FMT	Time in alternate era format (string can be used in strftime()).
LC_MONETARY Category Constants	
INT_CURR_SYMBOL	International currency symbol.
CURRENCY_SYMBOL	Local currency symbol.
CRNCYSTR	Same value as CURRENCY_SYMBOL.
MON_DECIMAL_POINT	Decimal point character.
MON_THOUSANDS_SEP	Thousands separator (groups of three digits).
MON_GROUPING	Like "grouping" element.
POSITIVE_SIGN	Sign for positive values.
NEGATIVE_SIGN	Sign for negative values.
INT_FRAC_DIGITS	International fractional digits.
FRAC_DIGITS	Local fractional digits.
P_CS_PRECEDES	Returns 1 if CURRENCY_SYMBOL precedes a positive value.
P_SEP_BY_SPACE	Returns 1 if a space separates CURRENCY_SYMBOL from a positive value.
N_CS_PRECEDES	Returns 1 if CURRENCY_SYMBOL precedes a negative value.
N_SEP_BY_SPACE	Returns 1 if a space separates CURRENCY_SYMBOL from a negative value.
P_SIGN_POSN	• Returns 0 if parentheses surround the quantity and CURRENCY_SYMBOL.

Constant	Description
	• Returns 1 if the sign string precedes the quantity and CURRENCY_SYMBOL. • Returns 2 if the sign string follows the quantity and CURRENCY_SYMBOL. • Returns 3 if the sign string immediately precedes the CURRENCY_SYMBOL. • Returns 4 if the sign string immediately follows the CURRENCY_SYMBOL.
N_SIGN_POSN	
LC_NUMERIC Category Constants	
DECIMAL_POINT	Decimal point character.
RADIXCHAR	Same value as DECIMAL_POINT.
THOUSANDS_SEP	Separator character for thousands (groups of three digits).
THOUSEP	Same value as THOUSANDS_SEP.
GROUPING	
LC_MESSAGES Category Constants	
YESEXPR	Regex string for matching "yes" input.
NOEXPR	Regex string for matching "no" input.
YESSTR	Output string for "yes".
NOSTR	Output string for "no".
LC_CTYPE Category Constants	
CODESET	Return a string with the name of the character encoding.

Return Values

Returns the element as a string, or `false` if `item` is not valid.

Examples

Example #1 nl_langinfo() example

```
<?php

var_dump(nl_langinfo(CODESET));
var_dump(nl_langinfo(YESEXPR));
?>
```

The above example will output something similar to:

```
string(14) "ANSI_X3.4-1968"
string(5) "^[yY]"
```

Notes

> **Note** :
>
> This function is not implemented on Windows platforms.

See Also

- setlocale() - Set locale information
- localeconv() - Get numeric formatting information

nl2br

nl2br

(PHP 4, PHP 5, PHP 7, PHP 8)

nl2br — Inserts HTML line breaks before all newlines in a string

Description

function **nl2br**(string `$string`, bool `$use_xhtml = true`): string

Returns `string` with `<br />` or `<br>` inserted before all newlines (`\r\n`, `\n\r`, `\n` and `\r`).

Parameters

`string`

The input string.

`use_xhtml`

Whether to use XHTML compatible line breaks or not.

Return Values

Returns the altered string.

Examples

Example #1 Using nl2br()

```php
<?php
echo nl2br("foo isn't\n bar");
?>
```

The above example will output:

```
foo isn't<br />
 bar
```

Example #2 Generating valid HTML markup using the `use_xhtml` parameter

```php
<?php
echo nl2br("Welcome\r\nThis is my HTML document", false);
?>
```

The above example will output:

```
Welcome<br>
This is my HTML document
```

Example #3 Various newline separators

```php
<?php
$string = "This\r\nis\n\ra\nstring\r";
echo nl2br($string);
?>
```

The above example will output:

```
This<br />
is<br />
a<br />
string<br />
```

See Also

- htmlspecialchars() - Convert special characters to HTML entities
- htmlentities() - Convert all applicable characters to HTML entities
- wordwrap() - Wraps a string to a given number of characters
- str_replace() - Replace all occurrences of the search string with the replacement string

number_format

number_format

(PHP 4, PHP 5, PHP 7, PHP 8)

number_format — Format a number with grouped thousands

Description

function **number_format**(
  float $num,
  int $decimals = 0,
  ?string $decimal_separator = ".",
  ?string $thousands_separator = ","
): string

Formats a number with grouped thousands and optionally decimal digits using the rounding half up rule.

Parameters

`num`

The number being formatted.

`decimals`

Sets the number of decimal digits. If `0`, the `decimal_separator` is omitted from the return value. As of PHP 8.3.0, when the value is negative, `num` is rounded to `decimals` significant digits before the decimal point. Prior to PHP 8.3.0, negative values were ignored and handled the same as `0`.

`decimal_separator`

Sets the separator for the decimal point.

`thousands_separator`

Sets the thousands separator.

Return Values

A formatted version of `num`.

Changelog

Version	Description
8.3.0	Added handling of negative values for `decimals`.
8.0.0	Prior to this version, **number_format()** accepted one, two, or four parameters (but not three).
7.2.0	**number_format()** was changed to not being able to return `-0`, previously `-0` could be returned for cases like where `num` would be `-0.01`.

Examples

Example #1 number_format() Example

For instance, French notation usually use two decimals, comma (',') as decimal separator, and space (' ') as thousand separator. The following example demonstrates various ways to format a number:

```php
<?php

$number = 1234.56;

// english notation (default)
echo number_format($number), PHP_EOL;
// 1,235

// French notation
echo number_format($number, 2, ',', ' '), PHP_EOL;
// 1 234,56

$number = 1234.5678;

// english notation without thousands separator
echo number_format($number, 2, '.', ''), PHP_EOL;
// 1234.57

?>
```

Example #2 A negative value for `decimals`

As of PHP 8.3.0, a negative value for `decimals` is used to round the number of significant digits before the decimal point.

```php
<?php
$number = "1234.5678";
var_dump(number_format($number, -1));
var_dump(number_format($number, -2));
var_dump(number_format($number, -3));
?>
```

The above example will output:

```
string(5) "1,230"
string(5) "1,200"
string(5) "1,000"
```

See Also

- money_format() - Formats a number as a currency string
- sprintf() - Return a formatted string
- printf() - Output a formatted string
- sscanf() - Parses input from a string according to a format

ord

ord

(PHP 4, PHP 5, PHP 7, PHP 8)

ord — Convert the first byte of a string to a value between 0 and 255

Description

function **ord**(string `$character`): int

Interprets the binary value of the first byte of `character` as an unsigned integer between 0 and 255.

If the string is in a single-byte encoding, such as ASCII, ISO-8859, or Windows 1252, this is equivalent to returning the position of a character in the character set's mapping table. However, note that this function is not aware of any string encoding, and in particular will never identify a Unicode code point in a multi-byte encoding such as UTF-8 or UTF-16.

This function complements chr().

Parameters

`character`

A character.

Return Values

An integer between 0 and 255.

Changelog

Version	Description
8.5.0	Passing a string which is not a single byte is now deprecated.

Examples

Example #1 ord() example

```php
<?php
$str = "\n";
if (ord($str) == 10) {
    echo "The first character of \$str is a line feed.\n";
}
?>
```

Example #2 Examining the individual bytes of a UTF-8 string

```php
<?php
$str = "🐘";
for ( $pos=0; $pos < strlen($str); $pos ++ ) {
 $byte = substr($str, $pos);
 echo 'Byte ' . $pos . ' of $str has value ' . ord($byte) . PHP_EOL;
}
?>
```

The above example will output:

```
Byte 0 of $str has value 240

Byte 1 of $str has value 159

Byte 2 of $str has value 144

Byte 3 of $str has value 152
```

See Also

- chr() - Generate a single-byte string from a number
- An » ASCII-table
- mb_ord() - Get Unicode code point of character
- IntlChar::ord() - Return Unicode code point value of character

parse_str

parse_str

(PHP 4, PHP 5, PHP 7, PHP 8)

parse_str — Parse a string as a URL query string

Description

function **parse_str**(string `$string`, array `&$result`): void

Parses `string` as if it were the query string passed via a URL and sets keys in the provided `result` array. If no `result` is passed, values are instead set as variables in the current scope.

Parameters

`string`

The input string.

`result`

A variable passed by reference, which will be set to an array containing the key-value pairs extracted from `string`. If the `result` parameter is not passed, a separate variable is set in the local scope for each key.

> **Warning**
>
> Using this function without the `result` parameter is highly *DISCOURAGED* and *DEPRECATED* as of PHP 7.2. As of PHP 8.0.0, the `result` parameter is *mandatory*.

Return Values

No value is returned.

Changelog

Version	Description
8.0.0	`result` is no longer optional.
7.2.0	Usage of **parse_str()** without a second parameter now emits an `E_DEPRECATED` notice.

Examples

Example #1 Using parse_str()

```php
<?php
$str = "first=value&arr[]=foo+bar&arr[]=baz";

// Recommended
parse_str($str, $output);
echo $output['first'], PHP_EOL;  // value
echo $output['arr'][0], PHP_EOL; // foo bar
echo $output['arr'][1], PHP_EOL; // baz
?>
```

Any spaces and dots in parameter names are converted to underscores when creating array keys or local variables. This is because variable names in PHP are not allowed to contain spaces or dots, but applies even when using this function with the recommended `result` parameter.

Example #2 parse_str() name mangling

```php
<?php
parse_str("My Value=Something", $output);
echo $output['My_Value']; // Something
?>
```

Notes

> **Note** :

> **parse_str()** is affected by the max_input_vars directive. Exceeding this limit triggers an `E_WARNING`, and any variables beyond the limit are not added to the result array. The default is 1000; adjust max_input_vars as needed.

> **Note** :
>
> All values populated in the `result` array (or variables created if second parameter is not set) are already URL-decoded using the same rules as urldecode().

> **Note** :
>
> To get the query string of the current request, you may use the variable *$_SERVER['QUERY_STRING']*. Also, you may want to read the section on variables from external sources.

See Also

- parse_url() - Parse a URL and return its components
- pathinfo() - Returns information about a file path
- http_build_query() - Generate URL-encoded query string
- urldecode() - Decodes URL-encoded string

print

print

(PHP 4, PHP 5, PHP 7, PHP 8)

print — Output a string

Description

function **print**(string `$expression`): int

Outputs `expression`.

`print` is not a function but a language construct. Its argument is the expression following the `print` keyword, and is not delimited by parentheses.

The major differences to echo are that `print` only accepts a single argument and always returns `1`.

Parameters

`expression`

The expression to be output. Non-string values will be coerced to strings, even when the `strict_types` directive is enabled.

Return Values

Returns `1`, always.

Examples

Example #1 `print` examples

```php
<?php
print "print does not require parentheses.";
print PHP_EOL;

// No newline or space is added; the below outputs "helloworld" all on one line
print "hello";
print "world";
print PHP_EOL;

print "This string spans
multiple lines. The newlines will be
output as well";
print PHP_EOL;
```

```php
print "This string spans\nmultiple lines. The newlines will be\noutput as well.";
print PHP_EOL;

// The argument can be any expression which produces a string
$foo = "example";
print "foo is $foo"; // foo is example
print PHP_EOL;

$fruits = ["lemon", "orange", "banana"];
print implode(" and ", $fruits); // lemon and orange and banana
print PHP_EOL;

// Non-string expressions are coerced to string, even if declare(strict_types=1) is used
print 6 * 7; // 42
print PHP_EOL;

// Because print has a return value, it can be used in expressions
// The following outputs "hello world"
if ( print "hello" ) {
    echo " world";
}
print PHP_EOL;

// The following outputs "true"
( 1 === 1 ) ? print 'true' : print 'false';
print PHP_EOL;
?>
```

Notes

> **Note** : **Using with parentheses**
>
> Surrounding the argument to `print` with parentheses will not raise a syntax error, and produces syntax which looks like a normal function call. However, this can be misleading, because the parentheses are actually part of the expression being output, not part of the `print` syntax itself.

```php
<?php
print "hello";
// outputs "hello"

print("hello");
// also outputs "hello", because ("hello") is a valid expression

print(1 + 2) * 3;
// outputs "9"; the parentheses cause 1+2 to be evaluated first, then 3*3
// the print statement sees the whole expression as one argument

if ( print("hello") && false ) {
    print " - inside if";
}
else {
    print " - inside else";
}
// outputs " - inside if"
// the expression ("hello") && false is first evaluated, giving false
// this is coerced to the empty string "" and printed
// the print construct then returns 1, so code in the if block is run
?>
```

When using `print` in a larger expression, placing both the keyword and its argument in parentheses may be necessary to give the intended result:

```php
<?php
if ( (print "hello") && false ) {
    print " - inside if";
}
else {
    print " - inside else";
}
```

```php
// outputs "hello - inside else"
// unlike the previous example, the expression (print "hello") is evaluated first
// after outputting "hello", print returns 1
// since 1 && false is false, code in the else block is run

print "hello " && print "world";
// outputs "world1"; print "world" is evaluated first,
// then the expression "hello " && 1 is passed to the left-hand print

(print "hello ") && (print "world");
// outputs "hello world"; the parentheses force the print expressions
// to be evaluated before the &&
?>
```

> **Note** :
>
> Because this is a language construct and not a function, it cannot be called using variable functions, or named arguments.

See Also

- echo - Output one or more strings
- printf() - Output a formatted string
- flush() - Flush system output buffer
- Ways to specify literal strings

printf

printf

(PHP 4, PHP 5, PHP 7, PHP 8)

printf — Output a formatted string

Description

function **printf**(string `$format`, mixed `...$values`): int

Produces output according to `format`.

Parameters

`format`

The format string is composed of zero or more directives: ordinary characters (excluding `%`) that are copied directly to the result and *conversion specifications*, each of which results in fetching its own parameter.

A conversion specification follows this prototype: `%[argnum$][flags][width][.precision]specifier`.

Argnum

An integer followed by a dollar sign `$`, to specify which number argument to treat in the conversion.

Flags

Flag	Description
-	Left-justify within the given field width; Right justification is the default
+	Prefix positive numbers with a plus sign +; Default only negative are prefixed with a negative sign.
(space)	Pads the result with spaces. This is the default.
`0`	Only left-pads numbers with zeros. With `s` specifiers this can also right-pad with zeros.
'(char)	Pads the result with the character (char).

Width

Either an integer that says how many characters (minimum) this conversion should result in, or `*`. If `*` is used, then the width is supplied as an additional integer value preceding the one formatted by the specifier.

Precision

A period `.` optionally followed by either an integer or `*`, whose meaning depends on the specifier:

- For `e`, `E`, `f` and `F` specifiers: this is the number of digits to be printed after the decimal point (by default, this is 6).
- For `g`, `G`, `h` and `H` specifiers: this is the maximum number of significant digits to be printed.
- For `s` specifier: it acts as a cutoff point, setting a maximum character limit to the string.

> **Note** :
>
> If the period is specified without an explicit value for precision, 0 is assumed. If `*` is used, the precision is supplied as an additional integer value preceding the one formatted by the specifier.

Specifiers

Specifier	Description
`%`	A literal percent character. No argument is required.
`b`	The argument is treated as an integer and presented as a binary number.
`c`	The argument is treated as an integer and presented as the character with that ASCII.
`d`	The argument is treated as an integer and presented as a (signed) decimal number.
`e`	The argument is treated as scientific notation (e.g. 1.2e+2).
`E`	Like the `e` specifier but uses uppercase letter (e.g. 1.2E+2).
`f`	The argument is treated as a float and presented as a floating-point number (locale aware).
`F`	The argument is treated as a float and presented as a floating-point number (non-locale aware).
`g`	General format. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of X: If P > X ≥ −4, the conversion is with style f and precision P − (X + 1). Otherwise, the conversion is with style e and precision P − 1.
`G`	Like the `g` specifier but uses `E` and `f`.
`h`	Like the `g` specifier but uses `F`. Available as of PHP 8.0.0.
`H`	Like the `g` specifier but uses `E` and `F`. Available as of PHP 8.0.0.
`o`	The argument is treated as an integer and presented as an octal number.
`s`	The argument is treated and presented as a string.
`u`	The argument is treated as an integer and presented as an unsigned decimal number.
`x`	The argument is treated as an integer and presented as a hexadecimal number (with lowercase letters).
`X`	The argument is treated as an integer and presented as a hexadecimal number (with uppercase letters).

> **Warning**
>
> The `c` type specifier ignores padding and width.

> **Warning**
>
> Attempting to use a combination of the string and width specifiers with character sets that require more than one byte per character may result in unexpected results.

Variables will be co-erced to a suitable type for the specifier:

Type Handling

Type	Specifiers
string	`s`
int	`d`, `u`, `c`, `o`, `x`, `X`, `b`
float	`e`, `E`, `f`, `F`, `g`, `G`, `h`, `H`

`values`

Return Values

Returns the length of the outputted string.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the number of arguments is zero. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[width]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[precision]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ArgumentCountError is thrown when less arguments are given than required. Prior to PHP 8.0.0, `false` was returned and a `E_WARNING` emitted instead.

Changelog

Version	Description
8.0.0	This function no longer returns `false` on failure.
8.0.0	Throw a ValueError if the number of arguments is zero; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[width]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[precision]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ArgumentCountError when less arguments are given than required; previously this function emitted a `E_WARNING` instead.

Examples

Example #1 printf(): various examples

```php
<?php
$n =  43951789;
$u = -43951789;
$c = 65; // ASCII 65 is 'A'

// notice the double %%, this prints a literal '%' character
printf("%%b = '%b'\n", $n); // binary representation
printf("%%c = '%c'\n", $c); // print the ascii character, same as chr() function
printf("%%d = '%d'\n", $n); // standard integer representation
printf("%%e = '%e'\n", $n); // scientific notation
printf("%%u = '%u'\n", $n); // unsigned integer representation of a positive integer
printf("%%u = '%u'\n", $u); // unsigned integer representation of a negative integer
printf("%%f = '%f'\n", $n); // floating point representation
printf("%%o = '%o'\n", $n); // octal representation
printf("%%s = '%s'\n", $n); // string representation
printf("%%x = '%x'\n", $n); // hexadecimal representation (lower-case)
printf("%%X = '%X'\n", $n); // hexadecimal representation (upper-case)

printf("%%+d = '%+d'\n", $n); // sign specifier on a positive integer
printf("%%+d = '%+d'\n", $u); // sign specifier on a negative integer
?>
```

The above example will output:

```
%b = '10100111101010011010101101'
%c = 'A'
%d = '43951789'
%e = '4.39518e+7'
%u = '43951789'
%u = '4251015507'
%f = '43951789.000000'
%o = '247523255'
%s = '43951789'
```

```
%x = '29ea6ad'
%X = '29EA6AD'
%+d = '+43951789'
%+d = '-43951789'
```

Example #2 printf(): string specifiers

```php
<?php
$s = 'monkey';
$t = 'many monkeys';

printf("[%s]\n",        $s); // standard string output
printf("[%10s]\n",      $s); // right-justification with spaces
printf("[%-10s]\n",     $s); // left-justification with spaces
printf("[%010s]\n",     $s); // zero-padding works on strings too
printf("[%'#10s]\n",    $s); // use the custom padding character '#'
printf("[%'#*s]\n", 10, $s); // Provide the padding width as an additional argument
printf("[%10.9s]\n",    $t); // right-justification but with a cutoff of 9 characters
printf("[%-10.9s]\n",   $t); // left-justification but with a cutoff of 9 characters
?>
```

The above example will output:

```
[monkey]
[    monkey]
[monkey    ]
[0000monkey]
[####monkey]
[####monkey]
[ many monk]
[many monk ]
```

See Also

- print - Output a string
- sprintf() - Return a formatted string
- fprintf() - Write a formatted string to a stream
- vprintf() - Output a formatted string
- vsprintf() - Return a formatted string
- vfprintf() - Write a formatted string to a stream
- sscanf() - Parses input from a string according to a format
- fscanf() - Parses input from a file according to a format
- number_format() - Format a number with grouped thousands
- date() - Format a Unix timestamp
- flush() - Flush system output buffer

quoted_printable_decode

quoted_printable_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

quoted_printable_decode — Convert a quoted-printable string to an 8 bit string

Description

function **quoted_printable_decode**(string `$string`): string

This function returns an 8-bit binary string corresponding to the decoded quoted printable string (according to » RFC2045, section 6.7, not » RFC2821, section 4.5.2, so additional periods are not stripped from the beginning of a line).

This function is similar to imap_qprint(), except this one does not require the IMAP module to work.

Parameters

`string`

The input string.

Return Values

Returns the 8-bit binary string.

Examples

Example #1 quoted_printable_decode() example

```php
<?php

$encoded = quoted_printable_encode('Möchten Sie ein paar Äpfel?');

var_dump($encoded);
var_dump(quoted_printable_decode($encoded));
?>
```

The above example will output:

```
string(37) "M=C3=B6chten Sie ein paar =C3=84pfel?"
string(29) "Möchten Sie ein paar Äpfel?"
```

See Also

- quoted_printable_encode() - Convert a 8 bit string to a quoted-printable string

quoted_printable_encode

quoted_printable_encode

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

quoted_printable_encode — Convert a 8 bit string to a quoted-printable string

Description

function **quoted_printable_encode**(string `$string`): string

Returns a quoted printable string created according to » RFC2045, section 6.7.

This function is similar to imap_8bit(), except this one does not require the IMAP module to work.

Parameters

`string`

The input string.

Return Values

Returns the encoded string.

Examples

Example #1 quoted_printable_encode() example

```php
<?php
```

```php
$encoded = quoted_printable_encode('Möchten Sie ein paar Äpfel?');

var_dump($encoded);
var_dump(quoted_printable_decode($encoded));
?>
```

The above example will output:

```
string(37) "M=C3=B6chten Sie ein paar =C3=84pfel?"
string(29) "Möchten Sie ein paar Äpfel?"
```

See Also

- quoted_printable_decode() - Convert a quoted-printable string to an 8 bit string
- iconv_mime_encode() - Composes a MIME header field

quotemeta

quotemeta

(PHP 4, PHP 5, PHP 7, PHP 8)

quotemeta — Quote meta characters

Description

function **quotemeta**(string `$string`): string

Returns a version of str with a backslash character (\) before every character that is among these:

```
. \ + * ? [ ^ ] ( $ )
```

Parameters

`string`

The input string.

Return Values

Returns the string with meta characters quoted, or `false` if an empty string is given as `string`.

Examples

Example #1 quotemeta() example

```php
<?php

var_dump(quotemeta('PHP is a popular scripting language. Fast, flexible, and pragmatic.'));
?>
```

The above example will output:

```
string(69) "PHP is a popular scripting language\. Fast, flexible, and pragmatic\."
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- addslashes() - Quote string with slashes
- addcslashes() - Quote string with slashes in a C style
- htmlentities() - Convert all applicable characters to HTML entities
- htmlspecialchars() - Convert special characters to HTML entities
- nl2br() - Inserts HTML line breaks before all newlines in a string
- stripslashes() - Un-quotes a quoted string
- stripcslashes() - Un-quote string quoted with addcslashes
- preg_quote() - Quote regular expression characters

rtrim

rtrim

(PHP 4, PHP 5, PHP 7, PHP 8)

rtrim — Strip whitespace (or other characters) from the end of a string

Description

function **rtrim**(string `$string`, string `$characters` = " \n\r\t\v\x00"): string

This function returns a string with whitespace (or other characters) stripped from the end of `string`.

Without the second parameter, **rtrim()** will strip these characters:

- `" "`: ASCII SP character `0x20`, an ordinary space.
- `"\t"`: ASCII HT character `0x09`, a tab.
- `"\n"`: ASCII LF character `0x0A`, a new line (line feed).
- `"\r"`: ASCII CR character `0x0D`, a carriage return.
- `"\0"`: ASCII NUL character `0x00`, the NUL-byte.
- `"\v"`: ASCII VT character `0x0B`, a vertical tab.

Parameters

`string`

The input string.

`characters`

Optionally, the stripped characters can also be specified using the `characters` parameter. Simply list all characters that need to be stripped. With `..` it is possible to specify an incrementing range of characters.

Return Values

Returns the modified string.

Examples

Example #1 Usage example of rtrim()

```php
<?php

$text = "\t\tThese are a few words :) ...  ";
$binary = "\x09Example string\x0A";
$hello  = "Hello World";
var_dump($text, $binary, $hello);
```

```php
print "\n";

$trimmed = rtrim($text);
var_dump($trimmed);

$trimmed = rtrim($text, " \t.");
var_dump($trimmed);

$trimmed = rtrim($hello, "Hdle");
var_dump($trimmed);

// trim the ASCII control characters at the end of $binary
// (from 0 to 31 inclusive)
$clean = rtrim($binary, "\x00..\x1F");
var_dump($clean);

?>
```

The above example will output:

```
string(32) "            These are a few words :) ...  "
string(16) "    Example string
"
string(11) "Hello World"

string(30) "            These are a few words :) ..."
string(26) "            These are a few words :)"
string(9) "Hello Wor"
string(15) "    Example string"
```

See Also

- trim() - Strip whitespace (or other characters) from the beginning and end of a string
- ltrim() - Strip whitespace (or other characters) from the beginning of a string

setlocale

setlocale

(PHP 4, PHP 5, PHP 7, PHP 8)

setlocale — Set locale information

Description

function **setlocale**(int `$category`, ?string `$locales`, string `...$rest`): string|false

Alternative signature (not supported with named arguments):

function **setlocale**(int `$category`, array `$locale_array`): string|false

Sets locale information.

> **Warning**
>
> The locale information is maintained per process, not per thread. If you are running PHP on a multithreaded server API, you may experience sudden changes in locale settings while a script is running, though the script itself never called **setlocale()**. This happens due to other scripts running in different threads of the same process at the same time, changing the process-wide locale using **setlocale()**. On Windows, locale information is maintained per thread as of PHP 7.0.5.

Parameters

`category`

`category` is a named constant specifying the category of the functions affected by the locale setting:

- `LC_ALL` for all of the below
- `LC_COLLATE` for string comparison, see strcoll()
- `LC_CTYPE` for character classification and conversion, for example ctype_alpha()
- `LC_MONETARY` for localeconv()
- `LC_NUMERIC` for decimal separator (See also localeconv())
- `LC_TIME` for date and time formatting with strftime()
- `LC_MESSAGES` for system responses (available if PHP was compiled with `libintl`)

`locales`

If `locales` is the empty string `""` or is `null`, the locale names will be set from the values of environment variables with the same names as the above categories, or from "LANG".

If `locales` is `"0"`, the locale setting is not affected, only the current setting is returned.

If `locales` is followed by additional parameters then each parameter is tried to be set as new locale until success. This is useful if a locale is known under different names on different systems or for providing a fallback for a possibly not available locale.

`rest`

Optional string parameters to try as locale settings until success.

`locale_array`

Each array element is tried to be set as new locale until success. This is useful if a locale is known under different names on different systems or for providing a fallback for a possibly not available locale.

> **Note** :
>
> On Windows, setlocale(LC_ALL, '') sets the locale names from the system's regional/language settings (accessible via Control Panel).

Return Values

Returns the new current locale, or `false` if the locale functionality is not implemented on your platform, the specified locale does not exist or the category name is invalid.

An invalid category name also causes a warning message. Category/locale names can be found in » RFC 1766 and » ISO 639. Different systems have different naming schemes for locales.

> **Note** :
>
> The return value of **setlocale()** depends on the system that PHP is running. It returns exactly what the system `setlocale` function returns.

Changelog

Version	Description
8.5.0	Passing an integer `0` as the `locales` argument is no longer supported and now throws a TypeError.

Examples

Example #1 setlocale() Examples

```
<?php
/* Set locale to Dutch */
setlocale(LC_ALL, 'nl_NL');

/* Output: vrijdag 22 december 1978 */
```

```
echo strftime("%A %e %B %Y", mktime(0, 0, 0, 12, 22, 1978));

/* try different possible locale names for german */
$loc_de = setlocale(LC_ALL, 'de_DE@euro', 'de_DE', 'de', 'ge');
echo "Preferred locale for german on this system is '$loc_de'";
?>
```

Example #2 setlocale() Retrieve current setting

```
<?php
/* Retrieve current setting */
$current = setlocale(LC_ALL, null);

echo "Current locale '$current'";
?>
```

Example #3 setlocale() Examples for Windows

```
<?php
/* Set locale to Dutch */
setlocale(LC_ALL, 'nld_nld');

/* Output: vrijdag 22 december 1978 */
echo strftime("%A %d %B %Y", mktime(0, 0, 0, 12, 22, 1978));

/* try different possible locale names for german */
$loc_de = setlocale(LC_ALL, 'de_DE@euro', 'de_DE', 'deu_deu');
echo "Preferred locale for german on this system is '$loc_de'";
?>
```

Notes

> **Tip**
>
> Windows users will find useful information about `locales` strings at Microsoft's MSDN website. Supported language strings can be found in the » language strings documentation and supported country/region strings in the » country/region strings documentation.

sha1

sha1

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

sha1 — Calculate the sha1 hash of a string

> **Warning**
>
> It is not recommended to use this function to secure passwords, due to the fast nature of this hashing algorithm. See the Password Hashing FAQ for details and best practices.

Description

function **sha1**(string `$string`, bool `$binary` = `false`): string

Calculates the sha1 hash of `string` using the » US Secure Hash Algorithm 1.

Parameters

`string`

The input string.

`binary`

If the optional `binary` is set to `true`, then the sha1 digest is instead returned in raw binary format with a length of 20, otherwise the returned value is a 40-character hexadecimal number.

Return Values

Returns the sha1 hash as a string.

Examples

Example #1 A sha1() example

```php
<?php
$str = 'apple';

if (sha1($str) === 'd0be2dc421be4fcd0172e5afceea3970e2f3d940') {
    echo "Would you like a green or red apple?";
}
?>
```

See Also

- hash() - Generate a hash value (message digest)
- password_hash() - Creates a password hash

sha1_file

sha1_file

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

sha1_file — Calculate the sha1 hash of a file

Description

function **sha1_file**(string `$filename`, bool `$binary` = **`false`**): string|false

Calculates the sha1 hash of the file specified by `filename` using the » US Secure Hash Algorithm 1, and returns that hash. The hash is a 40-character hexadecimal number.

Parameters

`filename`

The filename of the file to hash.

`binary`

When **`true`**, returns the digest in raw binary format with a length of 20.

Return Values

Returns a string on success, **`false`** otherwise.

Examples

Example #1 sha1_file() example

```php
<?php
foreach (glob('/examples/*.xml') as $ent)
{
    if (is_dir($ent)) {
        continue;
    }

    echo $ent . ' (SHA1: ' . sha1_file($ent) . ')', PHP_EOL;
}
?>
```

See Also

- hash_file() - Generate a hash value using the contents of a given file
- hash_init() - Initialize an incremental hashing context
- sha1() - Calculate the sha1 hash of a string

similar_text

similar_text

(PHP 4, PHP 5, PHP 7, PHP 8)

similar_text — Calculate the similarity between two strings

Description

function **similar_text**(string `$string1`, string `$string2`, float `&$percent = null`): int

This calculates the similarity between two strings as described in Programming Classics: Implementing the World's Best Algorithms by Oliver (ISBN 0-131-00413-1). Note that this implementation does not use a stack as in Oliver's pseudo code, but recursive calls which may or may not speed up the whole process. Note also that the complexity of this algorithm is O(N**3) where N is the length of the longest string.

Parameters

`string1`

The first string.

`string2`

The second string.

> **Note** :
>
> Swapping the `string1` and `string2` may yield a different result; see the example below.

`percent`

By passing a reference as third argument, **similar_text()** will calculate the similarity in percent, by dividing the result of **similar_text()** by the average of the lengths of the given strings times `100`.

Return Values

Returns the number of matching chars in both strings.

The number of matching characters is calculated by finding the longest first common substring, and then doing this for the prefixes and the suffixes, recursively. The lengths of all found common substrings are added.

Examples

Example #1 similar_text() argument swapping example

This example shows that swapping the `string1` and `string2` argument may yield different results.

```php
<?php
$sim = similar_text('bafoobar', 'barfoo', $perc);
echo "similarity: $sim ($perc %)\n";
$sim = similar_text('barfoo', 'bafoobar', $perc);
echo "similarity: $sim ($perc %)\n";
```

The above example will output something similar to:

```
similarity: 5 (71.428571428571 %)
similarity: 3 (42.857142857143 %)
```

See Also

- levenshtein() - Calculate Levenshtein distance between two strings
- metaphone() - Calculate the metaphone key of a string
- soundex() - Calculate the soundex key of a string

soundex

soundex

(PHP 4, PHP 5, PHP 7, PHP 8)

soundex — Calculate the soundex key of a string

Description

function **soundex**(string `$string`): string

Calculates the soundex key of `string`.

Soundex keys have the property that words pronounced similarly produce the same soundex key, and can thus be used to simplify searches in databases where you know the pronunciation but not the spelling.

This particular soundex function is one described by Donald Knuth in "The Art Of Computer Programming, vol. 3: Sorting And Searching", Addison-Wesley (1973), pp. 391-392.

Parameters

`string`

The input string.

Return Values

Returns the soundex key as a string with four characters. If at least one letter is contained in `string`, the returned string starts with a letter. Otherwise `"0000"` is returned.

Changelog

Version	Description
8.0.0	Prior to this version, calling the function with an empty string returned `false` for no particular reason.

Examples

Example #1 Soundex Examples

```php
<?php
echo soundex("Euler"), PHP_EOL, soundex("Ellery"), PHP_EOL;

soundex("Euler")       == soundex("Ellery");    // E460
soundex("Gauss")       == soundex("Ghosh");     // G200
soundex("Hilbert")     == soundex("Heilbronn"); // H416
soundex("Knuth")       == soundex("Kant");      // K530
soundex("Lloyd")       == soundex("Ladd");      // L300
soundex("Lukasiewicz") == soundex("Lissajous"); // L222
?>
```

See Also

- levenshtein() - Calculate Levenshtein distance between two strings
- metaphone() - Calculate the metaphone key of a string
- similar_text() - Calculate the similarity between two strings

sprintf

sprintf

(PHP 4, PHP 5, PHP 7, PHP 8)

sprintf — Return a formatted string

Description

function **sprintf**(string `$format`, mixed `...$values`): string

Returns a string produced according to the formatting string `format`.

Parameters

`format`

The format string is composed of zero or more directives: ordinary characters (excluding `%`) that are copied directly to the result and *conversion specifications*, each of which results in fetching its own parameter.

A conversion specification follows this prototype: `%[argnum$][flags][width][.precision]specifier`.

Argnum

An integer followed by a dollar sign `$`, to specify which number argument to treat in the conversion.

Flags

Flag	Description
`-`	Left-justify within the given field width; Right justification is the default
`+`	Prefix positive numbers with a plus sign `+`; Default only negative are prefixed with a negative sign.
(space)	Pads the result with spaces. This is the default.
`0`	Only left-pads numbers with zeros. With `s` specifiers this can also right-pad with zeros.
`'`(char)	Pads the result with the character (char).

Width

Either an integer that says how many characters (minimum) this conversion should result in, or `*`. If `*` is used, then the width is supplied as an additional integer value preceding the one formatted by the specifier.

Precision

A period `.` optionally followed by either an integer or `*`, whose meaning depends on the specifier:

- For `e`, `E`, `f` and `F` specifiers: this is the number of digits to be printed after the decimal point (by default, this is 6).
- For `g`, `G`, `h` and `H` specifiers: this is the maximum number of significant digits to be printed.
- For `s` specifier: it acts as a cutoff point, setting a maximum character limit to the string.

> **Note** :
>
> If the period is specified without an explicit value for precision, 0 is assumed. If `*` is used, the precision is supplied as an additional integer value preceding the one formatted by the specifier.

Specifiers

Specifier	Description
`%`	A literal percent character. No argument is required.
`b`	The argument is treated as an integer and presented as a binary number.
`c`	The argument is treated as an integer and presented as the character with that ASCII.
`d`	The argument is treated as an integer and presented as a (signed) decimal number.
`e`	The argument is treated as scientific notation (e.g. 1.2e+2).
`E`	Like the `e` specifier but uses uppercase letter (e.g. 1.2E+2).
`f`	The argument is treated as a float and presented as a floating-point number (locale aware).
`F`	The argument is treated as a float and presented as a floating-point number (non-locale aware).
`g`	General format. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of X: If P > X ≥ −4, the conversion is with style f and precision P − (X + 1). Otherwise, the conversion is with style e and precision P − 1.
`G`	Like the `g` specifier but uses `E` and `f`.
`h`	Like the `g` specifier but uses `F`. Available as of PHP 8.0.0.
`H`	Like the `g` specifier but uses `E` and `F`. Available as of PHP 8.0.0.

Specifier	Description
o	The argument is treated as an integer and presented as an octal number.
s	The argument is treated and presented as a string.
u	The argument is treated as an integer and presented as an unsigned decimal number.
x	The argument is treated as an integer and presented as a hexadecimal number (with lowercase letters).
X	The argument is treated as an integer and presented as a hexadecimal number (with uppercase letters).

> **Warning**
>
> The c type specifier ignores padding and width.

> **Warning**
>
> Attempting to use a combination of the string and width specifiers with character sets that require more than one byte per character may result in unexpected results.

Variables will be co-erced to a suitable type for the specifier:

Type Handling

Type	Specifiers
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Return Values

Returns a string produced according to the formatting string `format`.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the number of arguments is zero. Prior to PHP 8.0.0, a **`E_WARNING`** was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[width]` is less than zero or bigger than **`PHP_INT_MAX`**. Prior to PHP 8.0.0, a **`E_WARNING`** was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[precision]` is less than zero or bigger than **`PHP_INT_MAX`**. Prior to PHP 8.0.0, a **`E_WARNING`** was emitted instead.

As of PHP 8.0.0, a ArgumentCountError is thrown when less arguments are given than required. Prior to PHP 8.0.0, **`false`** was returned and a **`E_WARNING`** emitted instead.

Changelog

Version	Description
8.0.0	This function no longer returns **`false`** on failure.
8.0.0	Throw a ValueError if the number of arguments is zero; previously this function emitted a **`E_WARNING`** instead.
8.0.0	Throw a ValueError if `[width]` is less than zero or bigger than **`PHP_INT_MAX`**; previously this function emitted a **`E_WARNING`** instead.
8.0.0	Throw a ValueError if `[precision]` is less than zero or bigger than **`PHP_INT_MAX`**; previously this function emitted a **`E_WARNING`** instead.
8.0.0	Throw a ArgumentCountError when less arguments are given than required; previously this function emitted a **`E_WARNING`** instead.

Examples

Example #1 Argument swapping

The format string supports argument numbering/swapping.

```php
<?php
$num = 5;
$location = 'tree';

$format = 'There are %d monkeys in the %s';
echo sprintf($format, $num, $location);
?>
```

The above example will output:

```
There are 5 monkeys in the tree
```

However imagine we are creating a format string in a separate file, commonly because we would like to internationalize it and we rewrite it as:

Example #2 Wrong Argument Order

The format string supports argument numbering/swapping.

```php
<?php
$num = 5;
$location = 'tree';

$format = 'The %s contains %d monkeys';
echo sprintf($format, $num, $location);
?>
```

We now have a problem. The order of the placeholders in the format string does not match the order of the arguments in the code. We would like to leave the code as is and simply indicate in the format string which arguments the placeholders refer to. We would write the format string like this instead:

Example #3 Use Order Placeholder

```php
<?php
$num = 5;
$location = 'tree';

$format = 'The %2$s contains %1$d monkeys';
echo sprintf($format, $num, $location);
?>
```

An added benefit is that placeholders can be repeated without adding more arguments in the code.

Example #4 Repeated Placeholder

```php
<?php
$num = 5;
$location = 'tree';

$format = 'The %2$s contains %1$d monkeys.
           That\'s a nice %2$s full of %1$d monkeys.';
echo sprintf($format, $num, $location);
?>
```

When using argument swapping, the `n$` *position specifier* must come immediately after the percent sign (`%`), before any other specifiers, as shown below.

Example #5 Specifying padding character

```php
<?php
echo sprintf("%'.9d\n", 123);
```

```php
echo sprintf("%'.09d\n", 123);
?>
```

The above example will output:

```
......123
000000123
```

Example #6 Position specifier with other specifiers

```php
<?php
$num = 5;
$location = 'tree';

$format = 'The %2$s contains %1$04d monkeys';
echo sprintf($format, $num, $location);
?>
```

The above example will output:

```
The tree contains 0005 monkeys
```

Example #7 sprintf(): zero-padded integers

```php
<?php
$year = 2005;
$month = 5;
$day = 6;

$isodate = sprintf("%04d-%02d-%02d", $year, $month, $day);
echo $isodate, PHP_EOL;
?>
```

Example #8 sprintf(): formatting currency

```php
<?php
$money1 = 68.75;
$money2 = 54.35;
$money = $money1 + $money2;
echo $money, PHP_EOL;

$formatted = sprintf("%01.2f", $money);
echo $formatted, PHP_EOL;
?>
```

The above example will output:

```
123.1
123.10
```

Example #9 sprintf(): scientific notation

```php
<?php
$number = 362525200;

echo sprintf("%.3e", $number), PHP_EOL;
?>
```

The above example will output:

```
3.625e+8
```

See Also

- printf() - Output a formatted string
- fprintf() - Write a formatted string to a stream
- vprintf() - Output a formatted string
- vsprintf() - Return a formatted string
- vfprintf() - Write a formatted string to a stream
- sscanf() - Parses input from a string according to a format
- fscanf() - Parses input from a file according to a format
- number_format() - Format a number with grouped thousands
- date() - Format a Unix timestamp

sscanf

sscanf

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

sscanf — Parses input from a string according to a format

Description

function **sscanf**(string `$string`, string `$format`, mixed `&...$vars`): array|int|null

The function **sscanf()** is the input analog of printf(). **sscanf()** reads from the string `string` and interprets it according to the specified `format`.

Any whitespace in the format string matches any whitespace in the input string. This means that even a tab (`\t`) in the format string can match a single space character in the input string.

Parameters

`string`

The input string being parsed.

`format`

The interpreted format for `string`, which is described in the documentation for sprintf() with following differences:

- Function is not locale-aware.
- `F`, `g`, `G` and `b` are not supported.
- `D` stands for decimal number.
- `i` stands for integer with base detection.
- `n` stands for number of characters processed so far.
- `s` stops reading at any whitespace character.
- `*` instead of `argnum$` suppresses the assignment of this conversion specification.

`vars`

Optionally pass in variables by reference that will contain the parsed values.

Return Values

If only two parameters were passed to this function, the values parsed will be returned as an array. Otherwise, if optional parameters are passed, the function will return the number of assigned values. The optional parameters must be passed by reference.

If there are more substrings expected in the `format` than there are available within `string`, `null` will be returned.

When optional parameters are used and the end of the input `string` is reached before any value has been parsed, `-1` is returned.

Examples

Example #1 sscanf() Example

```
<?php
// getting the serial number
list($serial) = sscanf("SN/2350001", "SN/%d");
// and the date of manufacturing
$mandate = "January 01 2000";
list($month, $day, $year) = sscanf($mandate, "%s %d %d");
echo "Item $serial was manufactured on: $year-" . substr($month, 0, 3) . "-$day\n";
?>
```

If optional parameters are passed, the function will return the number of assigned values.

Example #2 sscanf() - using optional parameters

```
<?php
// get author info and generate DocBook entry
$auth = "24\tLewis Carroll";
$n = sscanf($auth, "%d\t%s %s", $id, $first, $last);
echo "<author id='$id'>
    <firstname>$first</firstname>
    <surname>$last</surname>
</author>\n";
?>
```

See Also

- printf() - Output a formatted string
- sprintf() - Return a formatted string
- fprintf() - Write a formatted string to a stream
- vprintf() - Output a formatted string
- vsprintf() - Return a formatted string
- vfprintf() - Write a formatted string to a stream
- fscanf() - Parses input from a file according to a format
- number_format() - Format a number with grouped thousands
- date() - Format a Unix timestamp

str_contains

str_contains

(PHP 8)

str_contains — Determine if a string contains a given substring

Description

function **str_contains**(string `$haystack`, string `$needle`): bool

Performs a case-sensitive check indicating if `needle` is contained in `haystack`.

Parameters

`haystack`

The string to search in.

`needle`

The substring to search for in the `haystack`.

Return Values

Returns `true` if `needle` is in `haystack`, `false` otherwise.

Examples

Example #1 Using the empty string `''`

```php
<?php
if (str_contains('abc', '')) {
    echo "Checking the existence of the empty string will always return true";
}
?>
```

The above example will output:

```
Checking the existence of the empty string will always return true
```

Example #2 Showing case-sensitivity

```php
<?php
$string = 'The lazy fox jumped over the fence';

if (str_contains($string, 'lazy')) {
    echo "The string 'lazy' was found in the string\n";
}

if (str_contains($string, 'Lazy')) {
    echo 'The string "Lazy" was found in the string';
} else {
    echo '"Lazy" was not found because the case does not match';
}

?>
```

The above example will output:

```
The string 'lazy' was found in the string
"Lazy" was not found because the case does not match
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- str_ends_with() - Checks if a string ends with a given substring
- str_starts_with() - Checks if a string starts with a given substring
- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- strrpos() - Find the position of the last occurrence of a substring in a string
- strripos() - Find the position of the last occurrence of a case-insensitive substring in a string
- strstr() - Find the first occurrence of a string
- strpbrk() - Search a string for any of a set of characters

- substr() - Return part of a string
- preg_match() - Perform a regular expression match

str_decrement

str_decrement

(PHP 8 >= 8.3.0)

str_decrement — Decrement an alphanumeric string

Description

function **str_decrement**(string `$string`): string

Returns the decremented alphanumeric ASCII `string`.

Parameters

`string`

The input string.

Return Values

Returns the decremented alphanumeric ASCII string.

Errors/Exceptions

A ValueError is thrown if `string` is empty.

A ValueError is thrown if `string` is not an alphanumeric ASCII string.

A ValueError is thrown if `string` cannot be decremented. For example, `"A"` or `"0"`.

Examples

Example #1 Basic str_decrement() example

```php
<?php
$str = 'ABC';
var_dump(str_decrement($str));
?>
```

The above example will output:

```
string(3) "ABB"
```

Example #2 str_decrement() example with a carry

```php
<?php
$str = 'ZA';
var_dump(str_decrement($str));

$str = 'AA';
var_dump(str_decrement($str));
?>
```

The above example will output:

```
string(2) "YZ"
```

```
string(1) "Z"
```

See Also

- str_increment() - Increment an alphanumeric string

str_ends_with

str_ends_with

(PHP 8)

str_ends_with — Checks if a string ends with a given substring

Description

function **str_ends_with**(string `$haystack`, string `$needle`): bool

Performs a case-sensitive check indicating if `haystack` ends with `needle`.

Parameters

`haystack`

The string to search in.

`needle`

The substring to search for in the `haystack`.

Return Values

Returns **`true`** if `haystack` ends with `needle`, **`false`** otherwise.

Examples

Example #1 Using the empty string `''`

```php
<?php
if (str_ends_with('abc', '')) {
    echo "All strings end with the empty string";
}
?>
```

The above example will output:

```
All strings end with the empty string
```

Example #2 Showing case-sensitivity

```php
<?php
$string = 'The lazy fox jumped over the fence';

if (str_ends_with($string, 'fence')) {
    echo "The string ends with 'fence'\n";
}

if (str_ends_with($string, 'Fence')) {
    echo 'The string ends with "Fence"';
} else {
    echo '"Fence" was not found because the case does not match';
```

```
}

?>
```

The above example will output:

```
The string ends with 'fence'
"Fence" was not found because the case does not match
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- str_contains() - Determine if a string contains a given substring
- str_starts_with() - Checks if a string starts with a given substring
- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- strrpos() - Find the position of the last occurrence of a substring in a string
- strripos() - Find the position of the last occurrence of a case-insensitive substring in a string
- strstr() - Find the first occurrence of a string
- strpbrk() - Search a string for any of a set of characters
- substr() - Return part of a string
- preg_match() - Perform a regular expression match

str_getcsv

str_getcsv

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

str_getcsv — Parse a CSV string into an array

Description

function **str_getcsv**(
  string `$string`,
  string `$separator = ","`,
  string `$enclosure = "\""`,
  string `$escape = "\\"`
): array

Parses a string input for fields in CSV format and returns an array containing the fields read.

> **Note** :
>
> The locale settings are taken into account by this function. For example, data encoded in certain one-byte encodings may be parsed incorrectly if **LC_CTYPE** is `en_US.UTF-8`.

Parameters

`string`

The string to parse.

`separator`

The `separator` parameter sets the field separator. It must be a single byte character.

`enclosure`

The `enclosure` parameter sets the field enclosure character. It must be a single byte character.

`escape`

The `escape` parameter sets the escape character. It must be a single byte character or the empty string. The empty string (`""`) disables the proprietary escape mechanism.

> **Warning**
>
> In the input stream, the `enclosure` character can always be escaped by doubling it inside a quoted string, resulting in a single `enclosure` character in the parsed result. The `escape` character works differently: If a sequence of `escape` and `enclosure` characters appear in the input, both characters will be present in the parsed result. So for the default parameters, a CSV line like `"a""b","c\"d"` will have the fields parsed as `a"b` and `c\"d`, respectively.

> **Warning**
>
> As of PHP 8.4.0, depending on the default value of `escape` is deprecated. It needs to be provided explicitly either positionally or by the use of named arguments.

> **Warning**
>
> When `escape` is set to anything other than an empty string (`""`) it can result in CSV that is not compliant with » RFC 4180 or unable to survive a roundtrip through the PHP CSV functions. The default for `escape` is `"\\"` so it is recommended to set it to the empty string explicitly. The default value will change in a future version of PHP, no earlier than PHP 9.0.

Return Values

Returns an indexed array containing the fields read.

Errors/Exceptions

Throws a ValueError if `separator` or `enclosure` is not one byte long.

Throws a ValueError if `escape` is not one byte long or the empty string.

Changelog

Version	Description
8.4.0	Relying on the default value of `escape` is now deprecated.
8.4.0	Now throws a ValueError if `separator`, `enclosure`, or `escape` is invalid. This mimics the behavior of fgetcsv() and fputcsv().
8.3.0	An empty string is returned instead of a string with a single null byte for the last field if it contains only an unterminated enclosure.
7.4.0	The `escape` parameter now interprets an empty string as signal to disable the proprietary escape mechanism. Formerly, an empty string was treated like the default parameter value.

Examples

Example #1 **str_getcsv()** example

```php
<?php

$string = 'PHP,Java,Python,Kotlin,Swift';
$data = str_getcsv($string, escape: '\\');

var_dump($data);
?>
```

The above example will output:

```
array(5) {
  [0]=>
  string(3) "PHP"
  [1]=>
  string(4) "Java"
  [2]=>
  string(6) "Python"
  [3]=>
  string(6) "Kotlin"
  [4]=>
  string(5) "Swift"
}
```

Example #2 str_getcsv() example with an empty string

> **Caution**
>
> On an empty string this function returns the value `[null]` instead of an empty array.

```php
<?php

$string = '';
$data = str_getcsv($string, escape: '\\');

var_dump($data);
?>
```

The above example will output:

```
array(1) {
  [0]=>
  NULL
}
```

See Also

- fputcsv() - Format line as CSV and write to file pointer
- fgetcsv() - Gets line from file pointer and parse for CSV fields
- SplFileObject::fgetcsv() - Gets line from file and parse as CSV fields
- SplFileObject::fputcsv() - Write a field array as a CSV line
- SplFileObject::setCsvControl() - Set the delimiter, enclosure and escape character for CSV
- SplFileObject::getCsvControl() - Get the delimiter, enclosure and escape character for CSV

str_increment

str_increment

(PHP 8 >= 8.3.0)

str_increment — Increment an alphanumeric string

Description

function **str_increment**(string `$string`): string

Returns the incremented alphanumeric ASCII `string`.

Parameters

`string`

The input string.

Return Values

Returns the incremented alphanumeric ASCII string.

Errors/Exceptions

A ValueError is thrown if `string` is empty.

A ValueError is thrown if `string` is not an alphanumeric ASCII string.

Examples

Example #1 Basic str_increment() example

```
<?php
$str = 'ABC';
var_dump(str_increment($str));
?>
```

The above example will output:

```
string(3) "ABD"
```

Example #2 str_increment() example with a carry

```
<?php
$str = 'DZ';
var_dump(str_increment($str));

$str = 'ZZ';
var_dump(str_increment($str));
?>
```

The above example will output:

```
string(2) "EA"
string(3) "AAA"
```

See Also

- str_decrement() - Decrement an alphanumeric string

str_ireplace

str_ireplace

(PHP 5, PHP 7, PHP 8)

str_ireplace — Case-insensitive version of str_replace()

Description

```
function str_ireplace(
    array|string $search,
    array|string $replace,
    string|array $subject,
    int &$count = null
): string|array
```

This function returns a string or an array with all occurrences of `search` in `subject` (ignoring case) replaced with the given `replace` value.

To replace text based on a pattern rather than a fixed string, use preg_replace() with the `i` pattern modifier.

Parameters

If `search` and `replace` are arrays, then **str_ireplace()** takes a value from each array and uses them to search and replace on `subject`. If `replace` has fewer values than `search`, then an empty string is used for the rest of replacement values. If `search` is an array and `replace` is a string, then this replacement string is used for every value of `search`. The converse would not make sense, though.

If `search` or `replace` are arrays, their elements are processed first to last.

`search`

The value being searched for, otherwise known as the *needle*. An array may be used to designate multiple needles.

`replace`

The replacement value that replaces found `search` values. An array may be used to designate multiple replacements.

`subject`

The string or array being searched and replaced on, otherwise known as the *haystack*.

If `subject` is an array, then the search and replace is performed with every entry of `subject`, and the return value is an array as well.

`count`

If passed, this will be set to the number of replacements performed.

Return Values

Returns a string or an array of replacements.

Changelog

Version	Description
8.2.0	Case folding no longer depends on the locale set with setlocale(). Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.

Examples

Example #1 str_ireplace() example

```php
<?php
$bodytag = str_ireplace("%body%", "black", "<body text=%BODY%>");
echo $bodytag; // <body text=black>
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

> **Caution**
>
> Replacement order gotcha

> Because **str_ireplace()** replaces left to right, it might replace a previously inserted value when doing multiple replacements. Example #2 in the str_replace() documentation demonstrates how this may affect you in practice.

See Also

- str_replace() - Replace all occurrences of the search string with the replacement string
- preg_replace() - Perform a regular expression search and replace
- strtr() - Translate characters or replace substrings

str_pad

str_pad

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

str_pad — Pad a string to a certain length with another string

Description

```
function str_pad(
    string $string,
    int $length,
    string $pad_string = " ",
    int $pad_type = STR_PAD_RIGHT
): string
```

This function returns the `string` string padded on the left, the right, or both sides to the specified padding length. If the optional argument `pad_string` is not supplied, the `string` is padded with spaces, otherwise it is padded with characters from `pad_string` up to the limit.

Parameters

`string`

The input string.

`length`

The desired length of the final padded string. If the value of `length` is negative, less than, or equal to the length of the input string, no padding takes place, and `string` will be returned.

`pad_string`

> **Note** :
>
> The `pad_string` may be truncated if the required number of padding characters can't be evenly divided by the `pad_string`'s length.

`pad_type`

Optional argument `pad_type` can be `STR_PAD_RIGHT`, `STR_PAD_LEFT`, or `STR_PAD_BOTH`. If `pad_type` is not specified it is assumed to be `STR_PAD_RIGHT`.

Return Values

Returns the padded string.

Examples

Example #1 str_pad() example

```
<?php
$input = "Alien";
echo str_pad($input, 10), PHP_EOL;                      // produces "Alien     "
echo str_pad($input, 10, "-=", STR_PAD_LEFT), PHP_EOL;  // produces "-=-=-Alien"
echo str_pad($input, 10, "_", STR_PAD_BOTH), PHP_EOL;   // produces "__Alien___"
echo str_pad($input,  6, "___"), PHP_EOL;               // produces "Alien_"
```

```
echo str_pad($input,  3, "*"), PHP_EOL;                // produces "Alien"
?>
```

See Also

- mb_str_pad() - Pad a multibyte string to a certain length with another multibyte string

str_repeat

str_repeat

(PHP 4, PHP 5, PHP 7, PHP 8)

str_repeat — Repeat a string

Description

function **str_repeat**(string `$string`, int `$times`): string

Returns `string` repeated `times` times.

Parameters

`string`

The string to be repeated.

`times`

Number of times the `string` string should be repeated.

`times` has to be greater than or equal to 0. If the `times` is set to 0, the function will return an empty string.

Return Values

Returns the repeated string.

Examples

Example #1 str_repeat() example

```
<?php
echo str_repeat("-=", 10);
?>
```

The above example will output:

```
-=-=-=-=-=-=-=-=-=-=
```

See Also

- for
- str_pad() - Pad a string to a certain length with another string
- substr_count() - Count the number of substring occurrences

str_replace

str_replace

(PHP 4, PHP 5, PHP 7, PHP 8)

str_replace — Replace all occurrences of the search string with the replacement string

Description

```
function str_replace(
    array|string $search,
    array|string $replace,
    string|array $subject,
    int &$count = null
): string|array
```

This function returns a string or an array with all occurrences of `search` in `subject` replaced with the given `replace` value.

To replace text based on a pattern rather than a fixed string, use preg_replace().

Parameters

If `search` and `replace` are arrays, then **str_replace()** takes a value from each array and uses them to search and replace on `subject`. If `replace` has fewer values than `search`, then an empty string is used for the rest of replacement values. If `search` is an array and `replace` is a string, then this replacement string is used for every value of `search`. The converse would not make sense, though.

If `search` or `replace` are arrays, their elements are processed first to last.

`search`

The value being searched for, otherwise known as the *needle*. An array may be used to designate multiple needles.

`replace`

The replacement value that replaces found `search` values. An array may be used to designate multiple replacements.

`subject`

The string or array being searched and replaced on, otherwise known as the *haystack*.

If `subject` is an array, then the search and replace is performed with every entry of `subject`, and the return value is an array as well.

`count`

If passed, this will be set to the number of replacements performed.

Return Values

This function returns a string or an array with the replaced values.

Examples

Example #1 Basic str_replace() examples

```php
<?php
// Provides: <body text='black'>
$bodytag = str_replace("%body%", "black", "<body text='%body%'>");
echo $bodytag, PHP_EOL;

// Provides: Hll Wrld f PHP
$vowels = array("a", "e", "i", "o", "u", "A", "E", "I", "O", "U");
$onlyconsonants = str_replace($vowels, "", "Hello World of PHP");
echo $onlyconsonants, PHP_EOL;

// Provides: You should eat pizza, beer, and ice cream every day
$phrase  = "You should eat fruits, vegetables, and fiber every day.";
$healthy = array("fruits", "vegetables", "fiber");
$yummy   = array("pizza", "beer", "ice cream");

$newphrase = str_replace($healthy, $yummy, $phrase);
echo $newphrase, PHP_EOL;

// Provides: 2
$str = str_replace("ll", "", "good golly miss molly!", $count);
echo $count, PHP_EOL;
?>
```

Example #2 Examples of potential str_replace() gotchas

```php
<?php
// Order of replacement
$str     = "Line 1\nLine 2\rLine 3\r\nLine 4\n";
$order   = array("\r\n", "\n", "\r");
$replace = '<br />';

// Processes \r\n's first so they aren't converted twice.
$newstr = str_replace($order, $replace, $str);
echo $newstr, PHP_EOL;

// Outputs F because A is replaced with B, then B is replaced with C, and so on...
// Finally E is replaced with F, because of left to right replacements.
$search  = array('A', 'B', 'C', 'D', 'E');
$replace = array('B', 'C', 'D', 'E', 'F');
$subject = 'A';
echo str_replace($search, $replace, $subject), PHP_EOL;

// Outputs: apearpearle pear
// For the same reason mentioned above
$letters = array('a', 'p');
$fruit   = array('apple', 'pear');
$text    = 'a p';
$output  = str_replace($letters, $fruit, $text);
echo $output, PHP_EOL;
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

> **Caution**
>
> #### Replacement order gotcha
>
> Because **str_replace()** replaces left to right, it might replace a previously inserted value when doing multiple replacements. See also the examples in this document.

> **Note** :
>
> This function is case-sensitive. Use str_ireplace() for case-insensitive replace.

See Also

- str_ireplace() - Case-insensitive version of str_replace
- substr_replace() - Replace text within a portion of a string
- preg_replace() - Perform a regular expression search and replace
- strtr() - Translate characters or replace substrings

str_rot13

str_rot13

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

str_rot13 — Perform the rot13 transform on a string

Description

function **str_rot13**(string `$string`): string

Performs the ROT13 encoding on the `string` argument and returns the resulting string.

The ROT13 encoding simply shifts every letter by 13 places in the alphabet while leaving non-alpha characters untouched. Encoding and decoding are done by the same function, passing an encoded string as argument will return the original version.

Parameters

`string`

The input string.

Return Values

Returns the ROT13 version of the given string.

Examples

Example #1 str_rot13() example

```php
<?php

echo str_rot13('PHP 4.3.0'); // CUC 4.3.0

?>
```

str_shuffle

str_shuffle

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

str_shuffle — Randomly shuffles a string

Description

function **str_shuffle**(string `$string`): string

str_shuffle() shuffles a string. One permutation of all possible is created.

> **Caution**
>
> This function does not generate cryptographically secure values, and *must not* be used for cryptographic purposes, or purposes that require returned values to be unguessable.
>
> If cryptographically secure randomness is required, the Random\Randomizer may be used with the Random\Engine\Secure engine. For simple use cases, the random_int() and random_bytes() functions provide a convenient and secure API that is backed by the operating system's CSPRNG.

> **Caution**
>
> This function uses the global Mt19937 ("Mersenne Twister") instance as the source of randomness and thus shares its state with all other functions using the global Mt19937. Using any of these functions advances the sequence for *all* the other functions, regardless of scope.
>
> Generating repeatable sequences by seeding mt_srand() or srand() with a known value will also yield repeatable output from this function.
>
> Prefer using Random\Randomizer methods in all newly written code.

Parameters

`string`

The input string.

Return Values

Returns the shuffled string.

Changelog

Version	Description
7.1.0	The internal randomization algorithm has been changed to use the » Mersenne Twister Random Number Generator instead of the libc rand function.

Examples

Example #1 str_shuffle() example

```
<?php
$str = 'abcdef';
$shuffled = str_shuffle($str);

// This will echo something like: bfdaec
echo $shuffled;
?>
```

See Also

- Random\Randomizer::shuffleBytes() - Get a byte-wise permutation of a string
- Random\Randomizer::shuffleArray() - Get a permutation of an array

str_split

str_split

(PHP 5, PHP 7, PHP 8)

str_split — Convert a string to an array

Description

function **str_split**(string `$string`, int `$length` = 1): array

Converts a string to an array.

Parameters

`string`

The input string.

`length`

Maximum length of the chunk.

Return Values

If the optional `length` parameter is specified, the returned array will be broken down into chunks with each being `length` in length, except the final chunk which may be shorter if the string does not divide evenly. The default `length` is `1`, meaning every chunk will be one byte in size.

Errors/Exceptions

If `length` is less than `1`, a ValueError will be thrown.

Changelog

Version	Description
8.2.0	If `string` is empty an empty array is now returned. Previously an array containing a single empty string was returned.
8.0.0	If `length` is less than `1`, a ValueError will be thrown now; previously, an error of level `E_WARNING` has been raised instead, and the function returned `false`.

Examples

Example #1 Example uses of str_split()

```
<?php
```

```php
$str = "Hello Friend";

$arr1 = str_split($str);
$arr2 = str_split($str, 3);

print_r($arr1);
print_r($arr2);

?>
```

The above example will output:

```
Array
(
    [0] => H
    [1] => e
    [2] => l
    [3] => l
    [4] => o
    [5] =>
    [6] => F
    [7] => r
    [8] => i
    [9] => e
    [10] => n
    [11] => d
)

Array
(
    [0] => Hel
    [1] => lo
    [2] => Fri
    [3] => end
)
```

Notes

> **Note** :
>
> **str_split()** will split into bytes, rather than characters when dealing with a multi-byte encoded string. mb_str_split() can be used to split the string into code points. grapheme_str_split() can be used to split the string into grapheme clusters.

See Also

- mb_str_split() - Given a multibyte string, return an array of its characters
- grapheme_str_split() - Split a string into an array
- chunk_split() - Split a string into smaller chunks
- preg_split() - Split string by a regular expression
- explode() - Split a string by a string
- count_chars() - Return information about characters used in a string
- str_word_count() - Return information about words used in a string
- for

str_starts_with

str_starts_with

(PHP 8)

str_starts_with — Checks if a string starts with a given substring

Description

function **str_starts_with**(string `$haystack`, string `$needle`): bool

Performs a case-sensitive check indicating if `haystack` begins with `needle`.

Parameters

`haystack`

The string to search in.

`needle`

The substring to search for in the `haystack`.

Return Values

Returns `true` if `haystack` begins with `needle`, `false` otherwise.

Examples

Example #1 Using the empty string `''`

```php
<?php
if (str_starts_with('abc', '')) {
    echo "All strings start with the empty string";
}
?>
```

The above example will output:

```
All strings start with the empty string
```

Example #2 Showing case-sensitivity

```php
<?php
$string = 'The lazy fox jumped over the fence';

if (str_starts_with($string, 'The')) {
    echo "The string starts with 'The'\n";
}

if (str_starts_with($string, 'the')) {
    echo 'The string starts with "the"';
} else {
    echo '"the" was not found because the case does not match';
}

?>
```

The above example will output:

```
The string starts with 'The'
"the" was not found because the case does not match
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- str_contains() - Determine if a string contains a given substring
- str_ends_with() - Checks if a string ends with a given substring
- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- strrpos() - Find the position of the last occurrence of a substring in a string
- strripos() - Find the position of the last occurrence of a case-insensitive substring in a string
- strstr() - Find the first occurrence of a string
- strpbrk() - Search a string for any of a set of characters
- substr() - Return part of a string
- preg_match() - Perform a regular expression match

str_word_count

str_word_count

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

str_word_count — Return information about words used in a string

Description

function **str_word_count**(string `$string`, int `$format` = 0, ?string `$characters` = `null`): array|int

Counts the number of words inside `string`. If the optional `format` is not specified, then the return value will be an integer representing the number of words found. In the event the `format` is specified, the return value will be an array, content of which is dependent on the `format`. The possible value for the `format` and the resultant outputs are listed below.

For the purpose of this function, 'word' is defined as a locale dependent string containing alphabetic characters, which also may contain, but not start with "'" and "-" characters. Note that multibyte locales are not supported.

Parameters

`string`

The string

`format`

Specify the return value of this function. The current supported values are:

- 0 - returns the number of words found
- 1 - returns an array containing all the words found inside the `string`
- 2 - returns an associative array, where the key is the numeric position of the word inside the `string` and the value is the actual word itself

`characters`

A list of additional characters which will be considered as 'word'

Return Values

Returns an array or an integer, depending on the `format` chosen.

Changelog

Version	Description
8.0.0	`characters` is nullable now.

Examples

Example #1 A str_word_count() example

```php
<?php

$str = "Hello fri3nd, you're
       looking          good today!";

print_r(str_word_count($str, 1));
print_r(str_word_count($str, 2));
print_r(str_word_count($str, 1, 'àáãç3'));

echo str_word_count($str);

?>
```

The above example will output:

```
Array
(
    [0] => Hello
    [1] => fri
    [2] => nd
    [3] => you're
    [4] => looking
    [5] => good
    [6] => today
)
Array
(
    [0] => Hello
    [6] => fri
    [10] => nd
    [14] => you're
    [28] => looking
    [45] => good
    [50] => today
)
Array
(
    [0] => Hello
    [1] => fri3nd
    [2] => you're
    [3] => looking
    [4] => good
    [5] => today
)
7
```

See Also

- explode() - Split a string by a string
- preg_split() - Split string by a regular expression
- count_chars() - Return information about characters used in a string
- substr_count() - Count the number of substring occurrences

strcasecmp

strcasecmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strcasecmp — Binary safe case-insensitive string comparison

Description

function **strcasecmp**(string `$string1`, string `$string2`): int

Binary safe case-insensitive string comparison. The comparison is not locale aware; only ASCII letters are compared in a case-insensitive way.

Parameters

`string1`

The first string

`string2`

The second string

Return Values

Returns a value less than 0 if `string1` is less than `string2`; a value greater than 0 if `string1` is greater than `string2`, and `0` if they are equal. No particular meaning can be reliably inferred from the value aside from its sign.

Changelog

Version	Description
8.2.0	This function is no longer guaranteed to return `strlen($string1) - strlen($string2)` when string lengths are not equal, but may now return `-1` or `1` instead.

Examples

Example #1 strcasecmp() example

```php
<?php
$var1 = "Hello";
$var2 = "hello";
if (strcasecmp($var1, $var2) == 0) {
    echo '$var1 is equal to $var2 in a case-insensitive string comparison';
}
?>
```

See Also

- strcmp() - Binary safe string comparison
- preg_match() - Perform a regular expression match
- substr_compare() - Binary safe comparison of two strings from an offset, up to length characters
- strncasecmp() - Binary safe case-insensitive string comparison of the first n characters
- stristr() - Case-insensitive strstr
- substr() - Return part of a string

strchr

strchr

(PHP 4, PHP 5, PHP 7, PHP 8)

strchr — Alias of strstr()

Description

This function is an alias of: strstr().

strcmp

strcmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strcmp — Binary safe string comparison

Description

function **strcmp**(string `$string1`, string `$string2`): int

Note that this comparison is case sensitive. For case-insensitive comparison, see strcasecmp().

Note that this comparison is not locale aware. For locale aware comparison see strcoll() or Collator::compare()

Parameters

`string1`

The first string.

`string2`

The second string.

Return Values

Returns a value less than 0 if `string1` is less than `string2`; a value greater than 0 if `string1` is greater than `string2`, and `0` if they are equal. No particular meaning can be reliably inferred from the value aside from its sign.

Changelog

Version	Description
8.2.0	This function is no longer guaranteed to return `strlen($string1) - strlen($string2)` when string lengths are not equal, but may now return `-1` or `1` instead.

Examples

Example #1 strcmp() example

```php
<?php
$var1 = "Hello";
$var2 = "hello";
if (strcmp($var1, $var2) !== 0) {
    echo '$var1 is not equal to $var2 in a case sensitive string comparison';
}
?>
```

See Also

- Full string comparison
  - strcasecmp() - Binary safe case-insensitive string comparison
  - Collator::compare() - Compare two Unicode strings
  - strcoll() - Locale based string comparison
- Partial string comparison
  - substr_compare() - Binary safe comparison of two strings from an offset, up to length characters
  - strncmp() - Binary safe string comparison of the first n characters
  - strstr() - Find the first occurrence of a string
- Similar / other string comparison
  - preg_match() - Perform a regular expression match
  - levenshtein() - Calculate Levenshtein distance between two strings
  - metaphone() - Calculate the metaphone key of a string
  - similar_text() - Calculate the similarity between two strings
  - soundex() - Calculate the soundex key of a string

strcoll

strcoll

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

strcoll — Locale based string comparison

Description

function **strcoll**(string `$string1`, string `$string2`): int

Note that this comparison is case sensitive, and unlike strcmp() this function is not binary safe.

strcoll() uses the current locale for doing the comparisons. If the current locale is C or POSIX, this function is equivalent to strcmp().

Parameters

`string1`

The first string.

`string2`

The second string.

Return Values

Returns < 0 if `string1` is less than `string2`; > 0 if `string1` is greater than `string2`, and 0 if they are equal.

See Also

- preg_match() - Perform a regular expression match
- strcmp() - Binary safe string comparison
- strcasecmp() - Binary safe case-insensitive string comparison
- substr() - Return part of a string
- stristr() - Case-insensitive strstr
- strncasecmp() - Binary safe case-insensitive string comparison of the first n characters
- strncmp() - Binary safe string comparison of the first n characters
- strstr() - Find the first occurrence of a string
- setlocale() - Set locale information

strcspn

strcspn

(PHP 4, PHP 5, PHP 7, PHP 8)

strcspn — Find length of initial segment not matching mask

Description

```
function strcspn(
    string $string,
    string $characters,
    int $offset = 0,
    ?int $length = null
): int
```

Returns the length of the initial segment of `string` which does *not* contain any of the characters in `characters`.

If `offset` and `length` are omitted, then all of `string` will be examined. If they are included, then the effect will be the same as calling `strcspn(substr($string, $offset, $length), $characters)` (see substr for more information).

Parameters

`string`

The string to examine.

`characters`

The string containing every disallowed character.

`offset`

The position in `string` to start searching.

If `offset` is given and is non-negative, then **strcspn()** will begin examining `string` at the `offset`'th position. For instance, in the string '`abcdef`', the character at position `0` is '`a`', the character at position `2` is '`c`', and so forth.

If `offset` is given and is negative, then **strcspn()** will begin examining `string` at the `offset`'th position from the end of `string`.

`length`

The length of the segment from `string` to examine.

If `length` is given and is non-negative, then `string` will be examined for `length` characters after the starting position.

If `length` is given and is negative, then `string` will be examined from the starting position up to `length` characters from the end of `string`.

Return Values

Returns the length of the initial segment of `string` which consists entirely of characters *not* in `characters`.

> **Note** :
>
> When a `offset` parameter is set, the returned length is counted starting from this position, not from the beginning of `string`.

Changelog

Version	Description
8.4.0	Prior to PHP 8.4.0, when `characters` was the empty string, the search would incorrectly stop at the first null byte in `string`.
8.0.0	`length` is nullable now.

Examples

Example #1 strcspn() example

```php
<?php
$a = strcspn('banana', 'a');
$b = strcspn('banana', 'abcd');
$c = strcspn('banana', 'z');
$d = strcspn('abcdhelloabcd', 'a', -9);
$e = strcspn('abcdhelloabcd', 'a', -9, -5);

var_dump($a);
var_dump($b);
var_dump($c);
var_dump($d);
var_dump($e);
?>
```

The above example will output:

```
int(1)
int(0)
int(6)
int(5)
int(4)
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- strspn() - Finds the length of the initial segment of a string consisting entirely of characters contained within a given mask

strip_tags

strip_tags

(PHP 4, PHP 5, PHP 7, PHP 8)

strip_tags — Strip HTML and PHP tags from a string

Description

function **strip_tags**(string `$string`, array|string|null `$allowed_tags = null`): string

This function tries to return a string with all NULL bytes, HTML and PHP tags stripped from a given `string`. It uses the same tag stripping state machine as the fgetss() function.

Parameters

`string`

The input string.

`allowed_tags`

You can use the optional second parameter to specify tags which should not be stripped. These are either given as string, or as of PHP 7.4.0, as array. Refer to the example below regarding the format of this parameter.

> **Note** :
>
> HTML comments and PHP tags are also stripped. This is hardcoded and can not be changed with `allowed_tags`.

> **Note** :
>
> Self-closing XHTML tags are ignored and only non-self-closing tags should be used in `allowed_tags`. For example, to allow both `<br>` and `<br/>`, you should use:
>
> ```
> <?php
> strip_tags($input, '<br>');
> ?>
> ```

Return Values

Returns the stripped string.

Changelog

Version	Description
8.0.0	`allowed_tags` is nullable now.
7.4.0	The `allowed_tags` now alternatively accepts an array.

Examples

Example #1 strip_tags() example

```
<?php
$text = '<p>Test paragraph.</p><!-- Comment --> <a href="#fragment">Other text</a>';
echo strip_tags($text);
echo "\n";

// Allow <p> and <a>
echo strip_tags($text, '<p><a>');

// as of PHP 7.4.0 the line above can be written as:
```

```
// echo strip_tags($text, ['p', 'a']);
?>
```

The above example will output:

```
Test paragraph. Other text
<p>Test paragraph.</p> <a href="#fragment">Other text</a>
```

Notes

> **Warning**
>
> This function should not be used to try to prevent XSS attacks. Use more appropriate functions like htmlspecialchars() or other means depending on the context of the output.

> **Warning**
>
> Because **strip_tags()** does not actually validate the HTML, partial or broken tags can result in the removal of more text/data than expected.

> **Warning**
>
> This function does not modify any attributes on the tags that you allow using `allowed_tags`, including the `style` and `onmouseover` attributes that a mischievous user may abuse when posting text that will be shown to other users.

> **Note** :
>
> Tag names within the input HTML that are greater than 1023 bytes in length will be treated as though they are invalid, regardless of the `allowed_tags` parameter.

See Also

- htmlspecialchars() - Convert special characters to HTML entities

stripcslashes

stripcslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

stripcslashes — Un-quote string quoted with addcslashes()

Description

function **stripcslashes**(string `$string`): string

Returns a string with backslashes stripped off. Recognizes C-like `\n`, `\r` ..., octal and hexadecimal representation.

Parameters

`string`

The string to be unescaped.

Return Values

Returns the unescaped string.

Examples

Example #1 stripcslashes() example

```php
<?php

var_dump(stripcslashes('I\'d have a coffee.\nNot a problem.') === "I'd have a coffee.
Not a problem."); // true
?>
```

See Also

- addcslashes() - Quote string with slashes in a C style

stripos

stripos

(PHP 5, PHP 7, PHP 8)

stripos — Find the position of the first occurrence of a case-insensitive substring in a string

Description

function **stripos**(string `$haystack`, string `$needle`, int `$offset` = 0): int|false

Find the numeric position of the first occurrence of `needle` in the `haystack` string.

Unlike strpos(), **stripos()** is case-insensitive.

Parameters

`haystack`

The string to search in.

`needle`

The string to search for.

Prior to PHP 8.0.0, if `needle` is not a string, it is converted to an integer and applied as the ordinal value of a character. This behavior is deprecated as of PHP 7.3.0, and relying on it is highly discouraged. Depending on the intended behavior, the `needle` should either be explicitly cast to string, or an explicit call to chr() should be performed.

`offset`

If specified, search will start this number of characters counted from the beginning of the string. If the offset is negative, the search will start this number of characters counted from the end of the string.

Return Values

Returns the position of where the needle exists relative to the beginning of the `haystack` string (independent of offset). Also note that string positions start at 0, and not 1.

Returns **`false`** if the needle was not found.

> **Warning**
>
> This function may return Boolean **`false`**, but may also return a non-Boolean value which evaluates to **`false`**. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.2.0	Case folding no longer depends on the locale set with setlocale(). Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing an int as `needle` is no longer supported.
7.3.0	Passing an int as `needle` has been deprecated.
7.1.0	Support for negative `offsets` has been added.

Examples

Example #1 stripos() examples

```php
<?php
$findme    = 'a';
$mystring1 = 'xyz';
$mystring2 = 'ABC';

$pos1 = stripos($mystring1, $findme);
$pos2 = stripos($mystring2, $findme);

// Nope, 'a' is certainly not in 'xyz'
if ($pos1 === false) {
    echo "The string '$findme' was not found in the string '$mystring1'", PHP_EOL;
}

// Note our use of !==.  Simply != would not work as expected
// because the position of 'a' is the 0th (first) character.
if ($pos2 !== false) {
    echo "We found '$findme' in '$mystring2' at position $pos2", PHP_EOL;
}
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- mb_stripos() - Finds position of first occurrence of a string within another, case insensitive
- str_contains() - Determine if a string contains a given substring
- str_ends_with() - Checks if a string ends with a given substring
- str_starts_with() - Checks if a string starts with a given substring
- strpos() - Find the position of the first occurrence of a substring in a string
- strrpos() - Find the position of the last occurrence of a substring in a string
- strripos() - Find the position of the last occurrence of a case-insensitive substring in a string
- stristr() - Case-insensitive strstr
- substr() - Return part of a string
- str_ireplace() - Case-insensitive version of str_replace

stripslashes

stripslashes

(PHP 4, PHP 5, PHP 7, PHP 8)

stripslashes — Un-quotes a quoted string

Description

function **stripslashes**(string `$string`): string

Un-quotes a quoted string.

stripslashes() can be used if you aren't inserting this data into a place (such as a database) that requires escaping. For example, if you're simply outputting data straight from an HTML form.

Parameters

`string`

The input string.

Return Values

Returns a string with backslashes stripped off. (`\'` becomes `'` and so on.) Double backslashes (`\\`) are made into a single backslash (`\`).

Examples

Example #1 A stripslashes() example

```php
<?php
$str = "Is your name O\'reilly?";

// Outputs: Is your name O'reilly?
echo stripslashes($str);
?>
```

> **Note** :
>
> **stripslashes()** is not recursive. If you want to apply this function to a multi-dimensional array, you need to use a recursive function.

Example #2 Using stripslashes() on an array

```php
<?php
function stripslashes_deep($value)
{
    $value = is_array($value) ?
                array_map('stripslashes_deep', $value) :
                stripslashes($value);

    return $value;
}

// Example
$array = array("f\\'oo", "b\\'ar", array("fo\\'o", "b\\'ar"));
$array = stripslashes_deep($array);

// Output
print_r($array);
?>
```

The above example will output:

```
Array
(
    [0] => f'oo
    [1] => b'ar
    [2] => Array
        (
            [0] => fo'o
            [1] => b'ar
        )

)
```

See Also

- addslashes() - Quote string with slashes
- get_magic_quotes_gpc() - Gets the current configuration setting of magic_quotes_gpc

stristr

stristr

(PHP 4, PHP 5, PHP 7, PHP 8)

stristr — Case-insensitive strstr()

Description

function **stristr**(string `$haystack`, string `$needle`, bool `$before_needle = false`): string|false

Returns all of `haystack` starting from and including the first occurrence of `needle` to the end.

Parameters

`haystack`

The string to search in

`needle`

The string to search for.

Prior to PHP 8.0.0, if `needle` is not a string, it is converted to an integer and applied as the ordinal value of a character. This behavior is deprecated as of PHP 7.3.0, and relying on it is highly discouraged. Depending on the intended behavior, the `needle` should either be explicitly cast to string, or an explicit call to chr() should be performed.

`before_needle`

If `true`, **stristr()** returns the part of the `haystack` before the first occurrence of the `needle` (excluding needle).

`needle` and `haystack` are examined in a case-insensitive manner.

Return Values

Returns the matched substring. If `needle` is not found, returns `false`.

Changelog

Version	Description
8.2.0	Case folding no longer depends on the locale set with setlocale(). Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing an int as `needle` is no longer supported.
7.3.0	Passing an int as `needle` has been deprecated.

Examples

Example #1 stristr() example

```
<?php
  $email = 'USER@EXAMPLE.com';
  echo stristr($email, 'e'), PHP_EOL; // outputs ER@EXAMPLE.com
  echo stristr($email, 'e', true), PHP_EOL; // outputs US
?>
```

Example #2 Testing if a string is found or not

```
<?php
  $string = 'Hello World!';
  if (stristr($string, 'earth') === FALSE) {
    echo '"earth" not found in string';
  }
// outputs: "earth" not found in string
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- strstr() - Find the first occurrence of a string
- strrchr() - Find the last occurrence of a character in a string
- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- strpbrk() - Search a string for any of a set of characters
- preg_match() - Perform a regular expression match

strlen

strlen

(PHP 4, PHP 5, PHP 7, PHP 8)

strlen — Get string length

Description

function **strlen**(string `$string`): int

Returns the length of the given `string`.

Parameters

`string`

The string being measured for length.

Return Values

The length of the `string` in bytes.

Examples

Example #1 A strlen() example

```php
<?php
$str = 'abcdef';
echo strlen($str), PHP_EOL; // 6

$str = ' ab cd ';
echo strlen($str), PHP_EOL; // 7
?>
```

Notes

> **Note** :
>
> **strlen()** returns the number of bytes rather than the number of characters in a string.

See Also

- count() - Counts all elements in an array or in a Countable object
- grapheme_strlen() - Get string length in grapheme units
- iconv_strlen() - Returns the character count of string
- mb_strlen() - Get string length

strnatcasecmp

strnatcasecmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strnatcasecmp — Case insensitive string comparisons using a "natural order" algorithm

Description

function **strnatcasecmp**(string `$string1`, string `$string2`): int

This function implements a comparison algorithm that orders alphanumeric strings in the way a human being would. The behaviour of this function is similar to strnatcmp(), except that the comparison is not case sensitive. For more information see: Martin Pool's » Natural Order String Comparison page.

Parameters

`string1`

The first string.

`string2`

The second string.

Return Values

Returns a value less than 0 if `string1` is less than `string2`; a value greater than 0 if `string1` is greater than `string2`, and `0` if they are equal. No particular meaning can be reliably inferred from the value aside from its sign.

Changelog

Version	Description
8.2.0	This function is no longer guaranteed to return `strlen($string1) - strlen($string2)` when string lengths are not equal, but may now return `-1` or `1` instead.

Examples

Example #1 strnatcasecmp() example

```php
<?php

var_dump(strnatcasecmp('Apple', 'Banana'));
var_dump(strnatcasecmp('Banana', 'Apple'));
var_dump(strnatcasecmp('apple', 'Apple'));
?>
```

The above example will output:

```
int(-1)
int(1)
int(0)
```

See Also

- preg_match() - Perform a regular expression match
- strcmp() - Binary safe string comparison
- strcasecmp() - Binary safe case-insensitive string comparison
- substr() - Return part of a string
- stristr() - Case-insensitive strstr
- strncasecmp() - Binary safe case-insensitive string comparison of the first n characters
- strncmp() - Binary safe string comparison of the first n characters
- strstr() - Find the first occurrence of a string
- setlocale() - Set locale information

strnatcmp

strnatcmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strnatcmp — String comparisons using a "natural order" algorithm

Description

function **strnatcmp**(string `$string1`, string `$string2`): int

This function implements a comparison algorithm that orders alphanumeric strings in the way a human being would, this is described as a "natural ordering". Note that this comparison is case sensitive.

Parameters

`string1`

The first string.

`string2`

The second string.

Return Values

Returns a value less than 0 if `string1` is less than `string2`; a value greater than 0 if `string1` is greater than `string2`, and `0` if they are equal. No particular meaning can be reliably inferred from the value aside from its sign.

Changelog

Version	Description
8.2.0	This function is no longer guaranteed to return `strlen($string1) - strlen($string2)` when string lengths are not equal, but may now return `-1` or `1` instead.

Examples

An example of the difference between this algorithm and the regular computer string sorting algorithms (used in strcmp()) can be seen below:

Example #1 strcmp()

```php
<?php
$arr1 = $arr2 = array("img12.png", "img10.png", "img2.png", "img1.png");
echo "Standard string comparison\n";
usort($arr1, "strcmp");
print_r($arr1);
echo "\nNatural order string comparison\n";
usort($arr2, "strnatcmp");
print_r($arr2);
?>
```

The above example will output:

```
Standard string comparison
Array
(
    [0] => img1.png
    [1] => img10.png
    [2] => img12.png
    [3] => img2.png
)

Natural order string comparison
Array
(
    [0] => img1.png
    [1] => img2.png
    [2] => img10.png
```

```
    [3] => img12.png
)
```

For more information see: Martin Pool's » Natural Order String Comparison page.

See Also

- preg_match() - Perform a regular expression match
- strcasecmp() - Binary safe case-insensitive string comparison
- substr() - Return part of a string
- stristr() - Case-insensitive strstr
- strcmp() - Binary safe string comparison
- strncmp() - Binary safe string comparison of the first n characters
- strncasecmp() - Binary safe case-insensitive string comparison of the first n characters
- strnatcasecmp() - Case insensitive string comparisons using a "natural order" algorithm
- strstr() - Find the first occurrence of a string
- natsort() - Sort an array using a "natural order" algorithm
- natcasesort() - Sort an array using a case insensitive "natural order" algorithm

strncasecmp

strncasecmp

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

strncasecmp — Binary safe case-insensitive string comparison of the first n characters

Description

function **strncasecmp**(string `$string1`, string `$string2`, int `$length`): int

This function is similar to strcasecmp(), with the difference that you can specify the (upper limit of the) number of characters from each string to be used in the comparison.

Parameters

`string1`

The first string.

`string2`

The second string.

`length`

The length of strings to be used in the comparison.

Return Values

Returns a value less than 0 if `string1` is less than `string2`; a value greater than 0 if `string1` is greater than `string2`, and `0` if they are equal. No particular meaning can be reliably inferred from the value aside from its sign.

Changelog

Version	Description
8.2.0	This function is no longer guaranteed to return `strlen($string1) - strlen($string2)` when string lengths are not equal, but may now return `-1` or `1` instead.

Examples

Example #1 **strncasecmp()** example

```
<?php

$var1 = 'Hello John';
```

```php
$var2 = 'hello Doe';
if (strncasecmp($var1, $var2, 5) === 0) {
    echo 'First 5 characters of $var1 and $var2 are equal in a case-insensitive string comparison';
}
?>
```

See Also

- strncmp() - Binary safe string comparison of the first n characters
- preg_match() - Perform a regular expression match
- substr_compare() - Binary safe comparison of two strings from an offset, up to length characters
- strcasecmp() - Binary safe case-insensitive string comparison
- stristr() - Case-insensitive strstr
- substr() - Return part of a string

strncmp

strncmp

(PHP 4, PHP 5, PHP 7, PHP 8)

strncmp — Binary safe string comparison of the first n characters

Description

function **strncmp**(string `$string1`, string `$string2`, int `$length`): int

This function is similar to strcmp(), with the difference that you can specify the (upper limit of the) number of characters from each string to be used in the comparison.

Note that this comparison is case sensitive.

Parameters

`string1`

The first string.

`string2`

The second string.

`length`

Number of characters to use in the comparison.

Return Values

Returns a value less than 0 if `string1` is less than `string2`; a value greater than 0 if `string1` is greater than `string2`, and `0` if they are equal. No particular meaning can be reliably inferred from the value aside from its sign.

Changelog

Version	Description
8.2.0	This function is no longer guaranteed to return `strlen($string1) - strlen($string2)` when string lengths are not equal, but may now return `-1` or `1` instead.

Examples

Example #1 strncmp() example

```php
<?php

$var1 = 'Hello John';
$var2 = 'Hello Doe';
if (strncmp($var1, $var2, 5) === 0) {
    echo 'First 5 characters of $var1 and $var2 are equal in a case-sensitive string comparison';
}
?>
```

See Also

- strncasecmp() - Binary safe case-insensitive string comparison of the first n characters
- preg_match() - Perform a regular expression match
- substr_compare() - Binary safe comparison of two strings from an offset, up to length characters
- strcmp() - Binary safe string comparison
- strstr() - Find the first occurrence of a string
- substr() - Return part of a string

strpbrk

strpbrk

(PHP 5, PHP 7, PHP 8)

strpbrk — Search a string for any of a set of characters

Description

function **strpbrk**(string `$string`, string `$characters`): string|false

strpbrk() searches the `string` string for any of the characters in `characters`.

Parameters

`string`

The string where `characters` is looked for.

`characters`

This parameter is case sensitive.

Return Values

Returns a string starting from the character found, or `false` if it is not found.

Examples

Example #1 strpbrk() example

```php
<?php

$text = 'This is a Simple text.';

// this echoes "is is a Simple text." because 'i' is matched first
echo strpbrk($text, 'mi'), PHP_EOL;

// this echoes "Simple text." because chars are case sensitive
echo strpbrk($text, 'S'), PHP_EOL;
?>
```

See Also

- strpos() - Find the position of the first occurrence of a substring in a string
- strstr() - Find the first occurrence of a string
- preg_match() - Perform a regular expression match

strpos

strpos

(PHP 4, PHP 5, PHP 7, PHP 8)

strpos — Find the position of the first occurrence of a substring in a string

Description

function **strpos**(string `$haystack`, string `$needle`, int `$offset` = 0): int|false

Find the numeric position of the first occurrence of `needle` in the `haystack` string.

Parameters

`haystack`

The string to search in.

`needle`

The string to search for.

Passing an empty string as `needle` matches at every position. **strpos()** returns `offset` when it is specified, or `0` otherwise.

Prior to PHP 8.0.0, if `needle` is not a string, it is converted to an integer and applied as the ordinal value of a character. This behavior is deprecated as of PHP 7.3.0, and relying on it is highly discouraged. Depending on the intended behavior, the `needle` should either be explicitly cast to string, or an explicit call to chr() should be performed.

`offset`

If specified, search will start this number of characters counted from the beginning of the string. If the offset is negative, the search will start this number of characters counted from the end of the string.

Return Values

Returns the position of where the needle exists relative to the beginning of the `haystack` string (independent of `offset`). Also note that string positions start at `0`, and not `1`.

Returns **`false`** if the needle was not found.

> **Warning**
>
> This function may return Boolean **`false`**, but may also return a non-Boolean value which evaluates to **`false`**. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing an int as `needle` is no longer supported.
7.3.0	Passing an int as `needle` has been deprecated.
7.1.0	Support for negative `offset`s has been added.

Examples

Example #1 Using `===`

```php
<?php
$mystring = 'abc';
$findme   = 'a';
$pos = strpos($mystring, $findme);

// Note our use of ===.  Simply == would not work as expected
// because the position of 'a' was the 0th (first) character.
if ($pos === false) {
    echo "The string '$findme' was not found in the string '$mystring'";
} else {
    echo "The string '$findme' was found in the string '$mystring'";
    echo " and exists at position $pos";
}
?>
```

Example #2 Using !==

```php
<?php
$mystring = 'abc';
$findme   = 'a';
$pos = strpos($mystring, $findme);

// The !== operator can also be used.  Using != would not work as expected
// because the position of 'a' is 0. The statement (0 != false) evaluates
// to false.
if ($pos !== false) {
     echo "The string '$findme' was found in the string '$mystring'";
         echo " and exists at position $pos";
} else {
     echo "The string '$findme' was not found in the string '$mystring'";
}
?>
```

Example #3 Using an offset

```php
<?php
// We can search for the character, ignoring anything before the offset
$newstring = 'abcdef abcdef';
$pos = strpos($newstring, 'a', 1); // $pos = 7, not 0

echo $pos, PHP_EOL;
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- str_contains() - Determine if a string contains a given substring
- str_ends_with() - Checks if a string ends with a given substring
- str_starts_with() - Checks if a string starts with a given substring
- strrpos() - Find the position of the last occurrence of a substring in a string
- strripos() - Find the position of the last occurrence of a case-insensitive substring in a string
- strstr() - Find the first occurrence of a string
- strpbrk() - Search a string for any of a set of characters
- substr() - Return part of a string
- preg_match() - Perform a regular expression match

strrchr

strrchr

(PHP 4, PHP 5, PHP 7, PHP 8)

strrchr — Find the last occurrence of a character in a string

Description

function **strrchr**(string `$haystack`, string `$needle`, bool `$before_needle = false`): string|false

This function returns the portion of `haystack` which starts at the last occurrence of `needle` and goes until the end of `haystack`.

Parameters

`haystack`

The string to search in

`needle`

If `needle` contains more than one character, only the first is used. This behavior is different from that of strstr().

Prior to PHP 8.0.0, if `needle` is not a string, it is converted to an integer and applied as the ordinal value of a character. This behavior is deprecated as of PHP 7.3.0, and relying on it is highly discouraged. Depending on the intended behavior, the `needle` should either be explicitly cast to string, or an explicit call to chr() should be performed.

`before_needle`

If `true`, **strrchr()** returns the part of the `haystack` before the last occurrence of the `needle` (excluding the needle).

Return Values

This function returns the portion of string, or `false` if `needle` is not found.

Changelog

Version	Description
8.3.0	The `before_needle` parameter was added.
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing an int as `needle` is no longer supported.
7.3.0	Passing an int as `needle` has been deprecated.

Examples

Example #1 strrchr() example

```php
<?php
$ext = strrchr('somefile.txt', '.');
echo "file extension: $ext \n";
$ext = $ext ? strtolower(substr($ext, 1)) : '';
echo "file extension: $ext";
?>
```

The above example will output something similar to:

```
file extension: .txt
file extension: txt
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- strstr() - Find the first occurrence of a string
- strrpos() - Find the position of the last occurrence of a substring in a string

strrev

strrev

(PHP 4, PHP 5, PHP 7, PHP 8)

strrev — Reverse a string

Description

function **strrev**(string `$string`): string

Returns `string`, reversed.

Parameters

`string`

The string to be reversed.

Return Values

Returns the reversed string.

Examples

Example #1 Reversing a string with strrev()

```php
<?php
echo strrev("Hello world!"); // outputs "!dlrow olleH"
?>
```

strripos

strripos

(PHP 5, PHP 7, PHP 8)

strripos — Find the position of the last occurrence of a case-insensitive substring in a string

Description

function **strripos**(string `$haystack`, string `$needle`, int `$offset` = 0): int|false

Find the numeric position of the last occurrence of `needle` in the `haystack` string.

Unlike strrpos(), **strripos()** is case-insensitive.

Parameters

`haystack`

The string to search in.

`needle`

The string to search for.

Prior to PHP 8.0.0, if `needle` is not a string, it is converted to an integer and applied as the ordinal value of a character. This behavior is deprecated as of PHP 7.3.0, and relying on it is highly discouraged. Depending on the intended behavior, the `needle` should either be explicitly cast to string, or an explicit call to chr() should be performed.

`offset`

If zero or positive, the search is performed left to right skipping the first `offset` bytes of the `haystack`.

If negative, the search is performed right to left skipping the last `offset` bytes of the `haystack` and searching for the first occurrence of `needle`.

> **Note** :
>
> This is effectively looking for the last occurrence of `needle` before the last `offset` bytes.

Return Values

Returns the position where the `needle` exists relative to the beginning of the `haystack` string (independent of search direction or `offset`).

> **Note** :
>
> String positions start at 0, and not 1.

Returns `false` if the `needle` was not found.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.2.0	Case folding no longer depends on the locale set with setlocale(). Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing an int as `needle` is no longer supported.
7.3.0	Passing an int as `needle` has been deprecated.

Examples

Example #1 A simple strripos() example

```php
<?php

$haystack = 'ababcd';
$needle   = 'aB';

$pos      = strripos($haystack, $needle);

if ($pos === false) {
    echo "Sorry, we did not find `$needle` in `$haystack`";
} else {
    echo "Congratulations!\n";
    echo "We found the last `$needle` in `$haystack` at position `$pos`";
}

?>
```

The above example will output:

```
Congratulations!
We found the last `aB` in `ababcd` at position `2`
```

See Also

- strpos() - Find the position of the first occurrence of a substring in a string
- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- strrpos() - Find the position of the last occurrence of a substring in a string
- strrchr() - Find the last occurrence of a character in a string
- stristr() - Case-insensitive strstr
- substr() - Return part of a string

strrpos

strrpos

(PHP 4, PHP 5, PHP 7, PHP 8)

strrpos — Find the position of the last occurrence of a substring in a string

Description

function **strrpos**(string `$haystack`, string `$needle`, int `$offset` = 0): int|false

Find the numeric position of the last occurrence of `needle` in the `haystack` string.

Parameters

`haystack`

The string to search in.

`needle`

The string to search for.

Prior to PHP 8.0.0, if `needle` is not a string, it is converted to an integer and applied as the ordinal value of a character. This behavior is deprecated as of PHP 7.3.0, and relying on it is highly discouraged. Depending on the intended behavior, the `needle` should either be explicitly cast to string, or an explicit call to chr() should be performed.

`offset`

If zero or positive, the search is performed left to right skipping the first `offset` bytes of the `haystack`.

If negative, the search starts `offset` bytes from the right instead of from the beginning of `haystack`. The search is performed right to left, searching for the first occurrence of `needle` from the selected byte.

> **Note** :
>
> This is effectively looking for the last occurrence of `needle` at or before the last `offset` bytes.

Return Values

Returns the position where the needle exists relative to the beginning of the `haystack` string (independent of search direction or offset).

> **Note** :
>
> String positions start at 0, and not 1.

Returns `false` if the needle was not found.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing an int as `needle` is no longer supported.
7.3.0	Passing an int as `needle` has been deprecated.

Examples

Example #1 Checking if a needle is in the haystack

It is easy to mistake the return values for "character found at position 0" and "character not found". Here's how to detect the difference:

```php
<?php
$mystring = 'Elephpant';

$pos = strrpos($mystring, "b");
if ($pos === false) { // note: three equal signs
    // not found...
}

?>
```

Example #2 Searching with offsets

```php
<?php
$foo = "0123456789a123456789b123456789c";

// Looking for '0' from the 0th byte (from the beginning)
var_dump(strrpos($foo, '0', 0));

// Looking for '0' from the 1st byte (after byte "0")
var_dump(strrpos($foo, '0', 1));

// Looking for '7' from the 21th byte (after byte 20)
var_dump(strrpos($foo, '7', 20));

// Looking for '7' from the 29th byte (after byte 28)
var_dump(strrpos($foo, '7', 28));

// Looking for '7' right to left from the 5th byte from the end
var_dump(strrpos($foo, '7', -5));

// Looking for 'c' right to left from the 2nd byte from the end
var_dump(strrpos($foo, 'c', -2));

// Looking for '9c' right to left from the 2nd byte from the end
var_dump(strrpos($foo, '9c', -2));
?>
```

The above example will output:

```
int(0)
bool(false)
int(27)
bool(false)
int(17)
bool(false)
int(29)
```

See Also

- strpos() - Find the position of the first occurrence of a substring in a string
- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- strripos() - Find the position of the last occurrence of a case-insensitive substring in a string
- strrchr() - Find the last occurrence of a character in a string
- substr() - Return part of a string

strspn

strspn

(PHP 4, PHP 5, PHP 7, PHP 8)

strspn — Finds the length of the initial segment of a string consisting entirely of characters contained within a given mask

Description

function **strspn**(
    string $string,
    string $characters,
    int $offset = 0,
    ?int $length = null
): int

Finds the length of the initial segment of `string` that contains *only* characters from `characters`.

If `offset` and `length` are omitted, then all of `string` will be examined. If they are included, then the effect will be the same as calling `strspn(substr($string, $offset, $length), $characters)` (see substr for more information).

The line of code:

```
<?php
$var = strspn("42 is the answer to the 128th question.", "1234567890");
?>
```

will assign `2` to *$var*, because the string "42" is the initial segment of `string` that consists only of characters contained within "1234567890".

Parameters

`string`

The string to examine.

`characters`

The list of allowable characters.

`offset`

The position in `string` to start searching.

If `offset` is given and is non-negative, then **strspn()** will begin examining `string` at the `offset`'th position. For instance, in the string '`abcdef`', the character at position `0` is '`a`', the character at position `2` is '`c`', and so forth.

If `offset` is given and is negative, then **strspn()** will begin examining `string` at the `offset`'th position from the end of `string`.

`length`

The length of the segment from `string` to examine.

If `length` is given and is non-negative, then `string` will be examined for `length` characters after the starting position.

If `length` is given and is negative, then `string` will be examined from the starting position up to `length` characters from the end of `string`.

Return Values

Returns the length of the initial segment of `string` which consists entirely of characters in `characters`.

> **Note** :
>
> When a `offset` parameter is set, the returned length is counted starting from this position, not from the beginning of `string`.

Changelog

Version	Description
8.0.0	`length` is nullable now.

Examples

Example #1 strspn() example

```
<?php
// subject does not start with any characters from mask
```

```php
var_dump(strspn("foo", "o"));

// examine two characters from subject starting at offset 1
var_dump(strspn("foo", "o", 1, 2));

// examine one character from subject starting at offset 1
var_dump(strspn("foo", "o", 1, 1));
?>
```

The above example will output:

```
int(0)
int(2)
int(1)
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- strcspn() - Find length of initial segment not matching mask

strstr

strstr

(PHP 4, PHP 5, PHP 7, PHP 8)

strstr — Find the first occurrence of a string

Description

function **strstr**(string `$haystack`, string `$needle`, bool `$before_needle = false`): string|false

Returns part of `haystack` string starting from and including the first occurrence of `needle` to the end of `haystack`.

> **Note** :
>
> This function is case-sensitive. For case-insensitive searches, use stristr().

> **Note** :
>
> If it is only required to determine if a particular `needle` occurs within `haystack`, the faster and less memory intensive str_contains() function should be used instead.

Parameters

`haystack`

The input string.

`needle`

The string to search for.

Prior to PHP 8.0.0, if `needle` is not a string, it is converted to an integer and applied as the ordinal value of a character. This behavior is deprecated as of PHP 7.3.0, and relying on it is highly discouraged. Depending on the intended behavior, the `needle` should either

be explicitly cast to string, or an explicit call to chr() should be performed.

`before_needle`

If `true`, **strstr()** returns the part of the `haystack` before the first occurrence of the `needle` (excluding the needle).

Return Values

Returns the portion of string, or `false` if `needle` is not found.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing an int as `needle` is no longer supported.
7.3.0	Passing an int as `needle` has been deprecated.

Examples

Example #1 strstr() example

```php
<?php
$email  = 'name@example.com';
$domain = strstr($email, '@');
echo $domain, PHP_EOL; // prints @example.com

$user = strstr($email, '@', true);
echo $user, PHP_EOL; // prints name
?>
```

See Also

- stristr() - Case-insensitive strstr
- strrchr() - Find the last occurrence of a character in a string
- strpos() - Find the position of the first occurrence of a substring in a string
- strpbrk() - Search a string for any of a set of characters
- preg_match() - Perform a regular expression match

strtok

strtok

(PHP 4, PHP 5, PHP 7, PHP 8)

strtok — Tokenize string

Description

function **strtok**(string `$string`, string `$token`): string|false

Alternative signature (not supported with named arguments):

function **strtok**(string `$token`): string|false

strtok() splits a string (`string`) into smaller strings (tokens), with each token being delimited by any character from `token`. That is, if you have a string like "This is an example string" you could tokenize this string into its individual words by using the space character as the `token`.

Note that only the first call to strtok uses the `string` argument. Every subsequent call to strtok only needs the `token` to use, as it keeps track of where it is in the current string. To start over, or to tokenize a new string you simply call strtok with the `string` argument again to initialize it. Note that you may put multiple tokens in the `token` parameter. The string will be tokenized when any one of the characters in the `token` argument is found.

> **Note** :
>
> This function behaves slightly different from what one may expect being familiar with explode(). First, a sequence of two or more contiguous `token` characters in the parsed string is considered to be a single delimiter. Also, a `token` situated at the start or end of the

string is ignored. For example, if a string `";aaa;;bbb;"` is used, successive calls to **strtok()** with `";"` as a `token` would return strings "aaa" and "bbb", and then `false`. As a result, the string will be split into only two elements, while `explode(";", $string)` would return an array of 5 elements.

Parameters

`string`

The string being split up into smaller strings (tokens).

`token`

The delimiter used when splitting up `string`.

Return Values

A string token, or `false` if no more tokens are available.

Changelog

Version	Description
8.3.0	Now emits `E_WARNING` when `token` is not provided.

Examples

Example #1 strtok() example

```php
<?php
$string = "This is\tan example\nstring";
/* Use tab and newline as tokenizing characters as well  */
$tok = strtok($string, " \n\t");

while ($tok !== false) {
    echo "Word={$tok}\n";
    $tok = strtok(" \n\t");
}
?>
```

Example #2 strtok() behavior on empty part found

```php
<?php
$first_token  = strtok('/something', '/');
$second_token = strtok('/');
var_dump($first_token, $second_token);
?>
```

The above example will output:

```
string(9) "something"
bool(false)
```

Example #3 The difference between strtok() and explode()

```php
<?php
$string = ";aaa;;bbb;";

$parts = [];
$tok = strtok($string, ";");
while ($tok !== false) {
    $parts[] = $tok;
    $tok = strtok(";");
}
```

```php
echo json_encode($parts),"\n";

$parts = explode(";", $string);
echo json_encode($parts),"\n";
```

The above example will output:

```
["aaa","bbb"]
["","aaa","","bbb",""]
```

Notes

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

See Also

- explode() - Split a string by a string

strtolower

strtolower

(PHP 4, PHP 5, PHP 7, PHP 8)

strtolower — Make a string lowercase

Description

function **strtolower**(string `$string`): string

Returns `string` with all ASCII alphabetic characters converted to lowercase.

Bytes in the range `"A"` (0x41) to `"Z"` (0x5a) will be converted to the corresponding lowercase letter by adding 32 to each byte value.

This can be used to convert ASCII characters within strings encoded with UTF-8, since multibyte UTF-8 characters will be ignored. To convert multibyte non-ASCII characters, use mb_strtolower().

Parameters

`string`

The input string.

Return Values

Returns the lowercased string.

Changelog

Version	Description
8.2.0	Case conversion no longer depends on the locale set with setlocale(). Only ASCII characters will be converted.

Examples

Example #1 strtolower() example

```php
<?php
$str = "Mary Had A Little Lamb and She LOVED It So";
$str = strtolower($str);
```

```
echo $str; // Prints mary had a little lamb and she loved it so
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- strtoupper() - Make a string uppercase
- ucfirst() - Make a string's first character uppercase
- ucwords() - Uppercase the first character of each word in a string
- mb_strtolower() - Make a string lowercase

strtoupper

strtoupper

(PHP 4, PHP 5, PHP 7, PHP 8)

strtoupper — Make a string uppercase

Description

function **strtoupper**(string `$string`): string

Returns `string` with all ASCII alphabetic characters converted to uppercase.

Bytes in the range `"a"` (0x61) to `"z"` (0x7a) will be converted to the corresponding uppercase letter by subtracting 32 from each byte value.

This can be used to convert ASCII characters within strings encoded with UTF-8, since multibyte UTF-8 characters will be ignored. To convert multibyte non-ASCII characters, use mb_strtoupper().

Parameters

`string`

The input string.

Return Values

Returns the uppercased string.

Changelog

Version	Description
8.2.0	Case conversion no longer depends on the locale set with setlocale(). Only ASCII characters will be converted.

Examples

Example #1 strtoupper() example

```
<?php
$str = "Mary Had A Little Lamb and She LOVED It So";
$str = strtoupper($str);
echo $str; // Prints MARY HAD A LITTLE LAMB AND SHE LOVED IT SO
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- strtolower() - Make a string lowercase
- ucfirst() - Make a string's first character uppercase
- ucwords() - Uppercase the first character of each word in a string
- mb_strtoupper() - Make a string uppercase

strtr

strtr

(PHP 4, PHP 5, PHP 7, PHP 8)

strtr — Translate characters or replace substrings

Description

function **strtr**(string `$string`, string `$from`, string `$to`): string

Alternative signature (not supported with named arguments):

function **strtr**(string `$string`, array `$replace_pairs`): string

If given three arguments, this function returns a copy of `string` where all occurrences of each (single-byte) character in `from` have been translated to the corresponding character in `to`, i.e., every occurrence of `$from[$n]` has been replaced with `$to[$n]`, where `$n` is a valid offset in both arguments.

If `from` and `to` have different lengths, the extra characters in the longer of the two are ignored. The length of `string` will be the same as the return value's.

If given two arguments, the second should be an array in the form `array('from' => 'to', ...)`. The return value is a string where all the occurrences of the array keys have been replaced by the corresponding values. The longest keys will be tried first. Once a substring has been replaced, its new value will not be searched again.

In this case, the keys and the values may have any length, provided that there is no empty key; additionally, the length of the return value may differ from that of `string`. However, this function will be the most efficient when all the keys have the same size.

Parameters

`string`

The string being translated.

`from`

The string being translated to `to`.

`to`

The string replacing `from`.

`replace_pairs`

The `replace_pairs` parameter may be used instead of `to` and `from`, in which case it's an array in the form `array('from' => 'to', ...)`.

If `replace_pairs` contains a key which is an empty string (`""`), the element is ignored; as of PHP 8.0.0 `E_WARNING` is raised in this case.

Return Values

Returns the translated string.

Examples

Example #1 strtr() example

```php
<?php

$original = "He1Lo, w0rld!";

// With three arguments, strtr() performs byte-by-byte translation.
```

```php
$result = strtr($original, "1L0", "llo");
echo $result;
```

The above example will output:

```
Hello, world!
```

The next example shows the behavior of **strtr()** when called with only two arguments. Note the preference of the replacements (`"h"` is not picked because there are longer matches) and how replaced text was not searched again.

Example #2 strtr() example with two arguments

```php
<?php

$replace_pairs = ["h" => "-", "hello" => "hi", "hi" => "hello"];
echo strtr("hi all, I said hello", $replace_pairs);
```

The above example will output:

```
hello all, I said hi
```

The two modes of behavior are substantially different. With three arguments, **strtr()** will replace bytes; with two, it may replace longer substrings.

Example #3 strtr() behavior comparison

```php
<?php

// Single-byte characters
$singlebyte_string = "baab";
echo strtr($singlebyte_string, "ab", "01"), "\n"; // Byte-by-byte translation

$replace_pairs = ["ab" => "01"];
echo strtr($singlebyte_string, $replace_pairs), "\n"; // Substring translation

// Multi-byte characters
$multibyte_string = "Ä"; // In UTF-8, encoded as two bytes: "\xC3\x84"

$result = strtr($multibyte_string, 'Ä', 'A');
echo bin2hex($result), "\n"; // "4184", where "c3" becomes "41" ("A"), while "84" is kept and breaks UTF-8

$replace_pairs = ['Ä' => 'A'];
echo strtr($multibyte_string, $replace_pairs); // A correct multibyte substring translation
```

The above example will output:

```
1001
ba01
4184
A
```

See Also

- str_replace() - Replace all occurrences of the search string with the replacement string
- preg_replace() - Perform a regular expression search and replace

substr

substr

(PHP 4, PHP 5, PHP 7, PHP 8)

substr — Return part of a string

Description

function **substr**(string $string, int $offset, ?int $length = null): string

Returns the portion of `string` specified by the `offset` and `length` parameters.

Parameters

`string`

The input string.

`offset`

If `offset` is non-negative, the returned string will start at the `offset`'th position in `string`, counting from zero. For instance, in the string 'abcdef', the character at position `0` is 'a', the character at position `2` is 'c', and so forth.

If `offset` is negative, the returned string will start at the `offset`'th character from the end of `string`.

If `string` is less than `offset` characters long, an empty string will be returned.

Example #1 Using a negative `offset`

```php
<?php
echo substr("abcdef", -1), PHP_EOL;    // returns "f"
echo substr("abcdef", -2), PHP_EOL;    // returns "ef"
echo substr("abcdef", -3, 1), PHP_EOL; // returns "d"
?>
```

`length`

If `length` is given and is positive, the string returned will contain at most `length` characters beginning from `offset` (depending on the length of `string`).

If `length` is given and is negative, then that many characters will be omitted from the end of `string`. If `offset` denotes the position of this truncation or beyond, an empty string will be returned.

If `length` is given and is `0`, an empty string will be returned.

If `length` is omitted or `null`, the substring starting from `offset` until the end of the string will be returned.

Example #2 Using a negative `length`

```php
<?php
echo substr("abcdef", 0, -1), PHP_EOL;  // returns "abcde"
echo substr("abcdef", 2, -1), PHP_EOL;  // returns "cde"
echo substr("abcdef", 4, -4), PHP_EOL;  // returns ""; prior to PHP 8.0.0, false was returned
echo substr("abcdef", -3, -1), PHP_EOL; // returns "de"
?>
```

Return Values

Returns the extracted part of `string`, or an empty string.

Changelog

Version	Description
8.0.0	`length` is nullable now. When `length` is explicitly set to `null`, the function returns a substring finishing at the end of the string, when it previously returned an empty string.

Version	Description
8.0.0	The function returns an empty string where it previously returned `false`.

Examples

Example #3 Basic **substr()** usage

```php
<?php
echo substr('abcdef', 1), PHP_EOL;     // bcdef
echo substr("abcdef", 1, null), PHP_EOL; // bcdef; prior to PHP 8.0.0, empty string was returned
echo substr('abcdef', 1, 3), PHP_EOL;  // bcd
echo substr('abcdef', 0, 4), PHP_EOL;  // abcd
echo substr('abcdef', 0, 8), PHP_EOL;  // abcdef
echo substr('abcdef', -1, 1), PHP_EOL; // f

// Accessing single characters in a string
// can also be achieved using "square brackets"
$string = 'abcdef';
echo $string[0], PHP_EOL;                 // a
echo $string[3], PHP_EOL;                 // d
echo $string[strlen($string)-1], PHP_EOL; // f

?>
```

Example #4 **substr()** casting behaviour

```php
<?php
class apple {
    public function __toString() {
        return "green";
    }
}

echo "1) ", var_export(substr("pear", 0, 2), true), PHP_EOL;
echo "2) ", var_export(substr(54321, 0, 2), true), PHP_EOL;
echo "3) ", var_export(substr(new apple(), 0, 2), true), PHP_EOL;
echo "4) ", var_export(substr(true, 0, 1), true), PHP_EOL;
echo "5) ", var_export(substr(false, 0, 1), true), PHP_EOL;
echo "6) ", var_export(substr("", 0, 1), true), PHP_EOL;
echo "7) ", var_export(substr(1.2e3, 0, 4), true), PHP_EOL;
?>
```

The above example will output:

```
1) 'pe'
2) '54'
3) 'gr'
4) '1'
5) ''
6) ''
7) '1200'
```

Example #5 Invalid Character Range

If an invalid character range is requested, **substr()** returns an empty string as of PHP 8.0.0; previously, `false` was returned instead.

```php
<?php
var_dump(substr('a', 2));
?>
```

Output of the above example in PHP 8:

```
string(0) ""
```

Output of the above example in PHP 7:

```
bool(false)
```

See Also

- strrchr() - Find the last occurrence of a character in a string
- substr_replace() - Replace text within a portion of a string
- preg_match() - Perform a regular expression match
- trim() - Strip whitespace (or other characters) from the beginning and end of a string
- mb_substr() - Get part of string
- wordwrap() - Wraps a string to a given number of characters
- String access and modification by character

substr_compare

substr_compare

(PHP 5, PHP 7, PHP 8)

substr_compare — Binary safe comparison of two strings from an offset, up to length characters

Description

```
function substr_compare(
    string $haystack,
    string $needle,
    int $offset,
    ?int $length = null,
    bool $case_insensitive = false
): int
```

substr_compare() compares `haystack` from position `offset` with `needle` up to `length` characters.

Parameters

`haystack`

The main string being compared.

`needle`

The secondary string being compared.

`offset`

The start position for the comparison. If negative, it starts counting from the end of the string.

`length`

The length of the comparison. The default value is the largest of the length of the `needle` compared to the length of `haystack` minus the `offset`.

`case_insensitive`

If `case_insensitive` is `true`, comparison is case insensitive.

Return Values

Returns a value less than 0 if `string1` is less than `string2`; a value greater than 0 if `string1` is greater than `string2`, and `0` if they are equal. No particular meaning can be reliably inferred from the value aside from its sign.

If `offset` is equal to (prior to PHP 7.2.18, 7.3.5) or greater than the length of `haystack`, or the `length` is set and is less than 0, **substr_compare()** prints a warning and returns `false`.

Changelog

Version	Description
8.2.0	This function is no longer guaranteed to return `strlen($string1) - strlen($string2)` when string lengths are not equal, but may now return `-1` or `1` instead.
8.0.0	`length` is nullable now.
7.2.18, 7.3.5	`offset` may now be equal to the length of `haystack`.

Examples

Example #1 A substr_compare() example

```php
<?php
echo substr_compare("abcde", "bc", 1, 2), PHP_EOL; // 0
echo substr_compare("abcde", "de", -2, 2), PHP_EOL; // 0
echo substr_compare("abcde", "bcg", 1, 2), PHP_EOL; // 0
echo substr_compare("abcde", "BC", 1, 2, true), PHP_EOL; // 0
echo substr_compare("abcde", "bc", 1, 3), PHP_EOL; // 1
echo substr_compare("abcde", "cd", 1, 2), PHP_EOL; // -1
echo substr_compare("abcde", "abc", 5, 1), PHP_EOL; // -1
?>
```

See Also

- strncmp() - Binary safe string comparison of the first n characters

substr_count

substr_count

(PHP 4, PHP 5, PHP 7, PHP 8)

substr_count — Count the number of substring occurrences

Description

```
function substr_count(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?int $length = null
): int
```

substr_count() returns the number of times the `needle` substring occurs in the `haystack` string. Please note that `needle` is case sensitive.

> **Note** :
>
> This function doesn't count overlapped substrings. See the example below!

Parameters

`haystack`

The string to search in

`needle`

The substring to search for

`offset`

The offset where to start counting. If the offset is negative, counting starts from the end of the string.

`length`

The maximum length after the specified offset to search for the substring. It outputs a warning if the offset plus the length is greater than the `haystack` length. A negative length counts from the end of `haystack`.

Return Values

This function returns an int.

Changelog

Version	Description
8.0.0	`length` is nullable now.
7.1.0	Support for negative `offset`s and `length`s has been added. `length` may also be `0` now.

Examples

Example #1 A substr_count() example

```php
<?php
$text = 'This is a test';
echo strlen($text), PHP_EOL; // 14

echo substr_count($text, 'is'), PHP_EOL; // 2

// the string is reduced to 's is a test', so it prints 1
echo substr_count($text, 'is', 3), PHP_EOL;

// the text is reduced to 's i', so it prints 0
echo substr_count($text, 'is', 3, 3), PHP_EOL;

// prints only 1, because it doesn't count overlapped substrings
$text2 = 'gcdgcdgcd';
echo substr_count($text2, 'gcdgcd'), PHP_EOL;

// throws an exception because 5+10 > 14
echo substr_count($text, 'is', 5, 10), PHP_EOL;
?>
```

See Also

- count_chars() - Return information about characters used in a string
- strpos() - Find the position of the first occurrence of a substring in a string
- substr() - Return part of a string
- strstr() - Find the first occurrence of a string

substr_replace

substr_replace

(PHP 4, PHP 5, PHP 7, PHP 8)

substr_replace — Replace text within a portion of a string

Description

```
function substr_replace(
    array|string $string,
    array|string $replace,
    array|int $offset,
    array|int|null $length = null
): string|array
```

substr_replace() replaces a copy of `string` delimited by the `offset` and (optionally) `length` parameters with the string given in `replace`.

Parameters

`string`

The input string.

An array of strings can be provided, in which case the replacements will occur on each string in turn. In this case, the `replace`, `offset` and `length` parameters may be provided either as scalar values to be applied to each input string in turn, or as arrays, in which case the corresponding array element will be used for each input string.

`replace`

The replacement string.

`offset`

If `offset` is non-negative, the replacing will begin at the `offset`'th offset into `string`.

If `offset` is negative, the replacing will begin at the `offset`'th character from the end of `string`.

`length`

If given and is positive, it represents the length of the portion of `string` which is to be replaced. If it is negative, it represents the number of characters from the end of `string` at which to stop replacing. If it is not given, then it will default to strlen(`string`); i.e. end the replacing at the end of `string`. Of course, if `length` is zero then this function will have the effect of inserting `replace` into `string` at the given `offset` offset.

Return Values

The result string is returned. If `string` is an array then array is returned.

Changelog

Version	Description
8.0.0	`length` is nullable now.

Examples

Example #1 Simple substr_replace() examples

```php
<?php
$var = 'ABCDEFGH:/MNRPQR/';
echo "Original: $var<hr />\n";

/* These two examples replace all of $var with 'bob'. */
echo substr_replace($var, 'bob', 0) . "<br />\n";
echo substr_replace($var, 'bob', 0, strlen($var)) . "<br />\n";

/* Insert 'bob' right at the beginning of $var. */
echo substr_replace($var, 'bob', 0, 0) . "<br />\n";

/* These next two replace 'MNRPQR' in $var with 'bob'. */
echo substr_replace($var, 'bob', 10, -1) . "<br />\n";
echo substr_replace($var, 'bob', -7, -1) . "<br />\n";

/* Delete 'MNRPQR' from $var. */
echo substr_replace($var, '', 10, -1) . "<br />\n";
?>
```

Example #2 Using substr_replace() to replace multiple strings at once

```php
<?php
$input = array('A: XXX', 'B: XXX', 'C: XXX');

// A simple case: replace XXX in each string with YYY.
echo implode('; ', substr_replace($input, 'YYY', 3, 3))."\n";

// A more complicated case where each replacement is different.
$replace = array('AAA', 'BBB', 'CCC');
```

```php
echo implode('; ', substr_replace($input, $replace, 3, 3))."\n";

// Replace a different number of characters each time.
$length = array(1, 2, 3);
echo implode('; ', substr_replace($input, $replace, 3, $length))."\n";
?>
```

The above example will output:

```
A: YYY; B: YYY; C: YYY
A: AAA; B: BBB; C: CCC
A: AAAXX; B: BBBX; C: CCC
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- str_replace() - Replace all occurrences of the search string with the replacement string
- substr() - Return part of a string
- String access and modification by character

trim

trim

(PHP 4, PHP 5, PHP 7, PHP 8)

trim — Strip whitespace (or other characters) from the beginning and end of a string

Description

function **trim**(string `$string`, string `$characters` = " \n\r\t\v\x00"): string

This function returns a string with whitespace stripped from the beginning and end of `string`. Without the second parameter, **trim()** will strip these characters:

- `" "`: ASCII SP character `0x20`, an ordinary space.
- `"\t"`: ASCII HT character `0x09`, a tab.
- `"\n"`: ASCII LF character `0x0A`, a new line (line feed).
- `"\r"`: ASCII CR character `0x0D`, a carriage return.
- `"\0"`: ASCII NUL character `0x00`, the NUL-byte.
- `"\v"`: ASCII VT character `0x0B`, a vertical tab.

Parameters

`string`

The string that will be trimmed.

`characters`

Optionally, the stripped characters can also be specified using the `characters` parameter. Simply list all characters that need to be stripped. With `..` it is possible to specify an incrementing range of characters.

Return Values

The trimmed string.

Examples

Example #1 Usage example of **trim()**

```php
<?php

$text   = "\t\tThese are a few words :) ...  ";
$binary = "\x09Example string\x0A";
$hello  = "Hello World";
var_dump($text, $binary, $hello);

print "\n";

$trimmed = trim($text);
var_dump($trimmed);

$trimmed = trim($text, " \t.");
var_dump($trimmed);

$trimmed = trim($hello, "Hdle");
var_dump($trimmed);

$trimmed = trim($hello, 'HdWr');
var_dump($trimmed);

// trim the ASCII control characters at the beginning and end of $binary
// (from 0 to 31 inclusive)
$clean = trim($binary, "\x00..\x1F");
var_dump($clean);

?>
```

The above example will output:

```
string(32) "            These are a few words :) ...  "
string(16) "    Example string
"
string(11) "Hello World"

string(28) "These are a few words :) ..."
string(24) "These are a few words :)"
string(5) "o Wor"
string(9) "ello Worl"
string(14) "Example string"
```

Example #2 Trimming array values with **trim()**

```php
<?php
function trim_value(&$value)
{
    $value = trim($value);
}

$fruit = array('apple','banana ', ' cranberry ');
var_dump($fruit);

array_walk($fruit, 'trim_value');
var_dump($fruit);

?>
```

The above example will output:

```
array(3) {
  [0]=>
  string(5) "apple"
  [1]=>
  string(7) "banana "
  [2]=>
  string(11) " cranberry "
}
array(3) {
  [0]=>
  string(5) "apple"
  [1]=>
  string(6) "banana"
  [2]=>
  string(9) "cranberry"
}
```

Notes

> **Note** : **Possible gotcha: removing middle characters**
>
> Because **trim()** trims characters from the beginning and end of a string, it may be confusing when characters are (or are not) removed from the middle. `trim('abc', 'bad')` removes both 'a' and 'b' because it trims 'a' thus moving 'b' to the beginning to also be trimmed. So, this is why it "works" whereas `trim('abc', 'b')` seemingly does not.

See Also

- ltrim() - Strip whitespace (or other characters) from the beginning of a string
- rtrim() - Strip whitespace (or other characters) from the end of a string
- str_replace() - Replace all occurrences of the search string with the replacement string

ucfirst

ucfirst

(PHP 4, PHP 5, PHP 7, PHP 8)

ucfirst — Make a string's first character uppercase

Description

function **ucfirst**(string `$string`): string

Returns a string with the first character of `string` capitalized, if that character is an ASCII character in the range from `"a"` (0x61) to `"z"` (0x7a).

Parameters

`string`

The input string.

Return Values

Returns the resulting string.

Changelog

Version	Description
8.2.0	Case conversion no longer depends on the locale set with setlocale(). Only ASCII characters will be converted.

Examples

Example #1 ucfirst() example

```php
<?php
$foo = 'hello world!';
echo ucfirst($foo), PHP_EOL;             // Hello world!

$bar = 'HELLO WORLD!';
echo ucfirst($bar), PHP_EOL;             // HELLO WORLD!
echo ucfirst(strtolower($bar)), PHP_EOL; // Hello world!
?>
```

See Also

- lcfirst() - Make a string's first character lowercase
- strtolower() - Make a string lowercase
- strtoupper() - Make a string uppercase
- ucwords() - Uppercase the first character of each word in a string
- mb_convert_case() - Perform case folding on a string

ucwords

ucwords

(PHP 4, PHP 5, PHP 7, PHP 8)

ucwords — Uppercase the first character of each word in a string

Description

function **ucwords**(string `$string`, string `$separators = " \t\r\n\f\v"`): string

Returns a string with the first character of each word in `string` capitalized, if that character is an ASCII character between `"a"` (0x61) and `"z"` (0x7a).

For this function, a word is a string of characters that are not listed in the `separators` parameter. By default, these are: space, horizontal tab, carriage return, newline, form-feed and vertical tab.

To do a similar conversion on multibyte strings, use mb_convert_case() with the `MB_CASE_TITLE` mode.

Parameters

`string`

The input string.

`separators`

The optional `separators` contains the word separator characters.

Return Values

Returns the modified string.

Changelog

Version	Description
8.2.0	Case conversion no longer depends on the locale set with setlocale(). Only ASCII characters will be converted.

Examples

Example #1 ucwords() example

```php
<?php
$foo = 'hello world!';
echo ucwords($foo), PHP_EOL;             // Hello World!

$bar = 'HELLO WORLD!';
```

```php
echo ucwords($bar), PHP_EOL;             // HELLO WORLD!
echo ucwords(strtolower($bar)), PHP_EOL; // Hello World!
?>
```

Example #2 ucwords() example with custom delimiter

```php
<?php
$foo = 'hello|world!';
echo ucwords($foo), PHP_EOL;             // Hello|world!

echo ucwords($foo, "|"), PHP_EOL;        // Hello|World!
?>
```

Example #3 ucwords() example with additional delimiters

```php
<?php
$foo = "mike o'hara";
echo ucwords($foo), PHP_EOL;                // Mike O'hara

echo ucwords($foo, " \t\r\n\f\v'"), PHP_EOL; // Mike O'Hara
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- strtoupper() - Make a string uppercase
- strtolower() - Make a string lowercase
- ucfirst() - Make a string's first character uppercase
- mb_convert_case() - Perform case folding on a string

utf8_decode

utf8_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

utf8_decode — Converts a string from UTF-8 to ISO-8859-1, replacing invalid or unrepresentable characters

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.2.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.2',
    message: 'visit the php.net documentation for various alternatives',
)]
function utf8_decode(string $string): string
```

This function converts the string `string` from the `UTF-8` encoding to `ISO-8859-1`. Bytes in the string which are not valid `UTF-8`, and `UTF-8` characters which do not exist in `ISO-8859-1` (that is, code points above `U+00FF`) are replaced with `?`.

> **Note** :
>
> Many web pages marked as using the `ISO-8859-1` character encoding actually use the similar `Windows-1252` encoding, and web browsers will interpret `ISO-8859-1` web pages as `Windows-1252`. `Windows-1252` features additional printable characters, such as the Euro sign (€) and curly quotes (“ ”), instead of certain `ISO-8859-1` control characters. This function will not convert such `Windows-1252` characters correctly. Use a different function if `Windows-1252` conversion is required.

Parameters

`string`

A UTF-8 encoded string.

Return Values

Returns the ISO-8859-1 translation of `string`.

Changelog

Version	Description
8.2.0	This function has been deprecated.
7.2.0	This function has been moved from the XML extension to the core of PHP. In previous versions, it was only available if the XML extension was installed.

Examples

Example #1 Basic examples

```php
<?php
// Convert the string 'Zoë' from UTF-8 to ISO 8859-1
$utf8_string = "\x5A\x6F\xC3\xAB";
$iso8859_1_string = utf8_decode($utf8_string);
echo bin2hex($iso8859_1_string), "\n";

// Invalid UTF-8 sequences are replaced with '?'
$invalid_utf8_string = "\xC3";
$iso8859_1_string = utf8_decode($invalid_utf8_string);
var_dump($iso8859_1_string);

// Characters which don't exist in ISO 8859-1, such as
// '€' (Euro Sign) are also replaced with '?'
$utf8_string = "\xE2\x82\xAC";
$iso8859_1_string = utf8_decode($utf8_string);
var_dump($iso8859_1_string);
?>
```

The above example will output:

```
5a6feb
string(1) "?"
string(1) "?"
```

Notes

> **Note** : **Deprecation and alternatives**
>
> This function is *deprecated* as of PHP 8.2.0, and will be removed in a future version. Existing uses should be checked and replaced with appropriate alternatives.
>
> Similar functionality can be achieved with mb_convert_encoding(), which supports ISO-8859-1 and many other character encodings.

```php
<?php
$utf8_string = "\xC3\xAB"; // 'ë' (e with diaeresis) in UTF-8
$iso8859_1_string = mb_convert_encoding($utf8_string, 'ISO-8859-1', 'UTF-8');
echo bin2hex($iso8859_1_string), "\n";

$utf8_string = "\xCE\xBB"; // 'λ' (Greek lower-case lambda) in UTF-8
$iso8859_7_string = mb_convert_encoding($utf8_string, 'ISO-8859-7', 'UTF-8');
echo bin2hex($iso8859_7_string), "\n";
```

```php
$utf8_string = "\xE2\x82\xAC"; // '€' (Euro sign) in UTF-8 (not present in ISO-8859-1)
$windows_1252_string = mb_convert_encoding($utf8_string, 'Windows-1252', 'UTF-8');
echo bin2hex($windows_1252_string), "\n";
?>
```

The above example will output:

```
eb
eb
80
```

Other options which may be available depending on the extensions installed are UConverter::transcode() and iconv().

The following all give the same result:

```php
<?php
$utf8_string = "\x5A\x6F\xC3\xAB"; // 'Zoë' in UTF-8
$iso8859_1_string = utf8_decode($utf8_string);
echo bin2hex($iso8859_1_string), "\n";

$iso8859_1_string = mb_convert_encoding($utf8_string, 'ISO-8859-1', 'UTF-8');
echo bin2hex($iso8859_1_string), "\n";

$iso8859_1_string = iconv('UTF-8', 'ISO-8859-1', $utf8_string);
echo bin2hex($iso8859_1_string), "\n";

$iso8859_1_string = UConverter::transcode($utf8_string, 'ISO-8859-1', 'UTF8');
echo bin2hex($iso8859_1_string), "\n";
?>
```

The above example will output:

```
5a6feb
5a6feb
5a6feb
5a6feb
```

Specifying `'?'` as the `'to_substr'` option to UConverter::transcode() gives the same result as **utf8_decode()** for strings which are invalid or which can not be represented in ISO 8859-1.

```php
<?php
$utf8_string = "\xE2\x82\xAC"; // € (Euro Sign) does not exist in ISO 8859-1
$iso8859_1_string = UConverter::transcode(
    $utf8_string, 'ISO-8859-1', 'UTF-8', ['to_substr' => '?']
);
var_dump($iso8859_1_string);
?>
```

The above example will output:

```
string(1) "?"
```

See Also

- utf8_encode() - Converts a string from ISO-8859-1 to UTF-8
- mb_convert_encoding() - Convert a string from one character encoding to another
- UConverter::transcode() - Convert a string from one character encoding to another
- iconv() - Convert a string from one character encoding to another

utf8_encode

utf8_encode

(PHP 4, PHP 5, PHP 7, PHP 8)

utf8_encode — Converts a string from ISO-8859-1 to UTF-8

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.2.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.2',
    message: 'visit the php.net documentation for various alternatives',
)]
function utf8_encode(string $string): string
```

This function converts the string `string` from the `ISO-8859-1` encoding to `UTF-8`.

> **Note** :
>
> This function does not attempt to guess the current encoding of the provided string, it assumes it is encoded as ISO-8859-1 (also known as "Latin 1") and converts to UTF-8. Since every sequence of bytes is a valid ISO-8859-1 string, this never results in an error, but will not result in a useful string if a different encoding was intended.
>
> Many web pages marked as using the `ISO-8859-1` character encoding actually use the similar `Windows-1252` encoding, and web browsers will interpret `ISO-8859-1` web pages as `Windows-1252`. `Windows-1252` features additional printable characters, such as the Euro sign (€) and curly quotes (“ ”), instead of certain `ISO-8859-1` control characters. This function will not convert such `Windows-1252` characters correctly. Use a different function if `Windows-1252` conversion is required.

Parameters

`string`

An ISO-8859-1 string.

Return Values

Returns the UTF-8 translation of `string`.

Changelog

Version	Description
8.2.0	This function has been deprecated.
7.2.0	This function has been moved from the XML extension to the core of PHP. In previous versions, it was only available if the XML extension was installed.

Examples

Example #1 Basic example

```php
<?php
// Convert the string 'Zoë' from ISO 8859-1 to UTF-8
$iso8859_1_string = "\x5A\x6F\xEB";
$utf8_string = utf8_encode($iso8859_1_string);
```

```php
echo bin2hex($utf8_string), "\n";
?>
```

The above example will output:

```
5a6fc3ab
```

Notes

> **Note** : **Deprecation and alternatives**
>
> This function is *deprecated* as of PHP 8.2.0, and will be removed in a future version. Existing uses should be checked and replaced with appropriate alternatives.
>
> Similar functionality can be achieved with mb_convert_encoding(), which supports ISO-8859-1 and many other character encodings.

```php
<?php
$iso8859_1_string = "\xEB"; // 'ë' (e with diaeresis) in ISO-8859-1
$utf8_string = mb_convert_encoding($iso8859_1_string, 'UTF-8', 'ISO-8859-1');
echo bin2hex($utf8_string), "\n";

$iso8859_7_string = "\xEB"; // the same string in ISO-8859-7 represents 'λ' (Greek lower-case lambda)
$utf8_string = mb_convert_encoding($iso8859_7_string, 'UTF-8', 'ISO-8859-7');
echo bin2hex($utf8_string), "\n";

$windows_1252_string = "\x80"; // '€' (Euro sign) in Windows-1252, but not in ISO-8859-1
$utf8_string = mb_convert_encoding($windows_1252_string, 'UTF-8', 'Windows-1252');
echo bin2hex($utf8_string), "\n";
?>
```

The above example will output:

```
c3ab
cebb
e282ac
```

Other options which may be available depending on the extensions installed are UConverter::transcode() and iconv().

The following all give the same result:

```php
<?php
$iso8859_1_string = "\x5A\x6F\xEB"; // 'Zoë' in ISO-8859-1

$utf8_string = utf8_encode($iso8859_1_string);
echo bin2hex($utf8_string), "\n";

$utf8_string = mb_convert_encoding($iso8859_1_string, 'UTF-8', 'ISO-8859-1');
echo bin2hex($utf8_string), "\n";

$utf8_string = UConverter::transcode($iso8859_1_string, 'UTF8', 'ISO-8859-1');
echo bin2hex($utf8_string), "\n";

$utf8_string = iconv('ISO-8859-1', 'UTF-8', $iso8859_1_string);
echo bin2hex($utf8_string), "\n";
?>
```

The above example will output:

```
5a6fc3ab
5a6fc3ab
5a6fc3ab
5a6fc3ab
```

See Also

- utf8_decode() - Converts a string from UTF-8 to ISO-8859-1, replacing invalid or unrepresentable characters
- mb_convert_encoding() - Convert a string from one character encoding to another
- UConverter::transcode() - Convert a string from one character encoding to another
- iconv() - Convert a string from one character encoding to another

vfprintf

vfprintf

(PHP 5, PHP 7, PHP 8)

vfprintf — Write a formatted string to a stream

Description

function **vfprintf**(resource `$stream`, string `$format`, array `$values`): int

Write a string produced according to `format` to the stream resource specified by `stream`.

Operates as fprintf() but accepts an array of arguments, rather than a variable number of arguments.

Parameters

`stream`

`format`

The format string is composed of zero or more directives: ordinary characters (excluding `%`) that are copied directly to the result and *conversion specifications*, each of which results in fetching its own parameter.

A conversion specification follows this prototype: `%[argnum$][flags][width][.precision]specifier`.

Argnum

An integer followed by a dollar sign `$`, to specify which number argument to treat in the conversion.

Flags

Flag	Description
-	Left-justify within the given field width; Right justification is the default
+	Prefix positive numbers with a plus sign +; Default only negative are prefixed with a negative sign.
(space)	Pads the result with spaces. This is the default.
`0`	Only left-pads numbers with zeros. With `s` specifiers this can also right-pad with zeros.
'(char)	Pads the result with the character (char).

Width

Either an integer that says how many characters (minimum) this conversion should result in, or `*`. If `*` is used, then the width is supplied as an additional integer value preceding the one formatted by the specifier.

Precision

A period `.` optionally followed by either an integer or `*`, whose meaning depends on the specifier:

- For `e`, `E`, `f` and `F` specifiers: this is the number of digits to be printed after the decimal point (by default, this is 6).
- For `g`, `G`, `h` and `H` specifiers: this is the maximum number of significant digits to be printed.
- For `s` specifier: it acts as a cutoff point, setting a maximum character limit to the string.

> **Note** :
>
> If the period is specified without an explicit value for precision, 0 is assumed. If `*` is used, the precision is supplied as an additional integer value preceding the one formatted by the specifier.

Specifiers

Specifier	Description
`%`	A literal percent character. No argument is required.
`b`	The argument is treated as an integer and presented as a binary number.
`c`	The argument is treated as an integer and presented as the character with that ASCII.
`d`	The argument is treated as an integer and presented as a (signed) decimal number.
`e`	The argument is treated as scientific notation (e.g. 1.2e+2).
`E`	Like the `e` specifier but uses uppercase letter (e.g. 1.2E+2).
`f`	The argument is treated as a float and presented as a floating-point number (locale aware).
`F`	The argument is treated as a float and presented as a floating-point number (non-locale aware).
`g`	General format. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of X: If P > X ≥ −4, the conversion is with style f and precision P − (X + 1). Otherwise, the conversion is with style e and precision P − 1.
`G`	Like the `g` specifier but uses `E` and `f`.
`h`	Like the `g` specifier but uses `F`. Available as of PHP 8.0.0.
`H`	Like the `g` specifier but uses `E` and `F`. Available as of PHP 8.0.0.
`o`	The argument is treated as an integer and presented as an octal number.
`s`	The argument is treated and presented as a string.
`u`	The argument is treated as an integer and presented as an unsigned decimal number.
`x`	The argument is treated as an integer and presented as a hexadecimal number (with lowercase letters).
`X`	The argument is treated as an integer and presented as a hexadecimal number (with uppercase letters).

> **Warning**
>
> The `c` type specifier ignores padding and width.

> **Warning**
>
> Attempting to use a combination of the string and width specifiers with character sets that require more than one byte per character may result in unexpected results.

Variables will be co-erced to a suitable type for the specifier:

Type Handling

Type	Specifiers
string	`s`
int	`d`, `u`, `c`, `o`, `x`, `X`, `b`
float	`e`, `E`, `f`, `F`, `g`, `G`, `h`, `H`

`values`

Return Values

Returns the length of the outputted string.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the number of arguments is zero. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[width]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[precision]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown when less arguments are given than required. Prior to PHP 8.0.0, `false` was returned and a `E_WARNING` emitted instead.

Changelog

Version	Description
8.0.0	This function no longer returns `false` on failure.
8.0.0	Throw a ValueError if the number of arguments is zero; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[width]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[precision]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError when less arguments are given than required; previously this function emitted a `E_WARNING` instead.

Examples

Example #1 vfprintf(): zero-padded integers

```php
<?php
if (!($fp = fopen('date.txt', 'w')))
    return;

$year = 2025;
$month = 5;
$day = 6;
vfprintf($fp, "%04d-%02d-%02d", array($year, $month, $day));
// will write the formatted ISO date to date.txt
?>
```

See Also

- printf() - Output a formatted string
- sprintf() - Return a formatted string
- fprintf() - Write a formatted string to a stream
- vprintf() - Output a formatted string
- vsprintf() - Return a formatted string
- sscanf() - Parses input from a string according to a format
- fscanf() - Parses input from a file according to a format
- number_format() - Format a number with grouped thousands
- date() - Format a Unix timestamp

vprintf

vprintf

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

vprintf — Output a formatted string

Description

function **vprintf**(string `$format`, array `$values`): int

Display array values as a formatted string according to `format` (which is described in the documentation for sprintf()).

Operates as printf() but accepts an array of arguments, rather than a variable number of arguments.

Parameters

`format`

The format string is composed of zero or more directives: ordinary characters (excluding `%`) that are copied directly to the result and *conversion specifications*, each of which results in fetching its own parameter.

A conversion specification follows this prototype: `%[argnum$][flags][width][.precision]specifier`.

Argnum

An integer followed by a dollar sign `$`, to specify which number argument to treat in the conversion.

Flags

Flag	Description
`-`	Left-justify within the given field width; Right justification is the default
`+`	Prefix positive numbers with a plus sign `+`; Default only negative are prefixed with a negative sign.
(space)	Pads the result with spaces. This is the default.
`0`	Only left-pads numbers with zeros. With `s` specifiers this can also right-pad with zeros.
`'`(char)	Pads the result with the character (char).

Width

Either an integer that says how many characters (minimum) this conversion should result in, or `*`. If `*` is used, then the width is supplied as an additional integer value preceding the one formatted by the specifier.

Precision

A period `.` optionally followed by either an integer or `*`, whose meaning depends on the specifier:

- For `e`, `E`, `f` and `F` specifiers: this is the number of digits to be printed after the decimal point (by default, this is 6).
- For `g`, `G`, `h` and `H` specifiers: this is the maximum number of significant digits to be printed.
- For `s` specifier: it acts as a cutoff point, setting a maximum character limit to the string.

> **Note** :
>
> If the period is specified without an explicit value for precision, 0 is assumed. If `*` is used, the precision is supplied as an additional integer value preceding the one formatted by the specifier.

Specifiers

Specifier	Description
`%`	A literal percent character. No argument is required.
`b`	The argument is treated as an integer and presented as a binary number.
`c`	The argument is treated as an integer and presented as the character with that ASCII.
`d`	The argument is treated as an integer and presented as a (signed) decimal number.
`e`	The argument is treated as scientific notation (e.g. 1.2e+2).
`E`	Like the `e` specifier but uses uppercase letter (e.g. 1.2E+2).
`f`	The argument is treated as a float and presented as a floating-point number (locale aware).
`F`	The argument is treated as a float and presented as a floating-point number (non-locale aware).
`g`	General format. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of X: If P > X ≥ −4, the conversion is with style f and precision P − (X + 1). Otherwise, the conversion is with style e and precision P − 1.
`G`	Like the `g` specifier but uses `E` and `f`.
`h`	Like the `g` specifier but uses `F`. Available as of PHP 8.0.0.
`H`	Like the `g` specifier but uses `E` and `F`. Available as of PHP 8.0.0.

Specifier	Description
o	The argument is treated as an integer and presented as an octal number.
s	The argument is treated and presented as a string.
u	The argument is treated as an integer and presented as an unsigned decimal number.
x	The argument is treated as an integer and presented as a hexadecimal number (with lowercase letters).
X	The argument is treated as an integer and presented as a hexadecimal number (with uppercase letters).

> **Warning**
>
> The c type specifier ignores padding and width.

> **Warning**
>
> Attempting to use a combination of the string and width specifiers with character sets that require more than one byte per character may result in unexpected results.

Variables will be co-erced to a suitable type for the specifier:

Type Handling

Type	Specifiers
string	s
int	d, u, c, o, x, X, b
float	e, E, f, F, g, G, h, H

`values`

Return Values

Returns the length of the outputted string.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the number of arguments is zero. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[width]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[precision]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown when less arguments are given than required. Prior to PHP 8.0.0, `false` was returned and a `E_WARNING` emitted instead.

Changelog

Version	Description
8.0.0	This function no longer returns `false` on failure.
8.0.0	Throw a ValueError if the number of arguments is zero; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[width]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[precision]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError when less arguments are given than required; previously this function emitted a `E_WARNING` instead.

Examples

Example #1 vprintf(): zero-padded integers

```php
<?php
vprintf("%04d-%02d-%02d", explode('-', '1988-8-1'));
?>
```

The above example will output:

```
1988-08-01
```

See Also

- printf() - Output a formatted string
- sprintf() - Return a formatted string
- fprintf() - Write a formatted string to a stream
- vsprintf() - Return a formatted string
- vfprintf() - Write a formatted string to a stream
- sscanf() - Parses input from a string according to a format
- fscanf() - Parses input from a file according to a format
- number_format() - Format a number with grouped thousands
- date() - Format a Unix timestamp

vsprintf

vsprintf

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

vsprintf — Return a formatted string

Description

function **vsprintf**(string `$format`, array `$values`): string

Operates as sprintf() but accepts an array of arguments, rather than a variable number of arguments.

Parameters

`format`

The format string is composed of zero or more directives: ordinary characters (excluding `%`) that are copied directly to the result and *conversion specifications*, each of which results in fetching its own parameter.

A conversion specification follows this prototype: `%[argnum$][flags][width][.precision]specifier`.

Argnum

An integer followed by a dollar sign `$`, to specify which number argument to treat in the conversion.

Flags

Flag	Description
`-`	Left-justify within the given field width; Right justification is the default
`+`	Prefix positive numbers with a plus sign `+`; Default only negative are prefixed with a negative sign.
(space)	Pads the result with spaces. This is the default.
`0`	Only left-pads numbers with zeros. With `s` specifiers this can also right-pad with zeros.
`'`(char)	Pads the result with the character (char).

Width

Either an integer that says how many characters (minimum) this conversion should result in, or `*`. If `*` is used, then the width is supplied as an additional integer value preceding the one formatted by the specifier.

Precision

A period `.` optionally followed by either an integer or `*`, whose meaning depends on the specifier:

- For `e`, `E`, `f` and `F` specifiers: this is the number of digits to be printed after the decimal point (by default, this is 6).
- For `g`, `G`, `h` and `H` specifiers: this is the maximum number of significant digits to be printed.
- For `s` specifier: it acts as a cutoff point, setting a maximum character limit to the string.

> **Note** :
>
> If the period is specified without an explicit value for precision, 0 is assumed. If `*` is used, the precision is supplied as an additional integer value preceding the one formatted by the specifier.

Specifiers

Specifier	Description
`%`	A literal percent character. No argument is required.
`b`	The argument is treated as an integer and presented as a binary number.
`c`	The argument is treated as an integer and presented as the character with that ASCII.
`d`	The argument is treated as an integer and presented as a (signed) decimal number.
`e`	The argument is treated as scientific notation (e.g. 1.2e+2).
`E`	Like the `e` specifier but uses uppercase letter (e.g. 1.2E+2).
`f`	The argument is treated as a float and presented as a floating-point number (locale aware).
`F`	The argument is treated as a float and presented as a floating-point number (non-locale aware).
`g`	General format. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style E would have an exponent of X: If P > X ≥ −4, the conversion is with style f and precision P − (X + 1). Otherwise, the conversion is with style e and precision P − 1.
`G`	Like the `g` specifier but uses `E` and `f`.
`h`	Like the `g` specifier but uses `F`. Available as of PHP 8.0.0.
`H`	Like the `g` specifier but uses `E` and `F`. Available as of PHP 8.0.0.
`o`	The argument is treated as an integer and presented as an octal number.
`s`	The argument is treated and presented as a string.
`u`	The argument is treated as an integer and presented as an unsigned decimal number.
`x`	The argument is treated as an integer and presented as a hexadecimal number (with lowercase letters).
`X`	The argument is treated as an integer and presented as a hexadecimal number (with uppercase letters).

> **Warning**
>
> The `c` type specifier ignores padding and width.

> **Warning**
>
> Attempting to use a combination of the string and width specifiers with character sets that require more than one byte per character may result in unexpected results.

Variables will be co-erced to a suitable type for the specifier:

Type Handling

Type	Specifiers
string	`s`
int	`d`, `u`, `c`, `o`, `x`, `X`, `b`
float	`e`, `E`, `f`, `F`, `g`, `G`, `h`, `H`

`values`

Return Values

Return array values as a formatted string according to `format`.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the number of arguments is zero. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[width]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown if `[precision]` is less than zero or bigger than `PHP_INT_MAX`. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

As of PHP 8.0.0, a ValueError is thrown when less arguments are given than required. Prior to PHP 8.0.0, `false` was returned and a `E_WARNING` emitted instead.

Changelog

Version	Description
8.0.0	This function no longer returns `false` on failure.
8.0.0	Throw a ValueError if the number of arguments is zero; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[width]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError if `[precision]` is less than zero or bigger than `PHP_INT_MAX`; previously this function emitted a `E_WARNING` instead.
8.0.0	Throw a ValueError when less arguments are given than required; previously this function emitted a `E_WARNING` instead.

Examples

Example #1 vsprintf(): zero-padded integers

```php
<?php
print vsprintf("%04d-%02d-%02d", explode('-', '1988-8-1'));
?>
```

The above example will output:

```
1988-08-01
```

See Also

- printf() - Output a formatted string
- sprintf() - Return a formatted string
- fprintf() - Write a formatted string to a stream
- vprintf() - Output a formatted string
- vfprintf() - Write a formatted string to a stream
- sscanf() - Parses input from a string according to a format
- fscanf() - Parses input from a file according to a format
- number_format() - Format a number with grouped thousands
- date() - Format a Unix timestamp

wordwrap

wordwrap

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

wordwrap — Wraps a string to a given number of characters

Description

function **wordwrap**(
  string `$string`,
  int `$width` = 75,

```
    string $break = "\n",
    bool $cut_long_words = false
): string
```

Wraps a string to a given number of characters using a string break character. Strings wrap after a space (U+0020) character unless `cut_long_words` is set to `true`.

Parameters

`string`

The input string.

`width`

The number of characters at which the string will be wrapped.

`break`

The line is broken using the optional `break` parameter. It must not be an empty string.

`cut_long_words`

If the `cut_long_words` is set to `true`, the string is always wrapped at or before the specified `width`. So if you have a word that is larger than the given width, it is broken apart. (See second example). When `false` the function does not split the word even if the `width` is smaller than the word width.

Return Values

Returns the given string wrapped at the specified length.

Errors/Exceptions

If `break` is an empty string, a ValueError is thrown.

Changelog

Version	Description
8.0.0	If `break` is empty string, a ValueError is thrown; previously in this case it emitted an `E_WARNING` and returned `false`.

Examples

Example #1 **wordwrap()** example

```php
<?php
$text = "The quick brown fox jumped over the lazy dog.";
$newtext = wordwrap($text, 20, "<br />\n");

echo $newtext;
?>
```

The above example will output:

```
The quick brown fox<br />
jumped over the lazy<br />
dog.
```

Example #2 **wordwrap()** example

```php
<?php
$text = "A very long woooooooooooord.";
$newtext = wordwrap($text, 8, "\n", true);
```

```
echo "$newtext\n";
?>
```

The above example will output:

```
A very
long
woooooo
ooooord.
```

Example #3 wordwrap() example

```
<?php
$text = "A very long woooooooooooooooooord. and something";
$newtext = wordwrap($text, 8, "\n", false);

echo "$newtext\n";
?>
```

The above example will output:

```
A very
long
woooooooooooooooooord.
and
something
```

See Also

- nl2br() - Inserts HTML line breaks before all newlines in a string
- chunk_split() - Split a string into smaller chunks

Changelog

The following changes have been made to classes/functions/methods of this extension.

Version	Function	Description
8.5.0	chr	Passing integers outside the interval [0, 255] is now deprecated.
	ord	Passing a string which is not a single byte is now deprecated.
	setlocale	Passing an integer 0 as the locales argument is no longer supported and now throws a TypeError.
8.4.0	str_getcsv	Relying on the default value of escape is now deprecated.
	str_getcsv	Now throws a ValueError if separator, enclosure, or escape is invalid. This mimics the behavior of fgetcsv and fputcsv.
	strcspn	Prior to PHP 8.4.0, when characters was the empty string, the search would incorrectly stop at the first null byte in string.
8.3.0	number_format	Added handling of negative values for decimals.
	str_getcsv	An empty string is returned instead of a string with a single null byte for the last field if it contains only an unterminated enclosure.
	strrchr	The before_needle parameter was added.
	strtok	Now emits E_WARNING when token is not provided.

Version	Function	Description
8.2.0	lcfirst	Case conversion no longer depends on the locale set with setlocale. Only ASCII characters will be converted.
	str_ireplace	Case folding no longer depends on the locale set with setlocale. Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.
	str_split	If string is empty an empty array is now returned. Previously an array containing a single empty string was returned.
	strcasecmp	This function is no longer guaranteed to return strlen($string1) - strlen($string2) when string lengths are not equal, but may now return -1 or 1 instead.
	strcmp	This function is no longer guaranteed to return strlen($string1) - strlen($string2) when string lengths are not equal, but may now return -1 or 1 instead.
	stripos	Case folding no longer depends on the locale set with setlocale. Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.
	stristr	Case folding no longer depends on the locale set with setlocale. Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.
	strnatcasecmp	This function is no longer guaranteed to return strlen($string1) - strlen($string2) when string lengths are not equal, but may now return -1 or 1 instead.
	strnatcmp	This function is no longer guaranteed to return strlen($string1) - strlen($string2) when string lengths are not equal, but may now return -1 or 1 instead.
	strncasecmp	This function is no longer guaranteed to return strlen($string1) - strlen($string2) when string lengths are not equal, but may now return -1 or 1 instead.
	strncmp	This function is no longer guaranteed to return strlen($string1) - strlen($string2) when string lengths are not equal, but may now return -1 or 1 instead.
	strripos	Case folding no longer depends on the locale set with setlocale. Only ASCII case folding will be done. Non-ASCII bytes will be compared by their byte value.
	strtolower	Case conversion no longer depends on the locale set with setlocale. Only ASCII characters will be converted.
	strtoupper	Case conversion no longer depends on the locale set with setlocale. Only ASCII characters will be converted.
	substr_compare	This function is no longer guaranteed to return strlen($string1) - strlen($string2) when string lengths are not equal, but may now return -1 or 1 instead.
	ucfirst	Case conversion no longer depends on the locale set with setlocale. Only ASCII characters will be converted.
	ucwords	Case conversion no longer depends on the locale set with setlocale. Only ASCII characters will be converted.
	utf8_decode	This function has been deprecated.
	utf8_encode	This function has been deprecated.
8.1.0	get_html_translation_table	flags changed from ENT_COMPAT to ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	html_entity_decode	flags changed from ENT_COMPAT to ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	htmlentities	flags changed from ENT_COMPAT to ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	htmlspecialchars	flags changed from ENT_COMPAT to ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
	htmlspecialchars_decode	flags changed from ENT_COMPAT to ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401.
8.0.0	convert_cyr_string	Removed this function.
	convert_uuencode	Prior to this version, trying to convert an empty string returned false for no particular reason.
	count_chars	Prior to this version, the function returned false on failure.
	crypt	The salt is no longer optional.
	explode	explode will now throw ValueError when separator parameter is given an empty string (""). Previously, explode returned false instead.
	fprintf	This function no longer returns false on failure.
	fprintf	Throw a ValueError if the number of arguments is zero; previously this function emitted a E_WARNING instead.

Version	Function	Description
	fprintf	Throw a ValueError if [width] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	fprintf	Throw a ValueError if [precision] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	fprintf	Throw a ArgumentCountError when less arguments are given than required; previously this function emitted a E_WARNING instead.
	hebrevc	Removed this function.
	html_entity_decode	encoding is nullable now.
	htmlentities	encoding is nullable now.
	implode	Passing the separator after the array is no longer supported.
	levenshtein	Prior to this version, levenshtein had to be called with either two or five arguments.
	levenshtein	Prior to this version, levenshtein would return -1 if one of the argument strings is longer than 255 characters.
	metaphone	The function returned false on failure.
	money_format	Removed this function.
	number_format	Prior to this version, number_format accepted one, two, or four parameters (but not three).
	parse_str	result is no longer optional.
	printf	This function no longer returns false on failure.
	printf	Throw a ValueError if the number of arguments is zero; previously this function emitted a E_WARNING instead.
	printf	Throw a ValueError if [width] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	printf	Throw a ValueError if [precision] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	printf	Throw a ArgumentCountError when less arguments are given than required; previously this function emitted a E_WARNING instead.
	soundex	Prior to this version, calling the function with an empty string returned false for no particular reason.
	sprintf	This function no longer returns false on failure.
	sprintf	Throw a ValueError if the number of arguments is zero; previously this function emitted a E_WARNING instead.
	sprintf	Throw a ValueError if [width] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	sprintf	Throw a ValueError if [precision] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	sprintf	Throw a ArgumentCountError when less arguments are given than required; previously this function emitted a E_WARNING instead.
	str_split	If length is less than 1, a ValueError will be thrown now; previously, an error of level E_WARNING has been raised instead, and the function returned false.
	str_word_count	characters is nullable now.
	strcspn	length is nullable now.
	strip_tags	allowed_tags is nullable now.
	stripos	needle now accepts an empty string.
	stripos	Passing an int as needle is no longer supported.
	stristr	needle now accepts an empty string.
	stristr	Passing an int as needle is no longer supported.
	strpos	needle now accepts an empty string.
	strpos	Passing an int as needle is no longer supported.

Version	Function	Description
	strrchr	needle now accepts an empty string.
	strrchr	Passing an int as needle is no longer supported.
	strripos	needle now accepts an empty string.
	strripos	Passing an int as needle is no longer supported.
	strrpos	needle now accepts an empty string.
	strrpos	Passing an int as needle is no longer supported.
	strspn	length is nullable now.
	strstr	needle now accepts an empty string.
	strstr	Passing an int as needle is no longer supported.
	substr	length is nullable now. When length is explicitly set to null, the function returns a substring finishing at the end of the string, when it previously returned an empty string.
	substr	The function returns an empty string where it previously returned false.
	substr_compare	length is nullable now.
	substr_count	length is nullable now.
	substr_replace	length is nullable now.
	vfprintf	This function no longer returns false on failure.
	vfprintf	Throw a ValueError if the number of arguments is zero; previously this function emitted a E_WARNING instead.
	vfprintf	Throw a ValueError if [width] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	vfprintf	Throw a ValueError if [precision] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	vfprintf	Throw a ValueError when less arguments are given than required; previously this function emitted a E_WARNING instead.
	vprintf	This function no longer returns false on failure.
	vprintf	Throw a ValueError if the number of arguments is zero; previously this function emitted a E_WARNING instead.
	vprintf	Throw a ValueError if [width] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	vprintf	Throw a ValueError if [precision] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	vprintf	Throw a ValueError when less arguments are given than required; previously this function emitted a E_WARNING instead.
	vsprintf	This function no longer returns false on failure.
	vsprintf	Throw a ValueError if the number of arguments is zero; previously this function emitted a E_WARNING instead.
	vsprintf	Throw a ValueError if [width] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	vsprintf	Throw a ValueError if [precision] is less than zero or bigger than PHP_INT_MAX; previously this function emitted a E_WARNING instead.
	vsprintf	Throw a ValueError when less arguments are given than required; previously this function emitted a E_WARNING instead.
	wordwrap	If break is empty string, a ValueError is thrown; previously in this case it emitted an E_WARNING and returned false.
7.4.0	chr	The function no longer silently accepts unsupported codepoints, and casts these to 0.
	convert_cyr_string	Deprecated this function.
	hebrevc	Deprecated this function.
	implode	Passing the separator after the array (i.e. using the legacy signature) has been deprecated.

Version	Function	Description
	money_format	Deprecated this function. Instead, use NumberFormatter::formatCurrency.
	str_getcsv	The escape parameter now interprets an empty string as signal to disable the proprietary escape mechanism. Formerly, an empty string was treated like the default parameter value.
	strip_tags	The allowed_tags now alternatively accepts an array.
7.3.5	substr_compare	offset may now be equal to the length of haystack.
7.3.0	stripos	Passing an int as needle has been deprecated.
	stristr	Passing an int as needle has been deprecated.
	strpos	Passing an int as needle has been deprecated.
	strrchr	Passing an int as needle has been deprecated.
	strripos	Passing an int as needle has been deprecated.
	strrpos	Passing an int as needle has been deprecated.
	strstr	Passing an int as needle has been deprecated.
7.2.18	substr_compare	offset may now be equal to the length of haystack.
7.2.0	number_format	number_format was changed to not being able to return -0, previously -0 could be returned for cases like where num would be -0.01.
	parse_str	Usage of parse_str without a second parameter now emits an E_DEPRECATED notice.
	utf8_decode	This function has been moved from the XML extension to the core of PHP. In previous versions, it was only available if the XML extension was installed.
	utf8_encode	This function has been moved from the XML extension to the core of PHP. In previous versions, it was only available if the XML extension was installed.
7.1.0	str_shuffle	The internal randomization algorithm has been changed to use the Mersenne Twister Random Number Generator instead of the libc rand function.
	stripos	Support for negative offsets has been added.
	strpos	Support for negative offsets has been added.
	substr_count	Support for negative offsets and lengths has been added. length may also be 0 now.

Arrays

Introduction

These functions allow you to interact with and manipulate arrays in various ways. Arrays are essential for storing, managing, and operating on sets of variables.

Simple and multi-dimensional arrays are supported, and may be either user created or created by another function. There are specific database handling functions for populating arrays from database queries, and several functions return arrays.

Please see the Arrays section of the manual for a detailed explanation of how arrays are implemented and used in PHP. See also Array operators for other ways how to manipulate the arrays.

Predefined Constants

The constants below are always available as part of the PHP core.

CASE_LOWER (int)

> **CASE_LOWER** is used with array_change_key_case() and is used to convert array keys to lower case. This is also the default case for array_change_key_case(). As of PHP 8.2.0, only ASCII characters will be converted.

CASE_UPPER (int)

> **CASE_UPPER** is used with array_change_key_case() and is used to convert array keys to upper case. As of PHP 8.2.0, only ASCII characters will be converted.

Sorting order flags:

SORT_ASC (int)

> **SORT_ASC** is used with array_multisort() to sort in ascending order.

SORT_DESC (int)

> **SORT_DESC** is used with array_multisort() to sort in descending order.

Sorting type flags: used by various sort functions

SORT_REGULAR (int)

> **SORT_REGULAR** is used to compare items normally.

SORT_NUMERIC (int)

> **SORT_NUMERIC** is used to compare items numerically.

SORT_STRING (int)

> **SORT_STRING** is used to compare items as strings.

SORT_LOCALE_STRING (int)

> **SORT_LOCALE_STRING** is used to compare items as strings, based on the current locale.

SORT_NATURAL (int)

> **SORT_NATURAL** is used to compare items as strings using "natural ordering" like natsort().

SORT_FLAG_CASE (int)

> **SORT_FLAG_CASE** can be combined (bitwise OR) with **SORT_STRING** or **SORT_NATURAL** to sort strings case-insensitively. As of PHP 8.2.0, only ASCII case folding will be done.

Filter flags:

ARRAY_FILTER_USE_KEY (int)

`ARRAY_FILTER_USE_KEY` is used with array_filter() to pass each key as the first argument to the given callback function.

`ARRAY_FILTER_USE_BOTH` (int)

`ARRAY_FILTER_USE_BOTH` is used with array_filter() to pass both value and key to the given callback function.

`COUNT_NORMAL` (int)

`COUNT_RECURSIVE` (int)

`EXTR_OVERWRITE` (int)

`EXTR_SKIP` (int)

`EXTR_PREFIX_SAME` (int)

`EXTR_PREFIX_ALL` (int)

`EXTR_PREFIX_INVALID` (int)

`EXTR_PREFIX_IF_EXISTS` (int)

`EXTR_IF_EXISTS` (int)

`EXTR_REFS` (int)

Sorting Arrays

PHP has several functions that deal with sorting arrays, and this document exists to help sort it all out.

The main differences are:

- Some sort based on the array keys, whereas others by the values: `$array['key'] = 'value';`
- Whether or not the correlation between the keys and values are maintained after the sort, which may mean the keys are reset numerically (0,1,2 ...)
- The order of the sort: alphabetical, ascending (low to high), descending (high to low), natural, random, or user defined
- Note: All of these sort functions act directly on the array variable itself, as opposed to returning a new sorted array
- If any of these sort functions evaluates two members as equal then they retain their original order. Prior to PHP 8.0.0, their order were undefined (the sorting was not stable).

Sorting function attributes

Function name	Sorts by	Maintains key association	Order of sort	Related functions
array_multisort()	value	string keys yes, int keys no	first array or sort options	array_walk()
asort()	value	yes	ascending	arsort()
arsort()	value	yes	descending	asort()
krsort()	key	yes	descending	ksort()
ksort()	key	yes	ascending	krsort()
natcasesort()	value	yes	natural, case insensitive	natsort()
natsort()	value	yes	natural	natcasesort()
rsort()	value	no	descending	sort()
shuffle()	value	no	random	array_rand()
sort()	value	no	ascending	rsort()
uasort()	value	yes	user defined	uksort()
uksort()	key	yes	user defined	uasort()
usort()	value	no	user defined	uasort()

Array Functions

See Also

See also is_array(), explode(), implode(), preg_split(), and unset().

Table of Contents

array

array

(PHP 4, PHP 5, PHP 7, PHP 8)

array — Create an array

Description

function **array**(mixed `...$values`): array

Creates an array. Read the section on the array type for more information on what an array is, including details about the alternative bracket syntax (`[]`).

Parameters

`values`

> Syntax "index => values", separated by commas, define index and values. index may be of type string or integer. When index is omitted, an integer index is automatically generated, starting at 0. If index is an integer, next generated index will be the biggest integer index + 1. Note that when two identical indices are defined, the last overwrites the first.
>
> Having a trailing comma after the last defined array entry, while unusual, is a valid syntax.

Return Values

Returns an array of the parameters. The parameters can be given an index with the `=>` operator. Read the section on the array type for more information on what an array is.

Examples

The following example demonstrates how to create a two-dimensional array, how to specify keys for associative arrays, and how to skip-and-continue numeric indices in normal arrays.

Example #1 array() example

```php
<?php
$fruits = array (
    "fruits"  => array("a" => "orange", "b" => "banana", "c" => "apple"),
    "numbers" => array(1, 2, 3, 4, 5, 6),
    "holes"   => array("first", 5 => "second", "third")
);
print_r($fruits);
?>
```

Example #2 Automatic index with array()

```php
<?php
$array = array(1, 1, 1, 1,  1, 8 => 1, 4 => 1, 19, 3 => 13);
print_r($array);
?>
```

The above example will output:

```
Array
(
    [0] => 1
    [1] => 1
    [2] => 1
    [3] => 13
    [4] => 1
    [8] => 1
    [9] => 19
)
```

Note that index '3' is defined twice, and keep its final value of 13. Index 4 is defined after index 8, and next generated index (value 19) is 9, since biggest index was 8.

This example creates a 1-based array.

Example #3 1-based index with array()

```php
<?php
$firstQuarter = array(1 => 'January', 'February', 'March');
print_r($firstQuarter);
?>
```

The above example will output:

```
Array
(
    [1] => January
    [2] => February
    [3] => March
)
```

As in Perl, you can access a value from the array inside double quotes. However, with PHP you'll need to enclose your array between curly braces.

Example #4 Accessing an array inside double quotes

```php
<?php
$foo = array('bar' => 'baz');
echo "Hello {$foo['bar']}!"; // Hello baz!
?>
```

Notes

> **Note** :
>
> **array()** is a language construct used to represent literal arrays, and not a regular function.

See Also

- array_pad() - Pad array to the specified length with a value
- list() - Assign variables as if they were an array
- count() - Counts all elements in an array or in a Countable object
- range() - Create an array containing a range of elements
- `foreach`
- The array type

array_all

array_all

(PHP 8 >= 8.4.0)

array_all — Checks if all array elements satisfy a callback function

Description

function **array_all**(array `$array`, callable `$callback`): bool

array_all() returns `true`, if the given `callback` returns `true` for all elements. Otherwise the function returns `false`.

Parameters

`array`

The array that should be searched.

`callback`

The callback function to call to check each element, which must be of the following signature:

function callback(mixed `$value`, mixed `$key`): bool
If this function returns `false`, `false` is returned from **array_all()** and the callback will not be called for further elements.

Return Values

The function returns `true`, if `callback` returns `true` for all elements. Otherwise the function returns `false`.

Examples

Example #1 array_all() example

```php
<?php
$array = [
    'a' => 'dog',
    'b' => 'cat',
    'c' => 'cow',
    'd' => 'duck',
    'e' => 'goose',
    'f' => 'elephant'
];

// Check, if all animal names are shorter than 12 letters.
var_dump(array_all($array, function (string $value) {
    return strlen($value) < 12;
}));

// Check, if all animal names are longer than 5 letters.
var_dump(array_all($array, function (string $value) {
    return strlen($value) > 5;
}));

// Check, if all array keys are strings.
var_dump(array_all($array, function (string $value, $key) {
   return is_string($key);
}));
?>
```

The above example will output:

```
bool(true)
bool(false)
bool(true)
```

See Also

- array_any() - Checks if at least one array element satisfies a callback function
- array_filter() - Filters elements of an array using a callback function
- array_find() - Returns the first element satisfying a callback function
- array_find_key() - Returns the key of the first element satisfying a callback function

array_any

array_any

(PHP 8 >= 8.4.0)

array_any — Checks if at least one array element satisfies a callback function

Description

function **array_any**(array `$array`, callable `$callback`): bool

array_any() returns `true`, if the given `callback` returns `true` for any element. Otherwise the function returns `false`.

Parameters

`array`

The array that should be searched.

`callback`

The callback function to call to check each element, which must be of the following signature:

function callback(mixed `$value`, mixed `$key`): bool
If this function returns `true`, `true` is returned from **array_any()** and the callback will not be called for further elements.

Return Values

The function returns `true`, if there is at least one element for which `callback` returns `true`. Otherwise the function returns `false`.

Examples

Example #1 array_any() example

```php
<?php
$array = [
    'a' => 'dog',
    'b' => 'cat',
    'c' => 'cow',
    'd' => 'duck',
    'e' => 'goose',
    'f' => 'elephant'
];

// Check, if any animal name is longer than 5 letters.
var_dump(array_any($array, function (string $value) {
    return strlen($value) > 5;
}));

// Check, if any animal name is shorter than 3 letters.
var_dump(array_any($array, function (string $value) {
    return strlen($value) < 3;
}));

// Check, if any array key is not a string.
var_dump(array_any($array, function (string $value, $key) {
   return !is_string($key);
}));
?>
```

The above example will output:

```
bool(true)
bool(false)
bool(false)
```

See Also

- array_all() - Checks if all array elements satisfy a callback function
- array_filter() - Filters elements of an array using a callback function
- array_find() - Returns the first element satisfying a callback function
- array_find_key() - Returns the key of the first element satisfying a callback function

array_change_key_case

array_change_key_case

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

array_change_key_case — Changes the case of all keys in an array

Description

function **array_change_key_case**(array `$array`, int `$case` = **`CASE_LOWER`**): array

Returns an array with all keys from `array` lowercased or uppercased. Numbered indices are left as is.

Parameters

`array`

The array to work on

`case`

Either **`CASE_UPPER`** or **`CASE_LOWER`** (default)

Return Values

Returns an array with its keys lower or uppercased.

Examples

Example #1 array_change_key_case() example

```php
<?php
$input_array = array("FirSt" => 1, "SecOnd" => 4);
print_r(array_change_key_case($input_array, CASE_UPPER));
?>
```

The above example will output:

```
Array
(
    [FIRST] => 1
    [SECOND] => 4
)
```

Notes

> **Note** :
>
> If an array has indices that will be the same once run through this function (e.g. "`keY`" and "`kEY`"), the value that is later in the array will override other indices.

array_chunk

array_chunk

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

array_chunk — Split an array into chunks

Description

function **array_chunk**(array `$array`, int `$length`, bool `$preserve_keys = false`): array

Chunks an array into arrays with `length` elements. The last chunk may contain less than `length` elements.

Parameters

`array`

The array to work on

`length`

The size of each chunk

`preserve_keys`

When set to `true` keys will be preserved. Default is `false` which will reindex the chunk numerically

Return Values

Returns a multidimensional numerically indexed array, starting with zero, with each dimension containing `length` elements.

Errors/Exceptions

If `length` is less than `1`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	If `length` is less than `1`, a ValueError will be thrown now; previously, an error of level `E_WARNING` has been raised instead, and the function returned `null`.

Examples

Example #1 array_chunk() example

```php
<?php
$input_array = array('a', 'b', 'c', 'd', 'e');
print_r(array_chunk($input_array, 2));
print_r(array_chunk($input_array, 2, true));
?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [0] => a
            [1] => b
        )

    [1] => Array
        (
            [0] => c
            [1] => d
        )

    [2] => Array
        (
            [0] => e
        )
```

```
)
Array
(
    [0] => Array
        (
            [0] => a
            [1] => b
        )

    [1] => Array
        (
            [2] => c
            [3] => d
        )

    [2] => Array
        (
            [4] => e
        )

)
```

See Also

- array_slice() - Extract a slice of the array

array_column

array_column

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

array_column — Return the values from a single column in the input array

Description

function **array_column**(array `$array`, int|string|null `$column_key`, int|string|null `$index_key = null`): array

array_column() returns the values from a single column of the `array`, identified by the `column_key`. Optionally, an `index_key` may be provided to index the values in the returned array by the values from the `index_key` column of the input array.

`array` is treated as a list of rows, each of which is itself an array or an object; a column is the entry or property that `column_key` names in every row. Rows that are neither an array nor an object, and rows in which that entry or property is absent, are skipped without a diagnostic. The keys of `array` are never preserved.

Parameters

`array`

A list whose rows are arrays or objects, from which a column of values is pulled. If a row is an object, public properties are read directly. For a property that is not publicly accessible to be read, the class must implement both the __get() and __isset() magic methods: without __isset() the row is skipped, and with __isset() but no __get() an Error is thrown.

`column_key`

The column of values to return. This value may be an integer key of the column to retrieve, or it may be a string key name for an associative array or property name. It may also be `null` to return complete rows (this is useful together with `index_key` to reindex the array).

`index_key`

The column to use as the index/keys for the returned array. This value may be the integer key of the column, or it may be the string key name. The value is cast as usual for array keys (however, prior to PHP 8.0.0, objects supporting conversion to string were also allowed).

Return Values

Returns an array of values representing a single column from the input array. If `column_key` is `null`, the rows themselves are returned.

Changelog

Version	Description
8.0.0	Objects in columns indicated by `index_key` parameter will no longer be cast to string and will now throw a TypeError instead.
7.1.0	Property visibility is now respected. Formerly, protected and private properties were read directly.
7.0.0	The rows of `array` may now be objects as well as arrays.

Examples

Example #1 Get the column of first names from a recordset

```php
<?php

// Array representing a possible record set returned from a database
$records = [
    [
        'id' => 2135,
        'first_name' => 'John',
        'last_name' => 'Doe',
    ],
    [
        'id' => 3245,
        'first_name' => 'Sally',
        'last_name' => 'Smith',
    ],
    [
        'id' => 5342,
        'first_name' => 'Jane',
        'last_name' => 'Jones',
    ],
    [
        'id' => 5623,
        'first_name' => 'Peter',
        'last_name' => 'Doe',
    ]
];

$first_names = array_column($records, 'first_name');
print_r($first_names);

?>
```

The above example will output:

```
Array
(
    [0] => John
    [1] => Sally
    [2] => Jane
    [3] => Peter
)
```

Example #2 Get the column of last names from a recordset, indexed by the "id" column

```php
<?php

// Using the $records array from Example #1
$records = [
    [
        'id' => 2135,
        'first_name' => 'John',
```

```
        'last_name' => 'Doe',
    ],
    [
        'id' => 3245,
        'first_name' => 'Sally',
        'last_name' => 'Smith',
    ],
    [
        'id' => 5342,
        'first_name' => 'Jane',
        'last_name' => 'Jones',
    ],
    [
        'id' => 5623,
        'first_name' => 'Peter',
        'last_name' => 'Doe',
    ]
];

$last_names = array_column($records, 'last_name', 'id');
print_r($last_names);

?>
```

The above example will output:

```
Array
(
    [2135] => Doe
    [3245] => Smith
    [5342] => Jones
    [5623] => Doe
)
```

Example #3 Get the column of usernames from the public "username" property of an object

```
<?php

class User
{
    public $username;

    public function __construct(string $username)
    {
        $this->username = $username;
    }
}

$users = [
    new User('user 1'),
    new User('user 2'),
    new User('user 3'),
];

print_r(array_column($users, 'username'));

?>
```

The above example will output:

```
Array
(
    [0] => user 1
```

```
    [1] => user 2
    [2] => user 3
)
```

Example #4 Get the column of names from the private "name" property of an object using the magic methods __isset() and __get()

```php
<?php

class Person
{
    private $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function __get($prop)
    {
        return $this->$prop;
    }

    public function __isset($prop) : bool
    {
        return isset($this->$prop);
    }
}

$people = [
    new Person('Fred'),
    new Person('Jane'),
    new Person('John'),
];

print_r(array_column($people, 'name'));
?>
```

The above example will output:

```
Array
(
    [0] => Fred
    [1] => Jane
    [2] => John
)
```

If __isset() is not provided, the object contributes no entry to the returned array.

array_combine

array_combine

(PHP 5, PHP 7, PHP 8)

array_combine — Creates an array by using one array for keys and another for its values

Description

function **array_combine**(array `$keys`, array `$values`): array

Creates an array by using the values from the `keys` array as keys and the values from the `values` array as the corresponding values.

Parameters

`keys`

Array of keys to be used. Illegal values for key will be converted to string.

`values`

Array of values to be used

Return Values

Returns the combined array.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the number of elements in `keys` and `values` does not match. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

Changelog

Version	Description
8.0.0	**array_combine()** will now throw a ValueError if the number of elements for each array is not equal; previously this function returned `false` instead.

Examples

Example #1 A simple array_combine() example

```
<?php
$a = array('green', 'red', 'yellow');
$b = array('avocado', 'apple', 'banana');
$c = array_combine($a, $b);

print_r($c);
?>
```

The above example will output:

```
Array
(
    [green] => avocado
    [red] => apple
    [yellow] => banana
)
```

See Also

- array_merge() - Merge one or more arrays
- array_walk() - Apply a user supplied function to every member of an array
- array_values() - Return all the values of an array
- array_map() - Applies the callback to the elements of the given arrays

array_count_values

array_count_values

(PHP 4, PHP 5, PHP 7, PHP 8)

array_count_values — Counts the occurrences of each distinct value in an array

Description

function **array_count_values**(array $array): array

array_count_values() returns an array using the values of array (which must be ints or strings) as keys and their frequency in array as values.

Parameters

array

The array of values to count

Return Values

Returns an associative array of values from array as keys and their count as value.

Errors/Exceptions

Throws E_WARNING for every element which is not string or int.

Examples

Example #1 array_count_values() example

```php
<?php
$array = array(1, "hello", 1, "world", "hello");
print_r(array_count_values($array));
?>
```

The above example will output:

```
Array
(
    [1] => 2
    [hello] => 2
    [world] => 1
)
```

See Also

- count() - Counts all elements in an array or in a Countable object
- array_unique() - Removes duplicate values from an array
- array_values() - Return all the values of an array
- count_chars() - Return information about characters used in a string

array_diff

array_diff

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_diff — Computes the difference of arrays

Description

function **array_diff**(array $array, array ...$arrays): array

Compares array against one or more other arrays and returns the values in array that are not present in any of the other arrays.

Parameters

array

The array to compare from

`arrays`

Arrays to compare against

Return Values

Returns an array containing all the entries from `array` that are not present in any of the other arrays. Keys in the `array` array are preserved.

Changelog

Version	Description
8.0.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_diff() example

```php
<?php
$array1 = array("a" => "green", "red", "blue", "red");
$array2 = array("b" => "green", "yellow", "red");
$result = array_diff($array1, $array2);

print_r($result);
?>
```

Multiple occurrences in *$array1* are all treated the same way. This will output :

```
Array
(
    [1] => blue
)
```

Two elements are considered equal if and only if `(string) $elem1 === (string) $elem2`. That is, when the string representation is the same.

Example #2 array_diff() example with non-matching types

```php
<?php
// This will generate a Notice that an array cannot be cast to a string.
$source = [1, 2, 3, 4];
$filter = [3, 4, [5], 6];
$result = array_diff($source, $filter);

// Whereas this is fine, since the objects can cast to a string.
class S {
  private $v;

  public function __construct(string $v) {
    $this->v = $v;
  }

  public function __toString() {
    return $this->v;
  }
}

$source = [new S('a'), new S('b'), new S('c')];
$filter = [new S('b'), new S('c'), new S('d')];

$result = array_diff($source, $filter);

// $result now contains one instance of S('a');
var_dump($result);
?>
```

To use an alternate comparison function, see array_udiff().

Notes

> **Note** :
>
> This function only checks one dimension of a n-dimensional array. Of course you can check deeper dimensions by using `array_diff($array1[0], $array2[0]);`.

See Also

- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check

array_diff_assoc

array_diff_assoc

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

array_diff_assoc — Computes the difference of arrays with additional index check

Description

function **array_diff_assoc**(array `$array`, array `...$arrays`): array

Compares `array` against `arrays` and returns the difference. Unlike array_diff() the array keys are also used in the comparison.

Parameters

`array`

The array to compare from

`arrays`

Arrays to compare against

Return Values

Returns an array containing all the values from `array` that are not present in any of the other arrays.

Changelog

Version	Description
8.0.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_diff_assoc() example

In this example the `"a" => "green"` pair is present in both arrays and thus it is not in the output from the function. Unlike this, the pair `0 => "red"` is in the output because the key of `"red"` is automatically assigned to `0` in the first array, whereas it is assigned to `1` in the second array as the key `0` is already taken by `yellow`.

```php
<?php
$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "green", "yellow", "red");
$result = array_diff_assoc($array1, $array2);
print_r($result);
?>
```

The above example will output:

```
Array
(
    [b] => brown
    [c] => blue
    [0] => red
)
```

Example #2 array_diff_assoc() example

Two values from *key* => *value* pairs are considered equal only if `(string) $elem1 === (string) $elem2` . In other words a strict check takes place so the string representations must be the same.

```php
<?php
$array1 = array(0, 1, 2);
$array2 = array("00", "01", "2");
$result = array_diff_assoc($array1, $array2);
print_r($result);
?>
```

The above example will output:

```
Array
(
    [0] => 0
    [1] => 1
)
```

Notes

> **Note** :
>
> This function only checks one dimension of an n-dimensional array. It is possible to check deeper dimensions by using, for example, `array_diff_assoc($array1[0], $array2[0]);`.

> **Note** :
>
> Ensure arguments are passed in the correct order when comparing similar arrays with more keys. The new array should be the first in the list.

See Also

- array_diff() - Computes the difference of arrays
- array_diff_uassoc() - Computes the difference of arrays with additional index check which is performed by a user supplied callback function
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check

array_diff_key

array_diff_key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_diff_key — Computes the difference of arrays using keys for comparison

Description

function **array_diff_key**(array `$array`, array `...$arrays`): array

Compares the keys from `array` against the keys from `arrays` and returns the difference. This function is like array_diff() except the comparison is done on the keys instead of the values.

Parameters

`array`

The array to compare from

`arrays`

Arrays to compare against

Return Values

Returns an array containing all the entries from `array` whose keys are absent from all of the other arrays.

Changelog

Version	Description
8.0.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_diff_key() example

The two keys from the `key => value` pairs are considered equal only if `(string) $key1 === (string) $key2` . In other words a strict type check is executed so the string representation must be the same.

```php
<?php
$array1 = array('blue' => 1, 'red' => 2, 'green' => 3, 'purple' => 4);
$array2 = array('green' => 5, 'yellow' => 7, 'cyan' => 8);

var_dump(array_diff_key($array1, $array2));
?>
```

The above example will output:

```
array(3) {
  ["blue"]=>
  int(1)
  ["red"]=>
  int(2)
  ["purple"]=>
  int(4)
}
```

```php
<?php
$array1 = array('blue' => 1, 'red'  => 2, 'green' => 3, 'purple' => 4);
$array2 = array('green' => 5, 'yellow' => 7, 'cyan' => 8);
$array3 = array('blue' => 6, 'yellow' => 7, 'mauve' => 8);

var_dump(array_diff_key($array1, $array2, $array3));
?>
```

The above example will output:

```
array(2) {
  ["red"]=>
  int(2)
  ["purple"]=>
  int(4)
}
```

Notes

> **Note** :
>
> This function only checks one dimension of a n-dimensional array. Of course you can check deeper dimensions by using `array_diff_key($array1[0], $array2[0]);`.

See Also

- array_diff() - Computes the difference of arrays
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_diff_uassoc() - Computes the difference of arrays with additional index check which is performed by a user supplied callback function
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_diff_ukey() - Computes the difference of arrays using a callback function on the keys for comparison
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_intersect_uassoc() - Computes the intersection of arrays with additional index check, compares indexes by a callback function
- array_intersect_key() - Computes the intersection of arrays using keys for comparison
- array_intersect_ukey() - Computes the intersection of arrays using a callback function on the keys for comparison

array_diff_uassoc

array_diff_uassoc

(PHP 5, PHP 7, PHP 8)

array_diff_uassoc — Computes the difference of arrays with additional index check which is performed by a user supplied callback function

Description

function **array_diff_uassoc**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

Compares `array` against `arrays` and returns the difference. Unlike array_diff() the array keys are used in the comparison.

Unlike array_diff_assoc() a user supplied callback function is used for the indices comparison, not internal function.

Parameters

`array`

The array to compare from

`arrays`

Arrays to compare against

`key_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Returns an array containing all the entries from `array` that are not present in any of the other arrays.

Examples

Example #1 array_diff_uassoc() example

In this example the `"a" => "green"` pair is present in both arrays and thus it is not in the output from the function. Unlike this, the pair `0 => "red"` is in the output because the key of `"red"` is automatically assigned to `0` in the first array, whereas it is assigned to `1` in the second array as the key `0` is already taken by `yellow`.

```php
<?php
function key_compare_func($a, $b)
{
    return $a <=> $b;
}

$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "green", "yellow", "red");
$result = array_diff_uassoc($array1, $array2, "key_compare_func");
print_r($result);
?>
```

The above example will output:

```
Array
(
    [b] => brown
    [c] => blue
    [0] => red
)
```

The equality of 2 indices is checked by the user supplied callback function.

Notes

> **Note** :
>
> This function only checks one dimension of an n-dimensional array. It is possible to check deeper dimensions by using, for example, `array_diff_uassoc($array1[0], $array2[0], "key_compare_func");`.

See Also

- array_diff() - Computes the difference of arrays
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_uintersect() - Computes the intersection of arrays, compares data by a callback function
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function

- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions

array_diff_ukey

array_diff_ukey

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_diff_ukey — Computes the difference of arrays using a callback function on the keys for comparison

Description

function **array_diff_ukey**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

Compares the keys from `array` against the keys from `arrays` and returns the difference. This function is like array_diff() except the comparison is done on the keys instead of the values.

Unlike array_diff_key() a user supplied callback function is used for the indices comparison, not internal function.

Parameters

`array`

The array to compare from

`arrays`

Arrays to compare against

`key_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Returns an array containing all the entries from `array` that are not present in any of the other arrays.

Examples

Example #1 array_diff_ukey() example

```php
<?php
function key_compare_func($key1, $key2)
{
    if ($key1 == $key2)
        return 0;
    else if ($key1 > $key2)
        return 1;
    else
        return -1;
}

$array1 = array('blue'  => 1, 'red'  => 2, 'green'  => 3, 'purple' => 4);
$array2 = array('green' => 5, 'blue' => 6, 'yellow' => 7, 'cyan'   => 8);

var_dump(array_diff_ukey($array1, $array2, 'key_compare_func'));
?>
```

The above example will output:

```
array(2) {
  ["red"]=>
  int(2)
  ["purple"]=>
  int(4)
}
```

Notes

> **Note** :
>
> This function only checks one dimension of a n-dimensional array. Of course you can check deeper dimensions by using `array_diff_ukey($array1[0], $array2[0], 'callback_func');`.

See Also

- array_diff() - Computes the difference of arrays
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_diff_uassoc() - Computes the difference of arrays with additional index check which is performed by a user supplied callback function
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_diff_key() - Computes the difference of arrays using keys for comparison
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_intersect_uassoc() - Computes the intersection of arrays with additional index check, compares indexes by a callback function
- array_intersect_key() - Computes the intersection of arrays using keys for comparison
- array_intersect_ukey() - Computes the intersection of arrays using a callback function on the keys for comparison

array_fill

array_fill

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

array_fill — Fill an array with values

Description

function **array_fill**(int `$start_index`, int `$count`, mixed `$value`): array

Fills an array with `count` entries of the value of the `value` parameter, keys starting at the `start_index` parameter.

Parameters

`start_index`

The first index of the returned array.

If `start_index` is negative, the first index of the returned array will be `start_index` and the following indices will start from zero prior to PHP 8.0.0; as of PHP 8.0.0, negative keys are incremented normally (see example).

`count`

Number of elements to insert. Must be greater than or equal to zero, and less than or equal to `2147483647`.

`value`

Value to use for filling

Return Values

Returns the filled array

Errors/Exceptions

Throws a ValueError if `count` is out of range.

Changelog

Version	Description
8.0.0	**array_fill()** now throws a ValueError if `count` is out of range; previously `E_WARNING` was raised, and the function returned `false`.

Examples

Example #1 array_fill() example

```php
<?php
$a = array_fill(5, 6, 'banana');
print_r($a);
?>
```

The above example will output:

```
Array
(
    [5] => banana
    [6] => banana
    [7] => banana
    [8] => banana
    [9] => banana
    [10] => banana
)
```

Example #2 array_fill() example with a negative start index

```php
<?php
$a = array_fill(-2, 4, 'pear');
print_r($a);
?>
```

Output of the above example in PHP 8:

```
Array
(
    [-2] => pear
    [-1] => pear
    [0] => pear
    [1] => pear
)
```

Output of the above example in PHP 7:

```
Array
(
    [-2] => pear
    [0] => pear
```

```
    [1] => pear
    [2] => pear
)
```

Note that index `-1` is not present prior to PHP 8.0.0.

Notes

See also the Arrays section of manual for a detailed explanation of negative keys.

See Also

- array_fill_keys() - Fill an array with values, specifying keys
- str_repeat() - Repeat a string
- range() - Create an array containing a range of elements

array_fill_keys

array_fill_keys

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

array_fill_keys — Fill an array with values, specifying keys

Description

function **array_fill_keys**(array `$keys`, mixed `$value`): array

Fills an array with the value of the `value` parameter, using the values of the `keys` array as keys.

Parameters

`keys`

Array of values that will be used as keys. Illegal values for key will be converted to string.

`value`

Value to use for filling

Return Values

Returns the filled array

Examples

Example #1 array_fill_keys() example

```php
<?php
$keys = array('foo', 5, 10, 'bar');
$a = array_fill_keys($keys, 'banana');
print_r($a);
?>
```

The above example will output:

```
Array
(
    [foo] => banana
    [5] => banana
    [10] => banana
    [bar] => banana
)
```

See Also

- array_fill() - Fill an array with values
- array_combine() - Creates an array by using one array for keys and another for its values

array_filter

array_filter

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

array_filter — Filters elements of an array using a callback function

Description

function **array_filter**(array `$array`, ?callable `$callback` = **`null`**, int `$mode` = 0): array

Iterates over each value in the `array` passing them to the `callback` function. If the `callback` function returns **`true`**, the current value from `array` is returned into the result array.

Array keys are preserved, and may result in gaps if the `array` was indexed. The result array can be reindexed using the array_values() function.

Parameters

`array`

The array to iterate over

`callback`

The callback function to use

If no `callback` is supplied, all empty entries of `array` will be removed. See empty() for how PHP defines empty in this case.

`mode`

Flag determining what arguments are sent to `callback`:

- **`ARRAY_FILTER_USE_KEY`** - pass key as the only argument to `callback` instead of the value
- **`ARRAY_FILTER_USE_BOTH`** - pass both value and key as arguments to `callback` instead of the value

Default is `0` which will pass value as the only argument to `callback` instead.

Return Values

Returns the filtered array.

Changelog

Version	Description
8.0.0	`callback` is nullable now.
8.0.0	If `callback` expects a parameter to be passed by reference, this function will now emit an **`E_WARNING`**.

Examples

Example #1 array_filter() example

```php
<?php
function odd($var)
{
    // returns whether the input integer is odd
    return $var & 1;
}

function even($var)
{
```

```php
    // returns whether the input integer is even
    return !($var & 1);
}

$array1 = ['a' => 1, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 5];
$array2 = [6, 7, 8, 9, 10, 11, 12];

echo "Odd :\n";
print_r(array_filter($array1, "odd"));
echo "Even:\n";
print_r(array_filter($array2, "even"));
?>
```

The above example will output:

```
Odd :
Array
(
    [a] => 1
    [c] => 3
    [e] => 5
)
Even:
Array
(
    [0] => 6
    [2] => 8
    [4] => 10
    [6] => 12
)
```

Example #2 array_filter() without `callback`

```php
<?php

$entry = [
    0 => 'foo',
    1 => false,
    2 => -1,
    3 => null,
    4 => '',
    5 => '0',
    6 => 0,
];

print_r(array_filter($entry));
?>
```

The above example will output:

```
Array
(
    [0] => foo
    [2] => -1
)
```

Example #3 array_filter() with `mode`

```php
<?php

$arr = ['a' => 1, 'b' => 2, 'c' => 3, 'd' => 4];

var_dump(array_filter($arr, function($k) {
    return $k == 'b';
}, ARRAY_FILTER_USE_KEY));

var_dump(array_filter($arr, function($v, $k) {
    return $k == 'b' || $v == 4;
}, ARRAY_FILTER_USE_BOTH));
?>
```

The above example will output:

```
array(1) {
  ["b"]=>
  int(2)
}
array(2) {
  ["b"]=>
  int(2)
  ["d"]=>
  int(4)
}
```

Notes

> **Caution**
>
> If the array is changed from the callback function (e.g. element added or unset) the behavior of this function is undefined.

See Also

- array_intersect() - Computes the intersection of arrays
- array_find() - Returns the first element satisfying a callback function
- array_any() - Checks if at least one array element satisfies a callback function
- array_map() - Applies the callback to the elements of the given arrays
- array_reduce() - Iteratively reduce the array to a single value using a callback function

array_find

array_find

(PHP 8 >= 8.4.0)

array_find — Returns the first element satisfying a callback function

Description

function **array_find**(array `$array`, callable `$callback`): mixed

array_find() returns the value of the first element of an array for which the given `callback` returns **`true`**. If no matching element is found the function returns **`null`**.

Parameters

`array`

The array that should be searched.

`callback`

The callback function to call to check each element, which must be of the following signature:

function callback(mixed `$value`, mixed `$key`): bool
If this function returns `true`, the value is returned from **array_find()** and the callback will not be called for further elements.

Return Values

The function returns the value of the first element for which the `callback` returns `true`. If no matching element is found the function returns `null`.

Examples

Example #1 array_find() example

```php
<?php
$array = [
    'a' => 'dog',
    'b' => 'cat',
    'c' => 'cow',
    'd' => 'duck',
    'e' => 'goose',
    'f' => 'elephant'
];

// Find the first animal with a name longer than 4 characters.
var_dump(array_find($array, function (string $value) {
    return strlen($value) > 4;
}));

// Find the first animal whose name begins with f.
var_dump(array_find($array, function (string $value) {
    return str_starts_with($value, 'f');
}));

// Find the first animal where the array key is the first symbol of the animal.
var_dump(array_find($array, function (string $value, $key) {
   return $value[0] === $key;
}));

// Find the first animal where the array key matching a RegEx.
var_dump(array_find($array, function ($value, $key) {
   return preg_match('/^([a-f])$/', $key);
}));
?>
```

The above example will output:

```
string(5) "goose"
NULL
string(3) "cow"
string(3) "dog"
```

See Also

- array_find_key() - Returns the key of the first element satisfying a callback function
- array_all() - Checks if all array elements satisfy a callback function
- array_any() - Checks if at least one array element satisfies a callback function
- array_filter() - Filters elements of an array using a callback function
- array_reduce() - Iteratively reduce the array to a single value using a callback function

array_find_key

array_find_key

(PHP 8 >= 8.4.0)

array_find_key — Returns the key of the first element satisfying a callback function

Description

function **array_find_key**(array `$array`, callable `$callback`): mixed

array_find_key() returns the key of the first element of an array for which the given `callback` returns **`true`**. If no matching element is found the function returns **`null`**.

Parameters

`array`

The array that should be searched.

`callback`

The callback function to call to check each element, which must be of the following signature:

function callback(mixed `$value`, mixed `$key`): bool
If this function returns **`true`**, the key is returned from **array_find_key()** and the callback will not be called for further elements.

Return Values

The function returns the key of the first element for which the `callback` returns **`true`**. If no matching element is found the function returns **`null`**.

Examples

Example #1 array_find_key() example

```php
<?php
$array = [
    'a' => 'dog',
    'b' => 'cat',
    'c' => 'cow',
    'd' => 'duck',
    'e' => 'goose',
    'f' => 'elephant'
];

// Find the first animal with a name longer than 4 characters.
var_dump(array_find_key($array, function (string $value) {
    return strlen($value) > 4;
}));

// Find the first animal whose name begins with f.
var_dump(array_find_key($array, function (string $value) {
    return str_starts_with($value, 'f');
}));

// Find the first animal where the array key is the first symbol of the animal.
var_dump(array_find_key($array, function (string $value, $key) {
   return $value[0] === $key;
}));

// Find the first animal where the array key matching a RegEx.
var_dump(array_find_key($array, function ($value, $key) {
   return preg_match('/^([a-f])$/', $key);
}));
?>
```

The above example will output:

```
string(1) "e"
NULL
```

```
string(1) "c"
string(1) "a"
```

See Also

- array_find() - Returns the first element satisfying a callback function
- array_all() - Checks if all array elements satisfy a callback function
- array_any() - Checks if at least one array element satisfies a callback function
- array_filter() - Filters elements of an array using a callback function
- array_reduce() - Iteratively reduce the array to a single value using a callback function

array_first

array_first

(PHP 8 >= 8.5.0)

array_first — Gets the first value of an array

Description

function **array_first**(array `$array`): mixed

Get the first value of the given `array`.

Parameters

`array`

An array.

Return Values

Returns the first value of `array` if the array is not empty; `null` otherwise.

Examples

Example #1 Basic array_first() Usage

```php
<?php
$array = [1 => 'a', 0 => 'b', 3 => 'c', 2 => 'd'];

$firstValue = array_first($array);

var_dump($firstValue);
?>
```

The above example will output:

```
string(1) "a"
```

See Also

- array_key_first() - Gets the first key of an array
- array_last() - Gets the last value of an array

array_flip

array_flip

(PHP 4, PHP 5, PHP 7, PHP 8)

array_flip — Exchanges all keys with their associated values in an array

Description

function **array_flip**(array $array): array

array_flip() returns an array in flip order, i.e. keys from `array` become values and values from `array` become keys.

Note that the values of `array` need to be valid keys, i.e. they need to be either int or string. A warning will be emitted if a value has the wrong type, and the key/value pair in question *will not be included in the result*.

If a value has several occurrences, the latest key will be used as its value, and all others will be lost.

Parameters

`array`

An array of key/value pairs to be flipped.

Return Values

Returns the flipped array.

Examples

Example #1 array_flip() example

```php
<?php
$input = array("oranges", "apples", "pears");
$flipped = array_flip($input);

print_r($flipped);
?>
```

The above example will output:

```
Array
(
    [oranges] => 0
    [apples] => 1
    [pears] => 2
)
```

Example #2 array_flip() example : collision

```php
<?php
$input = array("a" => 1, "b" => 1, "c" => 2);
$flipped = array_flip($input);

print_r($flipped);
?>
```

The above example will output:

```
Array
(
    [1] => b
    [2] => c
)
```

See Also

- array_values() - Return all the values of an array
- array_keys() - Return all the keys or a subset of the keys of an array
- array_reverse() - Return an array with elements in reverse order

array_intersect

array_intersect

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_intersect — Computes the intersection of arrays

Description

function **array_intersect**(array `$array`, array `...$arrays`): array

array_intersect() returns an array containing all the values of `array` that are present in all the arguments. Note that keys are preserved.

Parameters

`array`

The array with master values to check.

`arrays`

Arrays to compare values against.

Return Values

Returns an array containing all of the values in `array` whose values exist in all of the parameters.

Changelog

Version	Description
8.0.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_intersect() example

```php
<?php
$array1 = array("a" => "green", "red", "blue");
$array2 = array("b" => "green", "yellow", "red");
$result = array_intersect($array1, $array2);
print_r($result);
?>
```

The above example will output:

```
Array
(
    [a] => green
    [0] => red
)
```

Notes

> **Note** :
>
> Two elements are considered equal if and only if `(string) $elem1 === (string) $elem2`. In words: when the string representation is the same.

See Also

- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_diff() - Computes the difference of arrays
- array_diff_assoc() - Computes the difference of arrays with additional index check

array_intersect_assoc

array_intersect_assoc

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

array_intersect_assoc — Computes the intersection of arrays with additional index check

Description

function **array_intersect_assoc**(array `$array`, array `...$arrays`): array

array_intersect_assoc() returns an array containing all the values of `array` that are present in all the arguments. Note that the keys are also used in the comparison unlike in array_intersect().

Parameters

`array`

The array with master values to check.

`arrays`

Arrays to compare values against.

Return Values

Returns an associative array containing all the values in `array` that are present in all of the arguments.

Changelog

Version	Description
8.0.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_intersect_assoc() example

```php
<?php
$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "green", "b" => "yellow", "blue", "red");
$result_array = array_intersect_assoc($array1, $array2);
print_r($result_array);
?>
```

The above example will output:

```
Array
(
    [a] => green
)
```

In our example you see that only the pair `"a" => "green"` is present in both arrays and thus is returned. The value `"red"` is not returned because in *$array1* its key is `0` while the key of "red" in *$array2* is `1`, and the key `"b"` is not returned because its values are different in each array.

The two values from the `key => value` pairs are considered equal only if `(string) $elem1 === (string) $elem2` . In other words a strict type check is executed so the string representation must be the same.

See Also

- array_intersect() - Computes the intersection of arrays
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function
- array_intersect_uassoc() - Computes the intersection of arrays with additional index check, compares indexes by a callback function
- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions
- array_diff() - Computes the difference of arrays
- array_diff_assoc() - Computes the difference of arrays with additional index check

array_intersect_key

array_intersect_key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_intersect_key — Computes the intersection of arrays using keys for comparison

Description

function **array_intersect_key**(array `$array`, array `...$arrays`): array

array_intersect_key() returns an array containing all the entries of `array` which have keys that are present in all the arguments.

Parameters

`array`

The array with master keys to check.

`arrays`

Arrays to compare keys against.

Return Values

Returns an associative array containing all the entries of `array` which have keys that are present in all arguments.

Changelog

Version	Description
8.0.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_intersect_key() example

```php
<?php
$array1 = array('blue'  => 1, 'red'  => 2, 'green'  => 3, 'purple' => 4);
$array2 = array('green' => 5, 'blue' => 6, 'yellow' => 7, 'cyan'   => 8);

var_dump(array_intersect_key($array1, $array2));
?>
```

The above example will output:

```
array(2) {
  ["blue"]=>
  int(1)
  ["green"]=>
  int(3)
}
```

In our example you see that only the keys `'blue'` and `'green'` are present in both arrays and thus returned. Also notice that the values for the keys `'blue'` and `'green'` differ between the two arrays. A match still occurs because only the keys are checked. The values returned are those of `array`.

The two keys from the `key => value` pairs are considered equal only if `(string) $key1 === (string) $key2` . In other words a strict type check is executed so the string representation must be the same.

See Also

- array_diff() - Computes the difference of arrays
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_diff_uassoc() - Computes the difference of arrays with additional index check which is performed by a user supplied callback function
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_diff_key() - Computes the difference of arrays using keys for comparison
- array_diff_ukey() - Computes the difference of arrays using a callback function on the keys for comparison
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_intersect_uassoc() - Computes the intersection of arrays with additional index check, compares indexes by a callback function
- array_intersect_ukey() - Computes the intersection of arrays using a callback function on the keys for comparison

array_intersect_uassoc

array_intersect_uassoc

(PHP 5, PHP 7, PHP 8)

array_intersect_uassoc — Computes the intersection of arrays with additional index check, compares indexes by a callback function

Description

function **array_intersect_uassoc**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

array_intersect_uassoc() returns an array containing all the values of `array` that are present in all the arguments. Note that the keys are used in the comparison unlike in array_intersect().

Parameters

`array`

Initial array for comparison of the arrays.

`arrays`

Arrays to compare keys against.

`key_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**

> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Returns the values of `array` whose values exist in all of the arguments.

Examples

Example #1 array_intersect_uassoc() example

```
<?php
$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "GREEN", "B" => "brown", "yellow", "red");

print_r(array_intersect_uassoc($array1, $array2, "strcasecmp"));
?>
```

The above example will output:

```
Array
(
    [b] => brown
)
```

See Also

- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function
- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions
- array_intersect_key() - Computes the intersection of arrays using keys for comparison
- array_intersect_ukey() - Computes the intersection of arrays using a callback function on the keys for comparison

array_intersect_ukey

array_intersect_ukey

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_intersect_ukey — Computes the intersection of arrays using a callback function on the keys for comparison

Description

function **array_intersect_ukey**(array `$array`, array `...$arrays`, callable `$key_compare_func`): array

array_intersect_ukey() returns an array containing all the values of `array` which have matching keys that are present in all the arguments.

Parameters

`array`

Initial array for comparison of the arrays.

`arrays`

Arrays to compare keys against.

`key_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed $a, mixed $b): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Returns the values of `array` whose keys exist in all the arguments.

Examples

Example #1 array_intersect_ukey() example

```php
<?php
function key_compare_func($key1, $key2)
{
    if ($key1 == $key2)
        return 0;
    else if ($key1 > $key2)
        return 1;
    else
        return -1;
}

$array1 = array('blue'  => 1, 'red'  => 2, 'green'  => 3, 'purple' => 4);
$array2 = array('green' => 5, 'blue' => 6, 'yellow' => 7, 'cyan'   => 8);

var_dump(array_intersect_ukey($array1, $array2, 'key_compare_func'));
?>
```

The above example will output:

```
array(2) {
  ["blue"]=>
  int(1)
  ["green"]=>
  int(3)
}
```

In our example you see that only the keys `'blue'` and `'green'` are present in both arrays and thus returned. Also notice that the values for the keys `'blue'` and `'green'` differ between the two arrays. A match still occurs because only the keys are checked. The values returned are those of `array`.

See Also

- array_diff() - Computes the difference of arrays
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_diff_uassoc() - Computes the difference of arrays with additional index check which is performed by a user supplied callback function
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_diff_key() - Computes the difference of arrays using keys for comparison
- array_diff_ukey() - Computes the difference of arrays using a callback function on the keys for comparison
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_intersect_uassoc() - Computes the intersection of arrays with additional index check, compares indexes by a callback function
- array_intersect_key() - Computes the intersection of arrays using keys for comparison

array_is_list

array_is_list

(PHP 8 >= 8.1.0)

array_is_list — Checks whether a given `array` is a list

Description

function **array_is_list**(array `$array`): bool

Determines if the given `array` is a list. An array is considered a list if its keys consist of consecutive numbers from `0` to `count($array)-1`.

Parameters

`array`

The array being evaluated.

Return Values

Returns **`true`** if `array` is a list, **`false`** otherwise.

Examples

Example #1 array_is_list() example

```php
<?php
var_dump(array_is_list([])); // true
var_dump(array_is_list(['apple', 2, 3])); // true
var_dump(array_is_list([0 => 'apple', 'orange'])); // true

// The array does not start at 0
var_dump(array_is_list([1 => 'apple', 'orange'])); // false

// The keys are not in the correct order
var_dump(array_is_list([1 => 'apple', 0 => 'orange'])); // false

// Non-integer keys
var_dump(array_is_list([0 => 'apple', 'foo' => 'bar'])); // false

// Non-consecutive keys
var_dump(array_is_list([0 => 'apple', 2 => 'bar'])); // false
?>
```

Notes

> **Note** :
>
> This function returns **`true`** on empty arrays.

See Also

- array_values() - Return all the values of an array

array_key_exists

array_key_exists

(PHP 4 >= 4.0.7, PHP 5, PHP 7, PHP 8)

array_key_exists — Checks if the given key or index exists in the array

Description

function **array_key_exists**(string|int|float|bool|resource|null `$key`, array `$array`): bool

array_key_exists() returns **`true`** if the given `key` is set in the array. `key` can be any value possible for an array index.

Parameters

`key`

Value to check.

`array`

An array with keys to check.

Return Values

Returns `true` on success or `false` on failure.

> **Note** :
>
> **array_key_exists()** will search for the keys in the first dimension only. Nested keys in multidimensional arrays will not be found.

Changelog

Version	Description
8.5.0	Using null in the `key` parameter is deprecated, use an empty string instead.
8.0.0	The `key` parameter now accepts bool, float, int, null, resource, and string as arguments.
8.0.0	Passing an object to the `array` parameter is no longer supported.
7.4.0	Passing an object to the `array` parameter has been deprecated. Use property_exists() instead.

Examples

Example #1 **array_key_exists()** example

```php
<?php
$searchArray = ['first' => 1, 'second' => 4];
var_dump(array_key_exists('first', $searchArray));
?>
```

The above example will output:

```
bool(true)
```

Example #2 **array_key_exists()** vs isset()

isset() does not return `true` for array keys that correspond to a `null` value, while **array_key_exists()** does.

```php
<?php
$searchArray = ['first' => null, 'second' => 4];

var_dump(isset($searchArray['first']));
var_dump(array_key_exists('first', $searchArray));
?>
```

The above example will output:

```
bool(false)
bool(true)
```

See Also

- isset() - Determine if a variable is declared and is different than null
- array_keys() - Return all the keys or a subset of the keys of an array
- in_array() - Checks if a value exists in an array
- property_exists() - Checks if the object or class has a property

array_key_first

array_key_first

(PHP 7 >= 7.3.0, PHP 8)

array_key_first — Gets the first key of an array

Description

function **array_key_first**(array `$array`): int|string|null

Get the first key of the given `array` without affecting the internal array pointer.

Parameters

`array`

An array.

Return Values

Returns the first key of `array` if the array is not empty; `null` otherwise.

Examples

Example #1 Basic array_key_first() Usage

```php
<?php
$array = ['a' => 1, 'b' => 2, 'c' => 3];

$firstKey = array_key_first($array);

var_dump($firstKey);
?>
```

The above example will output:

```
string(1) "a"
```

Notes

> **Tip**
>
> There are several ways to provide this functionality for versions prior to PHP 7.3.0. It is possible to use array_keys(), but that may be rather inefficient. It is also possible to use reset() and key(), but that may change the internal array pointer. An efficient solution, which does not change the internal array pointer, written as polyfill:
>
> ```php
> <?php
> if (!function_exists('array_key_first')) {
>     function array_key_first(array $arr) {
>         foreach($arr as $key => $unused) {
>             return $key;
>         }
>         return NULL;
>     }
> ```

```
}
?>
```

See Also

- array_first() - Gets the first value of an array
- array_key_last() - Gets the last key of an array
- reset() - Set the internal pointer of an array to its first element

array_key_last

array_key_last

(PHP 7 >= 7.3.0, PHP 8)

array_key_last — Gets the last key of an array

Description

function **array_key_last**(array `$array`): int|string|null

Get the last key of the given `array` without affecting the internal array pointer.

Parameters

`array`

An array.

Return Values

Returns the last key of `array` if the array is not empty; `null` otherwise.

See Also

- array_last() - Gets the last value of an array
- array_key_first() - Gets the first key of an array
- end() - Set the internal pointer of an array to its last element

array_keys

array_keys

(PHP 4, PHP 5, PHP 7, PHP 8)

array_keys — Return all the keys or a subset of the keys of an array

Description

function **array_keys**(array `$array`): array
function **array_keys**(array `$array`, mixed `$filter_value`, bool `$strict = false`): array

array_keys() returns the keys, numeric and string, from the `array`.

If a `filter_value` is specified, then only the keys for that value are returned. Otherwise, all the keys from the `array` are returned.

Parameters

`array`

An array containing keys to return.

`filter_value`

If specified, then only keys containing this value are returned.

`strict`

Determines if strict comparison (===) should be used during the search.

Return Values

Returns an array of all the keys in `array`.

Examples

Example #1 array_keys() example

```php
<?php
$array = array(0 => 100, "color" => "red");
print_r(array_keys($array));

$array = array("blue", "red", "green", "blue", "blue");
print_r(array_keys($array, "blue"));

$array = array("color" => array("blue", "red", "green"),
               "size"  => array("small", "medium", "large"));
print_r(array_keys($array));
?>
```

The above example will output:

```
Array
(
    [0] => 0
    [1] => color
)
Array
(
    [0] => 0
    [1] => 3
    [2] => 4
)
Array
(
    [0] => color
    [1] => size
)
```

See Also

- array_values() - Return all the values of an array
- array_combine() - Creates an array by using one array for keys and another for its values
- array_key_exists() - Checks if the given key or index exists in the array
- array_search() - Searches the array for a given value and returns the first corresponding key if successful

array_last

array_last

(PHP 8 >= 8.5.0)

array_last — Gets the last value of an array

Description

function **array_last**(array `$array`): mixed

Get the last value of the given `array`.

Parameters

`array`

An array.

Return Values

Returns the last value of `array` if the array is not empty; `null` otherwise.

Examples

Example #1 Basic array_last() Usage

```php
<?php
$array = [1 => 'a', 0 => 'b', 3 => 'c', 2 => 'd'];

$lastValue = array_last($array);

var_dump($lastValue);
?>
```

The above example will output:

```
string(1) "d"
```

See Also

- array_key_last() - Gets the last key of an array
- array_first() - Gets the first value of an array

array_map

array_map

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

array_map — Applies the callback to the elements of the given arrays

Description

function **array_map**(?callable `$callback`, array `$array`, array `...$arrays`): array

array_map() returns an array containing the results of applying the `callback` to the corresponding value of `array` (and `arrays` if more arrays are provided) used as arguments for the callback. The number of parameters that the `callback` function accepts should match the number of arrays passed to **array_map()**. Excess input arrays are ignored. An ArgumentCountError is thrown if an insufficient number of arguments is provided.

Parameters

`callback`

A callable to run for each element in each array.

`null` can be passed as a value to `callback` to perform a zip operation on multiple arrays and return an array where each element is an array containing the elements from the input arrays at the same position of the internal array pointer (see example below). If only `array` is provided, **array_map()** will return the input array.

`array`

An array to run through the `callback` function.

`arrays`

Supplementary variable list of array arguments to run through the `callback` function.

Return Values

Returns an array containing the results of applying the `callback` function to the corresponding value of `array` (and `arrays` if more arrays are provided) used as arguments for the callback.

The returned array will preserve the keys of the array argument if and only if exactly one array is passed. If more than one array is passed, the returned array will have sequential integer keys.

Changelog

Version	Description
8.0.0	If `callback` expects a parameter to be passed by reference, this function will now emit an `E_WARNING`.

Examples

Example #1 array_map() example

```
<?php
function cube($n)
{
    return ($n * $n * $n);
}

$a = [1, 2, 3, 4, 5];
$b = array_map('cube', $a);
print_r($b);
?>
```

This makes *$b* have:

```
Array
(
    [0] => 1
    [1] => 8
    [2] => 27
    [3] => 64
    [4] => 125
)
```

Example #2 array_map() using a lambda function

```
<?php
$func = function(int $value): int {
    return $value * 2;
};

print_r(array_map($func, range(1, 5)));

// Or as of PHP 7.4.0:

print_r(array_map(fn($value): int => $value * 2, range(1, 5)));

?>
```

The above example will output:

```
Array
(
    [0] => 2
    [1] => 4
    [2] => 6
    [3] => 8
    [4] => 10
)
Array
(
```

```
    [0] => 2
    [1] => 4
    [2] => 6
    [3] => 8
    [4] => 10
)
```

Example #3 array_map() - using more arrays

```php
<?php
function show_Spanish(int $n, string $m): string
{
    return "The number {$n} is called {$m} in Spanish";
}

function map_Spanish(int $n, string $m): array
{
    return [$n => $m];
}

$a = [1, 2, 3, 4, 5];
$b = ['uno', 'dos', 'tres', 'cuatro', 'cinco'];

$c = array_map('show_Spanish', $a, $b);
print_r($c);

$d = array_map('map_Spanish', $a , $b);
print_r($d);
?>
```

The above example will output:

```
// printout of $c
Array
(
    [0] => The number 1 is called uno in Spanish
    [1] => The number 2 is called dos in Spanish
    [2] => The number 3 is called tres in Spanish
    [3] => The number 4 is called cuatro in Spanish
    [4] => The number 5 is called cinco in Spanish
)

// printout of $d
Array
(
    [0] => Array
        (
            [1] => uno
        )

    [1] => Array
        (
            [2] => dos
        )

    [2] => Array
        (
            [3] => tres
        )

    [3] => Array
        (
            [4] => cuatro
        )
```

```
    [4] => Array
        (
            [5] => cinco
        )

)
```

Usually when using two or more arrays, they should be of equal length because the callback function is applied in parallel to the corresponding elements. If the arrays are of unequal length, shorter ones will be extended with empty elements to match the length of the longest.

An interesting use of this function is to construct an array of arrays, which can be easily performed by using `null` as the name of the callback function

Example #4 Performing a zip operation of arrays

```php
<?php
$a = [1, 2, 3, 4, 5];
$b = ['one', 'two', 'three', 'four', 'five'];
$c = ['uno', 'dos', 'tres', 'cuatro', 'cinco'];

$d = array_map(null, $a, $b, $c);
print_r($d);
?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [0] => 1
            [1] => one
            [2] => uno
        )

    [1] => Array
        (
            [0] => 2
            [1] => two
            [2] => dos
        )

    [2] => Array
        (
            [0] => 3
            [1] => three
            [2] => tres
        )

    [3] => Array
        (
            [0] => 4
            [1] => four
            [2] => cuatro
        )

    [4] => Array
        (
            [0] => 5
            [1] => five
            [2] => cinco
        )

)
```

Example #5 `null` `callback` with only `array`

```php
<?php
$array = [1, 2, 3];
var_dump(array_map(null, $array));
?>
```

The above example will output:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

Example #6 array_map() - with string keys

```php
<?php
$arr = ['stringkey' => 'value'];
function cb1($a) {
    return [$a];
}
function cb2($a, $b) {
    return [$a, $b];
}
var_dump(array_map('cb1', $arr));
var_dump(array_map('cb2', $arr, $arr));
var_dump(array_map(null,  $arr));
var_dump(array_map(null, $arr, $arr));
?>
```

The above example will output:

```
array(1) {
  ["stringkey"]=>
  array(1) {
    [0]=>
    string(5) "value"
  }
}
array(1) {
  [0]=>
  array(2) {
    [0]=>
    string(5) "value"
    [1]=>
    string(5) "value"
  }
}
array(1) {
  ["stringkey"]=>
  string(5) "value"
}
array(1) {
  [0]=>
  array(2) {
```

```
    [0]=>
    string(5) "value"
    [1]=>
    string(5) "value"
  }
}
```

Example #7 array_map() - associative arrays

While **array_map()** does not directly support using the array key as an input, that may be simulated using array_keys().

```php
<?php
$arr = [
    'v1' => 'First release',
    'v2' => 'Second release',
    'v3' => 'Third release',
];

// Note: Before 7.4.0, use the longer syntax for anonymous functions instead.
$callback = fn(string $k, string $v): string => "$k was the $v";

$result = array_map($callback, array_keys($arr), array_values($arr));

var_dump($result);
?>
```

The above example will output:

```
array(3) {
  [0]=>
  string(24) "v1 was the First release"
  [1]=>
  string(25) "v2 was the Second release"
  [2]=>
  string(24) "v3 was the Third release"
}
```

See Also

- array_filter() - Filters elements of an array using a callback function
- array_reduce() - Iteratively reduce the array to a single value using a callback function
- array_walk() - Apply a user supplied function to every member of an array

array_merge

array_merge

(PHP 4, PHP 5, PHP 7, PHP 8)

array_merge — Merge one or more arrays

Description

function **array_merge**(array `...$arrays`): array

Merges the elements of one or more arrays together so that the values of one are appended to the end of the previous one. It returns the resulting array.

If the input arrays have the same string keys, then the later value for that key will overwrite the previous one. If, however, the arrays contain numeric keys, the later value will **not** overwrite the original value, but will be appended.

Values in the input arrays with numeric keys will be renumbered with incrementing keys starting from zero in the result array.

Parameters

`arrays`

Variable list of arrays to merge.

Return Values

Returns the resulting array. If called without any arguments, returns an empty array.

Changelog

Version	Description
7.4.0	This function can now be called without any parameter. Formerly, at least one parameter has been required.

Examples

Example #1 **array_merge()** example

```php
<?php

$array1 = ["color" => "red", 2, 4];
$array2 = ["a", "b", "color" => "green", "shape" => "trapezoid", 4];

$result = array_merge($array1, $array2);

print_r($result);
```

The above example will output:

```
Array
(
    [color] => green
    [0] => 2
    [1] => 4
    [2] => a
    [3] => b
    [shape] => trapezoid
    [4] => 4
)
```

Example #2 Simple **array_merge()** example

```php
<?php

$array1 = [];
$array2 = [1 => "data"];

$result = array_merge($array1, $array2);

print_r($result);
```

Don't forget that numeric keys will be renumbered!

```
Array
(
    [0] => data
)
```

If you want to append array elements from the second array to the first array while not overwriting the elements from the first array and not re-indexing, use the + array union operator:

```php
<?php

$array1 = [0 => 'zero_a', 2 => 'two_a', 3 => 'three_a'];
$array2 = [1 => 'one_b', 3 => 'three_b', 4 => 'four_b'];

$result = $array1 + $array2;

var_dump($result);
```

The keys from the first array will be preserved. If an array key exists in both arrays, then the element from the first array will be used and the matching key's element from the second array will be ignored.

```
array(5) {
  [0]=>
  string(6) "zero_a"
  [2]=>
  string(5) "two_a"
  [3]=>
  string(7) "three_a"
  [1]=>
  string(5) "one_b"
  [4]=>
  string(6) "four_b"
}
```

See Also

- array_merge_recursive() - Merge one or more arrays recursively
- array_replace() - Replaces elements from passed arrays into the first array
- array_combine() - Creates an array by using one array for keys and another for its values
- array operators

array_merge_recursive

array_merge_recursive

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_merge_recursive — Merge one or more arrays recursively

Description

function **array_merge_recursive**(array `...$arrays`): array

array_merge_recursive() merges the elements of one or more arrays together so that the values of one are appended to the end of the previous one. It returns the resulting array.

If the input arrays have the same string keys, then the values for these keys are merged together into an array, and this is done recursively, so that if one of the values is an array itself, the function will merge it with a corresponding entry in another array too. If, however, the arrays have the same numeric key, the later value will not overwrite the original value, but will be appended.

Parameters

`arrays`

Variable list of arrays to recursively merge.

Return Values

An array of values resulted from merging the arguments together. If called without any arguments, returns an empty array.

Changelog

Version	Description
7.4.0	This function can now be called without any parameter. Formerly, at least one parameter has been required.

Examples

Example #1 array_merge_recursive() example

```php
<?php
$ar1 = array("color" => array("favorite" => "red"), 5);
$ar2 = array(10, "color" => array("favorite" => "green", "blue"));
$result = array_merge_recursive($ar1, $ar2);
print_r($result);
?>
```

The above example will output:

```
Array
(
    [color] => Array
        (
            [favorite] => Array
                (
                    [0] => red
                    [1] => green
                )

            [0] => blue
        )

    [0] => 5
    [1] => 10
)
```

See Also

- array_merge() - Merge one or more arrays
- array_replace_recursive() - Replaces elements from passed arrays into the first array recursively

array_multisort

array_multisort

(PHP 4, PHP 5, PHP 7, PHP 8)

array_multisort — Sort multiple or multi-dimensional arrays

Description

function **array_multisort**(
  array &$array1,
  mixed $array1_sort_order = SORT_ASC,
  mixed $array1_sort_flags = SORT_REGULAR,
  mixed ...$rest
): true

array_multisort() can be used to sort several arrays at once, or a multi-dimensional array by one or more dimensions.

Associative (string) keys will be maintained, but numeric keys will be re-indexed.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array1`

An array being sorted.

`array1_sort_order`

The order used to sort the previous array argument. Either **`SORT_ASC`** to sort ascendingly or **`SORT_DESC`** to sort descendingly.

This argument can be swapped with `array1_sort_flags` or omitted entirely, in which case **`SORT_ASC`** is assumed.

`array1_sort_flags`

Sort options for the previous array argument:

Sorting type flags:

- **`SORT_REGULAR`** - compare items normally (don't change types)
- **`SORT_NUMERIC`** - compare items numerically
- **`SORT_STRING`** - compare items as strings
- **`SORT_LOCALE_STRING`** - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- **`SORT_NATURAL`** - compare items as strings using "natural ordering" like natsort()
- **`SORT_FLAG_CASE`** - can be combined (bitwise OR) with **`SORT_STRING`** or **`SORT_NATURAL`** to sort strings case-insensitively

This argument can be swapped with `array1_sort_order` or omitted entirely, in which case **`SORT_REGULAR`** is assumed.

`rest`

More arrays, optionally followed by sort order and flags. Only elements corresponding to equivalent elements in previous arrays are compared. In other words, the sort is lexicographical.

Return Values

Always returns **`true`**.

Changelog

Version	Description
8.5.0	The return type is **`true`** now; previously, it was bool.

Examples

Example #1 Sorting multiple arrays

```
<?php
$ar1 = array(10, 100, 100, 0);
$ar2 = array(1, 3, 2, 4);
array_multisort($ar1, $ar2);
```

```
var_dump($ar1);
var_dump($ar2);
?>
```

In this example, after sorting, the first array will contain 0, 10, 100, 100. The second array will contain 4, 1, 2, 3. The entries in the second array corresponding to the identical entries in the first array (100 and 100) were sorted as well.

```
array(4) {
  [0]=> int(0)
  [1]=> int(10)
  [2]=> int(100)
  [3]=> int(100)
}
array(4) {
  [0]=> int(4)
  [1]=> int(1)
  [2]=> int(2)
  [3]=> int(3)
}
```

Example #2 Sorting multi-dimensional array

```
<?php
$ar = array(
       array("10", 11, 100, 100, "a"),
       array(   1,  2, "2",   3,   1)
      );
array_multisort($ar[0], SORT_ASC, SORT_STRING,
                $ar[1], SORT_NUMERIC, SORT_DESC);
var_dump($ar);
?>
```

In this example, after sorting, the first array will transform to "10", 100, 100, 11, "a" (it was sorted as strings in ascending order). The second will contain 1, 3, "2", 2, 1 (sorted as numbers, in descending order).

```
array(2) {
  [0]=> array(5) {
    [0]=> string(2) "10"
    [1]=> int(100)
    [2]=> int(100)
    [3]=> int(11)
    [4]=> string(1) "a"
  }
  [1]=> array(5) {
    [0]=> int(1)
    [1]=> int(3)
    [2]=> string(1) "2"
    [3]=> int(2)
    [4]=> int(1)
  }
}
```

Example #3 Sorting database results

For this example, each element in the *data* array represents one row in a table. This type of dataset is typical of database records.

Example data:

```
volume | edition
-------+--------
    67 |       2
    86 |       1
    85 |       6
    98 |       2
    86 |       6
    67 |       7
```

The data as an array, called *data*. This would usually, for example, be obtained by looping with mysqli_fetch_assoc().

In this example, we will order by *volume* descending, *edition* ascending.

We have an array of rows, but **array_multisort()** requires an array of columns, so we use the below code to obtain the columns, then perform the sorting.

```php
<?php
// The data as created by looping over mysqli_fetch_assoc:
$data[] = array('volume' => 67, 'edition' => 2);
$data[] = array('volume' => 86, 'edition' => 1);
$data[] = array('volume' => 85, 'edition' => 6);
$data[] = array('volume' => 98, 'edition' => 2);
$data[] = array('volume' => 86, 'edition' => 6);
$data[] = array('volume' => 67, 'edition' => 7);

// Obtain a list of columns
foreach ($data as $key => $row) {
    $volume[$key]  = $row['volume'];
    $edition[$key] = $row['edition'];
}

// You can use array_column() instead of the above code
$volume  = array_column($data, 'volume');
$edition = array_column($data, 'edition');

// Sort the data with volume descending, edition ascending
// Add $data as the last parameter, to sort by the common key
array_multisort($volume, SORT_DESC, $edition, SORT_ASC, $data);

// Loop over the data and output the sorted values for each column
echo 'volume | edition', PHP_EOL;
echo '-------+--------', PHP_EOL;
for ($i = 0; $i < count($data); $i++) {
     printf("%6d | %7d\n", $volume[$i], $edition[$i]);
}
?>
```

The dataset is now sorted, and will look like this:

```
volume | edition
-------+--------
    98 |       2
    86 |       1
    86 |       6
    85 |       6
    67 |       2
    67 |       7
```

Example #4 Case insensitive sorting

Both `SORT_STRING` and `SORT_REGULAR` are case sensitive, strings starting with a capital letter will come before strings starting with a lowercase letter.

To perform a case insensitive sort, force the sorting order to be determined by a lowercase copy of the original array.

```php
<?php
$array = array('Alpha', 'atomic', 'Beta', 'bank');
$array_lowercase = array_map('strtolower', $array);

array_multisort($array_lowercase, SORT_ASC, SORT_STRING, $array);

print_r($array);
?>
```

The above example will output:

```
Array
(
    [0] => Alpha
    [1] => atomic
    [2] => bank
    [3] => Beta
)
```

See Also

- usort() - Sort an array by values using a user-defined comparison function
- The comparison of array sorting functions

array_pad

array_pad

(PHP 4, PHP 5, PHP 7, PHP 8)

array_pad — Pad array to the specified length with a value

Description

function **array_pad**(array `$array`, int `$length`, mixed `$value`): array

array_pad() returns a copy of the `array` padded to size specified by `length` with value `value`. If `length` is positive then the array is padded on the right, if it's negative then on the left. If the absolute value of `length` is less than or equal to the length of the `array` then no padding takes place.

Parameters

`array`

Initial array of values to pad.

`length`

New size of the array.

`value`

Value to pad if `array` is less than `length`.

Return Values

Returns a copy of the `array` padded to size specified by `length` with value `value`. If `length` is positive then the array is padded on the right, if it's negative then on the left. If the absolute value of `length` is less than or equal to the length of the `array` then no padding takes place.

Changelog

Version	Description
8.3.0	Prior to 8.3, only 1048576 elements could be added at once. Now it is limited only by the maximum size of an array instead.

Examples

Example #1 array_pad() example

```php
<?php
$input = array(12, 10, 9);

$result = array_pad($input, 5, 0);
// result is array(12, 10, 9, 0, 0)
echo join(', ', $result), PHP_EOL;

$result = array_pad($input, -7, -1);
// result is array(-1, -1, -1, -1, 12, 10, 9)
echo join(', ', $result), PHP_EOL;

$result = array_pad($input, 2, "noop");
// not padded
echo join(', ', $result), PHP_EOL;
?>
```

See Also

- array_fill() - Fill an array with values
- range() - Create an array containing a range of elements

array_pop

array_pop

(PHP 4, PHP 5, PHP 7, PHP 8)

array_pop — Pop the element off the end of array

Description

function **array_pop**(array `&$array`): mixed

array_pop() pops and returns the value of the last element of `array`, shortening the `array` by one element.

> **Note** :
>
> This function will reset() the array pointer of the input array after use.

Parameters

`array`

The array to get the value from.

Return Values

Returns the value of the last element of `array`. If `array` is empty, `null` will be returned.

Examples

Example #1 array_pop() example

```php
<?php
$stack = array("orange", "banana", "apple", "raspberry");
$fruit = array_pop($stack);
print_r($stack);
?>
```

After this, *$stack* will have only 3 elements:

```
Array
(
    [0] => orange
    [1] => banana
    [2] => apple
)
```

and `raspberry` will be assigned to *$fruit*.

See Also

- array_push() - Push one or more elements onto the end of array
- array_shift() - Shift an element off the beginning of array
- array_unshift() - Prepend one or more elements to the beginning of an array

array_product

array_product

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

array_product — Calculate the product of values in an array

Description

function **array_product**(array `$array`): int|float

array_product() returns the product of values in an array.

Parameters

`array`

The array.

Return Values

Returns the product as an integer or float.

Changelog

Version	Description
8.3.0	Now emits `E_WARNING` when `array` values cannot be converted to int or float. Previously arrays and objects where ignored whilst every other value was cast to int. Moreover, objects that define a numeric cast (e.g. GMP) are now cast instead of ignored.

Examples

Example #1 **array_product()** examples

```php
<?php

$a = array(2, 4, 6, 8);
echo "product(a) = " . array_product($a) . "\n";
echo "product(array()) = " . array_product(array()) . "\n";

?>
```

The above example will output:

```
product(a) = 384
product(array()) = 1
```

array_push

array_push

(PHP 4, PHP 5, PHP 7, PHP 8)

array_push — Push one or more elements onto the end of array

Description

function **array_push**(array `&$array`, mixed `...$values`): int

array_push() treats `array` as a stack, and pushes the passed variables onto the end of `array`. The length of `array` increases by the number of variables pushed. Has the same effect as:

```
<?php
$array[] = $var;
?>
```

repeated for each passed value.

> **Note** :
>
> If you use **array_push()** to add one element to the array, it's better to use `$array[] =` because in that way there is no overhead of calling a function.

> **Note** :
>
> **array_push()** will raise a warning if the first argument is not an array. This differed from the `$var[]` behaviour where a new array was created, prior to PHP 7.1.0.

Parameters

`array`

The input array.

`values`

The values to push onto the end of the `array`.

Return Values

Returns the new number of elements in the array.

Changelog

Version	Description
7.3.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_push() example

```
<?php
$stack = array("orange", "banana");
array_push($stack, "apple", "raspberry");
print_r($stack);
?>
```

The above example will output:

```
Array
(
    [0] => orange
    [1] => banana
    [2] => apple
    [3] => raspberry
)
```

See Also

- array_pop() - Pop the element off the end of array
- array_shift() - Shift an element off the beginning of array
- array_unshift() - Prepend one or more elements to the beginning of an array

array_rand

array_rand

(PHP 4, PHP 5, PHP 7, PHP 8)

array_rand — Pick one or more random keys out of an array

Description

function **array_rand**(array `$array`, int `$num` = 1): int|string|array

Picks one or more random entries out of an array, and returns the key (or keys) of the random entries.

> **Caution**
>
> This function does not generate cryptographically secure values, and *must not* be used for cryptographic purposes, or purposes that require returned values to be unguessable.
>
> If cryptographically secure randomness is required, the Random\Randomizer may be used with the Random\Engine\Secure engine. For simple use cases, the random_int() and random_bytes() functions provide a convenient and secure API that is backed by the operating system's CSPRNG.

> **Caution**
>
> This function uses the global Mt19937 ("Mersenne Twister") instance as the source of randomness and thus shares its state with all other functions using the global Mt19937. Using any of these functions advances the sequence for *all* the other functions, regardless of scope.
>
> Generating repeatable sequences by seeding mt_srand() or srand() with a known value will also yield repeatable output from this function.
>
> Prefer using Random\Randomizer methods in all newly written code.

Parameters

`array`

The input array. Cannot be empty.

`num`

Specifies how many entries should be picked. Must be greater than zero, and less than or equal to the length of `array`

Return Values

When picking only one entry, **array_rand()** returns the key for a random entry. Otherwise, an array of keys for the random entries is returned. This is done so that random keys can be picked from the array as well as random values. If multiple keys are returned, they will be returned in the order they were present in the original array.

Errors/Exceptions

Throws a ValueError if `array` is empty, or if `num` is out of range.

Changelog

Version	Description
8.0.0	**array_rand()** now throws a ValueError if `num` is out of range; previously an `E_WARNING` was raised, and the function returned `null`.
8.0.0	**array_rand()** now throws a ValueError if `array` is empty; previously an `E_WARNING` was raised, and the function returned `null`.
7.1.0	The internal randomization algorithm has been changed to use the » Mersenne Twister Random Number Generator instead of the libc rand function.

Examples

Example #1 array_rand() example

```php
<?php
$input = array("Neo", "Morpheus", "Trinity", "Cypher", "Tank");
$rand_keys = array_rand($input, 2);
echo $input[$rand_keys[0]] . "\n";
echo $input[$rand_keys[1]] . "\n";
?>
```

See Also

- Random\Randomizer::pickArrayKeys() - Select random array keys
- Random\Randomizer::shuffleArray() - Get a permutation of an array

array_reduce

array_reduce

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

array_reduce — Iteratively reduce the array to a single value using a callback function

Description

function **array_reduce**(array `$array`, callable `$callback`, mixed `$initial` = `null`): mixed

array_reduce() applies iteratively the `callback` function to the elements of the `array`, so as to reduce the array to a single value.

Parameters

`array`

The input array.

`callback`

function callback(mixed `$carry`, mixed `$item`): mixed

`carry`

Holds the return value of the previous iteration; in the case of the first iteration it instead holds the value of `initial`.

`item`

Holds the value of the current iteration.

`initial`

If the optional `initial` is available, it will be used at the beginning of the process, or as a final result in case the array is empty.

Return Values

Returns the resulting value.

If the array is empty and `initial` is not passed, **array_reduce()** returns `null`.

Changelog

Version	Description
8.0.0	If `callback` expects a parameter to be passed by reference, this function will now emit an `E_WARNING`.

Examples

Example #1 array_reduce() example

```php
<?php
function sum($carry, $item)
{
    $carry += $item;
    return $carry;
}

function product($carry, $item)
{
    $carry *= $item;
    return $carry;
}

$a = array(1, 2, 3, 4, 5);
$x = array();

var_dump(array_reduce($a, "sum")); // int(15)
var_dump(array_reduce($a, "product", 10)); // int(1200), because: 10*1*2*3*4*5
var_dump(array_reduce($x, "sum", "No data to reduce")); // string(17) "No data to reduce"
?>
```

See Also

- array_filter() - Filters elements of an array using a callback function
- array_map() - Applies the callback to the elements of the given arrays
- array_unique() - Removes duplicate values from an array
- array_count_values() - Counts the occurrences of each distinct value in an array

array_replace

array_replace

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

array_replace — Replaces elements from passed arrays into the first array

Description

function **array_replace**(array `$array`, array `...$replacements`): array

array_replace() creates a new array and assigns items into it for each key in each of the provided arrays. If a key appears in multiple input arrays, the value from the right-most input array will be used.

array_replace() does not process elements items recursively, it replaces the entire value for each key when it does a replacement.

Parameters

`array`

The array in which elements are replaced.

`replacements`

Arrays from which elements will be extracted. Values from later arrays overwrite the previous values.

Return Values

Returns an array.

Examples

Example #1 array_replace() example

```php
<?php
$base = array("orange", "banana", "apple", "raspberry");
$replacements = array(0 => "pineapple", 4 => "cherry");
$replacements2 = array(0 => "grape");

$basket = array_replace($base, $replacements, $replacements2);
var_dump($basket);
?>
```

The above example will output:

```
array(5) {
  [0]=>
  string(5) "grape"
  [1]=>
  string(6) "banana"
  [2]=>
  string(5) "apple"
  [3]=>
  string(9) "raspberry"
  [4]=>
  string(6) "cherry"
}
```

Example #2 Example of how nested arrays are handled

```php
<?php
$base = [ 'citrus' => [ 'orange', 'lemon' ], 'pome' => [ 'apple' ] ];
$replacements = [ 'citrus' => [ 'grapefruit' ] ];
$replacements2 = [ 'citrus' => [ 'kumquat', 'citron' ], 'pome' => [ 'loquat' ] ];

$basket = array_replace($base, $replacements, $replacements2);
var_dump($basket);
?>
```

The above example will output:

```
array(2) {
  ["citrus"]=>
  array(2) {
    [0]=>
    string(7) "kumquat"
    [1]=>
    string(6) "citron"
  }
  ["pome"]=>
  array(1) {
    [0]=>
    string(6) "loquat"
  }
}
```

See Also

- array_replace_recursive() - Replaces elements from passed arrays into the first array recursively
- array_merge() - Merge one or more arrays

array_replace_recursive

array_replace_recursive

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

array_replace_recursive — Replaces elements from passed arrays into the first array recursively

Description

function **array_replace_recursive**(array `$array`, array `...$replacements`): array

array_replace_recursive() replaces the values of `array` with the same values from all the following arrays. If a key from the first array exists in the second array, its value will be replaced by the value from the second array. If the key exists in the second array, and not the first, it will be created in the first array. If a key only exists in the first array, it will be left as is. If several arrays are passed for replacement, they will be processed in order, the later array overwriting the previous values.

array_replace_recursive() is recursive : it will recurse into arrays and apply the same process to the inner value.

When the value in the first array is scalar, it will be replaced by the value in the second array, may it be scalar or array. When the value in the first array and the second array are both arrays, **array_replace_recursive()** will replace their respective value recursively.

Parameters

`array`

The array in which elements are replaced.

`replacements`

Arrays from which elements will be extracted.

Return Values

Returns an array.

Examples

Example #1 array_replace_recursive() example

```php
<?php
$base = array('citrus' => array("orange"), 'berries' => array("blackberry", "raspberry"));
$replacements = array('citrus' => array('pineapple'), 'berries' => array('blueberry'));

$basket = array_replace_recursive($base, $replacements);
print_r($basket);

$basket = array_replace($base, $replacements);
print_r($basket);
?>
```

The above example will output:

```
Array
(
    [citrus] => Array
        (
            [0] => pineapple
        )

    [berries] => Array
        (
            [0] => blueberry
```

```
            [1] => raspberry
        )

)
Array
(
    [citrus] => Array
        (
            [0] => pineapple
        )

    [berries] => Array
        (
            [0] => blueberry
        )

)
```

Example #2 array_replace_recursive() and recursive behavior

```php
<?php
$base = array('citrus' => array("orange") , 'berries' => array("blackberry", "raspberry"), 'others' => 'banana' );
$replacements = array('citrus' => 'pineapple', 'berries' => array('blueberry'), 'others' => array('litchis'));
$replacements2 = array('citrus' => array('pineapple'), 'berries' => array('blueberry'), 'others' => 'litchis');

$basket = array_replace_recursive($base, $replacements, $replacements2);
print_r($basket);

?>
```

The above example will output:

```
Array
(
    [citrus] => Array
        (
            [0] => pineapple
        )

    [berries] => Array
        (
            [0] => blueberry
            [1] => raspberry
        )

    [others] => litchis
)
```

See Also

- array_replace() - Replaces elements from passed arrays into the first array
- array_merge_recursive() - Merge one or more arrays recursively

array_reverse

array_reverse

(PHP 4, PHP 5, PHP 7, PHP 8)

array_reverse — Return an array with elements in reverse order

Description

function **array_reverse**(array `$array`, bool `$preserve_keys = false`): array

Takes an input `array` and returns a new array with the order of the elements reversed.

Parameters

`array`

The input array.

`preserve_keys`

If set to `true` numeric keys are preserved. Non-numeric keys are not affected by this setting and will always be preserved.

Return Values

Returns the reversed array.

Examples

Example #1 array_reverse() example

```php
<?php
$input  = array("php", 4.0, array("green", "red"));
$reversed = array_reverse($input);
$preserved = array_reverse($input, true);

print_r($input);
print_r($reversed);
print_r($preserved);
?>
```

The above example will output:

```
Array
(
    [0] => php
    [1] => 4
    [2] => Array
        (
            [0] => green
            [1] => red
        )

)
Array
(
    [0] => Array
        (
            [0] => green
            [1] => red
        )

    [1] => 4
    [2] => php
)
Array
(
    [2] => Array
        (
            [0] => green
            [1] => red
        )

    [1] => 4
    [0] => php
)
```

See Also

- array_flip() - Exchanges all keys with their associated values in an array

array_search

array_search

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

array_search — Searches the array for a given value and returns the first corresponding key if successful

Description

function **array_search**(mixed `$needle`, array `$haystack`, bool `$strict = false`): int|string|false

Searches for `needle` in `haystack`.

Parameters

`needle`

The searched value.

> **Note** :
>
> If `needle` is a string, the comparison is done in a case-sensitive manner.

`haystack`

The array.

`strict`

If the third parameter `strict` is set to `true` then the **array_search()** function will search for *identical* elements in the `haystack`. This means it will also perform a strict type comparison of the `needle` in the `haystack`, and objects must be the same instance.

Return Values

Returns the key for `needle` if it is found in the array, `false` otherwise.

If `needle` is found in `haystack` more than once, the first matching key is returned. To return the keys for all matching values, use array_keys() with the optional `filter_value` parameter instead.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Examples

Example #1 array_search() example

```php
<?php
$array = array(0 => 'blue', 1 => 'red', 2 => 'green', 3 => 'red');

$key = array_search('green', $array); // $key = 2;
print_r($key);

$key = array_search('red', $array);   // $key = 1;
print_r($key);
?>
```

See Also

- array_keys() - Return all the keys or a subset of the keys of an array
- array_values() - Return all the values of an array
- array_key_exists() - Checks if the given key or index exists in the array
- in_array() - Checks if a value exists in an array

array_shift

array_shift

(PHP 4, PHP 5, PHP 7, PHP 8)

array_shift — Shift an element off the beginning of array

Description

function **array_shift**(array `&$array`): mixed

array_shift() shifts the first value of the `array` off and returns it, shortening the `array` by one element and moving everything down. All numerical array keys will be modified to start counting from zero while literal keys won't be affected.

> **Note** :
>
> This function will reset() the array pointer of the input array after use.

Parameters

`array`

The input array.

Return Values

Returns the shifted value, or `null` if `array` is empty or is not an array.

Examples

Example #1 array_shift() example

```php
<?php
$stack = array("orange", "banana", "apple", "raspberry");
$fruit = array_shift($stack);
print_r($stack);
?>
```

The above example will output:

```
Array
(
    [0] => banana
    [1] => apple
    [2] => raspberry
)
```

and `orange` will be assigned to *$fruit*.

See Also

- array_unshift() - Prepend one or more elements to the beginning of an array
- array_push() - Push one or more elements onto the end of array
- array_pop() - Pop the element off the end of array

array_slice

array_slice

(PHP 4, PHP 5, PHP 7, PHP 8)

array_slice — Extract a slice of the array

Description

```
function array_slice(
    array $array,
    int $offset,
    ?int $length = null,
    bool $preserve_keys = false
): array
```

array_slice() returns the sequence of elements from the array `array` as specified by the `offset` and `length` parameters.

Parameters

`array`

The input array.

`offset`

If `offset` is non-negative, the sequence will start at that offset in the `array`.

If `offset` is negative, the sequence will start that far from the end of the `array`.

> **Note** :
>
> The `offset` parameter denotes the position in the array, not the key.

`length`

If `length` is given and is positive, then the sequence will have up to that many elements in it.

If the array is shorter than the `length`, then only the available array elements will be present.

If `length` is given and is negative then the sequence will stop that many elements from the end of the array.

If it is omitted, then the sequence will have everything from `offset` up until the end of the `array`.

`preserve_keys`

> **Note** :
>
> **array_slice()** will reorder and reset the integer array indices by default. This behaviour can be changed by setting `preserve_keys` to `true`. String keys are always preserved, regardless of this parameter.

Return Values

Returns the slice. If the offset is larger than the size of the array, an empty array is returned.

Examples

Example #1 array_slice() examples

```php
<?php
$input = array("a", "b", "c", "d", "e");

$output = array_slice($input, 2);      // returns "c", "d", and "e"
$output = array_slice($input, -2, 1);  // returns "d"
$output = array_slice($input, 0, 3);   // returns "a", "b", and "c"

// note the differences in the array keys
print_r(array_slice($input, 2, -1));
```

```php
print_r(array_slice($input, 2, -1, true));
?>
```

The above example will output:

```
Array
(
    [0] => c
    [1] => d
)
Array
(
    [2] => c
    [3] => d
)
```

Example #2 array_slice() and one-based array

```php
<?php
$input = array(1 => "a", "b", "c", "d", "e");
print_r(array_slice($input, 1, 2));
?>
```

The above example will output:

```
Array
(
    [0] => b
    [1] => c
)
```

Example #3 array_slice() and array with mixed keys

```php
<?php
$ar = array('a'=>'apple', 'b'=>'banana', '42'=>'pear', 'd'=>'orange');
print_r(array_slice($ar, 0, 3));
print_r(array_slice($ar, 0, 3, true));
?>
```

The above example will output:

```
Array
(
    [a] => apple
    [b] => banana
    [0] => pear
)
Array
(
    [a] => apple
    [b] => banana
    [42] => pear
)
```

See Also

- array_chunk() - Split an array into chunks
- array_splice() - Remove a portion of the array and replace it with something else
- unset() - unset a given variable

array_splice

array_splice

(PHP 4, PHP 5, PHP 7, PHP 8)

array_splice — Remove a portion of the array and replace it with something else

Description

```
function array_splice(
    array &$array,
    int $offset,
    ?int $length = null,
    mixed $replacement = []
): array
```

Removes the elements designated by `offset` and `length` from the `array` array, and replaces them with the elements of the `replacement` array, if supplied.

> **Note** :
>
> Numerical keys in `array` are not preserved.

> **Note** :
>
> If `replacement` is not an array, it will be typecast to one (i.e. `(array) $replacement`). This may result in unexpected behavior when using an object or `null` `replacement`.

Parameters

`array`

The input array.

`offset`

If `offset` is positive then the start of the removed portion is at that offset from the beginning of the `array` array.

If `offset` is negative then the start of the removed portion is at that offset from the end of the `array` array.

`length`

If `length` is omitted, removes everything from `offset` to the end of the array.

If `length` is specified and is positive, then that many elements will be removed.

If `length` is specified and is negative, then the end of the removed portion will be that many elements from the end of the array.

If `length` is specified and is zero, no elements will be removed.

> **Tip**
>
> To remove everything from `offset` to the end of the array when `replacement` is also specified, use `count($input)` for `length`.

`replacement`

If `replacement` array is specified, then the removed elements are replaced with elements from this array.

If `offset` and `length` are such that nothing is removed, then the elements from the `replacement` array are inserted in the place specified by the `offset`.

> **Note** :
>
> Keys in the `replacement` array are not preserved.

If `replacement` is just one element it is not necessary to put `array()` or square brackets around it, unless the element is an array itself, an object or `null`.

Return Values

Returns an array consisting of the extracted elements.

Changelog

Version	Description
8.0.0	`length` is nullable now.

Examples

Example #1 array_splice() examples

```
<?php
$input = array("red", "green", "blue", "yellow");
array_splice($input, 2);
var_dump($input);

$input = array("red", "green", "blue", "yellow");
array_splice($input, 1, -1);
var_dump($input);

$input = array("red", "green", "blue", "yellow");
array_splice($input, 1, count($input), "orange");
var_dump($input);

$input = array("red", "green", "blue", "yellow");
array_splice($input, -1, 1, array("black", "maroon"));
var_dump($input);
?>
```

The above example will output:

```
array(2) {
  [0]=>
  string(3) "red"
  [1]=>
  string(5) "green"
}
array(2) {
  [0]=>
  string(3) "red"
  [1]=>
  string(6) "yellow"
}
array(2) {
  [0]=>
  string(3) "red"
  [1]=>
  string(6) "orange"
}
array(5) {
  [0]=>
  string(3) "red"
  [1]=>
  string(5) "green"
```

```
  [2]=>
  string(4) "blue"
  [3]=>
  string(5) "black"
  [4]=>
  string(6) "maroon"
}
```

Example #2 Equivalent statements to various array_splice() examples

The following statements are equivalent:

```php
<?php

// append two elements to $input
array_push($input, $x, $y);
array_splice($input, count($input), 0, array($x, $y));

// remove the last element of $input
array_pop($input);
array_splice($input, -1);

// remove the first element of $input
array_shift($input);
array_splice($input, 0, 1);

// insert an element at the start of $input
array_unshift($input, $x, $y);
array_splice($input, 0, 0, array($x, $y));

// replace the value in $input at index $x
$input[$x] = $y; // for arrays where key equals offset
array_splice($input, $x, 1, $y);

?>
```

See Also

- array_merge() - Merge one or more arrays
- array_slice() - Extract a slice of the array
- unset() - unset a given variable

array_sum

array_sum

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

array_sum — Calculate the sum of values in an array

Description

function **array_sum**(array `$array`): int|float

array_sum() returns the sum of values in an array.

Parameters

`array`

The input array.

Return Values

Returns the sum of values as an integer or float; `0` if the `array` is empty.

Changelog

Version	Description
8.3.0	Now emits `E_WARNING` when `array` values cannot be converted to int or float. Previously arrays and objects where ignored whilst every other value was cast to int. Moreover, objects that define a numeric cast (e.g. GMP) are now cast instead of ignored.

Examples

Example #1 array_sum() examples

```
<?php
$a = array(2, 4, 6, 8);
echo "sum(a) = " . array_sum($a) . "\n";

$b = array("a" => 1.2, "b" => 2.3, "c" => 3.4);
echo "sum(b) = " . array_sum($b) . "\n";
?>
```

The above example will output:

```
sum(a) = 20
sum(b) = 6.9
```

array_udiff

array_udiff

(PHP 5, PHP 7, PHP 8)

array_udiff — Computes the difference of arrays by using a callback function for data comparison

Description

function **array_udiff**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computes the difference of arrays by using a callback function for data comparison. This is unlike array_diff() which uses an internal function for comparing the data.

Parameters

`array`

The first array.

`arrays`

Arrays to compare against.

`value_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

> **Caution**
>
> The sorting callback must handle any value from any array in any order, regardless of the order they were originally provided. This is because each individual array is first sorted before being compared against other arrays. For example:

```php
<?php
$arrayA = ["string", 1];
$arrayB = [["value" => 1]];
// $item1 and $item2 can be any of "string", 1 or ["value" => 1]
$compareFunc = static function ($item1, $item2) {
    $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
    $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
    return $value1 <=> $value2;
};
?>
```

Return Values

Returns an array containing all the values of `array` that are not present in any of the other arguments.

Examples

Example #1 array_udiff() example using stdClass Objects

```php
<?php
// Arrays to compare
$array1 = array(new stdClass, new stdClass,
                new stdClass, new stdClass,
               );

$array2 = array(
                new stdClass, new stdClass,
               );

// Set some properties for each object
$array1[0]->width = 11; $array1[0]->height = 3;
$array1[1]->width = 7;  $array1[1]->height = 1;
$array1[2]->width = 2;  $array1[2]->height = 9;
$array1[3]->width = 5;  $array1[3]->height = 7;

$array2[0]->width = 7;  $array2[0]->height = 5;
$array2[1]->width = 9;  $array2[1]->height = 2;

function compare_by_area($a, $b) {
    $areaA = $a->width * $a->height;
    $areaB = $b->width * $b->height;

    if ($areaA < $areaB) {
        return -1;
    } elseif ($areaA > $areaB) {
        return 1;
    } else {
        return 0;
    }
}

print_r(array_udiff($array1, $array2, 'compare_by_area'));
?>
```

The above example will output:

```
Array
(
    [0] => stdClass Object
        (
            [width] => 11
            [height] => 3
        )

    [1] => stdClass Object
        (
            [width] => 7
            [height] => 1
```

```
        )

)
```

Example #2 array_udiff() example using DateTime Objects

```php
<?php
class MyCalendar {
    public $free = array();
    public $booked = array();

    public function __construct($week = 'now') {
        $start = new DateTime($week);
        $start->modify('Monday this week midnight');
        $end = clone $start;
        $end->modify('Friday this week midnight');
        $interval = new DateInterval('P1D');
        foreach (new DatePeriod($start, $interval, $end) as $freeTime) {
            $this->free[] = $freeTime;
        }
    }

    public function bookAppointment(DateTime $date, $note) {
        $this->booked[] = array('date' => $date->modify('midnight'), 'note' => $note);
    }

    public function checkAvailability() {
        return array_udiff($this->free, $this->booked, array($this, 'customCompare'));
    }

    public function customCompare($free, $booked) {
        if (is_array($free)) $a = $free['date'];
        else $a = $free;
        if (is_array($booked)) $b = $booked['date'];
        else $b = $booked;
        if ($a == $b) {
            return 0;
        } elseif ($a > $b) {
            return 1;
        } else {
            return -1;
        }
    }
}

// Create a calendar for weekly appointments
$myCalendar = new MyCalendar;

// Book some appointments for this week
$myCalendar->bookAppointment(new DateTime('Monday this week'), "Cleaning GoogleGuy's apartment.");
$myCalendar->bookAppointment(new DateTime('Wednesday this week'), "Going on a snowboarding trip.");
$myCalendar->bookAppointment(new DateTime('Friday this week'), "Fixing buggy code.");

// Check availability of days by comparing $booked dates against $free dates
echo "I'm available on the following days this week...\n\n";
foreach ($myCalendar->checkAvailability() as $free) {
    echo $free->format('l'), "\n";
}
echo "\n\n";
echo "I'm busy on the following days this week...\n\n";
foreach ($myCalendar->booked as $booked) {
    echo $booked['date']->format('l'), ": ", $booked['note'], "\n";
}
?>
```

The above example will output:

```
I'm available on the following days this week...

Tuesday
Thursday


I'm busy on the following days this week...

Monday: Cleaning GoogleGuy's apartment.
Wednesday: Going on a snowboarding trip.
Friday: Fixing buggy code.
```

Notes

> **Note** :
>
> Please note that this function only checks one dimension of a n-dimensional array. Of course you can check deeper dimensions by using `array_udiff($array1[0], $array2[0], "data_compare_func");`.

See Also

- array_diff() - Computes the difference of arrays
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_diff_uassoc() - Computes the difference of arrays with additional index check which is performed by a user supplied callback function
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_uintersect() - Computes the intersection of arrays, compares data by a callback function
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function
- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions

array_udiff_assoc

array_udiff_assoc

(PHP 5, PHP 7, PHP 8)

array_udiff_assoc — Computes the difference of arrays with additional index check, compares data by a callback function

Description

function **array_udiff_assoc**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computes the difference of arrays with additional index check, compares data by a callback function.

> **Note** :
>
> Please note that this function only checks one dimension of a n-dimensional array. Of course you can check deeper dimensions by using, for example, `array_udiff_assoc($array1[0], $array2[0], "some_comparison_func");`.

Parameters

`array`

The first array.

`arrays`

Arrays to compare against.

`value_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

> **Caution**
>
> The sorting callback must handle any value from any array in any order, regardless of the order they were originally provided. This is because each individual array is first sorted before being compared against other arrays. For example:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 and $item2 can be any of "string", 1 or ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

Return Values

array_udiff_assoc() returns an array containing all the values from `array` that are not present in any of the other arguments. Note that the keys are used in the comparison unlike array_diff() and array_udiff(). The comparison of arrays' data is performed by using an user-supplied callback. In this aspect the behaviour is opposite to the behaviour of array_diff_assoc() which uses internal function for comparison.

Examples

Example #1 **array_udiff_assoc()** example

```php
<?php
class cr {
    private $priv_member;
    function __construct($val)
    {
        $this->priv_member = $val;
    }

    static function comp_func_cr($a, $b)
    {
        if ($a->priv_member === $b->priv_member) return 0;
        return ($a->priv_member > $b->priv_member)? 1:-1;
    }
}

$a = array("0.1" => new cr(9), "0.5" => new cr(12), 0 => new cr(23), 1=> new cr(4), 2 => new cr(-15),);
$b = array("0.2" => new cr(9), "0.5" => new cr(22), 0 => new cr(3), 1=> new cr(4), 2 => new cr(-15),);

$result = array_udiff_assoc($a, $b, array("cr", "comp_func_cr"));
print_r($result);
?>
```

The above example will output:

```
```

```
Array
(
    [0.1] => cr Object
        (
            [priv_member:cr:private] => 9
        )

    [0.5] => cr Object
        (
            [priv_member:cr:private] => 12
        )

    [0] => cr Object
        (
            [priv_member:cr:private] => 23
        )

)
```

In our example above you see the `"1" => new cr(4)` pair is present in both arrays and thus it is not in the output from the function.

See Also

- array_diff() - Computes the difference of arrays
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_diff_uassoc() - Computes the difference of arrays with additional index check which is performed by a user supplied callback function
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_udiff_uassoc() - Computes the difference of arrays with additional index check, compares data and indexes by a callback function
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_uintersect() - Computes the intersection of arrays, compares data by a callback function
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function
- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions

array_udiff_uassoc

array_udiff_uassoc

(PHP 5, PHP 7, PHP 8)

array_udiff_uassoc — Computes the difference of arrays with additional index check, compares data and indexes by a callback function

Description

function **array_udiff_uassoc**(
  array `$array`,
  array `...$arrays`,
  callable `$value_compare_func`,
  callable `$key_compare_func`
): array

Computes the difference of arrays with additional index check, compares data and indexes by a callback function.

Note that the keys are used in the comparison unlike array_diff() and array_udiff().

Parameters

`array`

The first array.

`arrays`

Arrays to compare against.

`value_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

> **Caution**
>
> The sorting callback must handle any value from any array in any order, regardless of the order they were originally provided. This is because each individual array is first sorted before being compared against other arrays. For example:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 and $item2 can be any of "string", 1 or ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

`key_compare_func`

The comparison of keys (indices) is done also by the callback function `key_compare_func`. This behaviour is unlike what array_udiff_assoc() does, since the latter compares the indices by using an internal function.

Return Values

Returns an array containing all the values from `array` that are not present in any of the other arguments.

Examples

Example #1 array_udiff_uassoc() example

```php
<?php
class cr {
    private $priv_member;
    function __construct($val)
    {
        $this->priv_member = $val;
    }

    static function comp_func_cr($a, $b)
    {
        if ($a->priv_member === $b->priv_member) return 0;
        return ($a->priv_member > $b->priv_member)? 1:-1;
    }

    static function comp_func_key($a, $b)
    {
        if ($a === $b) return 0;
        return ($a > $b)? 1:-1;
    }
}
$a = array("0.1" => new cr(9), "0.5" => new cr(12), 0 => new cr(23), 1=> new cr(4), 2 => new cr(-15),);
$b = array("0.2" => new cr(9), "0.5" => new cr(22), 0 => new cr(3), 1=> new cr(4), 2 => new cr(-15),);

$result = array_udiff_uassoc($a, $b, array("cr", "comp_func_cr"), array("cr", "comp_func_key"));
print_r($result);
?>
```

The above example will output:

```
Array
(
    [0.1] => cr Object
        (
            [priv_member:cr:private] => 9
        )

    [0.5] => cr Object
        (
            [priv_member:cr:private] => 12
        )

    [0] => cr Object
        (
            [priv_member:cr:private] => 23
        )
)
```

In our example above you see the `"1" => new cr(4)` pair is present in both arrays and thus it is not in the output from the function. Keep in mind that you have to supply 2 callback functions.

Notes

> **Note** :
>
> Please note that this function only checks one dimension of a n-dimensional array. Of course you can check deeper dimensions by using, for example, `array_udiff_uassoc($array1[0], $array2[0], "data_compare_func", "key_compare_func");`.

See Also

- array_diff() - Computes the difference of arrays
- array_diff_assoc() - Computes the difference of arrays with additional index check
- array_udiff() - Computes the difference of arrays by using a callback function for data comparison
- array_udiff_assoc() - Computes the difference of arrays with additional index check, compares data by a callback function
- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_uintersect() - Computes the intersection of arrays, compares data by a callback function
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function
- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions

array_uintersect

array_uintersect

(PHP 5, PHP 7, PHP 8)

array_uintersect — Computes the intersection of arrays, compares data by a callback function

Description

function **array_uintersect**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computes the intersection of arrays, compares data by a callback function.

Parameters

`array`

The first array.

`arrays`

Arrays to compare against.

`value_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

> **Caution**
>
> The sorting callback must handle any value from any array in any order, regardless of the order they were originally provided. This is because each individual array is first sorted before being compared against other arrays. For example:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 and $item2 can be any of "string", 1 or ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

Return Values

Returns an array containing all the values of `array` that are present in all the arguments.

Examples

Example #1 **array_uintersect()** example

```php
<?php
$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "GREEN", "B" => "brown", "yellow", "red");

print_r(array_uintersect($array1, $array2, "strcasecmp"));
?>
```

The above example will output:

```
Array
(
    [a] => green
    [b] => brown
    [0] => red
)
```

See Also

- array_intersect() - Computes the intersection of arrays
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function

- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions

array_uintersect_assoc

array_uintersect_assoc

(PHP 5, PHP 7, PHP 8)

array_uintersect_assoc — Computes the intersection of arrays with additional index check, compares data by a callback function

Description

function **array_uintersect_assoc**(array `$array`, array `...$arrays`, callable `$value_compare_func`): array

Computes the intersection of arrays with additional index check, compares data by a callback function.

Note that the keys are used in the comparison unlike in array_uintersect(). The data is compared by using a callback function.

Parameters

`array`

The first array.

`arrays`

Arrays to compare against.

`value_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

> **Caution**
>
> The sorting callback must handle any value from any array in any order, regardless of the order they were originally provided. This is because each individual array is first sorted before being compared against other arrays. For example:
>
> ```php
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 and $item2 can be any of "string", 1 or ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

Return Values

Returns an array containing all the values of `array` that are present in all the arguments.

Examples

Example #1 array_uintersect_assoc() example

```php
<?php
$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "GREEN", "B" => "brown", "yellow", "red");
```

```
print_r(array_uintersect_assoc($array1, $array2, "strcasecmp"));
?>
```

The above example will output:

```
Array
(
    [a] => green
)
```

See Also

- array_uintersect() - Computes the intersection of arrays, compares data by a callback function
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_intersect_uassoc() - Computes the intersection of arrays with additional index check, compares indexes by a callback function
- array_uintersect_uassoc() - Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions

array_uintersect_uassoc

array_uintersect_uassoc

(PHP 5, PHP 7, PHP 8)

array_uintersect_uassoc — Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions

Description

function **array_uintersect_uassoc**(
  array `$array1`,
  array `...$arrays`,
  callable `$value_compare_func`,
  callable `$key_compare_func`
): array

Computes the intersection of arrays with additional index check, compares data and indexes by separate callback functions.

Parameters

`array1`

The first array.

`arrays`

Further arrays.

`value_compare_func`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

> **Caution**

> The sorting callback must handle any value from any array in any order, regardless of the order they were originally provided. This is because each individual array is first sorted before being compared against other arrays. For example:
>
> ```
> <?php
> $arrayA = ["string", 1];
> $arrayB = [["value" => 1]];
> // $item1 and $item2 can be any of "string", 1 or ["value" => 1]
> $compareFunc = static function ($item1, $item2) {
>     $value1 = is_string($item1) ? strlen($item1) : (is_array($item1) ? $item1["value"] : $item1);
>     $value2 = is_string($item2) ? strlen($item2) : (is_array($item2) ? $item2["value"] : $item2);
>     return $value1 <=> $value2;
> };
> ?>
> ```

`key_compare_func`

Key comparison callback function.

Return Values

Returns an array containing all the values of `array1` that are present in all the arguments.

Examples

Example #1 array_uintersect_uassoc() example

```
<?php
$array1 = array("a" => "green", "b" => "brown", "c" => "blue", "red");
$array2 = array("a" => "GREEN", "B" => "brown", "yellow", "red");

print_r(array_uintersect_uassoc($array1, $array2, "strcasecmp", "strcasecmp"));
?>
```

The above example will output:

```
Array
(
    [a] => green
    [b] => brown
)
```

See Also

- array_uintersect() - Computes the intersection of arrays, compares data by a callback function
- array_intersect_assoc() - Computes the intersection of arrays with additional index check
- array_intersect_uassoc() - Computes the intersection of arrays with additional index check, compares indexes by a callback function
- array_uintersect_assoc() - Computes the intersection of arrays with additional index check, compares data by a callback function

array_unique

array_unique

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

array_unique — Removes duplicate values from an array

Description

function **array_unique**(array `$array`, int `$flags` = **`SORT_STRING`**): array

Takes an input `array` and returns a new array without duplicate values.

Note that keys are preserved. If multiple elements compare equal under the given `flags`, then the key and value of the first equal element will be retained.

> **Note** :
>
> Two elements are considered equal if and only if `(string) $elem1 === (string) $elem2`, i.e. when the string representation is the same, the first element will be used.

Parameters

`array`

The input array.

`flags`

The optional second parameter `flags` may be used to modify the comparison behavior using these values:

Comparison type flags:

- `SORT_REGULAR` - compare items normally (don't change types)

  > **Note** :
  >
  > When combining `SORT_REGULAR` with arrays mixing numeric and non-numeric strings, the sort-then-compare algorithm may not place equal elements adjacent, which can leave duplicates in the result. Use `SORT_STRING` to avoid this.

- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings
- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale.

Return Values

Returns the filtered array.

Changelog

Version	Description
7.2.0	If `flags` is `SORT_STRING`, the returned array is built by adding the unique elements to a new array; formerly, it was a copy of `array` from which the non-unique elements were removed. Both hold the same elements under the same keys, but if the element with the largest integer key is among those removed, the index generated for an element appended to the returned array differs from the one `array` would have generated.

Examples

Example #1 array_unique() example

```php
<?php

$input = ["a" => "green", "red", "b" => "green", "blue", "red"];
$result = array_unique($input);
print_r($result);

?>
```

The above example will output:

```
Array
(
    [a] => green
    [0] => red
    [1] => blue
)
```

Example #2 array_unique() and types

```php
<?php

$input = [4, "4", "3", 4, 3, "3"];
$result = array_unique($input);
var_dump($result);

?>
```

The above example will output:

```
array(2) {
  [0]=>
  int(4)
  [2]=>
  string(1) "3"
}
```

Notes

> **Note** :
>
> Note that **array_unique()** is not intended to work on multi dimensional arrays.

See Also

- array_count_values() - Counts the occurrences of each distinct value in an array

array_unshift

array_unshift

(PHP 4, PHP 5, PHP 7, PHP 8)

array_unshift — Prepend one or more elements to the beginning of an array

Description

function **array_unshift**(array `&$array`, mixed `...$values`): int

array_unshift() prepends passed elements to the front of the `array`. Note that the list of elements is prepended as a whole, so that the prepended elements stay in the same order. All numerical array keys will be modified to start counting from zero while literal keys won't be changed.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`values`

The values to prepend.

Return Values

Returns the new number of elements in the `array`.

Changelog

Version	Description
7.3.0	This function can now be called with only one parameter. Formerly, at least two parameters have been required.

Examples

Example #1 array_unshift() example

```php
<?php

$queue = [
    "orange",
    "banana"
];

array_unshift($queue, "apple", "raspberry");

var_dump($queue);

?>
```

The above example will output:

```
array(4) {
  [0]=>
  string(5) "apple"
  [1]=>
  string(9) "raspberry"
  [2]=>
  string(6) "orange"
  [3]=>
  string(6) "banana"
}
```

Example #2 Usage with associative arrays

If one associative array is prepended to another associative array, the prepended array is numerically indexed into the former array.

```php
<?php

$foods = [
    'apples' => [
        'McIntosh' => 'red',
        'Granny Smith' => 'green',
    ],
    'oranges' => [
        'Navel' => 'orange',
        'Valencia' => 'orange',
    ],
];

$vegetables = [
    'lettuce' => [
        'Iceberg' => 'green',
        'Butterhead' => 'green',
    ],
    'carrots' => [
        'Deep Purple Hybrid' => 'purple',
        'Imperator' => 'orange',
```

```
    ],
    'cucumber' => [
        'Kirby' => 'green',
        'Gherkin' => 'green',
    ],
];

array_unshift($foods, $vegetables);

var_dump($foods);

?>
```

The above example will output:

```
array(3) {
  [0]=>
  array(3) {
    ["lettuce"]=>
    array(2) {
      ["Iceberg"]=>
      string(5) "green"
      ["Butterhead"]=>
      string(5) "green"
    }
    ["carrots"]=>
    array(2) {
      ["Deep Purple Hybrid"]=>
      string(6) "purple"
      ["Imperator"]=>
      string(6) "orange"
    }
    ["cucumber"]=>
    array(2) {
      ["Kirby"]=>
      string(5) "green"
      ["Gherkin"]=>
      string(5) "green"
    }
  }
  ["apples"]=>
  array(2) {
    ["McIntosh"]=>
    string(3) "red"
    ["Granny Smith"]=>
    string(5) "green"
  }
  ["oranges"]=>
  array(2) {
    ["Navel"]=>
    string(6) "orange"
    ["Valencia"]=>
    string(6) "orange"
  }
}
```

See Also

- array_merge() - Merge one or more arrays
- array_shift() - Shift an element off the beginning of array
- array_push() - Push one or more elements onto the end of array
- array_pop() - Pop the element off the end of array

array_values

array_values

(PHP 4, PHP 5, PHP 7, PHP 8)

array_values — Return all the values of an array

Description

function **array_values**(array `$array`): array

array_values() returns all the values from the `array` and indexes the array numerically.

Parameters

`array`

The array.

Return Values

Returns an indexed array of values.

Examples

Example #1 array_values() example

```
<?php
$array = array("size" => "XL", "color" => "gold");
print_r(array_values($array));
?>
```

The above example will output:

```
Array
(
    [0] => XL
    [1] => gold
)
```

See Also

- array_keys() - Return all the keys or a subset of the keys of an array
- array_combine() - Creates an array by using one array for keys and another for its values

array_walk

array_walk

(PHP 4, PHP 5, PHP 7, PHP 8)

array_walk — Apply a user supplied function to every member of an array

Description

function **array_walk**(array|object `&$array`, callable `$callback`, mixed `$arg` = `null`): true

Applies the user-defined `callback` function to each element of the `array` array.

array_walk() is not affected by the internal array pointer of `array`. **array_walk()** will walk through the entire array regardless of pointer position.

Parameters

`array`

The input array.

`callback`

Typically, `callback` takes on two parameters. The `array` parameter's value being the first, and the key/index second.

> **Note** :
>
> If `callback` needs to be working with the actual values of the array, specify the first parameter of `callback` as a reference. Then, any changes made to those elements will be made in the original array itself.

> **Note** :
>
> Many internal functions (for example strtolower()) will throw if more than the expected number of arguments are passed in and are not usable directly as a `callback`.

Only the values of the `array` may potentially be changed; its structure cannot be altered, i.e., the programmer cannot add, unset or reorder elements. If the callback does not respect this requirement, the behavior of this function is undefined, and unpredictable.

`arg`

If the optional `arg` parameter is supplied, it will be passed as the third parameter to the `callback`.

Return Values

Always returns `true`.

Errors/Exceptions

As of PHP 7.1.0, an ArgumentCountError will be thrown if the `callback` function requires more than 2 parameters (the value and key of the array member), or more than 3 parameters if the `arg` is also passed. Previously, in this case an error of level E_WARNING would be generated each time **array_walk()** calls `callback`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.
8.0.0	If `callback` expects the second or third parameter to be passed by reference, this function will now emit an `E_WARNING`.

Examples

Example #1 array_walk() example

```php
<?php
$fruits = array("d" => "lemon", "a" => "orange", "b" => "banana", "c" => "apple");

function test_alter(&$item1, $key, $prefix)
{
    $item1 = "$prefix: $item1";
}

function test_print($item2, $key)
{
    echo "$key. $item2\n";
}

echo "Before ...:\n";
array_walk($fruits, 'test_print');

array_walk($fruits, 'test_alter', 'fruit');
echo "... and after:\n";

array_walk($fruits, 'test_print');
?>
```

The above example will output:

```
Before ...:
d. lemon
a. orange
b. banana
c. apple
... and after:
d. fruit: lemon
a. fruit: orange
b. fruit: banana
c. fruit: apple
```

Example #2 array_walk() example using anonymous function

```php
<?php
$elements = ['a', 'b', 'c'];

array_walk($elements, function ($value, $key) {
  echo "{$key} => {$value}\n";
});

?>
```

The above example will output:

```
0 => a
1 => b
2 => c
```

See Also

- array_walk_recursive() - Apply a user function recursively to every member of an array
- iterator_apply() - Call a function for every element in an iterator
- list() - Assign variables as if they were an array
- each() - Return the current key and value pair from an array and advance the array cursor
- call_user_func_array() - Call a callback with an array of parameters
- array_map() - Applies the callback to the elements of the given arrays
- `foreach`

array_walk_recursive

array_walk_recursive

(PHP 5, PHP 7, PHP 8)

array_walk_recursive — Apply a user function recursively to every member of an array

Description

function **array_walk_recursive**(array|object `&$array`, callable `$callback`, mixed `$arg` = `null`): true

Applies the user-defined `callback` function to each element of the `array`. This function will recurse into deeper arrays.

Passing an object as the `array` is allowed, and will be treated as if it had been cast to an array first. Regardless of whether the function started with an array or an object, it will not recurse into inner objects.

Parameters

`array`

The input array.

`callback`

Typically, `callback` takes on two parameters. The `array` parameter's value being the first, and the key/index second.

> **Note** :
>
> If `callback` needs to be working with the actual values of the array, specify the first parameter of `callback` as a reference. Then, any changes made to those elements will be made in the original array itself.

`arg`

If the optional `arg` parameter is supplied, it will be passed as the third parameter to the `callback`.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 array_walk_recursive() example

```php
<?php
$sweet = array('a' => 'apple', 'b' => 'banana');
$fruits = array('sweet' => $sweet, 'sour' => 'lemon');

function test_print($item, $key)
{
    echo "$key holds $item\n";
}

array_walk_recursive($fruits, 'test_print');
?>
```

The above example will output:

```
a holds apple
b holds banana
sour holds lemon
```

You may notice that the key 'sweet' is never displayed. Any key that holds an array will not be passed to the function.

array_walk_recursive() can also be used to update values in place by passing the callback parameter by reference.

Example #2 Updating values in place

```php
<?php
$sweet = ['a' => 'apple', 'b' => 'banana'];
$fruits = ['sweet' => $sweet, 'sour' => 'lemon'];

function test_update(&$item)
{
    $item = strtoupper($item);
}

function test_print($item, $key)
```

```php
{
    echo "$key holds $item\n";
}

array_walk_recursive($fruits, 'test_update');
array_walk_recursive($fruits, 'test_print');
?>
```

The above example will output:

```
a holds APPLE
b holds BANANA
sour holds LEMON
```

See Also

- array_walk() - Apply a user supplied function to every member of an array

arsort

arsort

(PHP 4, PHP 5, PHP 7, PHP 8)

arsort — Sort an array in descending order and maintain index association

Description

function **arsort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Sorts `array` in place in descending order, such that its keys maintain their correlation with the values they are associated with.

This is used mainly when sorting associative arrays where the actual element order is significant.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- `SORT_REGULAR` - compare items normally; the details are described in the comparison operators section
- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings

- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- `SORT_NATURAL` - compare items as strings using "natural ordering" like natsort()
- `SORT_FLAG_CASE` - can be combined (bitwise OR) with `SORT_STRING` or `SORT_NATURAL` to sort strings case-insensitively

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 arsort() example

```php
<?php
$fruits = array("d" => "lemon", "a" => "orange", "b" => "banana", "c" => "apple");
arsort($fruits);
foreach ($fruits as $key => $val) {
    echo "$key = $val\n";
}
?>
```

The above example will output:

```
a = orange
d = lemon
b = banana
c = apple
```

The fruits have been sorted in reverse alphabetical order, and the index associated with each element has been maintained.

See Also

- sort() - Sort an array in ascending order
- asort() - Sort an array in ascending order and maintain index association
- The comparison of array sorting functions

asort

asort

(PHP 4, PHP 5, PHP 7, PHP 8)

asort — Sort an array in ascending order and maintain index association

Description

function **asort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Sorts `array` in place in ascending order, such that its keys maintain their correlation with the values they are associated with.

This is used mainly when sorting associative arrays where the actual element order is significant.

> **Note** :

If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- **SORT_REGULAR** - compare items normally; the details are described in the comparison operators section
- **SORT_NUMERIC** - compare items numerically
- **SORT_STRING** - compare items as strings
- **SORT_LOCALE_STRING** - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- **SORT_NATURAL** - compare items as strings using "natural ordering" like natsort()
- **SORT_FLAG_CASE** - can be combined (bitwise OR) with **SORT_STRING** or **SORT_NATURAL** to sort strings case-insensitively

Return Values

Always returns **true**.

Changelog

Version	Description
8.2.0	The return type is **true** now; previously, it was bool.

Examples

Example #1 asort() example

```php
<?php
$fruits = array("d" => "lemon", "a" => "orange", "b" => "banana", "c" => "apple");
asort($fruits);
foreach ($fruits as $key => $val) {
    echo "$key = $val\n";
}
?>
```

The above example will output:

```
c = apple
b = banana
d = lemon
a = orange
```

The fruits have been sorted in alphabetical order, and the index associated with each element has been maintained.

See Also

- sort() - Sort an array in ascending order
- arsort() - Sort an array in descending order and maintain index association
- The comparison of array sorting functions

compact

compact

(PHP 4, PHP 5, PHP 7, PHP 8)

compact — Create array containing variables and their values

Description

function **compact**(array|string `$var_name`, array|string `...$var_names`): array

Creates an array containing variables and their values.

For each of these, **compact()** looks for a variable with that name in the current symbol table and adds it to the output array such that the variable name becomes the key and the contents of the variable become the value for that key. In short, it does the opposite of extract().

> **Note** :
>
> Before PHP 7.3, any strings that are not set will silently be skipped.

Parameters

`var_name`
`var_names`

> **compact()** takes a variable number of parameters. Each parameter can be either a string containing the name of the variable, or an array of variable names. The array can contain other arrays of variable names inside it; **compact()** handles it recursively.

Return Values

Returns the output array with all the variables added to it.

Errors/Exceptions

compact() issues an `E_WARNING` level error if a given string refers to an unset variable.

Changelog

Version	Description
8.0.0	If a given string references an unset variable, an `E_WARNING` level error is now issued.
7.3.0	**compact()** now issues an `E_NOTICE` level error if a given string refers to an unset variable. Formerly, such strings have been silently skipped.

Examples

Example #1 **compact()** example

```php
<?php

$city  = "San Francisco";
$state = "CA";
$event = "SIGGRAPH";

$location_vars = array("city", "state");

$result = compact("event", $location_vars);
print_r($result);

?>
```

The above example will output:

```
Array
(
    [event] => SIGGRAPH
    [city] => San Francisco
    [state] => CA
)
```

Notes

> **Note** : **Gotcha**
>
> Because variable variables may not be used with PHP's Superglobal arrays within functions, the Superglobal arrays may not be passed into **compact()**.

See Also

- extract() - Import variables into the current symbol table from an array

count

count

(PHP 4, PHP 5, PHP 7, PHP 8)

count — Counts all elements in an array or in a Countable object

Description

function **count**(Countable|array `$value`, int `$mode` = `COUNT_NORMAL`): int

Counts all elements in an array when used with an array. When used with an object that implements the Countable interface, it returns the return value of the method Countable::count().

Parameters

`value`

An array or Countable object.

`mode`

If the optional `mode` parameter is set to `COUNT_RECURSIVE` (or 1), **count()** will recursively count the array. This is particularly useful for counting all the elements of a multidimensional array.

> **Caution**
>
> **count()** can detect recursion to avoid an infinite loop, but will emit an `E_WARNING` every time it does (in case the array contains itself more than once) and return a count higher than may be expected.

Return Values

Returns the number of elements in `value`. Prior to PHP 8.0.0, if the parameter was neither an array nor an object that implements the Countable interface, `1` would be returned, unless `value` was `null`, in which case `0` would be returned.

Changelog

Version	Description
8.0.0	**count()** will now throw TypeError on invalid countable types passed to the `value` parameter.
7.2.0	**count()** will now yield a warning on invalid countable types passed to the `value` parameter.

Examples

Example #1 **count()** example

```php
<?php
$a[0] = 1;
$a[1] = 3;
$a[2] = 5;
var_dump(count($a));

$b[0]  = 7;
$b[5]  = 9;
$b[10] = 11;
var_dump(count($b));
?>
```

The above example will output:

```
int(3)
int(3)
```

Example #2 **count()** non Countable|array example (bad example - don't do this)

```php
<?php
$b[0]  = 7;
$b[5]  = 9;
$b[10] = 11;
var_dump(count($b));

var_dump(count(null));

var_dump(count(false));
?>
```

The above example will output:

```
int(3)

Fatal error: Uncaught TypeError: count(): Argument #1 ($var) must be of type Countable .. on line 12
```

Example #3 Recursive **count()** example

```php
<?php
$food = array('fruits' => array('orange', 'banana', 'apple'),
              'veggie' => array('carrot', 'collard', 'pea'));

// recursive count
var_dump(count($food, COUNT_RECURSIVE));

// normal count
var_dump(count($food));

?>
```

The above example will output:

```
int(8)
int(2)
```

Example #4 Countable object

```php
<?php
class CountOfMethods implements Countable
{
    private function someMethod()
    {
    }

    public function count(): int
    {
        return count(get_class_methods($this));
    }
}

$obj = new CountOfMethods();
var_dump(count($obj));
?>
```

The above example will output:

```
int(2)
```

See Also

- is_array() - Finds whether a variable is an array
- isset() - Determine if a variable is declared and is different than null
- empty() - Determine whether a variable is empty
- strlen() - Get string length
- is_countable() - Verify that the contents of a variable is a countable value
- Arrays

current

current

(PHP 4, PHP 5, PHP 7, PHP 8)

current — Return the current element in an array

Description

function **current**(array|object `$array`): mixed

Every array has an internal pointer to its "current" element, which is initialized to the first element inserted into the array.

Parameters

`array`

The array.

Return Values

The **current()** function simply returns the value of the array element that's currently being pointed to by the internal pointer. It does not move the pointer in any way. If the internal pointer points beyond the end of the elements list or the array is empty, **current()** returns `false`.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Changelog

Version	Description
8.1.0	Calling this function on objects is deprecated. Either convert the object to an array using get_mangled_object_vars() first, or use the methods provided by a class that implements Iterator, such as ArrayIterator, instead.
7.4.0	Instances of SPL classes are now treated like empty objects that have no properties instead of calling the Iterator method with the same name as this function.

Examples

Example #1 Example use of current() and friends

```php
<?php
$transport = array('foot', 'bike', 'car', 'plane');
echo $mode = current($transport), PHP_EOL; // $mode = 'foot';
echo $mode = next($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = current($transport), PHP_EOL; // $mode = 'bike';
echo $mode = prev($transport), PHP_EOL;    // $mode = 'foot';
echo $mode = end($transport), PHP_EOL;     // $mode = 'plane';
echo $mode = current($transport), PHP_EOL; // $mode = 'plane';

$arr = array();
var_dump(current($arr)); // bool(false)

$arr = array(array());
var_dump(current($arr)); // array(0) { }
?>
```

Notes

> **Note** :
>
> The results of calling **current()** on an empty array and on an array, whose internal pointer points beyond the end of the elements, are indistinguishable from a bool `false` element. To properly traverse an array which may contain `false` elements, see the `foreach` control structure.
>
> To still use **current()** and properly check if the value is really an element of the array, the key() of the **current()** element should be checked to be strictly different from `null`.

See Also

- end() - Set the internal pointer of an array to its last element
- key() - Fetch a key from an array
- each() - Return the current key and value pair from an array and advance the array cursor
- prev() - Rewind the internal array pointer
- reset() - Set the internal pointer of an array to its first element
- next() - Advance the internal pointer of an array
- `foreach`

each

each

(PHP 4, PHP 5, PHP 7)

each — Return the current key and value pair from an array and advance the array cursor

> **Warning**

> This function has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **each**(array|object `&$array`): array

Return the current key and value pair from an array and advance the array cursor.

After **each()** has executed, the array cursor will be left on the next element of the array, or past the last element if it hits the end of the array. You have to use reset() if you want to traverse the array again using each.

Parameters

`array`

The input array.

Return Values

Returns the current key and value pair from the array `array`. This pair is returned in a four-element array, with the keys `0`, `1`, `key`, and `value`. Elements `0` and `key` contain the key name of the array element, and `1` and `value` contain the data.

If the internal pointer for the array points past the end of the array contents, **each()** returns `false`.

Examples

Example #1 each() examples

```php
<?php
$foo = array("bob", "fred", "jussi", "jouni", "egon", "marliese");
$bar = each($foo);
print_r($bar);
?>
```

$bar now contains the following key/value pairs:

```
Array
(
    [1] => bob
    [value] => bob
    [0] => 0
    [key] => 0
)
```

```php
<?php
$foo = array("Robert" => "Bob", "Seppo" => "Sepi");
$bar = each($foo);
print_r($bar);
?>
```

$bar now contains the following key/value pairs:

```
Array
(
    [1] => Bob
    [value] => Bob
    [0] => Robert
    [key] => Robert
)
```

each() is typically used in conjunction with list() to traverse an array, here's an example:

Example #2 Traversing an array with each()

```php
<?php
$fruit = array('a' => 'apple', 'b' => 'banana', 'c' => 'cranberry');

reset($fruit);
while (list($key, $val) = each($fruit)) {
    echo "$key => $val\n";
}
?>
```

The above example will output:

```
a => apple
b => banana
c => cranberry
```

> **Caution**
>
> Because assigning an array to another variable resets the original array's pointer, our example above would cause an endless loop had we assigned *$fruit* to another variable inside the loop.

> **Warning**
>
> **each()** will also accept objects, but may return unexpected results. It's therefore not recommended to iterate through object properties with **each()**.

See Also

- key() - Fetch a key from an array
- list() - Assign variables as if they were an array
- current() - Return the current element in an array
- reset() - Set the internal pointer of an array to its first element
- next() - Advance the internal pointer of an array
- prev() - Rewind the internal array pointer
- `foreach`
- Object Iteration

end

end

(PHP 4, PHP 5, PHP 7, PHP 8)

end — Set the internal pointer of an array to its last element

Description

function **end**(array|object `&$array`): mixed

end() advances `array`'s internal pointer to the last element, and returns its value.

Parameters

`array`

The array. This array is passed by reference because it is modified by the function. This means you must pass it a real variable and not a function returning an array because only actual variables may be passed by reference.

Return Values

Returns the value of the last element or `false` for empty array.

Changelog

Version	Description
8.1.0	Calling this function on objects is deprecated. Either convert the object to an array using get_mangled_object_vars() first, or use the methods provided by a class that implements Iterator, such as ArrayIterator, instead.
7.4.0	Instances of SPL classes are now treated like empty objects that have no properties instead of calling the Iterator method with the same name as this function.

Examples

Example #1 end() example

```php
<?php

$fruits = array('apple', 'banana', 'cranberry');
echo end($fruits); // cranberry

?>
```

See Also

- current() - Return the current element in an array
- each() - Return the current key and value pair from an array and advance the array cursor
- prev() - Rewind the internal array pointer
- reset() - Set the internal pointer of an array to its first element
- next() - Advance the internal pointer of an array
- array_key_last() - Gets the last key of an array

extract

extract

(PHP 4, PHP 5, PHP 7, PHP 8)

extract — Import variables into the current symbol table from an array

Description

function **extract**(array `&$array`, int `$flags = EXTR_OVERWRITE`, string `$prefix = ""`): int

Import variables from an array into the current symbol table.

Checks each key to see whether it has a valid variable name. It also checks for collisions with existing variables in the symbol table.

> **Warning**
>
> Do not use **extract()** on untrusted data, like user input (e.g. *$_GET*, *$_FILES*).

Parameters

`array`

An associative array. This function treats keys as variable names and values as variable values. For each key/value pair it will create a variable in the current symbol table, subject to `flags` and `prefix` parameters.

You must use an associative array; a numerically indexed array will not produce results unless you use `EXTR_PREFIX_ALL` or `EXTR_PREFIX_INVALID`.

`flags`

The way invalid/numeric keys and collisions are treated is determined by the extraction `flags`. It can be one of the following values:

EXTR_OVERWRITE

If there is a collision, overwrite the existing variable.

EXTR_SKIP

If there is a collision, don't overwrite the existing variable.

EXTR_PREFIX_SAME

If there is a collision, prefix the variable name with `prefix`.

EXTR_PREFIX_ALL

Prefix all variable names with `prefix`.

EXTR_PREFIX_INVALID

Only prefix invalid/numeric variable names with `prefix`.

EXTR_IF_EXISTS

Only overwrite the variable if it already exists in the current symbol table, otherwise do nothing. This is useful for defining a list of valid variables and then extracting only those variables you have defined out of *$_REQUEST*, for example.

EXTR_PREFIX_IF_EXISTS

Only create prefixed variable names if the non-prefixed version of the same variable exists in the current symbol table.

EXTR_REFS

Extracts variables as references. This effectively means that the values of the imported variables are still referencing the values of the `array` parameter. You can use this flag on its own or combine it with any other flag by OR'ing the `flags`.

If `flags` is not specified, it is assumed to be **EXTR_OVERWRITE**.

`prefix`

Note that `prefix` is only required if `flags` is **EXTR_PREFIX_SAME**, **EXTR_PREFIX_ALL**, **EXTR_PREFIX_INVALID** or **EXTR_PREFIX_IF_EXISTS**. If the prefixed result is not a valid variable name, it is not imported into the symbol table. Prefixes are automatically separated from the array key by an underscore character.

Return Values

Returns the number of variables successfully imported into the symbol table.

Examples

Example #1 extract() example

```php
<?php
$size = "large";
$var_array = array(
    "color" => "blue",
    "size"  => "medium",
    "shape" => "sphere"
);

extract($var_array, EXTR_PREFIX_SAME, "wddx");

echo "$color, $size, $shape, $wddx_size\n";

?>
```

The above example will output:

```
```

```
blue, large, sphere, medium
```

The *$size* wasn't overwritten because we specified **EXTR_PREFIX_SAME**, which resulted in *$wddx_size* being created. If **EXTR_SKIP** was specified, then *$wddx_size* wouldn't even have been created. **EXTR_OVERWRITE** would have caused *$size* to have value "medium", and **EXTR_PREFIX_ALL** would result in new variables being named *$wddx_color*, *$wddx_size*, and *$wddx_shape*.

Notes

> **Warning**
>
> Do not use **extract()** on untrusted data, like user input (i.e. *$_GET*, *$_FILES*, etc.). If you do, make sure you use one of the non-overwriting `flags` values such as **EXTR_SKIP** and be aware that you should extract in the same order that's defined in variables_order within the *php.ini*.

See Also

- compact() - Create array containing variables and their values
- list() - Assign variables as if they were an array

in_array

in_array

(PHP 4, PHP 5, PHP 7, PHP 8)

in_array — Checks if a value exists in an array

Description

function **in_array**(mixed `$needle`, array `$haystack`, bool `$strict = false`): bool

Searches for `needle` in `haystack` using loose comparison unless `strict` is set.

Parameters

`needle`

The searched value.

> **Note** :
>
> If `needle` is a string, the comparison is done in a case-sensitive manner.

`haystack`

The array.

`strict`

If the third parameter `strict` is set to **true** then the **in_array()** function will also check the types of the `needle` in the `haystack`.

> **Note** :
>
> Prior to PHP 8.0.0, a non-numeric string `needle` would loosely match a `haystack` value of `0`, and vice versa. As of PHP 8.0.0, the number is converted to a string and the two are compared as strings, so only a numeric string of the same value still matches.
>
> Non-strict mode nonetheless uses loose comparison, so values of different types can still compare as equal: **true** matches any truthy string, and **false** matches both `""` and `"0"`. Unless the types of all values involved are known with certainty, always pass `strict` to force an identity comparison.

Return Values

Returns `true` if `needle` is found in the array, `false` otherwise.

Examples

Example #1 in_array() example

```
<?php
$os = array("Mac", "NT", "Irix", "Linux");
if (in_array("Irix", $os)) {
    echo "Got Irix";
}
if (in_array("mac", $os)) {
    echo "Got mac";
}
?>
```

The second condition fails because **in_array()** is case-sensitive, so the program above will display:

```
Got Irix
```

Example #2 in_array() with strict example

```
<?php
$a = array('1.10', 12.4, 1.13);

if (in_array('12.4', $a, true)) {
    echo "'12.4' found with strict check\n";
}

if (in_array(1.13, $a, true)) {
    echo "1.13 found with strict check\n";
}
?>
```

The above example will output:

```
1.13 found with strict check
```

Example #3 in_array() with an array as needle

```
<?php
$a = array(array('p', 'h'), array('p', 'r'), 'o');

if (in_array(array('p', 'h'), $a)) {
    echo "'ph' was found\n";
}

if (in_array(array('f', 'i'), $a)) {
    echo "'fi' was found\n";
}

if (in_array('o', $a)) {
    echo "'o' was found\n";
}
?>
```

The above example will output:

```
'ph' was found
'o' was found
```

See Also

- array_search() - Searches the array for a given value and returns the first corresponding key if successful
- isset() - Determine if a variable is declared and is different than null
- array_key_exists() - Checks if the given key or index exists in the array

key

key

(PHP 4, PHP 5, PHP 7, PHP 8)

key — Fetch a key from an array

Description

function **key**(array|object `$array`): int|string|null

key() returns the index element of the current array position.

Parameters

`array`

The array.

Return Values

The **key()** function simply returns the key of the array element that's currently being pointed to by the internal pointer. It does not move the pointer in any way. If the internal pointer points beyond the end of the elements list or the array is empty, **key()** returns `null`.

Changelog

Version	Description
8.1.0	Calling this function on objects is deprecated. Either convert the object to an array using get_mangled_object_vars() first, or use the methods provided by a class that implements Iterator, such as ArrayIterator, instead.
7.4.0	Instances of SPL classes are now treated like empty objects that have no properties instead of calling the Iterator method with the same name as this function.

Examples

Example #1 key() example

```php
<?php
$array = array(
    'fruit1' => 'apple',
    'fruit2' => 'orange',
    'fruit3' => 'grape',
    'fruit4' => 'apple',
    'fruit5' => 'apple');

// this cycle echoes all associative array
// key where value equals "apple"
while ($fruit_name = current($array)) {
    if ($fruit_name == 'apple') {
        echo key($array), "\n";
    }
    next($array);
}
?>
```

The above example will output:

```
fruit1
fruit4
fruit5
```

See Also

- current() - Return the current element in an array
- next() - Advance the internal pointer of an array
- array_key_first() - Gets the first key of an array
- foreach

key_exists

key_exists

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

key_exists — Alias of array_key_exists()

Description

This function is an alias of: array_key_exists().

krsort

krsort

(PHP 4, PHP 5, PHP 7, PHP 8)

krsort — Sort an array by key in descending order

Description

function **krsort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Sorts `array` in place by keys in descending order.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- `SORT_REGULAR` - compare items normally; the details are described in the comparison operators section

- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings
- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- `SORT_NATURAL` - compare items as strings using "natural ordering" like natsort()
- `SORT_FLAG_CASE` - can be combined (bitwise OR) with `SORT_STRING` or `SORT_NATURAL` to sort strings case-insensitively

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.
8.2.0	This function now does numeric string comparison under `SORT_REGULAR` using the standard PHP 8 rules.

Examples

Example #1 krsort() example

```php
<?php
$fruits = array("d"=>"lemon", "a"=>"orange", "b"=>"banana", "c"=>"apple");
krsort($fruits);
foreach ($fruits as $key => $val) {
    echo "$key = $val\n";
}
?>
```

The above example will output:

```
d = lemon
c = apple
b = banana
a = orange
```

See Also

- sort() - Sort an array in ascending order
- ksort() - Sort an array by key in ascending order
- The comparison of array sorting functions

ksort

ksort

(PHP 4, PHP 5, PHP 7, PHP 8)

ksort — Sort an array by key in ascending order

Description

function **ksort**(array `&$array`, int `$flags` = `SORT_REGULAR`): true

Sorts `array` in place by keys in ascending order.

> **Note** :

If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- **SORT_REGULAR** - compare items normally; the details are described in the comparison operators section
- **SORT_NUMERIC** - compare items numerically
- **SORT_STRING** - compare items as strings
- **SORT_LOCALE_STRING** - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- **SORT_NATURAL** - compare items as strings using "natural ordering" like natsort()
- **SORT_FLAG_CASE** - can be combined (bitwise OR) with **SORT_STRING** or **SORT_NATURAL** to sort strings case-insensitively

Return Values

Always returns **true**.

Changelog

Version	Description
8.2.0	The return type is **true** now; previously, it was bool.
8.2.0	This function now does numeric string comparison under **SORT_REGULAR** using the standard PHP 8 rules.

Examples

Example #1 **ksort()** example

```php
<?php
$fruits = array("d"=>"lemon", "a"=>"orange", "b"=>"banana", "c"=>"apple");
ksort($fruits);
foreach ($fruits as $key => $val) {
    echo "$key = $val\n";
}
?>
```

The above example will output:

```
a = orange
b = banana
c = apple
d = lemon
```

Example #2 ksort() with int keys

```
<?php
$a = [0 => 'First', 2 => 'Last', 1 => 'Middle'];
var_dump($a);
ksort($a);
var_dump($a);
?>
```

The above example will output:

```
array(3) {
  [0]=>
  string(5) "First"
  [2]=>
  string(4) "Last"
  [1]=>
  string(6) "Middle"
}
array(3) {
  [0]=>
  string(5) "First"
  [1]=>
  string(6) "Middle"
  [2]=>
  string(4) "Last"
}
```

See Also

- sort() - Sort an array in ascending order
- krsort() - Sort an array by key in descending order
- The comparison of array sorting functions

list

list

(PHP 4, PHP 5, PHP 7, PHP 8)

list — Assign variables as if they were an array

Description

function **list**(mixed `$var`, mixed `...$vars = ?`): array

Like array(), this is not really a function, but a language construct. **list()** is used to assign a list of variables in one operation. Only arrays and objects that implement ArrayAccess can be unpacked. **list()** expressions can not be completely empty.

> **Note** :
>
> Before PHP 7.1.0, **list()** only worked on numerical arrays and assumes the numerical indices start at 0.

As of PHP 7.1.0, **list()** can also contain explicit keys, allowing for the destructuring of arrays with non-integer or non-sequential keys. For more details on array destructuring, see the array destructuring section.

> **Note** :
>
> Attempting to access an array key which has not been defined is the same as accessing any other undefined variable: an `E_WARNING`-level error message (`E_NOTICE`-level prior to PHP 8.0.0) will be issued, and the result will be `null`.

> Attempting to unpack a scalar assigns `null` to all variables. Attempting to unpack an object that does not implement ArrayAccess is a fatal error.

Parameters

`var`

A variable.

`vars`

Further variables.

Return Values

Returns the assigned array.

Changelog

Version	Description
8.5.0	Destructuring a non-array value (other than `null`) now emits an `E_WARNING`.
7.3.0	Support for reference assignments in array destructuring was added.
7.1.0	It is now possible to specify keys in **list()**. This enables destructuring of arrays with non-integer or non-sequential keys.

Examples

Example #1 list() examples

```php
<?php

$info = array('coffee', 'brown', 'caffeine');

// Listing all the variables
list($drink, $color, $power) = $info;
echo "$drink is $color and $power makes it special.\n";

// Listing some of them
list($drink, , $power) = $info;
echo "$drink has $power.\n";

// Or let's skip to only the third one
list( , , $power) = $info;
echo "I need $power!\n";

// list() doesn't work with strings
list($bar) = "abcde";
var_dump($bar); // NULL
?>
```

Example #2 An example use of list()

```php
<?php
$result = $pdo->query("SELECT id, name FROM employees");
while (list($id, $name) = $result->fetch(PDO::FETCH_NUM)) {
    echo "id: $id, name: $name\n";
}
?>
```

Example #3 Using nested list()

```php
<?php

list($a, list($b, $c)) = array(1, array(2, 3));

var_dump($a, $b, $c);

?>
```

The above example will output:

```
int(1)
int(2)
int(3)
```

The order in which the indices of the array to be consumed by **list()** are defined is irrelevant.

Example #4 list() and order of index definitions

```php
<?php
$foo = array(2 => 'a', 'foo' => 'b', 0 => 'c');
$foo[1] = 'd';
list($x, $y, $z) = $foo;
var_dump($foo, $x, $y, $z);
```

Gives the following output (note the order of the elements compared in which order they were written in the **list()** syntax):

```
array(4) {
  [2]=>
  string(1) "a"
  ["foo"]=>
  string(1) "b"
  [0]=>
  string(1) "c"
  [1]=>
  string(1) "d"
}
string(1) "c"
string(1) "d"
string(1) "a"
```

Example #5 list() with keys

As of PHP 7.1.0 **list()** can now also contain explicit keys, which can be given as arbitrary expressions. Mixing of integer and string keys is allowed; however, elements with and without keys cannot be mixed.

```php
<?php
$data = [
    ["id" => 1, "name" => 'Tom'],
    ["id" => 2, "name" => 'Fred'],
];
foreach ($data as ["id" => $id, "name" => $name]) {
    echo "id: $id, name: $name\n";
}
echo PHP_EOL;
list(1 => $second, 3 => $fourth) = [1, 2, 3, 4];
echo "$second, $fourth\n";
```

The above example will output:

```
id: 1, name: Tom
id: 2, name: Fred

2, 4
```

See Also

- each() - Return the current key and value pair from an array and advance the array cursor
- array() - Create an array
- extract() - Import variables into the current symbol table from an array

natcasesort

natcasesort

(PHP 4, PHP 5, PHP 7, PHP 8)

natcasesort — Sort an array using a case insensitive "natural order" algorithm

Description

function **natcasesort**(array `&$array`): true

natcasesort() is a case insensitive version of natsort().

This function implements a sort algorithm that orders alphanumeric strings in the way a human being would while maintaining key/value associations. This is described as a "natural ordering".

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 natcasesort() example

```php
<?php
$array1 = $array2 = array('IMG0.png', 'img12.png', 'img10.png', 'img2.png', 'img1.png', 'IMG3.png');

sort($array1);
echo "Standard sorting\n";
print_r($array1);

natcasesort($array2);
echo "\nNatural order sorting (case-insensitive)\n";
print_r($array2);
?>
```

The above example will output:

```
Standard sorting
Array
(
    [0] => IMG0.png
    [1] => IMG3.png
    [2] => img1.png
    [3] => img10.png
    [4] => img12.png
    [5] => img2.png
)

Natural order sorting (case-insensitive)
Array
(
    [0] => IMG0.png
    [4] => img1.png
    [3] => img2.png
    [5] => IMG3.png
    [2] => img10.png
    [1] => img12.png
)
```

For more information see: Martin Pool's » Natural Order String Comparison page.

See Also

- natsort() - Sort an array using a "natural order" algorithm
- The comparison of array sorting functions
- strnatcmp() - String comparisons using a "natural order" algorithm
- strnatcasecmp() - Case insensitive string comparisons using a "natural order" algorithm

natsort

natsort

(PHP 4, PHP 5, PHP 7, PHP 8)

natsort — Sort an array using a "natural order" algorithm

Description

function **natsort**(array `&$array`): true

This function implements a sort algorithm that orders alphanumeric strings in the way a human being would while maintaining key/value associations. This is described as a "natural ordering". An example of the difference between this algorithm and the regular computer string sorting algorithms (used in sort()) can be seen in the example below.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 natsort() examples demonstrating basic usage

```php
<?php
$array1 = $array2 = array("img12.png", "img10.png", "img2.png", "img1.png");

asort($array1);
echo "Standard sorting\n";
print_r($array1);

natsort($array2);
echo "\nNatural order sorting\n";
print_r($array2);
?>
```

The above example will output:

```
Standard sorting
Array
(
    [3] => img1.png
    [1] => img10.png
    [0] => img12.png
    [2] => img2.png
)

Natural order sorting
Array
(
    [3] => img1.png
    [2] => img2.png
    [1] => img10.png
    [0] => img12.png
)
```

For more information see: Martin Pool's » Natural Order String Comparison page.

Example #2 natsort() examples demonstrating potential gotchas

```php
<?php
echo "Negative numbers\n";
$negative = array('-5','3','-2','0','-1000','9','1');
print_r($negative);
natsort($negative);
print_r($negative);

echo "Zero padding\n";
$zeros = array('09', '8', '10', '009', '011', '0');
print_r($zeros);
natsort($zeros);
```

```
print_r($zeros);
?>
```

The above example will output:

```
Negative numbers
Array
(
    [0] => -5
    [1] => 3
    [2] => -2
    [3] => 0
    [4] => -1000
    [5] => 9
    [6] => 1
)
Array
(
    [2] => -2
    [0] => -5
    [4] => -1000
    [3] => 0
    [6] => 1
    [1] => 3
    [5] => 9
)
Zero padding
Array
(
    [0] => 09
    [1] => 8
    [2] => 10
    [3] => 009
    [4] => 011
    [5] => 0
)
Array
(
    [5] => 0
    [1] => 8
    [0] => 09
    [3] => 009
    [2] => 10
    [4] => 011
)
```

See Also

- natcasesort() - Sort an array using a case insensitive "natural order" algorithm
- The comparison of array sorting functions
- strnatcmp() - String comparisons using a "natural order" algorithm
- strnatcasecmp() - Case insensitive string comparisons using a "natural order" algorithm

next

next

(PHP 4, PHP 5, PHP 7, PHP 8)

next — Advance the internal pointer of an array

Description

function **next**(array|object `&$array`): mixed

next() behaves like current(), with one difference. It advances the internal array pointer one place forward before returning the element value. That means it returns the next array value and advances the internal array pointer by one.

Parameters

`array`

The array being affected.

Return Values

Returns the array value in the next place that's pointed to by the internal array pointer, or `false` if there are no more elements.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Changelog

Version	Description
8.1.0	Calling this function on objects is deprecated. Either convert the object to an array using get_mangled_object_vars() first, or use the methods provided by a class that implements Iterator, such as ArrayIterator, instead.
7.4.0	Instances of SPL classes are now treated like empty objects that have no properties instead of calling the Iterator method with the same name as this function.

Examples

Example #1 Example use of next() and friends

```php
<?php
$transport = array('foot', 'bike', 'car', 'plane');
echo $mode = current($transport), PHP_EOL; // $mode = 'foot';
echo $mode = next($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = next($transport), PHP_EOL;    // $mode = 'car';
echo $mode = prev($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = end($transport), PHP_EOL;     // $mode = 'plane';
?>
```

Notes

> **Note** :
>
> The end of an array is indistinguishable from a bool `false` element. To properly traverse an array which may contain `false` elements, see the `foreach` function.
>
> To still use **next()** and properly check if the end of the array has been reached, verify that the key() is `null`.

See Also

- current() - Return the current element in an array
- end() - Set the internal pointer of an array to its last element
- prev() - Rewind the internal array pointer
- reset() - Set the internal pointer of an array to its first element
- each() - Return the current key and value pair from an array and advance the array cursor

pos

pos

(PHP 4, PHP 5, PHP 7, PHP 8)

pos — Alias of current()

Description

This function is an alias of: current()

prev

prev

(PHP 4, PHP 5, PHP 7, PHP 8)

prev — Rewind the internal array pointer

Description

function **prev**(array|object `&$array`): mixed

Rewind the internal array pointer.

prev() behaves just like next(), except it rewinds the internal array pointer one place instead of advancing it.

Parameters

`array`

The input array.

Return Values

Returns the array value in the previous place that's pointed to by the internal array pointer, or `false` if there are no more elements.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Changelog

Version	Description
8.1.0	Calling this function on objects is deprecated. Either convert the object to an array using get_mangled_object_vars() first, or use the methods provided by a class that implements Iterator, such as ArrayIterator, instead.
7.4.0	Instances of SPL classes are now treated like empty objects that have no properties instead of calling the Iterator method with the same name as this function.

Examples

Example #1 Example use of prev() and friends

```php
<?php
$transport = array('foot', 'bike', 'car', 'plane');
echo $mode = current($transport), PHP_EOL; // $mode = 'foot';
echo $mode = next($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = next($transport), PHP_EOL;    // $mode = 'car';
echo $mode = prev($transport), PHP_EOL;    // $mode = 'bike';
echo $mode = end($transport), PHP_EOL;     // $mode = 'plane';
?>
```

Notes

> **Note** :
>
> The beginning of an array is indistinguishable from a bool `false` element. To make the distinction, check that the key() of the **prev()** element is not `null`.

See Also

- current() - Return the current element in an array
- end() - Set the internal pointer of an array to its last element
- next() - Advance the internal pointer of an array

- reset() - Set the internal pointer of an array to its first element
- each() - Return the current key and value pair from an array and advance the array cursor

range

range

(PHP 4, PHP 5, PHP 7, PHP 8)

range — Create an array containing a range of elements

Description

function **range**(string|int|float `$start`, string|int|float `$end`, int|float `$step` = 1): array

Create an array containing a range of elements.

If both `start` and `end` are strings, and `step` is int the produced array will be a sequence of bytes. Otherwise, the produced array will be a sequence of numbers.

The sequence is increasing if `start` is less than equal to `end`. Otherwise, the sequence is decreasing.

Parameters

`start`

First value of the sequence.

`end`

Last possible value of the sequence.

`step`

`step` indicates by how much is the produced sequence progressed between values of the sequence.

`step` may be negative for decreasing sequences.

If `step` is a float without a fractional part, it is interpreted as int.

Return Values

Returns a sequence of elements as an array with the first element being `start` going up to `end`, with each value of the sequence being `step` values apart.

The last element of the returned array is either `end` or the previous element of the sequence, depending on the value of `step`.

If both `start` and `end` are strings, and `step` is int the produced array will be a sequence of bytes, generally latin ASCII characters.

If at least one of `start`, `end`, or `step` is float the produced array will be a sequence of float.

Otherwise, the produced array will be a sequence of int.

Errors/Exceptions

- If `step` is `0`, a ValueError is thrown.
- If `start`, `end`, or `step` is not is_finite(), a ValueError is thrown.
- If `step` is negative, but the produced range is increasing (i.e. `$start <= $end`), a ValueError is thrown.
- If `start` or `end` is the empty string `''`, an **`E_WARNING`** is emitted and the empty string will be interpreted as `0`.
- If `start` or `end` is a non-numeric string with more than one byte, an **`E_WARNING`** is emitted.
- If `start` or `end` is a string that is implicitly cast to an int because the other boundary value is a number, an **`E_WARNING`** is emitted.
- If `step` is a float, and `start` and `end` are non-numeric string, an **`E_WARNING`** is emitted.

Changelog

Version	Description
8.3.0	If both `start` and `end` are strings then **range()** will now always produce an array of bytes. Previously if one of the boundary values was a numeric string, then the other boundary value was implicitly cast to int.
8.3.0	An `E_WARNING` is now emitted if `start` or `end` is a string that is implicitly cast to int because the other boundary value is a number.
8.3.0	An `E_WARNING` is now emitted if `start` or `end` is a non-numeric string with more than one byte.
8.3.0	An `E_WARNING` is now emitted if `start` or `end` is the empty string.
8.3.0	If `step` is a float with no fractional part, it will be interpreted as an int.
8.3.0	A ValueError is now thrown if `step` is negative when producing an increasing range.
8.3.0	A ValueError is now thrown if `step` is not finite.
8.3.0	A TypeError is now thrown if `start` or `end` is an array, object, or resource. Previously they were implicitly cast to int.

Examples

Example #1 range() examples

```php
<?php
echo implode(', ', range(0, 12)), PHP_EOL;

echo implode(', ', range(0, 100, 10)), PHP_EOL;

echo implode(', ', range('a', 'i')), PHP_EOL;

echo implode(', ', range('c', 'a')), PHP_EOL;

echo implode(', ', range('A', 'z')), PHP_EOL;
?>
```

The above example will output:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100
a, b, c, d, e, f, g, h, i
c, b, a
A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, [, \, ], ^, _, `, a, b, c, d, e, f, g, h, i,
j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z
```

See Also

- shuffle() - Shuffle an array
- array_fill() - Fill an array with values
- `foreach`

reset

reset

(PHP 4, PHP 5, PHP 7, PHP 8)

reset — Set the internal pointer of an array to its first element

Description

function **reset**(array|object `&$array`): mixed

reset() rewinds `array`'s internal pointer to the first element and returns the value of the first array element.

Parameters

`array`

The input array.

Return Values

Returns the value of the first array element, or `false` if the array is empty.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Changelog

Version	Description
8.1.0	Calling this function on objects is deprecated. Either convert the object to an array using get_mangled_object_vars() first, or use the methods provided by a class that implements Iterator, such as ArrayIterator, instead.
7.4.0	Instances of SPL classes are now treated like empty objects that have no properties instead of calling the Iterator method with the same name as this function.

Examples

Example #1 reset() example

```php
<?php

$array = array('step one', 'step two', 'step three', 'step four');

// by default, the pointer is on the first element
echo current($array) . "<br />\n"; // "step one"

// skip two steps
next($array);
next($array);
echo current($array) . "<br />\n"; // "step three"

// reset pointer, start again on step one
reset($array);
echo current($array) . "<br />\n"; // "step one"

?>
```

Notes

> **Note** :
>
> The return value for an empty array is indistinguishable from the return value in case of an array which has a bool `false` first element. To properly check the value of the first element of an array which may contain `false` elements, first check the count() of the array, or check that key() is not `null`, after calling **reset()**.

See Also

- current() - Return the current element in an array
- each() - Return the current key and value pair from an array and advance the array cursor
- end() - Set the internal pointer of an array to its last element
- next() - Advance the internal pointer of an array
- prev() - Rewind the internal array pointer
- array_key_first() - Gets the first key of an array

rsort

rsort

(PHP 4, PHP 5, PHP 7, PHP 8)

rsort — Sort an array in descending order

Description

function **rsort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Sorts `array` in place by values in descending order.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> This function assigns new keys to the elements in `array`. It will remove any existing keys that may have been assigned, rather than just reordering the keys.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- `SORT_REGULAR` - compare items normally; the details are described in the comparison operators section
- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings
- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- `SORT_NATURAL` - compare items as strings using "natural ordering" like natsort()
- `SORT_FLAG_CASE` - can be combined (bitwise OR) with `SORT_STRING` or `SORT_NATURAL` to sort strings case-insensitively

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 rsort() example

```php
<?php
$fruits = array("lemon", "orange", "banana", "apple");
rsort($fruits);
foreach ($fruits as $key => $val) {
    echo "$key = $val\n";
```

```
}
?>
```

The above example will output:

```
0 = orange
1 = lemon
2 = banana
3 = apple
```

The fruits have been sorted in reverse alphabetical order.

See Also

- sort() - Sort an array in ascending order
- arsort() - Sort an array in descending order and maintain index association
- krsort() - Sort an array by key in descending order
- The comparison of array sorting functions

shuffle

shuffle

(PHP 4, PHP 5, PHP 7, PHP 8)

shuffle — Shuffle an array

Description

function **shuffle**(array `&$array`): true

This function shuffles (randomizes the order of the elements in) an array.

> **Caution**
>
> This function does not generate cryptographically secure values, and *must not* be used for cryptographic purposes, or purposes that require returned values to be unguessable.
>
> If cryptographically secure randomness is required, the Random\Randomizer may be used with the Random\Engine\Secure engine. For simple use cases, the random_int() and random_bytes() functions provide a convenient and secure API that is backed by the operating system's CSPRNG.

> **Caution**
>
> This function uses the global Mt19937 ("Mersenne Twister") instance as the source of randomness and thus shares its state with all other functions using the global Mt19937. Using any of these functions advances the sequence for *all* the other functions, regardless of scope.
>
> Generating repeatable sequences by seeding mt_srand() or srand() with a known value will also yield repeatable output from this function.
>
> Prefer using Random\Randomizer methods in all newly written code.

Parameters

`array`

The array.

Return Values

Always returns `true`.

Changelog

Version	Description
7.1.0	The internal randomization algorithm has been changed to use the » Mersenne Twister Random Number Generator instead of the libc rand function.

Examples

Example #1 shuffle() example

```php
<?php
$numbers = range(1, 20);
shuffle($numbers);
foreach ($numbers as $number) {
    echo "$number ";
}
?>
```

Notes

> **Note** :
>
> This function assigns new keys to the elements in `array`. It will remove any existing keys that may have been assigned, rather than just reordering the keys.

> **Note** :
>
> Resets array's internal pointer to the first element.

See Also

- Random\Randomizer::shuffleArray() - Get a permutation of an array
- Random\Randomizer::shuffleBytes() - Get a byte-wise permutation of a string
- Random\Randomizer::pickArrayKeys() - Select random array keys
- The comparison of array sorting functions

sizeof

sizeof

(PHP 4, PHP 5, PHP 7, PHP 8)

sizeof — Alias of count()

Description

This function is an alias of: count().

sort

sort

(PHP 4, PHP 5, PHP 7, PHP 8)

sort — Sort an array in ascending order

Description

function **sort**(array `&$array`, int `$flags = SORT_REGULAR`): true

Sorts `array` in place by values in ascending order.

> **Note** :

If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> This function assigns new keys to the elements in `array`. It will remove any existing keys that may have been assigned, rather than just reordering the keys.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- **SORT_REGULAR** - compare items normally; the details are described in the comparison operators section
- **SORT_NUMERIC** - compare items numerically
- **SORT_STRING** - compare items as strings
- **SORT_LOCALE_STRING** - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- **SORT_NATURAL** - compare items as strings using "natural ordering" like natsort()
- **SORT_FLAG_CASE** - can be combined (bitwise OR) with **SORT_STRING** or **SORT_NATURAL** to sort strings case-insensitively

Return Values

Always returns **true**.

Changelog

Version	Description
8.2.0	The return type is **true** now; previously, it was bool.

Examples

Example #1 sort() example

```php
<?php

$fruits = array("lemon", "orange", "banana", "apple");
sort($fruits);
foreach ($fruits as $key => $val) {
    echo "fruits[" . $key . "] = " . $val . "\n";
}

?>
```

The above example will output:

```
```

```
fruits[0] = apple
fruits[1] = banana
fruits[2] = lemon
fruits[3] = orange
```

The fruits have been sorted in alphabetical order.

Example #2 sort() example using case-insensitive natural ordering

```php
<?php

$fruits = array(
    "Orange1", "orange2", "Orange3", "orange20"
);
sort($fruits, SORT_NATURAL | SORT_FLAG_CASE);
foreach ($fruits as $key => $val) {
    echo "fruits[" . $key . "] = " . $val . "\n";
}

?>
```

The above example will output:

```
fruits[0] = Orange1
fruits[1] = orange2
fruits[2] = Orange3
fruits[3] = orange20
```

The fruits have been sorted like natcasesort().

Notes

> **Note** :
>
> Like most PHP sorting functions, **sort()** uses an implementation of » Quicksort. The pivot is chosen in the middle of the partition resulting in an optimal time for already sorted arrays. This is however an implementation detail you shouldn't rely on.

> **Warning**
>
> Be careful when sorting arrays with mixed types values because **sort()** can produce unexpected results, if `flags` is `SORT_REGULAR`.

See Also

- rsort() - Sort an array in descending order
- The comparison of array sorting functions

uasort

uasort

(PHP 4, PHP 5, PHP 7, PHP 8)

uasort — Sort an array with a user-defined comparison function and maintain index association

Description

function **uasort**(array `&$array`, callable `$callback`): true

Sorts `array` in place such that its keys maintain their correlation with the values they are associated with, using a user-defined comparison function.

This is used mainly when sorting associative arrays where the actual element order is significant.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`callback`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Always returns **`true`**.

Changelog

Version	Description
8.2.0	The return type is **`true`** now; previously, it was bool.
8.0.0	If `callback` expects a parameter to be passed by reference, this function will now emit an **`E_WARNING`**.

Examples

Example #1 Basic uasort() example

```php
<?php
// Comparison function
function cmp($a, $b) {
    if ($a == $b) {
        return 0;
    }
    return ($a < $b) ? -1 : 1;
}

// Array to be sorted
$array = array('a' => 4, 'b' => 8, 'c' => -1, 'd' => -9, 'e' => 2, 'f' => 5, 'g' => 3, 'h' => -4);
print_r($array);

// Sort and print the resulting array
uasort($array, 'cmp');
print_r($array);
?>
```

The above example will output:

```
Array
(
    [a] => 4
    [b] => 8
    [c] => -1
    [d] => -9
    [e] => 2
    [f] => 5
    [g] => 3
    [h] => -4
)
Array
(
    [d] => -9
    [h] => -4
    [c] => -1
    [e] => 2
    [g] => 3
    [a] => 4
    [f] => 5
    [b] => 8
)
```

See Also

- usort() - Sort an array by values using a user-defined comparison function
- uksort() - Sort an array by keys using a user-defined comparison function
- The comparison of array sorting functions

uksort

uksort

(PHP 4, PHP 5, PHP 7, PHP 8)

uksort — Sort an array by keys using a user-defined comparison function

Description

function **uksort**(array `&$array`, callable `$callback`): true

Sorts `array` in place by keys using a user-supplied comparison function to determine the order.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> Resets array's internal pointer to the first element.

Parameters

`array`

The input array.

`callback`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.
8.0.0	If `callback` expects a parameter to be passed by reference, this function will now emit an `E_WARNING`.

Examples

Example #1 uksort() example

```php
<?php
function cmp($a, $b)
{
    $a = preg_replace('@^(a|an|the) @', '', $a);
    $b = preg_replace('@^(a|an|the) @', '', $b);
    return strcasecmp($a, $b);
}

$a = array("John" => 1, "the Earth" => 2, "an apple" => 3, "a banana" => 4);

uksort($a, "cmp");

foreach ($a as $key => $value) {
    echo "$key: $value\n";
}
?>
```

The above example will output:

```
an apple: 3
a banana: 4
the Earth: 2
John: 1
```

See Also

- usort() - Sort an array by values using a user-defined comparison function
- uasort() - Sort an array with a user-defined comparison function and maintain index association
- The comparison of array sorting functions

usort

usort

(PHP 4, PHP 5, PHP 7, PHP 8)

usort — Sort an array by values using a user-defined comparison function

Description

function **usort**(array `&$array`, callable `$callback`): true

Sorts `array` in place by values using a user-supplied comparison function to determine the order.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

> **Note** :
>
> This function assigns new keys to the elements in `array`. It will remove any existing keys that may have been assigned, rather than just reordering the keys.

Parameters

`array`

The input array.

`callback`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Always returns **`true`**.

Changelog

Version	Description
8.2.0	The return type is **`true`** now; previously, it was bool.
8.0.0	If `callback` expects a parameter to be passed by reference, this function will now emit an **`E_WARNING`**.

Examples

Example #1 **usort()** example

```php
<?php
function cmp($a, $b)
{
    if ($a == $b) {
        return 0;
    }
    return ($a < $b) ? -1 : 1;
}

$a = array(3, 2, 5, 6, 1);

usort($a, "cmp");

foreach ($a as $key => $value) {
    echo "$key: $value\n";
}
?>
```

The above example will output:

```
0: 1
1: 2
2: 3
3: 5
4: 6
```

The spaceship operator may be used to simplify the internal comparison even further.

```php
<?php
function cmp($a, $b)
{
    return $a <=> $b;
}

$a = array(3, 2, 5, 6, 1);

usort($a, "cmp");

foreach ($a as $key => $value) {
    echo "$key: $value\n";
}
?>
```

> **Note** :
>
> Obviously in this trivial case the sort() function would be more appropriate.

Example #2 usort() example using multi-dimensional array

```php
<?php
function cmp($a, $b)
{
    return strcmp($a["fruit"], $b["fruit"]);
}

$fruits[0]["fruit"] = "lemons";
$fruits[1]["fruit"] = "apples";
$fruits[2]["fruit"] = "grapes";

usort($fruits, "cmp");

foreach ($fruits as $key => $value) {
    echo "\$fruits[$key]: " . $value["fruit"] . "\n";
}
?>
```

The above example will output:

```
$fruits[0]: apples
$fruits[1]: grapes
$fruits[2]: lemons
```

When sorting a multi-dimensional array, *$a* and *$b* contain references to the first index of the array.

Example #3 usort() example using a member function of an object

```php
<?php
class TestObj {
    public string $name;

    function __construct($name)
    {
        $this->name = $name;
    }

    /* This is the static comparing function: */
    static function cmp_obj($a, $b)
    {
        return strtolower($a->name) <=> strtolower($b->name);
    }
}

$a[] = new TestObj("c");
$a[] = new TestObj("b");
$a[] = new TestObj("d");

usort($a, [TestObj::class, "cmp_obj"]);

foreach ($a as $item) {
    echo $item->name . "\n";
}
?>
```

The above example will output:

```
b
c
d
```

Example #4 usort() example using a closure to sort a multi-dimensional array

```php
<?php
$array[0] = array('key_a' => 'z', 'key_b' => 'c');
$array[1] = array('key_a' => 'x', 'key_b' => 'b');
$array[2] = array('key_a' => 'y', 'key_b' => 'a');

function build_sorter($key) {
    return function ($a, $b) use ($key) {
        return strnatcmp($a[$key], $b[$key]);
    };
}

usort($array, build_sorter('key_b'));

foreach ($array as $item) {
    echo $item['key_a'] . ', ' . $item['key_b'] . "\n";
}
?>
```

The above example will output:

```
y, a
x, b
z, c
```

Example #5 usort() example using the spaceship operator

The spaceship operator allows for straightforward comparison of compound values across multiple axes. The following example will sort `$people` by last name, then by first name if the last name matches.

```php
<?php
$people[0] = ['first' => 'Adam', 'last' => 'West'];
$people[1] = ['first' => 'Alec', 'last' => 'Baldwin'];
$people[2] = ['first' => 'Adam', 'last' => 'Baldwin'];

function sorter(array $a, array $b) {
    return [$a['last'], $a['first']] <=> [$b['last'], $b['first']];
}

usort($people, 'sorter');

foreach ($people as $person) {
    print $person['last'] . ', ' . $person['first'] . PHP_EOL;
}
?>
```

The above example will output:

```
Baldwin, Adam
Baldwin, Alec
West, Adam
```

See Also

- uasort() - Sort an array with a user-defined comparison function and maintain index association
- uksort() - Sort an array by keys using a user-defined comparison function
- The comparison of array sorting functions

Mathematical Functions

Introduction

These math functions will only handle values within the range of the int and float types. To handle larger numbers, take a look at the arbitrary precision math or GNU Multiple Precision functions functions.

See Also

- arithmetic operators
- Arbitrary precision math functions
- GNU Multiple Precision functions

Predefined Constants

The constants below are always available as part of the PHP core.

Mathematical Constants

`M_PI` (float)

Approximation of the number π (pi). (`3.1415926535897932384`6).

`M_E` (float)

Approximation of Euler's number `e` (`2.7182818284590452354`).

`M_LOG2E` (float)

Approximation of $\log_2(e)$ (`1.4426950408889634074`).

`M_LOG10E` (float)

Approximation of $\log_{10}(e)$ (`0.43429448190325182765`).

`M_LN2` (float)

Approximation of `ln(2)` (`0.69314718055994530942`).

`M_LN10` (float)

Approximation of `ln(10)` (`2.3025850929940456840`2).

`M_PI_2` (float)

Approximation of `π/2` (`1.5707963267948966192`3).

`M_PI_4` (float)

Approximation of `π/4` (`0.78539816339744830962`).

`M_1_PI` (float)

Approximation of `1/π` (`0.31830988618379067154`).

`M_2_PI` (float)

Approximation of `2/π` (`0.63661977236758134308`).

`M_SQRTPI` (float)

Approximation of `sqrt(π)` (`1.77245385090551602729`).

`M_2_SQRTPI` (float)

Approximation of `2/sqrt(π)` (`1.12837916709551257390`).

`M_SQRT2` (float)

Approximation of `sqrt(2)` (`1.41421356237309504880`).

`M_SQRT3` (float)

Approximation of `sqrt(3)` (`1.73205080756887729352`).

`M_SQRT1_2` (float)

Approximation of `1/sqrt(2)` (`0.70710678118654752440`).

`M_LNPI` (float)

Approximation of `ln(π)` (`1.14472988584940017414`).

`M_EULER` (float)

Approximation of Euler's constant γ (`0.57721566490153286061`).

IEEE 754 floating point constants

`NAN` (float)

Not A Number

`INF` (float)

Infinity

Rounding constants

> **Note** :
>
> As of PHP 8.4.0, it is recommended to use the RoundingMode enum instead.

`PHP_ROUND_HALF_UP` (int)

Rounding half away from zero.

`PHP_ROUND_HALF_DOWN` (int)

Rounding half toward zero.

`PHP_ROUND_HALF_EVEN` (int)

Round halves to even numbers

`PHP_ROUND_HALF_ODD` (int)

Round halves to odd numbers

RoundingMode

(PHP 8 >= 8.4.0)

Introduction

The **RoundingMode** enum is used to specify how rounding should be performed for round(), bcround(), and BcMath\Number::round().

Enum synopsis

enum **RoundingMode**
{
case HalfAwayFromZero ; // Round to the nearest integer. If the decimal part is 5, round to the integer with the larger absolute value.

case HalfTowardsZero ; // Round to the nearest integer. If the decimal part is 5, round to the integer with the smaller absolute value.

case HalfEven ; // Round to the nearest integer. If the decimal part is 5, round to the even integer.

case HalfOdd ; // Round to the nearest integer. If the decimal part is 5, round to the odd integer.

case TowardsZero ; // Round to the nearest integer with a smaller or equal absolute value.

case AwayFromZero ; // Round to the nearest integer with a greater or equal absolute value.

case NegativeInfinity ; // Round to the largest integer that is smaller or equal.

case PositiveInfinity ; // Round to the smallest integer that is greater or equal.

}

Math Functions

Table of Contents

abs

abs

(PHP 4, PHP 5, PHP 7, PHP 8)

abs — Absolute value

Description

function **abs**(int|float `$num`): int|float

Returns the absolute value of `num`.

Parameters

`num`

The numeric value to process

Return Values

The absolute value of `num`. If the argument `num` is of type float, the return type is also float, otherwise it is int (as float usually has a bigger value range than int).

Changelog

Version	Description
8.0.0	`num` no longer accepts internal objects which support numeric conversion.

Examples

Example #1 abs() example

```
<?php
var_dump(abs(-4.2));
var_dump(abs(5));
var_dump(abs(-5));
?>
```

The above example will output:

```
float(4.2)
int(5)
int(5)
```

See Also

- gmp_abs() - Absolute value
- gmp_sign() - Sign of number

acos

acos

(PHP 4, PHP 5, PHP 7, PHP 8)

acos — Arc cosine

Description

function **acos**(float `$num`): float

Returns the arc cosine of `num` in radians. **acos()** is the inverse function of cos(), which means that `$num == cos(acos($num))` for every value of `num` that is in the domain of **acos()**.

Parameters

`num`

The argument to process

Return Values

The arc cosine of `num` in radians.

See Also

- cos() - Cosine
- acosh() - Inverse hyperbolic cosine
- asin() - Arc sine
- atan() - Arc tangent

acosh

acosh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

acosh — Inverse hyperbolic cosine

Description

function **acosh**(float `$num`): float

Returns the inverse hyperbolic cosine of `num`, i.e. the value whose hyperbolic cosine is `num`.

Parameters

`num`

The value to process

Return Values

The inverse hyperbolic cosine of `num`

See Also

- cosh() - Hyperbolic cosine
- acos() - Arc cosine
- asinh() - Inverse hyperbolic sine
- atanh() - Inverse hyperbolic tangent

asin

asin

(PHP 4, PHP 5, PHP 7, PHP 8)

asin — Arc sine

Description

function **asin**(float $num): float

Returns the arc sine of num in radians. **asin()** is the inverse function of sin(), which means that $num == sin(asin($num)) for every value of num that is in the domain of **asin()**.

Parameters

num

The argument to process

Return Values

The arc sine of num in radians

See Also

- sin() - Sine
- asinh() - Inverse hyperbolic sine
- acos() - Arc cosine
- atan() - Arc tangent

asinh

asinh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

asinh — Inverse hyperbolic sine

Description

function **asinh**(float $num): float

Returns the inverse hyperbolic sine of num, i.e. the value whose hyperbolic sine is num.

Parameters

num

The argument to process

Return Values

The inverse hyperbolic sine of num

See Also

- sinh() - Hyperbolic sine
- asin() - Arc sine
- acosh() - Inverse hyperbolic cosine
- atanh() - Inverse hyperbolic tangent

atan

atan

(PHP 4, PHP 5, PHP 7, PHP 8)

atan — Arc tangent

Description

function **atan**(float $num): float

Returns the arc tangent of `num` in radians. **atan()** is the inverse function of tan(), which means that `$num == tan(atan($num))` for every value of `num` that is in the domain of **atan()**.

Parameters

`num`

The argument to process

Return Values

The arc tangent of `num` in radians.

See Also

- tan() - Tangent
- atanh() - Inverse hyperbolic tangent
- asin() - Arc sine
- acos() - Arc cosine

atan2

atan2

(PHP 4, PHP 5, PHP 7, PHP 8)

atan2 — Arc tangent of two variables

Description

function **atan2**(float `$y`, float `$x`): float

This function calculates the arc tangent of the two variables `x` and `y`. It is similar to calculating the arc tangent of `y / x`, except that the signs of both arguments are used to determine the quadrant of the result.

The function returns the result in radians, which is between -PI and PI (inclusive).

Parameters

y

Dividend parameter

x

Divisor parameter

Return Values

The arc tangent of `y/x` in radians.

See Also

- atan() - Arc tangent

atanh

atanh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

atanh — Inverse hyperbolic tangent

Description

function **atanh**(float `$num`): float

Returns the inverse hyperbolic tangent of `num`, i.e. the value whose hyperbolic tangent is `num`.

Parameters

`num`

The argument to process

Return Values

Inverse hyperbolic tangent of `num`

See Also

- tanh() - Hyperbolic tangent
- atan() - Arc tangent
- asinh() - Inverse hyperbolic sine
- acosh() - Inverse hyperbolic cosine

base_convert

base_convert

(PHP 4, PHP 5, PHP 7, PHP 8)

base_convert — Convert a number between arbitrary bases

Description

function **base_convert**(string `$num`, int `$from_base`, int `$to_base`): string

Returns a string containing `num` represented in base `to_base`. The base in which `num` is given is specified in `from_base`. Both `from_base` and `to_base` have to be between 2 and 36, inclusive. Digits in numbers with a base higher than 10 will be represented with the letters a-z, with a meaning 10, b meaning 11 and z meaning 35. The case of the letters doesn't matter, i.e. `num` is interpreted case-insensitively.

> **Warning**
>
> **base_convert()** may lose precision on large numbers due to properties related to the internal float type used. Please see the Floating point numbers section in the manual for more specific information and limitations.

Parameters

`num`

The number to convert. Any invalid characters in `num` are silently ignored. As of PHP 7.4.0 supplying any invalid characters is deprecated.

`from_base`

The base `num` is in

`to_base`

The base to convert `num` to

Return Values

`num` converted to base `to_base`

Changelog

Version	Description
7.4.0	Passing invalid characters will now generate a deprecation notice. The result will still be computed as if the invalid characters did not exist.

Examples

Example #1 base_convert() example

```
<?php
$hexadecimal = 'a37334';
```

```
echo base_convert($hexadecimal, 16, 2);
?>
```

The above example will output:

```
10100011011100110100
```

See Also

- intval() - Get the integer value of a variable

bindec

bindec

(PHP 4, PHP 5, PHP 7, PHP 8)

bindec — Binary to decimal

Description

function **bindec**(string `$binary_string`): int|float

Returns the decimal equivalent of the binary number represented by the `binary_string` argument.

bindec() converts a binary number to an int or, if needed for size reasons, float.

bindec() interprets all `binary_string` values as unsigned integers. This is because **bindec()** sees the most significant bit as another order of magnitude rather than as the sign bit.

Parameters

`binary_string`

The binary string to convert. Any invalid characters in `binary_string` are silently ignored. As of PHP 7.4.0 supplying any invalid characters is deprecated.

> **Warning**
>
> The parameter must be a string. Using other data types will produce unexpected results.

Return Values

The decimal value of `binary_string`

Changelog

Version	Description
7.4.0	Passing invalid characters will now generate a deprecation notice. The result will still be computed as if the invalid characters did not exist.

Examples

Example #1 bindec() example

```
<?php
echo bindec('110011') . "\n";
echo bindec('000110011') . "\n";

echo bindec('111');
?>
```

The above example will output:

```
51
51
7
```

Example #2 bindec() interprets input as unsigned integers

```php
<?php
/*
 * The lesson from this example is in the output
 * rather than the PHP code itself.
 */

$magnitude_lower = pow(2, (PHP_INT_SIZE * 8) - 2);
p($magnitude_lower - 1);
p($magnitude_lower, 'See the rollover?  Watch it next time around...');

p(PHP_INT_MAX, 'PHP_INT_MAX');
p(~PHP_INT_MAX, 'interpreted to be one more than PHP_INT_MAX');

if (PHP_INT_SIZE == 4) {
    $note = 'interpreted to be the largest unsigned integer';
} else {
    $note = 'interpreted to be the largest unsigned integer
              (18446744073709551615) but skewed by float precision';
}
p(-1, $note);


function p($input, $note = '') {
    echo "input:        $input\n";

    $format = '%0' . (PHP_INT_SIZE * 8) . 'b';
    $bin = sprintf($format, $input);
    echo "binary:       $bin\n";

    ini_set('precision', 20);  // For readability on 64 bit boxes.
    $dec = bindec($bin);
    echo 'bindec():     ' . $dec . "\n";

    if ($note) {
        echo "NOTE:         $note\n";
    }

    echo "\n";
}
?>
```

Output of the above example on 32 bit machines:

```
input:        1073741823
binary:       00111111111111111111111111111111
bindec():     1073741823

input:        1073741824
binary:       01000000000000000000000000000000
bindec():     1073741824
NOTE:         See the rollover?  Watch it next time around...

input:        2147483647
binary:       01111111111111111111111111111111
bindec():     2147483647
```

```
NOTE:         PHP_INT_MAX

input:        -2147483648
binary:       10000000000000000000000000000000
bindec():     2147483648
NOTE:         interpreted to be one more than PHP_INT_MAX

input:        -1
binary:       11111111111111111111111111111111
bindec():     4294967295
NOTE:         interpreted to be the largest unsigned integer
```

Output of the above example on 64 bit machines:

```
input:        4611686018427387903
binary:       0011111111111111111111111111111111111111111111111111111111111111
bindec():     4611686018427387903

input:        4611686018427387904
binary:       0100000000000000000000000000000000000000000000000000000000000000
bindec():     4611686018427387904
NOTE:         See the rollover?  Watch it next time around...

input:        9223372036854775807
binary:       0111111111111111111111111111111111111111111111111111111111111111
bindec():     9223372036854775807
NOTE:         PHP_INT_MAX

input:        -9223372036854775808
binary:       1000000000000000000000000000000000000000000000000000000000000000
bindec():     9223372036854775808
NOTE:         interpreted to be one more than PHP_INT_MAX

input:        -1
binary:       1111111111111111111111111111111111111111111111111111111111111111
bindec():     18446744073709551616
NOTE:         interpreted to be the largest unsigned integer
              (18446744073709551615) but skewed by float precision
```

Notes

> **Note** :
>
> The function can convert numbers that are too large to fit into the platforms int type, larger values are returned as float in that case.

See Also

- decbin() - Decimal to binary
- octdec() - Octal to decimal
- hexdec() - Hexadecimal to decimal
- base_convert() - Convert a number between arbitrary bases

ceil

ceil

(PHP 4, PHP 5, PHP 7, PHP 8)

ceil — Round fractions up

Description

function **ceil**(int|float $num): float

Returns the next highest integer value by rounding up num if necessary.

Parameters

num

The value to round

Return Values

num rounded up to the next highest integer. The return value of **ceil()** is still of type float as the value range of float is usually bigger than that of int.

Changelog

Version	Description
8.0.0	num no longer accepts internal objects which support numeric conversion.

Examples

Example #1 ceil() example

```
<?php
echo ceil(4.3), PHP_EOL;    // 5
echo ceil(9.999), PHP_EOL;  // 10
echo ceil(-3.14), PHP_EOL;  // -3
?>
```

See Also

- floor() - Round fractions down
- round() - Rounds a float

cos

cos

(PHP 4, PHP 5, PHP 7, PHP 8)

cos — Cosine

Description

function **cos**(float $num): float

cos() returns the cosine of the num parameter. The num parameter is in radians.

Parameters

num

An angle in radians

Return Values

The cosine of num

Examples

Example #1 cos() example

```
<?php

echo cos(M_PI); // -1

?>
```

See Also

- acos() - Arc cosine
- sin() - Sine
- tan() - Tangent
- deg2rad() - Converts the number in degrees to the radian equivalent

cosh

cosh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

cosh — Hyperbolic cosine

Description

function **cosh**(float `$num`): float

Returns the hyperbolic cosine of `num`, defined as `(exp($num) + exp(-$num))/2`.

Parameters

`num`

The argument to process

Return Values

The hyperbolic cosine of `num`

See Also

- cos() - Cosine
- acosh() - Inverse hyperbolic cosine
- sinh() - Hyperbolic sine
- tanh() - Hyperbolic tangent

decbin

decbin

(PHP 4, PHP 5, PHP 7, PHP 8)

decbin — Decimal to binary

Description

function **decbin**(int `$num`): string

Returns a string containing a binary representation of the given `num` argument.

Parameters

`num`

Decimal value to convert

Range of inputs on 32-bit machines

positive `num`	**negative** `num`	**return value**
0		0
1		1
2		10
... normal progression ...		
2147483646		11111111111111111111111111111110
2147483647 (largest signed integer)		11111111111111111111111111111111 (31 1's)

positive num	negative num	return value
2147483648	-2147483648	10000000000000000000000000000000
... normal progression ...		
4294967294	-2	11111111111111111111111111111110
4294967295 (largest unsigned integer)	-1	11111111111111111111111111111111 (32 1's)

Range of inputs on 64-bit machines

positive num	negative num	return value
0		0
1		1
2		10
... normal progression ...		
9223372036854775806		111111111111111111111111111111111111111111111111111111111111110
9223372036854775807 (largest signed integer)		111111111111111111111111111111111111111111111111111111111111111 (63 1's)
	-9223372036854775808	1000000000000000000000000000000000000000000000000000000000000000
... normal progression ...		
	-2	111111111111111111111111111111111111111111111111111111111111110
	-1	111111111111111111111111111111111111111111111111111111111111111 (64 1's)

Return Values

Binary string representation of `num`

Examples

Example #1 **decbin()** example

```
<?php
echo decbin(12) . "\n";
echo decbin(26);
?>
```

The above example will output:

```
1100
11010
```

See Also

- bindec() - Binary to decimal
- decoct() - Decimal to octal
- dechex() - Decimal to hexadecimal
- base_convert() - Convert a number between arbitrary bases
- printf() - Output a formatted string, using `%b`, `%032b` or `%064b` as the format
- sprintf() - Return a formatted string, using `%b`, `%032b` or `%064b` as the format

dechex

dechex

(PHP 4, PHP 5, PHP 7, PHP 8)

dechex — Decimal to hexadecimal

Description

function **dechex**(int `$num`): string

Returns a string containing a hexadecimal representation of the given unsigned `num` argument.

The largest number that can be converted is `PHP_INT_MAX * 2 + 1` (or `-1`): on 32-bit platforms, this will be `4294967295` in decimal, which results in **dechex()** returning `ffffffff`.

Parameters

`num`

The decimal value to convert.

As PHP's int type is signed, but **dechex()** deals with unsigned integers, negative integers will be treated as though they were unsigned.

Return Values

Hexadecimal string representation of `num`.

Examples

Example #1 dechex() example

```php
<?php
echo dechex(10) . "\n";
echo dechex(47);
?>
```

The above example will output:

```
a
2f
```

Example #2 dechex() example with large integers

```php
<?php
// The output below assumes a 32-bit platform.
// Note that the output is the same for all values.
echo dechex(-1)."\n";
echo dechex(PHP_INT_MAX * 2 + 1)."\n";
echo dechex(pow(2, 32) - 1)."\n";
?>
```

The above example will output:

```
ffffffff
ffffffff
ffffffff
```

See Also

- hexdec() - Hexadecimal to decimal
- decbin() - Decimal to binary
- decoct() - Decimal to octal
- base_convert() - Convert a number between arbitrary bases

decoct

decoct

(PHP 4, PHP 5, PHP 7, PHP 8)

decoct — Decimal to octal

Description

function **decoct**(int `$num`): string

Returns a string containing an octal representation of the given `num` argument. The largest number that can be converted depends on the platform in use. For 32-bit platforms this is usually `4294967295` in decimal resulting in `37777777777`. For 64-bit platforms this is usually `9223372036854775807` in decimal resulting in `777777777777777777777`.

Parameters

`num`

Decimal value to convert

Return Values

Octal string representation of `num`

Examples

Example #1 decoct() example

```
<?php
echo decoct(15) . "\n";
echo decoct(264);
?>
```

The above example will output:

```
17
410
```

See Also

- octdec() - Octal to decimal
- decbin() - Decimal to binary
- dechex() - Decimal to hexadecimal
- base_convert() - Convert a number between arbitrary bases

deg2rad

deg2rad

(PHP 4, PHP 5, PHP 7, PHP 8)

deg2rad — Converts the number in degrees to the radian equivalent

Description

function **deg2rad**(float `$num`): float

This function converts `num` from degrees to the radian equivalent.

Parameters

`num`

Angular value in degrees

Return Values

The radian equivalent of `num`

Examples

Example #1 deg2rad() example

```
<?php

echo deg2rad(45), PHP_EOL; // 0.785398163397
var_dump(deg2rad(45) === M_PI_4); // bool(true)

?>
```

See Also

- rad2deg() - Converts the radian number to the equivalent number in degrees

exp

exp

(PHP 4, PHP 5, PHP 7, PHP 8)

exp — Calculates the exponent of **e**

Description

function **exp**(float `$num`): float

Returns **e** raised to the power of `num`.

> **Note** :
>
> '**e**' is the base of the natural system of logarithms, or approximately 2.718282.

Parameters

`num`

The argument to process

Return Values

'e' raised to the power of `num`

Examples

Example #1 exp() example

```
<?php
echo exp(12), PHP_EOL;
echo exp(5.7);
?>
```

The above example will output:

```
162754.791419
298.86740096706
```

See Also

- log() - Natural logarithm
- pow() - Exponential expression

expm1

expm1

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

expm1 — Returns `exp($num) - 1`, computed in a way that is accurate even when the value of number is close to zero

Description

function **expm1**(float `$num`): float

expm1() returns the equivalent to `exp($num) - 1` computed in a way that is accurate even if the value of `num` is near zero, a case where `exp($num) - 1` would be inaccurate due to subtraction of two numbers that are nearly equal.

Parameters

`num`

The argument to process

Return Values

`e` to the power of `num`, minus one.

See Also

- log1p() - Returns log(1 + number), computed in a way that is accurate even when the value of number is close to zero
- exp() - Calculates the exponent of e

fdiv

fdiv

(PHP 8)

fdiv — Divides two numbers, according to IEEE 754

Description

function **fdiv**(float `$num1`, float `$num2`): float

Returns the floating point result of dividing the `num1` by the `num2`. If the `num2` is zero, then one of `INF`, `-INF`, or `NAN` will be returned.

Note that in comparisons, `NAN` will never be equal (`==`) or identical (`===`) to any value, including itself.

Parameters

`num1`

The dividend (numerator)

`num2`

The divisor

Return Values

The floating point result of `num1/num2`

Examples

Example #1 Using fdiv()

```
<?php
var_dump(fdiv(5.7, 1.3)); // float(4.384615384615385)
```

```
var_dump(fdiv(4, 2)); // float(2)
var_dump(fdiv(1.0, 0.0)); // float(INF)
var_dump(fdiv(-1.0, 0.0)); // float(-INF)
var_dump(fdiv(0.0, 0.0)); // float(NAN)
?>
```

See Also

- Division operator `/`
- intdiv() - Integer division - Integer division
- fmod() - Returns the floating point remainder (modulo) of the division of the arguments
- fpow() - Raise one number to the power of another, according to IEEE 754

floor

floor

(PHP 4, PHP 5, PHP 7, PHP 8)

floor — Round fractions down

Description

function **floor**(int|float `$num`): float

Returns the next lowest integer value (as float) by rounding down `num` if necessary.

Parameters

`num`

The numeric value to round

Return Values

`num` rounded to the next lowest integer. The return value of **floor()** is still of type float.

Changelog

Version	Description
8.0.0	`num` no longer accepts internal objects which support numeric conversion.

Examples

Example #1 floor() example

```
<?php
echo floor(4.3), PHP_EOL;   // 4
echo floor(9.999), PHP_EOL; // 9
echo floor(-3.14), PHP_EOL; // -4
?>
```

See Also

- ceil() - Round fractions up
- round() - Rounds a float

fmod

fmod

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

fmod — Returns the floating point remainder (modulo) of the division of the arguments

Description

function **fmod**(float `$num1`, float `$num2`): float

Returns the floating point remainder of dividing the dividend (`num1`) by the divisor (`num2`). The remainder (*r*) is defined as: num1 = i * num2 + r, for some integer *i*. If `num2` is non-zero, *r* has the same sign as `num1` and a magnitude less than the magnitude of `num2`.

Parameters

`num1`

The dividend

`num2`

The divisor

Return Values

The floating point remainder of `num1`/`num2`. **`NAN`** (float) if the second argument is 0.

Examples

Example #1 Using **fmod()**

```
<?php
$x = 5.7;
$y = 1.3;
$r = fmod($x, $y);
// $r equals 0.5, because 4 * 1.3 + 0.5 = 5.7

var_dump($x, $y, $r);
?>
```

See Also

- `/` - Floating-point division
- `%` - Integer modulus
- intdiv() - Integer division - Integer division

fpow

fpow

(PHP 8 >= 8.4.0)

fpow — Raise one number to the power of another, according to IEEE 754

Description

function **fpow**(float `$num`, float `$exponent`): float

Returns the floating point result of raising `num` to the power of `exponent`. If `num` is zero and `exponent` is less than zero, then **`INF`** is returned.

Parameters

`num`

The base to use.

`exponent`

The exponent.

Return Values

Returns a float corresponding to $num^{\$exponent}$.

Examples

Example #1 **fpow()** example

```
<?php
var_dump(fpow(10, 2));
var_dump(fpow(0, -3));
```

```php
var_dump(fpow(-1, 5.5));
?>
```

The above example will output:

```
float(100)
float(INF)
float(NAN)
```

See Also

- Exponentiation operator **
- pow() - Exponential expression
- fdiv() - Divides two numbers, according to IEEE 754
- fmod() - Returns the floating point remainder (modulo) of the division of the arguments

hexdec

hexdec

(PHP 4, PHP 5, PHP 7, PHP 8)

hexdec — Hexadecimal to decimal

Description

function **hexdec**(string `$hex_string`): int|float

Returns the decimal equivalent of the hexadecimal number represented by the `hex_string` argument. **hexdec()** converts a hexadecimal string to a decimal number.

hexdec() will ignore any non-hexadecimal characters it encounters. As of PHP 7.4.0 supplying any invalid characters is deprecated.

Parameters

`hex_string`

The hexadecimal string to convert

Return Values

The decimal representation of `hex_string`

Changelog

Version	Description
7.4.0	Passing invalid characters will now generate a deprecation notice. The result will still be computed as if the invalid characters did not exist.

Examples

Example #1 **hexdec()** example

```php
<?php
var_dump(hexdec("ee")); // prints "int(238)"
var_dump(hexdec("a0")); // prints "int(160)"
?>
```

Example #2 **hexdec()** with Invalid Characters

```php
<?php
var_dump(hexdec("See"));  // print "int(238)"
```

```
var_dump(hexdec("that")); // print "int(10)"
?>
```

Notes

> **Note** :
>
> The function can convert numbers that are too large to fit into the platforms int type, larger values are returned as float in that case.

See Also

- dechex() - Decimal to hexadecimal
- bindec() - Binary to decimal
- octdec() - Octal to decimal
- base_convert() - Convert a number between arbitrary bases

hypot

hypot

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

hypot — Calculate the length of the hypotenuse of a right-angle triangle

Description

function **hypot**(float $x, float $y): float

hypot() returns the length of the hypotenuse of a right-angle triangle with sides of length x and y, or the distance of the point (x, y) from the origin. This is equivalent to `sqrt($x*$x + $y*$y)`.

Parameters

x

Length of first side

y

Length of second side

Return Values

Calculated length of the hypotenuse

intdiv

intdiv

(PHP 7, PHP 8)

intdiv — Integer division

Description

function **intdiv**(int $num1, int $num2): int

Returns the integer quotient of the division of num1 by num2.

Parameters

num1

Number to be divided.

num2

Number which divides the num1.

Return Values

The integer quotient of the division of `num1` by `num2`.

Errors/Exceptions

If `num2` is `0`, a DivisionByZeroError exception is thrown. If the `num1` is **`PHP_INT_MIN`** and the `num2` is `-1`, then an ArithmeticError exception is thrown.

Examples

Example #1 intdiv() example

```
<?php
var_dump(intdiv(3, 2));
var_dump(intdiv(-3, 2));
var_dump(intdiv(3, -2));
var_dump(intdiv(-3, -2));
var_dump(intdiv(PHP_INT_MAX, PHP_INT_MAX));
var_dump(intdiv(PHP_INT_MIN, PHP_INT_MIN));
?>
```

The above example will output:

```
int(1)
int(-1)
int(-1)
int(1)
int(1)
int(1)
```

Example #2 intdiv() Example With Invalid Divisor

```
<?php
try {
    intdiv(PHP_INT_MIN, -1);
} catch (Error $e) {
    echo get_class($e), ': ', $e->getMessage(), PHP_EOL;
}

try {
    intdiv(1, 0);
} catch (Error $e) {
    echo get_class($e), ': ', $e->getMessage(), PHP_EOL;
}
?>
```

The above example will output:

```
ArithmeticError: Division of PHP_INT_MIN by -1 is not an integer
DivisionByZeroError: Division by zero
```

See Also

- `/` - Floating-point division
- `%` - Integer modulus
- fmod() - Returns the floating point remainder (modulo) of the division of the arguments - Floating-point modulo

is_finite

is_finite

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_finite — Checks whether a float is finite

Description

function **is_finite**(float $num): bool

Returns whether the given num is a finite float.

A finite float is neither NAN (is_nan()), nor infinite (is_infinite()).

Parameters

num

The float to check

Return Values

true if num is none of NAN, INF, -INF, else false.

Examples

Example #1 is_finite() example

```
<?php
$float = 1.2345;
var_dump($float, is_finite($float));

$nan = sqrt(-1);
var_dump($nan, is_finite($nan));

$inf = 1e308 * 2;
var_dump($inf, is_finite($inf));
?>
```

The above example will output:

```
float(1.2345)
bool(true)
float(NAN)
bool(false)
float(INF)
bool(false)
```

See Also

- is_infinite() - Checks whether a float is infinite
- is_nan() - Checks whether a float is NAN

is_infinite

is_infinite

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_infinite — Checks whether a float is infinite

Description

function **is_infinite**(float $num): bool

Returns whether the given num is either INF or -INF.

Parameters

num

The float to check

Return Values

true if num is either INF or -INF, else false.

Examples

Example #1 is_infinite() example

```
<?php
$inf = 1e308 * 2;

var_dump($inf, is_infinite($inf));

$negative_inf = -$inf;

var_dump($negative_inf, is_infinite($negative_inf));
?>
```

The above example will output:

```
float(INF)
bool(true)
float(-INF)
bool(true)
```

See Also

- is_finite() - Checks whether a float is finite
- is_nan() - Checks whether a float is NAN

is_nan

is_nan

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_nan — Checks whether a float is NAN

Description

function **is_nan**(float $num): bool

Returns whether the given num is NAN ("Not A Number").

NAN is returned from mathematical operations that are undefined, for example when passing parameters outside of function's input domain. The square root (sqrt()) is only defined for positive numbers, passing a negative number will result in NAN. Other examples of operations returning NAN are dividing INF by INF and any operation involving an existing NAN value.

> **Note** :
>
> Despite its name of "Not A Number", NAN is a valid value of type float.

> **Caution**

> `NAN` does not compare equal to `NAN`. To check whether a float is `NAN`, **is_nan()** must be used. Checking `$float === NAN` will not work.

Parameters

`num`

The float to check

Return Values

`true` if `num` is `NAN`, else `false`.

Examples

Example #1 is_nan() example

```
<?php
$nan = sqrt(-1);

var_dump($nan, is_nan($nan));
?>
```

The above example will output:

```
float(NAN)
bool(true)
```

See Also

- is_finite() - Checks whether a float is finite
- is_infinite() - Checks whether a float is infinite

log

log

(PHP 4, PHP 5, PHP 7, PHP 8)

log — Natural logarithm

Description

function **log**(float `$num`, float `$base` = `M_E`): float

If the optional `base` parameter is specified, **log()** returns $\log_{base}$ `num`, otherwise **log()** returns the natural logarithm of `num`.

Parameters

`num`

The value to calculate the logarithm for

`base`

The optional logarithmic base to use (defaults to 'e' and so to the natural logarithm).

Return Values

The logarithm of `num` to `base`, if given, or the natural logarithm.

See Also

- log10() - Base-10 logarithm

- exp() - Calculates the exponent of e
- pow() - Exponential expression
- error_log() - Send an error message to the defined error handling routines

log10

log10

(PHP 4, PHP 5, PHP 7, PHP 8)

log10 — Base-10 logarithm

Description

function **log10**(float $num): float

Returns the base-10 logarithm of num.

Parameters

num

The argument to process

Return Values

The base-10 logarithm of num

See Also

- log() - Natural logarithm

log1p

log1p

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

log1p — Returns log(1 + number), computed in a way that is accurate even when the value of number is close to zero

Description

function **log1p**(float $num): float

log1p() returns log(1 + num) computed in a way that is accurate even when the value of num is close to zero. log() might only return log(1) in this case due to lack of precision.

Parameters

num

The argument to process

Return Values

log(1 + num)

See Also

- expm1() - Returns exp($num) - 1, computed in a way that is accurate even when the value of number is close to zero
- log() - Natural logarithm
- log10() - Base-10 logarithm

max

max

(PHP 4, PHP 5, PHP 7, PHP 8)

max — Find highest value

Description

function **max**(mixed `$value`, mixed `...$values`): mixed

Alternative signature (not supported with named arguments):

function **max**(array `$value_array`): mixed

If the first and only parameter is an array, **max()** returns the highest value in that array. If at least two parameters are provided, **max()** returns the biggest of these values.

> **Note** :
>
> Values of different types will be compared using the standard comparison rules. For instance, a non-numeric string will be compared to an int as though it were `0`, but multiple non-numeric string values will be compared alphanumerically. The actual value returned will be of the original type with no conversion applied.

> **Caution**
>
> Be careful when passing arguments of different types because **max()** can produce unpredictable results.

Parameters

`value`

Any comparable value.

`values`

Any comparable values.

`value_array`

An array containing the values.

Return Values

max() returns the parameter value considered "highest" according to standard comparisons. If multiple values of different types evaluate as equal (e.g. `0` and `'abc'`) the first provided to the function will be returned.

Errors/Exceptions

If an empty array is passed, **max()** throws a ValueError.

Changelog

Version	Description
8.0.0	**max()** throws a ValueError on failure now; previously, `false` was returned and an `E_WARNING` error was emitted.
8.0.0	As string to number comparisons were changed, **max()** no longer returns a different value based on the argument order in those cases.

Examples

Example #1 Example uses of max()

```
<?php
echo max(2, 3, 1, 6, 7), PHP_EOL;  // 7
echo max(array(2, 4, 5)), PHP_EOL; // 5

// Here we are comparing -1 < 0, so 'hello' is the highest value
echo max('hello', -1), PHP_EOL;    // hello

// With multiple arrays of different lengths, max returns the longest
$val = max(array(2, 2, 2), array(1, 1, 1, 1)); // array(1, 1, 1, 1)
var_dump($val);

// Multiple arrays of the same length are compared from left to right
```

```php
// so in our example: 2 == 2, but 5 > 4
$val = max(array(2, 4, 8), array(2, 5, 1)); // array(2, 5, 1)
var_dump($val);

// If both an array and non-array are given, the array will be returned
// as comparisons treat arrays as greater than any other value
$val = max('string', array(2, 5, 7), 42);   // array(2, 5, 7)
var_dump($val);

// If one argument is NULL or a boolean, it will be compared against
// other values using the rule FALSE < TRUE regardless of the other types involved
// In the below example, -10 is treated as TRUE in the comparison
$val = max(-10, FALSE); // -10
var_dump($val);

// 0, on the other hand, is treated as FALSE, so is "lower than" TRUE
$val = max(0, TRUE); // TRUE
var_dump($val);
?>
```

See Also

- min() - Find lowest value
- count() - Counts all elements in an array or in a Countable object

min

min

(PHP 4, PHP 5, PHP 7, PHP 8)

min — Find lowest value

Description

function **min**(mixed `$value`, mixed `...$values`): mixed

Alternative signature (not supported with named arguments):

function **min**(array `$value_array`): mixed

If the first and only parameter is an array, **min()** returns the lowest value in that array. If at least two parameters are provided, **min()** returns the smallest of these values.

> **Note** :
>
> Values of different types will be compared using the standard comparison rules. For instance, a non-numeric string will be compared to an int as though it were `0`, but multiple non-numeric string values will be compared alphanumerically. The actual value returned will be of the original type with no conversion applied.

> **Caution**
>
> Be careful when passing arguments of different types because **min()** can produce unpredictable results.

Parameters

`value`

Any comparable value.

`values`

Any comparable values.

`value_array`

An array containing the values.

Return Values

min() returns the parameter value considered "lowest" according to standard comparisons. If multiple values of different types evaluate as equal (e.g. `0` and `'abc'`) the first provided to the function will be returned.

Errors/Exceptions

If an empty array is passed, **min()** throws a ValueError.

Changelog

Version	Description
8.0.0	**min()** throws a ValueError on failure now; previously, `false` was returned and an `E_WARNING` error was emitted.
8.0.0	As string to number comparisons were changed, **min()** no longer returns a different value based on the argument order in those cases.

Examples

Example #1 Example uses of min()

```php
<?php
echo min(2, 3, 1, 6, 7), PHP_EOL;  // 1
echo min(array(2, 4, 5)), PHP_EOL; // 2

// Here we are comparing -1 < 0, so -1 is the lowest value
echo min('hello', -1), PHP_EOL;    // -1

// With multiple arrays of different lengths, min returns the shortest
$val = min(array(2, 2, 2), array(1, 1, 1, 1)); // array(2, 2, 2)
var_dump($val);

// Multiple arrays of the same length are compared from left to right
// so in our example: 2 == 2, but 4 < 5
$val = min(array(2, 4, 8), array(2, 5, 1)); // array(2, 4, 8)
var_dump($val);

// If both an array and non-array are given, the array is never returned
// as comparisons treat arrays as greater than any other value
$val = min('string', array(2, 5, 7), 42);   // string
var_dump($val);

// If one argument is NULL or a boolean, it will be compared against
// other values using the rules FALSE < TRUE and NULL == FALSE regardless of the
// other types involved
// In the below examples, both -10 and 10 are treated as TRUE in the comparison
$val = min(-10, FALSE, 10); // FALSE
var_dump($val);

$val = min(-10, NULL, 10);  // NULL
var_dump($val);

// 0, on the other hand, is treated as FALSE, so is "lower than" TRUE
$val = min(0, TRUE); // 0
var_dump($val);
?>
```

See Also

- max() - Find highest value
- count() - Counts all elements in an array or in a Countable object

octdec

octdec

(PHP 4, PHP 5, PHP 7, PHP 8)

octdec — Octal to decimal

Description

function **octdec**(string `$octal_string`): int|float

Returns the decimal equivalent of the octal number represented by the `octal_string` argument.

Parameters

`octal_string`

The octal string to convert. Any invalid characters in `octal_string` are silently ignored. As of PHP 7.4.0 supplying any invalid characters is deprecated.

Return Values

The decimal representation of `octal_string`

Changelog

Version	Description
7.4.0	Passing invalid characters will now generate a deprecation notice. The result will still be computed as if the invalid characters did not exist.

Examples

Example #1 octdec() example

```php
<?php
echo octdec('77') . "\n";
echo octdec(decoct(45));
?>
```

The above example will output:

```
63
45
```

Notes

> **Note** :
>
> The function can convert numbers that are too large to fit into the platforms int type, larger values are returned as float in that case.

See Also

- decoct() - Decimal to octal
- bindec() - Binary to decimal
- hexdec() - Hexadecimal to decimal
- base_convert() - Convert a number between arbitrary bases

pi

pi

(PHP 4, PHP 5, PHP 7, PHP 8)

pi — Get value of pi

Description

function **pi**(): float

Returns an approximation of pi. Also, you can use the `M_PI` constant which yields identical results to **pi()**.

Parameters

This function has no parameters.

Return Values

The value of pi as float.

Examples

Example #1 pi() example

```
<?php
echo pi(), PHP_EOL; // 3.1415926535898
echo M_PI, PHP_EOL; // 3.1415926535898
?>
```

pow

pow

(PHP 4, PHP 5, PHP 7, PHP 8)

pow — Exponential expression

Description

function **pow**(mixed `$num`, mixed `$exponent`): int|float|object

Returns `num` raised to the power of `exponent`.

> **Note** :
>
> It is possible to use the ** operator instead.

Parameters

`num`

The base to use

`exponent`

The exponent

Return Values

`num` raised to the power of `exponent`. If both arguments are non-negative integers and the result can be represented as an integer, the result will be returned with int type, otherwise it will be returned as a float.

PHP-Extensions may override the behaviour of this operation and make it return an object.

Changelog

Version	Description
8.4.0	Raising `0` to a negative `exponent` is now deprecated.

Examples

Example #1 Some examples of pow()

```
<?php

var_dump(pow(2, 8)); // int(256)
echo pow(-1, 20), PHP_EOL; // 1
echo pow(0, 0), PHP_EOL; // 1
```

```php
echo pow(10, -1), PHP_EOL; // 0.1

echo pow(-1, 5.5), PHP_EOL; // NAN
?>
```

Example #2 Examples of pow() With GMP Extension Object

```php
<?php
var_dump(pow(new GMP("3"), new GMP("2"))); // object(GMP)
?>
```

Notes

> **Note** :
>
> This function will convert all input to a number, even non-scalar values, which could lead to *weird* results.

See Also

- Exponentiation operator **
- fpow() - Raise one number to the power of another, according to IEEE 754
- exp() - Calculates the exponent of e
- sqrt() - Square root
- bcpow() - Raise an arbitrary precision number to another
- gmp_pow() - Raise number into power

rad2deg

rad2deg

(PHP 4, PHP 5, PHP 7, PHP 8)

rad2deg — Converts the radian number to the equivalent number in degrees

Description

function **rad2deg**(float `$num`): float

This function converts `num` from radian to degrees.

Parameters

`num`

A radian value

Return Values

The equivalent of `num` in degrees

Examples

Example #1 rad2deg() example

```php
<?php

echo rad2deg(M_PI_4); // 45

?>
```

See Also

- deg2rad() - Converts the number in degrees to the radian equivalent

round

round

(PHP 4, PHP 5, PHP 7, PHP 8)

round — Rounds a float

Description

function **round**(int|float `$num`, int `$precision` = 0, int|RoundingMode `$mode` = **`RoundingMode::HalfAwayFromZero`**): float

Returns the rounded value of `num` to specified `precision` (number of digits after the decimal point). `precision` can also be negative or zero (default).

Parameters

`num`

The value to round.

`precision`

The optional number of decimal digits to round to.

If the `precision` is positive, `num` is rounded to `precision` significant digits after the decimal point.

If the `precision` is negative, `num` is rounded to `precision` significant digits before the decimal point, i.e. to the nearest multiple of `pow(10, -$precision)`, e.g. for a `precision` of -1 `num` is rounded to tens, for a `precision` of -2 to hundreds, etc.

`mode`

Use RoundingMode or one of the following constants to specify the mode in which rounding occurs.

Constants	Description
`PHP_ROUND_HALF_UP`	Rounds `num` away from zero when it is half way there, making 1.5 into 2 and -1.5 into -2.
`PHP_ROUND_HALF_DOWN`	Rounds `num` towards zero when it is half way there, making 1.5 into 1 and -1.5 into -1.
`PHP_ROUND_HALF_EVEN`	Rounds `num` towards the nearest even value when it is half way there, making both 1.5 and 2.5 into 2.
`PHP_ROUND_HALF_ODD`	Rounds `num` towards the nearest odd value when it is half way there, making 1.5 into 1 and 2.5 into 3.

However, please note that some newly added modes only exist in RoundingMode.

Return Values

The value rounded to the given `precision` as a float.

Errors/Exceptions

The function throws a ValueError if `mode` is invalid. Prior to PHP 8.4.0, an invalid mode would silently default to `PHP_ROUND_HALF_UP`.

Changelog

Version	Description
8.4.0	Four new rounding modes have been added.
8.4.0	Now throws a ValueError if `mode` is invalid.
8.0.0	`num` no longer accepts internal objects which support numeric conversion.

Examples

Example #1 **round()** examples

```
<?php
var_dump(round(3.4));
var_dump(round(3.5));
var_dump(round(3.6));
var_dump(round(3.6, 0));
var_dump(round(5.045, 2));
var_dump(round(5.055, 2));
var_dump(round(345, -2));
var_dump(round(345, -3));
var_dump(round(678, -2));
```

```php
var_dump(round(678, -3));
?>
```

The above example will output:

```
float(3)
float(4)
float(4)
float(4)
float(5.05)
float(5.06)
float(300)
float(0)
float(700)
float(1000)
```

Example #2 How `precision` affects a float

```php
<?php
$number = 135.79;

var_dump(round($number, 3));
var_dump(round($number, 2));
var_dump(round($number, 1));
var_dump(round($number, 0));
var_dump(round($number, -1));
var_dump(round($number, -2));
var_dump(round($number, -3));
?>
```

The above example will output:

```
float(135.79)
float(135.79)
float(135.8)
float(136)
float(140)
float(100)
float(0)
```

Example #3 `mode` examples

```php
<?php
echo 'Rounding modes with 9.5' . PHP_EOL;
var_dump(round(9.5, 0, PHP_ROUND_HALF_UP));
var_dump(round(9.5, 0, PHP_ROUND_HALF_DOWN));
var_dump(round(9.5, 0, PHP_ROUND_HALF_EVEN));
var_dump(round(9.5, 0, PHP_ROUND_HALF_ODD));

echo PHP_EOL;
echo 'Rounding modes with 8.5' . PHP_EOL;
var_dump(round(8.5, 0, PHP_ROUND_HALF_UP));
var_dump(round(8.5, 0, PHP_ROUND_HALF_DOWN));
var_dump(round(8.5, 0, PHP_ROUND_HALF_EVEN));
var_dump(round(8.5, 0, PHP_ROUND_HALF_ODD));
?>
```

The above example will output:

```
Rounding modes with 9.5
float(10)
float(9)
float(10)
float(9)

Rounding modes with 8.5
float(9)
float(8)
float(8)
float(9)
```

Example #4 `mode` with `precision` examples

```php
<?php
echo 'Using PHP_ROUND_HALF_UP with 1 decimal digit precision' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_UP));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_UP));

echo PHP_EOL;
echo 'Using PHP_ROUND_HALF_DOWN with 1 decimal digit precision' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_DOWN));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_DOWN));

echo PHP_EOL;
echo 'Using PHP_ROUND_HALF_EVEN with 1 decimal digit precision' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_EVEN));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_EVEN));

echo PHP_EOL;
echo 'Using PHP_ROUND_HALF_ODD with 1 decimal digit precision' . PHP_EOL;
var_dump(round( 1.55, 1, PHP_ROUND_HALF_ODD));
var_dump(round(-1.55, 1, PHP_ROUND_HALF_ODD));
?>
```

The above example will output:

```
Using PHP_ROUND_HALF_UP with 1 decimal digit precision
float(1.6)
float(-1.6)

Using PHP_ROUND_HALF_DOWN with 1 decimal digit precision
float(1.5)
float(-1.5)

Using PHP_ROUND_HALF_EVEN with 1 decimal digit precision
float(1.6)
float(-1.6)

Using PHP_ROUND_HALF_ODD with 1 decimal digit precision
float(1.5)
float(-1.5)
```

Example #5 Example of using RoundingMode

```php
<?php
foreach (RoundingMode::cases() as $mode) {
    foreach ([
        8.5,
```

```
        9.5,
        -3.5,
    ] as $number) {
        printf("%-17s: %+.17g -> %+.17g\n", $mode->name, $number, round($number, 0, $mode));
    }
    echo "\n";
}
?>
```

The above example will output:

```
HalfAwayFromZero : +8.5 -> +9
HalfAwayFromZero : +9.5 -> +10
HalfAwayFromZero : -3.5 -> -4

HalfTowardsZero  : +8.5 -> +8
HalfTowardsZero  : +9.5 -> +9
HalfTowardsZero  : -3.5 -> -3

HalfEven         : +8.5 -> +8
HalfEven         : +9.5 -> +10
HalfEven         : -3.5 -> -4

HalfOdd          : +8.5 -> +9
HalfOdd          : +9.5 -> +9
HalfOdd          : -3.5 -> -3

TowardsZero      : +8.5 -> +8
TowardsZero      : +9.5 -> +9
TowardsZero      : -3.5 -> -3

AwayFromZero     : +8.5 -> +9
AwayFromZero     : +9.5 -> +10
AwayFromZero     : -3.5 -> -4

NegativeInfinity : +8.5 -> +8
NegativeInfinity : +9.5 -> +9
NegativeInfinity : -3.5 -> -4

PositiveInfinity : +8.5 -> +9
PositiveInfinity : +9.5 -> +10
PositiveInfinity : -3.5 -> -3
```

See Also

- ceil() - Round fractions up
- floor() - Round fractions down
- number_format() - Format a number with grouped thousands

sin

sin

(PHP 4, PHP 5, PHP 7, PHP 8)

sin — Sine

Description

function **sin**(float $num): float

sin() returns the sine of the num parameter. The num parameter is in radians.

Parameters

num

A value in radians

Return Values

The sine of num

Examples

Example #1 sin() example

```
<?php
// Precision depends on your precision directive
echo sin(deg2rad(60)), PHP_EOL;  //  0.866025403 ...
echo sin(60), PHP_EOL;           // -0.304810621 ...
?>
```

See Also

- asin() - Arc sine
- sinh() - Hyperbolic sine
- cos() - Cosine
- tan() - Tangent
- deg2rad() - Converts the number in degrees to the radian equivalent

sinh

sinh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

sinh — Hyperbolic sine

Description

function **sinh**(float $num): float

Returns the hyperbolic sine of num, defined as (exp($num) - exp(-$num))/2.

Parameters

num

The argument to process

Return Values

The hyperbolic sine of num

See Also

- sin() - Sine
- asinh() - Inverse hyperbolic sine
- cosh() - Hyperbolic cosine
- tanh() - Hyperbolic tangent

sqrt

sqrt

(PHP 4, PHP 5, PHP 7, PHP 8)

sqrt — Square root

Description

function **sqrt**(float $num): float

Returns the square root of num.

Parameters

`num`

The argument to process

Return Values

The square root of `num` or the special value **`NAN`** for negative numbers.

Examples

Example #1 sqrt() example

```php
<?php
// Precision depends on your precision directive
echo sqrt(9), PHP_EOL; // 3
echo sqrt(10), PHP_EOL; // 3.16227766 ...
?>
```

See Also

- pow() - Exponential expression
- **`M_SQRTPI`** - `sqrt(pi)`
- **`M_2_SQRTPI`** - `2/sqrt(pi)`
- **`M_SQRT2`** - `sqrt(2)`
- **`M_SQRT3`** - `sqrt(3)`
- **`M_SQRT1_2`** - `1/sqrt(2)`

tan

tan

(PHP 4, PHP 5, PHP 7, PHP 8)

tan — Tangent

Description

function **tan**(float `$num`): float

tan() returns the tangent of the `num` parameter. The `num` parameter is in radians.

Parameters

`num`

The argument to process in radians

Return Values

The tangent of `num`

Examples

Example #1 tan() example

```php
<?php

echo tan(M_PI_4); // 1

?>
```

See Also

- atan() - Arc tangent
- atan2() - Arc tangent of two variables
- sin() - Sine
- cos() - Cosine

- tanh() - Hyperbolic tangent
- deg2rad() - Converts the number in degrees to the radian equivalent

tanh

tanh

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

tanh — Hyperbolic tangent

Description

function **tanh**(float `$num`): float

Returns the hyperbolic tangent of `num`, defined as `sinh($num)/cosh($num)`.

Parameters

`num`

The argument to process

Return Values

The hyperbolic tangent of `num`

See Also

- tan() - Tangent
- atanh() - Inverse hyperbolic tangent
- sinh() - Hyperbolic sine
- cosh() - Hyperbolic cosine

Date and Time

Introduction

The DateTimeImmutable and related classes allow you represent date/time information. The objects can be created by passing in a string presentation of date/time information, or from the current system's time.

A rich set of methods is supplied to modify and format this information as well, including handling timezones and DST transitions.

The date/time features in PHP implements the ISO 8601 calendar, which is a » proleptic Gregorian calendar which implements the current leap-day rules from before the Gregorian calendar was in place, and also includes the year `0` as the year number in between `-1 BCE` and `1 CE`. Leap seconds are not supported.

The date and time information is internally stored as a 64-bit number so all conceivably useful dates (including negative years) are supported. The range is from about 292 billion years in the past to the same in the future.

> **Note** :
>
> The timezones referenced in this section can be found in the List of Supported Timezones.

Installing/Configuring

Table of Contents

Installation

There is no installation needed to use these functions; they are part of the PHP core.

> **Note** : **Getting the latest timezone database**
>
> The latest version of the timezone database can be installed via PECL's » timezonedb.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Date/Time Configuration Options

Name	Default	Changeable	Changelog
date.default_latitude	"31.7667"	`INI_ALL`	
date.default_longitude	"35.2333"	`INI_ALL`	
date.sunrise_zenith	"90.833333"	`INI_ALL`	Prior to PHP 8.0.0, the default was "90.583333"
date.sunset_zenith	"90.833333"	`INI_ALL`	Prior to PHP 8.0.0, the default was "90.583333"
date.timezone	"UTC"	`INI_ALL`	From PHP 8.2, a warning is emitted when setting this to an invalid value or an empty string.

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`date.default_latitude` float

The default latitude ranging from `0` at the equator, to `+90` northward, and `-90` southward.

`date.default_longitude` float

The default longitude ranging from `0` at the prime meridian to `+180` eastward and `-180` westward.

`date.sunrise_zenith` float

The default sunrise zenith.

The default value is 90°50'. The additional 50' is due to two components: the Sun's radius, which is 16', and the atmospheric refraction, which is 34'.

`date.sunset_zenith` float

The default sunset zenith.

`date.timezone` string

The default timezone used by all date/time functions. The precedence order for which timezone is used if none is explicitly mentioned is described in the date_default_timezone_get() page. See List of Supported Timezones for a list of supported timezones.

> **Note** :
>
> The first four configuration options are currently only used by date_sunrise() and date_sunset().

Predefined Constants

The `DATE_*` constants are defined and offer standard date representations, which can be used along with the date format functions (like date()).

Available `returnFormat` for date_sunrise() and date_sunset()

> **Warning**
>
> These constants are deprecated as of PHP 8.4.0. The corresponding date_sunrise() and date_sunset() functions are deprecated as of PHP 8.1.0.

`SUNFUNCS_RET_TIMESTAMP` (int)

Timestamp

`SUNFUNCS_RET_STRING` (int)

Hours:minutes (example: `08:02`)

`SUNFUNCS_RET_DOUBLE` (int)

Hours as floating point number (example `8.75`)

`DATE_*` constants

`DATE_ATOM` (string)

Atom (example: `2005-08-15T15:52:01+00:00`); compatible with ISO-8601, RFC 3339, and XML Schema

`DATE_COOKIE` (string)

HTTP Cookies (example: `Monday, 15-Aug-2005 15:52:01 UTC`)

`DATE_ISO8601` (string)

ISO-8601-like (example: `2005-08-15T15:52:01+0000`)

> **Note** :
>
> This format is not compatible with ISO-8601, but is left this way for backward compatibility reasons. Use `DATE_ISO8601_EXPANDED`, `DATE_ATOM` for compatibility with ISO-8601 instead (ref ISO8601:2004 section 4.3.3 clause d).

`DATE_ISO8601_EXPANDED` (string)

ISO-8601 Expanded (example: `+10191-07-26T08:59:52+01:00`)

> **Note** :

> This format allows for year ranges outside of ISO-8601's normal range of `0000-9999` by always including a sign character. It also ensures that the timezone part (`+01:00`) is compatible with ISO-8601.

DATE_RFC822 (string)

RFC 822 (example: `Mon, 15 Aug 05 15:52:01 +0000`)

DATE_RFC850 (string)

RFC 850 (example: `Monday, 15-Aug-05 15:52:01 UTC`)

DATE_RFC1036 (string)

RFC 1036 (example: `Mon, 15 Aug 05 15:52:01 +0000`)

DATE_RFC1123 (string)

RFC 1123 (example: `Mon, 15 Aug 2005 15:52:01 +0000`)

DATE_RFC7231 (string)

RFC 7231 (as of PHP 7.0.19 and 7.1.5) (example: `Sat, 30 Apr 2016 17:52:13 GMT`)

DATE_RFC2822 (string)

RFC 2822 (example: `Mon, 15 Aug 2005 15:52:01 +0000`)

DATE_RFC3339 (string)

Same as **DATE_ATOM**.

DATE_RFC3339_EXTENDED (string)

RFC 3339 EXTENDED format (example: `2005-08-15T15:52:01.000+00:00`)

DATE_RSS (string)

RSS (example: `Mon, 15 Aug 2005 15:52:01 +0000`). Alias of **DATE_RFC1123**.

DATE_W3C (string)

World Wide Web Consortium (example: `2005-08-15T15:52:01+00:00`). Alias of **DATE_RFC3339**.

Examples

Table of Contents

Date/Time Arithmetic

The following examples show some pitfalls of Date/Time arithmetic with regard to DST transitions and months having different numbers of days.

Example #1 DateTimeImmutable::add/sub add intervals which cover elapsed time

Adding PT24H over a DST transition will appear to add 23/25 hours (for most timezones).

```
<?php
$dt = new DateTimeImmutable("2015-11-01 00:00:00", new DateTimeZone("America/New_York"));
echo "Start: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->add(new DateInterval("PT3H"));
echo "End:   ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

The above example will output:

```
Start: 2015-11-01 00:00:00 -04:00
End:   2015-11-01 02:00:00 -05:00
```

Example #2 DateTimeImmutable::modify and strtotime increment or decrement individual component values

Adding +24 hours over a DST transition will add exactly 24 hours as seen in the date/time string (unless the start or end time is on a transition point).

```php
<?php
$dt = new DateTimeImmutable("2015-11-01 00:00:00", new DateTimeZone("America/New_York"));
echo "Start: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("+24 hours");
echo "End:   ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

The above example will output:

```
Start: 2015-11-01 00:00:00 -04:00
End:   2015-11-02 00:00:00 -05:00
```

Example #3 DateInterval::createFromDateString creates intervals which increment or decrement component values

DateInterval::createFromDateString() uses the date/time parser, and the time part of the resulting interval increments or decrements the individual component values. For an interval created with DateInterval::__construct() the time part covers elapsed time instead, so the two are not interchangeable when a timezone transition falls within that time part. The date part is applied the same way by both, so `P1D` and `1 day` always give the same result.

```php
<?php
$dt = new DateTimeImmutable("2015-11-01 00:00:00", new DateTimeZone("America/New_York"));
echo "Start:    ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
echo "PT24H:    ", $dt->add(new DateInterval("PT24H"))->format("Y-m-d H:i:s P"), PHP_EOL;
echo "24 hours: ", $dt->add(DateInterval::createFromDateString("24 hours"))->format("Y-m-d H:i:s P"), PHP_EOL;
```

The above example will output:

```
Start:    2015-11-01 00:00:00 -04:00
PT24H:    2015-11-01 23:00:00 -05:00
24 hours: 2015-11-02 00:00:00 -05:00
```

Example #4 Adding or subtracting times can over- or underflow dates

Like where January 31st + 1 month will result in March 2nd (leap year) or 3rd (normal year).

```php
<?php
echo "Normal year:\n"; // February has 28 days
$dt = new DateTimeImmutable("2015-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Start: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("+1 month");
echo "End:   ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;

echo "Leap year:\n"; // February has 29 days
$dt = new DateTimeImmutable("2016-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Start: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("+1 month");
echo "End:   ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

The above example will output:

```
Normal year:
Start: 2015-01-31 00:00:00 -05:00
End:   2015-03-03 00:00:00 -05:00
Leap year:
Start: 2016-01-31 00:00:00 -05:00
End:   2016-03-02 00:00:00 -05:00
```

To get the last day of the next month (i.e. to prevent the overflow), the `last day of` format is available.

```php
<?php
echo "Normal year:\n"; // February has 28 days
$dt = new DateTimeImmutable("2015-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Start: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("last day of next month");
echo "End:   ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;

echo "Leap year:\n"; // February has 29 days
$dt = new DateTimeImmutable("2016-01-31 00:00:00", new DateTimeZone("America/New_York"));
echo "Start: ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
$dt = $dt->modify("last day of next month");
echo "End:   ", $dt->format("Y-m-d H:i:s P"), PHP_EOL;
```

The above example will output:

```
Normal year:
Start: 2015-01-31 00:00:00 -05:00
End:   2015-02-28 00:00:00 -05:00
Leap year:
Start: 2016-01-31 00:00:00 -05:00
End:   2016-02-29 00:00:00 -05:00
```

DateTime

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introduction

Representation of date and time.

This class behaves the same as DateTimeImmutable, except objects are modified itself when modification methods such as DateTime::modify() are called.

> **Warning**
>
> Calling methods on objects of the class **DateTime** will change the information encapsulated in these objects, if you want to prevent that you will have to use `clone` operator to create a new object. Use DateTimeImmutable instead of **DateTime** to obtain this recommended behaviour by default.

Class synopsis

class **DateTime** implements DateTimeInterface {
/* Inherited constants */
public const string *DateTimeInterface::ATOM* = "Y-m-d\\TH:i:sP";
public const string *DateTimeInterface::COOKIE* = "l, d-M-Y H:i:s T";
public const string *DateTimeInterface::ISO8601* = "Y-m-d\\TH:i:sO";
public const string *DateTimeInterface::ISO8601_EXPANDED* = "X-m-d\\TH:i:sP";
public const string *DateTimeInterface::RFC822* = "D, d M y H:i:s O";
public const string *DateTimeInterface::RFC850* = "l, d-M-y H:i:s T";
public const string *DateTimeInterface::RFC1036* = "D, d M y H:i:s O";
public const string *DateTimeInterface::RFC1123* = "D, d M Y H:i:s O";

```
public const string DateTimeInterface::RFC7231 = "D, d M Y H:i:s \\G\\M\\T";
public const string DateTimeInterface::RFC2822 = "D, d M Y H:i:s O";
public const string DateTimeInterface::RFC3339 = "Y-m-d\\TH:i:sP";
public const string DateTimeInterface::RFC3339_EXTENDED = "Y-m-d\\TH:i:s.vP";
public const string DateTimeInterface::RSS = "D, d M Y H:i:s O";
public const string DateTimeInterface::W3C = "Y-m-d\\TH:i:sP";
/* Methods */
public function __construct(string $datetime = "now", ?DateTimeZone $timezone = null)
public function add(DateInterval $interval): DateTime
public static function createFromFormat(string $format, string $datetime, ?DateTimeZone $timezone = null): DateTime|false
public static function createFromImmutable(DateTimeImmutable $object): static
public static function createFromInterface(DateTimeInterface $object): DateTime
public static function createFromTimestamp(int|float $timestamp): static
public function modify(string $modifier): DateTime
public static function __set_state(array $array): DateTime
public function setDate(int $year, int $month, int $day): DateTime
public function setISODate(int $year, int $week, int $dayOfWeek = 1): DateTime
public function setMicrosecond(int $microsecond): static
public function setTime(
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTime
public function setTimestamp(int $timestamp): DateTime
public function setTimezone(DateTimeZone $timezone): DateTime
public function sub(DateInterval $interval): DateTime
public function diff(DateTimeInterface $targetObject, bool $absolute = false): DateInterval
public function format(string $format): string
public function getMicrosecond(): int
public function getOffset(): int
public function getTimestamp(): int
public function getTimezone(): DateTimeZone|false
public function __serialize(): array
public function __unserialize(array $data): void
#[\Deprecated(
    since: '8.5',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function __wakeup(): void
}
```

Changelog

Version	Description
8.4.0	The class constants are now typed.
7.2.0	The class constants of **DateTime** are now defined on DateTimeInterface.
7.1.0	The **DateTime** constructor now includes the current microseconds in the constructed value. Before this, it would always initialise the microseconds to `0`.

Table of Contents

DateTime::add

DateTime::add

date_add

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::add -- date_add — Modifies a DateTime object, with added amount of days, months, years, hours, minutes and seconds

Description

Object-oriented style

public function **DateTime::add**(DateInterval `$interval`): DateTime

Procedural style

function date_add(DateTime `$object`, DateInterval `$interval`): DateTime

Adds the specified DateInterval object to the specified DateTime object.

Like DateTimeImmutable::add() but works with DateTime.

The procedural version takes the DateTime object as its first argument.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`interval`

A DateInterval object

Return Values

Returns the modified DateTime object for method chaining.

See Also

- DateTimeImmutable::add() - Returns a new object, with added amount of days, months, years, hours, minutes and seconds

DateTime::__construct

DateTime::__construct

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::__construct — Returns new DateTime object

Description

public function **DateTime::__construct**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`)

Like DateTimeImmutable::__construct() but works with DateTime. Consider using the DateTimeImmutable and features instead.

Returns a new DateTime object.

Parameters

`datetime`

A date/time string. Valid formats are explained in Date and Time Formats.

Enter `"now"` here to obtain the current time when using the `timezone` parameter.

`timezone`

A DateTimeZone object representing the timezone of `datetime`.

If `timezone` is omitted or `null`, the current timezone will be used.

> **Note** :
>
> The `timezone` parameter and the current timezone are ignored when the `datetime` parameter either is a UNIX timestamp (e.g. `@946684800`) or specifies a timezone (e.g. `2010-01-28T15:00:00+02:00`).

Return Values

Returns a new DateTime instance.

Errors/Exceptions

If an invalid Date/Time string is passed, DateMalformedStringException is thrown. Previous to PHP 8.3, this was Exception.

Changelog

Version	Description
8.3.0	Now throws DateMalformedStringException if an invalid string is passed, instead of Exception.

See Also

- DateTimeImmutable::__construct() - Returns new DateTimeImmutable object

DateTime::createFromFormat

DateTime::createFromFormat

date_create_from_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::createFromFormat -- date_create_from_format — Parses a time string according to a specified format

Description

Object-oriented style

public static function **DateTime::createFromFormat**(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTime|false

Procedural style

function date_create_from_format(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTime|false

Returns a new DateTime object representing the date and time specified by the `datetime` string, which was formatted in the given `format`.

Like DateTimeImmutable::createFromFormat() and date_create_immutable_from_format(), respectively, but creates a DateTime object.

This method, including parameters, examples, and considerations are documented on the DateTimeImmutable::createFromFormat page.

Parameters

See DateTimeImmutable::createFromFormat.

Return Values

Returns a new DateTime instance or `false` on failure.

Errors/Exceptions

This method throws ValueError when the `datetime` contains NULL-bytes.

Changelog

Version	Description
8.0.21, 8.1.8, 8.2.0	Now throws ValueError when NULL-bytes are passed into `datetime`, which previously was silently ignored.

Examples

For an extensive set of examples, see DateTimeImmutable::createFromFormat.

See Also

- DateTimeImmutable::createFromFormat() - Parses a time string according to a specified format

DateTime::createFromImmutable

DateTime::createFromImmutable

(PHP 7 >= 7.3.0, PHP 8)

DateTime::createFromImmutable — Returns new DateTime instance encapsulating the given DateTimeImmutable object

Description

public static function **DateTime::createFromImmutable**(DateTimeImmutable `$object`): static

Parameters

`object`

The immutable DateTimeImmutable object that needs to be converted to a mutable version. This object is not modified, but instead a new DateTime instance is created containing the same date, time, and timezone information.

Return Values

Returns a new DateTime instance.

Changelog

Version	Description
8.0.0	The method returns an instance of the currently invoked class now. Previously, it created a new instance of DateTime.

Examples

Example #1 Creating a mutable date time object

```
<?php
$date = new DateTimeImmutable("2014-06-20 11:45 Europe/London");
$mutable = DateTime::createFromImmutable( $date );
```

DateTime::createFromInterface

DateTime::createFromInterface

(PHP 8)

DateTime::createFromInterface — Returns new DateTime object encapsulating the given DateTimeInterface object

Description

public static function **DateTime::createFromInterface**(DateTimeInterface `$object`): DateTime

Parameters

`object`

The DateTimeInterface object that needs to be converted to a mutable version. This object is not modified, but instead a new DateTime object is created containing the same date, time, and timezone information.

Return Values

Returns a new DateTime instance.

Examples

Example #1 Creating a mutable date time object

```
<?php
$date = new DateTimeImmutable("2014-06-20 11:45 Europe/London");
$mutable = DateTime::createFromInterface($date);

$date = new DateTime("2014-06-20 11:45 Europe/London");
$also_mutable = DateTime::createFromInterface($date);
```

DateTime::createFromTimestamp

DateTime::createFromTimestamp

(PHP 8 >= 8.4.0)

DateTime::createFromTimestamp — Creates an instance from a Unix timestamp

Description

public static function **DateTime::createFromTimestamp**(int|float `$timestamp`): static

Creates an instance from a Unix timestamp.

Parameters

`timestamp`

Unix timestamp representing the date. A float value is also accepted which allows for microsecond precision.

Return Values

Returns a new DateTime instance.

Errors/Exceptions

If the `timestamp` is outside the range [`PHP_INT_MIN`, `PHP_INT_MAX`], a DateRangeError is thrown.

Examples

Example #1 DateTime::createFromTimestamp() example

```php
<?php
$date = DateTime::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u');
?>
```

The above example will output:

```
1970-01-01 00:02:03.456789
```

DateTime::getLastErrors

DateTime::getLastErrors

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::getLastErrors — Alias of DateTimeImmutable::getLastErrors()

Description

This function is an alias of: DateTimeImmutable::getLastErrors()

DateTime::modify

DateTime::modify

date_modify

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::modify -- date_modify — Alters the timestamp

Description

Object-oriented style

public function **DateTime::modify**(string `$modifier`): DateTime

Procedural style

function date_modify(DateTime `$object`, string `$modifier`): DateTime|false

Alter the timestamp of a DateTime object by incrementing or decrementing in a format accepted by DateTimeImmutable::__construct().

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`modifier`

A date/time string. Valid formats are explained in Date and Time Formats.

Return Values

Returns DateTime on success. Procedural style returns **`false`** on failure.

Errors/Exceptions

Object Oriented API only: If an invalid Date/Time string is passed, DateMalformedStringException is thrown.

Changelog

Version	Description
8.4.0	Now has a tentative return type of DateTime. Previously it was DateTime|false.
8.3.0	**DateTime::modify()** now throws DateMalformedStringException if an invalid string is passed. Previously, it returned **`false`**, and a warning was emitted. date_modify() has not been changed.

Examples

Example #1 DateTime::modify() example

Object-oriented style

```php
<?php
$date = new DateTime('2006-12-12');
$date->modify('+1 day');
echo $date->format('Y-m-d');
```

The above example will output:

```
2006-12-13
```

Procedural style

```php
<?php
$date = date_create('2006-12-12');
date_modify($date, '+1 day');
echo date_format($date, 'Y-m-d');
```

The above example will output:

```
2006-12-13
```

Example #2 Beware when adding or subtracting months

```php
<?php
$date = new DateTime('2000-12-31');

$date->modify('+1 month');
echo $date->format('Y-m-d') . "\n";

$date->modify('+1 month');
echo $date->format('Y-m-d') . "\n";
```

The above example will output:

```
2001-01-31
2001-03-03
```

Example #3 All formats of Date and Time are supported

```php
<?php
$date = new DateTime('2020-12-31');

$date->modify('July 1st, 2023');
echo $date->format('Y-m-d H:i') . "\n";

$date->modify('Monday next week');
echo $date->format('Y-m-d H:i') . "\n";

$date->modify('17:30');
echo $date->format('Y-m-d H:i') . "\n";
```

The above example will output:

```
2023-07-01 00:00
2023-07-03 00:00
2023-07-03 17:30
```

See Also

- strtotime() - Parse about any English textual datetime description into a Unix timestamp
- DateTimeImmutable::modify() - Creates a new object with modified timestamp
- DateTime::add() - Modifies a DateTime object, with added amount of days, months, years, hours, minutes and seconds
- DateTime::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds from a DateTime object
- DateTime::setDate() - Sets the date
- DateTime::setISODate() - Sets the ISO date
- DateTime::setTime() - Sets the time
- DateTime::setTimestamp() - Sets the date and time based on an Unix timestamp

DateTime::__set_state

DateTime::__set_state

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::__set_state — The __set_state handler

Description

public static function **DateTime::__set_state**(array `$array`): DateTime

The __set_state() handler.

Like DateTimeImmutable::__set_state() but works with DateTime.

Parameters

`array`

Initialization array.

Return Values

Returns a new instance of a DateTime object.

DateTime::setDate

DateTime::setDate

date_date_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setDate -- date_date_set — Sets the date

Description

Object-oriented style

public function **DateTime::setDate**(int `$year`, int `$month`, int `$day`): DateTime

Procedural style

```
function date_date_set(
    DateTime $object,
    int $year,
    int $month,
    int $day
): DateTime
```

Resets the current date of the DateTime object to a different date.

Like DateTimeImmutable::setDate() but works with DateTime, and changes the existing object.

The procedural version takes the DateTime object as its first argument.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`year`

Year of the date.

`month`

Month of the date.

`day`

Day of the date.

Return Values

Returns the modified DateTime object for method chaining.

See Also

- DateTimeImmutable::setDate() - Sets the date

DateTime::setISODate

DateTime::setISODate

date_isodate_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setISODate -- date_isodate_set — Sets the ISO date

Description

Object-oriented style

public function **DateTime::setISODate**(int `$year`, int `$week`, int `$dayOfWeek` = 1): DateTime

Procedural style

```
function date_isodate_set(
    DateTime $object,
    int $year,
    int $week,
    int $dayOfWeek = 1
): DateTime
```

Set a date according to the ISO 8601 standard - using weeks and day offsets rather than specific dates.

Like DateTimeImmutable::setISODate() but works with DateTime.

The procedural version takes the DateTime object as its first argument.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`year`

Year of the date.

`week`

Week of the date.

`dayOfWeek`

Offset from the first day of the week.

Return Values

Returns the modified DateTime object for method chaining.

See Also

- DateTimeImmutable::setISODate() - Sets the ISO date

DateTime::setMicrosecond

DateTime::setMicrosecond

(PHP 8 >= 8.4.0)

DateTime::setMicrosecond — Sets microsecond part of the time

Description

public function **DateTime::setMicrosecond**(int `$microsecond`): static

Sets microsecond part of the time.

Like DateTimeImmutable::setMicrosecond() but works with DateTime.

Parameters

`microsecond`

The microsecond value to set (`0` to `999999`).

Return Values

Returns the modified DateTime object for method chaining.

Errors/Exceptions

If the `microsecond` is outside the range [`0`, `999999`], a DateRangeError is thrown.

Examples

Example #1 DateTime::setMicrosecond() example

```php
<?php
$date = DateTime::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
$date->setMicrosecond(987654);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
?>
```

The above example will output:

```
1970-01-01 00:02:03.456789
1970-01-01 00:02:03.987654
```

See Also

- DateTimeInterface::getMicrosecond() - Gets the microsecond part of the Unix timestamp

DateTime::setTime

DateTime::setTime

date_time_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setTime -- date_time_set — Sets the time

Description

Object-oriented style

```
public function DateTime::setTime(
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTime
```

Procedural style

```
function date_time_set(
    DateTime $object,
```

```
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTime
```

Resets the current time of the DateTime object to a different time.

Like DateTimeImmutable::setTime() but works with DateTime.

The procedural version takes the DateTime object as its first argument.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`hour`

Hour of the time.

`minute`

Minute of the time.

`second`

Second of the time.

`microsecond`

Microsecond of the time.

Return Values

Returns the modified DateTime object for method chaining.

Changelog

Version	Description
8.1.0	The behaviour with double existing hours (during the fall-back DST transition) changed. Previously PHP would pick the second occurrence (after the DST transition), instead of the first occurrence (before DST transition).
7.1.0	The `microsecond` parameter was added.

See Also

- DateTimeImmutable::setTime() - Sets the time

DateTime::setTimestamp

DateTime::setTimestamp

date_timestamp_set

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::setTimestamp -- date_timestamp_set — Sets the date and time based on an Unix timestamp

Description

Object-oriented style

public function **DateTime::setTimestamp**(int `$timestamp`): DateTime

Procedural style

function date_timestamp_set(DateTime `$object`, int `$timestamp`): DateTime

Sets the date and time based on an Unix timestamp.

Like DateTimeImmutable::setTimestamp() but works with DateTime.

The procedural version takes the DateTime object as its first argument.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`timestamp`

Unix timestamp representing the date. Setting timestamps outside the range of int is possible by using DateTimeImmutable::modify() with the `@` format.

Return Values

Returns the modified DateTime object for method chaining.

See Also

- DateTimeImmutable::setTimestamp() - Sets the date and time based on a Unix timestamp
- DateTime::setMicrosecond() - Sets microsecond part of the time

DateTime::setTimezone

DateTime::setTimezone

date_timezone_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTime::setTimezone -- date_timezone_set — Sets the time zone for the DateTime object

Description

Object-oriented style

public function **DateTime::setTimezone**(DateTimeZone `$timezone`): DateTime

Procedural style

function date_timezone_set(DateTime `$object`, DateTimeZone `$timezone`): DateTime

Sets a new timezone for a DateTime object.

Like DateTimeImmutable::setTimezone() but works with DateTime.

The procedural version takes the DateTime object as its first argument.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`timezone`

A DateTimeZone object representing the desired time zone.

Return Values

Returns the DateTime object for method chaining. The underlying point-in-time is not changed when calling this method.

Examples

Example #1 DateTime::setTimeZone() example

Object-oriented style

```php
<?php
$date = new DateTime('2000-01-01', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

$date->setTimezone(new DateTimeZone('Pacific/Chatham'));
echo $date->format('Y-m-d H:i:sP') . "\n";
```

The above example will output:

```
2000-01-01 00:00:00+12:00
2000-01-01 01:45:00+13:45
```

Procedural style

```php
<?php
$date = date_create('2000-01-01', timezone_open('Pacific/Nauru'));
echo date_format($date, 'Y-m-d H:i:sP') . "\n";

date_timezone_set($date, timezone_open('Pacific/Chatham'));
echo date_format($date, 'Y-m-d H:i:sP') . "\n";
```

The above example will output:

```
2000-01-01 00:00:00+12:00
2000-01-01 01:45:00+13:45
```

See Also

- DateTimeImmutable::setTimezone() - Sets the time zone
- DateTime::getTimezone() - Return time zone relative to given DateTime
- DateTimeZone::__construct() - Creates new DateTimeZone object

DateTime::sub

DateTime::sub

date_sub

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::sub -- date_sub — Subtracts an amount of days, months, years, hours, minutes and seconds from a DateTime object

Description

Object-oriented style

public function **DateTime::sub**(DateInterval `$interval`): DateTime

Procedural style

function date_sub(DateTime `$object`, DateInterval `$interval`): DateTime

Modifies the specified DateTime object, by subtracting the specified DateInterval object.

Like DateTimeImmutable::sub() but works with DateTime.

The procedural version takes the DateTime object as its first argument.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`interval`

A DateInterval object

Return Values

Returns the modified DateTime object for method chaining.

Errors/Exceptions

Object Oriented API only: If an unsupported operation is attempted, such as using a DateInterval object representing relative time specifications such as `next weekday`, a DateInvalidOperationException is thrown.

Changelog

Version	Description
8.3.0	Now throws a DateInvalidOperationException with **DateTime::sub()**, instead of a warning when an unsupported operation is attempted. date_sub() has not been changed.

See Also

- DateTimeImmutable::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds

DateTimeImmutable

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introduction

Representation of date and time.

This class behaves the same as DateTime except new objects are returned when modification methods such as DateTime::modify() are called.

Class synopsis

class **DateTimeImmutable** implements DateTimeInterface {
/* Inherited constants */
public const string *DateTimeInterface::ATOM* = "Y-m-d\\TH:i:sP";
public const string *DateTimeInterface::COOKIE* = "l, d-M-Y H:i:s T";
public const string *DateTimeInterface::ISO8601* = "Y-m-d\\TH:i:sO";
public const string *DateTimeInterface::ISO8601_EXPANDED* = "X-m-d\\TH:i:sP";
public const string *DateTimeInterface::RFC822* = "D, d M y H:i:s O";
public const string *DateTimeInterface::RFC850* = "l, d-M-y H:i:s T";
public const string *DateTimeInterface::RFC1036* = "D, d M y H:i:s O";
public const string *DateTimeInterface::RFC1123* = "D, d M Y H:i:s O";
public const string *DateTimeInterface::RFC7231* = "D, d M Y H:i:s \\G\\M\\T";
public const string *DateTimeInterface::RFC2822* = "D, d M Y H:i:s O";
public const string *DateTimeInterface::RFC3339* = "Y-m-d\\TH:i:sP";
public const string *DateTimeInterface::RFC3339_EXTENDED* = "Y-m-d\\TH:i:s.vP";
public const string *DateTimeInterface::RSS* = "D, d M Y H:i:s O";
public const string *DateTimeInterface::W3C* = "Y-m-d\\TH:i:sP";
/* Methods */
public function __construct(string `$datetime` = "now", ?DateTimeZone `$timezone` = **`null`**)
#[\NoDiscard(`message`: "as DateTimeImmutable::add() does not modify the object itself")]
public function add(DateInterval `$interval`): DateTimeImmutable
public static function createFromFormat(string `$format`, string `$datetime`, ?DateTimeZone `$timezone` = null): DateTimeImmutable|false
public static function createFromInterface(DateTimeInterface `$object`): DateTimeImmutable
public static function createFromMutable(DateTime `$object`): static
public static function createFromTimestamp(int|float `$timestamp`): static

```
public static function getLastErrors(): array|false
#[\NoDiscard(message: "as DateTimeImmutable::modify() does not modify the object itself")]
public function modify(string $modifier): DateTimeImmutable
public static function __set_state(array $array): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setDate() does not modify the object itself")]
public function setDate(int $year, int $month, int $day): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setISODate() does not modify the object itself")]
public function setISODate(int $year, int $week, int $dayOfWeek = 1): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setMicrosecond() does not modify the object itself")]
public function setMicrosecond(int $microsecond): static
#[\NoDiscard(message: "as DateTimeImmutable::setTime() does not modify the object itself")]
public function setTime(
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setTimestamp() does not modify the object itself")]
public function setTimestamp(int $timestamp): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::setTimezone() does not modify the object itself")]
public function setTimezone(DateTimeZone $timezone): DateTimeImmutable
#[\NoDiscard(message: "as DateTimeImmutable::sub() does not modify the object itself")]
public function sub(DateInterval $interval): DateTimeImmutable
public function diff(DateTimeInterface $targetObject, bool $absolute = false): DateInterval
public function format(string $format): string
public function getMicrosecond(): int
public function getOffset(): int
public function getTimestamp(): int
public function getTimezone(): DateTimeZone|false
public function __serialize(): array
public function __unserialize(array $data): void
#[\Deprecated(
    since: '8.5',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function __wakeup(): void
}
```

Changelog

Version	Description
8.4.0	The class constants are now typed.
7.1.0	The **DateTimeImmutable** constructor now includes the current microseconds in the constructed value. Before this, it would always initialise the microseconds to `0`.

Table of Contents

DateTimeImmutable::add

DateTimeImmutable::add

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::add — Returns a new object, with added amount of days, months, years, hours, minutes and seconds

Description

```
#[\NoDiscard(message: "as DateTimeImmutable::add() does not modify the object itself")]
public function DateTimeImmutable::add(DateInterval $interval): DateTimeImmutable
```

Creates a new DateTimeImmutable object, and adds the specified DateInterval object to this, to represent the new value.

Parameters

`interval`

A DateInterval object

Return Values

Returns a new DateTimeImmutable object with the modified data.

Examples

Example #1 DateTimeImmutable::add() example

Object-oriented style

```
<?php
$date = new DateTimeImmutable('2000-01-01');
$newDate = $date->add(new DateInterval('P10D'));
echo $newDate->format('Y-m-d') . "\n";
?>
```

Example #2 Further DateTimeImmutable::add() examples

```
<?php
$date = new DateTimeImmutable('2000-01-01');
$newDate = $date->add(new DateInterval('PT10H30S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";

$date = new DateTimeImmutable('2000-01-01');
$newDate = $date->add(new DateInterval('P7Y5M4DT4H3M2S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

The above example will output:

```
2000-01-01 10:00:30
2007-06-05 04:03:02
```

Example #3 Beware when adding months

```
<?php
$date = new DateTimeImmutable('2000-12-31');
$interval = new DateInterval('P1M');

$newDate1 = $date->add($interval);
echo $newDate1->format('Y-m-d') . "\n";

$newDate2 = $newDate1->add($interval);
echo $newDate2->format('Y-m-d') . "\n";
?>
```

The above example will output:

```
2001-01-31
2001-03-03
```

See Also

- DateTimeImmutable::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds

- DateTimeImmutable::diff() - Returns the difference between two DateTime objects
- DateTimeImmutable::modify() - Creates a new object with modified timestamp
- Date/Time Arithmetic

DateTimeImmutable::__construct

DateTimeImmutable::__construct

date_create_immutable

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::__construct -- date_create_immutable — Returns new DateTimeImmutable object

Description

Object-oriented style

public function **DateTimeImmutable::__construct**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`)

Procedural style

function date_create_immutable(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`): DateTimeImmutable|false

Returns new a DateTimeImmutable object.

Parameters

`datetime`

A date/time string. Valid formats are explained in Date and Time Formats.

Enter `"now"` here to obtain the current time when using the `timezone` parameter.

`timezone`

A DateTimeZone object representing the timezone of `datetime`.

If `timezone` is omitted or `null`, the current timezone will be used.

> **Note** :
>
> The `timezone` parameter and the current timezone are ignored when the `datetime` parameter either is a UNIX timestamp (e.g. `@946684800`) or specifies a timezone (e.g. `2010-01-28T15:00:00+02:00`, or `2010-07-05T06:00:00Z`).

Return Values

Returns a new DateTimeImmutable instance.

Errors/Exceptions

If an invalid Date/Time string is passed, DateMalformedStringException is thrown. Previous to PHP 8.3, this was Exception.

Changelog

Version	Description
8.3.0	Now throws DateMalformedStringException if an invalid string is passed, instead of Exception.
7.1.0	From now on microseconds are filled with actual value. Not with '00000'.

Examples

Example #1 DateTimeImmutable::__construct() example

Object-oriented style

```
<?php
try {
```

```
    $date = new DateTimeImmutable('2000-01-01');
} catch (Exception $e) {
    echo $e->getMessage();
    exit(1);
}

echo $date->format('Y-m-d');
```

The above example will output:

```
2000-01-01
```

Procedural style

```
<?php
$date = date_create('2000-01-01');
if (!$date) {
    $e = date_get_last_errors();
    foreach ($e['errors'] as $error) {
        echo "$error\n";
    }
    exit(1);
}

echo date_format($date, 'Y-m-d');
```

The above example will output:

```
2000-01-01
```

Example #2 Intricacies of DateTimeImmutable::__construct()

```
<?php
date_default_timezone_set('America/Jamaica');

// Specified date/time in PHP's default time zone.
$date = new DateTimeImmutable('2000-01-01');
echo $date->format('Y-m-d H:i:sP') . "\n";

// Specified date/time in the specified time zone.
$date = new DateTimeImmutable('2000-01-01', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

// Current date/time in your PHP's default time zone.
$date = new DateTimeImmutable();
echo $date->format('Y-m-d H:i:sP') . "\n";

// Current date/time in the specified time zone.
$date = new DateTimeImmutable('now', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

// Using a UNIX timestamp.  Notice the result is in the UTC time zone.
$date = new DateTimeImmutable('@946684800');
echo $date->format('Y-m-d H:i:sP') . "\n";

// Non-existent values roll over.
$date = new DateTimeImmutable('2000-02-30');
echo $date->format('Y-m-d H:i:sP') . "\n";
```

The above example will output something similar to:

```
2000-01-01 00:00:00-05:00
2000-01-01 00:00:00+12:00
2010-04-24 10:24:16-04:00
2010-04-25 02:24:16+12:00
2000-01-01 00:00:00+00:00
2000-03-01 00:00:00-05:00
```

> **Note** :
>
> Rolled over dates can be detected by checking for warnings using DateTimeImmutable::getLastErrors().

Example #3 Changing the associated timezone

```php
<?php
$timeZone = new \DateTimeZone('Asia/Tokyo');

$time = new \DateTimeImmutable();
$time = $time->setTimezone($timeZone);

echo $time->format('Y/m/d H:i:s e'), "\n";
```

The above example will output something similar to:

```
2022/08/12 23:49:23 Asia/Tokyo
```

Example #4 Using a relative date/time string

```php
<?php
$time = new \DateTimeImmutable("-1 year");

echo $time->format('Y/m/d H:i:s'), "\n";
```

The above example will output something similar to:

```
2021/08/12 15:43:51
```

DateTimeImmutable::createFromFormat

DateTimeImmutable::createFromFormat

date_create_immutable_from_format

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::createFromFormat -- date_create_immutable_from_format — Parses a time string according to a specified format

Description

Object-oriented style

public static function **DateTimeImmutable::createFromFormat**(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTimeImmutable|false

Procedural style

function date_create_immutable_from_format(string `$format`, string `$datetime`, ?DateTimeZone `$timezone = null`): DateTimeImmutable|false

Returns a new DateTimeImmutable object representing the date and time specified by the `datetime` string, which was formatted in the given `format`.

Parameters

`format`

The format that the passed in string should be in. See the formatting options below. In most cases, the same letters as for the date() can be used.

All fields are initialised with the current date/time. In most cases you would want to reset these to "zero" (the Unix epoch, `1970-01-01 00:00:00 UTC`). You do that by including the `!` character as first character in your `format`, or `|` as your last. Please see the documentation for each character below for more information.

The format is parsed from left to right, which means that in some situations the order in which the format characters are present affects the result. In the case of `z` (the day of the year), it is required that a year has already been parsed, for example through the `Y` or `y` characters.

Letters that are used for parsing numbers allow a wide range of values, outside of what the logical range would be. For example, the `d` (day of the month) accepts values in the range from `00` to `99`. The only constraint is on the amount of digits. The date/time parser's overflow mechanism is used when out-of-range values are given. The examples below show some of this behaviour.

This also means that the data parsed for a format letter is greedy, and will read up to the amount of digits its format allows for. That can then also mean that there are no longer enough characters in the `datetime` string for following format characters. An example on this page also illustrates this issue.

The following characters are recognized in the `format` parameter string

`format` character	Description	Example parsable values
Day	---	---
`d` and `j`	Day of the month, 2 digits with or without leading zeros	`01` to `31` or `1` to `31`. (2 digit numbers higher than the number of days in the month are accepted, in which case they will make the month overflow. For example using 33 with January, means February 2nd)
`D` and `l`	A textual representation of a day	`Mon` through `Sun` or `Sunday` through `Saturday`. If the day name given is different than the day name belonging to a parsed (or default) date is different, then an overflow occurs to the *next* date with the given day name. See the examples below for an explanation.
`S`	English ordinal suffix for the day of the month, 2 characters. It's ignored while processing.	`st`, `nd`, `rd` or `th`.
`z`	The day of the year (starting from 0); must be preceded by `Y` or `y`.	`0` through `365`. (3 digit numbers higher than the numbers in a year are accepted, in which case they will make the year overflow. For example using 366 with 2022, means January 2nd, 2023)
Month	---	---
`F` and `M`	A textual representation of a month, such as January or Sept	`January` through `December` or `Jan` through `Dec`
`m` and `n`	Numeric representation of a month, with or without leading zeros	`01` through `12` or `1` through `12`. (2 digit numbers higher than 12 are accepted, in which case they will make the year overflow. For example using 13 means January in the next year)
Year	---	---
`X` and `x`	A full numeric representation of a year, *up to* 19 digits, optionally prefixed by `+` or `-`	Examples: `0055`, `787`, `1999`, `-2003`, `+10191`

`format` character	Description	Example parsable values
`Y`	A full numeric representation of a year, *up to* 4 digits	Examples: `25` (same as `0025`), `787`, `1999`, `2003`
`y`	A two digit representation of a year (which is assumed to be in the range 1970-2069, inclusive)	Examples: `99` or `03` (which will be interpreted as `1999` and `2003`, respectively)
Time	---	---
`a` and `A`	Ante meridiem and Post meridiem	`am` or `pm`
`g` and `h`	12-hour format of an hour with or without leading zero	`1` through `12` or `01` through `12` (2 digit numbers higher than 12 are accepted, in which case they will make the day overflow. For example using `14` means `02` in the next AM/PM period)
`G` and `H`	24-hour format of an hour with or without leading zeros	`0` through `23` or `00` through `23` (2 digit numbers higher than 24 are accepted, in which case they will make the day overflow. For example using `26` means `02:00` the next day)
`i`	Minutes with leading zeros	`00` to `59`. (2 digit numbers higher than 59 are accepted, in which case they will make the hour overflow. For example using `66` means `:06` the next hour)
`s`	Seconds, with leading zeros	`00` through `59` (2 digit numbers higher than 59 are accepted, in which case they will make the minute overflow. For example using `90` means `:30` the next minute)
`v`	Fraction in milliseconds (up to three digits)	Example: `12` (`0.12` seconds), `345` (`0.345` seconds)
`u`	Fraction in microseconds (up to six digits)	Example: `45` (`0.45` seconds), `654321` (`0.654321` seconds)
Timezone	---	---
`e`, `O`, `p`, `P` and `T`	Timezone identifier, or difference to UTC in hours, or difference to UTC with colon between hours and minutes, or timezone abbreviation	Examples: `UTC`, `GMT`, `Atlantic/Azores` or `+0200` or `+02:00` or `EST`, `MDT`
Full Date/Time	---	---
`U`	Seconds since the Unix Epoch (January 1 1970 00:00:00 GMT)	Example: `1292177455`
Whitespace and Separators	---	---
(space)	Zero or more spaces, tabs, NBSP (U+A0), or NNBSP (U+202F) characters	Example: `"\t"`, `" "`
`#`	One of the following separation symbol: `;`, `:`, `/`, `.`, `,`, `-`, `(` or `)`	Example: `/`
`;`, `:`, `/`, `.`, `,`, `-`, `(` or `)`	The specified character.	Example: `-`
`?`	A random byte	Example: `^` (Be aware that for UTF-8 characters you might need more than one `?`. In this case, using `*` is probably what you want instead)
`*`	Random bytes until the next separator or digit	Example: `*` in `Y-*-d` with the string `2009-aWord-08` will match `aWord`
`!`	Resets all fields (year, month, day, hour, minute, second, fraction and timezone information) to zero-like values (`0` for hour, minute, second and fraction, `1` for month and day, `1970` for year and the default timezone)	Without `!`, all fields will be set to the current date and time.
`|`	Resets all fields (year, month, day, hour, minute, second, fraction and timezone	`Y-m-d|` will set the year, month and day to the information found in the string to parse, and sets the hour, minute and second to 0.

`format` character	Description	Example parsable values
	information) to zero-like values if they have not been parsed yet	
`+`	If this format specifier is present, trailing data in the string will not cause an error, but a warning instead	Use DateTimeImmutable::getLastErrors() to find out whether trailing data was present.

Unrecognized characters in the format string will cause the parsing to fail and an error message is appended to the returned structure. You can query error messages with DateTimeImmutable::getLastErrors().

To include literal characters in `format`, you have to escape them with a backslash (`\`).

If `format` does not contain the character `!` then portions of the generated date/time which are not specified in `format` will be set to the current system time.

If `format` contains the character `!`, then portions of the generated date/time not provided in `format`, as well as values to the left-hand side of the `!`, will be set to corresponding values from the Unix epoch.

If any time character is parsed, then all other time-related fields are set to "0", unless also parsed.

The Unix epoch is 1970-01-01 00:00:00 UTC.

`datetime`

String representing the time.

`timezone`

A DateTimeZone object representing the desired time zone.

If `timezone` is omitted or **`null`** and `datetime` contains no timezone, the current timezone will be used.

> **Note** :
>
> The `timezone` parameter and the current timezone are ignored when the `datetime` parameter either contains a UNIX timestamp (e.g. `946684800`) or specifies a timezone (e.g. `2010-01-28T15:00:00+02:00`).

Return Values

Returns a new DateTimeImmutable instance or **`false`** on failure.

Errors/Exceptions

This method throws ValueError when the `datetime` contains NULL-bytes.

Changelog

Version	Description
8.2.9	The (space) specifier now also supports NBSP (U+A0) and NNBSP (U+202F) characters.
8.2.0	The `X` and `x` `format` specifiers have been added.
8.0.21, 8.1.8, 8.2.0	Now throws ValueError when NULL-bytes are passed into `datetime`, which previously was silently ignored.
7.3.0	The `v` `format` specifier has been added.

Examples

Example #1 DateTimeImmutable::createFromFormat() example

Object-oriented style

```php
<?php
$date = DateTimeImmutable::createFromFormat('j-M-Y', '15-Feb-2009');
echo $date->format('Y-m-d');
```

Example #2 Using predefined format constants with DateTimeImmutable::createFromFormat()

Object-oriented style

```
<?php
$date = DateTimeImmutable::createFromFormat(
    DateTimeInterface::ISO8601,
    '2004-02-12T15:19:21+00:00'
);
echo $date->format('c e') . "\n";

$date = DateTimeImmutable::createFromFormat(
    DateTimeInterface::RFC3339_EXTENDED,
    '2013-10-14T09:00:00.000+02:00'
);
echo $date->format('c e') . "\n";
```

The formatting constants as used in this example consist of a string of characters for formatting a DateTimeImmutable object. In most cases, these letters match with the same elements of date/time information as the ones defined in the parameters section above, but they tend to be more lenient.

Example #3 Intricacies of DateTimeImmutable::createFromFormat()

```
<?php
echo 'Current time: ' . date('Y-m-d H:i:s') . "\n";

$format = 'Y-m-d';
$date = DateTimeImmutable::createFromFormat($format, '2009-02-15');
echo "Format: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = 'Y-m-d H:i:s';
$date = DateTimeImmutable::createFromFormat($format, '2009-02-15 15:16:17');
echo "Format: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = 'Y-m-!d H:i:s';
$date = DateTimeImmutable::createFromFormat($format, '2009-02-15 15:16:17');
echo "Format: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = '!d';
$date = DateTimeImmutable::createFromFormat($format, '15');
echo "Format: $format; " . $date->format('Y-m-d H:i:s') . "\n";

$format = 'i';
$date = DateTimeImmutable::createFromFormat($format, '15');
echo "Format: $format; " . $date->format('Y-m-d H:i:s') . "\n";
```

The above example will output something similar to:

```
Current time: 2022-06-02 15:50:46
Format: Y-m-d; 2009-02-15 15:50:46
Format: Y-m-d H:i:s; 2009-02-15 15:16:17
Format: Y-m-!d H:i:s; 1970-01-15 15:16:17
Format: !d; 1970-01-15 00:00:00
Format: i; 2022-06-02 00:15:00
```

Example #4 Format string with literal characters

```
<?php
echo DateTimeImmutable::createFromFormat('H\h i\m s\s','23h 15m 03s')->format('H:i:s');
```

The above example will output something similar to:

```
23:15:03
```

Example #5 Overflow behaviour

```php
<?php
echo DateTimeImmutable::createFromFormat('Y-m-d H:i:s', '2021-17-35 16:60:97')->format(DateTimeImmutable::RFC2822);
```

The above example will output something similar to:

```
Sat, 04 Jun 2022 17:01:37 +0000
```

Although the result looks odd, it is correct, as the following overflows happen:

1. `97` seconds overflows to `1` minute, leaving `37` seconds.
2. `61` minutes overflows to `1` hour, leaving `1` minutes.
3. `35` days overflows to `1` month, leaving `4` days. The amount of days that are left over depends on the month, as not every month has the same amount of days.
4. `18` months overflows to `1` year, leaving `6` months.

Example #6 Overflowing day name behaviour

```php
<?php
$d = DateTime::createFromFormat(DateTimeInterface::RFC1123, 'Mon, 3 Aug 2020 25:00:00 +0000');
echo $d->format(DateTime::RFC1123), "\n";
```

The above example will output something similar to:

```
Mon, 10 Aug 2020 01:00:00 +0000
```

Although the result looks odd, it is correct, as the following overflows happen:

1. `3 Aug 2020 25:00:00` overflows to `(Tue) 4 Aug 2020 01:00`.
2. `Mon` gets applied, which advances the date to `Mon, 10 Aug 2020 01:00:00`. The explanation of relative keywords such as `Mon` is explained in the section on relative formats.

In order to detect overflows in dates, you can use DateTimeImmutable::getLastErrors(), which will include a warning if an overflow occurred.

Example #7 Detecting overflown dates

```php
<?php
$d = DateTimeImmutable::createFromFormat('Y-m-d H:i:s', '2021-17-35 16:60:97');
echo $d->format(DateTimeImmutable::RFC2822), "\n\n";

var_dump(DateTimeImmutable::getLastErrors());
```

The above example will output something similar to:

```
Sat, 04 Jun 2022 17:01:37 +0000

array(4) {
  'warning_count' =>
  int(2)
  'warnings' =>
  array(1) {
    [19] =>
    string(27) "The parsed date was invalid"
  }
  'error_count' =>
  int(0)
  'errors' =>
  array(0) {
  }
}
```

Example #8 Greedy parsing behaviour

```php
<?php
print_r(date_parse_from_format('Gis', '60101'));
```

The above example will output something similar to:

```
Array
(
    [year] =>
    [month] =>
    [day] =>
    [hour] => 60
    [minute] => 10
    [second] => 0
    [fraction] => 0
    [warning_count] => 1
    [warnings] => Array
        (
            [5] => The parsed time was invalid
        )

    [error_count] => 1
    [errors] => Array
        (
            [4] => A two digit second could not be found
        )

    [is_localtime] =>
)
```

The `G` format is to parse 24 hour clock hours, with or without leading zero. This requires to parse 1 or 2 digits. Because there are two following digits, it greedily reads this as `60`.

The following `i` and `s` format characters both require two digits. This means that `10` is passed as minute (`i`), and that there are then not enough digits left to parse for as second (`s`).

The `errors` array indicates this problem.

Additionally, an hour of `60` is outside the range `0-24`, which causes the `warnings` array to include a warning that the time is invalid.

See Also

- DateTimeImmutable::__construct() - Returns new DateTimeImmutable object

- DateTimeImmutable::getLastErrors() - Returns the warnings and errors
- checkdate() - Validate a Gregorian date
- strptime() - Parse a time/date generated with strftime

DateTimeImmutable::createFromInterface

DateTimeImmutable::createFromInterface

(PHP 8)

DateTimeImmutable::createFromInterface — Returns new DateTimeImmutable object encapsulating the given DateTimeInterface object

Description

public static function **DateTimeImmutable::createFromInterface**(DateTimeInterface `$object`): DateTimeImmutable

Parameters

`object`

The DateTimeInterface object that needs to be converted to an immutable version. This object is not modified, but instead a new DateTimeImmutable object is created containing the same date, time, and timezone information.

Return Values

Returns a new DateTimeImmutable instance.

Examples

Example #1 Creating an immutable date time object

```php
<?php
$date = new DateTime("2014-06-20 11:45 Europe/London");
$immutable = DateTimeImmutable::createFromInterface($date);

$date = new DateTimeImmutable("2014-06-20 11:45 Europe/London");
$also_immutable = DateTimeImmutable::createFromInterface($date);
```

DateTimeImmutable::createFromMutable

DateTimeImmutable::createFromMutable

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

DateTimeImmutable::createFromMutable — Returns new DateTimeImmutable instance encapsulating the given DateTime object

Description

public static function **DateTimeImmutable::createFromMutable**(DateTime `$object`): static

Parameters

`object`

The mutable DateTime object that you want to convert to an immutable version. This object is not modified, but instead a new DateTimeImmutable instance is created containing the same date time and timezone information.

Return Values

Returns a new DateTimeImmutable instance.

Changelog

Version	Description
8.0.0	The method returns an instance of the currently invoked class now. Previously, it created a new instance of DateTimeImmutable.

Examples

Example #1 Creating an immutable date time object

```php
<?php
$date = new DateTime("2014-06-20 11:45 Europe/London");
$immutable = DateTimeImmutable::createFromMutable( $date );
```

DateTimeImmutable::createFromTimestamp

DateTimeImmutable::createFromTimestamp

(PHP 8 >= 8.4.0)

DateTimeImmutable::createFromTimestamp — Creates an instance from a Unix timestamp

Description

public static function **DateTimeImmutable::createFromTimestamp**(int|float `$timestamp`): static

Creates an instance from a Unix timestamp.

Parameters

`timestamp`

Unix timestamp representing the date. A float value is also accepted which allows for microsecond precision.

Return Values

Returns a new DateTimeImmutable instance.

Errors/Exceptions

If the `timestamp` is outside the range [**PHP_INT_MIN**, **PHP_INT_MAX**], a DateRangeError is thrown.

Examples

Example #1 DateTimeImmutable::createFromTimestamp() example

```php
<?php
$date = DateTimeImmutable::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u');
?>
```

The above example will output:

```
1970-01-01 00:02:03.456789
```

DateTimeImmutable::getLastErrors

DateTimeImmutable::getLastErrors

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::getLastErrors — Returns the warnings and errors

Description

public static function **DateTimeImmutable::getLastErrors**(): array|false

Returns an array of warnings and errors found while parsing a date/time string.

Parameters

This function has no parameters.

Return Values

Returns array containing info about warnings and errors, or `false` if there are neither warnings nor errors.

Changelog

Version	Description
8.2.0	Before PHP 8.2.0, this function did not return `false` when there were no warnings or errors. Instead, it would always return the documented array structure.

Examples

Example #1 DateTimeImmutable::getLastErrors() example

```php
<?php
try {
    $date = new DateTimeImmutable('asdfasdf');
} catch (Exception $e) {
    // For demonstration purposes only...
    print_r(DateTimeImmutable::getLastErrors());

    // The real object-oriented way to do this is
    echo $e->getMessage();
}
?>
```

The above example will output:

```
Array
(
    [warning_count] => 1
    [warnings] => Array
        (
            [6] => Double timezone specification
        )

    [error_count] => 1
    [errors] => Array
        (
            [0] => The timezone could not be found in the database
        )
)
Failed to parse time string (asdfasdf) at position 0 (a): The timezone could not be found in the database
```

The indexes 6, and 0 in the example output refer to the character index in the string where the error occurred.

Example #2 Detecting rolled over dates

```php
<?php
$date = DateTimeImmutable::createFromFormat('!Y-m-d', '2020-02-30');
print_r(DateTimeImmutable::getLastErrors());
```

The above example will output:

```
Array
(
    [warning_count] => 1
    [warnings] => Array
        (
            [10] => The parsed date was invalid
        )
```

```
    [error_count] => 0
    [errors] => Array
        (
        )
)
```

DateTimeImmutable::modify

DateTimeImmutable::modify

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::modify — Creates a new object with modified timestamp

Description

#[\NoDiscard(`message`: "as DateTimeImmutable::modify() does not modify the object itself")]
public function **DateTimeImmutable::modify**(string `$modifier`): DateTimeImmutable

Creates a new DateTimeImmutable object with modified timestamp. The original object is not modified.

Parameters

`modifier`

A date/time string. Valid formats are explained in Date and Time Formats.

Return Values

Returns DateTimeImmutable on success. Procedural style returns **`false`** on failure.

Errors/Exceptions

If an invalid Date/Time string is passed, DateMalformedStringException is thrown. Previous to PHP 8.3, this was a warning.

Changelog

Version	Description
8.4.0	Now has a tentative return type of DateTimeImmutable. Previously it was DateTimeImmutable|false.
8.3.0	**DateTimeImmutable::modify()** now throws DateMalformedStringException if an invalid string is passed. Previously, it returned **`false`**, and a warning was emitted.

Examples

Example #1 DateTimeImmutable::modify() example

Object-oriented style

```php
<?php
$date = new DateTimeImmutable('2006-12-12');
$newDate = $date->modify('+1 day');
echo $newDate->format('Y-m-d');
```

The above example will output:

```
2006-12-13
```

Example #2 Beware when adding or subtracting months

```php
<?php
$date = new DateTimeImmutable('2000-12-31');

$newDate1 = $date->modify('+1 month');
echo $newDate1->format('Y-m-d') . "\n";

$newDate2 = $newDate1->modify('+1 month');
echo $newDate2->format('Y-m-d') . "\n";
```

The above example will output:

```
2001-01-31
2001-03-03
```

See Also

- DateTimeImmutable::add() - Returns a new object, with added amount of days, months, years, hours, minutes and seconds
- DateTimeImmutable::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds
- DateTimeImmutable::setDate() - Sets the date
- DateTimeImmutable::setISODate() - Sets the ISO date
- DateTimeImmutable::setTime() - Sets the time
- DateTimeImmutable::setTimestamp() - Sets the date and time based on a Unix timestamp

DateTimeImmutable::__set_state

DateTimeImmutable::__set_state

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::__set_state — The __set_state handler

Description

public static function **DateTimeImmutable::__set_state**(array `$array`): DateTimeImmutable

The __set_state() handler.

Parameters

`array`

Initialization array.

Return Values

Returns a new instance of a DateTimeImmutable object.

DateTimeImmutable::setDate

DateTimeImmutable::setDate

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setDate — Sets the date

Description

#[\NoDiscard(`message`: "as DateTimeImmutable::setDate() does not modify the object itself")]
public function **DateTimeImmutable::setDate**(int `$year`, int `$month`, int `$day`): DateTimeImmutable

Returns a new DateTimeImmutable object with the current date of the DateTimeImmutable object set to the given date.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create(). The function modifies this object.

`year`

Year of the date.

`month`

Month of the date.

`day`

Day of the date.

Return Values

Returns a new DateTimeImmutable object with the modified data.

Examples

Example #1 DateTimeImmutable::setDate() example

Object-oriented style

```
<?php
$date = new DateTimeImmutable();
$newDate = $date->setDate(2001, 2, 3);
echo $newDate->format('Y-m-d');
```

The above example will output:

```
2001-02-03
```

Example #2 Values exceeding ranges are added to their parent values

```
<?php
$date = new DateTimeImmutable();

$newDate = $date->setDate(2001, 2, 28);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setDate(2001, 2, 29);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setDate(2001, 14, 3);
echo $newDate->format('Y-m-d') . "\n";
```

The above example will output:

```
2001-02-28
2001-03-01
2002-02-03
```

See Also

- DateTimeImmutable::setISODate() - Sets the ISO date

- DateTimeImmutable::setTime() - Sets the time

DateTimeImmutable::setISODate

DateTimeImmutable::setISODate

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setISODate — Sets the ISO date

Description

#[\NoDiscard(message: "as DateTimeImmutable::setISODate() does not modify the object itself")]
public function **DateTimeImmutable::setISODate**(int $year, int $week, int $dayOfWeek = 1): DateTimeImmutable

Returns a new DateTimeImmutable object with the date set according to the ISO 8601 standard - using weeks and day offsets rather than specific dates.

Parameters

`year`

Year of the date.

`week`

Week of the date.

`dayOfWeek`

Offset from the first day of the week.

Return Values

Returns a new DateTimeImmutable object with the modified data.

Examples

Example #1 DateTimeImmutable::setISODate() example

Object-oriented style

```php
<?php

$date = new DateTimeImmutable();

$newDate = $date->setISODate(2008, 2);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setISODate(2008, 2, 7);
echo $newDate->format('Y-m-d') . "\n";
```

The above example will output:

```
2008-01-07
2008-01-13
```

Procedural style

```php
<?php

$date = date_create();

date_isodate_set($date, 2008, 2);
echo date_format($date, 'Y-m-d') . "\n";
```

```php
date_isodate_set($date, 2008, 2, 7);
echo date_format($date, 'Y-m-d') . "\n";
```

The above example will output:

```
2008-01-07
2008-01-13
```

Example #2 Values exceeding ranges are added to their parent values

```php
<?php

$date = new DateTimeImmutable();

$newDate = $date->setISODate(2008, 2, 7);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setISODate(2008, 2, 8);
echo $newDate->format('Y-m-d') . "\n";

$newDate = $date->setISODate(2008, 53, 7);
echo $newDate->format('Y-m-d') . "\n";
```

The above example will output:

```
2008-01-13
2008-01-14
2009-01-04
```

Example #3 Finding the month a week is in

```php
<?php

$date = new DateTimeImmutable();
$newDate = $date->setISODate(2008, 14);
echo $newDate->format('n');
```

The above example will output:

```
3
```

See Also

- DateTimeImmutable::setDate() - Sets the date
- DateTimeImmutable::setTime() - Sets the time

DateTimeImmutable::setMicrosecond

DateTimeImmutable::setMicrosecond

(PHP 8 >= 8.4.0)

DateTimeImmutable::setMicrosecond — Sets microsecond part of the time

Description

#[\NoDiscard(message: "as DateTimeImmutable::setMicrosecond() does not modify the object itself")]
public function **DateTimeImmutable::setMicrosecond**(int $microsecond): static

Returns a new DateTimeImmutable object constructed from the old one, with modified microsecond part.

Parameters

`microsecond`

The microsecond value to set (`0` to `999999`).

Return Values

Returns a new DateTimeImmutable object with the modified data.

Errors/Exceptions

If the `microsecond` is outside the range [`0`, `999999`], a DateRangeError is thrown.

Examples

Example #1 DateTimeImmutable::setMicrosecond() example

```php
<?php
$date = DateTimeImmutable::createFromTimestamp(123.456789);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
$date = $date->setMicrosecond(987654);
echo $date->format('Y-m-d H:i:s.u') . PHP_EOL;
?>
```

The above example will output:

```
1970-01-01 00:02:03.456789
1970-01-01 00:02:03.987654
```

See Also

- DateTimeInterface::getMicrosecond() - Gets the microsecond part of the Unix timestamp

DateTimeImmutable::setTime

DateTimeImmutable::setTime

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setTime — Sets the time

Description

#[\NoDiscard(message: "as DateTimeImmutable::setTime() does not modify the object itself")]
public function **DateTimeImmutable::setTime**(
    int $hour,
    int $minute,
    int $second = 0,
    int $microsecond = 0
): DateTimeImmutable

Returns a new DateTimeImmutable object with the time set to the given time.

Parameters

`hour`

Hour of the time.

`minute`

Minute of the time.

`second`

Second of the time.

`microsecond`

Microsecond of the time.

Return Values

Returns a new DateTimeImmutable object with the modified data.

Changelog

Version	Description
8.1.0	The behaviour with double existing hours (during the fall-back DST transition) changed. Previously PHP would pick the second occurrence (after the DST transition), instead of the first occurrence (before DST transition).
7.1.0	The `microsecond` parameter was added.

Examples

Example #1 DateTimeImmutable::setTime() example

Object-oriented style

```php
<?php
$date = new DateTimeImmutable('2001-01-01');

$newDate = $date->setTime(14, 55);
echo $newDate->format('Y-m-d H:i:s') . "\n";

$newDate = $date->setTime(14, 55, 24);
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

The above example will output something similar to:

```
2001-01-01 14:55:00
2001-01-01 14:55:24
```

Example #2 Values exceeding ranges are added to their parent values

```php
<?php
$date = new DateTimeImmutable('2001-01-01');

$newDate = $date->setTime(14, 55, 24);
echo $newDate->format('Y-m-d H:i:s') . "\n";

$newDate = $date->setTime(14, 55, 65);
echo $newDate->format('Y-m-d H:i:s') . "\n";

$newDate = $date->setTime(14, 65, 24);
echo $newDate->format('Y-m-d H:i:s') . "\n";
```

```php
$newDate = $date->setTime(25, 55, 24);
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

The above example will output:

```
2001-01-01 14:55:24
2001-01-01 14:56:05
2001-01-01 15:05:24
2001-01-02 01:55:24
```

See Also

- DateTimeImmutable::setDate() - Sets the date
- DateTimeImmutable::setISODate() - Sets the ISO date

DateTimeImmutable::setTimestamp

DateTimeImmutable::setTimestamp

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setTimestamp — Sets the date and time based on a Unix timestamp

Description

#[\NoDiscard(message: "as DateTimeImmutable::setTimestamp() does not modify the object itself")]
public function **DateTimeImmutable::setTimestamp**(int $timestamp): DateTimeImmutable

Returns a new DateTimeImmutable object constructed from the old one, with the date and time set based on an Unix timestamp.

Parameters

`timestamp`

> Unix timestamp representing the date. Setting timestamps outside the range of int is possible by using DateTimeImmutable::modify() with the @ format.

Return Values

Returns a new DateTimeImmutable object with the modified data.

Examples

Example #1 DateTimeImmutable::setTimestamp() example

Object-oriented style

```php
<?php
$date = new DateTimeImmutable();
echo $date->format('U = Y-m-d H:i:s') . "\n";

$newDate = $date->setTimestamp(1171502725);
echo $newDate->format('U = Y-m-d H:i:s') . "\n";
```

The above example will output something similar to:

```
1272508903 = 2010-04-28 22:41:43
1171502725 = 2007-02-14 20:25:25
```

See Also

- DateTimeImmutable::getTimestamp() - Gets the Unix timestamp
- DateTimeImmutable::setMicrosecond() - Sets microsecond part of the time

DateTimeImmutable::setTimezone

DateTimeImmutable::setTimezone

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::setTimezone — Sets the time zone

Description

#[\NoDiscard(message: "as DateTimeImmutable::setTimezone() does not modify the object itself")]
public function **DateTimeImmutable::setTimezone**(DateTimeZone `$timezone`): DateTimeImmutable

Returns a new DateTimeImmutable object with a new timezone set.

Parameters

`timezone`

A DateTimeZone object representing the desired time zone.

Return Values

Returns a new modified DateTimeImmutable object for method chaining. The underlying point-in-time is not changed when calling this method.

Examples

Example #1 DateTimeImmutable::setTimeZone() example

Object-oriented style

```
<?php
$date = new DateTimeImmutable('2000-01-01', new DateTimeZone('Pacific/Nauru'));
echo $date->format('Y-m-d H:i:sP') . "\n";

$newDate = $date->setTimezone(new DateTimeZone('Pacific/Chatham'));
echo $newDate->format('Y-m-d H:i:sP') . "\n";
?>
```

The above example will output:

```
2000-01-01 00:00:00+12:00
2000-01-01 01:45:00+13:45
```

See Also

- DateTimeImmutable::getTimezone() - Return time zone relative to given DateTime
- DateTimeZone::__construct() - Creates new DateTimeZone object

DateTimeImmutable::sub

DateTimeImmutable::sub

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

DateTimeImmutable::sub — Subtracts an amount of days, months, years, hours, minutes and seconds

Description

#[\NoDiscard(message: "as DateTimeImmutable::sub() does not modify the object itself")]
public function **DateTimeImmutable::sub**(DateInterval $interval): DateTimeImmutable

Returns a new DateTimeImmutable object, with the specified DateInterval object subtracted from the specified DateTimeImmutable object.

Parameters

`interval`

A DateInterval object

Return Values

Returns a new DateTimeImmutable object with the modified data.

Errors/Exceptions

If an unsupported operation is attempted, such as using a DateInterval object representing relative time specifications such as `next weekday`, a DateInvalidOperationException is thrown.

Changelog

Version	Description
8.3.0	Now throws a DateInvalidOperationException instead of a warning when an unsupported operation is attempted.

Examples

Example #1 DateTimeImmutable::sub() example

Object-oriented style

```php
<?php
$date = new DateTimeImmutable('2000-01-20');
$newDate = $date->sub(new DateInterval('P10D'));
echo $newDate->format('Y-m-d') . "\n";
?>
```

The above example will output:

```
2000-01-10
```

Example #2 Further DateTimeImmutable::sub() examples

```php
<?php
$date = new DateTimeImmutable('2000-01-20');
$newDate = $date->sub(new DateInterval('PT10H30S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";

$date = new DateTimeImmutable('2000-01-20');
$newDate = $date->sub(new DateInterval('P7Y5M4DT4H3M2S'));
echo $newDate->format('Y-m-d H:i:s') . "\n";
?>
```

The above example will output:

```
2000-01-19 13:59:30
1992-08-15 19:56:58
```

Example #3 Beware when subtracting months

```php
<?php
$date = new DateTimeImmutable('2001-04-30');
$interval = new DateInterval('P1M');

$newDate1 = $date->sub($interval);
echo $newDate1->format('Y-m-d') . "\n";

$newDate2 = $newDate1->sub($interval);
echo $newDate2->format('Y-m-d') . "\n";
?>
```

The above example will output:

```
2001-03-30
2001-03-02
```

See Also

- DateTimeImmutable::add() - Returns a new object, with added amount of days, months, years, hours, minutes and seconds
- DateTimeImmutable::diff() - Returns the difference between two DateTime objects
- DateTimeImmutable::modify() - Creates a new object with modified timestamp
- Date/Time Arithmetic

DateTimeInterface

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introduction

DateTimeInterface was created so that parameter, return, or property type declarations may accept either DateTimeImmutable or DateTime as a value. It is not possible to implement this interface with userland classes.

Common constants that allow for formatting DateTimeImmutable or DateTime objects through DateTimeImmutable::format() and DateTime::format() are also defined on this interface.

Interface synopsis

interface **DateTimeInterface** {
/* Constants */
public const string *ATOM* = "Y-m-d\\TH:i:sP";
public const string *COOKIE* = "l, d-M-Y H:i:s T";
public const string *ISO8601* = "Y-m-d\\TH:i:sO";
public const string *ISO8601_EXPANDED* = "X-m-d\\TH:i:sP";
public const string *RFC822* = "D, d M y H:i:s O";
public const string *RFC850* = "l, d-M-y H:i:s T";
public const string *RFC1036* = "D, d M y H:i:s O";
public const string *RFC1123* = "D, d M Y H:i:s O";
public const string *RFC7231* = "D, d M Y H:i:s \\G\\M\\T";
public const string *RFC2822* = "D, d M Y H:i:s O";
public const string *RFC3339* = "Y-m-d\\TH:i:sP";
public const string *RFC3339_EXTENDED* = "Y-m-d\\TH:i:s.vP";
public const string *RSS* = "D, d M Y H:i:s O";
public const string *W3C* = "Y-m-d\\TH:i:sP";
/* Methods */
public function diff(DateTimeInterface `$targetObject`, bool `$absolute` = **`false`**): DateInterval

```
public function format(string $format): string
public function getMicrosecond(): int
public function getOffset(): int
public function getTimestamp(): int
public function getTimezone(): DateTimeZone|false
public function __serialize(): array
public function __unserialize(array $data): void
#[\Deprecated(
    since: '8.5',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function __wakeup(): void
}
```

Predefined Constants

DateTimeInterface::ATOM string
DATE_ATOM

Atom (example: 2005-08-15T15:52:01+00:00); compatible with ISO-8601, RFC 3339, and XML Schema

DateTimeInterface::COOKIE string
DATE_COOKIE

HTTP Cookies (example: Monday, 15-Aug-2005 15:52:01 UTC)

DateTimeInterface::ISO8601 string
DATE_ISO8601

ISO-8601-like (example: 2005-08-15T15:52:01+0000)

> **Note** :
>
> This format is not compatible with ISO-8601, but is left this way for backward compatibility reasons. Use DateTimeInterface::ISO8601_EXPANDED, DateTimeInterface::ATOM for compatibility with ISO-8601 instead. (ref ISO8601:2004 section 4.3.3 clause d)

DateTimeInterface::ISO8601_EXPANDED string
DATE_ISO8601_EXPANDED

ISO-8601 Expanded (example: +10191-07-26T08:59:52+01:00)

> **Note** :
>
> This format allows for year ranges outside of ISO-8601's normal range of `0000-9999` by always including a sign character. It also ensures that the timezone part (`+01:00`) is compatible with ISO-8601.

DateTimeInterface::RFC822 string
DATE_RFC822

RFC 822 (example: Mon, 15 Aug 05 15:52:01 +0000)

DateTimeInterface::RFC850 string
DATE_RFC850

RFC 850 (example: Monday, 15-Aug-05 15:52:01 UTC)

DateTimeInterface::RFC1036 string
DATE_RFC1036

RFC 1036 (example: Mon, 15 Aug 05 15:52:01 +0000)

DateTimeInterface::RFC1123 string
DATE_RFC1123

RFC 1123 (example: Mon, 15 Aug 2005 15:52:01 +0000)

DateTimeInterface::RFC7231 string
DATE_RFC7231

RFC 7231 (since PHP 7.0.19 and 7.1.5) (example: Sat, 30 Apr 2016 17:52:13 GMT)

DateTimeInterface::RFC2822 string
DATE_RFC2822

RFC 2822 (example: Mon, 15 Aug 2005 15:52:01 +0000)

DateTimeInterface::RFC3339 string
DATE_RFC3339

Same as DATE_ATOM

DateTimeInterface::RFC3339_EXTENDED string
DATE_RFC3339_EXTENDED

RFC 3339 EXTENDED format (example: 2005-08-15T15:52:01.000+00:00)

DateTimeInterface::RSS string
DATE_RSS

RSS (example: Mon, 15 Aug 2005 15:52:01 +0000)

DateTimeInterface::W3C string
DATE_W3C

World Wide Web Consortium (example: 2005-08-15T15:52:01+00:00)

Changelog

Version	Description
8.5.0	The `DATE_RFC7231` and `DateTimeInterface::RFC7231` constants have been deprecated.
8.4.0	The class constants are now typed.
8.2.0	The constant `DateTimeInterface::ISO8601_EXPANDED` was added.
7.2.0	The class constants of DateTime are now defined on **DateTimeInterface**.

Table of Contents

DateTimeInterface::diff

DateTimeInterface::diff

DateTimeImmutable::diff

DateTime::diff

date_diff

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTimeInterface::diff -- DateTimeImmutable::diff -- DateTime::diff -- date_diff — Returns the difference between two DateTime objects

Description

Object-oriented style

public function **DateTimeInterface::diff**(DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval
public function **DateTimeImmutable::diff**(DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval
public function **DateTime::diff**(DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval

Procedural style

function date_diff(DateTimeInterface `$baseObject`, DateTimeInterface `$targetObject`, bool `$absolute = false`): DateInterval

Returns the difference between two DateTimeInterface objects.

Parameters

`datetime`

The date to compare to.

`absolute`

Should the interval be forced to be positive?

Return Values

The DateInterval object represents the difference between the two dates.

The `absolute` parameter only affects the `invert` property of a DateInterval object.

The return value more specifically represents the clock-time interval to apply to the original object (`$this` or `$originObject`) to arrive at the `$targetObject`. This process is not always reversible.

The method is aware of DST changeovers, and hence can return an interval of `24 hours and 30 minutes`, as per one of the examples. If you want to calculate with absolute time, you need to convert both the `$this`/`$baseObject`, and `$targetObject` to UTC first.

Examples

Example #1 DateTimeImmutable::diff() example

Object-oriented style

```
<?php
$origin = new DateTimeImmutable('2009-10-11');
$target = new DateTimeImmutable('2009-10-13');
$interval = $origin->diff($target);
echo $interval->format('%R%a days');
```

The above example will output:

```
+2 days
```

Procedural style

```
<?php
$origin = date_create('2009-10-11');
$target = date_create('2009-10-13');
$interval = date_diff($origin, $target);
echo $interval->format('%R%a days');
```

The above example will output:

```
+2 days
```

Example #2 DateTimeInterface::diff() during DST changeover

```
<?php
$originalTime = new DateTimeImmutable("2021-10-30 09:00:00 Europe/London");
$targetTime = new DateTimeImmutable("2021-10-31 08:30:00 Europe/London");
```

```php
$interval = $originalTime->diff($targetTime);
echo $interval->format("%H:%I:%S (Full days: %a)"), "\n";
```

The above example will output:

```
24:30:00 (Full days: 0)
```

Example #3 DateTimeInterface::diff() range

The value that the method returns is the exact amount of time to get from `$this` to `$targetObject`. Comparing January 1st to December 31st returns therefore 364, and not 365, days (for non-leap years).

```php
<?php
$originalTime = new DateTimeImmutable("2023-01-01 UTC");
$targetTime = new DateTimeImmutable("2023-12-31 UTC");
$interval = $originalTime->diff($targetTime);
echo "Full days: ", $interval->format("%a"), "\n";
```

The above example will output:

```
Full days: 364
```

Example #4 DateTime object comparison

> **Note** :
>
> DateTimeImmutable and DateTime objects can be compared using comparison operators.

```php
<?php
$date1 = new DateTime("now");
$date2 = new DateTime("tomorrow");

var_dump($date1 == $date2);
var_dump($date1 < $date2);
var_dump($date1 > $date2);
```

The above example will output:

```
bool(false)
bool(true)
bool(false)
```

See Also

- DateInterval::format() - Formats the interval
- DateTime::add() - Modifies a DateTime object, with added amount of days, months, years, hours, minutes and seconds
- DateTime::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds from a DateTime object

DateTimeInterface::format

DateTimeInterface::format

DateTimeImmutable::format

DateTime::format

date_format

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeInterface::format -- DateTimeImmutable::format -- DateTime::format -- date_format — Returns date formatted according to given format

Description

Object-oriented style

public function **DateTimeInterface::format**(string `$format`): string
public function **DateTimeImmutable::format**(string `$format`): string
public function **DateTime::format**(string `$format`): string

Procedural style

function date_format(DateTimeInterface `$object`, string `$format`): string

Returns date formatted according to given format.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create()

`format`

The format of the outputted date string. See the formatting options below. There are also several predefined date constants that may be used instead, so for example **`DATE_RSS`** contains the format string `'D, d M Y H:i:s'`.

The following characters are recognized in the `format` parameter string

`format` character	Description	Example returned values
Day	---	---
`d`	Day of the month, 2 digits with leading zeros	`01` to `31`
`D`	A textual representation of a day, three letters	`Mon` through `Sun`
`j`	Day of the month without leading zeros	`1` to `31`
`l` (lowercase 'L')	A full textual representation of the day of the week	`Sunday` through `Saturday`
`N`	ISO 8601 numeric representation of the day of the week	`1` (for Monday) through `7` (for Sunday)
`S`	English ordinal suffix for the day of the month, 2 characters	`st`, `nd`, `rd` or `th`. Works well with `j`
`w`	Numeric representation of the day of the week	`0` (for Sunday) through `6` (for Saturday)
`z`	The day of the year (starting from 0)	`0` through `365`
Week	---	---
`W`	ISO 8601 week number of year, weeks starting on Monday	Example: `42` (the 42nd week in the year)
Month	---	---
`F`	A full textual representation of a month, such as January or March	`January` through `December`
`m`	Numeric representation of a month, with leading zeros	`01` through `12`
`M`	A short textual representation of a month, three letters	`Jan` through `Dec`

`format` character	Description	Example returned values
`n`	Numeric representation of a month, without leading zeros	`1` through `12`
`t`	Number of days in the given month	`28` through `31`
Year	---	---
`L`	Whether it's a leap year	`1` if it is a leap year, `0` otherwise.
`o`	ISO 8601 week-numbering year. This has the same value as `Y`, except that if the ISO week number (`W`) belongs to the previous or next year, that year is used instead.	Examples: `1999` or `2003`
`X`	An expanded full numeric representation of a year, at least 4 digits, with `-` for years BCE, and `+` for years CE.	Examples: `-0055`, `+0787`, `+1999`, `+10191`
`x`	An expanded full numeric representation if required, or a standard full numeral representation if possible (like `Y`). At least four digits. Years BCE are prefixed with a `-`. Years beyond (and including) `10000` are prefixed by a `+`.	Examples: `-0055`, `0787`, `1999`, `+10191`
`Y`	A full numeric representation of a year, at least 4 digits, with `-` for years BCE.	Examples: `-0055`, `0787`, `1999`, `2003`, `10191`
`y`	A two digit representation of a year	Examples: `99` or `03`
Time	---	---
`a`	Lowercase Ante meridiem and Post meridiem	`am` or `pm`
`A`	Uppercase Ante meridiem and Post meridiem	`AM` or `PM`
`B`	Swatch Internet time	`000` through `999`
`g`	12-hour format of an hour without leading zeros	`1` through `12`
`G`	24-hour format of an hour without leading zeros	`0` through `23`
`h`	12-hour format of an hour with leading zeros	`01` through `12`
`H`	24-hour format of an hour with leading zeros	`00` through `23`
`i`	Minutes with leading zeros	`00` to `59`
`s`	Seconds with leading zeros	`00` through `59`
`u`	Microseconds. Note that date() will always generate `000000` since it takes an int parameter, whereas **DateTimeInterface::format()** does support microseconds if an object of type DateTimeInterface was created with microseconds.	Example: `654321`
`v`	Milliseconds. Same note applies as for `u`.	Example: `654`
Timezone	---	---
`e`	Timezone identifier	Examples: `UTC`, `GMT`, `Atlantic/Azores`
`I` (capital i)	Whether or not the date is in daylight saving time	`1` if Daylight Saving Time, `0` otherwise.
`O`	Difference to Greenwich time (GMT) without colon between hours and minutes	Example: `+0200`
`P`	Difference to Greenwich time (GMT) with colon between hours and minutes	Example: `+02:00`
`p`	The same as `P`, but returns `Z` instead of `+00:00` (available as of PHP 8.0.0)	Examples: `Z` or `+02:00`
`T`	Timezone abbreviation, if known; otherwise the GMT offset.	Examples: `EST`, `MDT`, `+05`
`Z`	Timezone offset in seconds. The offset for timezones west of UTC is always negative, and for those east of UTC is always positive.	`-43200` through `50400`
Full Date/Time	---	---
`c`	ISO 8601 date. Only compatible with the non-expanded format (up to year 9999). Later dates will result in an invalid string. For later dates and expanded format, see `x` and `X`.	2004-02-12T15:19:21+00:00
`r`	» RFC 2822/» RFC 5322 formatted date	Example: `Thu, 21 Dec 2000 16:01:07 +0200`
`U`	Seconds since the Unix Epoch (January 1 1970 00:00:00 GMT)	See also time()

Unrecognized characters in the format string will be printed as-is. The `z` format will always return `0` when using gmdate().

> **Note** :
>
> Since this function only accepts int timestamps the `u` format character is only useful when using the date_format() function with user based timestamps created with date_create().

Return Values

Returns the formatted date string on success.

Changelog

Version	Description
8.2.0	The format characters `X` and `x` have been added.
8.0.0	The format character `p` has been added.

Examples

Example #1 DateTimeInterface::format() example

Object-oriented style

```
<?php
$date = new DateTimeImmutable('2000-01-01');
echo $date->format('Y-m-d H:i:s');
?>
```

The above example will output:

```
2000-01-01 00:00:00
```

Procedural style

```
<?php
$date = date_create('2000-01-01');
echo date_format($date, 'Y-m-d H:i:s');
?>
```

The above example will output:

```
2000-01-01 00:00:00
```

Example #2 More examples

```
<?php
// set the default timezone to use.
date_default_timezone_set('UTC');

// now
$date = new DateTimeImmutable();

// Prints something like: Wednesday
echo $date->format('l'), "\n";
```

```php
// Prints something like: Wednesday 19th of October 2022 08:40:48 AM
echo $date->format('l jS \o\f F Y h:i:s A'), "\n";

/* use the constants in the format parameter */
// prints something like: Wed, 19 Oct 2022 08:40:48 +0000
echo $date->format(DateTimeInterface::RFC2822), "\n";
?>
```

You can prevent a recognized character in the format string from being expanded by escaping it with a preceding backslash. If the character with a backslash is already a special sequence, you may need to also escape the backslash.

Example #3 Escaping characters while formatting

```php
<?php
$date = new DateTimeImmutable();

// prints something like: Wednesday the 19th
echo $date->format('l \t\h\e jS');
?>
```

To format dates in other languages, IntlDateFormatter::format() can be used instead of **DateTimeInterface::format()**.

Notes

This method does not use locales. All output is in English.

See Also

- IntlDateFormatter::format() - Format the date/time value as a string

DateTimeInterface::getMicrosecond

DateTimeInterface::getMicrosecond

DateTimeImmutable::getMicrosecond

DateTime::getMicrosecond

(PHP 8 >= 8.4.0)

DateTimeInterface::getMicrosecond -- DateTimeImmutable::getMicrosecond -- DateTime::getMicrosecond — Gets the microsecond part of the Unix timestamp

Description

public function **DateTimeInterface::getMicrosecond**(): int
public function **DateTimeImmutable::getMicrosecond**(): int
public function **DateTime::getMicrosecond**(): int

Gets the microsecond part of the Unix timestamp.

Parameters

This function has no parameters.

Return Values

Returns the microsecond part of the Unix timestamp representing the date.

Examples

Example #1 DateTimeInterface::getMicrosecond() example

```php
<?php
$date = new DateTimeImmutable('2024-01-01 12:34:56.789123');
var_dump($date->format('u'));
var_dump($date->getMicrosecond());
?>
```

The above example will output:

```
string(6) "789123"
int(789123)
```

See Also

- DateTimeInterface::getTimestamp() - Gets the Unix timestamp
- DateTimeInterface::format() - Returns date formatted according to given format

DateTimeInterface::getOffset

DateTimeInterface::getOffset

DateTimeImmutable::getOffset

DateTime::getOffset

date_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeInterface::getOffset -- DateTimeImmutable::getOffset -- DateTime::getOffset -- date_offset_get — Returns the timezone offset

Description

Object-oriented style

public function **DateTimeInterface::getOffset**(): int
public function **DateTimeImmutable::getOffset**(): int
public function **DateTime::getOffset**(): int

Procedural style

function date_offset_get(DateTimeInterface `$object`): int

Returns the timezone offset.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create()

Return Values

Returns the timezone offset in seconds from UTC on success.

Examples

Example #1 DateTime::getOffset() example

Object-oriented style

```php
<?php
$winter = new DateTimeImmutable('2010-12-21', new DateTimeZone('America/New_York'));
$summer = new DateTimeImmutable('2008-06-21', new DateTimeZone('America/New_York'));

echo $winter->getOffset() . "\n";
echo $summer->getOffset() . "\n";
```

The above example will output:

```
-18000
-14400
```

Procedural style

```php
<?php
$winter = date_create('2010-12-21', timezone_open('America/New_York'));
$summer = date_create('2008-06-21', timezone_open('America/New_York'));

echo date_offset_get($winter) . "\n";
echo date_offset_get($summer) . "\n";
```

The above example will output:

```
-18000
-14400
```

Note: -18000 = -5 hours, -14400 = -4 hours.

DateTimeInterface::getTimestamp

DateTimeInterface::getTimestamp

DateTimeImmutable::getTimestamp

DateTime::getTimestamp

date_timestamp_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTimeInterface::getTimestamp -- DateTimeImmutable::getTimestamp -- DateTime::getTimestamp -- date_timestamp_get — Gets the Unix timestamp

Description

Object-oriented style

public function **DateTimeInterface::getTimestamp**(): int
public function **DateTimeImmutable::getTimestamp**(): int
public function **DateTime::getTimestamp**(): int

Procedural style

function date_timestamp_get(DateTimeInterface `$object`): int

Gets the Unix timestamp.

Parameters

This function has no parameters.

Return Values

Returns the Unix timestamp representing the date.

Errors/Exceptions

If the timestamp cannot be represented as int, a DateRangeError is thrown. Prior to PHP 8.3.0, a ValueError is thrown. And, prior to PHP 8.0.0, `false` was returned in this case. Still, the timestamp can be retrieved as string by using DateTimeInterface::format() with the `U` format.

Changelog

Version	Description
8.3.0	The out-of-range exception is now DateRangeError.
8.0.0	These functions no longer return `false` on failure.

Examples

Example #1 DateTime::getTimestamp() example

Object-oriented style

```
<?php
$date = new DateTimeImmutable();
echo $date->getTimestamp();
```

The above example will output something similar to:

```
1272509157
```

Procedural style

```
<?php
$date = date_create();
echo date_timestamp_get($date);
```

The above example will output something similar to:

```
1272509157
```

If you need to retrieve the timestamp with millisecond or microsecond resolution, then you can use the DateTimeInterface::format() function.

Example #2 Retrieving timestamp with milli and microsecond resolution

Object-oriented style

```
<?php
$date = new DateTimeImmutable();
$milli = (int) $date->format('Uv'); // Timestamp in milliseconds
$micro = (int) $date->format('Uu'); // Timestamp in microseconds

echo $milli, "\n", $micro, "\n";
```

The above example will output something similar to:

```
1674057635586
1674057635586918
```

See Also

- DateTime::setTimestamp() - Sets the date and time based on an Unix timestamp
- DateTimeImmutable::setTimestamp() - Sets the date and time based on a Unix timestamp
- DateTimeInterface::format() - Returns date formatted according to given format
- DateTimeInterface::getMicrosecond() - Gets the microsecond part of the Unix timestamp

DateTimeInterface::getTimezone

DateTimeInterface::getTimezone

DateTimeImmutable::getTimezone

DateTime::getTimezone

date_timezone_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeInterface::getTimezone -- DateTimeImmutable::getTimezone -- DateTime::getTimezone -- date_timezone_get — Return time zone relative to given DateTime

Description

Object-oriented style

public function **DateTimeInterface::getTimezone**(): DateTimeZone|false
public function **DateTimeImmutable::getTimezone**(): DateTimeZone|false
public function **DateTime::getTimezone**(): DateTimeZone|false

Procedural style

function date_timezone_get(DateTimeInterface `$object`): DateTimeZone|false

Return time zone relative to given DateTime.

Parameters

`object`

Procedural style only: A DateTime object returned by date_create()

Return Values

Returns a DateTimeZone object on success or `false` on failure.

Examples

Example #1 DateTime::getTimezone() example

Object-oriented style

```
<?php
$date = new DateTimeImmutable("now", new DateTimeZone('Europe/London'));
$tz = $date->getTimezone();
echo $tz->getName();
```

The above example will output:

```
Europe/London
```

Procedural style

```
<?php
$date = date_create("now", timezone_open('Europe/London'));
```

```php
$tz = date_timezone_get($date);
echo timezone_name_get($tz);
```

The above example will output:

```
Europe/London
```

See Also

- DateTime::setTimezone() - Sets the time zone for the DateTime object

DateTime::__serialize

DateTime::__serialize

DateTimeImmutable::__serialize

DateTimeInterface::__serialize

(PHP 8 >= 8.2.0)

DateTime::__serialize -- DateTimeImmutable::__serialize -- DateTimeInterface::__serialize — Serialize a DateTime

Description

public function **DateTime::__serialize**(): array
public function **DateTimeImmutable::__serialize**(): array
public function **DateTimeInterface::__serialize**(): array

The __serialize() handler.

Parameters

This function has no parameters.

Return Values

The serialized representation of the DateTime object.

Examples

Example #1 DateTime::__serialize() example

```php
<?php

class CustomDateTimeImmutable extends DateTimeImmutable
{
    #[\Override]
    public function __serialize(): array
    {
        return [
            'date' => $this->format(DateTimeInterface::W3C),
            'timestamp' => $this->getTimestamp(),
            'timezone' => $this->getTimeZone()->getName(),
            'to' => 'Drink a cup of coffee',
        ];
    }
}

$date = new CustomDateTimeImmutable('2025-03-27');
var_dump(serialize($date));
```

The above example will output:

```
string(171) "O:23:"CustomDateTimeImmutable":4:{s:4:"date";s:25:"2025-03-
27T00:00:00+00:00";s:9:"timestamp";i:1743033600;s:8:"timezone";s:3:"UTC";s:2:"to";s:21:"Drink a cup of coffee";}"
```

Notes

> **Warning**
>
> An Error is thrown when attempting to unserialize a custom DateTime object if __serialize() is defined but __unserialize() is missing.

See Also

- DateTime::__unserialize() - Unserialize an Datetime

DateTime::__unserialize

DateTime::__unserialize

DateTimeImmutable::__unserialize

DateTimeInterface::__unserialize

(PHP 8 >= 8.2.0)

DateTime::__unserialize -- DateTimeImmutable::__unserialize -- DateTimeInterface::__unserialize — Unserialize an Datetime

Description

public function **DateTime::__unserialize**(array `$data`): void
public function **DateTimeImmutable::__unserialize**(array `$data`): void
public function **DateTimeInterface::__unserialize**(array `$data`): void

The __unserialize() handler.

Parameters

`data`

The serialized DateTime.

Return Values

No value is returned.

Examples

Example #1 DateTime::__unserialize() example

```php
<?php

class CustomDateTimeImmutable extends DateTimeImmutable
{
    #[\Override]
    public function __unserialize(array $data): void
    {
        echo "Time to `{$data['to']}`:\n\n";

        parent::__construct($data['date'], new DateTimeZone($data['timezone']));
    }
}

$serializedData = 'O:23:"CustomDateTimeImmutable":4:{s:4:"date";s:25:"2025-03-
27T00:00:00+00:00";s:9:"timestamp";i:1743033600;s:8:"timezone";s:3:"UTC";s:2:"to";s:21:"Drink a cup of coffee";}';

var_dump(unserialize($serializedData));
```

The above example will output:

```
Time to `Drink a cup of coffee`:

object(CustomDateTimeImmutable)#1 (3) {
  ["date"]=>
  string(26) "2025-03-27 00:00:00.000000"
  ["timezone_type"]=>
  int(1)
  ["timezone"]=>
  string(6) "+00:00"
}
```

Notes

> **Warning**
>
> An Error is thrown when attempting to unserialize a custom DateTime object if __serialize() is defined but __unserialize() is missing.

See Also

- DateTime::__serialize() - Serialize a DateTime

DateTime::__wakeup

DateTime::__wakeup

DateTimeImmutable::__wakeup

DateTimeInterface::__wakeup

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTime::__wakeup -- DateTimeImmutable::__wakeup -- DateTimeInterface::__wakeup — The __wakeup handler

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.5',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function DateTime::__wakeup(): void
#[\Deprecated(
    since: '8.5',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function DateTimeImmutable::__wakeup(): void
#[\Deprecated(
    since: '8.5',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function DateTimeInterface::__wakeup(): void
```

The __wakeup() handler.

Parameters

This function has no parameters.

Return Values

Initializes a DateTime object.

DateTimeZone

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introduction

Representation of time zone.

Class synopsis

```
class DateTimeZone {
/* Constants */
public const int AFRICA;
public const int AMERICA;
public const int ANTARCTICA;
public const int ARCTIC;
public const int ASIA;
public const int ATLANTIC;
public const int AUSTRALIA;
public const int EUROPE;
public const int INDIAN;
public const int PACIFIC;
public const int UTC;
public const int ALL;
public const int ALL_WITH_BC;
public const int PER_COUNTRY;
/* Methods */
public function __construct(string $timezone)
public function getLocation(): array|false
public function getName(): string
public function getOffset(DateTimeInterface $datetime): int
public function getTransitions(int $timestampBegin = PHP_INT_MIN, int $timestampEnd = 2147483647): array|false
public static function listAbbreviations(): array
public static function listIdentifiers(int $timezoneGroup = DateTimeZone::ALL, ?string $countryCode = null): array
}
```

Predefined Constants

`DateTimeZone::AFRICA` int

Africa time zones.

`DateTimeZone::AMERICA` int

America time zones.

`DateTimeZone::ANTARCTICA` int

Antarctica time zones.

`DateTimeZone::ARCTIC` int

Arctic time zones.

`DateTimeZone::ASIA` int

Asia time zones.

`DateTimeZone::ATLANTIC` int

Atlantic time zones.

`DateTimeZone::AUSTRALIA` int

Australia time zones.

`DateTimeZone::EUROPE` int

Europe time zones.

`DateTimeZone::INDIAN` int

Indian time zones.

`DateTimeZone::PACIFIC` int

Pacific time zones.

`DateTimeZone::UTC` int

UTC time zones.

`DateTimeZone::ALL` int

All time zones.

`DateTimeZone::ALL_WITH_BC` int

All time zones including backwards compatible.

`DateTimeZone::PER_COUNTRY` int

Time zones per country.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

DateTimeZone::__construct

DateTimeZone::__construct

timezone_open

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::__construct -- timezone_open — Creates new DateTimeZone object

Description

Object-oriented style

public function **DateTimeZone::__construct**(string `$timezone`)

Procedural style

function timezone_open(string `$timezone`): DateTimeZone|false

Creates a new DateTimeZone object.

A DateTimeZone object provides access to three different types of timezone rules: UTC offset (type `1`), timezone abbreviation (type `2`), and timezone identifiers as published in the IANA timezone database (type `3`).

The DateTimeZone object can be attached to DateTime and DateTimeImmutable objects to be able to render the timezone encapsulated by these objects in a local timezone.

Parameters

`timezone`

One of the supported timezone names, an offset value (+0200), or a timezone abbreviation (BST).

Return Values

Returns DateTimeZone on success. Procedural style returns `false` on failure.

Errors/Exceptions

This method throws DateInvalidTimeZoneException if the timezone supplied is not recognised as a valid timezone. Prior to PHP 8.3, this was an Exception instead.

Changelog

Version	Description
8.3.0	Invalid values now return a DateInvalidTimeZoneException instead of a generic Exception.

Examples

Example #1 Creating and attaching DateTimeZone to a DateTimeImmutable

```php
<?php
$d = new DateTimeImmutable("2022-06-02 15:44:48 UTC");

$timezones = [ 'Europe/London', 'GMT+04:45', '-06:00', 'CEST' ];

foreach ($timezones as $tz) {
    $tzo = new DateTimeZone($tz);

    $local = $d->setTimezone($tzo);
    echo $local->format(DateTimeInterface::RFC2822 . ' — e') . "\n";
}
```

The above example will output:

```
Thu, 02 Jun 2022 16:44:48 +0100 — Europe/London
Thu, 02 Jun 2022 20:29:48 +0445 — +04:45
Thu, 02 Jun 2022 09:44:48 -0600 — -06:00
Thu, 02 Jun 2022 17:44:48 +0200 — CEST
```

Example #2 Catching errors when instantiating DateTimeZone

```php
<?php
// Error handling by catching exceptions
$timezones = array('Europe/London', 'Mars/Phobos', 'Jupiter/Europa');

foreach ($timezones as $tz) {
    try {
        $mars = new DateTimeZone($tz);
        echo $mars->getName() . "\n";
    } catch(Exception $e) {
        echo $e->getMessage() . "\n";
    }
}
```

The above example will output:

```
Europe/London
DateTimeZone::__construct() [datetimezone.--construct]: Unknown or bad timezone (Mars/Phobos)
DateTimeZone::__construct() [datetimezone.--construct]: Unknown or bad timezone (Jupiter/Europa)
```

DateTimeZone::getLocation

DateTimeZone::getLocation

timezone_location_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateTimeZone::getLocation -- timezone_location_get — Returns location information for a timezone

Description

Object-oriented style

public function **DateTimeZone::getLocation**(): array|false

Procedural style

function timezone_location_get(DateTimeZone `$object`): array|false

Returns location information for a timezone, including country code, latitude/longitude and comments.

Parameters

`object`

Procedural style only: A DateTimeZone object returned by timezone_open()

Return Values

Array containing location information about timezone or **`false`** on failure.

Examples

Example #1 DateTimeZone::getLocation() example

```php
<?php
$tz = new DateTimeZone("Asia/Jakarta");
print_r($tz->getLocation());
print_r(timezone_location_get($tz));
```

The above example will output:

```
Array
(
    [country_code] => ID
    [latitude] => -6.16667
    [longitude] => 106.8
    [comments] => Java, Sumatra
)
Array
(
    [country_code] => ID
    [latitude] => -6.16667
    [longitude] => 106.8
    [comments] => Java, Sumatra
)
```

The `country_code` elements contains the ISO 3166-1 alpha-2 country code for each entry. The `latitude` and `longitude` elements the coordinates of the named city from the time zone identifier, and `comments` contain (when not **`false`**) a hint of where in the given country this timezone applies. This information is suitable to present to end-users.

See Also

- DateTimeZone::listIdentifiers() - Returns a numerically indexed array containing all defined timezone identifiers to get a full or partial list of all supported timezone identifiers

DateTimeZone::getName

DateTimeZone::getName

timezone_name_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::getName -- timezone_name_get — Returns the name of the timezone

Description

Object-oriented style

public function **DateTimeZone::getName**(): string

Procedural style

function timezone_name_get(DateTimeZone `$object`): string

Returns the name of the timezone.

Parameters

`object`

The DateTimeZone for which to get a name.

Return Values

Depending on zone type, UTC offset (type 1), timezone abbreviation (type 2), and timezone identifiers as published in the IANA timezone database (type 3), the descriptor string to create a new DateTimeZone object with the same offset and/or rules. For example `02:00`, `CEST`, or one of the timezone names in the list of timezones.

DateTimeZone::getOffset

DateTimeZone::getOffset

timezone_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::getOffset -- timezone_offset_get — Returns the timezone offset from GMT

Description

Object-oriented style

public function **DateTimeZone::getOffset**(DateTimeInterface `$datetime`): int

Procedural style

function timezone_offset_get(DateTimeZone `$object`, DateTimeInterface `$datetime`): int

This function returns the offset to GMT for the date/time specified in the `datetime` parameter. The GMT offset is calculated with the timezone information contained in the DateTimeZone object being used.

Parameters

`object`

Procedural style only: A DateTimeZone object returned by timezone_open()

`datetime`

DateTime that contains the date/time to compute the offset from.

Return Values

Returns time zone offset in seconds.

Examples

Example #1 DateTimeZone::getOffset() examples

```php
<?php
// Create two timezone objects, one for Taipei (Taiwan) and one for
// Tokyo (Japan)
$dateTimeZoneTaipei = new DateTimeZone("Asia/Taipei");
$dateTimeZoneJapan = new DateTimeZone("Asia/Tokyo");

// Create two DateTime objects that will contain the same Unix timestamp, but
// have different timezones attached to them.
$dateTimeTaipei = new DateTime("now", $dateTimeZoneTaipei);
$dateTimeJapan = new DateTime("now", $dateTimeZoneJapan);

// Calculate the GMT offset for the date/time contained in the $dateTimeTaipei
// object, but using the timezone rules as defined for Tokyo
// ($dateTimeZoneJapan).
$timeOffset = $dateTimeZoneJapan->getOffset($dateTimeTaipei);

// Should show int(32400) (for dates after Sat Sep 8 01:00:00 1951 JST).
var_dump($timeOffset);
```

The above example will output:

```
int(32400)
```

DateTimeZone::getTransitions

DateTimeZone::getTransitions

timezone_transitions_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::getTransitions -- timezone_transitions_get — Returns all transitions for the timezone

Description

Object-oriented style

public function **DateTimeZone::getTransitions**(int `$timestampBegin` = **`PHP_INT_MIN`**, int `$timestampEnd` = 2147483647): array|false

Procedural style

function timezone_transitions_get(DateTimeZone `$object`, int `$timestampBegin` = **`PHP_INT_MIN`**, int `$timestampEnd` = 2147483647): array|false

Parameters

`object`

Procedural style only: A DateTimeZone object returned by timezone_open()

`timestampBegin`

Begin timestamp.

`timestampEnd`

End timestamp.

Return Values

Returns a numerically indexed array of transition arrays on success, or `false` on failure. DateTimeZone objects wrapping type 1 (UTC offsets) and type 2 (abbreviations) do not contain any transitions, and calling this method on them will return `false`.

If `timestampBegin` is given, the first entry in the returned array will contain a transition element at the time of `timestampBegin`.

Transition Array Structure

Key	Type	Description
`ts`	int	Unix timestamp
`time`	string	`DateTimeInterface::ISO8601_EXPANDED` (PHP 8.2 and later), or `DateTimeInterface::ISO8601` (PHP 8.1 and lower) time string
`offset`	int	Offset to UTC in seconds
`isdst`	bool	Whether daylight saving time is active
`abbr`	string	Timezone abbreviation

Changelog

Version	Description
8.1.0	The default value of `timestampEnd` has been changed to 2147483647. Previously, it was `PHP_INT_MAX`.

Examples

Example #1 A timezone_transitions_get() example

```php
<?php
$timezone = new DateTimeZone("Europe/London");
$transitions = $timezone->getTransitions();
print_r(array_slice($transitions, 0, 3));
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [ts] => -2147483648
            [time] => 1901-12-13T20:45:52+00:00
            [offset] => -75
            [isdst] =>
            [abbr] => LMT
        )

    [1] => Array
        (
            [ts] => 442304971
            [time] => 1847-12-01T00:01:15+00:00
            [offset] => 0
            [isdst] =>
            [abbr] => GMT
        )

    [2] => Array
        (
            [ts] => -1691964000
            [time] => 1916-05-21T02:00:00+00:00
            [offset] => 3600
            [isdst] => 1
            [abbr] => BST
        )

)
```

Example #2 A timezone_transitions_get() example with `timestampBegin` set

```php
<?php
$timezone = new DateTimeZone("Europe/London");
$transitions = $timezone->getTransitions(time());
print_r(array_slice($transitions, 0, 3));
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [ts] => 1759058251
            [time] => 2025-09-28T11:17:31+00:00
            [offset] => 3600
            [isdst] => 1
            [abbr] => BST
        )

    [1] => Array
        (
            [ts] => 1761440400
            [time] => 2025-10-26T01:00:00+00:00
            [offset] => 0
            [isdst] =>
            [abbr] => GMT
        )

    [2] => Array
        (
            [ts] => 1774746000
            [time] => 2026-03-29T01:00:00+00:00
            [offset] => 3600
            [isdst] => 1
            [abbr] => BST
        )

)
```

DateTimeZone::listAbbreviations

DateTimeZone::listAbbreviations

timezone_abbreviations_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::listAbbreviations -- timezone_abbreviations_list — Returns associative array containing dst, offset and the timezone name

Description

Object-oriented style

public static function **DateTimeZone::listAbbreviations**(): array

Procedural style

function timezone_abbreviations_list(): array

The returned list of abbreviations includes all historical use of abbreviations, which can lead to correct, but confusing entries. There are also conflicts, as PST is used both in the US and in the Philippines.

The list that this function returns is therefore not suitable for building an array with options to present a choice of timezone to users.

> **Note** :
>
> The data for this function are precompiled for performance reasons, and are not updated when using a newer » timezonedb.

Parameters

This function has no parameters.

Return Values

Returns the array of timezone abbreviations.

Examples

Example #1 A timezone_abbreviations_list() example

```
<?php
$timezone_abbreviations = DateTimeZone::listAbbreviations();
print_r($timezone_abbreviations["acst"]);
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/Adelaide
        )

    [1] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/Broken_Hill
        )

    [2] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/Darwin
        )

    [3] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/North
        )

    [4] => Array
        (
            [dst] =>
            [offset] => 34200
            [timezone_id] => Australia/South
        )

    [5] => Array
        (
            [dst] =>
```

```
            [offset] => 34200
            [timezone_id] => Australia/Yancowinna
        )

)
```

See Also

- timezone_identifiers_list() - Alias of DateTimeZone::listIdentifiers

DateTimeZone::listIdentifiers

DateTimeZone::listIdentifiers

timezone_identifiers_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

DateTimeZone::listIdentifiers -- timezone_identifiers_list — Returns a numerically indexed array containing all defined timezone identifiers

Description

Object-oriented style

public static function **DateTimeZone::listIdentifiers**(int `$timezoneGroup` = **`DateTimeZone::ALL`**, ?string `$countryCode` = **`null`**): array

Procedural style

function timezone_identifiers_list(int `$timezoneGroup` = **`DateTimeZone::ALL`**, ?string `$countryCode` = **`null`**): array

Return the list of » IANA Time Zone identifiers.

> **Note** :
>
> It's possible to detect the client (browser) timezone with JavaScript using » Intl.DateTimeFormat or » Temporal.ZonedDateTime.

Parameters

`timezoneGroup`

One of the DateTimeZone class constants (or a combination).

`countryCode`

A two-letter (uppercase) ISO 3166-1 compatible country code.

> **Note** :
>
> This option is only used when `timezoneGroup` is set to **`DateTimeZone::PER_COUNTRY`**.

Return Values

Returns the array of timezone identifiers. Only non-outdated items are returned. To get all, including outdated timezone identifiers, use the `DateTimeZone::ALL_WITH_BC` as value for `timezoneGroup`.

Changelog

Version	Description
8.0.0	Prior to this version, **`false`** was returned on failure.
7.1.0	`countryCode` is nullable now.

Examples

Example #1 List identifiers with location comments

```php
<?php
$identifiers = DateTimeZone::listIdentifiers(DateTimeZone::ALL);

foreach ($identifiers as $tzid) {
    $tz = new DateTimeZone($tzid);
    $comments = $tz->getLocation()['comments'];
    echo $tzid . " (" . ($comments ?: 'Whole region') . ")\n";
}
```

The above example will output something similar to:

```
America/Antigua (Whole region)
America/Araguaina (Tocantins)
America/Argentina/Buenos_Aires (Buenos Aires (BA, CF))
America/Argentina/Catamarca (Catamarca (CT), Chubut (CH))
America/Argentina/Cordoba (Argentina (most areas: CB, CC, CN, ER, FM, MN, SE, SF))
// (Output trimmed due to length)
```

Example #2 Listing identifiers for a specific region

```php
<?php
$timezone_identifiers = DateTimeZone::listIdentifiers( DateTimeZone::ASIA );
for ($i=0; $i < 5; $i++) {
    echo "$timezone_identifiers[$i]\n";
}
```

The above example will output something similar to:

```
Asia/Aden
Asia/Almaty
Asia/Amman
Asia/Anadyr
Asia/Aqtau
```

Example #3 Listing identifiers for multiple regions

```php
<?php
$timezone_identifiers = DateTimeZone::listIdentifiers( DateTimeZone::ASIA | DateTimeZone::PACIFIC );
echo join( ', ', $timezone_identifiers );
```

The above example will output something similar to:

```
Asia/Aden, Asia/Almaty, Asia/Amman, Asia/Anadyr, Asia/Aqtau, Asia/Aqtobe,
Asia/Ashgabat, Asia/Atyrau, Asia/Baghdad, Asia/Bahrain, Asia/Baku,
Asia/Bangkok, Asia/Barnaul, Asia/Beirut, Asia/Bishkek, Asia/Brunei,
Asia/Chita, Asia/Choibalsan, Asia/Colombo, Asia/Damascus, Asia/Dhaka,
Asia/Dili, Asia/Dubai, Asia/Dushanbe, Asia/Famagusta, Asia/Gaza, Asia/Hebron,
Asia/Ho_Chi_Minh, Asia/Hong_Kong, Asia/Hovd, Asia/Irkutsk, Asia/Jakarta,
Asia/Jayapura, Asia/Jerusalem, Asia/Kabul, Asia/Kamchatka, Asia/Karachi,
Asia/Kathmandu, Asia/Khandyga, Asia/Kolkata, Asia/Krasnoyarsk,
Asia/Kuala_Lumpur, Asia/Kuching, Asia/Kuwait, Asia/Macau, Asia/Magadan,
Asia/Makassar, Asia/Manila, Asia/Muscat, Asia/Nicosia, Asia/Novokuznetsk,
Asia/Novosibirsk, Asia/Omsk, Asia/Oral, Asia/Phnom_Penh, Asia/Pontianak,
```

```
Asia/Pyongyang, Asia/Qatar, Asia/Qostanay, Asia/Qyzylorda, Asia/Riyadh,
Asia/Sakhalin, Asia/Samarkand, Asia/Seoul, Asia/Shanghai, Asia/Singapore,
Asia/Srednekolymsk, Asia/Taipei, Asia/Tashkent, Asia/Tbilisi, Asia/Tehran,
Asia/Thimphu, Asia/Tokyo, Asia/Tomsk, Asia/Ulaanbaatar, Asia/Urumqi,
Asia/Ust-Nera, Asia/Vientiane, Asia/Vladivostok, Asia/Yakutsk, Asia/Yangon,
Asia/Yekaterinburg, Asia/Yerevan, Pacific/Apia, Pacific/Auckland,
Pacific/Bougainville, Pacific/Chatham, Pacific/Chuuk, Pacific/Easter,
Pacific/Efate, Pacific/Fakaofo, Pacific/Fiji, Pacific/Funafuti,
Pacific/Galapagos, Pacific/Gambier, Pacific/Guadalcanal, Pacific/Guam,
Pacific/Honolulu, Pacific/Kanton, Pacific/Kiritimati, Pacific/Kosrae,
Pacific/Kwajalein, Pacific/Majuro, Pacific/Marquesas, Pacific/Midway,
Pacific/Nauru, Pacific/Niue, Pacific/Norfolk, Pacific/Noumea,
Pacific/Pago_Pago, Pacific/Palau, Pacific/Pitcairn, Pacific/Pohnpei,
Pacific/Port_Moresby, Pacific/Rarotonga, Pacific/Saipan, Pacific/Tahiti,
Pacific/Tarawa, Pacific/Tongatapu, Pacific/Wake, Pacific/Wallis
```

Example #4 Listing identifiers for a single country

```php
<?php
$timezone_identifiers = DateTimeZone::listIdentifiers( DateTimeZone::PER_COUNTRY, "UA" );
foreach( $timezone_identifiers as $identifier ) {
    echo "$identifier\n";
}
```

The above example will output something similar to:

```
Europe/Kyiv
Europe/Simferopol
Europe/Uzhgorod
Europe/Zaporozhye
```

See Also

- timezone_abbreviations_list() - Alias of DateTimeZone::listAbbreviations

DateInterval

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

Represents a date interval.

A date interval stores either a fixed amount of time (in years, months, days, hours etc) or a relative time string in the format that DateTimeImmutable's and DateTime's constructors support.

More specifically, the information in an object of the **DateInterval** class is an instruction to get from one date/time to another date/time. This process is not always reversible.

A common way to create a **DateInterval** object is by calculating the difference between two date/time objects through DateTimeInterface::diff().

How the time part of an interval, that is hours, minutes, seconds and microseconds, is applied when the interval is added to, or subtracted from, a date/time object depends on how the interval was created. For an interval created with DateInterval::__construct() the time part covers elapsed time, whereas for an interval created with DateInterval::createFromDateString(), or returned by DateTimeInterface::diff(), it increments or decrements the individual component values. The date part, that is years, months and days, always increments or decrements the individual component values. The two therefore only give a different result when a timezone transition falls within the time part of the interval; see Date/Time Arithmetic.

Since there is no well defined way to compare date intervals, **DateInterval** instances are incomparable.

Class synopsis

```
class DateInterval {
/* Properties */
public int $y;
public int $m;
public int $d;
public int $h;
public int $i;
public int $s;
public float $f;
public int $invert;
public mixed $days;
public bool $from_string;
public string $date_string;
/* Methods */
public function __construct(string $duration)
public static function createFromDateString(string $datetime): DateInterval
public function format(string $format): string
}
```

Properties

> **Warning**
>
> The available properties listed below depend on PHP version, and should be considered as *readonly*.

y

Number of years.

m

Number of months.

d

Number of days.

h

Number of hours.

i

Number of minutes.

s

Number of seconds.

f

Number of microseconds, as a fraction of a second.

invert

Is `1` if the interval represents a negative time period and `0` otherwise. See DateInterval::format().

days

If the DateInterval object was created by DateTimeImmutable::diff() or DateTime::diff(), then this is the total number of full days between the start and end dates. Otherwise, *days* will be **`false`**.

from_string

If the DateInterval object was created by DateInterval::createFromDateString(), then this property's value will be `true`, and the *date_string* property will be populated. Otherwise, the value will be `false`, and the *y* to *f*, *invert*, and *days* properties will be populated.

date_string

The string used as argument to DateInterval::createFromDateString().

Changelog

Version	Description
8.2.0	The *from_string* and *date_string* properties were added for **DateInterval** instances that were created using the DateInterval::createFromDateString() method.
8.2.0	Only the `y` to `f`, `invert`, and `days` will be visible.
7.4.0	**DateInterval** instances are incomparable now; previously, all **DateInterval** instances were considered equal.
7.1.0	The *f* property was added.

Table of Contents

DateInterval::__construct

DateInterval::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateInterval::__construct — Creates a new DateInterval object

Description

public function **DateInterval::__construct**(string `$duration`)

Creates a new DateInterval object.

Parameters

`duration`

An interval specification.

The format starts with the letter `P`, for "period." Each duration period is represented by an integer value followed by a period designator. If the duration contains time elements, that portion of the specification is preceded by the letter `T`.

`duration` Period Designators

Period Designator	Description
`Y`	years
`M`	months
`D`	days
`W`	weeks. Converted into days. Prior to PHP 8.0.0, can not be combined with `D`.
`H`	hours
`M`	minutes
`S`	seconds

Here are some simple examples. Two days is `P2D`. Two seconds is `PT2S`. Six years and five minutes is `P6YT5M`.

> **Note** :
>
> The unit types must be entered from the largest scale unit on the left to the smallest scale unit on the right. So years before months, months before days, days before minutes, etc. Thus one year and four days must be represented as `P1Y4D`, not `P4D1Y`.

The specification can also be represented as a date time. A sample of one year and four days would be `P0001-00-04T00:00:00`. But the values in this format can not exceed a given period's roll-over-point (e.g. `25` hours is invalid).

These formats are based on the » ISO 8601 duration specification.

Errors/Exceptions

Throws an DateMalformedIntervalStringException when the `duration` cannot be parsed as an interval. Prior to PHP 8.3, this was Exception.

Changelog

Version	Description
8.3.0	Now throws DateMalformedIntervalStringException instead of Exception.
8.2.0	Only the `y` to `f`, `invert`, and `days` will be visible, including a new `from_string` boolean property.
8.0.0	`W` can be combined with `D`.

Examples

Example #1 Constructing and using DateInterval objects

```php
<?php
// Create a specific date
$someDate = \DateTime::createFromFormat("Y-m-d H:i", "2022-08-25 14:18");

// Create interval
$interval = new \DateInterval("P7D");

// Add interval
$someDate->add($interval);

// Convert interval to string
echo $interval->format("%d");
```

The above example will output:

```
7
```

Example #2 DateInterval example

```php
<?php
$interval = new DateInterval('P1W2D');
var_dump($interval);
```

Output of the above example in PHP 8.2:

```
object(DateInterval)#1 (10) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(0)
  ["d"]=>
  int(9)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["invert"]=>
  int(0)
  ["days"]=>
```

```
  bool(false)
  ["from_string"]=>
  bool(false)
}
```

Output of the above example in PHP 8:

```
object(DateInterval)#1 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(0)
  ["d"]=>
  int(9)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(0)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(0)
  ["special_amount"]=>
  int(0)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(0)
}
```

Output of the above example in PHP 7:

```
object(DateInterval)#1 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(0)
  ["d"]=>
  int(2)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
```

```
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(0)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(0)
  ["special_amount"]=>
  int(0)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(0)
}
```

See Also

- DateInterval::format() - Formats the interval
- DateTime::add() - Modifies a DateTime object, with added amount of days, months, years, hours, minutes and seconds
- DateTime::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds from a DateTime object
- DateTime::diff() - Returns the difference between two DateTime objects

DateInterval::createFromDateString

DateInterval::createFromDateString

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateInterval::createFromDateString — Sets up a DateInterval from the relative parts of the string

Description

Object-oriented style

public static function **DateInterval::createFromDateString**(string `$datetime`): DateInterval

Procedural style

function date_interval_create_from_date_string(string `$datetime`): DateInterval|false

Uses the date/time parsers as used in the DateTimeImmutable constructor to create a DateInterval from the relative parts of the parsed string.

Parameters

`datetime`

> A date with relative parts. Specifically, the relative formats supported by the parser used for DateTimeImmutable, DateTime, and strtotime() will be used to construct the DateInterval.
>
> To use an ISO-8601 format string like `P7D`, you must use the DateInterval::__construct().

Return Values

Returns DateInterval on success. Procedural style returns **`false`** on failure.

Errors/Exceptions

Object Oriented API only: If an invalid Date/Time string is passed, DateMalformedIntervalStringException is thrown.

Changelog

Version	Description
8.4.0	Now has a tentative return type of DateInterval. Previously it was DateInterval|false.
8.3.0	**DateInterval::createFromDateString()** now throws DateMalformedIntervalStringException if an invalid string is passed. Previously, it returned `false`, and a warning was emitted. date_interval_create_from_date_string() has not been changed.
8.2.0	Only the `from_string` and `date_string` properties will be visible when a DateInterval is created with this method.

Examples

Example #1 Parsing valid date intervals

```php
<?php
// Each set of intervals describes the same duration.
$i = new DateInterval('P1D');
$i = DateInterval::createFromDateString('1 day');

$i = new DateInterval('P2W');
$i = DateInterval::createFromDateString('2 weeks');

$i = new DateInterval('P3M');
$i = DateInterval::createFromDateString('3 months');

$i = new DateInterval('P4Y');
$i = DateInterval::createFromDateString('4 years');

$i = new DateInterval('P1Y1D');
$i = DateInterval::createFromDateString('1 year + 1 day');

$i = new DateInterval('P1DT12H');
$i = DateInterval::createFromDateString('1 day + 12 hours');

$i = new DateInterval('PT3600S');
$i = DateInterval::createFromDateString('3600 seconds');
```

> **Note** :
>
> The intervals in a set that have a time part, such as `P1DT12H` and `1 day + 12 hours`, are not interchangeable when they are added to a date/time object and a timezone transition falls within that time part: for an interval created with DateInterval::__construct() the time part covers elapsed time, whereas for an interval created with this method it increments or decrements the individual component values. See Date/Time Arithmetic.

Example #2 Parsing combinations and negative intervals

```php
<?php
$i = DateInterval::createFromDateString('62 weeks + 1 day + 2 weeks + 2 hours + 70 minutes');
echo $i->format('%d %h %i'), "\n";

$i = DateInterval::createFromDateString('1 year - 10 days');
echo $i->format('%y %d'), "\n";
```

The above example will output:

```
449 2 70
1 -10
```

Example #3 Parsing special relative date intervals

```php
<?php
$i = DateInterval::createFromDateString('last day of next month');
var_dump($i);
```

```php
$i = DateInterval::createFromDateString('last weekday');
var_dump($i);
```

Output of the above example in PHP 8.2:

```
object(DateInterval)#1 (2) {
  ["from_string"]=>
  bool(true)
  ["date_string"]=>
  string(22) "last day of next month"
}
object(DateInterval)#2 (2) {
  ["from_string"]=>
  bool(true)
  ["date_string"]=>
  string(12) "last weekday"
}
```

Output of the above example in PHP 8 is similar to:

```
object(DateInterval)#1 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(1)
  ["d"]=>
  int(0)
  ["h"]=>
  int(0)
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(2)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(0)
  ["special_amount"]=>
  int(0)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(0)
}
object(DateInterval)#2 (16) {
  ["y"]=>
  int(0)
  ["m"]=>
  int(0)
  ["d"]=>
  int(0)
  ["h"]=>
  int(0)
```

```
  ["i"]=>
  int(0)
  ["s"]=>
  int(0)
  ["f"]=>
  float(0)
  ["weekday"]=>
  int(0)
  ["weekday_behavior"]=>
  int(0)
  ["first_last_day_of"]=>
  int(0)
  ["invert"]=>
  int(0)
  ["days"]=>
  bool(false)
  ["special_type"]=>
  int(1)
  ["special_amount"]=>
  int(-1)
  ["have_weekday_relative"]=>
  int(0)
  ["have_special_relative"]=>
  int(1)
}
```

DateInterval::format

DateInterval::format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DateInterval::format — Formats the interval

Description

public function **DateInterval::format**(string `$format`): string

Formats the interval.

Parameters

`format`

The following characters are recognized in the `format` parameter string. Each format character must be prefixed by a percent sign (`%`).

`format` character	Description	Example values
`%`	Literal `%`	`%`
`Y`	Years, numeric, at least 2 digits with leading 0	`01`, `03`
`y`	Years, numeric	`1`, `3`
`M`	Months, numeric, at least 2 digits with leading 0	`01`, `03`, `12`
`m`	Months, numeric	`1`, `3`, `12`
`D`	Days, numeric, at least 2 digits with leading 0	`01`, `03`, `31`
`d`	Days, numeric	`1`, `3`, `31`
`a`	Total number of days as a result of a DateTime::diff() or `(unknown)` otherwise	`4`, `18`, `8123`
`H`	Hours, numeric, at least 2 digits with leading 0	`01`, `03`, `23`
`h`	Hours, numeric	`1`, `3`, `23`
`I`	Minutes, numeric, at least 2 digits with leading 0	`01`, `03`, `59`
`i`	Minutes, numeric	`1`, `3`, `59`
`S`	Seconds, numeric, at least 2 digits with leading 0	`01`, `03`, `57`

`format` character	Description	Example values
`s`	Seconds, numeric	`1`, `3`, `57`
`F`	Microseconds, numeric, at least 6 digits with leading 0	`007701`, `052738`, `428291`
`f`	Microseconds, numeric	`7701`, `52738`, `428291`
`R`	Sign "-" when negative, "+" when positive	`-`, `+`
`r`	Sign "-" when negative, empty when positive	`-`,

Return Values

Returns the formatted interval.

Changelog

Version	Description
7.2.12	The `F` and `f` format will now always be positive.
7.1.0	The `F` and `f` format characters were added.

Examples

Example #1 DateInterval example

```
<?php
$interval = new DateInterval('P2Y4DT6H8M');
echo $interval->format('%d days');
```

The above example will output:

```
4 days
```

Example #2 DateInterval and carry over points

```
<?php
$interval = new DateInterval('P32D');
echo $interval->format('%d days');
```

The above example will output:

```
32 days
```

Example #3 DateInterval and DateTime::diff() with the %a and %d modifiers

```
<?php
$january = new DateTime('2010-01-01');
$february = new DateTime('2010-02-01');
$interval = $february->diff($january);

// %a will output the total number of days.
echo $interval->format('%a total days')."\n";

// While %d will only output the number of days not already covered by the
// month.
echo $interval->format('%m month, %d days');
```

The above example will output:

```
31 total days
1 month, 0 days
```

Notes

> **Note** :
>
> The **DateInterval::format()** method does not recalculate carry over points in time strings nor in date segments. This is expected because it is not possible to overflow values like `"32 days"` which could be interpreted as anything from `"1 month and 4 days"` to `"1 month and 1 day"`.

See Also

- DateTime::diff() - Returns the difference between two DateTime objects

DatePeriod

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

Represents a date period.

A date period allows iteration over a set of dates and times, recurring at regular intervals, over a given period.

Class synopsis

```
class DatePeriod implements IteratorAggregate {
/* Constants */
public const int EXCLUDE_START_DATE;
public const int INCLUDE_END_DATE;
/* Properties */
public readonly ?DateTimeInterface $start;
public readonly ?DateTimeInterface $current;
public readonly ?DateTimeInterface $end;
public readonly ?DateInterval $interval;
public readonly int $recurrences;
public readonly bool $include_start_date;
public readonly bool $include_end_date;
/* Methods */
public function __construct(
    DateTimeInterface $start,
    DateInterval $interval,
    int $recurrences,
    int $options = 0
)
public function __construct(
    DateTimeInterface $start,
    DateInterval $interval,
    DateTimeInterface $end,
    int $options = 0
)
public function __construct(string $isostr, int $options = 0)
public static function createFromISO8601String(string $specification, int $options = 0): static
public function getDateInterval(): DateInterval
public function getEndDate(): ?DateTimeInterface
```

public function getRecurrences(): ?int
public function getStartDate(): DateTimeInterface
}

Predefined Constants

`DatePeriod::EXCLUDE_START_DATE` int

Exclude start date, used in DatePeriod::__construct().

`DatePeriod::INCLUDE_END_DATE` int

Include end date, used in DatePeriod::__construct().

Properties

recurrences

The minimum amount of instances as returned by the iterator.

If the number of recurrences has been explicitly passed through the *recurrences* parameter in the constructor of the **DatePeriod** instance, then this property contains this value, *plus* one if the start date has not been disabled through **`DatePeriod::EXCLUDE_START_DATE`**, *plus* one if the end date has been enabled through **`DatePeriod::INCLUDE_END_DATE`**.

If the number of recurrences has not been explicitly passed, then this property contains the minimum number of returned instances. This would be `0`, *plus* one if the start date has not been disabled through **`DatePeriod::EXCLUDE_START_DATE`**, *plus* one if the end date has been enabled through **`DatePeriod::INCLUDE_END_DATE`**.

```php
<?php
$start = new DateTime('2018-12-31 00:00:00');
$end   = new DateTime('2021-12-31 00:00:00');
$interval = new DateInterval('P1M');
$recurrences = 5;

// recurrences explicitly set through the constructor
$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::EXCLUDE_START_DATE);
echo $period->recurrences, "\n";

$period = new DatePeriod($start, $interval, $recurrences);
echo $period->recurrences, "\n";

$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::INCLUDE_END_DATE);
echo $period->recurrences, "\n";

// recurrences not set in the constructor
$period = new DatePeriod($start, $interval, $end);
echo $period->recurrences, "\n";

$period = new DatePeriod($start, $interval, $end, DatePeriod::EXCLUDE_START_DATE);
echo $period->recurrences, "\n";
```

The above example will output:

```
5
6
7
1
0
```

See also DatePeriod::getRecurrences().

include_end_date

Whether to include the end date in the set of recurring dates or not.

include_start_date

Whether to include the start date in the set of recurring dates or not.

start

The start date of the period.

current

During iteration this will contain the current date within the period.

end

The end date of the period.

interval

An ISO 8601 repeating interval specification.

Changelog

Version	Description
8.4.0	The class constants are now typed.
8.2.0	The `DatePeriod::INCLUDE_END_DATE` constant and include_end_date property have been added.
8.0.0	**DatePeriod** implements IteratorAggregate now. Previously, Traversable was implemented instead.

Table of Contents

DatePeriod::__construct

DatePeriod::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DatePeriod::__construct — Creates a new DatePeriod object

Description

```
public function DatePeriod::__construct(
    DateTimeInterface $start,
    DateInterval $interval,
    int $recurrences,
    int $options = 0
)
public function DatePeriod::__construct(
    DateTimeInterface $start,
    DateInterval $interval,
    DateTimeInterface $end,
    int $options = 0
)
```

> **Warning**
>
> The following constructor variant has been deprecated:
>
> public function **DatePeriod::__construct**(string `$isostr`, int `$options` = 0)
>
> This variant of the constructor is deprecated as of PHP 8.4.0, use DatePeriod::createFromISO8601String() instead.

Creates a new DatePeriod object.

DatePeriod objects can be used as an iterator to generate a number of DateTimeImmutable or DateTime object from a `start` date, a `interval`, and an `end` date or the number of `recurrences`.

The class of returned objects is equivalent to the DateTimeImmutable or DateTime ancestor class of the `start` object.

Parameters

`start`

The start date of the period. Included by default in the result set.

`interval`

The interval between recurrences within the period.

`recurrences`

The number of recurrences. The number of returned results is one higher than this, as the start date is included in the result set by default. Must be greater than `0`.

`end`

The end date of the period. Excluded by default from the result set.

`isostr`

A subset of the » ISO 8601 repeating interval specification.

Examples of some ISO 8601 interval specification features that PHP does not support are:

1. zero occurrences (`R0/`)
2. time offsets other than UTC (`Z`), such as `+02:00`.

`options`

A bit field which can be used to control certain behaviour with start- and end- dates.

With **`DatePeriod::EXCLUDE_START_DATE`** you exclude the start date from the set of recurring dates within the period.

With **`DatePeriod::INCLUDE_END_DATE`** you include the end date in the set of recurring dates within the period.

Errors/Exceptions

Throws an DateMalformedPeriodStringException when the `isostr` cannot be parsed as a valid ISO 8601 period. Prior to PHP 8.3, this was Exception.

Changelog

Version	Description
8.4.0	The signature using `isostr` has been deprecated, use DatePeriod::createFromISO8601String() instead.
8.3.0	Now throws DateMalformedPeriodStringException instead of Exception.
8.2.0	The **`DatePeriod::INCLUDE_END_DATE`** constant has been added.
7.2.19, 7.3.6, 7.4.0	`recurrences` must be greater than `0` now.

Examples

Example #1 DatePeriod example

```php
<?php
$start = new DateTime('2012-07-01');
$interval = new DateInterval('P7D');
$end = new DateTime('2012-07-31');
$recurrences = 4;
$iso = 'R4/2012-07-01T00:00:00Z/P7D';

// All of these periods are equivalent.
$period = new DatePeriod($start, $interval, $recurrences);
$period = new DatePeriod($start, $interval, $end);
$period = new DatePeriod($iso);

// By iterating over the DatePeriod object, all of the
// recurring dates within that period are printed.
foreach ($period as $date) {
    echo $date->format('Y-m-d')."\n";
}
```

The above example will output:

```
Deprecated: Calling DatePeriod::__construct(string $isostr, int $options = 0) is deprecated, use
DatePeriod::createFromISO8601String() instead in script on line 11
2012-07-01
2012-07-08
2012-07-15
2012-07-22
2012-07-29
```

Example #2 DatePeriod example with `DatePeriod::EXCLUDE_START_DATE`

```php
<?php
$start = new DateTime('2012-07-01');
$interval = new DateInterval('P7D');
$end = new DateTime('2012-07-31');

$period = new DatePeriod($start, $interval, $end,
                         DatePeriod::EXCLUDE_START_DATE);

// By iterating over the DatePeriod object, all of the
// recurring dates within that period are printed.
// Note that, in this case, 2012-07-01 is not printed.
foreach ($period as $date) {
    echo $date->format('Y-m-d')."\n";
}
```

The above example will output:

```
2012-07-08
2012-07-15
2012-07-22
2012-07-29
```

Example #3 DatePeriod example showing all last Thursdays in a year

```php
<?php
$begin = new DateTime('2021-12-31');
$end = new DateTime('2022-12-31 23:59:59');

$interval = DateInterval::createFromDateString('last thursday of next month');
$period = new DatePeriod($begin, $interval, $end, DatePeriod::EXCLUDE_START_DATE);

foreach ($period as $dt) {
    echo $dt->format('l Y-m-d'), "\n";
}
```

The above example will output:

```
Thursday 2022-01-27
Thursday 2022-02-24
Thursday 2022-03-31
Thursday 2022-04-28
Thursday 2022-05-26
Thursday 2022-06-30
Thursday 2022-07-28
Thursday 2022-08-25
Thursday 2022-09-29
```

```
Thursday 2022-10-27
Thursday 2022-11-24
Thursday 2022-12-29
```

Notes

Unbound numbers of repetitions as specified by ISO 8601 section 4.5 "Recurring time interval" are not supported, i.e. neither passing `"R/..."` as `isostr` nor passing `null` as `end` would work.

DatePeriod::createFromISO8601String

DatePeriod::createFromISO8601String

(PHP 8 >= 8.3.0)

DatePeriod::createFromISO8601String — Creates a new DatePeriod object from an ISO8601 string

Description

public static function **DatePeriod::createFromISO8601String**(string `$specification`, int `$options` = 0): static

Creates a new DatePeriod object from an ISO8601 string, as specified with `specification`.

Parameters

`specification`

A subset of the » ISO 8601 repeating interval specification.

An example of an accepted ISO 8601 interval specifications is `R5/2008-03-01T13:00:00Z/P1Y2M10DT2H30M`, which specifies:

- 5 iterations (`R5/`)
- Starting at `2008-03-01T13:00:00Z`.
- Each iteration is an 1 year, 2 months, 10 days, 2 hours, and 30 minute interval (`/P1Y2M10DT2H30M`).

Examples of some ISO 8601 interval specification features that PHP does not support are:

1. zero occurrences (`R0/`)
2. time offsets other than UTC (`Z`), such as `+02:00`.

`options`

A bit field which can be used to control certain behaviour with start- and end- dates.

With `DatePeriod::EXCLUDE_START_DATE` you exclude the start date from the set of recurring dates within the period.

With `DatePeriod::INCLUDE_END_DATE` you include the end date in the set of recurring dates within the period.

Return Values

Creates a new DatePeriod object.

DatePeriod objects created with this method can be used as an iterator to generate a number of DateTimeImmutable objects.

Errors/Exceptions

Throws an DateMalformedPeriodStringException when the `specification` cannot be parsed as a valid ISO 8601 period.

Examples

Example #1 DatePeriod::createFromISO8601String example

```php
<?php
$iso = 'R4/2023-07-01T00:00:00Z/P7D';
$period = DatePeriod::createFromISO8601String($iso);
```

```php
// By iterating over the DatePeriod object, all of the
// recurring dates within that period are printed.
foreach ($period as $date) {
    echo $date->format('Y-m-d'), "\n";
}
```

The above example will output:

```
2023-07-01
2023-07-08
2023-07-15
2023-07-22
2023-07-29
```

DatePeriod::getDateInterval

DatePeriod::getDateInterval

(PHP 5 >= 5.6.5, PHP 7, PHP 8)

DatePeriod::getDateInterval — Gets the interval

Description

Object-oriented style

public function **DatePeriod::getDateInterval**(): DateInterval

Gets a DateInterval object representing the interval used for the period.

Parameters

This function has no parameters.

Return Values

Returns a DateInterval object

Examples

Example #1 DatePeriod::getDateInterval() example

```php
<?php
$period = DatePeriod::createFromIso8601String('R7/2016-05-16T00:00:00Z/P1D');
$interval = $period->getDateInterval();
echo $interval->format('%d day');
```

The above example will output:

```
1 day
```

See Also

- DatePeriod::getStartDate() - Gets the start date
- DatePeriod::getEndDate() - Gets the end date

DatePeriod::getEndDate

DatePeriod::getEndDate

(PHP 5 >= 5.6.5, PHP 7, PHP 8)

DatePeriod::getEndDate — Gets the end date

Description

Object-oriented style

public function **DatePeriod::getEndDate**(): ?DateTimeInterface

Gets the end date of the period.

Parameters

This function has no parameters.

Return Values

Returns `null` if the DatePeriod does not have an end date. For example, when initialized with the `recurrences` parameter, or the `isostr` parameter without an end date.

Returns a DateTimeImmutable object when the DatePeriod is initialized with a DateTimeImmutable object as the `end` parameter.

Returns a cloned DateTime object representing the end date otherwise.

Examples

Example #1 DatePeriod::getEndDate() example

```php
<?php
$period = new DatePeriod(
    new DateTime('2016-05-16T00:00:00Z'),
    new DateInterval('P1D'),
    new DateTime('2016-05-20T00:00:00Z')
);
$start = $period->getEndDate();
echo $start->format(DateTime::ISO8601);
```

The above examples will output:

```
2016-05-20T00:00:00+0000
```

Example #2 DatePeriod::getEndDate() without an end date

```php
<?php
$period = new DatePeriod(
    new DateTime('2016-05-16T00:00:00Z'),
    new DateInterval('P1D'),
    7
);
var_dump($period->getEndDate());
```

The above example will output:

```
NULL
```

See Also

- DatePeriod::getStartDate() - Gets the start date
- DatePeriod::getDateInterval() - Gets the interval

DatePeriod::getRecurrences

DatePeriod::getRecurrences

(PHP 7 >= 7.2.17/7.3.4, PHP 8)

DatePeriod::getRecurrences — Gets the number of recurrences

Description

Object-oriented style

public function **DatePeriod::getRecurrences**(): ?int

Get the number of recurrences.

Parameters

This function has no parameters.

Return Values

The number of recurrences as set by explicitly passing the `$recurrences` to the constructor of the DatePeriod class, or `null` otherwise.

Examples

Example #1 Different values for DatePeriod::getRecurrences()

```php
<?php
$start = new DateTime('2018-12-31 00:00:00');
$end   = new DateTime('2021-12-31 00:00:00');
$interval = new DateInterval('P1M');
$recurrences = 5;

// recurrences explicitly set through the constructor
$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::EXCLUDE_START_DATE);
echo $period->getRecurrences(), "\n";

$period = new DatePeriod($start, $interval, $recurrences);
echo $period->getRecurrences(), "\n";

$period = new DatePeriod($start, $interval, $recurrences, DatePeriod::INCLUDE_END_DATE);
echo $period->getRecurrences(), "\n\n";

// recurrences not set in the constructor
$period = new DatePeriod($start, $interval, $end);
var_dump($period->getRecurrences());

$period = new DatePeriod($start, $interval, $end, DatePeriod::EXCLUDE_START_DATE);
var_dump($period->getRecurrences());
```

The above example will output:

```
5
5
5

NULL
NULL
```

See Also

- DatePeriod::$recurrences

DatePeriod::getStartDate

DatePeriod::getStartDate

(PHP 5 >= 5.6.5, PHP 7, PHP 8)

DatePeriod::getStartDate — Gets the start date

Description

Object-oriented style

public function **DatePeriod::getStartDate**(): DateTimeInterface

Gets the start date of the period.

Parameters

This function has no parameters.

Return Values

Returns a DateTimeImmutable object when the DatePeriod is initialized with a DateTimeImmutable object as the `start` parameter.

Returns a DateTime object otherwise.

Examples

Example #1 DatePeriod::getStartDate() example

```
<?php
$period = DatePeriod::createFromIso8601String('R7/2016-05-16T00:00:00Z/P1D');
$start = $period->getStartDate();
echo $start->format(DateTime::ISO8601);
```

The above example will output:

```
2016-05-16T00:00:00+0000
```

See Also

- DatePeriod::getEndDate() - Gets the end date
- DatePeriod::getDateInterval() - Gets the interval

Date/Time Functions

Table of Contents

checkdate

checkdate

(PHP 4, PHP 5, PHP 7, PHP 8)

checkdate — Validate a Gregorian date

Description

function **checkdate**(int `$month`, int `$day`, int `$year`): bool

Checks the validity of the date formed by the arguments. A date is considered valid if each parameter is properly defined.

Parameters

`month`

The month is between 1 and 12 inclusive.

`day`

The day is within the allowed number of days for the given `month`. Leap `year`s are taken into consideration.

`year`

The year is between 1 and 32767 inclusive.

Return Values

Returns `true` if the date given is valid; otherwise returns `false`.

Examples

Example #1 checkdate() example

```php
<?php
var_dump(checkdate(12, 31, 2000));
var_dump(checkdate(2, 29, 2001));
```

The above example will output:

```
bool(true)
bool(false)
```

See Also

- mktime() - Get Unix timestamp for a date
- strtotime() - Parse about any English textual datetime description into a Unix timestamp

date

date

(PHP 4, PHP 5, PHP 7, PHP 8)

date — Format a Unix timestamp

Description

function **date**(string `$format`, ?int `$timestamp` = `null`): string

Returns a string formatted according to the given format string using the given integer `timestamp` (Unix timestamp) or the current time if no timestamp is given. In other words, `timestamp` is optional and defaults to the value of time().

> **Warning**
>
> Unix timestamps do not handle timezones. Use the DateTimeImmutable class, and its DateTimeInterface::format() formatting method to format date/time information with a timezone attached.

Parameters

`format`

Format accepted by DateTimeInterface::format().

> **Note** :
>
> **date()** will always generate `000000` as microseconds since it takes an int parameter, whereas DateTimeInterface::format() does support microseconds if an object of type DateTimeInterface was created with microseconds.

`timestamp`

The optional `timestamp` parameter is an int Unix timestamp that defaults to the current local time if `timestamp` is omitted or `null`. In other words, it defaults to the value of time().

Return Values

Returns a formatted date string.

Errors/Exceptions

Every call to a date/time function will generate a `E_WARNING` if the time zone is not valid. See also date_default_timezone_set()

Changelog

Version	Description
8.0.0	`timestamp` is nullable now.

Examples

Example #1 date() examples

```php
<?php
// set the default timezone to use.
date_default_timezone_set('UTC');


// Prints something like: Monday
echo date("l") . "\n";

// Prints something like: Monday 8th of August 2005 03:12:46 PM
echo date('l jS \of F Y h:i:s A') . "\n";

// Prints: July 1, 2000 is on a Saturday
echo "July 1, 2000 is on a " . date("l", mktime(0, 0, 0, 7, 1, 2000)) . "\n";

/* use the constants in the format parameter */
// prints something like: Wed, 25 Sep 2013 15:28:57 -0700
echo date(DATE_RFC2822) . "\n";

// prints something like: 2000-07-01T00:00:00+00:00
echo date(DATE_ATOM, mktime(0, 0, 0, 7, 1, 2000));
```

You can prevent a recognized character in the format string from being expanded by escaping it with a preceding backslash. If the character with a backslash is already a special sequence, you may need to also escape the backslash.

Example #2 Escaping characters in date()

```php
<?php
// prints something like: Wednesday the 15th
echo date('l \t\h\e jS');
```

Some examples of **date()** formatting. Note that you should escape any other characters, as any which currently have a special meaning will produce undesirable results, and other characters may be assigned meaning in future PHP versions. When escaping, be sure to use single quotes to prevent characters like \n from becoming newlines.

Example #3 date() Formatting

```php
<?php
// Assuming today is March 10th, 2001, 5:16:18 pm, and that we are in the
// Mountain Standard Time (MST) Time Zone
date_default_timezone_set("America/Phoenix");

echo date("F j, Y, g:i a") . "\n";                 // March 10, 2001, 5:16 pm
echo date("m.d.y") . "\n";                         // 03.10.01
echo date("j, n, Y") . "\n";                       // 10, 3, 2001
echo date("Ymd") . "\n";                           // 20010310
echo date('h-i-s, j-m-y, it is w Day') . "\n";     // 05-16-18, 10-03-01, 1631 1618 6 Satpm01
echo date('\i\t \i\s \t\h\e jS \d\a\y.') . "\n";   // it is the 10th day.
echo date("D M j G:i:s T Y") . "\n";               // Sat Mar 10 17:16:18 MST 2001
echo date('H:m:s \m \i\s\ \m\o\n\t\h') . "\n";     // 17:03:18 m is month
echo date("H:i:s") . "\n";                         // 17:16:18
echo date("Y-m-d H:i:s") . "\n";                   // 2001-03-10 17:16:18 (the MySQL DATETIME format)
```

To format dates in other languages, IntlDateFormatter::format() can be used instead of **date()**.

Notes

> **Note** :
>
> To generate a timestamp from a string representation of the date, you may be able to use strtotime(). Additionally, some databases have functions to convert their date formats into timestamps (such as MySQL's » UNIX_TIMESTAMP function).

> **Tip**
>
> Timestamp of the start of the request is available in *$_SERVER['REQUEST_TIME']*.

See Also

- DateTimeImmutable::__construct() - Returns new DateTimeImmutable object
- DateTimeInterface::format() - Returns date formatted according to given format
- gmdate() - Format a GMT/UTC date/time
- idate() - Format a local time/date part as integer
- getdate() - Get date/time information
- getlastmod() - Gets time of last page modification
- mktime() - Get Unix timestamp for a date
- IntlDateFormatter::format() - Format the date/time value as a string
- time() - Return current Unix timestamp
- Predefined DateTime Constants

date_add

date_add

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_add — Alias of DateTime::add()

Description

This function is an alias of: DateTime::add()

date_create

date_create

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_create — create a new DateTime object

Description

function **date_create**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`): DateTime|false

This is the procedural version of DateTime::__construct().

Unlike the DateTime constructor, it will return `false` instead of an exception if the passed in `datetime` string is invalid.

Parameters

See DateTimeImmutable::__construct.

Return Values

Returns a new DateTime instance or `false` on failure

See Also

- DateTimeImmutable::__construct() - Returns new DateTimeImmutable object
- DateTimeImmutable::createFromFormat() - Parses a time string according to a specified format
- DateTime::__construct() - Returns new DateTime object

date_create_from_format

date_create_from_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_create_from_format — Alias of DateTime::createFromFormat()

Description

This function is an alias of: DateTime::createFromFormat()

date_create_immutable

date_create_immutable

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

date_create_immutable — create a new DateTimeImmutable object

Description

function **date_create_immutable**(string `$datetime` = "now", ?DateTimeZone `$timezone` = `null`): DateTimeImmutable|false

This is the procedural version of DateTimeImmutable::__construct().

Unlike the DateTimeImmutable constructor, it will return `false` instead of an exception if the passed in `datetime` string is invalid.

Parameters

See DateTimeImmutable::__construct.

Return Values

Returns a new DateTimeImmutable instance or `false` on failure

See Also

- DateTimeImmutable::createFromFormat() - Parses a time string according to a specified format

date_create_immutable_from_format

date_create_immutable_from_format

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

date_create_immutable_from_format — Alias of DateTimeImmutable::createFromFormat()

Description

This function is an alias of: DateTimeImmutable::createFromFormat()

date_date_set

date_date_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_date_set — Alias of DateTime::setDate()

Description

This function is an alias of: DateTime::setDate()

date_default_timezone_get

date_default_timezone_get

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

date_default_timezone_get — Gets the default timezone used by all date/time functions in a script

Description

function **date_default_timezone_get**(): string

In order of preference, this function returns the default timezone by:

- Reading the timezone set using the date_default_timezone_set() function (if any)
- Reading the value of the date.timezone ini option (if set)

If none of the above succeed, **date_default_timezone_get()** will return a default timezone of UTC.

Parameters

This function has no parameters.

Return Values

Returns a string.

Examples

Example #1 Getting the default timezone

```
<?php
date_default_timezone_set('Europe/London');

if (date_default_timezone_get()) {
    echo 'date_default_timezone_set: ' . date_default_timezone_get() . "\n";
}

if (ini_get('date.timezone')) {
    echo 'date.timezone: ' . ini_get('date.timezone');
}
```

The above example will output something similar to:

```
date_default_timezone_set: Europe/London
date.timezone: Europe/London
```

Example #2 Getting the abbreviation of a timezone

```
<?php
date_default_timezone_set('America/Los_Angeles');
$ts = mktime(0, 0, 0, 1, 1, 2024);
echo date_default_timezone_get() . ' => ' . date('e') . ' => ' . date('T', $ts);
```

The above example will output:

```
America/Los_Angeles => America/Los_Angeles => PST
```

See Also

- date_default_timezone_set() - Sets the default timezone used by all date/time functions in a script
- List of Supported Timezones

date_default_timezone_set

date_default_timezone_set

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

date_default_timezone_set — Sets the default timezone used by all date/time functions in a script

Description

function **date_default_timezone_set**(string `$timezoneId`): bool

date_default_timezone_set() sets the default timezone used by all date/time functions.

Instead of using this function to set the default timezone in your script, you can also use the INI setting date.timezone to set the default timezone.

Parameters

`timezoneId`

> The timezone identifier, like `UTC`, `Africa/Lagos`, `Asia/Hong_Kong`, or `Europe/Lisbon`. The list of valid identifiers is available in the List of Supported Timezones.

Return Values

This function returns **`false`** if the `timezoneId` isn't valid, or **`true`** otherwise.

Examples

Example #1 Getting the default timezone

```php
<?php
date_default_timezone_set('America/Los_Angeles');

$script_tz = date_default_timezone_get();
$ini_tz = ini_get('date.timezone');

if (strcmp($script_tz, $ini_tz)){
    echo 'Script timezone differs from ini-set timezone.';
} else {
    echo 'Script timezone and ini-set timezone match.';
}
```

See Also

- date_default_timezone_get() - Gets the default timezone used by all date/time functions in a script
- List of Supported Timezones

date_diff

date_diff

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_diff — Alias of DateTime::diff()

Description

This function is an alias of: DateTime::diff()

date_format

date_format

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_format — Alias of DateTime::format()

Description

This function is an alias of: DateTime::format()

date_get_last_errors

date_get_last_errors

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_get_last_errors — Alias of DateTimeImmutable::getLastErrors()

Description

This function is an alias of: DateTimeImmutable::getLastErrors()

date_interval_create_from_date_string

date_interval_create_from_date_string

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_interval_create_from_date_string — Alias of DateInterval::createFromDateString()

Description

This function is an alias of: DateInterval::createFromDateString()

date_interval_format

date_interval_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_interval_format — Alias of DateInterval::format()

Description

This function is an alias of: DateInterval::format()

date_isodate_set

date_isodate_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_isodate_set — Alias of DateTime::setISODate()

Description

This function is an alias of: DateTime::setISODate()

date_modify

date_modify

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_modify — Alias of DateTime::modify()

Description

This function is an alias of: DateTime::modify()

date_offset_get

date_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_offset_get — Alias of DateTime::getOffset()

Description

This function is an alias of: DateTime::getOffset()

date_parse

date_parse

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_parse — Returns associative array with detailed info about given date/time

Description

function **date_parse**(string `$datetime`): array

date_parse() parses the given `datetime` string according to the same rules as strtotime() and DateTimeImmutable::__construct(). Instead of returning a Unix timestamp (with strtotime()) or a DateTimeImmutable object (with DateTimeImmutable::__construct()), it returns an associative array with the information that it could detect in the given `datetime` string.

If no information about a certain group of elements can be found, these array elements will be set to **`false`** or are missing. If needed for constructing a timestamp or DateTimeImmutable object from the same `datetime` string, more fields can be set to a non-**`false`** value. See the examples for cases where that happens.

Parameters

`datetime`

Date/time in format accepted by DateTimeImmutable::__construct().

Return Values

Returns array with information about the parsed date/time.

The returned array has keys for `year`, `month`, `day`, `hour`, `minute`, `second`, `fraction`, and `is_localtime`.

If `is_localtime` is present then `zone_type` indicates the type of timezone. For type `1` (UTC offset) the `zone`, `is_dst` fields are added; for type `2` (abbreviation) the fields `tz_abbr`, `is_dst` are added; and for type `3` (timezone identifier) the `tz_abbr`, `tz_id` are added.

If relative time elements are present in the `datetime` string such as `+3 days`, the returned array includes a nested array with the key `relative`. This array then contains the keys `year`, `month`, `day`, `hour`, `minute`, `second`, and if necessary `weekday`, and `weekdays`, depending on the string that was passed in.

The array includes `warning_count` and `warnings` fields. The first one indicate how many warnings there were. The keys of elements `warnings` array indicate the position in the given `datetime` where the warning occurred, with the string value describing the warning itself.

The array also contains `error_count` and `errors` fields. The first one indicate how many errors were found. The keys of elements `errors` array indicate the position in the given `datetime` where the error occurred, with the string value describing the error itself.

> **Warning**
>
> The number of array elements in the `warnings` and `errors` arrays might be less than `warning_count` or `error_count` if they occurred at the same position.

Errors/Exceptions

In case the date/time format has an error, the element 'errors' will contain the error messages.

Changelog

Version	Description
7.2.0	The `zone` element of the returned array represents seconds instead of minutes now, and its sign is inverted. For instance `-120` is now `7200`.

Examples

Example #1 A date_parse() example with a comprehensive `datetime` string

```php
<?php
var_dump(date_parse("2006-12-12 10:00:00.5"));
```

The above example will output:

```
array(12) {
  ["year"]=>
  int(2006)
  ["month"]=>
  int(12)
  ["day"]=>
  int(12)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(0)
  ["second"]=>
  int(0)
  ["fraction"]=>
  float(0.5)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
}
```

The timezone elements only show up if they are included in the given `datetime` string. In that case there will always be a `zone_type` element and a few more depending on its value.

Example #2 date_parse() with timezone abbreviation information

```php
<?php
var_dump(date_parse("June 2nd, 2022, 10:28:17 BST"));
```

The above example will output:

```
array(16) {
  ["year"]=>
  int(2022)
```

```
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(28)
  ["second"]=>
  int(17)
  ["fraction"]=>
  float(0)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(true)
  ["zone_type"]=>
  int(2)
  ["zone"]=>
  int(0)
  ["is_dst"]=>
  bool(true)
  ["tz_abbr"]=>
  string(3) "BST"
}
```

Example #3 date_parse() with timezone identifier information

```php
<?php
var_dump(date_parse("June 2nd, 2022, 10:28:17 Europe/London"));
```

The above example will output:

```
array(14) {
  ["year"]=>
  int(2022)
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(28)
  ["second"]=>
  int(17)
  ["fraction"]=>
  float(0)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
```

```
  }
  ["is_localtime"]=>
  bool(true)
  ["zone_type"]=>
  int(3)
  ["tz_id"]=>
  string(13) "Europe/London"
}
```

If a more minimal `datetime` string is parsed, less information is available. In this example, all the time parts are returned as `false`.

Example #4 date_parse() with a minimal string

```php
<?php
var_dump(date_parse("June 2nd, 2022"));
```

The above example will output:

```
array(12) {
  ["year"]=>
  int(2022)
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  bool(false)
  ["minute"]=>
  bool(false)
  ["second"]=>
  bool(false)
  ["fraction"]=>
  bool(false)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
}
```

Relative formats do not influence the values parsed from absolute formats, but are parsed into the "relative" element.

Example #5 date_parse() with relative formats

```php
<?php
var_dump(date_parse("2006-12-12 10:00:00.5 +1 week +1 hour"));
```

The above example will output:

```
array(13) {
  ["year"]=>
  int(2006)
```

```
  ["month"]=>
  int(12)
  ["day"]=>
  int(12)
  ["hour"]=>
  int(10)
  ["minute"]=>
  int(0)
  ["second"]=>
  int(0)
  ["fraction"]=>
  float(0.5)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
  ["relative"]=>
  array(6) {
    ["year"]=>
    int(0)
    ["month"]=>
    int(0)
    ["day"]=>
    int(7)
    ["hour"]=>
    int(1)
    ["minute"]=>
    int(0)
    ["second"]=>
    int(0)
  }
}
```

Some stanzas, such as `Thursday` will set the time portion of the string to `0`. If `Thursday` is passed to DateTimeImmutable::__construct() it would also have resulted in the hour, minute, second, and fraction being set to `0`. In the example below, the year element is however left as `false`.

Example #6 date_parse() with side-effects

```php
<?php
var_dump(date_parse("Thursday, June 2nd"));
```

The above example will output:

```
array(13) {
  ["year"]=>
  bool(false)
  ["month"]=>
  int(6)
  ["day"]=>
  int(2)
  ["hour"]=>
  int(0)
  ["minute"]=>
  int(0)
  ["second"]=>
  int(0)
```

```
  ["fraction"]=>
  float(0)
  ["warning_count"]=>
  int(0)
  ["warnings"]=>
  array(0) {
  }
  ["error_count"]=>
  int(0)
  ["errors"]=>
  array(0) {
  }
  ["is_localtime"]=>
  bool(false)
  ["relative"]=>
  array(7) {
    ["year"]=>
    int(0)
    ["month"]=>
    int(0)
    ["day"]=>
    int(0)
    ["hour"]=>
    int(0)
    ["minute"]=>
    int(0)
    ["second"]=>
    int(0)
    ["weekday"]=>
    int(4)
  }
}
```

See Also

- date_parse_from_format() - Get info about given date formatted according to the specified format for parsing a `datetime` with a specific given format
- checkdate() - Validate a Gregorian date for Gregorian date validation
- getdate() - Get date/time information

date_parse_from_format

date_parse_from_format

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_parse_from_format — Get info about given date formatted according to the specified format

Description

function **date_parse_from_format**(string `$format`, string `$datetime`): array

Returns associative array with detailed info about given date/time.

Parameters

`format`

Documentation on how the `format` is used, please refer to the documentation of DateTimeImmutable::createFromFormat(). The same rules apply.

`datetime`

String representing the date/time.

Return Values

Returns associative array with detailed info about given date/time.

The returned array has keys for `year`, `month`, `day`, `hour`, `minute`, `second`, `fraction`, and `is_localtime`.

If `is_localtime` is present then `zone_type` indicates the type of timezone. For type `1` (UTC offset) the `zone`, `is_dst` fields are added; for type `2` (abbreviation) the fields `tz_abbr`, `is_dst` are added; and for type `3` (timezone identifier) the `tz_abbr`, `tz_id` are added.

The array includes `warning_count` and `warnings` fields. The first one indicate how many warnings there were. The keys of elements `warnings` array indicate the position in the given `datetime` where the warning occurred, with the string value describing the warning itself. An example below shows such a warning.

The array also contains `error_count` and `errors` fields. The first one indicate how many errors were found. The keys of elements `errors` array indicate the position in the given `datetime` where the error occurred, with the string value describing the error itself. An example below shows such an error.

> **Warning**
>
> The number of array elements in the `warnings` and `errors` arrays might be less than `warning_count` or `error_count` if they occurred at the same position.

Errors/Exceptions

This functions throws ValueError when the `datetime` contains NULL-bytes.

Changelog

Version	Description
8.0.21, 8.1.8, 8.2.0	Now throws ValueError when NULL-bytes are passed into `datetime`, which previously was silently ignored.
7.2.0	The `zone` element of the returned array represents seconds instead of minutes now, and its sign is inverted. For instance `-120` is now `7200`.

Examples

Example #1 **date_parse_from_format()** example

```php
<?php
$date = "6.1.2009 13:00+01:00";
print_r(date_parse_from_format("j.n.Y H:iP", $date));
```

The above example will output:

```
Array
(
    [year] => 2009
    [month] => 1
    [day] => 6
    [hour] => 13
    [minute] => 0
    [second] => 0
    [fraction] => 0
    [warning_count] => 0
    [warnings] => Array
        (
        )

    [error_count] => 0
    [errors] => Array
        (
        )

    [is_localtime] => 1
    [zone_type] => 1
    [zone] => 3600
    [is_dst] =>
)
```

Example #2 date_parse_from_format() with warnings example

```php
<?php
$date = "26 August 2022 22:30 pm";
$parsed = date_parse_from_format("j F Y G:i a", $date);

echo "Warnings count: ", $parsed['warning_count'], "\n";
foreach ($parsed['warnings'] as $position => $message) {
    echo "\tOn position {$position}: {$message}\n";
}
```

The above example will output:

```
Warnings count: 1
        On position 23: The parsed time was invalid
```

Example #3 date_parse_from_format() with errors example

```php
<?php
$date = "26 August 2022 CEST";
$parsed = date_parse_from_format("j F Y H:i", $date);

echo "Errors count: ", $parsed['error_count'], "\n";
foreach ($parsed['errors'] as $position => $message) {
    echo "\tOn position {$position}: {$message}\n";
}
```

The above example will output:

```
Errors count: 3
        On position 15: A two digit hour could not be found
        On position 19: Not enough data available to satisfy format
```

See Also

- DateTimeImmutable::createFromFormat() - Parses a time string according to a specified format
- checkdate() - Validate a Gregorian date

date_sub

date_sub

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_sub — Alias of DateTime::sub()

Description

This function is an alias of: DateTime::sub()

See Also

- DateTimeImmutable::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds
- DateTime::sub() - Subtracts an amount of days, months, years, hours, minutes and seconds from a DateTime object

date_sun_info

date_sun_info

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

date_sun_info — Returns an array with information about sunset/sunrise and twilight begin/end

Description

function **date_sun_info**(int `$timestamp`, float `$latitude`, float `$longitude`): array

Parameters

`timestamp`

Unix timestamp.

`latitude`

Latitude in degrees.

`longitude`

Longitude in degrees.

Return Values

Returns an array whose structure is detailed in the following list:

`sunrise`

The timestamp of the sunrise (zenith angle = 90°35').

`sunset`

The timestamp of the sunset (zenith angle = 90°35').

`transit`

The timestamp when the sun is at its zenith, i.e. has reached its topmost point.

`civil_twilight_begin`

The start of the civil dawn (zenith angle = 96°). It ends at `sunrise`.

`civil_twilight_end`

The end of the civil dusk (zenith angle = 96°). It starts at `sunset`.

`nautical_twilight_begin`

The start of the nautical dawn (zenith angle = 102°). It ends at `civil_twilight_begin`.

`nautical_twilight_end`

The end of the nautical dusk (zenith angle = 102°). It starts at `civil_twilight_end`.

`astronomical_twilight_begin`

The start of the astronomical dawn (zenith angle = 108°). It ends at `nautical_twilight_begin`.

`astronomical_twilight_end`

The end of the astronomical dusk (zenith angle = 108°). It starts at `nautical_twilight_end`.

The values of the array elements are either UNIX timestamps, **`false`** if the sun is below the respective zenith for the whole day, or **`true`** if the sun is above the respective zenith for the whole day.

Changelog

Version	Description
7.2.0	The calculation was fixed with regards to local midnight instead of local noon, which changes the results slightly.

Examples

Example #1 A **date_sun_info()** example

```php
<?php
date_default_timezone_set('Asia/Jerusalem');
$sun_info = date_sun_info(strtotime("2006-12-12"), 31.7667, 35.2333);
foreach ($sun_info as $key => $val) {
    echo "$key: " . date("H:i:s", $val) . "\n";
}
```

The above example will output:

```
sunrise: 06:29:21
sunset: 16:36:00
transit: 11:32:41
civil_twilight_begin: 06:02:36
civil_twilight_end: 17:02:45
nautical_twilight_begin: 05:32:14
nautical_twilight_end: 17:33:08
astronomical_twilight_begin: 05:02:31
astronomical_twilight_end: 18:02:51
```

Example #2 Polar night, with some processing

```php
<?php
$tz = new \DateTimeZone('America/Anchorage');

$si = date_sun_info(strtotime("2022-12-21"), 70.21, -148.51);
foreach ($si as $key => $value) {
    echo
        match ($value) {
            true => 'always',
            false => 'never',
            default => date_create("@{$value}")->setTimeZone($tz)->format( 'H:i:s T' ),
        },
        ": {$key}",
        "\n";
}
```

The above example will output:

```
never: sunrise
never: sunset
12:52:18 AKST: transit
10:53:19 AKST: civil_twilight_begin
14:51:17 AKST: civil_twilight_end
09:01:47 AKST: nautical_twilight_begin
16:42:48 AKST: nautical_twilight_end
07:40:47 AKST: astronomical_twilight_begin
18:03:49 AKST: astronomical_twilight_end
```

Example #3 Midnight sun (Tromsø, Norway)

```php
<?php
$si = date_sun_info(strtotime("2022-06-26"), 69.68, 18.94);
print_r($si);
```

The above example will output:

```
Array
(
    [sunrise] => 1
    [sunset] => 1
    [transit] => 1656240426
    [civil_twilight_begin] => 1
    [civil_twilight_end] => 1
    [nautical_twilight_begin] => 1
    [nautical_twilight_end] => 1
    [astronomical_twilight_begin] => 1
    [astronomical_twilight_end] => 1
)
```

Example #4 Calculating length of day (Kyiv)

```php
<?php
$si = date_sun_info(strtotime('2022-08-26'), 50.45, 30.52);
$diff = $si['sunset'] - $si['sunrise'];
echo "Length of day: ",
    floor($diff / 3600), "h ",
    floor(($diff % 3600) / 60), "m\n";
```

The above example will output:

```
Length of day: 13h 53m
```

date_sunrise

date_sunrise

(PHP 5, PHP 7, PHP 8)

date_sunrise — Returns time of sunrise for a given day and location

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.1.0. Relying on this function is highly discouraged. Use date_sun_info() instead.

Description

```
#[\Deprecated(
    since: '8.1',
    message: 'use date_sun_info() instead',
)]
function date_sunrise(
    int $timestamp,
    int $returnFormat = SUNFUNCS_RET_STRING,
    ?float $latitude = null,
    ?float $longitude = null,
    ?float $zenith = null,
```

```
    ?float $utcOffset = null
): string|int|float|false
```

date_sunrise() returns the sunrise time for a given day (specified as a `timestamp`) and location.

Parameters

`timestamp`

The `timestamp` of the day from which the sunrise time is taken.

`returnFormat`

`returnFormat` constants

constant	description	example
SUNFUNCS_RET_STRING	returns the result as string	16:46
SUNFUNCS_RET_DOUBLE	returns the result as float	16.78243132
SUNFUNCS_RET_TIMESTAMP	returns the result as int (timestamp)	1095034606

`latitude`

Defaults to North, pass in a negative value for South. See also: date.default_latitude

`longitude`

Defaults to East, pass in a negative value for West. See also: date.default_longitude

`zenith`

`zenith` is the angle between the center of the sun and a line perpendicular to earth's surface. It defaults to date.sunrise_zenith

Common `zenith` angles

Angle	Description
90°50'	Sunrise: the point where the sun becomes visible.
96°	Civil twilight: conventionally used to signify the start of dawn.
102°	Nautical twilight: the point at which the horizon starts being visible at sea.
108°	Astronomical twilight: the point at which the sun starts being the source of any illumination.

`utcOffset`

Specified in hours. The `utcOffset` is ignored, if `returnFormat` is **`SUNFUNCS_RET_TIMESTAMP`**.

Return Values

Returns the sunrise time in a specified `returnFormat` on success or **`false`** on failure. One potential reason for failure is that the sun does not rise at all, which happens inside the polar circles for part of the year.

Errors/Exceptions

Every call to a date/time function will generate a **`E_WARNING`** if the time zone is not valid. See also date_default_timezone_set()

Changelog

Version	Description
8.1.0	This function has been deprecated in favor of date_sun_info().
8.0.0	`latitude`, `longitude`, `zenith` and `utcOffset` are nullable now.

Examples

Example #1 date_sunrise() example

```php
<?php

/* calculate the sunrise time for Lisbon, Portugal
Latitude: 38.4 North
Longitude: 9 West
```

```
Zenith ~= 90
offset: +1 GMT
*/

echo date("D M d Y"). ', sunrise time : ' .date_sunrise(time(), SUNFUNCS_RET_STRING, 38.4, -9, 90, 1);
```

The above example will output something similar to:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 10
Deprecated: Function date_sunrise() is deprecated since 8.1, use date_sun_info() instead in script on line 10
Mon Dec 20 2004, sunrise time : 08:54
```

Example #2 No sunrise

```
<?php
$solstice = strtotime('2017-12-21');
var_dump(date_sunrise($solstice, SUNFUNCS_RET_STRING, 69.245833, -53.537222));
```

The above example will output:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 3
Deprecated: Function date_sunrise() is deprecated since 8.1, use date_sun_info() instead in script on line 3
bool(false)
```

See Also

- date_sun_info() - Returns an array with information about sunset/sunrise and twilight begin/end

date_sunset

date_sunset

(PHP 5, PHP 7, PHP 8)

date_sunset — Returns time of sunset for a given day and location

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.1.0. Relying on this function is highly discouraged. Use date_sun_info() instead.

Description

```
#[\Deprecated(
    since: '8.1',
    message: 'use date_sun_info() instead',
)]
function date_sunset(
    int $timestamp,
    int $returnFormat = SUNFUNCS_RET_STRING,
    ?float $latitude = null,
    ?float $longitude = null,
    ?float $zenith = null,
    ?float $utcOffset = null
): string|int|float|false
```

date_sunset() returns the sunset time for a given day (specified as a `timestamp`) and location.

Parameters

`timestamp`

The `timestamp` of the day from which the sunset time is taken.

`returnFormat`

`returnFormat` constants

constant	description	example
SUNFUNCS_RET_STRING	returns the result as string	16:46
SUNFUNCS_RET_DOUBLE	returns the result as float	16.78243132
SUNFUNCS_RET_TIMESTAMP	returns the result as int (timestamp)	1095034606

`latitude`

Defaults to North, pass in a negative value for South. See also: date.default_latitude

`longitude`

Defaults to East, pass in a negative value for West. See also: date.default_longitude

`zenith`

`zenith` is the angle between the center of the sun and a line perpendicular to earth's surface. It defaults to date.sunset_zenith

Common `zenith` angles

Angle	Description
90°50'	Sunset: the point where the sun becomes invisible.
96°	Civil twilight: conventionally used to signify the end of dusk.
102°	Nautical twilight: the point at which the horizon ends being visible at sea.
108°	Astronomical twilight: the point at which the sun ends being the source of any illumination.

`utcOffset`

Specified in hours. The `utcOffset` is ignored, if `returnFormat` is **`SUNFUNCS_RET_TIMESTAMP`**.

Return Values

Returns the sunset time in a specified `returnFormat` on success or **`false`** on failure. One potential reason for failure is that the sun does not set at all, which happens inside the polar circles for part of the year.

Errors/Exceptions

Every call to a date/time function will generate a **`E_WARNING`** if the time zone is not valid. See also date_default_timezone_set()

Changelog

Version	Description
8.1.0	This function has been deprecated in favor of date_sun_info().
8.0.0	`latitude`, `longitude`, `zenith` and `utcOffset` are nullable now.

Examples

Example #1 date_sunset() example

```php
<?php

/* calculate the sunset time for Lisbon, Portugal
Latitude: 38.4 North
Longitude: 9 West
Zenith ~= 90
offset: +1 GMT
*/
```

```php
echo date("D M d Y"). ', sunset time : ' .date_sunset(time(), SUNFUNCS_RET_STRING, 38.4, -9, 90, 1);
```

The above example will output something similar to:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 10
Deprecated: Function date_sunset() is deprecated since 8.1, use date_sun_info() instead in script on line 10
Mon Dec 20 2004, sunset time : 18:13
```

Example #2 No sunset

```php
<?php
$solstice = strtotime('2017-12-21');
var_dump(date_sunset($solstice, SUNFUNCS_RET_STRING, 69.245833, -53.537222));
```

The above example will output:

```
Deprecated: Constant SUNFUNCS_RET_STRING is deprecated in script on line 3
Deprecated: Function date_sunset() is deprecated since 8.1, use date_sun_info() instead in script on line 3
bool(false)
```

See Also

- date_sun_info() - Returns an array with information about sunset/sunrise and twilight begin/end

date_time_set

date_time_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_time_set — Alias of DateTime::setTime()

Description

This function is an alias of: DateTime::setTime()

date_timestamp_get

date_timestamp_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_timestamp_get — Alias of DateTime::getTimestamp()

Description

This function is an alias of: DateTime::getTimestamp()

date_timestamp_set

date_timestamp_set

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

date_timestamp_set — Alias of DateTime::setTimestamp()

Description

This function is an alias of: DateTime::setTimestamp()

date_timezone_get

date_timezone_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_timezone_get — Alias of DateTime::getTimezone()

Description

This function is an alias of: DateTime::getTimezone()

date_timezone_set

date_timezone_set

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

date_timezone_set — Alias of DateTime::setTimezone()

Description

This function is an alias of: DateTime::setTimezone()

getdate

getdate

(PHP 4, PHP 5, PHP 7, PHP 8)

getdate — Get date/time information

Description

function **getdate**(?int `$timestamp = null`): array

Returns an associative array containing the date information of the `timestamp`, or the current local time if `timestamp` is omitted or `null`.

Parameters

`timestamp`

> The optional `timestamp` parameter is an int Unix timestamp that defaults to the current local time if `timestamp` is omitted or `null`. In other words, it defaults to the value of time().

Return Values

Returns an associative array of information related to the `timestamp`. Elements from the returned associative array are as follows:

Key elements of the returned associative array

Key	Description	Example returned values
`"seconds"`	Numeric representation of seconds	`0` to `59`
`"minutes"`	Numeric representation of minutes	`0` to `59`
`"hours"`	Numeric representation of hours	`0` to `23`
`"mday"`	Numeric representation of the day of the month	`1` to `31`
`"wday"`	Numeric representation of the day of the week	`0` (for Sunday) through `6` (for Saturday)
`"mon"`	Numeric representation of a month	`1` through `12`
`"year"`	A full numeric representation of a year, 4 digits	Examples: `1999` or `2003`
`"yday"`	Numeric representation of the day of the year	`0` through `365`
`"weekday"`	A full textual representation of the day of the week	`Sunday` through `Saturday`

Key	Description	Example returned values
`"month"`	A full textual representation of a month, such as January or March	`January` through `December`
`0`	Seconds since the Unix Epoch, similar to the values returned by time() and used by date().	System Dependent, typically `-2147483648` through `2147483647`.

Changelog

Version	Description
8.0.0	`timestamp` is nullable now.

Examples

Example #1 **getdate()** example

```
<?php
$today = getdate();
print_r($today);
```

The above example will output something similar to:

```
Array
(
    [seconds] => 40
    [minutes] => 58
    [hours] => 21
    [mday] => 17
    [wday] => 2
    [mon] => 6
    [year] => 2003
    [yday] => 167
    [weekday] => Tuesday
    [month] => June
    [0] => 1055901520
)
```

See Also

- date() - Format a Unix timestamp
- idate() - Format a local time/date part as integer
- localtime() - Get the local time
- time() - Return current Unix timestamp
- setlocale() - Set locale information

gettimeofday

gettimeofday

(PHP 4, PHP 5, PHP 7, PHP 8)

gettimeofday — Get current time

Description

function **gettimeofday**(bool `$as_float = false`): array|float

This is an interface to gettimeofday(2). It returns an associative array containing the data returned from the system call.

Parameters

`as_float`

When set to `true`, a float instead of an array is returned.

Return Values

By default an array is returned. If `as_float` is set, then a float is returned.

Array keys:

- "sec" - seconds since the Unix Epoch
- "usec" - microseconds
- "minuteswest" - minutes west of Greenwich
- "dsttime" - type of dst correction

Examples

Example #1 gettimeofday() example

```
<?php
print_r(gettimeofday());

echo gettimeofday(true);
```

The above example will output something similar to:

```
Array
(
    [sec] => 1073504408
    [usec] => 238215
    [minuteswest] => 0
    [dsttime] => 1
)
1073504408.23910
```

gmdate

gmdate

(PHP 4, PHP 5, PHP 7, PHP 8)

gmdate — Format a GMT/UTC date/time

Description

function **gmdate**(string `$format`, ?int `$timestamp` = `null`): string

Identical to the date() function except that the time returned is Greenwich Mean Time (GMT).

Parameters

`format`

The format of the outputted date string. See the formatting options for the date() function.

`timestamp`

The optional `timestamp` parameter is an int Unix timestamp that defaults to the current local time if `timestamp` is omitted or `null`. In other words, it defaults to the value of time().

Return Values

Returns a formatted date string.

Changelog

Version	Description
8.0.0	`timestamp` is nullable now.

Examples

Example #1 **gmdate()** example

```php
<?php
date_default_timezone_set("Europe/Helsinki");

echo date("M d Y H:i:s e", mktime(0, 0, 0, 1, 1, 1998)) . "\n";
echo gmdate("M d Y H:i:s e", mktime(0, 0, 0, 1, 1, 1998));
```

The above example will output:

```
Jan 01 1998 00:00:00 Europe/Helsinki
Dec 31 1997 22:00:00 UTC
```

See Also

- DateTimeImmutable::__construct() - Returns new DateTimeImmutable object
- DateTimeInterface::format() - Returns date formatted according to given format
- date() - Format a Unix timestamp
- mktime() - Get Unix timestamp for a date
- gmmktime() - Get Unix timestamp for a GMT date
- IntlDateFormatter::format() - Format the date/time value as a string

gmmktime

gmmktime

(PHP 4, PHP 5, PHP 7, PHP 8)

gmmktime — Get Unix timestamp for a GMT date

Description

```
function gmmktime(
    int $hour,
    ?int $minute = null,
    ?int $second = null,
    ?int $month = null,
    ?int $day = null,
    ?int $year = null
): int|false
```

Identical to mktime() except the passed parameters represents a GMT date. **gmmktime()** internally uses mktime() so only times valid in derived local time can be used.

Like mktime(), optional arguments may be left out in order from right to left, with any omitted arguments being set to the current corresponding GMT value.

Calling **gmmktime()** without any arguments is not supported, and will result in an ArgumentCountError. time() can be used to get the current timestamp.

Parameters

`hour`

> The number of the hour relative to the start of the day determined by `month`, `day` and `year`. Negative values reference the hour before midnight of the day in question. Values greater than 23 reference the appropriate hour in the following day(s).

`minute`

The number of the minute relative to the start of the `hour`. Negative values reference the minute in the previous hour. Values greater than 59 reference the appropriate minute in the following hour(s).

`second`

The number of seconds relative to the start of the `minute`. Negative values reference the second in the previous minute. Values greater than 59 reference the appropriate second in the following minute(s).

`month`

The number of the month relative to the end of the previous year. Values 1 to 12 reference the normal calendar months of the year in question. Values less than 1 (including negative values) reference the months in the previous year in reverse order, so 0 is December, -1 is November, etc. Values greater than 12 reference the appropriate month in the following year(s).

`day`

The number of the day relative to the end of the previous month. Values 1 to 28, 29, 30 or 31 (depending upon the month) reference the normal days in the relevant month. Values less than 1 (including negative values) reference the days in the previous month, so 0 is the last day of the previous month, -1 is the day before that, etc. Values greater than the number of days in the relevant month reference the appropriate day in the following month(s).

`year`

The year

Return Values

Returns a int Unix timestamp on success, or `false` if the timestamp doesn't fit in a PHP integer.

Changelog

Version	Description
8.0.0	`hour` is no longer optional. If you need a Unix timestamp, use time().
8.0.0	`minute`, `second`, `month`, `day` and `year` are nullable now.

Examples

Example #1 **gmmktime()** basic example

```
<?php
date_default_timezone_set("Indian/Maldives");

echo "July 1, 2000 is on a " . date("l", gmmktime(0, 0, 0, 7, 1, 2000));
```

The above example will output:

```
July 1, 2000 is on a Saturday
```

See Also

- The DateTimeImmutable class
- mktime() - Get Unix timestamp for a date
- date() - Format a Unix timestamp
- time() - Return current Unix timestamp

gmstrftime

gmstrftime

(PHP 4, PHP 5, PHP 7, PHP 8)

gmstrftime — Format a GMT/UTC time/date according to locale settings

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.1.0. Relying on this function is highly discouraged.

Alternatives to this function include:

- gmdate()
- IntlDateFormatter::format()

Description

```
#[\Deprecated(
    since: '8.1',
    message: 'use IntlDateFormatter::format() instead',
)]
function gmstrftime(string $format, ?int $timestamp = null): string|false
```

Behaves the same as strftime() except that the time returned is Greenwich Mean Time (GMT). For example, when run in Eastern Standard Time (GMT -0500), the first line below prints "Dec 31 1998 20:00:00", while the second prints "Jan 01 1999 01:00:00".

> **Warning**
>
> This function depends on operating system locale information, which might be inconsistent with each other, or not available at all. Instead use the IntlDateFormatter::format() method.

Parameters

`format`

See description in strftime().

`timestamp`

The optional `timestamp` parameter is an int Unix timestamp that defaults to the current local time if `timestamp` is omitted or `null`. In other words, it defaults to the value of time().

Return Values

Returns a string formatted according to the given format string using the given `timestamp` or the current local time if no timestamp is given. Month and weekday names and other language dependent strings respect the current locale set with setlocale(). On failure, `false` is returned.

Changelog

Version	Description
8.0.0	`timestamp` is nullable now.

Examples

Example #1 gmstrftime() example

```php
<?php

setlocale(LC_TIME, 'es_ES.UTF-8');
date_default_timezone_set('EST');

echo strftime("%B %d %Y %H:%M:%S",   mktime(20, 0, 0, 12, 31, 98)) . "\n";
echo gmstrftime("%B %d %Y %H:%M:%S", mktime(20, 0, 0, 12, 31, 98));
```

The above example will output:

```
```

```
Deprecated: Function strftime() is deprecated since 8.1, use IntlDateFormatter::format() instead in script on line 6
diciembre 31 1998 20:00:00

Deprecated: Function gmstrftime() is deprecated since 8.1, use IntlDateFormatter::format() instead in script on line 7
enero 01 1999 01:00:00
```

See Also

- IntlDateFormatter::format() - Format the date/time value as a string
- DateTimeInterface::format() - Returns date formatted according to given format
- strftime() - Format a local time/date according to locale settings

idate

idate

(PHP 5, PHP 7, PHP 8)

idate — Format a local time/date part as integer

Description

function **idate**(string `$format`, ?int `$timestamp` = `null`): int|false

Returns a number formatted according to the given format string using the given integer `timestamp` or the current local time if no timestamp is given. In other words, `timestamp` is optional and defaults to the value of time().

Unlike the function date(), **idate()** accepts just one char in the `format` parameter.

Parameters

`format`

The following characters are recognized in the `format` parameter string

`format` character	Description
`B`	Swatch Beat/Internet Time
`d`	Day of the month
`h`	Hour (12 hour format)
`H`	Hour (24 hour format)
`i`	Minutes
`I` (uppercase i)	returns `1` if DST is activated, `0` otherwise
`L` (uppercase l)	returns `1` for leap year, `0` otherwise
`m`	Month number
`N`	ISO-8601 day of the week (`1` for Monday through `7` for Sunday)
`o`	ISO-8601 year (4 digits)
`s`	Seconds
`t`	Days in current month
`U`	Seconds since the Unix Epoch - January 1 1970 00:00:00 UTC - this is the same as time()
`w`	Day of the week (`0` on Sunday)
`W`	ISO-8601 week number of year, weeks starting on Monday
`y`	Year (1 or 2 digits - check note below)
`Y`	Year (4 digits)
`z`	Day of the year
`Z`	Timezone offset in seconds

`timestamp`

The optional `timestamp` parameter is an int Unix timestamp that defaults to the current local time if `timestamp` is omitted or `null`. In other words, it defaults to the value of time().

Return Values

Returns an int on success, or `false` on failure.

As **idate()** always returns an int and as they can't start with a "0", **idate()** may return fewer digits than you would expect. See the example below.

Errors/Exceptions

Every call to a date/time function will generate a `E_WARNING` if the time zone is not valid. See also date_default_timezone_set()

Changelog

Version	Description
8.2.0	Adds the `N` (ISO-8601 day of the week) and `o` (ISO-8601 year) format characters.
8.0.0	`timestamp` is nullable now.

Examples

Example #1 idate() example

```php
<?php
$timestamp = strtotime('1st January 2004'); // 1072915200

// this prints the year in a two digit format
// however, as this would start with a "0", it
// only prints "4"
echo idate('y', $timestamp) . "\n";

$timestamp = strtotime('1st January 2024'); // 1704067200
echo idate('y', $timestamp);
```

The above example will output:

```
4
24
```

See Also

- DateTimeInterface::format() - Returns date formatted according to given format
- date() - Format a Unix timestamp
- getdate() - Get date/time information
- time() - Return current Unix timestamp

localtime

localtime

(PHP 4, PHP 5, PHP 7, PHP 8)

localtime — Get the local time

Description

function **localtime**(?int `$timestamp = null`, bool `$associative = false`): array

The **localtime()** function returns an array identical to that of the structure returned by the C function call.

Parameters

`timestamp`

The optional `timestamp` parameter is an int Unix timestamp that defaults to the current local time if `timestamp` is omitted or `null`. In other words, it defaults to the value of time().

`associative`

Determines whether the function should return a regular, numerically indexed array, or an associative one.

Return Values

If `associative` is set to `false` or not supplied then the array is returned as a regular, numerically indexed array. If `associative` is set to `true` then **localtime()** returns an associative array containing the elements of the structure returned by the C function call to localtime. The keys of the associative array are as follows:

- "tm_sec" - seconds, `0` to `59`
- "tm_min" - minutes, `0` to `59`
- "tm_hour" - hours, `0` to `23`
- "tm_mday" - day of the month, `1` to `31`
- "tm_mon" - month of the year, `0` (Jan) to `11` (Dec)
- "tm_year" - years since 1900
- "tm_wday" - day of the week, `0` (Sun) to `6` (Sat)
- "tm_yday" - day of the year, `0` to `365`
- "tm_isdst" - is daylight savings time in effect?

  Positive if yes, `0` if not, negative if unknown.

Errors/Exceptions

Every call to a date/time function will generate a `E_WARNING` if the time zone is not valid. See also date_default_timezone_set()

Changelog

Version	Description
8.0.0	`timestamp` is nullable now.

Examples

Example #1 **localtime()** example

```php
<?php
$localtime = localtime();
$localtime_assoc = localtime(time(), true);
print_r($localtime);
print_r($localtime_assoc);
```

The above example will output something similar to:

```
Array
(
    [0] => 24
    [1] => 3
    [2] => 19
    [3] => 3
    [4] => 3
    [5] => 105
    [6] => 0
    [7] => 92
    [8] => 1
)
```

```
Array
(
    [tm_sec] => 24
    [tm_min] => 3
    [tm_hour] => 19
    [tm_mday] => 3
    [tm_mon] => 3
    [tm_year] => 105
    [tm_wday] => 0
    [tm_yday] => 92
    [tm_isdst] => 1
)
```

See Also

- getdate() - Get date/time information

microtime

microtime

(PHP 4, PHP 5, PHP 7, PHP 8)

microtime — Return current Unix timestamp with microseconds

Description

function **microtime**(bool `$as_float = false`): string|float

microtime() returns the current Unix timestamp with microseconds. This function is only available on operating systems that support the gettimeofday() system call.

For performance measurements, using hrtime() is recommended.

Parameters

`as_float`

If used and set to `true`, **microtime()** will return a float instead of a string, as described in the return values section below.

Return Values

By default, **microtime()** returns a string in the form "msec sec", where `sec` is the number of seconds since the Unix epoch (0:00:00 January 1,1970 GMT), and `msec` measures microseconds that have elapsed since `sec` and is also expressed in seconds as a decimal fraction.

If `as_float` is set to `true`, then **microtime()** returns a float, which represents the current time in seconds since the Unix epoch accurate to the nearest microsecond.

Examples

Example #1 Timing script execution

```php
<?php
$time_start = microtime(true);

// Sleep for a while
usleep(10_000);

$time_end = microtime(true);
$time = $time_end - $time_start;

print "Did nothing in $time seconds\n";
```

Example #2 microtime() and `REQUEST_TIME_FLOAT`

```php
<?php
// Randomize sleeping time
usleep(random_int(10_000, 1_000_000));

// REQUEST_TIME_FLOAT is available in the $_SERVER superglobal array.
// It contains the timestamp of the start of the request with microsecond precision.
$time = microtime(true) - $_SERVER["REQUEST_TIME_FLOAT"];

echo "Did nothing in $time seconds\n";
```

See Also

- time() - Return current Unix timestamp
- hrtime() - Get the system's high resolution time

mktime

mktime

(PHP 4, PHP 5, PHP 7, PHP 8)

mktime — Get Unix timestamp for a date

Description

function **mktime**(
  int $hour,
  ?int $minute = null,
  ?int $second = null,
  ?int $month = null,
  ?int $day = null,
  ?int $year = null
): int|false

Returns the Unix timestamp corresponding to the arguments given. This timestamp is a long integer containing the number of seconds between the Unix Epoch (January 1 1970 00:00:00 GMT) and the time specified.

Any optional arguments omitted or `null` will be set to the current value according to the local date and time.

> **Warning**
>
> Please note that the ordering of arguments is in an odd order: `month`, `day`, `year`, and not in the more reasonable order of `year`, `month`, `day`.

Calling **mktime()** without arguments is not supported, and will result in an ArgumentCountError. time() can be used to get the current timestamp.

Parameters

`hour`

The number of the hour relative to the start of the day determined by `month`, `day` and `year`. Negative values reference the hour before midnight of the day in question. Values greater than 23 reference the appropriate hour in the following day(s).

`minute`

The number of the minute relative to the start of the `hour`. Negative values reference the minute in the previous hour. Values greater than 59 reference the appropriate minute in the following hour(s).

`second`

The number of seconds relative to the start of the `minute`. Negative values reference the second in the previous minute. Values greater than 59 reference the appropriate second in the following minute(s).

`month`

The number of the month relative to the end of the previous year. Values 1 to 12 reference the normal calendar months of the year in question. Values less than 1 (including negative values) reference the months in the previous year in reverse order, so 0 is

December, -1 is November, etc. Values greater than 12 reference the appropriate month in the following year(s).

`day`

The number of the day relative to the end of the previous month. Values 1 to 28, 29, 30 or 31 (depending upon the month) reference the normal days in the relevant month. Values less than 1 (including negative values) reference the days in the previous month, so 0 is the last day of the previous month, -1 is the day before that, etc. Values greater than the number of days in the relevant month reference the appropriate day in the following month(s).

`year`

The number of the year, may be a two or four digit value, with values between 0-69 mapping to 2000-2069 and 70-100 to 1970-2000. On systems where time_t is a 32bit signed integer, as most common today, the valid range for `year` is somewhere between 1901 and 2038.

Return Values

mktime() returns the Unix timestamp of the arguments given, or `false` if the timestamp doesn't fit in a PHP integer.

Changelog

Version	Description
8.0.0	`hour` is no longer optional. If you need a Unix timestamp, use time().
8.0.0	`minute`, `second`, `month`, `day` and `year` are nullable now.

Examples

Example #1 **mktime()** basic example

```php
<?php
// Set the default timezone to use.
date_default_timezone_set('UTC');

// Prints: July 1, 2000 is on a Saturday
echo "July 1, 2000 is on a " . date("l", mktime(0, 0, 0, 7, 1, 2000)) . "\n";

// Prints something like: 2006-04-05T01:02:03+00:00
echo date('c', mktime(1, 2, 3, 4, 5, 2006)) . "\n";
```

The above example will output something similar to:

```
July 1, 2000 is on a Saturday
2006-04-05T01:02:03+00:00
```

Example #2 **mktime()** example

mktime() is useful for doing date arithmetic and validation, as it will automatically calculate the correct value for out-of-range input. For example, each of the following lines produces the string "Jan-01-1998".

```php
<?php
date_default_timezone_set('America/New_York');

echo date("c", mktime(0, 0, 0, 12, 32, 1997)) . "\n";
echo date("c", mktime(0, 0, 0, 13, 1, 1997)) . "\n";
echo date("c", mktime(0, 0, 0, 1, 1, 1998)) . "\n";
echo date("c", mktime(0, 0, 0, 1, 1, 98)) . "\n";
```

The above example will output something similar to:

```
1998-01-01T00:00:00-05:00
1998-01-01T00:00:00-05:00
1998-01-01T00:00:00-05:00
1998-01-01T00:00:00-05:00
```

Example #3 Using mktime to find relative dates

```php
<?php
date_default_timezone_set('Asia/Tokyo');

$tomorrow  = mktime(0, 0, 0, date("m")  , date("d")+1, date("Y"));
print date('c', $tomorrow) . "\n";

$lastmonth = mktime(0, 0, 0, date("m")-1, date("d"),   date("Y"));
print date('c', $lastmonth) . "\n";

$nextyear  = mktime(0, 0, 0, date("m"),   date("d"),   date("Y")+1) . "\n";
print date('c', $nextyear) . "\n";
```

The above example will output something similar to:

```
2025-09-30T00:00:00+09:00
2025-08-29T00:00:00+09:00
2026-09-29T00:00:00+09:00
```

> **Note** :
>
> This can be more reliable than simply adding or subtracting the number of seconds in a day or month to a timestamp because of daylight saving time.

Example #4 Last day of a month

The last day of any given month can be expressed as the "0" day of the next month, not the -1 day. Both of the following examples will produce the string "Last day in Feb 2000 is: 29".

```php
<?php

$lastday = mktime(0, 0, 0, 3, 0, 2000);
echo 'Last day in Feb 2000 is: ', date('d', $lastday) . "\n";

$lastday = mktime(0, 0, 0, 4, -31, 2000);
echo 'Last day in Feb 2000 is: ', date('d', $lastday) . "\n";
```

The above example will output:

```
Last day in Feb 2000 is: 29
Last day in Feb 2000 is: 29
```

See Also

- The DateTimeImmutable class
- checkdate() - Validate a Gregorian date

- gmmktime() - Get Unix timestamp for a GMT date
- date() - Format a Unix timestamp
- time() - Return current Unix timestamp

strftime

strftime

(PHP 4, PHP 5, PHP 7, PHP 8)

strftime — Format a local time/date according to locale settings

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.1.0. Relying on this function is highly discouraged.

Alternatives to this function include:

- date()
- IntlDateFormatter::format()

Description

```
#[\Deprecated(
    since: '8.1',
    message: 'use IntlDateFormatter::format() instead',
)]
function strftime(string $format, ?int $timestamp = null): string|false
```

Format the time and/or date according to locale settings. Month and weekday names and other language-dependent strings respect the current locale set with setlocale().

> **Warning**
>
> Not all conversion specifiers may be supported by your C library, in which case they will not be supported by PHP's **strftime()**. Additionally, not all platforms support negative timestamps, so your date range may be limited to no earlier than the Unix epoch. This means that %e, %T, %R and, %D (and possibly others) - as well as dates prior to `Jan 1, 1970` - will not work on Windows, some Linux distributions, and a few other operating systems. For Windows systems, a complete overview of supported conversion specifiers can be found at » MSDN. Instead use the IntlDateFormatter::format() method.

Parameters

`format`

The following characters are recognized in the `format` parameter string

`format`	Description	Example returned values
Day	---	---
`%a`	An abbreviated textual representation of the day	`Sun` through `Sat`
`%A`	A full textual representation of the day	`Sunday` through `Saturday`
`%d`	Two-digit day of the month (with leading zeros)	`01` to `31`
`%e`	Day of the month, with a space preceding single digits. Not implemented as described on Windows. See below for more information.	`1` to `31`
`%j`	Day of the year, 3 digits with leading zeros	`001` to `366`
`%u`	ISO-8601 numeric representation of the day of the week	`1` (for Monday) through `7` (for Sunday)
`%w`	Numeric representation of the day of the week	`0` (for Sunday) through `6` (for Saturday)
Week	---	---
`%U`	Week number of the given year, starting with the first Sunday as the first week	`13` (for the 13th full week of the year)

format	Description	Example returned values
%V	ISO-8601:1988 week number of the given year, starting with the first week of the year with at least 4 weekdays, with Monday being the start of the week	01 through 53 (where 53 accounts for an overlapping week)
%W	A numeric representation of the week of the year, starting with the first Monday as the first week	46 (for the 46th week of the year beginning with a Monday)
Month	---	---
%b	Abbreviated month name, based on the locale	Jan through Dec
%B	Full month name, based on the locale	January through December
%h	Abbreviated month name, based on the locale (an alias of %b)	Jan through Dec
%m	Two digit representation of the month	01 (for January) through 12 (for December)
Year	---	---
%C	Two digit representation of the century (year divided by 100, truncated to an integer)	19 for the 20th Century
%g	Two digit representation of the year going by ISO-8601:1988 standards (see %V)	Example: 09 for the week of January 6, 2009
%G	The full four-digit version of %g	Example: 2008 for the week of January 3, 2009
%y	Two digit representation of the year	Example: 09 for 2009, 79 for 1979
%Y	Four digit representation for the year	Example: 2038
Time	---	---
%H	Two digit representation of the hour in 24-hour format	00 through 23
%k	Hour in 24-hour format, with a space preceding single digits	0 through 23
%I	Two digit representation of the hour in 12-hour format	01 through 12
%l (lower-case 'L')	Hour in 12-hour format, with a space preceding single digits	1 through 12
%M	Two digit representation of the minute	00 through 59
%p	UPPER-CASE 'AM' or 'PM' based on the given time	Example: AM for 00:31, PM for 22:23. The exact result depends on the Operating System, and they can also return lower-case variants, or variants with dots (such as a.m.).
%P	lower-case 'am' or 'pm' based on the given time	Example: am for 00:31, pm for 22:23. Not supported by all Operating Systems.
%r	Same as "%I:%M:%S %p"	Example: 09:34:17 PM for 21:34:17
%R	Same as "%H:%M"	Example: 00:35 for 12:35 AM, 16:44 for 4:44 PM
%S	Two digit representation of the second	00 through 59
%T	Same as "%H:%M:%S"	Example: 21:34:17 for 09:34:17 PM
%X	Preferred time representation based on locale, without the date	Example: 03:59:16 or 15:59:16
%z	The time zone offset. Not implemented as described on Windows. See below for more information.	Example: -0500 for US Eastern Time
%Z	The time zone abbreviation. Not implemented as described on Windows. See below for more information.	Example: EST for Eastern Time
Time and Date Stamps	---	---
%c	Preferred date and time stamp based on locale	Example: Tue Feb 5 00:45:10 2009 for February 5, 2009 at 12:45:10 AM

format	Description	Example returned values
%D	Same as "%m/%d/%y"	Example: 02/05/09 for February 5, 2009
%F	Same as "%Y-%m-%d" (commonly used in database datestamps)	Example: 2009-02-05 for February 5, 2009
%s	Unix Epoch Time timestamp (same as the time() function)	Example: 305815200 for September 10, 1979 08:40:00 AM
%x	Preferred date representation based on locale, without the time	Example: 02/05/09 for February 5, 2009
Miscellaneous	---	---
%n	A newline character ("\n")	---
%t	A Tab character ("\t")	---
%%	A literal percentage character ("%")	---

> **Warning**
>
> Contrary to ISO-9899:1999, Sun Solaris starts with Sunday as 1. As a result, %u may not function as described in this manual.

> **Warning**
>
> *Windows only:*
>
> The %e modifier is not supported in the Windows implementation of this function. To achieve this value, the %#d modifier can be used instead. The example below illustrates how to write a cross platform compatible function.
>
> The %z and %Z modifiers both return the time zone name instead of the offset or abbreviation.

> **Warning**
>
> *macOS and musl only:* The %P modifier is not supported in the macOS implementation of this function.

`timestamp`

The optional `timestamp` parameter is an int Unix timestamp that defaults to the current local time if `timestamp` is omitted or **`null`**. In other words, it defaults to the value of time().

Return Values

Returns a string formatted according `format` using the given `timestamp` or the current local time if no timestamp is given. Month and weekday names and other language-dependent strings respect the current locale set with setlocale(). The function returns **`false`** if `format` is empty, contains unsupported conversion specifiers, or if the length of the returned string would be greater than `4095`.

Errors/Exceptions

Every call to a date/time function will generate a **`E_WARNING`** if the time zone is not valid. See also date_default_timezone_set()

As the output is dependent upon the underlying C library, some conversion specifiers are not supported. On Windows, supplying unknown conversion specifiers will result in 5 **`E_WARNING`** messages and return **`false`**. On other operating systems you may not get any **`E_WARNING`** messages and the output may contain the conversion specifiers unconverted.

Changelog

Version	Description
8.0.0	`timestamp` is nullable now.

Examples

This example will work if you have the respective locales installed in your system.

Example #1 strftime() locale examples

```
<?php
setlocale(LC_TIME, "C");
```

```
echo strftime("%A");
setlocale(LC_TIME, "fi_FI");
echo strftime(" in Finnish is %A,");
setlocale(LC_TIME, "fr_FR");
echo strftime(" in French %A and");
setlocale(LC_TIME, "de_DE");
echo strftime(" in German %A.\n");
```

Example #2 ISO 8601:1988 week number example

```
<?php
/*     December 2002 / January 2003
ISOWk  M   Tu  W   Thu F   Sa  Su
-----  ----------------------------
51     16  17  18  19  20  21  22
52     23  24  25  26  27  28  29
1      30  31   1   2   3   4   5
2       6   7   8   9  10  11  12
3      13  14  15  16  17  18  19   */

// Outputs: 12/28/2002 - %V,%G,%Y = 52,2002,2002
echo "12/28/2002 - %V,%G,%Y = " . strftime("%V,%G,%Y", strtotime("12/28/2002")) . "\n";

// Outputs: 12/30/2002 - %V,%G,%Y = 1,2003,2002
echo "12/30/2002 - %V,%G,%Y = " . strftime("%V,%G,%Y", strtotime("12/30/2002")) . "\n";

// Outputs: 1/3/2003 - %V,%G,%Y = 1,2003,2003
echo "1/3/2003 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/3/2003")) . "\n";

// Outputs: 1/10/2003 - %V,%G,%Y = 2,2003,2003
echo "1/10/2003 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/10/2003")) . "\n";



/*     December 2004 / January 2005
ISOWk  M   Tu  W   Thu F   Sa  Su
-----  ----------------------------
51     13  14  15  16  17  18  19
52     20  21  22  23  24  25  26
53     27  28  29  30  31   1   2
1       3   4   5   6   7   8   9
2      10  11  12  13  14  15  16   */

// Outputs: 12/23/2004 - %V,%G,%Y = 52,2004,2004
echo "12/23/2004 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("12/23/2004")) . "\n";

// Outputs: 12/31/2004 - %V,%G,%Y = 53,2004,2004
echo "12/31/2004 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("12/31/2004")) . "\n";

// Outputs: 1/2/2005 - %V,%G,%Y = 53,2004,2005
echo "1/2/2005 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/2/2005")) . "\n";

// Outputs: 1/3/2005 - %V,%G,%Y = 1,2005,2005
echo "1/3/2005 - %V,%G,%Y = " . strftime("%V,%G,%Y",strtotime("1/3/2005")) . "\n";
```

Example #3 Cross platform compatible example using the `%e` modifier

```
<?php

// Jan 1: results in: '%e%1%' (%%, e, %%, %e, %%)
$format = '%%e%%%e%%';

// Check for Windows to find and replace the %e
// modifier correctly
if (strtoupper(substr(PHP_OS, 0, 3)) == 'WIN') {
    $format = preg_replace('#(?<!%)((?:%%)*)%e#', '\1%#d', $format);
}

echo strftime($format);
```

Example #4 Display all known and unknown formats

```php
<?php

// Describe the formats
$strftimeFormats = array(
    'A' => 'A full textual representation of the day',
    'B' => 'Full month name, based on the locale',
    'C' => 'Two digit representation of the century (year divided by 100, truncated to an integer)',
    'D' => 'Same as "%m/%d/%y"',
    'E' => '',
    'F' => 'Same as "%Y-%m-%d"',
    'G' => 'The full four-digit version of %g',
    'H' => 'Two digit representation of the hour in 24-hour format',
    'I' => 'Two digit representation of the hour in 12-hour format',
    'J' => '',
    'K' => '',
    'L' => '',
    'M' => 'Two digit representation of the minute',
    'N' => '',
    'O' => '',
    'P' => 'lower-case "am" or "pm" based on the given time',
    'Q' => '',
    'R' => 'Same as "%H:%M"',
    'S' => 'Two digit representation of the second',
    'T' => 'Same as "%H:%M:%S"',
    'U' => 'Week number of the given year, starting with the first Sunday as the first week',
    'V' => 'ISO-8601:1988 week number of the given year, starting with the first week of the year with at least 4 weekdays,
with Monday being the start of the week',
    'W' => 'A numeric representation of the week of the year, starting with the first Monday as the first week',
    'X' => 'Preferred time representation based on locale, without the date',
    'Y' => 'Four digit representation for the year',
    'Z' => 'The time zone offset/abbreviation option NOT given by %z (depends on operating system)',
    'a' => 'An abbreviated textual representation of the day',
    'b' => 'Abbreviated month name, based on the locale',
    'c' => 'Preferred date and time stamp based on local',
    'd' => 'Two-digit day of the month (with leading zeros)',
    'e' => 'Day of the month, with a space preceding single digits',
    'f' => '',
    'g' => 'Two digit representation of the year going by ISO-8601:1988 standards (see %V)',
    'h' => 'Abbreviated month name, based on the locale (an alias of %b)',
    'i' => '',
    'j' => 'Day of the year, 3 digits with leading zeros',
    'k' => 'Hour in 24-hour format, with a space preceding single digits',
    'l' => 'Hour in 12-hour format, with a space preceding single digits',
    'm' => 'Two digit representation of the month',
    'n' => 'A newline character ("\n")',
    'o' => '',
    'p' => 'UPPER-CASE "AM" or "PM" based on the given time',
    'q' => '',
    'r' => 'Same as "%I:%M:%S %p"',
    's' => 'Unix Epoch Time timestamp',
    't' => 'A Tab character ("\t")',
    'u' => 'ISO-8601 numeric representation of the day of the week',
    'v' => '',
    'w' => 'Numeric representation of the day of the week',
    'x' => 'Preferred date representation based on locale, without the time',
    'y' => 'Two digit representation of the year',
    'z' => 'Either the time zone offset from UTC or the abbreviation (depends on operating system)',
    '%' => 'A literal percentage character ("%")',
);

// Results
$strftimeValues = array();

// Evaluate the formats whilst suppressing any errors
foreach ($strftimeFormats as $format => $description) {
    if (false !== ($value = @strftime("%{$format}"))) {
        $strftimeValues[$format] = $value;
    }
}

// Find the longest value
$maxValueLength = 2 + max(array_map('strlen', $strftimeValues));
```

```php
// Report known formats
foreach ($strftimeValues as $format => $value) {
    echo "Known format   : '{$format}' = ", str_pad("'{$value}'", $maxValueLength), " ( {$strftimeFormats[$format]} )\n";
}

// Report unknown formats
foreach (array_diff_key($strftimeFormats, $strftimeValues) as $format => $description) {
    echo "Unknown format : '{$format}'   ", str_pad(' ', $maxValueLength), ($description ? " ( {$description} )" : ''), "\n";
}
```

The above example will output something similar to:

```
Known format   : 'A' = 'Friday'           ( A full textual representation of the day )
Known format   : 'B' = 'December'         ( Full month name, based on the locale )
Known format   : 'H' = '11'               ( Two digit representation of the hour in 24-hour format )
Known format   : 'I' = '11'               ( Two digit representation of the hour in 12-hour format )
Known format   : 'M' = '24'               ( Two digit representation of the minute )
Known format   : 'S' = '44'               ( Two digit representation of the second )
Known format   : 'U' = '48'               ( Week number of the given year, starting with the first Sunday as the first week
)
Known format   : 'W' = '48'               ( A numeric representation of the week of the year, starting with the first
Monday as the first week )
Known format   : 'X' = '11:24:44'         ( Preferred time representation based on locale, without the date )
Known format   : 'Y' = '2010'             ( Four digit representation for the year )
Known format   : 'Z' = 'GMT Standard Time' ( The time zone offset/abbreviation option NOT given by %z (depends on operating
system) )
Known format   : 'a' = 'Fri'              ( An abbreviated textual representation of the day )
Known format   : 'b' = 'Dec'              ( Abbreviated month name, based on the locale )
Known format   : 'c' = '12/03/10 11:24:44' ( Preferred date and time stamp based on local )
Known format   : 'd' = '03'               ( Two-digit day of the month (with leading zeros) )
Known format   : 'j' = '337'              ( Day of the year, 3 digits with leading zeros )
Known format   : 'm' = '12'               ( Two digit representation of the month )
Known format   : 'p' = 'AM'               ( UPPER-CASE "AM" or "PM" based on the given time )
Known format   : 'w' = '5'                ( Numeric representation of the day of the week )
Known format   : 'x' = '12/03/10'         ( Preferred date representation based on locale, without the time )
Known format   : 'y' = '10'               ( Two digit representation of the year )
Known format   : 'z' = 'GMT Standard Time' ( Either the time zone offset from UTC or the abbreviation (depends on operating
system) )
Known format   : '%' = '%'                ( A literal percentage character ("%") )
Unknown format : 'C'                      ( Two digit representation of the century (year divided by 100, truncated to an
integer) )
Unknown format : 'D'                      ( Same as "%m/%d/%y" )
Unknown format : 'E'
Unknown format : 'F'                      ( Same as "%Y-%m-%d" )
Unknown format : 'G'                      ( The full four-digit version of %g )
Unknown format : 'J'
Unknown format : 'K'
Unknown format : 'L'
Unknown format : 'N'
Unknown format : 'O'
Unknown format : 'P'                      ( lower-case "am" or "pm" based on the given time )
Unknown format : 'Q'
Unknown format : 'R'                      ( Same as "%H:%M" )
Unknown format : 'T'                      ( Same as "%H:%M:%S" )
Unknown format : 'V'                      ( ISO-8601:1988 week number of the given year, starting with the first week of
the year with at least 4 weekdays, with Monday being the start of the week )
Unknown format : 'e'                      ( Day of the month, with a space preceding single digits )
Unknown format : 'f'
Unknown format : 'g'                      ( Two digit representation of the year going by ISO-8601:1988 standards (see %V)
)
Unknown format : 'h'                      ( Abbreviated month name, based on the locale (an alias of %b) )
Unknown format : 'i'
Unknown format : 'k'                      ( Hour in 24-hour format, with a space preceding single digits )
Unknown format : 'l'                      ( Hour in 12-hour format, with a space preceding single digits )
Unknown format : 'n'                      ( A newline character ("\n") )
Unknown format : 'o'
Unknown format : 'q'
Unknown format : 'r'                      ( Same as "%I:%M:%S %p" )
```

```
Unknown format : 's'                    ( Unix Epoch Time timestamp )
Unknown format : 't'                    ( A Tab character ("\t") )
Unknown format : 'u'                    ( ISO-8601 numeric representation of the day of the week )
Unknown format : 'v'
```

Notes

> **Note** :
>
> %G and %V, which are based on ISO 8601:1988 week numbers can give unexpected (albeit correct) results if the numbering system is not thoroughly understood. See %V examples in this manual page.

See Also

- IntlDateFormatter::format() - Format the date/time value as a string
- DateTimeInterface::format() - Returns date formatted according to given format
- » Online strftime() format design tool
- setlocale() - Set locale information
- mktime() - Get Unix timestamp for a date
- strptime() - Parse a time/date generated with strftime
- gmstrftime() - Format a GMT/UTC time/date according to locale settings
- » Open Group specification of **strftime()**

strptime

strptime

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

strptime — Parse a time/date generated with strftime()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.1.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.2',
    message: 'use date_parse_from_format() (for locale-independent parsing), or IntlDateFormatter::parse() (for locale-dependent parsing) instead',
)]
function strptime(string $timestamp, string $format): array|false
```

strptime() returns an array with the `timestamp` parsed, or false on error.

Month and weekday names and other language dependent strings respect the current locale set with setlocale() (LC_TIME).

Parameters

`timestamp` (string)

The string to parse (e.g. returned from strftime()).

`format` (string)

The format used in `timestamp` (e.g. the same as used in strftime()). Note that some of the format options available to strftime() may not have any effect within **strptime()**; the exact subset that are supported will vary based on the operating system and C library in use.

For more information about the format options, read the strftime() page.

Return Values

Returns an array or `false` on failure.

The following parameters are returned in the array

parameters	Description
`"tm_sec"`	Seconds after the minute (0-61)
`"tm_min"`	Minutes after the hour (0-59)
`"tm_hour"`	Hour since midnight (0-23)
`"tm_mday"`	Day of the month (1-31)
`"tm_mon"`	Months since January (0-11)
`"tm_year"`	Years since 1900
`"tm_wday"`	Days since Sunday (0-6)
`"tm_yday"`	Days since January 1 (0-365)
`"unparsed"`	the `timestamp` part which was not recognized using the specified `format`

Changelog

Version	Description
8.1.0	This function has been deprecated. Use date_parse_from_format() instead (for locale-independent parsing), or IntlDateFormatter::parse() (for locale-dependent parsing)

Examples

Example #1 strptime() example

```php
<?php
$format = '%d/%m/%Y %H:%M:%S';
$strf = strftime($format);

echo "$strf\n";

print_r(strptime($strf, $format));
```

The above example will output something similar to:

```
03/10/2004 15:54:19

Array
(
    [tm_sec] => 19
    [tm_min] => 54
    [tm_hour] => 15
    [tm_mday] => 3
    [tm_mon] => 9
    [tm_year] => 104
    [tm_wday] => 0
    [tm_yday] => 276
    [unparsed] =>
)
```

Notes

> **Note** :
>
> This function is not implemented on Windows platforms.

> **Note** :

> Internally, this function calls the `strptime()` function provided by the system's C library. This function can exhibit noticeably different behaviour across different operating systems. The use of date_parse_from_format(), which does not suffer from these issues, is recommended.

> **Note** :
>
> "`tm_sec`" includes any leap seconds (currently upto 2 a year). For more information on leap seconds, see the » Wikipedia article on leap seconds.

See Also

- IntlDateFormatter::parse() - Parse string to a timestamp value
- DateTime::createFromFormat() - Parses a time string according to a specified format
- checkdate() - Validate a Gregorian date
- strftime() - Format a local time/date according to locale settings
- date_parse_from_format() - Get info about given date formatted according to the specified format

strtotime

strtotime

(PHP 4, PHP 5, PHP 7, PHP 8)

strtotime — Parse about any English textual datetime description into a Unix timestamp

Description

function **strtotime**(string `$datetime`, ?int `$baseTimestamp` = `null`): int|false

The function expects to be given a string containing an English date format and will try to parse that format into a Unix timestamp (the number of seconds since January 1 1970 00:00:00 UTC), relative to the timestamp given in `baseTimestamp`, or the current time if `baseTimestamp` is not supplied. The date string parsing is defined in Date and Time Formats, and has several subtle considerations. Reviewing the full details there is strongly recommended.

> **Warning**
>
> The Unix timestamp that this function returns does not contain information about time zones. In order to do calculations with date/time information, you should use the more capable DateTimeImmutable.

Each parameter of this function uses the default time zone unless a time zone is specified in that parameter. Be careful not to use different time zones in each parameter unless that is intended. See date_default_timezone_get() on the various ways to define the default time zone.

Parameters

`datetime`

A date/time string. Valid formats are explained in Date and Time Formats.

`baseTimestamp`

The timestamp which is used as a base for the calculation of relative dates.

Return Values

Returns a timestamp on success, `false` otherwise.

Errors/Exceptions

Every call to a date/time function will generate a `E_WARNING` if the time zone is not valid. See also date_default_timezone_set()

Changelog

Version	Description
8.0.0	`baseTimestamp` is nullable now.

Examples

Example #1 A **strtotime()** example

```php
<?php
echo strtotime("now"), "\n";
echo strtotime("10 September 2000"), "\n";
echo strtotime("+1 day"), "\n";
echo strtotime("+1 week"), "\n";
echo strtotime("+1 week 2 days 4 hours 2 seconds"), "\n";
echo strtotime("next Thursday"), "\n";
echo strtotime("last Monday"), "\n";
```

Example #2 Checking for failure

```php
<?php
$str = 'Not Good';

if (($timestamp = strtotime($str)) === false) {
    echo "The string ($str) is bogus";
} else {
    echo "$str == " . date('l dS \o\f F Y h:i:s A', $timestamp);
}
```

Notes

> **Note** :
>
> "Relative" date in this case also means that if a particular component of the date/time stamp is not provided, it will be taken verbatim from the `baseTimestamp`. That is, `strtotime('February')`, if run on the 31st of May 2022, will be interpreted as `31 February 2022`, which will overflow into a timestamp on `3 March`. (In a leap year, it would be `2 March`.) Using `strtotime('1 February')` or `strtotime('first day of February')` would avoid that problem.

> **Note** :
>
> If the number of the year is specified in a two digit format, the values between 00-69 are mapped to 2000-2069 and 70-99 to 1970-1999. See the notes below for possible differences on 32bit systems (possible dates might end on 2038-01-19 03:14:07).

> **Note** :
>
> The valid range of a timestamp is typically from Fri, 13 Dec 1901 20:45:54 UTC to Tue, 19 Jan 2038 03:14:07 UTC. (These are the dates that correspond to the minimum and maximum values for a 32-bit signed integer.)
>
> For 64-bit versions of PHP, the valid range of a timestamp is effectively infinite, as 64 bits can represent approximately 293 billion years in either direction.

> **Note** :
>
> Using this function for mathematical operations is not advisable. It is better to use DateTime::add() and DateTime::sub().

See Also

- DateTimeImmutable
- DateTimeImmutable::createFromFormat() - Parses a time string according to a specified format
- Date and Time Formats
- checkdate() - Validate a Gregorian date
- strptime() - Parse a time/date generated with strftime

time

time

(PHP 4, PHP 5, PHP 7, PHP 8)

time — Return current Unix timestamp

Description

function **time**(): int

Returns the current time measured in the number of seconds since the Unix Epoch (January 1 1970 00:00:00 GMT).

> **Note** :
>
> Unix timestamps do not contain any information with regards to any local timezone. It is recommended to use the DateTimeImmutable class for handling date and time information in order to avoid the pitfalls that come with just Unix timestamps.

Parameters

This function has no parameters.

Return Values

Returns the current timestamp.

Examples

Example #1 time() example

```
<?php
echo 'Now: '. time();
```

The above example will output something similar to:

```
Now: 1660338149
```

Notes

> **Tip**
>
> Timestamp of the start of the request is available in *$_SERVER['REQUEST_TIME']*.

See Also

- DateTimeImmutable
- date() - Format a Unix timestamp
- microtime() - Return current Unix timestamp with microseconds

timezone_abbreviations_list

timezone_abbreviations_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_abbreviations_list — Alias of DateTimeZone::listAbbreviations()

Description

This function is an alias of: DateTimeZone::listAbbreviations()

timezone_identifiers_list

timezone_identifiers_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_identifiers_list — Alias of DateTimeZone::listIdentifiers()

Description

This function is an alias of: DateTimeZone::listIdentifiers()

timezone_location_get

timezone_location_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

timezone_location_get — Alias of DateTimeZone::getLocation()

Description

This function is an alias of: DateTimeZone::getLocation()

timezone_name_from_abbr

timezone_name_from_abbr

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

timezone_name_from_abbr — Returns a timezone name by guessing from abbreviation and UTC offset

Description

function **timezone_name_from_abbr**(string `$abbr`, int `$utcOffset = -1`, int `$isDST = -1`): string|false

Parameters

`abbr`

Time zone abbreviation.

`utcOffset`

Offset from GMT in seconds. Defaults to -1 which means that first found time zone corresponding to `abbr` is returned. Otherwise exact offset is searched and only if not found then the first time zone with any offset is returned.

`isDST`

Daylight saving time indicator. Defaults to -1, which means that whether the time zone has daylight saving or not is not taken into consideration when searching. If this is set to 1, then the `utcOffset` is assumed to be an offset with daylight saving in effect; if 0, then `utcOffset` is assumed to be an offset without daylight saving in effect. If `abbr` doesn't exist then the time zone is searched solely by the `utcOffset` and `isDST`.

Return Values

Returns time zone name on success or `false` on failure.

Examples

Example #1 A timezone_name_from_abbr() example

```php
<?php
echo timezone_name_from_abbr("CET") . "\n";
echo timezone_name_from_abbr("", 3600, 0) . "\n";
```

The above example will output something similar to:

```
Europe/Berlin
Europe/Paris
```

See Also

- timezone_abbreviations_list() - Alias of DateTimeZone::listAbbreviations

timezone_name_get

timezone_name_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_name_get — Alias of DateTimeZone::getName()

Description

This function is an alias of: DateTimeZone::getName()

timezone_offset_get

timezone_offset_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_offset_get — Alias of DateTimeZone::getOffset()

Description

This function is an alias of: DateTimeZone::getOffset()

timezone_open

timezone_open

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_open — Alias of DateTimeZone::__construct()

Description

This function is an alias of: DateTimeZone::__construct()

timezone_transitions_get

timezone_transitions_get

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

timezone_transitions_get — Alias of DateTimeZone::getTransitions()

Description

This function is an alias of: DateTimeZone::getTransitions()

timezone_version_get

timezone_version_get

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

timezone_version_get — Gets the version of the timezonedb

Description

function **timezone_version_get**(): string

Returns the current version of the timezonedb.

Parameters

This function has no parameters.

Return Values

Returns a string in the format `YYYY.increment`, such as `2022.2`.

If you have a timezone database version that is old (for example, it doesn't show the current year), then you can update the timezone information by either upgrading your PHP version, or installing the » timezonedb PECL package.

Some Linux distributions patch PHP's date/time support to use an alternative source for timezone information. In which case this function will return `0.system`. You are encouraged to install the » timezonedb PECL package in that case as well.

Examples

Example #1 Getting the timezonedb version

```php
<?php
echo timezone_version_get();
```

The above example will output something similar to:

```
2022.2
```

See Also

- List of Supported Timezones

Date/Time Errors and Exceptions

- DateError (extends Error)
    - DateObjectError

      DateRangeError
- DateException (extends Exception)
    - DateInvalidOperationException
    - DateInvalidTimezoneException
    - DateMalformedIntervalStringException
    - DateMalformedPeriodStringException
    - DateMalformedStringException

Supported Date and Time Formats

This section describes all the different formats in a BNF-like format, that the DateTimeImmutable, DateTime, date_create_immutable(), date_create(), date_parse(), and strtotime() parser understands. The formats are grouped by section. In most cases formats from different sections, separated by whitespace, comma or dot, can be used in the same date/time string. For each of the supported formats, one or more examples are given, as well as a description for the format. Characters in single quotes in the formats are case-insensitive (`'t'` could be `t` or `T`), characters in double quotes are case-sensitive (`"T"` is only `T`).

To format DateTimeImmutable and DateTime objects, please refer to the documentation of the DateTimeInterface::format() method.

A general set of rules should be taken into account.

1. The parser, allows for each unit (year, month, day, hour, minute, second) the full range of values. For a year that's just 4 digits, for a month that's 0-12, day is 0-31, hour is 0-24, and minute is 0-59.
2. 60 is allowed for seconds, as sometimes date strings with that leapsecond do show up. But PHP implements Unix time where "60" is not a valid second number and hence it overflows.
3. strtotime() returns `false` if any number is outside of the ranges, and DateTimeImmutable::__construct() throws an exception.
4. If a string contains a date, all time elements are reset to 0.
5. All less-significant time elements are reset to 0 if any part of a time is present in the given string.
6. The parser is dumb, and doesn't do any checks to make it faster (and more generic).
7. Besides rules for individual time elements, the parser also understand more specific compound formats, such as parsing Unix timestamps (`@1690388256`) and ISO Weekdates (`2008-W28-3`).
8. There is an additional check if an invalid date is provided:

```php
<?php
$res = date_parse("2015-09-31");
var_dump($res["warnings"]);
```

The above example will output:

```
array(1) {
  [11]=>
  string(27) "The parsed date was invalid"
}
```

9. It is already possible to handle the edge cases, but then DateTimeImmutable::createFromFormat() must be used while supplying the correct format.

```php
<?php
$res = DateTimeImmutable::createFromFormat("Y-m-d", "2015-09-34");
var_dump($res);
```

The above example will output:

```
object(DateTimeImmutable)#1 (3) {
  ["date"]=>
  string(26) "2015-10-04 17:24:43.000000"
  ["timezone_type"]=>
  int(3)
  ["timezone"]=>
  string(13) "Europe/London"
}
```

Time Formats

This page describes the different date/time formats in a BNF-like syntax, that the DateTimeImmutable, DateTime, date_create(), date_create_immutable(), and strtotime() parser understands.

To format DateTimeImmutable and DateTime objects, please refer to the documentation of the DateTimeInterface::format() method.

Used Symbols

Description	Formats	Examples
`frac`	. [0-9]+	".21342", ".85"

Description	Formats	Examples
hh	"0"?[1-9] | "1"[0-2]	"04", "7", "12"
HH	[01][0-9] | "2"[0-4]	"04", "07", "19"
meridian	[AaPp] .? [Mm] .? [\0\t]	"A.m.", "pM", "am."
MM	[0-5][0-9]	"00", "12", "59"
II	[0-5][0-9]	"00", "12", "59"
space	[\t]	
tz	"("? [A-Za-z]{1,6} ")"? | [A-Z][a-z]+([_/][A-Z][a-z]+)+	"CEST", "Europe/Amsterdam", "America/Indiana/Knox"
tzcorrection	"GMT"? [+-] hh ":"? MM?	"+0400", "GMT-07:00", "-07:00"

12 Hour Notation

Description	Format	Examples
Hour only, with meridian	hh space? meridian	"4 am", "5PM"
Hour and minutes, with meridian	hh [.:] MM space? meridian	"4:08 am", "7:19P.M."
Hour, minutes and seconds, with meridian	hh [.:] MM [.:] II space? meridian	"4:08:37 am", "7:19:19P.M."
MS SQL (Hour, minutes, seconds and fraction with meridian)	hh ":" MM ":" II [.:] [0-9]+ meridian	"4:08:39:12313am"

24 Hour Notation

Description	Format	Examples
Hour and minutes	't'? HH [.:] MM	"04:08", "19.19", "T23:43"
Hour and minutes, no colon	't'? HH MM	"0408", "t1919", "T2343"
Hour, minutes and seconds	't'? HH [.:] MM [.:] II	"04.08.37", "t19:19:19"
Hour, minutes and seconds, no colon	't'? HH MM II	"040837", "T191919"
Hour, minutes, seconds and timezone	't'? HH [.:] MM [.:] II space? (tzcorrection | tz)	"040837CEST", "T191919-0700"
Hour, minutes, seconds and fraction	't'? HH [.:] MM [.:] II frac	"04.08.37.81412", "19:19:19.532453"
Time zone information	tz | tzcorrection	"CEST", "Europe/Amsterdam", "+0430", "GMT-06:00"

Date Formats

This page describes the different date formats in a BNF-like syntax, that the DateTimeImmutable, DateTime, date_create(), date_create_immutable(), and strtotime() parser understands.

To format DateTimeImmutable and DateTime objects, please refer to the documentation of the DateTimeInterface::format() method.

Used Symbols

Description	Format	Examples
daysuf	"st" | "nd" | "rd" | "th"	
dd	([0-2]?[0-9] | "3"[01]) daysuf?	"7th", "22nd", "31"
DD	"0" [0-9] | [1-2][0-9] | "3" [01]	"07", "31"
m	'january' | 'february' | 'march' | 'april' | 'may' | 'june' | 'july' | 'august' | 'september' | 'october' | 'november' | 'december' | 'jan' | 'feb' | 'mar' | 'apr' | 'may' | 'jun' | 'jul' | 'aug' | 'sep' | 'sept' | 'oct' | 'nov' | 'dec' | "I" | "II" | "III" | "IV" | "V" | "VI" | "VII" | "VIII" | "IX" | "X" | "XI" | "XII"	
M	'jan' | 'feb' | 'mar' | 'apr' | 'may' | 'jun' | 'jul' | 'aug' | 'sep' | 'sept' | 'oct' | 'nov' | 'dec'	
mm	"0"? [0-9] | "1"[0-2]	"0", "04", "7", "12"
MM	"0" [0-9] | "1"[0-2]	"00", "04", "07", "12"
y	[0-9]{1,4}	"00", "78", "08", "8", "2008"

Description	Format	Examples
yy	[0-9]{2}	"00", "08", "78"
YY	[0-9]{4}	"2000", "2008", "1978"
YYY	[0-9]{5,19}	"81412", "20192"

Localized Notations

Description	Format	Examples
American month and day	mm "/" dd	"5/12", "10/27"
American month, day and year	mm "/" dd "/" y	"12/22/78", "1/17/2006", "1/17/6"
Four digit year, month and day with slashes	YY "/" mm "/" dd	"2008/6/30", "1978/12/22"
Four digit year and month (GNU)	YY "-" mm	"2008-6", "2008-06", "1978-12"
Year, month and day with dashes	y "-" mm "-" dd	"2008-6-30", "78-12-22", "8-6-21"
Day, month and four digit year, with dots, tabs or dashes	dd [.\t-] mm [.-] YY	"30-6-2008", "22.12.1978"
Day, month and two digit year, with dots or tabs	dd [.\t] mm "." yy	"30.6.08", "22\t12.78"
Day, textual month and year	dd ([\t.-])* m ([\t.-])* y	"30-June 2008", "22DEC78", "14 III 1879"
Textual month and four digit year (Day reset to 1)	m ([\t.-])* YY	"June 2008", "DEC1978", "March 1879"
Four digit year and textual month (Day reset to 1)	YY ([\t.-])* m	"2008 June", "1978-XII", "1879.MArCH"
Textual month, day and year	m ([.\t-])* dd [,.stndrh\t]+ y	"July 1st, 2008", "April 17, 1790", "May.9,78"
Textual month and day	m ([.\t-])* dd [,.stndrh\t]*	"July 1st,", "Apr 17", "May.9"
Day and textual month	dd ([.\t-])* m	"1 July", "17 Apr", "9.May"
Month abbreviation, day and year	M "-" DD "-" y	"May-09-78", "Apr-17-1790"
Year, month abbreviation and day	y "-" M "-" DD	"78-Dec-22", "1814-MAY-17"
Year (and just the year)	YY	"1978", "2008"
Year (expanded, 5-19 digits with sign)	[+-] YYY	"-81120", "+20192"
Textual month (and just the month)	m	"March", "jun", "DEC"

ISO8601 Notations

Description	Format	Examples
Eight digit year, month and day	YY MM DD	"15810726", "19780417", "18140517"
Four digit year, month and day with slashes	YY "/" MM "/" DD	"2008/06/30", "1978/12/22"
Two digit year, month and day with dashes	yy "-" MM "-" DD	"08-06-30", "78-12-22"
Four digit year with optional sign, month and day	[+-]? YY "-" MM "-" DD	"-0002-07-26", "+1978-04-17", "1814-05-17"
Five+ digit year with required sign, month and day	[+-] YYY "-" MM "-" DD	"-81120-02-26", "+20192-04-17"

> **Note** :
>
> For the y and yy formats, years below 100 are handled in a special way when the y or yy symbol is used. If the year falls in the range 0 (inclusive) to 69 (inclusive), 2000 is added. If the year falls in the range 70 (inclusive) to 99 (inclusive) then 1900 is added. This means that "00-01-01" is interpreted as "2000-01-01".

> **Note** :
>
> The "Day, month and two digit year, with dots or tabs" format (dd [.\t] mm "." yy) only works for the year values 61 (inclusive) to 99 (inclusive) - outside those years the *time format* "HH [.:] MM [.:] SS" has precedence.

> **Note** :
>
> The "Year (and just the year)" format only reliably works if a time string has already been found. Otherwise, if the four digit year matches HH MM then these two date elements are set instead.

To consistently parse just a year, use DateTimeImmutable::createFromFormat() with the `Y` specifier.

Caution

It is possible to over- and underflow the `dd` and `DD` format. Day 0 means the last day of previous month, whereas overflows count into the next month. This makes "2008-08-00" equivalent to "2008-07-31" and "2008-06-31" equivalent to "2008-07-01" (June only has 30 days).

Note that the day range is restricted to 0-31 as indicated by the regular expression above. Thus "2008-06-32" is not a valid date string, for instance.

It is also possible to underflow the `mm` and `MM` formats with the value 0. A month value of 0 means December of the previous year. As example "2008-00-22" is equivalent to "2007-12-22".

If you combine the previous two facts and underflow both the day and the month, the following happens: "2008-00-00" first gets converted to "2007-12-00" which then gets converted to "2007-11-30". This also happens with the string "0000-00-00", which gets transformed into "-0001-11-30" (the year -1 in the ISO 8601 calendar, which is 2 BC in the proleptic Gregorian calendar).

Compound Formats

This page describes the different compound date/time formats in a BNF-like syntax, that the DateTimeImmutable, DateTime, date_create(), date_create_immutable(), and strtotime() parser understands.

To format DateTimeImmutable and DateTime objects, please refer to the documentation of the DateTimeInterface::format() method.

Used Symbols

Description	Formats	Examples
`daysuf`	"st" | "nd" | "rd" | "th"	
`dd`	([0-2]?[0-9] | "3"[01]) `daysuf`?	"7th", "22nd", "31"
`DD`	"0" [0-9] | [1-2][0-9] | "3" [01]	"02", "12", "31"
`doy`	"00"[1-9] | "0"[1-9][0-9] | [1-2][0-9][0-9] | "3"[0-5][0-9] | "36"[0-6]	"001", "012", "180", "350", "366"
`frac`	. [0-9]+	".21342", ".85"
`hh`	[01]?[0-9] | "2"[0-4]	"04", "9", "19"
`HH`	[01][0-9] | "2"[0-4]	"04", "07", "19"
`meridian`	[AaPp] .? [Mm] .? [\0\t]	"A.m.", "pM", "am."
`ii`	[0-5]?[0-9]	"04", "8", "59"
`II`	[0-5][0-9]	"04", "08", "59"
`M`	'jan' | 'feb' | 'mar' | 'apr' | 'may' | 'jun' | 'jul' | 'aug' | 'sep' | 'sept' | 'oct' | 'nov' | 'dec'	
`mm`	"0"? [0-9] | "1"[0-2]	"0", "04", "7", "12"
`MM`	[0-1][0-9]	"00", "12"
`space`	[\t]	
`ss`	([0-5]?[0-9])|60	"04", "8", "59", "60" (leap second)
`SS`	[0-5][0-9]	"04", "08", "59"
`W`	"0"[1-9] | [1-4][0-9] | "5"[0-3]	"05", "17", "53"
`tzcorrection`	"GMT"? [+-] `hh` ":"? `II`?	"+0400", "GMT-07:00", "-07:00"
`YY`	[0-9]{4}	"2000", "2008", "1978"

Standards Formats

Description	Examples
ATOM	"2022-06-02T16:58:35+00:00"
COOKIE	"Thursday, 02-Jun-2022 16:58:35 UTC"
ISO8601	"2022-06-02T16:58:35+0000"
» RFC 822	"Thu, 02 Jun 22 16:58:35 +0000"
» RFC 850	"Thursday, 02-Jun-22 16:58:35 UTC"

Description	Examples
» RFC 1036	"Thu, 02 Jun 22 16:58:35 +0000"
» RFC 1123	"Thu, 02 Jun 2022 16:58:35 +0000"
» RFC 2822	"Thu, 02 Jun 2022 16:58:35 +0000"
» RFC 3339	"2022-06-02T16:58:35+00:00"
» RFC 3339 Extended	"2022-06-02T16:58:35.698+00:00"
» RFC 7231	"Thu, 02 Jun 2022 16:58:35 GMT"
RSS	"Thu, 02 Jun 2022 16:58:35 +0000"
W3C	"2022-06-02T16:58:35+00:00"

Localized Notations

Description	Format	Examples
Common Log Format	`dd "/" M "/" YY ":" HH ":" II ":" SS space tzcorrection`	"10/Oct/2000:13:55:36 -0700"
EXIF	`YY ":" MM ":" DD " " HH ":" II ":" SS`	"2008:08:07 18:11:31"
ISO year with ISO week	`YY "-"? "W" W`	"2008W27", "2008-W28"
ISO year with ISO week and day	`YY "-"? "W" W "-"? [0-7]`	"2008W273", "2008-W28-3"
MySQL	`YY "-" MM "-" DD " " HH ":" II ":" SS`	"2008-08-07 18:11:31"
PostgreSQL: Year with day-of-year	`YY "."? doy`	"2008.197", "2008197"
SOAP	`YY "-" MM "-" DD "T" HH ":" II ":" SS frac tzcorrection?`	"2008-07-01T22:35:17.02", "2008-07-01T22:35:17.03+08:00"
Unix Timestamp	"@" "-"? [0-9]+	"@1215282385"
Unix Timestamp with microseconds	"@" "-"? [0-9]+ "." [0-9]{0,6}	"@1607974647.503686"
XMLRPC	`YY MM DD "T" hh ":" II ":" SS`	"20080701T22:38:07", "20080701T9:38:07"
XMLRPC (Compact)	`YY MM DD 't' hh II SS`	"20080701t223807", "20080701T093807"
WDDX	`YY "-" mm "-" dd "T" hh ":" ii ":" ss`	"2008-7-1T9:3:37"

> **Note** :
>
> The "W" in the "ISO year with ISO week" and "ISO year with ISO week and day" formats is case-sensitive, you can only use the upper case "W".
>
> The "T" in the SOAP, XMLRPC and WDDX formats is case-sensitive, you can only use the upper case "T".
>
> The "Unix Timestamp" format sets the timezone to UTC.

Relative Formats

This page describes the different relative date/time formats in a BNF-like syntax, that the DateTimeImmutable, DateTime, date_create(), date_create_immutable(), and strtotime() parser understands.

To format DateTimeImmutable and DateTime objects, please refer to the documentation of the DateTimeInterface::format() method.

Used Symbols

Description	Format
`dayname`	'sunday' | 'monday' | 'tuesday' | 'wednesday' | 'thursday' | 'friday' | 'saturday' | 'sun' | 'mon' | 'tue' | 'wed' | 'thu' | 'fri' | 'sat'
`daytext`	'weekday' | 'weekdays'
`number`	[+-]?[0-9]+
`ordinal`	'first' | 'second' | 'third' | 'fourth' | 'fifth' | 'sixth' | 'seventh' | 'eighth' | 'ninth' | 'tenth' | 'eleventh' | 'twelfth' | 'next' | 'last' | 'previous' | 'this'
`reltext`	'next' | 'last' | 'previous' | 'this'

Description	Format
`space`	[\t]+
`unit`	'ms' | 'µs' | (('msec' | 'millisecond' | 'µsec' | 'microsecond' | 'usec' | 'sec' | 'second' | 'min' | 'minute' | 'hour' | 'day' | 'fortnight' | 'forthnight' | 'month' | 'year') 's'?) | 'weeks' | `daytext`

Day-based Notations

Format	Description	Examples
'yesterday'	Midnight of yesterday	"yesterday 14:00"
'midnight'	The time is set to 00:00:00	
'today'	The time is set to 00:00:00	
'now'	Now - this is simply ignored	
'noon'	The time is set to 12:00:00	"yesterday noon"
'tomorrow'	Midnight of tomorrow	
'back of' `hour`	15 minutes past the specified hour	"back of 7pm", "back of 15"
'front of' `hour`	15 minutes before the specified hour	"front of 5am", "front of 23"
'first day of'	Sets the day of the first of the current month. This phrase is usually best used together with a month name following it as it only effects the current month	"first day of January 2008"
'last day of'	Sets the day to the last day of the current month. This phrase is usually best used together with a month name following it as it only effects the current month	"last day of next month"
`ordinal space dayname space` 'of'	Calculates the `x`-th week day of the current month.	"first sat of July 2008"
'last' `space dayname space` 'of'	Calculates the *last* week day of the current month.	"last sat of July 2008"
`number space?` (`unit` | 'week')	Handles relative time items where the value is a number.	"+5 weeks", "12 day", "-7 weekdays"
(`ordinal` | `reltext`) `space unit`	Handles relative time items where the value is text. `last` and `previous` are equivalent to `-1`, `this` to nothing, and `next` to `+1`.	"fifth day", "second month", "last day", "previous year"
'ago'	Negates all the values of previously found relative time items.	"2 days ago", "8 days ago 14:00", "2 months 5 days ago", "2 months ago 5 days", "2 days ago"
`dayname`	Moves to the next day of this name. (See note)	"Monday"
`reltext space` 'week'	Handles the special format "weekday + last/this/next week".	"Monday next week"

> **Note** :
>
> Relative statements are always processed *after* non-relative statements. This makes "+1 week july 2008" and "july 2008 +1 week" equivalent.
>
> Exceptions to this rule are: "yesterday", "midnight", "today", "noon" and "tomorrow". Note that "tomorrow 11:00" and "11:00 tomorrow" are different. Considering today's date of "July 23rd, 2008" the first one produces "2008-07-24 11:00" where as the second one produces "2008-07-24 00:00". The reason for this is that those five statements directly influence the current time.
>
> Keywords such as "first day of" depend on the context in which the relative format string is used. If used with a static method or function, the referent is the current system timestamp. However, if used in DateTime::modify() or DateTimeImmutable::modify(), the referent is the object on which the `modify()` method is called.

> **Note** :
>
> Observe the following remarks when the current day-of-week is the same as the day-of-week used in the date/time string. The current day-of-week could have been (re-)calculated by non-relative parts of the date/time string however.
>
> 1. "`dayname`" does *not* advance to another day. (Example: "Wed July 23rd, 2008" means "2008-07-23").

2. "`number dayname`" does *not* advance to another day. (Example: "1 wednesday july 23rd, 2008" means "2008-07-23").
3. "`number` week `dayname`" will first add the number of weeks, but does *not* advance to another day. In this case "`number` week" and "`dayname`" are two distinct blocks. (Example: "+1 week wednesday july 23rd, 2008" means "2008-07-30").
4. "`ordinal dayname`" *does* advance to another day. (Example "first wednesday july 23rd, 2008" means "2008-07-30").
5. "`number` week `ordinal dayname`" will first add the number of weeks, and then *advances* to another day. In this case "`number` week" and "`ordinal dayname`" are two distinct blocks. (Example: "+1 week first wednesday july 23rd, 2008" means "2008-08-06").
6. "`ordinal dayname` 'of' " does *not* advance to another day. (Example: "first wednesday of july 23rd, 2008" means "2008-07-02" because the specific phrase with 'of' resets the day-of-month to '1' and the '23rd' is ignored here).

Also observe that the "of" in "`ordinal space dayname space` 'of' " and "'last' `space dayname space` 'of' " does something special.

1. It sets the day-of-month to 1.
2. "`ordinal dayname` 'of' " does *not* advance to another day. (Example: "first tuesday of july 2008" means "2008-07-01").
3. "`ordinal dayname` " *does* advance to another day. (Example: "first tuesday july 2008" means "2008-07-08", see also point 4 in the list above).
4. "'last' `dayname` 'of' " takes the last `dayname` of the current month. (Example: "last wed of july 2008" means "2008-07-30")
5. "'last' `dayname`" takes the last `dayname` from the current day. (Example: "last wed july 2008" means "2008-06-25"; "july 2008" first sets the current date to "2008-07-01" and then "last wed" moves to the previous Wednesday which is "2008-06-25").

> **Note** :
>
> Relative month values are calculated based on the length of months that they pass through. An example would be "+2 month 2011-11-30", which would produce "2012-01-30". This is due to November being 30 days in length, and December being 31 days in length, producing a total of 61 days.

> **Note** :
>
> `number` is an *integer* number; if a decimal number is given, the dot (or comma) is likely interpreted as delimiter. For instance, `'+1.5 hours'` is parsed like `'+1 5 hours'`, not as `'+1 hour +30 minutes'`.

Changelog

Version	Description
8.4.0	`number` now again accepts a plus sign followed by a minus sign, e.g. `+-2`, and other combinations of multiple signs.
8.2.0	`number` no longer accepts a plus sign followed by a minus sign, e.g. `+-2`.
7.0.8	Weeks always start on monday. Formerly, sunday would also be considered to start a week.

List of Supported Timezones

Table of Contents

Here you'll find the complete list of timezones supported by PHP, which are meant to be used with e.g. date_default_timezone_set().

> **Caution**
>
> The behavior of timezones not listed here is undefined.

> **Note** :
>
> The latest version of the timezone database can be installed via PECL's » timezonedb.

> **Note** :

> This list is based upon the timezone database version 2026.4.

Africa

Africa			
Africa/Abidjan	Africa/Accra	Africa/Addis_Ababa	Africa/Algiers
Africa/Asmara	Africa/Bamako	Africa/Bangui	Africa/Banjul
Africa/Bissau	Africa/Blantyre	Africa/Brazzaville	Africa/Bujumbura
Africa/Cairo	Africa/Casablanca	Africa/Ceuta	Africa/Conakry
Africa/Dakar	Africa/Dar_es_Salaam	Africa/Djibouti	Africa/Douala
Africa/El_Aaiun	Africa/Freetown	Africa/Gaborone	Africa/Harare
Africa/Johannesburg	Africa/Juba	Africa/Kampala	Africa/Khartoum
Africa/Kigali	Africa/Kinshasa	Africa/Lagos	Africa/Libreville
Africa/Lome	Africa/Luanda	Africa/Lubumbashi	Africa/Lusaka
Africa/Malabo	Africa/Maputo	Africa/Maseru	Africa/Mbabane
Africa/Mogadishu	Africa/Monrovia	Africa/Nairobi	Africa/Ndjamena
Africa/Niamey	Africa/Nouakchott	Africa/Ouagadougou	Africa/Porto-Novo
Africa/Sao_Tome	Africa/Tripoli	Africa/Tunis	Africa/Windhoek

America

America			
America/Adak	America/Anchorage	America/Anguilla	America/Antigua
America/Araguaina	America/Argentina/Buenos_Aires	America/Argentina/Catamarca	America/Argentina/Cordoba
America/Argentina/Jujuy	America/Argentina/La_Rioja	America/Argentina/Mendoza	America/Argentina/Rio_Gallegos
America/Argentina/Salta	America/Argentina/San_Juan	America/Argentina/San_Luis	America/Argentina/Tucuman
America/Argentina/Ushuaia	America/Aruba	America/Asuncion	America/Atikokan
America/Bahia	America/Bahia_Banderas	America/Barbados	America/Belem
America/Belize	America/Blanc-Sablon	America/Boa_Vista	America/Bogota
America/Boise	America/Cambridge_Bay	America/Campo_Grande	America/Cancun
America/Caracas	America/Cayenne	America/Cayman	America/Chicago
America/Chihuahua	America/Ciudad_Juarez	America/Costa_Rica	America/Coyhaique
America/Creston	America/Cuiaba	America/Curacao	America/Danmarkshavn
America/Dawson	America/Dawson_Creek	America/Denver	America/Detroit
America/Dominica	America/Edmonton	America/Eirunepe	America/El_Salvador
America/Fort_Nelson	America/Fortaleza	America/Glace_Bay	America/Goose_Bay
America/Grand_Turk	America/Grenada	America/Guadeloupe	America/Guatemala
America/Guayaquil	America/Guyana	America/Halifax	America/Havana
America/Hermosillo	America/Indiana/Indianapolis	America/Indiana/Knox	America/Indiana/Marengo
America/Indiana/Petersburg	America/Indiana/Tell_City	America/Indiana/Vevay	America/Indiana/Vincennes
America/Indiana/Winamac	America/Inuvik	America/Iqaluit	America/Jamaica
America/Juneau	America/Kentucky/Louisville	America/Kentucky/Monticello	America/Kralendijk
America/La_Paz	America/Lima	America/Los_Angeles	America/Lower_Princes
America/Maceio	America/Managua	America/Manaus	America/Marigot
America/Martinique	America/Matamoros	America/Mazatlan	America/Menominee
America/Merida	America/Metlakatla	America/Mexico_City	America/Miquelon
America/Moncton	America/Monterrey	America/Montevideo	America/Montserrat
America/Nassau	America/New_York	America/Nome	America/Noronha

America/North_Dakota/Beulah	America/North_Dakota/Center	America/North_Dakota/New_Salem	America/Nuuk
America/Ojinaga	America/Panama	America/Paramaribo	America/Phoenix
America/Port-au-Prince	America/Port_of_Spain	America/Porto_Velho	America/Puerto_Rico
America/Punta_Arenas	America/Rankin_Inlet	America/Recife	America/Regina
America/Resolute	America/Rio_Branco	America/Santarem	America/Santiago
America/Santo_Domingo	America/Sao_Paulo	America/Scoresbysund	America/Sitka
America/St_Barthelemy	America/St_Johns	America/St_Kitts	America/St_Lucia
America/St_Thomas	America/St_Vincent	America/Swift_Current	America/Tegucigalpa
America/Thule	America/Tijuana	America/Toronto	America/Tortola
America/Vancouver	America/Whitehorse	America/Winnipeg	America/Yakutat

Antarctica

Antarctica			
Antarctica/Casey	Antarctica/Davis	Antarctica/DumontDUrville	Antarctica/Macquarie
Antarctica/Mawson	Antarctica/McMurdo	Antarctica/Palmer	Antarctica/Rothera
Antarctica/Syowa	Antarctica/Troll	Antarctica/Vostok	

Arctic

Arctic
Arctic/Longyearbyen

Asia

Asia			
Asia/Aden	Asia/Almaty	Asia/Amman	Asia/Anadyr
Asia/Aqtau	Asia/Aqtobe	Asia/Ashgabat	Asia/Atyrau
Asia/Baghdad	Asia/Bahrain	Asia/Baku	Asia/Bangkok
Asia/Barnaul	Asia/Beirut	Asia/Bishkek	Asia/Brunei
Asia/Chita	Asia/Colombo	Asia/Damascus	Asia/Dhaka
Asia/Dili	Asia/Dubai	Asia/Dushanbe	Asia/Famagusta
Asia/Gaza	Asia/Hebron	Asia/Ho_Chi_Minh	Asia/Hong_Kong
Asia/Hovd	Asia/Irkutsk	Asia/Jakarta	Asia/Jayapura
Asia/Jerusalem	Asia/Kabul	Asia/Kamchatka	Asia/Karachi
Asia/Kathmandu	Asia/Khandyga	Asia/Kolkata	Asia/Krasnoyarsk
Asia/Kuala_Lumpur	Asia/Kuching	Asia/Kuwait	Asia/Macau
Asia/Magadan	Asia/Makassar	Asia/Manila	Asia/Muscat
Asia/Nicosia	Asia/Novokuznetsk	Asia/Novosibirsk	Asia/Omsk
Asia/Oral	Asia/Phnom_Penh	Asia/Pontianak	Asia/Pyongyang
Asia/Qatar	Asia/Qostanay	Asia/Qyzylorda	Asia/Riyadh
Asia/Sakhalin	Asia/Samarkand	Asia/Seoul	Asia/Shanghai
Asia/Singapore	Asia/Srednekolymsk	Asia/Taipei	Asia/Tashkent
Asia/Tbilisi	Asia/Tehran	Asia/Thimphu	Asia/Tokyo
Asia/Tomsk	Asia/Ulaanbaatar	Asia/Urumqi	Asia/Ust-Nera
Asia/Vientiane	Asia/Vladivostok	Asia/Yakutsk	Asia/Yangon
Asia/Yekaterinburg	Asia/Yerevan		

Atlantic

Atlantic

Atlantic/Azores	Atlantic/Bermuda	Atlantic/Canary	Atlantic/Cape_Verde
Atlantic/Faroe	Atlantic/Madeira	Atlantic/Reykjavik	Atlantic/South_Georgia
Atlantic/St_Helena	Atlantic/Stanley		

Australia

Australia

Australia/Adelaide	Australia/Brisbane	Australia/Broken_Hill	Australia/Darwin
Australia/Eucla	Australia/Hobart	Australia/Lindeman	Australia/Lord_Howe
Australia/Melbourne	Australia/Perth	Australia/Sydney	

Europe

Europe

Europe/Amsterdam	Europe/Andorra	Europe/Astrakhan	Europe/Athens
Europe/Belgrade	Europe/Berlin	Europe/Bratislava	Europe/Brussels
Europe/Bucharest	Europe/Budapest	Europe/Busingen	Europe/Chisinau
Europe/Copenhagen	Europe/Dublin	Europe/Gibraltar	Europe/Guernsey
Europe/Helsinki	Europe/Isle_of_Man	Europe/Istanbul	Europe/Jersey
Europe/Kaliningrad	Europe/Kirov	Europe/Kyiv	Europe/Lisbon
Europe/Ljubljana	Europe/London	Europe/Luxembourg	Europe/Madrid
Europe/Malta	Europe/Mariehamn	Europe/Minsk	Europe/Monaco
Europe/Moscow	Europe/Oslo	Europe/Paris	Europe/Podgorica
Europe/Prague	Europe/Riga	Europe/Rome	Europe/Samara
Europe/San_Marino	Europe/Sarajevo	Europe/Saratov	Europe/Simferopol
Europe/Skopje	Europe/Sofia	Europe/Stockholm	Europe/Tallinn
Europe/Tirane	Europe/Ulyanovsk	Europe/Vaduz	Europe/Vatican
Europe/Vienna	Europe/Vilnius	Europe/Volgograd	Europe/Warsaw
Europe/Zagreb	Europe/Zurich		

Indian

Indian

Indian/Antananarivo	Indian/Chagos	Indian/Christmas	Indian/Cocos
Indian/Comoro	Indian/Kerguelen	Indian/Mahe	Indian/Maldives
Indian/Mauritius	Indian/Mayotte	Indian/Reunion	

Pacific

Pacific

Pacific/Apia	Pacific/Auckland	Pacific/Bougainville	Pacific/Chatham
Pacific/Chuuk	Pacific/Easter	Pacific/Efate	Pacific/Fakaofo
Pacific/Fiji	Pacific/Funafuti	Pacific/Galapagos	Pacific/Gambier
Pacific/Guadalcanal	Pacific/Guam	Pacific/Honolulu	Pacific/Kanton
Pacific/Kiritimati	Pacific/Kosrae	Pacific/Kwajalein	Pacific/Majuro
Pacific/Marquesas	Pacific/Midway	Pacific/Nauru	Pacific/Niue
Pacific/Norfolk	Pacific/Noumea	Pacific/Pago_Pago	Pacific/Palau
Pacific/Pitcairn	Pacific/Pohnpei	Pacific/Port_Moresby	Pacific/Rarotonga
Pacific/Saipan	Pacific/Tahiti	Pacific/Tarawa	Pacific/Tongatapu
Pacific/Wake	Pacific/Wallis		

Others

> **Warning**
>
> Please do not use any of the timezones listed here (besides UTC), they only exist for backward compatible reasons, and may expose erroneous behavior. Furthermore, these timezones may be removed from the IANA timezone database at any time.

> **Warning**
>
> If you disregard the above warning, please also note that the IANA timezone database that provides PHP's timezone support uses POSIX style signs, which results in the `Etc/GMT+n` and `Etc/GMT-n` time zones being reversed from common usage.
>
> For example, the time zone 8 hours ahead of GMT that is used in China and Western Australia (among other places) is actually `Etc/GMT-8` in this database, not `Etc/GMT+8` as you would normally expect.
>
> Once again, it is strongly recommended that you use the correct time zone for your location, such as `Asia/Shanghai` or `Australia/Perth` for the above examples.

Others

Africa/Asmera	Africa/Timbuktu	America/Argentina/ComodRivadavia	America/Atka
America/Buenos_Aires	America/Catamarca	America/Coral_Harbour	America/Cordoba
America/Ensenada	America/Fort_Wayne	America/Godthab	America/Indianapolis
America/Jujuy	America/Knox_IN	America/Louisville	America/Mendoza
America/Montreal	America/Nipigon	America/Pangnirtung	America/Porto_Acre
America/Rainy_River	America/Rosario	America/Santa_Isabel	America/Shiprock
America/Thunder_Bay	America/Virgin	America/Yellowknife	Antarctica/South_Pole
Asia/Ashkhabad	Asia/Calcutta	Asia/Choibalsan	Asia/Chongqing
Asia/Chungking	Asia/Dacca	Asia/Harbin	Asia/Istanbul
Asia/Kashgar	Asia/Katmandu	Asia/Macao	Asia/Rangoon
Asia/Saigon	Asia/Tel_Aviv	Asia/Thimbu	Asia/Ujung_Pandang
Asia/Ulan_Bator	Atlantic/Faeroe	Atlantic/Jan_Mayen	Australia/ACT
Australia/Canberra	Australia/Currie	Australia/LHI	Australia/North
Australia/NSW	Australia/Queensland	Australia/South	Australia/Tasmania
Australia/Victoria	Australia/West	Australia/Yancowinna	Brazil/Acre
Brazil/DeNoronha	Brazil/East	Brazil/West	Canada/Atlantic
Canada/Central	Canada/Eastern	Canada/Mountain	Canada/Newfoundland
Canada/Pacific	Canada/Saskatchewan	Canada/Yukon	CET
Chile/Continental	Chile/EasterIsland	CST6CDT	Cuba
EET	Egypt	Eire	EST
EST5EDT	Etc/GMT	Etc/GMT+0	Etc/GMT+1
Etc/GMT+10	Etc/GMT+11	Etc/GMT+12	Etc/GMT+2
Etc/GMT+3	Etc/GMT+4	Etc/GMT+5	Etc/GMT+6
Etc/GMT+7	Etc/GMT+8	Etc/GMT+9	Etc/GMT-0
Etc/GMT-1	Etc/GMT-10	Etc/GMT-11	Etc/GMT-12
Etc/GMT-13	Etc/GMT-14	Etc/GMT-2	Etc/GMT-3
Etc/GMT-4	Etc/GMT-5	Etc/GMT-6	Etc/GMT-7
Etc/GMT-8	Etc/GMT-9	Etc/GMT0	Etc/Greenwich
Etc/UCT	Etc/Universal	Etc/UTC	Etc/Zulu
Europe/Belfast	Europe/Kiev	Europe/Nicosia	Europe/Tiraspol
Europe/Uzhgorod	Europe/Zaporozhye	Factory	GB
GB-Eire	GMT	GMT+0	GMT-0
GMT0	Greenwich	Hongkong	HST

Iceland	Iran	Israel	Jamaica
Japan	Kwajalein	Libya	MET
Mexico/BajaNorte	Mexico/BajaSur	Mexico/General	MST
MST7MDT	Navajo	NZ	NZ-CHAT
Pacific/Enderbury	Pacific/Johnston	Pacific/Ponape	Pacific/Samoa
Pacific/Truk	Pacific/Yap	Poland	Portugal
PRC	PST8PDT	ROC	ROK
Singapore	Turkey	UCT	Universal
US/Alaska	US/Aleutian	US/Arizona	US/Central
US/East-Indiana	US/Eastern	US/Hawaii	US/Indiana-Starke
US/Michigan	US/Mountain	US/Pacific	US/Samoa
UTC	W-SU	WET	Zulu

DateError

(PHP 8 >= 8.3.0)

Introduction

Thrown when the timezone database is not found, or contains invalid data.

This error should never occur, and is not dependent on code. There are two child-exceptions (DateObjectError and DateRangeError) which are thrown depending on programmer error or range related issues.

Class synopsis

class **DateError** extends Error {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

See Also

- DateObjectError
- DateRangeError

DateObjectError

(PHP 8 >= 8.3.0)

Introduction

Thrown when one of the Date/Time classes has not been correctly initialised.

Because Date/Time classes are not final, these classes can be inherit. When the parent constructor is not called, this error is thrown. This is always a programming error.

Class synopsis

```
class DateObjectError extends DateError {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

See Also

- DateError
- DateRangeError

DateRangeError

(PHP 8 >= 8.3.0)

Introduction

Thrown by DateTime::getTimestamp(), DateTimeImmutable::getTimestamp(), and date_timestamp_get(), on 32-bit platforms if the date object represents a date outside of the 32-bit signed range.

Class synopsis

```
class DateRangeError extends DateError {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

See Also

- DateError
- DateObjectError

DateException

(PHP 8 >= 8.3.0)

Introduction

Parent class of Date/Time exceptions, for issues that come to light due to user input, or free form text arguments that need to be parsed.

The following child exceptions are thrown by the extension:

- DateInvalidOperationException
- DateInvalidTimezoneException
- DateMalformedIntervalStringException
- DateMalformedPeriodStringException
- DateMalformedStringException

Class synopsis

```
class DateException extends Exception {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

DateInvalidOperationException

(PHP 8 >= 8.3.0)

Introduction

Thrown by DateTimeImmutable::sub() and DateTime::sub() when an unsupported operation is attempted.

An example of such an unsupported operation is using a DateInterval object representing relative time specifications such as `next weekday`, as no logical reversed statement can be constructed.

Class synopsis

```
class DateInvalidOperationException extends DateException {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
```

protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateInvalidTimeZoneException

(PHP 8 >= 8.3.0)

Introduction

Thrown when an incorrect value is passed to DateTimeZone::__construct().

Class synopsis

class **DateInvalidTimeZoneException** extends DateException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateMalformedIntervalStringException

(PHP 8 >= 8.3.0)

Introduction

Thrown when an invalid `duration` argument is passed to DateInterval::__construct().

Class synopsis

class **DateMalformedIntervalStringException** extends DateException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;

protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateMalformedPeriodStringException

(PHP 8 >= 8.3.0)

Introduction

Thrown when an invalid `isostr` argument is passed to DatePeriod::__construct().

Class synopsis

class **DateMalformedPeriodStringException** extends DateException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

DateMalformedStringException

(PHP 8 >= 8.3.0)

Introduction

Thrown when an invalid Date/Time string is detected.

This can be as value to DateTimeImmutable::__construct(), DateTimeImmutable::modify(), DateTime::__construct(), or DateTime::modify().

Class synopsis

class **DateMalformedStringException** extends DateException {
/* Inherited properties */
protected string *$message* = "";

```
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

JavaScript Object Notation

Introduction

This extension implements the » JavaScript Object Notation (JSON) data-interchange format. PHP comes with a parser that is specifically written for PHP and licensed under the PHP license.

> **Note** :
>
> PHP implements a superset of JSON as specified in the original » RFC 7159.

Installing/Configuring

Table of Contents

Installation

The JSON extension is a core PHP extension, so it is always enabled.

Prior to PHP 8.0.0, the JSON extension was bundled and compiled into PHP by default, but could be explicitly disabled using **--disable-json**.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

The following constants indicate the type of error returned by json_last_error() or stored as the *code* of a JsonException.

`JSON_ERROR_NONE` (int)

No error has occurred.

`JSON_ERROR_DEPTH` (int)

The maximum stack depth has been exceeded.

`JSON_ERROR_STATE_MISMATCH` (int)

Occurs with underflow or with the modes mismatch.

`JSON_ERROR_CTRL_CHAR` (int)

Control character error, possibly incorrectly encoded.

`JSON_ERROR_SYNTAX` (int)

Syntax error.

`JSON_ERROR_UTF8` (int)

Malformed UTF-8 characters, possibly incorrectly encoded.

`JSON_ERROR_RECURSION` (int)

The object or array passed to json_encode() include recursive references and cannot be encoded. If the `JSON_PARTIAL_OUTPUT_ON_ERROR` option was given, `null` will be encoded in the place of the recursive reference.

`JSON_ERROR_INF_OR_NAN` (int)

The value passed to json_encode() includes either `NAN` or `INF`. If the `JSON_PARTIAL_OUTPUT_ON_ERROR` option was given, `0` will be encoded in the place of these special numbers.

`JSON_ERROR_UNSUPPORTED_TYPE` (int)

A value of an unsupported type was given to json_encode(), such as a resource. If the JSON_PARTIAL_OUTPUT_ON_ERROR option was given, null will be encoded in the place of the unsupported value.

JSON_ERROR_INVALID_PROPERTY_NAME (int)

A key starting with \u0000 character was in the string passed to json_decode() when decoding a JSON object into a PHP object.

JSON_ERROR_UTF16 (int)

Single unpaired UTF-16 surrogate in unicode escape contained in the JSON string passed to json_decode().

JSON_ERROR_NON_BACKED_ENUM (int)

The value passed to json_encode() includes a non-backed enum which cannot be serialized. Available as of PHP 8.1.0.

The following constants can be combined to form options for json_decode().

JSON_BIGINT_AS_STRING (int)

Decodes large integers as their original string value.

JSON_OBJECT_AS_ARRAY (int)

Decodes JSON objects as PHP array. This option can be added automatically by calling json_decode() with the second parameter equal to true.

The following constants can be combined to form options for json_encode().

JSON_HEX_TAG (int)

All < and > are converted to \u003C and \u003E.

JSON_HEX_AMP (int)

All & are converted to \u0026.

JSON_HEX_APOS (int)

All ' are converted to \u0027.

JSON_HEX_QUOT (int)

All " are converted to \u0022.

JSON_FORCE_OBJECT (int)

Outputs an object rather than an array when a non-associative array is used. Especially useful when the recipient of the output is expecting an object and the array is empty.

JSON_NUMERIC_CHECK (int)

Encodes numeric strings as numbers.

JSON_PRETTY_PRINT (int)

Use whitespace in returned data to format it.

JSON_UNESCAPED_SLASHES (int)

Don't escape /.

JSON_UNESCAPED_UNICODE (int)

Encode multibyte Unicode characters literally (default is to escape as \uXXXX).

JSON_PARTIAL_OUTPUT_ON_ERROR (int)

Substitute some unencodable values instead of failing.

JSON_PRESERVE_ZERO_FRACTION (int)

Ensures that float values are always encoded as a float value.

`JSON_UNESCAPED_LINE_TERMINATORS` (int)

The line terminators are kept unescaped when `JSON_UNESCAPED_UNICODE` is supplied. It uses the same behaviour as it was before PHP 7.1 without this constant. Available as of PHP 7.1.0.

The following constants can be combined to form options for json_decode() and json_encode().

`JSON_INVALID_UTF8_IGNORE` (int)

Ignore invalid UTF-8 characters. Available as of PHP 7.2.0.

`JSON_INVALID_UTF8_SUBSTITUTE` (int)

Convert invalid UTF-8 characters to \0xfffd (Unicode Character 'REPLACEMENT CHARACTER') Available as of PHP 7.2.0.

`JSON_THROW_ON_ERROR` (int)

Throws JsonException if an error occurs instead of setting the global error state that is retrieved with json_last_error() and json_last_error_msg(). `JSON_PARTIAL_OUTPUT_ON_ERROR` takes precedence over `JSON_THROW_ON_ERROR`. Available as of PHP 7.3.0.

JsonException

(PHP 7 >= 7.3.0, PHP 8)

Introduction

Exception thrown if `JSON_THROW_ON_ERROR` option is set for json_encode() or json_decode(). *code* contains the error type, for possible values see json_last_error().

Class synopsis

```
class JsonException extends Exception {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

JsonSerializable

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introduction

Objects implementing **JsonSerializable** can customize their JSON representation when encoded with json_encode().

Interface synopsis

```
interface JsonSerializable {
/* Methods */
public function jsonSerialize(): mixed
}
```

Table of Contents

JsonSerializable::jsonSerialize

JsonSerializable::jsonSerialize

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

JsonSerializable::jsonSerialize — Specify data which should be serialized to JSON

Description

public function **JsonSerializable::jsonSerialize**(): mixed

Serializes the object to a value that can be serialized natively by json_encode().

Parameters

This function has no parameters.

Return Values

Returns data which can be serialized by json_encode(), which is a value of any type other than a resource.

Examples

Example #1 JsonSerializable::jsonSerialize() example returning an array

```php
<?php
class ArrayValue implements JsonSerializable {
    private $array;
    public function __construct(array $array) {
        $this->array = $array;
    }

    public function jsonSerialize(): mixed {
        return $this->array;
    }
}

$array = [1, 2, 3];
echo json_encode(new ArrayValue($array), JSON_PRETTY_PRINT);
?>
```

The above example will output:

```
[
    1,
    2,
    3
]
```

Example #2 JsonSerializable::jsonSerialize() example returning an associative array

```php
<?php
class ArrayValue implements JsonSerializable {
    private $array;
    public function __construct(array $array) {
        $this->array = $array;
    }

    public function jsonSerialize() {
        return $this->array;
    }
}
```

```php
$array = ['foo' => 'bar', 'quux' => 'baz'];
echo json_encode(new ArrayValue($array), JSON_PRETTY_PRINT);
?>
```

The above example will output:

```
{
    "foo": "bar",
    "quux": "baz"
}
```

Example #3 JsonSerializable::jsonSerialize() example returning an int

```php
<?php
class IntegerValue implements JsonSerializable {
    private $number;
    public function __construct($number) {
        $this->number = (int) $number;
    }

    public function jsonSerialize() {
        return $this->number;
    }
}

echo json_encode(new IntegerValue(1), JSON_PRETTY_PRINT);
?>
```

The above example will output:

```
1
```

Example #4 JsonSerializable::jsonSerialize() example returning a string

```php
<?php
class StringValue implements JsonSerializable {
    private $string;
    public function __construct($string) {
        $this->string = (string) $string;
    }

    public function jsonSerialize() {
        return $this->string;
    }
}

echo json_encode(new StringValue('Hello!'), JSON_PRETTY_PRINT);
?>
```

The above example will output:

```
"Hello!"
```

JSON Functions

Table of Contents

json_decode

json_decode

(PHP 5 >= 5.2.0, PHP 7, PHP 8, PECL json >= 1.2.0)

json_decode — Decodes a JSON string

Description

```
function json_decode(
    string $json,
    ?bool $associative = null,
    int $depth = 512,
    int $flags = 0
): mixed
```

Takes a JSON encoded string and converts it into a PHP value.

Parameters

`json`

The `json` string being decoded.

This function only works with UTF-8 encoded strings.

> **Note** :
>
> PHP implements a superset of JSON as specified in the original » RFC 7159.

`associative`

When **`true`**, JSON objects will be returned as associative arrays; when **`false`**, JSON objects will be returned as objects. When **`null`**, JSON objects will be returned as associative arrays or objects depending on whether **`JSON_OBJECT_AS_ARRAY`** is set in the `flags`.

`depth`

Maximum nesting depth of the structure being decoded. The value must be greater than `0`, and less than or equal to `2147483647`.

`flags`

Bitmask of **`JSON_BIGINT_AS_STRING`**, **`JSON_INVALID_UTF8_IGNORE`**, **`JSON_INVALID_UTF8_SUBSTITUTE`**, **`JSON_OBJECT_AS_ARRAY`**, **`JSON_THROW_ON_ERROR`**. The behaviour of these constants is described on the JSON constants page.

Return Values

Returns the value encoded in `json` as an appropriate PHP type. Unquoted values `true`, `false` and `null` are returned as **`true`**, **`false`** and **`null`** respectively. **`null`** is returned if the `json` cannot be decoded or if the encoded data is deeper than the nesting limit.

Errors/Exceptions

If `depth` is outside the allowed range, a ValueError is thrown as of PHP 8.0.0, while previously, an error of level **`E_WARNING`** was raised.

Changelog

Version	Description
7.3.0	**`JSON_THROW_ON_ERROR`** `flags` was added.
7.2.0	`associative` is nullable now.

Version	Description
7.2.0	`JSON_INVALID_UTF8_IGNORE`, and `JSON_INVALID_UTF8_SUBSTITUTE` `flags` were added.
7.1.0	An empty JSON key ("") can be encoded to the empty object property instead of using a key with value `_empty_`.

Examples

Example #1 **json_decode()** examples

```php
<?php
$json = '{"a":1,"b":2,"c":3,"d":4,"e":5}';

var_dump(json_decode($json));
var_dump(json_decode($json, true));

?>
```

The above example will output:

```
object(stdClass)#1 (5) {
    ["a"] => int(1)
    ["b"] => int(2)
    ["c"] => int(3)
    ["d"] => int(4)
    ["e"] => int(5)
}

array(5) {
    ["a"] => int(1)
    ["b"] => int(2)
    ["c"] => int(3)
    ["d"] => int(4)
    ["e"] => int(5)
}
```

Example #2 Accessing invalid object properties

Accessing elements within an object that contain characters not permitted under PHP's naming convention (e.g. the hyphen) can be accomplished by encapsulating the element name within braces and the apostrophe.

```php
<?php

$json = '{"foo-bar": 12345}';

$obj = json_decode($json);
print $obj->{'foo-bar'}; // 12345

?>
```

Example #3 common mistakes using **json_decode()**

```php
<?php

// the following strings are valid JavaScript but not valid JSON

// the name and value must be enclosed in double quotes
// single quotes are not valid
$bad_json = "{ 'bar': 'baz' }";
json_decode($bad_json); // null

// the name must be enclosed in double quotes
$bad_json = '{ bar: "baz" }';
json_decode($bad_json); // null
```

```php
// trailing commas are not allowed
$bad_json = '{ bar: "baz", }';
json_decode($bad_json); // null

?>
```

Example #4 `depth` errors

```php
<?php
// Encode some data with a maximum depth  of 4 (array -> array -> array -> string)
$json = json_encode(
    array(
        1 => array(
            'English' => array(
                'One',
                'January'
            ),
            'French' => array(
                'Une',
                'Janvier'
            )
        )
    )
);

// Show the errors for different depths.
var_dump(json_decode($json, true, 4));
echo 'Last error: ', json_last_error_msg(), PHP_EOL, PHP_EOL;

var_dump(json_decode($json, true, 3));
echo 'Last error: ', json_last_error_msg(), PHP_EOL, PHP_EOL;
?>
```

The above example will output:

```
array(1) {
  [1]=>
  array(2) {
    ["English"]=>
    array(2) {
      [0]=>
      string(3) "One"
      [1]=>
      string(7) "January"
    }
    ["French"]=>
    array(2) {
      [0]=>
      string(3) "Une"
      [1]=>
      string(7) "Janvier"
    }
  }
}
Last error: No error

NULL
Last error: Maximum stack depth exceeded
```

Example #5 `json_decode()` of large integers

```php
<?php
$json = '{"number": 12345678901234567890}';

var_dump(json_decode($json));
```

```php
var_dump(json_decode($json, false, 512, JSON_BIGINT_AS_STRING));

?>
```

The above example will output:

```
object(stdClass)#1 (1) {
  ["number"]=>
  float(1.2345678901235E+19)
}
object(stdClass)#1 (1) {
  ["number"]=>
  string(20) "12345678901234567890"
}
```

Notes

> **Note** :
>
> The JSON spec is not JavaScript, but a subset of JavaScript.

> **Note** :
>
> In the event of a failure to decode, json_last_error() can be used to determine the exact nature of the error.

See Also

- json_encode() - Returns the JSON representation of a value
- json_last_error() - Returns the last error occurred
- json_last_error_msg() - Returns the error string of the last json_validate(), json_encode() or json_decode() call

json_encode

json_encode

(PHP 5 >= 5.2.0, PHP 7, PHP 8, PECL json >= 1.2.0)

json_encode — Returns the JSON representation of a value

Description

function **json_encode**(mixed `$value`, int `$flags` = 0, int `$depth` = 512): string|false

Returns a string containing the JSON representation of the supplied `value`. If the parameter is an array or object, it will be serialized recursively.

If a value to be serialized is an object, then by default only publicly visible properties will be included. Alternatively, a class may implement JsonSerializable to control how its values are serialized to JSON.

The encoding is affected by the supplied `flags` and additionally the encoding of float values depends on the value of serialize_precision.

Parameters

`value`

The `value` being encoded. Can be any type except a resource.

All string data must be UTF-8 encoded.

> **Note** :

> PHP implements a superset of JSON as specified in the original » RFC 7159.

`flags`

Bitmask consisting of `JSON_FORCE_OBJECT`, `JSON_HEX_QUOT`, `JSON_HEX_TAG`, `JSON_HEX_AMP`, `JSON_HEX_APOS`, `JSON_INVALID_UTF8_IGNORE`, `JSON_INVALID_UTF8_SUBSTITUTE`, `JSON_NUMERIC_CHECK`, `JSON_PARTIAL_OUTPUT_ON_ERROR`, `JSON_PRESERVE_ZERO_FRACTION`, `JSON_PRETTY_PRINT`, `JSON_UNESCAPED_LINE_TERMINATORS`, `JSON_UNESCAPED_SLASHES`, `JSON_UNESCAPED_UNICODE`, `JSON_THROW_ON_ERROR`. The behaviour of these constants is described on the JSON constants page.

`depth`

Set the maximum depth. Must be greater than zero.

Return Values

Returns a JSON encoded string on success or `false` on failure.

Changelog

Version	Description
7.3.0	`JSON_THROW_ON_ERROR` `flags` was added.
7.2.0	`JSON_INVALID_UTF8_IGNORE`, and `JSON_INVALID_UTF8_SUBSTITUTE` `flags` were added.
7.1.0	`JSON_UNESCAPED_LINE_TERMINATORS` `flags` was added.
7.1.0	serialize_precision is used instead of precision when encoding float values.

Examples

Example #1 A json_encode() example

```php
<?php
$arr = array('a' => 1, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 5);

echo json_encode($arr);
?>
```

The above example will output:

```
{"a":1,"b":2,"c":3,"d":4,"e":5}
```

Example #2 A json_encode() example showing some flags in use

```php
<?php
$a = array('<foo>',"'bar'",'"baz"','&blong&', "\xc3\xa9");

echo "Normal: ",  json_encode($a), "\n";
echo "Tags: ",    json_encode($a, JSON_HEX_TAG), "\n";
echo "Apos: ",    json_encode($a, JSON_HEX_APOS), "\n";
echo "Quot: ",    json_encode($a, JSON_HEX_QUOT), "\n";
echo "Amp: ",     json_encode($a, JSON_HEX_AMP), "\n";
echo "Unicode: ", json_encode($a, JSON_UNESCAPED_UNICODE), "\n";
echo "All: ",     json_encode($a, JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_QUOT | JSON_HEX_AMP | JSON_UNESCAPED_UNICODE),
"\n\n";

$b = array();

echo "Empty array output as array: ", json_encode($b), "\n";
echo "Empty array output as object: ", json_encode($b, JSON_FORCE_OBJECT), "\n\n";

$c = array(array(1,2,3));

echo "Non-associative array output as array: ", json_encode($c), "\n";
```

```php
echo "Non-associative array output as object: ", json_encode($c, JSON_FORCE_OBJECT), "\n\n";

$d = array('foo' => 'bar', 'baz' => 'long');

echo "Associative array always output as object: ", json_encode($d), "\n";
echo "Associative array always output as object: ", json_encode($d, JSON_FORCE_OBJECT), "\n\n";
?>
```

The above example will output:

```
Normal: ["<foo>","'bar'","\"baz\"","&blong&","\u00e9"]
Tags: ["\u003Cfoo\u003E","'bar'","\"baz\"","&blong&","\u00e9"]
Apos: ["<foo>","\u0027bar\u0027","\"baz\"","&blong&","\u00e9"]
Quot: ["<foo>","'bar'","\u0022baz\u0022","&blong&","\u00e9"]
Amp: ["<foo>","'bar'","\"baz\"","\u0026blong\u0026","\u00e9"]
Unicode: ["<foo>","'bar'","\"baz\"","&blong&","é"]
All: ["\u003Cfoo\u003E","\u0027bar\u0027","\u0022baz\u0022","\u0026blong\u0026","é"]

Empty array output as array: []
Empty array output as object: {}

Non-associative array output as array: [[1,2,3]]
Non-associative array output as object: {"0":{"0":1,"1":2,"2":3}}

Associative array always output as object: {"foo":"bar","baz":"long"}
Associative array always output as object: {"foo":"bar","baz":"long"}
```

Example #3 JSON_NUMERIC_CHECK option example

```php
<?php
echo "Strings representing numbers automatically turned into numbers".PHP_EOL;
$numbers = array('+123123', '-123123', '1.2e3', '0.00001');
var_dump(
 $numbers,
 json_encode($numbers, JSON_NUMERIC_CHECK)
);
echo "Strings containing improperly formatted numbers".PHP_EOL;
$strings = array('+a33123456789', 'a123');
var_dump(
 $strings,
 json_encode($strings, JSON_NUMERIC_CHECK)
);
?>
```

The above example will output something similar to:

```
Strings representing numbers automatically turned into numbers
array(4) {
  [0]=>
  string(7) "+123123"
  [1]=>
  string(7) "-123123"
  [2]=>
  string(5) "1.2e3"
  [3]=>
  string(7) "0.00001"
}
string(28) "[123123,-123123,1200,1.0e-5]"
Strings containing improperly formatted numbers
array(2) {
  [0]=>
  string(13) "+a33123456789"
  [1]=>
```

```
  string(4) "a123"
}
string(24) "["+a33123456789","a123"]"
```

Example #4 Sequential versus non-sequential array example

```php
<?php
echo "Sequential array".PHP_EOL;
$sequential = array("foo", "bar", "baz", "blong");
var_dump(
 $sequential,
 json_encode($sequential)
);

echo PHP_EOL."Non-sequential array".PHP_EOL;
$nonsequential = array(1=>"foo", 2=>"bar", 3=>"baz", 4=>"blong");
var_dump(
 $nonsequential,
 json_encode($nonsequential)
);

echo PHP_EOL."Sequential array with one key unset".PHP_EOL;
unset($sequential[1]);
var_dump(
 $sequential,
 json_encode($sequential)
);
?>
```

The above example will output:

```
Sequential array
array(4) {
  [0]=>
  string(3) "foo"
  [1]=>
  string(3) "bar"
  [2]=>
  string(3) "baz"
  [3]=>
  string(5) "blong"
}
string(27) "["foo","bar","baz","blong"]"

Non-sequential array
array(4) {
  [1]=>
  string(3) "foo"
  [2]=>
  string(3) "bar"
  [3]=>
  string(3) "baz"
  [4]=>
  string(5) "blong"
}
string(43) "{"1":"foo","2":"bar","3":"baz","4":"blong"}"

Sequential array with one key unset
array(3) {
  [0]=>
  string(3) "foo"
  [2]=>
  string(3) "baz"
  [3]=>
  string(5) "blong"
```

```
}
string(33) "{"0":"foo","2":"baz","3":"blong"}"
```

Example #5 `JSON_PRESERVE_ZERO_FRACTION` option example

```php
<?php
var_dump(json_encode(12.0, JSON_PRESERVE_ZERO_FRACTION));
var_dump(json_encode(12.0));
?>
```

The above example will output:

```
string(4) "12.0"
string(2) "12"
```

Notes

> **Note** :
>
> In the event of a failure to encode, json_last_error() can be used to determine the exact nature of the error.

> **Note** :
>
> When encoding an array, if the keys are not a continuous numeric sequence starting from 0, all keys are encoded as strings, and specified explicitly for each key-value pair.

> **Note** :
>
> Like the reference JSON encoder, **json_encode()** will generate JSON that is a simple value (that is, neither an object nor an array) if given a string, int, float or bool as an input `value`. While most decoders will accept these values as valid JSON, some may not, as the specification is ambiguous on this point.
>
> To summarise, always test that your JSON decoder can handle the output you generate from **json_encode()**.

See Also

- JsonSerializable
- json_decode() - Decodes a JSON string
- json_last_error() - Returns the last error occurred
- json_last_error_msg() - Returns the error string of the last json_validate(), json_encode() or json_decode() call
- serialize() - Generates a storable representation of a value

json_last_error

json_last_error

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

json_last_error — Returns the last error occurred

Description

function **json_last_error**(): int

Returns the last error (if any) occurred during the last JSON validation/encoding/decoding, which did not specify `JSON_THROW_ON_ERROR`.

Parameters

This function has no parameters.

Return Values

Returns an integer, the value can be one of the following constants:

`JSON_ERROR_NONE`

No error has occurred

`JSON_ERROR_DEPTH`

The maximum stack depth has been exceeded

`JSON_ERROR_STATE_MISMATCH`

Invalid or malformed JSON

`JSON_ERROR_CTRL_CHAR`

Control character error, possibly incorrectly encoded

`JSON_ERROR_SYNTAX`

Syntax error

`JSON_ERROR_UTF8`

Malformed UTF-8 characters, possibly incorrectly encoded

`JSON_ERROR_RECURSION`

One or more recursive references in the value to be encoded

`JSON_ERROR_INF_OR_NAN`

One or more `NAN` or `INF` values in the value to be encoded

`JSON_ERROR_UNSUPPORTED_TYPE`

A value of a type that cannot be encoded was given

`JSON_ERROR_INVALID_PROPERTY_NAME`

A property name that cannot be encoded was given

`JSON_ERROR_UTF16`

Malformed UTF-16 characters, possibly incorrectly encoded

`JSON_ERROR_NON_BACKED_ENUM`

Value contains a non-backed enum which cannot be serialized

Changelog

Version	Description
8.1.0	`JSON_ERROR_NON_BACKED_ENUM` was added.

Examples

Example #1 json_last_error() example

```php
<?php
// A valid json string
$json[] = '{"Organization": "PHP Documentation Team"}';

// An invalid json string which will cause an syntax
// error, in this case we used ' instead of " for quotation
$json[] = "{'Organization': 'PHP Documentation Team'}";
```

```php
foreach ($json as $string) {
    echo 'Decoding: ' . $string;
    json_decode($string);

    switch (json_last_error()) {
        case JSON_ERROR_NONE:
            echo ' - No errors';
        break;
        case JSON_ERROR_DEPTH:
            echo ' - Maximum stack depth exceeded';
        break;
        case JSON_ERROR_STATE_MISMATCH:
            echo ' - Underflow or the modes mismatch';
        break;
        case JSON_ERROR_CTRL_CHAR:
            echo ' - Unexpected control character found';
        break;
        case JSON_ERROR_SYNTAX:
            echo ' - Syntax error, malformed JSON';
        break;
        case JSON_ERROR_UTF8:
            echo ' - Malformed UTF-8 characters, possibly incorrectly encoded';
        break;
        default:
            echo ' - Unknown error';
        break;
    }

    echo PHP_EOL;
}
?>
```

The above example will output:

```
Decoding: {"Organization": "PHP Documentation Team"} - No errors
Decoding: {'Organization': 'PHP Documentation Team'} - Syntax error, malformed JSON
```

Example #2 json_last_error() with json_encode()

```php
<?php
// An invalid UTF8 sequence
$text = "\xB1\x31";

$json  = json_encode($text);
$error = json_last_error();

var_dump($json, $error === JSON_ERROR_UTF8);
?>
```

The above example will output:

```
string(4) "null"
bool(true)
```

Example #3 json_last_error() and JSON_THROW_ON_ERROR

```php
<?php
// An invalid UTF8 sequence which causes JSON_ERROR_UTF8
json_encode("\xB1\x31");

// The following does not cause a JSON error
json_encode('okay', JSON_THROW_ON_ERROR);

// The global error state has not been changed by the former json_encode()
var_dump(json_last_error() === JSON_ERROR_UTF8);
?>
```

The above example will output:

```
bool(true)
```

See Also

- json_last_error_msg() - Returns the error string of the last json_validate(), json_encode() or json_decode() call
- json_decode() - Decodes a JSON string
- json_encode() - Returns the JSON representation of a value

json_last_error_msg

json_last_error_msg

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

json_last_error_msg — Returns the error string of the last json_validate(), json_encode() or json_decode() call

Description

function **json_last_error_msg**(): string

Returns the error string of the last json_validate(), json_encode() or json_decode() call, which did not specify `JSON_THROW_ON_ERROR`.

Parameters

This function has no parameters.

Return Values

Returns the error message on success, or `"No error"` if no error has occurred.

See Also

- json_last_error() - Returns the last error occurred

json_validate

json_validate

(PHP 8 >= 8.3.0)

json_validate — Checks if a string contains valid JSON

Description

function **json_validate**(string `$json`, int `$depth` = 512, int `$flags` = 0): bool

Returns whether the given string is syntactically valid JSON. If **json_validate()** returns `true`, json_decode() will successfully decode the given string when using the same `depth` and `flags`.

If **json_validate()** returns `false`, the cause can be retrieved using json_last_error() and json_last_error_msg().

json_validate() uses less memory than json_decode() if the decoded JSON payload is not used, because it does not need to build the array or object structure containing the payload.

> **Caution**
>
> Calling **json_validate()** immediately before json_decode() will unnecessarily parse the string twice, as json_decode() implicitly performs validation during decoding.
>
> **json_validate()** should therefore only be used if the decode JSON payload is not immediately used and knowing whether the string contains valid JSON is needed.

Parameters

`json`

The string to validate.

This function only works with UTF-8 encoded strings.

> **Note** :
>
> PHP implements a superset of JSON as specified in the original » RFC 7159.

`depth`

Maximum nesting depth of the structure being decoded. The value must be greater than `0`, and less than or equal to `2147483647`.

`flags`

Currently only `JSON_INVALID_UTF8_IGNORE` is accepted.

Return Values

Returns `true` if the given string is syntactically valid JSON, otherwise returns `false`.

Errors/Exceptions

If `depth` is outside the allowed range, a ValueError is thrown.

If `flags` is not a valid flag, a ValueError is thrown.

Examples

Example #1 json_validate() examples

```php
<?php
var_dump(json_validate('{ "test": { "foo": "bar" } }'));
var_dump(json_validate('{ "": "": "" } }'));
?>
```

The above example will output:

```
bool(true)
bool(false)
```

See Also

- json_decode() - Decodes a JSON string
- json_last_error() - Returns the last error occurred
- json_last_error_msg() - Returns the error string of the last json_validate(), json_encode() or json_decode() call

Session Handling

Introduction

Session support in PHP consists of a way to preserve certain data across subsequent accesses.

A visitor accessing your web site is assigned a unique id, the so-called session id. This is either stored in a cookie on the user side or is propagated in the URL.

The session support allows you to store data between requests in the *$_SESSION* superglobal array. When a visitor accesses your site, PHP will check automatically (if session.auto_start is set to 1) or on your request (explicitly through session_start()) whether a specific session id has been sent with the request. If this is the case, the prior saved environment is recreated.

> **Caution**
>
> If you turn on session.auto_start then the only way to put objects into your sessions is to load its class definition using auto_prepend_file in which you load the class definition else you will have to serialize() your object and unserialize() it afterwards.

$_SESSION (and all registered variables) are serialized internally by PHP using the serialization handler specified by the session.serialize_handler ini setting, after the request finishes. Registered variables which are undefined are marked as being not defined. On subsequent accesses, these are not defined by the session module unless the user defines them later.

> **Warning**
>
> Because session data is serialized, resource variables cannot be stored in the session.
>
> Serialize handlers (`php` and `php_binary`) inherit register_globals limitations. Therefore, numeric index or string index contains special characters (`|` and `!`) cannot be used. Using these will end up with errors at script shutdown. `php_serialize` does not have such limitations.

> **Note** :
>
> Please note when working with sessions that a record of a session is not created until a variable has been registered by adding a new key to the *$_SESSION* superglobal array. This holds true regardless of if a session has been started using the session_start() function.

Installing/Configuring

Table of Contents

Requirements

No external libraries are needed to build this extension.

> **Note** :
>
> Optionally you can use shared memory allocation (mm), developed by Ralf S. Engelschall, for session storage. You have to download » mm and install it. This option is not available for Windows platforms. Note that the session storage module for mm does not guarantee that concurrent accesses to the same session are properly locked. It might be more appropriate to use a shared memory based filesystem (such as tmpfs on Solaris/Linux, or */dev/md* on BSD) to store sessions in files, because they are properly locked. Session data is stored in memory thus web server restart deletes it.

Installation

This extension is enabled by default. It may be disabled by using the following option at compile time: **--disable-session**

To use shared memory allocation (mm) for session storage configure PHP **--with-mm[=DIR]**.

The Windows version of PHP has built-in support for this extension. You do not need to load any additional extensions in order to use these functions.

> **Note** :
>
> By default, all data related to a particular session will be stored in a file in the directory specified by the session.save_path INI option. A file for each session (regardless of if any data is associated with that session) will be created. This is due to the fact that a session is opened (a file is created) but no data is even written to that file. Note that this behavior is a side-effect of the limitations of working with the file system and it is possible that a custom session handler (such as one which uses a database) does not keep track of sessions which store no data.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Session configuration options

Name	Default	Changeable	Changelog
session.save_path	""	INI_ALL	
session.name	"PHPSESSID"	INI_ALL	
session.save_handler	"files"	INI_ALL	
session.auto_start	"0"	INI_PERDIR	
session.gc_probability	"1"	INI_ALL	
session.gc_divisor	"100"	INI_ALL	
session.gc_maxlifetime	"1440"	INI_ALL	
session.serialize_handler	"php"	INI_ALL	
session.cookie_lifetime	"0"	INI_ALL	
session.cookie_path	"/"	INI_ALL	
session.cookie_domain	""	INI_ALL	
session.cookie_secure	"0"	INI_ALL	Prior to PHP 7.2.0, the default was "".
session.cookie_httponly	"0"	INI_ALL	Prior to PHP 7.2.0, the default was "".
session.cookie_partitioned	"0"	INI_ALL	Available as of PHP 8.5.0.
session.cookie_samesite	""	INI_ALL	Available as of PHP 7.3.0.
session.use_strict_mode	"0"	INI_ALL	
session.use_cookies	"1"	INI_ALL	
session.use_only_cookies	"1"	INI_ALL	Disabling this setting is deprecated as of PHP 8.4.0.
session.referer_check	""	INI_ALL	Setting a non-empty value is deprecated as of PHP 8.4.0.
session.cache_limiter	"nocache"	INI_ALL	
session.cache_expire	"180"	INI_ALL	
session.use_trans_sid	"0"	INI_ALL	Enabling this setting is deprecated as of PHP 8.4.0.
session.trans_sid_tags	"a=href,area=href,frame=src,form="	INI_ALL	Available as of PHP 7.1.0. Changing this setting is deprecated as of PHP 8.4.0.
session.trans_sid_hosts	`$_SERVER['HTTP_HOST']`	INI_ALL	Available as of PHP 7.1.0. Setting a non-empty value is deprecated as of PHP 8.4.0.
session.sid_length	"32"	INI_ALL	Available as of PHP 7.1.0. Changing this setting is deprecated as of PHP 8.4.0.
session.sid_bits_per_character	"4"	INI_ALL	Available as of PHP 7.1.0. Changing this setting is deprecated as of PHP 8.4.0.
session.upload_progress.enabled	"1"	INI_PERDIR	
session.upload_progress.cleanup	"1"	INI_PERDIR	
session.upload_progress.prefix	"upload_progress_"	INI_PERDIR	

Name	Default	Changeable	Changelog
session.upload_progress.name	"PHP_SESSION_UPLOAD_PROGRESS"	INI_PERDIR	
session.upload_progress.freq	"1%"	INI_PERDIR	
session.upload_progress.min_freq	"1"	INI_PERDIR	
session.lazy_write	"1"	INI_ALL	
session.hash_function	"0"	INI_ALL	Removed as of PHP 7.1.0.
session.hash_bits_per_character	"4"	INI_ALL	Removed as of PHP 7.1.0.
session.entropy_file	""	INI_ALL	Removed as of PHP 7.1.0.
session.entropy_length	"0"	INI_ALL	Removed as of PHP 7.1.0.

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

The session management system supports a number of configuration options which you can place in your *php.ini* file. We will give a short overview.

`session.save_handler` string

> `session.save_handler` defines the name of the handler which is used for storing and retrieving data associated with a session. Defaults to `files`. Note that individual extensions may register their own `save_handlers`; registered handlers can be obtained on a per-installation basis by referring to phpinfo(). See also session_set_save_handler().

`session.save_path` string

> `session.save_path` defines the argument which is passed to the save handler. If you choose the default files handler, this is the path where the files are created. See also session_save_path().
>
> There is an optional `N` argument to this directive that determines the number of directory levels your session files will be spread around in. For example, setting to `'5;/tmp'` may end up creating a session file and location like `/tmp/4/b/1/e/3/sess_4b1e384ad74619bd212e236e52a5a174If` . In order to use `N` you must create all of these directories before use. A small shell script exists in *ext/session* to do this, it's called *mod_files.sh*, with a Windows version called *mod_files.bat*. Also note that if `N` is used and greater than 0 then automatic garbage collection will not be performed, see a copy of *php.ini* for further information. Also, if you use `N`, be sure to surround `session.save_path` in "quotes" because the separator (`;`) is also used for comments in *php.ini*.
>
> The file storage module creates files using mode 600 by default. This default can be changed with the optional `MODE` argument: `N;MODE;/path` where `MODE` is the octal representation of the mode. Setting `MODE` does not affect the process umask.
>
> > **Warning**
> >
> > If this is set to a world-readable directory, such as */tmp* (the default), other users on the server may be able to hijack sessions by getting the list of files in that directory.
>
> > **Caution**
> >
> > When using the optional directory level argument `N`, as described above, note that using a value higher than 1 or 2 is inappropriate for most sites due to the large number of directories required: for example, a value of 3 implies that `(2 ** session.sid_bits_per_character) ** 3` directories exist on the filesystem, which can result in a lot of wasted space and inodes.
> >
> > Only use `N` greater than 2 if you are absolutely certain that your site is large enough to require it.

`session.name` string

> `session.name` specifies the name of the session which is used as cookie name. It should only contain alphanumeric characters. Defaults to `PHPSESSID`. See also session_name().

`session.auto_start` bool

> `session.auto_start` specifies whether the session module starts a session automatically on request startup. Defaults to `0` (disabled).

`session.serialize_handler` string

> `session.serialize_handler` defines the name of the handler which is used to serialize/deserialize data. PHP serialize format (name `php_serialize`), PHP internal formats (name `php` and `php_binary`) and WDDX are supported (name `wddx`). WDDX is only available, if PHP is compiled with WDDX support. `php_serialize` uses plain serialize/unserialize function internally and does not have limitations

that `php` and `php_binary` have. Older serialize handlers cannot store numeric index nor string index contains special characters (`|` and `!`) in $_SESSION. Use `php_serialize` to avoid numeric index or special character errors at script shutdown. Defaults to `php`.

`session.gc_probability` int

`session.gc_probability` in conjunction with `session.gc_divisor` is used to manage probability that the gc (garbage collection) routine is started. Defaults to `1`. Must be greater than or equal to `0`. See session.gc_divisor for details.

`session.gc_divisor` int

`session.gc_divisor` coupled with `session.gc_probability` defines the probability that the gc (garbage collection) process is started on every session initialization. The probability is calculated by using gc_probability/gc_divisor, e.g. 1/100 means there is a 1% chance that the GC process starts on each request. `session.gc_divisor` defaults to `100`. Must be greater than `0`.

`session.gc_maxlifetime` int

`session.gc_maxlifetime` specifies the number of seconds after which data will be seen as 'garbage' and potentially cleaned up. Garbage collection may occur during session start (depending on session.gc_probability and session.gc_divisor). Defaults to `1440` (24 minutes).

> **Note** :
>
> If different scripts have different values of `session.gc_maxlifetime` but share the same place for storing the session data then the script with the minimum value will be cleaning the data. In this case, use this directive together with session.save_path.

`session.referer_check` string

`session.referer_check` contains the substring you want to check each HTTP Referer for. If the Referer was sent by the client and the substring was not found, the embedded session id will be marked as invalid. Defaults to the empty string.

> **Warning**
>
> Setting `session.referer_check` to a non-empty value is deprecated as of PHP 8.4.0.

`session.entropy_file` string

`session.entropy_file` gives a path to an external resource (file) which will be used as an additional entropy source in the session id creation process. Examples are `/dev/random` or `/dev/urandom` which are available on many Unix systems.

This feature is supported on Windows. Setting `session.entropy_length` to a non zero value will make PHP use the Windows Random API as entropy source.

> **Note** :
>
> Removed in PHP 7.1.0.
>
> `session.entropy_file` defaults to `/dev/urandom` or `/dev/arandom` if it is available.

`session.entropy_length` int

`session.entropy_length` specifies the number of bytes which will be read from the file specified above. Defaults to `32`.

Removed in PHP 7.1.0.

`session.use_strict_mode` bool

`session.use_strict_mode` specifies whether the module will use strict session id mode. If this mode is enabled, the module does not accept uninitialized session IDs. If an uninitialized session ID is sent from the browser, a new session ID is sent to the browser. Applications are protected from session fixation via session adoption with strict mode. Defaults to `0` (disabled).

> **Note** :
>
> Enabling `session.use_strict_mode` is mandatory for general session security. All sites are advised to enable this. See session_create_id() example code for more details.

> **Warning**
>
> If a custom session handler registered via session_set_save_handler() does not implement SessionUpdateTimestampHandlerInterface::validateId(), nor supplies the `validate_sid` callback, respectively, strict session ID mode is effectively disabled, regardless of the value of this directive. Particularly note that SessionHandler does *not* implement **SessionHandler::validateId()**.

`session.use_cookies` bool

`session.use_cookies` specifies whether the module will use cookies to store the session id on the client side. Defaults to `1` (enabled).

`session.use_only_cookies` bool

`session.use_only_cookies` specifies whether the module will **only** use cookies to store the session id on the client side. Enabling this setting prevents attacks that involve passing session IDs in URLs. Defaults to `1` (enabled).

> **Warning**
>
> Disabling `session.use_only_cookies` is deprecated as of PHP 8.4.0.

`session.cookie_lifetime` int

`session.cookie_lifetime` specifies the lifetime of the cookie in seconds which is sent to the browser. The value 0 means "until the browser is closed." Defaults to `0`. See also session_get_cookie_params() and session_set_cookie_params().

> **Note** :
>
> The expiration timestamp is set relative to the server time, which is not necessarily the same as the time in the client's browser.

`session.cookie_path` string

`session.cookie_path` specifies path to set in the session cookie. Defaults to `/`. See also session_get_cookie_params() and session_set_cookie_params().

`session.cookie_domain` string

`session.cookie_domain` specifies the domain to set in the session cookie. Default is none at all meaning the host name of the server which generated the cookie according to cookies specification. See also session_get_cookie_params() and session_set_cookie_params().

`session.cookie_secure` bool

`session.cookie_secure` specifies whether cookies should only be sent over secure connections. With this option set to `on`, sessions only work with HTTPS connections. If it is `off`, then sessions work with both HTTP and HTTPS connections. Defaults to `off`. See also session_get_cookie_params() and session_set_cookie_params().

`session.cookie_httponly` bool

Marks the cookie as accessible only through the HTTP protocol. This means that the cookie won't be accessible by scripting languages, such as JavaScript. This setting can effectively help to reduce identity theft through XSS attacks (although it is not supported by all browsers).

`session.cookie_partitioned` bool

Marks the cookie as partitioned (CHIPS), which means that the cookie will be isolated to a first-party context. This setting can help mitigate cross-site tracking (although it is not supported by all browsers).

If enabled, session.cookie_secure must also be enabled, otherwise the session cookie will not be sent and a warning is raised.

`session.cookie_samesite` string

Allows servers to assert that a cookie ought not to be sent along with cross-site requests. This assertion allows user agents to mitigate the risk of cross-origin information leakage, and provides some protection against cross-site request forgery attacks. Note that this is not supported by all browsers. An empty value means that no SameSite cookie attribute will be set. `Lax` and `Strict` mean that the cookie will not be sent cross-domain for POST requests; `Lax` will send the cookie for cross-domain GET requests, while `Strict` will not.

`session.cache_limiter` string

`session.cache_limiter` specifies the cache control method used for session pages. It may be one of the following values: `nocache`, `private`, `private_no_expire`, or `public`. Defaults to `nocache`. See also the session_cache_limiter() documentation for information about what these values mean.

`session.cache_expire` int

`session.cache_expire` specifies time-to-live for cached session pages in minutes, this has no effect for nocache limiter. Defaults to `180`. See also session_cache_expire().

`session.use_trans_sid` bool

`session.use_trans_sid` whether transparent sid support is enabled or not. Defaults to `0` (disabled).

> **Warning**
>
> Enabling `session.use_trans_sid` is deprecated as of PHP 8.4.0.

> **Note** :
>
> URL based session management has additional security risks compared to cookie based session management. Users may send a URL that contains an active session ID to their friends by email or users may save a URL that contains a session ID to their bookmarks and access your site with the same session ID always, for example.
>
> Since PHP 7.1.0, full URL path, e.g. https://php.net/, is handled by trans sid feature. Previous PHP handled relative URL path only. Rewrite target hosts are defined by session.trans_sid_hosts.

`session.trans_sid_tags` string

`session.trans_sid_tags` specifies which HTML tags are rewritten to include session id when transparent sid support is enabled. Defaults to `a=href,area=href,frame=src,form=`

`form` is special tag. `<input hidden="session_id" name="session_name">` is added as form variable.

> **Warning**
>
> Changing `session.trans_sid_tags` from its default value is deprecated as of PHP 8.4.0.

> **Note** :
>
> Before PHP 7.1.0, url_rewriter.tags was used for this purpose. Since PHP 7.1.0, `fieldset` is no longer considered as special tag.

`session.trans_sid_hosts` string

`session.trans_sid_hosts` specifies which hosts are rewritten to include session id when transparent sid support is enabled. Defaults to `$_SERVER['HTTP_HOST']` Multiple hosts can be specified by ",", no space is allowed between hosts. e.g. `php.net,wiki.php.net,bugs.php.net`

> **Warning**
>
> Setting `session.trans_sid_hosts` to a non-empty value is deprecated as of PHP 8.4.0.

`session.sid_length` int

`session.sid_length` allows you to specify the length of session ID string. Session ID length can be between 22 to 256.

The default is 32. A longer session ID is harder to guess.

> **Warning**
>
> Changing `session.sid_length` from its default value is deprecated as of PHP 8.4.0.

> **Note** :
>
> Available as of PHP 7.1.0.

`session.sid_bits_per_character` int

`session.sid_bits_per_character` allows you to specify the number of bits in encoded session ID character. The possible values are '4' (0-9, a-f), '5' (0-9, a-v), and '6' (0-9, a-z, A-Z, "-", ",").

The default is 4. More bits result in a stronger session ID.

> **Warning**
>
> Changing `session.sid_bits_per_character` from its default value is deprecated as of PHP 8.4.0.

> **Note** :
>
> Available as of PHP 7.1.0.

`session.hash_function` mixed

`session.hash_function` allows you to specify the hash algorithm used to generate the session IDs. '0' means MD5 (128 bits) and '1' means SHA-1 (160 bits).

It is also possible to specify any of the algorithms provided by the hash extension (if it is available), like `sha512` or `whirlpool`. A complete list of supported algorithms can be obtained with the hash_algos() function.

> **Note** :
>
> Removed in PHP 7.1.0.

`session.hash_bits_per_character` int

`session.hash_bits_per_character` allows you to define how many bits are stored in each character when converting the binary hash data to something readable. The possible values are '4' (0-9, a-f), '5' (0-9, a-v), and '6' (0-9, a-z, A-Z, "-", ",").

> **Note** :
>
> Removed in PHP 7.1.0.

`session.upload_progress.enabled` bool

Enables upload progress tracking, populating the *$_SESSION* variable. Defaults to 1, enabled.

`session.upload_progress.cleanup` bool

Cleanup the progress information as soon as all POST data has been read (i.e. upload completed). Defaults to 1, enabled.

> **Note** :
>
> It is highly recommended to keep this feature enabled.

`session.upload_progress.prefix` string

A prefix used for the upload progress key in the *$_SESSION*. This key will be concatenated with the value of `$_POST[ini_get("session.upload_progress.name")]` to provide a unique index.

Defaults to "upload_progress_".

`session.upload_progress.name` string

The name of the key to be used in *$_SESSION* storing the progress information. See also session.upload_progress.prefix.

If `$_POST[ini_get("session.upload_progress.name")]` is not passed or available, upload progressing will not be recorded.

Defaults to "PHP_SESSION_UPLOAD_PROGRESS".

`session.upload_progress.freq` mixed

Defines how often the upload progress information should be updated. This can be defined in bytes (i.e. "update progress information after every 100 bytes"), or in percentages (i.e. "update progress information after receiving every 1% of the whole filesize").

Defaults to "1%".

`session.upload_progress.min_freq` int

The minimum delay between updates, in seconds. Defaults to "1" (one second).

`session.lazy_write` bool

`session.lazy_write`, when set to 1, means that session data is only rewritten if it changes. Defaults to 1, enabled.

Upload progress will not be registered unless session.upload_progress.enabled is enabled, and the $_POST[ini_get("session.upload_progress.name")] variable is set. See Session Upload Progress for more details on this functionality.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

`SID` (string)

Constant containing either the session name and session ID in the form of `"name=ID"` or empty string if session ID was set in an appropriate session cookie. This is the same id as the one returned by session_id().

> **Warning**
>
> This constant has been deprecated as of PHP 8.4.0.

`PHP_SESSION_DISABLED` (int)

Return value of session_status() if sessions are disabled.

`PHP_SESSION_NONE` (int)

Return value of session_status() if sessions are enabled, but no session exists.

`PHP_SESSION_ACTIVE` (int)

Return value of session_status() if sessions are enabled, and a session exists.

Examples

Table of Contents

Basic usage

Sessions are a simple way to store data for individual users against a unique session ID. This can be used to persist state information between page requests. Session IDs are normally sent to the browser via session cookies and the ID is used to retrieve existing session data. The absence of an ID or session cookie lets PHP know to create a new session, and generate a new session ID.

Sessions follow a simple workflow. When a session is started, PHP will either retrieve an existing session using the ID passed (usually from a session cookie) or if no session is passed it will create a new session. PHP will populate the *$_SESSION* superglobal with any session data after the session has started. When PHP shuts down, it will automatically take the contents of the *$_SESSION* superglobal, serialize it, and send it for storage using the session save handler.

By default, PHP uses the internal `files` save handler which is set by session.save_handler. This saves session data on the server at the location specified by the session.save_path configuration directive.

Sessions can be started manually using the session_start() function. If the session.auto_start directive is set to 1, a session will automatically start on request startup.

Sessions normally shutdown automatically when PHP is finished executing a script, but can be manually shutdown using the session_write_close() function.

Example #1 Registering a variable with *$_SESSION*.

```
<?php
session_start();
if (!isset($_SESSION['count'])) {
  $_SESSION['count'] = 0;
} else {
  $_SESSION['count']++;
}
?>
```

Example #2 Unregistering a variable with *$_SESSION*.

```
<?php
session_start();
unset($_SESSION['count']);
?>
```

> **Caution**
>
> Do NOT unset the whole *$_SESSION* with `unset($_SESSION)` as this will disable the registering of session variables through the *$_SESSION* superglobal.

> **Warning**
>
> You can't use references in session variables as there is no feasible way to restore a reference to another variable.

> **Note** :
>
> File based sessions (the default in PHP) lock the session file once a session is opened via session_start() or implicitly via session.auto_start. Once locked, no other script can access the same session file until it has been closed by the first script terminating or calling session_write_close().
>
> This is most likely to be an issue on Web sites that use AJAX heavily and have multiple concurrent requests. The easiest way to deal with it is to call session_write_close() as soon as any required changes to the session have been made, preferably early in the script. Alternatively, a different session backend that does support concurrency could be used.

Passing the Session ID

There are two methods to propagate a session id:

- Cookies
- URL parameter

The session module supports both methods. Cookies are optimal, but because they are not always available, we also provide an alternative way. The second method embeds the session id directly into URLs.

PHP is capable of transforming links transparently. If the run-time option `session.use_trans_sid` is enabled, relative URIs will be changed to contain the session id automatically.

> **Note** :
>
> The arg_separator.output *php.ini* directive allows to customize the argument separator. For full XHTML conformance, specify & there.

Alternatively, you can use the constant **SID** which is defined if the session started. If the client did not send an appropriate session cookie, it has the form `session_name=session_id`. Otherwise, it expands to an empty string. Thus, you can embed it unconditionally into URLs.

The following example demonstrates how to register a variable, and how to link correctly to another page using **SID**.

Example #1 Counting the number of hits of a single user

```
<?php

session_start();

if (empty($_SESSION['count'])) {
   $_SESSION['count'] = 1;
} else {
   $_SESSION['count']++;
}
?>

<p>
Hello visitor, you have seen this page <?php echo $_SESSION['count']; ?> times.
</p>

<p>
To continue, <a href="nextpage.php?<?php echo htmlspecialchars(SID); ?>">click
here</a>.
</p>
```

The htmlspecialchars() may be used when printing the **SID** in order to prevent XSS related attacks.

Printing the **SID**, like shown above, is not necessary if --enable-trans-sid was used to compile PHP.

> **Note** :
>
> Non-relative URLs are assumed to point to external sites and hence don't append the **SID**, as it would be a security risk to leak the **SID** to a different server.

Custom Session Handlers

To implement database storage, or any other storage method, you will need to use session_set_save_handler() to create a set of user-level storage functions. A session handlers may be created using the SessionHandlerInterface or extending PHP's internal handlers by inheriting from SessionHandler.

The callbacks specified in session_set_save_handler() are methods called by PHP during the life-cycle of a session: `open`, `read`, `write` and `close` and for the housekeeping tasks: `destroy` for deleting a session and `gc` for periodic garbage collection.

Therefore, PHP always requires session save handlers. The default is usually the internal 'files' save handler. A custom save handler can be set using session_set_save_handler(). Alternative internal save handlers are also provided by PHP extensions, such as `sqlite`, `memcache` and `memcached` and can be set with session.save_handler.

When the session starts, PHP will internally call the `open` handler followed by the `read` callback which should return an encoded string exactly as it was originally passed for storage. Once the `read` callback returns the encoded string, PHP will decode it and then populate the resulting array into the *$_SESSION* superglobal.

When PHP shuts down (or when session_write_close() is called), PHP will internally encode the *$_SESSION* superglobal and pass this along with the session ID to the `write` callback. After the `write` callback has finished, PHP will internally invoke the `close` callback handler.

When a session is specifically destroyed, PHP will call the `destroy` handler with the session ID.

PHP will call the `gc` callback from time to time to expire any session records according to the set max lifetime of a session. This routine should delete all records from persistent storage which were last accessed longer than the `lifetime`.

Session Upload Progress

When the session.upload_progress.enabled INI option is enabled, PHP will be able to track the upload progress of individual files being uploaded. This information isn't particularly useful for the actual upload request itself, but during the file upload an application can send a POST request to a separate endpoint (via XHR for example) to check the status.

The upload progress will be available in the *$_SESSION* superglobal when an upload is in progress, and when POSTing a variable of the same name as the session.upload_progress.name INI setting is set to. When PHP detects such POST requests, it will populate an array in the *$_SESSION*, where the index is a concatenated value of the session.upload_progress.prefix and session.upload_progress.name INI options. The key is typically retrieved by reading these INI settings, i.e.

```
<?php
$key = ini_get("session.upload_progress.prefix") . $_POST[ini_get("session.upload_progress.name")];
var_dump($_SESSION[$key]);
?>
```

It is also possible to *cancel* the currently in-progress file upload, by setting the `$_SESSION[$key]["cancel_upload"]` key to **`true`**. When uploading multiple files in the same request, this will only cancel the currently in-progress file upload, and pending file uploads, but will not remove successfully completed uploads. When an upload is cancelled like this, the `error` key in *$_FILES* array will be set to **`UPLOAD_ERR_EXTENSION`**.

The session.upload_progress.freq and session.upload_progress.min_freq INI options control how frequent the upload progress information should be recalculated. With a reasonable amount for these two settings, the overhead of this feature is almost non-existent.

Example #1 Example information

Example of the structure of the progress upload array.

```
<form action="upload.php" method="POST" enctype="multipart/form-data">
 <input type="hidden" name="<?php echo ini_get("session.upload_progress.name"); ?>" value="123" />
 <input type="file" name="file1" />
 <input type="file" name="file2" />
 <input type="submit" />
</form>
```

The data stored in the session will look like this:

```
<?php
$_SESSION["upload_progress_123"] = array(
 "start_time" => 1234567890,   // The request time
 "content_length" => 57343257, // POST content length
 "bytes_processed" => 453489,  // Amount of bytes received and processed
 "done" => false,              // true when the POST handler has finished, successfully or not
 "files" => array(
  0 => array(
   "field_name" => "file1",       // Name of the <input/> field
   // The following 3 elements equals those in $_FILES
   "name" => "foo.avi",
   "tmp_name" => "/tmp/phpxxxxxx",
   "error" => 0,
   "done" => true,                // True when the POST handler has finished handling this file
   "start_time" => 1234567890,    // When this file has started to be processed
   "bytes_processed" => 57343250, // Number of bytes received and processed for this file
  ),
  // An other file, not finished uploading, in the same request
  1 => array(
   "field_name" => "file2",
   "name" => "bar.avi",
   "tmp_name" => NULL,
   "error" => 0,
   "done" => false,
   "start_time" => 1234567899,
   "bytes_processed" => 54554,
  ),
 )
);
```

> **Warning**
>
> The web server's request buffering has to be disabled for this to work properly, else PHP may see the file upload only once fully uploaded. Servers such as Nginx are known to buffer larger requests.

> **Caution**

> The upload progress information is written to the session before any scripts are executed. Therefore changing the session name via ini_set() or session_name() will give a session without the upload progress information.

Sessions and Security

Table of Contents

External links: » Session fixation

HTTP session management represents the core of web security. All possible mitigation measures *should* be adopted to ensure sessions are secured. Developers should also enable/use applicable security measures.

Session Management Basics

Session Security

The session module can not guarantee that the information stored in a session is only viewed by the user who created the session. Additional measures are needed to protect the confidentiality of the session, depending on the value associated with it.

The importance of the data carried in the session needs to be assessed and further protection may be deployed; this typically comes at a price, such as reduced convenience for the user. For example, to protect users from a simple social engineering tactic, session.use_only_cookies needs to be enabled. In that case, cookies must be enabled unconditionally on the client side, or sessions will not work.

There are several ways to leak an existing session ID to third parties. E.g. JavaScript injections, session IDs in URLs, packet sniffing, physical access to the device, etc. A leaked session ID enables the third party to access all resources associated with a specific ID. First, URLs carrying session IDs. If there are links to an external site or resource, the URL including the session ID might be stored in the external site's referrer logs. Second, a more active attacker might listen to network traffic. If it is unencrypted, session IDs will flow in plain text over the network. The solution is to implement SSL/TLS on the server and make it mandatory for users. HSTS should be used for improved security.

> **Note** :
>
> Even HTTPS can not protect confidential data at all times. For example the CRIME and Beast vulnerabilities may enable an attacker to read the data. Also, note that many networks employ HTTPS MITM proxies for audit purposes. Attackers may also set up such a proxy.

Non-adaptive Session Management

PHP's session manager is adaptive by default currently. An adaptive session manager bears additional risks.

When session.use_strict_mode is enabled, and the session save handler supports it, an uninitialized session ID is rejected and a new one is created. This prevents an attack that forces users to use a known session ID. An attacker may paste links or send emails that contains the session ID. E.g. `http://example.com/page.php?PHPSESSID=123456789` if session.use_trans_sid is enabled, the victim will start a session using the session ID provided by the attacker. session.use_strict_mode mitigate this risk.

> **Warning**
>
> User defined save handlers can also support strict session mode by implementing session ID validation. All user defined save handlers should implement session ID validation.

The session ID cookie may be set with the domain, path, httponly, secure and, as of PHP 7.3, SameSite attributes. There is precedence defined by browsers. By using the precedence, an attacker can set session ID that could be used permanently. Use of session.use_only_cookies will not solve this issue. session.use_strict_mode mitigates this risk. With session.use_strict_mode=On, the uninitialized session ID will be refused.

> **Note** :
>
> Even though session.use_strict_mode mitigates the risk of adaptive session management, an attacker can force users to use an initialized session ID which has been created by an attacker. E.g. JavaScript injection. This attack can be mitigated by this manual's recommendations.
>
> By following this manual, developers should enable, session.use_strict_mode, use timestamp based session management, and regenerate session IDs using session_regenerate_id() with recommended procedures. If developers follow all of the above, an attacker

generated session ID will eventually be deleted.

When access to an obsolete session occurs, developers should save all active session data of the user. As this information will be relevant for an ensuing investigation. The user should be forcefully logged out of all sessions, i.e. require them to reauthenticate. This prevents attackers from abusing stolen sessions.

> **Warning**
>
> Access to an obsolete session does not necessarily suggest an attack. An unstable network and/or immediate deletion of the active session will result in legitimate users using obsolete sessions.

As of PHP 7.1.0, session_create_id() has been added. This function may be operated to access all active sessions of a user efficiently by prefixing the session IDs with the user ID. Enabling session.use_strict_mode is vital with this setup. Otherwise, malicious users can set malicious session ID for other users.

> **Note** :
>
> Users prior to PHP 7.1.0 *should* use CSPRNG, e.g. */dev/urandom*, or random_bytes() and hash functions to generate a new session ID. session_create_id() has collision detection and generates a session ID according to the session's INI settings. Use of session_create_id() is preferred.

Session ID Regeneration

session.use_strict_mode is a good mitigation, however not sufficient. Developers must equally use session_regenerate_id() for session security.

Session ID regeneration reduces the risk of stolen session IDs, thus session_regenerate_id() must be called periodically. E.g. Regenerate the session ID every 15 minutes for security sensitive content. Even in the case that a session ID is stolen, both the legitimate user's and the attacker's session will expire. In other words access by the user or the attacker will generate an obsolete session access error.

Session IDs *must* be regenerated when user privileges are elevated, such as after authenticating. session_regenerate_id() must be called prior to setting the authentication information to *$_SESSION*. (session_regenerate_id() saves the current session data automatically in order to save timestamp/etc. to the current session.) Ensure only the new session contains the authenticated flag.

Developers *must not* rely on session ID expiration by session.gc_maxlifetime. Attackers may access a victim's session ID periodically to prevent its expiration and keep exploiting it, including an authenticated session.

Instead, developers must implement timestamp based session data management.

> **Warning**
>
> Although the session manager can manage timestamps transparently, this feature is not implemented. Old session data must be kept until GC. Simultaneously, developers must assure themselves obsolete session data is removed. However, developers must not remove active session data immediately. I.e. `session_regenerate_id(true);` and session_destroy() must never be called together for an active session. This may sound contradictory, but this is a mandatory requirement.

session_regenerate_id() does *not* delete outdated sessions by default. Obsolete authenticated sessions may be present for use. Developers must prevent outdated sessions to be consumed by anyone. They must prohibit access to obsolete session data by themselves using timestamps.

> **Warning**
>
> The sudden removal of an active session produces undesirable side effects. Sessions can vanish when there are concurrent connections to the web application and/or the network is unstable.
>
> Potential malicious access is undetectable with the sudden removal of active sessions.
>
> Instead of deleting outdated sessions immediately, developers must set a short-term expiration time (timestamp) in *$_SESSION*, and prohibit access to the session data by themselves.
>
> Developers must not prohibit access to old session data immediately after session_regenerate_id(). It must be prohibited at a later stage. E.g. a few seconds later for stable networks, like a wired network, and a few minutes later for unstable networks such as cell phones or Wi-Fi.

> If a user accesses an obsolete session (expired session), access to it should be denied. It is also recommended to remove the authenticated status from all of the user's sessions to as it is likely to represent an attack.

Proper use of session.use_only_cookies and session_regenerate_id() can cause personal DoS with undeletable cookies set by attackers. In this case, developers may invite users to remove cookies and advise them they may be affected by a security issue. Attackers may set malicious cookies via a vulnerable web application, an exposed/vicious browser plugin, a physically compromised device, etc.

> **Warning**
>
> Do not misunderstand the DoS risk. session.use_strict_mode=On is mandatory for general session ID security! All sites are advised to enable session.use_strict_mode.
>
> DoS can only happen when the account is under attack. A JavaScript injection vulnerability in an application represents the most common cause.

Session Data Deletion

Obsolete session data must be inaccessible and deleted. The current session module does not handle this well.

Obsolete session data should be removed as soon as possible. However, active sessions must not be removed instantly. To satisfy those requirements, developers must implement timestamp based session data management by themselves.

Set and manage expiration timestamp in $_SESSION. Prohibit access to outdated session data. When access to obsolete session data is detected, it is advised to remove all authenticated status from the user's sessions and force them to re-authenticate. Access to obsolete session data can represent an attack. To achieve this, developers must keep track of all active sessions of every user.

> **Note** :
>
> Access to an obsolete session can also happen because of an unstable network and/or concurrent access to the website. E.g. the server tried to set a new session ID via a cookie, but the Set-Cookie packet may not have reached the client due to loss of connection. One connection may issue a new session ID by session_regenerate_id(), but another concurrent connection may not have received the new session ID yet. Therefore, developers must prohibit access to obsolete session at a later stage. I.e. timestamp based session management is mandatory.

In summary, session data must not be destroyed with session_regenerate_id() nor session_destroy(), but timestamps must be used to control access to session data. Let session_gc() remove obsolete data from the session data storage.

Session and Locking

Session data is locked to avoid race conditions by default. Locking is mandatory to keep session data consistent across requests.

However, session-locking can be abused by attackers to perform DoS attacks. To mitigate the risk of a DoS attack by session-locking, minimize locks. Use read only sessions when session data does not need to be updated. Use the 'read_and_close' option with session_start(). `session_start(['read_and_close'=>1]);` Close the session as soon as possible after updating $_SESSION by using session_commit().

The current session module does *not* detect any modification of $_SESSION when the session is inactive. It is the developer's responsibility not to modify $_SESSION when the session is inactive.

Active Sessions

Developers should keep track of all active sessions for every user. And notify them of how many active sessions, from which IP (and area), how long it has been active, etc. PHP does not keep track of these. Developers are supposed to do so.

Various ways to implement this exist. One possible implementation is setting up a database that keeps track of the required data and stores any relevant information. Since session data is GCed, developers must take care of the GCed data to maintain the active session database consistency.

One of the simplest implementations is "User ID prefixed session ID" and store the required information in $_SESSION. Many databases posses good performance for selecting string prefix. Developers MAY use session_regenerate_id() and session_create_id() for this.

> **Warning**

> Never employ confidential data as a prefix. If the user ID is confidential, consider using hash_hmac().

> **Warning**
>
> Enabling session.use_strict_mode is mandatory for this setup. Ensure it is enabled. Otherwise, the active session database can be compromised.

Timestamp based session management is mandatory to detect access to obsolete sessions. When access to an obsolete session is detected, authentication flags should be removed from all active sessions of the user. This prevents attackers to keep exploiting stolen sessions.

Session and Auto-login

Developers must not use long life session IDs for auto-login because it increases the risk of stolen sessions. An auto-login feature should be implemented by the developer.

Use a secure one time hash key as an auto-login key using setcookie(). Use a secure hash stronger than SHA-2. E.g. SHA-256 or greater with random data from random_bytes() or */dev/urandom*.

If the user is unauthenticated, check whether the one-time auto-login key is valid or not. In the case it is valid, authenticate the user and set a new secure one-time hash key. An auto-login key must only be used once, i.e. never reuse an auto-login key, always generate a new one.

An auto-login key is a long life authentication key, it should be protected as much as possible. Use path/httponly/secure/SameSite cookie attributes to secure it. I.e. never transmit the auto-login key unless required.

Developer must implement the features that disables auto-login and removes unneeded auto-login key cookie.

CSRF (Cross-Site Request Forgeries) attacks

Sessions and authentication do not protect against CSRF attacks. Developers must implement CSRF protection by themselves.

output_add_rewrite_var() can be used for CSRF protection. Refer to the manual page for more details.

> **Note** :
>
> PHP prior to 7.2.0 uses the same output buffer and INI setting as trans sid. Therefore, use of output_add_rewrite_var() with PHP prior to version 7.2.0 is not advised.

Most web application frameworks support CSRF protection. Refer to the web application framework manual for more details.

As of PHP 7.3 the SameSite attribute for the session cookie can be set. This is an additional measure which can mitigate CSRF vulnerabilities.

Securing Session INI Settings

By securing session related INI settings, developers can improve session security. Some important INI settings do not have any recommended settings. Developers are responsible for hardening session settings.

- session.cookie_lifetime=0

  `0` possesses a particular meaning. It informs browsers not to store the cookie to permanent storage. Therefore, when the browser is terminated, the session ID cookie is deleted immediately. If developers set this other than 0, it may allow other users to use the session ID. Most applications should use "`0`" for this.

  If an auto-login feature is required, developers must implement their own secure auto-login feature. Do not use long life session IDs for this. More information can be found above in the relevant section.

- session.use_cookies=On

  session.use_only_cookies=On

  Although HTTP cookies suffer some problems, cookies remain the preferred way to manage session IDs. Only use cookies for session ID management when it is possible. Most applications should use a cookie for the session ID.

If session.use_only_cookies=Off, the session module will use the session ID values set by GET or POST provided the session ID cookie is uninitialized.

- session.use_strict_mode=On

  Although, enabling session.use_strict_mode is mandatory for secure sessions. It is disabled by default.

  This prevents the session module to use an uninitialized session ID. Put differently, the session module only accepts valid session IDs generated by the session module. It rejects any session ID supplied by users.

  Due to the cookie specification, attackers are capable to place non-removable session ID cookies by locally setting a cookie database or JavaScript injections. session.use_strict_mode can prevent an attacker-initialized session ID of being used.

  > **Note** :
  >
  > Attackers may initialize a session ID with their device and may set the session ID of the victim. They must keep the session ID active to abuse. Attackers require additional steps to perform an attack in this scenario. Therefore, session.use_strict_mode works as a mitigation.

- session.cookie_httponly=On

  Refuses access to the session cookie from JavaScript. This setting prevents cookies snatched by a JavaScript injection.

  It is possible to use a session ID as a CSRF token, but this is not recommended. For example, HTML sources may be saved and sent to other users. Developers should not write session IDs in web pages for better security. Almost all applications must use the httponly attribute for the session ID cookie.

  > **Note** :
  >
  > The CSRF token should be renewed periodically just like the session ID.

- session.cookie_secure=On

  Allow access to the session ID cookie only when the protocol is HTTPS. If a website is only accessible via HTTPS, it should enable this setting.

  HSTS should be considered for websites accessible only via HTTPS.

- session.cookie_samesite="Lax" or session.cookie_samesite="Strict"

  As of PHP 7.3 the `"SameSite"` attribute can be set for the session ID cookie. This attribute is a way to mitigate CSRF (Cross Site Request Forgery) attacks.

  The difference between Lax and Strict is the accessibility of the cookie in requests originating from another registrable domain employing the HTTP GET method. Cookies using Lax will be accessible in a GET request originated from another registrable domain, whereas cookies using Strict will not.

- session.gc_maxlifetime=[choose smallest possible]

  session.gc_maxlifetime is a setting for deleting obsolete session ID. Reliance on this setting is *not* recommended. Developers should manage the lifetime of sessions with a timestamp by themselves.

  Session GC (garbage collection) is best performed by using session_gc(). The session_gc() function should be executed by a task managers. E.g. cron on UNIX like systems.

  GC is performed by probability by default. This setting does *not* guarantee that an outdated session is deleted. Although developers cannot rely on this setting, specifying it to the smallest possible value is recommended. Adjusting session.gc_probability and session.gc_divisor so that obsolete sessions are deleted at an appropriate frequency. If an auto-login feature is required, developers must implement their own secure auto-login feature, see above for more information. Never use long life session ID for this feature.

  > **Note** :
  >
  > Some session save handler modules do not use this setting for probability based expiration. E.g. memcached, memcache. Refer to the session save handler documentation for details.

- session.use_trans_sid=Off

  Disabling transparent session ID management improves the general session ID security by eliminating the possibility of a session ID injection and/or leak.

  > **Warning**
  >
  > Enabling `session.use_trans_sid` is deprecated as of PHP 8.4.0.

  > **Note** :
  >
  > Session ID may leak from bookmarked URLs, e-mailed URLs, saved HTML source, etc.

- session.trans_sid_tags="a=href,area=href,frame=src,form="

  Unneeded HTML tags should not be rewritten. The default value is sufficient for most usages. Older PHP versions use url_rewriter.tags instead.

  > **Warning**
  >
  > Changing `session.trans_sid_tags` from its default value is deprecated as of PHP 8.4.0.

- session.trans_sid_hosts=""

  This setting lists the hosts for which the session ID may be added to URLs. Untrusted hosts must never be added. When it is empty, the session module only allows `$_SERVER['HTTP_HOST']`.

  > **Warning**
  >
  > Setting `session.trans_sid_hosts` to a non-empty value is deprecated as of PHP 8.4.0.

- session.referer_check=""

  When session.use_trans_sid is enabled, a non-empty value reduces the risk of session ID injection: the session ID sent by the client is discarded unless the `Referer` header contains that value. Note that with HTTPS browsers will not send the `Referer` header, and browsers may omit it by configuration. Therefore, this setting is not a reliable security measure.

  > **Warning**
  >
  > Setting `session.referer_check` to a non-empty value is deprecated as of PHP 8.4.0.

- session.cache_limiter=nocache

  Ensure HTTP content are uncached for authenticated sessions. Allow caching only when the content is not private. Otherwise, content may be exposed. `"private"` may be employed if HTTP content does not include security sensitive data. Note that `"private"` may transmit private data cached by shared clients. `"public"` must only be used when HTTP content does not contain any private data at all.

- session.save_path=[non world-readable directory]

  If this is set to a world-readable directory, such as */tmp* (the default), other users on the server may be able to hijack sessions by getting the list of files in that directory.

Session Functions

Table of Contents

session_abort

session_abort

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

session_abort — Discard session array changes and finish session

Description

function **session_abort**(): bool

session_abort() finishes session without saving data. Thus the original values in session data are kept.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
7.2.0	The return type of this function is bool now. Formerly, it has been void.

See Also

- *$_SESSION*
- The session.auto_start configuration directive
- session_start() - Start new or resume existing session
- session_reset() - Re-initialize session array with original values
- session_commit() - Alias of session_write_close

session_cache_expire

session_cache_expire

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

session_cache_expire — Get and/or set current cache expire

Description

function **session_cache_expire**(?int `$value` = `null`): int|false

session_cache_expire() returns the current setting of `session.cache_expire`.

The cache expire is reset to the default value of 180 stored in session.cache_expire at request startup time. Thus, you need to call **session_cache_expire()** for every request (and before session_start() is called).

Parameters

`value`

If `value` is given and not `null`, the current cache expire is replaced with `value`.

> **Note** :
>
> Setting `value` is of value only, if `session.cache_limiter` is set to a value *different* from `nocache`.

Return Values

Returns the current setting of `session.cache_expire`. The value returned should be read in minutes, defaults to 180. On failure to change the value, `false` is returned.

Changelog

Version	Description
8.0.0	`value` is nullable now.

Examples

Example #1 session_cache_expire() example

```php
<?php

/* set the cache limiter to 'private' */

session_cache_limiter('private');
$cache_limiter = session_cache_limiter();

/* set the cache expire to 30 minutes */
session_cache_expire(30);
$cache_expire = session_cache_expire();

/* start the session */

session_start();

echo "The cache limiter is now set to $cache_limiter<br />";
echo "The cached session pages expire after $cache_expire minutes";
?>
```

See Also

- session.cache_expire
- session.cache_limiter
- session_cache_limiter() - Get and/or set the current cache limiter

session_cache_limiter

session_cache_limiter

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

session_cache_limiter — Get and/or set the current cache limiter

Description

function **session_cache_limiter**(?string `$value` = `null`): string|false

session_cache_limiter() returns the name of the current cache limiter.

The cache limiter defines which cache control HTTP headers are sent to the client. These headers determine the rules by which the page content may be cached by the client and intermediate proxies. Setting the cache limiter to `nocache` disallows any client/proxy caching. A value of `public` permits caching by proxies and the client, whereas `private` disallows caching by proxies and permits the client to cache the contents.

In `private` mode, the Expire header sent to the client may cause confusion for some browsers, including Mozilla. You can avoid this problem by using `private_no_expire` mode. The `Expire` header is never sent to the client in this mode.

Setting the cache limiter to `''` will turn off automatic sending of cache headers entirely.

The cache limiter is reset to the default value stored in session.cache_limiter at request startup time. Thus, you need to call **session_cache_limiter()** for every request (and before session_start() is called).

Parameters

`value`

If `value` is specified and not `null`, the name of the current cache limiter is changed to the new value.

Possible values

Value	Headers sent

`public`	`Expires: (sometime in the future, according session.cache_expire)`
	`Cache-Control: public, max-age=(sometime in the future, according to session.cache_expire)`
	`Last-Modified: (the timestamp of the current script)`

Value	Headers sent
private_no_expire	`Cache-Control: private, max-age=(session.cache_expire in the future)` `Last-Modified: (the timestamp of the current script)`

private	`Expires: Thu, 19 Nov 1981 08:52:00 GMT`
	`Cache-Control: private, max-age=(session.cache_expire in the future)`
	`Last-Modified: (the timestamp of the current script)`

nocache	`Expires: Thu, 19 Nov 1981 08:52:00 GMT`
	`Cache-Control: no-store, no-cache, must-revalidate`
	`Pragma: no-cache`

Return Values

Returns the name of the current cache limiter. On failure to change the value, `false` is returned.

Changelog

Version	Description
8.0.0	`value` is nullable now.

Examples

Example #1 session_cache_limiter() example

```php
<?php

/* set the cache limiter to 'private' */

session_cache_limiter('private');
$cache_limiter = session_cache_limiter();

echo "The cache limiter is now set to $cache_limiter<br />";
?>
```

See Also

- session.cache_limiter

session_commit

session_commit

(PHP 4 >= 4.4.0, PHP 5, PHP 7, PHP 8)

session_commit — Alias of session_write_close()

Description

This function is an alias of: session_write_close().

session_create_id

session_create_id

(PHP 7 >= 7.1.0, PHP 8)

session_create_id — Create new session id

Description

function **session_create_id**(string `$prefix = ""`): string|false

session_create_id() is used to create new session id for the current session. It returns collision free session id.

If session is not active, collision check is omitted.

Session ID is created according to php.ini settings.

It is important to use the same user ID of your web server for GC task script. Otherwise, you may have permission problems especially with files save handler.

Parameters

`prefix`

> If `prefix` is specified, new session id is prefixed by `prefix`. Not all characters are allowed within the session id. Characters in the range `[a-zA-Z0-9,-]` are allowed. Maximum length is 256 characters.

Return Values

session_create_id() returns new collision free session id for the current session. If it is used without active session, it omits collision check. On failure, `false` is returned.

Examples

Example #1 session_create_id() example with session_regenerate_id()

```php
<?php
// My session start function support timestamp management
function my_session_start() {
    session_start();
    // Do not allow to use too old session ID
    if (!empty($_SESSION['deleted_time']) && $_SESSION['deleted_time'] < time() - 180) {
        session_destroy();
        session_start();
    }
}

// My session regenerate id function
function my_session_regenerate_id() {
    // Call session_create_id() while session is active to
    // make sure collision free.
    if (session_status() != PHP_SESSION_ACTIVE) {
        session_start();
    }
    // WARNING: Never use confidential strings for prefix!
    $newid = session_create_id('myprefix-');
    // Set deleted timestamp. Session data must not be deleted immediately for reasons.
    $_SESSION['deleted_time'] = time();
    // Finish session
    session_commit();
    // Make sure to accept user defined session ID
    // NOTE: You must enable use_strict_mode for normal operations.
    ini_set('session.use_strict_mode', 0);
    // Set new custom session ID
    session_id($newid);
    // Start with custom session ID
    session_start();
}

// Make sure use_strict_mode is enabled.
// use_strict_mode is mandatory for security reasons.
ini_set('session.use_strict_mode', 1);
my_session_start();

// Session ID must be regenerated when
//  - User logged in
//  - User logged out
//  - Certain period has passed
my_session_regenerate_id();

// Write useful codes
?>
```

See Also

- session_regenerate_id() - Update the current session id with a newly generated one
- session_start() - Start new or resume existing session
- session.use_strict_mode
- SessionHandler::create_sid() - Return a new session ID

session_decode

session_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

session_decode — Decodes session data from a session encoded string

Description

function **session_decode**(string `$data`): bool

session_decode() decodes the serialized session data provided in `data`, and populates the $_SESSION superglobal with the result.

By default, the unserialization method used is internal to PHP, and is not the same as unserialize(). The serialization method can be set using session.serialize_handler.

Parameters

`data`

The encoded data to be stored.

Return Values

Returns `true` on success or `false` on failure.

See Also

- session_encode() - Encodes the current session data as a session encoded string
- session.serialize_handler

session_destroy

session_destroy

(PHP 4, PHP 5, PHP 7, PHP 8)

session_destroy — Destroys all data registered to a session

Description

function **session_destroy**(): bool

session_destroy() destroys all of the data associated with the current session. It does not unset any of the global variables associated with the session, or unset the session cookie. To use the session variables again, session_start() has to be called.

> **Note** :
>
> You do not have to call **session_destroy()** from usual code. Cleanup $_SESSION array rather than destroying session data.

In order to kill the session altogether, the session ID must also be unset. If a cookie is used to propagate the session ID (default behavior), then the session cookie must be deleted. setcookie() may be used for that.

When session.use_strict_mode is enabled. You do not have to remove obsolete session ID cookie because session module will not accept session ID cookie when there is no data associated to the session ID and set new session ID cookie. Enabling session.use_strict_mode is recommended for all sites.

> **Warning**
>
> Immediate session deletion may cause unwanted results. When there is concurrent requests, other connections may see sudden session data loss. e.g. Requests from JavaScript and/or requests from URL links.

> Although current session module does not accept empty session ID cookie, but immediate session deletion may result in empty session ID cookie due to client(browser) side race condition. This will result that the client creates many session ID needlessly.
>
> To avoid these, you must set deletion time-stamp to $_SESSION and reject access while later. Or make sure your application does not have concurrent requests. This applies to session_regenerate_id() also.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Destroying a session with *$_SESSION*

```php
<?php
// Initialize the session.
// If you are using session_name("something"), don't forget it now!
session_start();

// Unset all of the session variables.
$_SESSION = array();

// If it's desired to kill the session, also delete the session cookie.
// Note: This will destroy the session, and not just the session data!
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

// Finally, destroy the session.
session_destroy();
?>
```

See Also

- session.use_strict_mode
- session_reset() - Re-initialize session array with original values
- session_regenerate_id() - Update the current session id with a newly generated one
- unset() - unset a given variable
- setcookie() - Send a cookie

session_encode

session_encode

(PHP 4, PHP 5, PHP 7, PHP 8)

session_encode — Encodes the current session data as a session encoded string

Description

function **session_encode**(): string|false

session_encode() returns a serialized string of the contents of the current session data stored in the $_SESSION superglobal.

By default, the serialization method used is internal to PHP, and is not the same as serialize(). The serialization method can be set using session.serialize_handler.

Parameters

This function has no parameters.

Return Values

Returns the contents of the current session encoded, or `false` on failure.

Notes

> **Warning**
>
> Must call session_start() before using **session_encode()**.

See Also

- session_decode() - Decodes session data from a session encoded string
- session.serialize_handler

session_gc

session_gc

(PHP 7 >= 7.1.0, PHP 8)

session_gc — Perform session data garbage collection

Description

function **session_gc**(): int|false

By default, PHP uses session.gc_probability to run the session garbage collector probabilistically on each request. There are some limitations with this approach:

- Low traffic sites may not have their session data deleted within the preferred duration.
- High traffic sites may have the garbage collector run too frequently, performing unnecessary extra work.
- Garbage collection is performed on the user's request, and the user may experience a delay.

For production systems, it is recommended to disable the probability-based garbage collection by setting session.gc_probability to `0` and explicitly trigger the garbage collector periodically, for example by using "cron" on UNIX-like systems to run a script that calls **session_gc()**.

> **Note** :
>
> When calling **session_gc()** from a command-line php script, the session.save_path must be set to the same value as web requests, and the script must have access and delete permissions for the session files. This may be affected by the user the script runs as, and container or sandboxing features such as systemd's `PrivateTmp=` option.

Parameters

This function has no parameters.

Return Values

session_gc() returns the number of deleted session entries on success, or `false` on failure.

> **Note** :
>
> Old session save handlers do not return the number of deleted session entries, but rather only a success/failure flag. If this is the case, `1` is returned regardless of how many session entries are actually deleted.

Examples

Example #1 session_gc() example for task managers like cron

```
<?php
// Note: This script should be executed by the same user of web server process.

// Need active session to initialize session data storage access.
```

```php
session_start();

// Executes GC immediately
session_gc();

// Clean up session ID created by session_start()
session_destroy();
?>
```

Example #2 session_gc() example for user accessible script

```php
<?php
// Note: session_gc() is recommended to be used by a task manager script, but
// it may be used as follows.

// Used for last GC time check
$gc_time = '/tmp/php_session_last_gc';
$gc_period = 1800;

session_start();
// Execute GC only when GC period elapsed.
// i.e. Calling session_gc() every request is waste of resources.
if (file_exists($gc_time)) {
    if (filemtime($gc_time) < time() - $gc_period) {
        session_gc();
        touch($gc_time);
    }
} else {
    touch($gc_time);
}
?>
```

See Also

- session_start() - Start new or resume existing session
- session_destroy() - Destroys all data registered to a session
- session.gc_probability

session_get_cookie_params

session_get_cookie_params

(PHP 4, PHP 5, PHP 7, PHP 8)

session_get_cookie_params — Get the session cookie parameters

Description

function **session_get_cookie_params**(): array

Gets the session cookie parameters.

Parameters

This function has no parameters.

Return Values

Returns an array with the current session cookie information, the array contains the following items:

- "lifetime" - The lifetime of the cookie in seconds.
- "path" - The path where information is stored.
- "domain" - The domain of the cookie.
- "secure" - The cookie should only be sent over secure connections.
- "partitioned" - The cookie is marked as Partitioned (CHIPS).
- "httponly" - The cookie can only be accessed through the HTTP protocol.

- "samesite" - Controls the cross-domain sending of the cookie.

Changelog

Version	Description
8.5.0	The "partitioned" entry was added in the returned array.
7.3.0	The "samesite" entry was added in the returned array.

See Also

- session.cookie_lifetime
- session.cookie_path
- session.cookie_domain
- session.cookie_secure
- session.cookie_httponly
- session.cookie_partitioned
- session.cookie_samesite
- session_set_cookie_params() - Set the session cookie parameters

session_id

session_id

(PHP 4, PHP 5, PHP 7, PHP 8)

session_id — Get and/or set the current session id

Description

function **session_id**(?string `$id` = `null`): string|false

session_id() is used to get or set the session id for the current session.

The constant `SID` can also be used to retrieve the current name and session id as a string suitable for adding to URLs. See also Session handling.

Parameters

`id`

If `id` is specified and not `null`, it will replace the current session id. **session_id()** needs to be called before session_start() for that purpose. Depending on the session handler, not all characters are allowed within the session id. For example, the file session handler only allows characters in the range `[a-zA-Z0-9,-]`!

> **Note** :
>
> When using session cookies, specifying an `id` for **session_id()** will always send a new cookie when session_start() is called, regardless if the current session id is identical to the one being set.

Return Values

session_id() returns the session id for the current session or the empty string (`""`) if there is no current session (no current session id exists). On failure, `false` is returned.

Changelog

Version	Description
8.0.0	`id` is nullable now.

See Also

- session_regenerate_id() - Update the current session id with a newly generated one
- session_start() - Start new or resume existing session
- session_set_save_handler() - Sets user-level session storage functions
- session.save_handler

session_module_name

session_module_name

(PHP 4, PHP 5, PHP 7, PHP 8)

session_module_name — Get and/or set the current session module

Description

function **session_module_name**(?string `$module = null`): string|false

session_module_name() gets the name of the current session module, which is also known as session.save_handler.

Parameters

`module`

> If `module` is specified and not `null`, that module will be used instead. Passing `"user"` to this parameter is forbidden. Instead session_set_save_handler() has to be called to set a user defined session handler.

Return Values

Returns the name of the current session module, or `false` on failure.

Changelog

Version	Description
8.0.0	`module` is nullable now.
7.2.0	It is now explicitly forbidden to set the module name to `"user"`. Formerly, this has been silently ignored.

session_name

session_name

(PHP 4, PHP 5, PHP 7, PHP 8)

session_name — Get and/or set the current session name

Description

function **session_name**(?string `$name = null`): string|false

session_name() returns the name of the current session. If `name` is given, **session_name()** will update the session name and return the *old* session name.

If a new session `name` is supplied, **session_name()** modifies the HTTP cookie (and outputs the content when session.use_trans_sid is enabled). Once the HTTP cookie has been sent, calling **session_name()** raises an `E_WARNING`. **session_name()** must be called before session_start() for the session to work properly.

The session name is reset to the default value stored in `session.name` at request startup time. Thus, you need to call **session_name()** for every request (and before session_start() is called).

Parameters

`name`

> The session name references the name of the session, which is used in cookies and URLs (e.g. `PHPSESSID`). It should contain only alphanumeric characters; it should be short and descriptive (i.e. for users with enabled cookie warnings). If `name` is specified and not `null`, the name of the current session is changed to its value.
>
> > **Warning**
> >
> > The session name can't consist of digits only, at least one letter must be present. Otherwise a new session id is generated every time.

Return Values

Returns the name of the current session. If `name` is given and function updates the session name, name of the *old* session is returned, or `false` on failure.

Changelog

Version	Description
8.0.0	`name` is nullable now.
7.2.0	**session_name()** checks session status, previously it only checked cookie status. Therefore, older **session_name()** allows to call **session_name()** after session_start() which may crash PHP and may result in misbehaviors.

Examples

Example #1 session_name() example

```php
<?php

/* set the session name to WebsiteID */

$previous_name = session_name("WebsiteID");

echo "The previous session name was $previous_name<br />";
?>
```

See Also

- The session.name configuration directive

session_regenerate_id

session_regenerate_id

(PHP 4 >= 4.3.2, PHP 5, PHP 7, PHP 8)

session_regenerate_id — Update the current session id with a newly generated one

Description

function **session_regenerate_id**(bool `$delete_old_session = false`): bool

session_regenerate_id() will replace the current session id with a new one, and keep the current session information.

When session.use_trans_sid is enabled, output must be started after **session_regenerate_id()** call. Otherwise, old session ID is used.

> **Warning**
>
> Currently, session_regenerate_id does not handle an unstable network well, e.g. Mobile and WiFi network. Therefore, you may experience a lost session by calling session_regenerate_id.
>
> You should not destroy old session data immediately, but should use destroy time-stamp and control access to old session ID. Otherwise, concurrent access to page may result in inconsistent state, or you may have lost session, or it may cause client (browser) side race condition and may create many session ID needlessly. Immediate session data deletion disables session hijack attack detection and prevention also.

Parameters

`delete_old_session`

Whether to delete the old associated session file or not. You should not delete old session if you need to avoid races caused by deletion or detect/avoid session hijack attacks.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 A session_regenerate_id() example

```php
<?php
// NOTE: This code is not fully working code, but an example!

session_start();

// Check destroyed time-stamp
if (isset($_SESSION['destroyed'])
    && $_SESSION['destroyed'] < time() - 300) {
    // Should not happen usually. This could be attack or due to unstable network.
    // Remove all authentication status of this users session.
    remove_all_authentication_flag_from_active_sessions($_SESSION['userid']);
    throw(new DestroyedSessionAccessException);
}

$old_sessionid = session_id();

// Set destroyed timestamp
$_SESSION['destroyed'] = time(); // session_regenerate_id() saves old session data

// Simply calling session_regenerate_id() may result in lost session, etc.
// See next example.
session_regenerate_id();

// New session does not need destroyed timestamp
unset($_SESSION['destroyed']);

$new_sessionid = session_id();

echo "Old Session: $old_sessionid<br />";
echo "New Session: $new_sessionid<br />";

print_r($_SESSION);
?>
```

Current session module does not handle unstable network well. You should manage session ID to avoid lost session by session_regenerate_id.

Example #2 Avoiding lost session by session_regenerate_id()

```php
<?php
// NOTE: This code is not fully working code, but an example!
// my_session_start() and my_session_regenerate_id() avoid lost sessions by
// unstable network. In addition, this code may prevent exploiting stolen
// session by attackers.

function my_session_start() {
    session_start();
    if (isset($_SESSION['destroyed'])) {
       if ($_SESSION['destroyed'] < time()-300) {
           // Should not happen usually. This could be attack or due to unstable network.
           // Remove all authentication status of this users session.
           remove_all_authentication_flag_from_active_sessions($_SESSION['userid']);
           throw(new DestroyedSessionAccessException);
       }
       if (isset($_SESSION['new_session_id'])) {
           // Not fully expired yet. Could be lost cookie by unstable network.
           // Try again to set proper session ID cookie.
           // NOTE: Do not try to set session ID again if you would like to remove
           // authentication flag.
           session_commit();
           session_id($_SESSION['new_session_id']);
           // New session ID should exist
           session_start();
           return;
       }
   }
}

function my_session_regenerate_id() {
    // New session ID is required to set proper session ID
    // when session ID is not set due to unstable network.
```

```
    $new_session_id = session_create_id();
    $_SESSION['new_session_id'] = $new_session_id;

    // Set destroy timestamp
    $_SESSION['destroyed'] = time();

    // Write and close current session;
    session_commit();

    // Start session with new session ID
    session_id($new_session_id);
    ini_set('session.use_strict_mode', 0);
    session_start();
    ini_set('session.use_strict_mode', 1);

    // New session does not need them
    unset($_SESSION['destroyed']);
    unset($_SESSION['new_session_id']);
}
?>
```

See Also

- session_id() - Get and/or set the current session id
- session_create_id() - Create new session id
- session_start() - Start new or resume existing session
- session_destroy() - Destroys all data registered to a session
- session_reset() - Re-initialize session array with original values
- session_name() - Get and/or set the current session name

session_register_shutdown

session_register_shutdown

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

session_register_shutdown — Session shutdown function

Description

function **session_register_shutdown**(): void

Registers session_write_close() as a shutdown function.

Parameters

This function has no parameters.

Return Values

No value is returned.

Errors/Exceptions

Emits `E_WARNING` if registering the shutdown function fails.

session_reset

session_reset

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

session_reset — Re-initialize session array with original values

Description

function **session_reset**(): bool

session_reset() reinitializes a session with original values stored in session storage. This function requires an active session and discards changes in $_SESSION.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
7.2.0	The return type of this function is bool now. Formerly, it has been void.

See Also

- *$_SESSION*
- The session.auto_start configuration directive
- session_start() - Start new or resume existing session
- session_abort() - Discard session array changes and finish session
- session_commit() - Alias of session_write_close

session_save_path

session_save_path

(PHP 4, PHP 5, PHP 7, PHP 8)

session_save_path — Get and/or set the current session save path

Description

function **session_save_path**(?string `$path` = `null`): string|false

session_save_path() returns the path of the current directory used to save session data.

Parameters

`path`

Session data path. If specified and not `null`, the path to which data is saved will be changed. **session_save_path()** needs to be called before session_start() for that purpose.

> **Note** :
>
> On some operating systems, you may want to specify a path on a filesystem that handles lots of small files efficiently.

Return Values

Returns the path of the current directory used for data storage, or `false` on failure.

Changelog

Version	Description
8.0.0	`path` is nullable now.

See Also

- The session.save_path configuration directive

session_set_cookie_params

session_set_cookie_params

(PHP 4, PHP 5, PHP 7, PHP 8)

session_set_cookie_params — Set the session cookie parameters

Description

function **session_set_cookie_params**(
  int $lifetime_or_options,
  ?string $path = null,
  ?string $domain = null,
  ?bool $secure = null,
  ?bool $httponly = null
): bool

Alternative signature available as of PHP 7.3.0:

function **session_set_cookie_params**(array $lifetime_or_options): bool

Set cookie parameters defined in the *php.ini* file. The effect of this function only lasts for the duration of the script. Thus, you need to call **session_set_cookie_params()** for every request and before session_start() is called.

This function updates the runtime ini values of the corresponding PHP ini configuration keys which can be retrieved with the ini_get().

Parameters

`lifetime_or_options`

When using the first signature, lifetime of the session cookie, defined in seconds.

When using the second signature, an associative array which may have any of the keys `lifetime`, `path`, `domain`, `secure`, `httponly`, `partitioned` and `samesite`. The values have the same meaning as described for the parameters with the same name. The value of the `samesite` element should be either `Lax` or `Strict`. If any of the allowed options are not given, their default values are the same as the default values of the explicit parameters. If the `samesite` element is omitted, no SameSite cookie attribute is set.

`path`

Path on the domain where the cookie will work. Use a single slash ('/') for all paths on the domain.

`domain`

Cookie domain, for example 'www.php.net'. To make cookies visible on all subdomains then the domain must be prefixed with a dot like '.php.net'.

`secure`

If **`true`** cookie will only be sent over secure connections.

`httponly`

If set to **`true`** then PHP will attempt to send the httponly flag when setting the session cookie.

Return Values

Returns **`true`** on success or **`false`** on failure.

Changelog

Version	Description
8.5.0	The "partitioned" entry was added in the `lifetime_or_options` array. If set to `true`, the cookie will be marked as Partitioned (CHIPS).
8.0.0	`path`, `domain`, `secure` and `httponly` are nullable now.
7.3.0	An alternative signature supporting an `lifetime_or_options` array has been added. This signature supports also setting of the SameSite cookie attribute.
7.2.0	Returns **`true`** on success or **`false`** on failure. Formerly the function returned void.

See Also

- session.cookie_lifetime
- session.cookie_path
- session.cookie_domain
- session.cookie_secure

- session.cookie_httponly
- session.cookie_partitioned
- session.cookie_samesite
- session_get_cookie_params() - Get the session cookie parameters

session_set_save_handler

session_set_save_handler

(PHP 4, PHP 5, PHP 7, PHP 8)

session_set_save_handler — Sets user-level session storage functions

Description

function **session_set_save_handler**(
  callable `$open`,
  callable `$close`,
  callable `$read`,
  callable `$write`,
  callable `$destroy`,
  callable `$gc`,
  callable `$create_sid = ?`,
  callable `$validate_sid = ?`,
  callable `$update_timestamp = ?`
): bool

It is possible to register the following prototype:

function **session_set_save_handler**(object `$sessionhandler`, bool `$register_shutdown = true`): bool

session_set_save_handler() sets the user-level session storage functions which are used for storing and retrieving data associated with a session. This is most useful when a storage method other than those supplied by PHP sessions is preferred, e.g. storing the session data in a local database.

Parameters

This function has two prototypes.

`sessionhandler`

An instance of a class implementing SessionHandlerInterface, and optionally SessionIdInterface and/or SessionUpdateTimestampHandlerInterface, such as SessionHandler, to register as the session handler.

`register_shutdown`

Register session_write_close() as a register_shutdown_function() function.

or

`open`

A callable with the following signature:

function open(string `$savePath`, string `$sessionName`): bool

The open callback works like a constructor in classes and is executed when the session is being opened. It is the first callback function executed when the session is started automatically or manually with session_start(). Return value is `true` for success, `false` for failure.

`close`

A callable with the following signature:

function close(): bool

The close callback works like a destructor in classes and is executed after the session write callback has been called. It is also invoked when session_write_close() is called. Return value should be `true` for success, `false` for failure.

`read`

A callable with the following signature:

function read(string `$sessionId`): string

The `read` callback must always return a session encoded (serialized) string, or an empty string if there is no data to read.

This callback is called internally by PHP when the session starts or when session_start() is called. Before this callback is invoked PHP will invoke the `open` callback.

The value this callback returns must be in exactly the same serialized format that was originally passed for storage to the `write` callback. The value returned will be unserialized automatically by PHP and used to populate the *$_SESSION* superglobal. While the data looks similar to serialize() please note it is a different format which is specified in the session.serialize_handler ini setting.

`write`

A callable with the following signature:

function write(string `$sessionId`, string `$data`): bool

The `write` callback is called when the session needs to be saved and closed. This callback receives the current session ID a serialized version the *$_SESSION* superglobal. The serialization method used internally by PHP is specified in the session.serialize_handler ini setting.

The serialized session data passed to this callback should be stored against the passed session ID. When retrieving this data, the `read` callback must return the exact value that was originally passed to the `write` callback.

This callback is invoked when PHP shuts down or explicitly when session_write_close() is called. Note that after executing this function PHP will internally execute the `close` callback.

> **Note** :
>
> The "write" handler is not executed until after the output stream is closed. Thus, output from debugging statements in the "write" handler will never be seen in the browser. If debugging output is necessary, it is suggested that the debug output be written to a file instead.

`destroy`

A callable with the following signature:

function destroy(string `$sessionId`): bool

This callback is executed when a session is destroyed with session_destroy() or with session_regenerate_id() with the destroy parameter set to **`true`**. Return value should be **`true`** for success, **`false`** for failure.

`gc`

A callable with the following signature:

function gc(int `$lifetime`): bool

The garbage collector callback is invoked internally by PHP periodically in order to purge old session data. The frequency is controlled by session.gc_probability and session.gc_divisor. The value of lifetime which is passed to this callback can be set in session.gc_maxlifetime. Return value should be **`true`** for success, **`false`** for failure.

`create_sid`

A callable with the following signature:

function create_sid(): string

This callback is executed when a new session ID is required. No parameters are provided, and the return value should be a string that is a valid session ID for your handler.

`validate_sid`

A callable with the following signature:

function validate_sid(string $key): bool

This callback is executed when a session is to be started, a session ID is supplied and session.use_strict_mode is enabled. The `key` is the session ID to validate. A session ID is valid, if a session with that ID already exists. The return value should be `true` for success, `false` for failure.

`update_timestamp`

A callable with the following signature:

function update_timestamp(string $key, string $val): bool

This callback is executed when a session is updated. `key` is the session ID, `val` is the session data. The return value should be `true` for success, `false` for failure.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.4.0	Calling **session_set_save_handler()** with more than two arguments is now deprecated. Use the two argument signature instead.

Examples

Example #1 Custom session handler: see full code in SessionHandlerInterface synopsis.

We just show the invocation here, the full example can be seen in the SessionHandlerInterface synopsis linked above.

Note we use the OOP prototype with **session_set_save_handler()** and register the shutdown function using the function's parameter flag. This is generally advised when registering objects as session save handlers.

```
<?php
class MySessionHandler implements SessionHandlerInterface
{
    // implement interfaces here
}

$handler = new MySessionHandler();
session_set_save_handler($handler, true);
session_start();

// proceed to set and retrieve values by key from $_SESSION
```

Notes

> **Warning**
>
> The `write` and `close` handlers are called after object destruction and therefore cannot use objects or throw exceptions. Exceptions are not able to be caught since will not be caught nor will any exception trace be displayed and the execution will just cease unexpectedly. The object destructors can however use sessions.
>
> It is possible to call session_write_close() from the destructor to solve this chicken and egg problem but the most reliable way is to register the shutdown function as described above.

> **Warning**
>
> Current working directory is changed with some SAPIs if session is closed in the script termination. It is possible to close the session earlier with session_write_close().

See Also

- The session.save_handler configuration directive
- The session.serialize_handler configuration directive.
- The register_shutdown_function() - Register a function for execution on shutdown

- The session_register_shutdown() - Session shutdown function
- Refer to » save_handler.inc for a full procedural reference implementation

session_start

session_start

(PHP 4, PHP 5, PHP 7, PHP 8)

session_start — Start new or resume existing session

Description

function **session_start**(array `$options = []`): bool

session_start() creates a session or resumes the current one based on a session identifier passed via a GET or POST request, or passed via a cookie.

When **session_start()** is called or when a session auto starts, PHP will call the open and read session save handlers. These will either be a built-in save handler provided by default or by PHP extensions (such as SQLite or Memcached); or can be custom handler as defined by session_set_save_handler(). The read callback will retrieve any existing session data (stored in a special serialized format) and will be unserialized and used to automatically populate the $_SESSION superglobal when the read callback returns the saved session data back to PHP session handling.

To use a named session, call session_name() before calling **session_start()**.

When session.use_trans_sid is enabled, the **session_start()** function will register an internal output handler for URL rewriting.

If a user uses `ob_gzhandler` or similar with ob_start(), the function order is important for proper output. For example, `ob_gzhandler` must be registered before starting the session.

Parameters

`options`

> If provided, this is an associative array of options that will override the currently set session configuration directives. The keys should not include the `session.` prefix.
>
> In addition to the normal set of configuration directives, a `read_and_close` option may also be provided. If set to **`true`**, this will result in the session being closed immediately after being read, thereby avoiding unnecessary locking if the session data won't be changed.

Return Values

This function returns **`true`** if a session was successfully started, otherwise **`false`**.

Errors/Exceptions

As of PHP 8.5.0, throws a ValueError if `options` has any non-string key.

As of PHP 8.5.0, throws a TypeError if the `read_and_close` option is not of type string, int or bool, or if its value cannot be interpreted as an int.

Changelog

Version	Description
8.5.0	Now throws a ValueError if the `options` array has non-string keys, and a TypeError if the `read_and_close` value is not compatible with int.
7.1.0	**session_start()** now returns **`false`** and no longer initializes *$_SESSION* when it failed to start the session.

Examples

A basic session example

Example #1 *page1.php*

```
<?php
// page1.php
```

```
session_start();

echo 'Welcome to page #1';

$_SESSION['favcolor'] = 'green';
$_SESSION['animal']   = 'cat';
$_SESSION['time']     = time();

// Works if session cookie was accepted
echo '<br /><a href="page2.php">page 2</a>';

// Or maybe pass along the session id, if needed
echo '<br /><a href="page2.php?' . SID . '">page 2</a>';
?>
```

After viewing *page1.php*, the second page *page2.php* will magically contain the session data. Read the session reference for information on propagating session ids as it, for example, explains what the constant **SID** is all about.

Example #2 *page2.php*

```
<?php
// page2.php

session_start();

echo 'Welcome to page #2<br />';

echo $_SESSION['favcolor']; // green
echo $_SESSION['animal'];   // cat
echo date('Y m d H:i:s', $_SESSION['time']);

// You may want to use SID here, like we did in page1.php
echo '<br /><a href="page1.php">page 1</a>';
?>
```

Providing options to **session_start()**

Example #3 Overriding the cookie lifetime

```
<?php
// This sends a persistent cookie that lasts a day.
session_start([
    'cookie_lifetime' => 86400,
]);
?>
```

Example #4 Reading the session and closing it

```
<?php
// If we know we don't need to change anything in the
// session, we can just read and close rightaway to avoid
// locking the session file and blocking other pages
session_start([
    'cookie_lifetime' => 86400,
    'read_and_close'  => true,
]);
```

Notes

> **Note** :
>
> To use cookie-based sessions, **session_start()** must be called before outputting anything to the browser.

> **Note** :

> Use of zlib.output_compression is recommended instead of ob_gzhandler()

> **Note** :
>
> This function sends out several HTTP headers depending on the configuration. See session_cache_limiter() to customize these headers.

See Also

- *$_SESSION*
- The session.auto_start configuration directive
- session_id() - Get and/or set the current session id

session_status

session_status

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

session_status — Returns the current session status

Description

function **session_status**(): int

session_status() is used to return the current session status.

Parameters

This function has no parameters.

Return Values

- `PHP_SESSION_DISABLED` if sessions are disabled.
- `PHP_SESSION_NONE` if sessions are enabled, but none exists.
- `PHP_SESSION_ACTIVE` if sessions are enabled, and one exists.

See Also

- session_start() - Start new or resume existing session

session_unset

session_unset

(PHP 4, PHP 5, PHP 7, PHP 8)

session_unset — Free all session variables

Description

function **session_unset**(): bool

The **session_unset()** function frees all session variables currently registered.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
7.2.0	The return type of this function is bool now. Formerly, it has been void.

Notes

> **Note** :
>
> If *$_SESSION* is used, use unset() to unregister a session variable, i.e. `unset($_SESSION['varname']);`.

> **Caution**
>
> Do NOT unset the whole *$_SESSION* with `unset($_SESSION)` as this will disable the registering of session variables through the *$_SESSION* superglobal.

> **Note** :
>
> The use of **session_unset()** is identical to `$_SESSION = []`.

> **Caution**
>
> This function works only if a session is active. It will not clear the *$_SESSION* array if the session has not been started yet or has already been destroyed. Use `$_SESSION = []` to unset all session variables even if the session is not active.

session_write_close

session_write_close

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

session_write_close — Write session data and end session

Description

function **session_write_close**(): bool

End the current session and store session data.

Session data is usually stored after your script terminated without the need to call **session_write_close()**, but as session data is locked to prevent concurrent writes only one script may operate on a session at any time. When using framesets together with sessions you will experience the frames loading one by one due to this locking. You can reduce the time needed to load all the frames by ending the session as soon as all changes to session variables are done.

Parameters

This function has no parameters.

Return Values

Returns **`true`** on success or **`false`** on failure.

Changelog

Version	Description
8.5.0	If *$_SESSION* contains a key with a pipe character (|), an **`E_WARNING`** is now emitted. Previously, this would silently fail to write the session data.
7.2.0	The return type of this function is bool now. Formerly, it has been void.

See Also

- session_register_shutdown() - Session shutdown function
- session_start() - Start new or resume existing session
- session_abort() - Discard session array changes and finish session

SessionHandler

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introduction

SessionHandler is a special class that can be used to expose the current internal PHP session save handler by inheritance. There are seven methods which wrap the seven internal session save handler callbacks (`open`, `close`, `read`, `write`, `destroy`, `gc` and `create_sid`). By default, this class will wrap whatever internal save handler is set as defined by the session.save_handler configuration directive which is usually `files` by default. Other internal session save handlers are provided by PHP extensions such as SQLite (as `sqlite`), Memcache (as `memcache`), and Memcached (as `memcached`).

When a plain instance of **SessionHandler** is set as the save handler using session_set_save_handler() it will wrap the current save handlers. A class extending from **SessionHandler** allows you to override the methods or intercept or filter them by calls the parent class methods which ultimately wrap the internal PHP session handlers.

This allows you, for example, to intercept the `read` and `write` methods to encrypt/decrypt the session data and then pass the result to and from the parent class. Alternatively one might chose to entirely override a method like the garbage collection callback `gc`.

Because the **SessionHandler** wraps the current internal save handler methods, the above example of encryption can be applied to any internal save handler without having to know the internals of the handlers.

To use this class, first set the save handler you wish to expose using session.save_handler and then pass an instance of **SessionHandler** or one extending it to session_set_save_handler().

Please note that the callback methods of this class are designed to be called internally by PHP and are not meant to be called from user-space code. The return values are equally processed internally by PHP. For more information on the session workflow, please refer to session_set_save_handler().

Class synopsis

class **SessionHandler** implements SessionHandlerInterface, SessionIdInterface {
/* Methods */
public function close(): bool
public function create_sid(): string
public function destroy(string `$id`): bool
public function gc(int `$max_lifetime`): int|false
public function open(string `$path`, string `$name`): bool
public function read(string `$id`): string|false
public function write(string `$id`, string `$data`): bool
}

Notes

> **Warning**
>
> This class is designed to expose the current internal PHP session save handler, if you want to write your own custom save handlers, please implement the SessionHandlerInterface interface instead of extending from **SessionHandler**.

Examples

Example #1 Using SessionHandler to add encryption to internal PHP save handlers.

```php
<?php

 /**
  * decrypt AES 256
  *
  * @param data $edata
  * @param string $password
  * @return decrypted data
  */
function decrypt($edata, $password) {
    $data = base64_decode($edata);
    $salt = substr($data, 0, 16);
    $ct = substr($data, 16);

    $rounds = 3; // depends on key length
    $data00 = $password.$salt;
    $hash = array();
    $hash[0] = hash('sha256', $data00, true);
    $result = $hash[0];
```

```
    for ($i = 1; $i < $rounds; $i++) {
        $hash[$i] = hash('sha256', $hash[$i - 1].$data00, true);
        $result .= $hash[$i];
    }
    $key = substr($result, 0, 32);
    $iv  = substr($result, 32,16);

    return openssl_decrypt($ct, 'AES-256-CBC', $key, true, $iv);
  }

/**
 * crypt AES 256
 *
 * @param data $data
 * @param string $password
 * @return base64 encrypted data
 */
function encrypt($data, $password) {
    // Generate a cryptographically secure random salt using random_bytes()
    $salt = random_bytes(16);

    $salted = '';
    $dx = '';
    // Salt the key(32) and iv(16) = 48
    while (strlen($salted) < 48) {
      $dx = hash('sha256', $dx.$password.$salt, true);
      $salted .= $dx;
    }

    $key = substr($salted, 0, 32);
    $iv  = substr($salted, 32,16);

    $encrypted_data = openssl_encrypt($data, 'AES-256-CBC', $key, true, $iv);
    return base64_encode($salt . $encrypted_data);
}

class EncryptedSessionHandler extends SessionHandler
{
    private $key;

    public function __construct($key)
    {
        $this->key = $key;
    }

    public function read($id)
    {
        $data = parent::read($id);

        if (!$data) {
            return "";
        } else {
            return decrypt($data, $this->key);
        }
    }

    public function write($id, $data)
    {
        $data = encrypt($data, $this->key);

        return parent::write($id, $data);
    }
}

// we'll intercept the native 'files' handler, but will equally work
// with other internal native handlers like 'sqlite', 'memcache' or 'memcached'
// which are provided by PHP extensions.
ini_set('session.save_handler', 'files');

$key = 'secret_string';
$handler = new EncryptedSessionHandler($key);
session_set_save_handler($handler, true);
```

```php
session_start();

// proceed to set and retrieve values by key from $_SESSION
```

> **Note** :
>
> Since this class' methods are designed to be called internally by PHP as part of the normal session workflow, child class calls to parent methods (i.e. the actual internal native handlers) will return `false` unless the session has actually been started (either automatically, or by explicit session_start()). This is important to consider when writing unit tests where the class methods might be invoked manually.

Table of Contents

SessionHandler::close

SessionHandler::close

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::close — Close the session

Description

public function **SessionHandler::close**(): bool

Closes the current session. This method is automatically executed internally by PHP when closing the session, or explicitly via session_write_close() (which first calls the SessionHandler::write()).

This method wraps the internal PHP save handler defined in the session.save_handler ini setting that was set before this handler was activated by session_set_save_handler().

If this class is extended by inheritance, calling the parent `close` method will invoke the wrapper for this method and therefore invoke the associated internal callback. This allows the method to be overridden and or intercepted.

For more information on what this method is expected to do, please refer to the documentation at SessionHandlerInterface::close().

Parameters

This function has no parameters.

Return Values

The return value (usually `true` on success, `false` on failure). Note this value is returned internally to PHP for processing.

SessionHandler::create_sid

SessionHandler::create_sid

(PHP 5 >= 5.5.1, PHP 7, PHP 8)

SessionHandler::create_sid — Return a new session ID

Description

public function **SessionHandler::create_sid**(): string

Generates and returns a new session ID.

Parameters

This function has no parameters.

Return Values

A session ID valid for the default session handler.

See Also

- session_id() - Get and/or set the current session id
- session_create_id() - Create new session id

SessionHandler::destroy

SessionHandler::destroy

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::destroy — Destroy a session

Description

public function **SessionHandler::destroy**(string `$id`): bool

Destroys a session. Called internally by PHP with session_regenerate_id() (assuming the `destroy` is set to **`true`**, by session_destroy() or when session_decode() fails.

This method wraps the internal PHP save handler defined in the session.save_handler ini setting that was set before this handler was set by session_set_save_handler().

If this class is extended by inheritance, calling the parent `destroy` method will invoke the wrapper for this method and therefore invoke the associated internal callback. This allows this method to be overridden and or intercepted and filtered.

For more information on what this method is expected to do, please refer to the documentation at SessionHandlerInterface::destroy().

Parameters

`id`

The session ID being destroyed.

Return Values

The return value (usually **`true`** on success, **`false`** on failure). Note this value is returned internally to PHP for processing.

SessionHandler::gc

SessionHandler::gc

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::gc — Cleanup old sessions

Description

public function **SessionHandler::gc**(int `$max_lifetime`): int|false

Cleans up expired sessions. Called randomly by PHP internally when a session starts or when session_start() is invoked. The frequency this is called is based on the session.gc_divisor and session.gc_probability configuration directives.

This method wraps the internal PHP save handler defined in the session.save_handler ini setting that was set before this handler was set by session_set_save_handler().

If this class is extended by inheritance, calling the parent `gc` method will invoke the wrapper for this method and therefore invoke the associated internal callback. This allows this method to be overridden and or intercepted and filtered.

For more information on what this method is expected to do, please refer to the documentation at SessionHandlerInterface::gc().

Parameters

`max_lifetime`

Sessions that have not updated for the last `max_lifetime` seconds will be removed.

Return Values

Returns the number of deleted sessions on success, or **`false`** on failure. Note this value is returned internally to PHP for processing.

Changelog

Version	Description
7.1.0	Prior to this version, the function returned **`true`** on success.

SessionHandler::open

SessionHandler::open

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::open — Initialize session

Description

public function **SessionHandler::open**(string `$path`, string `$name`): bool

Create new session, or re-initialize existing session. Called internally by PHP when a session starts either automatically or when session_start() is invoked.

This method wraps the internal PHP save handler defined in the session.save_handler ini setting that was set before this handler was set by session_set_save_handler().

If this class is extended by inheritance, calling the parent `open` method will invoke the wrapper for this method and therefore invoke the associated internal callback. This allows this method to be overridden and or intercepted and filtered.

For more information on what this method is expected to do, please refer to the documentation at SessionHandlerInterface::open().

Parameters

`path`

The path where to store/retrieve the session.

`name`

The session name.

Return Values

The return value (usually **`true`** on success, **`false`** on failure). Note this value is returned internally to PHP for processing.

See Also

- The session.auto-start configuration directive.

SessionHandler::read

SessionHandler::read

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::read — Read session data

Description

public function **SessionHandler::read**(string `$id`): string|false

Reads the session data from the session storage, and returns the result back to PHP for internal processing. This method is called automatically by PHP when a session is started (either automatically or explicitly with session_start() and is preceded by an internal call to the SessionHandler::open().

This method wraps the internal PHP save handler defined in the session.save_handler ini setting that was set before this handler was set by session_set_save_handler().

If this class is extended by inheritance, calling the parent `read` method will invoke the wrapper for this method and therefore invoke the associated internal callback. This allows the method to be overridden and or intercepted and filtered (for example, decrypting the `data` value returned by the parent `read` method).

For more information on what this method is expected to do, please refer to the documentation at SessionHandlerInterface::read().

Parameters

`id`

The session id to read data for.

Return Values

Returns an encoded string of the read data. If nothing was read, it must return `false`. Note this value is returned internally to PHP for processing.

See Also

- The session.serialize_handler configuration directive.

SessionHandler::write

SessionHandler::write

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandler::write — Write session data

Description

public function **SessionHandler::write**(string `$id`, string `$data`): bool

Writes the session data to the session storage. Called by normal PHP shutdown, by session_write_close(), or when session_register_shutdown() fails. PHP will call SessionHandler::close() immediately after this method returns.

This method wraps the internal PHP save handler defined in the session.save_handler ini setting that was set before this handler was set by session_set_save_handler().

If this class is extended by inheritance, calling the parent `write` method will invoke the wrapper for this method and therefore invoke the associated internal callback. This allows this method to be overridden and or intercepted and filtered (for example, encrypting the `data` value before sending it to the parent `write` method).

For more information on what this method is expected to do, please refer to the documentation at SessionHandlerInterface::write().

Parameters

`id`

The session id.

`data`

The encoded session data. This data is the result of the PHP internally encoding the *$_SESSION* superglobal to a serialized string and passing it as this parameter. Please note sessions use an alternative serialization method.

Return Values

The return value (usually `true` on success, `false` on failure). Note this value is returned internally to PHP for processing.

See Also

- The session.serialize_handler configuration directive.

SessionHandlerInterface

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introduction

SessionHandlerInterface is an interface which defines the minimal prototype for creating a custom session handler. In order to pass a custom session handler to session_set_save_handler() using its OOP invocation, the class can implement this interface.

Please note the callback methods of this class are designed to be called internally by PHP and are not meant to be called from user-space code.

Interface synopsis

interface **SessionHandlerInterface** {
/* Methods */
public function close(): bool

public function destroy(string $id): bool
public function gc(int $max_lifetime): int|false
public function open(string $path, string $name): bool
public function read(string $id): string|false
public function write(string $id, string $data): bool
}

Examples

Example #1 Example using SessionHandlerInterface

The following example provides file based session storage similar to the PHP sessions default save handler `files`. This example could easily be extended to cover database storage using your favorite PHP supported database engine.

Note we use the OOP prototype with session_set_save_handler() and register the shutdown function using the function's parameter flag. This is generally advised when registering objects as session save handlers.

> **Caution**
>
> For brevity, this example omits input validation. However, the `$id` parameters are actually user supplied values which require proper validation/sanitization to avoid vulnerabilities, such as path traversal issues. *So do not use this example unmodified in production environments.*

```php
<?php
class MySessionHandler implements SessionHandlerInterface
{
    private $savePath;

    public function open($savePath, $sessionName): bool
    {
        $this->savePath = $savePath;
        if (!is_dir($this->savePath)) {
            mkdir($this->savePath, 0777);
        }

        return true;
    }

    public function close(): bool
    {
        return true;
    }

    #[\ReturnTypeWillChange]
    public function read($id)
    {
        return (string) @file_get_contents("$this->savePath/sess_$id");
    }

    public function write($id, $data): bool
    {
        return file_put_contents("$this->savePath/sess_$id", $data) === false ? false : true;
    }

    public function destroy($id): bool
    {
        $file = "$this->savePath/sess_$id";
        if (file_exists($file)) {
            unlink($file);
        }

        return true;
    }

    #[\ReturnTypeWillChange]
    public function gc($maxlifetime)
    {
        foreach (glob("$this->savePath/sess_*") as $file) {
            if (filemtime($file) + $maxlifetime < time() && file_exists($file)) {
```

```
                unlink($file);
            }
        }

        return true;
    }
}

$handler = new MySessionHandler();
session_set_save_handler($handler, true);
session_start();

// proceed to set and retrieve values by key from $_SESSION
```

Table of Contents

SessionHandlerInterface::close

SessionHandlerInterface::close

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::close — Close the session

Description

public function **SessionHandlerInterface::close**(): bool

Closes the current session. This function is automatically executed when closing the session, or explicitly via session_write_close().

Parameters

This function has no parameters.

Return Values

The return value (usually `true` on success, `false` on failure). Note this value is returned internally to PHP for processing.

SessionHandlerInterface::destroy

SessionHandlerInterface::destroy

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::destroy — Destroy a session

Description

public function **SessionHandlerInterface::destroy**(string `$id`): bool

Destroys a session. Called by session_regenerate_id() (with $destroy = `true`), session_destroy() and when session_decode() fails.

Parameters

`id`

The session ID being destroyed.

Return Values

The return value (usually `true` on success, `false` on failure). Note this value is returned internally to PHP for processing.

SessionHandlerInterface::gc

SessionHandlerInterface::gc

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::gc — Cleanup old sessions

Description

public function **SessionHandlerInterface::gc**(int `$max_lifetime`): int|false

Cleans up expired sessions. Called by session_start(), based on session.gc_divisor, session.gc_probability and session.gc_maxlifetime settings.

Parameters

`max_lifetime`

Sessions that have not updated for the last `max_lifetime` seconds will be removed.

Return Values

Returns the number of deleted sessions on success, or **`false`** on failure. Note this value is returned internally to PHP for processing.

Changelog

Version	Description
7.1.0	Prior to this version, the function returned **`true`** on success.

SessionHandlerInterface::open

SessionHandlerInterface::open

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::open — Initialize session

Description

public function **SessionHandlerInterface::open**(string `$path`, string `$name`): bool

Re-initialize existing session, or creates a new one. Called when a session starts or when session_start() is invoked.

Parameters

`path`

The path where to store/retrieve the session.

`name`

The session name.

Return Values

The return value (usually **`true`** on success, **`false`** on failure). Note this value is returned internally to PHP for processing.

See Also

- session_name() - Get and/or set the current session name
- The session.auto-start configuration directive.

SessionHandlerInterface::read

SessionHandlerInterface::read

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::read — Read session data

Description

public function **SessionHandlerInterface::read**(string `$id`): string|false

Reads the session data from the session storage, and returns the results. Called right after the session starts or when session_start() is called. Please note that before this method is called SessionHandlerInterface::open() is invoked.

This method is called by PHP itself when the session is started. This method should retrieve the session data from storage by the session ID provided. The string returned by this method must be in the same serialized format as when originally passed to the

SessionHandlerInterface::write() If the record was not found, return `false`.

The data returned by this method will be decoded internally by PHP using the unserialization method specified in session.serialize_handler. The resulting data will be used to populate the *$_SESSION* superglobal.

Note that the serialization scheme is not the same as unserialize() and can be accessed by session_decode().

Parameters

`id`

The session id.

Return Values

Returns an encoded string of the read data. If nothing was read, it must return `false`. Note this value is returned internally to PHP for processing.

See Also

- The session.serialize_handler configuration directive.

SessionHandlerInterface::write

SessionHandlerInterface::write

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SessionHandlerInterface::write — Write session data

Description

public function **SessionHandlerInterface::write**(string `$id`, string `$data`): bool

Writes the session data to the session storage. Called by session_write_close(), when session_register_shutdown() fails, or during a normal shutdown. Note: SessionHandlerInterface::close() is called immediately after this function.

PHP will call this method when the session is ready to be saved and closed. It encodes the session data from the *$_SESSION* superglobal to a serialized string and passes this along with the session ID to this method for storage. The serialization method used is specified in the session.serialize_handler setting.

Note this method is normally called by PHP after the output buffers have been closed unless explicitly called by session_write_close()

Parameters

`id`

The session id.

`data`

The encoded session data. This data is the result of the PHP internally encoding the *$_SESSION* superglobal to a serialized string and passing it as this parameter. Please note sessions use an alternative serialization method.

Return Values

The return value (usually `true` on success, `false` on failure). Note this value is returned internally to PHP for processing.

See Also

- The session.serialize_handler configuration directive.

SessionIdInterface

(PHP 5 >= 5.5.1, PHP 7, PHP 8)

Introduction

SessionIdInterface is an interface which defines optional methods for creating a custom session handler. In order to pass a custom session handler to session_set_save_handler() using its OOP invocation, the class can implement this interface.

Note that the callback methods of classes implementing this interface are designed to be called internally by PHP and are not meant to be called from user-space code.

Interface synopsis

interface **SessionIdInterface** {
/* Methods */
public function create_sid(): string
}

Table of Contents

SessionIdInterface::create_sid

SessionIdInterface::create_sid

(PHP 5 >= 5.5.1, PHP 7, PHP 8)

SessionIdInterface::create_sid — Create session ID

Description

public function **SessionIdInterface::create_sid**(): string

Creates a new session ID.This function is automatically executed when a new session ID needs to be created.

Parameters

This function has no parameters.

Return Values

The new session ID. Note that this value is returned internally to PHP for processing.

See Also

- SessionHandler::create_sid() - Return a new session ID

SessionUpdateTimestampHandlerInterface

(PHP 7, PHP 8)

Introduction

SessionUpdateTimestampHandlerInterface is an interface which defines optional methods for creating a custom session handler. In order to pass a custom session handler to session_set_save_handler() using its OOP invocation, the class can implement this interface.

Note that the callback methods of classes implementing this interface are designed to be called internally by PHP and are not meant to be called from user-space code.

Class synopsis

interface **SessionUpdateTimestampHandlerInterface** {
/* Methods */
public function updateTimestamp(string `$id`, string `$data`): bool
public function validateId(string `$id`): bool
}

Table of Contents

SessionUpdateTimestampHandlerInterface::updateTimestamp

SessionUpdateTimestampHandlerInterface::updateTimestamp

(PHP 7, PHP 8)

SessionUpdateTimestampHandlerInterface::updateTimestamp — Update timestamp

Description

public function **SessionUpdateTimestampHandlerInterface::updateTimestamp**(string `$id`, string `$data`): bool

Updates the last modification timestamp of the session. This function is automatically executed when a session is updated.

Parameters

`id`

The session ID.

`data`

The session data. The serialized session data may be used to determine whether the session data has changed and therefore should be updated.

Return Values

Returns `true` if the timestamp was updated, `false` otherwise. Note that this value is returned internally to PHP for processing.

SessionUpdateTimestampHandlerInterface::validateId

SessionUpdateTimestampHandlerInterface::validateId

(PHP 7, PHP 8)

SessionUpdateTimestampHandlerInterface::validateId — Validate ID

Description

public function **SessionUpdateTimestampHandlerInterface::validateId**(string `$id`): bool

Validates a given session ID. A session ID is valid, if a session with that ID already exists. This function is automatically executed when a session is to be started, a session ID is supplied and session.use_strict_mode is enabled.

Parameters

`id`

The session ID.

Return Values

Returns `true` for valid ID, `false` otherwise. Note that this value is returned internally to PHP for processing.

Filesystem

Introduction

No external libraries are needed to build this extension, but if you want PHP to support LFS (large files) on Linux, then you need to have a recent glibc and you need compile PHP with the following compiler flags: `-D_LARGEFILE_SOURCE -D_FILE_OFFSET_BITS=64`.

Installing/Configuring

Table of Contents

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Filesystem and Streams Configuration Options

Name	Default	Changeable	Changelog
allow_url_fopen	"1"	INI_SYSTEM	
allow_url_include	"0"	INI_SYSTEM	Deprecated as of PHP 7.4.0.
user_agent	NULL	INI_ALL	
default_socket_timeout	"60"	INI_ALL	
from	""	INI_ALL	
auto_detect_line_endings	"0"	INI_ALL	Deprecated as of PHP 8.1.0.
sys_temp_dir	""	INI_SYSTEM	

Here's a short explanation of the configuration directives.

`allow_url_fopen` bool

This option enables the URL-aware fopen wrappers that enable accessing URL object like files. Default wrappers are provided for the access of remote files using the ftp or http protocol, some extensions like zlib may register additional wrappers.

`allow_url_include` bool

This option allows the use of URL-aware fopen wrappers with the following functions: include, include_once, require, require_once.

> **Note** :
>
> This setting requires allow_url_fopen to be on.

`user_agent` string

Define the user agent for PHP to send.

`default_socket_timeout` int

Default timeout (in seconds) for socket based streams. Specifying a negative value means an infinite timeout.

`from` string

The email address to be used on unauthenticated FTP connections and as the value of From header for HTTP connections, when using the ftp and http wrappers, respectively.

`auto_detect_line_endings` bool

When turned on, PHP will examine the data read by fgets() and file() to see if it is using Unix, MS-Dos or Macintosh line-ending conventions.

This enables PHP to interoperate with Macintosh systems, but defaults to Off, as there is a very small performance penalty when detecting the EOL conventions for the first line, and also because people using carriage-returns as item separators under Unix

systems would experience non-backwards-compatible behaviour.

> **Warning**
>
> This option has been deprecated as of PHP 8.1.0.

`sys_temp_dir` string

Resource Types

The file system uses `streams` as their resource type. Streams are documented in its own reference chapter, Streams.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

`SEEK_SET` (int)

`SEEK_CUR` (int)

`SEEK_END` (int)

`LOCK_SH` (int)

`LOCK_EX` (int)

`LOCK_UN` (int)

`LOCK_NB` (int)

Available `flags` for file()

`FILE_USE_INCLUDE_PATH` (int)

Search for `filename` in include_path.

`FILE_NO_DEFAULT_CONTEXT` (int)

`FILE_APPEND` (int)

Append content to existing file.

`FILE_IGNORE_NEW_LINES` (int)

Strip EOL characters.

`FILE_SKIP_EMPTY_LINES` (int)

Skip empty lines.

`FILE_BINARY` (int)

Binary mode.

> **Warning**
>
> This constant has no effect, and is deprecated as of PHP 8.1.0.

`FILE_TEXT` (int)

Text mode.

> **Warning**
>
> This constant has no effect, and is deprecated as of PHP 8.1.0.

Available `flags` for glob()

`GLOB_AVAILABLE_FLAGS` (int)

All GLOB_* flags combined. Equivalent to 0 | GLOB_BRACE | GLOB_MARK | GLOB_NOSORT | GLOB_NOCHECK | GLOB_NOESCAPE | GLOB_ERR | GLOB_ONLYDIR

GLOB_BRACE (int)

Expands {a,b,c} to match 'a', 'b', or 'c'

> **Note** :
>
> GLOB_BRACE is not available on some non GNU systems, like Solaris or Alpine Linux.

GLOB_ERR (int)

Stop on read errors (like unreadable directories), by default errors are ignored.

GLOB_MARK (int)

Adds a slash (a backslash on Windows) to each directory returned

GLOB_NOCHECK (int)

Return the search pattern if no files matching it were found

GLOB_NOESCAPE (int)

Backslashes do not quote metacharacters

GLOB_NOSORT (int)

Return files as they appear in the directory (no sorting). When this flag is not used, the pathnames are sorted alphabetically

GLOB_ONLYDIR (int)

Return only directory entries which match the pattern

Available `flags` for pathinfo()

PATHINFO_ALL (int)

All parts of the pathinfo returned as an associative array.

PATHINFO_DIRNAME (int)

The path of the directory or file.

PATHINFO_BASENAME (int)

The name of the directory or the name and extension of the file.

PATHINFO_EXTENSION (int)

The extension of the file.

PATHINFO_FILENAME (int)

The name of the file (without the extension) or directory.

Available `scanner_mode`s for parse_ini_file() and parse_ini_string()

INI_SCANNER_NORMAL (int)

Normal INI scanner mode.

INI_SCANNER_RAW (int)

Raw INI scanner mode.

INI_SCANNER_TYPED (int)

Typed INI scanner mode.

Available `flags` for fnmatch()

`FNM_NOESCAPE` (int)

Disable backslash escaping.

`FNM_PATHNAME` (int)

Slash in string only matches slash in the given pattern.

`FNM_PERIOD` (int)

Leading period in string must be exactly matched by period in the given pattern.

`FNM_CASEFOLD` (int)

Caseless match. Part of the GNU extension.

PHP File Upload Constants

`UPLOAD_ERR_CANT_WRITE` (int)

Failed to write file to disk. The value of the constant is `7`.

`UPLOAD_ERR_EXTENSION` (int)

A PHP extension stopped the file upload. PHP does not provide a way to ascertain which extension caused the file upload to stop; examining the list of loaded extensions with phpinfo() may help. The value of the constant is `8`.

`UPLOAD_ERR_FORM_SIZE` (int)

The uploaded file exceeds the *MAX_FILE_SIZE* directive that was specified in the HTML form. The value of the constant is `2`.

`UPLOAD_ERR_INI_SIZE` (int)

The uploaded file exceeds the upload_max_filesize directive in *php.ini*. The value of the constant is `1`.

`UPLOAD_ERR_NO_FILE` (int)

No file was uploaded. The value of the constant is `4`.

`UPLOAD_ERR_NO_TMP_DIR` (int)

Missing a temporary folder. The value of the constant is `6`.

`UPLOAD_ERR_OK` (int)

There is no error, the file uploaded with success. The value of the constant is `0`.

`UPLOAD_ERR_PARTIAL` (int)

The uploaded file was only partially uploaded. The value of the constant is `3`.

Filesystem Functions

See Also

For related functions, see also the Directory and Program Execution sections.

For a list and explanation of the various URL wrappers that can be used as remote files, see also Supported Protocols and Wrappers.

Table of Contents

basename

basename

(PHP 4, PHP 5, PHP 7, PHP 8)

basename — Returns trailing name component of path

Description

function **basename**(string `$path`, string `$suffix = ""`): string

Given a string containing the path to a file or directory, this function will return the trailing name component.

> **Note** :
>
> **basename()** operates naively on the input string, and is not aware of the actual filesystem, or path components such as "`..`".

> **Caution**
>
> **basename()** is locale aware, so for it to see the correct basename with multibyte character paths, the matching locale must be set using the setlocale() function. If `path` contains characters which are invalid for the current locale, the behavior of **basename()** is undefined.

Parameters

`path`

A path.

On Windows, both slash (`/`) and backslash (`\`) are used as directory separator character. In other environments, it is the forward slash (`/`).

`suffix`

If the name component ends in `suffix` this will also be cut off.

Return Values

Returns the base name of the given `path`.

Examples

Example #1 basename() example

```
<?php
echo "1) ".basename("/etc/sudoers.d", ".d").PHP_EOL;
echo "2) ".basename("/etc/sudoers.d").PHP_EOL;
echo "3) ".basename("/etc/passwd").PHP_EOL;
echo "4) ".basename("/etc/").PHP_EOL;
echo "5) ".basename(".").PHP_EOL;
echo "6) ".basename("/");
?>
```

The above example will output:

```
1) sudoers
2) sudoers.d
3) passwd
4) etc
5) .
6)
```

See Also

- dirname() - Returns a parent directory's path
- pathinfo() - Returns information about a file path

chgrp

chgrp

(PHP 4, PHP 5, PHP 7, PHP 8)

chgrp — Changes file group

Description

function **chgrp**(string `$filename`, string|int `$group`): bool

Attempts to change the group of the file `filename` to `group`.

Only the superuser may change the group of a file arbitrarily; other users may change the group of a file to any group of which that user is a member.

Parameters

`filename`

Path to the file.

`group`

A group name or number.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Changing a file's group

```php
<?php
$filename = 'shared_file.txt';
$format = "%s's Group ID @ %s: %d\n";
printf($format, $filename, date('r'), filegroup($filename));
chgrp($filename, 8);
clearstatcache(); // do not cache filegroup() results
printf($format, $filename, date('r'), filegroup($filename));
?>
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

> **Note** :
>
> On Windows, this function fails silently when applied on a regular file.

See Also

- chown() - Changes file owner
- chmod() - Changes file mode

chmod

chmod

(PHP 4, PHP 5, PHP 7, PHP 8)

chmod — Changes file mode

Description

function **chmod**(string `$filename`, int `$permissions`): bool

Attempts to change the mode of the specified file to that given in `permissions`.

Parameters

`filename`

Path to the file.

`permissions`

Note that `permissions` is not automatically assumed to be an octal value, so to ensure the expected operation, you need to prefix `permissions` with a zero (0). Strings such as "g+w" will not work properly.

```
<?php
chmod("/somedir/somefile", 755);   // decimal; probably incorrect
chmod("/somedir/somefile", "u+rwx,go+rx"); // string; incorrect
chmod("/somedir/somefile", 0755);  // octal; correct value of mode
?>
```

The `permissions` parameter consists of three octal number components specifying access restrictions for the owner, the user group in which the owner is in, and to everybody else in this order. One component can be computed by adding up the needed permissions for that target user base. Number 1 means that you grant execute rights, number 2 means that you make the file writeable, number 4 means that you make the file readable. Add up these numbers to specify needed rights. You can also read more about modes on Unix systems with '**man 1 chmod**' and '**man 2 chmod**'.

```
<?php
// Read and write for owner, nothing for everybody else
chmod("/somedir/somefile", 0600);

// Read and write for owner, read for everybody else
chmod("/somedir/somefile", 0644);

// Everything for owner, read and execute for others
chmod("/somedir/somefile", 0755);

// Everything for owner, read and execute for owner's group
chmod("/somedir/somefile", 0750);
?>
```

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

Upon failure, an **`E_WARNING`** is emitted.

Notes

> **Note** :
>
> The current user is the user under which PHP runs. It is probably not the same user you use for normal shell or FTP access. The mode can be changed only by user who owns the file on most systems.

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

See Also

- chown() - Changes file owner
- chgrp() - Changes file group
- fileperms() - Gets file permissions
- stat() - Gives information about a file

chown

chown

(PHP 4, PHP 5, PHP 7, PHP 8)

chown — Changes file owner

Description

function **chown**(string `$filename`, string|int `$user`): bool

Attempts to change the owner of the file `filename` to user `user`. Only the superuser may change the owner of a file.

Parameters

`filename`

Path to the file.

`user`

A user name or number.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Simple chown() usage

```php
<?php

// File name and username to use
$file_name= "foo.php";
$path = "/home/sites/php.net/public_html/sandbox/" . $file_name ;
$user_name = "root";

// Set the user
chown($path, $user_name);

// Check the result
$stat = stat($path);
print_r(posix_getpwuid($stat['uid']));

?>
```

The above example will output something similar to:

```
Array
(
    [name] => root
    [passwd] => x
    [uid] => 0
    [gid] => 0
    [gecos] => root
    [dir] => /root
    [shell] => /bin/bash
)
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

> **Note** :
>
> On Windows, this function fails silently when applied on a regular file.

See Also

- chmod() - Changes file mode
- chgrp() - Changes file group

clearstatcache

clearstatcache

(PHP 4, PHP 5, PHP 7, PHP 8)

clearstatcache — Clears file status cache

Description

function **clearstatcache**(bool `$clear_realpath_cache = false`, string `$filename = ""`): void

When you use stat(), lstat(), or any of the other functions listed in the affected functions list (below), PHP caches the information those functions return in order to provide faster performance. However, in certain cases, you may want to clear the cached information. For instance, if the same file is being checked multiple times within a single script, and that file is in danger of being removed or changed during that script's operation, you may elect to clear the status cache. In these cases, you can use the **clearstatcache()** function to clear the information that PHP caches about a file.

You should also note that PHP doesn't cache information about non-existent files. So, if you call file_exists() on a file that doesn't exist, it will return `false` until you create the file. If you create the file, it will return `true` even if you then delete the file. However unlink() clears the cache automatically.

> **Note** :
>
> This function caches information about specific filenames, so you only need to call **clearstatcache()** if you are performing multiple operations on the same filename and require the information about that particular file to not be cached.

Affected functions include stat(), lstat(), file_exists(), is_writable(), is_readable(), is_executable(), is_file(), is_dir(), is_link(), filectime(), fileatime(), filemtime(), fileinode(), filegroup(), fileowner(), filesize(), filetype(), and fileperms().

Parameters

`clear_realpath_cache`

Whether to *also* clear the realpath cache.

`filename`

Clear the realpath cache for a specific filename only; only used if `clear_realpath_cache` is `true`.

Return Values

No value is returned.

Examples

Example #1 clearstatcache() example

```
<?php
$file = 'output_log.txt';

function get_owner($file)
{
    $stat = stat($file);
    $user = posix_getpwuid($stat['uid']);
    return $user['name'];
}
```

```php
$format = "UID @ %s: %s\n";

printf($format, date('r'), get_owner($file));

chown($file, 'ross');
printf($format, date('r'), get_owner($file));

clearstatcache();
printf($format, date('r'), get_owner($file));
?>
```

The above example will output something similar to:

```
UID @ Sun, 12 Oct 2008 20:48:28 +0100: root
UID @ Sun, 12 Oct 2008 20:48:28 +0100: root
UID @ Sun, 12 Oct 2008 20:48:28 +0100: ross
```

copy

copy

(PHP 4, PHP 5, PHP 7, PHP 8)

copy — Copies file

Description

function **copy**(string `$from`, string `$to`, ?resource `$context = null`): bool

Makes a copy of the file `from` to `to`.

If you wish to move a file, use the rename() function.

Parameters

`from`

Path to the source file.

`to`

The destination path. If `to` is a URL, the copy operation may fail if the wrapper does not support overwriting of existing files.

> **Warning**
>
> If the destination file already exists, it will be overwritten.

`context`

A valid context resource created with stream_context_create().

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 copy() example

```php
<?php
$file = 'example.txt';
$newfile = 'example.txt.bak';
```

```
if (!copy($file, $newfile)) {
    echo "failed to copy $file...\n";
}
?>
```

See Also

- move_uploaded_file() - Moves an uploaded file to a new location
- rename() - Renames a file or directory
- The section of the manual about handling file uploads

delete

delete

delete — See unlink() or unset()

Description

There is no delete keyword or function in the PHP language. If you arrived at this page seeking to delete a file, try unlink(). To delete a variable from the local scope, check out unset().

See Also

- unlink() - Deletes a file
- unset() - unset a given variable

dirname

dirname

(PHP 4, PHP 5, PHP 7, PHP 8)

dirname — Returns a parent directory's path

Description

function **dirname**(string `$path`, int `$levels` = 1): string

Given a string containing the path of a file or directory, this function will return the parent directory's path that is `levels` up from the current directory.

> **Note** :
>
> **dirname()** operates naively on the input string, and is not aware of the actual filesystem, or path components such as "`..`".

> **Caution**
>
> On Windows, **dirname()** assumes the currently set codepage, so for it to see the correct directory name with multibyte character paths, the matching codepage must be set. If `path` contains characters which are invalid for the current codepage, the behavior of **dirname()** is undefined.
>
> On other systems, **dirname()** assumes `path` to be encoded in an ASCII compatible encoding. Otherwise, the behavior of the function is undefined.

Parameters

`path`

A path.

On Windows, both slash (`/`) and backslash (`\`) are used as directory separator character. In other environments, it is the forward slash (`/`).

`levels`

The number of parent directories to go up.

This must be an integer greater than 0.

Return Values

Returns the path of a parent directory. If there are no slashes in `path`, a dot ('.') is returned, indicating the current directory. Otherwise, the returned string is `path` with any trailing `/component` removed.

> **Caution**
>
> Be careful when using this function in a loop that can reach the top-level directory as this can result in an infinite loop.

```
<?php
dirname('.');    // Will return '.'.
dirname('/');    // Will return `\` on Windows and '/' on *nix systems.
dirname('\\');   // Will return `\` on Windows and '.' on *nix systems.
dirname('C:\\'); // Will return 'C:\' on Windows and '.' on *nix systems.
?>
```

Examples

Example #1 dirname() example

```
<?php
echo dirname("/etc/passwd") . PHP_EOL;
echo dirname("/etc/") . PHP_EOL;
echo dirname(".") . PHP_EOL;
echo dirname("C:\\") . PHP_EOL;
echo dirname("/usr/local/lib", 2);
```

The above example will output something similar to:

```
/etc
/ (or \ on Windows)
.
C:\
/usr
```

See Also

- basename() - Returns trailing name component of path
- pathinfo() - Returns information about a file path
- realpath() - Returns canonicalized absolute pathname

disk_free_space

disk_free_space

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

disk_free_space — Returns available space on filesystem or disk partition

Description

function **disk_free_space**(string `$directory`): float|false

Given a string containing a directory, this function will return the number of bytes available on the corresponding filesystem or disk partition.

Parameters

`directory`

A directory of the filesystem or disk partition.

> **Note** :
>
> Given a file name instead of a directory, the behaviour of the function is unspecified and may differ between operating systems and PHP versions.

Return Values

Returns the number of available bytes as a float or `false` on failure.

Examples

Example #1 disk_free_space() example

```
<?php
// $df contains the number of bytes available on "/"
$df = disk_free_space("/");

// On Windows:
$df_c = disk_free_space("C:");
$df_d = disk_free_space("D:");
?>
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

See Also

- disk_total_space() - Returns the total size of a filesystem or disk partition

disk_total_space

disk_total_space

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

disk_total_space — Returns the total size of a filesystem or disk partition

Description

function **disk_total_space**(string `$directory`): float|false

Given a string containing a directory, this function will return the total number of bytes on the corresponding filesystem or disk partition.

Parameters

`directory`

A directory of the filesystem or disk partition.

Return Values

Returns the total number of bytes as a float or `false` on failure.

Examples

Example #1 disk_total_space() example

```
<?php
// $ds contains the total number of bytes available on "/"
$ds = disk_total_space("/");

// On Windows:
$ds = disk_total_space("C:");
```

```
$ds = disk_total_space("D:");
?>
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

See Also

- disk_free_space() - Returns available space on filesystem or disk partition

diskfreespace

diskfreespace

(PHP 4, PHP 5, PHP 7, PHP 8)

diskfreespace — Alias of disk_free_space()

Description

This function is an alias of: disk_free_space().

fclose

fclose

(PHP 4, PHP 5, PHP 7, PHP 8)

fclose — Closes an open file pointer

Description

function **fclose**(resource `$stream`): bool

The file pointed to by `stream` is closed.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen().

Return Values

Returns **`true`** on success or **`false`** on failure.

Examples

Example #1 A simple fclose() example

```
<?php

$handle = fopen('somefile.txt', 'r');

fclose($handle);

?>
```

See Also

- fopen() - Opens file or URL
- fsockopen() - Open Internet or Unix domain socket connection

fdatasync

fdatasync

(PHP 8 >= 8.1.0)

fdatasync — Synchronizes data (but not meta-data) to the file

Description

function **fdatasync**(resource `$stream`): bool

This function synchronizes `stream` contents to storage media, just like fsync() does, but it does not synchronize file meta-data. Note that this function is only effectively different in POSIX systems. In Windows, this function is aliased to fsync().

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

Return Values

Returns **`true`** on success or **`false`** on failure.

Examples

Example #1 fdatasync() example

```php
<?php

$file = 'test.txt';

$stream = fopen($file, 'w');
fwrite($stream, 'test data');
fwrite($stream, "\r\n");
fwrite($stream, 'additional data');

fdatasync($stream);
fclose($stream);
?>
```

See Also

- fflush() - Flushes the output to a file
- fsync() - Synchronizes changes to the file (including meta-data)

feof

feof

(PHP 4, PHP 5, PHP 7, PHP 8)

feof — Tests for end-of-file on a file pointer

Description

function **feof**(resource `$stream`): bool

Tests for end-of-file on a file pointer.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

Return Values

Returns **`true`** if the file pointer is at EOF or an error occurs (including socket timeout); otherwise returns **`false`**.

Notes

> **Warning**
>
> If a connection opened by fsockopen() wasn't closed by the server, **feof()** will hang. To workaround this, see below example:
>
> **Example #1 Handling timeouts with feof()**

```php
<?php
function safe_feof($fp, &$start = NULL) {
 $start = microtime(true);

 return feof($fp);
}

/* Assuming $fp is previously opened by fsockopen() */

$start = NULL;
$timeout = ini_get('default_socket_timeout');

while(!safe_feof($fp, $start) && (microtime(true) - $start) < $timeout)
{
 /* Handle */
}
?>
```

> **Warning**
>
> If the passed file pointer is not valid you may get an infinite loop, because **feof()** fails to return `true`.
>
> **Example #2 feof() example with an invalid file pointer**

```php
<?php
// if file can not be read or doesn't exist fopen function returns FALSE
$file = @fopen("no_such_file", "r");

// FALSE from fopen will issue warning and result in infinite loop here
while (!feof($file)) {
}

fclose($file);
?>
```

fflush

fflush

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

fflush — Flushes the output to a file

Description

function **fflush**(resource `$stream`): bool

This function forces a write of all buffered output to the resource pointed to by the file `stream`.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 File write example using fflush()

```php
<?php
$filename = 'bar.txt';

$file = fopen($filename, 'r+');
rewind($file);
fwrite($file, 'Foo');
fflush($file);
ftruncate($file, ftell($file));
fclose($file);
?>
```

See Also

- clearstatcache() - Clears file status cache
- fwrite() - Binary-safe file write

fgetc

fgetc

(PHP 4, PHP 5, PHP 7, PHP 8)

fgetc — Gets character from file pointer

Description

function **fgetc**(resource `$stream`): string|false

Gets a character from the given file pointer.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

Return Values

Returns a string containing a single character read from the file pointed to by `stream`. Returns **`false`** on EOF.

> **Warning**
>
> This function may return Boolean **`false`**, but may also return a non-Boolean value which evaluates to **`false`**. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Examples

Example #1 A fgetc() example

```php
<?php
$fp = fopen('somefile.txt', 'r');
if (!$fp) {
    echo 'Could not open file somefile.txt';
}
while (false !== ($char = fgetc($fp))) {
    echo "$char\n";
}
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- fread() - Binary-safe file read
- fopen() - Opens file or URL
- popen() - Opens process file pointer
- fsockopen() - Open Internet or Unix domain socket connection
- fgets() - Gets line from file pointer

fgetcsv

fgetcsv

(PHP 4, PHP 5, PHP 7, PHP 8)

fgetcsv — Gets line from file pointer and parse for CSV fields

Description

```
function fgetcsv(
    resource $stream,
    ?int $length = null,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\"
): array|false
```

Similar to fgets() except that **fgetcsv()** parses the line it reads for fields in CSV format and returns an array containing the fields read.

> **Note** :
>
> The locale settings are taken into account by this function. For example, data encoded in certain one-byte encodings may be parsed incorrectly if `LC_CTYPE` is `en_US.UTF-8`.

Parameters

`stream`

A valid file pointer to a file successfully opened by fopen(), popen(), or fsockopen().

`length`

Must be greater than the longest line (in characters) to be found in the CSV file (allowing for trailing line-end characters). Otherwise the line is split in chunks of `length` characters, unless the split would occur inside an enclosure.

Omitting this parameter (or setting it to 0, or `null` in PHP 8.0.0 or later) the maximum line length is not limited, which is slightly slower.

`separator`

The `separator` parameter sets the field separator. It must be a single byte character.

`enclosure`

The `enclosure` parameter sets the field enclosure character. It must be a single byte character.

`escape`

The `escape` parameter sets the escape character. It must be a single byte character or the empty string. The empty string (`""`) disables the proprietary escape mechanism.

> **Warning**
>
> In the input stream, the `enclosure` character can always be escaped by doubling it inside a quoted string, resulting in a single `enclosure` character in the parsed result. The `escape` character works differently: If a sequence of `escape` and `enclosure` characters appear in the input, both characters will be present in the parsed result. So for the default parameters, a CSV line like `"a""b","c\"d"` will have the fields parsed as `a"b` and `c\"d`, respectively.

> **Warning**

> As of PHP 8.4.0, depending on the default value of `escape` is deprecated. It needs to be provided explicitly either positionally or by the use of named arguments.

> **Warning**
>
> When `escape` is set to anything other than an empty string (`""`) it can result in CSV that is not compliant with » RFC 4180 or unable to survive a roundtrip through the PHP CSV functions. The default for `escape` is `"\\"` so it is recommended to set it to the empty string explicitly. The default value will change in a future version of PHP, no earlier than PHP 9.0.

Return Values

Returns an indexed array containing the fields read on success, or `false` on failure.

> **Note** :
>
> A blank line in a CSV file will be returned as an array comprising a single null field, and will not be treated as an error.

> **Note** :
>
> Prior to PHP 8.1.0, the auto_detect_line_endings run-time configuration option could be enabled to help PHP properly recognize line endings when reading files created on Macintosh systems. This option has been deprecated as of PHP 8.1.0. If necessary, handle `"\r"` line breaks manually instead.

Errors/Exceptions

Throws a ValueError if `separator` or `enclosure` is not one byte long.

Throws a ValueError if `escape` is not one byte long or the empty string.

Changelog

Version	Description
8.4.0	Relying on the default value of `escape` is now deprecated.
8.3.0	An empty string is returned instead of a string with a single null byte for the last field if it contains only an unterminated enclosure.
8.0.0	`length` is now nullable.
7.4.0	The `escape` parameter now also accepts an empty string to disable the proprietary escape mechanism.

Examples

Example #1 Read and print the entire contents of a CSV file

```php
<?php

if (($handle = fopen("test.csv", "r")) !== false) {
    $row = 1;

    while (($fields = fgetcsv($handle, 1000, ",", "\"", "\\")) !== false) {
        $num = count($fields);

        echo "\n<p>$num fields in line $row:</p>\n<ol>";

        for ($c = 0; $c < $num; $c++) {
            echo "\n\t<li>", $fields[$c], "</li>";
        }

        echo "\n</ol>";

        $row++;
    }
```

```
    fclose($handle);
}
```

See Also

- fputcsv() - Format line as CSV and write to file pointer
- str_getcsv() - Parse a CSV string into an array
- SplFileObject::fgetcsv() - Gets line from file and parse as CSV fields
- SplFileObject::fputcsv() - Write a field array as a CSV line
- SplFileObject::setCsvControl() - Set the delimiter, enclosure and escape character for CSV
- SplFileObject::getCsvControl() - Get the delimiter, enclosure and escape character for CSV
- explode() - Split a string by a string
- file() - Reads entire file into an array
- pack() - Pack data into binary string

fgets

fgets

(PHP 4, PHP 5, PHP 7, PHP 8)

fgets — Gets line from file pointer

Description

function **fgets**(resource `$stream`, ?int `$length = null`): string|false

Gets a line from file pointer.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

`length`

Reading ends when `length` - 1 bytes have been read, or a newline (which is included in the return value), or an EOF (whichever comes first). If no length is specified, it will keep reading from the stream until it reaches the end of the line.

Return Values

Returns a string of up to `length` - 1 bytes read from the file pointed to by `stream`. If there is no more data to read in the file pointer, then `false` is returned.

If an error occurs, `false` is returned.

Examples

Example #1 Reading a file line by line

```
<?php

$fp = @fopen("/tmp/inputfile.txt", "r");

if ($fp) {
    while (($buffer = fgets($fp, 4096)) !== false) {
        echo $buffer, PHP_EOL;
    }

    if (!feof($fp)) {
        echo "Error: unexpected fgets() fail\n";
    }

    fclose($fp);
}

?>
```

Notes

> **Note** :
>
> Prior to PHP 8.1.0, the auto_detect_line_endings run-time configuration option could be enabled to help PHP properly recognize line endings when reading files created on Macintosh systems. This option has been deprecated as of PHP 8.1.0. If necessary, handle `"\r"` line breaks manually instead.

> **Note** :
>
> People used to the 'C' semantics of **fgets()** should note the difference in how `EOF` is returned.

See Also

- fgetss() - Gets line from file pointer and strip HTML tags
- fread() - Binary-safe file read
- fgetc() - Gets character from file pointer
- stream_get_line() - Gets line from stream resource up to a given delimiter
- fopen() - Opens file or URL
- popen() - Opens process file pointer
- fsockopen() - Open Internet or Unix domain socket connection
- stream_set_timeout() - Set timeout period on a stream

fgetss

fgetss

(PHP 4, PHP 5, PHP 7)

fgetss — Gets line from file pointer and strip HTML tags

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.3.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **fgetss**(resource `$handle`, int `$length = ?`, string `$allowable_tags = ?`): string

Identical to fgets(), except that **fgetss()** attempts to strip any NUL bytes, HTML and PHP tags from the text it reads. The function retains the parsing state from call to call, and as such is not equivalent to calling strip_tags() on the return value of fgets().

Parameters

`handle`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

`length`

Length of the data to be retrieved.

`allowable_tags`

You can use the optional third parameter to specify tags which should not be stripped. See strip_tags() for details regarding `allowable_tags`.

Return Values

Returns a string of up to `length` - 1 bytes read from the file pointed to by `handle`, with all HTML and PHP code stripped.

If an error occurs, returns `false`.

Examples

Example #1 Reading a PHP file line-by-line

```
<?php
$str = <<<EOD
<html><body>
 <p>Welcome! Today is the <?php echo(date('jS')); ?> of <?= date('F'); ?>.</p>
</body></html>
Text outside of the HTML block.
EOD;
file_put_contents('sample.php', $str);

$handle = @fopen("sample.php", "r");
if ($handle) {
    while (!feof($handle)) {
        $buffer = fgetss($handle, 4096);
        echo $buffer;
    }
    fclose($handle);
}
?>
```

The above example will output something similar to:

```
Welcome! Today is the  of .

Text outside of the HTML block.
```

Notes

> **Note** :
>
> Prior to PHP 8.1.0, the auto_detect_line_endings run-time configuration option could be enabled to help PHP properly recognize line endings when reading files created on Macintosh systems. This option has been deprecated as of PHP 8.1.0. If necessary, handle `"\r"` line breaks manually instead.

See Also

- fgets() - Gets line from file pointer
- fopen() - Opens file or URL
- popen() - Opens process file pointer
- fsockopen() - Open Internet or Unix domain socket connection
- strip_tags() - Strip HTML and PHP tags from a string
- SplFileObject::fgetss() - Gets line from file and strip HTML tags
- The string.strip_tags filter

file

file

(PHP 4, PHP 5, PHP 7, PHP 8)

file — Reads entire file into an array

Description

function **file**(string `$filename`, int `$flags` = 0, ?resource `$context` = `null`): array|false

Reads an entire file into an array.

> **Note** :

> You can use file_get_contents() to return the contents of a file as a string.

Parameters

`filename`

Path to the file.

> **Tip**
>
> A URL can be used as a filename with this function if the fopen wrappers have been enabled. See fopen() for more details on how to specify the filename. See the Supported Protocols and Wrappers for links to information about what abilities the various wrappers have, notes on their usage, and information on any predefined variables they may provide.

`flags`

The optional parameter `flags` can be one, or more, of the following constants:

`FILE_USE_INCLUDE_PATH`

Search for the file in the include_path.

`FILE_IGNORE_NEW_LINES`

Omit newline at the end of each array element.

`FILE_SKIP_EMPTY_LINES`

Skip empty lines.

`FILE_NO_DEFAULT_CONTEXT`

Don't use the default context.

`context`

A context stream resource.

Return Values

Returns the file in an array. Each element of the array corresponds to a line in the file, with the newline still attached. Upon failure, **file()** returns **`false`**.

> **Note** :
>
> Each line in the resulting array will include the line ending, unless **`FILE_IGNORE_NEW_LINES`** is used.

> **Note** :
>
> Prior to PHP 8.1.0, the auto_detect_line_endings run-time configuration option could be enabled to help PHP properly recognize line endings when reading files created on Macintosh systems. This option has been deprecated as of PHP 8.1.0. If necessary, handle `"\r"` line breaks manually instead.

Errors/Exceptions

As of PHP 8.3.0, throws a ValueError if `flags` includes any invalid values, such as **`FILE_APPEND`**.

Emits an **`E_WARNING`** level error if the file does not exist.

Changelog

Version	Description
8.3.0	ValueError is thrown for any invalid values of `flags`.

Examples

Example #1 file() example

```php
<?php
// Get a file into an array.  In this example we'll go through HTTP to get
// the HTML source of a URL.
$lines = file('http://www.example.com/');

// Loop through our array, show HTML source as HTML source; and line numbers too.
foreach ($lines as $line_num => $line) {
    echo "Line #<b>{$line_num}</b> : " . htmlspecialchars($line) . "<br />\n";
}

// Using the optional flags parameter
$trimmed = file('somefile.txt', FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
?>
```

Notes

> **Warning**
>
> When using SSL, Microsoft IIS will violate the protocol by closing the connection without sending a `close_notify` indicator. PHP will report this as "SSL: Fatal Protocol Error" when you reach the end of the data. To work around this, the value of error_reporting should be lowered to a level that does not include warnings. PHP can detect buggy IIS server software when you open the stream using the `https://` wrapper and will suppress the warning. When using fsockopen() to create an `ssl://` socket, the developer is responsible for detecting and suppressing this warning.

See Also

- file_get_contents() - Reads entire file into a string
- readfile() - Outputs a file
- fopen() - Opens file or URL
- fsockopen() - Open Internet or Unix domain socket connection
- popen() - Opens process file pointer
- include - include
- stream_context_create() - Creates a stream context

file_exists

file_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

file_exists — Checks whether a file or directory exists

Description

function **file_exists**(string `$filename`): bool

Checks whether a file or directory exists.

Parameters

`filename`

Path to the file or directory.

On windows, use *//computername/share/filename* or *\\computername\share\filename* to check files on network shares.

Return Values

Returns **`true`** if the file or directory specified by `filename` exists; **`false`** otherwise.

> **Note** :
>
> This function will return **`false`** for symlinks pointing to non-existing files.

> **Note** :
>
> The check is done using the real UID/GID instead of the effective one.

> **Note** :
>
> Because PHP's integer type is signed and many platforms use 32bit integers, some filesystem functions may return unexpected results for files which are larger than 2GB.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 Testing whether a file exists

```php
<?php
$filename = '/path/to/foo.txt';

if (file_exists($filename)) {
    echo "The file $filename exists";
} else {
    echo "The file $filename does not exist";
}
?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- is_readable() - Tells whether a file exists and is readable
- is_writable() - Tells whether the filename is writable
- is_file() - Tells whether the filename is a regular file
- file() - Reads entire file into an array
- SplFileInfo

file_get_contents

file_get_contents

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

file_get_contents — Reads entire file into a string

Description

```
function file_get_contents(
    string $filename,
    bool $use_include_path = false,
    ?resource $context = null,
    int $offset = 0,
    ?int $length = null
): string|false
```

This function is similar to file(), except that **file_get_contents()** returns the file in a string, starting at the specified `offset` up to `length` bytes. On failure, **file_get_contents()** will return `false`.

file_get_contents() is the preferred way to read the contents of a file into a string. It will use memory mapping techniques if supported by your OS to enhance performance.

> **Note** :
>
> If you're opening a URI with special characters, such as spaces, you need to encode the URI with urlencode().

Parameters

`filename`

Name of the file to read.

`use_include_path`

> **Note** :
>
> The `FILE_USE_INCLUDE_PATH` constant can be used to trigger include path search. This is not possible if strict typing is enabled, since `FILE_USE_INCLUDE_PATH` is an int. Use `true` instead.

`context`

A valid context resource created with stream_context_create(). If you don't need to use a custom context, you can skip this parameter by `null`.

`offset`

The offset where the reading starts on the original stream. Negative offsets count from the end of the stream.

Seeking (`offset`) is not supported with remote files. Attempting to seek on non-local files may work with small offsets, but this is unpredictable because it works on the buffered stream.

`length`

Maximum length of data read. The default is to read until end of file is reached. Note that this parameter is applied to the stream processed by the filters.

Return Values

The function returns the read data or `false` on failure.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Errors/Exceptions

An `E_WARNING` level error is generated if `filename` cannot be found, `length` is less than zero, or if seeking to the specified `offset` in the stream fails.

When **file_get_contents()** is called on a directory, an `E_WARNING` level error is generated on Windows, and as of PHP 7.4 on other operating systems as well.

Changelog

Version	Description
8.0.0	`length` is nullable now.
7.1.0	Support for negative `offset`s has been added.

Examples

Example #1 Get and output the source of the homepage of a website

```php
<?php
$homepage = file_get_contents('http://www.example.com/');
echo $homepage;
?>
```

Example #2 Searching within the include_path

```php
<?php
// If strict types are enabled i.e. declare(strict_types=1);
$file = file_get_contents('./people.txt', true);
// Otherwise
$file = file_get_contents('./people.txt', FILE_USE_INCLUDE_PATH);
?>
```

Example #3 Reading a section of a file

```php
<?php
// Read 14 characters starting from the 21st character
$section = file_get_contents('./people.txt', FALSE, NULL, 20, 14);
var_dump($section);
?>
```

The above example will output something similar to:

```
string(14) "lle Bjori Ro"
```

Example #4 Using stream contexts

```php
<?php
// Create a stream
$opts = [
  'http' => [
    'method' => "GET",
    'header' => "Accept-language: en\r\n" .
                "Cookie: foo=bar",
  ]
];

$context = stream_context_create($opts);

// Open the file using the HTTP headers set above
$file = file_get_contents('http://www.example.com/', false, $context);
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

> **Tip**
>
> A URL can be used as a filename with this function if the fopen wrappers have been enabled. See fopen() for more details on how to specify the filename. See the Supported Protocols and Wrappers for links to information about what abilities the various wrappers have, notes on their usage, and information on any predefined variables they may provide.

> **Warning**

> When using SSL, Microsoft IIS will violate the protocol by closing the connection without sending a `close_notify` indicator. PHP will report this as "SSL: Fatal Protocol Error" when you reach the end of the data. To work around this, the value of error_reporting should be lowered to a level that does not include warnings. PHP can detect buggy IIS server software when you open the stream using the `https://` wrapper and will suppress the warning. When using fsockopen() to create an `ssl://` socket, the developer is responsible for detecting and suppressing this warning.

See Also

- file() - Reads entire file into an array
- fgets() - Gets line from file pointer
- fread() - Binary-safe file read
- readfile() - Outputs a file
- file_put_contents() - Write data to a file
- stream_get_contents() - Reads remainder of a stream into a string
- stream_context_create() - Creates a stream context
- $http_response_header

file_put_contents

file_put_contents

(PHP 5, PHP 7, PHP 8)

file_put_contents — Write data to a file

Description

```
function file_put_contents(
    string $filename,
    mixed $data,
    int $flags = 0,
    ?resource $context = null
): int|false
```

This function is identical to calling fopen(), fwrite() and fclose() successively to write data to a file.

If `filename` does not exist, the file is created. Otherwise, the existing file is overwritten, unless the `FILE_APPEND` flag is set.

Parameters

`filename`

Path to the file where to write the data.

`data`

The data to write. Can be either a string, an array or a stream resource.

If `data` is a stream resource, the remaining buffer of that stream will be copied to the specified file. This is similar with using stream_copy_to_stream().

You can also specify the `data` parameter as a single dimension array. This is equivalent to `file_put_contents($filename, implode('', $array))`.

`flags`

The value of `flags` can be any combination of the following flags, joined with the binary OR (`|`) operator.

Available flags

Flag	Description
`FILE_USE_INCLUDE_PATH`	Search for `filename` in the include directory. See include_path for more information.
`FILE_APPEND`	If file `filename` already exists, append the data to the file instead of overwriting it.
`LOCK_EX`	Acquire an exclusive lock on the file while proceeding to the writing. In other words, a flock() call happens between the fopen() call and the fwrite() call. This is not identical to an fopen() call with mode "x".

`context`

A valid context resource created with stream_context_create().

Return Values

This function returns the number of bytes that were written to the file, or `false` on failure.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Examples

Example #1 Simple usage example

```php
<?php
$file = 'people.txt';
// Open the file to get existing content
$current = file_get_contents($file);
// Append a new person to the file
$current .= "John Smith\n";
// Write the contents back to the file
file_put_contents($file, $current);
?>
```

Example #2 Using flags

```php
<?php
$file = 'people.txt';
// The new person to add to the file
$person = "John Smith\n";
// Write the contents to the file,
// using the FILE_APPEND flag to append the content to the end of the file
// and the LOCK_EX flag to prevent anyone else writing to the file at the same time
file_put_contents($file, $person, FILE_APPEND | LOCK_EX);
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

> **Tip**
>
> A URL can be used as a filename with this function if the fopen wrappers have been enabled. See fopen() for more details on how to specify the filename. See the Supported Protocols and Wrappers for links to information about what abilities the various wrappers have, notes on their usage, and information on any predefined variables they may provide.

See Also

- fopen() - Opens file or URL
- fwrite() - Binary-safe file write
- file_get_contents() - Reads entire file into a string
- stream_context_create() - Creates a stream context

fileatime

fileatime

(PHP 4, PHP 5, PHP 7, PHP 8)

fileatime — Gets last access time of file

Description

function **fileatime**(string `$filename`): int|false

Gets the last access time of the given file.

Parameters

`filename`

Path to the file.

Return Values

Returns the time the file was last accessed, or `false` on failure. The time is returned as a Unix timestamp.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 fileatime() example

```php
<?php

// outputs e.g.  somefile.txt was last accessed: December 29 2002 22:16:23.

$filename = 'somefile.txt';
if (file_exists($filename)) {
    echo "$filename was last accessed: " . date("F d Y H:i:s.", fileatime($filename));
}

?>
```

Notes

> **Note** :
>
> The atime of a file is supposed to change whenever the data blocks of a file are being read. This can be costly performance-wise when an application regularly accesses a very large number of files or directories.
>
> Some Unix filesystems can be mounted with atime updates disabled to increase the performance of such applications; USENET news spools are a common example. On such filesystems this function will be useless.

> **Note** :
>
> Note that time resolution may differ from one file system to another.

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- filemtime() - Gets file modification time
- fileinode() - Gets file inode
- date() - Format a Unix timestamp

filectime

filectime

(PHP 4, PHP 5, PHP 7, PHP 8)

filectime — Gets inode change time of file

Description

function **filectime**(string `$filename`): int|false

Gets the inode change time of a file.

Parameters

`filename`

Path to the file.

Return Values

Returns the time the file was last changed, or `false` on failure. The time is returned as a Unix timestamp.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 A filectime() example

```php
<?php

// outputs e.g.  somefile.txt was last changed: December 29 2002 22:16:23.

$filename = 'somefile.txt';
if (file_exists($filename)) {
    echo "$filename was last changed: " . date("F d Y H:i:s.", filectime($filename));
}

?>
```

Notes

> **Note** :
>
> Note: In most Unix filesystems, a file is considered changed when its inode data is changed; that is, when the permissions, owner, group, or other metadata from the inode is updated. See also filemtime() (which is what you want to use when you want to create "Last Modified" footers on web pages) and fileatime().

> **Note** :
>
> Note also that in some Unix texts the ctime of a file is referred to as being the creation time of the file. This is wrong. There is no creation time for Unix files in most Unix filesystems.

> **Note** :
>
> Note that time resolution may differ from one file system to another.

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**

> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- filemtime() - Gets file modification time

filegroup

filegroup

(PHP 4, PHP 5, PHP 7, PHP 8)

filegroup — Gets file group

Description

function **filegroup**(string `$filename`): int|false

Gets the file group. The group ID is returned in numerical format, use posix_getgrgid() to resolve it to a group name.

Parameters

`filename`

Path to the file.

Return Values

Returns the group ID of the file, or `false` if an error occurs. The group ID is returned in numerical format, use posix_getgrgid() to resolve it to a group name. Upon failure, `false` is returned.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 Finding the group of a file

```php
<?php
$filename = 'index.php';
print_r(posix_getgrgid(filegroup($filename)));
?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- fileowner() - Gets file owner
- posix_getgrgid() - Return info about a group by group id

fileinode

fileinode

(PHP 4, PHP 5, PHP 7, PHP 8)

fileinode — Gets file inode

Description

function **fileinode**(string `$filename`): int|false

Gets the file inode.

Parameters

`filename`

Path to the file.

Return Values

Returns the inode number of the file, or `false` on failure.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 Comparing the inode of a file with the current file

```
<?php
$filename = 'index.php';
if (getmyinode() == fileinode($filename)) {
    echo 'You are checking the current file.';
}
?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- getmyinode() - Gets the inode of the current script
- stat() - Gives information about a file

filemtime

filemtime

(PHP 4, PHP 5, PHP 7, PHP 8)

filemtime — Gets file modification time

Description

function **filemtime**(string `$filename`): int|false

This function returns the time when the data blocks of a file were being written to, that is, the time when the content of the file was changed.

Parameters

`filename`

Path to the file.

Return Values

Returns the time the file was last modified, or `false` on failure. The time is returned as a Unix timestamp, which is suitable for the date() function.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 filemtime() example

```php
<?php
// outputs e.g.  somefile.txt was last modified: December 29 2002 22:16:23.

$filename = 'somefile.txt';
if (file_exists($filename)) {
    echo "$filename was last modified: " . date ("F d Y H:i:s.", filemtime($filename));
}
?>
```

Notes

> **Note** :
>
> Note that time resolution may differ from one file system to another.

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- filectime() - Gets inode change time of file
- stat() - Gives information about a file
- touch() - Sets access and modification time of file
- getlastmod() - Gets time of last page modification

fileowner

fileowner

(PHP 4, PHP 5, PHP 7, PHP 8)

fileowner — Gets file owner

Description

function **fileowner**(string `$filename`): int|false

Gets the file owner.

Parameters

`filename`

Path to the file.

Return Values

Returns the user ID of the owner of the file, or `false` on failure. The user ID is returned in numerical format, use posix_getpwuid() to resolve it to a username.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 Finding the owner of a file

```
<?php
$filename = 'index.php';
print_r(posix_getpwuid(fileowner($filename)));
?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- filegroup() - Gets file group
- stat() - Gives information about a file
- posix_getpwuid() - Return info about a user by user id

fileperms

fileperms

(PHP 4, PHP 5, PHP 7, PHP 8)

fileperms — Gets file permissions

Description

function **fileperms**(string `$filename`): int|false

Gets permissions for the given file.

Parameters

`filename`

Path to the file.

Return Values

Returns the file's permissions as a numeric mode. Lower bits of this mode are the same as the permissions expected by chmod(), however on most platforms the return value will also include information on the type of file given as `filename`. The examples below demonstrate how to test the return value for specific permissions and file types on POSIX systems, including Linux and macOS.

For local files, the specific return value is that of the `st_mode` member of the structure returned by the C library's stat() function. Exactly which bits are set can vary from platform to platform, and looking up your specific platform's documentation is recommended if parsing the non-permission bits of the return value is required.

Returns `false` on failure.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 Display permissions as an octal value

```php
<?php
echo substr(sprintf('%o', fileperms('/tmp')), -4);
echo substr(sprintf('%o', fileperms('/etc/passwd')), -4);
?>
```

The above example will output:

```
1777
0644
```

Example #2 Display full permissions

```php
<?php
$perms = fileperms('/etc/passwd');

switch ($perms & 0xF000) {
    case 0x1000: $info = 'p'; break; // FIFO pipe
    case 0x2000: $info = 'c'; break; // character special
    case 0x4000: $info = 'd'; break; // directory
    case 0x6000: $info = 'b'; break; // block special
    case 0x8000: $info = '-'; break; // regular
    case 0xA000: $info = 'l'; break; // symbolic link (reachable when using lstat function)
    case 0xC000: $info = 's'; break; // socket

    // unknown
    default: $info = 'u';
}

// Owner
$setuid = $perms & 0x0800;
$info .= (($perms & 0x0100) ? 'r' : '-');
$info .= (($perms & 0x0080) ? 'w' : '-');
$info .= (($perms & 0x0040) ? ($setuid ? 's' : 'x') : ($setuid ? 'S' : '-'));

// Group
$setgid = $perms & 0x0400;
$info .= (($perms & 0x0020) ? 'r' : '-');
$info .= (($perms & 0x0010) ? 'w' : '-');
$info .= (($perms & 0x0008) ? ($setgid ? 's' : 'x') : ($setgid ? 'S' : '-'));

// World
$sticky = $perms & 0x0200;
$info .= (($perms & 0x0004) ? 'r' : '-');
$info .= (($perms & 0x0002) ? 'w' : '-');
$info .= (($perms & 0x0001) ? ($sticky ? 't' : 'x') : ($sticky ? 'T' : '-'));

echo $info;
?>
```

The above example will output:

```
-rw-r--r--
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- chmod() - Changes file mode
- is_readable() - Tells whether a file exists and is readable
- stat() - Gives information about a file

filesize

filesize

(PHP 4, PHP 5, PHP 7, PHP 8)

filesize — Gets file size

Description

function **filesize**(string `$filename`): int|false

Gets the size for the given file.

Parameters

`filename`

Path to the file.

Return Values

Returns the size of the file in bytes, or **`false`** (and generates an error of level **`E_WARNING`**) in case of an error.

> **Note** :
>
> Because PHP's integer type is signed and many platforms use 32bit integers, some filesystem functions may return unexpected results for files which are larger than 2GB.

Errors/Exceptions

Upon failure, an **`E_WARNING`** is emitted.

Examples

Example #1 filesize() example

```php
<?php

// outputs e.g.  somefile.txt: 1024 bytes

$filename = 'somefile.txt';
echo $filename . ': ' . filesize($filename) . ' bytes';

?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- file_exists() - Checks whether a file or directory exists

filetype

filetype

(PHP 4, PHP 5, PHP 7, PHP 8)

filetype — Gets file type

Description

function **filetype**(string `$filename`): string|false

Returns the type of the given file.

Parameters

`filename`

Path to the file.

Return Values

Returns the type of the file. Possible values are fifo, char, dir, block, link, file, socket and unknown.

Returns `false` if an error occurs. **filetype()** will also produce an `E_NOTICE` message if the stat call fails or if the file type is unknown.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 filetype() example

```php
<?php

echo filetype('/etc/passwd');
echo "\n";
echo filetype('/etc/');

?>
```

The above example will output:

```
file
dir
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- is_dir() - Tells whether the filename is a directory
- is_file() - Tells whether the filename is a regular file
- is_link() - Tells whether the filename is a symbolic link
- file_exists() - Checks whether a file or directory exists
- mime_content_type() - Detect MIME Content-type for a file
- pathinfo() - Returns information about a file path
- stat() - Gives information about a file

flock

flock

(PHP 4, PHP 5, PHP 7, PHP 8)

flock — Portable advisory file locking

Description

function **flock**(resource `$stream`, int `$operation`, int `&$would_block` = `null`): bool

flock() allows you to perform a simple reader/writer model which can be used on virtually every platform (including most Unix derivatives and even Windows).

The lock is released also by fclose(), or when `stream` is garbage collected.

PHP supports a portable way of locking complete files in an advisory way (which means all accessing programs have to use the same way of locking or it will not work). By default, this function will block until the requested lock is acquired; this may be controlled with the `LOCK_NB` option documented below.

Parameters

`stream`

A file system pointer resource that is typically created using fopen().

`operation`

`operation` is one of the following:

- `LOCK_SH` to acquire a shared lock (reader).
- `LOCK_EX` to acquire an exclusive lock (writer).
- `LOCK_UN` to release a lock (shared or exclusive).

It is also possible to add `LOCK_NB` as a bitmask to one of the above operations, if **flock()** should not block during the locking attempt.

`would_block`

The optional third argument is set to 1 if the lock would block (EWOULDBLOCK errno condition).

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 flock() example

```php
<?php

$fp = fopen("/tmp/lock.txt", "r+");

if (flock($fp, LOCK_EX)) {  // acquire an exclusive lock
    ftruncate($fp, 0);      // truncate file
    rewind($fp);
    fwrite($fp, "Write something here\n");
    fflush($fp);            // flush output before releasing the lock
    flock($fp, LOCK_UN);    // release the lock
} else {
    echo "Couldn't get the lock!";
}

fclose($fp);

?>
```

Example #2 flock() using the LOCK_NB option

```php
<?php
$fp = fopen('/tmp/lock.txt', 'r+');

/* Activate the LOCK_NB option on an LOCK_EX operation */
if(!flock($fp, LOCK_EX | LOCK_NB)) {
    echo 'Unable to obtain lock';
    exit(-1);
}

/* ... */

fclose($fp);
?>
```

Notes

> **Note** :
>
> **flock()** uses mandatory locking instead of advisory locking on Windows. Mandatory locking is also supported on Linux and System V based operating systems via the usual mechanism supported by the fcntl() system call: that is, if the file in question has the setgid permission bit set and the group execution bit cleared. On Linux, the file system will also need to be mounted with the mand option for this to work.

> **Note** :
>
> Because **flock()** requires a file pointer, you may have to use a special lock file to protect access to a file that you intend to truncate by opening it in write mode (with a "w" or "w+" argument to fopen()).

> **Note** :
>
> May only be used on file pointers returned by fopen() for local files, or file pointers pointing to userspace streams that implement the streamWrapper::stream_lock() method.

> **Warning**
>
> Assigning another value to `stream` argument in subsequent code will release the lock.

> **Warning**
>
> On some operating systems **flock()** is implemented at the process level. When using a multithreaded server API you may not be able to rely on **flock()** to protect files against other PHP scripts running in parallel threads of the same server instance!

flock() is not supported on antiquated filesystems like `FAT` and its derivates and will therefore always return `false` under these environments.

On Windows, if PHP tries to execute a `LOCK_EX`'ed PHP source code file, it will hang until the file is unlocked. The following example will cause a deadlock:

```
<?php
if ($argv[1] ?? null === 'child') { die(); }
$fp = fopen(__FILE__, "rb");
flock($fp, LOCK_EX);
shell_exec("php " . escapeshellarg(__FILE__) . " child");
?>
```

> **Note** :
>
> On Windows, if the locking process opens the file a second time, it cannot access the file through this second handle until it unlocks the file.
>
> On Windows, if you try to require a LOCK_EX'ed file, it will fail with ErrorException: require(): Read of X bytes failed with errno=13 Permission denied

fnmatch

fnmatch

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

fnmatch — Match filename against a pattern

Description

function **fnmatch**(string `$pattern`, string `$filename`, int `$flags` = 0): bool

fnmatch() checks if the passed `filename` would match the given shell wildcard `pattern`.

Parameters

`pattern`

The `pattern` to match against. Usually, the `pattern` will contain wildcards like `'?'` and `'*'`.

Wildcards to be used in `pattern` parameter

Wildcard	Description
`?`	Question mark will match any single character. For example, pattern `"file?.txt"` will match `"file1.txt"` and `"fileA.txt"`, but will not match `"file10.txt"`.
`*`	Asterisk will match zero or more characters. For example, pattern `"foo*.xml"` will match `"foo.xml"` and `"foobar.xml"`.
`[ ]`	Square brackets are used to create ranges of ASCII codepoints or sets of characters. For example, pattern `"index.php[45]"` will match `"index.php4"` and `"index.php5"`, but will not match `"index.phpt"`. Well known ranges are `[0-9]`, `[a-z]`, and `[A-Z]`. Multiple sets and ranges can be used at the same time, for example `[0-9a-zABC]`.
`!`	Exclamation mark is used to negate characters within square brackets. For example, `"[!A-Z]*.html"` will match `"demo.html"`, but will not match `"Demo.html"`.
`\`	Backslash is used to escape special characters. For example, `"Name\?"` will match `"Name?"`, but will not match `"Names"`.

`filename`

The tested string. This function is especially useful for filenames, but may also be used on regular strings.

The average user may be used to shell patterns or at least in their simplest form to `'?'` and `'*'` wildcards so using **fnmatch()** instead of preg_match() for frontend search expression input may be way more convenient for non-programming users.

`flags`

The value of `flags` can be any combination of the following flags, joined with the binary OR (|) operator.

A list of possible flags for fnmatch()

Flag	Description
FNM_NOESCAPE	Disable backslash escaping.
FNM_PATHNAME	Slash in string only matches slash in the given pattern.
FNM_PERIOD	Leading period in string must be exactly matched by period in the given pattern.
FNM_CASEFOLD	Caseless match. Part of the GNU extension.

Return Values

Returns `true` if there is a match, `false` otherwise.

Examples

Example #1 Checking a color name against a shell wildcard pattern

```
<?php
if (fnmatch("*gr[ae]y", $color)) {
  echo "some form of gray ...";
}
?>
```

Notes

> **Warning**
>
> For now, this function is not available on non-POSIX compliant systems except Windows.

See Also

- glob() - Find pathnames matching a pattern
- preg_match() - Perform a regular expression match
- sscanf() - Parses input from a string according to a format
- printf() - Output a formatted string
- sprintf() - Return a formatted string

fopen

fopen

(PHP 4, PHP 5, PHP 7, PHP 8)

fopen — Opens file or URL

Description

function **fopen**(
  string $filename,
  string $mode,
  bool $use_include_path = false,
  ?resource $context = null
): resource|false

fopen() binds a named resource, specified by `filename`, to a stream.

Parameters

`filename`

If `filename` is of the form "scheme://...", it is assumed to be a URL and PHP will search for a protocol handler (also known as a wrapper) for that scheme. If no wrappers for that protocol are registered, PHP will emit a notice to help you track potential problems in your script and then continue as though `filename` specifies a regular file.

If PHP has decided that `filename` specifies a local file, then it will try to open a stream on that file. The file must be accessible to PHP, so you need to ensure that the file access permissions allow this access. If you have enabled open_basedir further restrictions may apply.

If PHP has decided that `filename` specifies a registered protocol, and that protocol is registered as a network URL, PHP will check to make sure that allow_url_fopen is enabled. If it is switched off, PHP will emit a warning and the **fopen()** call will fail.

> **Note** :
>
> The list of supported protocols can be found in Supported Protocols and Wrappers. Some protocols (also referred to as `wrappers`) support `context` and/or *php.ini* options. Refer to the specific page for the protocol in use for a list of options which can be set. (e.g. *php.ini* value `user_agent` used by the `http` wrapper).

On the Windows platform, be careful to escape any backslashes used in the path to the file, or use forward slashes.

```
<?php
$handle = fopen("c:\\folder\\resource.txt", "r");
?>
```

`mode`

The `mode` parameter specifies the type of access you require to the stream. It may be any of the following:

A list of possible modes for fopen() using `mode`

`mode`	Description
`'r'`	Open for reading only; place the file pointer at the beginning of the file.
`'r+'`	Open for reading and writing; place the file pointer at the beginning of the file.
`'w'`	Open for writing only; place the file pointer at the beginning of the file and truncate the file to zero length. If the file does not exist, attempt to create it.
`'w+'`	Open for reading and writing; otherwise it has the same behavior as `'w'`.
`'a'`	Open for writing only; place the file pointer at the end of the file. If the file does not exist, attempt to create it. In this mode, fseek() has no effect, writes are always appended.
`'a+'`	Open for reading and writing; place the file pointer at the end of the file. If the file does not exist, attempt to create it. In this mode, fseek() only affects the reading position, writes are always appended.
`'x'`	Create and open for writing only; place the file pointer at the beginning of the file. If the file already exists, the **fopen()** call will fail by returning `false` and generating an error of level `E_WARNING`. If the file does not exist, attempt to create it. This is equivalent to specifying `O_EXCL|O_CREAT` flags for the underlying `open(2)` system call.
`'x+'`	Create and open for reading and writing; otherwise it has the same behavior as `'x'`.
`'c'`	Open the file for writing only. If the file does not exist, it is created. If it exists, it is neither truncated (as opposed to `'w'`), nor the call to this function fails (as is the case with `'x'`). The file pointer is positioned on the beginning of the file. This may be useful if it's desired to get an advisory lock (see flock()) before attempting to modify the file, as using `'w'` could truncate the file before the lock was obtained (if truncation is desired, ftruncate() can be used after the lock is requested).
`'c+'`	Open the file for reading and writing; otherwise it has the same behavior as `'c'`.
`'e'`	Set close-on-exec flag on the opened file descriptor. Only available in PHP compiled on POSIX.1-2008 conformant systems.
`'n'`	Set non-blocking flag on the opened file descriptor. Only available in PHP compiled on POSIX.1-2008 conformant systems.

> **Note** :
>
> Different operating system families have different line-ending conventions. When you write a text file and want to insert a line break, you need to use the correct line-ending character(s) for your operating system. Unix based systems use `\n` as the line ending character, Windows based systems use `\r\n` as the line ending characters and Macintosh based systems (Mac OS Classic) used `\r` as the line ending character.
>
> If you use the wrong line ending characters when writing your files, you might find that other applications that open those files will "look funny".
>
> Windows offers a text-mode translation flag (`'t'`) which will transparently translate `\n` to `\r\n` when working with the file. In contrast, you can also use `'b'` to force binary mode, which will not translate your data. To use these flags, specify either `'b'` or `'t'` as the last character of the `mode` parameter.
>
> The default translation mode is `'b'`. You can use the `'t'` mode if you are working with plain-text files and you use `\n` to delimit your line endings in your script, but expect your files to be readable with applications such as old versions of notepad. You should use the `'b'` in all other cases.

> If you specify the 't' flag when working with binary files, you may experience strange problems with your data, including broken image files and strange problems with `\r\n` characters.

> **Note** :
>
> For portability, it is also strongly recommended that you re-write code that uses or relies upon the `'t'` mode so that it uses the correct line endings and `'b'` mode instead.

> **Note** :
>
> The `mode` is ignored for *php://output*, *php://input*, *php://stdin*, *php://stdout*, *php://stderr* and *php://fd* stream wrappers.

`use_include_path`

The optional third `use_include_path` parameter can be set to **`true`** if you want to search for the file in the include_path, too.

`context`

A context stream resource.

Return Values

Returns a file pointer resource on success, or **`false`** on failure

Errors/Exceptions

Upon failure, an **`E_WARNING`** is emitted.

Changelog

Version	Description
7.0.16, 7.1.2	The `'e'` option was added.

Examples

Example #1 **fopen()** examples

```
<?php
$handle = fopen("/home/rasmus/file.txt", "r");
$handle = fopen("/home/rasmus/file.gif", "wb");
$handle = fopen("http://www.example.com/", "r");
$handle = fopen("ftp://user:password@example.com/somefile.txt", "w");
?>
```

Notes

> **Warning**
>
> When using SSL, Microsoft IIS will violate the protocol by closing the connection without sending a `close_notify` indicator. PHP will report this as "SSL: Fatal Protocol Error" when you reach the end of the data. To work around this, the value of error_reporting should be lowered to a level that does not include warnings. PHP can detect buggy IIS server software when you open the stream using the `https://` wrapper and will suppress the warning. When using fsockopen() to create an `ssl://` socket, the developer is responsible for detecting and suppressing this warning.

> **Note** :
>
> If you are experiencing problems with reading and writing to files and you're using the server module version of PHP, remember to make sure that the files and directories you're using are accessible to the server process.

> **Note** :

> This function may also succeed when `filename` is a directory. If you are unsure whether `filename` is a file or a directory, you may need to use the is_dir() function before calling **fopen()**.

See Also

- Supported Protocols and Wrappers
- fclose() - Closes an open file pointer
- fgets() - Gets line from file pointer
- fread() - Binary-safe file read
- fwrite() - Binary-safe file write
- fsockopen() - Open Internet or Unix domain socket connection
- file() - Reads entire file into an array
- file_exists() - Checks whether a file or directory exists
- is_readable() - Tells whether a file exists and is readable
- stream_set_timeout() - Set timeout period on a stream
- popen() - Opens process file pointer
- stream_context_create() - Creates a stream context
- umask() - Changes the current umask
- SplFileObject

fpassthru

fpassthru

(PHP 4, PHP 5, PHP 7, PHP 8)

fpassthru — Output all remaining data on a file pointer

Description

function **fpassthru**(resource `$stream`): int

Reads to EOF on the given file pointer from the current position and writes the results to the output buffer.

You may need to call rewind() to reset the file pointer to the beginning of the file if you have already written data to the file.

If you just want to dump the contents of a file to the output buffer, without first modifying it or seeking to a particular offset, you may want to use the readfile(), which saves you the fopen() call.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

Return Values

Returns the number of characters read from `stream` and passed through to the output.

Examples

Example #1 Using **fpassthru()** with binary files

```
<?php

// open the file in a binary mode
$name = './img/ok.png';
$fp = fopen($name, 'rb');

// send the right headers
header("Content-Type: image/png");
header("Content-Length: " . filesize($name));

// dump the picture and stop the script
fpassthru($fp);
exit;

?>
```

Notes

> **Note** :
>
> When using **fpassthru()** on a binary file on Windows systems, you should make sure to open the file in binary mode by appending a `b` to the mode used in the call to fopen().
>
> You are encouraged to use the `b` flag when dealing with binary files, even if your system does not require it, so that your scripts will be more portable.

See Also

- readfile() - Outputs a file
- fopen() - Opens file or URL
- popen() - Opens process file pointer
- fsockopen() - Open Internet or Unix domain socket connection

fputcsv

fputcsv

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

fputcsv — Format line as CSV and write to file pointer

Description

```
function fputcsv(
    resource $stream,
    array $fields,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\",
    string $eol = "\n"
): int|false
```

fputcsv() formats a line (passed as a `fields` array) as CSV and writes it (terminated by a `eol`) to the specified `stream`.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

`fields`

An array of strings.

`separator`

The `separator` parameter sets the field separator. It must be a single byte character.

`enclosure`

The `enclosure` parameter sets the field enclosure character. It must be a single byte character.

`escape`

The `escape` parameter sets the escape character. It must be a single byte character or the empty string. The empty string (`""`) disables the proprietary escape mechanism.

> **Warning**
>
> In the input stream, the `enclosure` character can always be escaped by doubling it inside a quoted string, resulting in a single `enclosure` character in the parsed result. The `escape` character works differently: If a sequence of `escape` and `enclosure` characters

appear in the input, both characters will be present in the parsed result. So for the default parameters, a CSV line like `"a""b","c\"d"` will have the fields parsed as `a"b` and `c\"d`, respectively.

> **Warning**
>
> As of PHP 8.4.0, depending on the default value of `escape` is deprecated. It needs to be provided explicitly either positionally or by the use of named arguments.

`eol`

The optional `eol` parameter sets a custom End of Line sequence.

> **Warning**
>
> When `escape` is set to anything other than an empty string (`""`) it can result in CSV that is not compliant with » RFC 4180 or unable to survive a roundtrip through the PHP CSV functions. The default for `escape` is `"\\"` so it is recommended to set it to the empty string explicitly. The default value will change in a future version of PHP, no earlier than PHP 9.0.

> **Note** :
>
> If an `enclosure` character is contained in a field, it will be escaped by doubling it, unless it is immediately preceded by an `escape`.

Return Values

Returns the length of the written string or `false` on failure.

Errors/Exceptions

Throws a ValueError if `separator` or `enclosure` is not one byte long.

Throws a ValueError if `escape` is not one byte long or the empty string.

Changelog

Version	Description
8.4.0	Relying on the default value of `escape` is now deprecated.
8.1.0	The optional `eol` parameter has been added.
7.4.0	The `escape` parameter now also accepts an empty string to disable the proprietary escape mechanism.

Examples

Example #1 fputcsv() example

```php
<?php

$list = [
    ['aaa', 'bbb', 'ccc', 'dddd'],
    ['123', '456', '789'],
    ['"aaa"', '"bbb"']
];

$fp = fopen('file.csv', 'w');

foreach ($list as $fields) {
    fputcsv($fp, $fields, ',', '"', '');
}

fclose($fp);
?>
```

The above example will write the following to `file.csv`:

```
aaa,bbb,ccc,dddd
123,456,789
"""aaa""","""bbb"""
```

See Also

- fgetcsv() - Gets line from file pointer and parse for CSV fields
- str_getcsv() - Parse a CSV string into an array
- SplFileObject::fgetcsv() - Gets line from file and parse as CSV fields
- SplFileObject::fputcsv() - Write a field array as a CSV line
- SplFileObject::setCsvControl() - Set the delimiter, enclosure and escape character for CSV
- SplFileObject::getCsvControl() - Get the delimiter, enclosure and escape character for CSV

fputs

fputs

(PHP 4, PHP 5, PHP 7, PHP 8)

fputs — Alias of fwrite()

Description

This function is an alias of: fwrite().

fread

fread

(PHP 4, PHP 5, PHP 7, PHP 8)

fread — Binary-safe file read

Description

function **fread**(resource `$stream`, int `$length`): string|false

fread() reads up to `length` bytes from the file pointer referenced by `stream`. Reading stops as soon as one of the following conditions is met:

- `length` bytes have been read
- EOF (end of file) is reached
- a packet becomes available or the socket timeout occurs (for network streams)
- if the stream is read buffered and it does not represent a plain file, at most one read of up to a number of bytes equal to the chunk size (usually 8192) is made; depending on the previously buffered data, the size of the returned data may be larger than the chunk size.

Parameters

`stream`

A file system pointer resource that is typically created using fopen().

`length`

Up to `length` number of bytes read.

Return Values

Returns the read string or `false` on failure.

Examples

Example #1 A simple fread() example

```php
<?php
// get contents of a file into a string
$filename = "/usr/local/something.txt";
$handle = fopen($filename, "r");
$contents = fread($handle, filesize($filename));
fclose($handle);
?>
```

Example #2 Binary fread() example

> **Warning**
>
> On systems which differentiate between binary and text files (i.e. Windows) the file must be opened with 'b' included in fopen() mode parameter.

```php
<?php
$filename = "c:\\files\\somepic.gif";
$handle = fopen($filename, "rb");
$contents = fread($handle, filesize($filename));
fclose($handle);
?>
```

Example #3 Remote fread() examples

> **Warning**
>
> When reading from anything that is not a regular local file, such as streams returned when reading remote files or from popen() and fsockopen(), reading will stop after a packet is available. This means that you should collect the data together in chunks as shown in the examples below.

```php
<?php
$handle = fopen("http://www.example.com/", "rb");
$contents = stream_get_contents($handle);
fclose($handle);
?>
```

```php
<?php
$handle = fopen("http://www.example.com/", "rb");
if (FALSE === $handle) {
    exit("Failed to open stream to URL");
}

$contents = '';

while (!feof($handle)) {
    $contents .= fread($handle, 8192);
}
fclose($handle);
?>
```

Notes

> **Note** :
>
> If you just want to get the contents of a file into a string, use file_get_contents() as it has much better performance than the code above.

> **Note** :
>
> Note that **fread()** reads from the current position of the file pointer. Use ftell() to find the current position of the pointer and rewind() to rewind the pointer position.

See Also

- fwrite() - Binary-safe file write
- fopen() - Opens file or URL
- fsockopen() - Open Internet or Unix domain socket connection
- popen() - Opens process file pointer
- fgets() - Gets line from file pointer
- fgetss() - Gets line from file pointer and strip HTML tags
- fscanf() - Parses input from a file according to a format
- file() - Reads entire file into an array
- fpassthru() - Output all remaining data on a file pointer
- fseek() - Seeks on a file pointer
- ftell() - Returns the current position of the file read/write pointer
- rewind() - Rewind the position of a file pointer
- unpack() - Unpack data from binary string

fscanf

fscanf

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

fscanf — Parses input from a file according to a format

Description

function **fscanf**(resource `$stream`, string `$format`, mixed `&...$vars`): array|int|false|null

The function **fscanf()** is similar to sscanf(), but it takes its input from a file associated with `stream` and interprets the input according to the specified `format`.

Any whitespace in the format string matches any whitespace in the input stream. This means that even a tab (`\t`) in the format string can match a single space character in the input stream.

Each call to **fscanf()** reads one line from the file.

Parameters

`stream`

A file system pointer resource that is typically created using fopen().

`format`

The interpreted format for `string`, which is described in the documentation for sprintf() with following differences:

- Function is not locale-aware.
- `F`, `g`, `G` and `b` are not supported.
- `D` stands for decimal number.
- `i` stands for integer with base detection.
- `n` stands for number of characters processed so far.
- `s` stops reading at any whitespace character.
- `*` instead of `argnum$` suppresses the assignment of this conversion specification.

`vars`

The optional assigned values.

Return Values

If only two parameters were passed to this function, the values parsed will be returned as an array. Otherwise, if optional parameters are passed, the function will return the number of assigned values. The optional parameters must be passed by reference.

If there are more substrings expected in the `format` than there are available within `string`, **`null`** will be returned. On other errors, **`false`** will be returned.

When optional parameters are used and the end of the input read from `stream` is reached before any value has been parsed, `-1` is returned.

Examples

Example #1 fscanf() Example

```php
<?php
$handle = fopen("users.txt", "r");
while ($userinfo = fscanf($handle, "%s\t%s\t%s\n")) {
    list ($name, $profession, $countrycode) = $userinfo;
    //... do something with the values
}
fclose($handle);
?>
```

Example #2 Contents of users.txt

```
javier  argonaut        pe
hiroshi sculptor        jp
robert  slacker us
luigi   florist it
```

See Also

- fread() - Binary-safe file read
- fgets() - Gets line from file pointer
- fgetss() - Gets line from file pointer and strip HTML tags
- sscanf() - Parses input from a string according to a format
- printf() - Output a formatted string
- sprintf() - Return a formatted string

fseek

fseek

(PHP 4, PHP 5, PHP 7, PHP 8)

fseek — Seeks on a file pointer

Description

function **fseek**(resource `$stream`, int `$offset`, int `$whence` = **`SEEK_SET`**): int

Sets the file position indicator for the file referenced by `stream`. The new position, measured in bytes from the beginning of the file, is obtained by adding `offset` to the position specified by `whence`.

In general, it is allowed to seek past the end-of-file; if data is then written, reads in any unwritten region between the end-of-file and the sought position will yield bytes with value 0. However, certain streams may not support this behavior, especially when they have an underlying fixed size storage.

Parameters

`stream`

A file system pointer resource that is typically created using fopen().

`offset`

The offset.

To move to a position before the end-of-file, a negative value in the `offset` must be passed and the `whence` parameter must be set to **`SEEK_END`**.

`whence`

`whence` values are:

- **`SEEK_SET`** - Set position equal to `offset` bytes from the start of the file.
- **`SEEK_CUR`** - Set position to current location plus `offset` bytes.
- **`SEEK_END`** - Set position to end-of-file plus `offset` bytes.

Return Values

Upon success, returns `0`; otherwise, returns `-1`.

> **Warning**
>
> This function was created to imitate the C language function of the same name. Please pay attention to the return values as they differ from what one would expect in PHP.

Examples

Example #1 fseek() example

```php
<?php

$fp = fopen('somefile.txt', 'r');

// read some data
$data = fgets($fp, 4096);

// move back to the beginning of the file
// same as rewind($fp);
fseek($fp, 0);

?>
```

Notes

> **Note** :
>
> If you have opened the file in append (`a` or `a+`) mode, any data you write to the file will always be appended, regardless of the file position, and the result of calling **fseek()** will be undefined.

> **Note** :
>
> Not all streams support seeking. For those that do not support seeking, forward seeking from the current position is accomplished by reading and discarding data; other forms of seeking will fail.

See Also

- ftell() - Returns the current position of the file read/write pointer
- rewind() - Rewind the position of a file pointer

fstat

fstat

(PHP 4, PHP 5, PHP 7, PHP 8)

fstat — Gets information about a file using an open file pointer

Description

function **fstat**(resource `$stream`): array|false

Gathers the statistics of the file opened by the file pointer `stream`. This function is similar to the stat() function except that it operates on an open file pointer instead of a filename.

Parameters

`stream`

A file system pointer resource that is typically created using fopen().

Return Values

Returns an array with the statistics of the file; the format of the array is described in detail on the stat() manual page. Returns `false` on failure.

Examples

Example #1 fstat() example

```php
<?php

// open a file
$fp = fopen("/etc/passwd", "r");

// gather statistics
$fstat = fstat($fp);

// close the file
fclose($fp);

// print only the associative part
print_r(array_slice($fstat, 13));

?>
```

The above example will output something similar to:

```
Array
(
    [dev] => 771
    [ino] => 488704
    [mode] => 33188
    [nlink] => 1
    [uid] => 0
    [gid] => 0
    [rdev] => 0
    [size] => 1114
    [atime] => 1061067181
    [mtime] => 1056136526
    [ctime] => 1056136526
    [blksize] => 4096
    [blocks] => 8
)
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

fsync

fsync

(PHP 8 >= 8.1.0)

fsync — Synchronizes changes to the file (including meta-data)

Description

function **fsync**(resource `$stream`): bool

This function synchronizes changes to the file, including its meta-data. This is similar to fflush(), but it also instructs the operating system to write to the storage media.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or fsockopen() (and not yet closed by fclose()).

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 fsync() example

```php
<?php

$file = 'test.txt';

$stream = fopen($file, 'w');
fwrite($stream, 'test data');
fwrite($stream, "\r\n");
fwrite($stream, 'additional data');

fsync($stream);
fclose($stream);
?>
```

See Also

- fdatasync() - Synchronizes data (but not meta-data) to the file
- fflush() - Flushes the output to a file

ftell

ftell

(PHP 4, PHP 5, PHP 7, PHP 8)

ftell — Returns the current position of the file read/write pointer

Description

function **ftell**(resource `$stream`): int|false

Returns the position of the file pointer referenced by `stream`.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen() or popen(). **ftell()** gives undefined results for append-only streams (opened with "a" flag).

Return Values

Returns the position of the file pointer referenced by `stream` as an integer; i.e., its offset into the file stream.

If an error occurs, returns `false`.

> **Note** :
>
> Because PHP's integer type is signed and many platforms use 32bit integers, some filesystem functions may return unexpected results for files which are larger than 2GB.

Examples

Example #1 ftell() example

```php
<?php

// opens a file and read some data
$fp = fopen("/etc/passwd", "r");
```

```php
$data = fgets($fp, 12);

// where are we ?
echo ftell($fp); // 11

fclose($fp);

?>
```

See Also

- fopen() - Opens file or URL
- popen() - Opens process file pointer
- fseek() - Seeks on a file pointer
- rewind() - Rewind the position of a file pointer

ftruncate

ftruncate

(PHP 4, PHP 5, PHP 7, PHP 8)

ftruncate — Truncates a file to a given length

Description

function **ftruncate**(resource `$stream`, int `$size`): bool

Takes the filepointer, `stream`, and truncates the file to length, `size`.

Parameters

`stream`

The file pointer.

> **Note** :
>
> The `stream` must be open for writing.

`size`

The size to truncate to.

> **Note** :
>
> If `size` is larger than the file then the file is extended with null bytes.
>
> If `size` is smaller than the file then the file is truncated to that size.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 File truncation example

```php
<?php
$filename = 'lorem_ipsum.txt';

$handle = fopen($filename, 'r+');
ftruncate($handle, rand(1, filesize($filename)));
rewind($handle);
echo fread($handle, filesize($filename));
fclose($handle);
?>
```

Notes

> **Note** :
>
> The file pointer is *not* changed.

See Also

- fopen() - Opens file or URL
- fseek() - Seeks on a file pointer

fwrite

fwrite

(PHP 4, PHP 5, PHP 7, PHP 8)

fwrite — Binary-safe file write

Description

function **fwrite**(resource `$stream`, string `$data`, ?int `$length` = `null`): int|false

fwrite() writes the contents of `data` to the file stream pointed to by `stream`.

Parameters

`stream`

A file system pointer resource that is typically created using fopen().

`data`

The string that is to be written.

`length`

If `length` is an int, writing will stop after `length` bytes have been written or the end of `data` is reached, whichever comes first.

Return Values

fwrite() returns the number of bytes written, or `false` on failure.

Errors/Exceptions

fwrite() raises `E_WARNING` on failure.

Changelog

Version	Description
8.0.0	`length` is nullable now.

Examples

Example #1 A simple **fwrite()** example

```
<?php
$filename = 'test.txt';
$somecontent = "Add this to the file\n";

// Let's make sure the file exists and is writable first.
if (is_writable($filename)) {

    // In our example we're opening $filename in append mode.
    // The file pointer is at the bottom of the file hence
    // that's where $somecontent will go when we fwrite() it.
    if (!$fp = fopen($filename, 'a')) {
         echo "Cannot open file ($filename)";
         exit;
```

```
    }

    // Write $somecontent to our opened file.
    if (fwrite($fp, $somecontent) === FALSE) {
        echo "Cannot write to file ($filename)";
        exit;
    }

    echo "Success, wrote ($somecontent) to file ($filename)";

    fclose($fp);

} else {
    echo "The file $filename is not writable";
}
?>
```

Notes

> **Note** :
>
> Writing to a network stream may end before the whole string is written. Return value of **fwrite()** may be checked:

```
<?php
function fwrite_stream($fp, $string) {
    for ($written = 0; $written < strlen($string); $written += $fwrite) {
        $fwrite = fwrite($fp, substr($string, $written));
        if ($fwrite === false) {
            return $written;
        }
    }
    return $written;
}
?>
```

> **Note** :
>
> On systems which differentiate between binary and text files (i.e. Windows) the file must be opened with 'b' included in fopen() mode parameter.

> **Note** :
>
> If `stream` was fopen()ed in append mode, **fwrite()**s are atomic (unless the size of `data` exceeds the filesystem's block size, on some platforms, and as long as the file is on a local filesystem). That is, there is no need to flock() a resource before calling **fwrite()**; all of the data will be written without interruption.

> **Note** :
>
> If writing twice to the file pointer, then the data will be appended to the end of the file content:

```
<?php
$fp = fopen('data.txt', 'w');
fwrite($fp, '1');
fwrite($fp, '23');
fclose($fp);

// the content of 'data.txt' is now 123 and not 23!
?>
```

See Also

- fread() - Binary-safe file read
- fopen() - Opens file or URL
- fsockopen() - Open Internet or Unix domain socket connection

- popen() - Opens process file pointer
- file_get_contents() - Reads entire file into a string
- pack() - Pack data into binary string

glob

glob

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

glob — Find pathnames matching a pattern

Description

function **glob**(string `$pattern`, int `$flags` = 0): array|false

The **glob()** function searches for all the pathnames matching `pattern` according to the rules used by the libc glob() function, which is similar to the rules used by common shells.

The behavior on Unix systems and macOS is determined by the system's implementation of glob(). On Windows, an implementation that conforms to the POSIX 1003.2 definition for glob() is used, and it includes an extension to handle the `[!...]` convention for negating a range.

Parameters

`pattern`

The pattern. No tilde expansion or parameter substitution is done.

Special characters:

- `*` - Matches zero or more characters.
- `?` - Matches exactly one character (any character).
- `[...]` - Matches one character from a group of characters. If the first character is `!`, matches any character not in the group.
- `{a,b,c}` - Matches one string from a group of strings delimited by a comma when the **GLOB_BRACE** flag is used.
- `\` - Escapes the following character, except when the **GLOB_NOESCAPE** flag is used.

`flags`

Any of the **GLOB_*** constants.

Return Values

Returns an array containing the matched files/directories, an empty array if no file matched or **false** on error. Unless GLOB_NOSORT was used, the names will be sorted alphanumerically.

Examples

Example #1 Convenient way how glob() can replace opendir() and friends.

```php
<?php
foreach (glob("*.txt") as $filename) {
    echo "$filename size " . filesize($filename) . "\n";
}
?>
```

The above example will output something similar to:

```
funclist.txt size 44686
funcsummary.txt size 267625
quickref.txt size 137820
```

Example #2 Example with a more complex pattern

```php
<?php
foreach (glob("path/*/*.{txt,md}", \GLOB_BRACE) as $filename) {
    echo "$filename\n";
}
?>
```

The above example will output something similar to:

```
path/docs/mailinglist-rules.md
path/docs/README.md
path/docs/release-process.md
path/pear/install-pear.txt
path/Zend/README.md
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

> **Note** :
>
> This function isn't available on some systems (e.g. old Sun OS).

See Also

- opendir() - Open directory handle
- readdir() - Read entry from directory handle
- closedir() - Close directory handle
- fnmatch() - Match filename against a pattern

is_dir

is_dir

(PHP 4, PHP 5, PHP 7, PHP 8)

is_dir — Tells whether the filename is a directory

Description

function **is_dir**(string `$filename`): bool

Tells whether the given filename is a directory.

Parameters

`filename`

Path to the file. If `filename` is a relative filename, it will be checked relative to the current working directory. If `filename` is a symbolic or hard link then the link will be resolved and checked. If you have enabled open_basedir further restrictions may apply.

Return Values

Returns `true` if the filename exists and is a directory, `false` otherwise.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 is_dir() example

```php
<?php
var_dump(is_dir('a_file.txt'));
var_dump(is_dir('bogus_dir/abc'));

var_dump(is_dir('..')); //one dir up
?>
```

The above example will output:

```
bool(false)
bool(false)
bool(true)
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- chdir() - Change directory
- dir() - Return an instance of the Directory class
- opendir() - Open directory handle
- is_file() - Tells whether the filename is a regular file
- is_link() - Tells whether the filename is a symbolic link

is_executable

is_executable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_executable — Tells whether the filename is executable

Description

function **is_executable**(string `$filename`): bool

Tells whether the filename is executable.

Parameters

`filename`

Path to the file.

Return Values

Returns `true` if the filename exists and is executable, or `false` on error. On POSIX systems, a file is executable if the executable bit of the file permissions is set. For Windows, see the note below.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 is_executable() example

```
<?php

$file = '/home/vincent/somefile.sh';

if (is_executable($file)) {
    echo $file.' is executable';
} else {
    echo $file.' is not executable';
}

?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

> **Note** :
>
> On Windows, a file is considered executable, if it is a properly executable file as reported by the Win API `GetBinaryType()`; for BC reasons, files with a *.bat* or *.cmd* extension are also considered executable. Prior to PHP 7.4.0, any non-empty file with a *.exe* or *.com* extension was considered executable. Note that *PATHEXT* is irrelevant for **is_executable()**.

See Also

- is_file() - Tells whether the filename is a regular file
- is_link() - Tells whether the filename is a symbolic link

is_file

is_file

(PHP 4, PHP 5, PHP 7, PHP 8)

is_file — Tells whether the filename is a regular file

Description

function **is_file**(string `$filename`): bool

Tells whether the given file is a regular file. If `filename` is a symbolic link, it will resolve the symlink and will give information about the referred file.

Parameters

`filename`

Path to the file.

Return Values

Returns `true` if the filename exists and is a regular file, `false` otherwise.

> **Note** :
>
> Because PHP's integer type is signed and many platforms use 32bit integers, some filesystem functions may return unexpected results for files which are larger than 2GB.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 is_file() example

```
<?php
var_dump(is_file('a_file.txt')) . "\n";
var_dump(is_file('/usr/bin/')) . "\n";
?>
```

The above example will output:

```
bool(true)
bool(false)
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- is_dir() - Tells whether the filename is a directory
- is_link() - Tells whether the filename is a symbolic link
- SplFileInfo

is_link

is_link

(PHP 4, PHP 5, PHP 7, PHP 8)

is_link — Tells whether the filename is a symbolic link

Description

function **is_link**(string `$filename`): bool

Tells whether the given file is a symbolic link.

Parameters

`filename`

Path to the file.

Return Values

Returns `true` if the filename exists and is a symbolic link, `false` otherwise.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 Create and confirm if a file is a symbolic link

```
<?php
$link = 'uploads';

if (is_link($link)) {
    echo readlink($link);
} else {
    symlink('uploads.php', $link);
}
?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- is_dir() - Tells whether the filename is a directory
- is_file() - Tells whether the filename is a regular file
- readlink() - Returns the target of a symbolic link
- symlink() - Creates a symbolic link

is_readable

is_readable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_readable — Tells whether a file exists and is readable

Description

function **is_readable**(string `$filename`): bool

Tells whether a file exists and is readable.

Parameters

`filename`

Path to the file.

Return Values

Returns `true` if the file or directory specified by `filename` exists and is readable, `false` otherwise.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 is_readable() example

```php
<?php
$filename = 'test.txt';
if (is_readable($filename)) {
    echo 'The file is readable';
} else {
    echo 'The file is not readable';
}
?>
```

Notes

Keep in mind that PHP may be accessing the file as the user id that the web server runs as (often 'nobody').

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

> **Note** :
>
> The check is done using the real UID/GID instead of the effective one.

This function may return `true` for directories. Use is_dir() to distinguish file and directory.

See Also

- is_writable() - Tells whether the filename is writable
- file_exists() - Checks whether a file or directory exists
- fgets() - Gets line from file pointer

is_uploaded_file

is_uploaded_file

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

is_uploaded_file — Tells whether the file was uploaded via HTTP POST

Description

function **is_uploaded_file**(string `$filename`): bool

Returns `true` if the file named by `filename` was uploaded via HTTP POST. This is useful to help ensure that a malicious user hasn't tried to trick the script into working on files upon which it should not be working--for instance, */etc/passwd*.

This sort of check is especially important if there is any chance that anything done with uploaded files could reveal their contents to the user, or even to other users on the same system.

For proper working, the function **is_uploaded_file()** needs an argument like *$_FILES['userfile']['tmp_name']*, - the name of the uploaded file on the client's machine *$_FILES['userfile']['name']* does not work.

Parameters

`filename`

The filename being checked.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 is_uploaded_file() example

```php
<?php

if (is_uploaded_file($_FILES['userfile']['tmp_name'])) {
   echo "File ". $_FILES['userfile']['name'] ." uploaded successfully.\n";
   echo "Displaying contents\n";
   readfile($_FILES['userfile']['tmp_name']);
} else {
   echo "Possible file upload attack: ";
   echo "filename '". $_FILES['userfile']['tmp_name'] . "'.";
}

?>
```

See Also

- move_uploaded_file() - Moves an uploaded file to a new location
- *$_FILES*
- See Handling file uploads for a simple usage example.

is_writable

is_writable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_writable — Tells whether the filename is writable

Description

function **is_writable**(string `$filename`): bool

Returns `true` if the `filename` exists and is writable. The filename argument may be a directory name allowing you to check if a directory is writable.

Keep in mind that PHP may be accessing the file as the user id that the web server runs as (often 'nobody').

Parameters

`filename`

The filename being checked.

Return Values

Returns `true` if the `filename` exists and is writable.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 is_writable() example

```php
<?php
$filename = 'test.txt';
if (is_writable($filename)) {
    echo 'The file is writable';
} else {
    echo 'The file is not writable';
```

```
}
?>
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- is_readable() - Tells whether a file exists and is readable
- file_exists() - Checks whether a file or directory exists
- fwrite() - Binary-safe file write

is_writeable

is_writeable

(PHP 4, PHP 5, PHP 7, PHP 8)

is_writeable — Alias of is_writable()

Description

This function is an alias of: is_writable().

lchgrp

lchgrp

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

lchgrp — Changes group ownership of symlink

Description

function **lchgrp**(string `$filename`, string|int `$group`): bool

Attempts to change the group of the symlink `filename` to `group`.

Only the superuser may change the group of a symlink arbitrarily; other users may change the group of a symlink to any group of which that user is a member.

Parameters

`filename`

Path to the symlink.

`group`

The group specified by name or number.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Changing the group of a symbolic link

```php
<?php
$target = 'output.php';
$link = 'output.html';
symlink($target, $link);

lchgrp($link, 8);
?>
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

> **Note** :
>
> This function is not implemented on Windows platforms.

See Also

- chgrp() - Changes file group
- lchown() - Changes user ownership of symlink
- chown() - Changes file owner
- chmod() - Changes file mode

lchown

lchown

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

lchown — Changes user ownership of symlink

Description

function **lchown**(string `$filename`, string|int `$user`): bool

Attempts to change the owner of the symlink `filename` to user `user`.

Only the superuser may change the owner of a symlink.

Parameters

`filename`

Path to the file.

`user`

User name or number.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Changing the owner of a symbolic link

```php
<?php
$target = 'output.php';
$link = 'output.html';
symlink($target, $link);

lchown($link, 8);
?>
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

> **Note** :
>
> This function is not implemented on Windows platforms.

See Also

- chown() - Changes file owner
- lchgrp() - Changes group ownership of symlink
- chgrp() - Changes file group
- chmod() - Changes file mode

link

link

(PHP 4, PHP 5, PHP 7, PHP 8)

link — Create a hard link

Description

function **link**(string `$target`, string `$link`): bool

link() creates a hard link.

Parameters

`target`

Target of the link.

`link`

The link name.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

The function fails, and issues `E_WARNING`, if `link` already exists, or if `target` does not exist.

Examples

Example #1 Creating a simple hard link

```php
<?php
$target = 'source.ext'; // This is the file that already exists
$link = 'newfile.ext'; // This the filename that you want to link it to

link($target, $link);
?>
```

Notes

> **Note** :
>
> This function will not work on remote files as the file to be examined must be accessible via the server's filesystem.

> **Note** :
>
> For Windows only: This function requires PHP to run in an elevated mode or with the UAC disabled.

See Also

- symlink() - Creates a symbolic link
- readlink() - Returns the target of a symbolic link
- linkinfo() - Gets information about a link
- unlink() - Deletes a file

linkinfo

linkinfo

(PHP 4, PHP 5, PHP 7, PHP 8)

linkinfo — Gets information about a link

Description

function **linkinfo**(string `$path`): int|false

Gets information about a link.

This function is used to verify if a link (pointed to by `path`) really exists (using the same method as the S_ISLNK macro defined in *stat.h*).

Parameters

`path`

Path to the link.

Return Values

linkinfo() returns the `st_dev` field of the Unix C stat structure returned by the `lstat` system call. Returns a non-negative integer on success, -1 in case the link was not found, or `false` if an open.base_dir violation occurs.

Examples

Example #1 linkinfo() example

```php
<?php

echo linkinfo('/vmlinuz'); // 835

?>
```

See Also

- symlink() - Creates a symbolic link
- link() - Create a hard link
- readlink() - Returns the target of a symbolic link

lstat

lstat

(PHP 4, PHP 5, PHP 7, PHP 8)

lstat — Gives information about a file or symbolic link

Description

function **lstat**(string `$filename`): array|false

Gathers the statistics of the file or symbolic link named by `filename`.

Parameters

`filename`

Path to a file or a symbolic link.

Return Values

See the manual page for stat() for information on the structure of the array that **lstat()** returns. This function is identical to the stat() function except that if the `filename` parameter is a symbolic link, the status of the symbolic link is returned, not the status of the file pointed to by the symbolic link.

On failure, `false` is returned.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Examples

Example #1 Comparison of stat() and lstat()

```php
<?php
symlink('uploads.php', 'uploads');

// Contrast information for uploads.php and uploads
array_diff(stat('uploads'), lstat('uploads'));
?>
```

The above example will output something similar to:

Information that differs between the two files.

```
Array
(
    [ino] => 97236376
    [mode] => 33188
    [size] => 34
    [atime] => 1223580003
    [mtime] => 1223581848
    [ctime] => 1223581848
    [blocks] => 8
)
```

Notes

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support stat() family of functionality.

See Also

- stat() - Gives information about a file

mkdir

mkdir

(PHP 4, PHP 5, PHP 7, PHP 8)

mkdir — Makes directory

Description

```
function mkdir(
    string $directory,
    int $permissions = 0777,
    bool $recursive = false,
    ?resource $context = null
): bool
```

Attempts to create the directory specified by `directory`.

Parameters

`directory`

The directory path.

> **Tip**
>
> A URL can be used as a filename with this function if the fopen wrappers have been enabled. See fopen() for more details on how to specify the filename. See the Supported Protocols and Wrappers for links to information about what abilities the various wrappers have, notes on their usage, and information on any predefined variables they may provide.

`permissions`

The permissions are 0777 by default, which means the widest possible access. For more information on permissions, read the details on the chmod() page.

> **Note** :
>
> `permissions` is ignored on Windows.

Note that you probably want to specify the `permissions` as an octal number, which means it should have a leading zero. The `permissions` is also modified by the current umask, which you can change using umask().

`recursive`

If `true`, then any parent directories to the `directory` specified will also be created, with the same permissions.

`context`

A context stream resource.

Return Values

Returns `true` on success or `false` on failure.

> **Note** :
>
> If the directory to be created already exists, that is considered an error and `false` will still be returned. Use is_dir() or file_exists() to check if the directory already exists before trying to create it.

Errors/Exceptions

Emits an `E_WARNING` level error if the directory already exists.

Emits an `E_WARNING` level error if the relevant permissions prevent creating the directory.

Examples

Example #1 mkdir() example

```php
<?php
mkdir("/path/to/my/dir", 0700);
?>
```

Example #2 mkdir() using the `recursive` parameter

```php
<?php
// Desired directory structure
$structure = './depth1/depth2/depth3/';

// To create the nested structure, the $recursive parameter
// to mkdir() must be specified.

if (!mkdir($structure, 0777, true)) {
    die('Failed to create directories...');
}

// ...
?>
```

See Also

- is_dir() - Tells whether the filename is a directory
- rmdir() - Removes directory
- umask() - Changes the current umask

move_uploaded_file

move_uploaded_file

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

move_uploaded_file — Moves an uploaded file to a new location

Description

function **move_uploaded_file**(string `$from`, string `$to`): bool

This function checks to ensure that the file designated by `from` is a valid upload file (meaning that it was uploaded via PHP's HTTP POST upload mechanism). If the file is valid, it will be moved to the filename given by `to`.

This sort of check is especially important if there is any chance that anything done with uploaded files could reveal their contents to the user, or even to other users on the same system.

Parameters

`from`

The filename of the uploaded file.

`to`

The destination of the moved file.

Return Values

Returns `true` on success.

If `from` is not a valid upload file, then no action will occur, and **move_uploaded_file()** will return `false`.

If `from` is a valid upload file, but cannot be moved for some reason, no action will occur, and **move_uploaded_file()** will return `false`. Additionally, a warning will be issued.

Examples

Example #1 Uploading multiple files

```php
<?php
$uploads_dir = '/uploads';
foreach ($_FILES["pictures"]["error"] as $key => $error) {
    if ($error == UPLOAD_ERR_OK) {
        $tmp_name = $_FILES["pictures"]["tmp_name"][$key];
        // basename() may prevent filesystem traversal attacks;
        // further validation/sanitation of the filename may be appropriate
        $name = basename($_FILES["pictures"]["name"][$key]);
        move_uploaded_file($tmp_name, "$uploads_dir/$name");
    }
}
?>
```

Notes

> **Note** :
>
> **move_uploaded_file()** is open_basedir aware. However, restrictions are placed only on the `to` path as to allow the moving of uploaded files in which `from` may conflict with such restrictions. **move_uploaded_file()** ensures the safety of this operation by allowing only those files uploaded through PHP to be moved.

> **Warning**
>
> If the destination file already exists, it will be overwritten.

See Also

- is_uploaded_file() - Tells whether the file was uploaded via HTTP POST
- rename() - Renames a file or directory
- See Handling file uploads for a simple usage example

parse_ini_file

parse_ini_file

(PHP 4, PHP 5, PHP 7, PHP 8)

parse_ini_file — Parse a configuration file

Description

function **parse_ini_file**(string `$filename`, bool `$process_sections` = **`false`**, int `$scanner_mode` = **`INI_SCANNER_NORMAL`**): array|false

parse_ini_file() loads in the ini file specified in `filename`, and returns the settings in it in an associative array.

The structure of the ini file is the same as the *php.ini*'s.

> **Warning**
>
> This function must not be used with untrusted inputs, unless `scanner_mode` is **`INI_SCANNER_RAW`** since the parsed output might contain the values of sensitive constants, such as constants holding a database password.

Parameters

`filename`

> The filename of the ini file being parsed. If a relative path is used, it is evaluated relative to the current working directory, then the include_path.

`process_sections`

> By setting the `process_sections` parameter to **`true`**, you get a multidimensional array, with the section names and settings included. The default for `process_sections` is **`false`**

`scanner_mode`

Can either be `INI_SCANNER_NORMAL` (default) or `INI_SCANNER_RAW`. If `INI_SCANNER_RAW` is supplied, then option values will not be parsed.

As of PHP 5.6.1 can also be specified as `INI_SCANNER_TYPED`. In this mode boolean, null and integer types are preserved when possible. String values `"true"`, `"on"` and `"yes"` are converted to `true`. `"false"`, `"off"`, `"no"` and `"none"` are considered `false`. `"null"` is converted to `null` in typed mode. Also, all numeric strings are converted to integer type if it is possible.

Return Values

The settings are returned as an associative array on success, and `false` on failure.

Examples

Example #1 Contents of *sample.ini*

```
; This is a sample configuration file
; Comments start with ';', as in php.ini

[first_section]
one = 1
five = 5
animal = BIRD

[second_section]
path = "/usr/local/bin"
URL = "http://www.example.com/~username"

[third_section]
phpversion[] = "5.0"
phpversion[] = "5.1"
phpversion[] = "5.2"
phpversion[] = "5.3"

urls[svn] = "http://svn.php.net"
urls[git] = "http://git.php.net"
```

Example #2 parse_ini_file() example

Constants (but not "magic constants" like `__FILE__`) may also be parsed in the ini file so if you define a constant as an ini value before running **parse_ini_file()**, it will be integrated into the results. Only ini values are evaluated, and the value must be just the constant. For example:

```
<?php

define('BIRD', 'Dodo bird');

// Parse without sections
$ini_array = parse_ini_file("sample.ini");
print_r($ini_array);

// Parse with sections
$ini_array = parse_ini_file("sample.ini", true);
print_r($ini_array);

?>
```

The above example will output something similar to:

```
Array
(
    [one] => 1
    [five] => 5
    [animal] => Dodo bird
    [path] => /usr/local/bin
    [URL] => http://www.example.com/~username
    [phpversion] => Array
        (
            [0] => 5.0
            [1] => 5.1
```

```
            [2] => 5.2
            [3] => 5.3
        )

    [urls] => Array
        (
            [svn] => http://svn.php.net
            [git] => http://git.php.net
        )

)
Array
(
    [first_section] => Array
        (
            [one] => 1
            [five] => 5
            [animal] => Dodo bird
        )

    [second_section] => Array
        (
            [path] => /usr/local/bin
            [URL] => http://www.example.com/~username
        )

    [third_section] => Array
        (
            [phpversion] => Array
                (
                    [0] => 5.0
                    [1] => 5.1
                    [2] => 5.2
                    [3] => 5.3
                )

            [urls] => Array
                (
                    [svn] => http://svn.php.net
                    [git] => http://git.php.net
                )

        )

)
```

Example #3 parse_ini_file() parsing a php.ini file

```php
<?php
// A simple function used for comparing the results below
function yesno($expression)
{
    return($expression ? 'Yes' : 'No');
}

// Get the path to php.ini using the php_ini_loaded_file() function
$ini_path = php_ini_loaded_file();

// Parse php.ini
$ini = parse_ini_file($ini_path);

// Print and compare the values, note that using get_cfg_var()
// will give the same results for parsed and loaded here
echo '(parsed) magic_quotes_gpc = ' . yesno($ini['magic_quotes_gpc']) . PHP_EOL;
echo '(loaded) magic_quotes_gpc = ' . yesno(get_cfg_var('magic_quotes_gpc')) . PHP_EOL;
?>
```

The above example will output something similar to:

```
(parsed) magic_quotes_gpc = Yes
(loaded) magic_quotes_gpc = Yes
```

Example #4 Value Interpolation

In addition to evaluating constants, certain characters have special meaning in an ini value. Additionally, environment variables and previously defined configuration options (see get_cfg_var()) may be read using `${}` syntax.

```
; | is used for bitwise OR
three = 2|3

; & is used for bitwise AND
four = 6&5

; ^ is used for bitwise XOR
five = 3^6

; ~ is used for bitwise negate
negative_two = ~1

; () is used for grouping
seven = (8|7)&(6|5)

; Interpolate the PATH environment variable
path = ${PATH}

; Interpolate the configuration option 'memory_limit'
configured_memory_limit = ${memory_limit}
```

Example #5 Escaping Characters

Some characters have special meaning in double-quoted strings and must be escaped by the backslash prefix. First of all, these are the double quote `"` as the boundary marker, and the backslash `\` itself (if followed by one of the special characters):

```
quoted = "She said \"Exactly my point\"." ; Results in a string with quote marks in it.
hint = "Use \\\" to escape double quote" ; Results in: Use \" to escape double quote
```

There is an exception made for Windows-like paths: it's possible to not escape trailing backslash if the quoted string is directly followed by a linebreak:

```
save_path = "C:\Temp\"
```

If one does need to escape double quote followed by linebreak in a multiline value, it's possible to use value concatenation in the following way (there is one double-quoted string directly followed by another one):

```
long_text = "Lorem \"ipsum\"""
 dolor" ; Results in: Lorem "ipsum"\n dolor
```

Another character with special meaning is `$` (the dollar sign). It must be escaped if followed by the open curly brace:

```
code = "\${test}"
```

Escaping characters is not supported in the `INI_SCANNER_RAW` mode (in this mode all characters are processed "as is").

Note that the ini parser doesn't support standard escape sequences (`\n`, `\t`, etc.). If necessary, post-process the result of **parse_ini_file()** with stripcslashes() function.

Notes

> **Note** :

> This function has nothing to do with the *php.ini* file. It is already processed by the time you run your script. This function can be used to read in your own application's configuration files.

> **Note** :
>
> If a value in the ini file contains any non-alphanumeric characters it needs to be enclosed in double-quotes (").

> **Note** :
>
> There are reserved words which must not be used as keys for ini files. These include: `null`, `yes`, `no`, `true`, `false`, `on`, `off`, `none`. Values `null`, `off`, `no` and `false` result in `""`, and values `on`, `yes` and `true` result in `"1"`, unless **`INI_SCANNER_TYPED`** mode is used. Characters `?{}|&~!()^"` must not be used anywhere in the key and have a special meaning in the value.

> **Note** :
>
> Entries without an equal sign are ignored. For example, "foo" is ignored whereas "bar =" is parsed and added with an empty value. For example, MySQL has a "no-auto-rehash" setting in *my.cnf* that does not take a value, so it is ignored.

> **Note** :
>
> ini files are generally treated as plain text by web servers and thus served to browsers if requested. That means for security you must either keep your ini files outside of your docroot or reconfigure your web server to not serve them. Failure to do either of those may introduce a security risk.

See Also

- parse_ini_string() - Parse a configuration string

parse_ini_string

parse_ini_string

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

parse_ini_string — Parse a configuration string

Description

function **parse_ini_string**(string `$ini_string`, bool `$process_sections` = **`false`**, int `$scanner_mode` = **`INI_SCANNER_NORMAL`**): array|false

parse_ini_string() returns the settings in string `ini_string` in an associative array.

The structure of the ini string is the same as the *php.ini*'s.

> **Warning**
>
> This function must not be used with untrusted inputs, unless `scanner_mode` is **`INI_SCANNER_RAW`** since the parsed output might contain the values of sensitive constants, such as constants holding a database password.

Parameters

`ini_string`

The contents of the ini file being parsed.

`process_sections`

By setting the `process_sections` parameter to **`true`**, you get a multidimensional array, with the section names and settings included. The default for `process_sections` is **`false`**

`scanner_mode`

Can either be `INI_SCANNER_NORMAL` (default) or `INI_SCANNER_RAW`. If `INI_SCANNER_RAW` is supplied, then option values will not be parsed.

As of PHP 5.6.1 can also be specified as `INI_SCANNER_TYPED`. In this mode boolean, null and integer types are preserved when possible. String values `"true"`, `"on"` and `"yes"` are converted to `true`. `"false"`, `"off"`, `"no"` and `"none"` are considered `false`. `"null"` is converted to `null` in typed mode. Also, all numeric strings are converted to integer type if it is possible.

Return Values

The settings are returned as an associative array on success, and `false` on failure.

Notes

> **Note** :
>
> There are reserved words which must not be used as keys for ini files. These include: `null`, `yes`, `no`, `true`, `false`, `on`, `off`, `none`. Values `null`, `off`, `no` and `false` result in `""`, and values `on`, `yes` and `true` result in `"1"`, unless `INI_SCANNER_TYPED` mode is used. Characters `?{}|&~![()^"` must not be used anywhere in the key and have a special meaning in the value.

See Also

- parse_ini_file() - Parse a configuration file

pathinfo

pathinfo

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

pathinfo — Returns information about a file path

Description

function **pathinfo**(string `$path`, int `$flags = PATHINFO_ALL`): array|string

pathinfo() returns information about `path`: either an associative array or a string, depending on `flags`.

> **Note** :
>
> For information on retrieving the current path info, read the section on predefined reserved variables.

> **Note** :
>
> **pathinfo()** operates naively on the input string, and is not aware of the actual filesystem, or path components such as "`..`".

> **Note** :
>
> On Windows systems only, the `\` character will be interpreted as a directory separator. On other systems it will be treated like any other character.

> **Caution**
>
> **pathinfo()** is locale aware, so for it to parse a path containing multibyte characters correctly, the matching locale must be set using the setlocale() function.

Parameters

`path`

The path to be parsed.

`flags`

If present, specifies a specific element to be returned; one of `PATHINFO_DIRNAME`, `PATHINFO_BASENAME`, `PATHINFO_EXTENSION` or `PATHINFO_FILENAME`.

If `flags` is not specified, returns all available elements.

Return Values

If the `flags` parameter is not passed, an associative array containing the following elements is returned: `dirname`, `basename`, `extension` (if any), and `filename`.

> **Note** :
>
> If the `path` has more than one extension, `PATHINFO_EXTENSION` returns only the last one and `PATHINFO_FILENAME` only strips the last one. (see first example below).

> **Note** :
>
> If the `path` does not have an extension, no `extension` element will be returned (see second example below).

> **Note** :
>
> If the `basename` of the `path` starts with a dot, the following characters are interpreted as `extension`, and the `filename` is empty (see third example below).

If `flags` is present, returns a string containing the requested element.

Examples

Example #1 pathinfo() Example

```php
<?php
$path_parts = pathinfo('/www/htdocs/inc/lib.inc.php');

echo $path_parts['dirname'], "\n";
echo $path_parts['basename'], "\n";
echo $path_parts['extension'], "\n";
echo $path_parts['filename'], "\n";
?>
```

The above example will output:

```
/www/htdocs/inc
lib.inc.php
php
lib.inc
```

Example #2 pathinfo() example showing difference between null and no extension

```php
<?php
$path_parts = pathinfo('/path/emptyextension.');
var_dump($path_parts['extension']);

$path_parts = pathinfo('/path/noextension');
var_dump($path_parts['extension']);
?>
```

The above example will output something similar to:

```
string(0) ""

Notice: Undefined index: extension in test.php on line 6
NULL
```

Example #3 pathinfo() example for a dot-file

```php
<?php
print_r(pathinfo('/some/path/.test'));
?>
```

The above example will output something similar to:

```
Array
(
    [dirname] => /some/path
    [basename] => .test
    [extension] => test
    [filename] =>
)
```

Example #4 pathinfo() example with array dereferencing

The `flags` parameter is not a bitmask. Only a single value may be provided. To select only a limited set of parsed values, use array destructuring like so:

```php
<?php
['basename' => $basename, 'dirname' => $dirname] = pathinfo('/www/htdocs/inc/lib.inc.php');

var_dump($basename, $dirname);
?>
```

The above example will output something similar to:

```
string(11) "lib.inc.php"
string(15) "/www/htdocs/inc"
```

See Also

- dirname() - Returns a parent directory's path
- basename() - Returns trailing name component of path
- parse_url() - Parse a URL and return its components
- realpath() - Returns canonicalized absolute pathname

pclose

pclose

(PHP 4, PHP 5, PHP 7, PHP 8)

pclose — Closes process file pointer

Description

function **pclose**(resource `$handle`): int

Closes a file pointer to a pipe opened by popen().

Parameters

`handle`

The file pointer must be valid, and must have been returned by a successful call to popen().

Return Values

Returns the termination status of the process that was run. In case of an error then `-1` is returned.

> **Note** :
>
> If PHP has been compiled with --enable-sigchild, the return value of this function is undefined.

Examples

Example #1 pclose() example

```
<?php
$handle = popen('/bin/ls', 'r');
pclose($handle);
?>
```

Notes

> **Note** : **Unix Only:**
>
> **pclose()** is internally implemented using the `waitpid(3)` system call. To obtain the real exit status code the pcntl_wexitstatus() function should be used.

See Also

- popen() - Opens process file pointer

popen

popen

(PHP 4, PHP 5, PHP 7, PHP 8)

popen — Opens process file pointer

Description

function **popen**(string `$command`, string `$mode`): resource|false

Opens a pipe to a process executed by forking the command given by `command`.

Parameters

`command`

The command

`mode`

The mode. Either `'r'` for reading, or `'w'` for writing.

On Windows, **popen()** defaults to text mode, i.e. any `\n` characters written to or read from the pipe will be translated to `\r\n`. If this is not desired, binary mode can be enforced by setting `mode` to `'rb'` and `'wb'`, respectively.

Return Values

Returns a file pointer identical to that returned by fopen(), except that it is unidirectional (may only be used for reading or writing) and must be closed with pclose(). This pointer may be used with fgets(), fgetss(), and fwrite(). When the mode is 'r', the returned file pointer equals to the STDOUT of the command, when the mode is 'w', the returned file pointer equals to the STDIN of the command.

If an error occurs, returns `false`.

Examples

Example #1 popen() example

```php
<?php
$handle = popen("/bin/ls", "r");
?>
```

If the command to be executed could not be found, a valid resource is returned. This may seem odd, but makes sense; it allows you to access any error message returned by the shell:

Example #2 popen() example

```php
<?php
error_reporting(E_ALL);

/* Add redirection so we can get stderr. */
$handle = popen('/path/to/executable 2>&1', 'r');
echo "'$handle'; " . gettype($handle) . "\n";
$read = fread($handle, 2096);
echo $read;
pclose($handle);
?>
```

Notes

> **Note** :
>
> If you're looking for bi-directional support (two-way), use proc_open().

See Also

- pclose() - Closes process file pointer
- fopen() - Opens file or URL
- proc_open() - Execute a command and open file pointers for input/output

readfile

readfile

(PHP 4, PHP 5, PHP 7, PHP 8)

readfile — Outputs a file

Description

function **readfile**(string `$filename`, bool `$use_include_path = false`, ?resource `$context = null`): int|false

Reads a file and writes it to the output buffer.

Parameters

`filename`

The filename being read.

`use_include_path`

You can use the optional second parameter and set it to `true`, if you want to search for the file in the include_path, too.

`context`

A context stream resource.

Return Values

Returns the number of bytes read from the file on success, or `false` on failure

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

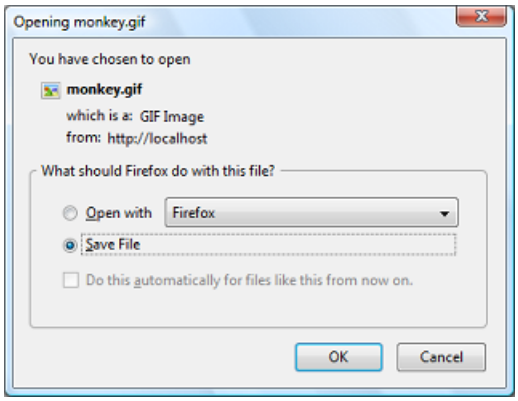
Examples

Example #1 Forcing a download using readfile()

```php
<?php
$file = 'monkey.gif';

if (file_exists($file)) {
    header('Content-Description: File Transfer');
    header('Content-Type: application/octet-stream');
    header('Content-Disposition: attachment; filename="'.basename($file).'"');
    header('Expires: 0');
    header('Cache-Control: must-revalidate');
    header('Pragma: public');
    header('Content-Length: ' . filesize($file));
    readfile($file);
    exit;
}
?>
```

The above example will output something similar to:



Notes

> **Note** :
>
> **readfile()** will not present any memory issues, even when sending large files, on its own. If you encounter an out of memory error ensure that output buffering is off with ob_get_level().

> **Tip**
>
> A URL can be used as a filename with this function if the fopen wrappers have been enabled. See fopen() for more details on how to specify the filename. See the Supported Protocols and Wrappers for links to information about what abilities the various wrappers have, notes on their usage, and information on any predefined variables they may provide.

See Also

- fpassthru() - Output all remaining data on a file pointer
- file() - Reads entire file into an array
- fopen() - Opens file or URL
- include - include
- require - require

- virtual() - Perform an Apache sub-request
- file_get_contents() - Reads entire file into a string
- Supported Protocols and Wrappers

readlink

readlink

(PHP 4, PHP 5, PHP 7, PHP 8)

readlink — Returns the target of a symbolic link

Description

function **readlink**(string `$path`): string|false

readlink() does the same as the readlink C function.

Parameters

`path`

The symbolic link path.

Return Values

Returns the contents of the symbolic link path or `false` on error.

> **Note** :
>
> The function fails if `path` is not a symlink, except on Windows, where the normalized path will be returned.

Examples

Example #1 readlink() example

```php
<?php

// output e.g. /boot/vmlinux-2.4.20-xfs
echo readlink('/vmlinuz');

?>
```

See Also

- is_link() - Tells whether the filename is a symbolic link
- symlink() - Creates a symbolic link
- linkinfo() - Gets information about a link

realpath

realpath

(PHP 4, PHP 5, PHP 7, PHP 8)

realpath — Returns canonicalized absolute pathname

Description

function **realpath**(string `$path`): string|false

realpath() expands all symbolic links and resolves references to `/./`, `/../` and extra `/` characters in the input `path` and returns the canonicalized absolute pathname.

Parameters

`path`

The path being checked.

> **Note** :
>
> Whilst a path must be supplied, the value can be an empty string. In this case, the value is interpreted as the current directory.

Return Values

Returns the canonicalized absolute pathname on success. The resulting path will have no symbolic link, `/./` or `/../` components. Trailing delimiters, such as `\` and `/`, are also removed.

realpath() returns `false` on failure, e.g. if the file does not exist.

> **Note** :
>
> The running script must have executable permissions on all directories in the hierarchy, otherwise **realpath()** will return `false`.

> **Note** :
>
> For case-insensitive filesystems **realpath()** may or may not normalize the character case.

> **Note** :
>
> The function **realpath()** will not work for a file which is inside a Phar as such path would be a virtual path, not a real one.

> **Note** :
>
> On Windows, junctions and symbolic links to directories are only expanded by one level.

> **Note** :
>
> Because PHP's integer type is signed and many platforms use 32bit integers, some filesystem functions may return unexpected results for files which are larger than 2GB.

Examples

Example #1 realpath() example

```php
<?php
chdir('/var/www/');
echo realpath('./../../etc/passwd') . PHP_EOL;

echo realpath('/tmp/') . PHP_EOL;
?>
```

The above example will output:

```
/etc/passwd
/tmp
```

Example #2 realpath() on Windows

On windows **realpath()** will change unix style paths to windows style.

```php
<?php
echo realpath('/windows/system32'), PHP_EOL;

echo realpath('C:\Program Files\\'), PHP_EOL;
?>
```

The above example will output:

```
C:\WINDOWS\System32
C:\Program Files
```

See Also

- basename() - Returns trailing name component of path
- dirname() - Returns a parent directory's path
- pathinfo() - Returns information about a file path

realpath_cache_get

realpath_cache_get

(PHP 5 >= 5.3.2, PHP 7, PHP 8)

realpath_cache_get — Get realpath cache entries

Description

function **realpath_cache_get**(): array

Get the contents of the realpath cache.

Parameters

This function has no parameters.

Return Values

Returns an array of realpath cache entries. The keys are original path entries, and the values are arrays of data items, containing the resolved path, expiration date, and other options kept in the cache.

Examples

Example #1 realpath_cache_get() example

```php
<?php
var_dump(realpath_cache_get());
?>
```

The above example will output something similar to:

```
array(2) {
  ["/test"]=>
  array(4) {
    ["key"]=>
    int(123456789)
    ["is_dir"]=>
    bool(true)
    ["realpath"]=>
    string(5) "/test"
    ["expires"]=>
    int(1260318939)
```

```
  }
  ["/test/test.php"]=>
  array(4) {
    ["key"]=>
    int(987654321)
    ["is_dir"]=>
    bool(false)
    ["realpath"]=>
    string(12) "/root/test.php"
    ["expires"]=>
    int(1260318939)
  }
}
```

See Also

- realpath_cache_size() - Get realpath cache size

realpath_cache_size

realpath_cache_size

(PHP 5 >= 5.3.2, PHP 7, PHP 8)

realpath_cache_size — Get realpath cache size

Description

function **realpath_cache_size**(): int

Get the amount of memory used by the realpath cache.

Parameters

This function has no parameters.

Return Values

Returns how much memory realpath cache is using.

Examples

Example #1 realpath_cache_size() example

```php
<?php
var_dump(realpath_cache_size());
?>
```

The above example will output something similar to:

```
int(412)
```

See Also

- realpath_cache_get() - Get realpath cache entries
- The realpath_cache_size configuration option

rename

rename

(PHP 4, PHP 5, PHP 7, PHP 8)

rename — Renames a file or directory

Description

function **rename**(string `$from`, string `$to`, ?resource `$context = null`): bool

Attempts to rename `from` to `to`, moving it between directories if necessary. If renaming a file and `to` exists, it will be overwritten. If renaming a directory and `to` exists, this function will emit a warning.

Parameters

`from`

The old name.

> **Note** :
>
> The wrapper used in `from` *must* match the wrapper used in `to`.

`to`

The new name.

> **Note** :
>
> On Windows, if `to` already exists, it must be writable. Otherwise **rename()** fails and issues `E_WARNING`.

`context`

A context stream resource.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Example with rename()

```
<?php
rename("/tmp/tmp_file.txt", "/home/user/login/docs/my_file.txt");
?>
```

See Also

- copy() - Copies file
- unlink() - Deletes a file
- move_uploaded_file() - Moves an uploaded file to a new location

rewind

rewind

(PHP 4, PHP 5, PHP 7, PHP 8)

rewind — Rewind the position of a file pointer

Description

function **rewind**(resource `$stream`): bool

Sets the file position indicator for `stream` to the beginning of the file stream.

> **Note** :

> If you have opened the file in append ("a" or "a+") mode, any data you write to the file will always be appended, regardless of the file pointer position.

Parameters

`stream`

The file pointer must be valid, and must point to a file successfully opened by fopen().

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 rewind() overwriting example

```
<?php
$handle = fopen('output.txt', 'r+');

fwrite($handle, 'Really long sentence.');
rewind($handle);
fwrite($handle, 'Foo');
rewind($handle);

echo fread($handle, filesize('output.txt'));

fclose($handle);
?>
```

The above example will output something similar to:

```
Foolly long sentence.
```

See Also

- fread() - Binary-safe file read
- fseek() - Seeks on a file pointer
- ftell() - Returns the current position of the file read/write pointer
- fwrite() - Binary-safe file write

rmdir

rmdir

(PHP 4, PHP 5, PHP 7, PHP 8)

rmdir — Removes directory

Description

function **rmdir**(string `$directory`, ?resource `$context = null`): bool

Attempts to remove the directory named by `directory`. The directory must be empty, and the relevant permissions must permit this. A `E_WARNING` level error will be generated on failure.

Parameters

`directory`

Path to the directory.

`context`

A context stream resource.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 rmdir() example

```
<?php
if (!is_dir('examples')) {
    mkdir('examples');
}

rmdir('examples');
?>
```

See Also

- is_dir() - Tells whether the filename is a directory
- mkdir() - Makes directory
- unlink() - Deletes a file

set_file_buffer

set_file_buffer

(PHP 4, PHP 5, PHP 7, PHP 8)

set_file_buffer — Alias of stream_set_write_buffer()

Description

This function is an alias of: stream_set_write_buffer().

stat

stat

(PHP 4, PHP 5, PHP 7, PHP 8)

stat — Gives information about a file

Description

function **stat**(string `$filename`): array|false

Gathers the statistics of the file named by `filename`. If `filename` is a symbolic link, statistics are from the file itself, not the symlink. Prior to PHP 7.4.0, on Windows NTS builds the `size`, `atime`, `mtime` and `ctime` statistics have been from the symlink, in this case.

lstat() is identical to **stat()** except it would instead be based off the symlinks status.

Parameters

`filename`

Path to the file.

Return Values

stat() and fstat() result format

Numeric	Associative	Description
0	dev	device number ***
1	ino	inode number ****
2	mode	inode protection mode *****
3	nlink	number of links

Numeric	Associative	Description
4	uid	userid of owner *
5	gid	groupid of owner *
6	rdev	device type, if inode device
7	size	size in bytes
8	atime	time of last access (Unix timestamp)
9	mtime	time of last modification (Unix timestamp)
10	ctime	time of last inode change (Unix timestamp)
11	blksize	blocksize of filesystem IO **
12	blocks	number of 512-byte blocks allocated **

* On Windows this will always be `0`.

** Only valid on systems supporting the st_blksize type - other systems (e.g. Windows) return `-1`.

*** On Windows, as of PHP 7.4.0, this is the serial number of the volume that contains the file, which is a 64-bit *unsigned* integer, so may overflow. Previously, it was the numeric representation of the drive letter (e.g. `2` for `C:`) for **stat()**, and `0` for lstat().

**** On Windows, as of PHP 7.4.0, this is the identifier associated with the file, which is a 64-bit *unsigned* integer, so may overflow. Previously, it was always `0`.

***** On Windows, the writable permission bit is set according to the read-only file attribute, and the same value is reported for all users, group and owner. The ACL is not taken into account, contrary to is_writable().

The value of `mode` contains information read by several functions. When written in octal, starting from the right, the first three digits are returned by chmod(). The next digit is ignored by PHP. The next two digits indicate the file type:

`mode` file types

`mode` in octal	Meaning
`0140000`	socket
`0120000`	link
`0100000`	regular file
`0060000`	block device
`0040000`	directory
`0020000`	character device
`0010000`	fifo

So for example a regular file could be `0100644` and a directory could be `0040755`.

In case of error, **stat()** returns `false`.

> **Note** :
>
> Because PHP's integer type is signed and many platforms use 32bit integers, some filesystem functions may return unexpected results for files which are larger than 2GB.

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

Changelog

Version	Description
7.4.0	On Windows, the device number is now the serial number of the volume that contains the file, and the inode number is the identifier associated with the file.
7.4.0	The `size`, `atime`, `mtime` and `ctime` statistics of symlinks are always those of the target. This was previously not the case for NTS builds on Windows.

Examples

Example #1 stat() example

```php
<?php
/* Get file stat */
$stat = stat('C:\php\php.exe');

/*
 * Print file access time, this is the same
 * as calling fileatime()
 */
echo 'Access time: ' . $stat['atime'];

/*
 * Print file modification time, this is the
 * same as calling filemtime()
 */
echo 'Modification time: ' . $stat['mtime'];

/* Print the device number */
echo 'Device number: ' . $stat['dev'];
?>
```

Example #2 Using stat() information together with touch()

```php
<?php
/* Get file stat */
$stat = stat('C:\php\php.exe');

/* Did we failed to get stat information? */
if (!$stat) {
    echo 'stat() call failed...';
} else {
    /*
     * We want the access time to be 1 week
     * after the current access time.
     */
    $atime = $stat['atime'] + 604800;

    /* Touch the file */
    if (!touch('some_file.txt', time(), $atime)) {
        echo 'Failed to touch file...';
    } else {
        echo 'touch() returned success...';
    }
}
?>
```

Notes

> **Note** :
>
> Note that time resolution may differ from one file system to another.

> **Note** :
>
> The results of this function are cached. See clearstatcache() for more details.

> **Tip**
>
> As of PHP 5.0.0, this function can also be used with *some* URL wrappers. Refer to Supported Protocols and Wrappers to determine which wrappers support **stat()** family of functionality.

See Also

- lstat() - Gives information about a file or symbolic link
- fstat() - Gets information about a file using an open file pointer
- filemtime() - Gets file modification time
- filegroup() - Gets file group
- SplFileInfo

symlink

symlink

(PHP 4, PHP 5, PHP 7, PHP 8)

symlink — Creates a symbolic link

Description

function **symlink**(string `$target`, string `$link`): bool

symlink() creates a symbolic link to the existing `target` with the specified name `link`.

Parameters

`target`

Target of the link.

`link`

The link name.

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

The function fails, and issues **`E_WARNING`**, if `link` already exists. On Windows, the function also fails, and issues **`E_WARNING`**, if `target` does not exist.

Examples

Example #1 Create a symbolic link

```
<?php
$target = 'uploads.php';
$link = 'uploads';
symlink($target, $link);

echo readlink($link);
?>
```

See Also

- is_link() - Tells whether the filename is a symbolic link
- link() - Create a hard link
- readlink() - Returns the target of a symbolic link
- linkinfo() - Gets information about a link
- unlink() - Deletes a file

tempnam

tempnam

(PHP 4, PHP 5, PHP 7, PHP 8)

tempnam — Create file with unique file name

Description

function **tempnam**(string `$directory`, string `$prefix`): string|false

Creates a file with a unique filename, with access permission set to 0600, in the specified directory. If the directory does not exist or is not writable, **tempnam()** may generate a file in the system's temporary directory, and return the full path to that file, including its name.

Parameters

`directory`

The directory where the temporary filename will be created.

`prefix`

The prefix of the generated temporary filename.

> **Note** :
>
> Only the first 63 characters of the prefix are used, the rest are ignored. Windows uses only the first three characters of the prefix.

Return Values

Returns the new temporary filename (with path), or `false` on failure.

Changelog

Version	Description
8.4.0	The name of files created by **tempnam()** are now 13 bytes longer. The total length is still platform-dependent.
7.1.0	**tempnam()** now emits a notice when falling back to the temp directory of the system.

Examples

Example #1 tempnam() example

```php
<?php
$tmpfname = tempnam("/tmp", "FOO");

$handle = fopen($tmpfname, "w");
fwrite($handle, "writing to tempfile");
fclose($handle);

// do something here

unlink($tmpfname);
?>
```

Notes

> **Note** :
>
> If PHP cannot create a file in the specified `directory` parameter, it falls back on the system default. On NTFS this also happens if the specified `directory` contains more than 65534 files.

See Also

- tmpfile() - Creates a temporary file
- sys_get_temp_dir() - Returns directory path used for temporary files
- unlink() - Deletes a file

tmpfile

tmpfile

(PHP 4, PHP 5, PHP 7, PHP 8)

tmpfile — Creates a temporary file

Description

function **tmpfile**(): resource|false

Creates a temporary file with a unique name in read-write-binary (w+b) mode and returns a file handle.

The file is automatically removed when closed (for example, by calling fclose(), or when there are no remaining references to the file handle returned by **tmpfile()**), or when the script ends.

> **Caution**
>
> If the script terminates unexpectedly, the temporary file may not be deleted.

Parameters

This function has no parameters.

Return Values

Returns a file handle, similar to the one returned by fopen(), for the new file or `false` on failure.

Examples

Example #1 tmpfile() example

```php
<?php
$temp = tmpfile();
fwrite($temp, "writing to tempfile");
fseek($temp, 0);
echo fread($temp, 1024);
fclose($temp); // this removes the file
?>
```

The above example will output:

```
writing to tempfile
```

See Also

- tempnam() - Create file with unique file name
- sys_get_temp_dir() - Returns directory path used for temporary files

touch

touch

(PHP 4, PHP 5, PHP 7, PHP 8)

touch — Sets access and modification time of file

Description

function **touch**(string `$filename`, ?int `$mtime = null`, ?int `$atime = null`): bool

Attempts to set the access and modification times of the file named in the `filename` parameter to the value given in `mtime`. Note that the access time is always modified, regardless of the number of parameters.

If the file does not exist, it will be created.

Parameters

`filename`

The name of the file being touched.

`mtime`

The touch time. If `mtime` is `null`, the current system time() is used.

`atime`

If not `null`, the access time of the given filename is set to the value of `atime`. Otherwise, it is set to the value passed to the `mtime` parameter. If both are `null`, the current system time is used.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.0.0	`mtime` and `atime` are now nullable.

Examples

Example #1 touch() example

```php
<?php
if (touch($filename)) {
    echo $filename . ' modification time has been changed to present time';
} else {
    echo 'Sorry, could not change modification time of ' . $filename;
}
?>
```

Example #2 touch() using the `mtime` parameter

```php
<?php
// This is the touch time, we'll set it to one hour in the past.
$time = time() - 3600;

// Touch the file
if (!touch('some_file.txt', $time)) {
    echo 'Whoops, something went wrong...';
} else {
    echo 'Touched file with success';
}
?>
```

Notes

> **Note** :
>
> Note that time resolution may differ from one file system to another.

umask

umask

(PHP 4, PHP 5, PHP 7, PHP 8)

umask — Changes the current umask

Description

function **umask**(?int `$mask` = `null`): int

umask() sets PHP's umask to `mask` & 0777 and returns the old umask. When PHP is being used as a server module, the umask is restored when each request is finished.

Parameters

`mask`

The new umask.

Return Values

If `mask` is `null`, **umask()** simply returns the current umask otherwise the old umask is returned.

Changelog

Version	Description
8.0.0	`mask` is nullable now.

Examples

Example #1 **umask()** example

```php
<?php
$old = umask(0);
chmod("/path/some_dir/some_file.txt", 0755);
umask($old);

// Checking
if ($old != umask()) {
    die('An error occurred while changing back the umask');
}
?>
```

Notes

> **Note** :
>
> Avoid using this function in multithreaded webservers. It is better to change the file permissions with chmod() after creating the file. Using **umask()** can lead to unexpected behavior of concurrently running scripts and the webserver itself because they all use the same umask.

unlink

unlink

(PHP 4, PHP 5, PHP 7, PHP 8)

unlink — Deletes a file

Description

function **unlink**(string `$filename`, ?resource `$context = null`): bool

Deletes `filename`. Similar to the Unix C unlink() function. An `E_WARNING` level error will be generated on failure.

Parameters

`filename`

Path to the file.

If the file is a symlink, the symlink will be deleted. On Windows, to delete a symlink to a directory, rmdir() has to be used instead.

`context`

A context stream resource.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
7.3.0	On Windows, it is now possible to **unlink()** files with handles in use, while formerly that would fail. However, it is still not possible to re-create the unlinked file, until all handles to it have been closed.

Examples

Example #1 Basic **unlink()** usage

```php
<?php
$fh = fopen('test.html', 'a');
fwrite($fh, '<h1>Hello world!</h1>');
fclose($fh);

unlink('test.html');
?>
```

See Also

- rmdir() - Removes directory

Directories

Predefined Constants

DIRECTORY_SEPARATOR (string)

PATH_SEPARATOR (string)

Semicolon on Windows, colon otherwise.

SCANDIR_SORT_ASCENDING (int)

SCANDIR_SORT_DESCENDING (int)

SCANDIR_SORT_NONE (int)

Directory

(PHP 4, PHP 5, PHP 7, PHP 8)

Introduction

Instances of **Directory** are created by calling the dir() function, not by the new operator.

Class synopsis

final class **Directory** {
/* Properties */
public readonly string *$path*;
public readonly resource *$handle*;
/* Methods */
public function close(): void
public function read(): string|false
public function rewind(): void
}

Properties

path

The directory that was opened.

handle

Can be used with other directory functions such as readdir(), rewinddir() and closedir().

Changelog

Version	Description
8.5.0	The class is now final.
8.1.0	The *path* and *handle* properties are now readonly.

Table of Contents

Directory::close

Directory::close

(PHP 4, PHP 5, PHP 7, PHP 8)

Directory::close — Close directory handle

Description

public function **Directory::close**(): void

Changelog

Version	Description
8.0.0	No parameter is accepted. Previously, a directory handle could be passed as argument.

Directory::read

Directory::read

(PHP 4, PHP 5, PHP 7, PHP 8)

Directory::read — Read entry from directory handle

Description

public function **Directory::read**(): string|false

Changelog

Version	Description
8.0.0	No parameter is accepted. Previously, a directory handle could be passed as argument.

Directory::rewind

Directory::rewind

(PHP 4, PHP 5, PHP 7, PHP 8)

Directory::rewind — Rewind directory handle

Description

public function **Directory::rewind**(): void

Changelog

Version	Description
8.0.0	No parameter is accepted. Previously, a directory handle could be passed as argument.

Directory Functions

See Also

For related functions such as dirname(), is_dir(), mkdir(), and rmdir(), see the Filesystem section.

Table of Contents

chdir

chdir

(PHP 4, PHP 5, PHP 7, PHP 8)

chdir — Change directory

Description

function **chdir**(string `$directory`): bool

Changes PHP's current directory to `directory`.

Parameters

`directory`

The new current directory

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

Throws an error of level `E_WARNING` on failure.

Examples

Example #1 chdir() example

```php
<?php

// current directory
echo getcwd() . "\n";

chdir('public_html');

// current directory
echo getcwd() . "\n";

?>
```

The above example will output something similar to:

```
/home/vincent
/home/vincent/public_html
```

Notes

> **Caution**
>
> If the PHP interpreter has been built with ZTS (Zend Thread Safety) enabled, any changes to the current directory made through **chdir()** will be invisible to the operating system. All built-in PHP functions will still respect the change in current directory; but external library functions called using FFI will not. You can tell whether your copy of PHP was built with ZTS enabled using **php -i** or the built-in constant `PHP_ZTS`.

See Also

- getcwd() - Gets the current working directory

chroot

chroot

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

chroot — Change the root directory

Description

function **chroot**(string `$directory`): bool

Changes the root directory of the current process to `directory`, and changes the current working directory to "/".

This function is only available to GNU and BSD systems, and only when using the CLI, CGI or Embed SAPI. Also, this function requires root privileges.

Calling this function does not change the values of the `__DIR__` and `__FILE__` magic constants.

Parameters

`directory`

The path to change the root directory to.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 chroot() example

```php
<?php
chroot("/path/to/your/chroot/");
echo getcwd();
?>
```

The above example will output:

```
/
```

Notes

> **Note** :
>
> This function is not implemented on Windows platforms.

> **Note** :
>
> This function is not available in PHP interpreters built with ZTS (Zend Thread Safety) enabled. To check whether your copy of PHP was built with ZTS enabled, use **php -i** or test the built-in constant `PHP_ZTS`.

closedir

closedir

(PHP 4, PHP 5, PHP 7, PHP 8)

closedir — Close directory handle

Description

function **closedir**(?resource `$dir_handle = null`): void

Closes the directory stream indicated by `dir_handle`.

Parameters

`dir_handle`

A directory handle resource previously opened with opendir(). If `dir_handle` is `null` the last handle opened using opendir() will be used.

Return Values

No value is returned.

Changelog

Version	Description
8.5.0	Using `null` for `dir_handle` is now deprecated. Instead, the last opened directory handle should be explicitly provided.
8.0.0	`dir_handle` is now nullable.

Examples

For a complete example refer to the opendir() documentation.

See Also

- opendir() - Open directory handle
- readdir() - Read entry from directory handle
- rewinddir() - Rewind directory handle
- dir() - Return an instance of the Directory class
- is_dir() - Tells whether the filename is a directory
- glob() - Find pathnames matching a pattern
- scandir() - List files and directories inside the specified path

dir

dir

(PHP 4, PHP 5, PHP 7, PHP 8)

dir — Return an instance of the Directory class

Description

function **dir**(string `$directory`, ?resource `$context = null`): Directory|false

A pseudo-object-oriented mechanism for reading a directory. The given `directory` is opened.

Parameters

`directory`

Directory to open

`context`

A context stream resource.

Return Values

Returns an instance of Directory, or `false` in case of error.

Changelog

Version	Description
8.0.0	`context` is now nullable.

Examples

Example #1 dir() example

Please note the fashion in which Directory::read()'s return value is checked in the example below. We are explicitly testing whether the return value is identical to (equal to and of the same type as - see Comparison Operators for more information) `false` since otherwise, any directory entry whose name evaluates to `false` will stop the loop.

```
<?php
$d = dir("/etc/php5");
echo "Handle: " . $d->handle . "\n";
echo "Path: " . $d->path . "\n";
while (false !== ($entry = $d->read())) {
   echo $entry."\n";
}
$d->close();
?>
```

The above example will output something similar to:

```
```

```
Handle: Resource id #2
Path: /etc/php5
.
..
apache
cgi
cli
```

Notes

> **Note** :
>
> The order in which directory entries are returned by the read method is system-dependent.

getcwd

getcwd

(PHP 4, PHP 5, PHP 7, PHP 8)

getcwd — Gets the current working directory

Description

function **getcwd**(): string|false

Gets the current working directory.

Parameters

This function has no parameters.

Return Values

Returns the current working directory on success, or `false` on failure.

On some Unix variants, **getcwd()** will return `false` if any one of the parent directories does not have the readable or search mode set, even if the current directory does. See chmod() for more information on modes and permissions.

Examples

Example #1 getcwd() example

```php
<?php

// current directory
echo getcwd() . "\n";

chdir('cvs');

// current directory
echo getcwd() . "\n";

?>
```

The above example will output something similar to:

```
/home/didou
/home/didou/cvs
```

Notes

> **Caution**
>
> If the PHP interpreter has been built with ZTS (Zend Thread Safety) enabled, the current working directory returned by **getcwd()** may be different from that returned by operating system interfaces. External libraries (invoked through FFI) which depend on the current working directory will be affected.

See Also

- chdir() - Change directory
- chmod() - Changes file mode

opendir

opendir

(PHP 4, PHP 5, PHP 7, PHP 8)

opendir — Open directory handle

Description

function **opendir**(string `$directory`, ?resource `$context = null`): resource|false

Opens up a directory handle to be used in subsequent closedir(), readdir(), and rewinddir() calls.

Parameters

`directory`

The directory path to open.

`context`

For a description of the `context` parameter, refer to the streams section of the manual.

Return Values

Returns a directory handle on success, or `false` on failure

Errors/Exceptions

Upon failure, an `E_WARNING` is emitted.

This may happen if `directory` is not a valid directory, the directory can not be opened due to permission restrictions, or due to filesystem errors.

Changelog

Version	Description
8.0.0	`context` is now nullable.

Examples

Example #1 List all entries in a directory, skipping the special . and .. directories

Because file and directory names can be strings that PHP considers "falsy" (e.g. a directory named `"0"`) and readdir() returns `false` when it has read all entries in a directory one needs to use the `===` comparison operator to properly distinguish between a directory entry whose name is "falsy" and having read all entries of the directory.

```
<?php

if ($handle = opendir('/path/to/files')) {
    echo "Entries:\n";

    /* Correctly handling directory entries that may be considered falsy */
    while (false !== ($entry = readdir($handle))) {
        if ($entry === '.' || $entry === '..') {
```

```
            continue;
        }
        echo "$entry\n";
    }

    closedir($handle);
}
?>
```

The above example will output something similar to:

```
Entries:
base
en
fr
output.md
test.php
```

See Also

- readdir() - Read entry from directory handle
- rewinddir() - Rewind directory handle
- closedir() - Close directory handle
- dir() - Return an instance of the Directory class
- is_dir() - Tells whether the filename is a directory
- glob() - Find pathnames matching a pattern
- scandir() - List files and directories inside the specified path

readdir

readdir

(PHP 4, PHP 5, PHP 7, PHP 8)

readdir — Read entry from directory handle

Description

function **readdir**(?resource `$dir_handle = null`): string|false

Returns the name of the next entry in the directory. The entries are returned in the order in which they are stored by the filesystem.

Parameters

`dir_handle`

A directory handle resource previously opened with opendir(). If `dir_handle` is `null` the last handle opened using opendir() will be used.

Return Values

Returns the entry name on success, or `false` on failure.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Changelog

Version	Description
8.5.0	Using `null` for `dir_handle` is now deprecated. Instead, the last opened directory handle should be explicitly provided.

Version	Description
8.0.0	`dir_handle` is now nullable.

Examples

For a complete example refer to the opendir() documentation.

See Also

- opendir() - Open directory handle
- rewinddir() - Rewind directory handle
- closedir() - Close directory handle
- dir() - Return an instance of the Directory class
- is_dir() - Tells whether the filename is a directory
- glob() - Find pathnames matching a pattern
- scandir() - List files and directories inside the specified path

rewinddir

rewinddir

(PHP 4, PHP 5, PHP 7, PHP 8)

rewinddir — Rewind directory handle

Description

function **rewinddir**(?resource `$dir_handle = null`): void

Resets the directory stream indicated by `dir_handle` to the beginning of the directory.

Parameters

`dir_handle`

A directory handle resource previously opened with opendir(). If `dir_handle` is `null` the last handle opened using opendir() will be used.

Return Values

No value is returned.

Changelog

Version	Description
8.5.0	Using `null` for `dir_handle` is now deprecated. Instead, the last opened directory handle should be explicitly provided.
8.0.0	`dir_handle` is now nullable.

Examples

For a complete example refer to the opendir() documentation.

See Also

- opendir() - Open directory handle
- readdir() - Read entry from directory handle
- closedir() - Close directory handle
- dir() - Return an instance of the Directory class
- is_dir() - Tells whether the filename is a directory
- glob() - Find pathnames matching a pattern
- scandir() - List files and directories inside the specified path

scandir

scandir

(PHP 5, PHP 7, PHP 8)

scandir — List files and directories inside the specified path

Description

function **scandir**(string `$directory`, int `$sorting_order` = **`SCANDIR_SORT_ASCENDING`**, ?resource `$context` = **`null`**): array|false

Returns an array of files and directories from the `directory`.

Parameters

`directory`

The directory that will be scanned.

`sorting_order`

By default, the sorted order is alphabetical in ascending order. If the optional `sorting_order` is set to **`SCANDIR_SORT_DESCENDING`**, then the sort order is alphabetical in descending order. If it is set to **`SCANDIR_SORT_NONE`** then the result is unsorted.

`context`

For a description of the `context` parameter, refer to the streams section of the manual.

Return Values

Returns an array of filenames on success, or **`false`** on failure. If `directory` is not a directory, then boolean **`false`** is returned, and an error of level **`E_WARNING`** is generated.

Changelog

Version	Description
8.0.0	`context` is now nullable.

Examples

Example #1 A simple **scandir()** example

```php
<?php
$dir    = '/tmp';
$files1 = scandir($dir);
$files2 = scandir($dir, SCANDIR_SORT_DESCENDING);

print_r($files1);
print_r($files2);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => .
    [1] => ..
    [2] => bar.php
    [3] => foo.txt
    [4] => somedir
)
Array
(
    [0] => somedir
    [1] => foo.txt
    [2] => bar.php
    [3] => ..
    [4] => .
)
```

Notes

> **Tip**
>
> A URL can be used as a filename with this function if the fopen wrappers have been enabled. See fopen() for more details on how to specify the filename. See the Supported Protocols and Wrappers for links to information about what abilities the various wrappers have, notes on their usage, and information on any predefined variables they may provide.

See Also

- opendir() - Open directory handle
- readdir() - Read entry from directory handle
- glob() - Find pathnames matching a pattern
- is_dir() - Tells whether the filename is a directory
- sort() - Sort an array in ascending order

Variable handling

Introduction

For information on how variables behave, see the Variables entry in the Language Reference section of the manual.

Installing/Configuring

Table of Contents

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Variables Configuration Options

Name	Default	Changeable	Changelog
unserialize_callback_func	`null`	`INI_ALL`	
unserialize_max_depth	"4096"	`INI_ALL`	Available as of PHP 7.4.0.

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`unserialize_callback_func` string

> The callback specified is called when unserialize() attempts to use an undefined class. A warning appears if the specified callback function is not defined, or if the callback function fails to define the missing class.
>
> See also unserialize() and Autoloading Classes.

`unserialize_max_depth` int

> The maximum depth of structures permitted during unserialization when using unserialize(), and is intended to prevent stack overflows. This can be disabled by setting `unserialize_max_depth=0`.
>
> See also unserialize() and Autoloading Classes.

Variable handling Functions

Table of Contents

boolval

boolval

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

boolval — Get the boolean value of a variable

Description

function **boolval**(mixed `$value`): bool

Returns the bool value of `value`.

Parameters

`value`

> The scalar value being converted to a bool.

Return Values

The bool value of `value`.

Examples

Example #1 boolval() examples

```
<?php
echo '0:        '.(boolval(0) ? 'true' : 'false')."\n";
echo '42:       '.(boolval(42) ? 'true' : 'false')."\n";
echo '0.0:      '.(boolval(0.0) ? 'true' : 'false')."\n";
echo '4.2:      '.(boolval(4.2) ? 'true' : 'false')."\n";
echo '"":       '.(boolval("") ? 'true' : 'false')."\n";
echo '"string": '.(boolval("string") ? 'true' : 'false')."\n";
echo '"0":      '.(boolval("0") ? 'true' : 'false')."\n";
echo '"1":      '.(boolval("1") ? 'true' : 'false')."\n";
echo '[1, 2]:   '.(boolval([1, 2]) ? 'true' : 'false')."\n";
echo '[]:       '.(boolval([]) ? 'true' : 'false')."\n";
echo 'stdClass: '.(boolval(new stdClass) ? 'true' : 'false')."\n";
?>
```

The above example will output:

```
0:        false
42:       true
0.0:      false
4.2:      true
"":       false
"string": true
"0":      false
"1":      true
[1, 2]:   true
[]:       false
stdClass: true
```

See Also

- floatval() - Get float value of a variable
- intval() - Get the integer value of a variable
- strval() - Get string value of a variable
- settype() - Set the type of a variable
- is_bool() - Finds out whether a variable is a boolean
- Type juggling

debug_zval_dump

debug_zval_dump

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

debug_zval_dump — Dumps a string representation of an internal zval structure to output

Description

function **debug_zval_dump**(mixed `$value`, mixed `...$values`): void

Dumps a string representation of an internal zval (Zend value) structure to output. This is mostly useful for understanding or debugging implementation details of the Zend Engine or PHP extensions.

Parameters

`value`

The variable or value to dump.

`values`

Further variables or values to dump.

Return Values

No value is returned.

Changelog

Version	Description
8.4.0	**debug_zval_dump()** now indicates whether an array is packed.

Examples

Example #1 **debug_zval_dump()** example

```php
<?php
$var1 = 'Hello';
$var1 .= ' World';
$var2 = $var1;

debug_zval_dump($var1);
?>
```

The above example will output:

```
string(11) "Hello World" refcount(3)
```

> **Note** : **Understanding the `refcount`**
>
> The `refcount` value shown by this function may be surprising without a detailed understanding of the engine's implementation.
>
> The Zend Engine uses reference counting for two different purposes:
>
> - Optimizing memory usage using a technique called "copy on write", where multiple variables holding the same value point to the same copy in memory. When any of the variables is modified, it is pointed to a new copy in memory, and the reference count on the original is decreased by 1.
> - Tracking variables which have been assigned or passed by reference (see References Explained). This refcount is stored on a separate reference zval, pointing to the zval for the current value. This additional zval is not currently shown by **debug_zval_dump()**.
>
> Because **debug_zval_dump()** takes its input as normal parameters, passed by value, the copy on write technique will be used to pass them: rather than copying the data, the refcount will be increased by one for the lifetime of the function call. If the function modified the parameter after receiving it, then a copy would be made; since it does not, it will show a refcount one higher than in the calling scope.
>
> The parameter passing also prevents **debug_zval_dump()** showing variables which have been assigned by reference. To illustrate, consider a slightly modified version of the above example:
>
> ```php
> <?php
> $var1 = 'Hello';
> $var1 .= ' World';
> // Point three variables as references to the same value
> $var2 =& $var1;
> $var3 =& $var1;
>
> debug_zval_dump($var1);
> ?>
> ```
>
> The above example will output:

```
string(11) "Hello World" refcount(2)
```

Although *$var1*, *$var2*, and *$var3* are linked as references, only the *value* is passed to **debug_zval_dump()**. That value is used once by the set of references, and once inside the **debug_zval_dump()**, so shows a refcount of 2.

Further complications arise because of optimisations made in the engine for different data types. Some types such as integers do not use "copy on write", so do not show a refcount at all. In other cases, the refcount shows extra copies used internally, such as when a literal string or array is stored as part of a code instruction.

See Also

- var_dump() - Dumps information about a variable
- debug_backtrace() - Generates a backtrace
- References Explained
- » References Explained (by Derick Rethans)

doubleval

doubleval

(PHP 4, PHP 5, PHP 7, PHP 8)

doubleval — Alias of floatval()

Description

This function is an alias of: floatval().

empty

empty

(PHP 4, PHP 5, PHP 7, PHP 8)

empty — Determine whether a variable is empty

Description

function **empty**(mixed `$var`): bool

Determine whether a variable is considered to be empty. A variable is considered empty if it does not exist or if its value equals `false`. **empty()** does not generate a warning if the variable does not exist.

Parameters

`var`

Variable to be checked

No warning is generated if the variable does not exist. That means **empty()** is essentially the concise equivalent to **!isset($var) || $var == false**. This also applies to nested structures, such as a multidimensional array or chained properties.

Return Values

Returns `true` if `var` does not exist or has a value that is empty or equal to zero, aka falsey, see conversion to boolean. Otherwise returns `false`.

Examples

Example #1 A simple empty() / isset() comparison.

```php
<?php
$var = 0;

// Evaluates to true because $var is empty
```

```php
if (empty($var)) {
    echo '$var is either 0, empty, or not set at all';
}

// Evaluates as true because $var is set
if (isset($var)) {
    echo '$var is set even though it is empty';
}
?>
```

Example #2 empty() on String Offsets

```php
<?php
$expected_array_got_string = 'somestring';
var_dump(empty($expected_array_got_string['some_key']));
var_dump(empty($expected_array_got_string[0]));
var_dump(empty($expected_array_got_string['0']));
var_dump(empty($expected_array_got_string['0.5']));
var_dump(empty($expected_array_got_string['0 Mostel']));
?>
```

The above example will output:

```
bool(true)
bool(false)
bool(false)
bool(true)
bool(true)
```

Example #3 empty() on multidimensional arrays

```php
<?php
$multidimensional = [
    'some' => [
        'deep' => [
            'nested' => 'value'
        ]
    ]
];

if (!empty($multidimensional['some']['some']['nested'])) {
    $someVariable = $multidimensional['some']['deep']['nested'];
}

var_dump(empty($multidimensional['some-undefined-key']));
var_dump(empty($multidimensional['some']['deep']['unknown']));
var_dump(empty($multidimensional['some']['deep']['nested']));
?>
```

The above example will output:

```
bool(true)
bool(true)
bool(false)
```

Notes

> **Note** :
>
> Because this is a language construct and not a function, it cannot be called using variable functions, or named arguments.

> **Note** :
>
> When using **empty()** on inaccessible object properties, the __isset() overloading method will be called, if declared.

See Also

- isset() - Determine if a variable is declared and is different than null
- __isset()
- unset() - unset a given variable
- array_key_exists() - Checks if the given key or index exists in the array
- count() - Counts all elements in an array or in a Countable object
- strlen() - Get string length
- The type comparison tables

floatval

floatval

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

floatval — Get float value of a variable

Description

function **floatval**(mixed `$value`): float

Gets the float value of `value`.

Parameters

`value`

May be any scalar type. **floatval()** should not be used on objects, as doing so will emit an `E_WARNING` level error and return 1.

Return Values

The float value of the given variable. Empty arrays return 0, non-empty arrays return 1.

Strings will most likely return 0 although this depends on the leftmost characters of the string. The common rules of float casting apply.

Changelog

Version	Description
8.0.0	The error level when converting from object was changed from `E_NOTICE` to `E_WARNING`.

Examples

Example #1 **floatval()** Example

```
<?php
$var = '122.34343The';
$float_value_of_var = floatval($var);
echo $float_value_of_var; // 122.34343
?>
```

Example #2 **floatval()** non-numeric leftmost characters Example

```
<?php
$var = 'The122.34343';
$float_value_of_var = floatval($var);
```

```
echo $float_value_of_var; // 0
?>
```

See Also

- boolval() - Get the boolean value of a variable
- intval() - Get the integer value of a variable
- strval() - Get string value of a variable
- settype() - Set the type of a variable
- Type juggling

get_debug_type

get_debug_type

(PHP 8)

get_debug_type — Gets the type name of a variable in a way that is suitable for debugging

Description

function **get_debug_type**(mixed `$value`): string

Returns the resolved name of the PHP variable `value`. This function will resolve objects to their class name, resources to their resource type name, and scalar values to their common name as would be used in type declarations.

This function differs from gettype() in that it returns type names that are more consistent with actual usage, rather than those present for historical reasons.

Parameters

`value`

The variable being type checked.

Return Values

Possible values for the returned string are:

Type + State	Return Value	Notes
null	`"null"`	-
Booleans (`true` or `false`)	`"bool"`	-
Integers	`"int"`	-
Floats	`"float"`	-
Strings	`"string"`	-
Arrays	`"array"`	-
Resources	`"resource (resourcename)"`	-
Resources (Closed)	`"resource (closed)"`	Example: A file stream after being closed with fclose().
Objects from Named Classes	The full name of the class including its namespace e.g. `Foo\Bar`	-
Objects from Anonymous Classes	`"class@anonymous"` or parent class name/interface name if the class extends another class or implements an interface e.g. `"Foo\Bar@anonymous"`	Anonymous classes are those created through the `$x = new class { ... }` syntax

Examples

Example #1 get_debug_type() example

```
<?php

namespace Foo;
```

```php
echo get_debug_type(null), PHP_EOL;
echo get_debug_type(true), PHP_EOL;
echo get_debug_type(1), PHP_EOL;
echo get_debug_type(0.1), PHP_EOL;
echo get_debug_type("foo"), PHP_EOL;
echo get_debug_type([]), PHP_EOL;

$fp = fopen('/examples/book.xml', 'rb');
echo get_debug_type($fp), PHP_EOL;

fclose($fp);
echo get_debug_type($fp), PHP_EOL;

echo get_debug_type(new \stdClass), PHP_EOL;
echo get_debug_type(new class {}), PHP_EOL;

interface A {}
interface B {}
class C {}

echo get_debug_type(new class implements A {}), PHP_EOL;
echo get_debug_type(new class implements A,B {}), PHP_EOL;
echo get_debug_type(new class extends C {}), PHP_EOL;
echo get_debug_type(new class extends C implements A {}), PHP_EOL;

?>
```

The above example will output something similar to:

```
null
bool
int
float
string
array
resource (stream)
resource (closed)
stdClass
class@anonymous
Foo\A@anonymous
Foo\A@anonymous
Foo\C@anonymous
Foo\C@anonymous
```

See Also

- gettype() - Get the type of a variable
- get_class() - Returns the name of the class of an object

get_defined_vars

get_defined_vars

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

get_defined_vars — Returns an array of all defined variables

Description

function **get_defined_vars**(): array

This function returns a multidimensional array containing a list of all defined variables, be them environment, server or user-defined variables, within the scope that **get_defined_vars()** is called.

Parameters

This function has no parameters.

Return Values

A multidimensional array with all the variables.

Examples

Example #1 get_defined_vars() Example

```php
<?php
$b = array(1, 1, 2, 3, 5, 8);

$arr = get_defined_vars();

// print $b
print_r($arr["b"]);

/* print path to the PHP interpreter (if used as a CGI)
 * e.g. /usr/local/bin/php */
echo $arr["_"];

// print the command-line parameters if any
print_r($arr["argv"]);

// print all the server vars
print_r($arr["_SERVER"]);

// print all the available keys for the arrays of variables
print_r(array_keys(get_defined_vars()));
?>
```

See Also

- isset() - Determine if a variable is declared and is different than null
- get_defined_functions() - Returns an array of all defined functions
- get_defined_constants() - Returns an associative array with the names of all the constants and their values

get_resource_id

get_resource_id

(PHP 8)

get_resource_id — Returns an integer identifier for the given resource

Description

function **get_resource_id**(resource `$resource`): int

This function provides a type-safe way for generating the integer identifier for a resource.

Parameters

`resource`

The evaluated resource handle.

Return Values

The int identifier for the given `resource`.

This function is essentially an int cast of `resource` to make it easier to retrieve the resource ID.

Examples

Example #1 get_resource_id() produces the same result as an int cast

```php
<?php
$handle = fopen("php://stdout", "w");

echo (int) $handle . "\n";

echo get_resource_id($handle);

?>
```

The above example will output something similar to:

```
698
698
```

See Also

- get_resource_type() - Returns the resource type

get_resource_type

get_resource_type

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

get_resource_type — Returns the resource type

Description

function **get_resource_type**(resource `$resource`): string

This function gets the type of the given resource.

Parameters

`resource`

The evaluated resource handle.

Return Values

If the given `resource` is a resource, this function will return a string representing its type. If the type is not identified by this function, the return value will be the string `Unknown`.

This function will return `null` and generate an error if `resource` is not a resource.

Examples

Example #1 get_resource_type() example

```php
<?php
$fp = fopen("foo", "w");
echo get_resource_type($fp) . "\n";
?>
```

Output of the above example in PHP 7:

```
stream
```

See Also

- get_resource_id() - Returns an integer identifier for the given resource

gettype

gettype

(PHP 4, PHP 5, PHP 7, PHP 8)

gettype — Get the type of a variable

Description

function **gettype**(mixed `$value`): string

Returns the type of the PHP variable `value`. For type checking, use `is_*` functions.

Parameters

`value`

The variable being type checked.

Return Values

Possible values for the returned string are:

- `"boolean"`
- `"integer"`
- `"double"` (for historical reasons `"double"` is returned in case of a float, and not simply `"float"`)
- `"string"`
- `"array"`
- `"object"`
- `"resource"`
- `"resource (closed)"` as of PHP 7.2.0
- `"NULL"`
- `"unknown type"`

Changelog

Version	Description
7.2.0	Closed resources are now reported as `'resource (closed)'`. Previously the returned value for closed resources were `'unknown type'`.

Examples

Example #1 gettype() example

```php
<?php

$data = array(1, 1., NULL, new stdClass, 'foo');

foreach ($data as $value) {
    echo gettype($value), "\n";
}

?>
```

The above example will output something similar to:

```
integer
double
NULL
object
string
```

See Also

- get_debug_type() - Gets the type name of a variable in a way that is suitable for debugging
- settype() - Set the type of a variable
- get_class() - Returns the name of the class of an object
- is_array() - Finds whether a variable is an array
- is_bool() - Finds out whether a variable is a boolean
- is_callable() - Verify that a value can be called as a function from the current scope
- is_float() - Finds whether the type of a variable is float
- is_int() - Find whether the type of a variable is integer
- is_null() - Finds whether a variable is null
- is_numeric() - Finds whether a variable is a number or a numeric string
- is_object() - Finds whether a variable is an object
- is_resource() - Finds whether a variable is a resource
- is_scalar() - Finds whether a variable is a scalar
- is_string() - Find whether the type of a variable is string
- function_exists() - Return true if the given function has been defined
- method_exists() - Checks if the class method exists

intval

intval

(PHP 4, PHP 5, PHP 7, PHP 8)

intval — Get the integer value of a variable

Description

function **intval**(mixed `$value`, int `$base` = 10): int

Returns the int value of `value`, using the specified `base` for the conversion (the default is base 10). **intval()** should not be used on objects, as doing so will emit an `E_WARNING` level error and return 1.

Parameters

`value`

The scalar value being converted to an integer

`base`

The base for the conversion

> **Note** :
>
> If `base` is 0, the base used is determined by the format of `value`:
>
> - if string includes a "0x" (or "0X") prefix, the base is taken as 16 (hex); otherwise,
> - if string starts with a "0b" (or "0B"), the base is taken as 2 (binary); otherwise,
> - if string starts with "0", the base is taken as 8 (octal); otherwise,
> - the base is taken as 10 (decimal).

Return Values

The integer value of `value` on success, or 0 on failure. Empty arrays return 0, non-empty arrays return 1.

The maximum value depends on the system. 32 bit systems have a maximum signed integer range of -2147483648 to 2147483647. So for example on such a system, `intval('1000000000000')` will return 2147483647. The maximum signed integer value for 64 bit systems is 9223372036854775807.

Strings will most likely return 0 although this depends on the leftmost characters of the string. The common rules of integer casting apply.

> **Note** :
>
> Numeric strings using scientific notation (containing the letter `e` or `E`) are first parsed as numbers before being converted to an integer.
>
> Because the numeric part of the string is parsed as a whole, the result is not simply the leading integer part. Large exponents can additionally overflow to `PHP_INT_MAX`:
>
> ```
> <?php
> echo intval('42.42e42'); // 9223372036854775807 on 64-bit systems
> ?>
> ```
>
> See Numeric strings for details on how such strings are interpreted.

Changelog

Version	Description
8.0.0	The error level when converting from object was changed from `E_NOTICE` to `E_WARNING`.

Examples

Example #1 intval() examples

The following examples are based on a 64 bit system.

```
<?php
echo intval(42), PHP_EOL;                      // 42
echo intval(4.7), PHP_EOL;                     // 4
echo intval('42'), PHP_EOL;                    // 42
echo intval('+42'), PHP_EOL;                   // 42
echo intval('-42'), PHP_EOL;                   // -42
echo intval(042), PHP_EOL;                     // 34
echo intval('042'), PHP_EOL;                   // 42
echo intval(1e10), PHP_EOL;                    // 10000000000
echo intval('1e10'), PHP_EOL;                  // 10000000000
echo intval(0x1A), PHP_EOL;                    // 26
echo intval('0x1A'), PHP_EOL;                  // 0
echo intval('0x1A', 0), PHP_EOL;               // 26
echo intval('1A', 16), PHP_EOL;                // 26
echo intval(42000000), PHP_EOL;                // 42000000
echo intval(420000000000000000000), PHP_EOL;   // -4275113695319687168
echo intval('420000000000000000000'), PHP_EOL; // 9223372036854775807
echo intval(42, 8), PHP_EOL;                   // 42
echo intval('42', 8), PHP_EOL;                 // 34
echo intval(42, 7), PHP_EOL;                   // 42
echo intval('42', 7), PHP_EOL;                 // 30
echo intval('1000', 11), PHP_EOL;              // 1331
echo intval('1ya', 36), PHP_EOL;               // 2530
echo intval(array()), PHP_EOL;                 // 0
echo intval(array('foo', 'bar')), PHP_EOL;     // 1
echo intval(false), PHP_EOL;                   // 0
echo intval(true), PHP_EOL;                    // 1
?>
```

Notes

> **Note** :
>
> The `base` parameter has no effect unless the `value` parameter is a string.

See Also

- boolval() - Get the boolean value of a variable
- floatval() - Get float value of a variable
- strval() - Get string value of a variable

- settype() - Set the type of a variable
- is_numeric() - Finds whether a variable is a number or a numeric string
- Type juggling
- BCMath Arbitrary Precision Mathematics Functions

is_array

is_array

(PHP 4, PHP 5, PHP 7, PHP 8)

is_array — Finds whether a variable is an array

Description

function **is_array**(mixed `$value`): bool

Finds whether the given variable is an array.

Parameters

`value`

The variable being evaluated.

Return Values

Returns **`true`** if `value` is an array, **`false`** otherwise.

Examples

Example #1 Check that variable is an array

```php
<?php
$yes = array('this', 'is', 'an array');

echo is_array($yes) ? 'Array' : 'not an Array';
echo "\n";

$no = 'this is a string';

echo is_array($no) ? 'Array' : 'not an Array';
?>
```

The above example will output:

```
Array
not an Array
```

See Also

- array_is_list() - Checks whether a given array is a list
- is_float() - Finds whether the type of a variable is float
- is_int() - Find whether the type of a variable is integer
- is_string() - Find whether the type of a variable is string
- is_object() - Finds whether a variable is an object

is_bool

is_bool

(PHP 4, PHP 5, PHP 7, PHP 8)

is_bool — Finds out whether a variable is a boolean

Description

function **is_bool**(mixed `$value`): bool

Finds whether the given variable is a boolean.

Parameters

`value`

The variable being evaluated.

Return Values

Returns `true` if `value` is a bool, `false` otherwise.

Examples

Example #1 is_bool() examples

```php
<?php
$a = false;
$b = 0;

// Since $a is a boolean, it will return true
if (is_bool($a) === true) {
    echo "Yes, this is a boolean\n";
}

// Since $b is not a boolean, it will return false
if (is_bool($b) === false) {
    echo "No, this is not a boolean\n";
}
?>
```

See Also

- is_float() - Finds whether the type of a variable is float
- is_int() - Find whether the type of a variable is integer
- is_string() - Find whether the type of a variable is string
- is_object() - Finds whether a variable is an object
- is_array() - Finds whether a variable is an array

is_callable

is_callable

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

is_callable — Verify that a value can be called as a function from the current scope

Description

function **is_callable**(mixed `$value`, bool `$syntax_only = false`, string `&$callable_name = null`): bool

Verifies that `value` is a callable, or that it can be called using the call_user_func() function.

Parameters

`value`

The value to be checked.

`syntax_only`

If set to `true` the function only verifies that `value` might be a function or method. It will reject any values that are not invokable objects, Closure, strings, or arrays that do not have a valid structure to be used as a callback. A valid callable array has 2 entries, the first of which is an object or a string, and the second a string.

`callable_name`

Receives the "callable name", e.g. `"SomeClass::someMethod"`. Note, however, that despite the implication that `SomeClass::someMethod()` is a callable static method, this is not the case.

Return Values

Returns **`true`** if `value` is callable, **`false`** otherwise.

Examples

Example #1 Checking whether a string can be called as a function

```php
<?php

function someFunction() {}

$functionVariable = 'someFunction';

var_dump(is_callable($functionVariable, false, $callable_name));

var_dump($callable_name);

?>
```

The above example will output:

```
bool(true)
string(12) "someFunction"
```

Example #2 Checking whether an array can be called as a function

```php
<?php

class SomeClass
{
    public function someMethod() {}
}

$anObject = new SomeClass();

$methodVariable = [$anObject, 'someMethod'];

var_dump(is_callable($methodVariable, true, $callable_name));

var_dump($callable_name);

?>
```

The above example will output:

```
bool(true)
string(21) "SomeClass::someMethod"
```

Example #3 is_callable() and constructors

Despite the fact that constructors are the methods that are called when an object is created, they are not static methods and **is_callable()** will return **`false`** for them. It's not possible to use **is_callable()** to check if a class can be instantiated from the current scope.

```php
<?php

class Foo
{
    public function __construct() {}

    public function foo() {}
}

var_dump(
    is_callable(['Foo', '__construct']),
    is_callable(['Foo', 'foo'])
);

$foo = new Foo();
var_dump(is_callable([$foo, '__construct']));

?>
```

The above example will output:

```
bool(false)
bool(false)
bool(true)
```

Notes

- An object is always callable if it implements __invoke(), and that method is visible in the current scope.
- A class name is callable if it implements __callStatic().
- If an object implements __call(), then this function will return `true` for any method on that object, even if the method is not defined.
- This function may trigger autoloading if called with the name of a class.

See Also

- call_user_func() - Call the callback given by the first parameter
- function_exists() - Return true if the given function has been defined
- method_exists() - Checks if the class method exists

is_countable

is_countable

(PHP 7 >= 7.3.0, PHP 8)

is_countable — Verify that the contents of a variable is a countable value

Description

function **is_countable**(mixed `$value`): bool

Verify that the contents of a variable is an array or an object implementing Countable

Parameters

`value`

The value to check

Return Values

Returns `true` if `value` is countable, `false` otherwise.

Changelog

Version	Description
7.3.0	**is_countable()** has been added.

Examples

Example #1 is_countable() examples

```
<?php
var_dump(is_countable([1, 2, 3])); // bool(true)
var_dump(is_countable(new ArrayIterator(['foo', 'bar', 'baz']))); // bool(true)
var_dump(is_countable(new ArrayIterator())); // bool(true)
var_dump(is_countable(new stdClass())); // bool(false)
?>
```

See Also

- is_array() - Finds whether a variable is an array
- is_object() - Finds whether a variable is an object
- is_iterable() - Verify that the contents of a variable is an iterable value
- is_bool() - Finds out whether a variable is a boolean

is_double

is_double

(PHP 4, PHP 5, PHP 7, PHP 8)

is_double — Alias of is_float()

Description

This function is an alias of: is_float().

is_float

is_float

(PHP 4, PHP 5, PHP 7, PHP 8)

is_float — Finds whether the type of a variable is float

Description

function **is_float**(mixed `$value`): bool

Finds whether the type of the given variable is float.

> **Note** :
>
> To test if a variable is a number or a numeric string (such as form input, which is always a string), you must use is_numeric().

Parameters

`value`

The variable being evaluated.

Return Values

Returns `true` if `value` is a float, `false` otherwise.

Examples

Example #1 is_float() example

```
<?php
```

```php
var_dump(is_float(27.25));
var_dump(is_float('abc'));
var_dump(is_float(23));
var_dump(is_float(23.5));
var_dump(is_float(1e7));  //Scientific Notation
var_dump(is_float(true));
?>
```

The above example will output:

```
bool(true)
bool(false)
bool(false)
bool(true)
bool(true)
bool(false)
```

See Also

- is_bool() - Finds out whether a variable is a boolean
- is_int() - Find whether the type of a variable is integer
- is_numeric() - Finds whether a variable is a number or a numeric string
- is_string() - Find whether the type of a variable is string
- is_array() - Finds whether a variable is an array
- is_object() - Finds whether a variable is an object

is_int

is_int

(PHP 4, PHP 5, PHP 7, PHP 8)

is_int — Find whether the type of a variable is integer

Description

function **is_int**(mixed `$value`): bool

Finds whether the type of the given variable is integer.

> **Note** :
>
> To test if a variable is a number or a numeric string (such as form input, which is always a string), you must use is_numeric().

Parameters

`value`

The variable being evaluated.

Return Values

Returns `true` if `value` is an int, `false` otherwise.

Examples

Example #1 is_int() example

```php
<?php
$values = array(23, "23", 23.5, "23.5", null, true, false);
foreach ($values as $value) {
    echo "is_int(";
```

```
    var_export($value);
    echo ") = ";
    var_dump(is_int($value));
}
?>
```

The above example will output:

```
is_int(23) = bool(true)
is_int('23') = bool(false)
is_int(23.5) = bool(false)
is_int('23.5') = bool(false)
is_int(NULL) = bool(false)
is_int(true) = bool(false)
is_int(false) = bool(false)
```

See Also

- is_bool() - Finds out whether a variable is a boolean
- is_float() - Finds whether the type of a variable is float
- is_numeric() - Finds whether a variable is a number or a numeric string
- is_string() - Find whether the type of a variable is string
- is_array() - Finds whether a variable is an array
- is_object() - Finds whether a variable is an object

is_integer

is_integer

(PHP 4, PHP 5, PHP 7, PHP 8)

is_integer — Alias of is_int()

Description

This function is an alias of: is_int().

is_iterable

is_iterable

(PHP 7 >= 7.1.0, PHP 8)

is_iterable — Verify that the contents of a variable is an iterable value

Description

function **is_iterable**(mixed `$value`): bool

Verify that the contents of a variable is accepted by the iterable pseudo-type, i.e. that it is either an array or an object implementing Traversable

Parameters

`value`

The value to check

Return Values

Returns `true` if `value` is iterable, `false` otherwise.

Examples

Example #1 is_iterable() examples

```php
<?php

var_dump(is_iterable([1, 2, 3]));  // bool(true)
var_dump(is_iterable(new ArrayIterator([1, 2, 3])));  // bool(true)
var_dump(is_iterable((function () { yield 1; })()));  // bool(true)
var_dump(is_iterable(1));  // bool(false)
var_dump(is_iterable(new stdClass()));  // bool(false)

?>
```

See Also

- is_array() - Finds whether a variable is an array

is_long

is_long

(PHP 4, PHP 5, PHP 7, PHP 8)

is_long — Alias of is_int()

Description

This function is an alias of: is_int().

is_null

is_null

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

is_null — Finds whether a variable is `null`

Description

function **is_null**(mixed `$value`): bool

Finds whether the given variable is `null`.

Parameters

`value`

The variable being evaluated.

Return Values

Returns `true` if `value` is null, `false` otherwise.

Examples

Example #1 is_null() example

```php
<?php

error_reporting(E_ALL);

$foo = NULL;
var_dump(is_null($inexistent), is_null($foo));

?>
```

See Also

- The `null` type
- isset() - Determine if a variable is declared and is different than null
- is_bool() - Finds out whether a variable is a boolean

- is_numeric() - Finds whether a variable is a number or a numeric string
- is_float() - Finds whether the type of a variable is float
- is_int() - Find whether the type of a variable is integer
- is_string() - Find whether the type of a variable is string
- is_object() - Finds whether a variable is an object
- is_array() - Finds whether a variable is an array

is_numeric

is_numeric

(PHP 4, PHP 5, PHP 7, PHP 8)

is_numeric — Finds whether a variable is a number or a numeric string

Description

function **is_numeric**(mixed `$value`): bool

Determines if the given variable is a number or a numeric string.

Parameters

`value`

The variable being evaluated.

Return Values

Returns **`true`** if `value` is a number or a numeric string, **`false`** otherwise.

Changelog

Version	Description
8.0.0	Numeric strings ending with whitespace (`"42 "`) will now return **`true`**. Previously, **`false`** was returned instead.

Examples

Example #1 is_numeric() examples

```php
<?php
$tests = array(
    "42",
    1337,
    0x539,
    02471,
    0b10100111001,
    1337e0,
    "0x539",
    "02471",
    "0b10100111001",
    "1337e0",
    "not numeric",
    array(),
    9.1,
    null,
    '',
);

foreach ($tests as $element) {
    if (is_numeric($element)) {
        echo var_export($element, true) . " is numeric", PHP_EOL;
    } else {
        echo var_export($element, true) . " is NOT numeric", PHP_EOL;
    }
}
?>
```

The above example will output:

```
'42' is numeric
1337 is numeric
1337 is numeric
1337 is numeric
1337 is numeric
1337.0 is numeric
'0x539' is NOT numeric
'02471' is numeric
'0b10100111001' is NOT numeric
'1337e0' is numeric
'not numeric' is NOT numeric
array (
) is NOT numeric
9.1 is numeric
NULL is NOT numeric
'' is NOT numeric
```

Example #2 is_numeric() with whitespace

```php
<?php
$tests = [
    " 42",
    "42 ",
    "\u{A0}9001", // non-breaking space
    "9001\u{A0}", // non-breaking space
];

foreach ($tests as $element) {
    if (is_numeric($element)) {
        echo var_export($element, true) . " is numeric", PHP_EOL;
    } else {
        echo var_export($element, true) . " is NOT numeric", PHP_EOL;
    }
}
?>
```

Output of the above example in PHP 8:

```
' 42' is numeric
'42 ' is numeric
' 9001' is NOT numeric
'9001 ' is NOT numeric
```

Output of the above example in PHP 7:

```
' 42' is numeric
'42 ' is NOT numeric
' 9001' is NOT numeric
'9001 ' is NOT numeric
```

See Also

- Numeric strings

- ctype_digit() - Check for numeric character(s)
- is_bool() - Finds out whether a variable is a boolean
- is_null() - Finds whether a variable is null
- is_float() - Finds whether the type of a variable is float
- is_int() - Find whether the type of a variable is integer
- is_string() - Find whether the type of a variable is string
- is_object() - Finds whether a variable is an object
- is_array() - Finds whether a variable is an array
- filter_var() - Filters a variable with a specified filter

is_object

is_object

(PHP 4, PHP 5, PHP 7, PHP 8)

is_object — Finds whether a variable is an object

Description

function **is_object**(mixed `$value`): bool

Finds whether the given variable is an object.

Parameters

`value`

The variable being evaluated.

Return Values

Returns `true` if `value` is an object, `false` otherwise.

Changelog

Version	Description
7.2.0	**is_object()** now returns `true` for unserialized objects without a class definition (class of __PHP_Incomplete_Class). Previously `false` was returned.

Examples

Example #1 is_object() example

```php
<?php
// Declare a simple function to return an
// array from our object
function get_students($obj)
{
    if (!is_object($obj)) {
        return false;
    }

    return $obj->students;
}

// Declare a new class instance and fill up
// some values
$obj = new stdClass();
$obj->students = array('Kalle', 'Ross', 'Felipe');

var_dump(get_students(null));
var_dump(get_students($obj));
?>
```

See Also

- is_bool() - Finds out whether a variable is a boolean
- is_int() - Find whether the type of a variable is integer

- is_float() - Finds whether the type of a variable is float
- is_string() - Find whether the type of a variable is string
- is_array() - Finds whether a variable is an array

is_real

is_real

(PHP 4, PHP 5, PHP 7, PHP 8)

is_real — Alias of is_float()

Description

This function is an alias of: is_float().

> **Warning**
>
> This alias was *DEPRECATED* in PHP 7.4.0, and *REMOVED* as of PHP 8.0.0.

is_resource

is_resource

(PHP 4, PHP 5, PHP 7, PHP 8)

is_resource — Finds whether a variable is a resource

Description

function **is_resource**(mixed `$value`): bool

Finds whether the given variable is a resource.

Parameters

`value`

The variable being evaluated.

Return Values

Returns **`true`** if `value` is a resource, **`false`** otherwise.

Examples

Example #1 is_resource() example

```
<?php

$handle = fopen("php://stdout", "w");
if (is_resource($handle)) {
    echo '$handle is a resource';
}

?>
```

The above example will output:

```
$handle is a resource
```

Notes

> **Note** :
>
> **is_resource()** is not a strict type-checking method: it will return `false` if `value` is a resource variable that has been closed.

See Also

- The resource-type documentation
- get_resource_type() - Returns the resource type

is_scalar

is_scalar

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

is_scalar — Finds whether a variable is a scalar

Description

function **is_scalar**(mixed `$value`): bool

Finds whether an expression is evaluated as a scalar value.

See scalar types for more information.

> **Note** :
>
> **is_scalar()** does not consider resource type values to be scalar as resources are abstract datatypes which are currently based on integers. This implementation detail should not be relied upon, as it may change.

> **Note** :
>
> **is_scalar()** does not consider NULL to be scalar.

Parameters

`value`

The variable being evaluated.

Return Values

Returns `true` if `value` is a scalar, `false` otherwise.

Examples

Example #1 is_scalar() example

```
<?php
function show_var($var)
{
    if (is_scalar($var)) {
        echo $var, PHP_EOL;
    } else {
        var_dump($var);
    }
}

$pi = 3.1416;
$proteins = array("hemoglobin", "cytochrome c oxidase", "ferredoxin");

show_var($pi);
show_var($proteins)

?>
```

The above example will output:

```
3.1416
array(3) {
  [0]=>
  string(10) "hemoglobin"
  [1]=>
  string(20) "cytochrome c oxidase"
  [2]=>
  string(10) "ferredoxin"
}
```

See Also

- is_float() - Finds whether the type of a variable is float
- is_int() - Find whether the type of a variable is integer
- is_numeric() - Finds whether a variable is a number or a numeric string
- is_real() - Alias of is_float
- is_string() - Find whether the type of a variable is string
- is_bool() - Finds out whether a variable is a boolean
- is_object() - Finds whether a variable is an object
- is_array() - Finds whether a variable is an array

is_string

is_string

(PHP 4, PHP 5, PHP 7, PHP 8)

is_string — Find whether the type of a variable is string

Description

function **is_string**(mixed `$value`): bool

Finds whether the type of the given variable is string.

Parameters

`value`

The variable being evaluated.

Return Values

Returns **`true`** if `value` is of type string, **`false`** otherwise.

Examples

Example #1 is_string() example

```php
<?php
$values = array(false, true, null, 'abc', '23', 23, '23.5', 23.5, '', ' ', '0', 0);
foreach ($values as $value) {
    echo "is_string(";
    var_export($value);
    echo ") = ";
    echo var_dump(is_string($value));
}
?>
```

The above example will output:

```
is_string(false) = bool(false)
is_string(true) = bool(false)
is_string(NULL) = bool(false)
is_string('abc') = bool(true)
is_string('23') = bool(true)
is_string(23) = bool(false)
is_string('23.5') = bool(true)
is_string(23.5) = bool(false)
is_string('') = bool(true)
is_string(' ') = bool(true)
is_string('0') = bool(true)
is_string(0) = bool(false)
```

See Also

- is_float() - Finds whether the type of a variable is float
- is_int() - Find whether the type of a variable is integer
- is_bool() - Finds out whether a variable is a boolean
- is_object() - Finds whether a variable is an object
- is_array() - Finds whether a variable is an array

isset

isset

(PHP 4, PHP 5, PHP 7, PHP 8)

isset — Determine if a variable is declared and is different than `null`

Description

function **isset**(mixed `$var`, mixed `...$vars`): bool

Determine if a variable is considered set, this means if a variable is declared and is different than `null`.

If a variable has been unset with the unset() function, it is no longer considered to be set.

isset() will return `false` when checking a variable that has been assigned to `null`. Also note that a null character (`"\0"`) is not equivalent to the PHP `null` constant.

If multiple parameters are supplied then **isset()** will return `true` only if all of the parameters are considered set. Evaluation goes from left to right and stops as soon as an unset variable is encountered.

Parameters

`var`

The variable to be checked.

`vars`

Further variables.

Return Values

Returns `true` if `var` exists and has any value other than `null`. `false` otherwise.

Examples

Example #1 isset() Examples

```
<?php

$var = '';
```

```php
// This will evaluate to TRUE so the text will be printed.
if (isset($var)) {
    echo "This var is set so I will print.", PHP_EOL;
}

// In the next examples we'll use var_dump to output
// the return value of isset().

$a = "test";
$b = "anothertest";

var_dump(isset($a));      // TRUE
var_dump(isset($a, $b)); // TRUE

unset ($a);

var_dump(isset($a));     // FALSE
var_dump(isset($a, $b)); // FALSE

$foo = NULL;
var_dump(isset($foo));   // FALSE

?>
```

This also work for elements in arrays:

Example #2 Example of isset() with array elements

```php
<?php

$a = array ('test' => 1, 'hello' => NULL, 'pie' => array('a' => 'apple'));

var_dump(isset($a['test']));            // TRUE
var_dump(isset($a['foo']));             // FALSE
var_dump(isset($a['hello']));           // FALSE

// The key 'hello' equals NULL so is considered unset
// If you want to check for NULL key values then try:
var_dump(array_key_exists('hello', $a)); // TRUE

// Checking deeper array values
var_dump(isset($a['pie']['a']));        // TRUE
var_dump(isset($a['pie']['b']));        // FALSE
var_dump(isset($a['cake']['a']['b']));  // FALSE

?>
```

Example #3 isset() on String Offsets

```php
<?php
$expected_array_got_string = 'somestring';
var_dump(isset($expected_array_got_string['some_key']));
var_dump(isset($expected_array_got_string[0]));
var_dump(isset($expected_array_got_string['0']));
var_dump(isset($expected_array_got_string[0.5]));
var_dump(isset($expected_array_got_string['0.5']));
var_dump(isset($expected_array_got_string['0 Mostel']));
?>
```

The above example will output:

```
bool(false)
bool(true)
bool(true)
bool(true)
bool(false)
bool(false)
```

Notes

> **Warning**
>
> **isset()** only works with variables as passing anything else will result in a parse error. For checking if constants are set use the defined() function.

> **Note** :
>
> Because this is a language construct and not a function, it cannot be called using variable functions, or named arguments.

> **Note** :
>
> When using **isset()** on inaccessible object properties, the __isset() overloading method will be called, if declared.

See Also

- empty() - Determine whether a variable is empty
- __isset()
- unset() - unset a given variable
- defined() - Checks whether a constant with the given name exists
- the type comparison tables
- array_key_exists() - Checks if the given key or index exists in the array
- is_null() - Finds whether a variable is null
- the error control @ operator

print_r

print_r

(PHP 4, PHP 5, PHP 7, PHP 8)

print_r — Prints human-readable information about a variable

Description

function **print_r**(mixed `$value`, bool `$return = false`): string|true

print_r() displays information about a variable in a way that's readable by humans.

print_r(), var_dump() and var_export() will also show protected and private properties of objects. Static class members will not be shown.

Parameters

`value`

The expression to be printed.

`return`

If you would like to capture the output of **print_r()**, use the `return` parameter. When this parameter is set to `true`, **print_r()** will return the information rather than print it.

Return Values

If given a string, int or float, the value itself will be printed. If given an array, values will be presented in a format that shows keys and elements. Similar notation is used for objects.

When the `return` parameter is `true`, this function will return a string. Otherwise, the return value is `true`.

Changelog

Version	Description
8.4.0	Return type changed from string|bool to string|true.

Examples

Example #1 print_r() example

```
<pre>
<?php
$a = array ('a' => 'apple', 'b' => 'banana', 'c' => array ('x', 'y', 'z'));
print_r($a);
?>
</pre>
```

The above example will output:

```
<pre>
Array
(
    [a] => apple
    [b] => banana
    [c] => Array
        (
            [0] => x
            [1] => y
            [2] => z
        )
)
</pre>
```

Example #2 `return` parameter example

```
<?php
$b = array ('m' => 'monkey', 'foo' => 'bar', 'x' => array ('x', 'y', 'z'));
$results = print_r($b, true); // $results now contains output from print_r

print_r($results);
?>
```

Notes

> **Note** :
>
> When the `return` parameter is used, this function uses internal output buffering prior to PHP 7.1.0, so it cannot be used inside an ob_start() callback function.

See Also

- ob_start() - Turn on output buffering
- var_dump() - Dumps information about a variable
- var_export() - Outputs or returns a parsable string representation of a variable

serialize

serialize

(PHP 4, PHP 5, PHP 7, PHP 8)

serialize — Generates a storable representation of a value

Description

function **serialize**(mixed `$value`): string

Generates a storable representation of a value.

This is useful for storing or passing PHP values around without losing their type and structure.

To make the serialized string into a PHP value again, use unserialize().

Parameters

`value`

The value to be serialized. **serialize()** handles all types, except the resource-type and some objects (see note below). You can even **serialize()** arrays that contain references to itself. Circular references inside the array/object you are serializing will also be stored. Any other reference will be lost.

When serializing objects, PHP will attempt to call the member functions __serialize() or __sleep() prior to serialization. This is to allow the object to do any last minute clean-up, etc. prior to being serialized. Likewise, when the object is restored using unserialize() the __unserialize() or __wakeup() member function is called.

> **Note** :
>
> Object's private members have the class name prepended to the member name; protected members have a '*' prepended to the member name. These prepended values have null bytes on either side.

Return Values

Returns a string containing a byte-stream representation of `value` that can be stored anywhere.

Note that this is a binary string which may include null bytes, and needs to be stored and handled as such. For example, **serialize()** output should generally be stored in a BLOB field in a database, rather than a CHAR or TEXT field.

Examples

Example #1 serialize() example

```php
<?php
// $session_data contains a multi-dimensional array with session
// information for the current user.  We use serialize() to store
// it in a database at the end of the request.

$conn = odbc_connect("webdb", "php", "chicken");
$stmt = odbc_prepare($conn,
      "UPDATE sessions SET data = ? WHERE id = ?");
$sqldata = array (serialize($session_data), $_SERVER['PHP_AUTH_USER']);
if (!odbc_execute($stmt, $sqldata)) {
    $stmt = odbc_prepare($conn,
     "INSERT INTO sessions (id, data) VALUES(?, ?)");
    if (!odbc_execute($stmt, array_reverse($sqldata))) {
        /* Something went wrong.. */
    }
}
?>
```

Notes

> **Note** :
>
> Note that many built-in PHP objects cannot be serialized. However, those with this ability either implement the Serializable interface or the magic __serialize()/__unserialize() or __sleep()/__wakeup() methods. If an internal class does not fulfill any of those requirements, it cannot reliably be serialized.
>
> There are some historical exceptions to the above rule, where some internal objects could be serialized without implementing the interface or exposing the methods.

> **Warning**

> When **serialize()** serializes objects, the leading backslash is not included in the class name of namespaced classes for maximum compatibility.

See Also

- unserialize() - Creates a PHP value from a stored representation
- var_export() - Outputs or returns a parsable string representation of a variable
- json_encode() - Returns the JSON representation of a value
- Serializing Objects
- __sleep()
- __wakeup()
- __serialize()
- __unserialize()

settype

settype

(PHP 4, PHP 5, PHP 7, PHP 8)

settype — Set the type of a variable

Description

function **settype**(mixed `&$var`, string `$type`): bool

Set the type of variable `var` to `type`.

Parameters

`var`

The variable being converted.

`type`

Possibles values of `type` are:

- "boolean" or "bool"
- "integer" or "int"
- "float" or "double"
- "string"
- "array"
- "object"
- "null"

Return Values

Returns **`true`** on success or **`false`** on failure.

Errors/Exceptions

Throws a ValueError if the value of `type` is not a valid type, as of PHP 8.0.0. Prior to PHP 8.0.0, a **`E_WARNING`** was emitted and **`false`** was returned.

Changelog

Version	Description
8.0.0	Now throws a ValueError when an invalid type is passed to `type`. Previously, a **`E_WARNING`** was emitted and the function returned **`false`**.

Examples

Example #1 settype() example

```php
<?php
$foo = "5bar"; // string
$bar = true;   // boolean

settype($foo, "integer"); // $foo is now 5   (integer)
settype($bar, "string");  // $bar is now "1" (string)

var_dump($foo, $bar);
?>
```

Notes

> **Note** :
>
> Maximum value for "int" is `PHP_INT_MAX`.

See Also

- gettype() - Get the type of a variable
- type-casting
- type-juggling

strval

strval

(PHP 4, PHP 5, PHP 7, PHP 8)

strval — Get string value of a variable

Description

function **strval**(mixed `$value`): string

Get the string value of a variable. See the documentation on string for more information on converting to string.

This function performs no formatting on the returned value. If you are looking for a way to format a numeric value as a string, please see sprintf() or number_format().

Parameters

`value`

The variable that is being converted to a string.

`value` may be any scalar type, null, or an object that implements the __toString() method. You cannot use **strval()** on arrays or on objects that do not implement the __toString() method.

Return Values

The string value of `value`.

Examples

Example #1 strval() example using PHP magic __toString() method.

```php
<?php
class StrValTest
{
    public function __toString()
    {
        return __CLASS__;
    }
}

// Prints 'StrValTest'
```

```php
echo strval(new StrValTest);
?>
```

See Also

- boolval() - Get the boolean value of a variable
- floatval() - Get float value of a variable
- intval() - Get the integer value of a variable
- settype() - Set the type of a variable
- sprintf() - Return a formatted string
- number_format() - Format a number with grouped thousands
- Type juggling
- __toString()

unserialize

unserialize

(PHP 4, PHP 5, PHP 7, PHP 8)

unserialize — Creates a PHP value from a stored representation

Description

function **unserialize**(string `$data`, array `$options` = []): mixed

unserialize() takes a single serialized variable and converts it back into a PHP value.

> **Warning**
>
> Do not pass untrusted user input to **unserialize()** regardless of the `options` value of `allowed_classes`. Unserialization can result in code being loaded and executed due to object instantiation and autoloading, and a malicious user may be able to exploit this. Use a safe, standard data interchange format such as JSON (via json_decode() and json_encode()) if you need to pass serialized data to the user.
>
> If you need to unserialize externally-stored serialized data, consider using hash_hmac() for data validation. Make sure data is not modified by anyone but you.

Parameters

`data`

The serialized string.

If the variable being unserialized is an object, after successfully reconstructing the object PHP will automatically attempt to call the __unserialize() or __wakeup() methods (if one exists).

> **Note** : **unserialize_callback_func directive**
>
> The callback specified in the unserialize_callback_func directive is called when an undefined class is unserialized. If no callback is specified, the object will be instantiated as __PHP_Incomplete_Class.

`options`

Any options to be provided to **unserialize()**, as an associative array.

Valid options

Name	Type	Description
`allowed_classes`	array|bool	Either an array of class names which should be accepted, `false` to accept no classes, or `true` to accept all classes. If this option is defined and **unserialize()** encounters an object of a class that isn't to be accepted, then the object will be instantiated as __PHP_Incomplete_Class instead. Omitting this option is the same as defining it as `true`: PHP will attempt to instantiate objects of any class. This option does not affect Enumerations.
`max_depth`	int	The maximum depth of structures permitted during unserialization, and is intended to prevent stack overflows. The default depth limit is `4096` and can be disabled by setting `max_depth` to `0`.

Return Values

The converted value is returned, and can be a bool, int, float, string, array or object.

In case the passed string is not unserializeable, `false` is returned and `E_WARNING` is issued.

Errors/Exceptions

Objects may throw Throwables in their unserialization handlers.

As of PHP 8.4.0, if the `allowed_classes` element of `options` is neither an array of class names nor a bool, **unserialize()** throws TypeError and ValueError.

Changelog

Version	Description
8.4.0	Now throws TypeError and ValueError if the `allowed_classes` element of `options` is neither an array of class names nor a bool.
8.4.0	Unserializing strings using the uppercase `"S"` tag is now deprecated; use the lowercase `"s"` tag instead.
8.3.0	Now emits `E_WARNING` when the input string has unconsumed data.
8.3.0	Now emits `E_WARNING` when the passed string is not unserializeable; previously `E_NOTICE` was emitted.
7.4.0	Added the `max_depth` element of `options` to set the maximum depth of structures permitted during unserialization.
7.1.0	The `allowed_classes` element of `options` is now strictly typed, i.e. if anything other than an array or a bool is given, **unserialize()** returns `false` and issues an `E_WARNING`.

Examples

Example #1 unserialize() example

```php
<?php
// Here, we use unserialize() to load session data to the
// $session_data array from the string selected from a database.
// This example complements the one described with serialize().

$conn = odbc_connect("webdb", "php", "chicken");
$stmt = odbc_prepare($conn, "SELECT data FROM sessions WHERE id = ?");
$sqldata = array($_SERVER['PHP_AUTH_USER']);
if (!odbc_execute($stmt, $sqldata) || !odbc_fetch_into($stmt, $tmp)) {
    // if the execute or fetch fails, initialize to empty array
    $session_data = array();
} else {
    // we should now have the serialized data in $tmp[0].
    $session_data = unserialize($tmp[0]);
    if (!is_array($session_data)) {
        // something went wrong, initialize to empty array
        $session_data = array();
    }
}
?>
```

Example #2 unserialize_callback_func example

```php
<?php
$serialized_object='O:1:"a":1:{s:5:"value";s:3:"100";}';

ini_set('unserialize_callback_func', 'mycallback'); // set your callback_function

function mycallback($classname)
{
    // just include a file containing your class definition
    // you get $classname to figure out which class definition is required
    var_dump($classname);
}

unserialize($serialized_object);
?>
```

Notes

> **Warning**
>
> `false` is returned both in the case of an error and if unserializing the serialized `false` value. It is possible to catch this special case by comparing `data` with `serialize(false)` or by catching the issued `E_WARNING`.

See Also

- json_encode() - Returns the JSON representation of a value
- json_decode() - Decodes a JSON string
- hash_hmac() - Generate a keyed hash value using the HMAC method
- serialize() - Generates a storable representation of a value
- Autoloading Classes
- unserialize_callback_func
- unserialize_max_depth
- __wakeup()
- __serialize()
- __unserialize()

unset

unset

(PHP 4, PHP 5, PHP 7, PHP 8)

unset — **unset()** a given variable

Description

function **unset**(mixed `$var`, mixed `...$vars`): void

unset() destroys the specified variables.

The behavior of **unset()** inside of a function can vary depending on what type of variable you are attempting to destroy.

If a globalized variable is **unset()** inside of a function, only the local variable is destroyed. The variable in the calling environment will retain the same value as before **unset()** was called.

Example #1 Using unset()

```php
<?php
function destroy_foo()
{
    global $foo;
    unset($foo);
}

$foo = 'bar';
destroy_foo();
echo $foo;
?>
```

To **unset()** a global variable inside of a function, then use the *$GLOBALS* array to do so:

Example #2 unset() a Global Variable

```php
<?php
function foo()
{
    unset($GLOBALS['bar']);
}

$bar = "something";
foo();
?>
```

If a variable that is PASSED BY REFERENCE is **unset()** inside of a function, only the local variable is destroyed. The variable in the calling environment will retain the same value as before **unset()** was called.

Example #3 unset() with Reference

```php
<?php
function foo(&$bar)
{
    unset($bar);
    $bar = "blah";
}

$bar = 'something';
echo "$bar\n";

foo($bar);
echo "$bar\n";
?>
```

If a static variable is **unset()** inside of a function, **unset()** destroys the variable only in the context of the rest of a function. Following calls will restore the previous value of a variable.

Example #4 unset() with Static Variable

```php
<?php
function foo()
{
    static $bar;
    $bar++;
    echo "Before unset: $bar, ";
    unset($bar);
    $bar = 23;
    echo "after unset: $bar\n";
}

foo();
foo();
foo();
?>
```

Parameters

`var`

The variable to be unset.

`vars`

Further variables.

Return Values

No value is returned.

Examples

Example #5 unset() example

```php
<?php
// destroy a single variable
unset($foo);

// destroy a single element of an array
unset($bar['quux']);

// destroy more than one variable
unset($foo1, $foo2, $foo3);
?>
```

Notes

> **Note** :
>
> Because this is a language construct and not a function, it cannot be called using variable functions, or named arguments.

> **Note** :
>
> It is possible to unset object properties visible in the current context.
>
> If declared, __get() is called when accessing an unset property, and __set() is called when setting an unset property.

> **Note** :
>
> It is not possible to unset `$this` inside an object method.

> **Note** :
>
> When using **unset()** on inaccessible object properties, the __unset() overloading method will be called, if declared.

See Also

- isset() - Determine if a variable is declared and is different than null
- empty() - Determine whether a variable is empty
- __unset()
- array_splice() - Remove a portion of the array and replace it with something else
- (unset) casting

var_dump

var_dump

(PHP 4, PHP 5, PHP 7, PHP 8)

var_dump — Dumps information about a variable

Description

function **var_dump**(mixed `$value`, mixed `...$values`): void

This function displays structured information about one or more expressions that includes its type and value. Arrays and objects are explored recursively with values indented to show structure.

All public, private and protected properties of objects will be returned in the output unless the object implements a __debugInfo() method.

> **Tip**
>
> As with anything that outputs its result directly to the browser, the output-control functions can be used to capture the output of this function, and save it in a string (for example).

Parameters

`value`

The expression to dump.

`values`

Further expressions to dump.

Return Values

No value is returned.

Examples

Example #1 var_dump() example

```php
<?php
$a = array(1, 2, array("a", "b", "c"));
var_dump($a);
?>
```

The above example will output:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  array(3) {
    [0]=>
    string(1) "a"
    [1]=>
    string(1) "b"
    [2]=>
    string(1) "c"
  }
}
```

```php
<?php

$b = 3.1;
$c = true;
var_dump($b, $c);

?>
```

The above example will output:

```
float(3.1)
bool(true)
```

See Also

- print_r() - Prints human-readable information about a variable
- debug_zval_dump() - Dumps a string representation of an internal zval structure to output
- var_export() - Outputs or returns a parsable string representation of a variable
- __debugInfo()

var_export

var_export

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

var_export — Outputs or returns a parsable string representation of a variable

Description

function **var_export**(mixed `$value`, bool `$return = false`): ?string

var_export() gets structured information about the given variable. It is similar to var_dump() with one exception: the returned representation is valid PHP code.

Parameters

`value`

The variable you want to export.

`return`

If used and set to `true`, **var_export()** will return the variable representation instead of outputting it.

Return Values

Returns the variable representation when the `return` parameter is used and evaluates to `true`. Otherwise, this function will return `null`.

Changelog

Version	Description
8.2.0	Exported class names are now fully qualified; previously, the leading backslash was omitted.
7.3.0	Now exports stdClass objects as an array cast to an object (`(object) array( ... )`), rather than using the nonexistent method **stdClass::__setState()**. The practical effect is that now stdClass is exportable, and the resulting code will even work on earlier versions of PHP.

Examples

Example #1 var_export() Examples

```php
<?php
$a = array (1, 2, array ("a", "b", "c"));
var_export($a);
?>
```

The above example will output:

```
array (
  0 => 1,
  1 => 2,
  2 =>
  array (
    0 => 'a',
    1 => 'b',
    2 => 'c',
  ),
)
```

```php
<?php

$b = 3.1;
$v = var_export($b, true);
echo $v;

?>
```

The above example will output:

```
3.1
```

Example #2 Exporting stdClass (since PHP 7.3.0)

```php
<?php
$person = new stdClass;
$person->name = 'ElePHPant ElePHPantsdotter';
$person->website = 'https://php.net/elephpant.php';

var_export($person);
```

The above example will output:

```
(object) array(
   'name' => 'ElePHPant ElePHPantsdotter',
   'website' => 'https://php.net/elephpant.php',
)
```

Example #3 Exporting classes

```php
<?php
class A { public $var; }
$a = new A;
$a->var = 5;
var_export($a);
?>
```

The above example will output:

```
\A::__set_state(array(
   'var' => 5,
))
```

Example #4 Using __set_state()

```php
<?php
class A
{
    public $var1;
    public $var2;

    public static function __set_state($an_array)
    {
        $obj = new A;
        $obj->var1 = $an_array['var1'];
        $obj->var2 = $an_array['var2'];
        return $obj;
    }
}

$a = new A;
$a->var1 = 5;
$a->var2 = 'foo';

eval('$b = ' . var_export($a, true) . ';'); // $b = A::__set_state(array(
                                            //    'var1' => 5,
                                            //    'var2' => 'foo',
                                            // ));
```

```
var_dump($b);
?>
```

The above example will output:

```
object(A)#2 (2) {
  ["var1"]=>
  int(5)
  ["var2"]=>
  string(3) "foo"
}
```

Notes

> **Note** :
>
> Variables of type resource couldn't be exported by this function.

> **Note** :
>
> **var_export()** does not handle circular references as it would be close to impossible to generate parsable PHP code for that. If you want to do something with the full representation of an array or object, use serialize().

> **Warning**
>
> Prior to PHP 8.2.0, when **var_export()** exports objects, the leading backslash is not included in the class name of namespaced classes for maximum compatibility.

> **Note** :
>
> To be able to evaluate the PHP generated by **var_export()**, all processed objects must implement the magic __set_state method. The only exception is stdClass, which is exported using an array cast to an object.

See Also

- print_r() - Prints human-readable information about a variable
- serialize() - Generates a storable representation of a value
- var_dump() - Dumps information about a variable

Regular Expressions (Perl-Compatible)

Introduction

This extension integrates regular expression pattern matching support into PHP. It is based on the free and open-source » PCRE2 library. This library implements regular expression pattern matching using syntax and semantics compatible with Perl, with a few » documented differences. See Pattern Syntax and Pattern Modifiers for a detailed explanation.

To improve performance, the extension caches compiled regular expressions. Each thread has its own dedicated cache, capable of holding up to 4096 expressions.

> **Note** :
>
> When the cache is full, a batch of the oldest entries that are not currently in use is discarded to make room for new ones. The cache size is not configurable.

> **Warning**
>
> PCRE2 imposes » size and other limits that may be relevant in some cases.

Installing/Configuring

Table of Contents

Installation

The PCRE extension is a core PHP extension and is always enabled.

By default, the extension uses a bundled version of the PCRE2 library. On non-Windows systems, an external PCRE2 library can be used instead with the **--with-external-pcre** configuration option. The minimum supported version is 10.30. Windows builds always use the bundled library.

PCRE2's just-in-time (JIT) compilation is enabled by default. It can be disabled with the **--without-pcre-jit** configuration option.

The Windows version of PHP has built-in support for this extension. You do not need to load any additional extensions in order to use these functions.

PCRE2 is an active project, and as it changes, so does the PHP functionality that relies upon it. It is possible that certain parts of the PHP documentation are outdated. For a list of changes, see the » PCRE2 library changelog. The upgrade history of the bundled library is listed below:

Upgrade history of the bundled PCRE library

PHP Version	Updated PCRE Version	Notes
8.4.0	10.44	
8.3.0	10.42	
8.2.0	10.40	
8.1.1	10.39	
8.1.0	10.37	
7.4.12, 8.0.0	10.35	
7.4.6	10.34	
7.4.0	10.33	
7.3.0	10.32	
7.2.0	8.41	
7.0.3	8.38	See CVE-2015-8383, CVE-2015-8386, CVE-2015-8387, CVE-2015-8389, CVE-2015-8390, CVE-2015-8391, CVE-2015-8393, CVE-2015-8394

PHP Version	Updated PCRE Version	Notes
7.0.0	8.37	See CVE-2015-2325, CVE-2015-2326

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

PCRE Configuration Options

Name	Default	Changeable	Changelog
pcre.backtrack_limit	"1000000"	INI_ALL	
pcre.recursion_limit	"100000"	INI_ALL	
pcre.jit	"1"	INI_ALL	

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`pcre.backtrack_limit` int

PCRE's backtracking limit. Defaults to 100000 for PHP < 5.3.7.

`pcre.recursion_limit` int

PCRE's recursion limit. Please note that if you set this value to a high number you may consume all the available process stack and eventually crash PHP (due to reaching the stack size limit imposed by the Operating System).

`pcre.jit` bool

Whether PCRE's just-in-time compilation is going to be used.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

PREG constants

Constants	Description	As of
PREG_PATTERN_ORDER (int)	Orders results so that *$matches[0]* is an array of full pattern matches, *$matches[1]* is an array of strings matched by the first parenthesized subpattern, and so on. This flag is only used with preg_match_all().	
PREG_SET_ORDER (int)	Orders results so that *$matches[0]* is an array of first set of matches, *$matches[1]* is an array of second set of matches, and so on. This flag is only used with preg_match_all().	
PREG_OFFSET_CAPTURE (int)	If this flag is set, for every occurring match the appendant byte offset will also be returned. Note that this changes the return values in an array where every element is an array consisting of the matched string at offset 0 and its string offset within subject at offset 1.	
PREG_SPLIT_NO_EMPTY (int)	This flag tells preg_split() to return only non-empty pieces.	
PREG_SPLIT_DELIM_CAPTURE (int)	This flag tells preg_split() to capture parenthesized expression in the delimiter pattern as well.	
PREG_SPLIT_OFFSET_CAPTURE (int)	See the description of PREG_OFFSET_CAPTURE. This flag is only used for preg_split().	
PREG_UNMATCHED_AS_NULL (int)	This flag tells preg_match() and preg_match_all() to include unmatched subpatterns in *$matches* as null values. Without this flag, unmatched subpatterns are reported as empty strings, as if they were empty matches. Setting this flag allows to distinguish between these two cases. With preg_match(), trailing unmatched subpatterns are omitted from *$matches* entirely unless this flag is used; as of PHP 7.4.0 the flag reports them as null, so that *$matches* always has the same size.	7.2.0
PREG_NO_ERROR (int)	Returned by preg_last_error() if there were no errors.	5.2.0
PREG_INTERNAL_ERROR (int)	Returned by preg_last_error() if there was an internal PCRE error.	5.2.0

Constants	Description	As of
PREG_BACKTRACK_LIMIT_ERROR (int)	Returned by preg_last_error() if backtrack limit was exhausted.	5.2.0
PREG_RECURSION_LIMIT_ERROR (int)	Returned by preg_last_error() if recursion limit was exhausted.	5.2.0
PREG_BAD_UTF8_ERROR (int)	Returned by preg_last_error() if the last error was caused by malformed UTF-8 data (only when running a regex in UTF-8 mode).	5.2.0
PREG_BAD_UTF8_OFFSET_ERROR (int)	Returned by preg_last_error() if the offset didn't correspond to the begin of a valid UTF-8 code point (only when running a regex in UTF-8 mode).	5.3.0
PREG_JIT_STACKLIMIT_ERROR (int)	Returned by preg_last_error() if the last PCRE function failed due to limited JIT stack space.	7.0.0
PCRE_VERSION (string)	PCRE version and release date (e.g. "`7.0 18-Dec-2006`").	5.2.4
PCRE_VERSION_MAJOR (int)	PCRE major version number.	
PCRE_VERSION_MINOR (int)	PCRE minor version number.	
PCRE_JIT_SUPPORT (bool)	Indicates whether PCRE JIT support is available.	
PREG_GREP_INVERT (int)	Return the elements that do not match a given pattern.	

Examples

Example #1 Examples of valid patterns

- `/<\/\w+>/`
- `|(\d{3})-\d+|Sm`
- `/^(?i)php[34]/`
- `{^\s+(\s+)?$}`

Example #2 Examples of invalid patterns

- `/href='(.*)'` - missing ending delimiter
- `/\w+\s*\w+/J` - unknown modifier 'J'
- `1-\d3-\d3-\d4|` - missing starting delimiter

PCRE Patterns

Table of Contents

PCRE regex syntax

Table of Contents

Introduction

The syntax and semantics of the regular expressions supported by PCRE are described in this section. Regular expressions are also described in the Perl documentation and in a number of other books, some of which have copious examples. Jeffrey Friedl's "Mastering Regular Expressions", published by O'Reilly (ISBN 1-56592-257-3), covers them in great detail. The description here is intended as reference documentation.

A regular expression is a pattern that is matched against a subject string from left to right. Most characters stand for themselves in a pattern, and match the corresponding characters in the subject. As a trivial example, the pattern `The quick brown fox` matches a portion of a subject string that is identical to itself.

Delimiters

When using the PCRE functions, it is required that the pattern is enclosed by *delimiters*. A delimiter can be any non-alphanumeric, non-backslash, non-NUL, non-whitespace single-byte character. Leading whitespace before a valid delimiter is silently ignored.

> **Note** :
>
> Multi-byte characters (such as UTF-8 encoded characters like §) cannot be used as delimiters. PHP reads exactly one byte as the delimiter; using a multi-byte character would cause the remaining bytes to be misinterpreted as unknown pattern modifiers.

Often used delimiters are forward slashes (`/`), hash signs (`#`) and tildes (`~`). The following are all examples of valid delimited patterns.

```
/foo bar/
#^[^0-9]$#
+php+
%[a-zA-Z0-9_-]%
```

It is also possible to use bracket style delimiters where the opening and closing brackets are the starting and ending delimiter, respectively. `()`, `{}`, `[]` and `<>` are all valid bracket style delimiter pairs.

```
(this [is] a (pattern))
{this [is] a (pattern)}
[this [is] a (pattern)]
<this [is] a (pattern)>
```

Bracket style delimiters do not need to be escaped when they are used as meta characters within the pattern, but as with other delimiters they must be escaped when they are used as literal characters.

If the delimiter needs to be matched inside the pattern it must be escaped using a backslash. If the delimiter appears often inside the pattern, it is a good idea to choose another delimiter in order to increase readability.

```
/http:\/\//
#http://#
```

The preg_quote() function may be used to escape a string for injection into a pattern and its optional second parameter may be used to specify the delimiter to be escaped.

You may add pattern modifiers after the ending delimiter. The following is an example of case-insensitive matching:

```
#[a-z]#i
```

Meta-characters

The power of regular expressions comes from the ability to include alternatives and repetitions in the pattern. These are encoded in the pattern by the use of *meta-characters*, which do not stand for themselves but instead are interpreted in some special way.

There are two different sets of meta-characters: those that are recognized anywhere in the pattern except within square brackets, and those that are recognized in square brackets. Outside square brackets, the meta-characters are as follows:

Meta-characters outside square brackets

Meta-character	Description
\	general escape character with several uses
^	assert start of subject (or line, in multiline mode)
$	assert end of subject or before a terminating newline (or end of line, in multiline mode)
.	match any character except newline (by default)
[	start character class definition
]	end character class definition
|	start of alternative branch
(	start subpattern
)	end subpattern
?	Introduces a special construct, such as a non-capturing group, a named subpattern, an assertion or an option setting, if used as the first character after `(`. If used as a quantifier it is the optional quantifier (equivalent to `{0,1}`). It also transforms greedy quantifiers into lazy quantifiers (see repetition).

Meta-character	Description
*	0 or more quantifier
+	1 or more quantifier
{	start min/max quantifier
}	end min/max quantifier

Part of a pattern that is in square brackets is called a character class. In a character class the only meta-characters are:

Meta-characters inside square brackets (*character classes*)

Meta-character	Description
\	general escape character
^	negate the class, but only if the first character
-	indicates character range

The following sections describe the use of each of the meta-characters.

Escape sequences

The backslash character has several uses. Firstly, if it is followed by a non-alphanumeric character, it takes away any special meaning that character may have. This use of backslash as an escape character applies both inside and outside character classes.

For example, if you want to match a "*" character, you write "*" in the pattern. This applies whether or not the following character would otherwise be interpreted as a meta-character, so it is always safe to precede a non-alphanumeric with "\" to specify that it stands for itself. In particular, if you want to match a backslash, you write "\\".

> **Note** :
>
> Single and double quoted PHP strings have special meaning of backslash. Thus if \ has to be matched with a regular expression \\, then "\\\\" or '\\\\' must be used in PHP code.

If a pattern is compiled with the PCRE_EXTENDED option, whitespace in the pattern (other than in a character class) and characters between a "#" outside a character class and the next newline character are ignored. An escaping backslash can be used to include a whitespace or "#" character as part of the pattern.

A second use of backslash provides a way of encoding non-printing characters in patterns in a visible manner. There is no restriction on the appearance of non-printing characters, apart from the binary zero that terminates a pattern, but when a pattern is being prepared by text editing, it is usually easier to use one of the following escape sequences than the binary character it represents:

\a

alarm, that is, the BEL character (hex 07)

\cx

"control-x", where x is any character

\e

escape (hex 1B)

\f

formfeed (hex 0C)

\n

newline (hex 0A)

\p{xx}

a character with the xx property, see unicode properties for more info

\P{xx}

a character without the xx property, see unicode properties for more info

\r

carriage return (hex 0D)

\R

line break: matches \n, \r and \r\n

\t

tab (hex 09)

\xhh

character with hex code hh

\ddd

character with octal code ddd, or backreference

The precise effect of "\cx" is as follows: if "x" is a lower case letter, it is converted to upper case. Then bit 6 of the character (hex 40) is inverted. Thus "\cz" becomes hex 1A, but "\c{" becomes hex 3B, while "\c;" becomes hex 7B.

After "\x", up to two hexadecimal digits are read (letters can be in upper or lower case). In *UTF-8 mode*, "\x{...}" is allowed, where the contents of the braces is a string of hexadecimal digits. It is interpreted as a UTF-8 character whose code number is the given hexadecimal number. The original hexadecimal escape sequence, \xhh, matches a two-byte UTF-8 character if the value is greater than 127.

After "\0" up to two further octal digits are read. In both cases, if there are fewer than two digits, just those that are present are used. Thus the sequence "\0\x\07" specifies two binary zeros followed by a BEL character. Make sure you supply two digits after the initial zero if the character that follows is itself an octal digit.

The handling of a backslash followed by a digit other than 0 is complicated. Outside a character class, PCRE reads it and any following digits as a decimal number. If the number is less than 10, or if there have been at least that many previous capturing left parentheses in the expression, the entire sequence is taken as a *back reference*. A description of how this works is given later, following the discussion of parenthesized subpatterns.

Inside a character class, or if the decimal number is greater than 9 and there have not been that many capturing subpatterns, PCRE re-reads up to three octal digits following the backslash, and generates a single byte from the least significant 8 bits of the value. Any subsequent digits stand for themselves. For example:

\040

is another way of writing a space

\40

is the same, provided there are fewer than 40 previous capturing subpatterns

\7

is always a back reference

\11

might be a back reference, or another way of writing a tab

\011

is always a tab

\0113

is a tab followed by the character "3"

\113

is the character with octal code 113 (since there can be no more than 99 back references)

\377

is a byte consisting entirely of 1 bits

\81

is either a back reference, or a binary zero followed by the two characters "8" and "1"

Note that octal values of 100 or greater must not be introduced by a leading zero, because no more than three octal digits are ever read.

All the sequences that define a single byte value can be used both inside and outside character classes. In addition, inside a character class, the sequence "\b" is interpreted as the backspace character (hex 08). Outside a character class it has a different meaning (see below).

The third use of backslash is for specifying generic character types:

\d

any decimal digit

\D

any character that is not a decimal digit

\h

any horizontal whitespace character

\H

any character that is not a horizontal whitespace character

\s

any whitespace character

\S

any character that is not a whitespace character

\v

any vertical whitespace character

\V

any character that is not a vertical whitespace character

\w

any "word" character

\W

any "non-word" character

Each pair of escape sequences partitions the complete set of characters into two disjoint sets. Any given character matches one, and only one, of each pair.

The "whitespace" characters are HT (9), LF (10), FF (12), CR (13), and space (32). However, if locale-specific matching is happening, characters with code points in the range 128-255 may also be considered as whitespace characters, for instance, NBSP (A0).

A "word" character is any letter or digit or the underscore character, that is, any character which can be part of a Perl "*word*". The definition of letters and digits is controlled by PCRE's character tables, and may vary if locale-specific matching is taking place. For example, in the "fr" (French) locale, some character codes greater than 128 are used for accented letters, and these are matched by \w.

These character type sequences can appear both inside and outside character classes. They each match one character of the appropriate type. If the current matching point is at the end of the subject string, all of them fail, since there is no character to match.

The fourth use of backslash is for certain simple assertions. An assertion specifies a condition that has to be met at a particular point in a match, without consuming any characters from the subject string. The use of subpatterns for more complicated assertions is described below. The backslashed assertions are

\b

word boundary

\B

not a word boundary

\A

start of subject (independent of multiline mode)

\Z

end of subject or newline at end (independent of multiline mode)

\z

end of subject (independent of multiline mode)

\G

first matching position in subject

These assertions may not appear in character classes (but note that "`\b`" has a different meaning, namely the backspace character, inside a character class).

A word boundary is a position in the subject string where the current character and the previous character do not both match `\w` or `\W` (i.e. one matches `\w` and the other matches `\W`), or the start or end of the string if the first or last character matches `\w`, respectively.

The `\A`, `\Z`, and `\z` assertions differ from the traditional circumflex and dollar (described in anchors) in that they only ever match at the very start and end of the subject string, whatever options are set. They are not affected by the PCRE_MULTILINE or PCRE_DOLLAR_ENDONLY options. The difference between `\Z` and `\z` is that `\Z` matches before a newline that is the last character of the string as well as at the end of the string, whereas `\z` matches only at the end.

The `\G` assertion is true only when the current matching position is at the start point of the match, as specified by the `offset` argument of preg_match(). It differs from `\A` when the value of `offset` is non-zero.

`\Q` and `\E` can be used to ignore regexp metacharacters in the pattern. For example: `\w+\Q.$.\E$` will match one or more word characters, followed by literals `.$.` and anchored at the end of the string. Note that this does not change the behavior of delimiters; for instance the pattern `#\Q#\E#$` is not valid, because the second `#` marks the end of the pattern, and the `\E#` is interpreted as invalid modifiers.

`\K` can be used to reset the match start. For example, the pattern `foo\Kbar` matches "foobar", but reports that it has matched "bar". The use of `\K` does not interfere with the setting of captured substrings. For example, when the pattern `(foo)\Kbar` matches "foobar", the first substring is still set to "foo".

Unicode character properties

Since 5.1.0, three additional escape sequences to match generic character types are available when *UTF-8 mode* is selected. They are:

\p{xx}

a character with the xx property

\P{xx}

a character without the xx property

\X

an extended Unicode sequence

The property names represented by `xx` above are limited to the Unicode general category properties. Each character has exactly one such property, specified by a two-letter abbreviation. For compatibility with Perl, negation can be specified by including a circumflex between the opening brace and the property name. For example, `\p{^Lu}` is the same as `\P{Lu}`.

If only one letter is specified with `\p` or `\P`, it includes all the properties that start with that letter. In this case, in the absence of negation, the curly brackets in the escape sequence are optional; these two examples have the same effect:

```
\p{L}
\pL
```

Supported property codes

Property	Matches	Notes
`C`	Other	
`Cc`	Control	
`Cf`	Format	
`Cn`	Unassigned	
`Co`	Private use	
`Cs`	Surrogate	
`L`	Letter	Includes the following properties: `Ll`, `Lm`, `Lo`, `Lt` and `Lu`.
`Ll`	Lower case letter	
`Lm`	Modifier letter	
`Lo`	Other letter	
`Lt`	Title case letter	
`Lu`	Upper case letter	
`M`	Mark	
`Mc`	Spacing mark	
`Me`	Enclosing mark	
`Mn`	Non-spacing mark	
`N`	Number	
`Nd`	Decimal number	
`Nl`	Letter number	
`No`	Other number	
`P`	Punctuation	
`Pc`	Connector punctuation	
`Pd`	Dash punctuation	
`Pe`	Close punctuation	
`Pf`	Final punctuation	
`Pi`	Initial punctuation	
`Po`	Other punctuation	
`Ps`	Open punctuation	
`S`	Symbol	
`Sc`	Currency symbol	
`Sk`	Modifier symbol	
`Sm`	Mathematical symbol	
`So`	Other symbol	Includes emojis
`Z`	Separator	
`Zl`	Line separator	
`Zp`	Paragraph separator	
`Zs`	Space separator	

Extended properties such as `InMusicalSymbols` are not supported by PCRE.

Specifying case-insensitive (caseless) matching does not affect these escape sequences. For example, `\p{Lu}` always matches only upper case letters.

Sets of Unicode characters are defined as belonging to certain scripts. A character from one of these sets can be matched using a script name. For example:

- `\p{Greek}`

- `\P{Han}`

Those that are not part of an identified script are lumped together as `Common`. The current list of scripts is:

Supported scripts

Arabic	Armenian	Avestan	Balinese	Bamum
Batak	Bengali	Bopomofo	Brahmi	Braille
Buginese	Buhid	Canadian_Aboriginal	Carian	Chakma
Cham	Cherokee	Common	Coptic	Cuneiform
Cypriot	Cyrillic	Deseret	Devanagari	Egyptian_Hieroglyphs
Ethiopic	Georgian	Glagolitic	Gothic	Greek
Gujarati	Gurmukhi	Han	Hangul	Hanunoo
Hebrew	Hiragana	Imperial_Aramaic	Inherited	Inscriptional_Pahlavi
Inscriptional_Parthian	Javanese	Kaithi	Kannada	Katakana
Kayah_Li	Kharoshthi	Khmer	Lao	Latin
Lepcha	Limbu	Linear_B	Lisu	Lycian
Lydian	Malayalam	Mandaic	Meetei_Mayek	Meroitic_Cursive
Meroitic_Hieroglyphs	Miao	Mongolian	Myanmar	New_Tai_Lue
Nko	Ogham	Old_Italic	Old_Persian	Old_South_Arabian
Old_Turkic	Ol_Chiki	Oriya	Osmanya	Phags_Pa
Phoenician	Rejang	Runic	Samaritan	Saurashtra
Sharada	Shavian	Sinhala	Sora_Sompeng	Sundanese
Syloti_Nagri	Syriac	Tagalog	Tagbanwa	Tai_Le
Tai_Tham	Tai_Viet	Takri	Tamil	Telugu
Thaana	Thai	Tibetan	Tifinagh	Ugaritic
Vai	Yi			

The `\X` escape matches a Unicode extended grapheme cluster. An extended grapheme cluster is one or more Unicode characters that combine to form a single glyph. In effect, this can be thought of as the Unicode equivalent of `.` as it will match one composed character, regardless of how many individual characters are actually used to render it.

In versions of PCRE older than 8.32 (which corresponds to PHP versions before 5.4.14 when using the bundled PCRE library), `\X` is equivalent to `(?>\PM\pM*)`. That is, it matches a character without the "mark" property, followed by zero or more characters with the "mark" property, and treats the sequence as an atomic group (see Once-only subpatterns). Characters with the "mark" property are typically accents that affect the preceding character.

Matching characters by Unicode property is not fast, because PCRE has to search a structure that contains data for over fifteen thousand characters. That is why the traditional escape sequences such as `\d` and `\w` do not use Unicode properties in PCRE.

Anchors

Outside a character class, in the default matching mode, the circumflex character (^) is an assertion which is true only if the current matching point is at the start of the subject string. Inside a character class, circumflex (^) has an entirely different meaning (see below).

Circumflex (^) need not be the first character of the pattern if a number of alternatives are involved, but it should be the first thing in each alternative in which it appears if the pattern is ever to match that branch. If all possible alternatives start with a circumflex (^), that is, if the pattern is constrained to match only at the start of the subject, it is said to be an "anchored" pattern. (There are also other constructs that can cause a pattern to be anchored.)

A dollar character ($) is an assertion which is **`true`** only if the current matching point is at the end of the subject string, or immediately before a newline character that is the last character in the string (by default). Dollar ($) need not be the last character of the pattern if a number of alternatives are involved, but it should be the last item in any branch in which it appears. Dollar has no special meaning in a character class.

The meaning of dollar can be changed so that it matches only at the very end of the string, by setting the PCRE_DOLLAR_ENDONLY option at compile or matching time. This does not affect the \Z assertion.

The meanings of the circumflex and dollar characters are changed if the PCRE_MULTILINE option is set. When this is the case, they match immediately after and immediately before an internal "\n" character, respectively, in addition to matching at the start and end of the subject string. For example, the pattern /^abc$/ matches the subject string "def\nabc" in multiline mode, but not otherwise. Consequently, patterns that are anchored in single line mode because all branches start with "^" are not anchored in multiline mode. The PCRE_DOLLAR_ENDONLY option is ignored if PCRE_MULTILINE is set.

Note that the sequences \A, \Z, and \z can be used to match the start and end of the subject in both modes, and if all branches of a pattern start with \A is it always anchored, whether PCRE_MULTILINE is set or not.

Dot

Outside a character class, a dot in the pattern matches any one character in the subject, including a non-printing character, but not (by default) newline. If the PCRE_DOTALL option is set, then dots match newlines as well. The handling of dot is entirely independent of the handling of circumflex and dollar, the only relationship being that they both involve newline characters. Dot has no special meaning in a character class.

\C can be used to match single byte. It makes sense in *UTF-8 mode* where full stop matches the whole character which can consist of multiple bytes.

Character classes

An opening square bracket introduces a character class, terminated by a closing square bracket. A closing square bracket on its own is not special. If a closing square bracket is required as a member of the class, it should be the first data character in the class (after an initial circumflex, if present) or escaped with a backslash.

A character class matches a single character in the subject; the character must be in the set of characters defined by the class, unless the first character in the class is a circumflex, in which case the subject character must not be in the set defined by the class. If a circumflex is actually required as a member of the class, ensure it is not the first character, or escape it with a backslash.

For example, the character class [aeiou] matches any lower case vowel, while [^aeiou] matches any character that is not a lower case vowel. Note that a circumflex is just a convenient notation for specifying the characters which are in the class by enumerating those that are not. It is not an assertion: it still consumes a character from the subject string, and fails if the current pointer is at the end of the string.

When case-insensitive (caseless) matching is set, any letters in a class represent both their upper case and lower case versions, so for example, an insensitive [aeiou] matches "A" as well as "a", and an insensitive [^aeiou] does not match "A", whereas a sensitive (caseful) version would.

The newline character is never treated in any special way in character classes, whatever the setting of the PCRE_DOTALL or PCRE_MULTILINE options is. A class such as [^a] will always match a newline.

The minus (hyphen) character can be used to specify a range of characters in a character class. For example, [d-m] matches any letter between d and m, inclusive. If a minus character is required in a class, it must be escaped with a backslash or appear in a position where it cannot be interpreted as indicating a range, typically as the first or last character in the class.

It is not possible to have the literal character "]" as the end character of a range. A pattern such as [W-]46] is interpreted as a class of two characters ("W" and "-") followed by a literal string "46]", so it would match "W46]" or "-46]". However, if the "]" is escaped with a backslash it is interpreted as the end of range, so [W-\]46] is interpreted as a single class containing a range followed by two separate characters. The octal or hexadecimal representation of "]" can also be used to end a range.

Ranges operate in ASCII collating sequence. They can also be used for characters specified numerically, for example [\000-\037]. If a range that includes letters is used when case-insensitive (caseless) matching is set, it matches the letters in either case. For example, [W-c] is equivalent to [][\^_`wxyzabc], matched case-insensitively, and if character tables for the "fr" locale are in use, [\xc8-\xcb] matches accented E characters in both cases.

The character types \d, \D, \s, \S, \w, and \W may also appear in a character class, and add the characters that they match to the class. For example, [\dABCDEF] matches any hexadecimal digit. A circumflex can conveniently be used with the upper case character types to specify a more restricted set of characters than the matching lower case type. For example, the class [^\W_] matches any letter or digit, but not underscore.

All non-alphanumeric characters other than \, -, ^ (at the start) and the terminating] are non-special in character classes, but it does no harm if they are escaped. The pattern terminator is always special and must be escaped when used within an expression.

Perl supports the POSIX notation for character classes. This uses names enclosed by `[:` and `:]` within the enclosing square brackets. PCRE also supports this notation. For example, `[01[:alpha:]%]` matches "0", "1", any alphabetic character, or "%". The supported class names are:

Character classes

`alnum`	letters and digits
`alpha`	letters
`ascii`	character codes 0 - 127
`blank`	space or tab only
`cntrl`	control characters
`digit`	decimal digits (same as \d)
`graph`	printing characters, excluding space
`lower`	lower case letters
`print`	printing characters, including space
`punct`	printing characters, excluding letters and digits
`space`	white space (not quite the same as \s)
`upper`	upper case letters
`word`	"word" characters (same as \w)
`xdigit`	hexadecimal digits

The `space` characters are HT (9), LF (10), VT (11), FF (12), CR (13), and space (32). Notice that this list includes the VT character (code 11). This makes "space" different to `\s`, which does not include VT (for Perl compatibility).

The name `word` is a Perl extension, and `blank` is a GNU extension from Perl 5.8. Another Perl extension is negation, which is indicated by a `^` character after the colon. For example, `[12[:^digit:]]` matches "1", "2", or any non-digit.

In UTF-8 mode, characters with values greater than 128 do not match any of the POSIX character classes. As of libpcre 8.10 some character classes are changed to use Unicode character properties, in which case the mentioned restriction does not apply. Refer to the » PCRE(3) manual for details.

Unicode character properties can appear inside a character class. They can not be part of a range. The minus (hyphen) character after a Unicode character class will match literally. Trying to end a range with a Unicode character property will result in a warning.

Alternation

Vertical bar characters are used to separate alternative patterns. For example, the pattern `gilbert|sullivan` matches either "gilbert" or "sullivan". Any number of alternatives may appear, and an empty alternative is permitted (matching the empty string). The matching process tries each alternative in turn, from left to right, and the first one that succeeds is used. If the alternatives are within a subpattern (defined below), "succeeds" means matching the rest of the main pattern as well as the alternative in the subpattern.

It is possible to record which alternative was matched by using `(*MARK:NAME)` or `(*:NAME)`. Any number of such `(*MARK)` verbs may appear and their names do not have to be unique. When a match succeeds, the name of the last-encountered `(*MARK:NAME)` will be placed among matches as if it were a capture group called `MARK` so it can be read from the `matches` of preg_match() and will be passed to the `callback` of preg_replace_callback() etc.

Internal option setting

The settings of PCRE_CASELESS, PCRE_MULTILINE, PCRE_DOTALL, PCRE_UNGREEDY, PCRE_EXTRA, PCRE_EXTENDED and PCRE_DUPNAMES can be changed from within the pattern by a sequence of Perl option letters enclosed between "(?" and ")". The option letters are:

Internal option letters

`i`	for PCRE_CASELESS
`m`	for PCRE_MULTILINE
`s`	for PCRE_DOTALL
`x`	for PCRE_EXTENDED
`U`	for PCRE_UNGREEDY
`X`	for PCRE_EXTRA (no longer supported as of PHP 7.3.0)
`J`	for PCRE_INFO_JCHANGED

For example, (?im) sets case-insensitive (caseless), multiline matching. It is also possible to unset these options by preceding the letter with a hyphen, and a combined setting and unsetting such as (?im-sx), which sets PCRE_CASELESS and PCRE_MULTILINE while unsetting PCRE_DOTALL and PCRE_EXTENDED, is also permitted. If a letter appears both before and after the hyphen, the option is unset.

When an option change occurs at top level (that is, not inside subpattern parentheses), the change applies to the remainder of the pattern that follows. So `/ab(?i)c/` matches only "abc" and "abC".

If an option change occurs inside a subpattern, the effect is different. This is a change of behaviour in Perl 5.005. An option change inside a subpattern affects only that part of the subpattern that follows it, so `(a(?i)b)c` matches "abc" and "aBc" and no other strings (assuming PCRE_CASELESS is not used). By this means, options can be made to have different settings in different parts of the pattern. Any changes made in one alternative do carry on into subsequent branches within the same subpattern. For example, `(a(?i)b|c)` matches "ab", "aB", "c", and "C", even though when matching "C" the first branch is abandoned before the option setting. This is because the effects of option settings happen at compile time. There would be some very weird behaviour otherwise.

The PCRE-specific options PCRE_UNGREEDY and PCRE_EXTRA can be changed in the same way as the Perl-compatible options by using the characters U and X respectively. The (?X) flag setting is special in that it must always occur earlier in the pattern than any of the additional features it turns on, even when it is at top level. It is best put at the start.

Subpatterns

Subpatterns are delimited by parentheses (round brackets), which can be nested. Marking part of a pattern as a subpattern does two things:

1. It localizes a set of alternatives. For example, the pattern `cat(aract|erpillar|)` matches one of the words "cat", "cataract", or "caterpillar". Without the parentheses, it would match "cataract", "erpillar" or the empty string.

2. It sets up the subpattern as a capturing subpattern (as defined above). When the whole pattern matches, that portion of the subject string that matched the subpattern is passed back to the caller via the *ovector* argument of **pcre_exec()**. Opening parentheses are counted from left to right (starting from 1) to obtain the numbers of the capturing subpatterns.

For example, if the string "the red king" is matched against the pattern `the ((red|white) (king|queen))` the captured substrings are "red king", "red", and "king", and are numbered 1, 2, and 3.

The fact that plain parentheses fulfill two functions is not always helpful. There are often times when a grouping subpattern is required without a capturing requirement. If an opening parenthesis is followed by "?:", the subpattern does not do any capturing, and is not counted when computing the number of any subsequent capturing subpatterns. For example, if the string "the white queen" is matched against the pattern `the ((?:red|white) (king|queen))` the captured substrings are "white queen" and "queen", and are numbered 1 and 2. The maximum number of captured substrings is 65535. It may not be possible to compile such large patterns, however, depending on the configuration options of libpcre.

As a convenient shorthand, if any option settings are required at the start of a non-capturing subpattern, the option letters may appear between the "?" and the ":". Thus the two patterns

```
(?i:saturday|sunday)
(?:(?i)saturday|sunday)
```

match exactly the same set of strings. Because alternative branches are tried from left to right, and options are not reset until the end of the subpattern is reached, an option setting in one branch does affect subsequent branches, so the above patterns match "SUNDAY" as well as "Saturday".

It is possible to name a subpattern using the syntax `(?P<name>pattern)`. This subpattern will then be indexed in the matches array by its normal numeric position and also by name. There are two alternative syntaxes `(?<name>pattern)` and `(?'name'pattern)`.

As of PHP 8.4.0, which bundles PCRE2 10.44, the maximum length of a subpattern name is 128 characters; previously it was 32 characters.

Sometimes it is necessary to have multiple matching, but alternating subgroups in a regular expression. Normally, each of these would be given their own backreference number even though only one of them would ever possibly match. To overcome this, the `(?|` syntax allows having duplicate numbers. Consider the following regex matched against the string `Sunday`:

```
(?:(Sat)ur|(Sun))day
```

Here `Sun` is stored in backreference 2, while backreference 1 is empty. Matching `Saturday` yields `Sat` in backreference 1 while backreference 2 does not exist. Changing the pattern to use the `(?|` fixes this problem:

```
(?|(Sat)ur|(Sun))day
```

Using this pattern, both `Sun` and `Sat` would be stored in backreference 1.

Repetition

Repetition is specified by quantifiers, which can follow any of the following items:

- a single character, possibly escaped
- the . metacharacter
- a character class
- a back reference (see next section)
- a parenthesized subpattern (unless it is an assertion - see below)

The general repetition quantifier specifies a minimum and maximum number of permitted matches, by giving the two numbers in curly brackets (braces), separated by a comma. The numbers must be less than 65536, and the first must be less than or equal to the second. For example: `z{2,4}` matches "zz", "zzz", or "zzzz". A closing brace on its own is not a special character. If the second number is omitted, but the comma is present, there is no upper limit; if the second number and the comma are both omitted, the quantifier specifies an exact number of required matches. Thus `[aeiou]{3,}` matches at least 3 successive vowels, but may match many more, while `\d{8}` matches exactly 8 digits.

Prior to PHP 8.4.0, an opening curly bracket that appears in a position where a quantifier is not allowed, or one that does not match the syntax of a quantifier, is taken as a literal character. For example, `{,6}` is not a quantifier, but a literal string of four characters. As of PHP 8.4.0, the PCRE extension is bundled with PCRE2 version 10.44, which allows patterns such as `\d{,8}` and they are interpreted as `\d{0,8}`. Further, as of PHP 8.4.0, space characters around quantifiers such as `\d{0 , 8}` and `\d{ 0 , 8 }` are allowed.

The quantifier {0} is permitted, causing the expression to behave as if the previous item and the quantifier were not present.

For convenience (and historical compatibility) the three most common quantifiers have single-character abbreviations:

Single-character quantifiers	
`*`	equivalent to `{0,}`
`+`	equivalent to `{1,}`
`?`	equivalent to `{0,1}`

It is possible to construct infinite loops by following a subpattern that can match no characters with a quantifier that has no upper limit, for example: `(a?)*`

Earlier versions of Perl and PCRE used to give an error at compile time for such patterns. However, because there are cases where this can be useful, such patterns are now accepted, but if any repetition of the subpattern does in fact match no characters, the loop is forcibly broken.

By default, the quantifiers are "greedy", that is, they match as much as possible (up to the maximum number of permitted times), without causing the rest of the pattern to fail. The classic example of where this gives problems is in trying to match comments in C programs. These appear between the sequences /* and */ and within the sequence, individual * and / characters may appear. An attempt to match C comments by applying the pattern `/\*.*\*/` to the string `/* first comment */ not comment /* second comment */` fails, because it matches the entire string due to the greediness of the .* item.

However, if a quantifier is followed by a question mark, then it becomes lazy, and instead matches the minimum number of times possible, so the pattern `/\*.*?\*/` does the right thing with the C comments. The meaning of the various quantifiers is not otherwise changed, just the preferred number of matches. Do not confuse this use of question mark with its use as a quantifier in its own right. Because it has two uses, it can sometimes appear doubled, as in `\d??\d` which matches one digit by preference, but can match two if that is the only way the rest of the pattern matches.

If the PCRE_UNGREEDY option is set (an option which is not available in Perl) then the quantifiers are not greedy by default, but individual ones can be made greedy by following them with a question mark. In other words, it inverts the default behaviour.

Quantifiers followed by + are "possessive". They eat as many characters as possible and don't return to match the rest of the pattern. Thus `.*abc` matches "aabc" but `.*+abc` doesn't because `.*+` eats the whole string. Possessive quantifiers can be used to speed up processing.

When a parenthesized subpattern is quantified with a minimum repeat count that is greater than 1 or with a limited maximum, more store is required for the compiled pattern, in proportion to the size of the minimum or maximum.

If a pattern starts with .* or .{0,} and the PCRE_DOTALL option (equivalent to Perl's /s) is set, thus allowing the . to match newlines, then the pattern is implicitly anchored, because whatever follows will be tried against every character position in the subject string, so there is no point in retrying the overall match at any position after the first. PCRE treats such a pattern as though it were preceded by \A. In cases

where it is known that the subject string contains no newlines, it is worth setting PCRE_DOTALL when the pattern begins with .* in order to obtain this optimization, or alternatively using ^ to indicate anchoring explicitly.

When a capturing subpattern is repeated, the value captured is the substring that matched the final iteration. For example, after `(tweedle[dume]{3}\s*)+` has matched "tweedledum tweedledee" the value of the captured substring is "tweedledee". However, if there are nested capturing subpatterns, the corresponding captured values may have been set in previous iterations. For example, after `/(a|(b))+/` matches "aba" the value of the second captured substring is "b".

Back references

Outside a character class, a backslash followed by a digit greater than 0 (and possibly further digits) is a back reference to a capturing subpattern earlier (i.e. to its left) in the pattern, provided there have been that many previous capturing left parentheses.

However, if the decimal number following the backslash is less than 10, it is always taken as a back reference, and causes an error only if there are not that many capturing left parentheses in the entire pattern. In other words, the parentheses that are referenced need not be to the left of the reference for numbers less than 10. A "forward back reference" can make sense when a repetition is involved and the subpattern to the right has participated in an earlier iteration. See the section escape sequences for further details of the handling of digits following a backslash.

A back reference matches whatever actually matched the capturing subpattern in the current subject string, rather than anything matching the subpattern itself. So the pattern `(sens|respons)e and \1ibility` matches "sense and sensibility" and "response and responsibility", but not "sense and responsibility". If case-sensitive (caseful) matching is in force at the time of the back reference, then the case of letters is relevant. For example, `((?i)rah)\s+\1` matches "rah rah" and "RAH RAH", but not "RAH rah", even though the original capturing subpattern is matched case-insensitively (caselessly).

There may be more than one back reference to the same subpattern. If a subpattern has not actually been used in a particular match, then any back references to it always fail. For example, the pattern `(a|(bc))\2` always fails if it starts to match "a" rather than "bc". Because there may be up to 99 back references, all digits following the backslash are taken as part of a potential back reference number. If the pattern continues with a digit character, then some delimiter must be used to terminate the back reference. If the PCRE_EXTENDED option is set, this can be whitespace. Otherwise an empty comment can be used.

A back reference that occurs inside the parentheses to which it refers fails when the subpattern is first used, so, for example, (a\1) never matches. However, such references can be useful inside repeated subpatterns. For example, the pattern `(a|b\1)+` matches any number of "a"s and also "aba", "ababba" etc. At each iteration of the subpattern, the back reference matches the character string corresponding to the previous iteration. In order for this to work, the pattern must be such that the first iteration does not need to match the back reference. This can be done using alternation, as in the example above, or by a quantifier with a minimum of zero.

The `\g` escape sequence can be used for absolute and relative referencing of subpatterns. This escape sequence must be followed by an unsigned number or a negative number, optionally enclosed in braces. The sequences `\1`, `\g1` and `\g{1}` are synonymous with one another. The use of this pattern with an unsigned number can help remove the ambiguity inherent when using digits following a backslash. The sequence helps to distinguish back references from octal characters and also makes it easier to have a back reference followed by a literal number, e.g. `\g{2}1`.

The use of the `\g` sequence with a negative number signifies a relative reference. For example, `(foo)(bar)\g{-1}` would match the sequence "foobarbar" and `(foo)(bar)\g{-2}` matches "foobarfoo". This can be useful in long patterns as an alternative to keeping track of the number of subpatterns in order to reference a specific previous subpattern.

Back references to the named subpatterns can be achieved by `(?P=name)`, `\k<name>`, `\k'name'`, `\k{name}`, `\g{name}`, `\g<name>` or `\g'name'`.

Assertions

An assertion is a test on the characters following or preceding the current matching point that does not actually consume any characters. The simple assertions coded as \b, \B, \A, \Z, \z, ^ and $ are described in escape sequences. More complicated assertions are coded as subpatterns. There are two kinds: those that *look ahead* of the current position in the subject string, and those that *look behind* it.

An assertion subpattern is matched in the normal way, except that it does not cause the current matching position to be changed. *Lookahead* assertions start with (?= for positive assertions and (?! for negative assertions. For example, `\w+(?=;)` matches a word followed by a semicolon, but does not include the semicolon in the match, and `foo(?!bar)` matches any occurrence of "foo" that is not followed by "bar". Note that the apparently similar pattern `(?!foo)bar` does not find an occurrence of "bar" that is preceded by something other than "foo"; it finds any occurrence of "bar" whatsoever, because the assertion (?!foo) is always `true` when the next three characters are "bar". A lookbehind assertion is needed to achieve this effect.

Lookbehind assertions start with (?<= for positive assertions and (?<! for negative assertions. For example, `(?<!foo)bar` does find an occurrence of "bar" that is not preceded by "foo". A lookbehind assertion may match strings of variable length, up to an implementation-defined maximum length. Alternatives of differing lengths, such as `(?<!dogs?|cats?)` and top-level branches that can match more than one length, such as `(?<=ab(c|de))` are permitted. The implementation of lookbehind assertions is, for each alternative, to temporarily move the

current position back by the fixed width and then try to match. If there are insufficient characters before the current position, the match is deemed to fail. Lookbehinds in conjunction with once-only subpatterns can be particularly useful for matching at the ends of strings; an example is given at the end of the section on once-only subpatterns.

Prior to PHP 8.4.0, which bundles PCRE2 10.44, the contents of a lookbehind assertion were restricted such that all the strings it matched had to have a fixed length. If there were several alternatives, they did not all have to have the same fixed length, thus `(?<=bullock|donkey)` was permitted, but `(?<!dogs?|cats?)` caused an error at compile time. Branches that matched different length strings were permitted only at the top level of a lookbehind assertion. This was an extension compared with Perl 5.005, which required all branches to match the same length of string. An assertion such as `(?<=ab(c|de))` was not permitted, because its single top-level branch could match two different lengths, but it was acceptable if rewritten to use two top-level branches: `(?<=abc|abde)`

Several assertions (of any sort) may occur in succession. For example, `(?<=\d{3})(?<!999)foo` matches "foo" preceded by three digits that are not "999". Notice that each of the assertions is applied independently at the same point in the subject string. First there is a check that the previous three characters are all digits, then there is a check that the same three characters are not "999". This pattern does not match "foo" preceded by six characters, the first of which are digits and the last three of which are not "999". For example, it doesn't match "123abcfoo". A pattern to do that is `(?<=\d{3}...)(?<!999)foo`

This time the first assertion looks at the preceding six characters, checking that the first three are digits, and then the second assertion checks that the preceding three characters are not "999".

Assertions can be nested in any combination. For example, `(?<=(?<!foo)bar)baz` matches an occurrence of "baz" that is preceded by "bar" which in turn is not preceded by "foo", while `(?<=\d{3}...(?<!999))foo` is another pattern which matches "foo" preceded by three digits and any three characters that are not "999".

Assertion subpatterns are not capturing subpatterns, and may not be repeated, because it makes no sense to assert the same thing several times. If any kind of assertion contains capturing subpatterns within it, these are counted for the purposes of numbering the capturing subpatterns in the whole pattern. However, substring capturing is carried out only for positive assertions, because it does not make sense for negative assertions.

Assertions count towards the maximum of 200 parenthesized subpatterns.

Once-only subpatterns

Once-only subpatterns are also known as *atomic groups*.

With both maximizing and minimizing repetition, failure of what follows normally causes the repeated item to be re-evaluated to see if a different number of repeats allows the rest of the pattern to match. Sometimes it is useful to prevent this, either to change the nature of the match, or to cause it fail earlier than it otherwise might, when the author of the pattern knows there is no point in carrying on.

Consider, for example, the pattern \d+foo when applied to the subject line `123456bar`

After matching all 6 digits and then failing to match "foo", the normal action of the matcher is to try again with only 5 digits matching the \d+ item, and then with 4, and so on, before ultimately failing. Once-only subpatterns provide the means for specifying that once a portion of the pattern has matched, it is not to be re-evaluated in this way, so the matcher would give up immediately on failing to match "foo" the first time. The notation is another kind of special parenthesis, starting with (?> as in this example: `(?>\d+)bar`

This kind of parenthesis "locks up" the part of the pattern it contains once it has matched, and a failure further into the pattern is prevented from backtracking into it. Backtracking past it to previous items, however, works as normal.

An alternative description is that a subpattern of this type matches the string of characters that an identical standalone pattern would match, if anchored at the current point in the subject string.

Once-only subpatterns are not capturing subpatterns. Simple cases such as the above example can be thought of as a maximizing repeat that must swallow everything it can. So, while both \d+ and \d+? are prepared to adjust the number of digits they match in order to make the rest of the pattern match, (?>\d+) can only match an entire sequence of digits.

This construction can of course contain arbitrarily complicated subpatterns, and it can be nested.

Once-only subpatterns can be used in conjunction with lookbehind assertions to specify efficient matching at the end of the subject string. Consider a simple pattern such as `abcd$` when applied to a long string which does not match. Because matching proceeds from left to right, PCRE will look for each "a" in the subject and then see if what follows matches the rest of the pattern. If the pattern is specified as `^.*abcd$` then the initial .* matches the entire string at first, but when this fails (because there is no following "a"), it backtracks to match all but the last character, then all but the last two characters, and so on. Once again the search for "a" covers the entire string, from right to left, so we are no better off. However, if the pattern is written as `^(?>.*)(?<=abcd)` then there can be no backtracking for the .* item; it can match only the entire string. The subsequent lookbehind assertion does a single test on the last four characters. If it fails, the match fails immediately. For long strings, this approach makes a significant difference to the processing time.

When a pattern contains an unlimited repeat inside a subpattern that can itself be repeated an unlimited number of times, the use of a once-only subpattern is the only way to avoid some failing matches taking a very long time indeed. The pattern `(\D+|<\d+>)*[!?]` matches an unlimited number of substrings that either consist of non-digits, or digits enclosed in <>, followed by either ! or ?. When it matches, it runs quickly. However, if it is applied to `aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa` it takes a long time before reporting failure. This is because the string can be divided between the two repeats in a large number of ways, and all have to be tried. (The example used [!?] rather than a single character at the end, because both PCRE and Perl have an optimization that allows for fast failure when a single character is used. They remember the last single character that is required for a match, and fail early if it is not present in the string.) If the pattern is changed to `((?>\D+)|<\d+>)*[!?]` sequences of non-digits cannot be broken, and failure happens quickly.

Conditional subpatterns

It is possible to cause the matching process to obey a subpattern conditionally or to choose between two alternative subpatterns, depending on the result of an assertion, or whether a previous capturing subpattern matched or not. The two possible forms of conditional subpattern are

```
(?(condition)yes-pattern)
(?(condition)yes-pattern|no-pattern)
```

If the condition is satisfied, the yes-pattern is used; otherwise the no-pattern (if present) is used. If there are more than two alternatives in the subpattern, a compile-time error occurs.

There are two kinds of condition. If the text between the parentheses consists of a sequence of digits, then the condition is satisfied if the capturing subpattern of that number has previously matched. Consider the following pattern, which contains non-significant white space to make it more readable (assume the PCRE_EXTENDED option) and to divide it into three parts for ease of discussion:

```
( \( )? [^()]+ (?(1) \) )
```

The first part matches an optional opening parenthesis, and if that character is present, sets it as the first captured substring. The second part matches one or more characters that are not parentheses. The third part is a conditional subpattern that tests whether the first set of parentheses matched or not. If they did, that is, if subject started with an opening parenthesis, the condition is `true`, and so the yes-pattern is executed and a closing parenthesis is required. Otherwise, since no-pattern is not present, the subpattern matches nothing. In other words, this pattern matches a sequence of non-parentheses, optionally enclosed in parentheses.

If the condition is the string `(R)`, it is satisfied if a recursive call to the pattern or subpattern has been made. At "top level", the condition is false.

If the condition is not a sequence of digits or (R), it must be an assertion. This may be a positive or negative lookahead or lookbehind assertion. Consider this pattern, again containing non-significant white space, and with the two alternatives on the second line:

```
(?(?=[^a-z]*[a-z])
\d{2}-[a-z]{3}-\d{2} | \d{2}-\d{2}-\d{2} )
```

The condition is a positive lookahead assertion that matches an optional sequence of non-letters followed by a letter. In other words, it tests for the presence of at least one letter in the subject. If a letter is found, the subject is matched against the first alternative; otherwise it is matched against the second. This pattern matches strings in one of the two forms dd-aaa-dd or dd-dd-dd, where aaa are letters and dd are digits.

Comments

The sequence (?# marks the start of a comment which continues up to the next closing parenthesis. Nested parentheses are not permitted. The characters that make up a comment play no part in the pattern matching at all.

If the PCRE_EXTENDED option is set, an unescaped # character outside a character class introduces a comment that continues up to the next newline character in the pattern.

Example #1 Usage of comments in PCRE pattern

```php
<?php

$subject = 'test';

/* (?# can be used to add comments without enabling PCRE_EXTENDED */
$match = preg_match('/te(?# this is a comment)st/', $subject);
var_dump($match);

/* Whitespace and # is treated as part of the pattern unless PCRE_EXTENDED is enabled */
```

```php
$match = preg_match('/te   #~~~~
st/', $subject);
var_dump($match);

/* When PCRE_EXTENDED is enabled, all whitespace data characters and anything
   that follows an unescaped # on the same line is ignored */
$match = preg_match('/te    #~~~~
st/x', $subject);
var_dump($match);
```

The above example will output:

```
int(1)
int(0)
int(1)
```

Recursive patterns

Consider the problem of matching a string in parentheses, allowing for unlimited nested parentheses. Without the use of recursion, the best that can be done is to use a pattern that matches up to some fixed depth of nesting. It is not possible to handle an arbitrary nesting depth. Perl 5.6 has provided an experimental facility that allows regular expressions to recurse (among other things). The special item (?R) is provided for the specific case of recursion. This PCRE pattern solves the parentheses problem (assume the PCRE_EXTENDED option is set so that white space is ignored): `\( ( (?>[^()]+) | (?R) )* \)`

First it matches an opening parenthesis. Then it matches any number of substrings which can either be a sequence of non-parentheses, or a recursive match of the pattern itself (i.e. a correctly parenthesized substring). Finally there is a closing parenthesis.

This particular example pattern contains nested unlimited repeats, and so the use of a once-only subpattern for matching strings of non-parentheses is important when applying the pattern to strings that do not match. For example, when it is applied to `(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa()` it yields "no match" quickly. However, if a once-only subpattern is not used, the match runs for a very long time indeed because there are so many different ways the + and * repeats can carve up the subject, and all have to be tested before failure can be reported.

The values set for any capturing subpatterns are those from the outermost level of the recursion at which the subpattern value is set. If the pattern above is matched against `(ab(cd)ef)` the value for the capturing parentheses is "ef", which is the last value taken on at the top level. If additional parentheses are added, giving `\( ( ( (?>[^()]+) | (?R) )* ) \)` then the string they capture is "ab(cd)ef", the contents of the top level parentheses. If there are more than 15 capturing parentheses in a pattern, PCRE has to obtain extra memory to store data during a recursion, which it does by using pcre_malloc, freeing it via pcre_free afterwards. If no memory can be obtained, it saves data for the first 15 capturing parentheses only, as there is no way to give an out-of-memory error from within a recursion.

`(?1)`, `(?2)` and so on can be used for recursive subpatterns too. It is also possible to use named subpatterns: `(?P>name)` or `(?&name)`.

If the syntax for a recursive subpattern reference (either by number or by name) is used outside the parentheses to which it refers, it operates like a subroutine in a programming language. An earlier example pointed out that the pattern `(sens|respons)e and \1ibility` matches "sense and sensibility" and "response and responsibility", but not "sense and responsibility". If instead the pattern `(sens|respons)e and (?1)ibility` is used, it does match "sense and responsibility" as well as the other two strings. Such references must, however, follow the subpattern to which they refer.

The maximum length of a subject string is the largest positive number that an integer variable can hold. However, PCRE uses recursion to handle subpatterns and indefinite repetition. This means that the available stack space may limit the size of a subject string that can be processed by certain patterns.

Performance

Certain items that may appear in patterns are more efficient than others. It is more efficient to use a character class like [aeiou] than a set of alternatives such as (a|e|i|o|u). In general, the simplest construction that provides the required behaviour is usually the most efficient. Jeffrey Friedl's book contains a lot of discussion about optimizing regular expressions for efficient performance.

When a pattern begins with .* and the PCRE_DOTALL option is set, the pattern is implicitly anchored by PCRE, since it can match only at the start of a subject string. However, if PCRE_DOTALL is not set, PCRE cannot make this optimization, because the . metacharacter does not then match a newline, and if the subject string contains newlines, the pattern may match from the character immediately following one of them instead of from the very start. For example, the pattern `(.*) second` matches the subject "first\nand second" (where \n stands for a

newline character) with the first captured substring being "and". In order to do this, PCRE has to retry the match starting after every newline in the subject.

If you are using such a pattern with subject strings that do not contain newlines, the best performance is obtained by setting PCRE_DOTALL, or starting the pattern with ^.* to indicate explicit anchoring. That saves PCRE from having to scan along the subject looking for a newline to restart at.

Beware of patterns that contain nested indefinite repeats. These can take a long time to run when applied to a string that does not match. Consider the pattern fragment `(a+)*`

This can match "aaaa" in 33 different ways, and this number increases very rapidly as the string gets longer. (The * repeat can match 0, 1, 2, 3, or 4 times, and for each of those cases other than 0, the + repeats can match different numbers of times.) When the remainder of the pattern is such that the entire match is going to fail, PCRE has in principle to try every possible variation, and this can take an extremely long time.

An optimization catches some of the more simple cases such as `(a+)*b` where a literal character follows. Before embarking on the standard matching procedure, PCRE checks that there is a "b" later in the subject string, and if there is not, it fails the match immediately. However, when there is no following literal this optimization cannot be used. You can see the difference by comparing the behaviour of `(a+)*\d` with the pattern above. The former gives a failure almost instantly when applied to a whole line of "a" characters, whereas the latter takes an appreciable time with strings longer than about 20 characters.

Possible modifiers in regex patterns

The current possible PCRE modifiers are listed below. The names in parentheses refer to internal PCRE names for these modifiers. Spaces and newlines are ignored in modifiers, other characters cause error.

> *i* (PCRE_CASELESS)
>
> If this modifier is set, letters in the pattern match both upper and lower case letters.
>
> *m* (PCRE_MULTILINE)
>
> By default, PCRE treats the subject string as consisting of a single "line" of characters (even if it actually contains several newlines). The "start of line" metacharacter (^) matches only at the start of the string, while the "end of line" metacharacter ($) matches only at the end of the string, or before a terminating newline (unless *D* modifier is set). This is the same as Perl.
>
> When this modifier is set, the "start of line" and "end of line" constructs match immediately following or immediately before any newline in the subject string, respectively, as well as at the very start and end. This is equivalent to Perl's /m modifier. If there are no "\n" characters in a subject string, or no occurrences of ^ or $ in a pattern, setting this modifier has no effect.
>
> *s* (PCRE_DOTALL)
>
> If this modifier is set, a dot metacharacter in the pattern matches all characters, including newlines. Without it, newlines are excluded. This modifier is equivalent to Perl's /s modifier. A negative class such as [^a] always matches a newline character, independent of the setting of this modifier.
>
> *x* (PCRE_EXTENDED)
>
> If this modifier is set, whitespace data characters in the pattern are totally ignored except when escaped or inside a character class, and characters between an unescaped # outside a character class and the next newline character, inclusive, are also ignored. This is equivalent to Perl's /x modifier, and makes it possible to include commentary inside complicated patterns. Note, however, that this applies only to data characters. Whitespace characters may never appear within special character sequences in a pattern, for example within the sequence (?(which introduces a conditional subpattern.
>
> *A* (PCRE_ANCHORED)
>
> If this modifier is set, the pattern is forced to be "anchored", that is, it is constrained to match only at the start of the string which is being searched (the "subject string"). This effect can also be achieved by appropriate constructs in the pattern itself, which is the only way to do it in Perl.
>
> *D* (PCRE_DOLLAR_ENDONLY)
>
> If this modifier is set, a dollar metacharacter in the pattern matches only at the end of the subject string. Without this modifier, a dollar also matches immediately before the final character if it is a newline (but not before any other newlines). This modifier is ignored if *m* modifier is set. There is no equivalent to this modifier in Perl.
>
> *S*

When a pattern is going to be used several times, it is worth spending more time analyzing it in order to speed up the time taken for matching. If this modifier is set, then this extra analysis is performed. At present, studying a pattern is useful only for non-anchored patterns that do not have a single fixed starting character. As of PHP 7.3.0 this flag has no effect.

U (PCRE_UNGREEDY)

This modifier inverts the "greediness" of the quantifiers so that they are not greedy by default, but become greedy if followed by ?. It is not compatible with Perl. It can also be set by a (?U) modifier setting within the pattern or by a question mark behind a quantifier (e.g. .*?).

> **Note** :
>
> It is usually not possible to match more than pcre.backtrack_limit characters in ungreedy mode.

X (PCRE_EXTRA)

This modifier turns on additional functionality of PCRE that is incompatible with Perl. Any backslash in a pattern that is followed by a letter that has no special meaning causes an error, thus reserving these combinations for future expansion. By default, as in Perl, a backslash followed by a letter with no special meaning is treated as a literal. There are at present no other features controlled by this modifier.

J (PCRE_INFO_JCHANGED)

The (?J) internal option setting changes the local PCRE_DUPNAMES option. Allow duplicate names for subpatterns. As of PHP 7.2.0 J is supported as modifier as well.

u (PCRE_UTF8)

This modifier turns on additional functionality of PCRE that is incompatible with Perl. Pattern and subject strings are treated as UTF-8. An invalid subject will cause the preg_* function to match nothing; an invalid pattern will trigger an error of level E_WARNING. Five and six octet UTF-8 sequences are regarded as invalid.

n (PCRE_NO_AUTO_CAPTURE)

This modifier makes simple (xyz) groups non-capturing. Only named groups like (?<name>xyz) are capturing. This only affects which groups are capturing, it is still possible to use numbered subpattern references, and the matches array will still contain numbered results. Available as of PHP 8.2.0.

r (PCRE2_EXTRA_CASELESS_RESTRICT)

When *u* (PCRE_UTF8) and *i* (PCRE_CASELESS) are in effect, this modifier prevents matching across ASCII and non-ASCII characters.

For example, preg_match('/\x{212A}/iu', "K") matches the Kelvin sign K (U+212A). When *r* is used (preg_match('/\x{212A}/iur', "K")), it does not match.

Available as of PHP 8.4.0.

Differences From Perl

Both Perl and PCRE2 change continually, so the exact set of differences depends on the version of the PCRE2 library in use. Refer to the » PCRE2 documentation on the differences between PCRE2 and Perl for an up-to-date list.

PCRE Functions

Table of Contents

preg_filter

preg_filter

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

preg_filter — Perform a regular expression search and replace

Description

```
function preg_filter(
    string|array $pattern,
    string|array $replacement,
    string|array $subject,
    int $limit = -1,
    int &$count = null
): string|array|null
```

preg_filter() is identical to preg_replace() except it only returns the (possibly transformed) subjects where there was a match. For details about how this function works, read the preg_replace() documentation.

Parameters

Parameters are described in the documentation for preg_replace().

Return Values

Returns an array if the `subject` parameter is an array, or a string otherwise.

If no matches are found or an error occurred, an empty array is returned when `subject` is an array or `null` otherwise.

Errors/Exceptions

If the regex pattern passed does not compile to a valid regex, an `E_WARNING` is emitted.

Examples

Example #1 Example comparing preg_filter() with preg_replace()

```php
<?php
$subject = array('1', 'a', '2', 'b', '3', 'A', 'B', '4');
$pattern = array('/\d/', '/[a-z]/', '/[1a]/');
$replace = array('A:$0', 'B:$0', 'C:$0');

echo "preg_filter returns\n";
print_r(preg_filter($pattern, $replace, $subject));

echo "preg_replace returns\n";
print_r(preg_replace($pattern, $replace, $subject));
?>
```

The above example will output:

```
preg_filter returns
Array
(
    [0] => A:C:1
    [1] => B:C:a
    [2] => A:2
    [3] => B:b
    [4] => A:3
    [7] => A:4
)
preg_replace returns
Array
(
    [0] => A:C:1
    [1] => B:C:a
    [2] => A:2
    [3] => B:b
    [4] => A:3
    [5] => A
    [6] => B
    [7] => A:4
)
```

See Also

- PCRE Patterns
- preg_quote() - Quote regular expression characters
- preg_replace() - Perform a regular expression search and replace
- preg_replace_callback() - Perform a regular expression search and replace using a callback
- preg_grep() - Return array entries that match the pattern
- preg_last_error() - Returns the error code of the last PCRE regex execution

preg_grep

preg_grep

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_grep — Return array entries that match the pattern

Description

function **preg_grep**(string `$pattern`, array `$array`, int `$flags` = 0): array|false

Returns the array consisting of the elements of the `array` array that match the given `pattern`.

Parameters

`pattern`

The pattern to search for, as a string.

`array`

The input array.

`flags`

If set to **`PREG_GREP_INVERT`**, this function returns the elements of the input array that do *not* match the given `pattern`.

Return Values

Returns an array indexed using the keys from the `array` array, or **`false`** on failure.

Errors/Exceptions

If the regex pattern passed does not compile to a valid regex, an **`E_WARNING`** is emitted.

Examples

Example #1 preg_grep() example

```
<?php
$array = [ "4", M_PI, "2.74", 42 ];

// return all array elements containing floating point numbers
$fl_array = preg_grep("/^(\d+)?\.\d+$/", $array);

var_dump($fl_array);
?>
```

See Also

- PCRE Patterns
- preg_quote() - Quote regular expression characters
- preg_match_all() - Perform a global regular expression match
- preg_filter() - Perform a regular expression search and replace
- preg_last_error() - Returns the error code of the last PCRE regex execution

preg_last_error

preg_last_error

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

preg_last_error — Returns the error code of the last PCRE regex execution

Description

function **preg_last_error**(): int

Returns the error code of the last PCRE regex execution.

Example #1 preg_last_error() example

```php
<?php

preg_match('/(?:\D+|<\d+>)*[!?]/', 'foobar foobar foobar');

if (preg_last_error() == PREG_BACKTRACK_LIMIT_ERROR) {
    echo 'Backtrack limit was exhausted!';
}

?>
```

The above example will output:

```
Backtrack limit was exhausted!
```

Parameters

This function has no parameters.

Return Values

Returns one of the following constants (explained on their own page):

- `PREG_NO_ERROR`
- `PREG_INTERNAL_ERROR`
- `PREG_BACKTRACK_LIMIT_ERROR` (see also pcre.backtrack_limit)
- `PREG_RECURSION_LIMIT_ERROR` (see also pcre.recursion_limit)
- `PREG_BAD_UTF8_ERROR`
- `PREG_BAD_UTF8_OFFSET_ERROR`
- `PREG_JIT_STACKLIMIT_ERROR`

See Also

- preg_last_error_msg() - Returns the error message of the last PCRE regex execution

preg_last_error_msg

preg_last_error_msg

(PHP 8)

preg_last_error_msg — Returns the error message of the last PCRE regex execution

Description

function **preg_last_error_msg**(): string

Returns the error message of the last PCRE regex execution.

Parameters

This function has no parameters.

Return Values

Returns the error message on success, or `"No error"` if no error has occurred.

Examples

Example #1 preg_last_error_msg() example

```php
<?php

preg_match('/(?:\D+|<\d+>)*[!?]/', 'foobar foobar foobar');

if (preg_last_error() !== PREG_NO_ERROR) {
    echo preg_last_error_msg();
}

?>
```

The above example will output:

```
Backtrack limit exhausted
```

See Also

- preg_last_error() - Returns the error code of the last PCRE regex execution

preg_match

preg_match

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_match — Perform a regular expression match

Description

```
function preg_match(
    string $pattern,
    string $subject,
    array &$matches = null,
    int $flags = 0,
    int $offset = 0
): int|false
```

Searches `subject` for a match to the regular expression given in `pattern`.

Parameters

`pattern`

The pattern to search for, as a string.

`subject`

The input string.

`matches`

If `matches` is provided, then it is filled with the results of search. *$matches[0]* will contain the text that matched the full pattern, *$matches[1]* will have the text that matched the first captured parenthesized subpattern, and so on.

`flags`

`flags` can be a combination of the following flags:

PREG_OFFSET_CAPTURE

If this flag is passed, for every occurring match the appendant string offset (in bytes) will also be returned. Note that this changes the value of `matches` into an array where every element is an array consisting of the matched string at offset `0` and its string offset into `subject` at offset `1`.

```php
<?php
preg_match('/(foo)(bar)(baz)/', 'foobarbaz', $matches, PREG_OFFSET_CAPTURE);
print_r($matches);
?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [0] => foobarbaz
            [1] => 0
        )

    [1] => Array
        (
            [0] => foo
            [1] => 0
        )

    [2] => Array
        (
            [0] => bar
            [1] => 3
        )

    [3] => Array
        (
            [0] => baz
            [1] => 6
        )

)
```

PREG_UNMATCHED_AS_NULL

If this flag is passed, unmatched subpatterns are reported as **null** and are always included in the results (including trailing ones). Without this flag, unmatched subpatterns that are followed by a matched subpattern are reported as an empty string, while trailing unmatched subpatterns are omitted from the results entirely.

```php
<?php
preg_match('/(a)(b)*(c)/', 'ac', $matches);
var_dump($matches);
preg_match('/(a)(b)*(c)/', 'ac', $matches, PREG_UNMATCHED_AS_NULL);
var_dump($matches);

// Trailing unmatched subpatterns:
preg_match('/(a)(b)?(c)?/', 'a', $matches);
var_dump($matches);
preg_match('/(a)(b)?(c)?/', 'a', $matches, PREG_UNMATCHED_AS_NULL);
var_dump($matches);
?>
```

The above example will output:

```
array(4) {
  [0]=>
  string(2) "ac"
  [1]=>
  string(1) "a"
  [2]=>
  string(0) ""
  [3]=>
  string(1) "c"
}
array(4) {
  [0]=>
  string(2) "ac"
  [1]=>
  string(1) "a"
  [2]=>
  NULL
  [3]=>
  string(1) "c"
}
array(2) {
  [0]=>
  string(1) "a"
  [1]=>
  string(1) "a"
}
array(4) {
  [0]=>
  string(1) "a"
  [1]=>
  string(1) "a"
  [2]=>
  NULL
  [3]=>
  NULL
}
```

`offset`

Normally, the search starts from the beginning of the subject string. The optional parameter `offset` can be used to specify the alternate place from which to start the search (in bytes).

> **Note** :
>
> Using `offset` is not equivalent to passing `substr($subject, $offset)` to **preg_match()** in place of the subject string, because `pattern` can contain assertions such as ^, *$* or *(?<=x)*. Compare:
>
> ```php
> <?php
> $subject = "abcdef";
> $pattern = '/^def/';
> preg_match($pattern, $subject, $matches, PREG_OFFSET_CAPTURE, 3);
> print_r($matches);
> ?>
> ```
>
> The above example will output:
>
> ```
> Array
> (
> )
> ```

while this example

```
<?php
$subject = "abcdef";
$pattern = '/^def/';
preg_match($pattern, substr($subject,3), $matches, PREG_OFFSET_CAPTURE);
print_r($matches);
?>
```

will produce

```
Array
(
    [0] => Array
        (
            [0] => def
            [1] => 0
        )

)
```

Alternatively, to avoid using substr(), use the `\G` assertion rather than the `^` anchor, or the `A` modifier; both of which do not work with the `offset` parameter.

Return Values

preg_match() returns 1 if the `pattern` matches given `subject`, 0 if it does not, or **`false`** on failure.

> **Warning**
>
> This function may return Boolean **`false`**, but may also return a non-Boolean value which evaluates to **`false`**. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Errors/Exceptions

If the regex pattern passed does not compile to a valid regex, an **`E_WARNING`** is emitted.

Changelog

Version	Description
7.4.0	When the **`PREG_UNMATCHED_AS_NULL`** flag is used, trailing unmatched capturing groups are now also included in the result with the value **`null`** (or `[null, -1]` when **`PREG_OFFSET_CAPTURE`** is also used), so that *$matches* always has the same size. Previously, they were omitted.
7.2.0	The **`PREG_UNMATCHED_AS_NULL`** is now supported for the `flags` parameter.

Examples

Example #1 Find the string of text "php"

```
<?php
// The "i" after the pattern delimiter indicates a case-insensitive search
if (preg_match("/php/i", "PHP is the web scripting language of choice.")) {
    echo "A match was found.";
} else {
    echo "A match was not found.";
}
?>
```

Example #2 Find the word "web"

```php
<?php
/* The \b in the pattern indicates a word boundary, so only the distinct
 * word "web" is matched, and not a word partial like "webbing" or "cobweb" */
if (preg_match("/\bweb\b/i", "PHP is the web scripting language of choice.")) {
    echo "A match was found.";
} else {
    echo "A match was not found.";
}

echo "\n";

if (preg_match("/\bweb\b/i", "PHP is the website scripting language of choice.")) {
    echo "A match was found.";
} else {
    echo "A match was not found.";
}
?>
```

Example #3 Getting the domain name out of a URL

```php
<?php
// get host name from URL
preg_match('@^(?:http://)?([^/]+)@i',
    "http://www.php.net/index.html", $matches);
$host = $matches[1];

// get last two segments of host name
preg_match('/[^.]+\.[^.]+$/', $host, $matches);
echo "domain name is: {$matches[0]}\n";
?>
```

The above example will output:

```
domain name is: php.net
```

Example #4 Using named subpattern

```php
<?php

$str = 'foobar: 2008';

preg_match('/(?P<name>\w+): (?P<digit>\d+)/', $str, $matches);

/* Alternative */
// preg_match('/(?<name>\w+): (?<digit>\d+)/', $str, $matches);

print_r($matches);

?>
```

The above example will output:

```
Array
(
    [0] => foobar: 2008
    [name] => foobar
    [1] => foobar
    [digit] => 2008
    [2] => 2008
)
```

Notes

> **Tip**
>
> Do not use **preg_match()** if you only want to check if one string is contained in another string. Use strpos() instead as it will be faster.

See Also

- PCRE Patterns
- preg_quote() - Quote regular expression characters
- preg_match_all() - Perform a global regular expression match
- preg_replace() - Perform a regular expression search and replace
- preg_split() - Split string by a regular expression
- preg_last_error() - Returns the error code of the last PCRE regex execution
- preg_last_error_msg() - Returns the error message of the last PCRE regex execution

preg_match_all

preg_match_all

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_match_all — Perform a global regular expression match

Description

```
function preg_match_all(
    string $pattern,
    string $subject,
    array &$matches = null,
    int $flags = 0,
    int $offset = 0
): int|false
```

Searches `subject` for all matches to the regular expression given in `pattern` and puts them in `matches` in the order specified by `flags`.

After the first match is found, the subsequent searches are continued on from end of the last match.

Parameters

`pattern`

The pattern to search for, as a string.

`subject`

The input string.

`matches`

Array of all matches in multi-dimensional array ordered according to `flags`.

`flags`

If no order flag is given, `PREG_PATTERN_ORDER` is assumed.

Can be a combination of the following flags (note that it doesn't make sense to use `PREG_PATTERN_ORDER` together with `PREG_SET_ORDER`):

`PREG_PATTERN_ORDER`

Orders results so that *$matches[0]* is an array of full pattern matches, *$matches[1]* is an array of strings matched by the first parenthesized subpattern, and so on.

```php
<?php
preg_match_all("|<[^>]+>(.*)</[^>]+>|U",
    "<b>example: </b><div align=left>this is a test</div>",
    $out, PREG_PATTERN_ORDER);
echo $out[0][0] . ", " . $out[0][1] . "\n";
echo $out[1][0] . ", " . $out[1][1] . "\n";
?>
```

The above example will output:

```
<b>example: </b>, <div align=left>this is a test</div>
example: , this is a test
```

So, *$out[0]* contains an array of strings that matched the full pattern, and *$out[1]* contains an array of strings enclosed by tags.

If the pattern contains named subpatterns, *$matches* additionally contains entries for keys with the subpattern name.

If the pattern contains duplicate named subpatterns, only the rightmost subpattern is stored in *$matches[NAME]*.

```php
<?php
preg_match_all(
    '/(?J)(?<match>foo)|(?<match>bar)/',
    'foo bar',
    $matches,
    PREG_PATTERN_ORDER
);
print_r($matches['match']);
?>
```

The above example will output:

```
Array
(
    [0] =>
    [1] => bar
)
```

PREG_SET_ORDER

Orders results so that *$matches[0]* is an array of first set of matches, *$matches[1]* is an array of second set of matches, and so on.

```php
<?php
preg_match_all("|<[^>]+>(.*)</[^>]+>|U",
    "<b>example: </b><div align=\"left\">this is a test</div>",
    $out, PREG_SET_ORDER);
echo $out[0][0] . ", " . $out[0][1] . "\n";
echo $out[1][0] . ", " . $out[1][1] . "\n";
?>
```

The above example will output:

```
<b>example: </b>, example:
<div align="left">this is a test</div>, this is a test
```

PREG_OFFSET_CAPTURE

If this flag is passed, for every occurring match the appendant string offset (in bytes) will also be returned. Note that this changes the value of `matches` into an array of arrays where every element is an array consisting of the matched string at offset `0` and its string offset into `subject` at offset `1`.

```
<?php
preg_match_all('/(foo)(bar)(baz)/', 'foobarbaz', $matches, PREG_OFFSET_CAPTURE);
print_r($matches);
?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [0] => Array
                (
                    [0] => foobarbaz
                    [1] => 0
                )

        )

    [1] => Array
        (
            [0] => Array
                (
                    [0] => foo
                    [1] => 0
                )

        )

    [2] => Array
        (
            [0] => Array
                (
                    [0] => bar
                    [1] => 3
                )

        )

    [3] => Array
        (
            [0] => Array
                (
                    [0] => baz
                    [1] => 6
                )

        )

)
```

PREG_UNMATCHED_AS_NULL

If this flag is passed, unmatched subpatterns are reported as **null**; otherwise they are reported as an empty string.

`offset`

Normally, the search starts from the beginning of the subject string. The optional parameter `offset` can be used to specify the alternate place from which to start the search (in bytes).

> **Note** :
>
> Using `offset` is not equivalent to passing `substr($subject, $offset)` to **preg_match_all()** in place of the subject string, because `pattern` can contain assertions such as ^, *$* or *(?<=x)*. See preg_match() for examples.

Return Values

Returns the number of full pattern matches (which might be zero), or `false` on failure.

Errors/Exceptions

If the regex pattern passed does not compile to a valid regex, an `E_WARNING` is emitted.

Changelog

Version	Description
7.2.0	The `PREG_UNMATCHED_AS_NULL` is now supported for the `flags` parameter.

Examples

Example #1 Getting all phone numbers out of some text.

```
<?php
preg_match_all("/\(?  (\d{3})?  \)?  (?(1)  [\-\s] ) \d{3}-\d{4}/x",
               "Call 555-1212 or 1-800-555-1212", $phones);
?>
```

Example #2 Find matching HTML tags (greedy)

```
<?php
// The \\2 is an example of backreferencing. This tells pcre that
// it must match the second set of parentheses in the regular expression
// itself, which would be the ([\w]+) in this case. The extra backslash is
// required because the string is in double quotes.
$html = "<b>bold text</b><a href=howdy.html>click me</a>";

preg_match_all("/(<([\w]+)[^>]*>)(.*?)(<\/\\2>)/", $html, $matches, PREG_SET_ORDER);

foreach ($matches as $val) {
    echo "matched: " . $val[0] . "\n";
    echo "part 1: " . $val[1] . "\n";
    echo "part 2: " . $val[2] . "\n";
    echo "part 3: " . $val[3] . "\n";
    echo "part 4: " . $val[4] . "\n\n";
}
?>
```

The above example will output:

```
matched: <b>bold text</b>
part 1: <b>
part 2: b
part 3: bold text
part 4: </b>

matched: <a href=howdy.html>click me</a>
part 1: <a href=howdy.html>
part 2: a
part 3: click me
part 4: </a>
```

Example #3 Using named subpattern

```php
<?php

$str = <<<FOO
a: 1
b: 2
c: 3
FOO;

preg_match_all('/(?P<name>\w+): (?P<digit>\d+)/', $str, $matches);

/* Alternative */
// preg_match_all('/(?<name>\w+): (?<digit>\d+)/', $str, $matches);

print_r($matches);

?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [0] => a: 1
            [1] => b: 2
            [2] => c: 3
        )

    [name] => Array
        (
            [0] => a
            [1] => b
            [2] => c
        )

    [1] => Array
        (
            [0] => a
            [1] => b
            [2] => c
        )

    [digit] => Array
        (
            [0] => 1
            [1] => 2
            [2] => 3
        )

    [2] => Array
        (
            [0] => 1
            [1] => 2
            [2] => 3
        )

)
```

See Also

- PCRE Patterns

- preg_quote() - Quote regular expression characters
- preg_match() - Perform a regular expression match
- preg_replace() - Perform a regular expression search and replace
- preg_split() - Split string by a regular expression
- preg_last_error() - Returns the error code of the last PCRE regex execution

preg_quote

preg_quote

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_quote — Quote regular expression characters

Description

function **preg_quote**(string `$str`, ?string `$delimiter` = `null`): string

preg_quote() takes `str` and puts a backslash in front of every character that is part of the regular expression syntax. This is useful if you have a run-time string that you need to match in some text and the string may contain special regex characters.

The special regular expression characters are: `. \ + * ? [ ^ ] $ ( ) { } = ! < > | : - #`

Note that `/` is not a special regular expression character.

> **Note** :
>
> Note that **preg_quote()** is not meant to be applied to the $replacement string(s) of preg_replace() etc.

Parameters

`str`

The input string.

`delimiter`

If the optional `delimiter` is specified, it will also be escaped. This is useful for escaping the delimiter that is required by the PCRE functions. The `/` is the most commonly used delimiter.

Return Values

Returns the quoted (escaped) string.

Changelog

Version	Description
7.3.0	The `#` character is now quoted
7.2.0	`delimiter` is nullable now.

Examples

Example #1 **preg_quote()** example

```php
<?php
$keywords = '$40 for a g3/400';
$keywords = preg_quote($keywords, '/');
echo $keywords; // returns \$40 for a g3\/400
?>
```

Example #2 Italicizing a word within some text

```php
<?php
// In this example, preg_quote($word) is used to keep the
// asterisks from having special meaning to the regular
// expression.
```

```php
$textbody = "This book is *very* difficult to find.";
$word = "*very*";
$textbody = preg_replace ("/" . preg_quote($word, '/') . "/",
                          "<i>" . $word . "</i>",
                          $textbody);
?>
```

Notes

> **Note** :
>
> This function is binary-safe.

See Also

- PCRE Patterns
- escapeshellcmd() - Escape shell metacharacters

preg_replace

preg_replace

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_replace — Perform a regular expression search and replace

Description

function **preg_replace**(
  string|array `$pattern`,
  string|array `$replacement`,
  string|array `$subject`,
  int `$limit` = -1,
  int `&$count` = **null**
): string|array|null

Searches `subject` for matches to `pattern` and replaces them with `replacement`.

To match an exact string, rather than a pattern, consider using str_replace() or str_ireplace() instead of this function.

Parameters

`pattern`

The pattern to search for. It can be either a string or an array with strings.

Several PCRE modifiers are also available.

`replacement`

The string or an array with strings to replace. If this parameter is a string and the `pattern` parameter is an array, all patterns will be replaced by that string. If both `pattern` and `replacement` parameters are arrays, each `pattern` will be replaced by the `replacement` counterpart. If there are fewer elements in the `replacement` array than in the `pattern` array, any extra `pattern`s will be replaced by an empty string.

`replacement` may contain references of the form `\n` or `$n`, with the latter form being the preferred one. Every such reference will be replaced by the text captured by the n'th parenthesized pattern. n can be from 0 to 99, and `\0` or `$0` refers to the text matched by the whole pattern. Opening parentheses are counted from left to right (starting from 1) to obtain the number of the capturing subpattern. Note that backslashes in string literals may require to be escaped.

When working with a replacement pattern where a backreference is immediately followed by another number (i.e.: placing a literal number immediately after a matched pattern), you cannot use the familiar `\1` notation for your backreference. `\11`, for example, would confuse **preg_replace()** since it does not know whether you want the `\1` backreference followed by a literal `1`, or the `\11` backreference followed by nothing. In this case the solution is to use `${1}1`. This creates an isolated `$1` backreference, leaving the `1` as a literal.

`subject`

The string or an array with strings to search and replace.

If `subject` is an array, then the search and replace is performed on every entry of `subject`, and the return value is an array as well.

If the `subject` array is associative, keys will be preserved in the returned value.

`limit`

The maximum possible replacements for each pattern in each `subject` string. Defaults to `-1` (no limit).

`count`

If specified, this variable will be filled with the number of replacements done.

Return Values

preg_replace() returns an array if the `subject` parameter is an array, or a string otherwise.

If matches are found, the new `subject` will be returned, otherwise `subject` will be returned unchanged or `null` if an error occurred.

Errors/Exceptions

Using the "\e" modifier is an error; an `E_WARNING` is emitted in this case.

If the regex pattern passed does not compile to a valid regex, an `E_WARNING` is emitted.

Examples

Example #1 Using backreferences followed by numeric literals

```php
<?php
$string = 'April 15, 2003';
$pattern = '/(\w+) (\d+), (\d+)/i';
$replacement = '${1}1,$3';
echo preg_replace($pattern, $replacement, $string);
?>
```

The above example will output:

```
April1,2003
```

Example #2 Using indexed arrays with preg_replace()

```php
<?php
$string = 'The quick brown fox jumps over the lazy dog.';
$patterns = array();
$patterns[0] = '/quick/';
$patterns[1] = '/brown/';
$patterns[2] = '/fox/';
$replacements = array();
$replacements[2] = 'bear';
$replacements[1] = 'black';
$replacements[0] = 'slow';
echo preg_replace($patterns, $replacements, $string);
?>
```

The above example will output:

```
The bear black slow jumps over the lazy dog.
```

By ksorting patterns and replacements, we should get what we wanted.

```php
<?php
$string = 'The quick brown fox jumps over the lazy dog.';
$patterns = array();
$patterns[0] = '/quick/';
$patterns[1] = '/brown/';
$patterns[2] = '/fox/';
$replacements = array();
$replacements[2] = 'bear';
$replacements[1] = 'black';
$replacements[0] = 'slow';
ksort($patterns);
ksort($replacements);
echo preg_replace($patterns, $replacements, $string);
?>
```

The above example will output:

```
The slow black bear jumps over the lazy dog.
```

Example #3 Replacing several values

```php
<?php
$patterns = array ('/(19|20)(\d{2})-(\d{1,2})-(\d{1,2})/',
                   '/^\s*{(\w+)}\s*=/');
$replace = array ('\3/\4/\1\2', '$\1 =');
echo preg_replace($patterns, $replace, '{startDate} = 1999-5-27');
?>
```

The above example will output:

```
$startDate = 5/27/1999
```

Example #4 Strip whitespace

This example strips excess whitespace from a string.

```php
<?php
$str = 'foo   o';
$str = preg_replace('/\s\s+/', ' ', $str);
// This will be 'foo o' now
echo $str;
?>
```

Example #5 Using the `count` parameter

```php
<?php
$count = 0;

echo preg_replace(array('/\d/', '/\s/'), '*', 'xp 4 to', -1 , $count);
echo "\n";
echo $count; //3
?>
```

The above example will output:

```
xp***to
3
```

Notes

> **Note** :
>
> When using arrays with `pattern` and `replacement`, the keys are processed in the order they appear in the array. This is *not necessarily* the same as the numerical index order. If you use indexes to identify which `pattern` should be replaced by which `replacement`, you should perform a ksort() on each array prior to calling **preg_replace()**.

> **Note** :
>
> When both `pattern` and `replacement` are arrays, matching rules will operate sequentially. That is, the second `pattern`/`replacement` pair will operate on the string that results from the first `pattern`/`replacement` pair, not the original string. If you want to simulate replacements operating in parallel, such as swapping two values, replace one pattern by an intermediary placeholder, then in a later pair replace that intermediary placeholder with the desired replacement.
>
> ```php
> <?php
> $p = array('/a/', '/b/', '/c/');
> $r = array('b', 'c', 'd');
> print_r(preg_replace($p, $r, 'a'));
> // prints d
> ?>
> ```

See Also

- PCRE Patterns
- preg_quote() - Quote regular expression characters
- preg_filter() - Perform a regular expression search and replace
- preg_match() - Perform a regular expression match
- preg_replace_callback() - Perform a regular expression search and replace using a callback
- preg_split() - Split string by a regular expression
- preg_last_error() - Returns the error code of the last PCRE regex execution
- str_replace() - Replace all occurrences of the search string with the replacement string

preg_replace_callback

preg_replace_callback

(PHP 4 >= 4.0.5, PHP 5, PHP 7, PHP 8)

preg_replace_callback — Perform a regular expression search and replace using a callback

Description

```
function preg_replace_callback(
    string|array $pattern,
    callable $callback,
    string|array $subject,
    int $limit = -1,
    int &$count = null,
    int $flags = 0
): string|array|null
```

The behavior of this function is almost identical to preg_replace(), except for the fact that instead of `replacement` parameter, one should specify a `callback`.

Parameters

`pattern`

The pattern to search for. It can be either a string or an array with strings.

`callback`

A callback that will be called and passed an array of matched elements in the `subject` string. The callback should return the replacement string. This is the callback signature:

function handler(array `$matches`): string

You'll often need the `callback` function for a **preg_replace_callback()** in just one place. In this case you can use an anonymous function to declare the callback within the call to **preg_replace_callback()**. By doing it this way you have all information for the call in one place and do not clutter the function namespace with a callback function's name not used anywhere else.

Example #1 preg_replace_callback() and anonymous function

```php
<?php
/* a unix-style command line filter to convert uppercase
 * letters at the beginning of paragraphs to lowercase */
$fp = fopen("php://stdin", "r") or die("can't read stdin");
while (!feof($fp)) {
    $line = fgets($fp);
    $line = preg_replace_callback(
        '|<p>\s*\w|',
        function ($matches) {
            return strtolower($matches[0]);
        },
        $line
    );
    echo $line;
}
fclose($fp);
?>
```

`subject`

The string or an array with strings to search and replace.

`limit`

The maximum possible replacements for each pattern in each `subject` string. Defaults to `-1` (no limit).

`count`

If specified, this variable will be filled with the number of replacements done.

`flags`

`flags` can be a combination of the `PREG_OFFSET_CAPTURE` and `PREG_UNMATCHED_AS_NULL` flags, which influence the format of the matches array. See the description in preg_match() for more details.

Return Values

preg_replace_callback() returns an array if the `subject` parameter is an array, or a string otherwise. On errors the return value is `null`

If matches are found, the new subject will be returned, otherwise `subject` will be returned unchanged.

Errors/Exceptions

If the regex pattern passed does not compile to a valid regex, an `E_WARNING` is emitted.

Changelog

Version	Description
7.4.0	The `flags` parameter was added.

Examples

Example #2 preg_replace_callback() example

```php
<?php
// this text was used in 2002
// we want to get this up to date for 2003
$text = "April fools day is 04/01/2002\n";
$text.= "Last christmas was 12/24/2001\n";
// the callback function
function next_year($matches)
{
  // as usual: $matches[0] is the complete match
  // $matches[1] the match for the first subpattern
  // enclosed in '(...)' and so on
  return $matches[1].($matches[2]+1);
}
echo preg_replace_callback(
            "|(\d{2}/\d{2}/)(\d{4})|",
            "next_year",
            $text);

?>
```

The above example will output:

```
April fools day is 04/01/2003
Last christmas was 12/24/2002
```

Example #3 preg_replace_callback() using recursive structure to handle encapsulated BB code

```php
<?php
$input = "plain [indent] deep [indent] deeper [/indent] deep [/indent] plain";

function parseTagsRecursive($input)
{

    $regex = '#\[indent]((?:[^[]|\[(?!/?indent])|(?R))+)\[/indent]#';

    if (is_array($input)) {
        $input = '<div style="margin-left: 10px">'.$input[1].'</div>';
    }

    return preg_replace_callback($regex, 'parseTagsRecursive', $input);
}

$output = parseTagsRecursive($input);

echo $output;
?>
```

See Also

- PCRE Patterns
- preg_replace_callback_array() - Perform a regular expression search and replace using callbacks
- preg_quote() - Quote regular expression characters
- preg_replace() - Perform a regular expression search and replace
- preg_last_error() - Returns the error code of the last PCRE regex execution
- Anonymous functions

preg_replace_callback_array

preg_replace_callback_array

(PHP 7, PHP 8)

preg_replace_callback_array — Perform a regular expression search and replace using callbacks

Description

function **preg_replace_callback_array**(
  array $pattern,
  string|array $subject,
  int $limit = -1,
  int &$count = null,
  int $flags = 0
): string|array|null

The behavior of this function is similar to preg_replace_callback(), except that callbacks are executed on a per-pattern basis.

Parameters

`pattern`

An associative array mapping patterns (keys) to callables (values).

`subject`

The string or an array with strings to search and replace.

`limit`

The maximum possible replacements for each pattern in each `subject` string. Defaults to `-1` (no limit).

`count`

If specified, this variable will be filled with the number of replacements done.

`flags`

`flags` can be a combination of the `PREG_OFFSET_CAPTURE` and `PREG_UNMATCHED_AS_NULL` flags, which influence the format of the matches array. See the description in preg_match() for more details.

Return Values

preg_replace_callback_array() returns an array if the `subject` parameter is an array, or a string otherwise. On errors the return value is `null`

If matches are found, the new subject will be returned, otherwise `subject` will be returned unchanged.

Errors/Exceptions

If the regex pattern passed does not compile to a valid regex, an `E_WARNING` is emitted.

Changelog

Version	Description
7.4.0	The `flags` parameter was added.

Examples

Example #1 **preg_replace_callback_array()** example

```php
<?php
$subject = 'Aaaaaa Bbb';

preg_replace_callback_array(
    [
        '~[a]+~i' => function ($match) {
            echo strlen($match[0]), ' matches for "a" found', PHP_EOL;
        },
        '~[b]+~i' => function ($match) {
            echo strlen($match[0]), ' matches for "b" found', PHP_EOL;
        }
    ],
    $subject
```

```
);
?>
```

The above example will output:

```
6 matches for "a" found
3 matches for "b" found
```

See Also

- PCRE Patterns
- preg_replace_callback() - Perform a regular expression search and replace using a callback
- preg_quote() - Quote regular expression characters
- preg_replace() - Perform a regular expression search and replace
- preg_last_error() - Returns the error code of the last PCRE regex execution
- Anonymous functions

preg_split

preg_split

(PHP 4, PHP 5, PHP 7, PHP 8)

preg_split — Split string by a regular expression

Description

function **preg_split**(
  string `$pattern`,
  string `$subject`,
  int `$limit` = -1,
  int `$flags` = 0
): array|false

Split the given string by a regular expression.

Parameters

`pattern`

The pattern to search for, as a string.

`subject`

The input string.

`limit`

If specified, then only substrings up to `limit` are returned with the rest of the string being placed in the last substring. A `limit` of -1 or 0 means "no limit".

`flags`

`flags` can be any combination of the following flags (combined with the | bitwise operator):

`PREG_SPLIT_NO_EMPTY`

If this flag is set, only non-empty pieces will be returned by **preg_split()**.

`PREG_SPLIT_DELIM_CAPTURE`

If this flag is set, parenthesized expression in the delimiter pattern will be captured and returned as well.

`PREG_SPLIT_OFFSET_CAPTURE`

If this flag is set, for every occurring match the appendant string offset will also be returned. Note that this changes the return value in an array where every element is an array consisting of the matched string at offset `0` and its string offset into `subject` at offset `1`.

Return Values

Returns an array containing substrings of `subject` split along boundaries matched by `pattern`, or `false` on failure.

Errors/Exceptions

If the regex pattern passed does not compile to a valid regex, an `E_WARNING` is emitted.

Examples

Example #1 preg_split() example : Get the parts of a search string

```php
<?php
// split the phrase by any number of commas or space characters,
// which include " ", \r, \t, \n and \f
$keywords = preg_split("/[\s,]+/", "hypertext language, programming");
print_r($keywords);
?>
```

The above example will output:

```
Array
(
    [0] => hypertext
    [1] => language
    [2] => programming
)
```

Example #2 Splitting a string into component characters

```php
<?php
$str = 'string';
$chars = preg_split('//', $str, -1, PREG_SPLIT_NO_EMPTY);
print_r($chars);
?>
```

The above example will output:

```
Array
(
    [0] => s
    [1] => t
    [2] => r
    [3] => i
    [4] => n
    [5] => g
)
```

Example #3 Splitting a string into matches and their offsets

```php
<?php
$str = 'hypertext language programming';
$chars = preg_split('/ /', $str, -1, PREG_SPLIT_OFFSET_CAPTURE);
```

```php
print_r($chars);
?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [0] => hypertext
            [1] => 0
        )

    [1] => Array
        (
            [0] => language
            [1] => 10
        )

    [2] => Array
        (
            [0] => programming
            [1] => 19
        )

)
```

Notes

> **Tip**
>
> If you don't need the power of regular expressions, you can choose faster (albeit simpler) alternatives like explode() or str_split().

> **Tip**
>
> If matching fails, an array with a single element containing the input string will be returned.

See Also

- PCRE Patterns
- preg_quote() - Quote regular expression characters
- explode() - Split a string by a string
- preg_match() - Perform a regular expression match
- preg_match_all() - Perform a global regular expression match
- preg_replace() - Perform a regular expression search and replace
- preg_last_error() - Returns the error code of the last PCRE regex execution

Multibyte String

Introduction

While there are many languages in which every necessary character can be represented by a one-to-one mapping to an 8-bit value, there are also several languages which require so many characters for written communication that they cannot be contained within the range a mere byte can code (A byte is made up of eight bits. Each bit can contain only two distinct values, one or zero. Because of this, a byte can only represent 256 unique values (two to the power of eight)). Multibyte character encoding schemes were developed to express more than 256 characters in the regular bytewise coding system.

When you manipulate (trim, split, splice, etc.) strings encoded in a multibyte encoding, you need to use special functions since two or more consecutive bytes may represent a single character in such encoding schemes. Otherwise, if you apply a non-multibyte-aware string function to the string, it probably fails to detect the beginning or ending of the multibyte character and ends up with a corrupted garbage string that most likely loses its original meaning.

`mbstring` provides multibyte specific string functions that help you deal with multibyte encodings in PHP. In addition to that, `mbstring` handles character encoding conversion between the possible encoding pairs. `mbstring` is designed to handle Unicode-based encodings such as UTF-8 and UCS-2 and many single-byte encodings for convenience (listed in Supported Character Encodings).

The Unicode data tables used by `mbstring` have been updated to Unicode 16.0 as of PHP 8.4.0, and to Unicode 17.0 as of PHP 8.5.0.

Installing/Configuring

Table of Contents

Installation

`mbstring` is a non-default extension. This means it is not enabled by default. You must explicitly enable the module with the `configure` option. See the Install section for details.

The following configure options are related to the `mbstring` module.

- **--enable-mbstring**: Enable `mbstring` functions. This option is required to use `mbstring` functions.

  libmbfl is necessary for `mbstring`. libmbfl is bundled with `mbstring`. Prior to PHP 7.3.0, if libmbfl is already installed on the system, **--with-libmbfl[=DIR]** can be specified to use the installed library.

- **--disable-mbregex**: Disable regular expression functions with multibyte character support.

  Oniguruma is necessary for the regular expression functions with multibyte character support. As of PHP 7.4.0, pkg-config is used to detect the libonig library. Prior to PHP 7.4.0, Oniguruma was bundled with `mbstring`, but it was possible to build against an already installed libonig by passing **--with-onig[=DIR]**.

  It is possible to disable the multibyte regex backtrack check by specifying **--disable-mbregex-backtrack**.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

mbstring configuration options

Name	Default	Changeable	Changelog
mbstring.language	"neutral"	`INI_ALL`	
mbstring.detect_order	NULL	`INI_ALL`	
mbstring.http_input	"pass"	`INI_ALL`	Deprecated
mbstring.http_output	"pass"	`INI_ALL`	Deprecated
mbstring.internal_encoding	NULL	`INI_ALL`	Deprecated
mbstring.substitute_character	NULL	`INI_ALL`	
mbstring.func_overload	"0"	`INI_SYSTEM`	Deprecated as of PHP 7.2.0; removed as of PHP 8.0.0.
mbstring.encoding_translation	"0"	`INI_PERDIR`	

Name	Default	Changeable	Changelog
mbstring.http_output_conv_mimetypes	"^(text/|application/xhtml\+xml)"	INI_ALL	
mbstring.strict_detection	"0"	INI_ALL	
mbstring.regex_retry_limit	"1000000"	INI_ALL	Available as of PHP 7.4.0.
mbstring.regex_stack_limit	"100000"	INI_ALL	Available as of PHP 7.3.5.

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`mbstring.language` string

The default national language setting (NLS) used in mbstring. Note that this option automagically defines `mbstring.internal_encoding` and `mbstring.internal_encoding` should be placed after `mbstring.language` in *php.ini*

`mbstring.encoding_translation` bool

Enables the transparent character encoding filter for the incoming HTTP queries, which performs detection and conversion of the input encoding to the internal character encoding.

`mbstring.internal_encoding` string

> **Warning**
>
> This deprecated feature *will* certainly be *removed* in the future.

Defines the default internal character encoding.

Users should leave this empty and set `default_charset` instead.

`mbstring.http_input` string

> **Warning**
>
> This deprecated feature *will* certainly be *removed* in the future.

Defines the default HTTP input character encoding.

Users should leave this empty and set `default_charset` instead.

`mbstring.http_output` string

> **Warning**
>
> This deprecated feature *will* certainly be *removed* in the future.

Defines the default HTTP output character encoding (output will be converted from the internal encoding to the HTTP output encoding upon output).

Users should leave this empty and set `default_charset` instead.

`mbstring.detect_order` string

Defines default character code detection order. See also mb_detect_order().

`mbstring.substitute_character` string

Defines character to substitute for invalid character encoding. See mb_substitute_character() for supported values.

`mbstring.func_overload` string

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. Relying on this feature is highly discouraged.

Overloads a set of single byte functions by the mbstring counterparts. See Function overloading for more information.

This setting can only be changed from the *php.ini* file.

`mbstring.http_output_conv_mimetypes` string

`mbstring.strict_detection` bool

Enables strict encoding detection. See mb_detect_encoding() for a description and examples.

`mbstring.regex_retry_limit` int

Limits the amount of backtracking that may be performed during one mbregex match.

This setting only takes effect when linking against oniguruma >= 6.8.0.

`mbstring.regex_stack_limit` int

Limits the stack depth of mbstring regular expressions.

According to the » HTML 4.01 specification, Web browsers are allowed to encode a form being submitted with a character encoding different from the one used for the page. See mb_http_input() to detect character encoding used by browsers.

Although popular browsers are capable of giving a reasonably accurate guess to the character encoding of a given HTML document, it would be better to set the `charset` parameter in the `Content-Type` HTTP header to the appropriate value by header() or default_charset ini setting.

Example #1 *php.ini* setting examples

```
; Set default language
mbstring.language        = Neutral; Set default language to Neutral(UTF-8) (default)
mbstring.language        = English; Set default language to English
mbstring.language        = Japanese; Set default language to Japanese

;; Set default internal encoding
;; Note: Make sure to use character encoding works with PHP
mbstring.internal_encoding    = UTF-8  ; Set internal encoding to UTF-8

;; HTTP input encoding translation is enabled.
mbstring.encoding_translation = On

;; Set default HTTP input character encoding
;; Note: Script cannot change http_input setting.
mbstring.http_input           = pass    ; No conversion.
mbstring.http_input           = auto    ; Set HTTP input to auto
                                ; "auto" is expanded according to mbstring.language
mbstring.http_input           = SJIS    ; Set HTTP input to SJIS
mbstring.http_input           = UTF-8,SJIS,EUC-JP ; Specify order

;; Set default HTTP output character encoding
mbstring.http_output          = pass    ; No conversion
mbstring.http_output          = UTF-8   ; Set HTTP output encoding to UTF-8

;; Set default character encoding detection order
mbstring.detect_order         = auto    ; Set detect order to auto
mbstring.detect_order         = ASCII,JIS,UTF-8,SJIS,EUC-JP ; Specify order

;; Set default substitute character
mbstring.substitute_character = 12307   ; Specify Unicode value
mbstring.substitute_character = none    ; Do not print character
mbstring.substitute_character = long    ; Long Example: U+3000,JIS+7E7E
```

Example #2 *php.ini* setting for `EUC-JP` users

```
;; Disable Output Buffering
output_buffering      = Off

;; Set HTTP header charset
default_charset       = EUC-JP

;; Set default language to Japanese
mbstring.language = Japanese
```

```
;; HTTP input encoding translation is enabled.
mbstring.encoding_translation = On

;; Set HTTP input encoding conversion to auto
mbstring.http_input   = auto

;; Convert HTTP output to EUC-JP
mbstring.http_output  = EUC-JP

;; Set internal encoding to EUC-JP
mbstring.internal_encoding = EUC-JP

;; Do not print invalid characters
mbstring.substitute_character = none
```

Example #3 *php.ini* setting for `SJIS` users

```
;; Enable Output Buffering
output_buffering     = On

;; Set mb_output_handler to enable output conversion
output_handler       = mb_output_handler

;; Set HTTP header charset
default_charset      = Shift_JIS

;; Set default language to Japanese
mbstring.language = Japanese

;; Set http input encoding conversion to auto
mbstring.http_input  = auto

;; Convert to SJIS
mbstring.http_output = SJIS

;; Set internal encoding to EUC-JP
mbstring.internal_encoding = EUC-JP

;; Do not print invalid characters
mbstring.substitute_character = none
```

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

`MB_OVERLOAD_MAIL` (int)

Removed as of PHP 8.0.0.

`MB_OVERLOAD_STRING` (int)

Removed as of PHP 8.0.0.

`MB_OVERLOAD_REGEX` (int)

Removed as of PHP 8.0.0.

`MB_CASE_UPPER` (int)

Performs a full upper-case folding. This may change the length of the string. This is the mode used by mb_strtoupper().

`MB_CASE_LOWER` (int)

Performs a full lower-case folding. This may change the length of the string. This is the mode used by mb_strtolower().

`MB_CASE_TITLE` (int)

Performs a full title-case conversion based on the Cased and CaseIgnorable derived Unicode properties. In particular this improves handling of quotes and apostrophes. This may change the length of the string.

`MB_CASE_FOLD` (int)

Performs a full case fold conversion which removes case distinctions present in the string. This is used for caseless matching. This may change the length of the string. Available since PHP 7.3.

`MB_CASE_LOWER_SIMPLE` (int)

Performs a simple lower-case fold conversion. This does not change the length of the string. Available as of PHP 7.3.

`MB_CASE_UPPER_SIMPLE` (int)

Performs simple upper-case fold conversion. This does not change the length of the string. Available as of PHP 7.3.

`MB_CASE_TITLE_SIMPLE` (int)

Performs simple title-case fold conversion. This does not change the length of the string. Available as of PHP 7.3.

`MB_CASE_FOLD_SIMPLE` (int)

Performs a simple case fold conversion which removes case distinctions present in the string. This is used for caseless matching. This does not change the length of the string. Used by case-insensitive operations internally by the MBString extension. Available as of PHP 7.3.

`MB_ONIGURUMA_VERSION` (string)

The Oniguruma version, e.g. `6.9.4`. Available as of PHP 7.4.

Summaries of supported encodings

Summaries of supported encodings

Name in the IANA character set registry	Underlying character set	Description	Additional note
ISO-10646-UCS-4	ISO 10646	The Universal Character Set with 31-bit code space, standardized as UCS-4 by ISO/IEC 10646. It is kept synchronized with the latest version of the Unicode code map.	If this name is used in the encoding conversion facility, the converter attempts to identify by the preceding BOM (byte order mark)in which endian the subsequent bytes are represented.
ISO-10646-UCS-4	UCS-4	See above.	In contrast to `UCS-4`, strings are always assumed to be in big endian form.
ISO-10646-UCS-4	UCS-4	See above.	In contrast to `UCS-4`, strings are always assumed to be in little endian form.
ISO-10646-UCS-2	UCS-2	The Universal Character Set with 16-bit code space, standardized as UCS-2 by ISO/IEC 10646. It is kept synchronized with the latest version of the unicode code map.	If this name is used in the encoding conversion facility, the converter attempts to identify by the preceding BOM (byte order mark)in which endian the subsequent bytes are represented.
ISO-10646-UCS-2	UCS-2	See above.	In contrast to `UCS-2`, strings are always

Name in the IANA character set registry	Underlying character set	Description	Additional note
			assumed to be in big endian form.
ISO-10646-UCS-2	UCS-2	See above.	In contrast to UCS-2, strings are always assumed to be in little endian form.
UTF-32	Unicode	Unicode Transformation Format of 32-bit unit width, whose encoding space refers to the Unicode's codeset standard. This encoding scheme wasn't identical to UCS-4 because the code space of Unicode were limited to a 21-bit value.	If this name is used in the encoding conversion facility, the converter attempts to identify by the preceding BOM (byte order mark)in which endian the subsequent bytes are represented.
UTF-32BE	Unicode	See above	In contrast to UTF-32, strings are always assumed to be in big endian form.
UTF-32LE	Unicode	See above	In contrast to UTF-32, strings are always assumed to be in little endian form.
UTF-16	Unicode	Unicode Transformation Format of 16-bit unit width. It's worth a note that UTF-16 is no longer the same specification as UCS-2 because the surrogate mechanism has been introduced since Unicode 2.0 and UTF-16 now refers to a 21-bit code space.	If this name is used in the encoding conversion facility, the converter attempts to identify by the preceding BOM (byte order mark)in which endian the subsequent bytes are represented.
UTF-16BE	Unicode	See above.	In contrast to UTF-16, strings are always assumed to be in big endian form.
UTF-16LE	Unicode	See above.	In contrast to UTF-16, strings are always assumed to be in little endian form.
UTF-8	Unicode / UCS	Unicode Transformation Format of 8-bit unit width.	none
UTF-7	Unicode	A mail-safe transformation format of Unicode, specified in » RFC2152.	none
(none)	Unicode	A variant of UTF-7 which is specialized for use in the » IMAP protocol.	none
US-ASCII (preferred MIME name) / iso-ir-6 / ANSI_X3.4-1986 / ISO_646.irv:1991 / ASCII / ISO646-US / us / IBM367 / CP367 / csASCII	ASCII / ISO 646	American Standard Code for Information Interchange is a commonly-used 7-bit encoding. Also standardized as an international standard, ISO 646.	(none)

Name in the IANA character set registry	Underlying character set	Description	Additional note
EUC-JP (preferred MIME name) / Extended_UNIX_Code_Packed_Format_for_Japanese / csEUCPkdFmtJapanese	Compound of US-ASCII / JIS X0201:1997 (hankaku kana part) / JIS X0208:1990 / JIS X0212:1990	As you see the name is derived from an abbreviation of Extended UNIX Code Packed Format for Japanese, this encoding is mostly used on UNIX or alike platforms. The original encoding scheme, Extended UNIX Code, is designed on the basis of ISO 2022.	The character set referred to by EUC-JP is different to IBM932 / CP932, which are used by OS/2® and Microsoft® Windows®. For information interchange with those platforms, use EUCJP-WIN instead.
Shift_JIS (preferred MIME name) / MS_Kanji / csShift_JIS	Compound of JIS X0201:1997 / JIS X0208:1997	Shift_JIS was developed in early 80's, at the time personal Japanese word processors were brought into the market, in order to maintain compatibilities with the legacy encoding scheme JIS X 0201:1976. According to the IANA definition the codeset of Shift_JIS is slightly different to IBM932 / CP932. However, the names "SJIS" / "Shift_JIS" are often wrongly used to refer to these codesets.	For the CP932 codemap, use SJIS-WIN instead.
(none)	Compound of JIS X0201:1997 / JIS X0208:1997 / IBM extensions / NEC extensions	While this "encoding" uses the same encoding scheme as EUC-JP, the underlying character set is different. That is, some code points map to different characters than EUC-JP.	none
Windows-31J / csWindows31J	Compound of JIS X0201:1997 / JIS X0208:1997 / IBM extensions / NEC extensions	While this "encoding" uses the same encoding scheme as Shift_JIS, the underlying character set is different. That means some code points map to different characters than Shift_JIS.	(none)
ISO-2022-JP (preferred MIME name) / csISO2022JP	US-ASCII / JIS X0201:1976 / JIS X0208:1978 / JIS X0208:1983	» RFC1468	(none)
JIS			
ISO-8859-1			
ISO-8859-2			
ISO-8859-3			
ISO-8859-4			
ISO-8859-5			
ISO-8859-6			
ISO-8859-7			
ISO-8859-8			
ISO-8859-9			
ISO-8859-10			
ISO-8859-13			

Name in the IANA character set registry	Underlying character set	Description	Additional note
ISO-8859-14			
ISO-8859-15			
ISO-8859-16			
byte2be			
byte2le			
byte4be			
byte4le			
BASE64			
HTML-ENTITIES			
7bit			
8bit			
EUC-CN			
CP936			
HZ			
EUC-TW			
CP950			
BIG-5			
EUC-KR			
UHC (CP949)			
ISO-2022-KR			
Windows-1251 (CP1251)			
Windows-1252 (CP1252)			
CP866 (IBM866)			
KOI8-R			
KOI8-U			

Basics of Japanese multi-byte encodings

Japanese characters can only be represented by multibyte encodings, and multiple encoding standards are used depending on platform and text purpose. To make matters worse, these encoding standards differ slightly from one another. In order to create a web application which would be usable in a Japanese environment, a developer has to keep these complexities in mind to ensure that the proper character encodings are used.

- Storage for a character can be up to six bytes
- Most Japanese multibyte characters appear twice as wide as single-byte characters. These characters are called "zen-kaku" in Japanese, which means "full width". Other, narrower, characters are called "han-kaku", which means "half width". The graphical properties of the characters, however, depends upon the type faces used to display them.
- Some character encodings use shift(escape) sequences defined in ISO-2022 to switch the code map of the specific code area (`00h` to `7fh`).
- ISO-2022-JP should be used in SMTP/NNTP, and headers and entities should be reencoded as per RFC requirements. Although those are not requisites, it's still a good idea because several popular user agents cannot recognize any other encoding methods.
- Web pages created for mobile phone services such as » i-mode or » EZweb are supposed to use Shift_JIS.
- Emoji used for mobile phone service such as » i-mode or » EZweb are supported.

HTTP Input and Output

HTTP input/output character encoding conversion may convert binary data also. Users are supposed to control character encoding conversion if binary data is used for HTTP input/output.

> **Note** :
>
> If `enctype` for HTML form is set to `multipart/form-data` and `mbstring.encoding_translation` is set to On in *php.ini* the POST'ed variables and the names of uploaded files will be converted to the internal character encoding as well. However, the conversion isn't applied to the query keys.

- HTTP Input

  There is no way to control HTTP input character conversion from a PHP script. To disable HTTP input character conversion, it has to be done in *php.ini*.

  Example #1 Disable HTTP input conversion in *php.ini*

  ```
  ;; Disable HTTP Input conversion
  mbstring.http_input = pass
  ;; Disable HTTP Input conversion
  mbstring.encoding_translation = Off
  ```

  When using PHP as an Apache module, it is possible to override those settings in each Virtual Host directive in *httpd.conf* or per directory with *.htaccess*. Refer to the Configuration section and Apache Manual for details.

- HTTP Output

  There are several ways to enable output character encoding conversion. One is using *php.ini*, another is using ob_start() with mb_output_handler() as the `ob_start` callback function.

Example #2 *php.ini* setting example

```
;; Enable output character encoding conversion for all PHP pages

;; Enable Output Buffering
output_buffering    = On

;; Set mb_output_handler to enable output conversion
output_handler      = mb_output_handler
```

Example #3 Script example

```
<?php

// Enable output character encoding conversion only for this page

// Set HTTP output character encoding to SJIS
mb_http_output('SJIS');

// Start buffering and specify "mb_output_handler" as
// callback function
ob_start('mb_output_handler');

?>
```

Supported Character Encodings

Currently the following character encodings are supported by the `mbstring` module. Any of those Character encodings can be specified in the `encoding` parameter of `mbstring` functions.

The following character encodings are supported in this PHP extension:

- UCS-4*
- UCS-4BE
- UCS-4LE*

- UCS-2
- UCS-2BE
- UCS-2LE
- UTF-32*
- UTF-32BE*
- UTF-32LE*
- UTF-16*
- UTF-16BE*
- UTF-16LE*
- UTF-7
- UTF7-IMAP
- UTF-8*
- ASCII*
- EUC-JP*
- SJIS*
- eucJP-win*
- SJIS-win*
- ISO-2022-JP
- ISO-2022-JP-MS
- CP932
- CP51932
- SJIS-mac (alias: MacJapanese)
- SJIS-Mobile#DOCOMO (alias: SJIS-DOCOMO)
- SJIS-Mobile#KDDI (alias: SJIS-KDDI)
- SJIS-Mobile#SOFTBANK (alias: SJIS-SOFTBANK)
- UTF-8-Mobile#DOCOMO (alias: UTF-8-DOCOMO)
- UTF-8-Mobile#KDDI-A
- UTF-8-Mobile#KDDI-B (alias: UTF-8-KDDI)
- UTF-8-Mobile#SOFTBANK (alias: UTF-8-SOFTBANK)
- ISO-2022-JP-MOBILE#KDDI (alias: ISO-2022-JP-KDDI)
- JIS
- JIS-ms
- CP50220
- CP50220raw
- CP50221
- CP50222
- ISO-8859-1*

- ISO-8859-2*
- ISO-8859-3*
- ISO-8859-4*
- ISO-8859-5*
- ISO-8859-6*
- ISO-8859-7*
- ISO-8859-8*
- ISO-8859-9*
- ISO-8859-10*
- ISO-8859-13*
- ISO-8859-14*
- ISO-8859-15*
- ISO-8859-16*
- byte2be
- byte2le
- byte4be
- byte4le
- BASE64
- HTML-ENTITIES (alias: HTML)
- 7bit
- 8bit
- EUC-CN*
- CP936
- GB18030
- HZ
- EUC-TW*
- CP950
- BIG-5*
- EUC-KR*
- UHC (alias: CP949)
- ISO-2022-KR
- Windows-1251 (alias: CP1251)
- Windows-1252 (alias: CP1252)
- CP866 (alias: IBM866)
- KOI8-R*
- KOI8-U*
- ArmSCII-8 (alias: ArmSCII8)

* denotes encodings usable also in regular expressions.

Any *php.ini* entry which accepts an encoding name can also use the values "`auto`" and "`pass`". `mbstring` functions which accept an encoding name can also use the value "`auto`".

If "`pass`" is set, no character encoding conversion is performed.

If "`auto`" is set, it is expanded to the list of encodings defined per the NLS. For instance, if the NLS is set to `Japanese`, the value is assumed to be "`ASCII,JIS,UTF-8,EUC-JP,SJIS`".

See also mb_detect_order()

Function Overloading Feature

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. Relying on this feature is highly discouraged.

You might often find it difficult to get an existing PHP application to work in a given multibyte environment. This happens because most PHP applications out there are written with the standard string functions such as substr(), which are known to not properly handle multibyte-encoded strings.

mbstring supports a 'function overloading' feature which enables you to add multibyte awareness to such an application without code modification by overloading multibyte counterparts on the standard string functions. For example, mb_substr() is called instead of substr() if function overloading is enabled. This feature makes it easy to port applications that only support single-byte encodings to a multibyte environment in many cases.

To use function overloading, set `mbstring.func_overload` in *php.ini* to a positive value that represents a combination of bitmasks specifying the categories of functions to be overloaded. It should be set to 1 to overload the mail() function. 2 for string functions, 4 for regular expression functions. For example, if it is set to 7, mail, strings and regular expression functions will be overloaded. The list of overloaded functions are shown below.

Functions to be overloaded

value of mbstring.func_overload	original function	overloaded function
1	mail()	mb_send_mail()
2	strlen()	mb_strlen()
2	strpos()	mb_strpos()
2	strrpos()	mb_strrpos()
2	substr()	mb_substr()
2	strtolower()	mb_strtolower()
2	strtoupper()	mb_strtoupper()
2	stripos()	mb_stripos()
2	strripos()	mb_strripos()
2	strstr()	mb_strstr()
2	stristr()	mb_stristr()
2	strrchr()	mb_strrchr()
2	substr_count()	mb_substr_count()

> **Note** :
>
> It is not recommended to use the function overloading option in the per-directory context, because it's not confirmed yet to be stable enough in a production environment and may lead to undefined behaviour.

PHP Character Encoding Requirements

Encodings of the following types are safely used with PHP.

- A singlebyte encoding,

    - which has ASCII-compatible (ISO646 compatible) mappings for the characters in range of `00h` to `7fh`.

- A multibyte encoding,
    - which has ASCII-compatible mappings for the characters in range of `00h` to `7fh`.
    - which don't use ISO2022 escape sequences.
    - which don't use a value from `00h` to `7fh` in any of the compounded bytes that represents a single character.

These are examples of character encodings that are unlikely to work with PHP.

```
JIS, SJIS, ISO-2022-JP, BIG-5
```

Although PHP scripts written in any of those encodings might not work, especially in the case where encoded strings appear as identifiers or literals in the script, you can almost avoid using these encodings by setting up the `mbstring`'s transparent encoding filter function for incoming HTTP queries.

> **Note** :
>
> It's highly discouraged to use SJIS, BIG5, CP936, CP949 and GB18030 for the internal encoding unless you are familiar with the parser, the scanner and the character encoding.

> **Note** :
>
> If you are connecting to a database with PHP, it is recommended that you use the same character encoding for both the database and the `internal encoding` for ease of use and better performance.
>
> If you are using PostgreSQL, the character encoding used in the database and the one used in PHP may differ as it supports automatic character set conversion between the backend and the frontend.

Multibyte String Functions

References

Multibyte character encoding schemes and their related issues are fairly complicated, and are beyond the scope of this documentation. Please refer to the following URLs and other resources for further information regarding these topics.

- Unicode materials

  » http://www.unicode.org/

- Japanese/Korean/Chinese character information

  » https://resources.oreilly.com/examples/9781565922242/blob/master/doc/cjk.inf

Table of Contents

mb_check_encoding

mb_check_encoding

(PHP 4 >= 4.4.3, PHP 5 >= 5.1.3, PHP 7, PHP 8)

mb_check_encoding — Check if strings are valid for the specified encoding

Description

function **mb_check_encoding**(array|string|null `$value` = **`null`**, ?string `$encoding` = **`null`**): bool

Checks if the specified byte stream is valid for the specified encoding. If `value` is of type array, all keys and values are validated recursively. It is useful to prevent so-called "Invalid Encoding Attack".

Parameters

`value`

The byte stream or array to check. If it is omitted, this function checks all the input from the beginning of the request.

> **Warning**
>
> As of PHP 8.1.0, omitting this parameter or passing `null` is deprecated.

`encoding`

The expected encoding.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.1.0	Calling this function with `null` as `value` or without argument is deprecated.
8.0.0	`value` and `encoding` are nullable now.
7.2.0	This function now also accepts an array as `value`. Formerly, only strings have been supported.

mb_chr

mb_chr

(PHP 7 >= 7.2.0, PHP 8)

mb_chr — Return character by Unicode code point value

Description

function **mb_chr**(int `$codepoint`, ?string `$encoding` = `null`): string|false

Returns a string containing the character specified by the Unicode code point value, encoded in the specified encoding.

This function complements mb_ord().

Parameters

`codepoint`

A Unicode codepoint value, e.g. `128024` for *U+1F418 ELEPHANT*

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

A string containing the requested character, if it can be represented in the specified encoding or `false` on failure.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.

Examples

Example #1 Testing different code points

```
<?php
$values = [65, 63, 0x20AC, 128024];
foreach ($values as $value) {
    var_dump(mb_chr($value, 'UTF-8'));
    var_dump(mb_chr($value, 'ISO-8859-1'));
}
?>
```

The above example will output:

```
string(1) "A"
string(1) "A"
string(1) "?"
string(1) "?"
string(3) "€"
bool(false)
string(4) "🐘"
bool(false)
```

See Also

- mb_internal_encoding() - Set/Get internal character encoding
- mb_ord() - Get Unicode code point of character
- IntlChar::ord() - Return Unicode code point value of character
- chr() - Generate a single-byte string from a number

mb_convert_case

mb_convert_case

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_convert_case — Perform case folding on a string

Description

function **mb_convert_case**(string `$string`, int `$mode`, ?string `$encoding` = `null`): string

Performs case folding on a string, converted in the way specified by `mode`.

Parameters

`string`

The string being converted.

`mode`

The mode of the conversion. It can be one of `MB_CASE_UPPER`, `MB_CASE_LOWER`, `MB_CASE_TITLE`, `MB_CASE_FOLD`, `MB_CASE_UPPER_SIMPLE`, `MB_CASE_LOWER_SIMPLE`, `MB_CASE_TITLE_SIMPLE`, `MB_CASE_FOLD_SIMPLE`.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

A case folded version of `string` converted in the way specified by `mode`.

Changelog

Version	Description
8.3.0	Implemented conditional casing rules for the Greek letter sigma which only apply to `MB_CASE_LOWER` and `MB_CASE_TITLE` modes, not to `MB_CASE_LOWER_SIMPLE` and `MB_CASE_TITLE_SIMPLE`.
7.3.0	Added support for `MB_CASE_FOLD`, `MB_CASE_UPPER_SIMPLE`, `MB_CASE_LOWER_SIMPLE`, `MB_CASE_TITLE_SIMPLE`, and `MB_CASE_FOLD_SIMPLE` as `mode`.

Examples

Example #1 mb_convert_case() example

```php
<?php
$str = "mary had a Little lamb and she loved it so";
$str = mb_convert_case($str, MB_CASE_UPPER, "UTF-8");
echo $str, PHP_EOL;
$str = mb_convert_case($str, MB_CASE_TITLE, "UTF-8");
echo $str, PHP_EOL;
?>
```

Example #2 mb_convert_case() example with non-Latin UTF-8 text

```php
<?php
$str = "Τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός";
$str = mb_convert_case($str, MB_CASE_UPPER, "UTF-8");
echo $str, PHP_EOL;
$str = mb_convert_case($str, MB_CASE_TITLE, "UTF-8");
echo $str, PHP_EOL;
?>
```

Notes

By contrast to the standard case folding functions such as strtolower() and strtoupper(), case folding is performed on the basis of the Unicode character properties. Thus the behaviour of this function is not affected by locale settings and it can convert any characters that have 'alphabetic' property, such a-umlaut (ä).

For more information about the Unicode properties, please see » http://www.unicode.org/reports/tr21/.

See Also

- mb_strtolower() - Make a string lowercase
- mb_strtoupper() - Make a string uppercase
- strtolower() - Make a string lowercase
- strtoupper() - Make a string uppercase
- ucfirst() - Make a string's first character uppercase
- ucwords() - Uppercase the first character of each word in a string

mb_convert_encoding

mb_convert_encoding

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_convert_encoding — Convert a string from one character encoding to another

Description

function **mb_convert_encoding**(array|string `$string`, string `$to_encoding`, array|string|null `$from_encoding` = **`null`**): array|string|false

Converts `string` from `from_encoding`, or the current internal encoding, to `to_encoding`. If `string` is an array, all its string values will be converted recursively.

Parameters

`string`

The string or array to be converted.

`to_encoding`

The desired encoding of the result.

`from_encoding`

The current encoding used to interpret `string`. Multiple encodings may be specified as an array or comma separated list, in which case the correct encoding will be guessed using the same algorithm as mb_detect_encoding().

If `from_encoding` is omitted or **`null`**, the mbstring.internal_encoding setting will be used if set, otherwise the default_charset setting.

See supported encodings for valid values of `to_encoding` and `from_encoding`.

Return Values

The encoded string or array on success, or `false` on failure.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if the value of `to_encoding` or `from_encoding` is an invalid encoding. Prior to PHP 8.0.0, a `E_WARNING` was emitted instead.

Changelog

Version	Description
8.2.0	**mb_convert_encoding()** will no longer return the following non text encodings: `"Base64"`, `"QPrint"`, `"UUencode"`, `"HTML entities"`, `"7 bit"` and `"8 bit"`.
8.0.0	**mb_convert_encoding()** will now throw a ValueError when `to_encoding` is passed an invalid encoding.
8.0.0	**mb_convert_encoding()** will now throw a ValueError when `from_encoding` is passed an invalid encoding.
8.0.0	`from_encoding` is nullable now.
7.2.0	This function now also accepts an array as `string`. Formerly, only strings have been supported.

Examples

Example #1 mb_convert_encoding() example

```php
<?php
/* Convert internal character encoding to SJIS */
$str = mb_convert_encoding($str, "SJIS");

/* Convert EUC-JP to UTF-7 */
$str = mb_convert_encoding($str, "UTF-7", "EUC-JP");

/* Auto detect encoding from JIS, eucjp-win, sjis-win, then convert str to UCS-2LE */
$str = mb_convert_encoding($str, "UCS-2LE", "JIS, eucjp-win, sjis-win");

/* If mbstring.language is "Japanese", "auto" is expanded to "ASCII,JIS,UTF-8,EUC-JP,SJIS" */
$str = mb_convert_encoding($str, "EUC-JP", "auto");
?>
```

See Also

- mb_detect_order() - Set/Get character encoding detection order
- UConverter::transcode() - Convert a string from one character encoding to another
- iconv() - Convert a string from one character encoding to another

mb_convert_kana

mb_convert_kana

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_convert_kana — Convert "kana" one from another ("zen-kaku", "han-kaku" and more)

Description

function **mb_convert_kana**(string `$string`, string `$mode` = "KV", ?string `$encoding` = `null`): string

Performs a "han-kaku" - "zen-kaku" conversion for string `string`. This function is only useful for Japanese.

Parameters

`string`

The string being converted.

`mode`

The conversion option.

Specify with a combination of following options.

Applicable Conversion Options

Option	Meaning
r	Convert "zen-kaku" alphabets to "han-kaku"
R	Convert "han-kaku" alphabets to "zen-kaku"
n	Convert "zen-kaku" numbers to "han-kaku"
N	Convert "han-kaku" numbers to "zen-kaku"
a	Convert "zen-kaku" alphabets and numbers to "han-kaku"
A	Convert "han-kaku" alphabets and numbers to "zen-kaku" (Characters included in "a", "A" options are U+0021 - U+007E excluding U+0022, U+0027, U+005C, U+007E)
s	Convert "zen-kaku" space to "han-kaku" (U+3000 -> U+0020)
S	Convert "han-kaku" space to "zen-kaku" (U+0020 -> U+3000)
k	Convert "zen-kaku kata-kana" to "han-kaku kata-kana"
K	Convert "han-kaku kata-kana" to "zen-kaku kata-kana"
h	Convert "zen-kaku hira-gana" to "han-kaku kata-kana"
H	Convert "han-kaku kata-kana" to "zen-kaku hira-gana"
c	Convert "zen-kaku kata-kana" to "zen-kaku hira-gana"
C	Convert "zen-kaku hira-gana" to "zen-kaku kata-kana"
V	Collapse voiced sound notation and convert them into a character. Use with "K","H"

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

The converted string.

Errors/Exceptions

Throws a ValueError if the combination of different `modes` is invalid. For example `"sS"`.

Changelog

Version	Description
8.2.0	A ValueError is now thrown if the combination of different `modes` is invalid.
8.0.0	`encoding` is nullable now.

Examples

Example #1 mb_convert_kana() example

```php
<?php
/* Convert all "han-kaku" "kata-kana" to "zen-kaku" "hira-gana" */
echo mb_convert_kana('ﾔﾏﾀﾞ ﾊﾅｺ', "HV") . "\n";

/* Convert "han-kaku" "kata-kana" to "zen-kaku" "kata-kana"
   and "zen-kaku" alphanumeric to "han-kaku" */
echo mb_convert_kana('ｺｳｻﾞﾊﾞﾝｺﾞｳ ０１２３４５６', "KVa") . "\n";
?>
```

The above example will output:

```
やまだ はなこ
コウザバンゴウ 0123456
```

mb_convert_variables

mb_convert_variables

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_convert_variables — Convert character code in variable(s)

Description

function **mb_convert_variables**(
  string `$to_encoding`,
  array|string `$from_encoding`,
  mixed `&$var`,
  mixed `&...$vars`
): string|false

Converts character encoding of variables `var` and `vars` in encoding `from_encoding` to encoding `to_encoding`.

mb_convert_variables() join strings in Array or Object to detect encoding, since encoding detection tends to fail for short strings. Therefore, it is impossible to mix encoding in single array or object.

Parameters

`to_encoding`

The encoding that the string is being converted to.

`from_encoding`

`from_encoding` is specified as an array or comma separated string, it tries to detect encoding from `from-coding`. When `from_encoding` is omitted, `detect_order` is used.

`var`

`var` is the reference to the variable being converted. String, Array and Object are accepted. **mb_convert_variables()** assumes all parameters have the same encoding.

`vars`

Additional `vars`.

Return Values

The character encoding before conversion for success, or `false` for failure.

Examples

Example #1 mb_convert_variables() example

```php
<?php
/* Convert variables $post1, $post2 to internal encoding */
$interenc = mb_internal_encoding();
$inputenc = mb_convert_variables($interenc, "ASCII,UTF-8,SJIS-win", $post1, $post2);
?>
```

mb_decode_mimeheader

mb_decode_mimeheader

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_decode_mimeheader — Decode string in MIME header field

Description

function **mb_decode_mimeheader**(string `$string`): string

Decodes encoded-word string `string` in MIME header.

Parameters

`string`

The string being decoded.

Return Values

The decoded string in internal character encoding.

Changelog

Version	Description
8.3.0	Underscores are converted to spaces as specified by » RFC 2047.

See Also

- mb_encode_mimeheader() - Encode string for MIME header

mb_decode_numericentity

mb_decode_numericentity

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_decode_numericentity — Decode HTML numeric string reference to character

Description

function **mb_decode_numericentity**(string `$string`, array `$map`, ?string `$encoding` = **`null`**): string

Convert numeric string reference of string `string` in a specified block to character.

Parameters

`string`

The string being decoded.

`map`

`map` is an array that specifies the code area to convert.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or **`null`**, the internal character encoding value will be used.

Return Values

The converted string.

Errors/Exceptions

Throws a ValueError if `map` contains any value that is not an int, float, bool, **`null`**, or a numeric string.

Changelog

Version	Description
8.4.0	The **mb_decode_numericentity()** function now throws a ValueError if `map` contains any value that cannot be implicitly converted to int.
8.0.0	`encoding` is nullable now.

Examples

Example #1 `map` example

```
<?php

$convmap = array(
```

```
    int start_code1, int end_code1, int offset1, int mask1,
    int start_code2, int end_code2, int offset2, int mask2,
    // ........
    int start_codeN, int end_codeN, int offsetN, int maskN
);
// Specify Unicode value for start_codeN and end_codeN
// Add offsetN to value and take bit-wise 'AND' with maskN,
// then convert value to numeric string reference.
?>
```

Example #2 `map` example escapes JavaScript string

```
<?php

function escape_javascript_string($str)
{
    $map = [
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,0,0, // 49
        0,0,0,0,0,0,0,0,1,1,
        1,1,1,1,1,0,0,0,0,0,
        0,0,0,0,0,0,0,0,0,0,
        0,0,0,0,0,0,0,0,0,0,
        0,1,1,1,1,1,1,0,0,0, // 99
        0,0,0,0,0,0,0,0,0,0,
        0,0,0,0,0,0,0,0,0,0,
        0,0,0,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1, // 149
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1, // 199
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1,
        1,1,1,1,1,1,1,1,1,1, // 249
        1,1,1,1,1,1,1, // 255
    ];

    // Char encoding is UTF-8
    $mblen = mb_strlen($str, 'UTF-8');
    $utf32 = bin2hex(mb_convert_encoding($str, 'UTF-32', 'UTF-8'));

    for ($i=0, $encoded=''; $i < $mblen; $i++) {
        $u = substr($utf32, $i * 8, 8);
        $v = base_convert($u, 16, 10);

        if ($v < 256 && $map[$v]) {
            $encoded .= '\\x' . substr($u, 6,2);
        } else if ($v == 2028) {
            $encoded .= '\\u2028';
        } else if ($v == 2029) {
            $encoded .= '\\u2029';
        } else {
            $encoded .= mb_convert_encoding(hex2bin($u), 'UTF-8', 'UTF-32');
        }
    }

    return $encoded;
}

// Test data
$convmap = [ 0x0, 0xffff, 0, 0xffff ];
$msg = '';

for ($i=0; $i < 1000; $i++) {
```

```
  // chr() cannot generate correct UTF-8 data larger value than 128, use mb_decode_numericentity().
  $msg .= mb_decode_numericentity('&#' . $i . ';', $convmap, 'UTF-8');
}

// var_dump($msg);
var_dump(escape_javascript_string($msg));
?>
```

See Also

- mb_encode_numericentity() - Encode character to HTML numeric string reference

mb_detect_encoding

mb_detect_encoding

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_detect_encoding — Detect character encoding

Description

function **mb_detect_encoding**(string `$string`, array|string|null `$encodings = null`, bool `$strict = false`): string|false

Detects the most likely character encoding for string `string` from a list of candidates.

As of PHP 8.1 this function uses heuristics to detect which of the valid text encodings in the specified list is most likely to be correct and may not be in order of `encodings` provided.

Automatic detection of the intended character encoding can never be entirely reliable; without some additional information, it is similar to decoding an encrypted string without the key. It is always preferable to use an indication of character encoding stored or transmitted with the data, such as a "Content-Type" HTTP header.

This function is most useful with multibyte encodings, where not all sequences of bytes form a valid string. If the input string contains such a sequence, that encoding will be rejected.

> **Warning**
>
> **The result is not accurate**
>
> The name of this function is misleading, it performs "guessing" rather than "detection".
>
> The guesses are far from accurate, and therefore you cannot use this function to accurately detect the correct character encoding.

Parameters

`string`

The string being inspected.

`encodings`

A list of character encodings to try. The list may be specified as an array of strings, or a single string separated by commas.

If `encodings` is omitted or `null`, the current detect_order (set with the mbstring.detect_order configuration option, or mb_detect_order() function) will be used.

`strict`

Controls the behaviour when `string` is not valid in any of the listed `encodings`. If `strict` is set to `false`, the closest matching encoding will be returned; if `strict` is set to `true`, `false` will be returned.

The default value for `strict` can be set with the mbstring.strict_detection configuration option.

Return Values

The detected character encoding, or `false` if the string is not valid in any of the listed encodings.

Changelog

Version	Description
8.2.0	**mb_detect_encoding()** will no longer return the following non text encodings: `"Base64"`, `"QPrint"`, `"UUencode"`, `"HTML entities"`, `"7 bit"` and `"8 bit"`.

Examples

Example #1 mb_detect_encoding() example

```php
<?php

$str = "\x95\xB6\x8E\x9A\x83\x52\x81\x5B\x83\x68";

// Detect character encoding with current detect_order
var_dump(mb_detect_encoding($str));

// "auto" is expanded according to mbstring.language
var_dump(mb_detect_encoding($str, "auto"));

// Specify "encodings" parameter by list separated by comma
var_dump(mb_detect_encoding($str, "JIS, eucjp-win, sjis-win"));

// Use array to specify "encodings" parameter
$encodings = [
  "ASCII",
  "JIS",
  "EUC-JP"
];
var_dump(mb_detect_encoding($str, $encodings));
?>
```

The above example will output:

```
string(5) "ASCII"
string(5) "ASCII"
string(8) "SJIS-win"
string(5) "ASCII"
```

Example #2 Effect of `strict` parameter

```php
<?php
// 'áéóú' encoded in ISO-8859-1
$str = "\xE1\xE9\xF3\xFA";

// The string is not valid ASCII or UTF-8, but UTF-8 is considered a closer match
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8'], false));
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8'], true));

// If a valid encoding is found, the strict parameter does not change the result
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8', 'ISO-8859-1'], false));
var_dump(mb_detect_encoding($str, ['ASCII', 'UTF-8', 'ISO-8859-1'], true));
?>
```

The above example will output:

```
string(5) "UTF-8"
bool(false)
string(10) "ISO-8859-1"
string(10) "ISO-8859-1"
```

In some cases, the same sequence of bytes may form a valid string in multiple character encodings, and it is impossible to know which interpretation was intended. For instance, among many others, the byte sequence "\xC4\xA2" could be:

- "Ä¢" (U+00C4 LATIN CAPITAL LETTER A WITH DIAERESIS followed by U+00A2 CENT SIGN) encoded in any of ISO-8859-1, ISO-8859-15, or Windows-1252
- "ФЂ" (U+0424 CYRILLIC CAPITAL LETTER EF followed by U+0402 CYRILLIC CAPITAL LETTER DJE) encoded in ISO-8859-5
- "Ģ" (U+0122 LATIN CAPITAL LETTER G WITH CEDILLA) encoded in UTF-8

Example #3 Effect of order when multiple encodings match

```
<?php
$str = "\xC4\xA2";

// The string is valid in all three encodings, but the first one listed may not always be the one returned
var_dump(mb_detect_encoding($str, ['UTF-8']));
var_dump(mb_detect_encoding($str, ['UTF-8', 'ISO-8859-1', 'ISO-8859-5'])); // as of php8.1 this returns ISO-8859-1 instead of
UTF-8
var_dump(mb_detect_encoding($str, ['ISO-8859-1', 'ISO-8859-5', 'UTF-8']));
var_dump(mb_detect_encoding($str, ['ISO-8859-5', 'UTF-8', 'ISO-8859-1']));
?>
```

The above example will output:

```
string(5) "UTF-8"
string(10) "ISO-8859-1"
string(10) "ISO-8859-1"
string(10) "ISO-8859-5"
```

See Also

- mb_detect_order() - Set/Get character encoding detection order

mb_detect_order

mb_detect_order

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_detect_order — Set/Get character encoding detection order

Description

function **mb_detect_order**(array|string|null `$encoding` = `null`): array|bool

Sets the automatic character encoding detection order to `encoding`.

Parameters

`encoding`

`encoding` is an array or comma separated list of character encoding. See supported encodings.

If `encoding` is omitted or `null`, it returns the current character encoding detection order as array.

This setting affects mb_detect_encoding() and mb_send_mail().

`mbstring` currently implements the following encoding detection filters. If there is an invalid byte sequence for the following encodings, encoding detection will fail.

`UTF-8`, `UTF-7`, `ASCII`, `EUC-JP`,`SJIS`, `eucJP-win`, `SJIS-win`, `JIS`, `ISO-2022-JP`

For `ISO-8859-*`, `mbstring` always detects as `ISO-8859-*`.

For `UTF-16`, `UTF-32`, `UCS2` and `UCS4`, encoding detection will fail always.

Return Values

When setting the encoding detection order, `true` is returned on success or `false` on failure.

When getting the encoding detection order, an ordered array of the encodings is returned.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.

Examples

Example #1 mb_detect_order() examples

```
<?php
/* Set detection order by enumerated list */
mb_detect_order("eucjp-win,sjis-win,UTF-8");

/* Set detection order by array */
$ary[] = "ASCII";
$ary[] = "JIS";
$ary[] = "EUC-JP";
mb_detect_order($ary);

/* Display current detection order */
echo implode(", ", mb_detect_order());
?>
```

Example #2 Example showing useless detect orders

```
; Always detect as ISO-8859-1
detect_order = ISO-8859-1, UTF-8

; Always detect as UTF-8, since ASCII/UTF-7 values are
; valid for UTF-8
detect_order = UTF-8, ASCII, UTF-7
```

See Also

- mb_internal_encoding() - Set/Get internal character encoding
- mb_http_input() - Detect HTTP input character encoding
- mb_http_output() - Set/Get HTTP output character encoding
- mb_send_mail() - Send encoded mail

mb_encode_mimeheader

mb_encode_mimeheader

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_encode_mimeheader — Encode string for MIME header

Description

```
function mb_encode_mimeheader(
    string $string,
    ?string $charset = null,
    ?string $transfer_encoding = null,
    string $newline = "\r\n",
    int $indent = 0
): string
```

Encodes a given string `string` by the MIME header encoding scheme.

Parameters

`string`

The string being encoded. Its encoding should be same as mb_internal_encoding().

`charset`

`charset` specifies the name of the character set in which `string` is represented in. The default value is determined by the current NLS setting (`mbstring.language`).

`transfer_encoding`

`transfer_encoding` specifies the scheme of MIME encoding. It should be either `"B"` (Base64) or `"Q"` (Quoted-Printable). Falls back to `"B"` if not given.

`newline`

`newline` specifies the EOL (end-of-line) marker with which **mb_encode_mimeheader()** performs line-folding (a » RFC term, the act of breaking a line longer than a certain length into multiple lines. The length is currently hard-coded to 74 characters). Falls back to `"\r\n"` (CRLF) if not given.

`indent`

Indentation of the first line (number of characters in the header before `string`).

Return Values

A converted version of the string represented in ASCII.

Changelog

Version	Description
8.3.0	`NUL` (0) bytes are no longer dropped when encoded using Quoted-Printable encoding, but encoded as `=00`.
8.0.0	`charset` and `transfer_encoding` are nullable now.

Examples

Example #1 mb_encode_mimeheader() example

```php
<?php
$name = "太郎"; // kanji
$mbox = "kru";
$doma = "gtinn.mon";
$addr = '"' . addcslashes(mb_encode_mimeheader($name, "UTF-7", "Q"), '"') . '" <' . $mbox . "@" . $doma . ">";
echo $addr;
?>
```

The above example will output:

```
"=?UTF-7?Q?+WSqQzg-?=" <kru@gtinn.mon>
```

Notes

> **Note** :
>
> This function isn't designed to break lines at higher-level contextual break points (word boundaries, etc.). This behaviour may clutter up the original string with unexpected spaces.

See Also

- mb_decode_mimeheader() - Decode string in MIME header field

mb_encode_numericentity

mb_encode_numericentity

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_encode_numericentity — Encode character to HTML numeric string reference

Description

function **mb_encode_numericentity**(
  string `$string`,
  array `$map`,
  ?string `$encoding` = `null`,
  bool `$hex` = `false`
): string

Converts specified character codes in string `string` from character code to HTML numeric character reference.

Parameters

`string`

The string being encoded.

`map`

`map` is array specifies code area to convert.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

`hex`

Whether the returned entity reference should be in hexadecimal notation (otherwise it is in decimal notation).

Return Values

The converted string.

Errors/Exceptions

Throws a ValueError if `map` contains any value that is not an int, float, bool, `null`, or a numeric string.

Changelog

Version	Description
8.4.0	The **mb_encode_numericentity()** function now throws a ValueError if `map` contains any value that cannot be implicitly converted to int.
8.0.0	`encoding` is nullable now.

Examples

Example #1 `map` example

```
<?php

$convmap = array(
    int start_code1, int end_code1, int offset1, int mask1,
    int start_code2, int end_code2, int offset2, int mask2,
    // ........
    int start_codeN, int end_codeN, int offsetN, int maskN
);
// Specify Unicode value for start_codeN and end_codeN
// Add offsetN to value and take bit-wise 'AND' with maskN,
// then converts value to numeric string reference.
?>
```

Example #2 mb_encode_numericentity() example

```php
<?php

$str = "aAæÆあア鮎";

/* Convert all UTF8 characters up to 4 bytes to HTML numeric character reference */
$convmap = [0, 0x1FFFFF, 0, 0x10FFFF];
var_dump(mb_encode_numericentity($str, $convmap, "utf8"));

/* Converts only 2-byte and 4-byte UTF8 characters to HTML numeric character reference */
$convmap = [
    0x80, 0x7FF, 0, 0x10FFFF,
    0x10000, 0x1FFFFF, 0, 0x10FFFF,
];

var_dump(mb_encode_numericentity($str, $convmap, "utf8"));
?>
```

The above example will output:

```
string(46) "&#97;&#65;&#230;&#198;&#12354;&#12450;&#40509;"
string(28) "aA&#230;&#198;あア&#40509;"
```

See Also

- mb_decode_numericentity() - Decode HTML numeric string reference to character

mb_encoding_aliases

mb_encoding_aliases

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

mb_encoding_aliases — Get aliases of a known encoding type

Description

function **mb_encoding_aliases**(string `$encoding`): array

Returns an array of aliases for a known `encoding` type.

Parameters

`encoding`

The encoding type being checked, for aliases.

Return Values

Returns a numerically indexed array of encoding aliases.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if `encoding` is an invalid encoding. Prior to PHP 8.0.0, an `E_WARNING` was emitted instead.

Changelog

Version	Description
8.0.0	Now throws a ValueError if `encoding` is an invalid encoding. Previously, an `E_WARNING` was emitted and `false` was returned.

Examples

Example #1 mb_encoding_aliases() example

```php
<?php
$encoding        = 'ASCII';
$known_encodings = mb_list_encodings();

if (in_array($encoding, $known_encodings)) {

    $aliases = mb_encoding_aliases($encoding);
    print_r($aliases);

} else {

    echo "Unknown ($encoding) encoding.\n";

}
?>
```

The above example will output something similar to:

```
Array
(
    [0] => ANSI_X3.4-1968
    [1] => iso-ir-6
    [2] => ANSI_X3.4-1986
    [3] => ISO_646.irv:1991
    [4] => US-ASCII
    [5] => ISO646-US
    [6] => us
    [7] => IBM367
    [8] => cp367
    [9] => csASCII
)
```

See Also

- mb_list_encodings() - Returns an array of all supported encodings

mb_ereg

mb_ereg

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg — Regular expression match with multibyte support

Description

function **mb_ereg**(string `$pattern`, string `$string`, array `&$matches = null`): bool

Executes the regular expression match with multibyte support.

Parameters

`pattern`

The search pattern.

`string`

The search string.

`matches`

If matches are found for parenthesized substrings of `pattern` and the function is called with the third argument `matches`, the matches will be stored in the elements of the array `matches`. If no matches are found, `matches` is set to an empty array.

$matches[1] will contain the substring which starts at the first left parenthesis; *$matches[2]* will contain the substring starting at the second, and so on. *$matches[0]* will contain a copy of the complete string matched.

Return Values

Returns whether `pattern` matches `string`.

Changelog

Version	Description
8.0.0	This function returns `true` on success now. Previously, it returned the byte length of the matched string if a match for `pattern` was found in `string` and `matches` was passed. If the optional parameter `matches` was not passed or the length of the matched string was `0`, this function returned `1`.
7.1.0	**mb_ereg()** will now set `matches` to an empty array, if nothing matched. Formerly, `matches` was not modified in that case.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_eregi() - Regular expression match ignoring case with multibyte support

mb_ereg_match

mb_ereg_match

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_match — Regular expression match for multibyte string

Description

function **mb_ereg_match**(string `$pattern`, string `$string`, ?string `$options = null`): bool

A regular expression match for a multibyte string

> **Note** :
>
> `pattern` is only matched at the beginning of `string`.

Parameters

`pattern`

The regular expression pattern.

`string`

The string being evaluated.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

Returns `true` if `string` matches the regular expression `pattern`, `false` if not.

Changelog

Version	Description
8.0.0	`options` is nullable now.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg() - Regular expression match with multibyte support

mb_ereg_replace

mb_ereg_replace

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_replace — Replace regular expression with multibyte support

Description

```
function mb_ereg_replace(
    string $pattern,
    string $replacement,
    string $string,
    ?string $options = null
): string|false|null
```

Scans `string` for matches to `pattern`, then replaces the matched text with `replacement`

Parameters

`pattern`

The regular expression pattern.

Multibyte characters may be used in `pattern`.

`replacement`

The replacement text.

`string`

The string being checked.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

The resultant string on success, or **`false`** on error. If `string` is not valid for the current encoding, **`null`** is returned.

Changelog

Version	Description
8.0.0	`options` is nullable now.
7.1.0	The function checks whether `string` is valid for the current encoding.
7.1.0	The `e` modifier has been deprecated.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

> **Warning**
>
> Never use the `e` modifier when working on untrusted input. No automatic escaping will happen (as known from preg_replace()). Not taking care of this will most likely create remote code execution vulnerabilities in your application.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_eregi_replace() - Replace regular expression with multibyte support ignoring case

mb_ereg_replace_callback

mb_ereg_replace_callback

(PHP 5 >= 5.4.1, PHP 7, PHP 8)

mb_ereg_replace_callback — Perform a regular expression search and replace with multibyte support using a callback

Description

```
function mb_ereg_replace_callback(
    string $pattern,
    callable $callback,
    string $string,
    ?string $options = null
): string|false|null
```

Scans `string` for matches to `pattern`, then replaces the matched text with the output of `callback` function.

The behavior of this function is almost identical to mb_ereg_replace(), except for the fact that instead of `replacement` parameter, one should specify a `callback`.

Parameters

`pattern`

The regular expression pattern.

Multibyte characters may be used in `pattern`.

`callback`

A callback that will be called and passed an array of matched elements in the `string` string. The callback should return the replacement string.

You'll often need the `callback` function for a **mb_ereg_replace_callback()** in just one place. In this case you can use an anonymous function to declare the callback within the call to **mb_ereg_replace_callback()**. By doing it this way you have all information for the call in one place and do not clutter the function namespace with a callback function's name not used anywhere else.

`string`

The string being checked.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

The resultant string on success, or **`false`** on error. If `string` is not valid for the current encoding, **`null`** is returned.

Changelog

Version	Description
8.0.0	`options` is nullable now.
7.1.0	The function checks whether `string` is valid for the current encoding.

Examples

Example #1 **mb_ereg_replace_callback()** example

```php
<?php
// this text was used in 2002
// we want to get this up to date for 2003
$text = "April fools day is 04/01/2002\n";
$text.= "Last christmas was 12/24/2001\n";
// the callback function
function next_year($matches)
{
  // as usual: $matches[0] is the complete match
  // $matches[1] the match for the first subpattern
  // enclosed in '(...)' and so on
  return $matches[1].($matches[2]+1);
}
echo mb_ereg_replace_callback(
            "(\d{2}/\d{2}/)(\d{4})",
            "next_year",
            $text);

?>
```

The above example will output:

```
April fools day is 04/01/2003
Last christmas was 12/24/2002
```

Example #2 **mb_ereg_replace_callback()** using anonymous function

```php
<?php
// this text was used in 2002
// we want to get this up to date for 2003
$text = "April fools day is 04/01/2002\n";
$text.= "Last christmas was 12/24/2001\n";

echo mb_ereg_replace_callback(
            "(\d{2}/\d{2}/)(\d{4})",
            function ($matches) {
               return $matches[1].($matches[2]+1);
            },
            $text);
?>
```

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_replace() - Replace regular expression with multibyte support
- Anonymous functions

mb_ereg_search

mb_ereg_search

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search — Multibyte regular expression match for predefined multibyte string

Description

function **mb_ereg_search**(?string `$pattern = null`, ?string `$options = null`): bool

Performs a multibyte regular expression match for a predefined multibyte string.

Parameters

`pattern`

The search pattern.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

mb_ereg_search() returns `true` if the multibyte string matches with the regular expression, or `false` otherwise. The string for matching is set by mb_ereg_search_init(). If `pattern` is not specified, the previous one is used.

Changelog

Version	Description
8.0.0	`pattern` and `options` are nullable now.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_search_init() - Setup string and regular expression for a multibyte regular expression match

mb_ereg_search_getpos

mb_ereg_search_getpos

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_getpos — Returns start point for next regular expression match

Description

function **mb_ereg_search_getpos**(): int

Returns the start point for the next regular expression match.

Parameters

This function has no parameters.

Return Values

mb_ereg_search_getpos() returns the point to start regular expression match for mb_ereg_search(), mb_ereg_search_pos(), mb_ereg_search_regs(). The position is represented by bytes from the head of string.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_search_setpos() - Set start point of next regular expression match

mb_ereg_search_getregs

mb_ereg_search_getregs

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_getregs — Retrieve the result from the last multibyte regular expression match

Description

function **mb_ereg_search_getregs**(): array|false

Retrieve the result from the last multibyte regular expression match

Parameters

This function has no parameters.

Return Values

An array including the sub-string of matched part by last mb_ereg_search(), mb_ereg_search_pos(), mb_ereg_search_regs(). If there are some matches, the first element will have the matched sub-string, the second element will have the first part grouped with brackets, the third element will have the second part grouped with brackets, and so on. It returns `false` on error.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_search_init() - Setup string and regular expression for a multibyte regular expression match

mb_ereg_search_init

mb_ereg_search_init

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_init — Setup string and regular expression for a multibyte regular expression match

Description

function **mb_ereg_search_init**(string `$string`, ?string `$pattern` = `null`, ?string `$options` = `null`): bool

mb_ereg_search_init() sets `string` and `pattern` for a multibyte regular expression. These values are used for mb_ereg_search(), mb_ereg_search_pos(), and mb_ereg_search_regs().

Parameters

`string`

The search string.

`pattern`

The search pattern.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.0.0	`pattern` and `options` are nullable now.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_search_regs() - Returns the matched part of a multibyte regular expression

mb_ereg_search_pos

mb_ereg_search_pos

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_pos — Returns position and length of a matched part of the multibyte regular expression for a predefined multibyte string

Description

function **mb_ereg_search_pos**(?string `$pattern = null`, ?string `$options = null`): array|false

Returns position and length of a matched part of the multibyte regular expression for a predefined multibyte string

The string for match is specified by mb_ereg_search_init(). If it is not specified, the previous one will be used.

Parameters

`pattern`

The search pattern.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

An array containing two elements. The first element is the offset, in bytes, where the match begins relative to the start of the search string, and the second element is the length in bytes of the match.

If an error occurs, `false` is returned.

Changelog

Version	Description
8.0.0	`pattern` and `options` are nullable now.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_search_init() - Setup string and regular expression for a multibyte regular expression match

mb_ereg_search_regs

mb_ereg_search_regs

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_regs — Returns the matched part of a multibyte regular expression

Description

function **mb_ereg_search_regs**(?string `$pattern = null`, ?string `$options = null`): array|false

Returns the matched part of a multibyte regular expression.

Parameters

`pattern`

The search pattern.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

mb_ereg_search_regs() executes the multibyte regular expression match, and if there are some matched part, it returns an array including substring of matched part as first element, the first grouped part with brackets as second element, the second grouped part as third element, and so on. It returns `false` on error.

Changelog

Version	Description
8.0.0	`pattern` and `options` are nullable now.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_search_init() - Setup string and regular expression for a multibyte regular expression match

mb_ereg_search_setpos

mb_ereg_search_setpos

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_ereg_search_setpos — Set start point of next regular expression match

Description

function **mb_ereg_search_setpos**(int `$offset`): bool

mb_ereg_search_setpos() sets the starting point of a match for mb_ereg_search().

Parameters

`offset`

The position to set. If it is negative, it counts from the end of the string.

Return Values

Returns **`true`** on success or **`false`** on failure.

Changelog

Version	Description
7.1.0	Support for negative `offset`s has been added.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_search_init() - Setup string and regular expression for a multibyte regular expression match

mb_eregi

mb_eregi

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_eregi — Regular expression match ignoring case with multibyte support

Description

function **mb_eregi**(string `$pattern`, string `$string`, array `&$matches = null`): bool

Executes the case insensitive regular expression match with multibyte support.

Parameters

`pattern`

The regular expression pattern.

`string`

The string being searched.

`matches`

If matches are found for parenthesized substrings of `pattern` and the function is called with the third argument `matches`, the matches will be stored in the elements of the array `matches`. If no matches are found, `matches` is set to an empty array.

$matches[1] will contain the substring which starts at the first left parenthesis; *$matches[2]* will contain the substring starting at the second, and so on. *$matches[0]* will contain a copy of the complete string matched.

Return Values

Returns whether `pattern` matches `string`.

Changelog

Version	Description
8.0.0	This function returns `true` on success now. Previously, it returned the byte length of the matched string if a match for `pattern` was found in `string` and `matches` was passed. If the optional parameter `matches` was not passed or the length of the matched string was `0`, this function returned `1`.
7.1.0	**mb_eregi()** will now set `matches` to an empty array, if nothing matched. Formerly, `matches` was not modified in that case.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg() - Regular expression match with multibyte support

mb_eregi_replace

mb_eregi_replace

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_eregi_replace — Replace regular expression with multibyte support ignoring case

Description

```
function mb_eregi_replace(
    string $pattern,
    string $replacement,
    string $string,
    ?string $options = null
): string|false|null
```

Scans `string` for matches to `pattern`, then replaces the matched text with `replacement`.

Parameters

`pattern`

The regular expression pattern. Multibyte characters may be used. The case will be ignored.

`replacement`

The replacement text.

`string`

The searched string.

`options`

The search option. See mb_regex_set_options() for explanation.

Return Values

The resultant string or `false` on error. If `string` is not valid for the current encoding, `null` is returned.

Changelog

Version	Description
8.0.0	`options` is nullable now.
7.1.0	The function checks whether `string` is valid for the current encoding.
7.1.0	The `e` modifier has been deprecated.

Notes

> **Note** :
>
> The internal encoding or the character encoding specified by mb_regex_encoding() will be used as the character encoding for this function.

> **Warning**
>
> Never use the `e` modifier when working on untrusted input. No automatic escaping will happen (as known from preg_replace()). Not taking care of this will most likely create remote code execution vulnerabilities in your application.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg_replace() - Replace regular expression with multibyte support

mb_get_info

mb_get_info

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_get_info — Get internal settings of mbstring

Description

function **mb_get_info**(string `$type` = "all"): array|string|int|false|null

mb_get_info() returns the internal setting parameters of mbstring.

Parameters

`type`

If `type` is not specified or is specified as `"all"`, `"internal_encoding"`, `"http_input"`, `"http_output"`, `"http_output_conv_mimetypes"`, `"mail_charset"`, `"mail_header_encoding"`, `"mail_body_encoding"`, `"illegal_chars"`, `"encoding_translation"`, `"language"`, `"detect_order"`, `"substitute_character"` and `"strict_detection"` will be returned.

If `type` is specified as `"internal_encoding"`, `"http_input"`, `"http_output"`, `"http_output_conv_mimetypes"`, `"mail_charset"`, `"mail_header_encoding"`, `"mail_body_encoding"`, `"illegal_chars"`, `"encoding_translation"`, `"language"`, `"detect_order"`, `"substitute_character"` or `"strict_detection"` the specified setting parameter will be returned.

Return Values

An array of type information if `type` is not specified, otherwise a specific `type`, or **`false`** on failure.

Changelog

Version	Description
8.0.0	The `types` `"func_overload"` and `"func_overload_list"` are no longer supported.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_http_output() - Set/Get HTTP output character encoding

mb_http_input

mb_http_input

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_http_input — Detect HTTP input character encoding

Description

function **mb_http_input**(?string `$type` = `null`): array|string|false

Detects the HTTP input character encoding.

Parameters

`type`

> Input string specifies the input type. `"G"` for GET, `"P"` for POST, `"C"` for COOKIE, `"S"` for string, `"L"` for list, and `"I"` for the whole list (will return array). If type is omitted, it returns the last input type processed.

Return Values

The character encoding name, as per the `type`, or an array of character encoding names, if `type` is `"I"`. If **mb_http_input()** does not process specified HTTP input, it returns `false`.

Errors/Exceptions

Throws a ValueError if `type` is invalid.

Changelog

Version	Description
8.0.0	**mb_http_input()** now throws a ValueError if `type` is invalid. `type` is nullable now.

See Also

- mb_internal_encoding() - Set/Get internal character encoding
- mb_http_output() - Set/Get HTTP output character encoding
- mb_detect_order() - Set/Get character encoding detection order

mb_http_output

mb_http_output

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_http_output — Set/Get HTTP output character encoding

Description

function **mb_http_output**(?string `$encoding` = `null`): string|bool

Set/Get the HTTP output character encoding. Output after this function is called will be converted from the set internal encoding to `encoding`.

Parameters

`encoding`

> If `encoding` is set, **mb_http_output()** sets the HTTP output character encoding to `encoding`.
>
> If `encoding` is omitted, **mb_http_output()** returns the current HTTP output character encoding.

Return Values

If `encoding` is omitted, **mb_http_output()** returns the current HTTP output character encoding. Otherwise, Returns `true` on success or `false` on failure.

Errors/Exceptions

Throws a ValueError if `encoding` contains null bytes.

Changelog

Version	Description
8.4.0	**mb_http_output()** now throws a ValueError if `encoding` contains null bytes.
8.0.0	`encoding` is nullable now.

See Also

- mb_internal_encoding() - Set/Get internal character encoding
- mb_http_input() - Detect HTTP input character encoding
- mb_detect_order() - Set/Get character encoding detection order

mb_internal_encoding

mb_internal_encoding

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_internal_encoding — Set/Get internal character encoding

Description

function **mb_internal_encoding**(?string `$encoding = null`): string|bool

Set/Get the internal character encoding

Parameters

`encoding`

> `encoding` is the character encoding name used for the HTTP input character encoding conversion, HTTP output character encoding conversion, and the default character encoding for string functions defined by the mbstring module. You should notice that the internal encoding is totally different from the one for multibyte regex.

Return Values

If `encoding` is set, then Returns `true` on success or `false` on failure. In this case, the character encoding for multibyte regex is NOT changed. If `encoding` is omitted, then the current character encoding name is returned.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if `encoding` is an invalid encoding. Prior to PHP 8.0.0, an `E_WARNING` was emitted instead.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.
8.0.0	Now throws a ValueError if `encoding` is an invalid encoding. Previously, an `E_WARNING` was emitted and `false` was returned.

Examples

Example #1 **mb_internal_encoding()** example

```php
<?php
/* Set internal character encoding to UTF-8 */
mb_internal_encoding("UTF-8");

/* Display current internal character encoding */
echo mb_internal_encoding();
?>
```

See Also

- mb_http_input() - Detect HTTP input character encoding
- mb_http_output() - Set/Get HTTP output character encoding
- mb_detect_order() - Set/Get character encoding detection order
- mb_regex_encoding() - Set/Get character encoding for multibyte regex

mb_language

mb_language

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_language — Set/Get current language

Description

function **mb_language**(?string `$language` = `null`): string|bool

Set/Get the current language.

Parameters

`language`

Used for encoding e-mail messages. The valid languages are listed in the following table. mb_send_mail() uses this setting to encode e-mail.

Language	Charset	Encoding	Alias
German/de	ISO-8859-15	Quoted-Printable	Deutsch
English/en	ISO-8859-1	Quoted-Printable	
Armenian/hy	ArmSCII-8	Quoted-Printable	
Japanese/ja	ISO-2022-JP	BASE64	
Korean/ko	ISO-2022-KR	BASE64	
neutral	UTF-8	BASE64	
Russian/ru	KOI8-R	Quoted-Printable	
Turkish/tr	ISO-8859-9	Quoted-Printable	
Ukrainian/ua	KOI8-U	Quoted-Printable	
uni	UTF-8	BASE64	universal
Simplified Chinese/zh-cn	HZ	BASE64	
Traditional Chinese/zh-tw	BIG-5	BASE64	

Return Values

If `language` is set and `language` is valid, it returns `true`. Otherwise, it returns `false`. When `language` is omitted or `null`, it returns the language name as a string.

Changelog

Version	Description
8.0.0	`language` is nullable now.

See Also

- mb_send_mail() - Send encoded mail

mb_lcfirst

mb_lcfirst

(PHP 8 >= 8.4.0)

mb_lcfirst — Make a string's first character lowercase

Description

function **mb_lcfirst**(string `$string`, ?string `$encoding` = `null`): string

Performs a multi-byte safe lcfirst() operation, and returns a string with the first character of `string` lowercased.

Parameters

`string`

The input string.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the resulting string.

See Also

- mb_ucfirst() - Make a string's first character uppercase
- lcfirst() - Make a string's first character lowercase

mb_list_encodings

mb_list_encodings

(PHP 5, PHP 7, PHP 8)

mb_list_encodings — Returns an array of all supported encodings

Description

function **mb_list_encodings**(): array

Returns an array containing all supported encodings.

Parameters

This function has no parameters.

Return Values

Returns a numerically indexed array.

Errors/Exceptions

This function does not emit any errors.

Examples

Example #1 mb_list_encodings() example

```php
<?php

print_r(mb_list_encodings());

?>
```

The above example will output something similar to:

```
Array
(
    [0] => pass
    [1] => auto
    [2] => wchar
    [3] => byte2be
    [4] => byte2le
    [5] => byte4be
    [6] => byte4le
    [7] => BASE64
    [8] => UUENCODE
    [9] => HTML-ENTITIES
    [10] => Quoted-Printable
    [11] => 7bit
    [12] => 8bit
    [13] => UCS-4
```

```
    [14] => UCS-4BE
    [15] => UCS-4LE
    [16] => UCS-2
    [17] => UCS-2BE
    [18] => UCS-2LE
    [19] => UTF-32
    [20] => UTF-32BE
    [21] => UTF-32LE
    [22] => UTF-16
    [23] => UTF-16BE
    [24] => UTF-16LE
    [25] => UTF-8
    [26] => UTF-7
    [27] => UTF7-IMAP
    [28] => ASCII
    [29] => EUC-JP
    [30] => SJIS
    [31] => eucJP-win
    [32] => SJIS-win
    [33] => JIS
    [34] => ISO-2022-JP
    [35] => Windows-1252
    [36] => ISO-8859-1
    [37] => ISO-8859-2
    [38] => ISO-8859-3
    [39] => ISO-8859-4
    [40] => ISO-8859-5
    [41] => ISO-8859-6
    [42] => ISO-8859-7
    [43] => ISO-8859-8
    [44] => ISO-8859-9
    [45] => ISO-8859-10
    [46] => ISO-8859-13
    [47] => ISO-8859-14
    [48] => ISO-8859-15
    [49] => EUC-CN
    [50] => CP936
    [51] => HZ
    [52] => EUC-TW
    [53] => BIG-5
    [54] => EUC-KR
    [55] => UHC
    [56] => ISO-2022-KR
    [57] => Windows-1251
    [58] => CP866
    [59] => KOI8-R
)
```

See Also

- mb_encoding_aliases() - Get aliases of a known encoding type

mb_ltrim

mb_ltrim

(PHP 8 >= 8.4.0)

mb_ltrim — Strip whitespace (or other characters) from the beginning of a string

Description

function **mb_ltrim**(string `$string`, ?string `$characters = null`, ?string `$encoding = null`): string

Performs a multi-byte safe ltrim() operation. Strip whitespace (or other characters) from the beginning of a string.

Without the second parameter, **mb_ltrim()** will strip these characters:

- `" "` (Unicode U+0020), an ordinary space.
- `"\t"` (Unicode U+0009), a tab.
- `"\n"` (Unicode U+000A), a new line (line feed).
- `"\r"` (Unicode U+000D), a carriage return.
- `"\0"` (Unicode U+0000), the NUL-byte.
- `"\v"` (Unicode U+000B), a vertical tab.
- `"\f"` (Unicode U+000C), a form feed.
- `"\u00A0"` (Unicode U+00A0), a NO-BREAK SPACE.
- `"\u1680"` (Unicode U+1680), a OGHAM SPACE MARK.
- `"\u2000"` (Unicode U+2000), a EN QUAD.
- `"\u2001"` (Unicode U+2001), a EM QUAD.
- `"\u2002"` (Unicode U+2002), a EN SPACE.
- `"\u2003"` (Unicode U+2003), a EM SPACE.
- `"\u2004"` (Unicode U+2004), a THREE-PER-EM SPACE.
- `"\u2005"` (Unicode U+2005), a FOUR-PER-EM SPACE.
- `"\u2006"` (Unicode U+2006), a SIX-PER-EM SPACE.
- `"\u2007"` (Unicode U+2007), a FIGURE SPACE.
- `"\u2008"` (Unicode U+2008), a PUNCTUATION SPACE.
- `"\u2009"` (Unicode U+2009), a THIN SPACE.
- `"\u200A"` (Unicode U+200A), a HAIR SPACE.
- `"\u2028"` (Unicode U+2028), a LINE SEPARATOR.
- `"\u2029"` (Unicode U+2029), a PARAGRAPH SEPARATOR.
- `"\u202F"` (Unicode U+202F), a NARROW NO-BREAK SPACE.
- `"\u205F"` (Unicode U+205F), a MEDIUM MATHEMATICAL SPACE.
- `"\u3000"` (Unicode U+3000), a IDEOGRAPHIC SPACE.
- `"\u0085"` (Unicode U+0085), a NEXT LINE (NEL).
- `"\u180E"` (Unicode U+180E), a MONGOLIAN VOWEL SEPARATOR.

Parameters

`string`

The input string.

`characters`

Optionally, the stripped characters can also be specified using the `characters` parameter. Simply list all characters that need to be stripped.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or **`null`**, the internal character encoding value will be used.

Return Values

This function returns a string with whitespace stripped from the beginning of `string`.

See Also

- mb_trim() - Strip whitespace (or other characters) from the beginning and end of a string
- mb_rtrim() - Strip whitespace (or other characters) from the end of a string
- ltrim() - Strip whitespace (or other characters) from the beginning of a string

mb_ord

mb_ord

(PHP 7 >= 7.2.0, PHP 8)

mb_ord — Get Unicode code point of character

Description

function **mb_ord**(string `$string`, ?string `$encoding` = `null`): int|false

Returns the Unicode code point value of the given character.

This function complements mb_chr().

Parameters

`string`

A string

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

The Unicode code point for the first character of `string` or `false` on failure.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.

Examples

Example #1 A basic mb_ord() example

```
<?php
var_dump(mb_ord("A", "UTF-8"));
var_dump(mb_ord("🐘", "UTF-8"));
var_dump(mb_ord("\x80", "ISO-8859-1"));
var_dump(mb_ord("\x80", "Windows-1252"));
?>
```

The above example will output:

```
int(65)
int(128024)
int(128)
int(8364)
```

See Also

- mb_internal_encoding() - Set/Get internal character encoding

- mb_chr() - Return character by Unicode code point value
- IntlChar::ord() - Return Unicode code point value of character
- ord() - Convert the first byte of a string to a value between 0 and 255

mb_output_handler

mb_output_handler

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_output_handler — Callback function converts character encoding in output buffer

Description

function **mb_output_handler**(string `$string`, int `$status`): string

mb_output_handler() is ob_start() callback function. **mb_output_handler()** converts characters in the output buffer from internal character encoding to HTTP output character encoding.

Parameters

`string`

The contents of the output buffer.

`status`

The status of the output buffer.

Return Values

The converted string.

Examples

Example #1 mb_output_handler() example

```php
<?php
mb_http_output("UTF-8");
ob_start("mb_output_handler");
?>
```

Notes

> **Note** :
>
> If you want to output binary data, such as an image, a Content-Type: header must be set using header() before any binary data is sent to the client (e.g. header("Content-Type: image/png")). If Content-Type: header is sent, output character encoding conversion will not be performed.
>
> Note that if 'Content-Type: text/*' is sent, the content body is regarded as text; conversion will take place.

See Also

- ob_start() - Turn on output buffering

mb_parse_str

mb_parse_str

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_parse_str — Parse GET/POST/COOKIE data and set global variable

Description

function **mb_parse_str**(string `$string`, array `&$result`): bool

Parses GET/POST/COOKIE data and sets global variables. Since PHP does not provide raw POST/COOKIE data, it can only be used for GET data for now. It parses URL encoded data, detects encoding, converts coding to internal encoding and set values to the `result` array or global variables.

Parameters

`string`

The URL encoded data.

`result`

An array containing decoded and character encoded converted values.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.0.0	The second parameter was no longer optional.
7.2.0	Calling **mb_parse_str()** without the second parameter was deprecated.

See Also

- mb_detect_order() - Set/Get character encoding detection order
- mb_internal_encoding() - Set/Get internal character encoding

mb_preferred_mime_name

mb_preferred_mime_name

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_preferred_mime_name — Get MIME charset string

Description

function **mb_preferred_mime_name**(string `$encoding`): string|false

Get a MIME charset string for a specific encoding.

Parameters

`encoding`

The encoding being checked.

Return Values

The MIME `charset` string for character encoding `encoding`, or `false` if no charset is preferred for the given `encoding`.

Examples

Example #1 mb_preferred_mime_name() example

```php
<?php
$outputenc = "sjis-win";
mb_http_output($outputenc);
ob_start("mb_output_handler");
header("Content-Type: text/html; charset=" . mb_preferred_mime_name($outputenc));
?>
```

mb_regex_encoding

mb_regex_encoding

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_regex_encoding — Set/Get character encoding for multibyte regex

Description

function **mb_regex_encoding**(?string `$encoding` = `null`): string|bool

Set/Get character encoding for a multibyte regex.

Parameters

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

If `encoding` is set, then Returns `true` on success or `false` on failure. In this case, the internal character encoding is NOT changed. If `encoding` is omitted, then the current character encoding name for a multibyte regex is returned.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.

See Also

- mb_internal_encoding() - Set/Get internal character encoding
- mb_ereg() - Regular expression match with multibyte support

mb_regex_set_options

mb_regex_set_options

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_regex_set_options — Set/Get the default options for mbregex functions

Description

function **mb_regex_set_options**(?string `$options` = `null`): string

Sets the default options described by `options` for multibyte regex functions.

Parameters

`options`

The options to set. This is a string where each character is an option. To set a mode, the mode character must be the last one set, however there can only be set one mode but multiple options.

Regex options

Option	Meaning	Changelog
i	Ambiguity match on	
x	Enables extended pattern form	
m	`'.'` matches with newlines	
s	`'^' -> '\A', '$' -> '\Z'`	
p	Same as both the `m` and `s` options	
l	Finds longest matches	
n	Ignores empty matches	
e	eval() resulting code	Deprecated as of PHP 7.1.0 and removed as of PHP 8.0.0

> **Note** :
>
> The `"e"` option has no effect when set through **mb_regex_set_options()**. Use it with mb_ereg_replace() or mb_eregi_replace().

Regex syntax modes (only one may be set)

Mode	Meaning
j	Java (Sun java.util.regex)
u	GNU regex
g	grep
c	Emacs
r	Ruby
z	Perl
b	POSIX Basic regex
d	POSIX Extended regex

Return Values

The previous options. If `options` is omitted or `null`, it returns the string that describes the current options.

Changelog

Version	Description
8.0.0	If the parameter `options` is given and not `null`, the *previous* options are returned. Formerly, the *current* options have been returned.
8.0.0	`options` is nullable now.
8.0.0	The `"e"` option now throws a ValueError.
7.1.0	The `"e"` option now emits an `E_DEPRECATED`.

See Also

- mb_split() - Split multibyte string using regular expression
- mb_ereg() - Regular expression match with multibyte support
- mb_eregi() - Regular expression match ignoring case with multibyte support

mb_rtrim

mb_rtrim

(PHP 8 >= 8.4.0)

mb_rtrim — Strip whitespace (or other characters) from the end of a string

Description

function **mb_rtrim**(string `$string`, ?string `$characters = null`, ?string `$encoding = null`): string

Performs a multi-byte safe rtrim() operation, and returns a string with whitespace (or other characters) stripped from the end of `string`.

Without the second parameter, **mb_rtrim()** will strip these characters:

- `" "` (Unicode U+0020), an ordinary space.
- `"\t"` (Unicode U+0009), a tab.
- `"\n"` (Unicode U+000A), a new line (line feed).
- `"\r"` (Unicode U+000D), a carriage return.
- `"\0"` (Unicode U+0000), the NUL-byte.
- `"\v"` (Unicode U+000B), a vertical tab.
- `"\f"` (Unicode U+000C), a form feed.
- `"\u00A0"` (Unicode U+00A0), a NO-BREAK SPACE.
- `"\u1680"` (Unicode U+1680), a OGHAM SPACE MARK.

- `"\u2000"` (Unicode U+2000), a EN QUAD.
- `"\u2001"` (Unicode U+2001), a EM QUAD.
- `"\u2002"` (Unicode U+2002), a EN SPACE.
- `"\u2003"` (Unicode U+2003), a EM SPACE.
- `"\u2004"` (Unicode U+2004), a THREE-PER-EM SPACE.
- `"\u2005"` (Unicode U+2005), a FOUR-PER-EM SPACE.
- `"\u2006"` (Unicode U+2006), a SIX-PER-EM SPACE.
- `"\u2007"` (Unicode U+2007), a FIGURE SPACE.
- `"\u2008"` (Unicode U+2008), a PUNCTUATION SPACE.
- `"\u2009"` (Unicode U+2009), a THIN SPACE.
- `"\u200A"` (Unicode U+200A), a HAIR SPACE.
- `"\u2028"` (Unicode U+2028), a LINE SEPARATOR.
- `"\u2029"` (Unicode U+2029), a PARAGRAPH SEPARATOR.
- `"\u202F"` (Unicode U+202F), a NARROW NO-BREAK SPACE.
- `"\u205F"` (Unicode U+205F), a MEDIUM MATHEMATICAL SPACE.
- `"\u3000"` (Unicode U+3000), a IDEOGRAPHIC SPACE.
- `"\u0085"` (Unicode U+0085), a NEXT LINE (NEL).
- `"\u180E"` (Unicode U+180E), a MONGOLIAN VOWEL SEPARATOR.

Parameters

`string`

The input string.

`characters`

Optionally, the stripped characters can also be specified using the `characters` parameter. Simply list all characters that need to be stripped.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the modified string.

See Also

- mb_trim() - Strip whitespace (or other characters) from the beginning and end of a string
- mb_ltrim() - Strip whitespace (or other characters) from the beginning of a string
- rtrim() - Strip whitespace (or other characters) from the end of a string

mb_scrub

mb_scrub

(PHP 7 >= 7.2.0, PHP 8)

mb_scrub — Replace ill-formed byte sequences with the substitute character

Description

function **mb_scrub**(string `$string`, ?string `$encoding` = `null`): string

Perform a character set conversion from the specified encoding, or the default encoding if no encoding was specified, to the same encoding. This has the effect of replacing any invalid byte sequences with the substitute character.

Parameters

`string`

The input string.

`encoding`

The encoding used to interpret `string`. If it is omitted or `null`, the mbstring.internal_encoding setting will be used if set, otherwise the default_charset setting will be used.

Return Values

The string result with invalid byte sequences replaced.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.

Examples

Example #1 Byte-level replacement performed by mb_scrub()

bin2hex() is used here because terminals, browsers and fonts may render an ill-formed byte sequence with a replacement character of their own, which hides what the string actually contains.

```php
<?php

// The byte 0xFF cannot appear in a valid UTF-8 string.
$input = "A\xFFB";
echo bin2hex($input), "\n";

// The default substitute character is "?" (0x3F).
echo bin2hex(mb_scrub($input, 'UTF-8')), "\n";

// U+FFFD REPLACEMENT CHARACTER is encoded as EF BF BD in UTF-8.
mb_substitute_character(0xFFFD);
echo bin2hex(mb_scrub($input, 'UTF-8')), "\n";

?>
```

The above example will output:

```
41ff42
413f42
41efbfbd42
```

Example #2 Using mb_scrub() before UTF-8 aware processing

PCRE patterns using the `u` modifier reject subjects that are not well-formed UTF-8. Scrubbing the input first makes it acceptable.

```php
<?php

$input = "A\xFFB";

var_dump(preg_match_all('/./us', $input));
echo preg_last_error_msg(), "\n";

$clean = mb_scrub($input, 'UTF-8');
```

```
var_dump(preg_match_all('/./us', $clean));

?>
```

The above example will output:

```
bool(false)
Malformed UTF-8 characters, possibly incorrectly encoded
int(3)
```

See Also

- mb_substitute_character() - Set/Get substitution character
- mb_check_encoding() - Check if strings are valid for the specified encoding
- mb_convert_encoding() - Convert a string from one character encoding to another

mb_send_mail

mb_send_mail

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_send_mail — Send encoded mail

Description

```
function mb_send_mail(
    string $to,
    string $subject,
    string $message,
    array|string $additional_headers = [],
    ?string $additional_params = null
): bool
```

Sends email. Headers and messages are converted and encoded according to the mb_language() setting. It's a wrapper function for mail(), so see also mail() for details.

Parameters

`to`

The mail addresses being sent to. Multiple recipients may be specified by putting a comma between each address in `to`. This parameter is not automatically encoded.

`subject`

The subject of the mail.

`message`

The message of the mail.

`additional_headers` (optional)

String or array to be inserted at the end of the email header.

This is typically used to add extra headers (From, Cc, and Bcc). Multiple extra headers should be separated with a CRLF (\r\n). Validate parameter not to be injected unwanted headers by attackers.

If an array is passed, its keys are the header names and its values are the respective header values.

> **Note** :

> When sending mail, the mail *must* contain a `From` header. This can be set with the `additional_headers` parameter, or a default can be set in *php.ini*.
>
> Failing to do this will result in an error message similar to `Warning: mail(): "sendmail_from" not set in php.ini or custom "From:" header missing`. The `From` header sets also `Return-Path` under Windows.

> **Note** :
>
> If messages are not received, try using a LF (\n) only. Some Unix mail transfer agents (most notably » qmail) replace LF by CRLF automatically (which leads to doubling CR if CRLF is used). This should be a last resort, as it does not comply with » RFC 2822.

`additional_params`

`additional_params` is a MTA command line parameter. It is useful when setting the correct Return-Path header when using sendmail.

This parameter is escaped by escapeshellcmd() internally to prevent command execution. escapeshellcmd() prevents command execution, but allows to add additional parameters. For security reason, this parameter should be validated.

Since escapeshellcmd() is applied automatically, some characters that are allowed as email addresses by internet RFCs cannot be used. Programs that are required to use these characters mail() cannot be used.

The user that the webserver runs as should be added as a trusted user to the sendmail configuration to prevent a 'X-Warning' header from being added to the message when the envelope sender (-f) is set using this method. For sendmail users, this file is */etc/mail/trusted-users*.

Return Values

Returns **`true`** on success or **`false`** on failure.

Changelog

Version	Description
8.0.0	`additional_params` is nullable now.
7.2.0	The `additional_headers` parameter now also accepts an array.

See Also

- mail() - Send mail
- mb_encode_mimeheader() - Encode string for MIME header
- mb_language() - Set/Get current language

mb_split

mb_split

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

mb_split — Split multibyte string using regular expression

Description

function **mb_split**(string `$pattern`, string `$string`, int `$limit` = -1): array|false

Split a multibyte `string` using regular expression `pattern` and returns the result as an array.

Parameters

`pattern`

The regular expression pattern.

`string`

The string being split.

`limit`

If optional parameter `limit` is specified, it will be split in `limit` elements as maximum.

Return Values

The result as an array, or `false` on failure.

Notes

> **Note** :
>
> The character encoding specified by mb_regex_encoding() will be used as the character encoding for this function by default.

See Also

- mb_regex_encoding() - Set/Get character encoding for multibyte regex
- mb_ereg() - Regular expression match with multibyte support
- explode() - Split a string by a string

mb_str_pad

mb_str_pad

(PHP 8 >= PHP 8.3.0)

mb_str_pad — Pad a multibyte string to a certain length with another multibyte string

Description

```
function mb_str_pad(
    string $string,
    int $length,
    string $pad_string = " ",
    int $pad_type = STR_PAD_RIGHT,
    ?string $encoding = null
): string
```

This function returns the `string` padded on the left, the right, or both sides to the specified padding length, where the length is measured in Unicode codepoints. If the optional argument `pad_string` is not supplied, the `string` is padded with spaces, otherwise it is padded with characters from `pad_string` up to the limit.

Parameters

`string`

The input string.

`length`

If the value of `length` is negative, less than, or equal to the length of the input string, no padding takes place, and `string` will be returned.

`pad_string`

> **Note** :
>
> The `pad_string` may be truncated if the required number of padding characters can't be evenly divided by the `pad_string`'s length.

`pad_type`

Optional argument `pad_type` can be `STR_PAD_RIGHT`, `STR_PAD_LEFT`, or `STR_PAD_BOTH`. By default `STR_PAD_RIGHT`.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the padded string.

Examples

Example #1 mb_str_pad() example

```
<?php
var_dump(mb_str_pad('▶▶', 6, '❤❓✳', STR_PAD_RIGHT)); // string(18) "▶▶❤❓✳❤"
var_dump(mb_str_pad('▶▶', 6, '❤❓✳', STR_PAD_LEFT));  // string(18) "❤❓✳❤▶▶"
var_dump(mb_str_pad('▶▶', 6, '❤❓✳', STR_PAD_BOTH));  // string(18) "❤❓▶▶❤❓"

var_dump(mb_str_pad("🎉", 3, "祝", STR_PAD_LEFT));   // string(10) "祝祝🎉"
?>
```

mb_str_split

mb_str_split

(PHP 7 >= 7.4.0, PHP 8)

mb_str_split — Given a multibyte string, return an array of its characters

Description

function **mb_str_split**(string `$string`, int `$length` = 1, ?string `$encoding` = **`null`**): array

This function will return an array of strings, it is a version of str_split() with support for encodings of variable character size as well as fixed-size encodings of 1,2 or 4 byte characters. If the `length` parameter is specified, the string is broken down into chunks of the specified length in characters (not bytes). The `encoding` parameter can be optionally specified and it is good practice to do so.

Parameters

`string`

The string to split into characters or chunks.

`length`

If specified, each element of the returned array will be composed of multiple characters instead of a single character.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or **`null`**, the internal character encoding value will be used.

A string specifying one of the supported encodings.

Return Values

mb_str_split() returns an array of strings.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.
8.0.0	This function no longer returns **`false`** on failure.

See Also

- str_split() - Convert a string to an array
- grapheme_str_split() - Split a string into an array
- explode() - Split a string by a string

mb_strcut

mb_strcut

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strcut — Get part of string

Description

function **mb_strcut**(
  string $string,
  int $start,
  ?int $length = null,
  ?string $encoding = null
): string

mb_strcut() extracts a substring from a string similarly to mb_substr(), but operates on bytes instead of characters. If the cut position happens to be between two bytes of a multi-byte character, the cut is performed starting from the first byte of that character. This is also the difference to the substr() function, which would simply cut the string between the bytes and thus result in a malformed byte sequence.

Parameters

`string`

The string being cut.

`start`

If `start` is non-negative, the returned string will start at the `start`'th *byte* position in `string`, counting from zero. For instance, in the string '`abcdef`', the byte at position `0` is '`a`', the byte at position `2` is '`c`', and so forth.

If `start` is negative, the returned string will start at the `start`'th byte counting back from the end of `string`. However, if the magnitude of a negative `start` is greater than the length of the string, the returned portion will start from the beginning of `string`.

`length`

Length in *bytes*. If omitted or `null` is passed, extract all bytes to the end of the string.

If `length` is negative, the returned string will end at the `length`'th byte counting back from the end of `string`. However, if the magnitude of a negative `length` is greater than the number of characters after the `start` position, an empty string will be returned.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

mb_strcut() returns the portion of `string` specified by the `start` and `length` parameters.

Changelog

Version	Description
8.4.0	The behavior is more consistent now on invalid `UTF-8` and `UTF-16` strings.
8.0.0	`encoding` is nullable now.

See Also

- mb_substr() - Get part of string
- mb_internal_encoding() - Set/Get internal character encoding

mb_strimwidth

mb_strimwidth

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strimwidth — Get truncated string with specified width

Description

function **mb_strimwidth**(
  string $string,
  int $start,
  int $width,
  string $trim_marker = "",

```
    ?string $encoding = null
): string
```

Truncates string `string` to specified `width`, where halfwidth characters count as `1`, and fullwidth characters count as `2`. See » http://www.unicode.org/reports/tr11/ for details regarding East Asian character widths.

Parameters

`string`

The string being decoded.

`start`

The start position offset. Number of characters from the beginning of string (first character is 0), or if start is negative, number of characters from the end of the string.

`width`

The width of the desired trim. If negative width is specified, count from the end of the string.

> **Note** :
>
> Passing negative width is deprecated as of PHP 8.3.0.

`trim_marker`

A string that is added to the end of string when string is truncated.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

The truncated string. If `trim_marker` is set, `trim_marker` replaces the last chars to match the `width`.

Changelog

Version	Description
8.3.0	Passing a negative `width` to **mb_strimwidth()** is now deprecated.
8.0.0	`encoding` is nullable now.
7.1.0	Support for negative `starts` and `widths` has been added.

Examples

Example #1 **mb_strimwidth()** example

```php
<?php
echo mb_strimwidth("Hello World", 0, 10, "...");
// output: "Hello W..."
?>
```

See Also

- mb_strwidth() - Return width of string
- mb_internal_encoding() - Set/Get internal character encoding

mb_stripos

mb_stripos

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_stripos — Finds position of first occurrence of a string within another, case insensitive

Description

```
function mb_stripos(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?string $encoding = null
): int|false
```

mb_stripos() returns the numeric position of the first occurrence of `needle` in the `haystack` string. Unlike mb_strpos(), **mb_stripos()** is case-insensitive. If `needle` is not found, it returns `false`.

Parameters

`haystack`

The string from which to get the position of the first occurrence of `needle`

`needle`

The string to find in `haystack`

`offset`

The position in `haystack` to start searching. A negative offset counts from the end of the string.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Return the numeric position of the first occurrence of `needle` in the `haystack` string, or `false` if `needle` is not found.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	`encoding` is nullable now.
7.1.0	Support for negative `offsets` has been added.

See Also

- stripos() - Find the position of the first occurrence of a case-insensitive substring in a string
- strpos() - Find the position of the first occurrence of a substring in a string
- mb_strpos() - Find position of first occurrence of string in a string

mb_stristr

mb_stristr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_stristr — Finds first occurrence of a string within another, case insensitive

Description

```
function mb_stristr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_stristr() finds the first occurrence of `needle` in `haystack` and returns the portion of `haystack`. Unlike mb_strstr(), **mb_stristr()** is case-insensitive. If `needle` is not found, it returns `false`.

Parameters

`haystack`

The string from which to get the first occurrence of `needle`

`needle`

The string to find in `haystack`

`before_needle`

Determines which portion of `haystack` this function returns. If set to **`true`**, it returns all of `haystack` from the beginning to the first occurrence of `needle` (excluding needle). If set to **`false`**, it returns all of `haystack` from the first occurrence of `needle` to the end (including needle).

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or **`null`**, the internal character encoding value will be used.

Return Values

Returns the portion of `haystack`, or **`false`** if `needle` is not found.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	`encoding` is nullable now.

See Also

- stristr() - Case-insensitive strstr
- strstr() - Find the first occurrence of a string
- mb_strstr() - Finds first occurrence of a string within another

mb_strlen

mb_strlen

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strlen — Get string length

Description

function **mb_strlen**(string `$string`, ?string `$encoding` = **`null`**): int

Gets the length of a string.

Parameters

`string`

The string being checked for length.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or **`null`**, the internal character encoding value will be used.

Return Values

Returns the number of characters in string `string` having character encoding `encoding`. A multi-byte character is counted as 1.

Errors/Exceptions

As of PHP 8.0.0, a ValueError is thrown if `encoding` is an invalid encoding. Prior to PHP 8.0.0, an **`E_WARNING`** was emitted instead.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.
8.0.0	Now throws a ValueError if `encoding` is an invalid encoding. Previously, an `E_WARNING` was emitted and `false` was returned.

See Also

- mb_internal_encoding() - Set/Get internal character encoding
- grapheme_strlen() - Get string length in grapheme units
- iconv_strlen() - Returns the character count of string
- strlen() - Get string length

mb_strpos

mb_strpos

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strpos — Find position of first occurrence of string in a string

Description

```
function mb_strpos(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?string $encoding = null
): int|false
```

Finds position of the first occurrence of `needle` in the `haystack` string.

Performs a multi-byte safe strpos() operation based on number of characters. The first character's position is 0, the second character position is 1, and so on.

Parameters

`haystack`

The string from which to get the position of the first occurrence of `needle`.

`needle`

The string to find in `haystack`. In contrast with strpos(), numeric values are not applied as the ordinal value of a character.

`offset`

The search offset in characters. If it is not specified, 0 is used. A negative offset counts from the end of the string.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the character position (not byte position) of the first occurrence of `needle` in the `haystack` string, counting from `0`. Returns `false` if `needle` is not found.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	`encoding` is nullable now.
7.1.0	Support for negative `offsets` has been added.

Examples

Example #1 mb_strpos() returns a character offset, not a byte offset

```php
<?php
// "🐘" is 4 bytes but 1 character; "é" is 2 bytes but 1 character
$str = "🐘H🐘é";

var_dump(mb_strpos($str, "é"));  // character 3, not byte 9
var_dump(mb_strpos($str, "🐘", 1)); // character 2, skipping the first "🐘"
?>
```

The above example will output:

```
int(3)
int(2)
```

See Also

- mb_strrpos() - Find position of last occurrence of a string in a string
- mb_stripos() - Finds position of first occurrence of a string within another, case insensitive
- mb_internal_encoding() - Set/Get internal character encoding
- strpos() - Find the position of the first occurrence of a substring in a string

mb_strrchr

mb_strrchr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strrchr — Finds the last occurrence of a character in a string within another

Description

```
function mb_strrchr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_strrchr() finds the last occurrence of `needle` in `haystack` and returns the portion of `haystack`. If `needle` is not found, it returns `false`.

Parameters

`haystack`

The string from which to get the last occurrence of `needle`.

`needle`

The string to find in `haystack`.

`before_needle`

Determines which portion of `haystack` this function returns. If set to `true`, it returns all of `haystack` from the beginning to the last occurrence of `needle`. If set to `false`, it returns all of `haystack` from the last occurrence of `needle` to the end.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the portion of `haystack`. or `false` if `needle` is not found.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	`encoding` is nullable now.

See Also

- strrchr() - Find the last occurrence of a character in a string
- mb_strstr() - Finds first occurrence of a string within another
- mb_strrichr() - Finds the last occurrence of a character in a string within another, case insensitive

mb_strrichr

mb_strrichr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strrichr — Finds the last occurrence of a character in a string within another, case insensitive

Description

```
function mb_strrichr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_strrichr() finds the last occurrence of `needle` in `haystack` and returns the portion of `haystack`. Unlike mb_strrchr(), **mb_strrichr()** is case-insensitive. If `needle` is not found, it returns `false`.

Parameters

`haystack`

The string from which to get the last occurrence of `needle`.

`needle`

The string to find in `haystack`.

`before_needle`

Determines which portion of `haystack` this function returns. If set to `true`, it returns all of `haystack` from the beginning to the last occurrence of `needle`. If set to `false`, it returns all of `haystack` from the last occurrence of `needle` to the end.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the portion of `haystack`. or `false` if `needle` is not found.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	`encoding` is nullable now.

See Also

- mb_stristr() - Finds first occurrence of a string within another, case insensitive
- mb_strrchr() - Finds the last occurrence of a character in a string within another

mb_strripos

mb_strripos

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strripos — Finds position of last occurrence of a string within another, case insensitive

Description

```
function mb_strripos(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?string $encoding = null
): int|false
```

mb_strripos() performs multi-byte safe strripos() operation based on number of characters. `needle` position is counted from the beginning of `haystack`. First character's position is 0. Second character position is 1. Unlike mb_strrpos(), **mb_strripos()** is case-insensitive.

Parameters

`haystack`

The string from which to get the position of the last occurrence of `needle`.

`needle`

The string to find in `haystack`.

`offset`

May be specified to begin searching an arbitrary number of characters into the `haystack`. Negative values will stop searching at an arbitrary point prior to the end of the `haystack`.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Return the numeric position of the last occurrence of `needle` in the `haystack` string, or `false` if `needle` is not found.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	`encoding` is nullable now.

See Also

- strripos() - Find the position of the last occurrence of a case-insensitive substring in a string
- strrpos() - Find the position of the last occurrence of a substring in a string
- mb_strrpos() - Find position of last occurrence of a string in a string

mb_strrpos

mb_strrpos

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strrpos — Find position of last occurrence of a string in a string

Description

```
function mb_strrpos(
    string $haystack,
    string $needle,
    int $offset = 0,
    ?string $encoding = null
): int|false
```

Performs a multibyte safe strrpos() operation based on the number of characters. `needle` position is counted from the beginning of `haystack`. First character's position is 0. Second character position is 1.

Parameters

`haystack`

The string being checked, for the last occurrence of `needle`

`needle`

The string to find in `haystack`.

`offset`

May be specified to begin searching an arbitrary number of characters into the `haystack`. Negative values will stop searching at an arbitrary point prior to the end of the `haystack`.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the numeric position of the last occurrence of `needle` in the `haystack` string. If `needle` is not found, it returns `false`.

Errors/Exceptions

- If `offset` is greater than the length of `haystack`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	`needle` now accepts an empty string.
8.0.0	Passing the `encoding` as the third argument instead of an `offset` has been removed.
8.0.0	`encoding` is nullable now.

See Also

- mb_strpos() - Find position of first occurrence of string in a string
- mb_internal_encoding() - Set/Get internal character encoding
- strrpos() - Find the position of the last occurrence of a substring in a string

mb_strstr

mb_strstr

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

mb_strstr — Finds first occurrence of a string within another

Description

```
function mb_strstr(
    string $haystack,
    string $needle,
    bool $before_needle = false,
    ?string $encoding = null
): string|false
```

mb_strstr() finds the first occurrence of `needle` in `haystack` and returns the portion of `haystack`. If `needle` is not found, it returns `false`.

Parameters

`haystack`

The string from which to get the first occurrence of `needle`

`needle`

The string to find in `haystack`

`before_needle`

Determines which portion of `haystack` this function returns. If set to `true`, it returns all of `haystack` from the beginning to the first occurrence of `needle` (excluding needle). If set to `false`, it returns all of `haystack` from the first occurrence of `needle` to the end (including needle).

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

Returns the portion of `haystack`, or `false` if `needle` is not found.

Changelog

Version	Description
8.3.2	On invalid strings (those with encoding errors) and on `SJIS-mac` (MacJapanese) strings, the portion of `haystack` returned when `before_needle` is `true` is now determined from the same character indices as the other mbstring functions.
8.0.0	`needle` now accepts an empty string.
8.0.0	`encoding` is nullable now.

See Also

- stristr() - Case-insensitive strstr
- strstr() - Find the first occurrence of a string
- mb_stristr() - Finds first occurrence of a string within another, case insensitive

mb_strtolower

mb_strtolower

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_strtolower — Make a string lowercase

Description

function **mb_strtolower**(string `$string`, ?string `$encoding = null`): string

Returns `string` with all alphabetic characters converted to lowercase.

Parameters

`string`

The string being lowercased.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

`string` with all alphabetic characters converted to lowercase.

Changelog

Version	Description
8.3.0	Implemented conditional casing rules for the Greek letter sigma.

Examples

Example #1 mb_strtolower() example

```php
<?php
$str = "Mary Had A Little Lamb and She LOVED It So";
$str = mb_strtolower($str);
echo $str; // Prints mary had a little lamb and she loved it so
?>
```

Example #2 mb_strtolower() example with non-Latin UTF-8 text

```php
<?php
$str = "Τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός";
$str = mb_strtolower($str, 'UTF-8');
echo $str; // Prints τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός
?>
```

Notes

By contrast to strtolower(), 'alphabetic' is determined by the Unicode character properties. Thus the behaviour of this function is not affected by locale settings and it can convert any characters that have 'alphabetic' property, such as a-umlaut (ä).

For more information about the Unicode properties, please see » http://www.unicode.org/reports/tr21/.

See Also

- mb_strtoupper() - Make a string uppercase
- mb_convert_case() - Perform case folding on a string
- strtolower() - Make a string lowercase

mb_strtoupper

mb_strtoupper

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_strtoupper — Make a string uppercase

Description

function **mb_strtoupper**(string `$string`, ?string `$encoding` = `null`): string

Returns `string` with all alphabetic characters converted to uppercase.

Parameters

`string`

The string being uppercased.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

`string` with all alphabetic characters converted to uppercase.

Examples

Example #1 mb_strtoupper() example

```php
<?php
$str = "Mary Had A Little Lamb and She LOVED It So";
$str = mb_strtoupper($str);
```

```
echo $str; // Prints MARY HAD A LITTLE LAMB AND SHE LOVED IT SO
?>
```

Example #2 mb_strtoupper() example with non-Latin UTF-8 text

```
<?php
$str = "Τάχιστη αλώπηξ βαφής ψημένη γη, δρασκελίζει υπέρ νωθρού κυνός";
$str = mb_strtoupper($str, 'UTF-8');
echo $str; // Prints ΤΆΧΙΣΤΗ ΑΛΏΠΗΞ ΒΑΦΗΣ ΨΗΜΕΝΗ ΓΗ, ΔΡΑΣΚΕΛΊΖΕΙ ΥΠΈΡ ΝΩΘΡΟΎ ΚΥΝΌΣ
?>
```

Notes

By contrast to strtoupper(), 'alphabetic' is determined by the Unicode character properties. Thus the behaviour of this function is not affected by locale settings and it can convert any characters that have 'alphabetic' property, such as a-umlaut (ä).

For more information about the Unicode properties, please see » http://www.unicode.org/reports/tr21/.

See Also

- mb_strtolower() - Make a string lowercase
- mb_convert_case() - Perform case folding on a string
- strtoupper() - Make a string uppercase

mb_strwidth

mb_strwidth

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_strwidth — Return width of string

Description

function **mb_strwidth**(string `$string`, ?string `$encoding` = `null`): int

Returns the width of string `string`, where halfwidth characters count as `1`, and fullwidth characters count as `2`. See » http://www.unicode.org/reports/tr11/ for details regarding East Asian character widths.

The fullwidth characters are: `U+1100-U+115F`, `U+11A3-U+11A7`, `U+11FA-U+11FF`, `U+2329-U+232A`, `U+2E80-U+2E99`, `U+2E9B-U+2EF3`, `U+2F00-U+2FD5`, `U+2FF0-U+2FFB`, `U+3000-U+303E`, `U+3041-U+3096`, `U+3099-U+30FF`, `U+3105-U+312D`, `U+3131-U+318E`, `U+3190-U+31BA`, `U+31C0-U+31E3`, `U+31F0-U+321E`, `U+3220-U+3247`, `U+3250-U+32FE`, `U+3300-U+4DBF`, `U+4E00-U+A48C`, `U+A490-U+A4C6`, `U+A960-U+A97C`, `U+AC00-U+D7A3`, `U+D7B0-U+D7C6`, `U+D7CB-U+D7FB`, `U+F900-U+FAFF`, `U+FE10-U+FE19`, `U+FE30-U+FE52`, `U+FE54-U+FE66`, `U+FE68-U+FE6B`, `U+FF01-U+FF60`, `U+FFE0-U+FFE6`, `U+1B000-U+1B001`, `U+1F200-U+1F202`, `U+1F210-U+1F23A`, `U+1F240-U+1F248`, `U+1F250-U+1F251`, `U+20000-U+2FFFD`, `U+30000-U+3FFFD`. All other characters are halfwidth characters.

Parameters

`string`

The string being decoded.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

The width of string `string`.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.

Examples

Example #1 mb_strwidth() example

```php
<?php
var_dump(
    mb_strwidth('a'),       // LATIN SMALL LETTER A
    mb_strwidth("\u{ff41}") // FULLWIDTH LATIN SMALL LETTER A
);
?>
```

The above example will output:

```
int(1)
int(2)
```

See Also

- mb_strimwidth() - Get truncated string with specified width
- mb_internal_encoding() - Set/Get internal character encoding

mb_substitute_character

mb_substitute_character

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_substitute_character — Set/Get substitution character

Description

function **mb_substitute_character**(string|int|null `$substitute_character = null`): string|int|bool

Specifies a substitution character when input character encoding is invalid or character code does not exist in output character encoding. Invalid characters may be substituted `"none"` (no output), string or int value (Unicode character code value).

This setting affects mb_convert_encoding(), mb_convert_variables(), mb_output_handler(), mb_scrub(), and mb_send_mail().

Parameters

`substitute_character`

Specify the Unicode value as an int, or as one of the following strings:

- `"none"`: no output
- `"long"`: Output character code value (Example: `U+3000`, `JIS+7E7E`)
- `"entity"`: Output character entity (Example: `Ȁ`)

Return Values

If `substitute_character` is set, it returns **`true`** for success, otherwise returns **`false`**. If `substitute_character` is not set, it returns the current setting.

Changelog

Version	Description
8.0.0	Passing an empty string to `substitute_character` is no longer supported; `"none"` should be passed instead.
8.0.0	`encoding` is nullable now.

Examples

Example #1 mb_substitute_character() example

```php
<?php
/* Set with Unicode U+3013 (GETA MARK) */
```

```php
mb_substitute_character(0x3013);

/* Set hex format */
mb_substitute_character("long");

/* Display current setting */
echo mb_substitute_character();
?>
```

mb_substr

mb_substr

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

mb_substr — Get part of string

Description

```
function mb_substr(
    string $string,
    int $start,
    ?int $length = null,
    ?string $encoding = null
): string
```

Performs a multi-byte safe substr() operation based on number of characters. Position is counted from the beginning of `string`. First character's position is 0. Second character position is 1, and so on.

Parameters

`string`

The string to extract the substring from.

`start`

If `start` is non-negative, the returned string will start at the `start`'th position in `string`, counting from zero. For instance, in the string '`abcdef`', the character at position `0` is '`a`', the character at position `2` is '`c`', and so forth.

If `start` is negative, the returned string will start at the `start`'th character from the end of `string`.

`length`

Maximum number of characters to use from `string`. If omitted or `null` is passed, extract all characters to the end of the string.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

mb_substr() returns the portion of `string` specified by the `start` and `length` parameters.

Changelog

Version	Description
8.3.2	On invalid strings (those with encoding errors), character indices are now interpreted in the same manner as most other mbstring functions. This means that character indices returned by mb_strpos() can be passed directly. For `SJIS-mac` (MacJapanese) strings, character indices now refer to the indices of the Unicode codepoints produced when the string is converted to Unicode, which differ from the character indices for the `SJIS-mac` characters that convert to a sequence of several codepoints.
8.0.0	`encoding` is nullable now.

See Also

- mb_strcut() - Get part of string
- mb_internal_encoding() - Set/Get internal character encoding

mb_substr_count

mb_substr_count

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

mb_substr_count — Count the number of substring occurrences

Description

function **mb_substr_count**(string `$haystack`, string `$needle`, ?string `$encoding = null`): int

Counts the number of times the `needle` substring occurs in the `haystack` string.

Parameters

`haystack`

The string being checked.

`needle`

The string being found.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

The number of times the `needle` substring occurs in the `haystack` string.

Changelog

Version	Description
8.0.0	`encoding` is nullable now.

Examples

Example #1 mb_substr_count() example

```
<?php
echo mb_substr_count("This is a test", "is"); // prints out 2
?>
```

See Also

- mb_strpos() - Find position of first occurrence of string in a string
- mb_substr() - Get part of string
- substr_count() - Count the number of substring occurrences

mb_trim

mb_trim

(PHP 8 >= 8.4.0)

mb_trim — Strip whitespace (or other characters) from the beginning and end of a string

Description

function **mb_trim**(string `$string`, ?string `$characters = null`, ?string `$encoding = null`): string

Performs a multi-byte safe trim() operation, and returns a string with whitespace stripped from the beginning and end of `string`. Without the second parameter, **mb_trim()** will strip these characters:

- `" "` (Unicode U+0020), an ordinary space.
- `"\t"` (Unicode U+0009), a tab.
- `"\n"` (Unicode U+000A), a new line (line feed).

- `"\r"` (Unicode U+000D), a carriage return.
- `"\0"` (Unicode U+0000), the NUL-byte.
- `"\v"` (Unicode U+000B), a vertical tab.
- `"\f"` (Unicode U+000C), a form feed.
- `"\u00A0"` (Unicode U+00A0), a NO-BREAK SPACE.
- `"\u1680"` (Unicode U+1680), a OGHAM SPACE MARK.
- `"\u2000"` (Unicode U+2000), a EN QUAD.
- `"\u2001"` (Unicode U+2001), a EM QUAD.
- `"\u2002"` (Unicode U+2002), a EN SPACE.
- `"\u2003"` (Unicode U+2003), a EM SPACE.
- `"\u2004"` (Unicode U+2004), a THREE-PER-EM SPACE.
- `"\u2005"` (Unicode U+2005), a FOUR-PER-EM SPACE.
- `"\u2006"` (Unicode U+2006), a SIX-PER-EM SPACE.
- `"\u2007"` (Unicode U+2007), a FIGURE SPACE.
- `"\u2008"` (Unicode U+2008), a PUNCTUATION SPACE.
- `"\u2009"` (Unicode U+2009), a THIN SPACE.
- `"\u200A"` (Unicode U+200A), a HAIR SPACE.
- `"\u2028"` (Unicode U+2028), a LINE SEPARATOR.
- `"\u2029"` (Unicode U+2029), a PARAGRAPH SEPARATOR.
- `"\u202F"` (Unicode U+202F), a NARROW NO-BREAK SPACE.
- `"\u205F"` (Unicode U+205F), a MEDIUM MATHEMATICAL SPACE.
- `"\u3000"` (Unicode U+3000), a IDEOGRAPHIC SPACE.
- `"\u0085"` (Unicode U+0085), a NEXT LINE (NEL).
- `"\u180E"` (Unicode U+180E), a MONGOLIAN VOWEL SEPARATOR.

Parameters

`string`

The string that will be trimmed.

`characters`

Optionally, the stripped characters can also be specified using the `characters` parameter. Simply list all characters that need to be stripped.

`encoding`

The `encoding` parameter is the character encoding. If it is omitted or `null`, the internal character encoding value will be used.

Return Values

The trimmed string.

See Also

- mb_ltrim() - Strip whitespace (or other characters) from the beginning of a string
- mb_rtrim() - Strip whitespace (or other characters) from the end of a string
- trim() - Strip whitespace (or other characters) from the beginning and end of a string

mb_ucfirst

mb_ucfirst

(PHP 8 >= 8.4.0)

mb_ucfirst — Make a string's first character uppercase

Description

function **mb_ucfirst**(string `$string`, ?string `$encoding` = `null`): string

Performs a multi-byte safe ucfirst() operation, and returns a string with the first character of `string` title-cased.

Parameters

`string`

The input string.

`encoding`

The string encoding.

Return Values

Returns the resulting string.

Notes

> **Note** :
>
> By contrast to the standard case folding functions such as strtolower() and strtoupper(), case folding is performed on the basis of the Unicode character properties. Thus the behaviour of this function is not affected by locale settings and it can convert any characters that have 'alphabetic' property, such a-umlaut (ä).

For more information about the Unicode properties, please see » http://www.unicode.org/reports/tr21/.

See Also

- mb_lcfirst() - Make a string's first character lowercase
- mb_convert_case() - Perform case folding on a string
- ucfirst() - Make a string's first character uppercase

Character type checking

Introduction

The functions provided by this extension check whether a character or string falls into a certain character class according to the current locale (see also setlocale()).

When called with an integer argument these functions behave exactly like their C counterparts from *ctype.h*. It means that if an integer smaller than 256 is passed, it will use the ASCII value of it to see if it fits in the specified range (digits are in 0x30-0x39). If the number is between -128 and -1 inclusive then 256 will be added and the check will be done on that.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

When called with a string argument they will check every character in the string and will only return `true` if every character in the string matches the requested criteria. When called with an empty string the result will always be `false`.

Passing anything else but a string or integer will return `false` immediately.

It should be noted that ctype functions are always preferred over regular expressions, and even to some equivalent `"str_*"` and `"is_*"` functions. This is because of the fact that ctype uses a native C library and thus processes significantly faster.

> **Note** :
>
> These functions are not related to the Python "ctypes" library at all. The extension name stems from the *ctype.h* C header file that their C equivalents are defined in.
>
> This extension also predates Python "ctypes" so any confusion caused by this naming is hardly PHP's fault.

Installing/Configuring

Table of Contents

Requirements

None besides functions from the standard C library which are always available.

Installation

This extension is enabled by default. It may be disabled by using the following option at compile time: **--disable-ctype**

The Windows version of PHP has built-in support for this extension. You do not need to load any additional extensions in order to use these functions.

Ctype Functions

Table of Contents

ctype_alnum

ctype_alnum

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_alnum — Check for alphanumeric character(s)

Description

function **ctype_alnum**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are alphanumeric.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is either a letter or a digit, `false` otherwise. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_alnum() example (using the default locale)

```php
<?php
$strings = array('AbCd1zyZ9', 'foo!#$bar');
foreach ($strings as $testcase) {
    if (ctype_alnum($testcase)) {
        echo "The string $testcase consists of all letters or digits.\n";
    } else {
        echo "The string $testcase does not consist of all letters or digits.\n";
    }
}
?>
```

The above example will output:

```
The string AbCd1zyZ9 consists of all letters or digits.
The string foo!#$bar does not consist of all letters or digits.
```

See Also

- ctype_alpha() - Check for alphabetic character(s)
- ctype_digit() - Check for numeric character(s)
- setlocale() - Set locale information
- IntlChar::isalnum() - Check if code point is an alphanumeric character

ctype_alpha

ctype_alpha

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_alpha — Check for alphabetic character(s)

Description

function **ctype_alpha**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are alphabetic. In the standard `C` locale letters are just `[A-Za-z]` and **ctype_alpha()** is equivalent to `(ctype_upper($text) || ctype_lower($text))` if $text is just a single character, but other languages have letters that are considered neither upper nor lower case.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is a letter from the current locale, `false` otherwise. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_alpha() example (using the default locale)

```php
<?php
$strings = array('KjgWZC', 'arf12');
foreach ($strings as $testcase) {
    if (ctype_alpha($testcase)) {
        echo "The string $testcase consists of all letters.\n";
    } else {
        echo "The string $testcase does not consist of all letters.\n";
    }
}
?>
```

The above example will output:

```
The string KjgWZC consists of all letters.
The string arf12 does not consist of all letters.
```

See Also

- ctype_upper() - Check for uppercase character(s)
- ctype_lower() - Check for lowercase character(s)
- setlocale() - Set locale information
- IntlChar::isalpha() - Check if code point is a letter character

ctype_cntrl

ctype_cntrl

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_cntrl — Check for control character(s)

Description

function **ctype_cntrl**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are control characters. Control characters are e.g. line feed, tab, escape.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is a control character from the current locale, `false` otherwise. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_cntrl() example

```php
<?php
$strings = array('string1' => "\n\r\t", 'string2' => 'arf12');
foreach ($strings as $name => $testcase) {
    if (ctype_cntrl($testcase)) {
        echo "The string '$name' consists of all control characters.\n";
    } else {
        echo "The string '$name' does not consist of all control characters.\n";
    }
}
?>
```

The above example will output:

```
The string 'string1' consists of all control characters.
The string 'string2' does not consist of all control characters.
```

See Also

- ctype_print() - Check for printable character(s)
- IntlChar::iscntrl() - Check if code point is a control character

ctype_digit

ctype_digit

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_digit — Check for numeric character(s)

Description

function **ctype_digit**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are numerical.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in the string `text` is a decimal digit, `false` otherwise. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_digit() example

```php
<?php
$strings = array('1820.20', '10002', 'wsl!12');
foreach ($strings as $testcase) {
    if (ctype_digit($testcase)) {
        echo "The string $testcase consists of all digits.\n";
    } else {
        echo "The string $testcase does not consist of all digits.\n";
    }
}
?>
```

The above example will output:

```
The string 1820.20 does not consist of all digits.
The string 10002 consists of all digits.
The string wsl!12 does not consist of all digits.
```

Example #2 A ctype_digit() example comparing strings with integers

```php
<?php

$numeric_string = '42';
$integer        = 42;

ctype_digit($numeric_string);  // true
ctype_digit($integer);         // false (ASCII 42 is the * character)

is_numeric($numeric_string);   // true
is_numeric($integer);          // true
?>
```

See Also

- ctype_alnum() - Check for alphanumeric character(s)
- ctype_xdigit() - Check for character(s) representing a hexadecimal digit
- is_numeric() - Finds whether a variable is a number or a numeric string
- is_int() - Find whether the type of a variable is integer
- is_string() - Find whether the type of a variable is string
- IntlChar::isdigit() - Check if code point is a digit character

ctype_graph

ctype_graph

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_graph — Check for any printable character(s) except space

Description

function **ctype_graph**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, creates visible output.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is printable and actually creates visible output (no white space), `false` otherwise. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_graph() example

```php
<?php
$strings = array('string1' => "asdf\n\r\t", 'string2' => 'arf12', 'string3' => 'LKA#@%.54');
foreach ($strings as $name => $testcase) {
    if (ctype_graph($testcase)) {
```

```
        echo "The string '$name' consists of all (visibly) printable characters.\n";
    } else {
        echo "The string '$name' does not consist of all (visibly) printable characters.\n";
    }
}
?>
```

The above example will output:

```
The string 'string1' does not consist of all (visibly) printable characters.
The string 'string2' consists of all (visibly) printable characters.
The string 'string3' consists of all (visibly) printable characters.
```

See Also

- ctype_alnum() - Check for alphanumeric character(s)
- ctype_print() - Check for printable character(s)
- ctype_punct() - Check for any printable character which is not whitespace or an alphanumeric character
- IntlChar::isgraph() - Check if code point is a graphic character

ctype_lower

ctype_lower

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_lower — Check for lowercase character(s)

Description

function **ctype_lower**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are lowercase letters.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is a lowercase letter in the current locale. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_lower() example (using the default locale)

```php
<?php
$strings = array('aac123', 'qiutoas', 'QASsdks');
foreach ($strings as $testcase) {
    if (ctype_lower($testcase)) {
        echo "The string $testcase consists of all lowercase letters.\n";
    } else {
        echo "The string $testcase does not consist of all lowercase letters.\n";
    }
}
?>
```

The above example will output:

```
The string aac123 does not consist of all lowercase letters.
The string qiutoas consists of all lowercase letters.
The string QASsdks does not consist of all lowercase letters.
```

See Also

- ctype_alpha() - Check for alphabetic character(s)
- ctype_upper() - Check for uppercase character(s)
- setlocale() - Set locale information
- IntlChar::islower() - Check if code point is a lowercase letter

ctype_print

ctype_print

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_print — Check for printable character(s)

Description

function **ctype_print**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are printable.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns **`true`** if every character in `text` will actually create output (including blanks). Returns **`false`** if `text` contains control characters or characters that do not have any output or control function at all. When called with an empty string the result will always be **`false`**.

Examples

Example #1 A ctype_print() example

```php
<?php
$strings = array('string1' => "asdf\n\r\t", 'string2' => 'arf12', 'string3' => 'LKA#@%.54');
foreach ($strings as $name => $testcase) {
    if (ctype_print($testcase)) {
        echo "The string '$name' consists of all printable characters.\n";
    } else {
        echo "The string '$name' does not consist of all printable characters.\n";
    }
}
?>
```

The above example will output:

```
The string 'string1' does not consist of all printable characters.
The string 'string2' consists of all printable characters.
The string 'string3' consists of all printable characters.
```

See Also

- ctype_cntrl() - Check for control character(s)
- ctype_graph() - Check for any printable character(s) except space
- ctype_punct() - Check for any printable character which is not whitespace or an alphanumeric character
- IntlChar::isprint() - Check if code point is a printable character

ctype_punct

ctype_punct

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_punct — Check for any printable character which is not whitespace or an alphanumeric character

Description

function **ctype_punct**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are punctuation character.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is printable, but neither letter, digit or blank, `false` otherwise. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_punct() example

```
<?php
$strings = array('ABasdk!@!$#', '!@ # $', '*&$()');
foreach ($strings as $testcase) {
    if (ctype_punct($testcase)) {
        echo "The string $testcase consists of all punctuation.\n";
    } else {
        echo "The string $testcase does not consist of all punctuation.\n";
    }
}
?>
```

The above example will output:

```
The string ABasdk!@!$# does not consist of all punctuation.
The string !@ # $ does not consist of all punctuation.
The string *&$() consists of all punctuation.
```

See Also

- ctype_cntrl() - Check for control character(s)
- ctype_graph() - Check for any printable character(s) except space
- IntlChar::ispunct() - Check if code point is punctuation character

ctype_space

ctype_space

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_space — Check for whitespace character(s)

Description

function **ctype_space**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, creates whitespace.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` creates some sort of white space, `false` otherwise. Besides the blank character this also includes tab, vertical tab, line feed, carriage return and form feed characters. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_space() example

```php
<?php
$strings = array(
    'string1' => "\n\r\t",
    'string2' => "\narf12",
    'string3' => '\n\r\t' // note the single quotes
);
foreach ($strings as $name => $testcase) {
    if (ctype_space($testcase)) {
        echo "The string '$name' consists of whitespace characters only.\n";
    } else {
        echo "The string '$name' contains non-whitespace characters.\n";
    }
}
?>
```

The above example will output:

```
The string 'string1' consists of whitespace characters only.
The string 'string2' contains non-whitespace characters.
The string 'string3' contains non-whitespace characters.
```

See Also

- ctype_cntrl() - Check for control character(s)
- ctype_graph() - Check for any printable character(s) except space
- ctype_punct() - Check for any printable character which is not whitespace or an alphanumeric character
- IntlChar::isspace() - Check if code point is a space character

ctype_upper

ctype_upper

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_upper — Check for uppercase character(s)

Description

function **ctype_upper**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are uppercase characters.

Parameters

`text`

The tested string.

> **Note** :
>
> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is an uppercase letter in the current locale. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_upper() example (using the default locale)

```
<?php
$strings = array('AKLWC139', 'LMNSDO', 'akwSKWsm');
foreach ($strings as $testcase) {
    if (ctype_upper($testcase)) {
        echo "The string $testcase consists of all uppercase letters.\n";
    } else {
        echo "The string $testcase does not consist of all uppercase letters.\n";
    }
}
?>
```

The above example will output:

```
The string AKLWC139 does not consist of all uppercase letters.
The string LMNSDO consists of all uppercase letters.
The string akwSKWsm does not consist of all uppercase letters.
```

See Also

- ctype_alpha() - Check for alphabetic character(s)
- ctype_lower() - Check for lowercase character(s)
- setlocale() - Set locale information
- IntlChar::isupper() - Check if code point has the general category "Lu" (uppercase letter)

ctype_xdigit

ctype_xdigit

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

ctype_xdigit — Check for character(s) representing a hexadecimal digit

Description

function **ctype_xdigit**(mixed `$text`): bool

Checks if all of the characters in the provided string, `text`, are hexadecimal 'digits'.

Parameters

`text`

The tested string.

> **Note** :

> If an int between -128 and 255 inclusive is provided, it is interpreted as the ASCII value of a single character (negative values have 256 added in order to allow characters in the Extended ASCII range). Any other integer is interpreted as a string containing the decimal digits of the integer.

> **Warning**
>
> As of PHP 8.1.0, passing a non-string argument is deprecated. In the future, the argument will be interpreted as a string instead of an ASCII codepoint. Depending on the intended behavior, the argument should either be cast to string or an explicit call to chr() should be made.

Return Values

Returns `true` if every character in `text` is a hexadecimal 'digit', that is a decimal digit or a character from `[A-Fa-f]`, `false` otherwise. When called with an empty string the result will always be `false`.

Examples

Example #1 A ctype_xdigit() example

```php
<?php
$strings = array('AB10BC99', 'AR1012', 'ab12bc99');
foreach ($strings as $testcase) {
    if (ctype_xdigit($testcase)) {
        echo "The string $testcase consists of all hexadecimal digits.\n";
    } else {
        echo "The string $testcase does not consist of all hexadecimal digits.\n";
    }
}
?>
```

The above example will output:

```
The string AB10BC99 consists of all hexadecimal digits.
The string AR1012 does not consist of all hexadecimal digits.
The string ab12bc99 consists of all hexadecimal digits.
```

See Also

- ctype_digit() - Check for numeric character(s)
- IntlChar::isxdigit() - Check if code point is a hexadecimal digit

URLs

Introduction

Dealing with URL strings: encoding, decoding and parsing.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

The following constants are meant to be used with parse_url().

PHP_URL_SCHEME (int)

PHP_URL_HOST (int)

Outputs the hostname of the URL parsed.

PHP_URL_PORT (int)

Outputs the port of the URL parsed.

PHP_URL_USER (int)

Outputs the user of the URL parsed.

PHP_URL_PASS (int)

Outputs the password of the URL parsed.

PHP_URL_PATH (int)

Outputs the path of the URL parsed.

PHP_URL_QUERY (int)

Outputs the query string of the URL parsed.

PHP_URL_FRAGMENT (int)

Outputs the fragment (string after the hashmark #) of the URL parsed.

The following constants are meant to be used with http_build_query().

PHP_QUERY_RFC1738 (int)

Encoding is performed per » RFC 1738 and the `application/x-www-form-urlencoded` media type, which implies that spaces are encoded as plus (+) signs.

PHP_QUERY_RFC3986 (int)

Encoding is performed according to » RFC 3986, and spaces will be percent encoded (`%20`).

URL Functions

Table of Contents

base64_decode

base64_decode

(PHP 4, PHP 5, PHP 7, PHP 8)

base64_decode — Decodes data encoded with MIME base64

Description

function **base64_decode**(string `$string`, bool `$strict` = `false`): string|false

Decodes a base64 encoded `string`.

Parameters

`string`

The encoded data.

`strict`

If the `strict` parameter is set to `true` then the **base64_decode()** function will return `false` if the input contains character from outside the base64 alphabet. Otherwise invalid characters will be silently discarded.

Return Values

Returns the decoded data or `false` on failure. The returned data may be binary.

Examples

Example #1 base64_decode() example

```php
<?php
$str = 'VGhpcyBpcyBhbiBlbmNvZGVkIHN0cmluZw==';
echo base64_decode($str);
?>
```

The above example will output:

```
This is an encoded string
```

See Also

- base64_encode() - Encodes data with MIME base64
- » RFC 2045 section 6.8

base64_encode

base64_encode

(PHP 4, PHP 5, PHP 7, PHP 8)

base64_encode — Encodes data with MIME base64

Description

function **base64_encode**(string `$string`): string

Encodes the given `string` with base64.

This encoding is designed to make binary data survive transport through transport layers that are not 8-bit clean, such as mail bodies.

Base64-encoded data takes about 33% more space than the original data.

Parameters

`string`

The data to encode.

Return Values

The encoded data, as a string.

Examples

Example #1 base64_encode() example

```
<?php
$str = 'This is an encoded string';
echo base64_encode($str);
?>
```

The above example will output:

```
VGhpcyBpcyBhbiBlbmNvZGVkIHN0cmluZw==
```

See Also

- base64_decode() - Decodes data encoded with MIME base64
- chunk_split() - Split a string into smaller chunks
- convert_uuencode() - Uuencode a string
- » RFC 2045 section 6.8

get_headers

get_headers

(PHP 5, PHP 7, PHP 8)

get_headers — Fetches all the headers sent by the server in response to an HTTP request

Description

function **get_headers**(string `$url`, bool `$associative = false`, ?resource `$context = null`): array|false

get_headers() returns an array with the headers sent by the server in response to a HTTP request.

Parameters

`url`

The target URL.

`associative`

If the optional `associative` parameter is set to true, **get_headers()** parses the response and sets the array's keys.

`context`

A valid context resource created with stream_context_create(), or `null` to use the default context.

Return Values

Returns an indexed or associative array with the headers, or `false` on failure.

Changelog

Version	Description
8.0.0	The `associative` has been changed from int to bool.
7.1.0	The `context` parameter was added.

Examples

Example #1 get_headers() example

```php
<?php
$url = 'http://www.example.com';

print_r(get_headers($url));

print_r(get_headers($url, true));
?>
```

The above example will output something similar to:

```
Array
(
    [0] => HTTP/1.1 200 OK
    [1] => Date: Sat, 29 May 2004 12:28:13 GMT
    [2] => Server: Apache/1.3.27 (Unix)  (Red-Hat/Linux)
    [3] => Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
    [4] => ETag: "3f80f-1b6-3e1cb03b"
    [5] => Accept-Ranges: bytes
    [6] => Content-Length: 438
    [7] => Connection: close
    [8] => Content-Type: text/html
)

Array
(
    [0] => HTTP/1.1 200 OK
    [Date] => Sat, 29 May 2004 12:28:14 GMT
    [Server] => Apache/1.3.27 (Unix)  (Red-Hat/Linux)
    [Last-Modified] => Wed, 08 Jan 2003 23:11:55 GMT
    [ETag] => "3f80f-1b6-3e1cb03b"
    [Accept-Ranges] => bytes
    [Content-Length] => 438
    [Connection] => close
    [Content-Type] => text/html
)
```

Example #2 get_headers() using HEAD example

```php
<?php
// By default get_headers uses a GET request to fetch the headers. If you
// want to send a HEAD request instead, you can do so using a stream context:
$context = stream_context_create(
    [
        'http' => array(
            'method' => 'HEAD'
        )
    ]
);
$headers = get_headers('http://example.com', false, $context);
?>
```

See Also

- apache_request_headers() - Fetch all HTTP request headers

get_meta_tags

get_meta_tags

(PHP 4, PHP 5, PHP 7, PHP 8)

get_meta_tags — Extracts all meta tag content attributes from a file and returns an array

Description

function **get_meta_tags**(string `$filename`, bool `$use_include_path = false`): array|false

Opens `filename` and parses it line by line for <meta> tags in the file. The parsing stops at `</head>`.

Parameters

`filename`

The path to the HTML file, as a string. This can be a local file or an URL.

Example #1 What get_meta_tags() parses

```
<meta name="author" content="name">
<meta name="keywords" content="php documentation">
<meta name="DESCRIPTION" content="a php manual">
<meta name="geo.position" content="49.33;-86.59">
</head> <!-- parsing stops here -->
```

`use_include_path`

Setting `use_include_path` to `true` will result in PHP trying to open the file along the standard include path as per the include_path directive. This is used for local files, not URLs.

Return Values

Returns an array with all the parsed meta tags.

The value of the name property becomes the key, the value of the content property becomes the value of the returned array, so you can easily use standard array functions to traverse it or access single values. Special characters in the value of the name property are substituted with '_', the rest is converted to lower case. If two meta tags have the same name, only the last one is returned.

Returns `false` on failure.

Examples

Example #2 What get_meta_tags() returns

```
<?php
// Assuming the above tags are at www.example.com
$tags = get_meta_tags('http://www.example.com/');

// Notice how the keys are all lowercase now, and
// how . was replaced by _ in the key.
echo $tags['author'];       // name
echo $tags['keywords'];     // php documentation
echo $tags['description'];  // a php manual
echo $tags['geo_position']; // 49.33;-86.59
?>
```

Notes

> **Note** :
>
> Only meta tags with name attributes will be parsed. Quotes are not required.

See Also

- htmlentities() - Convert all applicable characters to HTML entities
- urlencode() - URL-encodes string

http_build_query

http_build_query

(PHP 5, PHP 7, PHP 8)

http_build_query — Generate URL-encoded query string

Description

```
function http_build_query(
    array|object $data,
    string $numeric_prefix = "",
    ?string $arg_separator = null,
    int $encoding_type = PHP_QUERY_RFC1738
): string
```

Generates a URL-encoded query string from the associative (or indexed) array provided.

Parameters

`data`

May be an array or object containing properties.

If `data` is an array, it may be a simple one-dimensional structure, or an array of arrays (which in turn may contain other arrays).

If `data` is an object, then only public properties will be incorporated into the result.

> **Note** :
>
> The __toString() magic method is not called when an object is evaluated. To use the string representation of an object in the query string, the object must be explicitly cast to a string.

`numeric_prefix`

If numeric indices are used in the base array and this parameter is provided, it will be prepended to the numeric index for elements in the base array only.

This is meant to allow for legal variable names when the data is decoded by PHP or another CGI application later on.

`arg_separator`

The argument separator. If not set or `null`, arg_separator.output is used to separate arguments.

`encoding_type`

By default, `PHP_QUERY_RFC1738`.

If `encoding_type` is `PHP_QUERY_RFC1738`, then encoding is performed per » RFC 1738 and the `application/x-www-form-urlencoded` media type, which implies that spaces are encoded as plus (+) signs.

If `encoding_type` is `PHP_QUERY_RFC3986`, then encoding is performed according to » RFC 3986, and spaces will be percent encoded (`%20`).

Return Values

Returns a URL-encoded string.

Changelog

Version	Description
8.4.0	Prior to PHP 8.4.0, BackedEnum properties of `data` were converted to objects, rather than their scalar equivalents.
8.0.0	`arg_separator` is now nullable.

Examples

Example #1 Simple usage of **http_build_query()**

```php
<?php
$data = array(
    'foo' => 'bar',
    'baz' => 'boom',
    'cow' => 'milk',
    'null' => null,
    'php' => 'hypertext processor'
```

```
);

echo http_build_query($data) . "\n";
echo http_build_query($data, '', '&amp;');

?>
```

The above example will output:

```
foo=bar&baz=boom&cow=milk&php=hypertext+processor
foo=bar&amp;baz=boom&amp;cow=milk&amp;php=hypertext+processor
```

Example #2 http_build_query() with numerically index elements.

```
<?php
$data = array('foo', 'bar', 'baz', null, 'boom', 'cow' => 'milk', 'php' => 'hypertext processor');

echo http_build_query($data) . "\n";
echo http_build_query($data, 'myvar_');
?>
```

The above example will output:

```
0=foo&1=bar&2=baz&4=boom&cow=milk&php=hypertext+processor
myvar_0=foo&myvar_1=bar&myvar_2=baz&myvar_4=boom&cow=milk&php=hypertext+processor
```

Example #3 http_build_query() with complex arrays

```
<?php
$data = array(
    'user' => array(
        'name' => 'Bob Smith',
        'age'  => 47,
        'sex'  => 'M',
        'dob'  => '5/12/1956'
    ),
    'pastimes' => array('golf', 'opera', 'poker', 'rap'),
    'children' => array(
        'bobby' => array('age'=>12, 'sex'=>'M'),
        'sally' => array('age'=>8, 'sex'=>'F')
    ),
    'CEO'
);

echo http_build_query($data, 'flags_');
?>
```

The above example will output: (word wrapped for readability)

```
user%5Bname%5D=Bob+Smith&user%5Bage%5D=47&user%5Bsex%5D=M&
user%5Bdob%5D=5%2F12%2F1956&pastimes%5B0%5D=golf&pastimes%5B1%5D=opera&
pastimes%5B2%5D=poker&pastimes%5B3%5D=rap&children%5Bbobby%5D%5Bage%5D=12&
children%5Bbobby%5D%5Bsex%5D=M&children%5Bsally%5D%5Bage%5D=8&
children%5Bsally%5D%5Bsex%5D=F&flags_0=CEO
```

> **Note** :
>
> Only the numerically indexed element in the base array "CEO" received a prefix. The other numeric indices, found under pastimes, do not require a string prefix to be legal variable names.

Example #4 Using http_build_query() with an object

```php
<?php
class parentClass {
    public    $pub      = 'publicParent';
    protected $prot     = 'protectedParent';
    private   $priv     = 'privateParent';
    public    $pub_bar  = null;
    protected $prot_bar = null;
    private   $priv_bar = null;

    public function __construct(){
        $this->pub_bar  = new childClass();
        $this->prot_bar = new childClass();
        $this->priv_bar = new childClass();
    }
}

class childClass {
    public    $pub  = 'publicChild';
    protected $prot = 'protectedChild';
    private   $priv = 'privateChild';
}

$parent = new parentClass();

echo http_build_query($parent);
?>
```

The above example will output:

```
pub=publicParent&pub_bar%5Bpub%5D=publicChild
```

Example #5 Using http_build_query() with objects containing __toString()

```php
<?php
class Foo {
    public $publicProperty = 'visible';

    public function __toString() {
        return "bar";
    }
}

$params = array(
    'a' => 'b',
    'foo' => new Foo()
);

// Without casting, http_build_query reads the public properties
echo http_build_query($params) . "\n";

// With explicit casting, http_build_query uses the __toString() output
$params['foo'] = (string) new Foo();
```

```php
echo http_build_query($params) . "\n";
?>
```

The above example will output:

```
a=b&foo%5BpublicProperty%5D=visible
a=b&foo=bar
```

See Also

- parse_str() - Parse a string as a URL query string
- parse_url() - Parse a URL and return its components
- urlencode() - URL-encodes string
- array_walk() - Apply a user supplied function to every member of an array

parse_url

parse_url

(PHP 4, PHP 5, PHP 7, PHP 8)

parse_url — Parse a URL and return its components

Description

function **parse_url**(string `$url`, int `$component = -1`): int|string|array|null|false

This function parses a URL and returns an associative array containing any of the various components of the URL that are present. The values of the array elements are *not* URL decoded.

This function is **not** meant to validate the given URL, it only breaks it up into the parts listed below. Partial and invalid URLs are also accepted, **parse_url()** tries its best to parse them correctly.

> **Caution**
>
> This function does not follow any established URI or URL standard. It will return incorrect or non-sense results for relative or malformed URLs. Even for valid URLs the result may differ from that of a different URL parser, since there are multiple different URL-related standards that target different use cases and that differ in their requirements.
>
> Processing an URL with parsers following different URL standards is a common source of security vulnerabilities. As an example, validating an URL against an allow-list of acceptable hostnames with parser A might be ineffective when the actual retrieval of the resource uses parser B that extracts hostnames differently.
>
> The Uri\Rfc3986\Uri and Uri\WhatWg\Url classes strictly follow the RFC 3986 and WHATWG URL Standards respectively. It is strongly recommended to use these classes for all newly written code and to migrate existing uses of the **parse_url()** function to these classes, unless the **parse_url()** behavior needs to be preserved for compatibility reasons.

Parameters

`url`

The URL to parse.

`component`

Specify one of `PHP_URL_SCHEME`, `PHP_URL_HOST`, `PHP_URL_PORT`, `PHP_URL_USER`, `PHP_URL_PASS`, `PHP_URL_PATH`, `PHP_URL_QUERY` or `PHP_URL_FRAGMENT` to retrieve just a specific URL component as a string (except when `PHP_URL_PORT` is given, in which case the return value will be an int).

Return Values

On seriously malformed URLs, **parse_url()** may return `false`.

If the `component` parameter is omitted, an associative array is returned. At least one element will be present within the array. Potential keys within this array are:

- *scheme* - e.g. `http`
- *host*
- *port*
- *user*
- *pass*
- *path*
- *query* - after the question mark `?`
- *fragment* - after the hashmark `#`

If the `component` parameter is specified, **parse_url()** returns a string (or an int, in the case of `PHP_URL_PORT`) instead of an array. If the requested component doesn't exist within the given URL, `null` will be returned. As of PHP 8.0.0, **parse_url()** distinguishes absent and empty queries and fragments:

```
http://example.com/foo → query = null, fragment = null
http://example.com/foo? → query = "",   fragment = null
http://example.com/foo# → query = null, fragment = ""
http://example.com/foo?# → query = "",   fragment = ""
```

Previously all cases resulted in query and fragment being `null`.

Note that control characters (cf. ctype_cntrl()) in the components are replaced with underscores (_).

Changelog

Version	Description
8.0.0	**parse_url()** will now distinguish absent and empty queries and fragments.

Examples

Example #1 A **parse_url()** example

```php
<?php
$url = 'http://username:password@hostname:9090/path?arg=value#anchor';

var_dump(parse_url($url));
var_dump(parse_url($url, PHP_URL_SCHEME));
var_dump(parse_url($url, PHP_URL_USER));
var_dump(parse_url($url, PHP_URL_PASS));
var_dump(parse_url($url, PHP_URL_HOST));
var_dump(parse_url($url, PHP_URL_PORT));
var_dump(parse_url($url, PHP_URL_PATH));
var_dump(parse_url($url, PHP_URL_QUERY));
var_dump(parse_url($url, PHP_URL_FRAGMENT));
?>
```

The above example will output:

```
array(8) {
  ["scheme"]=>
  string(4) "http"
  ["host"]=>
  string(8) "hostname"
```

```
  ["port"]=>
  int(9090)
  ["user"]=>
  string(8) "username"
  ["pass"]=>
  string(8) "password"
  ["path"]=>
  string(5) "/path"
  ["query"]=>
  string(9) "arg=value"
  ["fragment"]=>
  string(6) "anchor"
}
string(4) "http"
string(8) "username"
string(8) "password"
string(8) "hostname"
int(9090)
string(5) "/path"
string(9) "arg=value"
string(6) "anchor"
```

Example #2 A parse_url() example with missing scheme

```php
<?php
$url = '//www.example.com/path?googleguy=googley';

// Prior to 5.4.7 this would show the path as "//www.example.com/path"
var_dump(parse_url($url));
?>
```

The above example will output:

```
array(3) {
  ["host"]=>
  string(15) "www.example.com"
  ["path"]=>
  string(5) "/path"
  ["query"]=>
  string(17) "googleguy=googley"
}
```

Notes

> **Note** :
>
> This function is intended specifically for the purpose of parsing URLs and not URIs. However, to comply with PHP's backwards compatibility requirements it makes an exception for the `file://` scheme where triple slashes (`file:///...`) are allowed. For any other scheme this is invalid.

See Also

- pathinfo() - Returns information about a file path
- parse_str() - Parse a string as a URL query string
- http_build_query() - Generate URL-encoded query string
- dirname() - Returns a parent directory's path
- basename() - Returns trailing name component of path
- » RFC 3986

rawurldecode

rawurldecode

(PHP 4, PHP 5, PHP 7, PHP 8)

rawurldecode — Decode URL-encoded strings

Description

function **rawurldecode**(string `$string`): string

Returns a string in which the sequences with percent (`%`) signs followed by two hex digits have been replaced with literal characters.

Parameters

`string`

The URL to be decoded.

Return Values

Returns the decoded URL, as a string.

Examples

Example #1 rawurldecode() example

```php
<?php

echo rawurldecode('foo%20bar%40baz'); // foo bar@baz

?>
```

Notes

> **Note** :
>
> **rawurldecode()** does not decode plus symbols ('+') into spaces. urldecode() does.

See Also

- rawurlencode() - URL-encode according to RFC 3986
- urldecode() - Decodes URL-encoded string
- urlencode() - URL-encodes string
- » RFC 3986

rawurlencode

rawurlencode

(PHP 4, PHP 5, PHP 7, PHP 8)

rawurlencode — URL-encode according to RFC 3986

Description

function **rawurlencode**(string `$string`): string

Encodes the given string according to » RFC 3986.

Parameters

`string`

The URL to be encoded.

Return Values

Returns a string in which all non-alphanumeric characters except -_.~ have been replaced with a percent (%) sign followed by two hex digits. This is the encoding described in » RFC 3986 for protecting literal characters from being interpreted as special URL delimiters, and for protecting URLs from being mangled by transmission media with character conversions (like some email systems).

Examples

Example #1 including a password in an FTP URL

```
<?php
echo '<a href="ftp://user:', rawurlencode('foo @+%/'),
     '@ftp.example.com/x.txt">';
?>
```

The above example will output:

```
<a href="ftp://user:foo%20%40%2B%25%2F@ftp.example.com/x.txt">
```

Or, if you pass information in a PATH_INFO component of the URL:

Example #2 rawurlencode() example 2

```
<?php
echo '<a href="http://example.com/department_list_script/',
    rawurlencode('sales and marketing/Miami'), '">';
?>
```

The above example will output:

```
<a href="http://example.com/department_list_script/sales%20and%20marketing%2FMiami">
```

See Also

- rawurldecode() - Decode URL-encoded strings
- urldecode() - Decodes URL-encoded string
- urlencode() - URL-encodes string
- » RFC 3986

urldecode

urldecode

(PHP 4, PHP 5, PHP 7, PHP 8)

urldecode — Decodes URL-encoded string

Description

function **urldecode**(string `$string`): string

Decodes any `%##` encoding in the given string. Plus symbols ('+') are decoded to a space character.

Parameters

`string`

The string to be decoded.

Return Values

Returns the decoded string.

Examples

Example #1 urldecode() example

```php
<?php
$query = "my=apples&are=green+and+red";

foreach (explode('&', $query) as $chunk) {
    $param = explode("=", $chunk);

    if ($param) {
        printf("Value for parameter \"%s\" is \"%s\"<br/>\n", urldecode($param[0]), urldecode($param[1]));
    }
}
?>
```

Notes

> **Warning**
>
> The superglobals *$_GET* and *$_REQUEST* are already decoded. Using **urldecode()** on an element in *$_GET* or *$_REQUEST* could have unexpected and dangerous results.

See Also

- urlencode() - URL-encodes string
- rawurlencode() - URL-encode according to RFC 3986
- rawurldecode() - Decode URL-encoded strings
- » RFC 3986

urlencode

urlencode

(PHP 4, PHP 5, PHP 7, PHP 8)

urlencode — URL-encodes string

Description

function **urlencode**(string `$string`): string

This function is convenient when encoding a string to be used in a query part of a URL, as a convenient way to pass variables to the next page.

Parameters

`string`

The string to be encoded.

Return Values

Returns a string in which all non-alphanumeric characters except `-_.` have been replaced with a percent (`%`) sign followed by two hex digits and spaces encoded as plus (`+`) signs. It is encoded the same way that the posted data from a WWW form is encoded, that is the same way as in `application/x-www-form-urlencoded` media type. This differs from the » RFC 3986 encoding (see rawurlencode()) in that for historical reasons, spaces are encoded as plus (`+`) signs.

Examples

Example #1 urlencode() example

```php
<?php
$userinput = 'Data123!@-_ +';
echo "UserInput: $userinput\n";
```

```php
echo '<a href="mycgi?foo=', urlencode($userinput), '">';
?>
```

The above example will output:

```
UserInput: Data123!@-_ +
<a href="mycgi?foo=Data123%21%40-_+%2B">
```

Example #2 urlencode() and htmlentities() example

```php
<?php
$foo = 'Data123!@-_ +';
$bar = "Not the same content as $foo";
echo "foo: $foo\n";
echo "bar: $bar\n";
$query_string = 'foo=' . urlencode($foo) . '&bar=' . urlencode($bar);
echo '<a href="mycgi?' . htmlentities($query_string) . '">';
?>
```

The above example will output:

```
foo: Data123!@-_ +
bar: Not the same content as Data123!@-_ +
<a href="mycgi?foo=Data123%21%40-_+%2B&amp;bar=Not+the+same+content+as+Data123%21%40-_+%2B">
```

Notes

> **Note** :
>
> Be careful about variables that may match HTML entities. Things like &, &copy and &pound are parsed by the browser and the actual entity is used instead of the desired variable name. This is an obvious hassle that the W3C has been telling people about for years. The reference is here: » http://www.w3.org/TR/html4/appendix/notes.html#h-B.2.2.
>
> PHP supports changing the argument separator to the W3C-suggested semi-colon through the arg_separator .ini directive. Unfortunately most user agents do not send form data in this semi-colon separated format. A more portable way around this is to use & instead of & as the separator. You don't need to change PHP's arg_separator for this. Leave it as &, but simply encode your URLs using htmlentities() or htmlspecialchars().

See Also

- urldecode() - Decodes URL-encoded string
- htmlentities() - Convert all applicable characters to HTML entities
- rawurlencode() - URL-encode according to RFC 3986
- rawurldecode() - Decode URL-encoded strings
- » RFC 3986

Network

Introduction

Provides various networking functions.

Installing/Configuring

Table of Contents

Requirements

Functions checkdnsrr(), getmxrr() and dns_get_record() requires Bind on Linux.

Resource Types

This extension defines a file pointer resource returned by fsockopen() and pfsockopen().

Predefined Constants

The constants below are always available as part of the PHP core.

Available `flags` for openlog()

`LOG_CONS` (int)

If there is an error while sending data to the system logger, write directly to the system console.

`LOG_NDELAY` (int)

Open the connection to the logger immediately.

`LOG_ODELAY` (int)

Delay opening the connection until the first message is logged. This is the default behaviour.

`LOG_NOWAIT` (int)

`LOG_PERROR` (int)

Additionally log messages to `STDERR`.

`LOG_PID` (int)

Include the PID with each log message.

Available `facility` for openlog()

`LOG_AUTH` (int)

For security/authorization messages.

> **Note** :
>
> Use `LOG_AUTHPRIV` instead when available.

`LOG_AUTHPRIV` (int)

For private security/authorization messages.

`LOG_CRON` (int)

For clock daemon messages. For example **cron** or **at**.

`LOG_DAEMON` (int)

For system daemon messages.

`LOG_KERN` (int)

For kernel messages.

`LOG_LOCAL0` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LOCAL1` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LOCAL2` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LOCAL3` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LOCAL4` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LOCAL5` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LOCAL6` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LOCAL7` (int)

Reserved for local use.

> **Warning**
>
> Not available on Windows.

`LOG_LPR` (int)

For messages from the line printer subsystem.

`LOG_MAIL` (int)

For messages from the mail subsystem.

`LOG_NEWS` (int)

For messages from the USENET news subsystem.

`LOG_SYSLOG` (int)

For messages generated internally by **syslogd**.

`LOG_USER` (int)

For generic user-level messages.

`LOG_UUCP` (int)

For messages from the UUCP subsystem.

Available `priority` for syslog()

The priority constants are listed from urgent, to debug messages.

`LOG_EMERG` (int)

Emergency, the system is unusable. This corresponds to a panic condition. Usually broadcast to all processes.

`LOG_ALERT` (int)

Alert, immediate action is required. For example, a corrupted system database.

`LOG_CRIT` (int)

Critical, action is required. For example, a hard device errors.

`LOG_ERR` (int)

Errors messages.

`LOG_WARNING` (int)

Warning messages.

`LOG_NOTICE` (int)

Notice messages, corresponds to conditions that are not error conditions, but that may require special handling.

`LOG_INFO` (int)

Informational messages.

`LOG_DEBUG` (int)

Debugging messages that contain information normally of use only when debugging a program.

Available `type`s for dns_get_record()

`DNS_ANY` (int)

Any Resource Record. On most systems this returns all resource records, however due to eccentricities in the performance of `libresolv` between platforms this is not guaranteed.

The slower `DNS_ALL` will collect all records more reliably.

DNS_ALL (int)

Iteratively query the name server for each available record type.

DNS_A (int)

IPv4 Address Resource.

DNS_AAAA (int)

IPv6 Address Resource.

DNS_A6 (int)

Defined as part of early IPv6, but downgraded to historic by » RFC 6563.

DNS_CAA (int)

Certification Authority Authorization Resource. Available as of PHP 7.0.16 and 7.1.2.

> **Warning**
>
> Not available on Windows.

DNS_CNAME (int)

Alias (Canonical Name) Resource.

DNS_HINFO (int)

Host Info Resource. For more explanations and meanings of these values, visit IANA's page on » Operating System Names.

DNS_MX (int)

Mail Exchanger Resource.

DNS_NAPTR (int)

Naming Authority Pointer.

DNS_NS (int)

Authoritative Name Server Resource.

DNS_PTR (int)

Pointer Resource.

DNS_SOA (int)

Start of Authority Resource.

DNS_SRV (int)

Service locator.

DNS_TXT (int)

Text Resource.

Network Functions

Table of Contents

checkdnsrr

checkdnsrr

(PHP 4, PHP 5, PHP 7, PHP 8)

checkdnsrr — Check DNS records corresponding to a given Internet host name or IP address

Description

function **checkdnsrr**(string `$hostname`, string `$type` = "MX"): bool

Searches DNS for records of type `type` corresponding to `hostname`.

Parameters

`hostname`

`hostname` may either be the IP address in dotted-quad notation or the host name.

`type`

`type` may be any one of: A, MX, NS, SOA, PTR, CNAME, AAAA, A6, SRV, NAPTR, TXT or ANY.

Return Values

Returns **`true`** if any records are found; returns **`false`** if no records were found or if an error occurred.

Notes

> **Note** :
>
> For compatibility with Windows before this was implemented, then try the » PEAR class » Net_DNS.

See Also

- dns_get_record() - Fetch DNS Resource Records associated with a hostname
- getmxrr() - Get MX records corresponding to a given Internet host name
- gethostbyaddr() - Get the Internet host name corresponding to a given IP address
- gethostbyname() - Get the IPv4 address corresponding to a given Internet host name
- gethostbynamel() - Get a list of IPv4 addresses corresponding to a given Internet host name
- the named(8) manual page

closelog

closelog

(PHP 4, PHP 5, PHP 7, PHP 8)

closelog — Close connection to system logger

Description

function **closelog**(): true

closelog() closes the descriptor being used to write to the system logger. The use of **closelog()** is optional.

Parameters

This function has no parameters.

Return Values

Always returns **`true`**.

See Also

- syslog() - Generate a system log message
- openlog() - Open connection to system logger

dns_check_record

dns_check_record

(PHP 5, PHP 7, PHP 8)

dns_check_record — Alias of checkdnsrr()

Description

This function is an alias of: checkdnsrr().

dns_get_mx

dns_get_mx

(PHP 5, PHP 7, PHP 8)

dns_get_mx — Alias of getmxrr()

Description

This function is an alias of: getmxrr().

dns_get_record

dns_get_record

(PHP 5, PHP 7, PHP 8)

dns_get_record — Fetch DNS Resource Records associated with a hostname

Description

```
function dns_get_record(
    string $hostname,
    int $type = DNS_ANY,
    array &$authoritative_name_servers = null,
    array &$additional_records = null,
    bool $raw = false
): array|false
```

Fetch DNS Resource Records associated with the given `hostname`.

Parameters

`hostname`

> `hostname` should be a valid DNS hostname such as "`www.example.com`". Reverse lookups can be generated using `in-addr.arpa` notation, but gethostbyaddr() is more suitable for the majority of reverse lookups.
>
> > **Note** :
> >
> > Per DNS standards, email addresses are given in `user.host` format (for example: `hostmaster.example.com` as opposed to `hostmaster@example.com`), be sure to check this value and modify if necessary before using it with a functions such as mail().

`type`

> By default, **dns_get_record()** will search for any resource records associated with `hostname`. To limit the query, use one of the `DNS_*` constants.

`authoritative_name_servers`

> Passed by reference and, if given, will be populated with Resource Records for the *Authoritative Name Servers*.

`additional_records`

> Passed by reference and, if given, will be populated with any *Additional Records*.

`raw`

> The `type` will be interpreted as a raw DNS type ID (the `DNS_*` constants cannot be used). The return value will contain a `data` key, which needs to be manually parsed.

Return Values

This function returns an array of associative arrays, or `false` on failure. Each associative array contains *at minimum* the following keys:

Basic DNS attributes

Attribute	Meaning
host	The record in the DNS namespace to which the rest of the associated data refers.
class	**dns_get_record()** only returns Internet class records and as such this parameter will always return `IN`.
type	String containing the record type. Additional attributes will also be contained in the resulting array dependent on the value of type. See table below.
ttl	`"Time To Live"` remaining for this record. This will *not* equal the record's original ttl, but will rather equal the original ttl minus whatever length of time has passed since the authoritative name server was queried.

Other keys in associative arrays dependent on `type`

Type	Extra Columns
`A`	`ip`: An IPv4 addresses in dotted decimal notation.
`MX`	`pri`: Priority of mail exchanger. Lower numbers indicate greater priority. `target`: FQDN of the mail exchanger. See also dns_get_mx().
`CNAME`	`target`: FQDN of location in DNS namespace to which the record is aliased.
`NS`	`target`: FQDN of the name server which is authoritative for this hostname.
`PTR`	`target`: Location within the DNS namespace to which this record points.
`TXT`	`txt`: Arbitrary string data associated with this record.
`HINFO`	`cpu`: IANA number designating the CPU of the machine referenced by this record. `os`: IANA number designating the Operating System on the machine referenced by this record. See IANA's » `Operating System Names` for the meaning of these values.
`CAA`	`flags`: A one-byte bitfield; currently only bit 0 is defined, meaning 'critical'; other bits are reserved and should be ignored. `tag`: The CAA tag name (alphanumeric ASCII string). `value`: The CAA tag value (binary string, may use subformats). For additional information see: » RFC 6844
`SOA`	`mname`: FQDN of the machine from which the resource records originated. `rname`: Email address of the administrative contact for this domain. `serial`: Serial # of this revision of the requested domain. `refresh`: Refresh interval (seconds) secondary name servers should use when updating remote copies of this domain. `retry`: Length of time (seconds) to wait after a failed refresh before making a second attempt. `expire`: Maximum length of time (seconds) a secondary DNS server should retain remote copies of the zone data without a successful refresh before discarding. `minimum-ttl`: Minimum length of time (seconds) a client can continue to use a DNS resolution before it should request a new resolution from the server. Can be overridden by individual resource records.
`AAAA`	`ipv6`: IPv6 address
`A6`	`masklen`: Length (in bits) to inherit from the target specified by `chain`. `ipv6`: Address for this specific record to merge with `chain`. `chain`: Parent record to merge with `ipv6` data.
`SRV`	`pri`: (Priority) lowest priorities should be used first. `weight`: Ranking to weight which of commonly prioritized `targets` should be chosen at random. `target` and `port`: hostname and port where the requested service can be found. For additional information see: » RFC 2782
`NAPTR`	`order` and `pref`: Equivalent to `pri` and `weight` above. `flags`, `services`, `regex`, and `replacement`: Parameters as defined by » RFC 2915.

Changelog

Version	Description
7.0.16, 7.1.2	Added support for CAA record type.

Examples

Example #1 Using **dns_get_record()**

```
<?php
$result = dns_get_record("php.net");
print_r($result);
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Array
        (
            [host] => php.net
            [type] => MX
            [pri] => 5
            [target] => pair2.php.net
            [class] => IN
            [ttl] => 6765
        )

    [1] => Array
        (
            [host] => php.net
            [type] => A
            [ip] => 64.246.30.37
            [class] => IN
            [ttl] => 8125
        )

)
```

Example #2 Using dns_get_record() and DNS_ANY

Since it's very common to want the IP address of a mail server once the MX record has been resolved, **dns_get_record()** also returns an array in `additional_records` which contains associate records. `authoritative_name_servers` is returned as well containing a list of authoritative name servers.

```php
<?php
/* Request "ANY" record for php.net,
   and create $authns and $addtl arrays
   containing list of name servers and
   any additional records which go with
   them */
$result = dns_get_record("php.net", DNS_ANY, $authns, $addtl);
echo "Result = ";
print_r($result);
echo "Auth NS = ";
print_r($authns);
echo "Additional = ";
print_r($addtl);
?>
```

The above example will output something similar to:

```
Result = Array
(
    [0] => Array
        (
            [host] => php.net
            [type] => MX
            [pri] => 5
            [target] => pair2.php.net
            [class] => IN
            [ttl] => 6765
        )

    [1] => Array
        (
```

```
            [host] => php.net
            [type] => A
            [ip] => 64.246.30.37
            [class] => IN
            [ttl] => 8125
        )

)
Auth NS = Array
(
    [0] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => remote1.easydns.com
            [class] => IN
            [ttl] => 10722
        )

    [1] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => remote2.easydns.com
            [class] => IN
            [ttl] => 10722
        )

    [2] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => ns1.easydns.com
            [class] => IN
            [ttl] => 10722
        )

    [3] => Array
        (
            [host] => php.net
            [type] => NS
            [target] => ns2.easydns.com
            [class] => IN
            [ttl] => 10722
        )

)
Additional = Array
(
    [0] => Array
        (
            [host] => pair2.php.net
            [type] => A
            [ip] => 216.92.131.5
            [class] => IN
            [ttl] => 6766
        )

    [1] => Array
        (
            [host] => remote1.easydns.com
            [type] => A
            [ip] => 64.39.29.212
            [class] => IN
            [ttl] => 100384
        )

    [2] => Array
        (
            [host] => remote2.easydns.com
            [type] => A
            [ip] => 212.100.224.80
```

```
            [class] => IN
            [ttl] => 81241
        )

    [3] => Array
        (
            [host] => ns1.easydns.com
            [type] => A
            [ip] => 216.220.40.243
            [class] => IN
            [ttl] => 81241
        )

    [4] => Array
        (
            [host] => ns2.easydns.com
            [type] => A
            [ip] => 216.220.40.244
            [class] => IN
            [ttl] => 81241
        )

)
```

See Also

- dns_get_mx() - Alias of getmxrr
- dns_check_record() - Alias of checkdnsrr

fsockopen

fsockopen

(PHP 4, PHP 5, PHP 7, PHP 8)

fsockopen — Open Internet or Unix domain socket connection

Description

```
function fsockopen(
    string $hostname,
    int $port = -1,
    int &$error_code = null,
    string &$error_message = null,
    ?float $timeout = null
): resource|false
```

Initiates a socket connection to the resource specified by `hostname`.

PHP supports targets in the Internet and Unix domains as described in List of Supported Socket Transports. A list of supported transports can also be retrieved using stream_get_transports().

The socket will by default be opened in blocking mode. You can switch it to non-blocking mode by using stream_set_blocking().

The function stream_socket_client() is similar but provides a richer set of options, including non-blocking connection and the ability to provide a stream context.

Parameters

`hostname`

If OpenSSL support is installed, you may prefix the `hostname` with either `ssl://` or `tls://` to use an SSL or TLS client connection over TCP/IP to connect to the remote host.

`port`

The port number. This can be omitted and skipped with `-1` for transports that do not use ports, such as `unix://`.

`error_code`

If provided, holds the system level error number that occurred in the system-level `connect()` call.

If the value returned in `error_code` is `0` and the function returned **`false`**, it is an indication that the error occurred before the `connect()` call. This is most likely due to a problem initializing the socket.

`error_message`

The error message as a string.

`timeout`

The connection timeout, in seconds. When **`null`**, the default_socket_timeout *php.ini* setting is used.

> **Note** :
>
> If you need to set a timeout for reading/writing data over the socket, use stream_set_timeout(), as the `timeout` parameter to **fsockopen()** only applies while connecting the socket.

Return Values

fsockopen() returns a file pointer which may be used together with the other file functions (such as fgets(), fgetss(), fwrite(), fclose(), and feof()). If the call fails, it will return **`false`**

Errors/Exceptions

Throws **`E_WARNING`** if `hostname` is not a valid domain.

Changelog

Version	Description
8.0.0	`timeout` is nullable now.

Examples

Example #1 fsockopen() Example

```
<?php
$fp = fsockopen("www.example.com", 80, $errno, $errstr, 30);
if (!$fp) {
    echo "$errstr ($errno)<br />\n";
} else {
    $out = "GET / HTTP/1.1\r\n";
    $out .= "Host: www.example.com\r\n";
    $out .= "Connection: Close\r\n\r\n";
    fwrite($fp, $out);
    while (!feof($fp)) {
        echo fgets($fp, 128);
    }
    fclose($fp);
}
?>
```

Example #2 Using UDP connection

The example below shows how to retrieve the day and time from the UDP service "daytime" (port 13) in your own machine.

```
<?php
$fp = fsockopen("udp://127.0.0.1", 13, $errno, $errstr);
if (!$fp) {
    echo "ERROR: $errno - $errstr<br />\n";
} else {
    fwrite($fp, "\n");
    echo fread($fp, 26);
    fclose($fp);
}
?>
```

Notes

> **Note** :
>
> Depending on the environment, the Unix domain or the optional connect timeout may not be available.

> **Warning**
>
> UDP sockets will sometimes appear to have opened without an error, even if the remote host is unreachable. The error will only become apparent when you read or write data to/from the socket. The reason for this is because UDP is a "connectionless" protocol, which means that the operating system does not try to establish a link for the socket until it actually needs to send or receive data.

> **Note** :
>
> When specifying a numerical IPv6 address (e.g. `fe80::1`), you must enclose the IP in square brackets—for example, `tcp://[fe80::1]:80`.

See Also

- pfsockopen() - Open persistent Internet or Unix domain socket connection
- stream_socket_client() - Open Internet or Unix domain socket connection
- stream_set_blocking() - Set blocking/non-blocking mode on a stream
- stream_set_timeout() - Set timeout period on a stream
- fgets() - Gets line from file pointer
- fgetss() - Gets line from file pointer and strip HTML tags
- fwrite() - Binary-safe file write
- fclose() - Closes an open file pointer
- feof() - Tests for end-of-file on a file pointer
- socket_connect() - Initiates a connection on a socket
- The Curl extension

gethostbyaddr

gethostbyaddr

(PHP 4, PHP 5, PHP 7, PHP 8)

gethostbyaddr — Get the Internet host name corresponding to a given IP address

Description

function **gethostbyaddr**(string `$ip`): string|false

Returns the host name of the Internet host specified by `ip`.

Parameters

`ip`

The host IP address.

Return Values

Returns the host name on success, the unmodified `ip` on failure, or **`false`** on malformed input.

Examples

Example #1 A simple **gethostbyaddr()** example

```php
<?php
$hostname = gethostbyaddr($_SERVER['REMOTE_ADDR']);

echo $hostname;
?>
```

See Also

- gethostbyname() - Get the IPv4 address corresponding to a given Internet host name
- gethostbynamel() - Get a list of IPv4 addresses corresponding to a given Internet host name

gethostbyname

gethostbyname

(PHP 4, PHP 5, PHP 7, PHP 8)

gethostbyname — Get the IPv4 address corresponding to a given Internet host name

Description

function **gethostbyname**(string `$hostname`): string

Returns the IPv4 address of the Internet host specified by `hostname`.

Parameters

`hostname`

The host name.

Return Values

Returns the IPv4 address or a string containing the unmodified `hostname` on failure.

Examples

Example #1 A simple gethostbyname() example

```php
<?php
$ip = gethostbyname('www.example.com');

echo $ip;
?>
```

See Also

- gethostbyaddr() - Get the Internet host name corresponding to a given IP address
- gethostbynamel() - Get a list of IPv4 addresses corresponding to a given Internet host name
- inet_pton() - Converts a human readable IP address to its packed in_addr representation
- inet_ntop() - Converts a packed internet address to a human readable representation

gethostbynamel

gethostbynamel

(PHP 4, PHP 5, PHP 7, PHP 8)

gethostbynamel — Get a list of IPv4 addresses corresponding to a given Internet host name

Description

function **gethostbynamel**(string `$hostname`): array|false

Returns a list of IPv4 addresses to which the Internet host specified by `hostname` resolves.

Parameters

`hostname`

The host name.

Return Values

Returns an array of IPv4 addresses or `false` if `hostname` could not be resolved.

Examples

Example #1 gethostbynamel() example

```php
<?php
$hosts = gethostbynamel('www.example.com');
print_r($hosts);
?>
```

The above example will output:

```
Array
(
    [0] => 192.0.34.166
)
```

See Also

- gethostbyname() - Get the IPv4 address corresponding to a given Internet host name
- gethostbyaddr() - Get the Internet host name corresponding to a given IP address
- checkdnsrr() - Check DNS records corresponding to a given Internet host name or IP address
- getmxrr() - Get MX records corresponding to a given Internet host name
- the `named(8)` manual page

gethostname

gethostname

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gethostname — Gets the host name

Description

function **gethostname**(): string|false

gethostname() gets the standard host name for the local machine.

Parameters

This function has no parameters.

Return Values

Returns a string with the hostname on success, otherwise `false` is returned.

Examples

Example #1 A simple gethostname() example

```php
<?php
echo gethostname(); // may output e.g,: sandie
?>
```

See Also

- gethostbyname() - Get the IPv4 address corresponding to a given Internet host name
- gethostbyaddr() - Get the Internet host name corresponding to a given IP address
- php_uname() - Returns information about the operating system PHP is running on

getmxrr

getmxrr

(PHP 4, PHP 5, PHP 7, PHP 8)

getmxrr — Get MX records corresponding to a given Internet host name

Description

function **getmxrr**(string `$hostname`, array `&$hosts`, array `&$weights = null`): bool

Searches DNS for MX records corresponding to `hostname`.

Parameters

`hostname`

The Internet host name.

`hosts`

A list of the MX records found is placed into the array `hosts`.

`weights`

If the `weights` array is given, it will be filled with the weight information gathered.

Return Values

Returns `true` if any records are found; returns `false` if no records were found or if an error occurred.

Notes

> **Note** :
>
> This function should not be used for the purposes of address verification. Only the mailexchangers found in DNS are returned, however, according to » RFC 2821 when no mail exchangers are listed, `hostname` itself should be used as the only mail exchanger with a priority of `0`.

> **Note** :
>
> For compatibility with Windows before this was implemented, then try the » PEAR class » Net_DNS.

See Also

- checkdnsrr() - Check DNS records corresponding to a given Internet host name or IP address
- dns_get_record() - Fetch DNS Resource Records associated with a hostname
- gethostbyname() - Get the IPv4 address corresponding to a given Internet host name
- gethostbynamel() - Get a list of IPv4 addresses corresponding to a given Internet host name
- gethostbyaddr() - Get the Internet host name corresponding to a given IP address
- the `named(8)` manual page

getprotobyname

getprotobyname

(PHP 4, PHP 5, PHP 7, PHP 8)

getprotobyname — Get protocol number associated with protocol name

Description

function **getprotobyname**(string `$protocol`): int|false

getprotobyname() returns the protocol number associated with the protocol `protocol` as per */etc/protocols*.

Parameters

`protocol`

The protocol name.

Return Values

Returns the protocol number, or `false` on failure.

Examples

Example #1 getprotobyname() example

```php
<?php
$protocol = 'tcp';
$get_prot = getprotobyname($protocol);
if ($get_prot === FALSE) {
    echo 'Invalid Protocol';
} else {
    echo 'Protocol #' . $get_prot;
}
?>
```

See Also

- getprotobynumber() - Get protocol name associated with protocol number

getprotobynumber

getprotobynumber

(PHP 4, PHP 5, PHP 7, PHP 8)

getprotobynumber — Get protocol name associated with protocol number

Description

function **getprotobynumber**(int `$protocol`): string|false

getprotobynumber() returns the protocol name associated with protocol `protocol` as per */etc/protocols*.

Parameters

`protocol`

The protocol number.

Return Values

Returns the protocol name as a string, or `false` on failure.

See Also

- getprotobyname() - Get protocol number associated with protocol name

getservbyname

getservbyname

(PHP 4, PHP 5, PHP 7, PHP 8)

getservbyname — Get port number associated with an Internet service and protocol

Description

function **getservbyname**(string `$service`, string `$protocol`): int|false

getservbyname() returns the Internet port which corresponds to `service` for the specified `protocol` as per */etc/services*.

Parameters

`service`

The Internet service name, as a string.

`protocol`

`protocol` is either `"tcp"` or `"udp"` (in lowercase).

Return Values

Returns the port number, or `false` if `service` or `protocol` is not found.

Examples

Example #1 getservbyname() example

```php
<?php
$services = array('http', 'ftp', 'ssh', 'telnet', 'imap',
'smtp', 'nicname', 'gopher', 'finger', 'pop3', 'www');

foreach ($services as $service) {
    $port = getservbyname($service, 'tcp');
    echo $service . ": " . $port . "<br />\n";
}
?>
```

See Also

- getservbyport() - Get Internet service which corresponds to port and protocol
- » http://www.iana.org/assignments/port-numbers for a complete list of port numbers.

getservbyport

getservbyport

(PHP 4, PHP 5, PHP 7, PHP 8)

getservbyport — Get Internet service which corresponds to port and protocol

Description

function **getservbyport**(int `$port`, string `$protocol`): string|false

getservbyport() returns the Internet service associated with `port` for the specified `protocol` as per */etc/services*.

Parameters

`port`

The port number.

`protocol`

`protocol` is either `"tcp"` or `"udp"` (in lowercase).

Return Values

Returns the Internet service name as a string, or `false` on failure.

See Also

- getservbyname() - Get port number associated with an Internet service and protocol

header

header

(PHP 4, PHP 5, PHP 7, PHP 8)

header — Send a raw HTTP header

Description

function **header**(string `$header`, bool `$replace = true`, int `$response_code = 0`): void

header() is used to send a raw HTTP header. See the » HTTP/1.1 specification for more information on HTTP headers.

Remember that **header()** must be called before any actual output is sent, either by normal HTML tags, blank lines in a file, or from PHP. It is a very common error to read code with include, or require, functions, or another file access function, and have spaces or empty lines that

are output before **header()** is called. The same problem exists when using a single PHP/HTML file.

```
<html>
<?php
/* This will give an error. Note the output
 * above, which is before the header() call */
header('Location: http://www.example.com/');
exit;
?>
```

Parameters

`header`

The header string.

There are two special-case header calls. The first is a header that starts with the string "`HTTP/`" (case is not significant), which will be used to figure out the HTTP status code to send. For example, if you have configured Apache to use a PHP script to handle requests for missing files (using the `ErrorDocument` directive), you may want to make sure that your script generates the proper status code.

```
<?php
// This example illustrates the "HTTP/" special case
// Better alternatives in typical use cases include:
// 1. header($_SERVER["SERVER_PROTOCOL"] . " 404 Not Found");
//    (to override http status messages for clients that are still using HTTP/1.0)
// 2. http_response_code(404); (to use the default message)
header("HTTP/1.1 404 Not Found");
?>
```

The second special case is the "Location:" header. Not only does it send this header back to the browser, but it also returns a `REDIRECT` (302) status code to the browser unless the `201` or a `3xx` status code has already been set.

```
<?php
header("Location: http://www.example.com/"); /* Redirect browser */

/* Make sure that code below does not get executed when we redirect. */
exit;
?>
```

`replace`

The optional `replace` parameter indicates whether the header should replace a previous similar header, or add a second header of the same type. By default it will replace, but if you pass in **`false`** as the second argument you can force multiple headers of the same type. For example:

```
<?php
header('WWW-Authenticate: Negotiate');
header('WWW-Authenticate: NTLM', false);
?>
```

`response_code`

Forces the HTTP response code to the specified value. Note that this parameter only has an effect if the `header` is not empty.

Return Values

No value is returned.

Errors/Exceptions

On failure to schedule the header to be sent, **header()** issues an **`E_WARNING`** level error.

Examples

Example #1 Download dialog

If you want the user to be prompted to save the data you are sending, such as a generated PDF file, you can use the » Content-Disposition header to supply a recommended filename and force the browser to display the save dialog.

```php
<?php
// We'll be outputting a PDF
header('Content-Type: application/pdf');

// It will be called downloaded.pdf
header('Content-Disposition: attachment; filename="downloaded.pdf"');

// The PDF source is in original.pdf
readfile('original.pdf');
?>
```

Example #2 Caching directives

PHP scripts often generate dynamic content that must not be cached by the client browser or any proxy caches between the server and the client browser. Many proxies and clients can be forced to disable caching with:

```php
<?php
header("Cache-Control: no-cache, must-revalidate"); // HTTP/1.1
header("Expires: Sat, 26 Jul 1997 05:00:00 GMT"); // Date in the past
?>
```

> **Note** :
>
> You may find that your pages aren't cached even if you don't output all of the headers above. There are a number of options that users may be able to set for their browser that change its default caching behavior. By sending the headers above, you should override any settings that may otherwise cause the output of your script to be cached.
>
> Additionally, session_cache_limiter() and the `session.cache_limiter` configuration setting can be used to automatically generate the correct caching-related headers when sessions are being used.

Example #3 Setting a cookie

setcookie() provides a convenient way to set cookies. To set a cookie that includes attributes which setcookie() doesn't support, **header()** can be used.

For example, the following sets a cookie that includes a `Partitioned` attribute.

```php
<?php
header('Set-Cookie: name=value; Secure; Path=/; SameSite=None; Partitioned;');
?>
```

Notes

> **Note** :
>
> Headers will only be accessible and output when a SAPI that supports them is in use.

> **Note** :
>
> You can use output buffering to get around this problem, with the overhead of all of your output to the browser being buffered in the server until you send it. You can do this by calling ob_start() and ob_end_flush() in your script, or setting the `output_buffering` configuration directive on in your *php.ini* or server configuration files.

> **Note** :
>
> The HTTP status header line will always be the first sent to the client, regardless of the actual **header()** call being the first or not. The status may be overridden by calling **header()** with a new status line at any time unless the HTTP headers have already been sent.

> **Note** :

> Most contemporary clients accept relative URIs as argument to » Location:, but some older clients require an absolute URI including the scheme, hostname and absolute path. You can usually use *$_SERVER['HTTP_HOST']*, *$_SERVER['PHP_SELF']* and dirname() to make an absolute URI from a relative one yourself:
>
> ```
> <?php
> /* Redirect to a different page in the current directory that was requested */
> $host  = $_SERVER['HTTP_HOST'];
> $uri   = rtrim(dirname($_SERVER['PHP_SELF']), '/\\');
> $extra = 'mypage.php';
> header("Location: http://$host$uri/$extra");
> exit;
> ?>
> ```

> **Note** :
>
> Session ID is not passed with Location header even if session.use_trans_sid is enabled. It must by passed manually using **SID** constant.

See Also

- headers_sent() - Checks if or where headers have been sent
- setcookie() - Send a cookie
- http_response_code() - Get or Set the HTTP response code
- header_remove() - Remove previously set headers
- headers_list() - Returns a list of response headers sent (or ready to send)
- The section on HTTP authentication

header_register_callback

header_register_callback

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

header_register_callback — Call a header function

Description

function **header_register_callback**(callable `$callback`): bool

Registers a function that will be called when PHP starts sending output.

The `callback` is executed just after PHP prepares all headers to be sent, and before any other output is sent, creating a window to manipulate the outgoing headers before being sent.

> **Warning**
>
> Callbacks are not stacked. Subsequent calls to **header_register_callback()** will silently de-register any existing `callback`, regardless of where it was set. Be aware that dependencies may overwrite the callback. Troubleshooting this scenario can be difficult.

Parameters

`callback`

Function called just before the headers are sent. It gets no parameters and the return value is ignored.

Return Values

Returns **true** on success or **false** on failure.

Examples

Example #1 header_register_callback() example

```
<?php

header('Content-Type: text/plain');
header('X-Test: foo');
```

```php
function foo() {
 foreach (headers_list() as $header) {
   if (strpos($header, 'X-Powered-By:') !== false) {
     header_remove('X-Powered-By');
   }
   header_remove('X-Test');
 }
}

$result = header_register_callback('foo');
echo "a";
?>
```

The above example will output something similar to:

```
Content-Type: text/plain

a
```

Notes

header_register_callback() is executed just as the headers are about to be sent out, so any output from this function can break output.

> **Note** :
>
> Headers will only be accessible and output when a SAPI that supports them is in use.

See Also

- headers_list() - Returns a list of response headers sent (or ready to send)
- header_remove() - Remove previously set headers
- header() - Send a raw HTTP header

header_remove

header_remove

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

header_remove — Remove previously set headers

Description

function **header_remove**(?string `$name` = `null`): void

Removes an HTTP header previously set using header().

Parameters

`name`

The header name to be removed. When `null`, all previously set headers are removed.

> **Note** :
>
> This parameter is case-insensitive.

Return Values

No value is returned.

Changelog

Version	Description
8.0.0	`name` is nullable now.

Examples

Example #1 Unsetting specific header.

```
<?php
header("X-Foo: Bar");
header("X-Bar: Baz");
header_remove("X-Foo");
?>
```

The above example will output something similar to:

```
X-Bar: Baz
```

Example #2 Unsetting all previously set headers.

```
<?php
header("X-Foo: Bar");
header("X-Bar: Baz");
header_remove();
?>
```

The above example will output something similar to:

```

```

Notes

> **Caution**
>
> This function will remove *all* headers set by PHP, including cookies, session and the `X-Powered-By` headers.

> **Note** :
>
> Headers will only be accessible and output when a SAPI that supports them is in use.

See Also

- header() - Send a raw HTTP header
- headers_sent() - Checks if or where headers have been sent

headers_list

headers_list

(PHP 5, PHP 7, PHP 8)

headers_list — Returns a list of response headers sent (or ready to send)

Description

function **headers_list**(): array

headers_list() will return a list of headers to be sent to the browser / client. To determine whether or not these headers have been sent yet, use headers_sent().

Parameters

This function has no parameters.

Return Values

Returns a numerically indexed array of headers.

Examples

Example #1 Example using headers_list()

```php
<?php

/* setcookie() will add a response header on its own */
setcookie('foo', 'bar');

/* Define a custom response header
   This will be ignored by most clients */
header("Example-Test: foo");

/* Specify plain text content in our response */
header('Content-Type: text/plain; charset=UTF-8');

/* What headers are going to be sent? */
var_dump(headers_list());

?>
```

The above example will output something similar to:

```
array(3) {
  [0]=>
  string(19) "Set-Cookie: foo=bar"
  [1]=>
  string(17) "Example-Test: foo"
  [2]=>
  string(39) "Content-Type: text/plain; charset=UTF-8"
}
```

Notes

> **Note** :
>
> Headers will only be accessible and output when a SAPI that supports them is in use.

See Also

- headers_sent() - Checks if or where headers have been sent
- header() - Send a raw HTTP header
- setcookie() - Send a cookie
- apache_response_headers() - Fetch all HTTP response headers
- http_response_code() - Get or Set the HTTP response code

headers_sent

headers_sent

(PHP 4, PHP 5, PHP 7, PHP 8)

headers_sent — Checks if or where headers have been sent

Description

function **headers_sent**(string `&$filename = null`, int `&$line = null`): bool

Checks if or where headers have been sent.

You can't add any more header lines using the header() function once the header block has already been sent. Using this function you can at least prevent getting HTTP header related error messages. Another option is to use Output Buffering.

Parameters

`filename`

If the optional `filename` and `line` parameters are set, **headers_sent()** will put the PHP source file name and line number where output started in the `filename` and `line` variables.

> **Note** :
>
> If the output has started before executing the PHP source file (for example due to a startup error), the `filename` parameter will be set to an empty string.

`line`

The line number where the output started.

Return Values

headers_sent() will return `false` if no HTTP headers have already been sent or `true` otherwise.

Examples

Example #1 Examples using headers_sent()

```php
<?php

// If no headers are sent, send one
if (!headers_sent()) {
    header('Location: http://www.example.com/');
    exit;
}

// An example using the optional file and line parameters
// Note that $filename and $linenum are passed in for later use.
// Do not assign them values beforehand.
if (!headers_sent($filename, $linenum)) {
    header('Location: http://www.example.com/');
    exit;

// You would most likely trigger an error here.
} else {

    echo "Headers already sent in $filename on line $linenum\n" .
         "Cannot redirect, for now please click this <a " .
         "href=\"http://www.example.com\">link</a> instead\n";
    exit;
}

?>
```

Notes

> **Note** :
>
> Headers will only be accessible and output when a SAPI that supports them is in use.

See Also

- ob_start() - Turn on output buffering
- trigger_error() - Generates a user-level error/warning/notice message
- headers_list() - Returns a list of response headers sent (or ready to send)
- header() - Send a raw HTTP header for a more detailed discussion of the matters involved.

http_clear_last_response_headers

http_clear_last_response_headers

(PHP 8 >= 8.4.0)

http_clear_last_response_headers — Clears the stored HTTP response headers

Description

function **http_clear_last_response_headers**(): void

Clears the HTTP response headers that were received using the HTTP wrapper.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- http_get_last_response_headers() - Retrieve last HTTP response headers

http_get_last_response_headers

http_get_last_response_headers

(PHP 8 >= 8.4.0)

http_get_last_response_headers — Retrieve last HTTP response headers

Description

function **http_get_last_response_headers**(): ?array

Returns an array containing the last HTTP response headers received via the HTTP wrapper. If there are none, `null` is returned instead.

Parameters

This function has no parameters.

Return Values

Returns an indexed array of HTTP headers which were received while using the HTTP wrapper. If there are none, `null` is returned instead.

Examples

Example #1 http_get_last_response_headers() example

Description.

```
<?php
file_get_contents("http://example.com");
var_dump(http_get_last_response_headers());
?>
```

The above example will output something similar to:

```
```

```
array(14) {
  [0]=>
  string(15) "HTTP/1.1 200 OK"
  [1]=>
  string(20) "Accept-Ranges: bytes"
  [2]=>
  string(11) "Age: 326940"
  [3]=>
  string(29) "Cache-Control: max-age=604800"
  [4]=>
  string(38) "Content-Type: text/html; charset=UTF-8"
  [5]=>
  string(35) "Date: Mon, 11 Nov 2024 13:34:09 GMT"
  [6]=>
  string(23) "Etag: "3147526947+gzip""
  [7]=>
  string(38) "Expires: Mon, 18 Nov 2024 13:34:09 GMT"
  [8]=>
  string(44) "Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT"
  [9]=>
  string(24) "Server: ECAcc (nyd/D16C)"
  [10]=>
  string(21) "Vary: Accept-Encoding"
  [11]=>
  string(12) "X-Cache: HIT"
  [12]=>
  string(20) "Content-Length: 1256"
  [13]=>
  string(17) "Connection: close"
}
```

See Also

- http_clear_last_response_headers() - Clears the stored HTTP response headers

http_response_code

http_response_code

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

http_response_code — Get or Set the HTTP response code

Description

function **http_response_code**(int `$response_code` = 0): int|bool

Gets or sets the HTTP response status code.

Parameters

`response_code`

The optional `response_code` will set the response code.

Return Values

If `response_code` is provided, then the previous status code will be returned. If `response_code` is not provided, then the current status code will be returned. Both of these values will default to a `200` status code if used in a web server environment.

`false` will be returned if `response_code` is not provided and it is not invoked in a web server environment (such as from a CLI application). `true` will be returned if `response_code` is provided and it is not invoked in a web server environment (but only when no previous response status has been set).

Examples

Example #1 Using http_response_code() in a web server environment

```php
<?php

// Get the current response code and set a new one
var_dump(http_response_code(404));

// Get the new response code
var_dump(http_response_code());
?>
```

The above example will output:

```
int(200)
int(404)
```

Example #2 Using http_response_code() in a CLI environment

```php
<?php

// Get the current default response code
var_dump(http_response_code());

// Set a response code
var_dump(http_response_code(201));

// Get the new response code
var_dump(http_response_code());
?>
```

The above example will output:

```
bool(false)
bool(true)
int(201)
```

See Also

- header() - Send a raw HTTP header
- headers_list() - Returns a list of response headers sent (or ready to send)

inet_ntop

inet_ntop

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

inet_ntop — Converts a packed internet address to a human readable representation

Description

function **inet_ntop**(string `$ip`): string|false

This function converts a 32bit IPv4, or 128bit IPv6 address (if PHP was built with IPv6 support enabled) into an address family appropriate string representation.

Parameters

`ip`

A 32bit IPv4, or 128bit IPv6 address.

Return Values

Returns a string representation of the address or `false` on failure.

Examples

Example #1 inet_ntop() Example

```
<?php
$packed = chr(127) . chr(0) . chr(0) . chr(1);
$expanded = inet_ntop($packed);

/* Outputs: 127.0.0.1 */
echo $expanded;

$packed = str_repeat(chr(0), 15) . chr(1);
$expanded = inet_ntop($packed);

/* Outputs: ::1 */
echo $expanded;
?>
```

See Also

- long2ip() - Converts a long integer address into a string in (IPv4) Internet standard dotted format
- ip2long() - Converts a string containing an (IPv4) Internet Protocol dotted address into a long integer
- inet_pton() - Converts a human readable IP address to its packed in_addr representation

inet_pton

inet_pton

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

inet_pton — Converts a human readable IP address to its packed in_addr representation

Description

function **inet_pton**(string `$ip`): string|false

This function converts a human readable IPv4 or IPv6 address (if PHP was built with IPv6 support enabled) into an address family appropriate 32bit or 128bit binary structure.

Parameters

`ip`

A human readable IPv4 or IPv6 address.

Return Values

Returns the `in_addr` representation of the given `ip`, or `false` if a syntactically invalid `ip` is given (for example, an IPv4 address without dots or an IPv6 address without colons).

Examples

Example #1 inet_pton() Example

```
<?php
$in_addr = inet_pton('127.0.0.1');

$in6_addr = inet_pton('::1');
?>
```

See Also

- ip2long() - Converts a string containing an (IPv4) Internet Protocol dotted address into a long integer
- long2ip() - Converts a long integer address into a string in (IPv4) Internet standard dotted format

- inet_ntop() - Converts a packed internet address to a human readable representation

ip2long

ip2long

(PHP 4, PHP 5, PHP 7, PHP 8)

ip2long — Converts a string containing an (IPv4) Internet Protocol dotted address into a long integer

Description

function **ip2long**(string `$ip`): int|false

The function **ip2long()** generates a long integer representation of IPv4 Internet network address from its Internet standard format (dotted string) representation.

ip2long() will also work with non-complete IP addresses. Read » http://ps-2.kev009.com/wisclibrary/aix52/usr/share/man/info/en_US/a_doc_lib/libs/commtrf2/inet_addr.htm for more info.

Parameters

`ip`

A standard format address.

Return Values

Returns the long integer or `false` if `ip` is invalid.

Examples

Example #1 ip2long() Example

```
<?php
$ip = gethostbyname('www.example.com');
$out = "The following URLs are equivalent:<br />\n";
$out .= 'http://www.example.com/, http://' . $ip . '/, and http://' . sprintf("%u", ip2long($ip)) . "/<br />\n";
echo $out;
?>
```

Example #2 Displaying an IP address

This second example shows how to print a converted address with the printf() function:

```
<?php
$ip   = gethostbyname('www.example.com');
$long = ip2long($ip);

if ($long == -1 || $long === FALSE) {
    echo 'Invalid IP, please try again';
} else {
    echo $ip   . "\n";            // 192.0.34.166
    echo $long . "\n";            // 3221234342 (-1073732954 on 32-bit systems, due to integer overflow)
    printf("%u\n", ip2long($ip)); // 3221234342
}
?>
```

Notes

> **Note** :
>
> Because PHP's int type is signed, and many IP addresses will result in negative integers on 32-bit architectures, you need to use the "%u" formatter of sprintf() or printf() to get the string representation of the unsigned IP address.

> **Note** :

> **ip2long()** will return `-1` for the IP `255.255.255.255` on 32-bit systems due to the integer value overflowing.

See Also

- long2ip() - Converts a long integer address into a string in (IPv4) Internet standard dotted format
- sprintf() - Return a formatted string

long2ip

long2ip

(PHP 4, PHP 5, PHP 7, PHP 8)

long2ip — Converts a long integer address into a string in (IPv4) Internet standard dotted format

Description

function **long2ip**(int `$ip`): string

The function **long2ip()** generates an Internet address in dotted format (i.e.: aaa.bbb.ccc.ddd) from the long integer representation.

Parameters

`ip`

A proper address representation in long integer.

Return Values

Returns the Internet IP address as a string.

Changelog

Version	Description
8.4.0	Return type changed from string|false to string.
7.1.0	The parameter type of `ip` has been changed from string to int.

Notes

> **Note** :
>
> On 32-bit architectures, casting integer representations of IP addresses from string to int will not give correct results for numbers which exceed `PHP_INT_MAX`.

See Also

- ip2long() - Converts a string containing an (IPv4) Internet Protocol dotted address into a long integer

net_get_interfaces

net_get_interfaces

(PHP 7 >= 7.3, PHP 8)

net_get_interfaces — Get network interfaces

Description

function **net_get_interfaces**(): array|false

Returns an enumeration of network interfaces (adapters) on the local machine.

Parameters

This function has no parameters.

Return Values

Returns an associative array where the key is the name of the interface and the value an associative array of interface attributes, or `false` on failure.

Each interface associative array contains:

Interface attributes

Name	Description
description	Optional string value for description of the interface. Windows only.
mac	Optional string value for MAC address of the interface. Windows only.
mtu	Integer value for Maximum transmission unit (MTU) of the interface. Windows only.
unicast	Array of associative arrays, see Unicast attributes below.
up	Boolean status (on/off) for interface.

Unicast attributes

Name	Description
flags	Integer value.
family	Integer value.
address	String value for address in either IPv4 or IPv6.
netmask	String value for netmask in either IPv4 or IPv6.

Errors/Exceptions

Emits an `E_WARNING` on failure to get interface information.

openlog

openlog

(PHP 4, PHP 5, PHP 7, PHP 8)

openlog — Open connection to system logger

Description

function **openlog**(string `$prefix`, int `$flags`, int `$facility`): true

openlog() opens a connection to the system logger for a program.

The use of **openlog()** is optional. It will automatically be called by syslog() if necessary, in which case `prefix` will default to the empty string.

Parameters

`prefix`

The string `prefix` is added to each message.

`flags`

Bitmask of the following constants:

- `LOG_CONS`
- `LOG_NDELAY`
- `LOG_ODELAY`
- `LOG_NOWAIT`
- `LOG_PERROR`
- `LOG_PID`

`facility`

The `facility` argument is used to specify what type of program is logging the message. This lets the configuration file specify that messages from different facilities will be handled differently. Must be one of the following constants:

- `LOG_AUTH`
- `LOG_AUTHPRIV`

- `LOG_CRON`
- `LOG_DAEMON`
- `LOG_KERN`
- `LOG_LOCAL[0-7]`
- `LOG_LPR`
- `LOG_MAIL`
- `LOG_NEWS`
- `LOG_SYSLOG`
- `LOG_USER`
- `LOG_UUCP`

> **Note** :
>
> This parameter is ignored on Windows.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The function now always returns `true`. Previously it returned `false` on failure.

See Also

- syslog() - Generate a system log message
- closelog() - Close connection to system logger

pfsockopen

pfsockopen

(PHP 4, PHP 5, PHP 7, PHP 8)

pfsockopen — Open persistent Internet or Unix domain socket connection

Description

```
function pfsockopen(
    string $hostname,
    int $port = -1,
    int &$error_code = null,
    string &$error_message = null,
    ?float $timeout = null
): resource|false
```

This function behaves exactly as fsockopen() with the difference that the connection is not closed after the script finishes. It is the persistent version of fsockopen().

Parameters

For parameter information, see the fsockopen() documentation.

Return Values

pfsockopen() returns a file pointer which may be used together with the other file functions (such as fgets(), fgetss(), fwrite(), fclose(), and feof()), or `false` on failure.

Changelog

Version	Description
8.0.0	`timeout` is nullable now.

See Also

- fsockopen() - Open Internet or Unix domain socket connection

request_parse_body

request_parse_body

(PHP 8 >= 8.4.0)

request_parse_body — Read and parse the request body and return the result

Description

function **request_parse_body**(?array `$options = null`): array

This function reads the request body and parses it according to the `Content-Type` header. Currently, two content types are supported:

- `application/x-www-form-urlencoded`
- `multipart/form-data`

This function is used primarily to parse `multipart/form-data` requests with HTTP verbs other than `POST` which do not automatically populate the *$_POST* and *$_FILES* superglobals.

> **Caution**
>
> The request body can only be consumed once. **request_parse_body()** consumes the request body without buffering it to the `php://input` stream. Conversely, if the body has already been read (e.g. via `php://input`), **request_parse_body()** will return empty data.

Parameters

`options`

The `options` parameter accepts an associative array to override the following global *php.ini* settings for parsing of the request body.

- `max_file_uploads`
- `max_input_vars`
- `max_multipart_body_parts`
- `post_max_size`
- `upload_max_filesize`

Return Values

request_parse_body() returns an array pair with the equivalent of *$_POST* at index `0` and *$_FILES* at index `1`.

Errors/Exceptions

When the request body is invalid, according to the `Content-Type` header, a RequestParseBodyException is thrown.

A ValueError is thrown when `options` contains invalid keys, or invalid values for the corresponding key.

Examples

Example #1 request_parse_body() example

```php
<?php
// Parse request and store result in the $_POST and $_FILES superglobals.
[$_POST, $_FILES] = request_parse_body();
// Echo the content of some transferred file
echo file_get_contents($_FILES['file_name']['tmp_name']);
?>
```

Example #2 request_parse_body() example with customized options

```php
<?php
// form.php
```

```php
assert_logged_in();

// Only for this form, we allow a bigger upload size.
[$_POST, $_FILES] = request_parse_body([
    'post_max_size' => '10M',
    'upload_max_filesize' => '10M',
]);

// Do something with the uploaded files.
?>
```

setcookie

setcookie

(PHP 4, PHP 5, PHP 7, PHP 8)

setcookie — Send a cookie

Description

function **setcookie**(
  string $name,
  string $value = "",
  int $expires_or_options = 0,
  string $path = "",
  string $domain = "",
  bool $secure = false,
  bool $httponly = false
): bool

Alternative signature available as of PHP 7.3.0 (not supported with named parameters):

function **setcookie**(string $name, string $value = "", array $options = []): bool

setcookie() defines a cookie to be sent along with the rest of the HTTP headers. Like other headers, cookies must be sent *before* any output from the script (this is a protocol restriction). This requires that this function is called prior to any output, including `<html>` and `<head>` tags as well as any whitespace.

Once the cookies have been set, they can be accessed on the next page load with the *$_COOKIE* array. Cookie values may also exist in *$_REQUEST*.

Parameters

» RFC 6265 provides the normative reference on how each **setcookie()** parameter is interpreted.

`name`

The name of the cookie.

`value`

The value of the cookie. This value is stored on the clients computer; do not store sensitive information. Assuming the `name` is `'cookiename'`, this value is retrieved through *$_COOKIE['cookiename']*

`expires_or_options`

The time the cookie expires. This is a Unix timestamp so is in number of seconds since the epoch. One way to set this is by adding the number of seconds before the cookie should expire to the result of calling time(). For instance, `time()+60*60*24*30` will set the cookie to expire in 30 days. Another option is to use the mktime() function. If set to `0`, or omitted, the cookie will expire at the end of the session (when the browser closes).

> **Note** :
>
> The `expires_or_options` parameter takes a Unix timestamp, as opposed to the date format `Wdy, DD-Mon-YYYY HH:MM:SS GMT`, because PHP does this conversion internally.

`path`

The path on the server in which the cookie will be available on. If set to `'/'`, the cookie will be available within the entire `domain`. If set to `'/foo/'`, the cookie will only be available within the `/foo/` directory and all sub-directories such as `/foo/bar/` of `domain`. The default value is the current directory that the cookie is being set in.

`domain`

The (sub)domain that the cookie is available to. Setting this to a subdomain (such as `'www.example.com'`) will make the cookie available to that subdomain and all other sub-domains of it (i.e. w2.www.example.com). To make the cookie available to the whole domain (including all subdomains of it), simply set the value to the domain name (`'example.com'`, in this case).

Older browsers still implementing the deprecated » RFC 2109 may require a leading `.` to match all subdomains.

`secure`

Indicates that the cookie should only be transmitted over a secure HTTPS connection from the client. When set to **`true`**, the cookie will only be set if a secure connection exists. On the server-side, it's on the programmer to send this kind of cookie only on secure connection (e.g. with respect to *$_SERVER["HTTPS"]*).

`httponly`

When **`true`** the cookie will be made accessible only through the HTTP protocol. This means that the cookie won't be accessible by scripting languages, such as JavaScript. It has been suggested that this setting can effectively help to reduce identity theft through XSS attacks (although it is not supported by all browsers), but that claim is often disputed. **`true`** or **`false`**

`options`

An associative array which may have any of the keys `expires`, `path`, `domain`, `secure`, `httponly`, `partitioned` and `samesite`.

The values have the same meaning as described for the parameters with the same name. The value of the `samesite` element should be either `None`, `Lax` or `Strict`. If any of the allowed options are not given, their default values are the same as the default values of the explicit parameters. If the `samesite` element is omitted, no SameSite cookie attribute is set.

> **Note** :
>
> To set a cookie that includes attributes that aren't among the keys listed, use header().

> **Note** :
>
> If `samesite` is `"None"` then `secure` must also be enabled or the cookie will be blocked by the client.

> **Note** :
>
> If `partitioned` is **`true`**, then `secure` must also be enabled, otherwise a ValueError is thrown.

Return Values

If output exists prior to calling this function, **setcookie()** will fail and return **`false`**. If **setcookie()** successfully runs, it will return **`true`**. This does not indicate whether the user accepted the cookie.

Errors/Exceptions

If the `options` array contains unsupported keys:

- Prior to PHP 8.0.0, an **`E_WARNING`** was generated.
- As of PHP 8.0.0, a ValueError is thrown.

Changelog

Version	Description
8.5.0	The "partitioned" entry was added in the `options` array. If set to `true`, the cookie will be marked as Partitioned (CHIPS).
8.2.0	The date format of the cookie is now `'D, d M Y H:i:s \G\M\T'`; previously it was `'D, d-M-Y H:i:s T'`.
8.0.0	Passing unsupported keys now throws a ValueError instead of emitting an **`E_WARNING`**.

Version	Description
7.3.0	An alternative signature supporting an `options` array has been added. This signature supports also setting of the SameSite cookie attribute.

Examples

The effects of the following examples can be observed using the browser developer tools cookie list (usually in the Storage or Application tab).

Example #1 setcookie() send example

```php
<?php

$value = 'something from somewhere';

// Set a "session cookie" that expires when the browser is closed
setcookie("TestCookie", $value);
// Set a cookie that expires in 1 hour
setcookie("TestCookie", $value, time()+3600);
// Set a cookie that applies only to a specific path on a specific domain
// Note that the domain used should match the site domain
setcookie("TestCookie", $value, time()+3600, "/~rasmus/", "example.com", true);

?>
```

Note that the value portion of the cookie will automatically be urlencoded and decoded by PHP. This can be avoided by using setrawcookie() instead.

To see the contents of the cookies set in the above example on a later request:

```php
<?php
// Print an individual cookie
echo $_COOKIE["TestCookie"];

// Another way to debug/test is to view all cookies
print_r($_COOKIE);
?>
```

Example #2 setcookie() delete example

To delete a cookie set the expiration date to a value in the past (but not zero, which is reserved for session cookies).

To delete the cookies set in previous example:

```php
<?php
// set the expiration date to one hour ago
setcookie("TestCookie", "", time() - 3600);
setcookie("TestCookie", "", time() - 3600, "/~rasmus/", "example.com", 1);
?>
```

Example #3 setcookie() and arrays

An "array of cookies" can be set by using array notation in the cookie name. This has the effect of setting as many cookies as there are array elements, but when the cookie is received by the script, the values are all placed in an array with the cookie's name:

```php
<?php
// set the cookies
setcookie("cookie[three]", "cookiethree");
setcookie("cookie[two]", "cookietwo");
setcookie("cookie[one]", "cookieone");

// after the page reloads, print them out
if (isset($_COOKIE['cookie'])) {
    foreach ($_COOKIE['cookie'] as $name => $value) {
        $name = htmlspecialchars($name);
        $value = htmlspecialchars($value);
        echo "$name : $value <br />\n";
    }
```

```
}
?>
```

The above example will output:

```
three : cookiethree
two : cookietwo
one : cookieone
```

> **Note** :
>
> Using separator characters such as `[` and `]` as part of the cookie name is not compliant to RFC 6265, section 4, but supposed to be supported by user agents according to RFC 6265, section 5.

Notes

> **Note** :
>
> Output buffering can be used to allow script output prior to the call of this function. All output will be buffered until it is flushed (either explicitly or at the end of the script execution). This is done by calling ob_start() and ob_end_flush() in the script, or setting the `output_buffering` configuration directive on in the *php.ini* or server configuration files.

Common Pitfalls:

- Cookies will not become visible until the next loading of a page that the cookie should be visible for. To test if a cookie was successfully set, check for the cookie on a next loading page before the cookie expires. Expire time is set via the `expires_or_options` parameter. A nice way to debug the existence of cookies is by simply calling `print_r($_COOKIE);`.
- Cookies must be deleted with the same parameters as they were set with. If the `value` argument is an empty string, and all other arguments match a previous call to **setcookie()**, then the cookie with the specified name will be deleted from the remote client. This is internally achieved by setting value to `'deleted'` and expiration time in the past.
- Because setting a cookie with a value of `false` will try to delete the cookie, boolean values should not be used. Instead, use *0* for `false` and *1* for `true`.
- Cookies names can be set as array names and will be available to the PHP scripts as arrays but separate cookies are stored by the browser. Consider json_encode() to set one cookie with multiple names and values. It is not recommended to use serialize() for this purpose because it can result in security holes.

Multiple calls to **setcookie()** are performed in the order called.

See Also

- header() - Send a raw HTTP header
- setrawcookie() - Send a cookie without urlencoding the cookie value
- cookies section
- » RFC 6265
- » RFC 2109

setrawcookie

setrawcookie

(PHP 5, PHP 7, PHP 8)

setrawcookie — Send a cookie without urlencoding the cookie value

Description

```
function setrawcookie(
    string $name,
    string $value = ?,
    int $expires_or_options = 0,
    string $path = ?,
    string $domain = ?,
    bool $secure = false,
    bool $httponly = false
): bool
```

Alternative signature available as of PHP 7.3.0 (not supported with named parameters):

function **setrawcookie**(string `$name`, string `$value` = ?, array `$options` = []): bool

setrawcookie() is exactly the same as setcookie() except that the cookie value will not be automatically urlencoded when sent to the browser.

Parameters

For parameter information, see the setcookie() documentation.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.5.0	The "partitioned" entry was added in the `options` array. If set to `true`, the cookie will be marked as Partitioned (CHIPS).
7.3.0	An alternative signature supporting an `options` array has been added. This signature supports also setting of the SameSite cookie attribute.

See Also

- setcookie() - Send a cookie

socket_get_status

socket_get_status

(PHP 4, PHP 5, PHP 7, PHP 8)

socket_get_status — Alias of stream_get_meta_data()

Description

This function is an alias of: stream_get_meta_data().

socket_set_blocking

socket_set_blocking

(PHP 4, PHP 5, PHP 7, PHP 8)

socket_set_blocking — Alias of stream_set_blocking()

Description

This function is an alias of: stream_set_blocking().

socket_set_timeout

socket_set_timeout

(PHP 4, PHP 5, PHP 7, PHP 8)

socket_set_timeout — Alias of stream_set_timeout()

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

This function is an alias of: stream_set_timeout().

See Also

- stream_set_timeout() - Set timeout period on a stream

syslog

syslog

(PHP 4, PHP 5, PHP 7, PHP 8)

syslog — Generate a system log message

Description

function **syslog**(int `$priority`, string `$message`): true

syslog() generates a log message that will be distributed by the system logger.

For information on setting up a user defined log handler, see the syslog.conf (5) Unix manual page. More information on the syslog facilities and option can be found in the man pages for syslog (3) on Unix machines.

Parameters

`priority`

One of the `LOG_EMERG`, `LOG_ALERT`, `LOG_CRIT`, `LOG_ERR`, `LOG_WARNING`, `LOG_NOTICE`, `LOG_INFO`, `LOG_DEBUG` constants.

`message`

The message to send.

Return Values

Always returns `true`.

Examples

Example #1 Using **syslog()**

```php
<?php
// open syslog, include the process ID and also send
// the log to standard error, and use a user defined
// logging mechanism
openlog("myScriptLog", LOG_PID | LOG_PERROR, LOG_LOCAL0);

// some code

if (authorized_client()) {
    // do something
} else {
    // unauthorized client!
    // log the attempt
    $access = date("Y/m/d H:i:s");
    syslog(LOG_WARNING, "Unauthorized client: $access {$_SERVER['REMOTE_ADDR']} ({$_SERVER['HTTP_USER_AGENT']})");
}

closelog();
?>
```

Notes

On Windows, the syslog service is emulated using the Event Log.

> **Note** :
>
> Use of **LOG_LOCAL0** through LOG_LOCAL7 for the facility parameter of openlog() is not available in Windows.

See Also

- openlog() - Open connection to system logger
- closelog() - Close connection to system logger
- syslog.filter INI setting (starting with PHP 7.3)

Password Hashing

Introduction

The password hashing API provides an easy to use wrapper around crypt() and some other password hashing algorithms, to make it easy to create and manage passwords in a secure manner.

Installing/Configuring

Table of Contents

Requirements

No external libraries are needed to build this extension.

For Argon2 password hashing, either » libargon2 is required or, as of PHP 8.4.0, OpenSSL version 3.2 or later. As of PHP 7.3.0, libargon2 version 20161029 or later is required if libargon2 is used.

Installation

There is no installation needed to use these functions; they are part of the PHP core.

However, to enable Argon2 password hashing, PHP must be built either with libargon2 support using the **--with-password-argon2** configure option or, starting from PHP 8.4.0, with OpenSSL using **--with-openssl** and **--with-openssl-argon2**.

Prior to PHP 8.1.0, it was possible to specify the argon2 directory with **--with-password-argon2[=DIR]**.

Predefined Constants

The constants below are always available as part of the PHP core.

`PASSWORD_BCRYPT` (string)

> `PASSWORD_BCRYPT` is used to create new password hashes using the `CRYPT_BLOWFISH` algorithm.
>
> This will always result in a hash using the "$2y$" crypt format, which is always 60 characters wide.
>
> Supported Options:
>
> - `salt` (string) - to manually provide a salt to use when hashing the password. Note that this will override and prevent a salt from being automatically generated.
>
> If omitted, a random salt will be generated by password_hash() for each password hashed. This is the intended mode of operation and as of PHP 7.0.0 the salt option has been deprecated.
>
> - `cost` (int) - which denotes the algorithmic cost that should be used. Examples of these values can be found on the crypt() page.
>
> If omitted, a default value of `12` will be used. This is a good baseline cost, but you may want to consider increasing it depending on your hardware.

`PASSWORD_BCRYPT_DEFAULT_COST` (int)

> Default algorithmic cost used by password_hash() when hashing passwords with `PASSWORD_BCRYPT`.
>
> Available as of PHP 7.0.0.

`PASSWORD_ARGON2I` (string)

> `PASSWORD_ARGON2I` is used to create new password hashes using the Argon2i algorithm.
>
> Supported Options:
>
> - `memory_cost` (int) - Maximum memory (in kibibytes) that may be used to compute the Argon2 hash. Defaults to `PASSWORD_ARGON2_DEFAULT_MEMORY_COST`.

- `time_cost` (int) - Number of iterations (passes) used to compute the Argon2 hash. Defaults to `PASSWORD_ARGON2_DEFAULT_TIME_COST`.
- `threads` (int) - Number of threads to use for computing the Argon2 hash. Defaults to `PASSWORD_ARGON2_DEFAULT_THREADS`. Only available with libargon2, not with libsodium implementation.

Available as of PHP 7.2.0.

`PASSWORD_ARGON2ID` (string)

`PASSWORD_ARGON2ID` is used to create new password hashes using the Argon2id algorithm. It supports the same options as `PASSWORD_ARGON2I`.

Available as of PHP 7.3.0.

`PASSWORD_ARGON2_DEFAULT_MEMORY_COST` (int)

Default amount of memory in kibibytes (KiB) that will be used to compute a hash.

Available as of PHP 7.2.0.

`PASSWORD_ARGON2_DEFAULT_TIME_COST` (int)

Default number of iterations (passes) used to compute a hash.

Available as of PHP 7.2.0.

`PASSWORD_ARGON2_DEFAULT_THREADS` (int)

Default number of threads that Argon2lib will use. Not available with libsodium implementation.

Available as of PHP 7.2.0.

`PASSWORD_ARGON2_PROVIDER` (string)

Available as of PHP 7.4.0.

`PASSWORD_DEFAULT` (string)

The default algorithm to use for hashing if no algorithm is provided. This may change in newer PHP releases when newer, stronger hashing algorithms are supported.

It is worth noting that over time this constant can change. It is thus important to be aware that the length of the resulting hash can change. Therefore, when using `PASSWORD_DEFAULT` the resulting hash must be stored in a way that can store arbitrary hashes, the recommended width is `255` bytes.

Currently is an alias for `PASSWORD_BCRYPT`.

Changelog

Version	Description
7.4.0	The values of the password algo IDs (`PASSWORD_BCRYPT`, `PASSWORD_ARGON2I`, `PASSWORD_ARGON2ID` and `PASSWORD_DEFAULT`) are now strings. Previously, they have been ints.

Password Hashing Functions

Table of Contents

password_algos

password_algos

(PHP 7 >= 7.4.0, PHP 8)

password_algos — Get available password hashing algorithm IDs

Description

function **password_algos**(): array

Returns a complete list of all registered password hashing algorithm IDs as an array of strings.

Parameters

This function has no parameters.

Return Values

Returns the available password hashing algorithm IDs.

Examples

Example #1 Basic password_algos() usage

```
<?php
print_r(password_algos());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => 2y
    [1] => argon2i
    [2] => argon2id
)
```

password_get_info

password_get_info

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_get_info — Returns information about the given hash

Description

function **password_get_info**(string `$hash`): array

When passed in a valid hash created by an algorithm supported by password_hash(), this function will return an array of information about that hash.

Parameters

`hash`

A hash created by password_hash().

Return Values

Returns an associative array with three elements:

- `algo`, which will match a password algorithm constant
- `algoName`, which has the human readable name of the algorithm
- `options`, which includes the options provided when calling password_hash()

password_hash

password_hash

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_hash — Creates a password hash

Description

function **password_hash**(#[\SensitiveParameter] string `$password`, string|int|null `$algo`, array `$options = []`): string

password_hash() creates a new password hash using a strong one-way hashing algorithm.

The following algorithms are currently supported:

- `PASSWORD_DEFAULT` - Use the bcrypt algorithm (default as of PHP 5.5.0). Note that this constant is designed to change over time as new and stronger algorithms are added to PHP. For that reason, the length of the result from using this identifier can change over time. Therefore, it is recommended to store the result in a database column that can expand beyond 60 bytes (255 bytes would be a good choice).
- `PASSWORD_BCRYPT` - Use the bcrypt algorithm to create the hash. This will produce a standard crypt() compatible hash using the `$2y$` identifier.
- `PASSWORD_ARGON2I` - Use the Argon2i hashing algorithm to create the hash. This algorithm is only available if PHP has been compiled with Argon2 support.
- `PASSWORD_ARGON2ID` - Use the Argon2id hashing algorithm to create the hash. This algorithm is only available if PHP has been compiled with Argon2 support.

Supported options for `PASSWORD_BCRYPT`:

- `salt` (string) - to manually provide a salt to use when hashing the password. Note that this will override and prevent a salt from being automatically generated.

  If omitted, a random salt will be generated by **password_hash()** for each password hashed. This is the intended mode of operation.

  > **Warning**
  >
  > The salt option is deprecated. It is now preferred to simply use the salt that is generated by default. As of PHP 8.0.0, an explicitly given salt is ignored.

- `cost` (int) - which denotes the algorithmic cost that should be used. Examples of these values can be found on the crypt() page.

  If omitted, a default value of `12` will be used. This is a good baseline cost, but it should be adjusted depending on hardware used.

Supported options for `PASSWORD_ARGON2I` and `PASSWORD_ARGON2ID`:

- `memory_cost` (int) - Maximum memory (in kibibytes) that may be used to compute the Argon2 hash. Defaults to `PASSWORD_ARGON2_DEFAULT_MEMORY_COST`.
- `time_cost` (int) - Number of iterations (passes) used to compute the Argon2 hash. Defaults to `PASSWORD_ARGON2_DEFAULT_TIME_COST`.
- `threads` (int) - Number of threads to use for computing the Argon2 hash. Defaults to `PASSWORD_ARGON2_DEFAULT_THREADS`.

  > **Warning**
  >
  > Only available when PHP uses libargon2, not with libsodium implementation.

Parameters

`password`

The user's password.

> **Caution**
>
> Using the `PASSWORD_BCRYPT` as the algorithm, will result in the `password` parameter being truncated to a maximum length of 72 bytes.

`algo`

A password algorithm constant denoting the algorithm to use when hashing the password.

`options`

An associative array containing options. See the password algorithm constants for documentation on the supported options for each algorithm.

Return Values

Returns the hashed password.

The used algorithm, cost and salt are returned as part of the hash. Therefore, all information that's needed to verify the hash is included in it. This allows the password_verify() function to verify the hash without needing separate storage for the salt or algorithm information.

Changelog

Version	Description
8.4.0	The default value of the `cost` option of the **PASSWORD_BCRYPT** algorithm was increased from `10` to `12`.
8.3.0	**password_hash()** now sets the underlying Random\RandomException as the Exception::$previous exception when a ValueError is thrown due to a failure in the salt generation.
8.0.0	**password_hash()** no longer returns **false** on failure, instead a ValueError will be thrown if the password hashing algorithm is not valid, or an Error if the password hashing failed for an unknown error.
8.0.0	The `algo` parameter is nullable now.
7.4.0	The `algo` parameter expects a string now, but still accepts ints for backward compatibility.
7.4.0	The sodium extension provides an alternative implementation for Argon2 passwords.
7.3.0	Support for Argon2id passwords using **PASSWORD_ARGON2ID** was added.
7.2.0	Support for Argon2i passwords using **PASSWORD_ARGON2I** was added.

Examples

Example #1 password_hash() example

```php
<?php
echo password_hash("rasmuslerdorf", PASSWORD_DEFAULT);
?>
```

The above example will output something similar to:

```
$2y$12$4Umg0rCJwMswRw/l.SwHvuQV01coP0eWmGzd61QH2RvAOMANUBGC.
```

Example #2 password_hash() example setting cost manually

```php
<?php
$options = [
    // Increase the bcrypt cost from 12 to 13.
    'cost' => 13,
];
echo password_hash("rasmuslerdorf", PASSWORD_BCRYPT, $options);
?>
```

The above example will output something similar to:

```
$2y$13$xeDfQumlmdm0Sco.4qmH1OGfUUmOcuRmfae0dPJhjX1Bq0yYhqbNi
```

Example #3 password_hash() example finding a good cost

This code will benchmark the machine to determine how high of a cost can be used without deteriorating user experience. It is recommended to set the highest cost that does not slow down other operations the machine needs to perform. 11 is a good baseline, and more is better if the machine is fast enough. The code below aims for ≤ 350 milliseconds stretching time, which is an appropriate delay for systems handling interactive logins.

```php
<?php
$timeTarget = 0.350; // 350 milliseconds

$cost = 11;
do {
    $cost++;
    $start = microtime(true);
    password_hash("test", PASSWORD_BCRYPT, ["cost" => $cost]);
    $end = microtime(true);
} while (($end - $start) < $timeTarget);

echo "Appropriate Cost Found: " . $cost - 1;
?>
```

The above example will output something similar to:

```
Appropriate Cost Found: 13
```

Example #4 password_hash() example using Argon2i

```php
<?php
echo 'Argon2i hash: ' . password_hash('rasmuslerdorf', PASSWORD_ARGON2I);
?>
```

The above example will output something similar to:

```
Argon2i hash: $argon2i$v=19$m=1024,t=2,p=2$YzJBSzV4TUhkMzc3d3laeg$zqU/1IN0/AogfP4cmSJI1vc8lpXRW9/S0sYY2i2jHT0
```

Notes

> **Caution**
>
> It is strongly recommended not to provide an explicit salt for this function. A secure salt will automatically be created if no salt is specified.
>
> As noted above, providing the `salt` option in PHP 7.0.0 will generate a deprecation warning. Support for providing a salt explicitly has been removed in PHP 8.0.0.

> **Note** :
>
> It is recommended to test this function on the machine used, adjusting the cost parameter(s) so that execution of the function takes less than 350 milliseconds for interactive logins. The script in the above example will help choosing an appropriate bcrypt cost for the given machine.

> **Note** :
>
> Updates to supported algorithms by this function (or changes to the default one) must follow the following rules:

> - Any new algorithm must be in core for at least 1 full release of PHP prior to becoming default. So if, for example, a new algorithm is added in 7.5.5, it would not be eligible for default until 7.7 (since 7.6 would be the first full release). But if a different algorithm was added in 7.6.0, it would also be eligible for default at 7.7.0.
> - The default should only change in a full release (7.3.0, 8.0.0, etc) and not in a revision release. The only exception to this is in an emergency when a critical security flaw is found in the current default.

See Also

- password_verify() - Verifies that a password matches a hash
- password_needs_rehash() - Checks if the given hash matches the given options
- crypt() - One-way string hashing
- sodium_crypto_pwhash_str() - Get an ASCII-encoded hash

password_needs_rehash

password_needs_rehash

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_needs_rehash — Checks if the given hash matches the given options

Description

function **password_needs_rehash**(string `$hash`, string|int|null `$algo`, array `$options = []`): bool

This function checks to see if the supplied hash implements the algorithm and options provided. If not, it is assumed that the hash needs to be rehashed.

Parameters

`hash`

A hash created by password_hash().

`algo`

A password algorithm constant denoting the algorithm to use when hashing the password.

`options`

An associative array containing options. See the password algorithm constants for documentation on the supported options for each algorithm.

Return Values

Returns **`true`** if the hash should be rehashed to match the given `algo` and `options`, or **`false`** otherwise.

Changelog

Version	Description
7.4.0	The `algo` parameter expects a string now, but still accepts ints for backward compatibility.

Examples

Example #1 Usage of password_needs_rehash()

```php
<?php

$password = 'rasmuslerdorf';
$hash = '$2y$12$4Umg0rCJwMswRw/l.SwHvuQV01coP0eWmGzd61QH2RvAOMANUBGC.';

$algorithm = PASSWORD_BCRYPT;
// bcrypt's cost parameter can change over time as hardware improves
$options = ['cost' => 13];

// Verify stored hash against plain-text password
if (password_verify($password, $hash)) {
```

```php
    // Check if either the algorithm or the options have changed
    if (password_needs_rehash($hash, $algorithm, $options)) {
        // If so, create a new hash, and replace the old one
        $newHash = password_hash($password, $algorithm, $options);

        // Update the user record with the $newHash
    }

    // Perform the login.
}
?>
```

password_verify

password_verify

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

password_verify — Verifies that a password matches a hash

Description

function **password_verify**(#[\SensitiveParameter] string `$password`, string `$hash`): bool

Verifies that the given hash matches the given password. **password_verify()** is compatible with crypt(). Therefore, password hashes created by crypt() can be used with **password_verify()**.

Note that password_hash() returns the algorithm, cost and salt as part of the returned hash. Therefore, all information that's needed to verify the hash is included in it. This allows the verify function to verify the hash without needing separate storage for the salt or algorithm information.

This function is safe against timing attacks.

Parameters

`password`

The user's password.

`hash`

A hash created by password_hash().

Return Values

Returns `true` if the password and hash match, or `false` otherwise.

Examples

Example #1 password_verify() example

This is a simplified example; it is recommended to rehash a correct password if necessary; see password_needs_rehash() for an example.

```php
<?php
// See the password_hash() example to see where this came from.
$hash = '$2y$12$4Umg0rCJwMswRw/l.SwHvuQV01coP0eWmGzd61QH2RvAOMANUBGC.';

if (password_verify('rasmuslerdorf', $hash)) {
    echo 'Password is valid!';
} else {
    echo 'Invalid password.';
}
?>
```

The above example will output:

```
Password is valid!
```

See Also

- password_needs_rehash() - Checks if the given hash matches the given options
- password_hash() - Creates a password hash
- sodium_crypto_pwhash_str_verify() - Verifies that a password matches a hash

Data Filtering

Introduction

This extension provides filters which can be used to validate or sanitize data. This is especially useful when the data source contains unknown (or foreign) data, like user supplied input. For example, this data may come from an HTML form.

There are two main types of filtering: *validation* and *sanitization*.

A validation filter is used to check if the data meets certain criteria. These filters are identified by the `FILTER_VALIDATE_*` constants. For example, the `FILTER_VALIDATE_EMAIL` filter can be used to determine if the data is a valid email address. However, it will never alter the input data.

Sanitization on the other hand will "clean up" the data, therefore it may alter the input data by adding or removing characters. These filters are identified by the `FILTER_SANITIZE_*` constants. For example, the `FILTER_SANITIZE_EMAIL` filter will remove characters that are inappropriate for an email address to contain. However, the sanitized data is not validated to check if it is a valid email address.

Most filters support optional *flags* that can tweak the behavior of the filter. These flags are identified by the `FILTER_FLAG_*` constants. For example, using the `FILTER_FLAG_PATH_REQUIRED` with the `FILTER_VALIDATE_URL` validation filter requires that the URL has a path (e.g. `/foo` in `https://example.org/foo`).

Installing/Configuring

Table of Contents

Installation

This extension is enabled by default. It may be disabled by using the following option at compile time: **--disable-filter**

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Filter Configuration Options

Name	Default	Changeable	Changelog
filter.default	"unsafe_raw"	`INI_PERDIR`	Deprecated as of PHP 8.1.0.
filter.default_flags	NULL	`INI_PERDIR`	

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`filter.default` string

Filter all *$_GET*, *$_POST*, *$_COOKIE*, *$_REQUEST* and *$_SERVER* data by this filter. Original data can be accessed through filter_input().

Must be the name of a filter which can be determined by using filter_list() and filter_id().

> **Note** :
>
> Be careful about the default flags for the default filters. They should be set explicitly. For example, to configure the default filter to behave exactly like htmlspecialchars() the default flags must be set to `0`, as shown in the example below.
>
> **Example #1 Configuring the default filter to act like htmlspecialchars**
>
> ```
> filter.default = full_special_chars
> filter.default_flags = 0
> ```

> **Warning**
>
> This INI setting is deprecated as of PHP 8.1.0.

`filter.default_flags` int

Default flags to apply when the default filter is set. This is set to `FILTER_FLAG_NO_ENCODE_QUOTES` by default for backwards compatibility reasons. See `FILTER_FLAG_*` constants for available flags.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

Input Constants

These constants are used by filter_input() and filter_input_array().

`INPUT_POST` (int)

POST variables.

`INPUT_GET` (int)

GET variables.

`INPUT_COOKIE` (int)

COOKIE variables.

`INPUT_ENV` (int)

ENV variables.

`INPUT_SERVER` (int)

SERVER variables.

`INPUT_SESSION` (int)

SESSION variables. (Removed as of PHP 8.0.0; was not implemented previously)

`INPUT_REQUEST` (int)

REQUEST variables. (Removed as of PHP 8.0.0; was not implemented previously)

Generic Filter Flags

`FILTER_FLAG_NONE` (int)

No flags.

`FILTER_REQUIRE_SCALAR` (int)

Flag used to require the input of the filter to be a scalar.

`FILTER_REQUIRE_ARRAY` (int)

Flag used to require the input of the filter to be an array.

`FILTER_FORCE_ARRAY` (int)

This flag wraps scalar inputs into a one element array for filters which operate on arrays.

`FILTER_NULL_ON_FAILURE` (int)

Use `null` instead of `false` on failure.

Usable with any validation `FILTER_VALIDATE_*` filter.

`FILTER_THROW_ON_FAILURE` (int)

Throws a Filter\FilterFailedException when a validation filter fails, instead of returning `false`.

Can be used with any validation `FILTER_VALIDATE_*` filter.

Available as of PHP 8.5.0.

Sanitization Filter Flags

`FILTER_FLAG_STRIP_LOW` (int)

Strip characters with ASCII value less than 32.

`FILTER_FLAG_STRIP_HIGH` (int)

Strip characters with ASCII value greater than 127.

`FILTER_FLAG_STRIP_BACKTICK` (int)

Strips backtick (`) characters.

`FILTER_FLAG_ENCODE_LOW` (int)

Encode characters with ASCII value less than 32.

`FILTER_FLAG_ENCODE_HIGH` (int)

Encode characters with ASCII value greater than 127.

`FILTER_FLAG_ENCODE_AMP` (int)

Encode `&`.

`FILTER_FLAG_NO_ENCODE_QUOTES` (int)

Single and double quotes (`'` and `"`) will not be encoded.

`FILTER_FLAG_EMPTY_STRING_NULL` (int)

If sanitizing a string results in an empty string, convert the value to `null`

Validation Filters

`FILTER_VALIDATE_BOOL` (int)

Returns `true` for `"1"`, `1` including binary, octal and hexadecimal notations, `1.0` including scientific notation, `"true"`, `true`, `"on"`, and `"yes"`.

Returns `false` for `"0"`, `0` including binary, octal and hexadecimal notations, `0.0` including scientific notation, `"false"`, `false`, `"off"`, `"no"`, and `""`.

String values are compared case-insensitively. The return value for non-boolean values depends on the `FILTER_NULL_ON_FAILURE`. If it is set, `null` is returned, otherwise `false` is returned.

Available options

`default`

Value to return in case the filter fails.

Available as of PHP 8.0.0.

`FILTER_VALIDATE_BOOLEAN` (int)

Alias of `FILTER_VALIDATE_BOOL`. The alias was available prior to the introduction of its canonical name in PHP 8.0.0.

`FILTER_VALIDATE_INT` (int)

Validates whether the value is an integer, on success it is converted to type int.

> **Note** :
>
> String values are trimmed using trim() before validation.

Available options

`default`

Value to return in case the filter fails.

`min_range`

Value is only valid if it is greater than or equal to the provided value.

`max_range`

Value is only valid if it is less than or equal to the provided value.

Optional Flags

`FILTER_FLAG_ALLOW_OCTAL` (int)

Allow integers in octal notation (`0[0-7]+`).

`FILTER_FLAG_ALLOW_HEX` (int)

Allow integers in hexadecimal notation (`0x[0-9a-fA-F]+`).

`FILTER_VALIDATE_FLOAT` (int)

Validates whether the value is a float, on success it is converted to type float.

> **Note** :
>
> String values are trimmed using trim() before validation.

Available options

`default`

Value to return in case the filter fails.

`decimal`

`min_range`

Value is only valid if it is greater than or equal to the provided value. Available as of PHP 7.4.0.

`max_range`

Value is only valid if it is less than or equal to the provided value. Available as of PHP 7.4.0.

Optional Flags

`FILTER_FLAG_ALLOW_THOUSAND` (int)

Accept commas (`,`), which usually represent the thousand separator.

`FILTER_VALIDATE_REGEXP` (int)

Validates value against the regular expression provided by the `regexp` option.

Available options

`default`

Value to return in case the filter fails.

`regexp`

Perl-compatible regular expression.

`FILTER_VALIDATE_URL` (int)

Validates whether the URL is valid according to » RFC 2396.

Available options

`default`

Value to return in case the filter fails.

Optional Flags

`FILTER_FLAG_SCHEME_REQUIRED` (int)

Requires the URL to contain a scheme part.

> **Warning**
>
> *DEPRECATED* as of PHP 7.3.0 and *REMOVED* as of PHP 8.0.0. This is because it is always implied by the `FILTER_VALIDATE_URL` filter.

`FILTER_FLAG_HOST_REQUIRED` (int)

Requires the URL to contain a host part.

> **Warning**
>
> *DEPRECATED* as of PHP 7.3.0 and *REMOVED* as of PHP 8.0.0. This is because it is always implied by the `FILTER_VALIDATE_URL` filter for most schemes.

`FILTER_FLAG_PATH_REQUIRED` (int)

Requires the URL to contain a path part.

`FILTER_FLAG_QUERY_REQUIRED` (int)

Requires the URL to contain a query part.

> **Warning**
>
> A valid URL may not specify the HTTP protocol (`http://`). Therefore, further validation may be required to determine if the URL uses an expected protocol, e.g. `ssh://` or `mailto:`.

> **Warning**
>
> This filter only works on ASCII URLs. This means that Internationalized Domain Names (IDN) will always be rejected.

> **Warning**
>
> This filter is very permissive. For any scheme other than `http://` or `https://`, host content will be accepted without any validation, though the presence of a host is still required unless the URL has a scheme of `mailto:`, `news:`, or `file:`. Schemes are not themselves validated and may be accepted despite not corresponding to a known URL scheme, or having some other meaning in PHP (like phar://). The filter also accepts » loopback addresses that correspond to the current host.
>
> ```php
> <?php
>
> var_dump(filter_var('javascript://x%0aalert(1)', FILTER_VALIDATE_URL));
> var_dump(filter_var('javascript://?demo', FILTER_VALIDATE_URL));
> var_dump(filter_var('news:?demo', FILTER_VALIDATE_URL));
> var_dump(filter_var('phar://demo', FILTER_VALIDATE_URL));
> var_dump(filter_var('123://demo', FILTER_VALIDATE_URL));
> var_dump(filter_var('-://demo', FILTER_VALIDATE_URL));
> var_dump(filter_var('http://127.0.0.1', FILTER_VALIDATE_URL));
> var_dump(filter_var('gopher://127.0.0.1', FILTER_VALIDATE_URL));
> ?>
> ```
>
> The above example will output:
>
> ```
> string(25) "javascript://x%0aalert(1)"
> bool(false)
> string(10) "news:?demo"
> string(11) "phar://demo"
> string(10) "123://demo"
> string(8) "-://demo"
> string(16) "http://127.0.0.1"
> string(18) "gopher://127.0.0.1"
> ```

`FILTER_VALIDATE_DOMAIN` (int)

Validates whether the domain name is valid according to » RFC 952, » RFC 1034, » RFC 1035, » RFC 1123, » RFC 2732, and » RFC 2181.

Available options

`default`

Value to return in case the filter fails.

Optional Flags

`FILTER_FLAG_HOSTNAME` (int)

Require hostnames to start with an alphanumeric character and contain only alphanumerics or hyphens.

`FILTER_VALIDATE_EMAIL` (int)

Validates whether the value is a "valid" e-mail address.

The validation is performed against the `addr-spec` syntax in » RFC 822. However, comments, whitespace folding, and dotless domain names are not supported, and thus will be rejected.

Available options

`default`

Value to return in case the filter fails.

Optional Flags

`FILTER_FLAG_EMAIL_UNICODE` (int)

Accepts Unicode characters in the local part. Available as of PHP 7.1.0.

> **Warning**
>
> Email validation is complex and the only true way to confirm an email is valid and exists is to send an email to the address.

`FILTER_VALIDATE_IP` (int)

Validates value as IP address.

Available options

`default`

Value to return in case the filter fails.

Optional Flags

`FILTER_FLAG_IPV4` (int)

Allow IPv4 address.

`FILTER_FLAG_IPV6` (int)

Allow IPv6 address.

`FILTER_FLAG_NO_RES_RANGE` (int)

Deny reserved addresses.

These are the ranges that are marked as `Reserved-By-Protocol` in » RFC 6890.

Which for IPv4 corresponds to the following ranges: `0.0.0.0/8`, `169.254.0.0/16`, `127.0.0.0/8`, `240.0.0.0/4`.

And for IPv6 corresponds to the following ranges: `::1/128`, `::/128`, `::FFFF:0:0/96`, `FE80::/10`.

`FILTER_FLAG_NO_PRIV_RANGE` (int)

Deny private addresses.

These are IPv4 addresses which are in the following ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`.

These are IPv6 addresses starting with `FD` or `FC`.

`FILTER_FLAG_GLOBAL_RANGE` (int)

Only allow global addresses. These can be found in » RFC 6890 where the `Global` attribute is `True`. Available as of PHP 8.2.0.

`FILTER_VALIDATE_MAC` (int)

Validates whether the value is a MAC address.

Available options

`default`

Value to return in case the filter fails.

Sanitizing Filters

`FILTER_UNSAFE_RAW` (int)

This filter does nothing.

However, it can strip or encode special characters if used together with the `FILTER_FLAG_STRIP_*` and `FILTER_FLAG_ENCODE_*` filter sanitization flags.

`FILTER_DEFAULT` (int)

Alias of `FILTER_UNSAFE_RAW`.

`FILTER_SANITIZE_STRING` (int)

This filter strips tags and HTML-encodes double and single quotes.

Optionally it can strip or encode specified characters if used together with the `FILTER_FLAG_STRIP_*` and `FILTER_FLAG_ENCODE_*` filter sanitization flags.

The behaviour of encoding quotes can be disabled by using the `FILTER_FLAG_NO_ENCODE_QUOTES` filter flag.

> **Warning**
>
> *Deprecated* as of PHP 8.1.0, use htmlspecialchars() instead.

> **Warning**
>
> The way this filter strips tags is not equivalent to strip_tags().

`FILTER_SANITIZE_STRIPPED` (int)

Alias of `FILTER_SANITIZE_STRING`.

> **Warning**
>
> *Deprecated* as of PHP 8.1.0, use htmlspecialchars() instead.

`FILTER_SANITIZE_ENCODED` (int)

This filter URL-encodes a string.

Optionally it can strip or encode specified characters if used together with the `FILTER_FLAG_STRIP_*` and `FILTER_FLAG_ENCODE_*` filter sanitization flags.

`FILTER_SANITIZE_SPECIAL_CHARS` (int)

This filter HTML-encodes `'`, `"`, `<`, `>`, `&` and characters with an ASCII value less than 32. Unlike the `FILTER_SANITIZE_FULL_SPECIAL_CHARS` filter, the `FILTER_SANITIZE_SPECIAL_CHARS` filter ignores the `FILTER_FLAG_NO_ENCODE_QUOTES` flag.

Optionally it can strip specified characters if used together with the `FILTER_FLAG_STRIP_*` filter sanitization flags, and it can encode characters with ASCII value greater than 127 using `FILTER_FLAG_ENCODE_HIGH`.

`FILTER_SANITIZE_FULL_SPECIAL_CHARS` (int)

This filter is equivalent to calling htmlspecialchars() with `ENT_QUOTES` set.

The behaviour of encoding quotes can be disabled by using the `FILTER_FLAG_NO_ENCODE_QUOTES` filter flag.

> **Warning**
>
> Like htmlspecialchars(), this filter is aware of the default_charset INI setting. If a sequence of bytes is detected that makes up an invalid character in the current character set then the entire string is rejected resulting in an empty string being returned.

`FILTER_SANITIZE_EMAIL` (int)

Sanitize the string by removing all characters except latin letters (`[a-zA-Z]`), digits (`[0-9]`), and the special characters `!#$%&'*+-=?^_`{|}~@.[]`.

`FILTER_SANITIZE_URL` (int)

Sanitize the string by removing all characters except latin letters (`[a-zA-Z]`), digits (`[0-9]`), and the special characters `$-_.+!*'(),{}|\\^~[]`<>#%";/?:@&=`.

`FILTER_SANITIZE_NUMBER_INT` (int)

Sanitize the string by removing all characters except digits (`[0-9]`), plus sign (`+`), and minus sign (`-`).

`FILTER_SANITIZE_NUMBER_FLOAT` (int)

Sanitize the string by removing all characters except digits (`[0-9]`), plus sign (`+`), and minus sign (`-`).

Optional Flags

`FILTER_FLAG_ALLOW_FRACTION` (int)

Accept dot (`.`) character, which usually represents the separator between the integer and fractional parts.

`FILTER_FLAG_ALLOW_THOUSAND` (int)

Accept commas (`,`) character, which usually represents the thousand separator.

`FILTER_FLAG_ALLOW_SCIENTIFIC` (int)

Accept numbers in scientific notation by allowing the `e` and `E` characters.

> **Warning**
>
> If the `FILTER_FLAG_ALLOW_FRACTION` flag is not used, then the decimal separator is removed, altering the value received.
>
> ```php
> <?php
> $number = '12.34';
>
> var_dump(filter_var($number, FILTER_SANITIZE_NUMBER_FLOAT));
> var_dump(filter_var($number, FILTER_SANITIZE_NUMBER_FLOAT, FILTER_FLAG_ALLOW_FRACTION));
> ?>
> ```
>
> The above example will output:
>
> ```
> string(4) "1234"
> string(5) "12.34"
> ```

`FILTER_SANITIZE_ADD_SLASHES` (int)

Apply addslashes() to the input. Available as of PHP 7.3.0.

`FILTER_SANITIZE_MAGIC_QUOTES` (int)

Alias of `FILTER_SANITIZE_ADD_SLASHES`.

> **Warning**

> *DEPRECATED* as of PHP 7.3.0 and *REMOVED* as of PHP 8.0.0.

User Defined Filter
`FILTER_CALLBACK` (int)

This filter delegates the filtering to a user defined function. The callable is passed via the `options` parameter as the value associated to the `'options'` key.

The callback should have the following signature:

function callback(string `$value`): mixed
`value`

The value that is being filtered.

> **Note** :
>
> The value returned by the callback will be the value returned by the invoked filter function.

Example #1 Example of using `FILTER_CALLBACK` to validate a login name

```php
<?php
function validate_login(string $value): ?string
{
    if (strlen($value) >= 5 && ctype_alnum($value)) {
        return $value;
    }
    return null;
}

$login = "val1dL0gin";
$filtered_login = filter_var($login, FILTER_CALLBACK, ['options' => 'validate_login']);
var_dump($filtered_login);

$login = "f&ke login";
$filtered_login = filter_var($login, FILTER_CALLBACK, ['options' => 'validate_login']);
var_dump($filtered_login);
?>
```

The above example will output:

```
string(10) "val1dL0gin"
NULL
```

> **Note** :
>
> As of PHP 8.1.25 and 8.2.12, the structural flags `FILTER_REQUIRE_ARRAY`, `FILTER_FORCE_ARRAY` and `FILTER_REQUIRE_SCALAR` apply to this filter, and `FILTER_NULL_ON_FAILURE` changes how such a structural failure is reported. In earlier versions all flags were ignored. Filter specific flags are always ignored, and since the return value of the callback is the result, the callback itself cannot fail: no flag turns a `false` returned by the callback into `null` or an exception.

Examples

Table of Contents

Validation

Example #1 Validating email addresses with filter_var()

```php
<?php
$email_a = 'joe@example.com';
$email_b = 'bogus';

if (filter_var($email_a, FILTER_VALIDATE_EMAIL)) {
    echo "Email address '$email_a' is considered valid.\n";
}
if (filter_var($email_b, FILTER_VALIDATE_EMAIL)) {
    echo "Email address '$email_b' is considered valid.\n";
} else {
    echo "Email address '$email_b' is considered invalid.\n";
}
?>
```

The above example will output:

```
Email address 'joe@example.com' is considered valid.
Email address 'bogus' is considered invalid.
```

Example #2 Validating IP addresses with filter_var()

```php
<?php
$ip_a = '127.0.0.1';
$ip_b = '42.42';

if (filter_var($ip_a, FILTER_VALIDATE_IP)) {
    echo "IP address '$ip_a' is considered valid.";
}
if (filter_var($ip_b, FILTER_VALIDATE_IP)) {
    echo "IP address '$ip_b' is considered valid.";
}
?>
```

The above example will output:

```
IP address '127.0.0.1' is considered valid.
```

Example #3 Passing options to filter_var()

```php
<?php
$int_a = '1';
$int_b = '-1';
$int_c = '4';
$options = array(
    'options' => array(
        'min_range' => 0,
        'max_range' => 3,
    )
);
if (filter_var($int_a, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "Integer A '$int_a' is considered valid (between 0 and 3).\n";
}
if (filter_var($int_b, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "Integer B '$int_b' is considered valid (between 0 and 3).\n";
}
if (filter_var($int_c, FILTER_VALIDATE_INT, $options) !== FALSE) {
    echo "Integer C '$int_c' is considered valid (between 0 and 3).\n";
}

$options['options']['default'] = 1;
```

```php
if (($int_c = filter_var($int_c, FILTER_VALIDATE_INT, $options)) !== FALSE) {
    echo "Integer C '$int_c' is considered valid (between 0 and 3).";
}
?>
```

The above example will output:

```
Integer A '1' is considered valid (between 0 and 3).
Integer C '1' is considered valid (between 0 and 3).
```

Sanitization

Example #1 Sanitizing and validating email addresses

```php
<?php
$a = 'joe@example.org';
$b = 'bogus - at - example dot org';
$c = '(bogus@example.org)';

$sanitized_a = filter_var($a, FILTER_SANITIZE_EMAIL);
if (filter_var($sanitized_a, FILTER_VALIDATE_EMAIL)) {
    echo "This (a) sanitized email address is considered valid.\n";
}

$sanitized_b = filter_var($b, FILTER_SANITIZE_EMAIL);
if (filter_var($sanitized_b, FILTER_VALIDATE_EMAIL)) {
    echo "This sanitized email address is considered valid.";
} else {
    echo "This (b) sanitized email address is considered invalid.\n";
}

$sanitized_c = filter_var($c, FILTER_SANITIZE_EMAIL);
if (filter_var($sanitized_c, FILTER_VALIDATE_EMAIL)) {
    echo "This (c) sanitized email address is considered valid.\n";
    echo "Before: $c\n";
    echo "After:  $sanitized_c\n";
}
?>
```

The above example will output:

```
This (a) sanitized email address is considered valid.
This (b) sanitized email address is considered invalid.
This (c) sanitized email address is considered valid.
Before: (bogus@example.org)
After: bogus@example.org
```

Filter Functions

Table of Contents

filter_has_var

filter_has_var

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_has_var — Checks if a variable of the specified type exists

Description

function **filter_has_var**(int `$input_type`, string `$var_name`): bool

Parameters

`input_type`

One of **`INPUT_GET`**, **`INPUT_POST`**, **`INPUT_COOKIE`**, **`INPUT_SERVER`**, or **`INPUT_ENV`**.

`var_name`

Name of a variable to check.

Return Values

Returns **`true`** on success or **`false`** on failure.

filter_id

filter_id

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_id — Returns the filter ID belonging to a named filter

Description

function **filter_id**(string `$name`): int|false

Parameters

`name`

Name of a filter to get.

Return Values

ID of a filter on success or **`false`** if filter doesn't exist.

See Also

- filter_list() - Returns a list of all supported filters

filter_input

filter_input

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_input — Gets a specific external variable by name and optionally filters it

Description

function **filter_input**(
  int `$type`,
  string `$var_name`,
  int `$filter` = **`FILTER_DEFAULT`**,
  array|int `$options` = 0
): mixed

Parameters

`type`

One of the **`INPUT_*`** constants.

> **Warning**

> The content of the superglobal that is being filtered is the original "raw" content provided by the SAPI, prior to any user modification to the superglobal. To filter a modified superglobal use filter_var() instead.

`var_name`

Name of a variable to filter inside the corresponding `type` superglobal.

`filter`

The filter to apply. Can be a validation filter by using one of the `FILTER_VALIDATE_*` constants, a sanitization filter by using one of the `FILTER_SANITIZE_*` or `FILTER_UNSAFE_RAW`, or a custom filter by using `FILTER_CALLBACK`.

> **Note** :
>
> The default is `FILTER_DEFAULT`, which is an alias of `FILTER_UNSAFE_RAW`. This will result in no filtering taking place by default.

`options`

Either an associative array of options, or a bitmask of filter flag constants `FILTER_FLAG_*`.

If the `filter` accepts options, flags can be provided by using the `"flags"` field of array.

Return Values

Value of the requested variable on success, `false` if the filter fails, or `null` if the `var_name` variable is not set. If the flag `FILTER_NULL_ON_FAILURE` is used, it returns `false` if the variable is not set and `null` if the filter fails.

Examples

Example #1 A filter_input() example

```php
<?php
$search_html = filter_input(INPUT_GET, 'search', FILTER_SANITIZE_SPECIAL_CHARS);
$search_url = filter_input(INPUT_GET, 'search', FILTER_SANITIZE_ENCODED);
echo "You have searched for $search_html.\n";
echo "<a href='?search=$search_url'>Search again.</a>";
?>
```

The above example will output something similar to:

```
You have searched for Me &#38; son.
<a href='?search=Me%20%26%20son'>Search again.</a>
```

See Also

- filter_input_array() - Gets external variables and optionally filters them
- filter_var() - Filters a variable with a specified filter
- filter_var_array() - Gets multiple variables and optionally filters them
- Validation filters `FILTER_VALIDATE_*`
- Sanitization filters `FILTER_SANITIZE_*`

filter_input_array

filter_input_array

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_input_array — Gets external variables and optionally filters them

Description

function **filter_input_array**(int `$type`, array|int `$options` = `FILTER_DEFAULT`, bool `$add_empty` = `true`): array|false|null

This function is useful for retrieving many values without repetitively calling filter_input().

Parameters

`type`

One of the `INPUT_*` constants.

> **Warning**
>
> The content of the superglobal that is being filtered is the original "raw" content provided by the SAPI, prior to any user modification to the superglobal. To filter a modified superglobal use filter_var_array() instead.

`options`

Either an associative array of options, or the filter to apply to each entry, which can either be a validation filter by using one of the `FILTER_VALIDATE_*` constants, or a sanitization filter by using one of the `FILTER_SANITIZE_*` constants.

The option array is an associative array where the key corresponds to a key in the input array and the associated value is either the filter to apply to this entry, or an associative array describing how and which filter should be applied to this entry.

The associative array describing how a filter should be applied must contain the `'filter'` key whose associated value is the filter to apply, which can be one of the `FILTER_VALIDATE_*`, `FILTER_SANITIZE_*`, `FILTER_UNSAFE_RAW`, or `FILTER_CALLBACK` constants. It can optionally contain the `'flags'` key which specifies any flags that apply to the filter, and the `'options'` key which specifies any options that apply to the filter.

`add_empty`

Add missing keys as `null` to the return value.

Return Values

On success, an array containing the values of the requested variables.

On failure, `false` is returned. If the input array designated by `type` is not populated, `null` is returned instead.

Missing entries from the input array are added to the returned array as `null` if `add_empty` is `true`, and are omitted entirely if it is `false`. Unlike filter_input(), the `FILTER_NULL_ON_FAILURE` flag does not change this: a missing entry is always `null`.

An entry of the returned array will be `false` if the filter fails, unless the `FILTER_NULL_ON_FAILURE` flag is used, in which case it will be `null`. With the `FILTER_FORCE_ARRAY` flag, that failure value is wrapped in a one element array like any other result.

Examples

Example #1 A filter_input_array() example

This example assumes a GET request to `?email=user@example.com&age=twenty&url=https://example.com`. The `age` entry fails because `twenty` is not an integer; a value outside the `1` to `120` range would fail the same way.

```php
<?php
$filters = [
    'email' => FILTER_VALIDATE_EMAIL,
    'age'   => [
        'filter'  => FILTER_VALIDATE_INT,
        'options' => ['min_range' => 1, 'max_range' => 120],
    ],
    'url'   => FILTER_VALIDATE_URL,
];

$result = filter_input_array(INPUT_GET, $filters);

var_dump($result);
?>
```

The above example will output something similar to:

```
```

```
array(3) {
  ["email"]=>
  string(16) "user@example.com"
  ["age"]=>
  bool(false)
  ["url"]=>
  string(19) "https://example.com"
}
```

Example #2 Filtering POST data with filter_input_array()

This example assumes a POST request with fields `username=<script>alert</script>` and `comment=Hello World`. No `missing` field is submitted: because `add_empty` defaults to `true`, it is still present in the result, set to `null`.

```
<?php
$filters = [
    'username' => FILTER_SANITIZE_SPECIAL_CHARS,
    'comment'  => FILTER_SANITIZE_SPECIAL_CHARS,
    'missing'  => FILTER_VALIDATE_INT,
];

$result = filter_input_array(INPUT_POST, $filters);

var_dump($result);
?>
```

The above example will output something similar to:

```
array(3) {
  ["username"]=>
  string(38) "&#60;script&#62;alert&#60;/script&#62;"
  ["comment"]=>
  string(11) "Hello World"
  ["missing"]=>
  NULL
}
```

Example #3 Requesting an input type that is not populated

This example assumes a GET request. Because the request carried no POST fields, the input array designated by `INPUT_POST` is not populated and `null` is returned instead of an array. This applies to every input type: a request with no query string yields `null` for `INPUT_GET` as well.

```
<?php
var_dump(filter_input_array(INPUT_POST, ['a' => FILTER_VALIDATE_INT]));
?>
```

The above example will output something similar to:

```
NULL
```

Notes

> **Note** :
>
> There is no `REQUEST_TIME` key in `INPUT_SERVER` array because it is inserted into the *$_SERVER* later.

See Also

- filter_input() - Gets a specific external variable by name and optionally filters it
- filter_var() - Filters a variable with a specified filter
- filter_var_array() - Gets multiple variables and optionally filters them
- Validation filters `FILTER_VALIDATE_*`
- Sanitization filters `FILTER_SANITIZE_*`

filter_list

filter_list

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_list — Returns a list of all supported filters

Description

function **filter_list**(): array

Parameters

This function has no parameters.

Return Values

Returns an array of names of all supported filters, empty array if there are no such filters. Indexes of this array are not filter IDs, they can be obtained with filter_id() from a name instead.

Examples

Example #1 A filter_list() example

```php
<?php
print_r(filter_list());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => int
    [1] => boolean
    [2] => float
    [3] => validate_regexp
    [4] => validate_url
    [5] => validate_email
    [6] => validate_ip
    [7] => string
    [8] => stripped
    [9] => encoded
    [10] => special_chars
    [11] => unsafe_raw
    [12] => email
    [13] => url
    [14] => number_int
    [15] => number_float
    [16] => magic_quotes
    [17] => callback
)
```

filter_var

filter_var

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_var — Filters a variable with a specified filter

Description

function **filter_var**(mixed `$value`, int `$filter` = `FILTER_DEFAULT`, array|int `$options` = 0): mixed

Filter a variable using a `FILTER_VALIDATE_*` validation filters, a `FILTER_SANITIZE_*` sanitization filters, or a custom filter.

Parameters

`value`

Value to filter.

> **Warning**
>
> Scalar values are converted to string internally before they are filtered.

`filter`

The filter to apply. Can be a validation filter by using one of the `FILTER_VALIDATE_*` constants, a sanitization filter by using one of the `FILTER_SANITIZE_*` or `FILTER_UNSAFE_RAW`, or a custom filter by using `FILTER_CALLBACK`.

> **Note** :
>
> The default is `FILTER_DEFAULT`, which is an alias of `FILTER_UNSAFE_RAW`. This will result in no filtering taking place by default.

`options`

Either an associative array of options, or a bitmask of filter flag constants `FILTER_FLAG_*`.

If the `filter` accepts options, flags can be provided by using the `"flags"` field of array.

Return Values

On success returns the filtered data. On failure `false` is returned, unless the `FILTER_NULL_ON_FAILURE` flag is used, in which case `null` is returned.

Examples

Example #1 A filter_var() example

```
<?php
var_dump(filter_var('bob@example.com', FILTER_VALIDATE_EMAIL));
var_dump(filter_var('https://example.com', FILTER_VALIDATE_URL, FILTER_FLAG_PATH_REQUIRED));
?>
```

The above example will output:

```
string(15) "bob@example.com"
bool(false)
```

Example #2 Example validating entries of an array

```php
<?php
$emails = [
    "bob@example.com",
    "test@example.local",
    "invalidemail"
];

var_dump(filter_var($emails, FILTER_VALIDATE_EMAIL, FILTER_REQUIRE_ARRAY));
?>
```

The above example will output:

```
array(3) {
  [0]=>
  string(15) "bob@example.com"
  [1]=>
  string(18) "test@example.local"
  [2]=>
  bool(false)
}
```

Example #3 Example of passing an array for `options`

```php
<?php

$options = [
    'options' => [
        'min_range' => 10,
    ],
    'flags' => FILTER_FLAG_ALLOW_OCTAL,
];

var_dump(filter_var('0755', FILTER_VALIDATE_INT, $options));
var_dump(filter_var('011', FILTER_VALIDATE_INT, $options));

?>
```

The above example will output:

```
int(493)
bool(false)
```

Example #4 Providing flags either directly or via an array

```php
<?php

$str = 'string';

var_dump(filter_var($str, FILTER_VALIDATE_BOOLEAN, FILTER_NULL_ON_FAILURE));
var_dump(filter_var($str, FILTER_VALIDATE_BOOLEAN, ['flags' => FILTER_NULL_ON_FAILURE]));

?>
```

The above example will output:

```
NULL
NULL
```

See Also

- filter_var_array() - Gets multiple variables and optionally filters them
- filter_input() - Gets a specific external variable by name and optionally filters it
- filter_input_array() - Gets external variables and optionally filters them
- Validation filters `FILTER_VALIDATE_*`
- Sanitization filters `FILTER_SANITIZE_*`

filter_var_array

filter_var_array

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

filter_var_array — Gets multiple variables and optionally filters them

Description

function **filter_var_array**(array `$array`, array|int `$options` = `FILTER_DEFAULT`, bool `$add_empty` = `true`): array|false|null

Filter an associative array of values using `FILTER_VALIDATE_*` validation filters, `FILTER_SANITIZE_*` sanitization filters, or custom filters.

Parameters

`array`

An associative array containing the data to filter.

`options`

Either an associative array of options, or the filter to apply to each entry, which can either be a validation filter by using one of the `FILTER_VALIDATE_*` constants, or a sanitization filter by using one of the `FILTER_SANITIZE_*` constants.

The option array is an associative array where the key corresponds to a key in the input array and the associated value is either the filter to apply to this entry, or an associative array describing how and which filter should be applied to this entry.

The associative array describing how a filter should be applied must contain the `'filter'` key whose associated value is the filter to apply, which can be one of the `FILTER_VALIDATE_*`, `FILTER_SANITIZE_*`, `FILTER_UNSAFE_RAW`, or `FILTER_CALLBACK` constants. It can optionally contain the `'flags'` key which specifies any flags that apply to the filter, and the `'options'` key which specifies any options that apply to the filter.

`add_empty`

Add missing keys as `null` to the return value.

Return Values

On success, an array containing the values of the requested variables.

On failure, `false` is returned.

Missing entries from the input array are added to the returned array as `null` if `add_empty` is `true`, and are omitted entirely if it is `false`.

An entry of the returned array will be `false` if the filter fails, unless the `FILTER_NULL_ON_FAILURE` flag is used, in which case it will be `null`. With the `FILTER_FORCE_ARRAY` flag, that failure value is wrapped in a one element array like any other result.

Examples

Example #1 A filter_var_array() example

Entries are filtered as scalars unless `FILTER_REQUIRE_ARRAY` or `FILTER_FORCE_ARRAY` is used. The `FILTER_REQUIRE_SCALAR` flag on `testscalar` below therefore only states that default explicitly.

```php
<?php

$data = [
    'product_id' => 'libgd<script>',
    'component'  => '10',
    'versions'   => '2.0.33',
    'testscalar' => ['2', '23', '10', '12'],
    'testarray'  => '2',
];

$filters = [
    'product_id'   => FILTER_SANITIZE_ENCODED,
    'component'    => [
        'filter'   => FILTER_VALIDATE_INT,
        'flags'    => FILTER_FORCE_ARRAY,
        'options'  => [
            'min_range' => 1,
            'max_range' => 10,
        ],
    ],
    'versions'     => [
        'filter' => FILTER_SANITIZE_ENCODED
    ],
    'testscalar'   => [
        'filter' => FILTER_VALIDATE_INT,
        'flags'  => FILTER_REQUIRE_SCALAR,
    ],
    'testarray'    => [
        'filter' => FILTER_VALIDATE_INT,
        'flags'  => FILTER_FORCE_ARRAY,
    ],
    'doesnotexist' => FILTER_VALIDATE_INT,
];

var_dump(filter_var_array($data, $filters));

?>
```

The above example will output:

```
array(6) {
  ["product_id"]=>
  string(17) "libgd%3Cscript%3E"
  ["component"]=>
  array(1) {
    [0]=>
    int(10)
  }
  ["versions"]=>
  string(6) "2.0.33"
  ["testscalar"]=>
  bool(false)
  ["testarray"]=>
  array(1) {
    [0]=>
    int(2)
  }
  ["doesnotexist"]=>
  NULL
}
```

Example #2 Applying a single filter to all values

When `options` is an int, the same filter is applied to every entry in the array.

```php
<?php
$data = [
    'name'  => '<b>John</b>',
    'email' => 'john@example<script>.com',
    'bio'   => 'Developer & writer',
];

var_dump(filter_var_array($data, FILTER_SANITIZE_SPECIAL_CHARS));
?>
```

The above example will output:

```
array(3) {
  ["name"]=>
  string(27) "&#60;b&#62;John&#60;/b&#62;"
  ["email"]=>
  string(32) "john@example&#60;script&#62;.com"
  ["bio"]=>
  string(22) "Developer &#38; writer"
}
```

Example #3 Using `FILTER_CALLBACK`

```php
<?php
$data = [
    'name'  => '  John Doe  ',
    'city'  => '  New York  ',
];

$options = [
    'name' => [
        'filter'  => FILTER_CALLBACK,
        'options' => 'trim',
    ],
    'city' => [
        'filter'  => FILTER_CALLBACK,
        'options' => function ($value) {
            return strtoupper(trim($value));
        },
    ],
];

var_dump(filter_var_array($data, $options));
?>
```

The above example will output:

```
array(2) {
  ["name"]=>
  string(8) "John Doe"
  ["city"]=>
  string(8) "NEW YORK"
}
```

See Also

- filter_input_array() - Gets external variables and optionally filters them
- filter_var() - Filters a variable with a specified filter

- filter_input() - Gets a specific external variable by name and optionally filters it
- Validation filters `FILTER_VALIDATE_*`
- Sanitization filters `FILTER_SANITIZE_*`

Filter\FilterException

(No version information available, might only be in Git)

Introduction

The base class for Exceptions thrown by the Filter extension.

Class synopsis

```
namespace Filter;
class FilterException extends Exception {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

Filter\FilterFailedException

(No version information available, might only be in Git)

Introduction

Thrown when a validation filter fails and the `FILTER_THROW_ON_FAILURE` flag is set.

Class synopsis

```
namespace Filter;
class FilterFailedException extends Filter\FilterException {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
```

```
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

Function Handling

Introduction

These functions all handle various operations involved in working with functions.

Function handling Functions

Table of Contents

call_user_func

call_user_func

(PHP 4, PHP 5, PHP 7, PHP 8)

call_user_func — Call the callback given by the first parameter

Description

function **call_user_func**(callable `$callback`, mixed `...$args`): mixed

Calls the `callback` given by the first parameter and passes the remaining parameters as arguments.

Parameters

`callback`

The callable to be called.

`args`

Zero or more parameters to be passed to the callback.

> **Note** :
>
> Note that the parameters for **call_user_func()** are not passed by reference.
>
> **Example #1 call_user_func() example and references**

```php
<?php
error_reporting(E_ALL);
function increment(&$var)
{
    $var++;
}

$a = 0;
call_user_func('increment', $a);
echo $a."\n";

// it is possible to use this instead
call_user_func_array('increment', array(&$a));
echo $a."\n";

// it is also possible to use a variable function
$increment = 'increment';
$increment($a);
echo $a."\n";
?>
```

The above example will output:

```
Warning: Parameter 1 to increment() expected to be a reference, value given in …
0
1
2
```

Return Values

Returns the return value of the callback.

Examples

Example #2 call_user_func() example

```php
<?php
function barber($type)
{
    echo "You wanted a $type haircut, no problem\n";
}
call_user_func('barber', "mushroom");
call_user_func('barber', "shave");
?>
```

The above example will output:

```
You wanted a mushroom haircut, no problem
You wanted a shave haircut, no problem
```

Example #3 call_user_func() using namespace name

```php
<?php

namespace Foobar;

class Foo {
    static public function test() {
        print "Hello world!\n";
    }
}

call_user_func(__NAMESPACE__ .'\Foo::test');
call_user_func(array(__NAMESPACE__ .'\Foo', 'test'));

?>
```

The above example will output:

```
Hello world!
Hello world!
```

Example #4 Using a class method with call_user_func()

```php
<?php

class myclass {
```

```php
    static function say_hello()
    {
        echo "Hello!\n";
    }
}

$classname = "myclass";

call_user_func(array($classname, 'say_hello'));
call_user_func($classname .'::say_hello');

$myobject = new myclass();

call_user_func(array($myobject, 'say_hello'));

?>
```

The above example will output:

```
Hello!
Hello!
Hello!
```

Example #5 Using lambda function with call_user_func()

```php
<?php
call_user_func(function($arg) { print "[$arg]\n"; }, 'test');
?>
```

The above example will output:

```
[test]
```

Notes

> **Note** :
>
> Callbacks registered with functions such as **call_user_func()** and call_user_func_array() will not be called if there is an uncaught exception thrown in a previous callback.

See Also

- call_user_func_array() - Call a callback with an array of parameters
- is_callable() - Verify that a value can be called as a function from the current scope
- Variable functions
- ReflectionFunction::invoke() - Invokes function
- ReflectionMethod::invoke() - Invoke

call_user_func_array

call_user_func_array

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

call_user_func_array — Call a callback with an array of parameters

Description

function **call_user_func_array**(callable `$callback`, array `$args`): mixed

Calls the `callback` given by the first parameter with the parameters in `args`.

Parameters

`callback`

The callable to be called.

`args`

The parameters to be passed to the callback, as an array.

If the keys of `args` are all numeric, the keys are ignored and each element will be passed to `callback` as a positional argument, in order.

If any keys of `args` are strings, those elements will be passed to `callback` as named arguments, with the name given by the key.

It is a fatal error to have a numeric key in `args` appear after a string key, or to have a string key that does not match the name of any parameter of `callback`.

Return Values

Returns the return value of the callback, or **`false`** on error.

Changelog

Version	Description
8.0.0	`args` keys will now be interpreted as parameter names, instead of being silently ignored.

Examples

Example #1 call_user_func_array() example

```php
<?php
function foobar($arg, $arg2) {
    echo __FUNCTION__, " got $arg and $arg2\n";
}
class foo {
    function bar($arg, $arg2) {
        echo __METHOD__, " got $arg and $arg2\n";
    }
}


// Call the foobar() function with 2 arguments
call_user_func_array("foobar", array("one", "two"));

// Call the $foo->bar() method with 2 arguments
$foo = new foo;
call_user_func_array(array($foo, "bar"), array("three", "four"));
?>
```

The above example will output something similar to:

```
foobar got one and two
foo::bar got three and four
```

Example #2 call_user_func_array() using namespace name

```php
<?php

namespace Foobar;

class Foo {
    static public function test($name) {
        print "Hello {$name}!\n";
    }
}

call_user_func_array(__NAMESPACE__ .'\Foo::test', array('Hannes'));

call_user_func_array(array(__NAMESPACE__ .'\Foo', 'test'), array('Philip'));

?>
```

The above example will output something similar to:

```
Hello Hannes!
Hello Philip!
```

Example #3 Using lambda function

```php
<?php

$func = function($arg1, $arg2) {
    return $arg1 * $arg2;
};

var_dump(call_user_func_array($func, array(2, 4)));

?>
```

The above example will output:

```
int(8)
```

Example #4 Passing values by reference

```php
<?php

function mega(&$a){
    $a = 55;
    echo "function mega \$a=$a\n";
}
$bar = 77;
call_user_func_array('mega',array(&$bar));
echo "global \$bar=$bar\n";

?>
```

The above example will output:

```
function mega $a=55
global $bar=55
```

Example #5 call_user_func_array() using named arguments

```php
<?php
function foobar($first, $second) {
    echo __FUNCTION__, " got $first and $second\n";
}

// Call the foobar() function with named arguments in non-positional order
call_user_func_array("foobar", array("second" => "two", "first" => "one"));

// Call the foobar() function with one named argument
call_user_func_array("foobar", array("foo", "second" => "bar"));

// Fatal error: Cannot use positional argument after named argument
call_user_func_array("foobar", array("first" => "one", "bar"));

?>
```

The above example will output something similar to:

```
foobar got one and two
foobar got foo and bar

Fatal error: Uncaught Error: Cannot use positional argument after named argument
```

Notes

> **Note** :
>
> Callbacks registered with functions such as call_user_func() and **call_user_func_array()** will not be called if there is an uncaught exception thrown in a previous callback.

See Also

- call_user_func() - Call the callback given by the first parameter
- ReflectionFunction::invokeArgs() - Invokes function args
- ReflectionMethod::invokeArgs() - Invoke args

create_function

create_function

(PHP 4 >= 4.0.1, PHP 5, PHP 7)

create_function — Create a function dynamically by evaluating a string of code

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **create_function**(string `$args`, string `$code`): string

Creates a function dynamically from the parameters passed, and returns a unique name for it.

> **Caution**

> This function internally performs an eval() and as such has the same security issues as eval(). It also has bad performance and memory usage characteristics, because the created functions are global and can not be freed.
>
> A native anonymous function should be used instead.

Parameters

It is normally advisable to pass these parameters as single quoted strings. If using double quoted strings, variable names in the code need to be escaped carefully, e.g. `\$somevar`.

`args`

The function arguments, as a single comma-separated string.

`code`

The function code.

Return Values

Returns a unique function name as a string, or `false` on failure. Note that the name contains a non-printable character (`"\0"`), so care should be taken when printing the name or incorporating it in any other string.

Examples

Example #1 Creating a function dynamically, with create_function() or anonymous functions

You can use a dynamically created function, to (for example) create a function from information gathered at run time. First, using **create_function()**:

```
<?php
$newfunc = create_function('$a,$b', 'return "ln($a) + ln($b) = " . log($a * $b);');
echo $newfunc(2, M_E) . "\n";
?>
```

Now the same code, using an anonymous function; note that the code and arguments are no longer contained in strings:

```
<?php
$newfunc = function($a,$b) { return "ln($a) + ln($b) = " . log($a * $b); };
echo $newfunc(2, M_E) . "\n";
?>
```

The above example will output:

```
ln(2) + ln(2.718281828459) = 1.6931471805599
```

Example #2 Making a general processing function, with create_function() or anonymous functions

Another use could be to have general handler function that can apply a set of operations to a list of parameters:

```
<?php
function process($var1, $var2, $farr)
{
    foreach ($farr as $f) {
        echo $f($var1, $var2) . "\n";
    }
}

// create a bunch of math functions
$farr = array(
    create_function('$x,$y', 'return "some trig: ".(sin($x) + $x*cos($y));'),
    create_function('$x,$y', 'return "a hypotenuse: ".sqrt($x*$x + $y*$y);'),
    create_function('$a,$b', 'if ($a >=0) {return "b*a^2 = ".$b*sqrt($a);} else {return false;}'),
```

```
    create_function('$a,$b', "return \"min(b^2+a, a^2,b) = \".min(\$a*\$a+\$b,\$b*\$b+\$a);"),
    create_function('$a,$b', 'if ($a > 0 && $b != 0) {return "ln(a)/b = ".log($a)/$b; } else { return false; }')
);

echo "\nUsing the first array of dynamic functions\n";
echo "parameters: 2.3445, M_PI\n";
process(2.3445, M_PI, $farr);

// now make a bunch of string processing functions
$garr = array(
    create_function('$b,$a', 'if (strncmp($a, $b, 3) == 0) return "** \"$a\" '.
        'and \"$b\"\n** Look the same to me! (looking at the first 3 chars)";'),
    create_function('$a,$b', 'return "CRCs: " . crc32($a) . ", ".crc32($b);'),
    create_function('$a,$b', 'return "similar(a,b) = " . similar_text($a, $b, $p) . "($p%)";')
);
echo "\nUsing the second array of dynamic functions\n";
process("Twas brilling and the slithy toves", "Twas the night", $garr);
?>
```

Again, here is the same code using anonymous functions. Note that variable names in the code no longer need to be escaped, because they are not enclosed in a string.

```
<?php
function process($var1, $var2, $farr)
{
    foreach ($farr as $f) {
        echo $f($var1, $var2) . "\n";
    }
}

// create a bunch of math functions
$farr = array(
    function($x,$y) { return "some trig: ".(sin($x) + $x*cos($y)); },
    function($x,$y) { return "a hypotenuse: ".sqrt($x*$x + $y*$y); },
    function($a,$b) { if ($a >=0) {return "b*a^2 = ".$b*sqrt($a);} else {return false;} },
    function($a,$b) { return "min(b^2+a, a^2,b) = " . min($a*$a+$b, $b*$b+$a); },
    function($a,$b) { if ($a > 0 && $b != 0) {return "ln(a)/b = ".log($a)/$b; } else { return false; } }
);

echo "\nUsing the first array of dynamic functions\n";
echo "parameters: 2.3445, M_PI\n";
process(2.3445, M_PI, $farr);

// now make a bunch of string processing functions
$garr = array(
    function($b,$a) { if (strncmp($a, $b, 3) == 0) return "** \"$a\" " .
        "and \"$b\"\n** Look the same to me! (looking at the first 3 chars)"; },
    function($a,$b) { return "CRCs: " . crc32($a) . ", ".crc32($b); },
    function($a,$b) { return "similar(a,b) = " . similar_text($a, $b, $p) . "($p%)"; }
);
echo "\nUsing the second array of dynamic functions\n";
process("Twas brilling and the slithy toves", "Twas the night", $garr);
?>
```

The above example will output:

```
Using the first array of dynamic functions
parameters: 2.3445, M_PI
some trig: -1.6291725057799
a hypotenuse: 3.9199852871011
b*a^2 = 4.8103313314525
min(b^2+a, a^2,b) = 8.6382729035898
ln(a)/b = 0.27122299212594

Using the second array of dynamic functions
** "Twas the night" and "Twas brilling and the slithy toves"
** Look the same to me! (looking at the first 3 chars)
```

```
CRCs: 3569586014, 342550513
similar(a,b) = 11(45.833333333333%)
```

Example #3 Using dynamic functions as callback functions

Perhaps the most common use for dynamic functions is to pass them as callbacks, for example when using array_walk() or usort().

```php
<?php
$av = array("the ", "a ", "that ", "this ");
array_walk($av, create_function('&$v,$k', '$v = $v . "mango";'));
print_r($av);
?>
```

Converted to an anonymous function:

```php
<?php
$av = array("the ", "a ", "that ", "this ");
array_walk($av, function(&$v,$k) { $v = $v . "mango"; });
print_r($av);
?>
```

The above example will output:

```
Array
(
  [0] => the mango
  [1] => a mango
  [2] => that mango
  [3] => this mango
)
```

Sorting strings from longest to shortest with **create_function()**:

```php
<?php
$sv = array("small", "a big string", "larger", "it is a string thing");
echo "Original:\n";
print_r($sv);
echo "Sorted:\n";
usort($sv, create_function('$a,$b','return strlen($b) - strlen($a);'));
print_r($sv);
?>
```

Converted to an anonymous function:

```php
<?php
$sv = array("small", "a big string", "larger", "it is a string thing");
echo "Original:\n";
print_r($sv);
echo "Sorted:\n";
usort($sv, function($a,$b) { return strlen($b) - strlen($a); });
print_r($sv);
?>
```

The above example will output:

```
Original:
Array
```

```
(
  [0] => small
  [1] => a big string
  [2] => larger
  [3] => it is a string thing
)
Sorted:
Array
(
  [0] => it is a string thing
  [1] => a big string
  [2] => larger
  [3] => small
)
```

See Also

- Anonymous functions

forward_static_call

forward_static_call

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

forward_static_call — Call a callback while preserving the current called class

Description

function **forward_static_call**(callable `$callback`, mixed `...$args`): mixed

Calls a user defined function or method given by the `callback` parameter, with the following arguments. This function must be called within a method context, it can't be used outside a class. When calling a method callback, it makes a forwarding call, so `static::` inside the method resolves to the same called class as in the caller. This is useful for static method callbacks that should behave like `self::` or `parent::`.

Parameters

`callback`

The function or method to be called. This parameter may be an array, with the name of the class, and the method, or a string, with a function name.

`args`

Zero or more parameters to be passed to the callback.

Return Values

Returns the function result, or **`false`** on error.

Examples

Example #1 forward_static_call() example

```php
<?php

class A
{
    const NAME = 'A';
    public static function test() {
        $args = func_get_args();
        echo static::NAME, " ".join(',', $args)." \n";
    }
}

class B extends A
```

```php
{
    const NAME = 'B';

    public static function test() {
        echo self::NAME, "\n";
        forward_static_call(array('A', 'test'), 'more', 'args');
        forward_static_call( 'test', 'other', 'args');
    }
}

B::test('foo');

function test() {
        $args = func_get_args();
        echo "C ".join(',', $args)." \n";
    }

?>
```

The above example will output:

```
B
B more,args
C other,args
```

See Also

- forward_static_call_array() - Call a callback with an array of parameters, preserving the current called class
- call_user_func_array() - Call a callback with an array of parameters
- call_user_func() - Call the callback given by the first parameter
- is_callable() - Verify that a value can be called as a function from the current scope

forward_static_call_array

forward_static_call_array

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

forward_static_call_array — Call a callback with an array of parameters, preserving the current called class

Description

function **forward_static_call_array**(callable `$callback`, array `$args`): mixed

Calls a user defined function or method given by the `callback` parameter, with arguments passed as an array, similarly to call_user_func_array(). When called from within a method context and calling a method callback, it makes a forwarding call, so `static::` inside the method resolves to the same called class as in the caller. This is useful for static method callbacks that should behave like `self::` or `parent::`.

> **Note** :
>
> Unlike forward_static_call(), this function does not require an active class scope: calling forward_static_call() outside a class throws an Error, while this function does not.

Parameters

`callback`

> The function or method to be called. This parameter may be an array, with the name of the class, and the method, or a string, with a function name.

`args`

The parameters to be passed to the callback, as an array.

> **Note** :
>
> To pass a parameter by reference, the corresponding element in `args` must be a reference.

Return Values

Returns the function result, or `false` on error.

Examples

Example #1 forward_static_call_array() example

```php
<?php

class A
{
    const NAME = 'A';
    public static function test() {
        $args = func_get_args();
        echo static::NAME, " ".join(',', $args)." \n";
    }
}

class B extends A
{
    const NAME = 'B';

    public static function test() {
        echo self::NAME, "\n";
        forward_static_call_array(array('A', 'test'), array('more', 'args'));
        forward_static_call_array( 'test', array('other', 'args'));
    }
}

B::test('foo');

function test() {
        $args = func_get_args();
        echo "C ".join(',', $args)." \n";
    }

?>
```

The above example will output:

```
B
B more,args
C other,args
```

See Also

- forward_static_call() - Call a callback while preserving the current called class
- call_user_func() - Call the callback given by the first parameter
- call_user_func_array() - Call a callback with an array of parameters
- is_callable() - Verify that a value can be called as a function from the current scope

func_get_arg

func_get_arg

(PHP 4, PHP 5, PHP 7, PHP 8)

func_get_arg — Return an item from the argument list

Description

function **func_get_arg**(int `$position`): mixed

Gets the specified argument from a user-defined function's argument list.

This function may be used in conjunction with func_get_args() and func_num_args() to allow user-defined functions to accept variable-length argument lists.

Parameters

`position`

The argument offset. Function arguments are counted starting from zero.

Return Values

Returns the specified argument, or `false` on error.

Errors/Exceptions

Generates a warning if called from outside of a user-defined function, or if `position` is greater than the number of arguments actually passed.

Examples

Example #1 **func_get_arg()** example

```php
<?php
function foo()
{
    $numargs = func_num_args();
    echo "Number of arguments: $numargs\n";
    if ($numargs >= 2) {
        echo "Second argument is: " . func_get_arg(1) . "\n";
    }
}

foo(1, 2, 3);
?>
```

The above example will output:

```
Number of arguments: 3
Second argument is: 2
```

Example #2 **func_get_arg()** example of byref and byval arguments

```php
<?php
function byVal($arg) {
    echo 'As passed     : ', var_export(func_get_arg(0)), PHP_EOL;
    $arg = 'baz';
    echo 'After change  : ', var_export(func_get_arg(0)), PHP_EOL;
}

function byRef(&$arg) {
    echo 'As passed     : ', var_export(func_get_arg(0)), PHP_EOL;
    $arg = 'baz';
    echo 'After change  : ', var_export(func_get_arg(0)), PHP_EOL;
}
```

```
$arg = 'bar';
byVal($arg);
byRef($arg);
?>
```

The above example will output:

```
As passed     : 'bar'
After change  : 'baz'
As passed     : 'bar'
After change  : 'baz'
```

Notes

> **Note** :
>
> As of PHP 8.0.0, the func_*() family of functions is intended to be mostly transparent with regard to named arguments, by treating the arguments as if they were all passed positionally, and missing arguments are replaced with their defaults. This function ignores the collection of unknown named variadic arguments. Unknown named arguments which are collected can only be accessed through the variadic parameter.

> **Note** :
>
> If the arguments are passed by reference, any changes to the arguments will be reflected in the values returned by this function. As of PHP 7 the current values will also be returned if the arguments are passed by value.

> **Note** :
>
> This function returns a copy of the passed arguments only, and does not account for default (non-passed) arguments.

See Also

- ... syntax
- func_get_args() - Returns an array comprising a function's argument list
- func_num_args() - Returns the number of arguments passed to the function

func_get_args

func_get_args

(PHP 4, PHP 5, PHP 7, PHP 8)

func_get_args — Returns an array comprising a function's argument list

Description

function **func_get_args**(): array

Gets an array of the function's argument list.

This function may be used in conjunction with func_get_arg() and func_num_args() to allow user-defined functions to accept variable-length argument lists.

Parameters

This function has no parameters.

Return Values

Returns an array in which each element is a copy of the corresponding member of the current user-defined function's argument list.

Errors/Exceptions

Generates a warning if called from outside of a user-defined function.

Examples

Example #1 func_get_args() example

```php
<?php
function foo()
{
    $numargs = func_num_args();
    echo "Number of arguments: $numargs \n";
    if ($numargs >= 2) {
        echo "Second argument is: " . func_get_arg(1) . "\n";
    }
    $arg_list = func_get_args();
    for ($i = 0; $i < $numargs; $i++) {
        echo "Argument $i is: " . $arg_list[$i] . "\n";
    }
}

foo(1, 2, 3);
?>
```

The above example will output:

```
Number of arguments: 3
Second argument is: 2
Argument 0 is: 1
Argument 1 is: 2
Argument 2 is: 3
```

Example #2 func_get_args() example of byref and byval arguments

```php
<?php
function byVal($arg) {
    echo 'As passed     : ', var_export(func_get_args()), PHP_EOL;
    $arg = 'baz';
    echo 'After change  : ', var_export(func_get_args()), PHP_EOL;
}

function byRef(&$arg) {
    echo 'As passed     : ', var_export(func_get_args()), PHP_EOL;
    $arg = 'baz';
    echo 'After change  : ', var_export(func_get_args()), PHP_EOL;
}

$arg = 'bar';
byVal($arg);
byRef($arg);
?>
```

The above example will output:

```
As passed     : array (
```

```
  0 => 'bar',
)
After change  : array (
  0 => 'baz',
)
As passed     : array (
  0 => 'bar',
)
After change  : array (
  0 => 'baz',
)
```

Notes

> **Note** :
>
> As of PHP 8.0.0, the func_*() family of functions is intended to be mostly transparent with regard to named arguments, by treating the arguments as if they were all passed positionally, and missing arguments are replaced with their defaults. This function ignores the collection of unknown named variadic arguments. Unknown named arguments which are collected can only be accessed through the variadic parameter.

> **Note** :
>
> If the arguments are passed by reference, any changes to the arguments will be reflected in the values returned by this function. As of PHP 7 the current values will also be returned if the arguments are passed by value.

> **Note** :
>
> This function returns a copy of the passed arguments only, and does not account for default (non-passed) arguments.

See Also

- ... syntax
- func_get_arg() - Return an item from the argument list
- func_num_args() - Returns the number of arguments passed to the function
- ReflectionFunctionAbstract::getParameters() - Gets parameters

func_num_args

func_num_args

(PHP 4, PHP 5, PHP 7, PHP 8)

func_num_args — Returns the number of arguments passed to the function

Description

function **func_num_args**(): int

Gets the number of arguments passed to the function.

This function may be used in conjunction with func_get_arg() and func_get_args() to allow user-defined functions to accept variable-length argument lists.

Parameters

This function has no parameters.

Return Values

Returns the number of arguments passed into the current user-defined function.

Errors/Exceptions

Generates a warning if called from outside of a user-defined function.

Examples

Example #1 func_num_args() example

```php
<?php
function foo()
{
    echo "Number of arguments: ", func_num_args(), PHP_EOL;
}

foo(1, 2, 3);
?>
```

The above example will output:

```
Number of arguments: 3
```

Notes

> **Note** :
>
> As of PHP 8.0.0, the func_*() family of functions is intended to be mostly transparent with regard to named arguments, by treating the arguments as if they were all passed positionally, and missing arguments are replaced with their defaults. This function ignores the collection of unknown named variadic arguments. Unknown named arguments which are collected can only be accessed through the variadic parameter.

See Also

- ... syntax
- func_get_arg() - Return an item from the argument list
- func_get_args() - Returns an array comprising a function's argument list
- ReflectionFunctionAbstract::getNumberOfParameters() - Gets number of parameters

function_exists

function_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

function_exists — Return `true` if the given function has been defined

Description

function **function_exists**(string `$function`): bool

Checks the list of defined functions, both built-in (internal) and user-defined, for `function`.

Parameters

`function`

The function name, as a string.

Return Values

Returns `true` if `function` exists and is a function, `false` otherwise.

> **Note** :
>
> This function will return `false` for constructs, such as include_once and echo.

Examples

Example #1 function_exists() example

```
<?php
if (function_exists('imap_open')) {
    echo "IMAP functions are available.<br />\n";
} else {
    echo "IMAP functions are not available.<br />\n";
}
?>
```

Notes

> **Note** :
>
> A function name may exist even if the function itself is unusable due to configuration or compiling options (with the image functions being an example).

See Also

- method_exists() - Checks if the class method exists
- is_callable() - Verify that a value can be called as a function from the current scope
- get_defined_functions() - Returns an array of all defined functions
- class_exists() - Checks if the class has been defined
- extension_loaded() - Find out whether an extension is loaded

get_defined_functions

get_defined_functions

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

get_defined_functions — Returns an array of all defined functions

Description

function **get_defined_functions**(bool `$exclude_disabled = true`): array

Gets an array of all defined functions.

Parameters

`exclude_disabled`

Whether disabled functions should be excluded from the return value. This parameter has no effect since PHP 8.0.0.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.5.0. Relying on this feature is highly discouraged.

Return Values

Returns a multidimensional array containing a list of all defined functions, both built-in (internal) and user-defined. The internal functions will be accessible via *$arr["internal"]*, and the user defined ones using *$arr["user"]* (see example below).

Changelog

Version	Description
8.5.0	The `exclude_disabled` parameter has been deprecated, as it no longer has any effect.
8.0.0	The default value of the `exclude_disabled` parameter has been changed from `false` to `true`. However, it will not have any effect as disabled functions are removed from the function table at compile time.
7.0.15, 7.1.1	The `exclude_disabled` parameter has been added.

Examples

Example #1 get_defined_functions() example

```php
<?php
function myrow($id, $data)
{
    return "<tr><th>$id</th><td>$data</td></tr>\n";
}

$arr = get_defined_functions();

print_r($arr);
?>
```

The above example will output something similar to:

```
Array
(
    [internal] => Array
        (
            [0] => zend_version
            [1] => func_num_args
            [2] => func_get_arg
            [3] => func_get_args
            [4] => strlen
            [5] => strcmp
            [6] => strncmp
            ...
            [750] => bcscale
            [751] => bccomp
        )

    [user] => Array
        (
            [0] => myrow
        )

)
```

See Also

- function_exists() - Return true if the given function has been defined
- get_defined_vars() - Returns an array of all defined variables
- get_defined_constants() - Returns an associative array with the names of all the constants and their values
- get_declared_classes() - Returns an array with the name of the defined classes

register_shutdown_function

register_shutdown_function

(PHP 4, PHP 5, PHP 7, PHP 8)

register_shutdown_function — Register a function for execution on shutdown

Description

function **register_shutdown_function**(callable `$callback`, mixed `...$args`): void

Registers a `callback` to be executed after script execution finishes or exit() is called.

Multiple calls to **register_shutdown_function()** can be made, and each will be called in the same order as they were registered. If you call exit() within one registered shutdown function, processing will stop completely and no other registered shutdown functions will be called.

> **Caution**
>
> Since PHP 8.4.0, a parameterless exit() call within a registered shutdown function resets the exit code to `0`. Calling exit() with an explicit status overwrites the previous exit code in all versions.

Shutdown functions may also call **register_shutdown_function()** themselves to add a shutdown function to the end of the queue.

Parameters

`callback`

The shutdown callback to register.

The shutdown callbacks are executed as the part of the request, so it's possible to send output from them and access output buffers.

`args`

It is possible to pass parameters to the shutdown function by passing additional parameters.

Return Values

No value is returned.

Examples

Example #1 register_shutdown_function() example

```php
<?php
function shutdown()
{
    // This is our shutdown function, in
    // here we can do any last operations
    // before the script is complete.

    echo 'Script executed with success', PHP_EOL;
}

register_shutdown_function('shutdown');
?>
```

Notes

> **Note** :
>
> The working directory of the script can change inside the shutdown function under some web servers, e.g. Apache.

> **Note** :
>
> Shutdown functions will not be executed if the process is killed with a SIGTERM or SIGKILL signal. While you cannot intercept a SIGKILL, you can use pcntl_signal() to install a handler for a SIGTERM which uses exit() to end cleanly.

> **Note** :
>
> Shutdown functions run separately from the time tracked by max_execution_time. That means even if a process is terminated for running too long, shutdown functions will still be called. Additionally, if the `max_execution_time` runs out while a shutdown function is running it will not be terminated.

See Also

- auto_append_file
- exit() - Terminate the current script with a status code or message
- fastcgi_finish_request() - Flushes all response data to the client
- The section on connection handling

register_tick_function

register_tick_function

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

register_tick_function — Register a function for execution on each tick

Description

function **register_tick_function**(callable `$callback`, mixed `...$args`): bool

Registers the given `callback` to be executed when a tick is called.

Parameters

`callback`

The function to register.

`args`

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.5.0	Tick handlers are now deactivated later during request shutdown: after all shutdown functions and destructors have run, and after the output handlers have been cleaned up. Previously they were deactivated at the very beginning of request shutdown, so tick functions were no longer called during shutdown functions, destructors, or output handling. See the migration guide for details.

Examples

Example #1 register_tick_function() example

```php
<?php
declare(ticks=1);

function my_tick_function($param) {
    echo "Tick callback function called with param: $param\n";
}

register_tick_function('my_tick_function', true);
?>
```

See Also

- declare
- unregister_tick_function() - De-register a function for execution on each tick

unregister_tick_function

unregister_tick_function

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

unregister_tick_function — De-register a function for execution on each tick

Description

function **unregister_tick_function**(callable `$callback`): void

De-registers the function `callback` so it is no longer executed when a tick is called.

Parameters

`callback`

The function to de-register.

Return Values

No value is returned.

See Also

- register_tick_function() - Register a function for execution on each tick

Class/Object Information

Introduction

These functions allow you to obtain information about classes and instance objects. You can obtain the name of the class to which an object belongs, as well as its member properties and methods. Using these functions, you can find out not only the class membership of an object, but also its parentage (i.e. what class is the object class extending).

Please see the Objects section of the manual for a detailed explanation of how classes and objects are implemented and used in PHP.

Examples

In this example, we first define a base class and an extension of the class. The base class describes a general vegetable, whether it is edible, and what is its color. The subclass *Spinach* adds a method to cook it and another to find out if it is cooked.

Example #1 Class Definitions

Vegetable

```php
<?php

class Vegetable {
    public $edible;

    public $color;

    public function __construct($edible, $color = "green")
    {
        $this->edible = $edible;
        $this->color = $color;
    }

    public function isEdible()
    {
        return $this->edible;
    }

    public function getColor()
    {
        return $this->color;
    }
}

?>
```

Spinach

```php
<?php

class Spinach extends Vegetable {
    public $cooked = false;

    public function __construct()
    {
        parent::__construct(true, "green");
    }

    public function cook()
    {
        $this->cooked = true;
    }

    public function isCooked()
    {
        return $this->cooked;
    }
}
```

```
?>
```

We then instantiate 2 objects from these classes and print out information about them, including their class parentage. We also define some utility functions, mainly to have a nice printout of the variables.

Example #2 test_script.php

```php
<?php

// register autoloader to load classes
spl_autoload_register();

function printProperties($obj)
{
    foreach (get_object_vars($obj) as $prop => $val) {
        echo "\t$prop = $val\n";
    }
}

function printMethods($obj)
{
    $arr = get_class_methods(get_class($obj));
    foreach ($arr as $method) {
        echo "\tfunction $method()\n";
    }
}

function objectBelongsTo($obj, $class)
{
    if (is_subclass_of($obj, $class)) {
        echo "Object belongs to class " . get_class($obj);
        echo ", a subclass of $class\n";
    } else {
        echo "Object does not belong to a subclass of $class\n";
    }
}

// instantiate 2 objects
$veggie = new Vegetable(true, "blue");
$leafy = new Spinach();

// print out information about objects
echo "veggie: CLASS " . get_class($veggie) . "\n";
echo "leafy: CLASS " . get_class($leafy);
echo ", PARENT " . get_parent_class($leafy) . "\n";

// show veggie properties
echo "\nveggie: Properties\n";
printProperties($veggie);

// and leafy methods
echo "\nleafy: Methods\n";
printMethods($leafy);

echo "\nParentage:\n";
objectBelongsTo($leafy, Spinach::class);
objectBelongsTo($leafy, Vegetable::class);

?>
```

The above examples will output:

```
veggie: CLASS Vegetable
leafy: CLASS Spinach, PARENT Vegetable

veggie: Properties
        edible = 1
        color = blue
```

```
leafy: Methods
        function __construct()
        function cook()
        function isCooked()
        function isEdible()
        function getColor()

Parentage:
Object does not belong to a subclass of Spinach
Object belongs to class Spinach, a subclass of Vegetable
```

One important thing to note in the example above is that the object *$leafy* is an instance of the class **Spinach** which is a subclass of **Vegetable**.

Classes/Object Functions

Table of Contents

__autoload

__autoload

(PHP 5, PHP 7)

__autoload — Attempt to load undefined class

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **__autoload**(string `$class`): void

You can define this function to enable classes autoloading.

Parameters

`class`

Name of the class to load

Return Values

No value is returned.

See Also

- spl_autoload_register() - Register given function as __autoload() implementation

class_alias

class_alias

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

class_alias — Creates an alias for a class

Description

function **class_alias**(string `$class`, string `$alias`, bool `$autoload` = **`true`**): bool

Creates an alias named `alias` based on the user defined class `class`. The aliased class is exactly the same as the original class.

The class alias cannot be any of the PHP reserved words.

> **Note** :
>
> As of PHP 8.3.0, **class_alias()** also supports creating an alias of a PHP internal class.

Parameters

`class`

The original class.

`alias`

The alias name for the class.

`autoload`

Whether to autoload if the original class is not found.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.3.0	**class_alias()** now supports creating an alias of an internal class.

Examples

Example #1 class_alias() example

```php
<?php

class Foo { }

class_alias('Foo', 'Bar');

$a = new Foo;
$b = new Bar;

// the objects are the same
var_dump($a == $b, $a === $b);
var_dump($a instanceof $b);

// the classes are the same
var_dump($a instanceof Foo);
var_dump($a instanceof Bar);

var_dump($b instanceof Foo);
var_dump($b instanceof Bar);

?>
```

The above example will output:

```
bool(true)
bool(false)
bool(true)
bool(true)
bool(true)
bool(true)
bool(true)
```

Notes

> **Note** :
>
> Class names are case-insensitive in PHP, and this is reflected in this function. Aliases created by **class_alias()** are declared in lowercase. This means that for a class `MyClass`, the `class_alias('MyClass', 'MyClassAlias')` call will declare a new class alias named `myclassalias`.

See Also

- get_parent_class() - Retrieves the parent class name for object or class
- is_subclass_of() - Checks if the object has this class as one of its parents or implements it

class_exists

class_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

class_exists — Checks if the class has been defined

Description

function **class_exists**(string `$class`, bool `$autoload` = `true`): bool

This function checks whether or not the given class has been defined.

Parameters

`class`

The class name. The name is matched in a case-insensitive manner.

`autoload`

Whether to autoload if not already loaded.

Return Values

Returns `true` if `class` is a defined class, `false` otherwise.

Examples

Example #1 class_exists() example

```php
<?php
// Check that the class exists before trying to use it
if (class_exists('MyClass')) {
    $myclass = new MyClass();
}

?>
```

Example #2 `autoload` parameter example

```php
<?php
spl_autoload_register(function ($class_name) {
    include $class_name . '.php';

    // Check to see whether the include declared the class
    if (!class_exists($class_name, false)) {
        throw new LogicException("Unable to load class: $class_name");
    }
});

if (class_exists(MyClass::class)) {
    $myclass = new MyClass();
}

?>
```

See Also

- function_exists() - Return true if the given function has been defined
- enum_exists() - Checks if the enum has been defined
- interface_exists() - Checks if the interface has been defined
- get_declared_classes() - Returns an array with the name of the defined classes

enum_exists

enum_exists

(PHP 8 >= 8.1.0)

enum_exists — Checks if the enum has been defined

Description

function **enum_exists**(string `$enum`, bool `$autoload = true`): bool

This function checks whether or not the given enum has been defined.

Parameters

`enum`

The enum name. The name is matched in a case-insensitive manner.

`autoload`

Whether to autoload if not already loaded.

Return Values

Returns `true` if `enum` is a defined enum, `false` otherwise.

Examples

Example #1 enum_exists() example

```php
<?php
// Check that the enum exists before trying to use it
if (enum_exists(Suit::class)) {
    $myclass = Suit::Hearts;
}
?>
```

See Also

- function_exists() - Return true if the given function has been defined
- class_exists() - Checks if the class has been defined
- interface_exists() - Checks if the interface has been defined
- get_declared_classes() - Returns an array with the name of the defined classes

get_called_class

get_called_class

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

get_called_class — The "Late Static Binding" class name

Description

function **get_called_class**(): string

Gets the name of the class the static method is called in.

Parameters

This function has no parameters.

Return Values

Returns the class name.

Errors/Exceptions

If **get_called_class()** is called from outside a class, an Error is thrown. Prior to PHP 8.0.0, an `E_WARNING` level error was raised.

Changelog

Version	Description
8.0.0	Calling this function from outside a class, will now throw an Error. Previously, an `E_WARNING` was raised and the function returned `false`.

Examples

Example #1 Using get_called_class()

```php
<?php

class foo {
    static public function test() {
        var_dump(get_called_class());
    }
}

class bar extends foo {
}

foo::test();
bar::test();

?>
```

The above example will output:

```
string(3) "foo"
string(3) "bar"
```

See Also

- get_parent_class() - Retrieves the parent class name for object or class
- get_class() - Returns the name of the class of an object
- is_subclass_of() - Checks if the object has this class as one of its parents or implements it

get_class

get_class

(PHP 4, PHP 5, PHP 7, PHP 8)

get_class — Returns the name of the class of an object

Description

function **get_class**(object `$object` = ?): string

Gets the name of the class of the given `object`.

Parameters

`object`

The tested object.

> **Note** :
>
> Explicitly passing `null` as the `object` is no longer allowed as of PHP 7.2.0 and emits an `E_WARNING`. As of PHP 8.0.0, a TypeError is emitted when `null` is used.

Return Values

Returns the name of the class of which `object` is an instance.

If the `object` is an instance of a class which exists in a namespace, the qualified namespaced name of that class is returned.

Errors/Exceptions

If **get_class()** is called with anything other than an object, TypeError is raised. Prior to PHP 8.0.0, an `E_WARNING` level error was raised.

If **get_class()** is called with no arguments from outside a class, an Error is thrown. Prior to PHP 8.0.0, an `E_WARNING` level error was raised.

Changelog

Version	Description
8.3.0	Calling **get_class()** without an argument now emits an `E_DEPRECATED` warning; previously, calling this function inside a class returned the name of that class.
8.0.0	Calling this function from outside a class, without any arguments, will now throw an Error. Previously, an `E_WARNING` was raised and the function returned `false`.
7.2.0	Prior to this version the default value for `object` was `null` and it had the same effect as not passing any value. Now `null` has been removed as the default value for `object`, and is no longer a valid input.

Examples

Example #1 Using get_class()

```php
<?php

class foo {
    function name()
    {
        echo "My name is " , get_class($this) , "\n";
    }
}

// create an object
$bar = new foo();

// external call
echo "Its name is " , get_class($bar) , "\n";

// internal call
$bar->name();

?>
```

The above example will output:

```
Its name is foo
My name is foo
```

Example #2 Using get_class() in superclass

```php
<?php

abstract class bar {
    public function __construct()
    {
        var_dump(get_class($this));
        var_dump(get_class());
    }
}

class foo extends bar {
}

new foo;

?>
```

The above example will output:

```
string(3) "foo"
string(3) "bar"
```

Example #3 Using get_class() with namespaced classes

```php
<?php

namespace Foo\Bar;

class Baz {
    public function __construct()
    {

    }
}

$baz = new \Foo\Bar\Baz;

var_dump(get_class($baz));
?>
```

The above example will output:

```
string(11) "Foo\Bar\Baz"
```

See Also

- get_called_class() - The "Late Static Binding" class name
- get_parent_class() - Retrieves the parent class name for object or class
- gettype() - Get the type of a variable
- get_debug_type() - Gets the type name of a variable in a way that is suitable for debugging
- is_subclass_of() - Checks if the object has this class as one of its parents or implements it

get_class_methods

get_class_methods

(PHP 4, PHP 5, PHP 7, PHP 8)

get_class_methods — Gets the class methods' names

Description

function **get_class_methods**(object|string `$object_or_class`): array

Gets the class methods names.

Parameters

`object_or_class`

The class name or an object instance

Return Values

Returns an array of method names defined for the class specified by `object_or_class`.

Changelog

Version	Description
8.0.0	The `object_or_class` parameter now only accepts objects or valid class names.

Examples

Example #1 **get_class_methods()** example

```php
<?php

class myclass {
    // constructor
    function __construct()
    {
        return(true);
    }

    // method 1
    function myfunc1()
    {
        return(true);
    }

    // method 2
    function myfunc2()
    {
        return(true);
    }
}

$class_methods = get_class_methods('myclass');
// or
$class_methods = get_class_methods(new myclass());

foreach ($class_methods as $method_name) {
    echo "$method_name\n";
}

?>
```

The above example will output:

```
__construct
myfunc1
myfunc2
```

See Also

- get_class() - Returns the name of the class of an object
- get_class_vars() - Get the default properties of the class
- get_object_vars() - Gets the properties of the given object

get_class_vars

get_class_vars

(PHP 4, PHP 5, PHP 7, PHP 8)

get_class_vars — Get the default properties of the class

Description

function **get_class_vars**(string `$class`): array

Get the default properties of the given class.

Parameters

`class`

The class name

Return Values

Returns an associative array of declared properties visible from the current scope, with their default value. The resulting array elements are in the form of `varname => value`. In case of an error, it returns **`false`**.

Examples

Example #1 get_class_vars() example

```php
<?php

class MyClass
{
    public $var1; // This has no explicit default value (technically it has NULL as a default)...
    public $var2 = "xyz";
    public $var3 = 100;
    private $var4;

    public function __construct()
    {
        // Change some properties
        $this->var1 = "foo";
        $this->var2 = "bar";
        return true;
    }
}

$my_class = new MyClass();

$class_vars = get_class_vars(get_class($my_class));

foreach ($class_vars as $name => $value) {
    echo "{$name}: ", var_export($value, true), "\n";
}

?>
```

The above example will output:

```
var1: NULL
var2: 'xyz'
var3: 100
```

Example #2 get_class_vars() and scoping behaviour

```php
<?php

function format($array)
{
    return implode('|', array_keys($array)) . "\r\n";
}

class TestCase
{
    public $a    = 1;
    protected $b = 2;
    private $c   = 3;

    public static function expose()
    {
        echo format(get_class_vars(__CLASS__));
    }
}

TestCase::expose();
echo format(get_class_vars('TestCase'));

?>
```

The above example will output:

```
// 5.0.0
a| * b| TestCase c
a| * b| TestCase c

// 5.0.1 - 5.0.2
a|b|c
a|b|c

// 5.0.3 +
a|b|c
a
```

See Also

- get_class_methods() - Gets the class methods' names
- get_object_vars() - Gets the properties of the given object

get_declared_classes

get_declared_classes

(PHP 4, PHP 5, PHP 7, PHP 8)

get_declared_classes — Returns an array with the name of the defined classes

Description

function **get_declared_classes**(): array

Gets the declared classes.

Parameters

This function has no parameters.

Return Values

Returns an array of the names of the declared classes in the current script.

> **Note** :
>
> Note that depending on what extensions you have compiled or loaded into PHP, additional classes could be present. This means that you will not be able to define your own classes using these names. There is a list of predefined classes in the Predefined Classes section of the appendices.

Changelog

Version	Description
7.4.0	Previously **get_declared_classes()** always returned parent classes before child classes. This is no longer the case. No particular order is guaranteed for the **get_declared_classes()** return value.

Examples

Example #1 get_declared_classes() example

```php
<?php
print_r(get_declared_classes());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => stdClass
    [1] => __PHP_Incomplete_Class
    [2] => Directory
)
```

See Also

- class_exists() - Checks if the class has been defined
- get_declared_interfaces() - Returns an array of all declared interfaces
- get_defined_functions() - Returns an array of all defined functions

get_declared_interfaces

get_declared_interfaces

(PHP 5, PHP 7, PHP 8)

get_declared_interfaces — Returns an array of all declared interfaces

Description

function **get_declared_interfaces**(): array

Gets the declared interfaces.

Parameters

This function has no parameters.

Return Values

Returns an array of the names of the declared interfaces in the current script.

Examples

Example #1 get_declared_interfaces() example

```php
<?php
print_r(get_declared_interfaces());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Traversable
    [1] => IteratorAggregate
    [2] => Iterator
    [3] => ArrayAccess
    [4] => reflector
    [5] => RecursiveIterator
    [6] => SeekableIterator
)
```

See Also

- interface_exists() - Checks if the interface has been defined
- get_declared_classes() - Returns an array with the name of the defined classes
- class_implements() - Return the interfaces which are implemented by the given class or interface

get_declared_traits

get_declared_traits

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

get_declared_traits — Returns an array of all declared traits

Description

function **get_declared_traits**(): array

Parameters

This function has no parameters.

Return Values

Returns an array with names of all declared traits in values.

See Also

- class_uses() - Return the traits used by the given class
- trait_exists() - Checks if the trait exists

get_mangled_object_vars

get_mangled_object_vars

(PHP 7 >= 7.4.0, PHP 8)

get_mangled_object_vars — Returns an array of mangled object properties

Description

function **get_mangled_object_vars**(object `$object`): array

Returns an array whose elements are the `object`'s properties. The keys are the member variable names, with a few notable exceptions: private variables have the class name prepended to the variable name, and protected variables have a `*` prepended to the variable name. These prepended values have `NUL` bytes on either side. Uninitialized typed properties are silently discarded.

Parameters

`object`

An object instance.

Return Values

Returns an array containing all properties, regardless of visibility, of `object`.

Examples

Example #1 get_mangled_object_vars() example

```php
<?php

class A
{
    public $public = 1;

    protected $protected = 2;

    private $private = 3;
}

class B extends A
{
    private $private = 4;
}

$object = new B;
$object->dynamic = 5;
$object->{'6'} = 6;

var_dump(get_mangled_object_vars($object));

class AO extends ArrayObject
{
    private $private = 1;
}

$arrayObject = new AO(['x' => 'y']);
$arrayObject->dynamic = 2;

var_dump(get_mangled_object_vars($arrayObject));
```

The above example will output:

```
array(6) {
  ["Bprivate"]=>
  int(4)
  ["public"]=>
  int(1)
  ["*protected"]=>
  int(2)
  ["Aprivate"]=>
  int(3)
  ["dynamic"]=>
  int(5)
  [6]=>
  int(6)
}
array(2) {
```

```
  ["AOprivate"]=>
  int(1)
  ["dynamic"]=>
  int(2)
}
```

See Also

- get_class_vars() - Get the default properties of the class
- get_object_vars() - Gets the properties of the given object

get_object_vars

get_object_vars

(PHP 4, PHP 5, PHP 7, PHP 8)

get_object_vars — Gets the properties of the given object

Description

function **get_object_vars**(object `$object`): array

Gets the accessible non-static properties of the given `object` according to scope.

Parameters

`object`

An object instance.

Return Values

Returns an associative array of defined object accessible non-static properties for the specified `object` in scope.

Examples

Example #1 Use of get_object_vars()

```php
<?php

class foo {
    private $a;
    public $b = 1;
    public $c;
    private $d;
    static $e;

    public function test() {
        var_dump(get_object_vars($this));
    }
}

$test = new foo;
var_dump(get_object_vars($test));

$test->test();

?>
```

The above example will output:

```
array(2) {
  ["b"]=>
  int(1)
```

```
  ["c"]=>
  NULL
}
array(4) {
  ["a"]=>
  NULL
  ["b"]=>
  int(1)
  ["c"]=>
  NULL
  ["d"]=>
  NULL
}
```

> **Note** :
>
> Uninitialized properties are considered inaccessible, and thus will not be included in the array.

See Also

- get_class_methods() - Gets the class methods' names
- get_class_vars() - Get the default properties of the class

get_parent_class

get_parent_class

(PHP 4, PHP 5, PHP 7, PHP 8)

get_parent_class — Retrieves the parent class name for object or class

Description

function **get_parent_class**(object|string `$object_or_class` = ?): string|false

Retrieves the parent class name for object or class.

Parameters

`object_or_class`

The tested object or class name.

Return Values

Returns the name of the parent class of the class of which `object_or_class` is an instance or the name.

If the object does not have a parent or the given class does not exist, `false` will be returned.

Changelog

Version	Description
8.3.0	Calling **get_parent_class()** without an argument now emits an `E_DEPRECATED` warning; previously, calling this function inside a class returned the name of that class.
8.0.0	The `object_or_class` parameter now only accepts objects or valid class names.

Examples

Example #1 Using **get_parent_class()**

```php
<?php

class Dad {
    function __construct()
    {
```

```
    // implements some logic
    }
}

class Child extends Dad {
    function __construct()
    {
        echo "I'm " , get_parent_class($this) , "'s son\n";
    }
}

class Child2 extends Dad {
    function __construct()
    {
        echo "I'm " , get_parent_class('child2') , "'s son too\n";
    }
}

$foo = new child();
$bar = new child2();

?>
```

The above example will output:

```
I'm Dad's son
I'm Dad's son too
```

See Also

- get_class() - Returns the name of the class of an object
- is_subclass_of() - Checks if the object has this class as one of its parents or implements it
- class_parents() - Return the parent classes of the given class

interface_exists

interface_exists

(PHP 5 >= 5.0.2, PHP 7, PHP 8)

interface_exists — Checks if the interface has been defined

Description

function **interface_exists**(string `$interface`, bool `$autoload` = **`true`**): bool

Checks if the given interface has been defined.

Parameters

`interface`

The interface name

`autoload`

Whether to autoload if not already loaded.

Return Values

Returns **`true`** if the interface given by `interface` has been defined, **`false`** otherwise.

Examples

Example #1 interface_exists() example

```php
<?php
// Check the interface exists before trying to use it
if (interface_exists('MyInterface')) {
    class MyClass implements MyInterface
    {
        // Methods
    }
}

?>
```

See Also

- get_declared_interfaces() - Returns an array of all declared interfaces
- class_implements() - Return the interfaces which are implemented by the given class or interface
- class_exists() - Checks if the class has been defined
- enum_exists() - Checks if the enum has been defined

is_a

is_a

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

is_a — Checks whether the object is of a given type or subtype

Description

function **is_a**(mixed `$object_or_class`, string `$class`, bool `$allow_string = false`): bool

Determines if the given `object_or_class` is the `class` object type, or has `class` as one of its supertypes.

Parameters

`object_or_class`

A class name or an object instance.

`class`

The class or interface name

`allow_string`

If this parameter set to `false`, string class name as `object_or_class` is not allowed. This also prevents from calling autoloader if the class doesn't exist.

Return Values

Returns `true` if `object_or_class` is the `class` object type, or has `class` as one of its supertypes, `false` otherwise.

Examples

Example #1 is_a() example

```php
<?php
// define a class
class WidgetFactory
{
  var $oink = 'moo';
}

// create a new object
$WF = new WidgetFactory();

if (is_a($WF, 'WidgetFactory')) {
  echo "yes, \$WF is still a WidgetFactory\n";
}
?>
```

Example #2 Using the *instanceof* operator

```
<?php
// define a class
class WidgetFactory
{
  var $oink = 'moo';
}

// create a new object
$WF = new WidgetFactory();

if ($WF instanceof WidgetFactory) {
    echo 'Yes, $WF is a WidgetFactory';
}
?>
```

See Also

- get_class() - Returns the name of the class of an object
- get_parent_class() - Retrieves the parent class name for object or class
- is_subclass_of() - Checks if the object has this class as one of its parents or implements it

is_subclass_of

is_subclass_of

(PHP 4, PHP 5, PHP 7, PHP 8)

is_subclass_of — Checks if the object has this class as one of its parents or implements it

Description

function **is_subclass_of**(mixed `$object_or_class`, string `$class`, bool `$allow_string = true`): bool

Checks if the given `object_or_class` has the class `class` as one of its parents or implements it.

Parameters

`object_or_class`

A class name or an object instance. No error is generated if the class does not exist.

`class`

The class name

`allow_string`

If this parameter set to false, string class name as `object_or_class` is not allowed. This also prevents from calling autoloader if the class doesn't exist.

Return Values

This function returns `true` if the object `object_or_class`, belongs to a class which is a subclass of `class`, `false` otherwise.

Examples

Example #1 is_subclass_of() example

```
<?php
// define a class
class WidgetFactory
{
  var $oink = 'moo';
}

// define a child class
class WidgetFactory_Child extends WidgetFactory
{
  var $oink = 'oink';
```

```php
}

// create a new object
$WF = new WidgetFactory();
$WFC = new WidgetFactory_Child();

if (is_subclass_of($WFC, 'WidgetFactory')) {
  echo "yes, \$WFC is a subclass of WidgetFactory\n";
} else {
  echo "no, \$WFC is not a subclass of WidgetFactory\n";
}


if (is_subclass_of($WF, 'WidgetFactory')) {
  echo "yes, \$WF is a subclass of WidgetFactory\n";
} else {
  echo "no, \$WF is not a subclass of WidgetFactory\n";
}


if (is_subclass_of('WidgetFactory_Child', 'WidgetFactory')) {
  echo "yes, WidgetFactory_Child is a subclass of WidgetFactory\n";
} else {
  echo "no, WidgetFactory_Child is not a subclass of WidgetFactory\n";
}
?>
```

The above example will output:

```
yes, $WFC is a subclass of WidgetFactory
no, $WF is not a subclass of WidgetFactory
yes, WidgetFactory_Child is a subclass of WidgetFactory
```

Example #2 is_subclass_of() using interface example

```php
<?php
// Define the Interface
interface MyInterface
{
  public function MyFunction();
}

// Define the class implementation of the interface
class MyClass implements MyInterface
{
  public function MyFunction()
  {
    return "MyClass Implements MyInterface!";
  }
}

// Instantiate the object
$my_object = new MyClass;

// Works since 5.3.7

// Test using the object instance of the class
if (is_subclass_of($my_object, 'MyInterface')) {
  echo "Yes, \$my_object is a subclass of MyInterface\n";
} else {
  echo "No, \$my_object is not a subclass of MyInterface\n";
}

// Test using a string of the class name
if (is_subclass_of('MyClass', 'MyInterface')) {
```

```
  echo "Yes, MyClass is a subclass of MyInterface\n";
} else {
  echo "No, MyClass is not a subclass of MyInterface\n";
}
?>
```

The above example will output:

```
Yes, $my_object is a subclass of MyInterface
Yes, MyClass is a subclass of MyInterface
```

Notes

> **Note** :
>
> Using this function will use any registered autoloaders if the class is not already known.

See Also

- get_class() - Returns the name of the class of an object
- get_parent_class() - Retrieves the parent class name for object or class
- is_a() - Checks whether the object is of a given type or subtype
- class_parents() - Return the parent classes of the given class

method_exists

method_exists

(PHP 4, PHP 5, PHP 7, PHP 8)

method_exists — Checks if the class method exists

Description

function **method_exists**(object|string `$object_or_class`, string `$method`): bool

Checks if the class method exists in the given `object_or_class`.

Parameters

`object_or_class`

An object instance or a class name

`method`

The method name

Return Values

Returns `true` if the method given by `method` has been defined for the given `object_or_class`, `false` otherwise.

Changelog

Version	Description
7.4.0	Class checks against inherited private methods now return `false`.

Examples

Example #1 method_exists() example

```php
<?php
$directory = new Directory('.');
var_dump(method_exists($directory,'read'));
?>
```

The above example will output:

```
bool(true)
```

Example #2 Static method_exists() example

```php
<?php
var_dump(method_exists('Directory','read'));
?>
```

The above example will output:

```
bool(true)
```

Notes

> **Note** :
>
> Using this function will use any registered autoloaders if the class is not already known.

> **Note** :
>
> The **method_exists()** function cannot detect methods that are magically accessible using the `__call` magic method.

See Also

- function_exists() - Return true if the given function has been defined
- is_callable() - Verify that a value can be called as a function from the current scope
- class_exists() - Checks if the class has been defined

property_exists

property_exists

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

property_exists — Checks if the object or class has a property

Description

function **property_exists**(object|string `$object_or_class`, string `$property`): bool

This function checks if the given `property` exists in the specified class.

> **Note** :
>
> As opposed with isset(), **property_exists()** returns `true` even if the property has the value `null`.

Parameters

`object_or_class`

The class name or an object of the class to test for

`property`

The name of the property

Return Values

Returns `true` if the property exists, `false` if it doesn't exist.

Examples

Example #1 A property_exists() example

```php
<?php

class myClass {
    public $mine;
    private $xpto;
    static protected $test;

    static function test() {
        var_dump(property_exists('myClass', 'xpto')); //true
    }
}

var_dump(property_exists('myClass', 'mine'));   //true
var_dump(property_exists(new myClass, 'mine')); //true
var_dump(property_exists('myClass', 'xpto'));   //true
var_dump(property_exists('myClass', 'bar'));    //false
var_dump(property_exists('myClass', 'test'));   //true
myClass::test();

?>
```

property_exists() returns `true` regardless of property visibility, detects dynamic properties added to instances (when the class uses `#[AllowDynamicProperties]`), and ignores `__isset` and `__get` magic methods.

Example #2 Visibility, dynamic properties, and magic methods

```php
<?php

#[AllowDynamicProperties]
class MyClass {
    public $public;
    private $private;
    static protected $protectedStatic;

    // ignored by property_exists
    public function __isset(string $name): bool {
        return true;
    }

    // ignored by property_exists
    public function __get(string $name): string {
        return '';
    }
}

$instance = new MyClass();
$instance->instanced = 'abc';

var_dump(property_exists($instance, 'public'));          // true  (public)
var_dump(property_exists($instance, 'private'));         // true  (visibility ignored)
var_dump(property_exists($instance, 'protectedStatic')); // true  (visibility ignored)
var_dump(property_exists($instance, 'instanced'));       // true  (dynamic property on instance)
var_dump(property_exists($instance, 'undefined'));       // false (not declared, __isset ignored)
```

```php
var_dump(property_exists(MyClass::class, 'public'));    // true
var_dump(property_exists(MyClass::class, 'instanced')); // false (dynamic props not on class)
var_dump(property_exists(MyClass::class, 'undefined')); // false

?>
```

The above example will output:

```
bool(true)
bool(true)
bool(true)
bool(true)
bool(false)
bool(true)
bool(false)
bool(false)
```

Notes

> **Note** :
>
> Using this function will use any registered autoloaders if the class is not already known.

> **Note** :
>
> The **property_exists()** function cannot detect properties that are magically accessible using the `__get` magic method.

See Also

- method_exists() - Checks if the class method exists

trait_exists

trait_exists

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

trait_exists — Checks if the trait exists

Description

function **trait_exists**(string `$trait`, bool `$autoload = true`): bool

Parameters

`trait`

Name of the trait to check

`autoload`

Whether to autoload if not already loaded.

Return Values

Returns `true` if trait exists, and `false` otherwise.

Error Handling and Logging

Introduction

These are functions dealing with error handling and logging. They allow you to define your own error handling rules, as well as modify the way the errors can be logged. This allows you to change and enhance error reporting to suit your needs.

With the logging functions, you can send messages directly to other machines, to an email (or email to pager gateway!), to system logs, etc., so you can selectively log and monitor the most important parts of your applications and websites.

The error reporting functions allow you to customize what level and kind of error feedback is given, ranging from simple notices to customized functions returned during errors.

Installing/Configuring

Table of Contents

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Errors and Logging Configuration Options

Name	Default	Changeable	Changelog
error_reporting	NULL	INI_ALL	
fatal_error_backtraces	"1"	INI_ALL	Available as of PHP 8.5.0
display_errors	"1"	INI_ALL	
display_startup_errors	"1"	INI_ALL	Prior to PHP 8.0.0, the default value was "0".
log_errors	"0"	INI_ALL	
log_errors_max_len	"1024"	INI_ALL	Had no effect as of PHP 8.0.0, removed as of PHP 8.1.0.
ignore_repeated_errors	"0"	INI_ALL	
ignore_repeated_source	"0"	INI_ALL	
report_memleaks	"1"	INI_ALL	Deprecated as of PHP 8.5.0
track_errors	"0"	INI_ALL	Deprecated as of PHP 7.2.0, removed as of PHP 8.0.0.
html_errors	"1"	INI_ALL	
xmlrpc_errors	"0"	INI_SYSTEM	
xmlrpc_error_number	"0"	INI_ALL	
docref_root	""	INI_ALL	
docref_ext	""	INI_ALL	
error_prepend_string	NULL	INI_ALL	
error_append_string	NULL	INI_ALL	
error_log	NULL	INI_ALL	
error_log_mode	0o644	INI_ALL	Available as of PHP 8.2.0
syslog.facility	"LOG_USER"	INI_SYSTEM	Available as of PHP 7.3.0.
syslog.filter	"no-ctrl"	INI_ALL	Available as of PHP 7.3.0.
syslog.ident	"php"	INI_SYSTEM	Available as of PHP 7.3.0.

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`error_reporting` int

> Set the error reporting level. The parameter is either an integer representing a bit field, or named constants. The error_reporting levels and constants are described in Predefined Constants, and in *php.ini*. To set at runtime, use the error_reporting() function. See

also the display_errors directive.

The default value is **E_ALL**.

Prior to PHP 8.0.0, the default value was: **E_ALL & ~E_NOTICE & ~E_STRICT & ~E_DEPRECATED**. This means diagnostics of level **E_NOTICE**, **E_STRICT** and **E_DEPRECATED** were not shown.

> **Note** : **PHP Constants outside of PHP**
>
> Using PHP Constants outside of PHP, like in *httpd.conf*, will have no useful meaning so in such cases the int values are required. And since error levels will be added over time, the maximum value (for **E_ALL**) will likely change. So in place of **E_ALL** consider using a larger value to cover all bit fields from now and well into the future, a numeric value like `2147483647` (includes all errors, not just **E_ALL**).

`fatal_error_backtraces` bool

Controls whether fatal errors should include a backtrace. Fatal errors are **E_ERROR**, **E_CORE_ERROR**, **E_COMPILE_ERROR**, **E_USER_ERROR**, **E_RECOVERABLE_ERROR**, and **E_PARSE**.

`display_errors` string

This determines whether errors should be printed to the screen as part of the output or if they should be hidden from the user.

- `Off` - do not display any errors.
- `On` or `stdout` - display errors to `stdout`. This is the default.
- `stderr` - display errors to `stderr`. Only effective for the `CLI`, `phpdbg` and `CGI` SAPIs.

The bundled `php.ini-development` sets this to `On`; `php.ini-production` sets it to `Off`.

> **Note** :
>
> This is a feature to support your development and should never be used on production systems (e.g. systems connected to the internet).

> **Note** :
>
> Although display_errors may be set at runtime (with ini_set()), it won't have any effect if the script has fatal errors. This is because the desired runtime action does not get executed.

`display_startup_errors` bool

Even when display_errors is on, errors that occur during PHP's startup sequence are not displayed. It's strongly recommended to keep display_startup_errors off, except for debugging.

`log_errors` bool

Tells whether script error messages should be logged to the server's error log or error_log. This option is thus server-specific.

> **Note** :
>
> You're strongly advised to use error logging in place of error displaying on production web sites.

`log_errors_max_len` int

Set the maximum length of log_errors in bytes. In error_log information about the source is added. The default is 1024 and 0 allows to not apply any maximum length at all. This length is applied to logged errors, displayed errors and also to *$php_errormsg*, but not to explicitly called functions such as error_log().

When an int is used, the value is measured in bytes. Shorthand notation, as described in this FAQ, may also be used.

`ignore_repeated_errors` bool

Do not log repeated messages. Repeated errors must occur in the same file on the same line unless ignore_repeated_source is set true.

`ignore_repeated_source` bool

Ignore source of message when ignoring repeated messages. When this setting is On you will not log errors with repeated messages from different files or sourcelines.

`report_memleaks` bool

If this parameter is set to On (the default), this parameter will show a report of memory leaks detected by the Zend memory manager. This report will be sent to stderr on Posix platforms. On Windows, it will be sent to the debugger using OutputDebugString() and can be viewed with tools like » DbgView. This parameter only has effect in a debug build and if error_reporting includes **`E_WARNING`** in the allowed list.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.5.0. Relying on this feature is highly discouraged.

`track_errors` bool

If enabled, the last error message will always be present in the variable *$php_errormsg*.

`html_errors` bool

If enabled, error messages will include HTML tags. The format for HTML errors produces clickable messages that direct the user to a page describing the error or function in causing the error. These references are affected by docref_root and docref_ext.

If disabled, error message will be solely plain text.

`xmlrpc_errors` bool

If enabled, turns off normal error reporting and formats errors as XML-RPC error message.

`xmlrpc_error_number` int

Used as the value of the XML-RPC faultCode element.

`docref_root` string

The new error format contains a reference to a page describing the error or function causing the error. In case of manual pages you can download the manual in your language and set this ini directive to the URL of your local copy. If your local copy of the manual can be reached by `"/manual/"` you can simply use **`docref_root=/manual/`**. Additional you have to set docref_ext to match the fileextensions of your copy **`docref_ext=.html`**. It is possible to use external references. For example you can use **`docref_root=http://manual/en/`** or **`docref_root="http://landonize.it/?how=url&theme=classic&filter=Landon&url=http%3A%2F%2Fwww.php.net%2F"`**

Most of the time you want the docref_root value to end with a slash `"/"`. But see the second example above which does not have nor need it.

> **Note** :
>
> This is a feature to support your development since it makes it easy to lookup a function description. However it should never be used on production systems (e.g. systems connected to the internet).

`docref_ext` string

See docref_root.

> **Note** :
>
> The value of docref_ext must begin with a dot `"."`.

`error_prepend_string` string

String to output before an error message. Only used when the error message is displayed on screen. The main purpose is to be able to prepend additional HTML markup to the error message.

`error_append_string` string

String to output after an error message. Only used when the error message is displayed on screen. The main purpose is to be able to append additional HTML markup to the error message.

`error_log` string

Name of the file where script errors should be logged. The file should be writable by the web server's user. If the special value `syslog` is used, the errors are sent to the system logger instead. On Unix, this means syslog(3) and on Windows it means the event log. See also: syslog(). If this directive is not set, errors are sent to the SAPI error logger. For example, it is an error log in Apache or `stderr` in CLI. See also error_log().

`error_log_mode` int

File mode for the file described set in error_log.

`syslog.facility` string

Specifies what type of program is logging the message. Only effective if error_log is set to "syslog".

`syslog.filter` string

Specifies the filter type to filter the logged messages. Allowed characters are passed unmodified; all others are written in their hexadecimal representation prefixed with `\x`.

- `all` – the logged string will be split at newline characters, and all characters are passed unaltered
- `ascii` – the logged string will be split at newline characters, and any non-printable 7-bit ASCII characters will be escaped
- `no-ctrl` – the logged string will be split at newline characters, and any non-printable characters will be escaped
- `raw` – all characters are passed to the system logger unaltered, without splitting at newlines (identical to PHP before 7.3)

This setting will affect logging via error_log set to "syslog" and calls to syslog().

> **Note** :
>
> The `raw` filter type is available as of PHP 7.3.8 and PHP 7.4.0.

This directive is not supported on Windows.

`syslog.ident` string

Specifies the ident string which is prepended to every message. Only effective if error_log is set to "syslog".

Predefined Constants

The constants below are always available as part of the PHP core.

The following constants (either the corresponding numerical value or their symbolic name) are used as a bitmask to specify which errors to report. It is possible to use bitwise operators to combine these values or mask out certain types of errors.

> **Tip**
>
> The name of the constants can be used within *php.ini*, instead of the raw numerical values to which they correspond to. However, only the `|`, `~`, `^`, `!`, `&` operators are understood within *php.ini*.

> **Warning**
>
> It is not possible to use the symbolic names outside of PHP. For example in *httpd.conf* the computed bitmask value must be used instead.

`E_ERROR` (int)

Fatal run-time errors. These indicate errors that can not be recovered from, such as a memory allocation problem. Execution of the script is halted.

Value of the constant: 1

E_WARNING (int)

Run-time warnings (non-fatal errors). Execution of the script is not halted.

Value of the constant: 2

E_PARSE (int)

Compile-time parse errors. Parse errors should only be generated by the parser.

Value of the constant: 4

E_NOTICE (int)

Run-time notices. Indicate that the script encountered something that could indicate an error, but could also happen in the normal course of running a script.

Value of the constant: 8

E_CORE_ERROR (int)

Fatal errors that occur during PHP's initial startup. This is like an **E_ERROR**, except it is generated by the core of PHP.

Value of the constant: 16

E_CORE_WARNING (int)

Warnings (non-fatal errors) that occur during PHP's initial startup. This is like an **E_WARNING**, except it is generated by the core of PHP.

Value of the constant: 32

E_COMPILE_ERROR (int)

Fatal compile-time errors. This is like an **E_ERROR**, except it is generated by the Zend Scripting Engine.

Value of the constant: 64

E_COMPILE_WARNING (int)

Compile-time warnings (non-fatal errors). This is like an **E_WARNING**, except it is generated by the Zend Scripting Engine.

Value of the constant: 128

E_DEPRECATED (int)

Run-time deprecation notices. Enable this to receive warnings about code that will not work in future versions.

Value of the constant: 8192

E_USER_ERROR (int)

User-generated error message. This is like an **E_ERROR**, except it is generated in PHP code by using the PHP function trigger_error().

Value of the constant: 256

> **Warning**
>
> Usage of this constant with trigger_error() is deprecated as of PHP 8.4.0. It is recommended to either `throw` an Exception or call exit() instead.

E_USER_WARNING (int)

User-generated warning message. This is like an **E_WARNING**, except it is generated in PHP code by using the PHP function trigger_error().

Value of the constant: 512

`E_USER_NOTICE` (int)

User-generated notice message. This is like an `E_NOTICE`, except it is generated in PHP code by using the PHP function trigger_error().

Value of the constant: `1024`

`E_USER_DEPRECATED` (int)

User-generated deprecation message. This is like an `E_DEPRECATED`, except it is generated in PHP code by using the PHP function trigger_error().

Value of the constant: `16384`

`E_STRICT` (int)

Run-time suggestions emitted by PHP about the executed code to ensure forward compatibility.

Value of the constant: `2048`

> **Warning**
>
> This error level is unused, and has been deprecated as of PHP 8.4.0.

`E_RECOVERABLE_ERROR` (int)

Legacy engine "exceptions" which correspond to catchable fatal error. Similar to Error but must be caught via a user defined error handler (see set_error_handler()). If not handled, this behaves like `E_ERROR`.

Value of the constant: `4096`

> **Note** :
>
> This error level is effectively unused, the only case where this can happen is when interpreting an object as a bool fails. This can only happen for internal objects.
>
> The most common example, prior to PHP 8.4.0, is using a GMP instance in a conditional.

`E_ALL` (int)

Bit-mask that contains every single error, warning, and notice.

Value of the constant: `30719`

> **Warning**
>
> Prior to PHP 8.4, the constant value was: `32767`

Examples

Below we can see an example of using the error handling capabilities in PHP. We define an error handling function which logs the information into a file (using an XML format), and e-mails the developer if a critical error in the logic happens.

Example #1 Using error handling in a script

```php
<?php
// we will do our own error handling
error_reporting(0);

// user defined error handling function
function userErrorHandler($errno, $errmsg, $filename, $linenum, $vars)
{
    // timestamp for the error entry
    $dt = date("Y-m-d H:i:s (T)");

    // define an assoc array of error string
    // in reality the only entries we should
```

```
    // consider are E_WARNING, E_NOTICE, E_USER_ERROR,
    // E_USER_WARNING and E_USER_NOTICE
    $errortype = array (
                E_ERROR              => 'Error',
                E_WARNING            => 'Warning',
                E_PARSE              => 'Parsing Error',
                E_NOTICE             => 'Notice',
                E_CORE_ERROR         => 'Core Error',
                E_CORE_WARNING       => 'Core Warning',
                E_COMPILE_ERROR      => 'Compile Error',
                E_COMPILE_WARNING    => 'Compile Warning',
                E_USER_ERROR         => 'User Error',
                E_USER_WARNING       => 'User Warning',
                E_USER_NOTICE        => 'User Notice',
                E_STRICT             => 'Runtime Notice',
                E_RECOVERABLE_ERROR  => 'Catchable Fatal Error'
                );
    // set of errors for which a var trace will be saved
    $user_errors = array(E_USER_ERROR, E_USER_WARNING, E_USER_NOTICE);

    $err = "<errorentry>\n";
    $err .= "\t<datetime>" . $dt . "</datetime>\n";
    $err .= "\t<errornum>" . $errno . "</errornum>\n";
    $err .= "\t<errortype>" . $errortype[$errno] . "</errortype>\n";
    $err .= "\t<errormsg>" . $errmsg . "</errormsg>\n";
    $err .= "\t<scriptname>" . $filename . "</scriptname>\n";
    $err .= "\t<scriptlinenum>" . $linenum . "</scriptlinenum>\n";

    if (in_array($errno, $user_errors)) {
        $err .= "\t<vartrace>" . wddx_serialize_value($vars, "Variables") . "</vartrace>\n";
    }
    $err .= "</errorentry>\n\n";

    // for testing
    // echo $err;

    // save to the error log, and e-mail me if there is a critical user error
    error_log($err, 3, "/usr/local/php4/error.log");
    if ($errno == E_USER_ERROR) {
        mail("phpdev@example.com", "Critical User Error", $err);
    }
}


function distance($vect1, $vect2)
{
    if (!is_array($vect1) || !is_array($vect2)) {
        trigger_error("Incorrect parameters, arrays expected", E_USER_ERROR);
        return NULL;
    }

    if (count($vect1) != count($vect2)) {
        trigger_error("Vectors need to be of the same size", E_USER_ERROR);
        return NULL;
    }

    for ($i=0; $i<count($vect1); $i++) {
        $c1 = $vect1[$i]; $c2 = $vect2[$i];
        $d = 0.0;
        if (!is_numeric($c1)) {
            trigger_error("Coordinate $i in vector 1 is not a number, using zero",
                          E_USER_WARNING);
            $c1 = 0.0;
        }
        if (!is_numeric($c2)) {
            trigger_error("Coordinate $i in vector 2 is not a number, using zero",
                          E_USER_WARNING);
            $c2 = 0.0;
        }
        $d += $c2*$c2 - $c1*$c1;
    }
    return sqrt($d);
```

```
}

$old_error_handler = set_error_handler("userErrorHandler");

// undefined constant, generates a warning
$t = I_AM_NOT_DEFINED;

// define some "vectors"
$a = array(2, 3, "foo");
$b = array(5.5, 4.3, -1.6);
$c = array(1, -3);

// generate a user error
$t1 = distance($c, $b) . "\n";

// generate another user error
$t2 = distance($b, "i am not an array") . "\n";

// generate a warning
$t3 = distance($a, $b) . "\n";

?>
```

Error Handling Functions

See Also

See also syslog().

Table of Contents

debug_backtrace

debug_backtrace

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

debug_backtrace — Generates a backtrace

Description

function **debug_backtrace**(int `$options` = `DEBUG_BACKTRACE_PROVIDE_OBJECT`, int `$limit` = 0): array

debug_backtrace() generates a PHP backtrace.

Parameters

`options`

This parameter is a bitmask for the following options:

debug_backtrace() options

DEBUG_BACKTRACE_PROVIDE_OBJECT	Whether or not to populate the "object" index.
DEBUG_BACKTRACE_IGNORE_ARGS	Whether or not to omit the "args" index, and thus all the function/method arguments, to save memory.

> **Note** :
>
> There are four possible combinations:
>
> **debug_backtrace() options**
>
> | | |
> |---|---|
> | `debug_backtrace()` | Populates both indexes |
> | `debug_backtrace(DEBUG_BACKTRACE_PROVIDE_OBJECT)` | |
> | `debug_backtrace(~DEBUG_BACKTRACE_PROVIDE_OBJECT)` | Omits index `"object"` and populates index `"args"`. |
> | `debug_backtrace(0)` | |

`debug_backtrace(DEBUG_BACKTRACE_IGNORE_ARGS)`	Omits *both* index `"object"` *and* index `"args"`.
`debug_backtrace(~DEBUG_BACKTRACE_PROVIDE_OBJECT | DEBUG_BACKTRACE_IGNORE_ARGS)`	
`debug_backtrace(DEBUG_BACKTRACE_PROVIDE_OBJECT|DEBUG_BACKTRACE_IGNORE_ARGS)`	Populate index `"object"` *and* omit index `"args"`.

`limit`

This parameter can be used to limit the number of stack frames returned. By default (`limit=0`) it returns all stack frames.

Return Values

Returns an array of associative arrays. The possible returned elements are as follows:

Possible returned elements from debug_backtrace()

Name	Type	Description
function	string	The current function name. See also __FUNCTION__.
line	int	The current line number. See also __LINE__.
file	string	The current file name. See also __FILE__.
class	string	The current class name. See also __CLASS__
object	object	The current object if **DEBUG_BACKTRACE_PROVIDE_OBJECT** is given.
type	string	The current call type. If a method call, "->" is returned. If a static method call, "::" is returned. If a function call, nothing is returned.
args	array	If inside a function, this lists the functions arguments. If inside an included file, this lists the included file name(s). Unless **DEBUG_BACKTRACE_IGNORE_ARGS** is given.

Examples

Example #1 debug_backtrace() example

```php
<?php
// filename: /tmp/a.php

function a_test($str)
{
    echo "\nHi: $str";
    var_dump(debug_backtrace());
}

a_test('friend');
?>

<?php
// filename: /tmp/b.php
include_once '/tmp/a.php';
?>
```

Results similar to the following when executing */tmp/b.php*:

```
Hi: friend
array(2) {
[0]=>
array(4) {
    ["file"] => string(10) "/tmp/a.php"
    ["line"] => int(10)
    ["function"] => string(6) "a_test"
    ["args"]=>
    array(1) {
      [0] => &string(6) "friend"
    }
}
[1]=>
```

```
array(4) {
    ["file"] => string(10) "/tmp/b.php"
    ["line"] => int(2)
    ["args"] =>
    array(1) {
      [0] => string(10) "/tmp/a.php"
    }
    ["function"] => string(12) "include_once"
  }
}
```

See Also

- trigger_error() - Generates a user-level error/warning/notice message
- debug_print_backtrace() - Prints a backtrace

debug_print_backtrace

debug_print_backtrace

(PHP 5, PHP 7, PHP 8)

debug_print_backtrace — Prints a backtrace

Description

function **debug_print_backtrace**(int `$options` = 0, int `$limit` = 0): void

debug_print_backtrace() prints a PHP backtrace. It prints the function calls, included/required files and eval()ed stuff.

Parameters

`options`

This parameter is a bitmask for the following options:

debug_print_backtrace() options

DEBUG_BACKTRACE_IGNORE_ARGS	Whether or not to omit the "args" index, and thus all the function/method arguments, to save memory.
DEBUG_BACKTRACE_PROVIDE_OBJECT	Accepted for compatibility with debug_backtrace(), but has no effect: the generated backtrace is rendered as a string and the "object" entry is never printed.

`limit`

This parameter can be used to limit the number of stack frames printed. By default (`limit=0`) it prints all stack frames.

Return Values

No value is returned.

Examples

Example #1 debug_print_backtrace() example

```php
<?php
// include.php file

function a() {
    b();
}

function b() {
    c();
}

function c(){
    debug_print_backtrace();
```

```php
}

a();

?>
```

```php
<?php
// test.php file
// this is the file you should run

include 'include.php';
?>
```

The above example will output something similar to:

```
#0  c() called at [/tmp/include.php:10]
#1  b() called at [/tmp/include.php:6]
#2  a() called at [/tmp/include.php:17]
#3  include(/tmp/include.php) called at [/tmp/test.php:3]
```

See Also

- debug_backtrace() - Generates a backtrace

error_clear_last

error_clear_last

(PHP 7, PHP 8)

error_clear_last — Clear the most recent error

Description

function **error_clear_last**(): void

Parameters

This function has no parameters.

Return Values

Clears the most recent errors, making it unable to be retrieved with error_get_last().

Examples

Example #1 An error_clear_last() example

```php
<?php
var_dump(error_get_last());
error_clear_last();
var_dump(error_get_last());

@$a = $b;

var_dump(error_get_last());
error_clear_last();
var_dump(error_get_last());
?>
```

The above example will output something similar to:

```
NULL
NULL
array(4) {
  ["type"]=>
  int(8)
  ["message"]=>
  string(21) "Undefined variable: b"
  ["file"]=>
  string(9) "%s"
  ["line"]=>
  int(6)
}
NULL
```

See Also

- Error constants

error_get_last

error_get_last

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

error_get_last — Get the last occurred error

Description

function **error_get_last**(): ?array

Gets information about the last error that occurred.

Parameters

This function has no parameters.

Return Values

Returns an associative array describing the last error with keys "type", "message", "file" and "line". If the error has been caused by a PHP internal function then the "message" begins with its name. Returns `null` if there hasn't been an error yet.

Examples

Example #1 An error_get_last() example

```php
<?php
echo $a;
print_r(error_get_last());
?>
```

The above example will output something similar to:

```
Array
(
    [type] => 8
    [message] => Undefined variable: a
    [file] => C:\WWW\index.php
    [line] => 2
)
```

See Also

- Error constants
- Variable *$php_errormsg*
- error_clear_last() - Clear the most recent error
- Directive `display_errors`
- Directive `html_errors`
- Directive `xmlrpc_errors`

error_log

error_log

(PHP 4, PHP 5, PHP 7, PHP 8)

error_log — Send an error message to the defined error handling routines

Description

```
function error_log(
    string $message,
    int $message_type = 0,
    ?string $destination = null,
    ?string $additional_headers = null
): bool
```

Sends an error message to the web server's error log or to a file.

Parameters

`message`

The error message that should be logged.

`message_type`

Says where the error should go. The possible message types are as follows:

error_log() log types

0	`message` is sent to PHP's system logger, using the Operating System's system logging mechanism or a file, depending on what the error_log configuration directive is set to. This is the default option.
1	`message` is sent by email to the address in the `destination` parameter. This is the only message type where the fourth parameter, `additional_headers` is used.
2	No longer an option.
3	`message` is appended to the file `destination`. A newline is not automatically added to the end of the `message` string.
4	`message` is sent directly to the SAPI logging handler.

`destination`

The destination. Its meaning depends on the `message_type` parameter as described above.

`additional_headers`

The extra headers. It's used when the `message_type` parameter is set to `1`. This message type uses the same internal function as mail() does.

Return Values

Returns `true` on success or `false` on failure. If `message_type` is zero, this function always returns `true`, regardless of whether the error could be logged or not.

Changelog

Version	Description
8.0.0	`destination` and `additional_headers` are now nullable.

Examples

Example #1 error_log() examples

```php
<?php
// Send notification through the server log if we can not
// connect to the database.
if (!Ora_Logon($username, $password)) {
    error_log("Oracle database not available!", 0);
}

// Notify administrator by email if we run out of FOO
if (!($foo = allocate_new_foo())) {
    error_log("Big trouble, we're all out of FOOs!", 1,
              "operator@example.com");
}

// another way to call error_log():
error_log("You messed up!", 3, "/var/tmp/my-errors.log");
?>
```

Notes

> **Warning**
>
> **error_log()** is not binary safe. `message` will be truncated by null character.

> **Tip**
>
> `message` should not contain null character. Note that `message` may be sent to file, mail, syslog, etc. Use appropriate conversion/escape function, base64_encode(), rawurlencode() or addslashes() before calling **error_log()**.

error_reporting

error_reporting

(PHP 4, PHP 5, PHP 7, PHP 8)

error_reporting — Sets which PHP errors are reported

Description

function **error_reporting**(?int `$error_level = null`): int

The **error_reporting()** function sets the error_reporting directive at runtime. PHP has many levels of errors, using this function sets that level for the duration (runtime) of your script. If the optional `error_level` is not set, **error_reporting()** will just return the current error reporting level.

Parameters

`error_level`

The new error_reporting level. It takes on either a bitmask, or named constants. Using named constants is strongly encouraged to ensure compatibility for future versions. As error levels are added, the range of integers increases, so older integer-based error levels will not always behave as expected.

The available error level constants and the actual meanings of these error levels are described in the predefined constants.

Return Values

Returns the error_reporting level, *before* it is changed to `error_level`.

> **Note** :
>
> The error control `@`-operator changes the `error_level` during error handling.

Changelog

Version	Description
8.0.0	`error_level` is nullable now.

Examples

Example #1 error_reporting() examples

```php
<?php

// Turn off all error reporting
error_reporting(0);

// Report simple running errors
error_reporting(E_ERROR | E_WARNING | E_PARSE);

// Reporting E_NOTICE can be good too (to report uninitialized
// variables or catch variable name misspellings ...)
error_reporting(E_ERROR | E_WARNING | E_PARSE | E_NOTICE);

// Report all errors except E_NOTICE
error_reporting(E_ALL & ~E_NOTICE);

// Report all PHP errors
error_reporting(E_ALL);

// Report all PHP errors
error_reporting(-1);

// Same as error_reporting(E_ALL);
ini_set('error_reporting', E_ALL);

?>
```

Notes

> **Tip**
>
> Passing in the value `-1` will show every possible error, even when new levels and constants are added in future PHP versions. The behavior is equivalent to passing `E_ALL` constant.

See Also

- The display_errors directive
- The html_errors directive
- The xmlrpc_errors directive
- The error control operator
- ini_set() - Sets the value of a configuration option

get_error_handler

get_error_handler

(PHP 8 >= 8.5.0)

get_error_handler — Gets the user-defined error handler function

Description

function **get_error_handler**(): ?callable

Returns the current error handler function, if any.

Parameters

This function has no parameters.

Return Values

Returns the currently defined error handler (if any). If the built-in error handler is used `null` is returned.

The returned handler is the exact callable value that was passed to set_error_handler() to define it.

Examples

Example #1 get_error_handler() example

```php
<?php

$handler = function (int $errno, string $errstr, ?string $errfile, ?int $errline) {
     echo "Error: " . $errstr . "\n";
};

var_dump(get_error_handler()); // NULL

set_error_handler($handler);

var_dump(get_error_handler() === $handler); // bool(true)

?>
```

Notes

> **Tip**
>
> Prior to PHP 8.5.0, this functionality can be provided by the following polyfill:
>
> ```php
> <?php
> if (!function_exists('get_error_handler')) {
>     function noop_error_handler() {
>     }
>     function get_error_handler(): ?callable {
>         $handler = set_error_handler('noop_error_handler');
>         restore_error_handler();
>         return $handler;
>     }
> }
> ?>
> ```

See Also

- error_reporting() - Sets which PHP errors are reported
- set_error_handler() - Sets a user-defined error handler function
- restore_error_handler() - Restores the previous error handler function
- trigger_error() - Generates a user-level error/warning/notice message
- error level constants

get_exception_handler

get_exception_handler

(PHP 8 >= 8.5.0)

get_exception_handler — Gets the user-defined exception handler function

Description

function **get_exception_handler**(): ?callable

Returns the current exception handler function, if any.

Parameters

This function has no parameters.

Return Values

Returns the currently defined exception handler. If no handler is defined, `null` is returned.

The returned handler is the exact callable value that was passed to set_exception_handler() to define it.

Examples

Example #1 get_exception_handler() example

```php
<?php

$handler = function (Throwable $ex) {
     echo "Exception: " . $ex::class . ": " . $ex->getMessage() . "\n";
};

var_dump(get_exception_handler()); // NULL

set_exception_handler($handler);

var_dump(get_exception_handler() === $handler); // bool(true)

?>
```

Notes

> **Tip**
>
> Prior to PHP 8.5.0, this functionality can be provided by the following polyfill:
>
> ```php
> <?php
> if (!function_exists('get_exception_handler')) {
>     function noop_exception_handler() {
>     }
>     function get_exception_handler(): ?callable {
>         $handler = set_exception_handler('noop_exception_handler');
>         restore_exception_handler();
>         return $handler;
>     }
> }
> ?>
> ```

See Also

- set_exception_handler() - Sets a user-defined exception handler function
- restore_exception_handler() - Restores the previously defined exception handler function
- restore_error_handler() - Restores the previous error handler function
- error_reporting() - Sets which PHP errors are reported
- Exceptions

restore_error_handler

restore_error_handler

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

restore_error_handler — Restores the previous error handler function

Description

function **restore_error_handler**(): true

Pops the error handler that was active before the most recent call to set_error_handler() off the internal stack of error handlers and makes it active again, together with the `error_levels` mask it was registered with. The restored handler could be the built-in or a user defined function.

Calling this function more often than set_error_handler() leaves the built-in error handler active; no error is raised.

Parameters

This function has no parameters.

Return Values

Always returns `true`.

Examples

Example #1 restore_error_handler() example

Decide if unserialize() caused an error, then restore the original error handler.

```php
<?php
function unserialize_handler($errno, $errstr)
{
    echo "Invalid serialized value.\n";
}

$serialized = 'foo';
set_error_handler('unserialize_handler');
$original = unserialize($serialized);
restore_error_handler();
?>
```

The above example will output:

```
Invalid serialized value.
```

See Also

- error_reporting() - Sets which PHP errors are reported
- set_error_handler() - Sets a user-defined error handler function
- get_error_handler() - Gets the user-defined error handler function
- restore_exception_handler() - Restores the previously defined exception handler function
- trigger_error() - Generates a user-level error/warning/notice message

restore_exception_handler

restore_exception_handler

(PHP 5, PHP 7, PHP 8)

restore_exception_handler — Restores the previously defined exception handler function

Description

function **restore_exception_handler**(): true

Used after changing the exception handler function using set_exception_handler(), to revert to the previous exception handler (which could be the built-in or a user defined function).

> **Note** :
>
> While an exception handler is running, no exception handler is active: as of PHP 8.3.0 the engine unsets it before invoking it, so that an exception thrown by the handler is not passed back to it.
>
> As of PHP 8.3.5 the engine also pushes the handler it is about to invoke onto the handler stack. A **restore_exception_handler()** call made from within an exception handler therefore pops that entry and re-installs the handler that is currently running; a second call is needed to install the handler that was active before it.
>
> The running handler is re-installed automatically when it returns, but only if no exception handler is active at that point: a handler that calls set_exception_handler() or **restore_exception_handler()** is left in charge of the handler stack itself.

Parameters

This function has no parameters.

Return Values

Always returns `true`.

Changelog

Version	Description
8.3.5	The exception handler being invoked is now pushed onto the handler stack, so **restore_exception_handler()** called from within an exception handler re-installs the handler that is currently running; restoring the handler that was active before it now requires two calls.
8.3.0	The active exception handler is now unset while it runs.

Examples

Example #1 restore_exception_handler() example

```php
<?php
    function exception_handler_1(Exception $e)
    {
        echo '[' . __FUNCTION__ . '] ' . $e->getMessage();
    }

    function exception_handler_2(Exception $e)
    {
        echo '[' . __FUNCTION__ . '] ' . $e->getMessage();
    }

    set_exception_handler('exception_handler_1');
    set_exception_handler('exception_handler_2');

    restore_exception_handler();

    throw new Exception('This triggers the first exception handler...');
?>
```

The above example will output:

```
[exception_handler_1] This triggers the first exception handler...
```

See Also

- set_exception_handler() - Sets a user-defined exception handler function
- get_exception_handler() - Gets the user-defined exception handler function
- set_error_handler() - Sets a user-defined error handler function
- restore_error_handler() - Restores the previous error handler function
- error_reporting() - Sets which PHP errors are reported

set_error_handler

set_error_handler

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

set_error_handler — Sets a user-defined error handler function

Description

function **set_error_handler**(?callable `$callback`, int `$error_levels` = `E_ALL`): ?callable

Sets a user function (`callback`) to handle errors in a script. The handler that was active until then is saved on an internal stack of error handlers, from which restore_error_handler() pops it back.

This function can be used to define custom error handlers during runtime, for example in applications which need to do file/data cleanup when a critical error happens, or when triggering an error in response to certain conditions (using trigger_error()).

It is important to remember that the standard PHP error handler is completely bypassed for the error types specified by `error_levels` unless the callback function returns **`false`**. error_reporting() settings will have no effect and the error handler will be called regardless - however, it's still possible to read the current value of error_reporting and act appropriately.

Also note that it is the handler's responsibility to stop the script's execution if necessary by calling exit(). If the error-handler function returns, script execution will continue with the next statement after the one that caused an error.

The following error types cannot be handled with a user defined function: **`E_ERROR`**, **`E_PARSE`**, **`E_CORE_ERROR`**, **`E_CORE_WARNING`**, **`E_COMPILE_ERROR`**, **`E_COMPILE_WARNING`** independent of where they were raised, and most of **`E_STRICT`** raised in the file where **set_error_handler()** is called.

If errors occur before the script is executed (e.g. on file uploads) the custom error handler cannot be called since it is not registered at that time.

Parameters

`callback`

If **`null`** is passed, no user-defined handler is left active and errors fall back to the built-in error handler. The handler that was active until then is still saved on the stack and can be brought back with restore_error_handler(). Otherwise, the handler is a callback with the following signature:

```
function handler(
    int $errno,
    string $errstr,
    string $errfile = ?,
    int $errline = ?,
    array $errcontext = ?
): bool
```

`errno`

The first parameter, `errno`, will be passed the level of the error raised, as an integer.

`errstr`

The second parameter, `errstr`, will be passed the error message, as a string.

`errfile`

If the callback accepts a third parameter, `errfile`, it will be passed the filename that the error was raised in, as a string.

`errline`

If the callback accepts a fourth parameter, `errline`, it will be passed the line number where the error was raised, as an integer.

`errcontext`

If the callback accepts a fifth parameter, `errcontext`, it will be passed an array that points to the active symbol table at the point the error occurred. In other words, `errcontext` will contain an array of every variable that existed in the scope the error was triggered in. User error handlers must not modify the error context.

> **Warning**
>
> This parameter has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. If the function defines this parameter without a default, an error of "too few arguments" will be raised when it is called.

If the function returns **`false`** then the normal error handler continues.

`error_levels`

Can be used to mask the triggering of the `callback` function just like the error_reporting ini setting controls which errors are shown. Without this mask set the `callback` will be called for every error regardless to the setting of the error_reporting setting.

Return Values

Returns the previously defined error handler (if any) as a callable. If the built-in error handler is used **`null`** is returned.

Changelog

Version	Description
8.0.0	`errcontext` was removed, and will no longer be passed to user callbacks.
7.2.0	`errcontext` became deprecated. Usage of this parameter now emits an `E_DEPRECATED` notice.

Examples

Example #1 Error handling with set_error_handler() and trigger_error()

The example below shows the handling of internal exceptions by triggering errors and handling them with a user defined function:

```php
<?php
// error handler function
function myErrorHandler($errno, $errstr, $errfile, $errline)
{
    if (!(error_reporting() & $errno)) {
        // This error code is not included in error_reporting, so let it fall
        // through to the standard PHP error handler
        return false;
    }

    // $errstr may need to be escaped:
    $errstr = htmlspecialchars($errstr);

    switch ($errno) {
    case E_USER_ERROR:
        echo "<b>My ERROR</b> [$errno] $errstr<br />\n";
        echo "  Fatal error on line $errline in file $errfile";
        echo ", PHP " . PHP_VERSION . " (" . PHP_OS . ")<br />\n";
        echo "Aborting...<br />\n";
        exit(1);

    case E_USER_WARNING:
        echo "<b>My WARNING</b> [$errno] $errstr<br />\n";
        break;

    case E_USER_NOTICE:
        echo "<b>My NOTICE</b> [$errno] $errstr<br />\n";
        break;

    default:
        echo "Unknown error type: [$errno] $errstr<br />\n";
        break;
    }

    /* Don't execute PHP internal error handler */
    return true;
}

// function to test the error handling
function scale_by_log($vect, $scale)
{
    if (!is_numeric($scale) || $scale <= 0) {
        trigger_error("log(x) for x <= 0 is undefined, you used: scale = $scale", E_USER_ERROR);
    }

    if (!is_array($vect)) {
        trigger_error("Incorrect input vector, array of values expected", E_USER_WARNING);
        return null;
    }

    $temp = array();
    foreach($vect as $pos => $value) {
        if (!is_numeric($value)) {
            trigger_error("Value at position $pos is not a number, using 0 (zero)", E_USER_NOTICE);
            $value = 0;
        }
        $temp[$pos] = log($scale) * $value;
    }

    return $temp;
```

```php
}

// set to the user defined error handler
$old_error_handler = set_error_handler("myErrorHandler");

// trigger some errors, first define a mixed array with a non-numeric item
echo "vector a\n";
$a = array(2, 3, "foo", 5.5, 43.3, 21.11);
print_r($a);

// now generate second array
echo "----\nvector b - a notice (b = log(PI) * a)\n";
/* Value at position $pos is not a number, using 0 (zero) */
$b = scale_by_log($a, M_PI);
print_r($b);

// this is trouble, we pass a string instead of an array
echo "----\nvector c - a warning\n";
/* Incorrect input vector, array of values expected */
$c = scale_by_log("not array", 2.3);
var_dump($c); // NULL

// this is a critical error, log of zero or negative number is undefined
echo "----\nvector d - fatal error\n";
/* log(x) for x <= 0 is undefined, you used: scale = $scale" */
$d = scale_by_log($a, -2.5);
var_dump($d); // Never reached
?>
```

The above example will output something similar to:

```
vector a
Array
(
    [0] => 2
    [1] => 3
    [2] => foo
    [3] => 5.5
    [4] => 43.3
    [5] => 21.11
)
----
vector b - a notice (b = log(PI) * a)
<b>My NOTICE</b> [1024] Value at position 2 is not a number, using 0 (zero)<br />
Array
(
    [0] => 2.2894597716988
    [1] => 3.4341896575482
    [2] => 0
    [3] => 6.2960143721717
    [4] => 49.566804057279
    [5] => 24.165247890281
)
----
vector c - a warning
<b>My WARNING</b> [512] Incorrect input vector, array of values expected<br />
NULL
----
vector d - fatal error
<b>My ERROR</b> [256] log(x) for x <= 0 is undefined, you used: scale = -2.5<br />
  Fatal error on line 35 in file trigger_error.php, PHP 5.2.1 (FreeBSD)<br />
Aborting...<br />
```

Example #2 Calling the previous handler from the new one

The return value of **set_error_handler()** is the handler that was active until then. Calling it from the new handler keeps the existing behaviour while adding to it. This only works if that handler is a user defined function; `null` is returned when the built-in one was active.

```php
<?php
function logging_handler(int $errno, string $errstr): bool
{
    echo "logged: $errstr\n";
    return true;
}

set_error_handler('logging_handler');

$previous = set_error_handler(function (int $errno, string $errstr) use (&$previous): bool {
    echo "counted: $errstr\n";
    return $previous !== null ? $previous($errno, $errstr) : false;
});

trigger_error("something happened", E_USER_WARNING);

restore_error_handler();
trigger_error("only logged now", E_USER_WARNING);

restore_error_handler();
?>
```

The above example will output:

```
counted: something happened
logged: something happened
logged: only logged now
```

Notes

> **Caution**
>
> Do not revert to the previous handler by passing the return value of **set_error_handler()** back to it. Doing so pushes a further entry onto the stack instead of removing one, so the stack grows without bound in loops and long-running processes, and `error_levels` is reset to `E_ALL`, silently widening the range of errors the restored handler receives. restore_error_handler() is the intended way to revert to the previous handler.

See Also

- ErrorException
- error_reporting() - Sets which PHP errors are reported
- restore_error_handler() - Restores the previous error handler function
- get_error_handler() - Gets the user-defined error handler function
- trigger_error() - Generates a user-level error/warning/notice message
- error level constants

set_exception_handler

set_exception_handler

(PHP 5, PHP 7, PHP 8)

set_exception_handler — Sets a user-defined exception handler function

Description

function **set_exception_handler**(?callable `$callback`): ?callable

Sets the default exception handler if an exception is not caught within a try/catch block. Execution will stop after the `callback` is called.

> **Note** :
>
> While an exception handler is running, no exception handler is active: as of PHP 8.3.0 the engine unsets it before invoking it, so that an exception thrown by the handler is not passed back to it. Called from within a handler, **set_exception_handler()** therefore reports no previously defined handler.
>
> As of PHP 8.3.5 the engine also pushes the handler it is about to invoke onto the handler stack, so restore_exception_handler() called from within a handler re-installs the handler that is currently running rather than the one that was active before it.
>
> The running handler is re-installed automatically when it returns, but only if no exception handler is active at that point. As soon as the handler calls **set_exception_handler()** or restore_exception_handler(), this automatic restoration is skipped and the active handler is whatever the handler itself left in place.
>
> Modifying the exception handler from within itself is therefore discouraged.

Parameters

`callback`

The function to be called when an uncaught exception occurs. This handler function needs to accept one parameter, which will be the Throwable object that was thrown. Both Error and Exception implement the Throwable interface. This is the handler signature:

function handler(Throwable $ex): void

`null` may be passed instead, to reset this handler to its default state.

Return Values

Returns the previously defined exception handler, or `null` on error. If no previous handler was defined, `null` is also returned.

Changelog

Version	Description
8.3.5	The exception handler being invoked is now pushed onto the handler stack and re-installed once it returns, unless the handler modified the stack itself.
8.3.0	The active exception handler is now unset while it runs, so **set_exception_handler()** called from within a handler reports no previously defined handler.

Examples

Example #1 set_exception_handler() example

```php
<?php
function exception_handler(Throwable $exception) {
  echo "Uncaught exception: " , $exception->getMessage(), "\n";
}

set_exception_handler('exception_handler');

throw new Exception('Uncaught Exception');
echo "Not Executed\n";
?>
```

See Also

- get_exception_handler() - Gets the user-defined exception handler function
- restore_exception_handler() - Restores the previously defined exception handler function
- restore_error_handler() - Restores the previous error handler function
- error_reporting() - Sets which PHP errors are reported
- Exceptions

trigger_error

trigger_error

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

trigger_error — Generates a user-level error/warning/notice message

Description

function **trigger_error**(string `$message`, int `$error_level` = `E_USER_NOTICE`): true

Used to trigger a user error condition, it can be used in conjunction with the built-in error handler, or with a user defined function that has been set as the new error handler (set_error_handler()).

This function is useful when you need to generate a particular response to an exception at runtime.

Parameters

`message`

The designated error message for this error. It's limited to 1024 bytes in length. Any additional characters beyond 1024 bytes will be truncated.

`error_level`

The designated error type for this error. It only works with the `E_USER_*` family of constants, and will default to `E_USER_NOTICE`.

> **Warning**
>
> Passing `E_USER_ERROR` as the `error_level` is now deprecated. Throw an Exception or call exit() instead.

Return Values

Always returns `true`.

Errors/Exceptions

This function throws a ValueError if `error_level` is not one of `E_USER_ERROR`, `E_USER_WARNING`, `E_USER_NOTICE`, `E_USER_DEPRECATED`.

Changelog

Version	Description
8.4.0	Passing `E_USER_ERROR` as the `error_level` is now deprecated. Throw an Exception or call exit() instead.
8.4.0	The function now has a return type of true instead of bool.
8.0.0	The function now throws a ValueError if an invalid `error_level` is specified. Previously, it returned `false`.

Examples

Example #1 **trigger_error()** example

See set_error_handler() for a more extensive example.

```php
<?php
$password = $_POST['password'] ?? '';
if ($password === '') {
  trigger_error("Using an empty password is unsafe", E_USER_WARNING);
}
$hash = password_hash($password, PASSWORD_DEFAULT);
?>
```

Notes

> **Warning**
>
> HTML entities in `message` are not escaped. Use htmlentities() on the message if the error is to be displayed in a browser.

See Also

- error_reporting() - Sets which PHP errors are reported
- set_error_handler() - Sets a user-defined error handler function

- restore_error_handler() - Restores the previous error handler function
- The error level constants
- The Deprecated attribute

user_error

user_error

(PHP 4, PHP 5, PHP 7, PHP 8)

user_error — Alias of trigger_error()

Description

This function is an alias of: trigger_error().

Output Buffering Control

Introduction

The Output Control functions allow you to control when output is sent from the script. This can be useful in several different situations, especially if you need to send headers to the browser after your script has begun outputting data. The Output Control functions do not affect headers sent using header() or setcookie(), only functions such as echo and data between blocks of PHP code.

Installing/Configuring

Table of Contents

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Output Control configuration options

Name	Default	Changeable	Changelog
output_buffering	`"0"`	`INI_PERDIR`	
output_handler	`null`	`INI_PERDIR`	
implicit_flush	`"0"`	`INI_ALL`	
url_rewriter.tags	`"form="`	`INI_ALL`	As of PHP 7.1.0, this INI setting only affects output_add_rewrite_var(). Prior to PHP 7.1.0, this INI setting enabled transparent session id support (see session.trans_sid_tags).
url_rewriter.hosts	`$_SERVER['HTTP_HOST']` is used as default.	`INI_ALL`	Available as of PHP 7.1.0

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`output_buffering` bool/int

Output buffering for all files can be enabled by setting this directive to `"On"`. To limit the size of the buffer, a number/quantity corresponding to the maximum numbers of bytes allowed can be used instead of `"On"` for the value of this directive. For example `output_buffering=4096`. This directive is always Off in PHP-CLI.

`output_handler` string

The output of scripts can be redirected to a function. For example setting `output_handler` to mb_output_handler(), character encoding will be transparently converted to the specified encoding. Setting any output handler automatically turns on output buffering.

> **Note** :
>
> mb_output_handler() and ob_iconv_handler() cannot be used together and ob_gzhandler() and zlib.output_compression cannot be used with any of the following: mb_output_handler(), ob_gzhandler(), zlib.output_compression, the 'URL-Rewriter' handler (see session.use_trans_sid and output_add_rewrite_var()).

> **Note** :
>
> Only built-in functions can be used with this directive. For user defined functions, use ob_start().

`implicit_flush` bool

`false` by default. Changing this to `true` tells PHP to tell the output layer to flush itself automatically after every output block. This is equivalent to calling the PHP function flush() after each and every call to any function producing output (such as print or echo) and each and every HTML block.

When using PHP within an web environment, turning this option on has serious performance implications and is generally recommended for debugging purposes only. This value defaults to `true` when operating under the `CLI SAPI`.

See also ob_implicit_flush().

`url_rewriter.tags` string

`url_rewriter.tags` specifies HTML tags and attributes in which URLs are rewritten by output_add_rewrite_var() values. Defaults to `"form="`.

Adding `"form="` or any `form` attribute will add a hidden `input` element to the `form` with a name and value attribute for each name-value pair passed to output_add_rewrite_var().

> **Caution**
>
> Adding the same tag more than once to `url_rewriter.tags` will only use the first occurrence during the URL rewriting process.

> **Note** :
>
> Prior to PHP 7.1.0, url_rewriter.tags was used to specify session.trans_sid_tags.

`url_rewriter.hosts` string

`url_rewriter.hosts` specifies which hosts are rewritten to include output_add_rewrite_var() values. Defaults to `$_SERVER['HTTP_HOST']`. Multiple hosts can be specified by a comma separated list that does not include spaces. For example `"php.net,wiki.php.net,bugs.php.net"`.

Predefined Constants

The constants below are always available as part of the PHP core.

Status flags passed to output handler

The following flags are passed to the second (`phase`) parameter of the output handler set by ob_start() as part of a bitmask:

`PHP_OUTPUT_HANDLER_START` (int)

Indicates that output buffering has begun.

`PHP_OUTPUT_HANDLER_WRITE` (int)

Indicates that the output buffer is being flushed, and had data to output.

`PHP_OUTPUT_HANDLER_FLUSH` (int)

Indicates that the buffer has been flushed.

`PHP_OUTPUT_HANDLER_CLEAN` (int)

Indicates that the output buffer has been cleaned.

`PHP_OUTPUT_HANDLER_FINAL` (int)

Indicates that this is the final output buffering operation.

`PHP_OUTPUT_HANDLER_CONT` (int)

Indicates that the buffer has been flushed, but output buffering will continue.

This is an alias for `PHP_OUTPUT_HANDLER_WRITE`.

`PHP_OUTPUT_HANDLER_END` (int)

Indicates that output buffering has ended.

This is an alias for `PHP_OUTPUT_HANDLER_FINAL`.

Output buffer control flags

The following flags can be passed to the third (`flags`) parameter of the output handler set by ob_start() as a bitmask:

PHP_OUTPUT_HANDLER_CLEANABLE (int)

Controls whether an output buffer created by ob_start() can be cleaned by ob_clean(). This flag does not control the behaviour of ob_end_clean() or ob_get_clean().

PHP_OUTPUT_HANDLER_FLUSHABLE (int)

Controls whether an output buffer created by ob_start() can be flushed by ob_flush(). This flag does not control the behaviour of ob_end_flush() or ob_get_flush().

PHP_OUTPUT_HANDLER_REMOVABLE (int)

Controls whether an output buffer created by ob_start() can be removed before the end of the script or when calling ob_end_clean(), ob_end_flush(), ob_get_clean() or ob_get_flush().

PHP_OUTPUT_HANDLER_STDFLAGS (int)

The default set of output buffer flags; currently equivalent to **PHP_OUTPUT_HANDLER_CLEANABLE** | **PHP_OUTPUT_HANDLER_FLUSHABLE** | **PHP_OUTPUT_HANDLER_REMOVABLE**.

Output handler status flags

The following flags are part of the `flags` bitmask returned by ob_get_status():

PHP_OUTPUT_HANDLER_STARTED (int)

Indicates that the output handler was called.

PHP_OUTPUT_HANDLER_DISABLED (int)

Indicates that the output handler is disabled. This flag is set when the output handler returns **false** or fails while processing the buffer. Prior to PHP 8.4.0, this flag could be set when starting an output buffer.

PHP_OUTPUT_HANDLER_PROCESSED (int)

Indicates that the output handler successfully processed the buffer. Available as of PHP 8.4.0.

Output Buffering

Output buffering is the buffering (temporary storage) of output before it is flushed (sent and discarded) to the browser (in a web context) or to the shell (on the command line). While output buffering is active no output is sent from the script, instead the output is stored in an internal buffer.

Buffering Affecting PHP

PHP relies on the underlying software/hardware infrastructure when flushing output. Buffering implemented by consoles on the command line (e.g. line buffered) or web servers and browser in a web context (e.g. fully buffered) do affect when output is displayed to the end-user. Some of these effects can be eliminated by fine-tuning server settings and/or aligning buffer sizes of the various layers.

Output Buffering Control In PHP

PHP provides a fully buffered user-level output buffer with functions to start, manipulate and turn off the buffer (most ob_* functions), and two functions to flush the underlying system buffers (flush() and ob_implicit_flush()). Some of this functionality can be set and/or configured using the appropriate *php.ini* settings as well.

Use Cases

Output buffering is generally useful in situations when the buffered output is modified or inspected, or it is used more than once in a request; or when the controlled flushing of output is desired. Specific use cases include:

- caching the result of compute/time intensive scripts for example by generating static `HTML` pages
- re-using the generated output by displaying it, saving it to a file and/or sending it by email
- flushing the `head` of an `HTML` page separate from the `body` allows browsers to load external resources while the script executes potentially more time consuming processes (e.g. database/file access, external network connection). This is only useful if the `HTTP` status code cannot change after the headers are sent

- extracting information from functions that would otherwise produce output (e.g. phpinfo())
- controlling the output of third-party code by modifying/using parts (e.g. extracting data, replacing words/phrases, adding missing `HTML` tags), or discarding it entirely under certain conditions (e.g. errors)
- polyfilling certain unavailable web server functionality (e.g. compressing or encoding output)

Flushing System Buffers

PHP provides two related ways to flush (send and discard the contents of) system buffers: through calling flush() and through enabling implicit flushing with ob_implicit_flush() or the implicit_flush *php.ini* setting.

Output Flushing Behaviour

With implicit flushing disabled, PHP will flush output only when flush() is called or when the script ends.

With implicit flushing enabled, PHP will attempt to flush after every block of code resulting in output. Output in this context is non-zero length data that is:

- outside of the `<?php ?>` tags
- printed by language constructs and functions whose explicit purpose is to output user provided variables or strings such as echo, print, printf(), var_dump(), var_export(), vprintf()
- printed by functions whose purpose is to collect and output data/information on the running script or PHP such as debug_print_backtrace(), phpcredits(), phpinfo(), ReflectionExtension::info()
- printed by PHP on an uncaught exception or an unhandled error (subject to the settings of display_errors and error_reporting)
- anything written to `php://output`

> **Note** :
>
> Printing empty strings or sending headers is not considered output and will not result in a flush operation.

> **Warning**
>
> If implicit flushing is enabled, control characters (e.g. `"\n"`, `"\r"`, `"\0"`) will trigger flushing as well.

Limitations

This functionality cannot flush user-level output buffers. To use these together, user-level output buffers must be flushed before flushing system buffers in order for PHP to produce any output.

> **Warning**
>
> Calling flush() or implicit flushing being enabled can interfere with output handlers of user-level output buffers that set and send headers in a web context (e.g. ob_gzhandler()) by sending headers before these handlers can do so.

Buffering implemented by the underlying software/hardware cannot be overridden by PHP and should be taken into account when working with PHP's buffer control functions. Checking the web servers/browsers/consoles buffering settings and working with these can alleviate possible issues. Working in a web context, either the web server's buffering settings or the script's buffering could be adjusted to work in tandem whereas working around the buffering strategies of various browsers can be achieved by adjusting buffering in the PHP script. On consoles that implement line buffering, newline characters could be inserted in the appropriate places before flushing output.

SAPI Differences In Flushing

Although flushing is implemented by each SAPI in a slightly different way, these implementations fall in one of two categories:

- SAPIs used in a web context will flush headers first followed by the output. `Apache2Handler`, `CGI`, `FastCGI` and `FPM` are such SAPIs
- other SAPIs such as `CLI` and `embed` will flush output only

User-Level Output Buffers

Table of Contents

User-level output buffers can be started, manipulated and terminated from PHP code. Each of these buffers includes an output buffer and an associated output handler function.

Turning Output Buffering On

Output buffering can be turned on by using the ob_start() function or by setting the output_buffering and output_handler *php.ini* settings. While both can create output buffers, ob_start() is more flexible as it accepts user-defined functions as output handlers and the operations allowed on the buffer (flush, clean, remove) can be set as well. Buffers started with ob_start() will be active from the line the function was called, while those started with output_buffering will be buffering output from the first line of the script.

PHP is also shipped with a built-in `"URL-Rewriter"` output handler which starts its own output buffer and only allows up to two instances of it running at any time (one for user-level URL-rewriting and one for transparent session id support). These buffers can be started by calling the output_add_rewrite_var() function and/or by enabling the session.use_trans_sid *php.ini* setting.

The bundled `zlib` extension has its own output buffer which can be enabled by using the zlib.output_compression *php.ini* setting.

> **Note** :
>
> While `"URL-Rewriter"` is special in that it only allows up to two instances of it running at any one time, all user-level output buffers use the same underlying buffers used by ob_start() with their functionality implemented by a custom output handler function. As such, all of their functionality can be emulated by userland code.

Flushing, Accessing And Cleaning Buffer Contents

Flushing sends and discards the contents of the active buffer. Output buffers get flushed when the size of the output exceeds the size of the buffer; the script ends or ob_flush(), ob_end_flush() or ob_get_flush() is called.

> **Caution**
>
> Calling ob_end_flush() or ob_get_flush() will turn off the active buffer.

> **Caution**
>
> Flushing buffers will flush the return value of the output handler which can differ from the contents of the buffer. For example, using ob_gzhandler() will compress the output and flush the compressed output.

The contents of the active buffer can be retrieved by calling ob_get_contents(), ob_get_clean() or ob_get_flush().

If only the length of the buffer's contents are needed, ob_get_length() or ob_get_status() will return the length of the contents in bytes.

> **Caution**
>
> Calling ob_get_clean() or ob_get_flush() will turn off the active buffer after returning the its contents.

The contents of the active buffer can be cleaned by calling ob_clean(), ob_end_clean() or ob_get_clean().

> **Caution**
>
> Calling ob_end_clean() or ob_get_clean() will turn off the active buffer.

Turning Buffers Off

Output buffers can be turned off by calling ob_end_clean(), ob_end_flush(), ob_get_flush() or ob_get_clean().

> **Warning**
>
> Output buffers started without the **`PHP_OUTPUT_HANDLER_REMOVABLE`** flag cannot be turned off and may generate an **`E_NOTICE`**.

Every output buffer that has not been closed by the end of the script or when exit() is called will be flushed and turned off by PHP's shutdown process. The buffers will be flushed and turned off in reverse order of their starting up. The last buffered started will be first, the

first buffer started will be last to be flushed and turned off.

> **Caution**
>
> If flushing of the buffer's contents is not desired, a custom output handler should be used to prevent flushing during shutdown.

Exceptions Thrown In Output Handlers

If an uncaught exception is thrown in an output handler the program terminates and the handler is invoked by the shutdown process after which the `"Uncaught Exception"` error message is flushed.

If the uncaught exception is thrown in a handler invoked by ob_flush(), ob_end_flush() or ob_get_flush(), the contents of the buffer are flushed before the error message.

If an uncaught exception is thrown in an output handler during shutdown, the handler is terminated and neither the contents of the buffer nor the error message is flushed.

> **Note** :
>
> If a handler throws an exception its **`PHP_OUTPUT_HANDLER_DISABLED`** status flag is set.

Errors Raised In Output Handlers

If a non-fatal error is raised in an output handler the program continues execution.

If the non-fatal error is raised in a handler invoked by ob_flush(), ob_end_flush() or ob_get_flush(), the buffer flushes certain data depending on the return value of the handler. If the handler returns **`false`** the buffer and the error message are flushed. If the returns anything else the handler return value is flushed but not the error message.

> **Note** :
>
> If a handler returns **`false`** its **`PHP_OUTPUT_HANDLER_DISABLED`** status flag is set.

If a fatal error is raised in an output handler the program terminates and the handler is invoked by the shutdown process after which the error message is flushed.

If the fatal error is raised in a handler invoked by ob_flush(), ob_end_flush() or ob_get_flush(), the contents of the buffers are flushed before the error message.

If a fatal error is raised in an output handler during shutdown the program terminates without flushing the buffer or the error message.

Output In Output Handlers

In specific circumstances, output produced in the handler is flushed along with the contents of the buffer. This output is not appended to the buffer and is not part of the string returned by ob_get_flush().

During flush operations (calling ob_flush(), ob_end_flush(), ob_get_flush() and during shutdown) if the return value of a handler is **`false`** the contents of the buffer are flushed followed by the output. If the handler is not invoked during shutdown the handler throwing an exception or exit() being called results in the same behavior.

> **Note** :
>
> If a handler returns **`false`** its **`PHP_OUTPUT_HANDLER_DISABLED`** status flag is set.

Output Handler Status Flags

The handler status flags of the buffer's `flags` bitmask are set every time to the output handler is invoked and are part of the `flags` returned by ob_get_status(). If the handler successfully executes and does not return **`false`**, **`PHP_OUTPUT_HANDLER_STARTED`** and **`PHP_OUTPUT_HANDLER_PROCESSED`** is set. If the handler returns **`false`** or throws an exception while executing, **`PHP_OUTPUT_HANDLER_STARTED`** and **`PHP_OUTPUT_HANDLER_DISABLED`** is set.

> **Note** :

> If the **PHP_OUTPUT_HANDLER_DISABLED** of a handler is set, the handler will not be invoked by calling ob_end_clean(), ob_end_flush(), ob_get_clean(), ob_get_flush(), ob_clean(), ob_flush() or during PHP's shutdown process. Prior to PHP 8.4.0, this flag had no effect when calling ob_clean() or ob_flush().

What Output Is Buffered?

PHP's user-level output buffers buffer all output after they are started until they are turned off or the script ends. Output in the context of PHP's user-level output buffer is everything that PHP would display or send back to the browser. In practical terms, output is non-zero length data that is:

- outside of the `<?php ?>` tags
- printed by language constructs and functions whose explicit purpose is to output user provided variables or strings such as echo, print, printf(), var_dump(), var_export(), vprintf()
- printed by functions whose purpose is to collect and output data/information on the running script or PHP such as debug_print_backtrace(), phpcredits(), phpinfo(), ReflectionExtension::info()
- printed by PHP on an uncaught exception or an unhandled error (subject to the settings of display_errors and error_reporting)
- anything written to `php://output`

> **Note** :
>
> Data that is written directly to `stdout` or passed to an SAPI function with a similar functionality will not be captured by user-level output buffers. This includes writing data to `stdout` with fwrite() or sending headers using header() or setcookie().

Nesting Output Buffers

If there is an output buffer active when a new buffer is started, the new buffer will be nested inside the previously active buffer. The inner buffer will behave the same way regardless whether it is nested but output buffered by it will not be buffered by the outer buffer. Only output flushed by the inner buffer will be buffered by the outer buffer.

Most `ob_*` functions only work with the active output buffer (the last one started) therefore only the active buffer can be flushed, cleaned and turned off. The functions that work with other buffers are ob_list_handlers() which returns the list of all output handlers in use and ob_get_status() which can return information on the active buffer only or on all buffers in use.

Calling ob_get_level() or ob_get_status() will return the nesting level of the active output buffer.

> **Caution**
>
> The value for identical levels between ob_get_level() and ob_get_status() is off by one. For ob_get_level() the first level is `1`, whereas for ob_get_status() the first level is `0`.

Buffer Size

Buffer sizes are expressed by integers and represent the number of bytes the buffer can store without flushing. When the size of output in the buffer exceeds the size of the buffer, the contents of the buffer are sent to the output handler, its return value is flushed and the buffer is cleared.

With the exception of `"URL-Rewriter"`, the size of output buffers can be set when the buffer is started. If set to `0`, the output buffer is only limited by the memory available to PHP. If set to `1`, the buffer is flushed after every block of code producing any non-zero length output.

The size of output buffers can be retrieved by calling ob_get_status().

Output buffers started with ob_start() will have their buffer sizes set to the integer value passed to the function's second `chunk_size` parameter. If omitted, it is set to `0`.

The output buffer started with output_buffering set to `"On"` will have its buffer size set to 0. If set to an integer than buffer size will correspond to that number.

`"URL-Rewriter"`'s buffer size is set to `0`, therefore it is only limited by the memory available to PHP.

The size of `zlib`'s output buffer is controlled by the zlib.output_compression *php.ini* setting. If set to `"On"` the buffer size will be `"16K"`/`16384`. If set to an integer then buffer size will correspond to that number in bytes.

Operations Allowed On Buffers

The operations allowed on buffers can be controlled by passing one of the buffer control flags to ob_start()'s third `flags` parameter. If omitted, all operations are allowed by default. If `0` is used instead, the buffer cannot be flushed, cleaned or removed but its contents can still be retrieved.

PHP_OUTPUT_HANDLER_CLEANABLE allows ob_clean() to clean the contents of the buffer.

> **Warning**
>
> The absence of the **PHP_OUTPUT_HANDLER_CLEANABLE** flag will not prevent ob_end_clean() or ob_get_clean() from clearing the contents of the buffer.

PHP_OUTPUT_HANDLER_FLUSHABLE allows ob_flush() to flush the contents of the buffer.

> **Warning**
>
> The absence of the **PHP_OUTPUT_HANDLER_FLUSHABLE** flag will not prevent ob_end_flush() or ob_get_flush() from flushing the contents of the buffer.

PHP_OUTPUT_HANDLER_REMOVABLE allows ob_end_clean(), ob_end_flush(), ob_get_clean() or ob_get_flush() to turn off the buffer.

PHP_OUTPUT_HANDLER_STDFLAGS, the combination of the three flags will allow each of the three operations to be performed on the buffer.

Output Handlers

Output handlers are callables associated with output buffers that are invoked by calling ob_clean(), ob_flush(), ob_end_flush(), ob_get_flush(), ob_end_clean(), ob_get_clean() or during PHP's shutdown process.

> **Note** :
>
> The shutdown process will flush the return value of the handler.

If omitted or **null** when starting the output buffer the internal `"default output handler"` will be used which returns the unmodified contents of the buffer when invoked. Output handlers can be used to return a modified version of the buffer's contents and/or have side-effects (e.g. send headers).

PHP comes with two internal output handlers: `"default output handler"` and `"URL-Rewriter"` (which is integrated into its own output buffer and only up to two instances of it can be started).

The bundled extensions include four additional output handlers: mb_output_handler(), ob_gzhandler(), ob_iconv_handler(), ob_tidyhandler().

Working With Output Handlers

When invoked, output handlers are passed the contents of the buffer and a bitmask indicating the status of output buffering.

function handler (string `$buffer` , int `$phase` = ?): string

`buffer`

Contents of the output buffer.

`phase`

Bitmask of **PHP_OUTPUT_HANDLER_*** constants .

> **Warning**
>
> Calling any of the following functions from within an output handler will result in a fatal error: ob_clean(), ob_end_clean(), ob_end_flush(), ob_flush(), ob_get_clean(), ob_get_flush(), ob_start().

> **Note** :
>
> If the `PHP_OUTPUT_HANDLER_DISABLED` of a handler is set, the handler will not be invoked by calling ob_end_clean(), ob_end_flush(), ob_get_clean(), ob_get_flush(), ob_clean(), ob_flush() or during PHP's shutdown process. Prior to PHP 8.4.0, this flag had no effect when calling ob_clean() or ob_flush().

> **Note** :
>
> The working directory of the script can change inside the shutdown function under some web servers, e.g. Apache or the built-in web server.

Flags Passed To Output Handlers

The bitmask passed to the second `phase` parameter of the output handler provides information on the invocation of the handler.

> **Note** :
>
> The bitmask can include more than one flag and the bitwise `&` operator should be used to check whether a flag is set.

> **Warning**
>
> The value of `PHP_OUTPUT_HANDLER_WRITE` and its alias `PHP_OUTPUT_HANDLER_CONT` is `0` therefore whether it is set can only be determined by using an equality operator (`==` or `===`).

The following flags are set in a specific phase of the handler's lifecycle: `PHP_OUTPUT_HANDLER_START` is set when a handler is invoked for the first time. `PHP_OUTPUT_HANDLER_FINAL` or its alias `PHP_OUTPUT_HANDLER_END` is set when a handler is invoked for the last time, i.e. it is being turned off. This flag is also set when buffers are being turned off by PHP's shutdown process.

The following flags are set by a specific invocation of the handler: `PHP_OUTPUT_HANDLER_FLUSH` is set when the handler is invoked by calling ob_flush(). `PHP_OUTPUT_HANDLER_WRITE` or its alias `PHP_OUTPUT_HANDLER_CONT` is set when the size of its contents equals or exceeds the size of the buffer and the handler is invoked while the buffer is being automatically flushed. `PHP_OUTPUT_HANDLER_FLUSH` is set when the handler is invoked by calling ob_clean(), ob_end_clean() or ob_get_clean(). When ob_end_clean() or ob_get_clean() is called, `PHP_OUTPUT_HANDLER_FINAL` is set as well.

> **Note** :
>
> When ob_end_flush() or ob_get_flush() is called, `PHP_OUTPUT_HANDLER_FINAL` is set but `PHP_OUTPUT_HANDLER_FLUSH` is not.

Output Handler Return Values

The return value of the output handler is internally coerced into a string following standard PHP type semantics, with two exceptions: arrays and booleans.

Arrays are converted into the string `"Array"` but the `Array to string conversion` warning is not triggered.

If the handler returns `false` the contents of the buffer are returned. If the handler returns `true` an empty string is returned.

> **Note** :
>
> If a handler returns `false` or throws an exception its `PHP_OUTPUT_HANDLER_DISABLED` status flag is set.

Examples

Table of Contents

Basic usage

Example #1 Output Control example

```php
<?php

ob_start();
echo "Hello\n";

setcookie("cookiename", "cookiedata");

ob_end_flush();

?>
```

In the above example, the output from echo would be stored in the output buffer until ob_end_flush() was called. In the mean time, the call to setcookie() successfully stored a cookie without causing an error. (Headers cannot normally be sent to the browser after data has already been sent.)

Output rewrite usage

Since PHP 7.1.0, output_add_rewrite_var(), output_reset_rewrite_vars() use dedicated output buffer. i.e. It does not use trans sid output buffer.

Example #1 Output rewrite example

```php
<?php
// This code works with PHP 7.1.0, 7.0.10, 5.6.25 and up.

// HTTP_HOST is default target host. Set manually to make sample code works.
$_SERVER['HTTP_HOST'] = 'php.net';

// Output rewriter only rewrite form. Add a=href.
// Tags can be specified tag_name=url_attr, e.g. img=src,iframe=src
// No space allowed between settings.
// Form tag is special tag that add hidden input.
ini_set('url_rewriter.tags','a=href,form=');
var_dump(ini_get('url_rewriter.tags'));

// This is added to URL and form
output_add_rewrite_var('test', 'value');
?>
<a href="//php.net/index.php?bug=1234">bug1234</a>
<form action="https://php.net/?bug=1234&edit=1" method="post">
 <input type="text" name="title" />
</form>
```

The above example will output:

```
<a href="//php.net/?bug=1234&test=value">bug1234</a>
<form action="https://php.net/?bug=1234&edit=1" method="post"><input type="hidden" name="test" value="value" />
 <input type="text" name="title" />
</form>
```

Since PHP 7.1.0, output rewrite functions have its own INI settings, url_rewriter.tags and url_rewriter.hosts.

Output Control Functions

See Also

See also header() and setcookie().

Table of Contents

flush

flush

(PHP 4, PHP 5, PHP 7, PHP 8)

flush — Flush system output buffer

Description

function **flush**(): void

Flushes the system write buffers of PHP and the backend used by PHP (e.g.: CGI, a web server). In a command line environment **flush()** will attempt to flush the contents of the buffers only whereas in a web context headers and the contents of the buffers are flushed.

> **Note** :
>
> **flush()** may not be able to override the buffering scheme of the web server and it has no effect on any client-side buffering in the browser.

> **Note** :
>
> This function does not have any effect on user level output handlers such as those started by ob_start() or output_add_rewrite_var().

> **Warning**
>
> **flush()** can interfere with output handlers that set and send headers in a web context (e.g. ob_gzhandler()) by sending headers before these handlers can do so.

Parameters

This function has no parameters.

Return Values

No value is returned.

Changelog

Version	Description
8.4.0	Flushing headers without a body will now succeed in FastCGI.

See Also

- ob_flush() - Flush (send) the return value of the active output handler
- ob_clean() - Clean (erase) the contents of the active output buffer
- ob_end_flush() - Flush (send) the return value of the active output handler and turn the active output buffer off
- ob_end_clean() - Clean (erase) the contents of the active output buffer and turn it off

ob_clean

ob_clean

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_clean — Clean (erase) the contents of the active output buffer

Description

function **ob_clean**(): bool

This function calls the output handler (with the `PHP_OUTPUT_HANDLER_CLEAN` flag), discards it's return value and cleans (erases) the contents of the active output buffer.

This function does not turn off the active output buffer like ob_end_clean() or ob_get_clean() does.

ob_clean() will fail without an active output buffer started with the `PHP_OUTPUT_HANDLER_CLEANABLE` flag.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If the function fails it generates an `E_NOTICE`.

See Also

- ob_start() - Turn on output buffering
- ob_get_contents() - Return the contents of the output buffer
- ob_end_clean() - Clean (erase) the contents of the active output buffer and turn it off
- ob_get_clean() - Get the contents of the active output buffer and turn it off
- ob_flush() - Flush (send) the return value of the active output handler

ob_end_clean

ob_end_clean

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_end_clean — Clean (erase) the contents of the active output buffer and turn it off

Description

function **ob_end_clean**(): bool

This function calls the output handler (with the `PHP_OUTPUT_HANDLER_CLEAN` and `PHP_OUTPUT_HANDLER_FINAL` flags), discards its return value, discards the contents of the active output buffer and turns off the active output buffer.

ob_end_clean() will fail without an active output buffer started with the `PHP_OUTPUT_HANDLER_REMOVABLE` flag.

ob_end_clean() will discard the contents of the active output buffer even if it was started without the `PHP_OUTPUT_HANDLER_CLEANABLE` flag.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If the function fails it generates an `E_NOTICE`.

Examples

The following example shows an easy way to get rid of the contents of the active output buffer:

Example #1 ob_end_clean() example

```
<?php
ob_start();
echo 'Text that won\'t get displayed.';
ob_end_clean();
?>
```

See Also

- ob_start() - Turn on output buffering
- ob_get_contents() - Return the contents of the output buffer
- ob_clean() - Clean (erase) the contents of the active output buffer
- ob_get_clean() - Get the contents of the active output buffer and turn it off
- ob_end_flush() - Flush (send) the return value of the active output handler and turn the active output buffer off

ob_end_flush

ob_end_flush

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_end_flush — Flush (send) the return value of the active output handler and turn the active output buffer off

Description

function **ob_end_flush**(): bool

This function calls the output handler (with the `PHP_OUTPUT_HANDLER_FINAL` flag), flushes (sends) it's return value, discards the contents of the active output buffer and turns off the active output buffer.

ob_end_flush() will fail without an active output buffer started with the `PHP_OUTPUT_HANDLER_REMOVABLE` flag.

ob_end_flush() will flush (send) the return value of the output handler even if the active output buffer was started without the `PHP_OUTPUT_HANDLER_FLUSHABLE` flag.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If the function fails it generates an `E_NOTICE`.

Examples

Example #1 ob_end_flush() example

The following example shows an easy way to flush and end all output buffers:

```
<?php
  while (@ob_end_flush());
?>
```

See Also

- ob_start() - Turn on output buffering
- ob_get_contents() - Return the contents of the output buffer
- ob_flush() - Flush (send) the return value of the active output handler
- ob_get_flush() - Flush (send) the return value of the active output handler, return the contents of the active output buffer and turn it off
- ob_end_clean() - Clean (erase) the contents of the active output buffer and turn it off

ob_flush

ob_flush

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_flush — Flush (send) the return value of the active output handler

Description

function **ob_flush**(): bool

This function calls the output handler (with the `PHP_OUTPUT_HANDLER_FLUSH` flag), flushes (sends) its return value and discards the contents of the active output buffer.

This function does not turn off the active output buffer like ob_end_flush() or ob_get_flush() does.

ob_flush() will fail without an active output buffer started with the `PHP_OUTPUT_HANDLER_FLUSHABLE` flag.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If the function fails it generates an `E_NOTICE`.

See Also

- ob_start() - Turn on output buffering
- ob_get_contents() - Return the contents of the output buffer
- ob_end_flush() - Flush (send) the return value of the active output handler and turn the active output buffer off
- ob_get_flush() - Flush (send) the return value of the active output handler, return the contents of the active output buffer and turn it off
- ob_clean() - Clean (erase) the contents of the active output buffer

ob_get_clean

ob_get_clean

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

ob_get_clean — Get the contents of the active output buffer and turn it off

Description

function **ob_get_clean**(): string|false

This function calls the output handler (with the `PHP_OUTPUT_HANDLER_CLEAN` and `PHP_OUTPUT_HANDLER_FINAL` flags), discards it's return value, returns the contents of the active output buffer and turns off the active output buffer.

ob_get_clean() will fail without an active output buffer started with the `PHP_OUTPUT_HANDLER_REMOVABLE` flag.

ob_get_clean() will discard the contents of the active output buffer even if it was started without the `PHP_OUTPUT_HANDLER_CLEANABLE` flag.

Parameters

This function has no parameters.

Return Values

Returns the contents of the active output buffer on success or `false` on failure.

> **Caution**
>
> **ob_get_clean()** will return false but will not generate an `E_NOTICE` if there is no active output buffer.

Errors/Exceptions

If the function fails it generates an `E_NOTICE`.

Examples

Example #1 A simple ob_get_clean() example

```
<?php

ob_start();

echo "Hello World";

$out = ob_get_clean();
$out = strtolower($out);
```

```
var_dump($out);
?>
```

The above example will output:

```
string(11) "hello world"
```

See Also

- ob_start() - Turn on output buffering
- ob_get_contents() - Return the contents of the output buffer
- ob_clean() - Clean (erase) the contents of the active output buffer
- ob_end_clean() - Clean (erase) the contents of the active output buffer and turn it off
- ob_get_flush() - Flush (send) the return value of the active output handler, return the contents of the active output buffer and turn it off

ob_get_contents

ob_get_contents

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_get_contents — Return the contents of the output buffer

Description

function **ob_get_contents**(): string|false

Gets the contents of the output buffer without clearing it.

Parameters

This function has no parameters.

Return Values

This will return the contents of the output buffer or `false`, if output buffering isn't active.

Examples

Example #1 A simple **ob_get_contents()** example

```
<?php

ob_start();

echo "Hello ";

$out1 = ob_get_contents();

echo "World";

$out2 = ob_get_contents();

ob_end_clean();

var_dump($out1, $out2);
?>
```

The above example will output:

```
string(6) "Hello "
string(11) "Hello World"
```

See Also

- ob_start() - Turn on output buffering
- ob_get_length() - Return the length of the output buffer

ob_get_flush

ob_get_flush

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

ob_get_flush — Flush (send) the return value of the active output handler, return the contents of the active output buffer and turn it off

Description

function **ob_get_flush**(): string|false

This function calls the output handler (with the `PHP_OUTPUT_HANDLER_FINAL` flag), flushes (sends) it's return value, returns the contents of the active output buffer and turns off the active output buffer.

ob_get_flush() will fail without an active output buffer started with the `PHP_OUTPUT_HANDLER_REMOVABLE` flag.

ob_get_flush() will flush (send) the return value of the output handler even if the active output buffer was started without the `PHP_OUTPUT_HANDLER_FLUSHABLE` flag.

Parameters

This function has no parameters.

Return Values

Returns the contents of the active output buffer on success or `false` on failure.

Errors/Exceptions

If the function fails it generates an `E_NOTICE`.

Examples

Example #1 ob_get_flush() example

```php
<?php
//using output_buffering=On
print_r(ob_list_handlers());

//save buffer in a file
$buffer = ob_get_flush();
file_put_contents('buffer.txt', $buffer);

print_r(ob_list_handlers());
?>
```

The above example will output:

```
Array
(
    [0] => default output handler
)
Array
(
)
```

See Also

- ob_start() - Turn on output buffering
- ob_get_contents() - Return the contents of the output buffer
- ob_flush() - Flush (send) the return value of the active output handler
- ob_end_flush() - Flush (send) the return value of the active output handler and turn the active output buffer off
- ob_get_clean() - Get the contents of the active output buffer and turn it off

ob_get_length

ob_get_length

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

ob_get_length — Return the length of the output buffer

Description

function **ob_get_length**(): int|false

This will return the length of the contents in the output buffer, in bytes.

Parameters

This function has no parameters.

Return Values

Returns the length of the output buffer contents, in bytes, or `false` if no buffering is active.

Examples

Example #1 A simple ob_get_length() example

```php
<?php

ob_start();

echo "Hello ";

$len1 = ob_get_length();

echo "World";

$len2 = ob_get_length();

ob_end_clean();

echo $len1 . ", " . $len2;
?>
```

The above example will output:

```
6, 11
```

See Also

- ob_start() - Turn on output buffering
- ob_get_contents() - Return the contents of the output buffer

ob_get_level

ob_get_level

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_get_level — Return the nesting level of the output buffering mechanism

Description

function **ob_get_level**(): int

Returns the nesting level of the output buffering mechanism.

Parameters

This function has no parameters.

Return Values

Returns the level of nested output buffering handlers or zero if output buffering is not active.

> **Caution**
>
> The value for identical levels between **ob_get_level()** and ob_get_status() is off by one. For **ob_get_level()** the first level is `1`. Whereas for ob_get_status() the first level is `0`.

See Also

- ob_start() - Turn on output buffering
- ob_get_status() - Get status of output buffers
- ob_get_contents() - Return the contents of the output buffer

ob_get_status

ob_get_status

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ob_get_status — Get status of output buffers

Description

function **ob_get_status**(bool `$full_status = false`): array

ob_get_status() returns status information on either the top level output buffer or all active output buffer levels if `full_status` is set to `true`.

Parameters

`full_status`

`true` to return all active output buffer levels. If `false` or not set, only the top level output buffer is returned.

Return Values

If `full_status` is omitted or `false` a simple array holding status information on the active output level is returned.

If `full_status` is `true` an array with one element for each active output buffer level is returned. The output level is used as key of the top level array and each array element itself is another array holding status information on one active output level.

An empty array is returned if output buffering is not turned on.

ob_get_status() return array elements

Key	Value
name	Name of active output handler (see the return values of ob_list_handlers() for details)
type	`0` (internal handler) or `1` (user supplied handler)

Key	Value
flags	Bitmask of flags set by ob_start(), the type of output handler (see above) and the status of the buffering process (`PHP_OUTPUT_HANDLER_*` constants). If the handler successfully processed the buffer and did not return `false`, `PHP_OUTPUT_HANDLER_STARTED` and `PHP_OUTPUT_HANDLER_PROCESSED` will be set. If the handler failed while processing the buffer or returned `false`, `PHP_OUTPUT_HANDLER_STARTED` and `PHP_OUTPUT_HANDLER_DISABLED` will be set.
level	Output nesting level (zero based). Note that the value returned for the same level by ob_get_level() is off by one. The first level is `0` for **ob_get_status()**, and `1` for ob_get_level().
chunk_size	Chunk size in bytes. Set by ob_start() or output_buffering is enabled and its value is set to a positive integer
buffer_size	Currently allocated size of the output buffer in bytes. It starts at `16384` bytes with the default `chunk_size`, or at `chunk_size` rounded up to the next multiple of `4096` when one is given, and grows in multiples of `4096` as more output is buffered. This is not the amount of data in the buffer (see `buffer_used`).
buffer_used	Size of data in output buffer in bytes (the same as the integer return value of ob_get_length())

Examples

Example #1 Array returned when `full_status` is `false`

```
Array
(
    [name] => URL-Rewriter
    [type] => 0
    [flags] => 112
    [level] => 2
    [chunk_size] => 0
    [buffer_size] => 16384
    [buffer_used] => 1024
)
```

Example #2 Array returned when `full_status` is `true`

```
Array
(
    [0] => Array
        (
            [name] => default output handler
            [type] => 0
            [flags] => 112
            [level] => 1
            [chunk_size] => 0
            [buffer_size] => 16384
            [buffer_used] => 2048
        )

    [1] => Array
        (
            [name] => URL-Rewriter
            [type] => 0
            [flags] => 112
            [level] => 2
            [chunk_size] => 0
            [buffer_size] => 16384
            [buffer_used] => 1024
        )

)
```

See Also

- ob_get_level() - Return the nesting level of the output buffering mechanism
- ob_list_handlers() - List all output handlers in use
- ob_get_length() - Return the length of the output buffer
- ob_start() - Turn on output buffering

ob_implicit_flush

ob_implicit_flush

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_implicit_flush — Turn implicit flush on/off

Description

function **ob_implicit_flush**(bool `$enable = true`): void

ob_implicit_flush() will turn implicit flushing on or off. Implicit flushing will result in a flush operation after every block of code resulting in output, so that explicit calls to flush() will no longer be needed.

> **Note** :
>
> Printing empty strings or sending headers is not considered output and will not result in a flush operation.

> **Note** :
>
> This function does not have any effect on user level output handlers such as those started by ob_start() or output_add_rewrite_var().

Parameters

`enable`

`true` to turn implicit flushing on, `false` otherwise.

Return Values

No value is returned.

Changelog

Version	Description
8.0.0	The `enable` expects a bool value now; previously, an int was expected.

See Also

- flush() - Flush system output buffer
- ob_start() - Turn on output buffering
- ob_end_flush() - Flush (send) the return value of the active output handler and turn the active output buffer off

ob_list_handlers

ob_list_handlers

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

ob_list_handlers — List all output handlers in use

Description

function **ob_list_handlers**(): array

Lists all output handlers in use.

Parameters

This function has no parameters.

Return Values

This will return an array with the output handlers in use (if any).

If output_buffering is enabled and no output_handler is set, or no callback or `null` was passed to ob_start(), `"default output handler"` is returned. Enabling output_buffering and setting an output_handler is equivalent to passing an internal (built-in) function to ob_start().

If a callable was passed to ob_start(), the fully qualified name of the callable is returned. If the callable is an object implementing __invoke(), the fully qualified name of the object's __invoke() method is returned. If the callable is a Closure, `"Closure::__invoke"` is returned.

Examples

Example #1 ob_list_handlers() example

```php
<?php
// using output_buffering=On, no output_handler set
var_dump(ob_list_handlers());
ob_end_flush();

// no callback or null
ob_start();
var_dump(ob_list_handlers());
ob_end_flush();

// anonymous function
ob_start(function($string) { return $string; });
var_dump(ob_list_handlers());
ob_end_flush();

// arrow function
ob_start(fn($string) => $string);
var_dump(ob_list_handlers());
ob_end_flush();

// first class callable
$firstClassCallable = userDefinedFunction(...);

ob_start([$firstClassCallable, '__invoke']);
var_dump(ob_list_handlers());
ob_end_flush();

// internal (built-in) function
ob_start('print_r');
var_dump(ob_list_handlers());
ob_end_flush();

// user-defined function
function userDefinedFunction($string, $flags) { return $string; };

ob_start('userDefinedFunction');
var_dump(ob_list_handlers());
ob_end_flush();

class MyClass {
    public static function staticHandle($string) {
        return $string;
    }

    public static function handle($string) {
        return $string;
    }

    public function __invoke($string) {
        return $string;
    }
}

// class and static method
ob_start(['MyClass','staticHandle']);
```

```php
var_dump(ob_list_handlers());
ob_end_flush();

// object and non-static method
ob_start([new MyClass,'handle']);
var_dump(ob_list_handlers());
ob_end_flush();

// invokable object
ob_start(new MyClass);
var_dump(ob_list_handlers());
ob_end_flush();
?>
```

The above example will output:

```
array(1) {
  [0]=>
  string(22) "default output handler"
}
array(1) {
  [0]=>
  string(22) "default output handler"
}
array(1) {
  [0]=>
  string(7) "print_r"
}
array(1) {
  [0]=>
  string(19) "userDefinedFunction"
}
array(1) {
  [0]=>
  string(17) "Closure::__invoke"
}
array(1) {
  [0]=>
  string(17) "Closure::__invoke"
}
array(1) {
  [0]=>
  string(17) "Closure::__invoke"
}
array(1) {
  [0]=>
  string(21) "MyClass::staticHandle"
}
array(1) {
  [0]=>
  string(15) "MyClass::handle"
}
array(1) {
  [0]=>
  string(17) "MyClass::__invoke"
}
```

See Also

- ob_end_clean() - Clean (erase) the contents of the active output buffer and turn it off
- ob_end_flush() - Flush (send) the return value of the active output handler and turn the active output buffer off
- ob_get_flush() - Flush (send) the return value of the active output handler, return the contents of the active output buffer and turn it off
- ob_start() - Turn on output buffering

ob_start

ob_start

(PHP 4, PHP 5, PHP 7, PHP 8)

ob_start — Turn on output buffering

Description

function **ob_start**(?callable `$callback` = `null`, int `$chunk_size` = 0, int `$flags` = `PHP_OUTPUT_HANDLER_STDFLAGS`): bool

This function will turn output buffering on. While output buffering is active no output is sent from the script, instead the output is stored in an internal buffer. See What Output Is Buffered? on exactly what output is affected.

Output buffers are stackable, that is, **ob_start()** may be called while another buffer is active. If multiple output buffers are active, output is being filtered sequentially through each of them in nesting order. See Nesting Output Buffers for more details.

See User-Level Output Buffers for a detailed description of output buffers.

Parameters

`callback`

An optional `callback` callable may be specified. It can also be bypassed by passing `null`.

`callback` is invoked when the output buffer is flushed (sent), cleaned, or when the output buffer is flushed at the end of the script.

The signature of the `callback` is as follows:

function handler(string `$buffer`, int `$phase` = ?): string

`buffer`

Contents of the output buffer.

`phase`

Bitmask of `PHP_OUTPUT_HANDLER_*` constants . See Flags Passed To Output Handlers for more details.

If `callback` returns `false` the contents of the buffer are returned. See Output Handler Return Values for more details.

> **Warning**
>
> Calling any of the following functions from within an output handler will result in a fatal error: ob_clean(), ob_end_clean(), ob_end_flush(), ob_flush(), ob_get_clean(), ob_get_flush(), **ob_start()**.

See Output Handlers and Working With Output Handlers for more details on `callback`s (output handlers).

`chunk_size`

If the optional parameter `chunk_size` is passed, the buffer will be flushed after any block of code resulting in output that causes the buffer's length to equal or exceed `chunk_size`. The default value `0` means that all output is buffered until the buffer is turned off. A value of `1` means the buffer will be flushed after every output operation, effectively disabling buffering. `chunk_size` also determines the initial size of the buffer reported by ob_get_status(): `16384` bytes when it is `0`, and `chunk_size` rounded up to the next multiple of `4096` otherwise. See Buffer Size for more details.

`flags`

The `flags` parameter is a bitmask that controls the operations that can be performed on the output buffer. The default is to allow output buffers to be cleaned, flushed and removed, which can be set explicitly via the buffer control flags . See Operations Allowed On Buffers for more details.

Each flag controls access to a set of functions, as described below:

Constant	Functions
`PHP_OUTPUT_HANDLER_CLEANABLE`	ob_clean()
`PHP_OUTPUT_HANDLER_FLUSHABLE`	ob_flush()
`PHP_OUTPUT_HANDLER_REMOVABLE`	ob_end_clean(), ob_end_flush(), ob_get_clean(), ob_get_flush()

> **Note** :
>
> Prior to PHP 8.4.0, the flags parameter could set the output handler status flags as well.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.5.0	Trying to produce output (e.g. with echo) within the user output handler callback is now deprecated. The deprecation warning bypasses the handler and is output directly.

Examples

Example #1 User defined callback function example

```php
<?php

function callback($buffer)
{
  // replace all the apples with oranges
  return (str_replace("apples", "oranges", $buffer));
}

ob_start("callback");

?>
<html>
<body>
<p>It's like comparing apples to oranges.</p>
</body>
</html>
<?php

ob_end_flush();

?>
```

The above example will output:

```
<html>
<body>
<p>It's like comparing oranges to oranges.</p>
</body>
</html>
```

Example #2 Creating an unerasable output buffer

```php
<?php

ob_start(null, 0, PHP_OUTPUT_HANDLER_STDFLAGS ^ PHP_OUTPUT_HANDLER_REMOVABLE);

?>
```

See Also

- ob_get_contents() - Return the contents of the output buffer
- ob_end_clean() - Clean (erase) the contents of the active output buffer and turn it off
- ob_end_flush() - Flush (send) the return value of the active output handler and turn the active output buffer off

- ob_implicit_flush() - Turn implicit flush on/off
- ob_gzhandler() - ob_start callback function to gzip output buffer
- ob_iconv_handler() - Convert character encoding as output buffer handler
- mb_output_handler() - Callback function converts character encoding in output buffer
- ob_tidyhandler() - ob_start callback function to repair the buffer

output_add_rewrite_var

output_add_rewrite_var

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

output_add_rewrite_var — Add URL rewriter values

Description

function **output_add_rewrite_var**(string `$name`, string `$value`): bool

This function starts the `'URL-Rewriter'` output buffer handler if it is not active, stores the `name` and `value` parameters, and when the buffer is flushed rewrites the URLs and forms based on the applicable ini settings. Subsequent calls to this function will store all additional name/value pairs until the handler is turned off.

When the output buffer is flushed (by calling ob_flush(), ob_end_flush(), ob_get_flush() or at the end of the script) the `'URL-Rewriter'` handler adds the name/value pairs as query parameters to URLs in attributes of HTML tags and adds hidden fields to forms based on the values of the url_rewriter.tags and url_rewriter.hosts configuration directives.

Each name/value pair added to the `'URL-Rewriter'` handler is added to the URLs and/or forms even if this results in duplicate URL query parameters or elements with the same name attributes.

> **Note** :
>
> Once the `'URL-Rewriter'` handler has been turned off it cannot be started again.

> **Warning**
>
> Prior to PHP 8.4.0, the hosts to be rewritten were set in session.trans_sid_hosts instead of url_rewriter.hosts.

Parameters

`name`

The variable name.

`value`

The variable value.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
7.1.0	As of PHP 7.1.0, a dedicated output buffer is used, url_rewriter.tags is used solely for output functions and url_rewriter.hosts is available. Prior to PHP 7.1.0, rewrite variables set by **output_add_rewrite_var()** shared an output buffer with transparent session id support (see session.trans_sid_tags).

Examples

Example #1 output_add_rewrite_var() example

```
<?php
ini_set('url_rewriter.tags', 'a=href,form=');

output_add_rewrite_var('var', 'value');
```

```php
// some links
echo '<a href="file.php">link</a>
<a href="http://example.com">link2</a>';

// a form
echo '<form action="script.php" method="post">
<input type="text" name="var2" />
</form>';

print_r(ob_list_handlers());
?>
```

The above example will output:

```
<a href="file.php?var=value">link</a>
<a href="http://example.com">link2</a>

<form action="script.php" method="post">
<input type="hidden" name="var" value="value" />
<input type="text" name="var2" />
</form>

Array
(
    [0] => URL-Rewriter
)
```

See Also

- output_reset_rewrite_vars() - Reset URL rewriter values
- ob_flush() - Flush (send) the return value of the active output handler
- ob_list_handlers() - List all output handlers in use
- url_rewriter.tags
- url_rewriter.hosts

output_reset_rewrite_vars

output_reset_rewrite_vars

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

output_reset_rewrite_vars — Reset URL rewriter values

Description

function **output_reset_rewrite_vars**(): bool

This function removes all rewrite variables previously set by the output_add_rewrite_var() function.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
7.1.0	Before PHP 7.1.0, rewrite vars set by output_add_rewrite_var() use the same Session module trans sid output buffer. Since PHP 7.1.0, dedicated output buffer is used and **output_reset_rewrite_vars()** only removes rewrite vars defined by output_add_rewrite_var().

Examples

Example #1 **output_reset_rewrite_vars()** example

```php
<?php
ini_set('url_rewriter.tags', 'a=href');

output_add_rewrite_var('var', 'value');

echo '<a href="file.php">link</a>';
ob_flush();

output_reset_rewrite_vars();
echo '<a href="file.php">link</a>';
?>
```

The above example will output:

```
<a href="file.php?var=value">link</a>
<a href="file.php">link</a>
```

See Also

- output_add_rewrite_var() - Add URL rewriter values
- ob_flush() - Flush (send) the return value of the active output handler
- ob_list_handlers() - List all output handlers in use
- url_rewriter.tags
- url_rewriter.hosts
- session.trans_sid_tags
- session.trans_sid_hosts

PHP Options and Information

Introduction

This functions enable you to get a lot of information about PHP itself, e.g. runtime configuration, loaded extensions, version and much more. You'll also find functions to set options for your running PHP. The probably best known function of PHP - phpinfo() - can be found here.

Installing/Configuring

Table of Contents

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

PHP Options/Info Configuration Options

Name	Default	Changeable	Changelog
assert.active	"1"	INI_ALL	Deprecated as of PHP 8.3.0
assert.bail	"0"	INI_ALL	Deprecated as of PHP 8.3.0
assert.warning	"1"	INI_ALL	Deprecated as of PHP 8.3.0
assert.callback	NULL	INI_ALL	Deprecated as of PHP 8.3.0
assert.quiet_eval	"0"	INI_ALL	Removed as of PHP 8.0.0
assert.exception	"1"	INI_ALL	Prior to PHP 8.0.0, defaults to "0". Deprecated as of PHP 8.3.0
enable_dl	"1"	INI_SYSTEM	This deprecated feature *will* certainly be *removed* in the future.
max_execution_time	"30"	INI_ALL	
max_input_time	"-1"	INI_PERDIR	
max_input_nesting_level	"64"	INI_PERDIR	
max_input_vars	1000	INI_PERDIR	
zend.enable_gc	"1"	INI_ALL	
zend.max_allowed_stack_size	"0"	INI_SYSTEM	Available as of PHP 8.3.0.
zend.reserved_stack_size	"0"	INI_SYSTEM	Available as of PHP 8.3.0.
fiber.stack_size		INI_ALL	Available as of PHP 8.1.0.

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`assert.active` bool

Enable assert() evaluation. zend.assertions should be used instead to control the behaviour of assert().

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

`assert.bail` bool

Terminate script execution on failed assertions.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

`assert.warning` bool

Issue a PHP warning for each failed assertion.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

`assert.callback` string

User function to call on failed assertions.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

`assert.quiet_eval` bool

> **Warning**
>
> This feature was *REMOVED* as of PHP 8.0.0.

Use the current setting of error_reporting() during assertion expression evaluation. If enabled, no errors are shown (implicit error_reporting(0)) while evaluation. If disabled, errors are shown according to the settings of error_reporting()

`assert.exception` bool

Issue an AssertionError exception for the failed assertion.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

`enable_dl` bool

This directive allows to turn dynamic loading of PHP extensions with dl() on and off.

The main reason for turning dynamic loading off is security. With dynamic loading, it's possible to ignore all open_basedir restrictions. The default is to allow dynamic loading.

`max_execution_time` int

This sets the maximum time in seconds a script is allowed to run before it is terminated by the parser. This helps prevent poorly written scripts from tying up the server. The default setting is `30`. When running PHP from the command line the default setting is `0`.

On non Windows systems, the maximum execution time is not affected by system calls, stream operations etc. Please see the set_time_limit() function for more details.

When PHP is compiled with --enable-zend-max-execution-timers (the default for ZTS builds since PHP 8.3.0), the limit is tracked with POSIX per-thread timers that measure elapsed time, so time spent sleeping or waiting on I/O counts towards it. The default `setitimer(ITIMER_PROF)` based tracking measures CPU time instead.

Your web server can have other timeout configurations that may also interrupt PHP execution. Apache has a `Timeout` directive and IIS has a CGI timeout function. Both default to 300 seconds. See your web server documentation for specific details.

`max_input_time` int

This sets the maximum time in seconds a script is allowed to parse input data, like POST and GET. Timing begins at the moment PHP is invoked at the server and ends when execution begins. The default setting is `-1`, which means that max_execution_time is used instead. Set to `0` to allow unlimited time.

`max_input_nesting_level` int

Sets the max nesting depth of input variables (i.e. *$_GET*, *$_POST*.)

`max_input_vars` int

How many input variables may be accepted (limit is applied to $_GET, $_POST and $_COOKIE superglobal separately). Use of this directive mitigates the possibility of denial of service attacks which use hash collisions. If there are more input variables than specified by this directive, an **`E_WARNING`** is issued, and further input variables are truncated from the request.

`zend.enable_gc` bool

Enables or disables the circular reference collector.

`zend.max_allowed_stack_size` int

The maximum native stack space that the operating system allows the program to consume. Trying to consume more than the operating system allows typically results in a hard crash with no easily available debugging information. To make debugging easier, the engine throws an Error before it happens (when the program uses more than zend.max_allowed_stack_size-zend.reserved_stack_size bytes of stack).

Recursion in user-defined code does not consume native stack space. However, internal functions and magic methods do. Very deep recursion involving these functions can cause the program to exhaust all available native stack space.

Possible values for this parameter are:

- `0`: Auto-detect the maximum native stack space that the operating system allows the program to consume. This is the default. When detection is not possible, a known system default is used.
- `-1`: Disables stack size checking in the engine.
- Positive integer: A fixed size, in bytes. Setting this value too high has the same effect as disabling stack size checking.

As the stack size of fibers is determined by fiber.stack_size, the value of this parameter is used instead of zend.max_allowed_stack_size when checking stack usage during the execution of a Fiber.

> **Note** :
>
> This is not related to stack *buffer* overflows, and is not a security feature.

`zend.reserved_stack_size` int

The reserved stack size, in bytes. This is subtracted from the max allowed stack size, as a buffer, when checking the stack size.

Possible values for this parameter are:

- `0`: Auto-detect a sensible size.
- Positive integer: A fixed size, in bytes.

`fiber.stack_size` int

The native stack size, in bytes, allocated for each Fiber.

The default value is 1MiB on systems with a pointer size lower than 8 bytes, or 2MiB otherwise.

Predefined Constants

The constants below are always available as part of the PHP core.

Pre-defined phpcredits() constants

Constants	Description
`CREDITS_GROUP` (int)	A list of the core developers
`CREDITS_GENERAL` (int)	General credits: Language design and concept, PHP authors and SAPI module.
`CREDITS_SAPI` (int)	A list of the server API modules for PHP, and their authors.
`CREDITS_MODULES` (int)	A list of the extension modules for PHP, and their authors.
`CREDITS_DOCS` (int)	The credits for the documentation team.

CREDITS_FULLPAGE (int)	Usually used in combination with the other flags. Indicates that a complete stand-alone HTML page needs to be printed including the information indicated by the other flags.
CREDITS_QA (int)	The credits for the quality assurance team.
CREDITS_ALL (int)	All the credits, equivalent to using: CREDITS_DOCS | CREDITS_GENERAL | CREDITS_GROUP | CREDITS_MODULES | CREDITS_QA | CREDITS_FULLPAGE. It generates a complete stand-alone HTML page with the appropriate tags. This is the default value.

phpinfo() constants

Constants	Description
INFO_GENERAL (int)	The configuration line, *php.ini* location, build date, Web Server, System and more.
INFO_CREDITS (int)	PHP Credits. See also phpcredits().
INFO_CONFIGURATION (int)	Current Local and Master values for PHP directives. See also ini_get().
INFO_MODULES (int)	Loaded modules and their respective settings.
INFO_ENVIRONMENT (int)	Environment Variable information that's also available in *$_ENV*.
INFO_VARIABLES (int)	Shows all predefined variables from EGPCS (Environment, GET, POST, Cookie, Server).
INFO_LICENSE (int)	PHP License information. See also the » license faq.
INFO_ALL (int)	Shows all of the above. This is the default value.

INI mode constants

Constants	Description
INI_USER (int)	Entry can be set in user scripts (like with ini_set()) or in the Windows registry. Entry can be set in *.user.ini*
INI_PERDIR (int)	Entry can be set in *php.ini*, *.htaccess*, *httpd.conf* or *.user.ini*
INI_SYSTEM (int)	Entry can be set in *php.ini* or *httpd.conf*
INI_ALL (int)	Entry can be set anywhere

Assert constants, these values are used to set the assertion options in assert_options().

assert() constants

Constants	Description
ASSERT_ACTIVE (int)	Enable assert() evaluation. **Warning** This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

ASSERT_CALLBACK (int)	Callback to call on failed assertions. **Warning** This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.
ASSERT_BAIL (int)	Terminate execution on failed assertions. **Warning** This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.
ASSERT_EXCEPTION (int)	Throws an AssertionError for each failed assertion **Warning** This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.
ASSERT_WARNING (int)	Issues a PHP warning for each failed assertion **Warning** This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.
ASSERT_QUIET_EVAL (int)	Disable `error_reporting` during assertion expression evaluation. **Warning** This feature was *REMOVED* as of PHP 8.0.0.

The following constants are only available if the host operating system is Windows, and can tell different versioning information so its possible to detect various features and make use of them.

Windows specific constants

Constants	Description
PHP_WINDOWS_VERSION_MAJOR (int)	The major version of Windows, this can be either `4` (NT4/Me/98/95), `5` (XP/2003 R2/2003/2000) or `6` (Vista/2008/7/8/8.1).
PHP_WINDOWS_VERSION_MINOR (int)	The minor version of Windows, this can be either `0` (Vista/2008/2000/NT4/95), `1` (XP), `2` (2003 R2/2003/XP x64), `10` (98) or `90` (ME).
PHP_WINDOWS_VERSION_BUILD (int)	The Windows build number (for example, Windows Vista with SP1 applied is build 6001)
PHP_WINDOWS_VERSION_PLATFORM (int)	The platform that PHP currently is running on, this value is `2` on Windows Vista/XP/2000/NT4, Server 2008/2003 and on Windows ME/98/95 this value is `1`.
PHP_WINDOWS_VERSION_SP_MAJOR (int)	The major version of the service pack installed, this value is `0` if no service pack is installed. For example, Windows XP with service pack 3 installed will make this value `3`.
PHP_WINDOWS_VERSION_SP_MINOR (int)	The minor version of the service pack installed, this value is `0` if no service pack is installed.

PHP_WINDOWS_VERSION_SUITEMASK (int)	The suitemask is a bitmask that can tell if various features of Windows is installed, see the table below for possible bitfield values.
PHP_WINDOWS_VERSION_PRODUCTTYPE (int)	This contains the value used to determine the PHP_WINDOWS_NT_* constants. This value may be one of the PHP_WINDOWS_NT_* constants indicating the platform type.
PHP_WINDOWS_NT_DOMAIN_CONTROLLER (int)	This is a domain controller
PHP_WINDOWS_NT_SERVER (int)	This is a server system (eg. Server 2008/2003/2000), note that if this is a domain controller its reported as PHP_WINDOWS_NT_DOMAIN_CONTROLLER.
PHP_WINDOWS_NT_WORKSTATION (int)	This is a workstation system (eg. Vista/XP/2000/NT4)

This table shows a list of features that can be checked for using the PHP_WINDOWS_VERSION_SUITEMASK bitmask.

Windows suitemask bitfields

Bits	Description
0x00000004	Microsoft BackOffice components are installed.
0x00000400	Windows Server 2003, Web Edition is installed.
0x00004000	Windows Server 2003, Compute Cluster Edition is installed.
0x00000080	Windows Server 2008 Datacenter, Windows Server 2003, Datacenter Edition or Windows 2000 Datacenter Server is installed.
0x00000002	Windows Server 2008 Enterprise, Windows Server 2003, Enterprise Edition, Windows 2000 Advanced Server, or Windows NT Server 4.0 Enterprise Edition is installed.
0x00000040	Windows XP Embedded is installed.
0x00000200	Windows Vista Home Premium, Windows Vista Home Basic, or Windows XP Home Edition is installed.
0x00000100	Remote Desktop is supported, but only one interactive session is supported. This value is set unless the system is running in application server mode.
0x00000001	Microsoft Small Business Server was once installed on the system, but may have been upgraded to another version of Windows.
0x00000020	Microsoft Small Business Server is installed with the restrictive client license in force.
0x00002000	Windows Storage Server 2003 R2 or Windows Storage Server 2003 is installed.
0x00000010	Terminal Services is installed. This value is always set. If this value is set but 0x00000100 is not set, then the system is running in application server mode.
0x00008000	Windows Home Server is installed.

PHP Options/Info Functions

Table of Contents

assert

assert

(PHP 4, PHP 5, PHP 7, PHP 8)

assert — Checks an assertion

Description

function **assert**(mixed $assertion, Throwable|string|null $description = null): bool

assert() allows for the definition of expectations: assertions that take effect in development and testing environments, but are optimised away to have zero cost in production.

Assertions can be used to aid debugging. One use case for them is to act as sanity-checks for preconditions that should always be `true` and that if they aren't upheld this indicates some programming errors. Another use case is to ensure the presence of certain features like extension functions or certain system limits and features.

As assertions can be configured to be eliminated, they should *not* be used for normal runtime operations like input parameter checks. As a rule of thumb code should behave as expected even if assertion checking is deactivated.

assert() will check that the expectation given in `assertion` holds. If not, and thus the result is `false`, it will take the appropriate action depending on how **assert()** was configured.

The behaviour of **assert()** is dictated by the following INI settings:

Assert Configure Options

Name	Default	Description	Changelog
zend.assertions	`1`	• `1`: generate and execute code (development mode) • `0`: generate code but jump around it at runtime • `-1`: do not generate code (production mode)	
assert.active	`true`	If `false`, **assert()** does not check the expectation and returns `true`, unconditionally.	Deprecated as of PHP 8.3.0.
assert.callback	`null`	A user defined function to call when an assertion fails. Its signature should be: function **assert_callback**(string `$file`, int `$line`, null `$assertion`, string `$description = ?`): void	Prior to PHP 8.0.0, the signature of the callback should be: function **assert_callback**(string `$file`, int `$line`, string `$assertion`, string `$description = ?`): void Deprecated as of PHP 8.3.0.
assert.exception	`true`	If `true` will throw an AssertionError if the expectation isn't upheld.	Deprecated as of PHP 8.3.0.
assert.bail	`false`	If `true` will abort execution of the PHP script if the expectation isn't upheld.	Deprecated as of PHP 8.3.0.
assert.warning	`true`	If `true`, will emit an `E_WARNING` if the expectation isn't upheld. This INI setting is ineffective if assert.exception is enabled.	Deprecated as of PHP 8.3.0.

Parameters

`assertion`

This is any expression that returns a value, which will be executed and the result is used to indicate whether the assertion succeeded or failed.

> **Warning**
>
> Prior to PHP 8.0.0, if `assertion` was a string it was interpreted as PHP code and executed via eval(). This string would be passed to the callback as the third argument. This behaviour was *DEPRECATED* in PHP 7.2.0, and *REMOVED* in PHP 8.0.0.

`description`

If `description` is an instance of Throwable, it will be thrown only if the `assertion` is executed and fails.

> **Note** :
>
> As of PHP 8.0.0, this is done *prior* to calling the potentially defined assertion callback.

> **Note** :
>
> As of PHP 8.0.0, the object will be thrown regardless of the configuration of assert.exception.

> **Note** :
>
> As of PHP 8.0.0, the assert.bail setting has no effect in this case.

If `description` is a string this message will be used if an exception or a warning is emitted. An optional description that will be included in the failure message if the `assertion` fails.

If `description` is omitted. A default description equal to the source code for the invocation of **assert()** is created at compile time.

Return Values

assert() will always return `true` if at least one of the following is true:

- `zend.assertions=0`
- `zend.assertions=-1`
- `assert.active=0`

If none of the conditions are true **assert()** will return `true` if `assertion` is truthy and `false` otherwise.

Changelog

Version	Description
8.3.0	All `assert.` INI settings have been deprecated.
8.0.0	**assert()** will no longer evaluate string arguments, instead they will be treated like any other argument. `assert($a == $b)` should be used instead of `assert('$a == $b')`. The `assert.quiet_eval` *php.ini* directive and the `ASSERT_QUIET_EVAL` constant have also been removed, as they would no longer have any effect.
8.0.0	If `description` is an instance of Throwable, the object is thrown if the assertion fails, regardless of the value of assert.exception.
8.0.0	If `description` is an instance of Throwable, no user callback is called even if it set.
8.0.0	Declaring a function called `assert()` inside a namespace is no longer allowed, and issues `E_COMPILE_ERROR`.
7.3.0	Declaring a function called `assert()` inside a namespace became deprecated. Such declaration now emits an `E_DEPRECATED`.
7.2.0	Usage of a string as the `assertion` became deprecated. It now emits an `E_DEPRECATED` notice when both assert.active and zend.assertions are set to `1`.

Examples

Example #1 assert() example

```php
<?php
assert(1 > 2);
echo 'Hi!';
```

If assertions are enabled (`zend.assertions=1`) the above example will output:

```
Fatal error: Uncaught AssertionError: assert(1 > 2) in example.php:2
Stack trace:
#0 example.php(2): assert(false, 'assert(1 > 2)')
#1 {main}
  thrown in example.php on line 2
```

If assertions are disabled (`zend.assertions=0` or `zend.assertions=-1`) the above example will output:

```
Hi!
```

Example #2 Using a custom message

```php
<?php
assert(1 > 2, "Expected one to be greater than two");
echo 'Hi!';
```

If assertions are enabled the above example will output:

```
Fatal error: Uncaught AssertionError: Expected one to be greater than two in example.php:2
Stack trace:
#0 example.php(2): assert(false, 'Expected one to...')
#1 {main}
  thrown in example.php on line 2
```

If assertions are disabled the above example will output:

```
Hi!
```

Example #3 Using a custom exception class

```php
<?php
class ArithmeticAssertionError extends AssertionError {}

assert(1 > 2, new ArithmeticAssertionError("Expected one to be greater than two"));
echo 'Hi!';
```

If assertions are enabled the above example will output:

```
Fatal error: Uncaught ArithmeticAssertionError: Expected one to be greater than two in example.php:4
Stack trace:
#0 {main}
  thrown in example.php on line 4
```

If assertions are disabled the above example will output:

```
Hi!
```

See Also

- assert_options() - Set/get the various assert flags

assert_options

assert_options

(PHP 4, PHP 5, PHP 7, PHP 8)

assert_options — Set/get the various assert flags

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.3.0. Relying on this function is highly discouraged.

Description

function **assert_options**(int `$option`, mixed `$value` = ?): mixed

Set the various assert() control options or just query their current settings.

> **Note** :
>
> The use of **assert_options()** is discouraged in favor of setting and getting the *php.ini* directives zend.assertions and assert.exception with ini_set() and ini_get(), respectively.

Parameters

`option`

Assert Options

Option	INI Setting	Default value	Description
ASSERT_ACTIVE	assert.active	1	enable assert() evaluation
ASSERT_EXCEPTION	assert.exception	1	throws an AssertionError for each failed assertions
ASSERT_WARNING	assert.warning	1	issue a PHP warning for each failed assertion
ASSERT_BAIL	assert.bail	0	terminate execution on failed assertions
ASSERT_QUIET_EVAL	assert.quiet_eval	0	disable error_reporting during assertion expression evaluation. Removed as of PHP 8.0.0.
ASSERT_CALLBACK	assert.callback	(`null`)	Callback to call on failed assertions

`value`

An optional new value for the option.

The callback function set via `ASSERT_CALLBACK` or assert.callback should have the following signature:

```
function assert_callback(
    string $file,
    int $line,
    ?string $assertion,
    string $description = ?
): void
```

`file`

The file where assert() has been called.

`line`

The line where assert() has been called.

`assertion`

Prior to PHP 8.0.0, the assertion which has been passed to assert(), but only when the assertion is given as a string. (If the assertion is a boolean condition, this parameter will be an empty string.) As of PHP 8.0.0, this parameter is always `null`.

`description`

The description that has been passed to assert().

Passing an empty string as `value` resets the assert callback.

Return Values

Returns the original setting of any option.

Errors/Exceptions

If `option` is not a valid option a ValueError is thrown.

Changelog

Version	Description
8.3.0	**assert_options()** is now deprecated.
8.0.0	If `option` is not a valid option, a ValueError is now thrown. Previously `false` was returned.

Examples

Example #1 assert_options() example

```php
<?php
// This is our function to handle
// assert failures
function assert_failure($file, $line, $assertion, $message)
{
    echo "The assertion $assertion in $file on line $line has failed: $message";
}

// This is our test function
function test_assert($parameter)
{
    assert(is_bool($parameter));
}

// Set our assert options
assert_options(ASSERT_ACTIVE,   true);
assert_options(ASSERT_BAIL,     true);
assert_options(ASSERT_WARNING,  false);
assert_options(ASSERT_CALLBACK, 'assert_failure');

// Make an assert that would fail
test_assert(1);

// This is never reached due to ASSERT_BAIL
// being true
echo 'Never reached';
?>
```

See Also

- assert() - Checks an assertion

cli_get_process_title

cli_get_process_title

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

cli_get_process_title — Returns the current process title

Description

function **cli_get_process_title**(): ?string

Returns the current process title, as set by cli_set_process_title(). Note that this may not exactly match what is shown in **ps** or **top**, depending on your operating system.

This function is available only in CLI mode.

Parameters

This function has no parameters.

Return Values

Return a string with the current process title or `null` on error.

Errors/Exceptions

An `E_WARNING` will be generated if the operating system is unsupported.

Examples

Example #1 cli_get_process_title() example

```
<?php
echo "Process title: " . cli_get_process_title() . "\n";
?>
```

See Also

- cli_set_process_title() - Sets the process title

cli_set_process_title

cli_set_process_title

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

cli_set_process_title — Sets the process title

Description

function **cli_set_process_title**(string `$title`): bool

Sets the process title visible in tools such as **top** and **ps**. This function is available only in CLI mode.

Parameters

`title`

The new title.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

An `E_WARNING` will be generated if the operating system is unsupported.

Changelog

Version	Description
8.5.0	**cli_set_process_title()** will now emit an `E_WARNING` when setting a process title that is too long; previously the title would be truncated.

Examples

```
<?php
$title = "My Amazing PHP Script";
$pid = getmypid(); // you can use this to see your process title in ps

if (!cli_set_process_title($title)) {
    echo "Unable to set process title for PID $pid...\n";
    exit(1);
} else {
    echo "The process title '$title' for PID $pid has been set for your process!\n";
    sleep(5);
```

```
}
?>
```

See Also

- cli_get_process_title() - Returns the current process title

dl

dl

(PHP 4, PHP 5, PHP 7, PHP 8)

dl — Loads a PHP extension at runtime

Description

function **dl**(string `$extension_filename`): bool

Loads the PHP extension given by the parameter `extension_filename`.

Use extension_loaded() to test whether a given extension is already available or not. This works on both built-in extensions and dynamically loaded ones (either through *php.ini* or **dl()**).

> **Warning**
>
> This function is only available for the CLI and embed SAPIs, and the CGI SAPI when run from the command line.

Parameters

`extension_filename`

This parameter is *only* the filename of the extension to load which also depends on your platform. For example, the sockets extension (if compiled as a shared module, not the default!) would be called *sockets.so* on Unix platforms whereas it is called *php_sockets.dll* on the Windows platform.

The directory where the extension is loaded from depends on your platform:

Windows - If not explicitly set in the *php.ini*, the extension is loaded from *C:\php5* by default.

Unix - If not explicitly set in the *php.ini*, the default extension directory depends on

- whether PHP has been built with `--enable-debug` or not
- whether PHP has been built with ZTS (Zend Thread Safety) support or not
- the current internal `ZEND_MODULE_API_NO` (Zend internal module API number, which is basically the date on which a major module API change happened, e.g. `20010901`)

Taking into account the above, the directory then defaults to `<install-dir>/lib/php/extensions/ <debug-or-not>-<zts-or-not>-ZEND_MODULE_API_NO`, e.g. */usr/local/php/lib/php/extensions/debug-non-zts-20010901* or */usr/local/php/lib/php/extensions/no-debug-zts-20010901*.

Return Values

Returns `true` on success or `false` on failure. If the functionality of loading modules is not available or has been disabled (by setting enable_dl off in *php.ini*) an `E_WARNING` is emitted and `false` is returned. If **dl()** fails because the specified library couldn't be loaded, in addition to `false` an `E_WARNING` message is emitted.

Examples

Example #1 dl() examples

```
<?php
// Example loading an extension based on OS
if (!extension_loaded('sqlite')) {
    if (strtoupper(substr(PHP_OS, 0, 3)) === 'WIN') {
        dl('php_sqlite.dll');
    } else {
```

```
        dl('sqlite.so');
    }
}

// Or using PHP_SHLIB_SUFFIX constant
if (!extension_loaded('sqlite')) {
    $prefix = (PHP_SHLIB_SUFFIX === 'dll') ? 'php_' : '';
    dl($prefix . 'sqlite.' . PHP_SHLIB_SUFFIX);
}
?>
```

Notes

> **Note** :
>
> **dl()** is case sensitive on Unix platforms.

See Also

- Extension Loading Directives
- extension_loaded() - Find out whether an extension is loaded

extension_loaded

extension_loaded

(PHP 4, PHP 5, PHP 7, PHP 8)

extension_loaded — Find out whether an extension is loaded

Description

function **extension_loaded**(string `$extension`): bool

Finds out whether the extension is loaded.

Parameters

`extension`

The extension name. This parameter is case-insensitive.

You can see the names of various extensions by using phpinfo() or if you're using the `CGI` or `CLI` version of PHP you can use the **-m** switch to list all available extensions:

```
$ php -m
[PHP Modules]
xml
tokenizer
standard
sockets
session
posix
pcre
overload
mysql
mbstring
ctype

[Zend Modules]
```

Return Values

Returns `true` if the extension identified by `extension` is loaded, `false` otherwise.

Examples

Example #1 extension_loaded() example

```
<?php
if (!extension_loaded('gd')) {
    if (!dl('gd.so')) {
        exit;
    }
}
?>
```

See Also

- get_loaded_extensions() - Returns an array with the names of all modules compiled and loaded
- get_extension_funcs() - Returns an array with the names of the functions of a module
- phpinfo() - Outputs information about PHP's configuration
- dl() - Loads a PHP extension at runtime
- function_exists() - Return true if the given function has been defined

gc_collect_cycles

gc_collect_cycles

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_collect_cycles — Forces collection of any existing garbage cycles

Description

function **gc_collect_cycles**(): int

Forces collection of any existing garbage cycles.

Parameters

This function has no parameters.

Return Values

Returns number of collected cycles.

Changelog

Version	Description
8.5.0	The return value no longer includes strings and resources that were indirectly collected through cycles.

See Also

- Garbage Collection

gc_disable

gc_disable

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_disable — Deactivates the circular reference collector

Description

function **gc_disable**(): void

Deactivates the circular reference collector, setting zend.enable_gc to `0`.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- Garbage Collection

gc_enable

gc_enable

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_enable — Activates the circular reference collector

Description

function **gc_enable**(): void

Activates the circular reference collector, setting zend.enable_gc to 1.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- Garbage Collection

gc_enabled

gc_enabled

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

gc_enabled — Returns status of the circular reference collector

Description

function **gc_enabled**(): bool

Returns status of the circular reference collector.

Parameters

This function has no parameters.

Return Values

Returns `true` if the garbage collector is enabled, `false` otherwise.

Examples

Example #1 A gc_enabled() example

```
<?php
if(gc_enabled()) gc_collect_cycles();
?>
```

See Also

- Garbage Collection

gc_mem_caches

gc_mem_caches

(PHP 7, PHP 8)

gc_mem_caches — Reclaims memory used by the Zend Engine memory manager

Description

function **gc_mem_caches**(): int

Reclaims memory used by the Zend Engine memory manager.

Parameters

This function has no parameters.

Return Values

Returns the number of bytes freed.

See Also

- Garbage Collection

gc_status

gc_status

(PHP 7 >= 7.3.0, PHP 8)

gc_status — Gets information about the garbage collector

Description

function **gc_status**(): array

Gets information about the current state of the garbage collector.

Parameters

This function has no parameters.

Return Values

Returns an associative array with the following elements:

- `"runs"`
- `"collected"`
- `"threshold"`
- `"roots"`
- `"running"`
- `"protected"`
- `"full"`
- `"buffer_size"`
- `"application_time"`
- `"collector_time"`
- `"destructor_time"`
- `"free_time"`

Changelog

Version	Description
8.3.0	**gc_status()** now returns the following additional fields: `"running"`, `"protected"`, `"full"`, `"buffer_size"`, `"application_time"`, `"collector_time"`, `"destructor_time"`, and `"free_time"`.

Examples

Example #1 gc_status() Usage

```php
<?php

// create object tree that needs gc collection
$a = new stdClass();
$a->b = [];
for ($i = 0; $i < 100000; $i++) {
    $b = new stdClass();
    $b->a = $a;
    $a->b[] = $b;
}
unset($a);
unset($b);
gc_collect_cycles();

var_dump(gc_status());
```

The above example will output something similar to:

```
array(4) {
  ["runs"]=>
  int(5)
  ["collected"]=>
  int(100002)
  ["threshold"]=>
  int(50001)
  ["roots"]=>
  int(0)
}
```

Output of the above example in PHP 8.3 is similar to:

```
array(12) {
  ["running"]=>
  bool(false)
  ["protected"]=>
  bool(false)
  ["full"]=>
  bool(false)
  ["runs"]=>
  int(5)
  ["collected"]=>
  int(100002)
  ["threshold"]=>
  int(50001)
  ["buffer_size"]=>
  int(131072)
  ["roots"]=>
  int(0)
  ["application_time"]=>
  float(0.031182458)
  ["collector_time"]=>
  float(0.020106291)
  ["destructor_time"]=>
  float(0)
```

```
  ["free_time"]=>
  float(0.003707167)
}
```

See Also

- Garbage Collection

get_cfg_var

get_cfg_var

(PHP 4, PHP 5, PHP 7, PHP 8)

get_cfg_var — Gets the value of a PHP configuration option

Description

function **get_cfg_var**(string `$option`): string|array|false

Gets the value of a PHP configuration `option`.

This function will not return configuration information set when the PHP was compiled, or read from an Apache configuration file.

To check whether the system is using a configuration file, try retrieving the value of the cfg_file_path configuration setting. If this is available, a configuration file is being used.

Parameters

`option`

The configuration option name.

Return Values

Returns the current value of the PHP configuration variable specified by `option`, or **`false`** if an error occurs.

See Also

- ini_get() - Gets the value of a configuration option
- ini_get_all() - Gets all configuration options

get_current_user

get_current_user

(PHP 4, PHP 5, PHP 7, PHP 8)

get_current_user — Gets the name of the owner of the current PHP script

Description

function **get_current_user**(): string

Returns the name of the owner of the current PHP script.

Parameters

This function has no parameters.

Return Values

Returns the username as a string.

Examples

Example #1 get_current_user() example

```php
<?php
echo 'Current script owner: ' . get_current_user();
?>
```

The above example will output something similar to:

```
Current script owner: SYSTEM
```

See Also

- getmyuid() - Gets PHP script owner's UID
- getmygid() - Get PHP script owner's GID
- getmypid() - Gets PHP's process ID
- getmyinode() - Gets the inode of the current script
- getlastmod() - Gets time of last page modification

get_defined_constants

get_defined_constants

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

get_defined_constants — Returns an associative array with the names of all the constants and their values

Description

function **get_defined_constants**(bool `$categorize = false`): array

Returns the names and values of all currently defined constants. This includes constants defined by extensions and those created using the define() function. However, it excludes magic constants, which resolve at compile time.

Parameters

`categorize`

Causing this function to return a multi-dimensional array with categories in the keys of the first dimension and constants and their values in the second dimension.

```php
<?php
define("MY_CONSTANT", 1);
print_r(get_defined_constants(true));
?>
```

The above example will output something similar to:

```
Array
(
    [Core] => Array
        (
            [E_ERROR] => 1
            [E_WARNING] => 2
            [E_PARSE] => 4
            [E_NOTICE] => 8
            [E_CORE_ERROR] => 16
            [E_CORE_WARNING] => 32
            [E_COMPILE_ERROR] => 64
            [E_COMPILE_WARNING] => 128
            [E_USER_ERROR] => 256
            [E_USER_WARNING] => 512
            [E_USER_NOTICE] => 1024
            [E_ALL] => 30719
```

```
            [TRUE] => 1
        )

    [pcre] => Array
        (
            [PREG_PATTERN_ORDER] => 1
            [PREG_SET_ORDER] => 2
            [PREG_OFFSET_CAPTURE] => 256
            [PREG_SPLIT_NO_EMPTY] => 1
            [PREG_SPLIT_DELIM_CAPTURE] => 2
            [PREG_SPLIT_OFFSET_CAPTURE] => 4
            [PREG_GREP_INVERT] => 1
        )

    [user] => Array
        (
            [MY_CONSTANT] => 1
        )

)
```

Return Values

Returns an array of constant name => constant value array, optionally grouped by extension name registering the constant.

Examples

Example #1 get_defined_constants() Example

```php
<?php
print_r(get_defined_constants());
?>
```

The above example will output something similar to:

```
Array
(
    [E_ERROR] => 1
    [E_WARNING] => 2
    [E_PARSE] => 4
    [E_NOTICE] => 8
    [E_CORE_ERROR] => 16
    [E_CORE_WARNING] => 32
    [E_COMPILE_ERROR] => 64
    [E_COMPILE_WARNING] => 128
    [E_USER_ERROR] => 256
    [E_USER_WARNING] => 512
    [E_USER_NOTICE] => 1024
    [E_ALL] => 30719
    [TRUE] => 1
)
```

See Also

- defined() - Checks whether a constant with the given name exists
- constant() - Returns the value of a constant
- get_loaded_extensions() - Returns an array with the names of all modules compiled and loaded
- get_defined_functions() - Returns an array of all defined functions
- get_defined_vars() - Returns an array of all defined variables

get_extension_funcs

get_extension_funcs

(PHP 4, PHP 5, PHP 7, PHP 8)

get_extension_funcs — Returns an array with the names of the functions of a module

Description

function **get_extension_funcs**(string `$extension`): array|false

This function returns the names of all the functions defined in the module indicated by `extension`.

Parameters

`extension`

The module name.

> **Note** :
>
> This parameter must be in *lowercase*.

Return Values

Returns an array with all the functions, or **`false`** if `extension` is not a valid extension.

Examples

Example #1 Prints the XML functions

```php
<?php
print_r(get_extension_funcs("xml"));
?>
```

The above example will output something similar to:

```
Array
(
    [0] => xml_parser_create
    [1] => xml_parser_create_ns
    [2] => xml_set_object
    [3] => xml_set_element_handler
    [4] => xml_set_character_data_handler
    [5] => xml_set_processing_instruction_handler
    [6] => xml_set_default_handler
    [7] => xml_set_unparsed_entity_decl_handler
    [8] => xml_set_notation_decl_handler
    [9] => xml_set_external_entity_ref_handler
    [10] => xml_set_start_namespace_decl_handler
    [11] => xml_set_end_namespace_decl_handler
    [12] => xml_parse
    [13] => xml_parse_into_struct
    [14] => xml_get_error_code
    [15] => xml_error_string
    [16] => xml_get_current_line_number
    [17] => xml_get_current_column_number
    [18] => xml_get_current_byte_index
    [19] => xml_parser_free
    [20] => xml_parser_set_option
    [21] => xml_parser_get_option
)
```

See Also

- get_loaded_extensions() - Returns an array with the names of all modules compiled and loaded
- ReflectionExtension::getFunctions() - Gets extension functions

get_include_path

get_include_path

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

get_include_path — Gets the current include_path configuration option

Description

function **get_include_path**(): string|false

Gets the current include_path configuration option value.

Parameters

This function has no parameters.

Return Values

Returns the path, as a string, or `false` on failure.

Examples

Example #1 get_include_path() example

```
<?php
echo get_include_path();

// Or using ini_get()
echo ini_get('include_path');
?>
```

See Also

- ini_get() - Gets the value of a configuration option
- restore_include_path() - Restores the value of the include_path configuration option
- set_include_path() - Sets the include_path configuration option
- include - include

get_included_files

get_included_files

(PHP 4, PHP 5, PHP 7, PHP 8)

get_included_files — Returns an array with the names of included or required files

Description

function **get_included_files**(): array

Gets the names of all files that have been included using include, include_once, require or require_once.

Parameters

This function has no parameters.

Return Values

Returns an array of the names of all files.

The script originally called is considered an "included file," so it will be listed together with the files referenced by include and family.

Files that are included or required multiple times only show up once in the returned array.

Examples

Example #1 get_included_files() example

```php
<?php
// This file is abc.php

include 'test1.php';
include_once 'test2.php';
require 'test3.php';
require_once 'test4.php';

$included_files = get_included_files();

foreach ($included_files as $filename) {
    echo "$filename\n";
}

?>
```

The above example will output:

```
/path/to/abc.php
/path/to/test1.php
/path/to/test2.php
/path/to/test3.php
/path/to/test4.php
```

See Also

- include - include
- include_once - include_once
- require - require
- require_once - require_once
- get_required_files() - Alias of get_included_files

get_loaded_extensions

get_loaded_extensions

(PHP 4, PHP 5, PHP 7, PHP 8)

get_loaded_extensions — Returns an array with the names of all modules compiled and loaded

Description

function **get_loaded_extensions**(bool `$zend_extensions = false`): array

This function returns the names of all the modules compiled and loaded in the PHP interpreter.

Parameters

`zend_extensions`

Only return Zend extensions, if not then regular extensions, like mysqli are listed. Defaults to `false` (return regular extensions).

Return Values

Returns an indexed array of all the modules names.

Examples

Example #1 get_loaded_extensions() Example

```php
<?php
print_r(get_loaded_extensions());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => Core
    [1] => date
    [2] => libxml
    [3] => pcre
    [4] => sqlite3
    [5] => zlib
    [6] => ctype
    [7] => dom
    [8] => fileinfo
    [9] => filter
    [10] => hash
    [11] => json
    [12] => mbstring
    [13] => SPL
    [14] => PDO
    [15] => session
    [16] => posix
    [17] => Reflection
    [18] => standard
    [19] => SimpleXML
    [20] => pdo_sqlite
    [21] => Phar
    [22] => tokenizer
    [23] => xml
    [24] => xmlreader
    [25] => xmlwriter
    [26] => gmp
    [27] => iconv
    [28] => intl
    [29] => bcmath
    [30] => sodium
    [31] => Zend OPcache
)
```

See Also

- get_extension_funcs() - Returns an array with the names of the functions of a module
- extension_loaded() - Find out whether an extension is loaded
- dl() - Loads a PHP extension at runtime
- phpinfo() - Outputs information about PHP's configuration

get_magic_quotes_gpc

get_magic_quotes_gpc

(PHP 4, PHP 5, PHP 7)

get_magic_quotes_gpc — Gets the current configuration setting of magic_quotes_gpc

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.4.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **get_magic_quotes_gpc**(): false

Always returns `false`.

Parameters

This function has no parameters.

Return Values

Always returns `false`.

Changelog

Version	Description
8.0.0	Removed this function.
7.4.0	Deprecated this function.

See Also

- addslashes() - Quote string with slashes
- stripslashes() - Un-quotes a quoted string
- get_magic_quotes_runtime() - Gets the current active configuration setting of magic_quotes_runtime
- ini_get() - Gets the value of a configuration option

get_magic_quotes_runtime

get_magic_quotes_runtime

(PHP 4, PHP 5, PHP 7)

get_magic_quotes_runtime — Gets the current active configuration setting of magic_quotes_runtime

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.4.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **get_magic_quotes_runtime**(): false

Always returns `false`.

Parameters

This function has no parameters.

Return Values

Always returns `false`.

Changelog

Version	Description
8.0.0	Removed this function.
7.4.0	Deprecated this function.

See Also

- get_magic_quotes_gpc() - Gets the current configuration setting of magic_quotes_gpc

get_required_files

get_required_files

(PHP 4, PHP 5, PHP 7, PHP 8)

get_required_files — Alias of get_included_files()

Description

This function is an alias of: get_included_files().

get_resources

get_resources

(PHP 7, PHP 8)

get_resources — Returns active resources

Description

function **get_resources**(?string `$type = null`): array

Returns an array of all currently active resources, optionally filtered by resource type.

> **Note** :
>
> This function is meant for debugging and testing purposes. It is not supposed to be used in production environments, especially not to access or even manipulate resources which are normally not accessible (e.g. the underlying stream resource of SplFileObject instances).

Parameters

`type`

If defined, this will cause **get_resources()** to only return resources of the given type. A list of resource types is available.

If the string `Unknown` is provided as the type, then only resources that are of an unknown type will be returned.

If omitted, all resources will be returned.

Return Values

Returns an array of currently active resources, indexed by resource number.

Changelog

Version	Description
8.0.0	`type` is nullable now.

Examples

Example #1 Unfiltered get_resources()

```php
<?php
$fp = tmpfile();
var_dump(get_resources());
?>
```

The above example will output something similar to:

```
array(1) {
  [1]=>
  resource(1) of type (stream)
}
```

Example #2 Filtered get_resources()

```php
<?php
$fp = tmpfile();
var_dump(get_resources('stream'));
var_dump(get_resources('curl'));
?>
```

The above example will output something similar to:

```
array(1) {
  [1]=>
  resource(1) of type (stream)
}
array(0) {
}
```

See Also

- get_loaded_extensions() - Returns an array with the names of all modules compiled and loaded
- get_defined_constants() - Returns an associative array with the names of all the constants and their values
- get_defined_functions() - Returns an array of all defined functions
- get_defined_vars() - Returns an array of all defined variables

getenv

getenv

(PHP 4, PHP 5, PHP 7, PHP 8)

getenv — Gets the value of a single or all environment variables

Description

function **getenv**(?string `$name` = **`null`**, bool `$local_only` = **`false`**): string|array|false

Gets the value of a single or all environment variables.

You can see a list of all the environmental variables by using phpinfo(). Many of these variables are listed within » RFC 3875, specifically section 4.1, "Request Meta-Variables".

Parameters

`name`

The variable name as a string or **`null`**.

`local_only`

When set to **`true`**, only local environment variables are returned, set by the operating system or putenv. It only has an effect when `name` is a string.

Return Values

Returns the value of the environment variable `name`, or **`false`** if the environment variable `name` does not exist. If `name` is **`null`**, all environment variables are returned as an associative array.

Changelog

Version	Description
8.0.0	The `name` is now nullable.
7.1.0	The `name` can now be omitted to retrieve an associative array of all environment variables.
7.0.9	The `local_only` parameter has been added.

Examples

Example #1 getenv() Example

```php
<?php
// Example use of getenv()
$ip = getenv('REMOTE_ADDR');

// Or simply use a Superglobal ($_SERVER or $_ENV)
$ip = $_SERVER['REMOTE_ADDR'];

// Safely get the value of an environment variable, ignoring whether
// or not it was set by a SAPI or has been changed with putenv
$ip = getenv('REMOTE_ADDR', true) ?: getenv('REMOTE_ADDR')
?>
```

Notes

> **Warning**
>
> If PHP is running in a SAPI such as Fast CGI, this function will always return the value of an environment variable set by the SAPI, even if putenv() has been used to set a local environment variable of the same name. Use the `local_only` parameter to return the value of locally-set environment variables.

See Also

- putenv() - Sets the value of an environment variable
- apache_getenv() - Get an Apache subprocess_env variable
- Superglobals

getlastmod

getlastmod

(PHP 4, PHP 5, PHP 7, PHP 8)

getlastmod — Gets time of last page modification

Description

function **getlastmod**(): int|false

Gets the time of the last modification of the main script of execution.

If you're interested in getting the last modification time of a different file, consider using filemtime().

Parameters

This function has no parameters.

Return Values

Returns the time of the last modification of the current page. The value returned is a Unix timestamp, suitable for feeding to date(). Returns `false` on error.

Examples

Example #1 getlastmod() example

```php
<?php
// outputs e.g. 'Last modified: March 04 1998 20:43:59.'
echo "Last modified: " . date ("F d Y H:i:s.", getlastmod());
?>
```

See Also

- date() - Format a Unix timestamp
- getmyuid() - Gets PHP script owner's UID

- getmygid() - Get PHP script owner's GID
- get_current_user() - Gets the name of the owner of the current PHP script
- getmyinode() - Gets the inode of the current script
- getmypid() - Gets PHP's process ID
- filemtime() - Gets file modification time

getmygid

getmygid

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

getmygid — Get PHP script owner's GID

Description

function **getmygid**(): int|false

Gets the group ID of the current script.

Parameters

This function has no parameters.

Return Values

Returns the group ID of the current script, or `false` on error.

See Also

- getmyuid() - Gets PHP script owner's UID
- getmypid() - Gets PHP's process ID
- get_current_user() - Gets the name of the owner of the current PHP script
- getmyinode() - Gets the inode of the current script
- getlastmod() - Gets time of last page modification

getmyinode

getmyinode

(PHP 4, PHP 5, PHP 7, PHP 8)

getmyinode — Gets the inode of the current script

Description

function **getmyinode**(): int|false

Gets the inode of the current script.

Parameters

This function has no parameters.

Return Values

Returns the current script's inode as an integer, or `false` on error.

See Also

- getmygid() - Get PHP script owner's GID
- getmyuid() - Gets PHP script owner's UID
- getmypid() - Gets PHP's process ID
- get_current_user() - Gets the name of the owner of the current PHP script
- getlastmod() - Gets time of last page modification

getmypid

getmypid

(PHP 4, PHP 5, PHP 7, PHP 8)

getmypid — Gets PHP's process ID

Description

function **getmypid**(): int|false

Gets the current PHP process ID.

Parameters

This function has no parameters.

Return Values

Returns the current PHP process ID, or `false` on error.

Notes

> **Warning**
>
> Process IDs are not unique, thus they are a weak entropy source. We recommend against relying on pids in security-dependent contexts.

See Also

- getmygid() - Get PHP script owner's GID
- getmyuid() - Gets PHP script owner's UID
- get_current_user() - Gets the name of the owner of the current PHP script
- getmyinode() - Gets the inode of the current script
- getlastmod() - Gets time of last page modification

getmyuid

getmyuid

(PHP 4, PHP 5, PHP 7, PHP 8)

getmyuid — Gets PHP script owner's UID

Description

function **getmyuid**(): int|false

Gets the user ID of the current script.

Parameters

This function has no parameters.

Return Values

Returns the user ID of the current script, or `false` on error.

See Also

- getmygid() - Get PHP script owner's GID
- getmypid() - Gets PHP's process ID
- get_current_user() - Gets the name of the owner of the current PHP script
- getmyinode() - Gets the inode of the current script
- getlastmod() - Gets time of last page modification

getopt

getopt

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

getopt — Gets options from the command line argument list

Description

function **getopt**(string `$short_options`, array `$long_options = []`, int `&$rest_index = null`): array|false

Parses options passed to the script.

Parameters

`short_options`

> Each character in this string will be used as option characters and matched against options passed to the script starting with a single hyphen (`-`).
>
> For example, an option string `"x"` recognizes an option `-x`.
>
> Only a-z, A-Z and 0-9 are allowed.

`long_options`

> An array of options. Each element in this array will be used as option strings and matched against options passed to the script starting with two hyphens (`--`).
>
> For example, an longopts element `"opt"` recognizes an option `--opt`.

`rest_index`

> If the `rest_index` parameter is present, then the index where argument parsing stopped will be written to this variable.

The `short_options` parameter may contain the following elements:

- Individual characters (do not accept values)
- Characters followed by a colon (parameter requires value)
- Characters followed by two colons (optional value)

Option values are the first argument after the string. If a value is required, it does not matter whether the value has leading white space or not. See note.

> **Note** :
>
> Optional values do not accept `" "` (space) as a separator.

The `long_options` array values may contain:

- String (parameter does not accept any value)
- String followed by a colon (parameter requires value)
- String followed by two colons (optional value)

> **Note** :
>
> The format for the `short_options` and `long_options` is almost the same, the only difference is that `long_options` takes an array of options (where each element is the option) whereas `short_options` takes a string (where each character is the option).

Return Values

This function will return an array of option / argument pairs, or `false` on failure.

> **Note** :
>
> The parsing of options will end at the first non-option found, anything that follows is discarded.

Changelog

Version	Description
7.1.0	Added the `rest_index` parameter.

Examples

Example #1 **getopt()** example: The basics

```
<?php
// Script example.php
$options = getopt("f:hp:");
var_dump($options);
?>
```

```
shell> php example.php -fvalue -h
```

The above example will output:

```
array(2) {
  ["f"]=>
  string(5) "value"
  ["h"]=>
  bool(false)
}
```

Example #2 **getopt()** example: Introducing long options

```
<?php
// Script example.php
$shortopts  = "";
$shortopts .= "f:";  // Required value
$shortopts .= "v::"; // Optional value
$shortopts .= "abc"; // These options do not accept values

$longopts  = array(
    "required:",     // Required value
    "optional::",    // Optional value
    "option",        // No value
    "opt",           // No value
);
$options = getopt($shortopts, $longopts);
var_dump($options);
?>
```

```
shell> php example.php -f "value for f" -v -a --required value --optional="optional value" --option
```

The above example will output:

```
array(6) {
  ["f"]=>
  string(11) "value for f"
  ["v"]=>
  bool(false)
  ["a"]=>
  bool(false)
  ["required"]=>
  string(5) "value"
  ["optional"]=>
  string(14) "optional value"
  ["option"]=>
```

```
  bool(false)
}
```

Example #3 getopt() example: Passing multiple options as one

```php
<?php
// Script example.php
$options = getopt("abc");
var_dump($options);
?>
```

```
shell> php example.php -aaac
```

The above example will output:

```
array(2) {
  ["a"]=>
  array(3) {
    [0]=>
    bool(false)
    [1]=>
    bool(false)
    [2]=>
    bool(false)
  }
  ["c"]=>
  bool(false)
}
```

Example #4 getopt() example: Using `rest_index`

```php
<?php
// Script example.php
$rest_index = null;
$opts = getopt('a:b:', [], $rest_index);
$pos_args = array_slice($argv, $rest_index);
var_dump($pos_args);
```

```
shell> php example.php -a 1 -b 2 -- test
```

The above example will output:

```
array(1) {
  [0]=>
  string(4) "test"
}
```

See Also

- $argv

getrusage

getrusage

(PHP 4, PHP 5, PHP 7, PHP 8)

getrusage — Gets the current resource usages

Description

function **getrusage**(int `$mode` = 0): array|false

This is an interface to **getrusage(2)**. It gets data returned from the system call.

Parameters

`mode`

If `mode` is 1, getrusage will be called with **`RUSAGE_CHILDREN`**.

Return Values

Returns an associative array containing the data returned from the system call. All entries are accessible by using their documented field names. Returns `false` on failure.

Examples

Example #1 getrusage() example

```php
<?php
$dat = getrusage();
echo $dat["ru_oublock"];       // number of block output operations
echo $dat["ru_inblock"];       // number of block input operations
echo $dat["ru_msgsnd"];        // number of IPC messages sent
echo $dat["ru_msgrcv"];        // number of IPC messages received
echo $dat["ru_maxrss"];        // maximum resident set size
echo $dat["ru_ixrss"];         // integral shared memory size
echo $dat["ru_idrss"];         // integral unshared data size
echo $dat["ru_minflt"];        // number of page reclaims (soft page faults)
echo $dat["ru_majflt"];        // number of page faults (hard page faults)
echo $dat["ru_nsignals"];      // number of signals received
echo $dat["ru_nvcsw"];         // number of voluntary context switches
echo $dat["ru_nivcsw"];        // number of involuntary context switches
echo $dat["ru_nswap"];         // number of swaps
echo $dat["ru_utime.tv_usec"]; // user time used (microseconds)
echo $dat["ru_utime.tv_sec"];  // user time used (seconds)
echo $dat["ru_stime.tv_usec"]; // system time used (microseconds)
echo $dat["ru_stime.tv_sec"];  // system time used (seconds)
?>
```

Notes

> **Note** :
>
> On Windows **getrusage()** will only return the following members:
>
> - `"ru_stime.tv_sec"`
> - `"ru_stime.tv_usec"`
> - `"ru_utime.tv_sec"`
> - `"ru_utime.tv_usec"`
> - `"ru_majflt"` (only if `mode` is **`RUSAGE_SELF`**)
> - `"ru_maxrss"` (only if `mode` is **`RUSAGE_SELF`**)
>
> If **getrusage()** is called with `mode` set to `1` (**`RUSAGE_CHILDREN`**), then resource usage for child processes are collected.

> **Note** :
>
> on BeOS 2000, only the following members are returned:
>
> - `"ru_stime.tv_sec"`

- "ru_stime.tv_usec"
- "ru_utime.tv_sec"
- "ru_utime.tv_usec"

See Also

- Your system's man page on **getrusage(2)**

ini_alter

ini_alter

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_alter — Alias of ini_set()

Description

This function is an alias of: ini_set().

ini_get

ini_get

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_get — Gets the value of a configuration option

Description

function **ini_get**(string `$option`): string|false

Returns the value of the configuration option on success.

Parameters

`option`

The configuration option name.

Return Values

Returns the value of the configuration option as a string on success, or an empty string for `null` values. Returns `false` if the configuration option doesn't exist.

Examples

Example #1 A few ini_get() examples

```php
<?php

/*
Our php.ini contains the following settings:

display_errors = On
opcache.enable_cli = Off
post_max_size = 8M
*/

echo 'display_errors = ' . ini_get('display_errors') . "\n";
echo 'opcache.enable_cli = ' . (int) ini_get('opcache.enable_cli') . "\n";
echo 'post_max_size = ' . ini_get('post_max_size') . "\n";
echo 'post_max_size + 1 = ' . (rtrim(ini_get('post_max_size'), 'KMG') + 1) . "\n";
echo 'post_max_size in bytes = ' . ini_parse_quantity(ini_get('post_max_size'));

?>
```

The above example will output something similar to:

```
display_errors = 1
opcache.enable_cli = 0
post_max_size = 8M
post_max_size+1 = 9
post_max_size in bytes = 8388608
```

Notes

> **Note** : **When querying boolean values**
>
> A boolean ini value of `off` will be returned as an empty string or "0" while a boolean ini value of `on` will be returned as "1". The function can also return the literal string of INI value.

> **Note** : **When querying memory size values**
>
> Many ini memory size values, such as upload_max_filesize, are stored in the *php.ini* file in shorthand notation. **ini_get()** will return the exact string stored in the *php.ini* file and *NOT* its int equivalent. Attempting normal arithmetic functions on these values will not have otherwise expected results. The ini_parse_quantity() function can be used to convert the shorthand notation into bytes.

> **Note** :
>
> **ini_get()** can't read "array" ini options such as `pdo.dsn.*`, and returns `false` in this case.

See Also

- get_cfg_var() - Gets the value of a PHP configuration option
- ini_get_all() - Gets all configuration options
- ini_parse_quantity() - Get interpreted size from ini shorthand syntax
- ini_restore() - Restores the value of a configuration option
- ini_set() - Sets the value of a configuration option

ini_get_all

ini_get_all

(PHP 4 >= 4.2.0, PHP 5, PHP 7, PHP 8)

ini_get_all — Gets all configuration options

Description

function **ini_get_all**(?string `$extension` = `null`, bool `$details` = `true`): array|false

Returns all the registered configuration options.

Parameters

`extension`

An optional extension name. If not `null` or the string `core`, the function returns only options specific for that extension.

`details`

Retrieve details settings or only the current value for each setting. Default is `true` (retrieve details).

Return Values

Returns an associative array with directive name as the array key. Returns `false` and raises an `E_WARNING` level error if the `extension` doesn't exist.

When `details` is **`true`** (default) the array will contain `global_value` (set in *php.ini*), `local_value` (perhaps set with ini_set() or *.htaccess*), and `access` (the access level).

When `details` is **`false`** the value will be the current value of the option.

See the manual section for information on what access levels mean.

> **Note** :
>
> It's possible for a directive to have multiple access levels, which is why `access` shows the appropriate bitmask values.

Examples

Example #1 ini_get_all() examples

```php
<?php
print_r(ini_get_all("pcre"));
print_r(ini_get_all());
?>
```

The above example will output something similar to:

```
Array
(
    [pcre.backtrack_limit] => Array
        (
            [global_value] => 100000
            [local_value] => 100000
            [access] => 7
        )

    [pcre.recursion_limit] => Array
        (
            [global_value] => 100000
            [local_value] => 100000
            [access] => 7
        )

)
Array
(
    [allow_call_time_pass_reference] => Array
        (
            [global_value] => 0
            [local_value] => 0
            [access] => 6
        )

    [allow_url_fopen] => Array
        (
            [global_value] => 1
            [local_value] => 1
            [access] => 4
        )

    ...

)
```

Example #2 Disabling `details`

```php
<?php
print_r(ini_get_all("pcre", false)); // Added in PHP 5.3.0
```

```php
print_r(ini_get_all(null, false)); // Added in PHP 5.3.0
?>
```

The above example will output something similar to:

```
Array
(
    [pcre.backtrack_limit] => 100000
    [pcre.recursion_limit] => 100000
)
Array
(
    [allow_call_time_pass_reference] => 0
    [allow_url_fopen] => 1
    ...
)
```

Notes

> **Note** :
>
> **ini_get_all()** ignores "array" ini options such as `pdo.dsn.*`.

See Also

- How to change configuration settings
- ini_get() - Gets the value of a configuration option
- ini_restore() - Restores the value of a configuration option
- ini_set() - Sets the value of a configuration option
- get_loaded_extensions() - Returns an array with the names of all modules compiled and loaded
- phpinfo() - Outputs information about PHP's configuration
- ReflectionExtension::getINIEntries() - Gets extension ini entries

ini_parse_quantity

ini_parse_quantity

(PHP 8 >= 8.2.0)

ini_parse_quantity — Get interpreted size from ini shorthand syntax

Description

function **ini_parse_quantity**(string `$shorthand`): int

Returns the interpreted size in bytes on success from an ini shorthand.

Parameters

`shorthand`

> Ini shorthand to parse, must be a number followed by an optional multiplier. The following multipliers are supported: `k`/`K` (`1024`), `m`/`M` (`1048576`), `g`/`G` (`1073741824`). The number can be a decimal, hex (prefixed with `0x` or `0X`), octal (prefixed with `0o`, `0O` or `0`) or binary (prefixed with `0b` or `0B`)

Return Values

Returns the interpreted size in bytes as an int.

Errors/Exceptions

If the value cannot be parsed, or an invalid multiplier is used, an `E_WARNING` is raised.

Examples

Example #1 A few ini_parse_quantity() examples

```php
<?php

var_dump(ini_parse_quantity('1024'));
var_dump(ini_parse_quantity('1024M'));
var_dump(ini_parse_quantity('512K'));
var_dump(ini_parse_quantity('0xFFk'));
var_dump(ini_parse_quantity('0b1010k'));
var_dump(ini_parse_quantity('0o1024'));
var_dump(ini_parse_quantity('01024'));
var_dump(ini_parse_quantity('Foobar'));
var_dump(ini_parse_quantity('10F'));

?>
```

The above example will output:

```
int(1024)
int(1073741824)
int(524288)
int(261120)
int(10240)
int(532)
int(532)

Warning: Invalid quantity "Foobar": no valid leading digits, interpreting as "0" for backwards compatibility
int(0)

Warning: Invalid quantity "10F": unknown multiplier "F", interpreting as "10" for backwards compatibility
int(10)
```

See Also

- ini_get() - Gets the value of a configuration option

ini_restore

ini_restore

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_restore — Restores the value of a configuration option

Description

function **ini_restore**(string `$option`): void

Restores a given configuration option to its original value.

Parameters

`option`

The configuration option name.

Return Values

No value is returned.

Examples

Example #1 ini_restore() example

```php
<?php
$setting = 'html_errors';

echo 'Current value for \'' . $setting . '\': ' . ini_get($setting), PHP_EOL;

ini_set($setting, ini_get($setting) ? 0 : 1);
echo 'New value for \'' . $setting . '\': ' . ini_get($setting), PHP_EOL;

ini_restore($setting);
echo 'Original value for \'' . $setting . '\': ' . ini_get($setting), PHP_EOL;
?>
```

The above example will output:

```
Current value for 'html_errors': 1
New value for 'html_errors': 0
Original value for 'html_errors': 1
```

See Also

- ini_get() - Gets the value of a configuration option
- ini_get_all() - Gets all configuration options
- ini_set() - Sets the value of a configuration option

ini_set

ini_set

(PHP 4, PHP 5, PHP 7, PHP 8)

ini_set — Sets the value of a configuration option

Description

function **ini_set**(string `$option`, string|int|float|bool|null `$value`): string|false

Sets the value of the given configuration option. The configuration option will keep this new value during the script's execution, and will be restored at the script's ending.

Parameters

`option`

Not all the available options can be changed using **ini_set()**. There is a list of all available options in the appendix.

`value`

The new value for the option.

Return Values

Returns the old value on success, `false` on failure.

Changelog

Version	Description
8.1.0	`value` now accepts any scalar type (including `null`). Previously, only string values were accepted.

Examples

Example #1 Setting an ini option

```php
<?php
echo ini_get('display_errors');
```

```php
if (!ini_get('display_errors')) {
    ini_set('display_errors', '1');
}

echo ini_get('display_errors');
?>
```

See Also

- get_cfg_var() - Gets the value of a PHP configuration option
- ini_get() - Gets the value of a configuration option
- ini_get_all() - Gets all configuration options
- ini_restore() - Restores the value of a configuration option
- How to change configuration settings

memory_get_peak_usage

memory_get_peak_usage

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

memory_get_peak_usage — Returns the peak of memory allocated by PHP

Description

function **memory_get_peak_usage**(bool `$real_usage = false`): int

Returns the peak of memory, in bytes, that's been allocated to your PHP script.

Parameters

`real_usage`

> Set this to `true` to get the real size of memory allocated from system. If not set or `false` only the memory used by `emalloc()` is reported.

Return Values

Returns the memory peak in bytes.

See Also

- memory_get_usage() - Returns the amount of memory allocated to the PHP script, or reserved from the system by PHP's memory manager
- memory_reset_peak_usage() - Reset the peak memory usage
- memory_limit

memory_get_usage

memory_get_usage

(PHP 4 >= 4.3.2, PHP 5, PHP 7, PHP 8)

memory_get_usage — Returns the amount of memory allocated to the PHP script, or reserved from the system by PHP's memory manager

Description

function **memory_get_usage**(bool `$real_usage = false`): int

Returns the amount of memory, in bytes, that is currently allocated to the PHP script by PHP's memory manager. The reported value is rounded up to the granularity of the allocator, so it is larger than the exact number of bytes that were requested.

Parameters

`real_usage`

> Set this to `true` to get the total amount of memory that PHP's memory manager has reserved from the system, including pages that are not currently in use. This is the value that memory_limit is enforced against. It is not the amount of memory the operating system has given to the PHP process, which is typically much larger.

If not set or `false` only the memory currently allocated to the PHP script is reported.

> **Note** :
>
> PHP only tracks memory allocated through its own memory manager, that is through the internal `emalloc()` function and the related ones. Memory allocated persistently, or directly with `malloc()` by an extension, is not reported, even when `real_usage` is `true`.

Return Values

Returns the memory amount in bytes.

Examples

Example #1 A memory_get_usage() example

```php
<?php

// This is only an example, the numbers below will
// differ depending on the specific system

echo memory_get_usage(), "\n"; // 36640

$a = str_repeat("Hello", 4242);

echo memory_get_usage(), "\n"; // 57960

unset($a);

echo memory_get_usage(), "\n"; // 36744

?>
```

See Also

- memory_get_peak_usage() - Returns the peak of memory allocated by PHP
- memory_limit

memory_reset_peak_usage

memory_reset_peak_usage

(PHP 8 >= 8.2.0)

memory_reset_peak_usage — Reset the peak memory usage

Description

function **memory_reset_peak_usage**(): void

Resets the peak memory usage returned by the memory_get_peak_usage() function.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 memory_reset_peak_usage() example

```php
<?php

var_dump(memory_get_peak_usage());

$a = str_repeat("Hello", 424242);
var_dump(memory_get_peak_usage());
```

```php
unset($a);
memory_reset_peak_usage();

$a = str_repeat("Hello", 2424);
var_dump(memory_get_peak_usage());

?>
```

The above example will output something similar to:

```
int(422440)
int(2508672)
int(399208)
```

See Also

- memory_get_peak_usage() - Returns the peak of memory allocated by PHP

php_ini_loaded_file

php_ini_loaded_file

(PHP 5 >= 5.2.4, PHP 7, PHP 8)

php_ini_loaded_file — Retrieve a path to the loaded php.ini file

Description

function **php_ini_loaded_file**(): string|false

Check if a *php.ini* file is loaded, and retrieve its path.

Parameters

This function has no parameters.

Return Values

The loaded *php.ini* path, or `false` if one is not loaded.

Examples

Example #1 php_ini_loaded_file() example

```php
<?php
$inipath = php_ini_loaded_file();

if ($inipath) {
    echo 'Loaded php.ini: ' . $inipath;
} else {
    echo 'A php.ini file is not loaded';
}
?>
```

The above example will output something similar to:

```
Loaded php.ini: /usr/local/php/php.ini
```

See Also

- php_ini_scanned_files() - Return a list of .ini files parsed from the additional ini dir
- phpinfo() - Outputs information about PHP's configuration
- The configuration file

php_ini_scanned_files

php_ini_scanned_files

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

php_ini_scanned_files — Return a list of .ini files parsed from the additional ini dir

Description

function **php_ini_scanned_files**(): string|false

php_ini_scanned_files() returns a comma-separated list of configuration files parsed after *php.ini*. The directories searched are set by a compile time option and, optionally, by an environment variable at run time: more information can be found in the installation guide.

The returned configuration files include the full path.

Parameters

This function has no parameters.

Return Values

Returns a comma-separated string of .ini files on success. Each comma is followed by a newline. If the configure directive **--with-config-file-scan-dir** wasn't set and the *PHP_INI_SCAN_DIR* environment variable isn't set, `false` is returned. If it was set and the directory was empty, an empty string is returned. If a file is unrecognizable, the file will still make it into the returned string but a PHP error will also result. This PHP error will be seen both at compile time and while using **php_ini_scanned_files()**.

Examples

Example #1 A simple example to list the returned ini files

```php
<?php
if ($filelist = php_ini_scanned_files()) {
    if (strlen($filelist) > 0) {
        $files = explode(',', $filelist);

        foreach ($files as $file) {
            echo "<li>" . trim($file) . "</li>\n";
        }
    }
}
?>
```

See Also

- ini_set() - Sets the value of a configuration option
- phpinfo() - Outputs information about PHP's configuration
- php_ini_loaded_file() - Retrieve a path to the loaded php.ini file

php_sapi_name

php_sapi_name

(PHP 4 >= 4.0.1, PHP 5, PHP 7, PHP 8)

php_sapi_name — Returns the type of interface between web server and PHP

Description

function **php_sapi_name**(): string|false

Returns a lowercase string that describes the type of interface (the Server API, SAPI) that PHP is using. For example, in CLI PHP this string will be "cli" whereas with Apache it may have several different values depending on the exact SAPI used. Possible values are listed below.

Parameters

This function has no parameters.

Return Values

Returns the interface type, as a lowercase string, or `false` on failure.

Although not exhaustive, the possible return values include `apache`, `apache2handler`, `cgi` (until PHP 5.3), `cgi-fcgi`, `cli`, `cli-server`, `embed`, `fpm-fcgi`, `litespeed`, `phpdbg`.

Examples

Example #1 php_sapi_name() example

This example checks for the substring `cgi` because it may also be `cgi-fcgi`.

```
<?php
$sapi_type = php_sapi_name();
if (substr($sapi_type, 0, 3) == 'cgi') {
    echo "You are using CGI PHP\n";
} else {
    echo "You are not using CGI PHP\n";
}
?>
```

Notes

> **Note** : **An alternative approach**
>
> The PHP constant `PHP_SAPI` has the same value as **php_sapi_name()**.

> **Tip**
>
> #### A potential gotcha
>
> The defined SAPI may not be obvious, because for example instead of `apache` it may be defined as `apache2handler`.

See Also

- PHP_SAPI

php_uname

php_uname

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

php_uname — Returns information about the operating system PHP is running on

Description

function **php_uname**(string `$mode` = "a"): string

php_uname() returns a description of the operating system PHP is running on. This is the same string you see at the very top of the phpinfo() output. For the name of just the operating system, consider using the `PHP_OS` constant, but keep in mind this constant will contain the operating system PHP was *built* on.

On some older UNIX platforms, it may not be able to determine the current OS information in which case it will revert to displaying the OS PHP was built on. This will only happen if your uname() library call either doesn't exist or doesn't work.

Parameters

`mode`

`mode` is a single character that defines what information is returned:

- `'a'`: This is the default. Returns the same information as the individual modes `'s'`, `'n'`, `'r'`, `'v'`, `'m'` separated by spaces.

- `'s'`: Operating system name. eg. `FreeBSD`.
- `'n'`: Host name. eg. `localhost.example.com`.
- `'r'`: Release name. eg. `5.1.2-RELEASE`.
- `'v'`: Version information. Varies a lot between operating systems.
- `'m'`: Machine type. eg. `i386`.

Return Values

Returns the description, as a string.

Changelog

Version	Description
8.4.0	Throws a ValueError when an invalid `mode` is specified.

Examples

Example #1 Some php_uname() examples

```
<?php
echo php_uname();
echo PHP_OS;

/* Some possible outputs:
Linux localhost 2.4.21-0.13mdk #1 Fri Mar 14 15:08:06 EST 2003 i686
Linux

FreeBSD localhost 3.2-RELEASE #15: Mon Dec 17 08:46:02 GMT 2001
FreeBSD

Windows NT XN1 5.1 build 2600
WINNT
*/

if (strtoupper(substr(PHP_OS, 0, 3)) === 'WIN') {
    echo 'This is a server using Windows!';
} else {
    echo 'This is a server not using Windows!';
}

?>
```

There are also some related Predefined PHP constants that may come in handy, for example:

Example #2 A few OS related constant examples

```
<?php
// *nix
echo DIRECTORY_SEPARATOR; // /
echo PHP_SHLIB_SUFFIX;    // so
echo PATH_SEPARATOR;      // :

// Win*
echo DIRECTORY_SEPARATOR; // \
echo PHP_SHLIB_SUFFIX;    // dll
echo PATH_SEPARATOR;      // ;
?>
```

See Also

- phpversion() - Gets the current PHP version
- php_sapi_name() - Returns the type of interface between web server and PHP
- phpinfo() - Outputs information about PHP's configuration

phpcredits

phpcredits

(PHP 4, PHP 5, PHP 7, PHP 8)

phpcredits — Prints out the credits for PHP

Description

function **phpcredits**(int `$flags` = `CREDITS_ALL`): true

This function prints out the credits listing the PHP developers, modules, etc. It generates the appropriate HTML codes to insert the information in a page.

Parameters

`flags`

To generate a custom credits page, you may want to use the `flags` parameter.

Pre-defined phpcredits() flags

name	description
CREDITS_ALL	All the credits, equivalent to using: `CREDITS_DOCS` + `CREDITS_GENERAL` + `CREDITS_GROUP` + `CREDITS_MODULES` + `CREDITS_FULLPAGE`. It generates a complete stand-alone HTML page with the appropriate tags.
CREDITS_DOCS	The credits for the documentation team
CREDITS_FULLPAGE	Usually used in combination with the other flags. Indicates that a complete stand-alone HTML page needs to be printed including the information indicated by the other flags.
CREDITS_GENERAL	General credits: Language design and concept, PHP authors and SAPI module.
CREDITS_GROUP	A list of the core developers
CREDITS_MODULES	A list of the extension modules for PHP, and their authors
CREDITS_SAPI	A list of the server API modules for PHP, and their authors

Return Values

Always returns `true`.

Examples

Example #1 Prints the general credits

```
<?php
phpcredits(CREDITS_GENERAL);
?>
```

Example #2 Prints the core developers and the documentation group

```
<?php
phpcredits(CREDITS_GROUP | CREDITS_DOCS | CREDITS_FULLPAGE);
?>
```

Example #3 Printing all the credits

```
<html>
 <head>
  <title>My credits page</title>
 </head>
 <body>
<?php
// some code of your own
phpcredits(CREDITS_ALL - CREDITS_FULLPAGE);
// some more code
?>
 </body>
</html>
```

Notes

> **Note** :
>
> **phpcredits()** outputs plain text instead of HTML when using the CLI mode.

See Also

- phpversion() - Gets the current PHP version
- phpinfo() - Outputs information about PHP's configuration

phpinfo

phpinfo

(PHP 4, PHP 5, PHP 7, PHP 8)

phpinfo — Outputs information about PHP's configuration

Description

function **phpinfo**(int `$flags` = `INFO_ALL`): true

Outputs a large amount of information about the current state of PHP. This includes information about PHP compilation options and extensions, the PHP version, server information and environment (if compiled as a module), the PHP environment, OS version information, paths, master and local values of configuration options, HTTP headers, and the PHP License.

Because every system is setup differently, **phpinfo()** is commonly used to check configuration settings and for available predefined variables on a given system.

phpinfo() is also a valuable debugging tool as it contains all EGPCS (Environment, GET, POST, Cookie, Server) data.

Parameters

`flags`

The output may be customized by passing one or more of the following *constants* bitwise values summed together in the optional `flags` parameter. One can also combine the respective constants or bitwise values together with the bitwise or operator.

phpinfo() options

Name (constant)	Value	Description
INFO_GENERAL	1	The configuration line, *php.ini* location, build date, Web Server, System and more.
INFO_CREDITS	2	PHP Credits. See also phpcredits().
INFO_CONFIGURATION	4	Current Local and Master values for PHP directives. See also ini_get().
INFO_MODULES	8	Loaded modules and their respective settings. See also get_loaded_extensions().
INFO_ENVIRONMENT	16	Environment Variable information that's also available in *$_ENV*.
INFO_VARIABLES	32	Shows all predefined variables from EGPCS (Environment, GET, POST, Cookie, Server).
INFO_LICENSE	64	PHP License information. See also the » license FAQ.
INFO_ALL	-1	Shows all of the above.

Return Values

Always returns `true`.

Examples

Example #1 phpinfo() Example

```
<?php

// Show all information, defaults to INFO_ALL
phpinfo();

// Show just the module information.
// phpinfo(8) yields identical results.
phpinfo(INFO_MODULES);
```

```
?>
```

Notes

> **Note** :
>
> In versions of PHP before 5.5, parts of the information displayed are disabled when the expose_php configuration setting is set to `off`. This includes the PHP and Zend logos, and the credits.

> **Note** :
>
> **phpinfo()** outputs plain text instead of HTML when using the CLI mode.

See Also

- phpversion() - Gets the current PHP version
- phpcredits() - Prints out the credits for PHP
- ini_get() - Gets the value of a configuration option
- ini_set() - Sets the value of a configuration option
- get_loaded_extensions() - Returns an array with the names of all modules compiled and loaded
- Predefined Variables

phpversion

phpversion

(PHP 4, PHP 5, PHP 7, PHP 8)

phpversion — Gets the current PHP version

Description

function **phpversion**(?string `$extension = null`): string|false

Returns a string containing the version of the currently running PHP parser or extension.

Parameters

`extension`

An optional extension name.

Return Values

Returns the current PHP version as a string. If a string argument is provided for `extension` parameter, **phpversion()** returns the version of that extension, or `false` if there is no version information associated or the extension isn't enabled.

Changelog

Version	Description
8.0.0	`extension` is nullable now.

Examples

Example #1 **phpversion()** example

```
<?php

// Prints e.g. 'Current PHP version: 8.3.12'
echo 'Current PHP version: ' . phpversion();

// Prints e.g. '1.22.3' or nothing if the extension isn't enabled
echo phpversion('zip');

?>
```

Example #2 `PHP_VERSION_ID` example and usage

```php
<?php

/**
 * PHP_VERSION_ID is defined as a number, where the higher the number
 * is, the newer a PHP version is used. It's defined as used in the above
 * expression:
 *
 * $version_id = $major_version * 10000 + $minor_version * 100 + $release_version;
 *
 * Now with PHP_VERSION_ID we can check for features this PHP version
 * may have, this doesn't require to use version_compare() every time
 * you check if the current PHP version may not support a feature.
 *
 * For example, we may here define the PHP_*_VERSION constants that's
 * not available in versions prior to 5.2.7
 */

if (PHP_VERSION_ID < 50207) {
    define('PHP_MAJOR_VERSION',   $version[0]);
    define('PHP_MINOR_VERSION',   $version[1]);
    define('PHP_RELEASE_VERSION', $version[2]);

    // and so on, ...
}

?>
```

Notes

> **Note** :
>
> This information is also available in the predefined constant `PHP_VERSION`. More versioning information is available using the `PHP_*_VERSION` constants.

> **Note** :
>
> Some extensions may define their own version number. However, most bundled extension will use the PHP version as their version number.

See Also

- PHP_VERSION constants
- version_compare() - Compares two "PHP-standardized" version number strings
- phpinfo() - Outputs information about PHP's configuration
- phpcredits() - Prints out the credits for PHP
- zend_version() - Gets the version of the current Zend engine

putenv

putenv

(PHP 4, PHP 5, PHP 7, PHP 8)

putenv — Sets the value of an environment variable

Description

function **putenv**(string `$assignment`): bool

Adds `assignment` to the server environment. The environment variable will only exist for the duration of the current request. At the end of the request the environment is restored to its original state.

Parameters

`assignment`

The setting, like `"FOO=BAR"`

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Setting an environment variable

```
<?php
putenv("UNIQID=$uniqid");
?>
```

See Also

- getenv() - Gets the value of a single or all environment variables
- apache_setenv() - Set an Apache subprocess_env variable

restore_include_path

restore_include_path

(PHP 4 >= 4.3.0, PHP 5, PHP 7)

restore_include_path — Restores the value of the include_path configuration option

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.4.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **restore_include_path**(): void

Restores the include_path configuration option back to its original master value as set in *php.ini*

Parameters

This function has no parameters.

Return Values

No value is returned.

Changelog

Version	Description
8.0.0	Removed this function.
7.4.0	Deprecated this function.

Examples

Example #1 restore_include_path() example

```
<?php

echo get_include_path();  // .:/usr/local/lib/php

set_include_path('/inc');

echo get_include_path();  // /inc

restore_include_path();

// Or using ini_restore()
ini_restore('include_path');
```

```php
echo get_include_path();  // .:/usr/local/lib/php

?>
```

See Also

- ini_restore() - Restores the value of a configuration option
- get_include_path() - Gets the current include_path configuration option
- set_include_path() - Sets the include_path configuration option
- include - include

set_include_path

set_include_path

(PHP 4 >= 4.3.0, PHP 5, PHP 7, PHP 8)

set_include_path — Sets the include_path configuration option

Description

function **set_include_path**(string `$include_path`): string|false

Sets the include_path configuration option for the duration of the script.

Parameters

`include_path`

The new value for the include_path

Return Values

Returns the old include_path on success or **`false`** on failure.

Examples

Example #1 set_include_path() example

```php
<?php
set_include_path('/usr/lib/pear');

// Or using ini_set()
ini_set('include_path', '/usr/lib/pear');
?>
```

Example #2 Adding to the include path

Making use of the **`PATH_SEPARATOR`** constant, it is possible to extend the include path regardless of the operating system.

In this example we add */usr/lib/pear* to the end of the existing `include_path`.

```php
<?php
$path = '/usr/lib/pear';
set_include_path(get_include_path() . PATH_SEPARATOR . $path);
?>
```

See Also

- ini_set() - Sets the value of a configuration option
- get_include_path() - Gets the current include_path configuration option
- restore_include_path() - Restores the value of the include_path configuration option
- include - include

set_time_limit

set_time_limit

(PHP 4, PHP 5, PHP 7, PHP 8)

set_time_limit — Limits the maximum execution time

Description

function **set_time_limit**(int `$seconds`): bool

Set the number of seconds a script is allowed to run. If this is reached, the script returns a fatal error. The default limit is 30 seconds or, if it exists, the `max_execution_time` value defined in the *php.ini*.

When called, **set_time_limit()** restarts the timeout counter from zero. In other words, if the timeout is the default 30 seconds, and 25 seconds into script execution a call such as `set_time_limit(20)` is made, the script will run for a total of 45 seconds before timing out.

Parameters

`seconds`

The maximum execution time, in seconds. If set to zero, no time limit is imposed.

Return Values

Returns `true` on success, or `false` on failure.

Notes

> **Note** :
>
> The **set_time_limit()** function and the configuration directive max_execution_time only affect the execution time of the script itself. Any time spent on activity that happens outside the execution of the script such as system calls using system(), stream operations, database queries, etc. is not included when determining the maximum time that the script has been running. This is not true on Windows where the measured time is real.

See Also

- max_execution_time
- max_input_time

sys_get_temp_dir

sys_get_temp_dir

(PHP 5 >= 5.2.1, PHP 7, PHP 8)

sys_get_temp_dir — Returns directory path used for temporary files

Description

function **sys_get_temp_dir**(): string

Returns the path of the directory PHP stores temporary files in by default.

Parameters

This function has no parameters.

Return Values

Returns the path of the temporary directory.

Examples

Example #1 sys_get_temp_dir() example

```php
<?php
// Create a temporary file in the temporary
// files directory using sys_get_temp_dir()
$temp_file = tempnam(sys_get_temp_dir(), 'Tux');

echo $temp_file;
?>
```

The above example will output something similar to:

```
C:\Windows\Temp\TuxA318.tmp
```

See Also

- tmpfile() - Creates a temporary file
- tempnam() - Create file with unique file name

version_compare

version_compare

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

version_compare — Compares two "PHP-standardized" version number strings

Description

function **version_compare**(string $version1, string $version2, ?string $operator = null): int|bool

version_compare() compares two "PHP-standardized" version number strings.

The function first replaces _, - and + with a dot . in the version strings and also inserts dots . before and after any non number so that for example '4.3.2RC1' becomes '4.3.2.RC.1'. Then it compares the parts starting from left to right. If a part contains special version strings these are handled in the following order: any string not found in this list < dev < alpha = a < beta = b < RC = rc < # < pl = p. This way not only versions with different levels like '4.1' and '4.1.2' can be compared but also any PHP specific version containing development state.

Parameters

version1

First version number.

version2

Second version number.

operator

An optional operator. The possible operators are: <, lt, <=, le, >, gt, >=, ge, ==, =, eq, !=, <>, ne respectively.

This parameter is case-sensitive, values should be lowercase.

Return Values

By default, **version_compare()** returns -1 if the first version is lower than the second, 0 if they are equal, and 1 if the second is lower.

When using the optional operator argument, the function will return true if the relationship is the one specified by the operator, false otherwise.

Examples

The examples below use the PHP_VERSION constant, because it contains the value of the PHP version that is executing the code.

Example #1 version_compare() examples

```php
<?php
if (version_compare(PHP_VERSION, '7.0.0') >= 0) {
    echo 'I am at least PHP version 7.0.0, my version: ' . PHP_VERSION . "\n";
}

if (version_compare(PHP_VERSION, '5.3.0') >= 0) {
    echo 'I am at least PHP version 5.3.0, my version: ' . PHP_VERSION . "\n";
```

```
}

if (version_compare(PHP_VERSION, '5.0.0', '>=')) {
    echo 'I am at least PHP version 5.0.0, my version: ' . PHP_VERSION . "\n";
}

if (version_compare(PHP_VERSION, '5.0.0', '<')) {
    echo 'I am still PHP 4, my version: ' . PHP_VERSION . "\n";
}
?>
```

Notes

> **Note** :
>
> The `PHP_VERSION` constant holds current PHP version.

> **Note** :
>
> Note that pre-release versions, such as 5.3.0-dev, are considered lower than their final release counterparts (like 5.3.0).

> **Note** :
>
> Special version strings such as `alpha` and `beta` are case sensitive. Version strings from arbitrary sources that do not adhere to the PHP standard may need to be lowercased via strtolower() before calling **version_compare()**.

See Also

- phpversion() - Gets the current PHP version
- php_uname() - Returns information about the operating system PHP is running on
- function_exists() - Return true if the given function has been defined

zend_thread_id

zend_thread_id

(PHP 5, PHP 7, PHP 8)

zend_thread_id — Returns a unique identifier for the current thread

Description

function **zend_thread_id**(): int

This function returns a unique identifier for the current thread.

Parameters

This function has no parameters.

Return Values

Returns the thread id as an integer.

Examples

Example #1 zend_thread_id() example

```
<?php
$thread_id = zend_thread_id();

echo 'Current thread id is: ' . $thread_id;
?>
```

The above example will output something similar to:

```
Current thread id is: 7864
```

Notes

> **Note** :
>
> This function is only available if PHP has been built with ZTS (Zend Thread Safety) support and debug mode (`--enable-debug`).

zend_version

zend_version

(PHP 4, PHP 5, PHP 7, PHP 8)

zend_version — Gets the version of the current Zend engine

Description

function **zend_version**(): string

Returns a string containing the version of the currently running Zend Engine.

Parameters

This function has no parameters.

Return Values

Returns the Zend Engine version number, as a string.

Examples

Example #1 zend_version() example

```php
<?php
echo "Zend engine version: " . zend_version();
?>
```

The above example will output something similar to:

```
Zend engine version: 2.2.0
```

See Also

- phpinfo() - Outputs information about PHP's configuration
- phpcredits() - Prints out the credits for PHP
- phpversion() - Gets the current PHP version

Miscellaneous Functions

Introduction

These functions were placed here because none of the other categories seemed to fit.

Installing/Configuring

Table of Contents

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Misc. Configuration Options

Name	Default	Changeable	Changelog
ignore_user_abort	"0"	INI_ALL	
highlight.string	"#DD0000"	INI_ALL	
highlight.comment	"#FF8000"	INI_ALL	
highlight.keyword	"#007700"	INI_ALL	
highlight.default	"#0000BB"	INI_ALL	
highlight.html	"#000000"	INI_ALL	
browscap	NULL	INI_SYSTEM	

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`ignore_user_abort` bool

> `false` by default. If changed to `true` scripts will not be terminated after a client has aborted their connection.
>
> See also ignore_user_abort().

`highlight.bg` string
`highlight.comment` string
`highlight.default` string
`highlight.html` string
`highlight.keyword` string
`highlight.string` string

> Colors for Syntax Highlighting mode. Anything that's acceptable in <font color="??????"> would work.

`browscap` string

> Name (e.g.: *browscap.ini*) and location of browser capabilities file. See also get_browser().

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

`CONNECTION_ABORTED` (int)

`CONNECTION_NORMAL` (int)

`CONNECTION_TIMEOUT` (int)

`__COMPILER_HALT_OFFSET__` (int)

Misc. Functions

Table of Contents

connection_aborted

connection_aborted

(PHP 4, PHP 5, PHP 7, PHP 8)

connection_aborted — Check whether client disconnected

Description

function **connection_aborted**(): int

Checks whether the client disconnected.

Parameters

This function has no parameters.

Return Values

Returns 1 if client disconnected, 0 otherwise.

See Also

- connection_status() - Returns connection status bitfield
- ignore_user_abort() - Set whether a client disconnect should abort script execution
- Connection Handling for a complete description of connection handling in PHP.

connection_status

connection_status

(PHP 4, PHP 5, PHP 7, PHP 8)

connection_status — Returns connection status bitfield

Description

function **connection_status**(): int

Gets the connection status bitfield.

Parameters

This function has no parameters.

Return Values

Returns the connection status bitfield, which can be used against the `CONNECTION_*` constants to determine the connection status.

See Also

- connection_aborted() - Check whether client disconnected
- ignore_user_abort() - Set whether a client disconnect should abort script execution
- Connection Handling for a complete description of connection handling in PHP.

constant

constant

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

constant — Returns the value of a constant

Description

function **constant**(string `$name`): mixed

Return the value of the constant indicated by `name`.

constant() is useful if you need to retrieve the value of a constant, but do not know its name. I.e. it is stored in a variable or returned by a function.

This function works also with class constants and enum cases.

Parameters

`name`

The constant name.

> **Note** :
>
> **constant()** takes the name as an ordinary string and does not resolve it against the current namespace: the name is looked up in the global namespace, so an unqualified name may silently return a different constant from the one the same bare identifier would refer to. To retrieve the value of a constant declared in a namespace, the namespace must be part of the name, as in `constant('My\Namespace\MYCONST')` or `constant(__NAMESPACE__ . '\MYCONST')`. Because the name is a plain string, aliases introduced by `use` statements and the `namespace\` prefix are not applied to it. In a double-quoted string the namespace separators must be escaped, as in `"My\\Namespace\\MYCONST"`. The namespace part of the name is case-insensitive, the constant name is not.

Return Values

Returns the value of the constant.

Errors/Exceptions

If the constant is not defined, an Error exception is thrown. Prior to PHP 8.0.0, an `E_WARNING` level error was generated in that case.

Changelog

Version	Description
8.0.0	If the constant is not defined, **constant()** now throws an Error exception; previously an `E_WARNING` was generated, and `null` was returned.

Examples

Example #1 Using constant() with Constants

```php
<?php

define("MAXSIZE", 100);

echo MAXSIZE;
echo constant("MAXSIZE"); // same thing as the previous line


interface bar {
    const test = 'foobar!';
}

class foo {
    const test = 'foobar!';
}

$const = 'test';

var_dump(constant('bar::'. $const)); // string(7) "foobar!"
var_dump(constant('foo::'. $const)); // string(7) "foobar!"

?>
```

Example #2 Using constant() with Enum Cases (as of PHP 8.1.0)

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

$case = 'Hearts';

var_dump(constant('Suit::'. $case)); // enum(Suit::Hearts)

?>
```

See Also

- define() - Defines a named constant
- defined() - Checks whether a constant with the given name exists
- get_defined_constants() - Returns an associative array with the names of all the constants and their values
- The section on Constants

define

define

(PHP 4, PHP 5, PHP 7, PHP 8)

define — Defines a named constant

Description

function **define**(string `$constant_name`, mixed `$value`, bool `$case_insensitive = false`): bool

Defines a named constant at runtime.

Parameters

`constant_name`

The name of the constant.

> **Note** :
>
> It is possible to **define()** constants with reserved or even invalid names, whose value can (only) be retrieved with constant(). However, doing so is not recommended.
>
> **define()** takes the name as an ordinary string and does not resolve it against the current namespace. Inside `namespace My\Namespace;`, `define('MYCONST', 1)` therefore defines `MYCONST` in the global namespace, silently and without error. To place the constant in a namespace, the namespace must be part of the name, written without a leading namespace separator, as in `define('My\Namespace\MYCONST', 1)` or `define(__NAMESPACE__ . '\MYCONST', 1)`. Unlike defined() and constant(), **define()** does not strip a leading separator: `define('\My\Namespace\MYCONST', 1)` returns `true` but creates a constant that can never be retrieved. Because the name is a plain string, aliases introduced by `use` statements are not applied to it. In a double-quoted string the namespace separators must be escaped, as in `"My\\Namespace\\MYCONST"`.

`value`

The value of the constant.

> **Warning**
>
> While it is possible to define resource constants, it is not recommended and may cause unpredictable behavior.

`case_insensitive`

If set to `true`, the constant will be defined case-insensitive. The default behavior is case-sensitive; i.e. `CONSTANT` and `Constant` represent different values.

> **Warning**
>
> Defining case-insensitive constants is deprecated as of PHP 7.3.0. As of PHP 8.0.0, only `false` is an acceptable value, passing `true` will produce a warning.

> **Note** :
>
> Case-insensitive constants are stored as lower-case.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.1.0	`value` can now be an object.
8.0.0	Passing `true` to `case_insensitive` now emits an `E_WARNING`. Passing `false` is still allowed.
7.3.0	`case_insensitive` has been deprecated and will be removed in version 8.0.0.

Examples

Example #1 Defining Constants

```
<?php
define("CONSTANT", "Hello world.");
echo CONSTANT; // outputs "Hello world."
echo Constant; // outputs "Constant" and issues a notice.

define("GREETING", "Hello you.", true);
echo GREETING; // outputs "Hello you."
echo Greeting; // outputs "Hello you."

// Works as of PHP 7
define('ANIMALS', array(
    'dog',
    'cat',
    'bird'
));
echo ANIMALS[1]; // outputs "cat"

?>
```

Example #2 Constants with Reserved Names

This example illustrates the *possibility* to define a constant with the same name as a magic constant. Since the resulting behavior is obviously confusing, it is not recommended to do this in practise, though.

```
<?php
var_dump(defined('__LINE__'));
var_dump(define('__LINE__', 'test'));
var_dump(constant('__LINE__'));
var_dump(__LINE__);
?>
```

The above example will output:

```
bool(false)
bool(true)
```

```
string(4) "test"
int(5)
```

See Also

- defined() - Checks whether a constant with the given name exists
- constant() - Returns the value of a constant
- The section on Constants

defined

defined

(PHP 4, PHP 5, PHP 7, PHP 8)

defined — Checks whether a constant with the given name exists

Description

function **defined**(string `$constant_name`): bool

Checks whether a constant with the given `constant_name` is defined.

This function works also with class constants and enum cases.

> **Note** :
>
> If you want to see if a variable exists, use isset() as **defined()** only applies to constants. If you want to see if a function exists, use function_exists().

Parameters

`constant_name`

The constant name.

> **Note** :
>
> **defined()** takes the name as an ordinary string and does not resolve it against the current namespace: the name is looked up in the global namespace, so an unqualified name may silently report on a different constant from the one the same bare identifier would refer to. To check whether a constant declared in a namespace is defined, the namespace must be part of the name, as in `defined('My\Namespace\MYCONST')` or `defined(__NAMESPACE__ . '\MYCONST')`. Because the name is a plain string, aliases introduced by `use` statements and the `namespace\` prefix are not applied to it. In a double-quoted string the namespace separators must be escaped, as in `"My\\Namespace\\MYCONST"`. The namespace part of the name is case-insensitive, the constant name is not.

Return Values

Returns **`true`** if the named constant given by `constant_name` has been defined, **`false`** otherwise.

Examples

Example #1 Checking Constants

```php
<?php

/* Note the use of quotes, this is important. This example is checking
 * if the string 'TEST' is the name of a constant named TEST */
if (defined('TEST')) {
    echo TEST;
}


interface bar {
```

```
    const test = 'foobar!';
}

class foo {
    const test = 'foobar!';
}

var_dump(defined('bar::test')); // bool(true)
var_dump(defined('foo::test')); // bool(true)

?>
```

Example #2 Checking Enum Cases (as of PHP 8.1.0)

```
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

var_dump(defined('Suit::Hearts')); // bool(true)

?>
```

See Also

- define() - Defines a named constant
- constant() - Returns the value of a constant
- get_defined_constants() - Returns an associative array with the names of all the constants and their values
- function_exists() - Return true if the given function has been defined
- The section on Constants

die

die

(PHP 4, PHP 5, PHP 7, PHP 8)

die — Alias of exit()

Description

This function is an alias of: exit().

eval

eval

(PHP 4, PHP 5, PHP 7, PHP 8)

eval — Evaluate a string as PHP code

Description

function **eval**(string `$code`): mixed

Evaluates the given `code` as PHP.

The code being evaluated inherits the variable scope of the line on which the **eval()** call occurs. Any variables available at that line will be available for reading and modification in the evaluated code. However, all functions and classes defined will be defined in the global namespace. In other words, the compiler considers the evaluated code as if it were a separate included file.

> **Caution**

> The **eval()** language construct is *very dangerous* because it allows execution of arbitrary PHP code. *Its use thus is discouraged.* If you have carefully verified that there is no other option than to use this construct, pay special attention *not to pass any user provided data* into it without properly validating it beforehand.

Parameters

`code`

Valid PHP code to be evaluated.

The code must not be wrapped in opening and closing PHP tags, i.e. `'echo "Hi!";'` must be passed instead of `'<?php echo "Hi!"; ?>'`. It is still possible to leave and re-enter PHP mode though using the appropriate PHP tags, e.g. `'echo "In PHP mode!"; ?>In HTML mode!<?php echo "Back in PHP mode!";'`.

Apart from that the passed code must be valid PHP. This includes that all statements must be properly terminated using a semicolon. `'echo "Hi!"'` for example will cause a parse error, whereas `'echo "Hi!";'` will work.

A `return` statement will immediately terminate the evaluation of the code.

The code will be executed in the scope of the code calling **eval()**. Thus any variables defined or changed in the **eval()** call will remain visible after it terminates.

Return Values

eval() returns `null` unless `return` is called in the evaluated code, in which case the value passed to `return` is returned. As of PHP 7, if there is a parse error in the evaluated code, **eval()** throws a ParseError exception. Before PHP 7, in this case **eval()** returned `false` and execution of the following code continued normally. It is not possible to catch a parse error in **eval()** using set_error_handler().

Examples

Example #1 eval() example - simple text merge

```php
<?php
$string = 'cup';
$name = 'coffee';
$str = 'This is a $string with my $name in it.';
echo $str. "\n";
eval("\$str = \"$str\";");
echo $str. "\n";
?>
```

The above example will output:

```
This is a $string with my $name in it.
This is a cup with my coffee in it.
```

Notes

> **Note** :

Because this is a language construct and not a function, it cannot be called using variable functions, or named arguments.

> **Tip**

As with anything that outputs its result directly to the browser, the output-control functions can be used to capture the output of this function, and save it in a string (for example).

> **Note** :

> In case of a fatal error in the evaluated code, the whole script exits.

See Also

- call_user_func() - Call the callback given by the first parameter

exit

exit

(PHP 4, PHP 5, PHP 7, PHP 8)

exit — Terminate the current script with a status code or message

Description

function **exit**(string|int `$status` = 0): never

Terminates execution of the script. Shutdown functions and object destructors will always be executed even if **exit()** is called. However, `finally` blocks are never executed.

An exit code of `0` is used to indicate that the program succeeded in its tasks. Any other value indicates some sort of error occurred during execution.

exit() is a special function, because it has a dedicated token in the parser, as such it can be used like a statement (i.e. without parentheses) to terminate the script with the default status code.

> **Caution**
>
> It is not possible to disable, or create a namespaced function shadowing the global **exit()** function.

Parameters

`status`

If `status` is a string, this function prints the `status` just before exiting. The exit code returned by PHP is `0`.

If `status` is an int, the exit code returned by PHP will be `status`.

> **Note** :
>
> Exit codes should be in the range `0` to `254`, the exit code `255` is reserved by PHP and should not be used.

> **Warning**
>
> Prior to PHP 8.4.0, **exit()** did not follow PHP's standard type juggling semantics, nor respect the `strict_types` declare.
>
> Any value not of type int was cast to string including resource and array values. As of PHP 8.4.0, it follows the usual type juggling semantics and throws a TypeError on invalid values.

Return Values

As this terminates the PHP script, no value is returned.

Changelog

Version	Description
8.4.0	**exit()** is now a proper function, therefore it follows the usual type juggling semantics is affected by the `strict_types` declare, can be called with named arguments, and be a variable functions.
8.4.0	A parameterless **exit()** called within shutdown functions or object destructors now resets the exit code to `0`; previously the exit code set by an earlier **exit()** call was kept.

Examples

Example #1 Basic exit() example

```php
<?php

// exit program normally
exit();
exit(0);

// exit with an error code
exit(1);

?>
```

Example #2 exit() example with a string

```php
<?php

$filename = '/path/to/data-file';
$file = fopen($filename, 'r')
    or exit("unable to open file ($filename)");

?>
```

Example #3 Shutdown functions and destructors run regardless

```php
<?php
class Foo
{
    public function __destruct()
    {
        echo 'Destruct: ' . __METHOD__ . '()' . PHP_EOL;
    }
}

function shutdown()
{
    echo 'Shutdown: ' . __FUNCTION__ . '()' . PHP_EOL;
}

$foo = new Foo();
register_shutdown_function('shutdown');

exit();
echo 'This will not be output.';
?>
```

The above example will output:

```
Shutdown: shutdown()
Destruct: Foo::__destruct()
```

Example #4 exit() as a statement

```php
<?php

// exit program normally with exit code 0
exit;

?>
```

Notes

> **Warning**

> Prior to PHP 8.4.0, **exit()** was a language construct and not a function, therefore it was not possible to call it using variable functions, or named arguments.

See Also

- register_shutdown_function() - Register a function for execution on shutdown
- Shutdown functions
- object destructors

get_browser

get_browser

(PHP 4, PHP 5, PHP 7, PHP 8)

get_browser — Tells what the user's browser is capable of

Description

function **get_browser**(?string `$user_agent = null`, bool `$return_array = false`): object|array|false

Attempts to determine the capabilities of the user's browser, by looking up the browser's information in the *browscap.ini* file.

Parameters

`user_agent`

> The User Agent to be analyzed. By default, the value of HTTP User-Agent header is used; however, you can alter this (i.e., look up another browser's info) by passing this parameter.
>
> You can bypass this parameter with a `null` value.

`return_array`

> If set to `true`, this function will return an array instead of an object.

Return Values

The information is returned in an object or an array which will contain various data elements representing, for instance, the browser's major and minor version numbers and ID string; `true`/`false` values for features such as frames, JavaScript, and cookies; and so forth.

The `cookies` value simply means that the browser itself is capable of accepting cookies and does not mean the user has enabled the browser to accept cookies or not. The only way to test if cookies are accepted is to set one with setcookie(), reload, and check for the value.

Returns `false` when no information can be retrieved, such as when the browscap configuration setting in *php.ini* has not been set.

Examples

Example #1 Listing all information about the users browser

```php
<?php
echo $_SERVER['HTTP_USER_AGENT'] . "\n\n";

$browser = get_browser(null, true);
print_r($browser);
?>
```

The above example will output something similar to:

```
Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.7) Gecko/20040803 Firefox/0.9.3

Array
(
    [browser_name_regex] => ^mozilla/5\.0 (windows; .; windows nt 5\.1; .*rv:.*) gecko/.* firefox/0\.9.*$
    [browser_name_pattern] => Mozilla/5.0 (Windows; ?; Windows NT 5.1; *rv:*) Gecko/* Firefox/0.9*
    [parent] => Firefox 0.9
```

```
    [platform] => WinXP
    [browser] => Firefox
    [version] => 0.9
    [majorver] => 0
    [minorver] => 9
    [cssversion] => 2
    [frames] => 1
    [iframes] => 1
    [tables] => 1
    [cookies] => 1
    [backgroundsounds] =>
    [vbscript] =>
    [javascript] => 1
    [javaapplets] => 1
    [activexcontrols] =>
    [cdf] =>
    [aol] =>
    [beta] => 1
    [win16] =>
    [crawler] =>
    [stripper] =>
    [wap] =>
    [netclr] =>
)
```

Notes

> **Note** :
>
> In order for this to work, your browscap configuration setting in *php.ini* must point to the correct location of the *browscap.ini* file on your system.
>
> *browscap.ini* is not bundled with PHP, but you may find an up-to-date » php_browscap.ini file here.
>
> While *browscap.ini* contains information on many browsers, it relies on user updates to keep the database current. The format of the file is fairly self-explanatory.

__halt_compiler

__halt_compiler

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

__halt_compiler — Halts the compiler execution

Description

function **__halt_compiler**(): void

Halts the execution of the compiler. This can be useful to embed data in PHP scripts, like the installation files.

Byte position of the data start can be determined by the `__COMPILER_HALT_OFFSET__` constant which is defined only if there is a **__halt_compiler()** presented in the file.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 A __halt_compiler() example

```php
<?php

// open this file
$fp = fopen(__FILE__, 'r');

// seek file pointer to data
fseek($fp, __COMPILER_HALT_OFFSET__);

// and output it
var_dump(stream_get_contents($fp));

// the end of the script execution
__halt_compiler(); the installation data (eg. tar, gz, PHP, etc.)
```

Notes

> **Note** :
>
> **__halt_compiler()** can only be used from the outermost scope.

highlight_file

highlight_file

(PHP 4, PHP 5, PHP 7, PHP 8)

highlight_file — Syntax highlighting of a file

Description

function **highlight_file**(string `$filename`, bool `$return` = **`false`**): string|bool

Prints out or returns a syntax highlighted version of the code contained in `filename` using the colors defined in the built-in syntax highlighter for PHP.

Many servers are configured to automatically highlight files with a *phps* extension. For example, *example.phps* when viewed will show the syntax highlighted source of the file. To enable this, add this line to the *httpd.conf*:

```
AddType application/x-httpd-php-source .phps
```

Parameters

`filename`

Path to the PHP file to be highlighted.

`return`

Set this parameter to **`true`** to make this function return the highlighted code.

Return Values

If `return` is set to **`true`**, returns the highlighted code as a string instead of printing it out. Otherwise, it will return **`true`** on success, **`false`** on failure.

Changelog

Version	Description
8.3.0	The resulting HTML has changed.

Notes

> **Caution**
>
> Care should be taken when using the **highlight_file()** function to make sure that you do not inadvertently reveal sensitive information such as passwords or any other type of information that might create a potential security risk.

> **Note** :
>
> When the `return` parameter is used, this function uses internal output buffering so it cannot be used inside an ob_start() callback function.

See Also

- highlight_string() - Syntax highlighting of a string
- Highlighting INI directives

highlight_string

highlight_string

(PHP 4, PHP 5, PHP 7, PHP 8)

highlight_string — Syntax highlighting of a string

Description

function **highlight_string**(string `$string`, bool `$return = false`): string|true

Outputs or returns html markup for a syntax highlighted version of the given PHP code using the colors defined in the built-in syntax highlighter for PHP.

Parameters

`string`

The PHP code to be highlighted. This should include the opening tag.

`return`

Set this parameter to `true` to make this function return the highlighted code.

Return Values

If `return` is set to `true`, returns the highlighted code as a string instead of printing it out. Otherwise, it will return `true`.

Changelog

Version	Description
8.4.0	Return type changed from string|bool to string|true.
8.3.0	The resulting HTML has changed.

Examples

Example #1 highlight_string() example

```
<?php
highlight_string('<?php phpinfo(); ?>');
?>
```

The above example will output:

```
<code><span style="color: #000000">
<span style="color: #0000BB">&lt;?php phpinfo</span><span style="color: #007700">(); </span><span style="color: #0000BB">?
&gt;</span>
```

```
</span>
</code>
```

Output of the above example in PHP 8.3:

```
<pre><code style="color: #000000"><span style="color: #0000BB">&lt;?php phpinfo</span><span style="color: #007700">();
</span><span style="color: #0000BB">?&gt;</span></code></pre>
```

Notes

> **Note** :
>
> When the `return` parameter is used, this function uses internal output buffering so it cannot be used inside an ob_start() callback function.

The HTML markup generated is subject to change.

See Also

- highlight_file() - Syntax highlighting of a file
- Highlighting INI directives

hrtime

hrtime

(PHP 7 >= 7.3.0, PHP 8)

hrtime — Get the system's high resolution time

Description

function **hrtime**(bool `$as_number` = **`false`**): array|int|float|false

Returns the system's high resolution time, counted from an arbitrary point in time. The delivered timestamp is monotonic and can not be adjusted.

Parameters

`as_number`

Whether the high resolution time should be returned as array or number.

Return Values

Returns an array of integers in the form [seconds, nanoseconds], if the parameter `as_number` is false. Otherwise the nanoseconds are returned as int (64bit platforms) or float (32bit platforms). Returns **`false`** on failure.

Changelog

Version	Description
8.5.0	On macOS, the implementation now uses `clock_gettime_nsec_np(CLOCK_UPTIME_RAW)` instead of `mach_absolute_time()`.

Examples

Example #1 **hrtime()** usage

```php
<?php
echo hrtime(true), PHP_EOL;
```

```php
print_r(hrtime());
?>
```

The above example will output something similar to:

```
10444739687370679
Array
(
    [0] => 10444739
    [1] => 687464812
)
```

See Also

- The High resolution timing extension
- microtime() - Return current Unix timestamp with microseconds
- time_nanosleep() - Delay for a number of seconds and nanoseconds

ignore_user_abort

ignore_user_abort

(PHP 4, PHP 5, PHP 7, PHP 8)

ignore_user_abort — Set whether a client disconnect should abort script execution

Description

function **ignore_user_abort**(?bool `$enable = null`): int

Sets whether a client disconnect should cause a script to be aborted.

When running PHP as a command line script, and the script's tty goes away without the script being terminated then the script will die the next time it tries to write anything, unless `enable` is set to `true`

Parameters

`enable`

> If set and not `null`, this function will set the ignore_user_abort ini setting to the given `enable`. Otherwise, this function will only return the previous setting without changing it.

Return Values

Returns the previous setting, as an integer.

Changelog

Version	Description
8.0.0	`enable` is nullable now.

Examples

Example #1 A ignore_user_abort() example

```php
<?php
// Ignore user aborts and allow the script
// to run forever
ignore_user_abort(true);
set_time_limit(0);

echo 'Testing connection handling in PHP';

// Run a pointless loop that sometime
```

```
// hopefully will make us click away from
// page or click the "Stop" button.
while(1)
{
    // Did the connection fail?
    if(connection_status() != CONNECTION_NORMAL)
    {
        break;
    }

    // Sleep for 10 seconds
    sleep(10);
}

// If this is reached, then the 'break'
// was triggered from inside the while loop

// So here we can log, or perform any other tasks
// we need without actually being dependent on the
// browser.
?>
```

Notes

PHP will not detect that the user has aborted the connection until an attempt is made to send information to the client. Simply using an echo statement does not guarantee that information is sent, see flush().

See Also

- connection_aborted() - Check whether client disconnected
- connection_status() - Returns connection status bitfield
- Connection Handling for a complete description of connection handling in PHP.

pack

pack

(PHP 4, PHP 5, PHP 7, PHP 8)

pack — Pack data into binary string

Description

function **pack**(string `$format`, mixed `...$values`): string

Pack given arguments into a binary string according to `format`.

The idea for this function was taken from Perl and all formatting codes work the same as in Perl. However, there are some formatting codes that are missing such as Perl's "u" format code.

Note that the distinction between signed and unsigned values only affects the function unpack(), where as function **pack()** gives the same result for signed and unsigned format codes.

Parameters

`format`

The `format` string consists of format codes followed by an optional repeater argument. The repeater argument can be either an integer value or `*` for repeating to the end of the input data. For a, A, h, H the repeat count specifies how many characters of one data argument are taken, for @ it is the absolute position where to put the next data, for everything else the repeat count specifies how many data arguments are consumed and packed into the resulting binary string.

Currently implemented formats are:

pack() format characters

Code	Description
a	NUL-padded string
A	SPACE-padded string

Code	Description
h	Hex string, low nibble first
H	Hex string, high nibble first
c	signed char
C	unsigned char
s	signed short (always 16 bit, machine byte order)
S	unsigned short (always 16 bit, machine byte order)
n	unsigned short (always 16 bit, big endian byte order)
v	unsigned short (always 16 bit, little endian byte order)
i	signed integer (machine dependent size and byte order)
I	unsigned integer (machine dependent size and byte order)
l	signed long (always 32 bit, machine byte order)
L	unsigned long (always 32 bit, machine byte order)
N	unsigned long (always 32 bit, big endian byte order)
V	unsigned long (always 32 bit, little endian byte order)
q	signed long long (always 64 bit, machine byte order)
Q	unsigned long long (always 64 bit, machine byte order)
J	unsigned long long (always 64 bit, big endian byte order)
P	unsigned long long (always 64 bit, little endian byte order)
f	float (machine dependent size and representation)
g	float (machine dependent size, little endian byte order)
G	float (machine dependent size, big endian byte order)
d	double (machine dependent size and representation)
e	double (machine dependent size, little endian byte order)
E	double (machine dependent size, big endian byte order)
x	NUL byte
X	Back up one byte
Z	NUL-terminated (ASCIIZ) string, will be NUL padded
@	NUL-fill to absolute position

`values`

Return Values

Returns a binary string containing data.

Changelog

Version	Description
8.0.0	This function no longer returns `false` on failure.
7.2.0	float and double types supports both Big Endian and Little Endian.
7.0.15, 7.1.1	The "e", "E", "g" and "G" codes were added to enable byte order support for float and double.

Examples

Example #1 **pack()** example

```
<?php
$binarydata = pack("nvc*", 0x1234, 0x5678, 65, 66);
?>
```

The resulting binary string will be 6 bytes long and contain the byte sequence 0x12, 0x34, 0x78, 0x56, 0x41, 0x42.

Notes

> **Caution**
>
> Format codes `q`, `Q`, `J` and `P` are not available on 32-bit PHP builds.

> **Caution**
>
> Note that PHP internally stores int values as signed values of a machine-dependent size. Integer literals and operations that yield numbers outside the bounds of the int type will be stored as float. When packing these floats as integers, they are first cast into the integer type. This may or may not result in the desired byte pattern.
>
> The most relevant case is when packing unsigned numbers that would be representable with the int type if it were unsigned. In systems where the int type has a 32-bit size, the cast usually results in the same byte pattern as if the int were unsigned (although this relies on implementation-defined unsigned to signed conversions, as per the C standard). In systems where the int type has 64-bit size, the float most likely does not have a mantissa large enough to hold the value without loss of precision. If those systems also have a native 64-bit C `int` type (most UNIX-like systems don't), the only way to use the `I` pack format in the upper range is to create int negative values with the same byte representation as the desired unsigned value.

See Also

- unpack() - Unpack data from binary string

php_strip_whitespace

php_strip_whitespace

(PHP 5, PHP 7, PHP 8)

php_strip_whitespace — Return source with stripped comments and whitespace

Description

function **php_strip_whitespace**(string `$filename`): string

Returns the PHP source code in `filename` with PHP comments and whitespace removed. This may be useful for determining the amount of actual code in your scripts compared with the amount of comments. This is similar to using **php -w** from the commandline.

Parameters

`filename`

Path to the PHP file.

Return Values

The stripped source code will be returned on success, or an empty string on failure.

> **Note** :
>
> This function respects the value of the short_open_tag ini directive.

Examples

Example #1 php_strip_whitespace() example

```
<?php
// PHP comment here

/*
 * Another PHP comment
 */

echo        php_strip_whitespace(__FILE__);
// Newlines are considered whitespace, and are removed too:
```

```
do_nothing();
?>
```

The above example will output:

```
<?php
 echo php_strip_whitespace(__FILE__); do_nothing(); ?>
```

Notice the PHP comments are gone, as are the whitespace and newline after the first echo statement.

sapi_windows_cp_conv

sapi_windows_cp_conv

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_conv — Convert string from one codepage to another

Description

function **sapi_windows_cp_conv**(int|string `$in_codepage`, int|string `$out_codepage`, string `$subject`): ?string

Convert string from one codepage to another.

Parameters

`in_codepage`

The codepage of the `subject` string. Either the codepage name or identifier.

`out_codepage`

The codepage to convert the `subject` string to. Either the codepage name or identifier.

`subject`

The string to convert.

Return Values

The `subject` string converted to `out_codepage`, or `null` on failure.

Errors/Exceptions

This function issues E_WARNING level errors, if invalid codepages are given, or if the subject is not valid for `in_codepage`.

See Also

- sapi_windows_cp_get() - Get current codepage
- iconv() - Convert a string from one character encoding to another

sapi_windows_cp_get

sapi_windows_cp_get

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_get — Get current codepage

Description

function **sapi_windows_cp_get**(string `$kind` = ""): int

Gets the current codepage.

Parameters

`kind`

The kind of operating system codepage to get, either `'ansi'` or `'oem'`. Any other value refers to the current codepage of the process.

Return Values

If `kind` is `'ansi'`, the current ANSI code page of the operating system is returned. If `kind` is `'oem'`, the current OEM code page of the operating system is returned. Otherwise, the current codepage of the process is returned.

See Also

- sapi_windows_cp_set() - Set process codepage

sapi_windows_cp_is_utf8

sapi_windows_cp_is_utf8

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_is_utf8 — Indicates whether the codepage is UTF-8 compatible

Description

function **sapi_windows_cp_is_utf8**(): bool

Indicates whether the codepage of the current process is UTF-8 compatible.

Parameters

This function has no parameters.

Return Values

Returns whether the codepage of the current process is UTF-8 compatible.

See Also

- sapi_windows_cp_get() - Get current codepage

sapi_windows_cp_set

sapi_windows_cp_set

(PHP 7 >= 7.1.0, PHP 8)

sapi_windows_cp_set — Set process codepage

Description

function **sapi_windows_cp_set**(int `$codepage`): bool

Set the codepage of the current process.

Parameters

`codepage`

A codepage identifier.

Return Values

Returns `true` on success or `false` on failure.

See Also

- sapi_windows_cp_get() - Get current codepage

sapi_windows_generate_ctrl_event

sapi_windows_generate_ctrl_event

(PHP 7 >= 7.4.0, PHP 8)

sapi_windows_generate_ctrl_event — Send a CTRL event to another process

Description

function **sapi_windows_generate_ctrl_event**(int `$event`, int `$pid` = 0): bool

Sends a CTRL event to another process in the same process group.

Parameters

`event`

The `CTRL` event to send; either `PHP_WINDOWS_EVENT_CTRL_C` or `PHP_WINDOWS_EVENT_CTRL_BREAK`.

`pid`

The ID of the process to which to send the event to. If `0` is given, the event is sent to all processes of the process group.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 Basic sapi_windows_generate_ctrl_event() Usage

This example shows how to pass along `CTRL+BREAK` events to a child process. In this case the child process echoes `I'm still alive` every second, until the user presses `CTRL+BREAK`, what causes only the child process to be terminated.

```
<?php
// forward CTRL+BREAK events to the child process
sapi_windows_set_ctrl_handler('sapi_windows_generate_ctrl_event');

// create a child process which echoes every second
$cmd = ['php', '-r', 'while (true) { echo "I\'m still alive\n"; sleep(1); }'];
$descspec = array(['pipe', 'r'], ['pipe', 'w'], ['pipe', 'w']);
$options = ['create_process_group' => true];
$proc = proc_open($cmd, $descspec, $pipes, null, null, $options);
while (true) {
    echo fgets($pipes[1]);
}
?>
```

See Also

- proc_open() - Execute a command and open file pointers for input/output
- sapi_windows_set_ctrl_handler() - Set or remove a CTRL event handler

sapi_windows_set_ctrl_handler

sapi_windows_set_ctrl_handler

(PHP 7 >= 7.4.0, PHP 8)

sapi_windows_set_ctrl_handler — Set or remove a CTRL event handler

Description

function **sapi_windows_set_ctrl_handler**(?callable `$handler`, bool `$add` = `true`): bool

Sets or removes a `CTRL` event handler, which allows Windows CLI processes to intercept or ignore `CTRL+C` and `CTRL+BREAK` events. Note that in multithreaded environments, this is only possible when called from the main thread.

Parameters

`handler`

A callback function to set or remove. If set, this function will be called whenever a `CTRL +C` or `CTRL +BREAK` event occurs. The function is supposed to have the following signature:

function **handler**(int `$event`): void

`event`

The `CTRL` event which has been received; either **`PHP_WINDOWS_EVENT_CTRL_C`** or **`PHP_WINDOWS_EVENT_CTRL_BREAK`**.

Setting a **`null`** `handler` causes the process to ignore `CTRL +C` events, but not `CTRL +BREAK` events.

`add`

If **`true`**, the handler is set. If **`false`**, the handler is removed.

Return Values

Returns **`true`** on success or **`false`** on failure.

Examples

Example #1 Basic sapi_windows_set_ctrl_handler() Usage

This example shows how to intercept `CTRL` events.

```
<?php
function ctrl_handler(int $event)
{
    switch ($event) {
        case PHP_WINDOWS_EVENT_CTRL_C:
            echo "You have pressed CTRL+C\n";
            break;
        case PHP_WINDOWS_EVENT_CTRL_BREAK:
            echo "You have pressed CTRL+BREAK\n";
            break;
    }
}

sapi_windows_set_ctrl_handler('ctrl_handler');
while (true); // infinite loop, so the handler can be triggered
?>
```

See Also

- sapi_windows_generate_ctrl_event() - Send a CTRL event to another process

sapi_windows_vt100_support

sapi_windows_vt100_support

(PHP 7 >= 7.2.0, PHP 8)

sapi_windows_vt100_support — Get or set VT100 support for the specified stream associated to an output buffer of a Windows console.

Description

function **sapi_windows_vt100_support**(resource `$stream`, ?bool `$enable` = **`null`**): bool

If `enable` is **`null`**, the function returns **`true`** if the stream `stream` has VT100 control codes enabled, **`false`** otherwise.

If `enable` is a bool, the function will try to enable or disable the VT100 features of the stream `stream`. If the feature has been successfully enabled (or disabled), the function will return **`true`**, or **`false`** otherwise.

At startup, PHP tries to enable the VT100 feature of the **`STDOUT`**/**`STDERR`** streams. By the way, if those streams are redirected to a file, the VT100 features may not be enabled.

If VT100 support is enabled, it is possible to use control sequences as they are known from the VT100 terminal. They allow the modification of the terminal's output. On Windows these sequences are called Console Virtual Terminal Sequences.

> **Warning**

> This function uses the **ENABLE_VIRTUAL_TERMINAL_PROCESSING** flag implemented in the Windows 10 API, so the VT100 feature may not be available on older Windows versions.

Parameters

`stream`

The stream on which the function will operate.

`enable`

If bool, the VT100 feature will be enabled (if **true**) or disabled (if **false**).

Return Values

If `enable` is **null**: returns **true** if the VT100 feature is enabled, **false** otherwise.

If `enable` is a bool: Returns **true** on success or **false** on failure.

Changelog

Version	Description
8.0.0	`enable` is now nullable.

Examples

Example #1 sapi_windows_vt100_support() default state

By default, **STDOUT** and **STDERR** have the VT100 feature enabled.

```
php -r "var_export(sapi_windows_vt100_support(STDOUT));echo ' ';var_export(sapi_windows_vt100_support(STDERR));"
```

The above example will output something similar to:

```
true true
```

By the way, if a stream is redirected, the VT100 feature will not be enabled:

```
php -r "var_export(sapi_windows_vt100_support(STDOUT));echo ' ';var_export(sapi_windows_vt100_support(STDERR));" 2>NUL
```

The above example will output something similar to:

```
true false
```

Example #2 sapi_windows_vt100_support() changing state

You won't be able to enable the VT100 feature of **STDOUT** or **STDERR** if the stream is redirected.

```
php -r "var_export(sapi_windows_vt100_support(STDOUT, true));echo ' ';var_export(sapi_windows_vt100_support(STDERR, true));"
2>NUL
```

The above example will output something similar to:

```
true false
```

Example #3 Example usage of VT100 support enabled

```php
<?php
$out = fopen('php://stdout','w');
fwrite($out, 'Just forgot a lettr.');
// Moves the cursor two characters backwards
fwrite($out, "\033[2D");
// Inserts one blank, shifting existing text to the right -> Just forgot a lett r.
fwrite($out, "\033[1@");
fwrite($out, 'e');
?>
```

The above example will output:

```
Just forgot a letter.
```

show_source

show_source

(PHP 4, PHP 5, PHP 7, PHP 8)

show_source — Alias of highlight_file()

Description

This function is an alias of: highlight_file().

sleep

sleep

(PHP 4, PHP 5, PHP 7, PHP 8)

sleep — Delay execution

Description

function **sleep**(int `$seconds`): int

Delays the program execution for the given number of `seconds`.

> **Note** :
>
> In order to delay program execution for a fraction of a second, use usleep() as the **sleep()** function expects an int. For example, `sleep(0.25)` will pause program execution for `0` seconds.

Parameters

`seconds`

Halt time in seconds (must be greater than or equal to `0`).

Return Values

Returns zero on success.

If the call was interrupted by a signal, **sleep()** returns a non-zero value. On Windows, this value will always be `192` (the value of the **`WAIT_IO_COMPLETION`** constant within the Windows API). On other platforms, the return value will be the number of seconds left to sleep.

Errors/Exceptions

If the specified number of `seconds` is negative, this function will throw a ValueError.

Changelog

Version	Description
8.0.0	The function throws a ValueError on negative `seconds`; previously, an `E_WARNING` was raised instead, and the function returned `false`.

Examples

Example #1 sleep() example

```php
<?php

// current time
echo date('h:i:s') . "\n";

// sleep for 10 seconds
sleep(10);

// wake up !
echo date('h:i:s') . "\n";

?>
```

This example will output (after 10 seconds)

```
05:31:23
05:31:33
```

See Also

- usleep() - Delay execution in microseconds
- time_nanosleep() - Delay for a number of seconds and nanoseconds
- time_sleep_until() - Make the script sleep until the specified time
- set_time_limit() - Limits the maximum execution time

sys_getloadavg

sys_getloadavg

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

sys_getloadavg — Gets system load average

Description

function **sys_getloadavg**(): array|false

Returns three samples representing the average system load (the number of processes in the system run queue) over the last 1, 5 and 15 minutes, respectively. Returns `false` on failure.

Parameters

This function has no parameters.

Return Values

Returns an array with three samples (last 1, 5 and 15 minutes).

Examples

Example #1 A sys_getloadavg() example

```php
<?php
$load = sys_getloadavg();
if ($load[0] > 0.80) {
    header('HTTP/1.1 503 Too busy, try again later');
    die('Server too busy. Please try again later.');
}
?>
```

Notes

> **Note** :
>
> This function is not implemented on Windows platforms.

time_nanosleep

time_nanosleep

(PHP 5, PHP 7, PHP 8)

time_nanosleep — Delay for a number of seconds and nanoseconds

Description

function **time_nanosleep**(int $seconds, int $nanoseconds): array|bool

Delays program execution for the given number of `seconds` and `nanoseconds`.

Parameters

`seconds`

Must be a non-negative integer.

`nanoseconds`

Must be a non-negative integer less than 1 billion.

> **Note** :
>
> On Windows, the system may sleep longer that the given number of nanoseconds, depending on the hardware.

Return Values

Returns `true` on success or `false` on failure.

If the delay was interrupted by a signal, an associative array will be returned with the components:

- `seconds` - number of seconds remaining in the delay
- `nanoseconds` - number of nanoseconds remaining in the delay

Examples

Example #1 time_nanosleep() example

```php
<?php
// Careful! This won't work as expected if an array is returned
if (time_nanosleep(0, 500000000)) {
    echo "Slept for half a second.\n";
}

// This is better:
if (time_nanosleep(0, 500000000) === true) {
    echo "Slept for half a second.\n";
```

```php
}

// And this is the best:
$nano = time_nanosleep(2, 100000);

if ($nano === true) {
    echo "Slept for 2 seconds, 100 microseconds.\n";
} elseif ($nano === false) {
    echo "Sleeping failed.\n";
} elseif (is_array($nano)) {
    $seconds = $nano['seconds'];
    $nanoseconds = $nano['nanoseconds'];
    echo "Interrupted by a signal.\n";
    echo "Time remaining: $seconds seconds, $nanoseconds nanoseconds.";
}
?>
```

See Also

- sleep() - Delay execution
- usleep() - Delay execution in microseconds
- time_sleep_until() - Make the script sleep until the specified time
- set_time_limit() - Limits the maximum execution time

time_sleep_until

time_sleep_until

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

time_sleep_until — Make the script sleep until the specified time

Description

function **time_sleep_until**(float `$timestamp`): bool

Makes the script sleep until the specified `timestamp`.

Parameters

`timestamp`

The timestamp when the script should wake.

Return Values

Returns `true` on success or `false` on failure.

Errors/Exceptions

If the specified `timestamp` is in the past, this function will generate a `E_WARNING`.

Examples

Example #1 A time_sleep_until() example

```php
<?php

//returns false and generates a warning
var_dump(time_sleep_until(time()-1));

// may only work on faster computers, will sleep up to 0.2 seconds
var_dump(time_sleep_until(microtime(true)+0.2));

?>
```

Notes

> **Note** :

> All signals will be delivered after the script wakes up.

See Also

- sleep() - Delay execution
- usleep() - Delay execution in microseconds
- time_nanosleep() - Delay for a number of seconds and nanoseconds
- set_time_limit() - Limits the maximum execution time

uniqid

uniqid

(PHP 4, PHP 5, PHP 7, PHP 8)

uniqid — Generate a time-based identifier

Description

function **uniqid**(string `$prefix = ""`, bool `$more_entropy = false`): string

Gets an identifier based on the current time with microsecond precision, prefixed with the given `prefix` and optionally appending a randomly generated value.

> **Caution**
>
> This function does not generate cryptographically secure values, and *must not* be used for cryptographic purposes, or purposes that require returned values to be unguessable.
>
> If cryptographically secure randomness is required, the Random\Randomizer may be used with the Random\Engine\Secure engine. For simple use cases, the random_int() and random_bytes() functions provide a convenient and secure API that is backed by the operating system's CSPRNG.

> **Warning**
>
> This function does not guarantee the uniqueness of the return value because the value is based on the current time in microseconds or the current time with a small amount of random data appended if `more_entropy` is `true`.

Parameters

`prefix`

> Can be useful, for instance, if you generate identifiers simultaneously on several hosts that could generate the same identifier at the same microsecond. (This can happen even on a single host if the system clock is moved backwards, such as by an NTP adjustment.)
>
> With an empty `prefix`, the returned string will be 13 characters long. If `more_entropy` is `true`, it will be 23 characters.

`more_entropy`

> If set to `true`, **uniqid()** will add additional entropy (using the combined linear congruential generator) at the end of the return value, which increases the likelihood that the result will be unique.

Return Values

Returns timestamp based identifier as a string.

> **Warning**
>
> This function does not guarantee the uniqueness of the return value.

Examples

Example #1 uniqid() Example

```
<?php
/* A uniqid, like: 4b3403665fea6 */
```

```php
printf("uniqid(): %s\r\n", uniqid());

/* We can also prefix the uniqid, this the same as
 * doing:
 *
 * $uniqid = $prefix . uniqid();
 * $uniqid = uniqid($prefix);
 */
printf("uniqid('php_'): %s\r\n", uniqid('php_'));

/* We can also activate the more_entropy parameter, which is
 * required on some systems, like Cygwin. This makes uniqid()
 * produce a value like: 4b340550242239.64159797
 */
printf("uniqid('', true): %s\r\n", uniqid('', true));
?>
```

Notes

> **Note** :
>
> Under Cygwin, the `more_entropy` must be set to `true` for this function to work.

See Also

- random_bytes() - Get cryptographically secure random bytes

unpack

unpack

(PHP 4, PHP 5, PHP 7, PHP 8)

unpack — Unpack data from binary string

Description

function **unpack**(string `$format`, string `$string`, int `$offset` = 0): array|false

Unpacks from a binary string into an array according to the given `format`.

The unpacked data is stored in an associative array. To accomplish this you have to name the different format codes and separate them by a slash /. If a repeater argument is present, then each of the array keys will have a sequence number behind the given name.

Changes were made to bring this function into line with Perl:

- The "a" code now retains trailing NULL bytes.
- The "A" code now strips all trailing ASCII whitespace (spaces, tabs, newlines, carriage returns, and NULL bytes).
- The "Z" code was added for NULL-padded strings, and removes trailing NULL bytes.

Parameters

`format`

See pack() for an explanation of the format codes.

`string`

The packed data.

`offset`

The offset to begin unpacking from.

Return Values

Returns an associative array containing unpacked elements of binary string, or `false` on failure.

Changelog

Version	Description
7.2.0	float and double types supports both Big Endian and Little Endian.
7.1.0	The optional `offset` has been added.

Examples

Example #1 unpack() example

```
<?php
$binarydata = "\x04\x00\xa0\x00";
$array = unpack("cchars/nint", $binarydata);
print_r($array);
?>
```

The above example will output:

```
Array
(
    [chars] => 4
    [int] => 160
)
```

Example #2 unpack() example with a repeater

```
<?php
$binarydata = "\x04\x00\xa0\x00";
$array = unpack("c2chars/nint", $binarydata);
print_r($array);
?>
```

The above example will output:

```
Array
(
    [chars1] => 4
    [chars2] => 0
    [int] => 40960
)
```

Notes

> **Caution**
>
> Note that PHP internally stores integral values as signed. If you unpack a large unsigned long and it is of the same size as PHP internally stored values the result will be a negative number even though unsigned unpacking was specified.

> **Caution**
>
> If you do not name an element, numeric indices starting from `1` are used. Be aware that if you have more than one unnamed element, some data is overwritten because the numbering restarts from `1` for each element.
>
> **Example #3 unpack() example with unnamed keys**
>
> ```
> <?php
> $binarydata = "\x32\x42\x00\xa0";
> ```

```php
$array = unpack("c2/n", $binarydata);
var_dump($array);
?>
```

The above example will output:

```
array(2) {
  [1]=>
  int(160)
  [2]=>
  int(66)
}
```

Note that the first value from the `c` specifier is overwritten by the first value from the `n` specifier.

See Also

- pack() - Pack data into binary string

usleep

usleep

(PHP 4, PHP 5, PHP 7, PHP 8)

usleep — Delay execution in microseconds

Description

function **usleep**(int `$microseconds`): void

Delays program execution for the given number of microseconds.

Parameters

`microseconds`

Halt time in microseconds. A microsecond is one millionth of a second.

> **Note** :
>
> Values larger than `1000000` (i.e. sleeping for more than a second) may not be supported by the operating system. Use sleep() instead.

> **Note** :
>
> The sleep may be lengthened slightly (i.e. may be longer than `microseconds`) by any system activity or by the time spent processing the call or by the granularity of system timers.

Return Values

No value is returned.

Examples

Example #1 usleep() example

```php
<?php

// Current time
```

```php
echo (new DateTime('now'))->format('H:i:s.v'), "\n";

// wait for 2 milliseconds
usleep(2000);

// back!
echo (new DateTime('now'))->format('H:i:s.v'), "\n";

// wait for 30 milliseconds
usleep(30000);

// back again!
echo (new DateTime('now'))->format('H:i:s.v'), "\n";

?>
```

The above example will output:

```
11:13:28.005
11:13:28.007
11:13:28.037
```

See Also

- sleep() - Delay execution
- time_nanosleep() - Delay for a number of seconds and nanoseconds
- time_sleep_until() - Make the script sleep until the specified time
- set_time_limit() - Limits the maximum execution time

Changelog

The following changes have been made to classes/functions/methods of this extension.

Version	Function	Description
8.5.0	hrtime	On macOS, the implementation now uses clock_gettime_nsec_np(CLOCK_UPTIME_RAW) instead of mach_absolute_time().
8.4.0	exit	exit is now a proper function, therefore it follows the usual type juggling semantics is affected by the strict_types declare, can be called with named arguments, and be a variable functions.
	exit	A parameterless exit called within shutdown functions or object destructors now resets the exit code to 0; previously the exit code set by an earlier exit call was kept.
	highlight_string	Return type changed from stringbool to stringtrue.
8.3.0	highlight_file	The resulting HTML has changed.
	highlight_string	The resulting HTML has changed.
8.1.0	define	value can now be an object.
8.0.0	constant	If the constant is not defined, constant now throws an Error exception; previously an E_WARNING was generated, and null was returned.
	define	Passing true to case_insensitive now emits an E_WARNING. Passing false is still allowed.
	ignore_user_abort	enable is nullable now.
	pack	This function no longer returns false on failure.
	sapi_windows_vt100_support	enable is now nullable.
	sleep	The function throws a ValueError on negative seconds; previously, an E_WARNING was raised instead, and the function returned false.
7.3.0	define	case_insensitive has been deprecated and will be removed in version 8.0.0.
7.2.0	pack	float and double types supports both Big Endian and Little Endian.

Version	Function	Description
	unpack	float and double types supports both Big Endian and Little Endian.
7.1.1	pack	The "e", "E", "g" and "G" codes were added to enable byte order support for float and double.
7.1.0	unpack	The optional offset has been added.
7.0.15	pack	The "e", "E", "g" and "G" codes were added to enable byte order support for float and double.

Mail

Introduction

The mail() function allows you to send mail.

Installing/Configuring

Table of Contents

Requirements

For the Mail functions to be available, PHP must have access to the `sendmail` binary on your system during compile time. If you use another mail program, such as qmail or postfix, be sure to use the appropriate sendmail wrappers that come with them. PHP will first look for sendmail in your *PATH*, and then in the following: `/usr/bin:/usr/sbin:/usr/etc:/etc:/usr/ucblib:/usr/lib`. It's highly recommended to have sendmail available from your *PATH*. Also, the user that compiled PHP must have permission to access the sendmail binary.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

Mail configuration options

Name	Default	Changeable	Changelog
mail.add_x_header	"0"	`INI_PERDIR`	
mail.mixed_lf_and_crlf	"0"	`INI_SYSTEM`|`INI_PERDIR`	Added in PHP 8.2.4
mail.log	NULL	`INI_SYSTEM`|`INI_PERDIR`	
mail.force_extra_parameters	NULL	`INI_SYSTEM`	
SMTP	"localhost"	`INI_ALL`	
smtp_port	"25"	`INI_ALL`	
sendmail_from	NULL	`INI_ALL`	
sendmail_path	"/usr/sbin/sendmail -t -i"	`INI_SYSTEM`	

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`mail.add_x_header` bool

Add `X-PHP-Originating-Script` that will include UID of the script followed by the filename.

`mail.log` string

The path to a log file that will log all mail() calls. Log entries include the full path of the script, line number, `To` address and headers.

`mail.mixed_lf_and_crlf` bool

Allows reverting the line separator for email headers and message bodies to LF (Line Feed), mimicking the non-compliant behavior of PHP 7. It is provided as a compatibility measure for certain non-compliant Mail Transfer Agents (MTAs) that fail to correctly process CRLF (Carriage Return + Line Feed) as a line separator in email headers and message content.

`mail.force_extra_parameters` string

Force the addition of the specified parameters to be passed as extra parameters to the sendmail binary. These parameters will always replace the value of the 5th parameter to mail().

In addition to the default behavior for `INI_SYSTEM`, this value can also be set with `php_value` in *httpd.conf* (but this is not recommended).

`SMTP` string

Used under Windows only: host name or IP address of the SMTP server PHP should use for mail sent with the mail() function.

`smtp_port` int

Used under Windows only: Number of the port to connect to the server specified with the `SMTP` setting when sending mail with mail(); defaults to 25.

`sendmail_from` string

Which `"From:"` mail address should be used in mail sent directly via SMTP (Windows only). This directive also sets the `"Return-Path:"` header.

`sendmail_path` string

Where the **sendmail** program can be found, usually */usr/sbin/sendmail* or */usr/lib/sendmail*. **configure** does an honest attempt of locating this one for you and set a default, but if it fails, you can set it here.

Systems not using **sendmail** should set this directive to the sendmail wrapper/replacement their mail system offers, if any. For example, » Qmail users can normally set it to */var/qmail/bin/sendmail* or */var/qmail/bin/qmail-inject*.

qmail-inject does not require any option to process mail correctly.

This directive works also under Windows. If set, *smtp*, *smtp_port* and *sendmail_from* are ignored and the specified command is executed.

Mail Functions

Table of Contents

ezmlm_hash

ezmlm_hash

(PHP 4 >= 4.0.2, PHP 5, PHP 7)

ezmlm_hash — Calculate the hash value needed by EZMLM

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.4.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

function **ezmlm_hash**(string `$addr`): int

ezmlm_hash() calculates the hash value needed when keeping EZMLM mailing lists in a MySQL database.

Parameters

`addr`

The email address that's being hashed.

Return Values

The hash value of `addr`.

Changelog

Version	Description
8.0.0	Removed this function.
7.2.0	Deprecated this function.

Examples

Example #1 Calculating the hash and subscribing a user

```
<?php

$user = "joecool@example.com";
```

```php
$hash = ezmlm_hash($user);
$query = sprintf("INSERT INTO sample VALUES (%s, '%s')", $hash, $user);
$db->query($query); // using PHPLIB db interface

?>
```

mail

mail

(PHP 4, PHP 5, PHP 7, PHP 8)

mail — Send mail

Description

function **mail**(
  string $to,
  string $subject,
  string $message,
  array|string $additional_headers = [],
  string $additional_params = ""
): bool

Sends an email.

Parameters

`to`

Receiver, or receivers of the mail.

The formatting of this string must comply with » RFC 2822. Some examples are:

- user@example.com
- user@example.com, anotheruser@example.com
- User
- User , Another User

`subject`

Subject of the email to be sent.

> **Caution**
>
> Subject must satisfy » RFC 2047.

`message`

Message to be sent.

Each line should be separated with a CRLF (\r\n). Lines should not be larger than 70 characters.

> **Caution**
>
> (Windows only) When PHP is talking to a SMTP server directly, if a full stop is found on the start of a line, it is removed. To counter-act this, replace these occurrences with a double dot.
>
> ```php
> <?php
> $text = str_replace("\n.", "\n..", $text);
> ?>
> ```

`additional_headers` (optional)

String or array to be inserted at the end of the email header.

This is typically used to add extra headers (From, Cc, and Bcc). Multiple extra headers should be separated with a CRLF (\r\n). If outside data are used to compose this header, the data should be sanitized so that no unwanted headers could be injected.

If an array is passed, its keys are the header names and its values are the respective header values.

> **Note** :
>
> When sending mail, the mail *must* contain a `From` header. This can be set with the `additional_headers` parameter, or a default can be set in *php.ini*.
>
> Failing to do this will result in an error message similar to `Warning: mail(): "sendmail_from" not set in php.ini or custom "From:" header missing`. The `From` header sets also `Return-Path` when sending directly via SMTP (Windows only).

> **Note** :
>
> If messages are not received, try using a LF (\n) only. Some Unix mail transfer agents (most notably » qmail) replace LF by CRLF automatically (which leads to doubling CR if CRLF is used). This should be a last resort, as it does not comply with » RFC 2822.

`additional_params` (optional)

The `additional_params` parameter can be used to pass additional flags as command line options to the program configured to be used when sending mail, as defined by the `sendmail_path` configuration setting. For example, this can be used to set the envelope sender address when using sendmail with the `-f` sendmail option.

This parameter is escaped by escapeshellcmd() internally to prevent command execution. escapeshellcmd() prevents command execution, but allows to add additional parameters. For security reasons, it is recommended for the user to sanitize this parameter to avoid adding unwanted parameters to the shell command.

Since escapeshellcmd() is applied automatically, some characters that are allowed as email addresses by internet RFCs cannot be used. **mail()** can not allow such characters, so in programs where the use of such characters is required, alternative means of sending emails (such as using a framework or a library) is recommended.

The user that the webserver runs as should be added as a trusted user to the sendmail configuration to prevent a 'X-Warning' header from being added to the message when the envelope sender (-f) is set using this method. For sendmail users, this file is */etc/mail/trusted-users*.

Return Values

Returns `true` if the mail was successfully accepted for delivery, `false` otherwise.

It is important to note that just because the mail was accepted for delivery, it does NOT mean the mail will actually reach the intended destination.

Changelog

Version	Description
7.2.0	The `additional_headers` parameter now also accepts an array.

Examples

Example #1 Sending mail.

Using **mail()** to send a simple email:

```
<?php
// The message
$message = "Line 1\r\nLine 2\r\nLine 3";

// In case any of our lines are larger than 70 characters, we should use wordwrap()
$message = wordwrap($message, 70, "\r\n");

// Send
mail('caffeinated@example.com', 'My Subject', $message);
?>
```

Example #2 Sending mail with extra headers.

The addition of basic headers, telling the MUA the From and Reply-To addresses:

```php
<?php
$to      = 'nobody@example.com';
$subject = 'the subject';
$message = 'hello';
$headers = 'From: webmaster@example.com' . "\r\n" .
    'Reply-To: webmaster@example.com' . "\r\n" .
    'X-Mailer: PHP/' . phpversion();

mail($to, $subject, $message, $headers);
?>
```

Example #3 Sending mail with extra headers as array

This example sends the same mail as the example immediately above, but passes the additional headers as array (available as of PHP 7.2.0).

```php
<?php
$to      = 'nobody@example.com';
$subject = 'the subject';
$message = 'hello';
$headers = array(
    'From' => 'webmaster@example.com',
    'Reply-To' => 'webmaster@example.com',
    'X-Mailer' => 'PHP/' . phpversion()
);

mail($to, $subject, $message, $headers);
?>
```

Example #4 Sending mail with an additional command line parameter.

The `additional_params` parameter can be used to pass an additional parameter to the program configured to use when sending mail using the `sendmail_path`.

```php
<?php
mail('nobody@example.com', 'the subject', 'the message', null,
   '-fwebmaster@example.com');
?>
```

Example #5 Sending HTML email

It is also possible to send HTML email with **mail()**.

```php
<?php
// Multiple recipients
$to = 'johny@example.com, sally@example.com'; // note the comma

// Subject
$subject = 'Birthday Reminders for August';

// Message
$message = '
<html>
<head>
  <title>Birthday Reminders for August</title>
</head>
<body>
  <p>Here are the birthdays upcoming in August!</p>
  <table>
    <tr>
      <th>Person</th><th>Day</th><th>Month</th><th>Year</th>
    </tr>
    <tr>
      <td>Johny</td><td>10th</td><td>August</td><td>1970</td>
    </tr>
    <tr>
      <td>Sally</td><td>17th</td><td>August</td><td>1973</td>
    </tr>
  </table>
</body>
```

```
</html>
';

// To send HTML mail, the Content-type header must be set
$headers[] = 'MIME-Version: 1.0';
$headers[] = 'Content-type: text/html; charset=iso-8859-1';

// Additional headers
$headers[] = 'To: Mary <mary@example.com>, Kelly <kelly@example.com>';
$headers[] = 'From: Birthday Reminder <birthday@example.com>';
$headers[] = 'Cc: birthdayarchive@example.com';
$headers[] = 'Bcc: birthdaycheck@example.com';

// Mail it
mail($to, $subject, $message, implode("\r\n", $headers));
?>
```

> **Note** :
>
> If intending to send HTML or otherwise Complex mails, it is recommended to use the PEAR package » PEAR::Mail_Mime.

Notes

> **Note** :
>
> The SMTP implementation (Windows only) of **mail()** differs in many ways from the sendmail implementation. First, it doesn't use a local binary for composing messages but only operates on direct sockets which means a `MTA` is needed listening on a network socket (which can either on the localhost or a remote machine).
>
> Second, the custom headers like `From:`, `Cc:`, `Bcc:` and `Date:` are **not** interpreted by the `MTA` in the first place, but are parsed by PHP.
>
> As such, the `to` parameter should not be an address in the form of "Something <someone@example.com>". The mail command may not parse this properly while talking with the MTA.

> **Note** :
>
> It is worth noting that the **mail()** function is not suitable for larger volumes of email in a loop. This function opens and closes an SMTP socket for each email, which is not very efficient.
>
> For the sending of large amounts of email, see the » PEAR::Mail, and » PEAR::Mail_Queue packages.

> **Note** :
>
> The following RFCs may be useful: » RFC 1896, » RFC 2045, » RFC 2046, » RFC 2047, » RFC 2048, » RFC 2049, and » RFC 2822.

See Also

- mb_send_mail() - Send encoded mail
- imap_mail() - Send an email message
- » PEAR::Mail
- » PEAR::Mail_Mime

Client URL Library

Introduction

PHP supports libcurl, a library created by Daniel Stenberg, that allows you to connect and communicate to many different types of servers with many different types of protocols. libcurl currently supports the http, https, ftp, gopher, telnet, dict, file, and ldap protocols. libcurl also supports HTTPS certificates, HTTP POST, HTTP PUT, FTP uploading (this can also be done with PHP's ftp extension), HTTP form based upload, proxies, cookies, and user+password authentication.

Installing/Configuring

Table of Contents

Requirements

In order to use PHP's cURL functions you need to install the » libcurl package. PHP requires libcurl version 7.10.5 or later. As of PHP 7.3.0, version 7.15.5 or later is required. As of PHP 8.0.0, version 7.29.0 or later is required. As of PHP 8.4.0, version 7.61.0 or later is required.

Installation

To use PHP's cURL support you must also compile PHP **--with-curl[=DIR]** where DIR is the location of the directory containing the *lib* and *include* directories. In the *include* directory there should be a folder named *curl* which should contain the *easy.h* and *curl.h* files. There should be a file named *libcurl.a* located in the *lib* directory.

> **Note** : **Note to Win32 Users**
>
> In order to enable this module on a Windows environment, *libeay32.dll* and *ssleay32.dll*, or, as of OpenSSL 1.1 *libcrypto-*.dll* and *libssl-*.dll*, must be present in your *PATH*. Also *libssh2.dll* must be present in your *PATH*.
>
> You don't need *libcurl.dll* from the cURL site.

Runtime Configuration

The behaviour of these functions is affected by settings in *php.ini*.

cURL Configuration Options

Name	Default	Changeable	Changelog
curl.cainfo	NULL	`INI_SYSTEM`	

For further details and definitions of the INI_* modes, see the Where a configuration setting may be set.

Here's a short explanation of the configuration directives.

`curl.cainfo` string

A default value for the `CURLOPT_CAINFO` option. This is required to be an absolute path.

Resource Types

Prior to PHP 8.0.0, this extension defined three resource types: a `curl` handle, a `curl_multi` handle, and a `curl_share` handle.

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

Descriptions and uses for these constants are described within the curl_setopt(), curl_multi_setopt() and curl_getinfo() documentation.

Constants	Description

CURLALTSVC_H1 (int)	Available as of PHP 8.2.0 and cURL 7.64.1.
CURLALTSVC_H2 (int)	Available as of PHP 8.2.0 and cURL 7.64.1.
CURLALTSVC_H3 (int)	Available as of PHP 8.2.0 and cURL 7.64.1.
CURLALTSVC_READONLYFILE (int)	Available as of PHP 8.2.0 and cURL 7.64.1.
CURLAUTH_ANY (int)	
CURLAUTH_ANYSAFE (int)	
CURLAUTH_AWS_SIGV4 (int)	Available as of PHP 8.2.0 and cURL 7.75.0.
CURLAUTH_BASIC (int)	
CURLAUTH_BEARER (int)	Available as of PHP 7.3.0 and cURL 7.61.0.
CURLAUTH_DIGEST (int)	
CURLAUTH_DIGEST_IE (int)	Use HTTP Digest authentication with an IE flavor. Available as of cURL 7.19.3.
CURLAUTH_GSSAPI (int)	Available as of PHP 7.3.0 and cURL 7.54.1
CURLAUTH_GSSNEGOTIATE (int)	
CURLAUTH_NEGOTIATE (int)	Available as of PHP 7.0.7 and cURL 7.38.0.
CURLAUTH_NONE (int)	Available as of cURL 7.10.6.
CURLAUTH_NTLM (int)	
CURLAUTH_NTLM_WB (int)	Available as of PHP 7.0.7 and cURL 7.22.0
CURLAUTH_ONLY (int)	This is a meta symbol. OR this value together with a single specific auth value to force libcurl to probe for unrestricted auth and if not, only that single auth algorithm is acceptable. Available as of cURL 7.21.3.
CURLFOLLOW_ALL (int)	Value for CURLOPT_FOLLOWLOCATION that enables following redirects and keeps a custom request method set with CURLOPT_CUSTOMREQUEST in use for all requests, even after redirects. Available as of PHP 8.5.0 and cURL 8.13.0.
CURLFOLLOW_OBEYCODE (int)	Value for CURLOPT_FOLLOWLOCATION that enables following redirects while obeying the HTTP response code: a custom request method set with CURLOPT_CUSTOMREQUEST is kept, except that it is changed to GET on the redirect status codes that require it (such as 301, 302 and 303). Available as of PHP 8.5.0 and cURL 8.13.0.
CURLFOLLOW_FIRSTONLY (int)	Value for CURLOPT_FOLLOWLOCATION that enables following redirects but uses a custom request method set with CURLOPT_CUSTOMREQUEST for the first request only; subsequent requests follow the method dictated by the redirect response code. Available as of PHP 8.5.0 and cURL 8.13.0.
CURLFTPAUTH_DEFAULT (int)	
CURLFTPAUTH_SSL (int)	

CURLFTPAUTH_TLS (int)	
CURLFTPMETHOD_DEFAULT (int)	Available as of PHP 8.2.0 and cURL 7.15.3.
CURLFTPMETHOD_MULTICWD (int)	Do a single CWD operation for each path part in the given URL. Available as of cURL 7.15.3.
CURLFTPMETHOD_NOCWD (int)	libcurl makes no CWD at all. libcurl does SIZE, RETR, STOR etc. and gives a full path to the server for all these commands. Available as of cURL 7.15.3.
CURLFTPMETHOD_SINGLECWD (int)	libcurl does one CWD with the full target directory and then operates on the file like in the multicwd case. Available as of cURL 7.15.3.
CURLFTPSSL_ALL (int)	
CURLFTPSSL_CCC_ACTIVE (int)	Initiate the shutdown and wait for a reply. Available as of cURL 7.16.2.
CURLFTPSSL_CCC_NONE (int)	Do not attempt to use CCC (Clear Command Channel). Available as of cURL 7.16.2.
CURLFTPSSL_CCC_PASSIVE (int)	Do not initiate the shutdown, but wait for the server to do it. Do not send a reply. Available as of cURL 7.16.1.
CURLFTPSSL_CONTROL (int)	
CURLFTPSSL_NONE (int)	
CURLFTPSSL_TRY (int)	
CURLFTP_CREATE_DIR (int)	Available as of PHP 7.0.7 and cURL 7.19.3
CURLFTP_CREATE_DIR_NONE (int)	Available as of PHP 7.0.7 and cURL 7.19.3
CURLFTP_CREATE_DIR_RETRY (int)	Available as of PHP 7.0.7 and cURL 7.19.3
CURLGSSAPI_DELEGATION_FLAG (int)	Allow unconditional GSSAPI credential delegation. Available as of cURL 7.22.0.
CURLGSSAPI_DELEGATION_POLICY_FLAG (int)	Delegate only if the OK-AS-DELEGATE flag is set in the service ticket if this feature is supported by the GSS-API implementation and the definition of GSS_C_DELEG_POLICY_FLAG was available at compile-time. Available as of cURL 7.22.0.
CURLHEADER_SEPARATE (int)	Available as of PHP 7.0.7 and cURL 7.37.0.
CURLHEADER_UNIFIED (int)	Available as of PHP 7.0.7 and cURL 7.37.0.
CURLHSTS_ENABLE (int)	Available as of PHP 8.2.0 and cURL 7.74.0
CURLHSTS_READONLYFILE (int)	Available as of PHP 8.2.0 and cURL 7.74.0
CURLKHMATCH_LAST (int)	Available as of PHP 8.3.0 and cURL 7.19.6
CURLKHMATCH_MISMATCH (int)	Available as of PHP 8.3.0 and cURL 7.19.6

CURLKHMATCH_MISSING (int)	Available as of PHP 8.3.0 and cURL 7.19.6
CURLKHMATCH_OK (int)	Available as of PHP 8.3.0 and cURL 7.19.6
CURLMIMEOPT_FORMESCAPE (int)	Available as of PHP 8.3.0 and cURL 7.81.0
CURLMSG_DONE (int)	
CURLPIPE_HTTP1 (int)	Available as of cURL 7.43.0.
CURLPIPE_MULTIPLEX (int)	Available as of cURL 7.43.0.
CURLPIPE_NOTHING (int)	Available as of cURL 7.43.0.
CURLPROXY_HTTP (int)	Available as of cURL 7.10.
CURLPROXY_HTTPS (int)	Available as of PHP 7.3.0 and cURL 7.52.0
CURLPROXY_HTTP_1_0 (int)	Available as of PHP 7.0.7 and cURL 7.19.3
CURLPROXY_SOCKS4 (int)	Available as of cURL 7.10.
CURLPROXY_SOCKS4A (int)	Available as of cURL 7.18.0.
CURLPROXY_SOCKS5 (int)	Available as of cURL 7.10.
CURLPROXY_SOCKS5_HOSTNAME (int)	Available as of cURL 7.18.0.
CURLPX_BAD_ADDRESS_TYPE (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_BAD_VERSION (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_CLOSED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_GSSAPI (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_GSSAPI_PERMSG (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_GSSAPI_PROTECTION (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_IDENTD (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_IDENTD_DIFFER (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_LONG_HOSTNAME (int)	Available as of PHP 8.2.0 and cURL 7.73.0

CURLPX_LONG_PASSWD (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_LONG_USER (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_NO_AUTH (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_OK (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_RECV_ADDRESS (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_RECV_AUTH (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_RECV_CONNECT (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_RECV_REQACK (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_ADDRESS_TYPE_NOT_SUPPORTED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_COMMAND_NOT_SUPPORTED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_CONNECTION_REFUSED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_GENERAL_SERVER_FAILURE (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_HOST_UNREACHABLE (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_NETWORK_UNREACHABLE (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_NOT_ALLOWED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_TTL_EXPIRED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REPLY_UNASSIGNED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_REQUEST_FAILED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_RESOLVE_HOST (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_SEND_AUTH (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_SEND_CONNECT (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_SEND_REQUEST (int)	Available as of PHP 8.2.0 and cURL 7.73.0

CURLPX_UNKNOWN_FAIL (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_UNKNOWN_MODE (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLPX_USER_REJECTED (int)	Available as of PHP 8.2.0 and cURL 7.73.0
CURLSSH_AUTH_AGENT (int)	Available as of PHP 7.0.7 and cURL 7.28.0
CURLSSH_AUTH_ANY (int)	
CURLSSH_AUTH_DEFAULT (int)	
CURLSSH_AUTH_GSSAPI (int)	Available as of PHP 7.3.0 and cURL 7.58.0
CURLSSH_AUTH_HOST (int)	
CURLSSH_AUTH_KEYBOARD (int)	
CURLSSH_AUTH_NONE (int)	
CURLSSH_AUTH_PASSWORD (int)	
CURLSSH_AUTH_PUBLICKEY (int)	
CURLSSLOPT_ALLOW_BEAST (int)	Available as of cURL 7.25.0
CURLSSLOPT_AUTO_CLIENT_CERT (int)	Available as of PHP 8.2.0 and cURL 7.77.0
CURLSSLOPT_NATIVE_CA (int)	Available as of PHP 8.2.0 and cURL 7.71.0
CURLSSLOPT_NO_PARTIALCHAIN (int)	Available as of PHP 8.2.0 and cURL 7.68.0
CURLSSLOPT_NO_REVOKE (int)	Available as of PHP 7.0.7 and cURL 7.44.0
CURLSSLOPT_REVOKE_BEST_EFFORT (int)	Available as of PHP 8.2.0 and cURL 7.70.0
CURLUSESSL_ALL (int)	Require SSL for all communication or fail with **CURLE_USE_SSL_FAILED**. Available as of cURL 7.17.0.
CURLUSESSL_CONTROL (int)	Require SSL for the control connection or fail with **CURLE_USE_SSL_FAILED**. Available as of cURL 7.17.0.
CURLUSESSL_NONE (int)	Do not attempt to use SSL. Available as of cURL 7.17.0.
CURLUSESSL_TRY (int)	Try using SSL, proceed as normal otherwise. Note that server may close the connection if the negotiation does not succeed. Available as of cURL 7.17.0.
CURLVERSION_NOW (int)	
CURLWS_RAW_MODE (int)	Available as of PHP 8.3.0 and cURL 7.86.0
CURL_FNMATCHFUNC_FAIL (int)	Returned by the wildcard match callback function if an error occurred. Available as of cURL 7.21.0.

CURL_FNMATCHFUNC_MATCH (int)	Returned by the wildcard match callback function if pattern matches the string. Available as of cURL 7.21.0.
CURL_FNMATCHFUNC_NOMATCH (int)	Returned by the wildcard match callback function if pattern does not match the string. Available as of cURL 7.21.0.
CURL_HTTP_VERSION_1_0 (int)	
CURL_HTTP_VERSION_1_1 (int)	
CURL_HTTP_VERSION_2 (int)	Available as of PHP 7.0.7 and cURL 7.43.0
CURL_HTTP_VERSION_2TLS (int)	Available as of PHP 7.0.7 and cURL 7.47.0
CURL_HTTP_VERSION_2_0 (int)	Available as of cURL 7.33.0
CURL_HTTP_VERSION_2_PRIOR_KNOWLEDGE (int)	Available as of PHP 7.0.7 and cURL 7.49.0
CURL_HTTP_VERSION_3 (int)	Available as of PHP 8.4.0 and cURL 7.66.0.
CURL_HTTP_VERSION_3ONLY (int)	Available as of PHP 8.4.0 and cURL 7.88.0.
CURL_HTTP_VERSION_NONE (int)	
CURL_IPRESOLVE_V4 (int)	Use only IPv4 addresses when establishing a connection or choosing one from the connection pool. Available as of cURL 7.10.8.
CURL_IPRESOLVE_V6 (int)	Use only IPv6 addresses when establishing a connection or choosing one from the connection pool. Available as of cURL 7.10.8.
CURL_IPRESOLVE_WHATEVER (int)	Use addresses of all IP versions allowed by the system. Available as of cURL 7.10.8.
CURL_MAX_READ_SIZE (int)	Available as of PHP 7.3.0 and cURL 7.53.0
CURL_NETRC_IGNORED (int)	
CURL_NETRC_OPTIONAL (int)	
CURL_NETRC_REQUIRED (int)	
CURL_PUSH_DENY (int)	Available as of PHP 7.1.0 and cURL 7.44.0
CURL_PUSH_OK (int)	Available as of PHP 7.1.0 and cURL 7.44.0
CURL_READFUNC_PAUSE (int)	Available as of cURL 7.18.0.
CURL_REDIR_POST_301 (int)	Available as of PHP 7.0.7 and cURL 7.18.2
CURL_REDIR_POST_302 (int)	Available as of PHP 7.0.7 and cURL 7.18.2
CURL_REDIR_POST_303 (int)	Available as of PHP 7.0.7 and cURL 7.25.1

CURL_REDIR_POST_ALL (int)	Available as of PHP 7.0.7 and cURL 7.18.2
CURL_RTSPREQ_ANNOUNCE (int)	When sent by a client, this method changes the description of the session. ANNOUNCE acts like an HTTP PUT or POST just like CURL_RTSPREQ_SET_PARAMETER. Available as of cURL 7.20.0.
CURL_RTSPREQ_DESCRIBE (int)	Used to get the low level description of a stream. The application should note what formats it understands in the Accept: header. Unless set manually, libcurl automatically adds in Accept: application/sdp. Time-condition headers are added to DESCRIBE requests if the CURLOPT_TIMECONDITION option is used. Available as of cURL 7.20.0.
CURL_RTSPREQ_GET_PARAMETER (int)	Retrieve a parameter from the server. By default, libcurl adds a Content-Type: text/parameters header on all non-empty requests unless a custom one is set. GET_PARAMETER acts just like an HTTP PUT or POST. Applications wishing to send a heartbeat message should use an empty GET_PARAMETER request. Available as of cURL 7.20.0.
CURL_RTSPREQ_OPTIONS (int)	Used to retrieve the available methods of the server. Available as of cURL 7.20.0.
CURL_RTSPREQ_PAUSE (int)	Send a PAUSE command to the server. Use the CURLOPT_RANGE option with a single value to indicate when the stream should be halted (e.g. npt=25). Available as of cURL 7.20.0.
CURL_RTSPREQ_PLAY (int)	Send a PLAY command to the server. Use the CURLOPT_RANGE option to modify the playback time (e.g. npt=10-15). Available as of cURL 7.20.0.
CURL_RTSPREQ_RECEIVE (int)	Set the RTSP request type to this value to receive interleaved RTP data. Available as of cURL 7.20.0.
CURL_RTSPREQ_RECORD (int)	Used to tell the server to record a session. Use the CURLOPT_RANGE option to modify the record time. Available as of cURL 7.20.0.
CURL_RTSPREQ_SETUP (int)	Used to initialize the transport layer for the session. Available as of cURL 7.20.0.
CURL_RTSPREQ_SET_PARAMETER (int)	Set a parameter on the server. Available as of cURL 7.20.0.
CURL_RTSPREQ_TEARDOWN (int)	Terminates an RTSP session. Simply closing a connection does not terminate the RTSP session since it is valid to control an RTSP session over different connections. Available as of cURL 7.20.0.
CURL_SSLVERSION_DEFAULT (int)	
CURL_SSLVERSION_MAX_DEFAULT (int)	Available as of PHP 7.3.0 and cURL 7.54.0
CURL_SSLVERSION_MAX_NONE (int)	Available as of PHP 7.3.0 and cURL 7.54.0
CURL_SSLVERSION_MAX_TLSv1_0 (int)	Available as of PHP 7.3.0 and cURL 7.54.0
CURL_SSLVERSION_MAX_TLSv1_1 (int)	Available as of PHP 7.3.0 and cURL 7.54.0
CURL_SSLVERSION_MAX_TLSv1_2 (int)	Available as of PHP 7.3.0 and cURL 7.54.0

`CURL_SSLVERSION_MAX_TLSv1_3` (int)	Available as of PHP 7.3.0 and cURL 7.54.0
`CURL_SSLVERSION_SSLv2` (int)	
`CURL_SSLVERSION_SSLv3` (int)	
`CURL_SSLVERSION_TLSv1` (int)	
`CURL_SSLVERSION_TLSv1_0` (int)	
`CURL_SSLVERSION_TLSv1_1` (int)	
`CURL_SSLVERSION_TLSv1_2` (int)	
`CURL_SSLVERSION_TLSv1_3` (int)	Available as of PHP 7.3.0 and cURL 7.52.0
`CURL_TIMECOND_IFMODSINCE` (int)	
`CURL_TIMECOND_IFUNMODSINCE` (int)	
`CURL_TIMECOND_LASTMOD` (int)	
`CURL_TIMECOND_NONE` (int)	
`CURL_TLSAUTH_SRP` (int)	Available as of cURL 7.21.4.
`CURL_VERSION_ALTSVC` (int)	Available as of PHP 7.3.6 and cURL 7.64.1
`CURL_VERSION_ASYNCHDNS` (int)	Asynchronous DNS resolves. Available as of PHP 7.3.0 and cURL 7.10.7
`CURL_VERSION_BROTLI` (int)	Available as of PHP 7.3.0 and cURL 7.57.0
`CURL_VERSION_CONV` (int)	Character conversions supported. Available as of PHP 7.3.0 and cURL 7.15.4
`CURL_VERSION_CURLDEBUG` (int)	Debug memory tracking supported. Available as of PHP 7.3.6 and cURL 7.19.6
`CURL_VERSION_DEBUG` (int)	Built with debug capabilities. Available as of PHP 7.3.0 and cURL 7.10.6
`CURL_VERSION_GSASL` (int)	Available as of PHP 8.2.0 and cURL 7.76.0
`CURL_VERSION_GSSAPI` (int)	Built against a GSS-API library. Available as of PHP 7.3.0 and cURL 7.38.0
`CURL_VERSION_GSSNEGOTIATE` (int)	Negotiate auth is supported. Available as of PHP 7.3.0 and cURL 7.10.6 (deprecated as of cURL 7.38.0)
`CURL_VERSION_HSTS` (int)	Available as of PHP 8.2.0 and cURL 7.74.0
`CURL_VERSION_HTTP2` (int)	HTTP2 support built-in. Available as of cURL 7.33.0
`CURL_VERSION_HTTP3` (int)	Available as of PHP 8.2.0 and cURL 7.66.0
`CURL_VERSION_HTTPS_PROXY` (int)	Available as of PHP 7.3.0 and cURL 7.52.0
`CURL_VERSION_IDN` (int)	Internationalized Domain Names are supported. Available as of PHP 7.3.0 and cURL 7.12.0

`CURL_VERSION_IPV6` (int)	IPv6-enabled.
`CURL_VERSION_KERBEROS4` (int)	Kerberos V4 auth is supported.
`CURL_VERSION_KERBEROS5` (int)	Kerberos V5 auth is supported. Available as of PHP 7.0.7 and cURL 7.40.0
`CURL_VERSION_LARGEFILE` (int)	Supports files larger than 2GB. Available as of cURL 7.33.0
`CURL_VERSION_LIBZ` (int)	libz features are present.
`CURL_VERSION_MULTI_SSL` (int)	Available as of PHP 7.3.0 and cURL 7.56.0
`CURL_VERSION_NTLM` (int)	NTLM auth is supported. Available as of PHP 7.3.0 and cURL 7.10.6
`CURL_VERSION_NTLM_WB` (int)	NTLM delegation to winbind helper is supported. Available as of PHP 7.3.0 and cURL 7.22.0
`CURL_VERSION_PSL` (int)	Mozilla's Public Suffix List, used for cookie domain verification. Available as of PHP 7.3.6 and cURL 7.47.0
`CURL_VERSION_SPNEGO` (int)	SPNEGO auth is supported. Available as of PHP 7.3.0 and cURL 7.10.8
`CURL_VERSION_SSL` (int)	SSL options are present.
`CURL_VERSION_SSPI` (int)	Built against Windows SSPI. Available as of PHP 7.3.0 and cURL 7.13.2
`CURL_VERSION_TLSAUTH_SRP` (int)	TLS-SRP auth is supported. Available as of PHP 7.3.0 and cURL 7.21.4
`CURL_VERSION_UNICODE` (int)	Available as of PHP 8.2.0 and cURL 7.72.0
`CURL_VERSION_UNIX_SOCKETS` (int)	Unix domain sockets support. Available as of PHP 7.0.7 and cURL 7.40.0
`CURL_VERSION_ZSTD` (int)	Available as of PHP 8.2.0 and cURL 7.72.0
`CURL_WRITEFUNC_PAUSE` (int)	Available as of cURL 7.18.0.
`CURL_PREREQFUNC_OK` (int)	Available as of PHP 8.4.0 and cURL 7.80.0.
`CURL_PREREQFUNC_ABORT` (int)	Available as of PHP 8.4.0 and cURL 7.80.0.

curl_setopt()

Constants	Description
`CURLOPT_ABSTRACT_UNIX_SOCKET` (int)	Enables the use of an abstract Unix domain socket instead of establishing a TCP connection to a host and sets the path to the given string. This option shares the same semantics as `CURLOPT_UNIX_SOCKET_PATH`. These two options share the same storage and therefore only one of them can be set per handle. Available as of PHP 7.3.0 and cURL 7.53.0.

CURLOPT_ACCEPT_ENCODING (int)	Sets a string with the contents of the `Accept-Encoding:` header sent in an HTTP request. Set to **null** to disable sending the `Accept-Encoding:` header. Defaults to **null**. Available as of cURL 7.21.6.
CURLOPT_ACCEPTTIMEOUT_MS (int)	The maximum number of milliseconds to wait for a server to connect back to cURL when an active FTP connection is used. This option accepts any value that can be cast to a valid int. Defaults to `60000` milliseconds. Available as of cURL 7.24.0.
CURLOPT_ADDRESS_SCOPE (int)	The scope id value to use when connecting to IPv6 addresses. This option accepts any value that can be cast to a valid int. Defaults to `0`. Available as of cURL 7.19.0.
CURLOPT_ALTSVC (int)	Pass a string with the filename for cURL to use as the Alt-Svc cache file to read existing cache contents from and possibly also write it back to a after a transfer, unless **CURLALTSVC_READONLYFILE** is set via **CURLOPT_ALTSVC_CTRL**. Available as of PHP 8.2.0 and cURL 7.64.1.
CURLOPT_ALTSVC_CTRL (int)	Populate the bitmask with the correct set of features to instruct cURL how to handle Alt-Svc for the transfers using this handle. cURL only accepts Alt-Svc headers over HTTPS. It will also only complete a request to an alternative origin if that origin is properly hosted over HTTPS. Setting any bit will enable the alt-svc engine. Set to any of the **CURLALTSVC_*** constants. Defaults to Alt-Svc handling being disabled. Available as of PHP 8.2.0 and cURL 7.64.1.
CURLOPT_APPEND (int)	Setting this option to `1` will cause FTP uploads to append to the remote file instead of overwriting it. Defaults to `0`. Available as of cURL 7.17.0.
CURLOPT_AUTOREFERER (int)	**true** to automatically set the `Referer:` field in requests where it follows a `Location:` redirect. Defaults to `0`. Available as of cURL 7.1.0.
CURLOPT_AWS_SIGV4 (int)	Provides AWS V4 signature authentication on HTTP(S) header as a string. This option overrides any other authentication types that have been set in **CURLOPT_HTTPAUTH**. This method cannot be combined with other authentication types. Available as of PHP 8.2.0 and cURL 7.75.0.
CURLOPT_BINARYTRANSFER (int)	This constant is no longer used as of PHP 5.5.0. Deprecated as of PHP 8.4.0.
CURLOPT_BUFFERSIZE (int)	The size of the buffer to use for each read. There is no guarantee this request will be fulfilled, however. This option accepts any value that can be cast to a valid int. Defaults to **CURL_MAX_WRITE_SIZE** (currently, 16kB). Available as of cURL 7.10.
CURLOPT_CAINFO (int)	A string with the name of a file holding one or more certificates to verify the peer with. This only makes sense when used in combination with **CURLOPT_SSL_VERIFYPEER**. Might require an absolute path. Available as of cURL 7.4.2.
CURLOPT_CAINFO_BLOB (int)	A binary string of PEM encoded content holding one or more certificates to verify the peer with. This option overrides **CURLOPT_CAINFO**. Available as of PHP 8.2.0 and cURL 7.77.0.
CURLOPT_CAPATH (int)	A string with a directory that holds multiple CA certificates. Use this option alongside **CURLOPT_SSL_VERIFYPEER**. Available as of cURL 7.9.8.
CURLOPT_CA_CACHE_TIMEOUT (int)	Sets the maximum time in seconds any in memory cached CA certificate store may be kept and reused for new connections. This option accepts any value that can be cast to a valid int. Defaults to `86400` (24 hours). Available as of PHP 8.3.0 and cURL 7.87.0

CURLOPT_CERTINFO (int)	**true** to output SSL certification information to **STDERR** on secure transfers. Requires **CURLOPT_VERBOSE** to be on to have an effect. Defaults to **false**. Available as of cURL 7.19.1.
CURLOPT_CONNECTTIMEOUT (int)	The number of seconds to wait while trying to connect. Use `0` to wait indefinitely. This option accepts any value that can be cast to a valid int. Defaults to `300`. Available as of cURL 7.7.0.
CURLOPT_CONNECTTIMEOUT_MS (int)	The number of milliseconds to wait while trying to connect. Use `0` to wait indefinitely. If cURL is built to use the standard system name resolver, that portion of the connect will still use full-second resolution for timeouts with a minimum timeout allowed of one second. This option accepts any value that can be cast to a valid int. Defaults to `300000`. Available as of cURL 7.16.2.
CURLOPT_CONNECT_ONLY (int)	**true** tells the library to perform all the required proxy authentication and connection setup, but no data transfer. This option is implemented for HTTP, SMTP and POP3. Defaults to **false**. Available as of cURL 7.15.2.
CURLOPT_CONNECT_TO (int)	Connect to a specific host and port instead of the URL's host and port. Accepts an array of strings with the format `HOST:PORT:CONNECT-TO-HOST:CONNECT-TO-PORT`. Available as of PHP 7.0.7 and cURL 7.49.0.
CURLOPT_COOKIE (int)	A string with the contents of the `Cookie:` header to be used in the HTTP request. Note that multiple cookies are separated with a semicolon followed by a space (e.g., `fruit=apple; colour=red`). Available as of cURL 7.1.0.
CURLOPT_COOKIEFILE (int)	A string with the name of the file containing the cookie data. The cookie file can be in Netscape format, or just plain HTTP-style headers dumped into a file. If the name is an empty string, no cookies are loaded, but cookie handling is still enabled. Available as of cURL 7.1.0.
CURLOPT_COOKIEJAR (int)	A string with the name of a file to save all internal cookies to when the handle's destructor is called. Available as of cURL 7.9.0. **Warning** As of PHP 8.0.0, curl_close() is a no-op and does *not* destroy the handle. If cookies need to be written prior to the handle being automatically destroyed, run `curl_setopt($ch, CURLOPT_COOKIELIST, "FLUSH");`.
CURLOPT_COOKIELIST (int)	A cookie string (i.e. a single line in Netscape/Mozilla format, or a regular HTTP-style Set-Cookie header) adds that single cookie to the internal cookie store. `ALL` erases all cookies held in memory , `SESS` erases all session cookies held in memory , `FLUSH` writes all known cookies to the file specified by **CURLOPT_COOKIEJAR** , `RELOAD` loads all cookies from the files specified by **CURLOPT_COOKIEFILE** . Available as of cURL 7.14.1.
CURLOPT_COOKIESESSION (int)	**true** to mark this as a new cookie "session". It will force cURL to ignore all cookies it is about to load that are "session cookies" from the previous session. By default, cURL always stores and loads all cookies, independent if they are session cookies or not. Session cookies are cookies without expiry date and they are meant to be alive and existing for this "session" only. Available as of cURL 7.9.7.
CURLOPT_CRLF (int)	**true** to convert Unix newlines to CRLF newlines on transfers. Available as of cURL 7.1.0.
CURLOPT_CRLFILE (int)	Pass a string naming a file with the concatenation of CRL (Certificate Revocation List) (in PEM format) to use in the certificate validation that occurs during the SSL exchange. When cURL is built to use GnuTLS, there is no way to influence the use of CRL passed to help in the verification

	process. When cURL is built with OpenSSL support, `X509_V_FLAG_CRL_CHECK` and `X509_V_FLAG_CRL_CHECK_ALL` are both set, requiring CRL check against all the elements of the certificate chain if a CRL file is passed. Also note that **`CURLOPT_CRLFILE`** implies **`CURLSSLOPT_NO_PARTIALCHAIN`** as of cURL 7.71.0 due to an OpenSSL bug. Available as of cURL 7.19.0.
`CURLOPT_CUSTOMREQUEST` (int)	A custom request method to use instead of `GET` or `HEAD` when doing a HTTP request. This is useful for doing `DELETE` or other, more obscure HTTP requests. Valid values are things like `GET`, `POST`, `CONNECT` and so on; i.e. Do not enter a whole HTTP request line here. For instance, entering `GET /index.html HTTP/1.0\r\n\r\n` would be incorrect. This option accepts a string or **`null`**. Available as of cURL 7.1.0. > **Note** : Don't do this without making sure the server supports the custom request method first.
`CURLOPT_DEFAULT_PROTOCOL` (int)	A string with the default protocol to use if the URL is missing a scheme name. Available as of PHP 7.0.7 and cURL 7.45.0.
`CURLOPT_DIRLISTONLY` (int)	Setting this option to `1` will have different effects based on the protocol it is used with. FTP and SFTP based URLs will list only the names of files in a directory. POP3 will list the email message or messages on the POP3 server. For FILE, this option has no effect as directories are always listed in this mode. Using this option with **`CURLOPT_WILDCARDMATCH`** will prevent the latter from having any effect. Defaults to `0`. Available as of cURL 7.17.0.
`CURLOPT_DISALLOW_USERNAME_IN_URL` (int)	**`true`** to not allow URLs that include a username. Usernames are allowed by default. Available as of PHP 7.3.0 and cURL 7.61.0.
`CURLOPT_DNS_CACHE_TIMEOUT` (int)	The number of seconds to keep DNS entries in memory. This option is set to `120` (2 minutes) by default. This option accepts any value that can be cast to a valid int. Available as of cURL 7.9.3.
`CURLOPT_DNS_INTERFACE` (int)	Set the name of the network interface that the DNS resolver should bind to. This must be an interface name (not an address). This option accepts a string or **`null`**. Available as of PHP 7.0.7 and cURL 7.33.0
`CURLOPT_DNS_LOCAL_IP4` (int)	Set the local IPv4 address that the resolver should bind to. The argument should contain a single numerical IPv4 address. This option accepts a string or **`null`**. Available as of PHP 7.0.7 and cURL 7.33.0.
`CURLOPT_DNS_LOCAL_IP6` (int)	Set the local IPv6 address that the resolver should bind to. The argument should contain a single numerical IPv6 address. This option accepts a string or **`null`**. Available as of PHP 7.0.7 and cURL 7.33.0.
`CURLOPT_DNS_SERVERS` (int)	Pass a string with a comma-separated list of DNS servers to be used instead of the system default (e.g.: `192.168.1.100,192.168.1.101:8080`). Available as of cURL 7.24.0.
`CURLOPT_DNS_SHUFFLE_ADDRESSES` (int)	**`true`** to shuffle the order of all returned addresses so that they will be used in a random order, when a name is resolved and more than one IP address is returned. This may cause IPv4 to be used before IPv6 or vice versa. Available as of PHP 7.3.0 and cURL 7.60.0.

CURLOPT_DNS_USE_GLOBAL_CACHE (int)	**true** to use a global DNS cache. This option is not thread-safe. It is conditionally enabled by default if PHP is built for non-threaded use (CLI, FCGI, Apache2-Prefork, etc.). Available as of cURL 7.9.3 and deprecated as of cURL 7.11.1. As of PHP 8.4, this option no longer has any effect.
CURLOPT_DOH_SSL_VERIFYHOST (int)	Set to `2` to verify the DNS-over-HTTPS server's SSL certificate name fields against the host name. Available as of PHP 8.2.0 and cURL 7.76.0.
CURLOPT_DOH_SSL_VERIFYPEER (int)	Set to `1` to enable and `0` to disable verification of the authenticity of the DNS-over-HTTPS server's SSL certificate. Available as of PHP 8.2.0 and cURL 7.76.0.
CURLOPT_DOH_SSL_VERIFYSTATUS (int)	Set to `1` to enable and `0` to disable the verification of the status of the DNS-over-HTTPS server certificate using the "Certificate Status Request" TLS extension (OCSP stapling). Available as of PHP 8.2.0 and cURL 7.76.0.
CURLOPT_DOH_URL (int)	Provides the DNS-over-HTTPS URL. This option accepts a string or **null**. Available as of PHP 8.1.0 and cURL 7.62.0.
CURLOPT_EGDSOCKET (int)	Like **CURLOPT_RANDOM_FILE**, except a filename to an Entropy Gathering Daemon socket. Available as of cURL 7.7.0 and deprecated as of cURL 7.84.0.
CURLOPT_ENCODING (int)	The contents of the `Accept-Encoding:` header as a string. This enables decoding of the response. Supported encodings are: `identity`, `deflate`, `gzip`. If an empty string is set, a header containing all supported encoding types is sent. Available as of cURL 7.10 and deprecated as of cURL 7.21.6.
CURLOPT_EXPECT_100_TIMEOUT_MS (int)	The timeout for `Expect: 100-continue` responses in milliseconds. Defaults to `1000` milliseconds. This option accepts any value that can be cast to a valid int. Available as of PHP 7.0.7 and cURL 7.36.0.
CURLOPT_FAILONERROR (int)	**true** to fail verbosely if the HTTP code returned is greater than or equal to `400`. The default behavior is to return the page normally, ignoring the code. Available as of cURL 7.1.0.
CURLOPT_FILE (int)	Accepts a file handle resource to the file that the transfer should be written to. The default is **STDOUT** (the browser window). Available as of cURL 7.1.0 and deprecated as of cURL 7.9.7.
CURLOPT_FILETIME (int)	Set to **true** to attempt to retrieve the modification date of the remote document. This value can be retrieved using the **CURLINFO_FILETIME** option with curl_getinfo(). Available as of cURL 7.5.0.

CURLOPT_FNMATCH_FUNCTION (int)	Pass a callable that will be used for wildcard matching. The signature of the callback should be:
	
	function callback(resource `$curlHandle`, string `$pattern`, string `$string`): int
	
	`curlHandle`
	The cURL handle.
	
	`pattern`
	The wildcard pattern.
	
	`string`
	The string to run the wildcard pattern matching on.
	
	The callback should return **CURL_FNMATCHFUNC_MATCH** if pattern matches the string, **CURL_FNMATCHFUNC_NOMATCH** if not or **CURL_FNMATCHFUNC_FAIL** if an error occurred. Available as of

	cURL 7.21.0.
CURLOPT_FOLLOWLOCATION (int)	Set to **true** to follow any `Location:` header that the server sends as part of the HTTP header. See also **CURLOPT_MAXREDIRS**. This constant is not available when open_basedir is enabled. Available as of cURL 7.1.0.
CURLOPT_FORBID_REUSE (int)	Set to **true** to force the connection to explicitly close when it has finished processing, and not be pooled for reuse. Available as of cURL 7.7.0.
CURLOPT_FRESH_CONNECT (int)	Set to **true** to force the use of a new connection instead of a cached one. Available as of cURL 7.7.0.
CURLOPT_FTPAPPEND (int)	Set to **true** to append to the remote file instead of overwriting it. Available as of cURL 7.1.0 and deprecated as of cURL 7.16.4.
CURLOPT_FTPASCII (int)	An alias of **CURLOPT_TRANSFERTEXT**. Use that instead. Available as of cURL 7.1, deprecated as of cURL 7.11.1 and last available in cURL 7.15.5. Removed as of PHP 7.3.0.
CURLOPT_FTPLISTONLY (int)	Set to **true** to only list the names of an FTP directory. Available as of cURL 7.1.0 and deprecated as of cURL 7.16.4.
CURLOPT_FTPPORT (int)	A string which will be used to get the IP address to use for the FTP `PORT` instruction. The `PORT` instruction tells the remote server to connect to our specified IP address. The string may be a plain IP address, a hostname, a network interface name (under Unix), or just a plain `-` to use the system's default IP address. This option accepts a string or **null**. Available as of cURL 7.1.0.
CURLOPT_FTPSSLAUTH (int)	Set the FTP over SSL authentication method (if activated) to any of the **CURLFTPAUTH_*** constants. Defaults to **CURLFTPAUTH_DEFAULT**. Available as of cURL 7.12.2.
CURLOPT_FTP_ACCOUNT (int)	Pass a string that will be sent as account information over FTP (using the `ACCT` command) after username and password has been provided to the server. Set to **null** to disable sending the account information. Defaults to **null**. Available as of cURL 7.13.0.
CURLOPT_FTP_ALTERNATIVE_TO_USER (int)	Pass a string that will be used to try to authenticate over FTP if the `USER/PASS` negotiation fails. Available as of cURL 7.15.5.
CURLOPT_FTP_CREATE_MISSING_DIRS (int)	Set to **true** to create missing directories when an FTP operation encounters a path that currently doesn't exist. Available as of cURL 7.10.7.
CURLOPT_FTP_FILEMETHOD (int)	Tell cURL which method to use to reach a file on a FTP(S) server. Possible values are any of the **CURLFTPMETHOD_*** constants. Defaults to **CURLFTPMETHOD_MULTICWD**. Available as of cURL 7.15.1.
CURLOPT_FTP_RESPONSE_TIMEOUT (int)	A timeout in seconds cURL will wait for a response from an FTP server. This option overrides **CURLOPT_TIMEOUT**. This option accepts any value that can be cast to a valid int. This option name is replaced with **CURLOPT_SERVER_RESPONSE_TIMEOUT**, available as of PHP 8.4.0. Available as of cURL 7.10.8 and deprecated as of cURL 7.85.0.
CURLOPT_FTP_SKIP_PASV_IP (int)	If this option is set to `1` cURL will not use the IP address the server suggests in its 227-response to cURL's `PASV` command but will use the IP address it used for the connection. The port number received from the 227-response will not be ignored by cURL. Defaults to `1` as of cURL 7.74.0 and `0` prior to that. Available as of cURL 7.15.0.

CURLOPT_FTP_SSL (int)	Available as of cURL 7.11.0 and deprecated as of cURL 7.16.4.
CURLOPT_FTP_SSL_CCC (int)	This option makes cURL use CCC (Clear Command Channel) which shuts down the SSL/TLS layer after authenticating making the rest of the control channel communication unencrypted. Use one of the **CURLFTPSSL_CCC_*** constants. Defaults to **CURLFTPSSL_CCC_NONE**. Available as of cURL 7.16.1.
CURLOPT_FTP_USE_EPRT (int)	Set to **true** to use EPRT (and LPRT) when doing active FTP downloads. Set to **false** to disable EPRT and LPRT and use PORT only. Available as of cURL 7.10.5.
CURLOPT_FTP_USE_EPSV (int)	Set to **true** to first try an EPSV command for FTP transfers before reverting back to PASV. Set to **false** to disable EPSV. Available as of cURL 7.9.2.
CURLOPT_FTP_USE_PRET (int)	Set to 1 to send a PRET command before PASV (and EPSV). Has no effect when using the active FTP transfers mode. Defaults to 0. Available as of cURL 7.20.0.
CURLOPT_GSSAPI_DELEGATION (int)	Set to **CURLGSSAPI_DELEGATION_FLAG** to allow unconditional GSSAPI credential delegation. Set to **CURLGSSAPI_DELEGATION_POLICY_FLAG** to delegate only if the OK-AS-DELEGATE flag is set in the service ticket. Defaults to **CURLGSSAPI_DELEGATION_NONE**. Available as of cURL 7.22.0.
CURLOPT_HAPPY_EYEBALLS_TIMEOUT_MS (int)	Head start for IPv6 for the happy eyeballs algorithm. Happy eyeballs attempts to connect to both IPv4 and IPv6 addresses for dual-stack hosts, preferring IPv6 first for timeout milliseconds. Defaults to **CURL_HET_DEFAULT**, which is currently 200 milliseconds. This option accepts any value that can be cast to a valid int. Available as of PHP 7.3.0 and cURL 7.59.0
CURLOPT_HAPROXYPROTOCOL (int)	**true** to send an HAProxy PROXY protocol v1 header at the start of the connection. The default action is not to send this header. Available as of PHP 7.3.0 and cURL 7.60.0.
CURLOPT_HEADER (int)	Set to **true** to include the headers in the output sent to the callback defined by **CURLOPT_WRITEFUNCTION**. Available as of cURL 7.1.0.
CURLOPT_HEADERFUNCTION (int)	A callable with the following signature: function callback(resource $curlHandle, string $headerData): int curlHandle The cURL handle. headerData The header data which must be written by the callback. The callback should return the number of bytes written. Available as of cURL 7.7.2.
CURLOPT_HEADEROPT (int)	Send HTTP headers to both proxy and host or separately. Possible values are any of the **CURLHEADER_*** constants. Defaults to **CURLHEADER_SEPARATE** as of cURL 7.42.1, and **CURLHEADER_UNIFIED** prior to that. Available as of PHP 7.0.7 and cURL 7.37.0.
CURLOPT_HSTS (int)	string with HSTS (HTTP Strict Transport Security) cache file name or **null** to allow HSTS without reading from or writing to any file and clear the list of files to read HSTS data from. Available as of PHP 8.2.0 and cURL 7.74.0.
CURLOPT_HSTS_CTRL (int)	Accepts a bitmask of HSTS (HTTP Strict Transport Security) features defined by the **CURLHSTS_*** constants. Available as of PHP 8.2.0 and cURL 7.74.0.

CURLOPT_HTTP09_ALLOWED (int)	Whether to allow HTTP/0.9 responses. Defaults to **false** as of cURL 7.66.0; formerly it defaulted to **true**. Available as of PHP 7.3.15 and 7.4.3, respectively, and cURL 7.64.0.
CURLOPT_HTTP200ALIASES (int)	An array of HTTP 200 responses that will be treated as valid responses and not as errors. Available as of cURL 7.10.3.
CURLOPT_HTTPAUTH (int)	A bitmask of HTTP authentication method(s) to use. The options are: **CURLAUTH_BASIC**, **CURLAUTH_DIGEST**, **CURLAUTH_GSSNEGOTIATE**, **CURLAUTH_NTLM**, **CURLAUTH_AWS_SIGV4**, **CURLAUTH_ANY**, **CURLAUTH_ANYSAFE**. If more than one method is used, cURL will poll the server to see what methods it supports and pick the best one. **CURLAUTH_ANY** sets all bits. cURL will automatically select the one it finds most secure. **CURLAUTH_ANYSAFE** sets all bits except **CURLAUTH_BASIC**. cURL will automatically select the one it finds most secure. Available as of cURL 7.10.6.
CURLOPT_HTTPGET (int)	Set to **true** to reset the HTTP request method to GET. Since GET is the default, this is only necessary if the request method has been changed. Available as of cURL 7.8.1.
CURLOPT_HTTPHEADER (int)	An array of HTTP header fields to set, in the format `array('Content-type: text/plain', 'Content-length: 100')` Available as of cURL 7.1.0.
CURLOPT_HTTPPROXYTUNNEL (int)	**true** to tunnel through a given HTTP proxy. Available as of cURL 7.3.0.
CURLOPT_HTTP_CONTENT_DECODING (int)	**false** to get the raw HTTP response body. Available as of cURL 7.16.2.
CURLOPT_HTTP_TRANSFER_DECODING (int)	If set to 0, transfer decoding is disabled. If set to 1, transfer decoding is enabled. cURL does chunked transfer decoding by default unless this option is set to 0. Defaults to 1. Available as of cURL 7.16.2.
CURLOPT_HTTP_VERSION (int)	Set to one of the **CURL_HTTP_VERSION_*** constants for cURL to use the specified HTTP version. Available as of cURL 7.9.1.
CURLOPT_IGNORE_CONTENT_LENGTH (int)	If set to 1, ignore the Content-Length header in the HTTP response and ignore asking for or relying on it for FTP transfers. Defaults to 0. Available as of cURL 7.14.1.
CURLOPT_INFILE (int)	Accepts a file handle resource to the file that the transfer should be read from when uploading. Available as of cURL 7.1.0 and deprecated as of cURL 7.9.7. Use **CURLOPT_READDATA** instead.
CURLOPT_INFILESIZE (int)	The expected size, in bytes, of the file when uploading a file to a remote site. Note that using this option will not stop cURL from sending more data, as exactly what is sent depends on **CURLOPT_READFUNCTION**. This option accepts any value that can be cast to a valid int. Available as of cURL 7.1.0.
CURLOPT_INFILESIZE_LARGE (int)	The expected size, in bytes, of the file when uploading a file to a remote site. This is the 64-bit variant of **CURLOPT_INFILESIZE**, allowing sizes larger than 2 GB to be specified. Available as of PHP 8.5.0.
CURLOPT_INTERFACE (int)	Set to a string with the name of the outgoing network interface to use. This can be an interface name, an IP address or a host name. Available as of cURL 7.1.0.
CURLOPT_IPRESOLVE (int)	Allows an application to select what kind of IP addresses to use when resolving host names. This is only interesting when using host names that resolve addresses using more than one version of

	IP. Set to one of the `CURL_IPRESOLVE_*` constants. Defaults to `CURL_IPRESOLVE_WHATEVER`. Available as of cURL 7.10.8.
`CURLOPT_ISSUERCERT` (int)	If set to a string naming a file holding a CA certificate in PEM format, an additional check against the peer certificate is performed to verify the issuer is indeed the one associated with the certificate provided by the option. For the result of the check to be considered a failure, this option should be used in combination with the `CURLOPT_SSL_VERIFYPEER` option. Available as of cURL 7.19.0.
`CURLOPT_ISSUERCERT_BLOB` (int)	Pass a string with binary data of a CA SSL certificate in PEM format. If set, an additional check against the peer certificate is performed to verify the issuer is the one associated with the certificate provided by the option. Available as of PHP 8.1.0 and cURL 7.71.0.
`CURLOPT_KEEP_SENDING_ON_ERROR` (int)	Set to `true` to keep sending the request body if the HTTP code returned is equal to or larger than `300`. The default action would be to stop sending and close the stream or connection. Suitable for manual NTLM authentication. Most applications do not need this option. Available as of PHP 7.3.0 and cURL 7.51.0.
`CURLOPT_KEYPASSWD` (int)	Set to a string with the password required to use the `CURLOPT_SSLKEY` or `CURLOPT_SSH_PRIVATE_KEYFILE` private key. Setting this option to `null` disables using a password for these options. Available as of cURL 7.17.0.
`CURLOPT_KRB4LEVEL` (int)	The KRB4 (Kerberos 4) security level. Any of the following string values (in order from least to most powerful) are valid: `clear`, `safe`, `confidential`, `private`. If the string does not match one of these, `private` is used. Setting this option to `null` will disable KRB4 security. Currently KRB4 security only works with FTP transactions. Available as of cURL 7.3.0 and deprecated as of cURL 7.17.0.
`CURLOPT_KRBLEVEL` (int)	Set the kerberos security level for FTP and also enables kerberos awareness. This should be set to one of the following strings: `clear`, `safe`, `confidential`, `private`. If the string is set but does not match one of these, `private` is used. Setting this option to `null` will disable kerberos support for FTP. Defaults to `null`. Available as of cURL 7.16.4 and deprecated as of cURL 8.17.0.
`CURLOPT_LOCALPORT` (int)	Sets the local port number of the socket used for the connection. This option accepts any value that can be cast to a valid int. Defaults to `0`. Available as of cURL 7.15.2.
`CURLOPT_LOCALPORTRANGE` (int)	The number of attempts cURL makes to find a working local port number, starting with the one set with `CURLOPT_LOCALPORT`. This option accepts any value that can be cast to a valid int. Defaults to `1`. Available as of cURL 7.15.2.
`CURLOPT_LOGIN_OPTIONS` (int)	Can be used to set protocol specific login options, such as the preferred authentication mechanism via `AUTH=NTLM` or `AUTH=*`, and should be used in conjunction with the `CURLOPT_USERNAME` option. Available as of PHP 7.0.7 and cURL 7.34.0.
`CURLOPT_LOW_SPEED_LIMIT` (int)	The transfer speed, in bytes per second, that the transfer should be below during the count of `CURLOPT_LOW_SPEED_TIME` seconds before PHP considers the transfer too slow and aborts. This option accepts any value that can be cast to a valid int. Available as of cURL 7.1.0.
`CURLOPT_LOW_SPEED_TIME` (int)	The number of seconds the transfer speed should be below `CURLOPT_LOW_SPEED_LIMIT` before PHP considers the transfer too slow and aborts. This option accepts any value that can be cast to a valid int. Available as of cURL 7.1.0.

CURLOPT_MAIL_AUTH (int)	Set a string with the authentication address (identity) of a submitted message that is being relayed to another server. The address should not be specified within a pair of angled brackets (<>). If an empty string is used then a pair of brackets are sent by cURL as required by RFC 2554. Available as of cURL 7.25.0.
CURLOPT_MAIL_FROM (int)	Set a string with the sender's email address when sending SMTP mail. The email address should be specified with angled brackets (<>) around it, which if not specified are added automatically. If this parameter is not specified then an empty address is sent to the SMTP server which might cause the email to be rejected. Available as of cURL 7.20.0.
CURLOPT_MAIL_RCPT (int)	Set to an array of strings with the recipients to pass to the server in an SMTP mail request. Each recipient should be specified within a pair of angled brackets (<>). If an angled bracket is not used as the first character, cURL assumes a single email address has been provided and encloses that address within brackets. Available as of cURL 7.20.0.
CURLOPT_MAIL_RCPT_ALLLOWFAILS (int)	Set to `1` to allow `RCPT TO` command to fail for some recipients which makes cURL ignore errors for individual recipients and proceed with the remaining accepted recipients. If all recipients trigger failures and this flag is specified, cURL aborts the SMTP conversation and returns the error received from to the last `RCPT TO` command. Replaced by **CURLOPT_MAIL_RCPT_ALLOWFAILS** as of cURL 8.2.0. Available as of PHP 8.2.0 and cURL 7.69.0. Deprecated as of cURL 8.2.0.
CURLOPT_MAXAGE_CONN (int)	The maximum idle time allowed for an existing connection to be considered for reuse. Default maximum age is set to `118` seconds. This option accepts any value that can be cast to a valid int. Available as of PHP 8.2.0 and cURL 7.65.0.
CURLOPT_MAXCONNECTS (int)	The maximum amount of persistent connections that are allowed. When the limit is reached, the oldest one in the cache is closed to prevent increasing the number of open connections. This option accepts any value that can be cast to a valid int. Available as of cURL 7.7.0.
CURLOPT_MAXFILESIZE (int)	Sets the maximum accepted size (in bytes) of a file to download. If the file requested is found larger than this value, the transfer is aborted and **CURLE_FILESIZE_EXCEEDED** is returned. Passing `0` disables this option, and passing a negative size returns a **CURLE_BAD_FUNCTION_ARGUMENT**. If the file size is not known prior to the start of download, this option has no effect. For setting a size limit above `2GB`, **CURLOPT_MAXFILESIZE_LARGE** should be used. As of cURL 8.4.0, this option also stops ongoing transfers if they reach this threshold. This option accepts any value that can be cast to a valid int. Defaults to `0`. Available as of cURL 7.10.8.
CURLOPT_MAXFILESIZE_LARGE (int)	The maximum file size in bytes allowed to download. If the file requested is found larger than this value, the transfer will not start and **CURLE_FILESIZE_EXCEEDED** will be returned. The file size is not always known prior to download, and for such files this option has no effect even if the file transfer ends up being larger than this given limit. This option accepts any value that can be cast to a valid int. Available as of PHP 8.2.0 and cURL 7.11.0.
CURLOPT_MAXLIFETIME_CONN (int)	The maximum time in seconds, since the creation of the connection, that is allowed for an existing connection to have for it to be considered for reuse. If a connection is found in the cache that is older than this value, it will instead be closed once any in-progress transfers are complete. Default is `0` seconds, meaning the option is disabled and all connections are eligible for reuse. This option accepts any value that can be cast to a valid int. Available as of PHP 8.2.0 and cURL 7.80.0.
CURLOPT_MAXREDIRS (int)	The maximum amount of HTTP redirections to follow. Use this option alongside **CURLOPT_FOLLOWLOCATION**. Default value of `20` is set to prevent infinite redirects. Setting to `-1` allows infinite redirects, and `0` refuses all redirects. Available as of cURL 7.5.0.

`CURLOPT_MAX_RECV_SPEED_LARGE` (int)	If a download exceeds this speed (counted in bytes per second) on cumulative average during the transfer, the transfer will pause to keep the average rate less than or equal to the parameter value. Defaults to unlimited speed. This option accepts any value that can be cast to a valid int. Available as of cURL 7.15.5.
`CURLOPT_MAX_SEND_SPEED_LARGE` (int)	If an upload exceeds this speed (counted in bytes per second) on cumulative average during the transfer, the transfer will pause to keep the average rate less than or equal to the parameter value. Defaults to unlimited speed. This option accepts any value that can be cast to a valid int. Available as of cURL 7.15.5.
`CURLOPT_MIME_OPTIONS` (int)	Set to a bitmask of `CURLMIMEOPT_*` constants. Currently there is only one available option: `CURLMIMEOPT_FORMESCAPE`. Available as of PHP 8.3.0 and cURL 7.81.0.
`CURLOPT_MUTE` (int)	Set to `true` to be completely silent with regards to the cURL functions. Use `CURLOPT_RETURNTRANSFER` instead. Available as of cURL 7.1.0, deprecated as of cURL 7.8.0 and last available in cURL 7.15.5. Removed as of PHP 7.3.0.
`CURLOPT_NETRC` (int)	Set to `true` to scan the *~/.netrc* file to find a username and password for the remote site that a connection is being established with. Available as of cURL 7.1.0.
`CURLOPT_NETRC_FILE` (int)	Set a string containing the full path name to a `.netrc` file. If this option is omitted and `CURLOPT_NETRC` is set, cURL checks for a `.netrc` file in the current user's home directory. Available as of cURL 7.11.0.
`CURLOPT_NEW_DIRECTORY_PERMS` (int)	Sets the value of the permissions (int) that is set on newly created directories on the remote server. The default value is `0755`. The only protocols that can use this are `sftp://`, `scp://` and `file://`. Available as of cURL 7.16.4.
`CURLOPT_NEW_FILE_PERMS` (int)	Sets the value of the permissions (as an int) that are set on newly created files on the remote server. The default value is `0644`. The only protocols that can use this are `sftp://`, `scp://` and `file://`. Available as of cURL 7.16.4.
`CURLOPT_NOBODY` (int)	Set to `true` to exclude the body from the output. For HTTP(S), cURL makes a HEAD request. For most other protocols, cURL is not asking for the body data at all. Changing this to `false` will result in body data being included in the output. Available as of cURL 7.1.0.
`CURLOPT_NOPROGRESS` (int)	Set to `true` to disable the progress meter for cURL transfers. **Note** : PHP automatically sets this option to `true`, this should only be changed for debugging purposes. Available as of cURL 7.1.0.
`CURLOPT_NOPROXY` (int)	Set a string consisting of a comma separated list of hostnames that do not require a proxy to get reached. Each name in this list is matched as either a domain which contains the hostname or the hostname itself. The only wildcard available in the string is a single `*` character which matches all hosts, effectively disabling the proxy. Setting this option to an empty string enables the proxy for all hostnames. Since cURL 7.86.0, IP addresses set with this option can be provided using CIDR notation. Available as of cURL 7.19.4.

CURLOPT_NOSIGNAL (int)	**true** to ignore any cURL function that causes a signal to be sent to the PHP process. This is turned on by default in multi-threaded SAPIs so timeout options can still be used. Available as of cURL 7.10.

CURLOPT_PASSWDFUNCTION (int)	A callable with the following signature:
	
	function callback(resource `$curlHandle`, string `$passwordPrompt`, int `$maximumPasswordLength`): string
	
	`curlHandle`
	The cURL handle.
	
	`passwordPrompt`
	A password prompt.
	
	`maximumPasswordLength`
	The maximum length of the password.
	
	The callback should return a string containing the password. Available as of cURL 7.4.2, deprecated as of cURL 7.11.1 and last available in cURL 7.15.5. Removed as of PHP 7.3.0.

CURLOPT_PASSWORD (int)	Set to a string with the password to use in authentication. Available as of cURL 7.19.1.
CURLOPT_PATH_AS_IS (int)	Set to **true** for cURL not alter URL paths before passing them on to the server. Defaults to **false**, which squashes sequences of `/../` or `/./` that may exist in the URL's path part which is supposed to be removed according to RFC 3986 section 5.2.4. Available as of PHP 7.0.7 and cURL 7.42.0.
CURLOPT_PINNEDPUBLICKEY (int)	Set a string with the pinned public key. The string can be the file name of the pinned public key in a PEM or DER file format. The string can also be any number of base64 encoded sha256 hashes preceded by `sha256//` and separated by `;`. Available as of PHP 7.0.7 and cURL 7.39.0.
CURLOPT_PIPEWAIT (int)	Set to **true** to wait for an existing connection to confirm whether it can do multiplexing and use it if it can before creating and using a new connection. Available as of PHP 7.0.7 and cURL 7.43.0.
CURLOPT_PORT (int)	An int with an alternative port number to connect to instead of the one specified in the URL or the default port for the used protocol. Available as of cURL 7.1.0.
CURLOPT_POST (int)	Set to **true** to do a HTTP `POST` request. This request uses the `application/x-www-form-urlencoded` header. Defaults to **false**. Available as of cURL 7.1.0.
CURLOPT_POSTFIELDS (int)	The full data to post in a HTTP `POST` operation. This parameter can either be passed as a urlencoded string like '`para1=val1&para2=val2&...`' or as an array with the field name as key and field data as value. If `value` is an array, the `Content-Type` header will be set to `multipart/form-data`. Files can be sent using CURLFile or CURLStringFile, in which case `value` must be an array. Available as of cURL 7.1.0.
CURLOPT_POSTQUOTE (int)	An array of FTP command strings to execute on the server after the FTP request has been performed. Available as of cURL 7.1.0.
CURLOPT_POSTREDIR (int)	Set to a bitmask of **CURL_REDIR_POST_301**, **CURL_REDIR_POST_302** and **CURL_REDIR_POST_303** if the HTTP `POST` method should be maintained when **CURLOPT_FOLLOWLOCATION** is set and a specific type of redirect occurs. Available as of cURL 7.19.1.

CURLOPT_PRE_PROXY (int)	Set a string holding the host name or dotted numerical IP address to be used as the preproxy that cURL connects to before it connects to the HTTP(S) proxy specified in the **CURLOPT_PROXY** option for the upcoming request. The preproxy can only be a SOCKS proxy and it should be prefixed with `[scheme]://` to specify which kind of socks is used. A numerical IPv6 address must be written within [brackets]. Setting the preproxy to an empty string explicitly disables the use of a preproxy. To specify port number in this string, append `:[port]` to the end of the host name. The proxy's port number may optionally be specified with the separate option **CURLOPT_PROXYPORT**. Defaults to using port 1080 for proxies if a port is not specified. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PREQUOTE (int)	Set an array of FTP command strings to pass to the server after the transfer type is set. These commands are not performed when a directory listing is performed, only for file transfers. Available as of cURL 7.9.5.
CURLOPT_PRIVATE (int)	Any data that should be associated with this cURL handle. This data can subsequently be retrieved with the **CURLINFO_PRIVATE** option of curl_getinfo(). cURL does nothing with this data. When using a cURL multi handle, this private data is typically a unique key to identify a standard cURL handle. Available as of cURL 7.10.3.

CURLOPT_PROGRESSFUNCTION (int)	A callable with the following signature:
	
	function callback(
	  resource `$curlHandle`,
	  int `$bytesToDownload`,
	  int `$bytesDownloaded`,
	  int `$bytesToUpload`,
	  int `$bytesUploaded`
	): int
	
	`curlHandle`
	  The cURL handle.
	
	`bytesToDownload`
	  The total number of bytes expected to be downloaded in this transfer.
	
	`bytesDownloaded`
	  The number of bytes downloaded so far.
	
	`bytesToUpload`
	  The total number of bytes expected to be uploaded in this transfer.
	
	`bytesUploaded`
	  The number of bytes uploaded so far.
	
	The callback should return an int with a non-zero value to abort the transfer and set a **CURLE_ABORTED_BY_CALLBACK** error.
	
	> **Note** :
	> The callback is only called when the **CURLOPT_NOPROGRESS** option is set to `false`.
	
	Available as of cURL 7.1.0 and deprecated as of cURL 7.32.0. Use **CURLOPT_XFERINFOFUNCTION** instead.

CURLOPT_PROTOCOLS (int)	Bitmask of **CURLPROTO_*** values. If used, this bitmask limits what protocols cURL may use in the transfer. Defaults to **CURLPROTO_ALL**, ie. cURL will accept all protocols it supports. See also **CURLOPT_REDIR_PROTOCOLS_STR**. Available as of cURL 7.19.4 and deprecated as of cURL 7.85.0.
CURLOPT_PROTOCOLS_STR (int)	Set to a string with a comma separated list of case insensitive protocol names (URL schemes) to allow in the transfer. Set to `ALL` to enable all protocols. By default, cURL accepts all protocols it was built with support for. Available protocols are: `DICT`, `FILE`, `FTP`, `FTPS`, `GOPHER`, `GOPHERS`, `HTTP`, `HTTPS`, `IMAP`, `IMAPS`, `LDAP`, `LDAPS`, `MQTT`, `POP3`, `POP3S`, `RTMP`, `RTMPE`, `RTMPS`, `RTMPT`, `RTMPTE`, `RTMPTS`, `RTSP`, `SCP`, `SFTP`, `SMB`, `SMBS`, `SMTP`, `SMTPS`, `TELNET`, `TFTP`, `WS`, `WSS`. Available as of PHP 8.3.0 and cURL 7.85.0.
CURLOPT_PROXY (int)	A string with the HTTP proxy to tunnel requests through. This should be the hostname, the dotted numerical IP address or a numerical IPv6 address written within [brackets]. Available as of cURL 7.1.0.
CURLOPT_PROXYAUTH (int)	A bitmask of the HTTP authentication method(s) (**CURLAUTH_*** constants) to use for the proxy connection. For proxy authentication, only **CURLAUTH_BASIC** and **CURLAUTH_NTLM** are currently supported. Defaults to **CURLAUTH_BASIC**. Available as of cURL 7.10.7.
CURLOPT_PROXYHEADER (int)	An array of custom HTTP header strings to pass to proxies. Available as of PHP 7.0.7 and cURL 7.37.0.
CURLOPT_PROXYPASSWORD (int)	Set a string with the password to be used for authentication with the proxy. Available as of cURL 7.19.1.
CURLOPT_PROXYPORT (int)	An int with the port number of the proxy to connect to. This port number can also be set in **CURLOPT_PROXY**. Setting this to zero makes cURL use the default proxy port number or the port number specified in the proxy URL string. Available as of cURL 7.1.0.
CURLOPT_PROXYTYPE (int)	Sets the type of the proxy to one of the **CURLPROXY_*** constants. Defaults to **CURLPROXY_HTTP**. Available as of cURL 7.10.
CURLOPT_PROXYUSERNAME (int)	Set a string with the username to be used for authentication with the proxy. Available as of cURL 7.19.1.
CURLOPT_PROXYUSERPWD (int)	A string with a username and password formatted as `[username]:[password]` to use for the connection to the proxy. Available as of cURL 7.1.0.
CURLOPT_PROXY_CAINFO (int)	The path to proxy Certificate Authority (CA) bundle. Set the path as a string naming a file holding one or more certificates to verify the HTTPS proxy with. This option is for connecting to an HTTPS proxy, not an HTTPS server. Defaults set to the system path where cURL's cacert bundle is assumed to be stored. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_CAINFO_BLOB (int)	A string with the name of a PEM file holding one or more certificates to verify the HTTPS proxy with. This option is for connecting to an HTTPS proxy, not an HTTPS server. Defaults set to the system path where cURL's cacert bundle is assumed to be stored. Available as of PHP 8.2.0 and cURL 7.77.0.
CURLOPT_PROXY_CAPATH (int)	A string with the directory holding multiple CA certificates to verify the HTTPS proxy with. Available as of PHP 7.3.0 and cURL 7.52.0.

CURLOPT_PROXY_CRLFILE (int)	Set to a string with the file name with the concatenation of CRL (Certificate Revocation List) in PEM format to use in the certificate validation that occurs during the SSL exchange. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_ISSUERCERT (int)	Proxy issuer SSL certificate filename string. Available as of PHP 8.1.0 and cURL 7.71.0.
CURLOPT_PROXY_ISSUERCERT_BLOB (int)	A string with the proxy issuer SSL certificate. Available as of PHP 8.1.0 and cURL 7.71.0.
CURLOPT_PROXY_KEYPASSWD (int)	Set the string be used as the password required to use the **CURLOPT_PROXY_SSLKEY** private key. A passphrase is not needed to load a certificate but one is needed to load a private key. This option is for connecting to an HTTPS proxy, not an HTTPS server. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_PINNEDPUBLICKEY (int)	Set the pinned public key for HTTPS proxy. The string can be the file name of the pinned public key which is expected to be in a PEM or DER file format. The string can also be any number of base64 encoded sha256 hashes preceded by `sha256//` and separated by `;`. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_SERVICE_NAME (int)	A string with the proxy authentication service name. Available as of PHP 7.0.7, cURL 7.43.0 (for HTTP proxies) and cURL 7.49.0 (for SOCKS5 proxies).
CURLOPT_PROXY_SSLCERT (int)	A string with the file name of the client certificate used to connect to the HTTPS proxy. The default format is P12 on Secure Transport and PEM on other engines, and can be changed with **CURLOPT_PROXY_SSLCERTTYPE**. With NSS or Secure Transport, this can also be the nickname of the certificate used for authentication as it is named in the security database. If a file from the current directory is to be used, it must be prefixed with `./` in order to avoid confusion with a nickname. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_SSLCERTTYPE (int)	A string with the format of the client certificate used when connecting to an HTTPS proxy. Supported formats are PEM and DER, except with Secure Transport. OpenSSL (versions 0.9.3 and later) and Secure Transport (on iOS 5 or later, or OS X 10.7 or later) also support P12 for PKCS#12-encoded files. Defaults to PEM. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_SSLCERT_BLOB (int)	A string with the SSL proxy client certificate. Available as of PHP 8.1.0 and cURL 7.71.0.
CURLOPT_PROXY_SSLKEY (int)	A string with the file name of the private key used for connecting to the HTTPS proxy. The default format is PEM and can be changed with **CURLOPT_PROXY_SSLKEYTYPE**. (iOS and Mac OS X only) This option is ignored if cURL was built against Secure Transport. Available if built with TLS enabled. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_SSLKEYTYPE (int)	A string with the format of the private key. Supported formats are: PEM, DER, ENG. Available as of PHP 7.3.0 and cURL 7.52.0.
CURLOPT_PROXY_SSLKEY_BLOB (int)	A string with the private key for connecting to the HTTPS proxy. Available as of PHP 8.1.0 and cURL 7.71.0.
CURLOPT_PROXY_SSLVERSION (int)	Set the preferred HTTPS proxy TLS version to one of the **CURL_SSLVERSION_*** constants. Defaults to **CURL_SSLVERSION_DEFAULT**. **Warning**

	> It is better to not set this option and leave the default `CURL_SSLVERSION_DEFAULT` which will attempt to figure out the remote SSL protocol version. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_SSL_CIPHER_LIST` (int)	A string with a colon-separated list of ciphers to use for the connection to the HTTPS proxy. When used with OpenSSL commas and spaces are also acceptable as separators, and `!`, `-` and `+` can be used as operators. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_SSL_OPTIONS` (int)	Set proxy SSL behavior options, which is a bitmask of the `CURLSSLOPT_*` constants. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_SSL_VERIFYHOST` (int)	Set to `2` to verify in the HTTPS proxy's certificate name fields against the proxy name. When set to `0` the connection succeeds regardless of the names used in the certificate. Use that ability with caution! Set to `1` in cURL 7.28.0 and earlier as a debug option. Set to `1` in cURL 7.28.1 to 7.65.3 `CURLE_BAD_FUNCTION_ARGUMENT` is returned. As of cURL 7.66.0 `1` and `2` is treated as the same value. Defaults to `2`. In production environments the value of this option should be kept at `2`. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_SSL_VERIFYPEER` (int)	Set to `false` to stop cURL from verifying the peer's certificate. Alternate certificates to verify against can be specified with the `CURLOPT_CAINFO` option or a certificate directory can be specified with the `CURLOPT_CAPATH` option. When set to `false`, the peer certificate verification succeeds regardless. `true` by default. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_TLS13_CIPHERS` (int)	A string with a colon-separated list of ciphers to use for the connection to the TLS 1.3 connection to a proxy. This option is currently used only when cURL is built to use OpenSSL 1.1.1 or later. When using a different SSL backend the TLS 1.3 cipher suites can be set with the `CURLOPT_PROXY_SSL_CIPHER_LIST` option. Available as of PHP 7.3.0 and cURL 7.61.0.
`CURLOPT_PROXY_TLSAUTH_PASSWORD` (int)	A string with the password to use for the TLS authentication method specified with the `CURLOPT_PROXY_TLSAUTH_TYPE` option. Requires that the `CURLOPT_PROXY_TLSAUTH_USERNAME` option to also be set. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_TLSAUTH_TYPE` (int)	The method of the TLS authentication used for the HTTPS connection. Supported method is `SRP`. > **Note** : Secure Remote Password (SRP) authentication for TLS provides mutual authentication if both sides have a shared secret. To use TLS-SRP, the `CURLOPT_PROXY_TLSAUTH_USERNAME` and `CURLOPT_PROXY_TLSAUTH_PASSWORD` options must also be set. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_TLSAUTH_USERNAME` (int)	The username to use for the HTTPS proxy TLS authentication method specified with the `CURLOPT_PROXY_TLSAUTH_TYPE` option. Requires that the `CURLOPT_PROXY_TLSAUTH_PASSWORD` option to also be set. Available as of PHP 7.3.0 and cURL 7.52.0.
`CURLOPT_PROXY_TRANSFER_MODE` (int)	Set to `1` to set the transfer mode (binary or ASCII) for FTP transfers done via an HTTP proxy, by appending `type=a` or `type=i` to the URL. Without this setting or it being set to `0`, `CURLOPT_TRANSFERTEXT` has no effect when doing FTP via a proxy. Defaults to `0`. Available as of cURL 7.18.0.
`CURLOPT_PUT` (int)	`true` to HTTP PUT a file. The file to PUT must be set with `CURLOPT_READDATA` and `CURLOPT_INFILESIZE`. Available as of cURL 7.1.0 and deprecated as of cURL 7.12.1.

CURLOPT_QUICK_EXIT (int)	Set to **true** for cURL to skip cleanup of resources when recovering from a timeout. This allows for a swift termination of the cURL process at the expense of a possible leak of associated resources. Available as of PHP 8.3.0 and cURL 7.87.0.
CURLOPT_QUOTE (int)	An array of FTP command strings to execute on the server prior to the FTP request. Available as of cURL 7.1.0.
CURLOPT_RANDOM_FILE (int)	A string with a filename to be used to seed the random number generator for SSL. Available as of cURL 7.7.0 and deprecated as of cURL 7.84.0.
CURLOPT_RANGE (int)	A string with the range(s) of data to retrieve in the format `X-Y` where X or Y are optional. HTTP transfers also support several intervals, separated with commas in the format `X-Y,N-M`. Set to **null** to disable requesting a byte range. Available as of cURL 7.1.0.
CURLOPT_READDATA (int)	Sets a file pointer resource that will be used by the file read function set with **CURLOPT_READFUNCTION**. Available as of cURL 7.9.7.

CURLOPT_READFUNCTION (int)	A callable with the following signature:
	
	function callback(resource `$curlHandle`, resource `$streamResource`, int `$maxAmountOfDataToRead`): string
	
	`curlHandle`
	The cURL handle.
	
	`streamResource`
	Stream resource provided to cURL through the option **CURLOPT_READDATA**.
	
	`maxAmountOfDataToRead`
	The maximum amount of data to be read.
	
	The callback should return a string with a length equal or smaller than the amount of data requested, typically by reading it from the passed stream resource. It should return an empty string to signal `EOF`. Available as of cURL 7.1.0.

CURLOPT_REDIR_PROTOCOLS (int)	Bitmask of **CURLPROTO_*** values which limit what protocols cURL may use in a transfer that it follows to in a redirect when **CURLOPT_FOLLOWLOCATION** is enabled. This allows limiting specific transfers to only be allowed to use a subset of protocols in redirections. As of cURL 7.19.4, by default cURL will allow all protocols except for `FILE` and `SCP`. Prior to cURL 7.19.4, cURL would unconditionally follow to all supported protocols. See also **CURLOPT_PROTOCOLS** for protocol constant values. Available as of cURL 7.19.4 and deprecated as of cURL 7.85.0.
CURLOPT_REDIR_PROTOCOLS_STR (int)	Set to a string with a comma separated list of case insensitive protocol names (URL schemes) to allow to follow to in a redirect when **CURLOPT_FOLLOWLOCATION** is enabled. Set to `ALL` to enable all protocols. As of cURL 7.65.2 it defaults to `FTP`, `FTPS`, `HTTP` and `HTTPS`. From cURL 7.40.0 to 7.65.1, this defaults to all protocols except `FILE`, `SCP`, `SMB` and `SMBS`. Prior to cURL 7.40.0, this defaults to all protocols except `FILE` and `SCP`. Available protocols are: `DICT`, `FILE`, `FTP`, `FTPS`, `GOPHER`, `GOPHERS`, `HTTP`, `HTTPS`, `IMAP`, `IMAPS`, `LDAP`, `LDAPS`, `MQTT`, `POP3`, `POP3S`, `RTMP`, `RTMPE`, `RTMPS`, `RTMPT`, `RTMPTE`, `RTMPTS`, `RTSP`, `SCP`, `SFTP`, `SMB`, `SMBS`, `SMTP`, `SMTPS`, `TELNET`, `TFTP`, `WS`, `WSS`. Available as of PHP 8.3.0 and cURL 7.85.0.
CURLOPT_REFERER (int)	A string with the contents of the `Referer:` header to be used in a HTTP request. Available as of cURL 7.1.0.

CURLOPT_REQUEST_TARGET (int)	A string to use in the upcoming request instead of the path as extracted from the URL. Available as of PHP 7.3.0 and cURL 7.55.0.
CURLOPT_RESOLVE (int)	Provide an array of colon-separated strings with custom addresses for specific host and port pairs in the following format: `array( "example.com:80:127.0.0.1", "example2.com:443:127.0.0.2", )` Available as of cURL 7.21.3.
CURLOPT_RESUME_FROM (int)	The offset, in bytes, to resume a transfer from. This option accepts any value that can be cast to a valid int. Available as of cURL 7.1.0.
CURLOPT_RETURNTRANSFER (int)	**true** to return the transfer as a string of the return value of curl_exec() instead of outputting it directly.
CURLOPT_RTSP_CLIENT_CSEQ (int)	Set an int with the CSEQ number to issue for the next RTSP request. Useful if the application is resuming a previously broken connection. The CSEQ increments from this new number henceforth. Defaults to `0`. Available as of cURL 7.20.0.
CURLOPT_RTSP_REQUEST (int)	Sets the kind of RTSP request to make. Must be one of the **CURL_RTSPREQ_*** constants. Available as of cURL 7.20.0.
CURLOPT_RTSP_SERVER_CSEQ (int)	Set an int with the CSEQ number to expect for the next RTSP Server to Client request. This feature (listening for Server requests) is unimplemented. Defaults to `0`. Available as of cURL 7.20.0.
CURLOPT_RTSP_SESSION_ID (int)	Set a string with the value of the current RTSP Session ID for the handle. Once this value is set to any non-**null** value, cURL returns **CURLE_RTSP_SESSION_ERROR** if the ID received from the server does not match. If set to **null**, cURL automatically sets the ID the first time the server sets it in a response. Defaults to **null** Available as of cURL 7.20.0.
CURLOPT_RTSP_STREAM_URI (int)	Sets a string with the stream URI to operate on. If not set, cURL defaults to operating on generic server options by passing `*` in the place of the RTSP Stream URI. When working with RTSP, `CURLOPT_RTSP_STREAM_URI` indicates what URL to send to the server in the request header while the `CURLOPT_URL` indicates where to make the connection to. Available as of cURL 7.20.0.
CURLOPT_RTSP_TRANSPORT (int)	Set the `Transport:` header for this RTSP session. Available as of cURL 7.20.0.
CURLOPT_SAFE_UPLOAD (int)	Always **true**, what disables support for the `@` prefix for uploading files in **CURLOPT_POSTFIELDS**, which means that values starting with `@` can be safely passed as fields. CURLFile may be used for uploads instead.
CURLOPT_SASL_AUTHZID (int)	The authorization identity (authzid) string for the transfer. Only applicable to the PLAIN SASL authentication mechanism where it is optional. When not specified, only the authentication identity (authcid) as specified by the username will be sent to the server, along with the password. The server will derive the authzid from the authcid when not provided, which it will then use internally. Available as of PHP 8.2.0 and cURL 7.66.0.
CURLOPT_SASL_IR (int)	**true** to enable sending the initial response in the first packet. Available as of PHP 7.0.7 and cURL 7.31.0.
CURLOPT_SERVICE_NAME (int)	A string with the authentication service name. Available as of PHP 7.0.7 and cURL 7.43.0.

CURLOPT_SHARE (int)	A result of curl_share_init(). Makes the cURL handle to use the data from the shared handle. Available as of cURL 7.10.
CURLOPT_SOCKS5_AUTH (int)	The SOCKS5 authentication method(s) to use. The options are: **CURLAUTH_BASIC**, **CURLAUTH_GSSAPI**, **CURLAUTH_NONE**. When more than one method is set, cURL will poll the server to see what methods it supports and pick the best one. Defaults to `CURLAUTH_BASIC|CURLAUTH_GSSAPI`. Set the actual username and password with the **CURLOPT_PROXYUSERPWD** option. Available as of PHP 7.3.0 and cURL 7.55.0.
CURLOPT_SOCKS5_GSSAPI_NEC (int)	Set to `1` to enable and `0` to disable the unprotected exchange of the protection mode negotiation as part of the GSSAPI negotiation. Available as of cURL 7.19.4.
CURLOPT_SOCKS5_GSSAPI_SERVICE (int)	Set a string holding the name of the SOCKS5 service. Defaults to `rcmd`. Available as of cURL 7.19.4 and deprecated as of cURL 7.49.0. Use **CURLOPT_PROXY_SERVICE_NAME** instead.
CURLOPT_SSH_AUTH_TYPES (int)	A bitmask consisting of one or more of the following constants: **CURLSSH_AUTH_PUBLICKEY**, **CURLSSH_AUTH_PASSWORD**, **CURLSSH_AUTH_HOST**, **CURLSSH_AUTH_KEYBOARD**, **CURLSSH_AUTH_AGENT**, **CURLSSH_AUTH_ANY**. Defaults to **CURLSSH_AUTH_ANY**. Available as of cURL 7.16.1.
CURLOPT_SSH_COMPRESSION (int)	**true** to enable, **false** to disable built-in SSH compression. Note that the server can disregard this request. Defaults to **false**. Available as of PHP 7.3.0 and cURL 7.56.0.

CURLOPT_SSH_HOSTKEYFUNCTION (int)	A callable that will be called when SSH host key verification is needed. The callback must have the following signature:
	
	function callback(
	  resource `$curlHandle`,
	  int `$keyType`,
	  string `$key`,
	  int `$keyLength`
	): int
	
	`curlHandle`
	  The cURL handle.
	
	`keyType`
	  One of the **CURLKHTYPE_*** key types.
	
	`key`
	  The key to check.
	
	`keyLength`
	  The length of the key in bytes.
	
	This callback overrides **CURLOPT_SSH_KNOWNHOSTS**. Available as of PHP 8.3.0 and cURL 7.84.0.

CURLOPT_SSH_HOST_PUBLIC_KEY_MD5 (int)	A string containing 32 hexadecimal digits which should contain the MD5 checksum of the remote host's public key, and cURL will reject the connection to the host unless the md5sums match. This option is only for SCP and SFTP transfers. Available as of cURL 7.17.1.
CURLOPT_SSH_HOST_PUBLIC_KEY_SHA256 (int)	A string with the base64-encoded SHA256 hash of the remote host's public key. The transfer will fail if the given hash does not match the hash the remote host provides. Available as of PHP 8.2.0 and cURL 7.80.0.

CURLOPT_SSH_KNOWNHOSTS (int)	Set to the filename of the *known_host* file to use which should use the OpenSSH file format as supported by libssh2. Available as of cURL 7.19.6.
CURLOPT_SSH_PRIVATE_KEYFILE (int)	The file name for a private key. If not used, cURL defaults to *$HOME/.ssh/id_dsa* if the HOME environment variable is set, and just `id_dsa` in the current directory if HOME is not set. If the file is password-protected, set the password with **CURLOPT_KEYPASSWD**. Available as of cURL 7.16.1.
CURLOPT_SSH_PUBLIC_KEYFILE (int)	The file name for a public key. If not used, cURL defaults to *$HOME/.ssh/id_dsa.pub* if the HOME environment variable is set, and just `id_dsa.pub` in the current directory if HOME is not set. Available as of cURL 7.16.1.
CURLOPT_SSLCERT (int)	The name of a file containing a PEM formatted certificate. Available as of cURL 7.1.0.
CURLOPT_SSLCERTPASSWD (int)	The password required to use the **CURLOPT_SSLCERT** certificate. Available as of cURL 7.1.0 and deprecated as of cURL 7.17.0.
CURLOPT_SSLCERTTYPE (int)	A string with the format of the certificate. Supported formats are: `PEM`, `DER`, `ENG`, `P12`. `P12` (for PKCS#12-encoded files) is available as of OpenSSL 0.9.3. Defaults to `PEM`. Available as of cURL 7.9.3.
CURLOPT_SSLCERT_BLOB (int)	A string with the SSL client certificate. Available as of PHP 8.1.0 and cURL 7.71.0.
CURLOPT_SSLENGINE (int)	The string identifier for the crypto engine of the private SSL key specified in **CURLOPT_SSLKEY**. Available as of cURL 7.9.3.
CURLOPT_SSLENGINE_DEFAULT (int)	The string identifier for the crypto engine used for asymmetric crypto operations. Available as of cURL 7.9.3.
CURLOPT_SSLKEY (int)	The name of a file containing a private SSL key. Available as of cURL 7.9.3.
CURLOPT_SSLKEYPASSWD (int)	The secret password needed to use the private SSL key specified in **CURLOPT_SSLKEY**. > **Note** : > Since this option contains a sensitive password, remember to keep the PHP script it is contained within safe. Available as of cURL 7.9.3 and deprecated as of cURL 7.17.0.
CURLOPT_SSLKEYTYPE (int)	The key type of the private SSL key specified in **CURLOPT_SSLKEY**. Supported key types are: `PEM`, `DER`, `ENG`. Defaults to `PEM`. Available as of cURL 7.9.3.
CURLOPT_SSLKEY_BLOB (int)	A string private key for client cert. Available as of PHP 8.1.0 and cURL 7.71.0.
CURLOPT_SSLVERSION (int)	One of the **CURL_SSLVERSION_*** constants. > **Warning** > It is better to not set this option and leave the defaults. As setting this to **CURL_SSLVERSION_SSLv2** or **CURL_SSLVERSION_SSLv3** is very dangerous, given the known vulnerabilities in SSLv2 and SSLv3.

	Defaults to `CURL_SSLVERSION_DEFAULT`. Available as of cURL 7.1.0.
`CURLOPT_SSL_CIPHER_LIST` (int)	A colon-separated string of ciphers to use for the TLS 1.2 (1.1, 1.0) connection. Available as of cURL 7.9.
`CURLOPT_SSL_EC_CURVES` (int)	A colon delimited list of elliptic curve algorithms. For example, `X25519:P-521` is a valid list of two elliptic curves. This option defines the client's key exchange algorithms in the SSL handshake, if the SSL backend cURL is built to use supports it. Available as of PHP 8.2.0 and cURL 7.73.0.
`CURLOPT_SSL_ENABLE_ALPN` (int)	`false` to disable ALPN in the SSL handshake (if the SSL backend cURL is built to use supports it), which can be used to negotiate http2. Available as of PHP 7.0.7 and cURL 7.36.0.
`CURLOPT_SSL_ENABLE_NPN` (int)	`false` to disable NPN in the SSL handshake (if the SSL backend cURL is built to use supports it), which can be used to negotiate http2. Available as of PHP 7.0.7 and cURL 7.36.0, and deprecated as of cURL 7.86.0.
`CURLOPT_SSL_FALSESTART` (int)	`true` to enable and `false` to disable TLS false start which is a mode where a TLS client starts sending application data before verifying the server's `Finished` message. Available as of PHP 7.0.7 and cURL 7.42.0. Deprecated since cURL 8.15.0.
`CURLOPT_SSL_OPTIONS` (int)	Set SSL behavior options, which is a bitmask of the `CURLSSLOPT_*` constants. Defaults to none of the bits being set. Available as of PHP 7.0.7. and cURL 7.25.0.
`CURLOPT_SSL_SESSIONID_CACHE` (int)	Set to `0` to disable and `1` to enable SSL session-ID caching. By default all transfers are done using the cache enabled. Available as of cURL 7.16.0.
`CURLOPT_SSL_VERIFYHOST` (int)	`2` to verify that a Common Name field or a Subject Alternate Name field in the SSL peer certificate matches the provided hostname. `0` to not check the names. `1` should not be used. In production environments the value of this option should be kept at `2` (default value). Support for value `1` removed in cURL 7.28.1. Available as of cURL 7.8.1.
`CURLOPT_SSL_VERIFYPEER` (int)	`false` to stop cURL from verifying the peer's certificate. Alternate certificates to verify against can be specified with the `CURLOPT_CAINFO` option or a certificate directory can be specified with the `CURLOPT_CAPATH` option. Defaults to `true` as of cURL 7.10. Default bundle of CA certificates installed as of cURL 7.10. Available as of cURL 7.4.2.
`CURLOPT_SSL_VERIFYSTATUS` (int)	`true` to enable and `false` to disable verification of the certificate's status. Available as of PHP 7.0.7 and cURL 7.41.0.
`CURLOPT_STDERR` (int)	Accepts a file handle resource pointing to an alternative location to output errors to instead of `STDERR`. Available as of cURL 7.1.0.
`CURLOPT_STREAM_WEIGHT` (int)	Set the numerical stream weight (a number between `1` and `256`). Available as of PHP 7.0.7 and cURL 7.46.0.
`CURLOPT_SUPPRESS_CONNECT_HEADERS` (int)	`true` to suppress proxy CONNECT response headers from the user callback functions `CURLOPT_HEADERFUNCTION` and `CURLOPT_WRITEFUNCTION`, when `CURLOPT_HTTPPROXYTUNNEL` is used and a CONNECT request is made. Defaults to `false`. Available as of PHP 7.3.0 and cURL 7.54.0.
`CURLOPT_TCP_FASTOPEN` (int)	`true` to enable and `false` to disable TCP Fast Open. Available as of PHP 7.0.7 and cURL 7.49.0.

CURLOPT_TCP_KEEPALIVE (int)	If set to 1, TCP keepalive probes will be sent. The delay and frequency of these probes can be controlled by the CURLOPT_TCP_KEEPIDLE and CURLOPT_TCP_KEEPINTVL options, provided the operating system supports them. If set to 0 (default) keepalive probes are disabled. The maximum number of probes can be set with the CURLOPT_TCP_KEEPCNT option. Available as of cURL 7.25.0.
CURLOPT_TCP_KEEPIDLE (int)	Sets the delay, in seconds, that the operating system will wait while the connection is idle before sending keepalive probes, if CURLOPT_TCP_KEEPALIVE is enabled. Not all operating systems support this option. The default is 60. Available as of cURL 7.25.0.
CURLOPT_TCP_KEEPINTVL (int)	Sets the interval, in seconds, that the operating system will wait between sending keepalive probes, if CURLOPT_TCP_KEEPALIVE is enabled. Not all operating systems support this option. The default is 60. Available as of cURL 7.25.0.
CURLOPT_TCP_KEEPCNT (int)	Sets the maximum number of TCP keep-alive probes. The default is 9. Available as of PHP 8.4.0 and cURL 8.9.0.
CURLOPT_TCP_NODELAY (int)	true to disable TCP's Nagle algorithm, which tries to minimize the number of small packets on the network. Defaults to true. Available as of cURL 7.11.2.
CURLOPT_TELNETOPTIONS (int)	Set an array of strings to pass to the telnet negotiations. The variables should be in the format <option=value>. cURL supports the options TTYPE, XDISPLOC and NEW_ENV. Available as of cURL 7.7.0.
CURLOPT_TFTP_BLKSIZE (int)	Set the blocksize to use for TFTP data transmission. Valid range is 8-65464 bytes. The default of 512 bytes is used if this option is not specified. The specified block size is only used if supported by the remote server. If the server does not return an option acknowledgment or returns an option acknowledgment with no block size, the default of 512 bytes is used. Available as of cURL 7.19.4.
CURLOPT_TFTP_NO_OPTIONS (int)	true to not send TFTP options requests. Defaults to false. Available as of PHP 7.0.7 and cURL 7.48.0.
CURLOPT_TIMECONDITION (int)	Set how CURLOPT_TIMEVALUE is treated. Use CURL_TIMECOND_IFMODSINCE to return the page only if it has been modified since the time specified in CURLOPT_TIMEVALUE. If it hasn't been modified, a 304 Not Modified header will be returned assuming CURLOPT_HEADER is true. Use CURL_TIMECOND_IFUNMODSINCE for the reverse effect. Use CURL_TIMECOND_NONE to ignore CURLOPT_TIMEVALUE and always return the page. CURL_TIMECOND_NONE is the default. Prior to cURL 7.46.0 the default was CURL_TIMECOND_IFMODSINCE. Available as of cURL 7.1.0.
CURLOPT_TIMEOUT (int)	The maximum number of seconds to allow cURL functions to execute. Defaults to 0, meaning that functions never time out during transfer. Available as of cURL 7.1.0.
CURLOPT_TIMEOUT_MS (int)	The maximum number of milliseconds to allow cURL functions to execute. If cURL is built to use the standard system name resolver, that portion of the connect will still use full-second resolution for timeouts with a minimum timeout allowed of one second. Defaults to 0, meaning that functions never time out during transfer. Available as of cURL 7.16.2.
CURLOPT_TIMEVALUE (int)	The time in seconds since January 1st, 1970. The time will be used by CURLOPT_TIMECONDITION. Defaults to 0. Available as of cURL 7.1.0.
CURLOPT_TIMEVALUE_LARGE (int)	The time in seconds since January 1st, 1970. The time will be used by CURLOPT_TIMECONDITION. Defaults to zero. The difference between this option and CURLOPT_TIMEVALUE is the type of the

	argument. On systems where 'long' is only 32 bit wide, this option has to be used to set dates beyond the year 2038. Available as of PHP 7.3.0 and cURL 7.59.0.
CURLOPT_TLS13_CIPHERS (int)	A string with a colon-separated list of ciphers to use for the connection to the TLS 1.3 connection. This option is currently used only when cURL is built to use OpenSSL 1.1.1 or later. When using a different SSL backend the TLS 1.3 cipher suites can be set with the **CURLOPT_SSL_CIPHER_LIST** option. Available as of PHP 7.3.0 and cURL 7.61.0.
CURLOPT_TLSAUTH_PASSWORD (int)	Set a password to use for the TLS authentication method specified with the **CURLOPT_TLSAUTH_TYPE** option. Requires that the **CURLOPT_TLSAUTH_USERNAME** option also be set. This feature relies on TLS SRP which does not work with TLS 1.3. Available as of cURL 7.21.4.
CURLOPT_TLSAUTH_TYPE (int)	Set a string with the method of the TLS authentication. Supported method is `SRP` (TLS Secure Remote Password authentication). Available as of cURL 7.21.4.
CURLOPT_TLSAUTH_USERNAME (int)	Set a string with the username to use for the TLS authentication method specified with the **CURLOPT_TLSAUTH_TYPE** option. Requires that the **CURLOPT_TLSAUTH_PASSWORD** option also be set. This feature relies on TLS SRP which does not work with TLS 1.3. Available as of cURL 7.21.4.
CURLOPT_TRANSFER_ENCODING (int)	Set to `1` to enable and `0` to disable requesting compressed `Transfer Encoding` in the outgoing HTTP request. If the server responds with a compressed `Transfer Encoding`, cURL will automatically uncompress it on reception. Defaults to `0`. Available as of cURL 7.21.6.
CURLOPT_TRANSFERTEXT (int)	**true** to use ASCII mode for FTP transfers. For LDAP, it retrieves data in plain text instead of HTML. On Windows systems, it will not set **STDOUT** to binary mode. Defaults to **false**. Available as of cURL 7.1.1.
CURLOPT_UNIX_SOCKET_PATH (int)	Enables the use of Unix domain sockets as connection endpoint and sets the path to the given string. Set to **null** to disable. Defaults to **null**. Available as of PHP 7.0.7 and cURL 7.40.0.
CURLOPT_UNRESTRICTED_AUTH (int)	**true** to keep sending the username and password when following locations (using **CURLOPT_FOLLOWLOCATION**), even when the hostname has changed. Defaults to **false**. Available as of cURL 7.10.4.
CURLOPT_UPKEEP_INTERVAL_MS (int)	Some protocols have "connection upkeep" mechanisms. These mechanisms usually send some traffic on existing connections in order to keep them alive. This option defines the connection upkeep interval. Currently, the only protocol with a connection upkeep mechanism is HTTP/2. When the connection upkeep interval is exceeded, an HTTP/2 PING frame is sent on the connection. Defaults to **CURL_UPKEEP_INTERVAL_DEFAULT** which is currently `60` seconds. Available as of PHP 8.2.0 and cURL 7.62.0.
CURLOPT_UPLOAD (int)	**true** to prepare for and perform an upload. Defaults to **false**. Available as of cURL 7.1.0.
CURLOPT_UPLOAD_BUFFERSIZE (int)	Preferred buffer size in bytes for the cURL upload buffer. The upload buffer size by default is 64 kilobytes. The maximum buffer size allowed to be set is 2 megabytes. The minimum buffer size allowed to be set is 16 kilobytes. Available as of PHP 8.2.0 and cURL 7.62.0.
CURLOPT_URL (int)	The URL to fetch. This can also be set when initializing a session with curl_init(). Available as of cURL 7.1.0.
CURLOPT_USE_SSL (int)	Sets the desired level of SSL/TLS for the transfer when using FTP, SMTP, POP3, IMAP, etc. These are all protocols that start out plain text and get "upgraded" to SSL using the STARTTLS

	command. Set to one of the **CURLUSESSL_*** constants. Available as of cURL 7.17.0.
CURLOPT_USERAGENT (int)	The contents of the `User-Agent:` header to be used in a HTTP request. Available as of cURL 7.1.0.
CURLOPT_USERNAME (int)	The user name to use in authentication. Available as of cURL 7.19.1.
CURLOPT_USERPWD (int)	A username and password formatted as `[username]:[password]` to use for the connection. Available as cURL 7.1.0.
CURLOPT_VERBOSE (int)	**true** to output verbose information. Writes output to **STDERR**, or the file specified using **CURLOPT_STDERR**. Defaults to **false**. Available as of cURL 7.1.0.
CURLOPT_WILDCARDMATCH (int)	Set to `1` to transfer multiple files according to a filename pattern. The pattern can be specified as part of the **CURLOPT_URL** option, using an fnmatch-like pattern (Shell Pattern Matching) in the last part of URL (filename). Available as of cURL 7.21.0.

CURLOPT_WRITEFUNCTION (int)	A callable with the following signature:
	
	function callback(resource `$curlHandle`, string `$data`): int
	
	`curlHandle`
	
	The cURL handle.
	
	`data`
	
	The data to be written.
	
	The data must be saved by the callback and the callback must return the exact number of bytes written or the transfer will be aborted with an error. Available as of cURL 7.1.0.

CURLOPT_WRITEHEADER (int)	Accepts a file handle resource to the file that the header part of the transfer is written to. Available as of cURL 7.1.0.
CURLOPT_WS_OPTIONS (int)	Accepts a bitmask setting WebSocket behavior options. The only available option is **CURLWS_RAW_MODE**. Defaults to `0`. Available as of PHP 8.3.0 and cURL 7.86.0.

CURLOPT_XFERINFOFUNCTION (int)	A callable with the following signature:
	
	function callback(
	resource `$curlHandle`,
	int `$bytesToDownload`,
	int `$bytesDownloaded`,
	int `$bytesToUpload`,
	int `$bytesUploaded`
	): int
	
	`curlHandle`
	
	The cURL handle.
	
	`bytesToDownload`
	
	The total number of bytes expected to be downloaded in this transfer.
	
	`bytesDownloaded`
	
	The number of bytes downloaded so far.
	
	`bytesToUpload`

	The total number of bytes expected to be uploaded in this transfer. `bytesUploaded` The number of bytes uploaded so far. Return `1` to abort the transfer and set a **`CURLE_ABORTED_BY_CALLBACK`** error. Available as of PHP 8.2.0 and cURL 7.32.0.
`CURLOPT_SERVER_RESPONSE_TIMEOUT` (int)	A timeout in seconds cURL will wait for a response from an FTP, SFTP, IMAP, SCP, SMTP, or a POP3 server. This option replaces the existing **`CURLOPT_FTP_RESPONSE_TIMEOUT`** option which is deprecated in cURL 7.85.0. Available as of PHP 8.4.0.
`CURLOPT_XOAUTH2_BEARER` (int)	Specifies the OAuth 2.0 access token. Set to **`null`** to disable. Defaults to **`null`**. Available as of PHP 7.0.7 and cURL 7.33.0.

`CURLOPT_PREREQFUNCTION` (int)	A callable with the following signature that gets called after the connection is established, but before the request payload (for example, the GET/POST/DELETE request of an HTTP connection) is sent, and can be used to abort or allow the connection depending on the source and destination IP address and port numbers:
	
	function callback(
	CurlHandle `$curlHandle`,
	string `$destination_ip`,
	string `$local_ip`,
	int `$destination_port`,
	int `$local_port`
	): int
	
	`curlHandle`
	
	The cURL handle.
	
	`destination_ip`
	
	The primary IP of the remote server established with this connection. For FTP, this is the IP for the control connection. IPv6 addresses are represented without surrounding brackets.
	
	`local_ip`
	
	The originating IP for this connection. IPv6 addresses are represented without surrounding brackets.
	
	`destination_port`
	
	The primary port number on the remote server established with this connection. For FTP, this is the port for the control connection. This can be a TCP or a UDP port number depending on the protocol.
	
	`local_port`
	
	The originating port number for this connection. This can be a TCP or a UDP port number depending on the protocol.
	
	Return **`CURL_PREREQFUNC_OK`** to allow the request, or **`CURL_PREREQFUNC_ABORT`** to abort the transfer. Available as of PHP 8.4.0 and cURL 7.80.0.

`CURLOPT_DEBUGFUNCTION` (int)	Available as of PHP 8.4.0. This option requires **`CURLOPT_VERBOSE`** option enabled. A callable to replace the standard cURL verbose output. This callback gets called during various stages of the request with verbose debug information. The callback should match the following signature: function callback(CurlHandle `$curlHandle`, int `$type`, string `$data`): void `curlHandle` The cURL handle.

`type`

One of the following constants indicating the type of the `data` value:

Constants	Description
CURLINFO_TEXT (int)	Informational text.
CURLINFO_HEADER_IN (int)	Header (or header-like) data received from the peer.
CURLINFO_HEADER_OUT (int)	Header (or header-like) data sent to the peer.
CURLINFO_DATA_IN (int)	Unprocessed protocol data received from the peer. Even if the data is encoded or compressed, it is not provided decoded nor decompressed to this callback.
CURLINFO_DATA_OUT (int)	Protocol data sent to the peer.
CURLINFO_SSL_DATA_IN (int)	SSL/TLS (binary) data received from the peer.
CURLINFO_SSL_DATA_OUT (int)	SSL/TLS (binary) data sent to the peer.

`data`

Verbose debug data of the type indicate by the `type` parameter.

curl_share_setopt()

Constants	Description
CURL_LOCK_DATA_CONNECT (int)	Shares/unshares the connection cache. Available as of PHP 7.3.0 and cURL 7.10.3.
CURL_LOCK_DATA_COOKIE (int)	Shares/unshares cookie data. Available as of cURL 7.10.3.
CURL_LOCK_DATA_DNS (int)	Shares/unshares DNS cache. Note that when you use cURL multi handles, all handles added to the same multi handle will share DNS cache by default. Available as of cURL 7.10.3.
CURL_LOCK_DATA_PSL (int)	Shares/unshares the Public Suffix List. Available as of PHP 7.3.0 and cURL 7.61.0.
CURL_LOCK_DATA_SSL_SESSION (int)	Shares/unshares SSL session IDs, reducing the time spent on the SSL handshake when reconnecting to the same server. Note that SSL session IDs are reused within the same handle by default. Available as of cURL 7.10.3.
CURLSHOPT_NONE (int)	Available as of cURL 7.10.3.
CURLSHOPT_SHARE (int)	Specifies a type of data that should be shared. Available as of cURL 7.10.3.
CURLSHOPT_UNSHARE (int)	Specifies a type of data that will be no longer shared. Available as of cURL 7.10.3.

curl_getinfo()

Constants	Description

CURLINFO_APPCONNECT_TIME (int)	Time in seconds it took from the start until the SSL/SSH connect/handshake to the remote host was completed
CURLINFO_APPCONNECT_TIME_T (int)	Time, in microseconds, it took from the start until the SSL/SSH connect/handshake to the remote host was completed. Available as of PHP 7.3.0 and cURL 7.61.0
CURLINFO_CAINFO (int)	Default built-in CA certificate path. Available as of PHP 8.3.0 and cURL 7.84.0
CURLINFO_CAPATH (int)	Default built-in CA path string. Available as of PHP 8.3.0 and cURL 7.84.0
CURLINFO_CERTINFO (int)	TLS certificate chain
CURLINFO_CONDITION_UNMET (int)	Info on unmet time conditional
CURLINFO_CONN_ID (int)	The ID of the last connection used by the transfer. The connection ID is unique among all connections using the same connection cache and is useful to tell connection reuse apart. Available as of PHP 8.5.0 and cURL 8.2.0.
CURLINFO_CONNECT_TIME (int)	Time in seconds it took to establish the connection
CURLINFO_CONNECT_TIME_T (int)	Total time taken, in microseconds, from the start until the connection to the remote host (or proxy) was completed. Available as of PHP 7.3.0 and cURL 7.61.0
CURLINFO_CONTENT_LENGTH_DOWNLOAD (int)	Content length of download, read from Content-Length: field. Deprecated since cURL 7.55.0. Use **CURLINFO_CONTENT_LENGTH_DOWNLOAD_T** instead.
CURLINFO_CONTENT_LENGTH_DOWNLOAD_T (int)	The content-length of the download. This is the value read from the Content-Length: field. -1 if the size isn't known. Available as of PHP 7.3.0 and cURL 7.55.0
CURLINFO_CONTENT_LENGTH_UPLOAD (int)	Specified size of upload. Deprecated since cURL 7.55.0. Use **CURLINFO_CONTENT_LENGTH_UPLOAD_T** instead.
CURLINFO_CONTENT_LENGTH_UPLOAD_T (int)	The specified size of the upload. -1 if the size isn't known. Available as of PHP 7.3.0 and cURL 7.55.0
CURLINFO_CONTENT_TYPE (int)	`Content-Type` of the requested document. NULL indicates server did not send valid `Content-Type` header
CURLINFO_COOKIELIST (int)	All known cookies
CURLINFO_EFFECTIVE_METHOD (int)	Get the last used HTTP method.
CURLINFO_EFFECTIVE_URL (int)	Last effective URL
CURLINFO_FILETIME (int)	Remote time of the retrieved document, with the **CURLOPT_FILETIME** enabled; if -1 is returned the time of the document is unknown

CURLINFO_FILETIME_T (int)	Remote time of the retrieved document (as Unix timestamp), an alternative to **CURLINFO_FILETIME** to allow systems with 32 bit long variables to extract dates outside of the 32bit timestamp range. Available as of PHP 7.3.0 and cURL 7.59.0
CURLINFO_FTP_ENTRY_PATH (int)	Entry path in FTP server
CURLINFO_HEADER_OUT (int)	The request string sent. For this to work, add the **CURLINFO_HEADER_OUT** option to the handle by calling curl_setopt()
CURLINFO_HEADER_SIZE (int)	Total size of all headers received
CURLINFO_HTTPAUTH_AVAIL (int)	Bitmask indicating the authentication method(s) available according to the previous response
CURLINFO_HTTPAUTH_USED (int)	Bitmask indicating the HTTP authentication method(s) actually used in the previous request. Available as of PHP 8.5.0 and cURL 8.12.0.
CURLINFO_HTTP_CODE (int)	The last response code. As of cURL 7.10.8, this is a legacy alias of **CURLINFO_RESPONSE_CODE**.
CURLINFO_HTTP_CONNECTCODE (int)	The CONNECT response code
CURLINFO_HTTP_VERSION (int)	The version used in the last HTTP connection. The return value will be one of the defined **CURL_HTTP_VERSION_*** constants or 0 if the version can't be determined. Available as of PHP 7.3.0 and cURL 7.50.0
CURLINFO_LASTONE (int)	The last enum value in the underlying `CURLINFO` enum in `libcurl`.
CURLINFO_LOCAL_IP (int)	Local (source) IP address of the most recent connection
CURLINFO_LOCAL_PORT (int)	Local (source) port of the most recent connection
CURLINFO_NAMELOOKUP_TIME (int)	Time in seconds until name resolving was complete
CURLINFO_NAMELOOKUP_TIME_T (int)	Time in microseconds from the start until the name resolving was completed. Available as of PHP 7.3.0 and cURL 7.61.0
CURLINFO_NUM_CONNECTS (int)	Number of connections curl had to create to achieve the previous transfer
CURLINFO_OS_ERRNO (int)	Errno from a connect failure. The number is OS and system specific.
CURLINFO_PRETRANSFER_TIME (int)	Time in seconds from start until just before file transfer begins
CURLINFO_PRETRANSFER_TIME_T (int)	Time taken from the start until the file transfer is just about to begin, in microseconds. Available as of PHP 7.3.0 and cURL 7.61.0
CURLINFO_PRIMARY_IP (int)	IP address of the most recent connection

CURLINFO_PRIMARY_PORT (int)	Destination port of the most recent connection
CURLINFO_PRIVATE (int)	Private data associated with this cURL handle, previously set with the CURLOPT_PRIVATE option of curl_setopt()
CURLINFO_PROTOCOL (int)	The protocol used in the last HTTP connection. The returned value will be exactly one of the CURLPROTO_* values. Available as of PHP 7.3.0 and cURL 7.52.0. Deprecated since cURL 7.85.0. Use CURLINFO_SCHEME instead.
CURLINFO_PROXYAUTH_AVAIL (int)	Bitmask indicating the proxy authentication method(s) available according to the previous response
CURLINFO_PROXYAUTH_USED (int)	Bitmask indicating the proxy authentication method(s) actually used in the previous request. Available as of PHP 8.5.0 and cURL 8.12.0.
CURLINFO_PROXY_ERROR (int)	The detailed (SOCKS) proxy error code when the most recent transfer returned a CURLE_PROXY error. The returned value will be exactly one of the CURLPX_* values. The error code will be CURLPX_OK if no response code was available. Available as of PHP 8.2.0 and cURL 7.73.0
CURLINFO_PROXY_SSL_VERIFYRESULT (int)	The result of the certificate verification that was requested (using the CURLOPT_PROXY_SSL_VERIFYPEER option). Only used for HTTPS proxies. Available as of PHP 7.3.0 and cURL 7.52.0
CURLINFO_QUEUE_TIME_T (int)	The time, in microseconds, that the transfer was held in a waiting queue before it started, due to limits set with CURLMOPT_MAX_TOTAL_CONNECTIONS or similar options. Available as of PHP 8.5.0 and cURL 8.6.0.
CURLINFO_REDIRECT_COUNT (int)	Number of redirects, with the CURLOPT_FOLLOWLOCATION option enabled
CURLINFO_REDIRECT_TIME (int)	Time in seconds of all redirection steps before final transaction was started, with the CURLOPT_FOLLOWLOCATION option enabled
CURLINFO_REDIRECT_TIME_T (int)	Total time, in microseconds, it took for all redirection steps include name lookup, connect, pretransfer and transfer before final transaction was started. Available as of PHP 7.3.0 and cURL 7.61.0
CURLINFO_REDIRECT_URL (int)	With the CURLOPT_FOLLOWLOCATION option disabled: redirect URL found in the last transaction, that should be requested manually next. With the CURLOPT_FOLLOWLOCATION option enabled: this is empty. The redirect URL in this case is available in CURLINFO_EFFECTIVE_URL
CURLINFO_REFERER (int)	The `Referer` header. Available as of PHP 8.2.0 and cURL 7.76.0
CURLINFO_REQUEST_SIZE (int)	Total size of issued requests, currently only for HTTP requests
CURLINFO_RESPONSE_CODE (int)	The last response code. Available as of cURL 7.10.8
CURLINFO_RETRY_AFTER (int)	The information from the `Retry-After` header, or zero if there was no valid header. Available as of PHP 8.2.0 and cURL 7.66.0

CURLINFO_RTSP_CLIENT_CSEQ (int)	Next RTSP client CSeq
CURLINFO_RTSP_CSEQ_RECV (int)	Recently received CSeq
CURLINFO_RTSP_SERVER_CSEQ (int)	Next RTSP server CSeq
CURLINFO_RTSP_SESSION_ID (int)	RTSP session ID
CURLINFO_SCHEME (int)	The URL scheme used for the most recent connection. Available as of PHP 7.3.0 and cURL 7.52.0
CURLINFO_SIZE_DOWNLOAD (int)	Total number of bytes downloaded. Deprecated since cURL 7.55.0. Use **CURLINFO_SIZE_DOWNLOAD_T** instead.
CURLINFO_SIZE_DOWNLOAD_T (int)	Total number of bytes that were downloaded. The number is only for the latest transfer and will be reset again for each new transfer. Available as of PHP 7.3.0 and cURL 7.50.0
CURLINFO_SIZE_UPLOAD (int)	Total number of bytes uploaded. Deprecated since cURL 7.55.0. Use **CURLINFO_SIZE_UPLOAD_T** instead.
CURLINFO_SIZE_UPLOAD_T (int)	Total number of bytes that were uploaded. Available as of PHP 7.3.0 and cURL 7.50.0
CURLINFO_SPEED_DOWNLOAD (int)	Average download speed. Deprecated since cURL 7.55.0. Use **CURLINFO_SPEED_DOWNLOAD_T** instead.
CURLINFO_SPEED_DOWNLOAD_T (int)	The average download speed in bytes/second that curl measured for the complete download. Available as of PHP 7.3.0 and cURL 7.50.0
CURLINFO_SPEED_UPLOAD (int)	Average upload speed. Deprecated since cURL 7.55.0. Use **CURLINFO_SPEED_UPLOAD_T** instead.
CURLINFO_SPEED_UPLOAD_T (int)	The average upload speed in bytes/second that curl measured for the complete upload. Available as of PHP 7.3.0 and cURL 7.50.0
CURLINFO_SSL_ENGINES (int)	OpenSSL crypto-engines supported
CURLINFO_SSL_VERIFYRESULT (int)	Result of SSL certification verification requested by setting **CURLOPT_SSL_VERIFYPEER**
CURLINFO_STARTTRANSFER_TIME (int)	Time in seconds until the first byte is about to be transferred
CURLINFO_STARTTRANSFER_TIME_T (int)	Time, in microseconds, it took from the start until the first byte is received. Available as of PHP 7.3.0 and cURL 7.61.0
CURLINFO_TOTAL_TIME (int)	Total transaction time in seconds for last transfer
CURLINFO_TOTAL_TIME_T (int)	Total time in microseconds for the previous transfer, including name resolving, TCP connect etc. Available as of PHP 7.3.0 and cURL 7.61.0

CURLINFO_USED_PROXY (int)	Whether the previous transfer used a proxy; returns `1` if a proxy was used and `0` otherwise. Available as of PHP 8.5.0 and cURL 8.7.0.
CURLINFO_POSTTRANSFER_TIME_T (int)	Time it took from the start until the last byte is sent, in microseconds. Available as of PHP 8.4.0 and cURL 8.10.0

curl_multi_setopt()

Constants	Description
CURLMOPT_CHUNK_LENGTH_PENALTY_SIZE (int)	Specifies the chunk length threshold for pipelining in bytes. Available as of PHP 7.0.7 and cURL 7.30.0
CURLMOPT_CONTENT_LENGTH_PENALTY_SIZE (int)	Specifies the size threshold for pipelining penalty in bytes. Available as of PHP 7.0.7 and cURL 7.30.0
CURLMOPT_MAXCONNECTS (int)	Specifies the maximum amount of simultaneously open connections that libcurl may cache. By default the size will be enlarged to fit four times the number of handles added via curl_multi_add_handle(). When the cache is full, curl closes the oldest one in the cache to prevent the number of open connections from increasing. Available as of cURL 7.16.3.
CURLMOPT_MAX_CONCURRENT_STREAMS (int)	Specifies the maximum number of concurrent streams for connections that cURL should support on connections using HTTP/2. Valid values range from `1` to `2147483647` (`2^31 - 1`). The value passed here would be honored based on other system resources properties. Default is `100`. Available as of PHP 8.2.0 and cURL 7.67.0.
CURLMOPT_MAX_HOST_CONNECTIONS (int)	Specifies the maximum number of connections to a single host. Available as of PHP 7.0.7 and cURL 7.30.0
CURLMOPT_MAX_PIPELINE_LENGTH (int)	Specifies the maximum number of requests in a pipeline. Available as of PHP 7.0.7 and cURL 7.30.0
CURLMOPT_MAX_TOTAL_CONNECTIONS (int)	Specifies the maximum number of simultaneously open connections. Available as of PHP 7.0.7 and cURL 7.30.0
CURLMOPT_PIPELINING (int)	Pass 1 to enable or 0 to disable. Enabling pipelining on a multi handle will make it attempt to perform HTTP Pipelining as far as possible for transfers using this handle. This means that adding a second request that can use an already existing connection will "pipe" the second request on the same connection. As of cURL 7.43.0, the value is a bitmask, and passing 2 will try to multiplex the new transfer over an existing HTTP/2 connection. Passing 3 instructs cURL to ask for pipelining and multiplexing independently of each other. As of cURL 7.62.0, setting the pipelining bit has no effect. Instead of integer literals, the CURLPIPE_* constants can also be used. Available as of cURL 7.16.0.

CURLMOPT_PUSHFUNCTION (int)	Pass a callable that will be registered to handle server pushes and should have the following signature:
	
	function pushfunction(resource `$parent_ch`, resource `$pushed_ch`, array `$headers`): int
	
	`parent_ch`
	
	The parent cURL handle (the request the client made).
	
	`pushed_ch`
	
	A new cURL handle for the pushed request.

	`headers` The push promise headers. The push function is supposed to return either `CURL_PUSH_OK` if it can handle the push, or `CURL_PUSH_DENY` to reject it. Available as of PHP 7.1.0 and cURL 7.44.0

cURL protocol constants

Constants	Description
`CURLPROTO_ALL` (int)	Available as of cURL 7.19.4.
`CURLPROTO_DICT` (int)	Available as of cURL 7.19.4.
`CURLPROTO_FILE` (int)	Available as of cURL 7.19.4.
`CURLPROTO_FTP` (int)	Available as of cURL 7.19.4.
`CURLPROTO_FTPS` (int)	Available as of cURL 7.19.4.
`CURLPROTO_GOPHER` (int)	Available as of cURL 7.21.2.
`CURLPROTO_HTTP` (int)	Available as of cURL 7.19.4.
`CURLPROTO_HTTPS` (int)	Available as of cURL 7.19.4.
`CURLPROTO_IMAP` (int)	Available as of cURL 7.20.0.
`CURLPROTO_IMAPS` (int)	Available as of cURL 7.20.0.
`CURLPROTO_LDAP` (int)	Available as of cURL 7.19.4.
`CURLPROTO_LDAPS` (int)	Available as of cURL 7.19.4.
`CURLPROTO_MQTT` (int)	Available as of PHP 8.2.0 and cURL 7.71.0.
`CURLPROTO_POP3` (int)	Available as of cURL 7.20.0.
`CURLPROTO_POP3S` (int)	Available as of cURL 7.20.0.
`CURLPROTO_RTMP` (int)	Available as of cURL 7.21.0.
`CURLPROTO_RTMPE` (int)	Available as of cURL 7.21.0.
`CURLPROTO_RTMPS` (int)	Available as of cURL 7.21.0.
`CURLPROTO_RTMPT` (int)	Available as of cURL 7.21.0.

CURLPROTO_RTMPTE (int)	Available as of cURL 7.21.0.
CURLPROTO_RTMPTS (int)	Available as of cURL 7.21.0.
CURLPROTO_RTSP (int)	Available as of cURL 7.20.0.
CURLPROTO_SCP (int)	Available as of cURL 7.19.4.
CURLPROTO_SFTP (int)	Available as of cURL 7.19.4.
CURLPROTO_SMB (int)	Available as of PHP 7.0.7 and cURL 7.40.0.
CURLPROTO_SMBS (int)	Available as of PHP 7.0.7 and cURL 7.40.0.
CURLPROTO_SMTP (int)	Available as of cURL 7.20.0.
CURLPROTO_SMTPS (int)	Available as of cURL 7.20.0.
CURLPROTO_TELNET (int)	Available as of cURL 7.19.4.
CURLPROTO_TFTP (int)	Available as of cURL 7.19.4.

cURL error constants

Constants	Description
CURLE_ABORTED_BY_CALLBACK (int)	Aborted by callback. A callback returned "abort" to libcurl.
CURLE_BAD_CALLING_ORDER (int)	
CURLE_BAD_CONTENT_ENCODING (int)	Unrecognized transfer encoding.
CURLE_BAD_DOWNLOAD_RESUME (int)	The download could not be resumed because the specified offset was out of the file boundary.
CURLE_BAD_FUNCTION_ARGUMENT (int)	A function was called with a bad parameter.
CURLE_BAD_PASSWORD_ENTERED (int)	
CURLE_COULDNT_CONNECT (int)	Failed to connect to host or proxy.
CURLE_COULDNT_RESOLVE_HOST (int)	Could not resolve host. The given remote host was not resolved.
CURLE_COULDNT_RESOLVE_PROXY (int)	Could not resolve proxy. The given proxy host could not be resolved.
CURLE_FAILED_INIT (int)	Early initialization code failed. This is likely to be an internal error or problem, or a resource problem where something fundamental could not get done at init time.
CURLE_FILESIZE_EXCEEDED (int)	Maximum file size exceeded.

CURLE_FILE_COULDNT_READ_FILE (int)	A file given with FILE:// could not be opened. Most likely because the file path does not identify an existing file or due to the lack of appropriate file permissions.
CURLE_FTP_ACCESS_DENIED (int)	
CURLE_FTP_BAD_DOWNLOAD_RESUME (int)	
CURLE_FTP_CANT_GET_HOST (int)	An internal failure to lookup the host used for the new connection.
CURLE_FTP_CANT_RECONNECT (int)	
CURLE_FTP_COULDNT_GET_SIZE (int)	
CURLE_FTP_COULDNT_RETR_FILE (int)	This was either a unexpected reply to a 'RETR' command or a zero byte transfer complete.
CURLE_FTP_COULDNT_SET_ASCII (int)	
CURLE_FTP_COULDNT_SET_BINARY (int)	
CURLE_FTP_COULDNT_STOR_FILE (int)	
CURLE_FTP_COULDNT_USE_REST (int)	The FTP REST command returned error. This should never happen if the server is sane.
CURLE_FTP_PARTIAL_FILE (int)	
CURLE_FTP_PORT_FAILED (int)	The FTP PORT command returned error. This mostly happens when a good enough address has not been specified for libcurl to use. See CURLOPT_FTPPORT.
CURLE_FTP_QUOTE_ERROR (int)	
CURLE_FTP_SSL_FAILED (int)	
CURLE_FTP_USER_PASSWORD_INCORRECT (int)	
CURLE_FTP_WEIRD_227_FORMAT (int)	FTP servers return a 227-line as a response to a PASV command. If libcurl fails to parse that line, this return code is passed back.
CURLE_FTP_WEIRD_PASS_REPLY (int)	After having sent the FTP password to the server, libcurl expects a proper reply. This error code indicates that an unexpected code was returned.
CURLE_FTP_WEIRD_PASV_REPLY (int)	libcurl failed to get a sensible result back from the server as a response to either a PASV or a EPSV command. The server is flawed.
CURLE_FTP_WEIRD_SERVER_REPLY (int)	The server sent data libcurl could not parse. This error code is known as CURLE_WEIRD_SERVER_REPLY as of cURL 7.51.0.
CURLE_FTP_WEIRD_USER_REPLY (int)	
CURLE_FTP_WRITE_ERROR (int)	
CURLE_FUNCTION_NOT_FOUND (int)	Function not found. A required zlib function was not found.
CURLE_GOT_NOTHING (int)	Nothing was returned from the server, and under the circumstances, getting nothing is considered an error.
CURLE_HTTP_NOT_FOUND (int)	
CURLE_HTTP_PORT_FAILED (int)	
CURLE_HTTP_POST_ERROR (int)	This is an odd error that mainly occurs due to internal confusion.
CURLE_HTTP_RANGE_ERROR (int)	

CURLE_HTTP_RETURNED_ERROR (int)	This is returned if CURLOPT_FAILONERROR is set true and the HTTP server returns an error code that is greater than or equal to 400.
CURLE_LDAP_CANNOT_BIND (int)	LDAP cannot bind. LDAP bind operation failed.
CURLE_LDAP_INVALID_URL (int)	
CURLE_LDAP_SEARCH_FAILED (int)	LDAP search failed.
CURLE_LIBRARY_NOT_FOUND (int)	
CURLE_MALFORMAT_USER (int)	
CURLE_OBSOLETE (int)	
CURLE_OK (int)	All fine. Proceed as usual.
CURLE_OPERATION_TIMEDOUT (int)	Operation timeout. The specified time-out period was reached according to the conditions.
CURLE_OPERATION_TIMEOUTED (int)	
CURLE_OUT_OF_MEMORY (int)	A memory allocation request failed.
CURLE_PARTIAL_FILE (int)	A file transfer was shorter or larger than expected. This happens when the server first reports an expected transfer size, and then delivers data that does not match the previously given size.
CURLE_PROXY (int)	Proxy handshake error. CURLINFO_PROXY_ERROR provides extra details on the specific problem. Available as of PHP 8.2.0 and cURL 7.73.0
CURLE_READ_ERROR (int)	There was a problem reading a local file or an error returned by the read callback.
CURLE_RECV_ERROR (int)	Failure with receiving network data.
CURLE_SEND_ERROR (int)	Failed sending network data.
CURLE_SHARE_IN_USE (int)	
CURLE_SSH (int)	An unspecified error occurred during the SSH session. Available as of cURL 7.16.1.
CURLE_SSL_CACERT (int)	
CURLE_SSL_CACERT_BADFILE (int)	Problem with reading the SSL CA cert.
CURLE_SSL_CERTPROBLEM (int)	Problem with the local client certificate.
CURLE_SSL_CIPHER (int)	Could not use specified cipher.
CURLE_SSL_CONNECT_ERROR (int)	A problem occurred somewhere in the SSL/TLS handshake. Reading the message in the error buffer provides more details on the problem. Could be certificates (file formats, paths, permissions), passwords, and others.
CURLE_SSL_ENGINE_NOTFOUND (int)	The specified crypto engine was not found.

CURLE_SSL_ENGINE_SETFAILED (int)	Failed setting the selected SSL crypto engine as default.
CURLE_SSL_PEER_CERTIFICATE (int)	
CURLE_SSL_PINNEDPUBKEYNOTMATCH (int)	Failed to match the pinned key specified with CURLOPT_PINNEDPUBLICKEY.
CURLE_TELNET_OPTION_SYNTAX (int)	
CURLE_TOO_MANY_REDIRECTS (int)	Too many redirects. When following redirects, libcurl hit the maximum amount. The limit can be set with CURLOPT_MAXREDIRS.
CURLE_UNKNOWN_TELNET_OPTION (int)	
CURLE_UNSUPPORTED_PROTOCOL (int)	The URL passed to libcurl used a protocol that libcurl does not support. The issue might be a compile-time option that was not used, a misspelled protocol string or just a protocol libcurl has no code for.
CURLE_URL_MALFORMAT (int)	The URL was not properly formatted.
CURLE_URL_MALFORMAT_USER (int)	
CURLE_WEIRD_SERVER_REPLY (int)	The server sent data libcurl could not parse. This error code was known as CURLE_FTP_WEIRD_SERVER_REPLY before cURL 7.51.0. Available as of PHP 7.3.0 and cURL 7.51.0
CURLE_WRITE_ERROR (int)	An error occurred when writing received data to a local file, or an error was returned to libcurl from a write callback.

curl_multi_* status constants

Constants	Description
CURLM_ADDED_ALREADY (int)	An easy handle already added to a multi handle was attempted to get added a second time. Available as of cURL 7.32.1.
CURLM_BAD_EASY_HANDLE (int)	An easy handle was not good/valid. It could mean that it is not an easy handle at all, or possibly that the handle already is in use by this or another multi handle. Available as of cURL 7.9.6.
CURLM_BAD_HANDLE (int)	The passed-in handle is not a valid multi handle. Available as of cURL 7.9.6.
CURLM_CALL_MULTI_PERFORM (int)	As of cURL 7.20.0, this constant is not used. Before cURL 7.20.0, this status could be returned by curl_multi_exec() when curl_multi_select() or a similar function was called before it returned any other constant. Available as of cURL 7.9.6.
CURLM_INTERNAL_ERROR (int)	Internal `libcurl` error. Available as of cURL 7.9.6.
CURLM_OK (int)	No errors. Available as of cURL 7.9.6.
CURLM_OUT_OF_MEMORY (int)	Ran out of memory while processing multi handles. Available as of cURL 7.9.6.

curl_pause()

Constants	Description
CURLPAUSE_ALL (int)	Pause sending and receiving data. Available as of cURL 7.18.0.

CURLPAUSE_CONT (int)	Unpause sending and receiving data. Available as of cURL 7.18.0.
CURLPAUSE_RECV (int)	Pause receiving data. Available as of cURL 7.18.0.
CURLPAUSE_RECV_CONT (int)	Unpause receiving data. Available as of cURL 7.18.0.
CURLPAUSE_SEND (int)	Pause sending data. Available as of cURL 7.18.0.
CURLPAUSE_SEND_CONT (int)	Unpause sending data. Available as of cURL 7.18.0.

Examples

Table of Contents

Basic curl examples

Once you've compiled PHP with cURL support, you can begin using the cURL functions. The basic idea behind the cURL functions is that you initialize a cURL session using the curl_init(), then you can set all your options for the transfer via the curl_setopt(), then you can execute the session with the curl_exec().

Here is a simple example that uses the cURL functions to send a POST request:

Example #1 Using PHP's cURL module to send a POST request

```php
<?php
$data = ['foo' => 'bar', 'baz' => 48];
$url = "http://www.example.com/handler.php";

$ch = curl_init($url);

// tell CURL to return the response instead of sending it to stdout
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// set the POST data, corresponding method and headers
curl_setopt($ch, CURLOPT_POSTFIELDS, http_build_query($data));

// send the request and get the response
$response = curl_exec($ch);

if(curl_error($ch)) {
    // handle the error, or just
    throw new RuntimeException(curl_error($ch));
}
```

Another example that uses the cURL functions to send a raw JSON POST request:

Example #2 Using PHP's cURL module to send a JSON POST request

```php
<?php
$post_data = ['foo' => 'bar', 'baz' => 48];
$url = "http://www.example.com/api/endpoint";

$ch = curl_init($url);

// tell CURL to return the response instead of sending it to stdout
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// set the POST data and corresponding method
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($post_data));

// set the required header
curl_setopt($ch, CURLOPT_HTTPHEADER, ['Content-Type: application/json']);
```

```php
// send the request and get the response
$response = curl_exec($ch);

if(curl_error($ch)) {
    // handle the error, or just
    throw new RuntimeException(curl_error($ch));
}
```

Here is another example that uses the cURL functions to fetch the example.com homepage into a file:

Example #3 Using PHP's cURL module to fetch the example.com homepage

```php
<?php

$ch = curl_init("http://www.example.com/");
$fp = fopen("example_homepage.txt", "w");

curl_setopt($ch, CURLOPT_FILE, $fp);
curl_setopt($ch, CURLOPT_HEADER, 0);

curl_exec($ch);
if(curl_error($ch)) {
    fwrite($fp, curl_error($ch));
}
fclose($fp);
```

cURL Functions

Table of Contents

curl_close

curl_close

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_close — Close a cURL session

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.5',
    message: "as it has no effect since PHP 8.0",
)]
function curl_close(CurlHandle $handle): void
```

> **Note** :
>
> This function has no effect. Prior to PHP 8.0.0, this function was used to close the resource.

Closes a cURL session and frees all resources. The cURL handle, `handle`, is also deleted.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

No value is returned.

Changelog

Version	Description
8.5.0	This function has been deprecated.
8.0.0	This function is a NOP now.
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 Initializing a new cURL session and fetching a web page

```php
<?php
// create a new cURL resource
$ch = curl_init();

// set URL and other appropriate options
curl_setopt($ch, CURLOPT_URL, "http://www.example.com/");
curl_setopt($ch, CURLOPT_HEADER, 0);

// grab URL and pass it to the browser
curl_exec($ch);

// close cURL resource, and free up system resources
curl_close($ch);
?>
```

See Also

- curl_init() - Initialize a cURL session
- curl_multi_close() - Remove all cURL handles from a multi handle

curl_copy_handle

curl_copy_handle

(PHP 5, PHP 7, PHP 8)

curl_copy_handle — Copy a cURL handle along with all of its preferences

Description

function **curl_copy_handle**(CurlHandle `$handle`): CurlHandle|false

Copies a cURL handle keeping the same preferences.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

Returns a new cURL handle, or `false` on failure.

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.
8.0.0	On success, this function returns a CurlHandle instance now; previously, a resource was returned.

Examples

Example #1 Copying a cURL handle

```php
<?php
// create a new cURL resource
$ch = curl_init();
```

```php
// set URL and other appropriate options
curl_setopt($ch, CURLOPT_URL, 'http://www.example.com/');
curl_setopt($ch, CURLOPT_HEADER, 0);

// copy the handle
$ch2 = curl_copy_handle($ch);

// grab URL (http://www.example.com/) and pass it to the browser
curl_exec($ch2);
?>
```

curl_errno

curl_errno

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

curl_errno — Return the last error number

Description

function **curl_errno**(CurlHandle `$handle`): int

Returns the error number for the last cURL operation.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

Returns the error number or `0` (zero) if no error occurred.

> **Note** :
>
> When a CurlHandle is executed via curl_multi_exec(), this function always returns `0`. The error code for each individual handle is available as the `result` key returned by curl_multi_info_read().

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_errno() example

```php
<?php
// Create a curl handle to a non-existing location
$ch = curl_init('http://404.php.net/');

// Execute
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_exec($ch);

// Check if any error occurred
if(curl_errno($ch))
{
    echo 'Curl error: ' . curl_error($ch);
}
?>
```

See Also

- curl_error() - Return a string containing the last error for the current session

- curl_multi_info_read() - Get information about the current transfers
- » Curl error codes

curl_error

curl_error

(PHP 4 >= 4.0.3, PHP 5, PHP 7, PHP 8)

curl_error — Return a string containing the last error for the current session

Description

function **curl_error**(CurlHandle `$handle`): string

Returns a clear text error message for the last cURL operation.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

Returns the error message or `''` (the empty string) if no error occurred.

> **Note** :
>
> When a CurlHandle is executed via curl_multi_exec(), this function always returns `''` (the empty string). To retrieve the error string for each individual handle, pass the `result` key returned by curl_multi_info_read() to curl_strerror().

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_error() example

```php
<?php
// Create a curl handle to a non-existing location
$ch = curl_init('http://404.php.net/');
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

if(curl_exec($ch) === false)
{
    echo 'Curl error: ' . curl_error($ch);
}
else
{
    echo 'Operation completed without any errors';
}
?>
```

See Also

- curl_errno() - Return the last error number
- curl_strerror() - Return string describing the given error code
- curl_multi_info_read() - Get information about the current transfers
- » Curl error codes

curl_escape

curl_escape

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_escape — URL encodes the given string

Description

function **curl_escape**(CurlHandle `$handle`, string `$string`): string|false

This function URL encodes the given string according to » RFC 3986.

Parameters

`handle`

A cURL handle returned by curl_init().

`string`

The string to be encoded.

Return Values

Returns escaped string or `false` on failure.

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_escape() example

```php
<?php

// Create a curl handle
$ch = curl_init();

// Escape a string used as a GET parameter
$location = curl_escape($ch, 'Hofbräuhaus / München');
// Result: Hofbr%C3%A4uhaus%20%2F%20M%C3%BCnchen

// Compose an URL with the escaped string
$url = "http://example.com/add_location.php?location={$location}";
// Result: http://example.com/add_location.php?location=Hofbr%C3%A4uhaus%20%2F%20M%C3%BCnchen

// Set options and send HTTP request
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_exec($ch);

?>
```

See Also

- curl_unescape() - Decodes the given URL encoded string
- urlencode() - URL-encodes string
- rawurlencode() - URL-encode according to RFC 3986
- » RFC 3986

curl_exec

curl_exec

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_exec — Perform a cURL session

Description

function **curl_exec**(CurlHandle `$handle`): string|bool

Execute the given cURL session.

This function should be called after initializing a cURL session and all the options for the session are set.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

On success, this function flushes the result directly to the `stdout` and returns **`true`**, or **`false`** on failure. However, if the **`CURLOPT_RETURNTRANSFER`** option is set, it will return the result on success, **`false`** on failure.

> **Warning**
>
> This function may return Boolean **`false`**, but may also return a non-Boolean value which evaluates to **`false`**. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

> **Note** :
>
> Note that response status codes which indicate errors (such as `404 Not found`) are not regarded as failure. curl_getinfo() can be used to check for these.

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 Fetching a web page

```php
<?php
// create a new cURL resource
$ch = curl_init();

// set URL and other appropriate options
curl_setopt($ch, CURLOPT_URL, "http://www.example.com/");
curl_setopt($ch, CURLOPT_HEADER, 0);

// grab URL and pass it to the browser
curl_exec($ch);
?>
```

See Also

- curl_multi_exec() - Processes each of the handles in the stack

curl_getinfo

curl_getinfo

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

curl_getinfo — Get information regarding a specific transfer

Description

function **curl_getinfo**(CurlHandle `$handle`, ?int `$option` = **`null`**): mixed

Gets information about the last transfer.

Parameters

`handle`

A cURL handle returned by curl_init().

`option`

One of the **`CURLINFO_*`** constants.

Return Values

If `option` is given, returns its value. Otherwise, returns an associative array with the following elements (which correspond to `option`), or **`false`** on failure:

- "url"
- "content_type"
- "http_code"
- "header_size"
- "request_size"
- "filetime"
- "ssl_verify_result"
- "redirect_count"
- "total_time"
- "namelookup_time"
- "connect_time"
- "pretransfer_time"
- "size_upload"
- "size_download"
- "speed_download"
- "speed_upload"
- "download_content_length"
- "upload_content_length"
- "starttransfer_time"
- "redirect_time"
- "certinfo"
- "primary_ip"
- "primary_port"
- "local_ip"
- "local_port"
- "redirect_url"
- "request_header" (This is only set if the **`CURLINFO_HEADER_OUT`** is set by a previous call to curl_setopt())
- "posttransfer_time_us" (Available as of PHP 8.4.0 and cURL 8.10.0)

Note that private data is not included in the associative array and must be retrieved individually with the **`CURLINFO_PRIVATE`** option.

Changelog

Version	Description
8.4.0	Introduced CURLINFO_POSTTRANSFER_TIME_T constant and posttransfer_time_us (Curl 8.10.0 or later).
8.3.0	Introduced CURLINFO_CAINFO and CURLINFO_CAPATH.
8.2.0	Introduced CURLINFO_PROXY_ERROR, CURLINFO_REFERER, CURLINFO_RETRY_AFTER.
8.0.0	handle expects a CurlHandle instance now; previously, a resource was expected.
8.0.0	option is nullable now; previously, the default was 0.
7.3.0	Introduced CURLINFO_CONTENT_LENGTH_DOWNLOAD_T, CURLINFO_CONTENT_LENGTH_UPLOAD_T, CURLINFO_HTTP_VERSION, CURLINFO_PROTOCOL, CURLINFO_PROXY_SSL_VERIFYRESULT, CURLINFO_SCHEME, CURLINFO_SIZE_DOWNLOAD_T, CURLINFO_SIZE_UPLOAD_T, CURLINFO_SPEED_DOWNLOAD_T, CURLINFO_SPEED_UPLOAD_T, CURLINFO_APPCONNECT_TIME_T, CURLINFO_CONNECT_TIME_T, CURLINFO_FILETIME_T, CURLINFO_NAMELOOKUP_TIME_T, CURLINFO_PRETRANSFER_TIME_T, CURLINFO_REDIRECT_TIME_T, CURLINFO_STARTTRANSFER_TIME_T, CURLINFO_TOTAL_TIME_T.

Examples

Example #1 curl_getinfo() example

```php
<?php
// Create a cURL handle
$ch = curl_init('http://www.example.com/');

// Execute
curl_exec($ch);

// Check if any error occurred
if (!curl_errno($ch)) {
  $info = curl_getinfo($ch);
  echo 'Took ', $info['total_time'], ' seconds to send a request to ', $info['url'], "\n";
}
?>
```

Example #2 curl_getinfo() example with option parameter

```php
<?php
// Create a cURL handle
$ch = curl_init('http://www.example.com/');

// Execute
curl_exec($ch);

// Check HTTP status code
if (!curl_errno($ch)) {
  switch ($http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE)) {
    case 200:  # OK
      break;
    default:
      echo 'Unexpected HTTP code: ', $http_code, "\n";
  }
}
?>
```

Notes

> **Note** :
>
> Information gathered by this function is kept if the handle is re-used. This means that unless a statistic is overridden internally by this function, the previous info is returned.

curl_init

curl_init

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_init — Initialize a cURL session

Description

function **curl_init**(?string `$url` = `null`): CurlHandle|false

Initializes a new session and returns a cURL handle.

Parameters

`url`

If provided, the `CURLOPT_URL` option will be set to its value. This can be set manually using the curl_setopt() function.

> **Note** :
>
> The `file` protocol is disabled by cURL if open_basedir is set.

Return Values

Returns a cURL handle on success, `false` on errors.

Changelog

Version	Description
8.0.0	On success, this function returns a CurlHandle instance now; previously, a resource was returned.
8.0.0	`url` is nullable now.

Examples

Example #1 Initializing a new cURL session and fetching a web page

```php
<?php

// Initializes a new cURL session
$ch = curl_init();

// Set URL and other appropriate options
curl_setopt($ch, CURLOPT_URL, "http://www.example.com/");
curl_setopt($ch, CURLOPT_HEADER, 0);

// Grab URL and pass it to the browser
curl_exec($ch);

?>
```

See Also

- curl_multi_init() - Returns a new cURL multi handle

curl_multi_add_handle

curl_multi_add_handle

(PHP 5, PHP 7, PHP 8)

curl_multi_add_handle — Add a normal cURL handle to a cURL multi handle

Description

function **curl_multi_add_handle**(CurlMultiHandle `$multi_handle`, CurlHandle `$handle`): int

Adds the `handle` handle to the multi handle `multi_handle`

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

`handle`

A cURL handle returned by curl_init().

Return Values

Returns 0 on success, or one of the **CURLM_*** errors code.

Changelog

Version	Description
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

See Also

- curl_multi_remove_handle() - Remove a handle from a set of cURL handles
- curl_multi_init() - Returns a new cURL multi handle
- curl_init() - Initialize a cURL session

curl_multi_close

curl_multi_close

(PHP 5, PHP 7, PHP 8)

curl_multi_close — Remove all cURL handles from a multi handle

Description

function **curl_multi_close**(CurlMultiHandle `$multi_handle`): void

Removes all CurlHandles attached to the CurlMultiHandle, as if curl_multi_remove_handle() was called for each of them.

Prior to PHP 8.0.0 this function also closed the cURL multi handle resource, making it unusable.

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

Return Values

No value is returned.

Changelog

Version	Description
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.

See Also

- curl_multi_init() - Returns a new cURL multi handle

curl_multi_errno

curl_multi_errno

(PHP 7 >= 7.1.0, PHP 8)

curl_multi_errno — Return the last multi curl error number

Description

function **curl_multi_errno**(CurlMultiHandle `$multi_handle`): int

Return an integer containing the last multi curl error number.

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

Return Values

Return an integer containing the last multi curl error number.

Changelog

Version	Description
8.0.0	The function no longer returns `false` on failure.
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.

See Also

- curl_errno() - Return the last error number

curl_multi_exec

curl_multi_exec

(PHP 5, PHP 7, PHP 8)

curl_multi_exec — Processes each of the handles in the stack

Description

function **curl_multi_exec**(CurlMultiHandle `$multi_handle`, int `&$still_running`): int

Processes each of the handles in the stack. This method can be called whether or not a handle needs to read or write data.

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

`still_running`

A reference to a flag to tell whether the operations are still running.

Return Values

A cURL code defined in the cURL Predefined Constants.

> **Note** :
>
> This only returns errors regarding the whole multi stack. There might still have occurred problems on individual transfers even when this function returns `CURLM_OK`.

Changelog

Version	Description
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_multi_exec() example

This example will create curl handles for a list of URLs, add them to a multi handle, and process them asynchronously.

```php
<?php

$urls = [
    "https://www.php.net/",
    "https://www.example.com/",
];
```

```php
$mh = curl_multi_init();
$map = new WeakMap();

foreach ($urls as $url) {
    $ch = curl_init($url);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
    curl_multi_add_handle($mh, $ch);
    $map[$ch] = $url;
}

do {
    $status = curl_multi_exec($mh, $unfinishedHandles);
    if ($status !== CURLM_OK) {
        throw new \Exception(curl_multi_strerror(curl_multi_errno($mh)));
    }

    while (($info = curl_multi_info_read($mh)) !== false) {
        if ($info['msg'] === CURLMSG_DONE) {
            $handle = $info['handle'];
            curl_multi_remove_handle($mh, $handle);
            $url = $map[$handle];

            if ($info['result'] === CURLE_OK) {
                $statusCode = curl_getinfo($handle, CURLINFO_HTTP_CODE);

                echo "Request to {$url} finished with HTTP status {$statusCode}:", PHP_EOL;
                echo curl_multi_getcontent($handle);
                echo PHP_EOL, PHP_EOL;
            } else {
                echo "Request to {$url} failed with error: ", PHP_EOL;
                echo curl_strerror($info['result']);
                echo PHP_EOL, PHP_EOL;
            }
        }
    }

    if ($unfinishedHandles) {
        if (($updatedHandles = curl_multi_select($mh)) === -1) {
            throw new \Exception(curl_multi_strerror(curl_multi_errno($mh)));
        }
    }
} while ($unfinishedHandles);

curl_multi_close($mh);

?>
```

See Also

- curl_multi_init() - Returns a new cURL multi handle
- curl_multi_select() - Wait until reading or writing is possible for any cURL multi handle connection
- curl_exec() - Perform a cURL session

curl_multi_getcontent

curl_multi_getcontent

(PHP 5, PHP 7, PHP 8)

curl_multi_getcontent — Return the content of a cURL handle if `CURLOPT_RETURNTRANSFER` is set

Description

function **curl_multi_getcontent**(CurlHandle `$handle`): ?string

If `CURLOPT_RETURNTRANSFER` is an option that is set for a specific handle, then this function will return the content of that cURL handle in the form of a string.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

Return the content of a cURL handle if `CURLOPT_RETURNTRANSFER` is set or `null` if not set.

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

See Also

- curl_multi_init() - Returns a new cURL multi handle

curl_multi_info_read

curl_multi_info_read

(PHP 5, PHP 7, PHP 8)

curl_multi_info_read — Get information about the current transfers

Description

function **curl_multi_info_read**(CurlMultiHandle `$multi_handle`, int `&$queued_messages = null`): array|false

Ask the multi handle if there are any messages or information from the individual transfers. Messages may include information such as an error code from the transfer or just the fact that a transfer is completed.

Repeated calls to this function will return a new result each time, until a `false` is returned as a signal that there is no more to get at this point. The integer pointed to with `queued_messages` will contain the number of remaining messages after this function was called.

> **Warning**
>
> The data the returned resource points to will not survive calling curl_multi_remove_handle().

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

`queued_messages`

Number of messages that are still in the queue

Return Values

On success, returns an associative array for the message, `false` on failure.

Contents of the returned array

Key:	Value:
`msg`	The `CURLMSG_DONE` constant. Other return values are currently not available.
`result`	One of the `CURLE_*` constants. If everything is OK, the `CURLE_OK` will be the result.
`handle`	Resource of type curl indicates the handle which it concerns.

Changelog

Version	Description
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.

See Also

- curl_multi_init() - Returns a new cURL multi handle

curl_multi_init

curl_multi_init

(PHP 5, PHP 7, PHP 8)

curl_multi_init — Returns a new cURL multi handle

Description

function **curl_multi_init**(): CurlMultiHandle

Allows the processing of multiple cURL handles asynchronously.

Parameters

This function has no parameters.

Return Values

Returns a cURL multi handle.

Changelog

Version	Description
8.0.0	On success, this function returns a CurlMultiHandle instance now; previously, a resource was returned.

See Also

- curl_init() - Initialize a cURL session
- curl_multi_close() - Remove all cURL handles from a multi handle

curl_multi_remove_handle

curl_multi_remove_handle

(PHP 5, PHP 7, PHP 8)

curl_multi_remove_handle — Remove a handle from a set of cURL handles

Description

function **curl_multi_remove_handle**(CurlMultiHandle `$multi_handle`, CurlHandle `$handle`): int

Removes a given `handle` from the given `multi_handle`. When the `handle` has been removed, it is again perfectly legal to run curl_exec() on this handle. Removing the `handle` while being used, will effectively halt the transfer in progress involving that handle.

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

`handle`

A cURL handle returned by curl_init().

Return Values

Returns 0 on success, or one of the **`CURLM_*`** error codes.

Changelog

Version	Description
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

See Also

- curl_init() - Initialize a cURL session

- curl_multi_init() - Returns a new cURL multi handle
- curl_multi_add_handle() - Add a normal cURL handle to a cURL multi handle

curl_multi_select

curl_multi_select

(PHP 5, PHP 7, PHP 8)

curl_multi_select — Wait until reading or writing is possible for any cURL multi handle connection

Description

function **curl_multi_select**(CurlMultiHandle `$multi_handle`, float `$timeout` = 1.0): int

Blocks the execution of the script until a cURL handle attached to the cURL multi handle would be able to make progress on the next call to curl_multi_exec() or until the timeout strikes (whichever comes first).

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

`timeout`

Time, in seconds, to wait for a response from the active cURL multi handle connections.

Return Values

On success, returns the number of active descriptors contained in the descriptor sets. This may be `0` if there was no activity on any of the descriptors. On failure, this function will return `-1` on a select failure (from the underlying `select()` system call).

Errors/Exceptions

Throws a ValueError if `timeout` is less than `0` or greater than `PHP_INT_MAX`.

Changelog

Version	Description
8.4.0	Now throws a ValueError if `timeout` is less than `0` or greater than `PHP_INT_MAX`.
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.

See Also

- curl_multi_init() - Returns a new cURL multi handle

curl_multi_setopt

curl_multi_setopt

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_multi_setopt — Set a cURL multi option

Description

function **curl_multi_setopt**(CurlMultiHandle `$multi_handle`, int `$option`, mixed `$value`): bool

Sets an option on the given cURL multi handle.

Parameters

`multi_handle`

A cURL multi handle returned by curl_multi_init().

`option`

One of the `CURLMOPT_*` constants.

`value`

The value to be set on `option`. See the description of the `CURLMOPT_*` constants for details on the type of values each constant expects.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.2.0	Introduced `CURLMOPT_MAX_CONCURRENT_STREAMS`.
8.0.0	`multi_handle` expects a CurlMultiHandle instance now; previously, a resource was expected.
7.1.0	Introduced `CURLMOPT_PUSHFUNCTION`.
7.0.7	Introduced `CURLMOPT_CHUNK_LENGTH_PENALTY_SIZE`, `CURLMOPT_CONTENT_LENGTH_PENALTY_SIZE`, `CURLMOPT_MAX_HOST_CONNECTIONS`, `CURLMOPT_MAX_PIPELINE_LENGTH` and `CURLMOPT_MAX_TOTAL_CONNECTIONS`.

curl_multi_strerror

curl_multi_strerror

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_multi_strerror — Return string describing error code

Description

function **curl_multi_strerror**(int `$error_code`): ?string

Returns a text error message describing the given `CURLM_*` error code.

Parameters

`error_code`

One of the `CURLM_*` constants.

Return Values

Returns error string for valid error code, `null` otherwise.

See Also

- curl_strerror() - Return string describing the given error code
- » cURL error codes

curl_pause

curl_pause

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_pause — Pause and unpause a connection

Description

function **curl_pause**(CurlHandle `$handle`, int `$flags`): int

Pause or unpause a cURL session. A session can be paused while a transfer is in progress, in either the read, write or both directions, by calling this function from a callback registered with curl_setopt().

Parameters

`handle`

A cURL handle returned by curl_init().

`flags`

One of `CURLPAUSE_*` constants.

Return Values

Returns an error code (`CURLE_OK` for no error).

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

curl_reset

curl_reset

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_reset — Reset all options of a libcurl session handle

Description

function **curl_reset**(CurlHandle `$handle`): void

This function re-initializes all options set on the given cURL handle to the default values.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

No value is returned.

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_reset() example

```php
<?php
// Create a curl handle
$ch = curl_init();

// Set CURLOPT_USERAGENT option
curl_setopt($ch, CURLOPT_USERAGENT, "My test user-agent");

// Reset all previously set options
curl_reset($ch);

// Send HTTP request
curl_setopt($ch, CURLOPT_URL, 'http://example.com/');
curl_exec($ch); // the previously set user-agent will be not sent, it has been reset by curl_reset
?>
```

Notes

> **Note** :
>
> **curl_reset()** also resets the URL given as the curl_init() parameter.

See Also

- curl_setopt() - Set an option for a cURL transfer

curl_setopt

curl_setopt

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_setopt — Set an option for a cURL transfer

Description

function **curl_setopt**(CurlHandle `$handle`, int `$option`, mixed `$value`): bool

Sets an option on the given cURL session handle.

Parameters

`handle`

A cURL handle returned by curl_init().

`option`

The `CURLOPT_*` option to set.

`value`

The value to be set on `option`. See the description of the `CURLOPT_*` constants for details on the type of values each constant expects.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.4.0	`CURLOPT_DNS_USE_GLOBAL_CACHE` no longer has any effect, and enabling it on thread-safe PHP builds no longer triggers a warning.
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.
7.3.15, 7.4.3	Introduced `CURLOPT_HTTP09_ALLOWED`.
7.3.0	Introduced `CURLOPT_ABSTRACT_UNIX_SOCKET`, `CURLOPT_KEEP_SENDING_ON_ERROR`, `CURLOPT_PRE_PROXY`, `CURLOPT_PROXY_CAINFO`, `CURLOPT_PROXY_CAPATH`, `CURLOPT_PROXY_CRLFILE`, `CURLOPT_PROXY_KEYPASSWD`, `CURLOPT_PROXY_PINNEDPUBLICKEY`, `CURLOPT_PROXY_SSLCERT`, `CURLOPT_PROXY_SSLCERTTYPE`, `CURLOPT_PROXY_SSL_CIPHER_LIST`, `CURLOPT_PROXY_SSLKEY`, `CURLOPT_PROXY_SSLKEYTYPE`, `CURLOPT_PROXY_SSL_OPTIONS`, `CURLOPT_PROXY_SSL_VERIFYHOST`, `CURLOPT_PROXY_SSL_VERIFYPEER`, `CURLOPT_PROXY_SSLVERSION`, `CURLOPT_PROXY_TLSAUTH_PASSWORD`, `CURLOPT_PROXY_TLSAUTH_TYPE`, `CURLOPT_PROXY_TLSAUTH_USERNAME`, `CURLOPT_SOCKS5_AUTH`, `CURLOPT_SUPPRESS_CONNECT_HEADERS`, `CURLOPT_DISALLOW_USERNAME_IN_URL`, `CURLOPT_DNS_SHUFFLE_ADDRESSES`, `CURLOPT_HAPPY_EYEBALLS_TIMEOUT_MS`, `CURLOPT_HAPROXYPROTOCOL`, `CURLOPT_PROXY_TLS13_CIPHERS`, `CURLOPT_SSH_COMPRESSION`, `CURLOPT_TIMEVALUE_LARGE` and `CURLOPT_TLS13_CIPHERS`.
7.0.7	Introduced `CURL_HTTP_VERSION_2`, `CURL_HTTP_VERSION_2_PRIOR_KNOWLEDGE`, `CURL_HTTP_VERSION_2TLS`, `CURL_REDIR_POST_301`, `CURL_REDIR_POST_302`, `CURL_REDIR_POST_303`, `CURL_REDIR_POST_ALL`, `CURL_VERSION_KERBEROS5`, `CURL_VERSION_PSL`, `CURL_VERSION_UNIX_SOCKETS`, `CURLAUTH_NEGOTIATE`, `CURLAUTH_NTLM_WB`, `CURLFTP_CREATE_DIR`, `CURLFTP_CREATE_DIR_NONE`, `CURLFTP_CREATE_DIR_RETRY`, `CURLHEADER_SEPARATE`, `CURLHEADER_UNIFIED`, `CURLMOPT_CHUNK_LENGTH_PENALTY_SIZE`, `CURLMOPT_CONTENT_LENGTH_PENALTY_SIZE`, `CURLMOPT_MAX_HOST_CONNECTIONS`, `CURLMOPT_MAX_PIPELINE_LENGTH`, `CURLMOPT_MAX_TOTAL_CONNECTIONS`, `CURLOPT_CONNECT_TO`, `CURLOPT_DEFAULT_PROTOCOL`, `CURLOPT_DNS_INTERFACE`, `CURLOPT_DNS_LOCAL_IP4`, `CURLOPT_DNS_LOCAL_IP6`, `CURLOPT_EXPECT_100_TIMEOUT_MS`, `CURLOPT_HEADEROPT`, `CURLOPT_LOGIN_OPTIONS`, `CURLOPT_PATH_AS_IS`, `CURLOPT_PINNEDPUBLICKEY`, `CURLOPT_PIPEWAIT`, `CURLOPT_PROXY_SERVICE_NAME`, `CURLOPT_PROXYHEADER`, `CURLOPT_SASL_IR`, `CURLOPT_SERVICE_NAME`, `CURLOPT_SSL_ENABLE_ALPN`, `CURLOPT_SSL_ENABLE_NPN`, `CURLOPT_SSL_FALSESTART`, `CURLOPT_SSL_VERIFYSTATUS`, `CURLOPT_STREAM_WEIGHT`, `CURLOPT_TCP_FASTOPEN`, `CURLOPT_TFTP_NO_OPTIONS`, `CURLOPT_UNIX_SOCKET_PATH`, `CURLOPT_XOAUTH2_BEARER`, `CURLPROTO_SMB`, `CURLPROTO_SMBS`, `CURLPROXY_HTTP_1_0`, `CURLSSH_AUTH_AGENT` and `CURLSSLOPT_NO_REVOKE`.

Examples

Example #1 Initializing a new cURL session and fetching a web page

```php
<?php
// create a new cURL resource
$ch = curl_init();

// set URL and other appropriate options
curl_setopt($ch, CURLOPT_URL, "http://www.example.com/");
curl_setopt($ch, CURLOPT_HEADER, false);

// grab URL and pass it to the browser
curl_exec($ch);
?>
```

Notes

> **Note** :
>
> Passing an array to `CURLOPT_POSTFIELDS` will encode the data as *multipart/form-data*, while passing a URL-encoded string will encode the data as *application/x-www-form-urlencoded*.

See Also

- curl_setopt_array() - Set multiple options for a cURL transfer
- CURLFile
- CURLStringFile

curl_setopt_array

curl_setopt_array

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

curl_setopt_array — Set multiple options for a cURL transfer

Description

function **curl_setopt_array**(CurlHandle `$handle`, array `$options`): bool

Sets multiple options for a cURL session. This function is useful for setting a large number of cURL options without repetitively calling curl_setopt().

Parameters

`handle`

A cURL handle returned by curl_init().

`options`

An array specifying which options to set and their values. The keys should be valid curl_setopt() constants or their integer equivalents.

Return Values

Returns `true` if all options were successfully set. If an option could not be successfully set, `false` is immediately returned, ignoring any future options in the `options` array.

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 Initializing a new cURL session and fetching a web page

```php
<?php
// create a new cURL resource
$ch = curl_init();
```

```php
// set URL and other appropriate options
$options = array(CURLOPT_URL => 'http://www.example.com/',
                 CURLOPT_HEADER => false
                );

curl_setopt_array($ch, $options);

// grab URL and pass it to the browser
curl_exec($ch);
?>
```

Notes

> **Note** :
>
> As with curl_setopt(), passing an array to **CURLOPT_POST** will encode the data as *multipart/form-data*, while passing a URL-encoded string will encode the data as *application/x-www-form-urlencoded*.

See Also

- curl_setopt() - Set an option for a cURL transfer

curl_share_close

curl_share_close

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_share_close — Close a cURL share handle

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.5',
    message: "as it has no effect since PHP 8.0",
)]
function curl_share_close(CurlShareHandle $share_handle): void
```

> **Note** :
>
> This function has no effect. Prior to PHP 8.0.0, this function was used to close the resource.

Closes a cURL share handle and frees all resources.

Parameters

`share_handle`

A cURL share handle returned by curl_share_init().

Return Values

No value is returned.

Changelog

Version	Description
8.5.0	This function has been deprecated.
8.0.0	This function is a NOP now.

Version	Description
8.0.0	`share_handle` expects a CurlShareHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_share_setopt() example

This example will create a cURL share handle, add two cURL handles to it, and then run them with cookie data sharing.

```php
<?php
// Create cURL share handle and set it to share cookie data
$sh = curl_share_init();
curl_share_setopt($sh, CURLSHOPT_SHARE, CURL_LOCK_DATA_COOKIE);

// Initialize the first cURL handle and assign the share handle to it
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Execute the first cURL handle
curl_exec($ch1);

// Initialize the second cURL handle and assign the share handle to it
$ch2 = curl_init("http://php.net/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Execute the second cURL handle
//  all cookies from $ch1 handle are shared with $ch2 handle
curl_exec($ch2);

// Close the cURL share handle
curl_share_close($sh);
?>
```

See Also

- curl_share_init() - Initialize a cURL share handle

curl_share_errno

curl_share_errno

(PHP 7 >= 7.1.0, PHP 8)

curl_share_errno — Return the last share curl error number

Description

function **curl_share_errno**(CurlShareHandle `$share_handle`): int

Return an integer containing the last share curl error number.

Parameters

`share_handle`

A cURL share handle returned by curl_share_init().

Return Values

Returns an integer containing the last share curl error number.

Changelog

Version	Description
8.0.0	The function no longer returns `false` on failure.
8.0.0	`share_handle` expects a CurlShareHandle instance now; previously, a resource was expected.

See Also

- curl_errno() - Return the last error number

curl_share_init

curl_share_init

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_share_init — Initialize a cURL share handle

Description

function **curl_share_init**(): CurlShareHandle

Allows to share data between cURL handles.

Parameters

This function has no parameters.

Return Values

Returns a cURL share handle.

Changelog

Version	Description
8.0.0	This function returns a CurlShareHandle instance now; previously, a resource was returned.

Examples

Example #1 curl_share_init() example

This example will create a cURL share handle, add two cURL handles to it, and then run them with cookie data sharing.

```php
<?php
// Create cURL share handle and set it to share cookie data
$sh = curl_share_init();
curl_share_setopt($sh, CURLSHOPT_SHARE, CURL_LOCK_DATA_COOKIE);

// Initialize the first cURL handle and assign the share handle to it
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Execute the first cURL handle
curl_exec($ch1);

// Initialize the second cURL handle and assign the share handle to it
$ch2 = curl_init("http://php.net/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Execute the second cURL handle
//  all cookies from $ch1 handle are shared with $ch2 handle
curl_exec($ch2);
?>
```

See Also

- curl_share_setopt() - Set an option for a cURL share handle
- curl_share_init_persistent() - Initialize a persistent cURL share handle

curl_share_init_persistent

curl_share_init_persistent

(PHP 8 >= 8.5.0)

curl_share_init_persistent — Initialize a **persistent** cURL share handle

Description

function **curl_share_init_persistent**(array `$share_options`): CurlSharePersistentHandle

Initialize a **persistent** cURL share handle with the given share options. Unlike curl_share_init(), handles created by this function will not be destroyed at the end of the PHP request. If a persistent share handle with the same set of `share_options` is found, it will be reused.

Parameters

`share_options`

A non-empty array of **`CURL_LOCK_DATA_*`** constants.

> **Note** :
>
> **`CURL_LOCK_DATA_COOKIE`** is not allowed and, if specified, this function will throw a ValueError. Sharing cookies between PHP requests may lead to inadvertently mixing up sensitive cookies between users.

Return Values

Returns a CurlSharePersistentHandle.

Errors/Exceptions

- If `share_options` is empty, this function throws a ValueError.
- If `share_options` contains a value not matching a **`CURL_LOCK_DATA_*`**, this function throws a ValueError.
- If `share_options` contains **`CURL_LOCK_DATA_COOKIE`**, this function throws a ValueError.
- If `share_options` contains a non-integer value, this function throws a TypeError.

Examples

Example #1 curl_share_init_persistent() example

This example will create a persistent cURL share handle and demonstrate sharing connections between them. If this is executed in a long-lived PHP SAPI, `$sh` will survive between SAPI requests.

```php
<?php

// Create or retrieve a persistent cURL share handle set to share DNS lookups and connections
$sh = curl_share_init_persistent([CURL_LOCK_DATA_DNS, CURL_LOCK_DATA_CONNECT]);

// Initialize the first cURL handle and assign the share handle to it
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Execute the first cURL handle. This may reuse the connection from an earlier SAPI request
curl_exec($ch1);

// Initialize the second cURL handle and assign the share handle to it
$ch2 = curl_init("http://example.com/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Execute the second cURL handle. This will reuse the connection from $ch1
curl_exec($ch2);

?>
```

See Also

- curl_setopt() - Set an option for a cURL transfer
- curl_share_init() - Initialize a cURL share handle

curl_share_setopt

curl_share_setopt

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_share_setopt — Set an option for a cURL share handle

Description

function **curl_share_setopt**(CurlShareHandle `$share_handle`, int `$option`, mixed `$value`): bool

Sets an option on the given cURL share handle.

Parameters

`share_handle`

A cURL share handle returned by curl_share_init().

`option`

One of the `CURLSHOPT_*` constants.

`value`

One of the `CURL_LOCK_DATA_*` constants.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.0.0	`share_handle` expects a CurlShareHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_share_setopt() example

This example will create a cURL share handle, add two cURL handles to it, and then run them with cookie data sharing.

```php
<?php
// Create cURL share handle and set it to share cookie data
$sh = curl_share_init();
curl_share_setopt($sh, CURLSHOPT_SHARE, CURL_LOCK_DATA_COOKIE);

// Initialize the first cURL handle and assign the share handle to it
$ch1 = curl_init("http://example.com/");
curl_setopt($ch1, CURLOPT_SHARE, $sh);

// Execute the first cURL handle
curl_exec($ch1);

// Initialize the second cURL handle and assign the share handle to it
$ch2 = curl_init("http://php.net/");
curl_setopt($ch2, CURLOPT_SHARE, $sh);

// Execute the second cURL handle
//  all cookies from $ch1 handle are shared with $ch2 handle
curl_exec($ch2);
?>
```

curl_share_strerror

curl_share_strerror

(PHP 7 >= 7.1.0, PHP 8)

curl_share_strerror — Return string describing the given error code

Description

function **curl_share_strerror**(int `$error_code`): ?string

Returns a text error message describing the given error code.

Parameters

`error_code`

One of the » cURL error codes constants.

Return Values

Returns error description or `null` for invalid error code.

See Also

- curl_share_errno() - Return the last share curl error number
- curl_strerror() - Return string describing the given error code

curl_strerror

curl_strerror

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_strerror — Return string describing the given error code

Description

function **curl_strerror**(int `$error_code`): ?string

Returns a text error message describing the given error code.

Parameters

`error_code`

One of the » cURL error codes constants.

Return Values

Returns error description or `null` for invalid error code.

Examples

Example #1 curl_errno() example

```php
<?php
// Create a curl handle with a misspelled protocol in URL
$ch = curl_init("http://example.com/");

// Send request
curl_exec($ch);

// Check for errors and display the error message
if($errno = curl_errno($ch)) {
    $error_message = curl_strerror($errno);
    echo "cURL error ({$errno}):\n {$error_message}";
}
?>
```

The above example will output:

```
cURL error (1):
 Unsupported protocol
```

See Also

- curl_errno() - Return the last error number

- curl_error() - Return a string containing the last error for the current session
- » Curl error codes

curl_unescape

curl_unescape

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

curl_unescape — Decodes the given URL encoded string

Description

function **curl_unescape**(CurlHandle `$handle`, string `$string`): string|false

This function decodes the given URL encoded string.

Parameters

`handle`

A cURL handle returned by curl_init().

`string`

The URL encoded string to be decoded.

Return Values

Returns decoded string or `false` on failure.

Changelog

Version	Description
8.0.0	`handle` expects a CurlHandle instance now; previously, a resource was expected.

Examples

Example #1 curl_escape() example

```php
<?php
// Create a curl handle
$ch = curl_init('http://example.com/redirect.php');

// Send HTTP request and follow redirections
curl_setopt($ch, CURLOPT_FOLLOWLOCATION, 1);
curl_exec($ch);

// Get the last effective URL
$effective_url = curl_getinfo($ch, CURLINFO_EFFECTIVE_URL);
// ie. "http://example.com/show_location.php?loc=M%C3%BCnchen"

// Decode the URL
$effective_url_decoded = curl_unescape($ch, $effective_url);
// "http://example.com/show_location.php?loc=München"
?>
```

Notes

> **Note** :
>
> **curl_unescape()** does not decode plus symbols (+) into spaces. urldecode() does.

See Also

- curl_escape() - URL encodes the given string
- urlencode() - URL-encodes string
- urldecode() - Decodes URL-encoded string

- rawurlencode() - URL-encode according to RFC 3986
- rawurldecode() - Decode URL-encoded strings

curl_upkeep

curl_upkeep

(PHP 8 >= 8.2.0)

curl_upkeep — Performs any connection upkeep checks

Description

function **curl_upkeep**(CurlHandle `$handle`): bool

Available if built against libcurl >= 7.62.0.

Some protocols have "connection upkeep" mechanisms. These mechanisms usually send some traffic on existing connections in order to keep them alive; this can prevent connections from being closed due to overzealous firewalls, for example.

Connection upkeep is currently available only for HTTP/2 connections. A small amount of traffic is usually sent to keep a connection alive. HTTP/2 maintains its connection by sending a HTTP/2 PING frame.

Parameters

`handle`

A cURL handle returned by curl_init().

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 curl_upkeep() example

```php
<?php
$url = "https://example.com";

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_HTTP_VERSION,CURL_HTTP_VERSION_2_0);
curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
curl_setopt($ch, CURLOPT_UPKEEP_INTERVAL_MS, 200);
if (curl_exec($ch)) {
    usleep(300);
    var_dump(curl_upkeep($ch));
}
?>
```

See Also

- curl_init() - Initialize a cURL session

curl_version

curl_version

(PHP 4 >= 4.0.2, PHP 5, PHP 7, PHP 8)

curl_version — Gets cURL version information

Description

function **curl_version**(): array|false

Returns information about the cURL version.

Parameters

This function has no parameters.

Return Values

Returns an associative array with the following elements:

Key	Value description
version_number	cURL 24 bit version number
version	cURL version number, as a string
ssl_version_number	OpenSSL 24 bit version number
ssl_version	OpenSSL version number, as a string
libz_version	zlib version number, as a string
host	Information about the host where cURL was built
age	
features	A bitmask of the `CURL_VERSION_*` constants
protocols	An array of protocols names supported by cURL
feature_list	An associative array of all known cURL features, and whether they are supported (`true`) or not (`false`)

Changelog

Version	Description
8.4.0	`features_list` added.
8.0.0	The optional `age` parameter has been removed.
7.4.0	The optional `age` parameter has been deprecated; if a value is passed, it is ignored.

Examples

Example #1 curl_version() example

This example will check which features that's available in cURL build by using the `'features'` bitmask returned by **curl_version()**.

```php
<?php
// Get curl version array
$version = curl_version();

// These are the bitfields that can be used
// to check for features in the curl build
$bitfields = Array(
            'CURL_VERSION_IPV6',
            'CURL_VERSION_KERBEROS4',
            'CURL_VERSION_SSL',
            'CURL_VERSION_LIBZ'
            );


foreach($bitfields as $feature)
{
    echo $feature . ($version['features'] & constant($feature) ? ' matches' : ' does not match');
    echo PHP_EOL;
}
?>
```

CurlHandle

(PHP 8)

Introduction

A fully opaque class which replaces `curl` resources as of PHP 8.0.0.

Class synopsis

final class **CurlHandle** {

}

CurlMultiHandle

(PHP 8)

Introduction

A fully opaque class which replaces `curl_multi` resources as of PHP 8.0.0.

Class synopsis

final class **CurlMultiHandle** {
}

CurlShareHandle

(PHP 8)

Introduction

A fully opaque class which replaces `curl_share` resources as of PHP 8.0.0.

Class synopsis

final class **CurlShareHandle** {
}

CurlSharePersistentHandle

(PHP 8 >= 8.5.0)

Introduction

Represents a persistent cURL "share" handle.

Class synopsis

final class **CurlSharePersistentHandle** {
/* Properties */
public readonly array *$options*;
}

Properties

options

The `CURL_LOCK_DATA_*` caches shared between requests using this handle.

CURLFile

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introduction

This class or CURLStringFile should be used to upload a file with `CURLOPT_POSTFIELDS`.

Unserialization of **CURLFile** instances is not allowed. As of PHP 7.4.0, serialization is forbidden in the first place.

Class synopsis

class **CURLFile** {
/* Properties */
public string *$name* = "";
public string *$mime* = "";
public string *$postname* = "";
/* Methods */
public function __construct(string `$filename`, ?string `$mime_type` = `null`, ?string `$posted_filename` = `null`)

public function getFilename(): string
public function getMimeType(): string
public function getPostFilename(): string
public function setMimeType(string $mime_type): void
public function setPostFilename(string $posted_filename): void
}

Properties

name

Name of the file to be uploaded.

mime

MIME type of the file (default is `application/octet-stream`).

postname

The name of the file in the upload data (defaults to the *name* property).

See Also

- curl_setopt()
- CURLStringFile

Table of Contents

CURLFile::__construct

CURLFile::__construct

curl_file_create

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::__construct -- curl_file_create — Create a CURLFile object

Description

Object-oriented style

public function **CURLFile::__construct**(string $filename, ?string $mime_type = **null**, ?string $posted_filename = **null**)

Procedural style

function **curl_file_create**(string $filename, ?string $mime_type = **null**, ?string $posted_filename = **null**): CURLFile

Creates a CURLFile object, used to upload a file with **CURLOPT_POSTFIELDS**.

Parameters

`filename`

Path to the file which will be uploaded.

`mime_type`

Mimetype of the file.

`posted_filename`

Name of the file to be used in the upload data.

Return Values

Returns a CURLFile object.

Changelog

Version	Description
8.0.0	`mime_type` and `posted_filename` are nullable now; previously their default was `0`.

Examples

Example #1 CURLFile::__construct() example

Object-oriented style

```
<?php
/* http://example.com/upload.php:
<?php var_dump($_FILES); ?>
*/

// Create a cURL handle
$ch = curl_init('http://example.com/upload.php');

// Create a CURLFile object
$cfile = new CURLFile('cats.jpg','image/jpeg','test_name');

// Assign POST data
$data = array('test_file' => $cfile);
curl_setopt($ch, CURLOPT_POST,1);
curl_setopt($ch, CURLOPT_POSTFIELDS, $data);

// Execute the handle
curl_exec($ch);
?>
```

Procedural style

```
<?php
/* http://example.com/upload.php:
<?php var_dump($_FILES); ?>
*/

// Create a cURL handle
$ch = curl_init('http://example.com/upload.php');

// Create a CURLFile object
$cfile = curl_file_create('cats.jpg','image/jpeg','test_name');

// Assign POST data
$data = array('test_file' => $cfile);
curl_setopt($ch, CURLOPT_POST,1);
curl_setopt($ch, CURLOPT_POSTFIELDS, $data);

// Execute the handle
curl_exec($ch);
?>
```

The above example will output:

```
array(1) {
  ["test_file"]=>
  array(5) {
    ["name"]=>
    string(9) "test_name"
    ["type"]=>
    string(10) "image/jpeg"
    ["tmp_name"]=>
    string(14) "/tmp/phpPC9Kbx"
    ["error"]=>
    int(0)
    ["size"]=>
    int(46334)
```

```
    }
}
```

Example #2 CURLFile::__construct() uploading multiple files example

Object-oriented style

```php
<?php
$request = curl_init('http://www.example.com/upload.php');
curl_setopt($request, CURLOPT_POST, true);
curl_setopt($request, CURLOPT_SAFE_UPLOAD, true);
curl_setopt($request, CURLOPT_POSTFIELDS, [
    'blob[0]' => new CURLFile(realpath('first-file.jpg'), 'image/jpeg'),
    'blob[1]' => new CURLFile(realpath('second-file.txt'), 'text/plain'),
    'blob[2]' => new CURLFile(realpath('third-file.exe'), 'application/octet-stream'),
]);
curl_setopt($request, CURLOPT_RETURNTRANSFER, true);

echo curl_exec($request);

var_dump(curl_getinfo($request));
```

Procedural style

```php
<?php
// procedural
$request = curl_init('http://www.example.com/upload.php');
curl_setopt($request, CURLOPT_POST, true);
curl_setopt($request, CURLOPT_SAFE_UPLOAD, true);
curl_setopt($request, CURLOPT_POSTFIELDS, [
    'blob[0]' => curl_file_create(realpath('first-file.jpg'), 'image/jpeg'),
    'blob[1]' => curl_file_create(realpath('second-file.txt'), 'text/plain'),
    'blob[2]' => curl_file_create(realpath('third-file.exe'), 'application/octet-stream'),
]);
curl_setopt($request, CURLOPT_RETURNTRANSFER, true);

echo curl_exec($request);

var_dump(curl_getinfo($request));
```

The above example will output:

```
array(26) {
  ["url"]=>
  string(31) "http://www.example.com/upload.php"
  ["content_type"]=>
  string(24) "text/html; charset=UTF-8"
  ["http_code"]=>
  int(200)
  ["header_size"]=>
  int(198)
  ["request_size"]=>
  int(196)
  ["filetime"]=>
  int(-1)
  ["ssl_verify_result"]=>
  int(0)
  ["redirect_count"]=>
  int(0)
  ["total_time"]=>
  float(0.060062)
  ["namelookup_time"]=>
  float(0.028575)
  ["connect_time"]=>
  float(0.029011)
```

```
  ["pretransfer_time"]=>
  float(0.029121)
  ["size_upload"]=>
  float(3230730)
  ["size_download"]=>
  float(811)
  ["speed_download"]=>
  float(13516)
  ["speed_upload"]=>
  float(53845500)
  ["download_content_length"]=>
  float(811)
  ["upload_content_length"]=>
  float(3230730)
  ["starttransfer_time"]=>
  float(0.030355)
  ["redirect_time"]=>
  float(0)
  ["redirect_url"]=>
  string(0) ""
  ["primary_ip"]=>
  string(13) "0.0.0.0"
  ["certinfo"]=>
  array(0) {
  }
  ["primary_port"]=>
  int(80)
  ["local_ip"]=>
  string(12) "0.0.0.0"
  ["local_port"]=>
  int(34856)
}
```

See Also

- curl_setopt() - Set an option for a cURL transfer

CURLFile::getFilename

CURLFile::getFilename

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::getFilename — Get file name

Description

public function **CURLFile::getFilename**(): string

Parameters

This function has no parameters.

Return Values

Returns file name.

CURLFile::getMimeType

CURLFile::getMimeType

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::getMimeType — Get MIME type

Description

public function **CURLFile::getMimeType**(): string

Parameters

This function has no parameters.

Return Values

Returns MIME type.

CURLFile::getPostFilename

CURLFile::getPostFilename

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::getPostFilename — Get file name for POST

Description

public function **CURLFile::getPostFilename**(): string

Parameters

This function has no parameters.

Return Values

Returns file name for POST.

CURLFile::setMimeType

CURLFile::setMimeType

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::setMimeType — Set MIME type

Description

public function **CURLFile::setMimeType**(string `$mime_type`): void

Parameters

`mime_type`

MIME type to be used in POST data.

Return Values

No value is returned.

CURLFile::setPostFilename

CURLFile::setPostFilename

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

CURLFile::setPostFilename — Set file name for POST

Description

public function **CURLFile::setPostFilename**(string `$posted_filename`): void

Parameters

`posted_filename`

Filename to be used in POST data.

Return Values

No value is returned.

CURLStringFile

(PHP 8 >= 8.1.0)

Introduction

CURLStringFile makes it possible to upload a file directly from a variable. This is similar to CURLFile, but works with the contents of the file, not filename. This class or CURLFile should be used to upload the contents of the file with `CURLOPT_POSTFIELDS`.

Class synopsis

class **CURLStringFile** {
/* Properties */
public string *$data*;
public string *$postname*;
public string *$mime*;
/* Methods */
public function __construct(string `$data`, string `$postname`, string `$mime` = "application/octet-stream")
}

Properties

data

The contents to be uploaded.

postname

The name of the file to be used in the upload data.

mime

MIME type of the file (default is `application/octet-stream`).

See Also

- curl_setopt()
- CURLFile

Table of Contents

CURLStringFile::__construct

CURLStringFile::__construct

(PHP 8 >= 8.1.0)

CURLStringFile::__construct — Create a CURLStringFile object

Description

public function **CURLStringFile::__construct**(string `$data`, string `$postname`, string `$mime` = "application/octet-stream")

Creates a CURLStringFile object, used to upload a file with `CURLOPT_POSTFIELDS`.

Parameters

`data`

The contents to be uploaded.

`postname`

The name of the file to be used in the upload data.

`mime`

MIME type of the file (default is `application/octet-stream`).

Examples

Example #1 CURLStringFile::__construct() example

```php
<?php
/* http://example.com/upload.php:
<?php
var_dump($_FILES);
var_dump(file_get_contents($_FILES['test_string']['tmp_name']));
?>
*/

// Create a cURL handle
$ch = curl_init('http://example.com/upload.php');

// Create a CURLStringFile object
$cstringfile = new CURLStringFile('test upload contents','test.txt','text/plain');

// Assign POST data
$data = array('test_string' => $cstringfile);
curl_setopt($ch, CURLOPT_POST, 1);
curl_setopt($ch, CURLOPT_POSTFIELDS, $data);

// Execute the handle
curl_exec($ch);
?>
```

The above example will output:

```
array(1) {
  ["test_string"]=>
  array(5) {
    ["name"]=>
    string(8) "test.txt"
    ["type"]=>
    string(10) "text/plain"
    ["tmp_name"]=>
    string(14) "/tmp/phpTtaoCz"
    ["error"]=>
    int(0)
    ["size"]=>
    int(20)
  }
}
string(20) "test upload contents"
```

See Also

- curl_setopt() - Set an option for a cURL transfer

HASH Message Digest Framework

Introduction

This extension provides functions that can be used for direct or incremental processing of arbitrary length messages using a variety of hashing algorithms, including the generation of HMAC values and key derivations including HKDF and PBKDF2.

There are roughly three categories of hashing algorithms, and a complete list of algorithms can be found in the documentation for hash_algos().

- Checksum algorithms (such as `"crc32b"` or `"adler32"`): These are used to calculate checksums, useful in situations such as when transmission errors are to be detected. They are often very fast. These algorithms often generate values that are easily "guessable" or can be manipulated to create collisions, so they are entirely unsuitable for cryptographic use.
- Non-cryptographic algorithms (such as the xxHash family): These are often used to calculate hash values for hash tables, as they are designed to yield a good distribution over arbitrary string inputs. They are also generally fast, but also not suitable for cryptographic use.
- Cryptographic algorithms (such as the SHA-2 family): These are designed to yield hash values that are representative of their inputs but are not guessable nor prone to collisions. Performance is often a secondary concern, but modern hardware often supports special handling for these algorithms that PHP tries to use when available.

  The Computer Security Resource Center of the NIST has » explanation of the algorithms currently approved by United States Federal Information Processing Standards.

  > **Caution**
  >
  > Some of the early cryptographic algorithms, such as `"md4"`, `"md5"`, and `"sha1"`, have been proven to be prone to collision attacks and it is generally recommended to no longer use these for cryptographic applications.

See also the FAQ on Safe Password Hashing for information on best practices for using hash functions in handling passwords.

Installing/Configuring

Table of Contents

Installation

The Hash extension is a core PHP extension, so it is always enabled.

Prior to PHP 7.4.0, the Hash extension was bundled and compiled into PHP by default, but could be explicitly disabled using **--disable-hash**.

Prior to 5.1.2, the Hash extension was installed as a » PECL module.

Resource Types

This extension defines a Hashing Context resource returned by hash_init().

Predefined Constants

The constants below are defined by this extension, and will only be available when the extension has either been compiled into PHP or dynamically loaded at runtime.

`HASH_HMAC` (int)

Optional flag for hash_init(). Indicates that the HMAC digest-keying algorithm should be applied to the current hashing context.

HashContext

(PHP 7 >= 7.2.0, PHP 8)

Introduction

Class synopsis

final class **HashContext** {
/* Methods */
private function __construct()
public function __debugInfo(): array
public function __serialize(): array
public function __unserialize(array $data): void
}

Table of Contents

HashContext::__construct

HashContext::__construct

(PHP 7 >= 7.2.0, PHP 8)

HashContext::__construct — Private constructor to disallow direct instantiation

Description

private function **HashContext::__construct**()

Parameters

This function has no parameters.

HashContext::__debugInfo

HashContext::__debugInfo

(PHP 8 >= 8.4.0)

HashContext::__debugInfo — Returns debugging information about the hashing context

Description

public function **HashContext::__debugInfo**(): array

This method is not meant to be called directly; it is invoked by var_dump() and related functions when inspecting a HashContext instance.

Parameters

This function has no parameters.

Return Values

Returns an associative array of debugging information. It contains an `algo` key holding the name of the hashing algorithm in use by the context.

Examples

Example #1 **HashContext::__debugInfo()** example

```php
<?php
$ctx = hash_init('sha256');
var_dump($ctx);
?>
```

The above example will output:

```
object(HashContext)#1 (1) {
  ["algo"]=>
  string(6) "sha256"
}
```

See Also

- hash_init() - Initialize an incremental hashing context
- var_dump() - Dumps information about a variable

HashContext::__serialize

HashContext::__serialize

(PHP 8)

HashContext::__serialize — Serializes the HashContext object

Description

public function **HashContext::__serialize**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

HashContext::__unserialize

HashContext::__unserialize

(PHP 8)

HashContext::__unserialize — Deserializes the `data` parameter into a HashContext object

Description

public function **HashContext::__unserialize**(array `$data`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

The value being deserialized.

Return Values

No value is returned.

Hash Functions

Table of Contents

hash

hash

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash — Generate a hash value (message digest)

Description

```
function hash(
    string $algo,
    string $data,
    bool $binary = false,
    array $options = []
): string
```

Parameters

`algo`

Name of selected hashing algorithm (e.g. `"sha256"`). For a list of supported algorithms see hash_algos().

`data`

Message to be hashed.

`binary`

When set to `true`, outputs raw binary data. `false` outputs lowercase hexits.

`options`

An array of options for the various hashing algorithms. Currently, only the `"seed"` parameter is supported by the MurmurHash variants.

Return Values

Returns a string containing the calculated message digest as lowercase hexits unless `binary` is set to true in which case the raw binary representation of the message digest is returned.

Errors/Exceptions

Throws a ValueError exception if `algo` is unknown.

Changelog

Version	Description
8.1.0	The `options` parameter has been added.
8.0.0	Now throws a ValueError exception if `algo` is unknown; previously, `false` was returned and an `E_WARNING` message was emitted.

Examples

Example #1 A hash() example

```php
<?php
echo hash('sha256', 'The quick brown fox jumped over the lazy dog.');
?>
```

The above example will output:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

See Also

- hash_init() - Initialize an incremental hashing context
- hash_file() - Generate a hash value using the contents of a given file
- hash_hmac() - Generate a keyed hash value using the HMAC method

hash_algos

hash_algos

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_algos — Return a list of registered hashing algorithms

Description

function **hash_algos**(): array

Parameters

This function has no parameters.

Return Values

Returns a numerically indexed array containing the list of supported hashing algorithms.

> **Note** :
>
> Not all algorithms are suitable for every use case. See the Hash extension introduction for a description of the different categories (checksum, non-cryptographic, and cryptographic).

Changelog

Version	Description
8.1.0	Support for MurmurHash3 and xxHash algorithms has been added.
7.4.0	Support for crc32c has been added.
7.1.0	Support for sha512/224, sha512/256, sha3-224, sha3-256, sha3-384 and sha3-512 has been added.

Examples

Example #1 **hash_algos()** example

As of PHP 8.1.0, **hash_algos()** will return the following list of algorithm names.

```php
<?php
print_r(hash_algos());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => md2
    [1] => md4
    [2] => md5
    [3] => sha1
    [4] => sha224
    [5] => sha256
    [6] => sha384
    [7] => sha512/224
    [8] => sha512/256
    [9] => sha512
    [10] => sha3-224
    [11] => sha3-256
    [12] => sha3-384
    [13] => sha3-512
    [14] => ripemd128
    [15] => ripemd160
    [16] => ripemd256
    [17] => ripemd320
    [18] => whirlpool
```

```
    [19] => tiger128,3
    [20] => tiger160,3
    [21] => tiger192,3
    [22] => tiger128,4
    [23] => tiger160,4
    [24] => tiger192,4
    [25] => snefru
    [26] => snefru256
    [27] => gost
    [28] => gost-crypto
    [29] => adler32
    [30] => crc32
    [31] => crc32b
    [32] => crc32c
    [33] => fnv132
    [34] => fnv1a32
    [35] => fnv164
    [36] => fnv1a64
    [37] => joaat
    [38] => murmur3a
    [39] => murmur3c
    [40] => murmur3f
    [41] => xxh32
    [42] => xxh64
    [43] => xxh3
    [44] => xxh128
    [45] => haval128,3
    [46] => haval160,3
    [47] => haval192,3
    [48] => haval224,3
    [49] => haval256,3
    [50] => haval128,4
    [51] => haval160,4
    [52] => haval192,4
    [53] => haval224,4
    [54] => haval256,4
    [55] => haval128,5
    [56] => haval160,5
    [57] => haval192,5
    [58] => haval224,5
    [59] => haval256,5
)
```

See Also

- hash() - Generate a hash value (message digest)
- hash_hmac_algos() - Return a list of registered hashing algorithms suitable for hash_hmac

hash_copy

hash_copy

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

hash_copy — Copy hashing context

Description

function **hash_copy**(HashContext `$context`): HashContext

Parameters

`context`

Hashing context returned by hash_init().

Return Values

Returns a copy of Hashing Context.

Changelog

Version	Description
7.2.0	Accept and return HashContext instead of resource.

Examples

Example #1 hash_copy() example

```php
<?php
$context = hash_init("sha256");
hash_update($context, "The quick brown fox ");

/* copy context to be able to continue using it */
$copy_context = hash_copy($context);

echo hash_final($context), "\n";

hash_update($copy_context, "jumped over the lazy dog.");
echo hash_final($copy_context), "\n";
?>
```

The above example will output:

```
b29d66e56ed90cce9b0165c43fedec612b60a071974d8be4513e18580d55b5bd
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

hash_equals

hash_equals

(PHP 5 >= 5.6.0, PHP 7, PHP 8)

hash_equals — Timing attack safe string comparison

Description

function **hash_equals**(#[\SensitiveParameter] string `$known_string`, #[\SensitiveParameter] string `$user_string`): bool

Checks whether two strings are equal without leaking information about the contents of `known_string` via the execution time.

This function can be used to mitigate timing attacks. Performing a regular comparison with `===` will take more or less time to execute depending on whether the two values are different or not and at which position the first difference can be found, thus leaking information about the contents of the secret `known_string`.

> **Caution**
>
> It is important to provide the user-supplied string as the second parameter, rather than the first.

Parameters

`known_string`

The known string that must be kept secret.

`user_string`

The user-supplied string to compare against.

Return Values

Returns `true` when the two strings are equal, `false` otherwise.

Examples

Example #1 hash_equals() example

```php
<?php
$secretKey = '8uRhAeH89naXfFXKGOEj';

// Value and signature are provided by the user, e.g. within the URL
// and retrieved using $_GET.
$value = 'username=rasmuslerdorf';
$signature = '8c35009d3b50caf7f5d2c1e031842e6b7823a1bb781d33c5237cd27b57b5f327';

if (hash_equals(hash_hmac('sha256', $value, $secretKey), $signature)) {
    echo "The value is correctly signed.", PHP_EOL;
} else {
    echo "The value was tampered with.", PHP_EOL;
}
?>
```

The above example will output:

```
The value is correctly signed.
```

Notes

> **Note** :
>
> Both arguments must be of the same length to be compared successfully. When arguments of differing length are supplied, `false` is returned immediately and the length of the known string may be leaked in case of a timing attack.

See Also

- hash_hmac() - Generate a keyed hash value using the HMAC method

hash_file

hash_file

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_file — Generate a hash value using the contents of a given file

Description

function **hash_file**(
  string `$algo`,
  string `$filename`,
  bool `$binary = false`,
  array `$options = []`
): string|false

Parameters

`algo`

Name of selected hashing algorithm (e.g. `"sha256"`). For a list of supported algorithms see hash_algos().

`filename`

URL describing location of file to be hashed; Supports fopen() wrappers.

`binary`

When set to `true`, outputs raw binary data. `false` outputs lowercase hexits.

`options`

An array of options for the various hashing algorithms. Currently, only the `"seed"` parameter is supported by the MurmurHash variants.

Return Values

Returns a string containing the calculated message digest as lowercase hexits unless `binary` is set to true in which case the raw binary representation of the message digest is returned, or `false` on failure.

Changelog

Version	Description
8.1.0	The `options` parameter has been added.

Examples

Example #1 Using **hash_file()**

```php
<?php
/* Create a file to calculate hash of */
file_put_contents('example.txt', 'The quick brown fox jumped over the lazy dog.');

echo hash_file('sha256', 'example.txt');
?>
```

The above example will output:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

See Also

- hash_init() - Initialize an incremental hashing context
- hash_hmac_file() - Generate a keyed hash value using the HMAC method and the contents of a given file

hash_final

hash_final

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_final — Finalize an incremental hash and return resulting digest

Description

function **hash_final**(HashContext `$context`, bool `$binary` = `false`): string

Parameters

`context`

Hashing context returned by hash_init().

`binary`

When set to `true`, outputs raw binary data. `false` outputs lowercase hexits.

Return Values

Returns a string containing the calculated message digest as lowercase hexits unless `binary` is set to true in which case the raw binary representation of the message digest is returned.

Changelog

Version	Description
7.2.0	Accept HashContext instead of resource.

See Also

- hash_init() - Initialize an incremental hashing context
- hash_update() - Pump data into an active hashing context
- hash_update_stream() - Pump data into an active hashing context from an open stream
- hash_update_file() - Pump data into an active hashing context from a file

hash_hkdf

hash_hkdf

(PHP 7 >= 7.1.2, PHP 8)

hash_hkdf — Generate a HKDF key derivation of a supplied key input

Description

```
function hash_hkdf(
    string $algo,
    #[\SensitiveParameter] string $key,
    int $length = 0,
    string $info = "",
    string $salt = ""
): string
```

Parameters

`algo`

Name of selected hashing algorithm (e.g. `"sha256"`). For a list of supported algorithms see hash_hmac_algos().

> **Note** :
>
> Non-cryptographic hash functions are not allowed.

`key`

Input keying material (raw binary). Cannot be empty.

`length`

Desired output length in bytes. Cannot be greater than 255 times the chosen hash function size.

If `length` is `0`, the output length will default to the chosen hash function size.

`info`

Application/context-specific info string.

`salt`

Salt to use during derivation.

While optional, adding random salt significantly improves the strength of HKDF.

Return Values

Returns a string containing a raw binary representation of the derived key (also known as output keying material - OKM).

Errors/Exceptions

Throws a ValueError exception if `key` is empty, `algo` is unknown/non-cryptographic, `length` is less than `0` or too large (greater than 255 times the size of the hash function).

Changelog

Version	Description
8.0.0	Now throws a ValueError exception on error. Previously, `false` was returned and an `E_WARNING` message was emitted.

Examples

The example below produces a pair of separate keys, suitable for creation of an encrypt-then-HMAC construct, using AES-256 and SHA-256 for encryption and authentication respectively.

Example #1 hash_hkdf() example

```
<?php
// Generate a random key, and salt to strengthen it during derivation.
$inputKey = random_bytes(32);
$salt = random_bytes(16);

// Derive a pair of separate keys, using the same input created above.
$encryptionKey = hash_hkdf('sha256', $inputKey, 32, 'aes-256-encryption', $salt);
$authenticationKey = hash_hkdf('sha256', $inputKey, 32, 'sha-256-authentication', $salt);

var_dump($encryptionKey !== $authenticationKey); // bool(true)
?>
```

See Also

- hash_pbkdf2() - Generate a PBKDF2 key derivation of a supplied password
- » RFC 5869
- » userland implementation

hash_hmac

hash_hmac

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_hmac — Generate a keyed hash value using the HMAC method

Description

function **hash_hmac**(
  string `$algo`,
  string `$data`,
  #[\SensitiveParameter] string `$key`,
  bool `$binary` = `false`
): string

Parameters

`algo`

Name of selected hashing algorithm (e.g. `"sha256"`). For a list of supported algorithms see hash_hmac_algos().

> **Note** :
>
> Non-cryptographic hash functions are not allowed.

`data`

Message to be hashed.

`key`

Shared secret key used for generating the HMAC variant of the message digest.

`binary`

When set to `true`, outputs raw binary data. `false` outputs lowercase hexits.

Return Values

Returns a string containing the calculated message digest as lowercase hexits unless `binary` is set to true in which case the raw binary representation of the message digest is returned.

Errors/Exceptions

Throws a ValueError exception if `algo` is unknown or is a non-cryptographic hash function.

Changelog

Version	Description
8.0.0	Now throws a ValueError exception if `algo` is unknown or is a non-cryptographic hash function; previously, `false` was returned and an `E_WARNING` message was emitted.
7.2.0	Usage of non-cryptographic hash functions (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) was disabled.

Examples

Example #1 hash_hmac() example

```php
<?php
echo hash_hmac('sha256', 'The quick brown fox jumped over the lazy dog.', 'secret');
?>
```

The above example will output:

```
9c5c42422b03f0ee32949920649445e417b2c634050833c5165704b825c2a53b
```

See Also

- hash_hmac_algos() - Return a list of registered hashing algorithms suitable for hash_hmac
- hash_hmac_file() - Generate a keyed hash value using the HMAC method and the contents of a given file
- hash_equals() - Timing attack safe string comparison

hash_hmac_algos

hash_hmac_algos

(PHP 7 >= 7.2.0, PHP 8)

hash_hmac_algos — Return a list of registered hashing algorithms suitable for hash_hmac

Description

function **hash_hmac_algos**(): array

Parameters

This function has no parameters.

Return Values

Returns a numerically indexed array containing the list of supported hashing algorithms suitable for hash_hmac().

Examples

Example #1 hash_hmac_algos() example

```php
<?php
print_r(hash_hmac_algos());
```

The above example will output something similar to:

```
Array
(
    [0] => md2
    [1] => md4
    [2] => md5
    [3] => sha1
    [4] => sha224
    [5] => sha256
    [6] => sha384
    [7] => sha512/224
    [8] => sha512/256
    [9] => sha512
    [10] => sha3-224
    [11] => sha3-256
    [12] => sha3-384
    [13] => sha3-512
    [14] => ripemd128
    [15] => ripemd160
    [16] => ripemd256
    [17] => ripemd320
    [18] => whirlpool
    [19] => tiger128,3
    [20] => tiger160,3
    [21] => tiger192,3
    [22] => tiger128,4
    [23] => tiger160,4
    [24] => tiger192,4
    [25] => snefru
    [26] => snefru256
    [27] => gost
    [28] => gost-crypto
    [29] => haval128,3
    [30] => haval160,3
    [31] => haval192,3
    [32] => haval224,3
    [33] => haval256,3
    [34] => haval128,4
    [35] => haval160,4
    [36] => haval192,4
    [37] => haval224,4
    [38] => haval256,4
    [39] => haval128,5
    [40] => haval160,5
    [41] => haval192,5
    [42] => haval224,5
    [43] => haval256,5
)
```

Notes

> **Note** :
>
> Before PHP 7.2.0 the only means to get a list of supported hash algorithms has been to call hash_algos() which also returns hash algorithms that are not suitable for hash_hmac().

See Also

- hash_hmac() - Generate a keyed hash value using the HMAC method

- hash_algos() - Return a list of registered hashing algorithms

hash_hmac_file

hash_hmac_file

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_hmac_file — Generate a keyed hash value using the HMAC method and the contents of a given file

Description

```
function hash_hmac_file(
    string $algo,
    string $filename,
    #[\SensitiveParameter] string $key,
    bool $binary = false
): string|false
```

Parameters

`algo`

Name of selected hashing algorithm (e.g. `"sha256"`). For a list of supported algorithms see hash_hmac_algos().

> **Note** :
>
> Non-cryptographic hash functions are not allowed.

`filename`

URL describing location of file to be hashed; Supports fopen wrappers.

`key`

Shared secret key used for generating the HMAC variant of the message digest.

`binary`

When set to `true`, outputs raw binary data. `false` outputs lowercase hexits.

Return Values

Returns a string containing the calculated message digest as lowercase hexits unless `binary` is set to true in which case the raw binary representation of the message digest is returned. Returns `false` if the file `filename` cannot be read.

Errors/Exceptions

Throws a ValueError exception if `algo` is unknown or is a non-cryptographic hash function.

Changelog

Version	Description
8.0.0	Now throws a ValueError exception if `algo` is unknown or is a non-cryptographic hash function; previously, `false` was returned and an `E_WARNING` message was emitted.
7.2.0	Usage of non-cryptographic hash functions (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) was disabled.

Examples

Example #1 **hash_hmac_file()** example

```php
<?php
/* Create a file to calculate hash of */
file_put_contents('example.txt', 'The quick brown fox jumped over the lazy dog.');

echo hash_hmac_file('sha256', 'example.txt', 'secret');
?>
```

The above example will output:

```
9c5c42422b03f0ee32949920649445e417b2c634050833c5165704b825c2a53b
```

See Also

- hash_hmac() - Generate a keyed hash value using the HMAC method
- hash_hmac_algos() - Return a list of registered hashing algorithms suitable for hash_hmac
- hash_init() - Initialize an incremental hashing context
- hash_equals() - Timing attack safe string comparison

hash_init

hash_init

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_init — Initialize an incremental hashing context

Description

```
function hash_init(
    string $algo,
    int $flags = 0,
    #[\SensitiveParameter] string $key = "",
    array $options = []
): HashContext
```

Parameters

`algo`

Name of selected hashing algorithm (e.g. `"sha256"`). For a list of supported algorithms see hash_algos().

> **Note** :
>
> Non-cryptographic hash functions are not allowed if the `HASH_HMAC` flag is specified.

`flags`

Optional settings for hash generation, currently supports only one option: `HASH_HMAC`. When specified, the `key` *must* be specified.

`key`

When `HASH_HMAC` is specified for `flags`, a shared secret key to be used with the HMAC hashing method must be supplied in this parameter.

`options`

An array of options for the various hashing algorithms. Currently, only the `"seed"` parameter is supported by the MurmurHash variants.

Return Values

Returns a Hashing Context for use with hash_update(), hash_update_stream(), hash_update_file(), and hash_final().

Errors/Exceptions

- Throws a ValueError exception if `algo` is unknown or is a non-cryptographic hash function, or if `key` is empty.
- Passing configurations options of the wrong type in `options` will now emit an `E_DEPRECATED` error because they can be interpreted incorrectly. This will become a ValueError in the future.

Changelog

Version	Description
8.4.0	Passing options of a wrong type is now deprecated.
8.1.0	The `options` parameter has been added.
8.0.0	Now throws an ValueError exception if the `algo` is unknown or is a non-cryptographic hash function, or if `key` is empty. Previously, `false` was returned and an `E_WARNING` message was emitted.
7.2.0	Usage of non-cryptographic hash functions (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) with `HASH_HMAC` was disabled.
7.2.0	Return HashContext instead of resource.

Examples

Example #1 Incremental hashing example

```php
<?php
$hash = hash('sha256', 'The quick brown fox jumped over the lazy dog.');

$ctx = hash_init('sha256');
hash_update($ctx, 'The quick brown fox ');
hash_update($ctx, 'jumped over the lazy dog.');
$incremental_hash = hash_final($ctx);

echo $incremental_hash, PHP_EOL;
var_dump($hash === $incremental_hash);
?>
```

The above example will output:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
bool(true)
```

See Also

- hash_algos() - Return a list of registered hashing algorithms
- hash_update() - Pump data into an active hashing context
- hash_update_file() - Pump data into an active hashing context from a file
- hash_update_stream() - Pump data into an active hashing context from an open stream
- hash_final() - Finalize an incremental hash and return resulting digest

hash_pbkdf2

hash_pbkdf2

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

hash_pbkdf2 — Generate a PBKDF2 key derivation of a supplied password

Description

```
function hash_pbkdf2(
    string $algo,
    #[\SensitiveParameter] string $password,
    string $salt,
    int $iterations,
    int $length = 0,
    bool $binary = false,
    array $options = []
): string
```

Parameters

`algo`

Name of selected hashing algorithm (e.g. `"sha256"`). For a list of supported algorithms see hash_hmac_algos().

> **Note** :
>
> Non-cryptographic hash functions are not allowed.

`password`

The password to use for the derivation.

`salt`

The salt to use for the derivation. This value should be generated randomly.

`iterations`

The number of internal iterations to perform for the derivation.

`length`

The length of the output string. If `binary` is **`true`** this corresponds to the byte-length of the derived key, if `binary` is **`false`** this corresponds to twice the byte-length of the derived key (as every byte of the key is returned as two hexits).

If `0` is passed, the entire output of the supplied algorithm is used.

`binary`

When set to **`true`**, outputs raw binary data. **`false`** outputs lowercase hexits.

`options`

An array of options for the various hashing algorithms. Currently, only the `"seed"` key is supported by the MurmurHash variants.

Return Values

Returns a string containing the derived key as lowercase hexits unless `binary` is set to **`true`** in which case the raw binary representation of the derived key is returned.

Errors/Exceptions

Throws a ValueError exception if the algorithm is unknown, the `iterations` parameter is less than or equal to `0`, the `length` is less than `0` or the `salt` is too long (greater than **`INT_MAX`** `- 4`).

Changelog

Version	Description
8.0.0	Now throws a ValueError exception on error. Previously, **`false`** was returned and an **`E_WARNING`** message was emitted.
7.2.0	Usage of non-cryptographic hash functions (adler32, crc32, crc32b, fnv132, fnv1a32, fnv164, fnv1a64, joaat) was disabled.

Examples

Example #1 **hash_pbkdf2()** example, basic usage

```php
<?php
$password = "password";
$iterations = 600000;

// Generate a cryptographically secure random salt using random_bytes()
$salt = random_bytes(16);

$hash = hash_pbkdf2("sha256", $password, $salt, $iterations, 20);
var_dump($hash);

// for raw binary, the $length needs to be halved for equivalent results
```

```
$hash = hash_pbkdf2("sha256", $password, $salt, $iterations, 10, true);
var_dump(bin2hex($hash));?>
```

The above example will output something similar to:

```
string(20) "120fb6cffcf8b32c43e7"
string(20) "120fb6cffcf8b32c43e7"
```

Notes

> **Caution**
>
> The PBKDF2 method can be used for hashing passwords for storage. However, it should be noted that password_hash() or crypt() with `CRYPT_BLOWFISH` are better suited for password storage.

See Also

- password_hash() - Creates a password hash
- hash_hkdf() - Generate a HKDF key derivation of a supplied key input
- sodium_crypto_pwhash() - Derive a key from a password, using Argon2

hash_update

hash_update

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_update — Pump data into an active hashing context

Description

function **hash_update**(HashContext `$context`, string `$data`): true

Parameters

`context`

Hashing context returned by hash_init().

`data`

Message to be included in the hash digest.

Return Values

Always returns `true`.

Changelog

Version	Description
8.4.0	Now has a return type of true instead of bool.
7.2.0	Accept HashContext instead of resource.

See Also

- hash_init() - Initialize an incremental hashing context
- hash_update_file() - Pump data into an active hashing context from a file
- hash_update_stream() - Pump data into an active hashing context from an open stream
- hash_final() - Finalize an incremental hash and return resulting digest

hash_update_file

hash_update_file

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_update_file — Pump data into an active hashing context from a file

Description

function **hash_update_file**(HashContext `$context`, string `$filename`, ?resource `$stream_context = null`): bool

Parameters

`context`

Hashing context returned by hash_init().

`filename`

URL describing location of file to be hashed; Supports fopen() wrappers.

`stream_context`

Stream context as returned by stream_context_create().

Return Values

Returns **`true`** on success or **`false`** on failure.

Changelog

Version	Description
8.0.0	`stream_context` is now nullable.
7.2.0	Accept HashContext instead of resource.

See Also

- hash_init() - Initialize an incremental hashing context
- hash_update() - Pump data into an active hashing context
- hash_update_stream() - Pump data into an active hashing context from an open stream
- hash_final() - Finalize an incremental hash and return resulting digest
- hash_file() - Generate a hash value using the contents of a given file

hash_update_stream

hash_update_stream

(PHP 5 >= 5.1.2, PHP 7, PHP 8, PECL hash >= 1.1)

hash_update_stream — Pump data into an active hashing context from an open stream

Description

function **hash_update_stream**(HashContext `$context`, resource `$stream`, int `$length = -1`): int

Parameters

`context`

Hashing context returned by hash_init().

`stream`

Open file handle as returned by any stream creation function.

`length`

Maximum number of characters to copy from `stream` into the hashing context.

Return Values

Actual number of bytes added to the hashing context from `stream`.

Changelog

Version	Description
7.2.0	Accept HashContext instead of resource.

Examples

Example #1 hash_update_stream() example

```php
<?php
$fp = tmpfile();
fwrite($fp, 'jumped over the lazy dog.');
rewind($fp);

$ctx = hash_init('sha256');
hash_update($ctx, 'The quick brown fox ');
hash_update_stream($ctx, $fp);
echo hash_final($ctx);
?>
```

The above example will output:

```
68b1282b91de2c054c36629cb8dd447f12f096d3e3c587978dc2248444633483
```

See Also

- hash_init() - Initialize an incremental hashing context
- hash_update() - Pump data into an active hashing context
- hash_final() - Finalize an incremental hash and return resulting digest

Random Number Generators and Functions Related to Randomness

Introduction

Predefined Constants

The constants below are always available as part of the PHP core.

`MT_RAND_MT19937` (int)

Indicates that the correct » Mt19937 (Mersenne Twister) implementation will be used by the algorithm, when creating a Random\Engine\Mt19937 instance using Random\Engine\Mt19937::__construct() or seeding the global Mersenne Twister with mt_srand().

`MT_RAND_PHP` (int)

Indicates that an incorrect Mersenne Twister implementation will be used by the algorithm, when creating a Random\Engine\Mt19937 instance using Random\Engine\Mt19937::__construct() or seeding the global Mersenne Twister with mt_srand().

The incorrect implementation is available for backwards compatibility with mt_srand() prior to PHP 7.1.0.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

Examples

Example #1 Random Example

```php
<?php
$r = new \Random\Randomizer();

// Generating a random domain name
printf(
    "%s.example.com\n",
    $r->getBytesFromString('abcdefghijklmnopqrstuvwxyz0123456789', 16)
);

// Shuffle array:
$fruits = [ 'red' => '🍎', 'green' => '🥝', 'yellow' => '🍌', 'pink' => '🍑', 'purple' => '🍇' ];
echo "Salad: ", implode(', ', $r->shuffleArray($fruits)), "\n";

// Shuffeling array keys
$fruits = [ 'red' => '🍎', 'green' => '🥝', 'yellow' => '🍌', 'pink' => '🍑', 'purple' => '🍇' ];

$keys = $r->pickArrayKeys($fruits, 2);
// Look up the values for the picked keys.
$selection = array_map(
    static fn ($key) => $fruits[$key],
    $keys
);

echo "Values: ", implode(', ', $selection), "\n";
?>
```

The above example will output something similar to:

```
j87fzv1p0daiwmlo.example.com
Salad: 🥝, 🍇, 🍎, 🍌, 🍑
```

```
Values: 🍌, 🍑
```

Random Functions

Table of Contents

getrandmax

getrandmax

(PHP 4, PHP 5, PHP 7, PHP 8)

getrandmax — Show largest possible random value

Description

function **getrandmax**(): int

Returns the maximum value that can be returned by a call to rand().

Parameters

This function has no parameters.

Return Values

The largest possible random value returned by rand()

See Also

- rand() - Generate a random integer
- srand() - Seed the random number generator
- mt_getrandmax() - Show largest possible random value

lcg_value

lcg_value

(PHP 4, PHP 5, PHP 7, PHP 8)

lcg_value — Combined linear congruential generator

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.4.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.4',
    message: "use \\Random\\Randomizer::getFloat() instead",
)]
function lcg_value(): float
```

lcg_value() returns a pseudo random number in the range of (0, 1). The function combines two CGs with periods of 2^31 - 85 and 2^31 - 249. The period of this function is equal to the product of both primes.

> **Caution**
>
> This function does not generate cryptographically secure values, and *must not* be used for cryptographic purposes, or purposes that require returned values to be unguessable.

> If cryptographically secure randomness is required, the Random\Randomizer may be used with the Random\Engine\Secure engine. For simple use cases, the random_int() and random_bytes() functions provide a convenient and secure API that is backed by the operating system's CSPRNG.

> **Caution**
>
> Scaling the return value to a different interval using multiplication or addition (a so-called affine transformation) might result in a bias in the resulting value as floats are not equally dense across the number line. As not all values can be exactly represented by a float, the result of the affine transformation might also result in values outside of the requested interval.
>
> Use Random\Randomizer::getFloat() to generate a random float within an arbitrary interval. Use Random\Randomizer::getInt() to generate a random integer within an arbitrary interval.

Parameters

This function has no parameters.

Return Values

A pseudo random float value between 0.0 and 1.0, inclusive.

Changelog

Version	Description
8.4.0	This function has been deprecated.

See Also

- Random\Randomizer::getFloat() - Get a uniformly selected float
- Random\Randomizer::getInt() - Get a uniformly selected integer
- random_int() - Get a cryptographically secure, uniformly selected integer

mt_getrandmax

mt_getrandmax

(PHP 4, PHP 5, PHP 7, PHP 8)

mt_getrandmax — Show largest possible random value

Description

function **mt_getrandmax**(): int

Returns the maximum value that can be returned by a call to mt_rand().

Parameters

This function has no parameters.

Return Values

Returns the maximum random value returned by a call to mt_rand() without arguments, which is the maximum value that can be used for its `max` parameter without the result being scaled up (and therefore less random).

See Also

- mt_rand() - Generate a random value via the Mersenne Twister Random Number Generator
- mt_srand() - Seeds the Mersenne Twister Random Number Generator
- getrandmax() - Show largest possible random value

mt_rand

mt_rand

(PHP 4, PHP 5, PHP 7, PHP 8)

mt_rand — Generate a random value via the Mersenne Twister Random Number Generator

Description

function **mt_rand**(): int
function **mt_rand**(int `$min`, int `$max`): int

Many random number generators of older libcs have dubious or unknown characteristics and are slow. The **mt_rand()** function is a drop-in replacement for the older rand(). It uses a random number generator with known characteristics using the » Mersenne Twister, which will produce random numbers four times faster than what the average libc rand() provides.

If called without the optional `min`, `max` arguments **mt_rand()** returns a pseudo-random value between 0 and mt_getrandmax(). If you want a random number between 5 and 15 (inclusive), for example, use `mt_rand(5, 15)`.

> **Caution**
>
> This function does not generate cryptographically secure values, and *must not* be used for cryptographic purposes, or purposes that require returned values to be unguessable.
>
> If cryptographically secure randomness is required, the Random\Randomizer may be used with the Random\Engine\Secure engine. For simple use cases, the random_int() and random_bytes() functions provide a convenient and secure API that is backed by the operating system's CSPRNG.

> **Caution**
>
> This function uses the global Mt19937 ("Mersenne Twister") instance as the source of randomness and thus shares its state with all other functions using the global Mt19937. Using any of these functions advances the sequence for *all* the other functions, regardless of scope.
>
> Generating repeatable sequences by seeding mt_srand() or srand() with a known value will also yield repeatable output from this function.
>
> Prefer using Random\Randomizer methods in all newly written code.

Parameters

`min`

Optional lowest value to be returned (default: 0)

`max`

Optional highest value to be returned (default: mt_getrandmax())

Return Values

A random integer value between `min` (or 0) and `max` (or mt_getrandmax(), inclusive).

Errors/Exceptions

- If `max` is less than `min`, a ValueError will be thrown.

Changelog

Version	Description
8.0.0	A ValueError will be thrown if `max` is less than `min`; previously an `E_WARNING` was raised, and the function returned `false`.
7.2.0	**mt_rand()** has received a bug fix for a modulo bias bug. This means that sequences generated with a specific seed may differ from PHP 7.1 on 64-bit machines.
7.1.0	rand() has been made an alias of **mt_rand()**.
7.1.0	**mt_rand()** has been updated to use the fixed, correct, version of the Mersenne Twister algorithm. To fall back to the old behaviour, use mt_srand() with `MT_RAND_PHP` as the second parameter.

Examples

Example #1 mt_rand() example

```php
<?php
echo mt_rand(), "\n";
echo mt_rand(), "\n";

echo mt_rand(5, 15), "\n";
?>
```

The above example will output something similar to:

```
1604716014
1478613278
6
```

Notes

> **Warning**
>
> `min` `max` range must be within the range mt_getrandmax(). i.e. (`max` - `min`) <= mt_getrandmax() Otherwise, **mt_rand()** may return poorer random numbers than it should.

See Also

- mt_srand() - Seeds the Mersenne Twister Random Number Generator
- mt_getrandmax() - Show largest possible random value
- random_int() - Get a cryptographically secure, uniformly selected integer
- random_bytes() - Get cryptographically secure random bytes

mt_srand

mt_srand

(PHP 4, PHP 5, PHP 7, PHP 8)

mt_srand — Seeds the Mersenne Twister Random Number Generator

Description

function **mt_srand**(?int `$seed` = `null`, int `$mode` = `MT_RAND_MT19937`): void

Seeds the random number generator with `seed` or with a random value if no `seed` is given.

> **Note** :
>
> There is no need to seed the random number generator with srand() or **mt_srand()** as this is done automatically.

> **Caution**
>
> Because the Mt19937 (“Mersenne Twister”) engine accepts only a single 32 bit integer as the seed, the number of possible random sequences is limited to just 2^{32} (i.e. 4,294,967,296), despite Mt19937’s huge period of 2^{19937}-1.
>
> When relying on either implicit or explicit random seeding, duplications will appear much earlier. Duplicated seeds are expected with 50% probability after less than 80,000 randomly generated seeds according to the birthday problem. A 10% probability of a duplicated seed happens after randomly generating roughly 30,000 seeds.
>
> This makes Mt19937 unsuitable for applications where duplicated sequences must not happen with more than a negligible probability. If reproducible seeding is required, both the Random\Engine\Xoshiro256StarStar and Random\Engine\PcgOneseq128XslRr64 engines support much larger seeds that are unlikely to collide randomly. If reproducibility is not required, the Random\Engine\Secure engine provides cryptographically secure randomness.

Parameters

`seed`

Fills the state with values generated with a linear congruential generator that was seeded with `seed` interpreted as an unsigned 32 bit integer.

If `seed` is omitted or `null`, a random unsigned 32-bit integer will be used.

`mode`

Use one of the following constants to specify the implementation of the algorithm to use.

- `MT_RAND_MT19937`: The correct Mt19937 implementation, available as of PHP 7.1.0.
- `MT_RAND_PHP` Uses an incorrect Mersenne Twister implementation which was used as the default up till PHP 7.1.0. This mode is available for backward compatibility.

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

Return Values

No value is returned.

Changelog

Version	Description
8.3.0	`seed` is now nullable.
7.1.0	srand() has been made an alias of **mt_srand()**.
7.1.0	mt_rand() has been updated to use the fixed, correct, version of the Mersenne Twister algorithm. To fall back to the old behaviour, use **mt_srand()** with `MT_RAND_PHP` as the second parameter.

See Also

- mt_rand() - Generate a random value via the Mersenne Twister Random Number Generator
- mt_getrandmax() - Show largest possible random value
- srand() - Seed the random number generator

rand

rand

(PHP 4, PHP 5, PHP 7, PHP 8)

rand — Generate a random integer

Description

function **rand**(): int
function **rand**(int `$min`, int `$max`): int

If called without the optional `min`, `max` arguments **rand()** returns a pseudo-random integer between 0 and getrandmax(). If you want a random number between 5 and 15 (inclusive), for example, use `rand(5, 15)`.

> **Caution**
>
> This function does not generate cryptographically secure values, and *must not* be used for cryptographic purposes, or purposes that require returned values to be unguessable.
>
> If cryptographically secure randomness is required, the Random\Randomizer may be used with the Random\Engine\Secure engine. For simple use cases, the random_int() and random_bytes() functions provide a convenient and secure API that is backed by the operating system's CSPRNG.

> **Caution**

This function uses the global Mt19937 ("Mersenne Twister") instance as the source of randomness and thus shares its state with all other functions using the global Mt19937. Using any of these functions advances the sequence for *all* the other functions, regardless of scope.

Generating repeatable sequences by seeding mt_srand() or srand() with a known value will also yield repeatable output from this function.

Prefer using Random\Randomizer methods in all newly written code.

> **Note** :
>
> Prior to PHP 7.1.0, getrandmax() was only 32767 on some platforms (such as Windows). If you require a range larger than 32767, specifying `min` and `max` will allow you to create a range larger than this, or consider using mt_rand() instead.

> **Note** :
>
> As of PHP 7.1.0, **rand()** uses the same random number generator as mt_rand(). To preserve backwards compatibility **rand()** allows `max` to be smaller than `min` as opposed to returning `false` as mt_rand().

Parameters

`min`

The lowest value to return (default: 0)

`max`

The highest value to return (default: getrandmax())

Return Values

A pseudo random value between `min` (or 0) and `max` (or getrandmax(), inclusive).

Changelog

Version	Description
7.2.0	**rand()** has received a bug fix for a modulo bias bug. This means that sequences generated with a specific seed may differ from PHP 7.1 on 64-bit machines.
7.1.0	**rand()** has been made an alias of mt_rand().

Examples

Example #1 rand() example

```php
<?php
echo rand(), "\n";
echo rand(), "\n";

echo rand(5, 15), "\n";
?>
```

The above example will output something similar to:

```
7771
22264
11
```

Notes

> **Warning**
>
> `min max` range must be within the range getrandmax(). i.e. abs(`max - min`) <= getrandmax(). Otherwise, **rand()** may return poor-quality random numbers.

See Also

- srand() - Seed the random number generator
- getrandmax() - Show largest possible random value
- mt_rand() - Generate a random value via the Mersenne Twister Random Number Generator
- random_int() - Get a cryptographically secure, uniformly selected integer
- random_bytes() - Get cryptographically secure random bytes

random_bytes

random_bytes

(PHP 7, PHP 8)

random_bytes — Get cryptographically secure random bytes

Description

function **random_bytes**(int `$length`): string

Generates a string containing uniformly selected random bytes with the requested `length`.

As the returned bytes are selected completely randomly, the resulting string is likely to contain unprintable characters or invalid UTF-8 sequences. It may be necessary to encode it before transmission or display.

The randomness generated by this function is suitable for all applications, including the generation of long-term secrets, such as encryption keys.

The sources of randomness in the order of priority are as follows:

- Linux: » getrandom(), */dev/urandom*
- FreeBSD >= 12 (PHP >= 7.3): » getrandom(), */dev/urandom*
- Windows (PHP >= 7.2): » CNG-API

  Windows: » CryptGenRandom
- macOS (PHP >= 8.2; >= 8.1.9; >= 8.0.22 if CCRandomGenerateBytes is available at compile time): CCRandomGenerateBytes()

  macOS (PHP >= 8.1; >= 8.0.2): arc4random_buf(), */dev/urandom*
- NetBSD >= 7 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- OpenBSD >= 5.5 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- DragonflyBSD (PHP >= 8.1): » getrandom(), */dev/urandom*
- Solaris (PHP >= 8.1): » getrandom(), */dev/urandom*
- Any combination of operating system and PHP version not previously mentioned: */dev/urandom*
- If none of the sources are available or they all fail to generate randomness, then a Random\RandomException will be thrown.

> **Note** :
>
> Although this function was added to PHP in PHP 7.0, a » userland implementation is available for PHP 5.2 to 5.6, inclusive.

Parameters

`length`

The length of the random string that should be returned in bytes; must be `1` or greater.

Return Values

A string containing the requested number of cryptographically secure random bytes.

Errors/Exceptions

- If an appropriate source of randomness cannot be found, a Random\RandomException will be thrown.
- If the value of `length` is less than `1`, a ValueError will be thrown.

Changelog

Version	Description
8.2.0	In case of a CSPRNG failure, this function will now throw a Random\RandomException. Previously a plain Exception was thrown.

Examples

Example #1 random_bytes() example

```
<?php
$bytes = random_bytes(5);
var_dump(bin2hex($bytes));
?>
```

The above example will output something similar to:

```
string(10) "385e33f741"
```

See Also

- Random\Randomizer::getBytes() - Get random bytes
- random_int() - Get a cryptographically secure, uniformly selected integer
- bin2hex() - Convert binary data into hexadecimal representation
- base64_encode() - Encodes data with MIME base64

random_int

random_int

(PHP 7, PHP 8)

random_int — Get a cryptographically secure, uniformly selected integer

Description

function **random_int**(int `$min`, int `$max`): int

Generates a uniformly selected integer between the given minimum and maximum.

The randomness generated by this function is suitable for all applications, including the generation of long-term secrets, such as encryption keys.

The sources of randomness in the order of priority are as follows:

- Linux: » getrandom(), */dev/urandom*
- FreeBSD >= 12 (PHP >= 7.3): » getrandom(), */dev/urandom*
- Windows (PHP >= 7.2): » CNG-API

  Windows: » CryptGenRandom
- macOS (PHP >= 8.2; >= 8.1.9; >= 8.0.22 if CCRandomGenerateBytes is available at compile time): CCRandomGenerateBytes()

macOS (PHP >= 8.1; >= 8.0.2): arc4random_buf(), */dev/urandom*

- NetBSD >= 7 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- OpenBSD >= 5.5 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- DragonflyBSD (PHP >= 8.1): » getrandom(), */dev/urandom*
- Solaris (PHP >= 8.1): » getrandom(), */dev/urandom*
- Any combination of operating system and PHP version not previously mentioned: */dev/urandom*
- If none of the sources are available or they all fail to generate randomness, then a Random\RandomException will be thrown.

> **Note** :
>
> Although this function was added to PHP in PHP 7.0, a » userland implementation is available for PHP 5.2 to 5.6, inclusive.

Parameters

`min`

The lowest value to be returned.

`max`

The highest value to be returned.

Return Values

A cryptographically secure, uniformly selected integer from the closed interval [`min`, `max`]. Both `min` and `max` are possible return values.

Errors/Exceptions

- If an appropriate source of randomness cannot be found, a Random\RandomException will be thrown.
- If `max` is less than `min`, a ValueError will be thrown.

Changelog

Version	Description
8.2.0	In case of a CSPRNG failure, this function will now throw a Random\RandomException. Previously a plain Exception was thrown.

Examples

Example #1 **random_int()** example

```php
<?php
var_dump(random_int(100, 999));
var_dump(random_int(-1000, 0));
?>
```

The above example will output something similar to:

```
int(248)
int(-898)
```

See Also

- Random\Randomizer::getInt() - Get a uniformly selected integer
- random_bytes() - Get cryptographically secure random bytes

srand

srand

(PHP 4, PHP 5, PHP 7, PHP 8)

srand — Seed the random number generator

Description

function **srand**(?int `$seed` = `null`, int `$mode` = `MT_RAND_MT19937`): void

Seeds the random number generator with `seed` or with a random value if no `seed` is given.

> **Note** :
>
> There is no need to seed the random number generator with **srand()** or mt_srand() as this is done automatically.

> **Caution**
>
> Because the Mt19937 ("Mersenne Twister") engine accepts only a single 32 bit integer as the seed, the number of possible random sequences is limited to just 2^{32} (i.e. 4,294,967,296), despite Mt19937's huge period of 2^{19937}-1.
>
> When relying on either implicit or explicit random seeding, duplications will appear much earlier. Duplicated seeds are expected with 50% probability after less than 80,000 randomly generated seeds according to the birthday problem. A 10% probability of a duplicated seed happens after randomly generating roughly 30,000 seeds.
>
> This makes Mt19937 unsuitable for applications where duplicated sequences must not happen with more than a negligible probability. If reproducible seeding is required, both the Random\Engine\Xoshiro256StarStar and Random\Engine\PcgOneseq128XslRr64 engines support much larger seeds that are unlikely to collide randomly. If reproducibility is not required, the Random\Engine\Secure engine provides cryptographically secure randomness.

> **Note** :
>
> As of PHP 7.1.0, **srand()** has been made an alias of mt_srand().

Parameters

`seed`

Fills the state with values generated with a linear congruential generator that was seeded with `seed` interpreted as an unsigned 32 bit integer.

If `seed` is omitted or `null`, a random unsigned 32-bit integer will be used.

Return Values

No value is returned.

Changelog

Version	Description
8.3.0	`seed` is now nullable.
7.1.0	**srand()** has been made an alias of mt_srand().

See Also

- rand() - Generate a random integer
- getrandmax() - Show largest possible random value
- mt_srand() - Seeds the Mersenne Twister Random Number Generator

Random\Randomizer

(PHP 8 >= 8.2.0)

Introduction

Provides a high-level API to the randomness provided by an Random\Engine.

Class synopsis

namespace **Random**;
final class **Randomizer** {
/* Properties */
public readonly Random\Engine *$engine*;
/* Methods */
public function __construct(?Random\Engine `$engine` = **`null`**)
public function getBytes(int `$length`): string
public function getBytesFromString(string `$string`, int `$length`): string
public function getFloat(float `$min`, float `$max`, Random\IntervalBoundary `$boundary` = **`Random\IntervalBoundary::ClosedOpen`**): float
public function getInt(int `$min`, int `$max`): int
public function nextFloat(): float
public function nextInt(): int
public function pickArrayKeys(array `$array`, int `$num`): array
public function __serialize(): array
public function shuffleArray(array `$array`): array
public function shuffleBytes(string `$bytes`): string
public function __unserialize(array `$data`): void
}

Properties

engine

The low-level source of randomness for the Random\Randomizer's methods.

Table of Contents

Random\Randomizer::__construct

Random\Randomizer::__construct

(PHP 8 >= 8.2.0)

Random\Randomizer::__construct — Constructs a new Randomizer

Description

public function **Random\Randomizer::__construct**(?Random\Engine `$engine` = **`null`**)

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`engine`

The Random\Engine to use to generate randomness.

If `engine` is omitted or **`null`**, a new Random\Engine\Secure object will be used.

Examples

Example #1 Random\Randomizer::__construct() example

```
<?php
$r = new \Random\Randomizer();
$r = new \Random\Randomizer(new \Random\Engine\Mt19937());
?>
```

Random\Randomizer::getBytes

Random\Randomizer::getBytes

(PHP 8 >= 8.2.0)

Random\Randomizer::getBytes — Get random bytes

Description

public function **Random\Randomizer::getBytes**(int `$length`): string

Generates a string containing uniformly selected random bytes with the requested `length`.

As the returned bytes are selected completely randomly, the resulting string is likely to contain unprintable characters or invalid UTF-8 sequences. It may be necessary to encode it before transmission or display.

Parameters

`length`

The length of the random string that should be returned in bytes; must be `1` or greater.

Return Values

A string containing the requested number of random bytes.

Errors/Exceptions

- If the value of `length` is less than `1`, a ValueError will be thrown.
- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::getBytes() example

```php
<?php
$r = new \Random\Randomizer();

echo bin2hex($r->getBytes(8)), "\n";
?>
```

The above example will output something similar to:

```
ebdbe93cd56682c2
```

See Also

- random_bytes() - Get cryptographically secure random bytes
- bin2hex() - Convert binary data into hexadecimal representation
- base64_encode() - Encodes data with MIME base64
- Random\Randomizer::getBytesFromString() - Get random bytes from a source string

Random\Randomizer::getBytesFromString

Random\Randomizer::getBytesFromString

(PHP 8 >= 8.3.0)

Random\Randomizer::getBytesFromString — Get random bytes from a source string

Description

public function **Random\Randomizer::getBytesFromString**(string `$string`, int `$length`): string

Generates a string containing uniformly selected random bytes from the input `string` with the requested `length`.

The chance for a byte to be selected is proportional to its share of the input `string`. If each byte occurs the same amount of times, each byte is equally likely to be selected.

Parameters

`string`

The string from which the returned bytes are selected.

`length`

The length of the random string that should be returned in bytes; must be `1` or greater.

Return Values

A string containing the requested number of random bytes taken from the input `string`.

Errors/Exceptions

- If `string` is empty, a ValueError will be thrown.
- If the value of `length` is less than `1`, a ValueError will be thrown.
- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 **Random\Randomizer::getBytesFromString()** example

```php
<?php
$randomizer = new \Random\Randomizer();

printf(
    "%s.example.com",
    $randomizer->getBytesFromString('abcdefghijklmnopqrstuvwxyz0123456789', 16)
);
?>
```

The above example will output something similar to:

```
3zsw04eiubcf82jd.example.com
```

Example #2 Generate a random code for multi-factor authentication

```php
<?php
// The Secure engine is the default, but we make it explicit, because
// multi-factor codes are security sensitive.
$randomizer = new \Random\Randomizer(new \Random\Engine\Secure());

echo implode('-', str_split($randomizer->getBytesFromString('0123456789', 20), 5));
?>
```

The above example will output something similar to:

```
11551-80418-27047-42075
```

Example #3 Select from a string with a non-uniform distribution

```php
<?php
$randomizer = new \Random\Randomizer();

echo $randomizer->getBytesFromString('aaaaabcdef', 20);
?>
```

The above example will output something similar to:

```
fddacbeaaeaaacaaaaca
```

See Also

- Random\Randomizer::getBytes() - Get random bytes

Random\Randomizer::getFloat

Random\Randomizer::getFloat

(PHP 8 >= 8.3.0)

Random\Randomizer::getFloat — Get a uniformly selected float

Description

public function **Random\Randomizer::getFloat**(float `$min`, float `$max`, Random\IntervalBoundary `$boundary` = **`Random\IntervalBoundary::ClosedOpen`**): float

Returns a uniformly selected, equidistributed float from a requested interval.

Due to the limited precision, not all real numbers can be exactly represented as a floating point number. If a number cannot be represented exactly, it is rounded to the nearest representable exact value. Furthermore, floats are not equally dense across the whole number line. Because floats use a binary exponent, the distance between two neighboring floats doubles at each power of two. In other words: There are the same number of representable floats between `1.0` and `2.0` as they are between `2.0` and `4.0`, `4.0` and `8.0`, `8.0` and `16.0`, and so on.

Randomly sampling an arbitrary number within the requested interval, for example by dividing two integers, might result in a biased distribution for this reason. The necessary rounding will cause some floats to be returned more often than others, especially around powers of two when the density of floats changes.

Random\Randomizer::getFloat() implements an algorithm that will return a uniformly selected float from the largest possible set of exactly representable and equidistributed floats within the requested interval. The distance between the selectable floats ("step size") matches the distance between the floats with the lowest density, i.e. the distance between floats at interval boundary with the larger absolute value. This means that not all representable floats within a given interval may be returned if the interval crosses one or more powers of two. Stepping will start from the interval boundary with the larger absolute value to ensure the steps align with the exactly representable floats.

Closed interval boundaries will always be included in the set of selectable floats. Thus, if the size of the interval is not an exact multiple of the step size and the boundary with the smaller absolute value is a closed boundary, the distance between that boundary and its nearest selectable float will be smaller than the step size.

> **Caution**
>
> Post-processing the returned floats is likely going to break the uniform equidistribution, because the intermediate floats within a mathematical operation are experiencing implicit rounding. The requested interval should match the desired interval as closely as possible and rounding should only be performed as an explicit operation right before displaying the selected number to a user.

Explanation of the Algorithm Using Example Values

To give an example of how the algorithm works, consider a floating point representation that uses a 3-bit mantissa. This representation is capable of representing 8 different floating point values between consecutive powers of two. This means that between `1.0` and `2.0` all steps of size `0.125` are exactly representable and between `2.0` and `4.0` all steps of size `0.25` are exactly representable. In reality PHP's floats use a 52-bit mantissa and can represent 2^{52} different values between each power of two. This means that

- `1.0`
- `1.125`
- `1.25`
- `1.375`
- `1.5`
- `1.625`
- `1.75`
- `1.875`
- `2.0`
- `2.25`
- `2.5`
- `2.75`
- `3.0`
- `3.25`
- `3.5`
- `3.75`
- `4.0`

are the exactly representable floats between `1.0` and `4.0`.

Now consider that `$randomizer->getFloat(1.625, 2.5, IntervalBoundary::ClosedOpen)` is called, i.e. a random float starting at `1.625` until, but not including, `2.5` is requested. The algorithm first determines the step size at the boundary with the larger absolute value (`2.5`). The step size at that boundary is `0.25`.

Note that the size of the requested interval is `0.875`, which is not an exact multiple of `0.25`. If the algorithm would start stepping at the lower bound `1.625`, it would encounter `2.125`, which is not exactly representable and would experience implicit rounding. Thus the algorithm starts stepping at the upper boundary `2.5`. The selectable values are:

- `2.25`
- `2.0`
- `1.75`
- `1.625`

`2.5` is not included, because the upper boundary of the requested interval is an open boundary. `1.625` is included, even though its distance to the nearest value `1.75` is `0.125`, which is smaller than the previously determined step size of `0.25`. The reason for that is that the requested interval is closed at the lower boundary (`1.625`) and closed boundaries are always included.

Finally the algorithm uniformly selects one of the four selectable values at random and returns it.

Why Dividing Two Integers Does Not Work

In the previous example, there are eight representable floating point numbers between each sub-interval delimited by a power of two. To give an example why dividing two integers would not work well to generate a random float, consider that there are 16 equidistributed floating point numbers in the right-open interval from `0.0` until, but not including, `1.0`. Half of them are the eight exactly representable values between `0.5` and `1.0`, the other half are the values between `0.0` and `1.0` that the step size of `0.0625`. These can easily be generated by dividing a random integer between `0` and `15` by `16` to obtain one of:

- `0.0`
- `0.0625`
- `0.125`
- `0.1875`
- `0.25`
- `0.3125`
- `0.375`
- `0.4375`
- `0.5`
- `0.5625`
- `0.625`
- `0.6875`

- `0.75`
- `0.8125`
- `0.875`
- `0.9375`

This random float could be scaled to right-open interval from `1.625` until, but not including, `2.75` by multiplying it with the size of the interval (`0.875`) and adding the minimum `1.625`. This so-called affine transformation would result in the values:

- `1.625` rounded to `1.625`
- `1.679` rounded to `1.625`
- `1.734` rounded to `1.75`
- `1.789` rounded to `1.75`
- `1.843` rounded to `1.875`
- `1.898` rounded to `1.875`
- `1.953` rounded to `2.0`
- `2.007` rounded to `2.0`
- `2.062` rounded to `2.0`
- `2.117` rounded to `2.0`
- `2.171` rounded to `2.25`
- `2.226` rounded to `2.25`
- `2.281` rounded to `2.25`
- `2.335` rounded to `2.25`
- `2.390` rounded to `2.5`
- `2.445` rounded to `2.5`

Note how the upper boundary of `2.5` would be returned, despite being an open boundary and thus being excluded. Also note how `2.0` and `2.25` are twice as likely to be returned compared to the other values.

Parameters

`min`

The lower bound of the interval.

`max`

The upper bound of the interval.

`boundary`

Specifies whether the interval boundaries are possible return values.

Return Values

A uniformly selected, equidistributed float from the interval specified by `min`, `max`, and `boundary`. Whether `min` and `max` are possible return values depends on the value of `boundary`.

Errors/Exceptions

- If the value of `min` is not finite (is_finite()), a ValueError will be thrown.
- If the value of `max` is not finite (is_finite()), a ValueError will be thrown.
- If the requested interval does not contain any values, a ValueError will be thrown.
- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::getFloat() example

```php
<?php
$randomizer = new \Random\Randomizer();

// Note that the latitude granularity is double the
// longitude's granularity.
//
// For the latitude the value may be both -90 and 90.
```

```php
// For the longitude the value may be 180, but not -180, because
// -180 and 180 refer to the same longitude.
printf(
    "Lat: %+.6f Lng: %+.6f",
    $randomizer->getFloat(-90, 90, \Random\IntervalBoundary::ClosedClosed),
    $randomizer->getFloat(-180, 180, \Random\IntervalBoundary::OpenClosed),
);
?>
```

The above example will output something similar to:

```
Lat: +69.244304 Lng: -53.548951
```

Notes

> **Note** :
>
> This method implements the γ-section algorithm as published in » Drawing Random Floating-Point Numbers from an Interval. Frédéric Goualard, ACM Trans. Model. Comput. Simul., 32:3, 2022 to obtain the desired behavioral properties.

> **Caution**
>
> Underflow is intentionally left unhandled in the γ-section algorithm. This may result in incorrect values being returned for intervals with boundaries in the subnormal range of floating point numbers, i.e. for boundaries with an absolute value smaller than approximately `2^-1020` (about `8.9e-308`).

See Also

- Random\Randomizer::nextFloat() - Get a float from the right-open interval [0.0, 1.0)
- Random\Randomizer::getInt() - Get a uniformly selected integer

Random\Randomizer::getInt

Random\Randomizer::getInt

(PHP 8 >= 8.2.0)

Random\Randomizer::getInt — Get a uniformly selected integer

Description

public function **Random\Randomizer::getInt**(int `$min`, int `$max`): int

Parameters

`min`

The lowest value to be returned.

`max`

The highest value to be returned.

Return Values

A uniformly selected integer from the closed interval [`min`, `max`]. Both `min` and `max` are possible return values.

Errors/Exceptions

- If `max` is less than `min`, a ValueError will be thrown.
- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::getInt() example

```php
<?php
$r = new \Random\Randomizer();

// Random integer in range:
echo $r->getInt(1, 100), "\n";
?>
```

The above example will output something similar to:

```
42
```

See Also

- random_int() - Get a cryptographically secure, uniformly selected integer
- Random\Randomizer::getFloat() - Get a uniformly selected float

Random\Randomizer::nextFloat

Random\Randomizer::nextFloat

(PHP 8 >= 8.3.0)

Random\Randomizer::nextFloat — Get a float from the right-open interval [0.0, 1.0)

Description

public function **Random\Randomizer::nextFloat**(): float

Returns a uniformly selected, equidistributed float from the right-open interval from `0.0` until, but not including, `1.0`.

The chance for returned float to be within a given right-open sub-interval is proportional to the size of the sub-interval. This means that the chance for a float to be *less than* `0.5` is 50 %, which is equal to the chance for a float to be *at least* `0.5`. Likewise the chance for a float to be within the right-open interval from `0.2` until, but not including, `0.25` is exactly 5 %.

This property makes it easy to leverage **Random\Randomizer::nextFloat()** to generate a random boolean with a given chance by checking if the returned float is *less than* a given chance.

> **Note** :
>
> The domain of the returnable floats of **Random\Randomizer::nextFloat()** is identical to that of `Randomizer::getFloat(0.0, 1.0, IntervalBoundary::ClosedOpen)`.
>
> The internal implementation of **Random\Randomizer::nextFloat()** is more efficient.

> **Caution**
>
> Scaling the return value to a different interval using multiplication or addition (a so-called affine transformation) might result in a bias in the resulting value as floats are not equally dense across the number line. As not all values can be exactly represented by a float, the result of the affine transformation might also result in values outside of the requested interval due to implicit rounding. An in-depth explanation of the problems with the affine transformation is given in the documentation for Random\Randomizer::getFloat().
>
> Use Random\Randomizer::getFloat() to generate a random float within an arbitrary interval. Use Random\Randomizer::getInt() to generate a random integer within an arbitrary interval.

Parameters

This function has no parameters.

Return Values

A uniformly selected, equidistributed float from the right-open (`IntervalBoundary::ClosedOpen`) interval [0.0, 1.0). `0.0` is a possible return value, `1.0` is not.

Errors/Exceptions

- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::nextFloat() example

```php
<?php
$r = new \Random\Randomizer();

// The resulting bool will be true with the given chance.
$chance = 0.5;

$bool = $r->nextFloat() < $chance;

echo ($bool ? "You won" : "You lost"), "\n";
?>
```

The above example will output something similar to:

```
You won
```

Example #2 Incorrect scaling using an affine transformation

```php
<?php
final class MaxEngine implements Random\Engine {
    public function generate(): string {
        return "\xff";
    }
}

$randomizer = new \Random\Randomizer(new MaxEngine);

$min = 3.5;
$max = 4.5;

// DO NOT DO THIS:
//
// This will output 4.5, despite nextFloat() sampling from
// a right-open interval, which will never return 1.
printf("Wrong scaling: %.17g", $randomizer->nextFloat() * ($max - $min) + $min);

// Correct:
// $randomizer->getFloat($min, $max, \Random\IntervalBoundary::ClosedOpen);
?>
```

The above example will output:

```
Wrong scaling: 4.5
```

See Also

- Random\Randomizer::getFloat() - Get a uniformly selected float

Random\Randomizer::nextInt

Random\Randomizer::nextInt

(PHP 8 >= 8.2.0)

Random\Randomizer::nextInt — Get a positive integer

Description

public function **Random\Randomizer::nextInt**(): int

Parameters

This function has no parameters.

Return Values

A positive integer between 0 and a maximum value depending on the number of bytes returned from Random\Engine::generate(). The exact maximum can be calculated as $2^{\$engine_bytes * 8 - 1} - 1$.

Errors/Exceptions

- To avoid inconsistencies, 32 bit PHP will throw Random\RandomException if the output size of Random\Engine::generate() exceeds 32 bits, as the selected integer cannot be returned losslessly. This affects the native 64 bit engines Random\Engine\PcgOneseq128XslRr64 and Random\Engine\Xoshiro256StarStar. Any userland engine returning more than 4 bytes of randomness is also affected.
- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::nextInt() example

```php
<?php
$r = new \Random\Randomizer();

// Random "next" integer:
echo $r->nextInt(), "\n";
?>
```

The above example will output something similar to:

```
8041689838856078718
```

Random\Randomizer::pickArrayKeys

Random\Randomizer::pickArrayKeys

(PHP 8 >= 8.2.0)

Random\Randomizer::pickArrayKeys — Select random array keys

Description

public function **Random\Randomizer::pickArrayKeys**(array `$array`, int `$num`): array

Uniformly selects `num` distinct array keys of the input `array`.

Each key of the input `array` is equally likely to be returned.

> **Caution**

> The selection of the array keys depends on the internal structure of the input `array`. The returned array keys might be different for two equal input arrays and two Random\Engines with identical state, depending on how the input arrays have been created.

Parameters

`array`

The array whose array keys are selected.

`num`

The number of array keys to return; must be between `1` and the number of elements in `array`.

Return Values

An array containing `num` distinct array keys of `array`.

The returned array will be a list (array_is_list()). It will be a subset of the array returned by array_keys().

Errors/Exceptions

- If `num` is less than `1` or greater than the number of elements in `array`, a ValueError will be thrown.
- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::pickArrayKeys() example

```php
<?php
$r = new \Random\Randomizer();

$fruits = [ 'red' => '🍎', 'green' => '🥝', 'yellow' => '🍌', 'pink' => '🍑', 'purple' => '🍇' ];

// Pick 2 random array keys:
echo "Keys: ", implode(', ', $r->pickArrayKeys($fruits, 2)), "\n";

// Pick another 3:
echo "Keys: ", implode(', ', $r->pickArrayKeys($fruits, 3)), "\n";
?>
```

The above example will output something similar to:

```
Keys: yellow, purple
Keys: red, green, yellow
```

Example #2 Picking random values

```php
<?php
$r = new \Random\Randomizer();

$fruits = [ 'red' => '🍎', 'green' => '🥝', 'yellow' => '🍌', 'pink' => '🍑', 'purple' => '🍇' ];

$keys = $r->pickArrayKeys($fruits, 2);
// Look up the values for the picked keys.
$selection = array_map(
    static fn ($key) => $fruits[$key],
    $keys
);

echo "Values: ", implode(', ', $selection), "\n";
?>
```

The above example will output something similar to:

```
Values: 🍎, 🍆
```

See Also

- array_keys() - Return all the keys or a subset of the keys of an array

Random\Randomizer::__serialize

Random\Randomizer::__serialize

(PHP 8 >= 8.2.0)

Random\Randomizer::__serialize — Serializes the Randomizer object

Description

public function **Random\Randomizer::__serialize**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 Random\Randomizer::__serialize() example

```php
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Randomizer::shuffleArray

Random\Randomizer::shuffleArray

(PHP 8 >= 8.2.0)

Random\Randomizer::shuffleArray — Get a permutation of an array

Description

public function **Random\Randomizer::shuffleArray**(array `$array`): array

Returns a uniformly selected permutation of the input `array`.

Each possible permutation of the input `array` is equally likely to be returned.

Parameters

`array`

The array whose values are shuffled.

The input array will not be modified.

Return Values

A permutation of the values of `array`.

Array keys of the input `array` will not be preserved; the returned array will be a list (array_is_list()).

Errors/Exceptions

- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::shuffleArray() example

```php
<?php
$r = new \Random\Randomizer();

$fruits = [ 'red' => '🍅', 'green' => '🥝', 'yellow' => '🍌', 'pink' => '🍑', 'purple' => '🍇' ];

// Shuffle array:
echo "Salad: ", implode(', ', $r->shuffleArray($fruits)), "\n";

// Shuffle again:
echo "Another Salad: ", implode(', ', $r->shuffleArray($fruits)), "\n";
?>
```

The above example will output something similar to:

```
Salad: 🍅, 🥝, 🍇, 🍌, 🍑
Another Salad: 🍑, 🍇, 🥝, 🍅, 🍌
```

Random\Randomizer::shuffleBytes

Random\Randomizer::shuffleBytes

(PHP 8 >= 8.2.0)

Random\Randomizer::shuffleBytes — Get a byte-wise permutation of a string

Description

public function **Random\Randomizer::shuffleBytes**(string `$bytes`): string

Returns a uniformly selected permutation of the input `bytes`.

Each possible permutation of the input `bytes` is equally likely to be returned.

Parameters

`bytes`

The string whose bytes are shuffled.

The input string will not be modified.

Return Values

A permutation of the bytes of `bytes`.

Errors/Exceptions

- Any Throwables thrown by the Random\Engine::generate() method of the underlying `Random\Randomizer::$engine`.

Examples

Example #1 Random\Randomizer::shuffleBytes() example

```
<?php
$r = new \Random\Randomizer();

// Shuffle bytes in a string:
echo "«", $r->shuffleBytes("PHP is great!"), "»\n";
?>
```

The above example will output something similar to:

```
« ga rHs!PPiet»
```

Example #2 Byte-wise shuffling breaks Unicode characters

```
<?php
$r = new \Random\Randomizer();

$unicode = "🍅, 🥝, 🍌, 🍑, 🍇";
$shuffled = $r->shuffleBytes( $unicode );

// Byte-wise shuffling of non-ASCII characters destroys them,
// resulting in invalid sequences (indicated by the Unicode
// replacement character) or even entirely different characters
// appearing in the output.
echo "Original: ", $unicode, "\n";
echo "Shuffled: «", $shuffled, "»\n";
echo "Shuffled Bytes: ", bin2hex($shuffled), "\n";
?>
```

The above example will output something similar to:

```
Original: 🍅, 🥝, 🍌, 🍑, 🍇
Shuffled: «🍟 ,,,
```

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

Return Values

No value is returned.

Examples

Example #1 Random\Randomizer::__unserialize() example

```
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\IntervalBoundary

(PHP 8 >= 8.3.0)

Introduction

The **Random\IntervalBoundary** enum specifies whether an interval includes the boundary values within the set of values lying in the interval.

Enum synopsis

namespace **Random**;
enum **IntervalBoundary**
{
case ClosedOpen ; // A right-open interval. The lower boundary is included in the interval, the upper boundary is not.

case ClosedClosed ; // A closed interval. Both boundary values are included in the interval.

case OpenClosed ; // A left-open interval. The upper boundary is included in the interval, the lower boundary is not.

case OpenOpen ; // An open interval. Neither boundary value is included in the interval.

}

Random\Engine

(PHP 8 >= 8.2.0)

Introduction

A **Random\Engine** provides a low-level source of randomness by returning random bytes that are consumed by high-level APIs to perform their operations. The **Random\Engine** interface allows swapping out the algorithm used to generate randomness, because each algorithm makes different tradeoffs to fit specific use-cases. Some algorithms are very fast, but generate lower-quality randomness,

whereas other algorithms are slower, but generate better randomness, up to cryptographically secure randomness as provided by the Random\Engine\Secure engine.

PHP provides several **Random\Engine**s out of the box to accommodate different use-cases. The Random\Engine\Secure engine that is backed by a CSPRNG is the recommended safe default choice, unless the application requires either reproducible sequences or very high performance.

Interface synopsis

```
namespace Random;
interface Engine {
/* Methods */
public function generate(): string
}
```

Table of Contents

Random\Engine::generate

Random\Engine::generate

(PHP 8 >= 8.2.0)

Random\Engine::generate — Generates randomness

Description

public function **Random\Engine::generate**(): string

Returns randomness and advances the algorithm's state by one step.

The randomness is represented by a binary string containing random bytes. This representation allows to unambiguously interpret the random bits generated by the algorithm, for example to accommodate different output sizes used by different algorithms.

Algorithms that natively operate on integer values should return the integer in little-endian byte order, for example by leveraging the pack() function with the P format code. The high-level interface provided by the Random\Randomizer will interpret the returned random bytes as unsigned little-endian integers if a numeric representation is required.

It is strongly recommended that each bit of the returned string is uniformly and independently selected, as some applications require randomness based on the bit-level to work correctly. For example linear congruential generators often generate lower-quality randomness for the less significant bits of the return integer value and thus would not be appropriate for applications that require bit-level randomness.

Parameters

This function has no parameters.

Return Values

A non-empty string containing random bytes.

> **Note** :
>
> The Random\Randomizer works with unsigned 64 bit integers internally. If the returned string contains more than 64 bit (8 byte) of randomness the exceeding bytes will be ignored. Other applications may be able to process more than 64 bit at once.

Errors/Exceptions

- If generating randomness fails, a Random\RandomException should be thrown. Any other Exception thrown during generation should be caught and wrapped into a Random\RandomException.
- If the returned string is empty, a Random\BrokenRandomEngineError will be thrown by the Random\Randomizer.
- If the implemented algorithm is severely biased, a Random\BrokenRandomEngineError may be thrown by the Random\Randomizer to prevent infinite loops if rejection sampling is required to return unbiased results.

Examples

Example #1 Random\Engine::generate() example

```php
<?php
/**
 * Implements a Linear Congruential Generator with modulus 65536,
 * multiplier 61 and increment 17 returning an 8 Bit integer.
 *
 * Note: This engine is suitable for demonstration purposes only.
 *       Linear Congruential Generators generally generate low
 *       quality randomness and this specific implementation has
 *       a very short 16 Bit period that is unsuitable for
 *       almost any real-world use case.
 */
final class LinearCongruentialGenerator implements \Random\Engine
{
    private int $state;

    public function __construct(?int $seed = null)
    {
        if ($seed === null) {
            $seed = random_int(0, 0xffff);
        }

        $this->state = $seed & 0xffff;
    }

    public function generate(): string
    {
        $this->state = (61 * $this->state + 17) & 0xffff;

        return pack('C', $this->state >> 8);
    }
}

$r = new \Random\Randomizer(
    new LinearCongruentialGenerator(seed: 1)
);

echo "Lucky Number: ", $r->getInt(0, 99), "\n";
?>
```

The above example will output:

```
Lucky Number: 4
```

Random\CryptoSafeEngine

(PHP 8 >= 8.2.0)

Introduction

A marker interface indicating that the Random\Engine returns cryptographically secure randomness.

Interface synopsis

namespace **Random**;
interface **CryptoSafeEngine** extends Random\Engine {
/* Inherited methods */
public function Random\Engine::generate(): string
}

Random\Engine\Secure

(PHP 8 >= 8.2.0)

Introduction

Generates cryptographically secure randomness using the operating system's CSPRNG.

The randomness generated by this Random\Engine is suitable for all applications, including the generation of long-term secrets, such as encryption keys.

The **Random\Engine\Secure** engine is the recommended safe default choice, unless the application requires either reproducible sequences or very high performance.

Class synopsis

```
namespace Random\Engine;
final class Secure implements Random\CryptoSafeEngine {
/* Methods */
public function generate(): string
}
```

Table of Contents

Random\Engine\Secure::generate

Random\Engine\Secure::generate

(PHP 8 >= 8.2.0)

Random\Engine\Secure::generate — Generate cryptographically secure randomness

Description

public function **Random\Engine\Secure::generate**(): string

Returns cryptographically secure randomness.

The sources of randomness in the order of priority are as follows:

- Linux: » getrandom(), */dev/urandom*
- FreeBSD >= 12 (PHP >= 7.3): » getrandom(), */dev/urandom*
- Windows (PHP >= 7.2): » CNG-API

  Windows: » CryptGenRandom
- macOS (PHP >= 8.2; >= 8.1.9; >= 8.0.22 if CCRandomGenerateBytes is available at compile time): CCRandomGenerateBytes()

  macOS (PHP >= 8.1; >= 8.0.2): arc4random_buf(), */dev/urandom*
- NetBSD >= 7 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- OpenBSD >= 5.5 (PHP >= 7.1; >= 7.0.1): arc4random_buf(), */dev/urandom*
- DragonflyBSD (PHP >= 8.1): » getrandom(), */dev/urandom*
- Solaris (PHP >= 8.1): » getrandom(), */dev/urandom*
- Any combination of operating system and PHP version not previously mentioned: */dev/urandom*
- If none of the sources are available or they all fail to generate randomness, then a Random\RandomException will be thrown.

Parameters

This function has no parameters.

Return Values

A string containing `PHP_INT_SIZE` cryptographically secure random bytes.

Errors/Exceptions

- If an appropriate source of randomness cannot be found, a Random\RandomException will be thrown.

Random\Engine\Mt19937

(PHP 8 >= 8.2.0)

Introduction

Implements the » Mt19937 ("Mersenne Twister") algorithm.

Class synopsis

namespace **Random\Engine**;
final class **Mt19937** implements Random\Engine {
/* Methods */
public function __construct(?int $seed = **null**, int $mode = **MT_RAND_MT19937**)
public function __debugInfo(): array
public function generate(): string
public function __serialize(): array
public function __unserialize(array $data): void
}

Table of Contents

Random\Engine\Mt19937::__construct

Random\Engine\Mt19937::__construct

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__construct — Constructs a new Mt19937 engine

Description

public function **Random\Engine\Mt19937::__construct**(?int $seed = **null**, int $mode = **MT_RAND_MT19937**)

> **Caution**
>
> Because the Mt19937 ("Mersenne Twister") engine accepts only a single 32 bit integer as the seed, the number of possible random sequences is limited to just 2^{32} (i.e. 4,294,967,296), despite Mt19937's huge period of 2^{19937}-1.
>
> When relying on either implicit or explicit random seeding, duplications will appear much earlier. Duplicated seeds are expected with 50% probability after less than 80,000 randomly generated seeds according to the birthday problem. A 10% probability of a duplicated seed happens after randomly generating roughly 30,000 seeds.
>
> This makes Mt19937 unsuitable for applications where duplicated sequences must not happen with more than a negligible probability. If reproducible seeding is required, both the Random\Engine\Xoshiro256StarStar and Random\Engine\PcgOneseq128XslRr64 engines support much larger seeds that are unlikely to collide randomly. If reproducibility is not required, the Random\Engine\Secure engine provides cryptographically secure randomness.

Parameters

`seed`

Fills the state with values generated with a linear congruential generator that was seeded with `seed` interpreted as an unsigned 32 bit integer.

If `seed` is omitted or **null**, a random unsigned 32 bit integer will be used.

`mode`

Use one of the following constants to specify the implementation of the algorithm to use.

- **MT_RAND_MT19937**: The correct Mt19937 implementation.
- **MT_RAND_PHP**: An incorrect implementation for backwards compatibility with mt_srand() prior to PHP 7.1.0.

> **Warning**

> This feature has been *DEPRECATED* as of PHP 8.3.0. Relying on this feature is highly discouraged.

Examples

Example #1 Random\Engine\Mt19937::__construct() example

```php
<?php
// Uses a random 32 Bit seed.
$e = new \Random\Engine\Mt19937();

$r = new \Random\Randomizer($e);
?>
```

Random\Engine\Mt19937::__debugInfo

Random\Engine\Mt19937::__debugInfo

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__debugInfo — Returns the internal state of the engine

Description

public function **Random\Engine\Mt19937::__debugInfo**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 Random\Engine\Mt19937::__debugInfo() example

```php
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Engine\Mt19937::generate

Random\Engine\Mt19937::generate

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::generate — Generate 32 bits of randomness

Description

public function **Random\Engine\Mt19937::generate**(): string

> **Warning**

> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A string representing an unsigned 32 bit integer in little-endian order.

Random\Engine\Mt19937::__serialize

Random\Engine\Mt19937::__serialize

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__serialize — Serializes the Mt19937 object

Description

public function **Random\Engine\Mt19937::__serialize**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 Random\Engine\Mt19937::__serialize() example

```
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Engine\Mt19937::__unserialize

Random\Engine\Mt19937::__unserialize

(PHP 8 >= 8.2.0)

Random\Engine\Mt19937::__unserialize — Deserializes the `data` parameter into a Mt19937 object

Description

public function **Random\Engine\Mt19937::__unserialize**(array `$data`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

Return Values

No value is returned.

Examples

Example #1 Random\Engine\Mt19937::__unserialize() example

```
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Engine\PcgOneseq128XslRr64

(PHP 8 >= 8.2.0)

Introduction

Implements a » Permuted congruential generator (PCG) with 128 bits of state, XSL and RR output transformations, and 64 bits of output.

Class synopsis

namespace **Random\Engine**;
final class **PcgOneseq128XslRr64** implements Random\Engine {
/* Methods */
public function __construct(string|int|null $seed = **null**)
public function __debugInfo(): array
public function generate(): string
public function jump(int $advance): void
public function __serialize(): array
public function __unserialize(array $data): void
}

Table of Contents

Random\Engine\PcgOneseq128XslRr64::__construct

Random\Engine\PcgOneseq128XslRr64::__construct

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__construct — Constructs a new PCG Oneseq 128 XSL RR 64 engine

Description

public function **Random\Engine\PcgOneseq128XslRr64::__construct**(string|int|null $seed = **null**)

Parameters

`seed`

How the internal 128 bit (16 byte) state consisting of one unsigned 128 bit integer is seeded depends on the type used as the `seed`.

Type	Description
null	Fills the state with 16 random bytes generated using the CSPRNG.
int	Fills the state by setting the state to 0, advancing the engine one step, adding the value of seed interpreted as an unsigned 64 bit integer, and advancing the engine another step.
string	Fills the state by interpreting a 16 byte string as a little-endian unsigned 128 bit integer.

Errors/Exceptions

- If the length of a string seed is not 16 bytes, a ValueError will be thrown.

Examples

Example #1 Random\Engine\PcgOneseq128XslRr64::__construct() example

```
<?php
// Uses a random 128 Bit seed.
$e = new \Random\Engine\PcgOneseq128XslRr64();

$r = new \Random\Randomizer($e);
?>
```

Example #2 Deriving a seed from a string

```
<?php
$string = "My string seed";

// Hash the string with truncated SHA-256 using binary output
// to turn the $string into a 128 Bit seed. Using the same
// string will result in the same sequence of randomness.
$e = new \Random\Engine\PcgOneseq128XslRr64(
    substr(hash('sha256', $string, binary: true), 0, 16)
);

echo bin2hex($e->generate()), "\n";
?>
```

The above example will output:

```
8333ef59315b16d8
```

Random\Engine\PcgOneseq128XslRr64::__debugInfo

Random\Engine\PcgOneseq128XslRr64::__debugInfo

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__debugInfo — Returns the internal state of the engine

Description

public function **Random\Engine\PcgOneseq128XslRr64::__debugInfo**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 Random\Engine\PcgOneseq128XslRr64::__debugInfo() example

```php
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Engine\PcgOneseq128XslRr64::generate

Random\Engine\PcgOneseq128XslRr64::generate

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::generate — Generate 64 bits of randomness

Description

public function **Random\Engine\PcgOneseq128XslRr64::generate**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A string representing an unsigned 64 bit integer in little-endian order.

Random\Engine\PcgOneseq128XslRr64::jump

Random\Engine\PcgOneseq128XslRr64::jump

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::jump — Efficiently move the engine ahead multiple steps

Description

public function **Random\Engine\PcgOneseq128XslRr64::jump**(int `$advance`): void

Moves the algorithm's state ahead by the number of steps given by `advance`, as if Random\Engine\PcgOneseq128XslRr64::generate() was called that many times.

Parameters

`advance`

The number of steps to move ahead; must be `0` or greater.

Return Values

No value is returned.

Errors/Exceptions

- If `advance` is less than `0`, a ValueError will be thrown.

Examples

Example #1 Random\Engine\PcgOneseq128XslRr64::jump() example

```php
<?php
$a = new \Random\Engine\PcgOneseq128XslRr64(0);
$b = clone $a;

for ($i = 0; $i < 1_000; $i++) {
    $a->generate();
}
$b->jump(1_000);

echo "A: ", bin2hex($a->generate()), "\n";
echo "B: ", bin2hex($b->generate()), "\n";
?>
```

The above example will output:

```
A: e6d0d5813913a424
B: e6d0d5813913a424
```

Example #2 Randomizer methods may call the engine more than once

```php
<?php
$a = new \Random\Randomizer(new \Random\Engine\PcgOneseq128XslRr64(42659));
$b = new \Random\Randomizer(clone $a->engine);

$a->getInt(1, 1572864); // Performs two calls to generate().
$a->getInt(1, 1572864);

$b->engine->jump(2);

// Because the first call to ->getInt() called ->generate() twice
// the engines do not match up after performing a ->jump(2).
echo "A: ", bin2hex($a->engine->generate()), "\n";
echo "B: ", bin2hex($b->engine->generate()), "\n";

// Now the B engine matches the A engine.
echo "B: ", bin2hex($b->engine->generate()), "\n";
?>
```

The above example will output:

```
A: 1e9f3107d56653d0
B: a156c0086dd79d44
B: 1e9f3107d56653d0
```

Random\Engine\PcgOneseq128XslRr64::__serialize

Random\Engine\PcgOneseq128XslRr64::__serialize

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__serialize — Serializes the PcgOneseq128XslRr64 object

Description

public function **Random\Engine\PcgOneseq128XslRr64::__serialize**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 Random\Engine\PcgOneseq128XslRr64::__serialize() example

```
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Engine\PcgOneseq128XslRr64::__unserialize

Random\Engine\PcgOneseq128XslRr64::__unserialize

(PHP 8 >= 8.2.0)

Random\Engine\PcgOneseq128XslRr64::__unserialize — Deserializes the `data` parameter into a PcgOneseq128XslRr64 object

Description

public function **Random\Engine\PcgOneseq128XslRr64::__unserialize**(array `$data`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

Return Values

No value is returned.

Examples

Example #1 Random\Engine\PcgOneseq128XslRr64::__unserialize() example

```
<?php

/* ... */
```

```
?>
```

The above example will output something similar to:

```
...
```

Random\Engine\Xoshiro256StarStar

(PHP 8 >= 8.2.0)

Introduction

Implements the » xoshiro256** algorithm.

Class synopsis

namespace **Random\Engine**;
final class **Xoshiro256StarStar** implements Random\Engine {
/* Methods */
public function __construct(string|int|null `$seed` = **`null`**)
public function __debugInfo(): array
public function generate(): string
public function jump(): void
public function jumpLong(): void
public function __serialize(): array
public function __unserialize(array `$data`): void
}

Table of Contents

Random\Engine\Xoshiro256StarStar::__construct

Random\Engine\Xoshiro256StarStar::__construct

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__construct — Constructs a new xoshiro256** engine

Description

public function **Random\Engine\Xoshiro256StarStar::__construct**(string|int|null `$seed` = **`null`**)

Parameters

`seed`

How the internal 256 bit (32 byte) state consisting of four unsigned 64 bit integers is seeded depends on the type used as the `seed`.

Type	Description
null	Fills the state with 32 random bytes generated using the CSPRNG.
int	Fills the state with four consecutive values generated with the SplitMix64 algorithm that was seeded with `seed` interpreted as an unsigned 64 bit integer.
string	Fills the state by interpreting a 32 byte string as four little-endian unsigned 64 bit integers.

Errors/Exceptions

- If the length of a string `seed` is not 32 bytes, a ValueError will be thrown.
- If a string `seed` consists of 32 NUL bytes (`"\x00"`), a ValueError will be thrown.

Examples

Example #1 Random\Engine\Xoshiro256StarStar::__construct() example

```php
<?php
// Uses a random 256 Bit seed.
$e = new \Random\Engine\Xoshiro256StarStar();

$r = new \Random\Randomizer($e);
?>
```

Example #2 Deriving a seed from a string

```php
<?php
$string = "My string seed";

// Hash the string with SHA-256 using binary output to turn the
// $string into a 256 Bit seed. Using the same string will result
// in the same sequence of randomness.
$e = new \Random\Engine\Xoshiro256StarStar(
    hash('sha256', $string, binary: true)
);

echo bin2hex($e->generate()), "\n";
?>
```

The above example will output:

```
6e013453678388c2
```

Random\Engine\Xoshiro256StarStar::__debugInfo

Random\Engine\Xoshiro256StarStar::__debugInfo

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__debugInfo — Returns the internal state of the engine

Description

public function **Random\Engine\Xoshiro256StarStar::__debugInfo**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 Random\Engine\Xoshiro256StarStar::__debugInfo() example

```php
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Engine\Xoshiro256StarStar::generate

Random\Engine\Xoshiro256StarStar::generate

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::generate — Generate 64 bits of randomness

Description

public function **Random\Engine\Xoshiro256StarStar::generate**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A string representing an unsigned 64 bit integer in little-endian order.

Random\Engine\Xoshiro256StarStar::jump

Random\Engine\Xoshiro256StarStar::jump

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::jump — Efficiently move the engine ahead by 2^128 steps

Description

public function **Random\Engine\Xoshiro256StarStar::jump**(): void

Moves the algorithm's state ahead by 2^{128} steps, as if Random\Engine\Xoshiro256StarStar::generate() was called 2^{128} times.

The purpose of a jump is to facilitate the creation of a new Random\Engine\Xoshiro256StarStar engine from an existing seeded Random\Engine\Xoshiro256StarStar engine. The seeded engine acts as a blueprint, which can be cloned and repeatedly jumped to create 2^{128} non-overlapping sequences with 2^{128} values each.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 **Random\Engine\Xoshiro256StarStar::jump()** example

```php
<?php
use Random\Engine\Xoshiro256StarStar;
use Random\Randomizer;

$blueprintRng = new Xoshiro256StarStar(0);

$fibers = [];
```

```php
for ($i = 0; $i < 8; $i++) {
    $fiberRng = clone $blueprintRng;
    $blueprintRng->jump();

    $fiber = new Fiber(static function () use ($fiberRng, $i): void {
        $randomizer = new Randomizer($fiberRng);

        while (true) {
            Fiber::suspend();

            echo "{$i}: ", $randomizer->getInt(0, 100), "\n";
        }
    });
    $fiber->start();

    $fibers[] = $fiber;
}

// Even though the fibers execute in a random order, they will print the same value
// each time, because each has its own unique instance of the RNG.
$randomizer = new Randomizer();

$fibers = $randomizer->shuffleArray($fibers);
foreach ($fibers as $fiber) {
    $fiber->resume();
}

$fibers = $randomizer->shuffleArray($fibers);
foreach ($fibers as $fiber) {
    $fiber->resume();
}
?>
```

The above example will output something similar to:

```
4: 89
3: 10
2: 63
1: 75
6: 41
5: 56
0: 16
7: 60
7: 34
6: 58
1: 74
4: 63
3: 3
5: 42
2: 45
0: 86
```

See Also

- Random\Engine\Xoshiro256StarStar::jumpLong() - Efficiently move the engine ahead by 2^192 steps

Random\Engine\Xoshiro256StarStar::jumpLong

Random\Engine\Xoshiro256StarStar::jumpLong

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::jumpLong — Efficiently move the engine ahead by 2^192 steps

Description

public function **Random\Engine\Xoshiro256StarStar::jumpLong**(): void

Moves the algorithm's state ahead by 2^{192} steps, as if Random\Engine\Xoshiro256StarStar::generate() was called 2^{192} times.

The purpose of a long jump is to facilitate the creation of a new Random\Engine\Xoshiro256StarStar engine from an existing seeded Random\Engine\Xoshiro256StarStar engine. The seeded engine acts as a blueprint, which can be cloned and repeatedly jumped to create 2^{64} non-overlapping sequences with 2^{192} values each.

Long jumping may be combined with Random\Engine\Xoshiro256StarStar::jump()ing to further split each of the 2^{64} sequences generated by long jumping, into 2^{64} sequences of 2^{128} values each.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 Random\Engine\Xoshiro256StarStar::jumpLong() example

```php
<?php
$blueprintRng = new \Random\Engine\Xoshiro256StarStar(0);

// Each parent engine will have its own chunk of 2**192 values.
$parent1 = clone $blueprintRng;
$blueprintRng->jumpLong();

$parent2 = clone $blueprintRng;
$blueprintRng->jumpLong();

// Each of the child engines will have its own chunk of 2**128 values
// taken from their parent engine's chunk of 2**192 values.
$child1a = clone $parent1;
$parent1->jump();
$child1b = clone $parent1;
$parent1->jump();

$child2a = clone $parent2;
$parent2->jump();
$child2b = clone $parent2;
$parent2->jump();

echo "Child 1A: ", bin2hex($child1a->generate()), "\n";
echo "Child 1B: ", bin2hex($child1b->generate()), "\n";
echo "Child 2A: ", bin2hex($child2a->generate()), "\n";
echo "Child 2B: ", bin2hex($child2b->generate()), "\n";
?>
```

The above example will output:

```
Child 1A: b4f275cb365fec99
Child 1B: 2cd646c8ed156237
Child 2A: eb3729a722a504e7
Child 2B: d4208dc85bdd6dc3
```

See Also

- Random\Engine\Xoshiro256StarStar::jump() - Efficiently move the engine ahead by 2^128 steps

Random\Engine\Xoshiro256StarStar::__serialize

Random\Engine\Xoshiro256StarStar::__serialize

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__serialize — Serializes the Xoshiro256StarStar object

Description

public function **Random\Engine\Xoshiro256StarStar::__serialize**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Examples

Example #1 Random\Engine\Xoshiro256StarStar::__serialize() example

```php
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\Engine\Xoshiro256StarStar::__unserialize

Random\Engine\Xoshiro256StarStar::__unserialize

(PHP 8 >= 8.2.0)

Random\Engine\Xoshiro256StarStar::__unserialize — Deserializes the `data` parameter into a Xoshiro256StarStar object

Description

public function **Random\Engine\Xoshiro256StarStar::__unserialize**(array `$data`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

Return Values

No value is returned.

Examples

Example #1 Random\Engine\Xoshiro256StarStar::__unserialize() example

```php
<?php

/* ... */

?>
```

The above example will output something similar to:

```
...
```

Random\RandomError

(PHP 8 >= 8.2.0)

Introduction

The base class for Errors that occur during generation or use of randomness.

Class synopsis

namespace **Random**;
class **RandomError** extends Error {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Random\BrokenRandomEngineError

(PHP 8 >= 8.2.0)

Introduction

Indicates that the used Random\Engine is broken, e.g. because it is severely biased.

Class synopsis

namespace **Random**;
class **BrokenRandomEngineError** extends Random\RandomError {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;

protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Random\RandomException

(PHP 8 >= 8.2.0)

Introduction

The base class for Exceptions that occur during generation or use of randomness.

Class synopsis

namespace **Random**;
class **RandomException** extends Exception {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Standard PHP Library (SPL)

Introduction

The Standard PHP Library (SPL) extension defines interfaces and classes that are meant to solve common problems.

The extension provides various datastructures, generic exceptions, iterators, classes to work with filesystems using an object oriented API, and utility functions.

Interfaces

Table of Contents

SPL provides a set of interfaces to enhance iterators, and a pair of interfaces to implement the Observer Design Pattern.

See Also

- Predefined Interfaces and Classes

OuterIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Classes implementing **OuterIterator** can be used to iterate over iterators.

Interface synopsis

interface **OuterIterator** extends Iterator {
/* Methods */
public function getInnerIterator(): ?Iterator
/* Inherited methods */
public function Iterator::current(): mixed
public function Iterator::key(): mixed
public function Iterator::next(): void
public function Iterator::rewind(): void
public function Iterator::valid(): bool
}

Table of Contents

OuterIterator::getInnerIterator

OuterIterator::getInnerIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

OuterIterator::getInnerIterator — Returns the inner iterator for the current entry

Description

public function **OuterIterator::getInnerIterator**(): ?Iterator

Returns the inner iterator for the current iterator entry.

Parameters

This function has no parameters.

Return Values

Returns the inner iterator for the current entry if it exists, or `null` otherwise.

RecursiveIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Classes implementing **RecursiveIterator** can be used to iterate over iterators recursively.

Interface synopsis

```
interface RecursiveIterator extends Iterator {
/* Methods */
public function getChildren(): ?RecursiveIterator
public function hasChildren(): bool
/* Inherited methods */
public function Iterator::current(): mixed
public function Iterator::key(): mixed
public function Iterator::next(): void
public function Iterator::rewind(): void
public function Iterator::valid(): bool
}
```

Table of Contents

RecursiveIterator::getChildren

RecursiveIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIterator::getChildren — Returns an iterator for the current entry

Description

public function **RecursiveIterator::getChildren**(): ?RecursiveIterator

Returns an iterator for the current iterator entry.

Parameters

This function has no parameters.

Return Values

Returns an iterator for the current entry if it exists, or `null` otherwise.

See Also

- RecursiveIterator::hasChildren() - Returns if an iterator can be created for the current entry

RecursiveIterator::hasChildren

RecursiveIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIterator::hasChildren — Returns if an iterator can be created for the current entry

Description

public function **RecursiveIterator::hasChildren**(): bool

Returns if an iterator can be created for the current entry. RecursiveIterator::getChildren().

Parameters

This function has no parameters.

Return Values

Returns `true` if the current entry can be iterated over, otherwise returns `false`.

See Also

- RecursiveIterator::getChildren() - Returns an iterator for the current entry

SeekableIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The Seekable iterator.

Interface synopsis

interface **SeekableIterator** extends Iterator {
/* Methods */
public function seek(int `$offset`): void
/* Inherited methods */
public function Iterator::current(): mixed
public function Iterator::key(): mixed
public function Iterator::next(): void
public function Iterator::rewind(): void
public function Iterator::valid(): bool
}

Examples

Example #1 Basic usage

This example demonstrates creating a custom **SeekableIterator**, seeking to a position and handling an invalid position.

```php
<?php
class MySeekableIterator implements SeekableIterator {

    private $position;

    private $array = array(
        "first element",
        "second element",
        "third element",
        "fourth element"
    );

    /* Method required for SeekableIterator interface */

    public function seek($position) {
      if (!isset($this->array[$position])) {
          throw new OutOfBoundsException("invalid seek position ($position)");
      }

      $this->position = $position;
    }

    /* Methods required for Iterator interface */

    public function rewind() {
        $this->position = 0;
    }

    public function current() {
        return $this->array[$this->position];
    }

    public function key() {
        return $this->position;
    }

    public function next() {
        ++$this->position;
    }

    public function valid() {
```

```
        return isset($this->array[$this->position]);
    }
}

try {

    $it = new MySeekableIterator;
    echo $it->current(), "\n";

    $it->seek(2);
    echo $it->current(), "\n";

    $it->seek(1);
    echo $it->current(), "\n";

    $it->seek(10);

} catch (OutOfBoundsException $e) {
    echo $e->getMessage();
}
?>
```

The above example will output something similar to:

```
first element
third element
second element
invalid seek position (10)
```

Table of Contents

SeekableIterator::seek

SeekableIterator::seek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SeekableIterator::seek — Seeks to a position

Description

public function **SeekableIterator::seek**(int `$offset`): void

Seeks to a given position in the iterator.

Parameters

`offset`

The position to seek to.

Return Values

No value is returned.

Errors/Exceptions

Implementations should throw an OutOfBoundsException if the `offset` is not seekable.

Examples

Example #1 SeekableIterator::seek() example

Seek to the item at position 3 in the iterator (ArrayIterator implements SeekableIterator).

```
<?php
$array = array("apple", "banana", "cherry", "damson", "elderberry");
```

```php
$iterator = new ArrayIterator($array);
$iterator->seek(3);
echo $iterator->current();
?>
```

The above example will output something similar to:

```
damson
```

See Also

- SeekableIterator
- Iterator

SplObserver

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The **SplObserver** interface is used alongside SplSubject to implement the Observer Design Pattern.

Interface synopsis

interface **SplObserver** {
/* Methods */
public function update(SplSubject `$subject`): void
}

Table of Contents

SplObserver::update

SplObserver::update

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObserver::update — Receive update from subject

Description

public function **SplObserver::update**(SplSubject `$subject`): void

This method is called when any SplSubject to which the observer is attached calls SplSubject::notify().

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`subject`

The SplSubject notifying the observer of an update.

Return Values

No value is returned.

SplSubject

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The **SplSubject** interface is used alongside SplObserver to implement the Observer Design Pattern.

Interface synopsis

interface **SplSubject** {
/* Methods */
public function attach(SplObserver `$observer`): void
public function detach(SplObserver `$observer`): void
public function notify(): void
}

Table of Contents

SplSubject::attach

SplSubject::attach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplSubject::attach — Attach an SplObserver

Description

public function **SplSubject::attach**(SplObserver `$observer`): void

Attaches an SplObserver so that it can be notified of updates.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`observer`

The SplObserver to attach.

Return Values

No value is returned.

SplSubject::detach

SplSubject::detach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplSubject::detach — Detach an observer

Description

public function **SplSubject::detach**(SplObserver `$observer`): void

Detaches an observer from the subject to no longer notify it of updates.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`observer`

The SplObserver to detach.

Return Values

No value is returned.

SplSubject::notify

SplSubject::notify

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplSubject::notify — Notify an observer

Description

public function **SplSubject::notify**(): void

Notifies all attached observers.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

Datastructures

Table of Contents

SPL provides a set of standard datastructures. They are grouped here by their underlying implementation which usually defines their general field of application.

Doubly Linked Lists

A Doubly Linked List (DLL) is a list of nodes linked in both directions to each other. Iterator's operations, access to both ends, addition or removal of nodes have a cost of O(1) when the underlying structure is a DLL. It hence provides a decent implementation for stacks and queues.

- SplDoublyLinkedList
    - SplStack
    - SplQueue

Heaps

Heaps are tree-like structures that follow the heap-property: each node is greater than or equal to its children, when compared using the implemented compare method which is global to the heap.

- SplHeap
    - SplMaxHeap
    - SplMinHeap
- SplPriorityQueue

Arrays

Arrays are structures that store the data in a contiguous way, accessible via indexes.

> **Note** :
>
> Do not confuse this with PHP's native array type. PHP arrays are in reality ordered hashtables. However, SPL provides the ArrayObject class to wrap PHP arrays into an object.

- SplFixedArray

Map

A map is a datastructure holding key-value pairs. PHP arrays can be seen as maps from integers/strings to values. SPL provides a map from objects to data. This map can also be used as an object set.

- SplObjectStorage

SplDoublyLinkedList

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplDoublyLinkedList class provides the main functionalities of a doubly linked list.

Class synopsis

class **SplDoublyLinkedList** implements Iterator, Countable, ArrayAccess, Serializable {
/* Constants */
public const int *IT_MODE_LIFO*;
public const int *IT_MODE_FIFO*;
public const int *IT_MODE_DELETE*;
public const int *IT_MODE_KEEP*;
/* Methods */
public function add(int `$index`, mixed `$value`): void
public function bottom(): mixed
public function count(): int
public function current(): mixed
public function getIteratorMode(): int
public function isEmpty(): bool
public function key(): int
public function next(): void
public function offsetExists(int `$index`): bool
public function offsetGet(int `$index`): mixed
public function offsetSet(?int `$index`, mixed `$value`): void
public function offsetUnset(int `$index`): void
public function pop(): mixed
public function prev(): void
public function push(mixed `$value`): void
public function rewind(): void
public function serialize(): string
public function setIteratorMode(int `$mode`): int
public function shift(): mixed
public function top(): mixed
public function unserialize(string `$data`): void
public function unshift(mixed `$value`): void
public function valid(): bool
}

Predefined Constants

Iteration Direction

`SplDoublyLinkedList::IT_MODE_LIFO`

The list will be iterated in a last in, first out order, like a stack.

`SplDoublyLinkedList::IT_MODE_FIFO`

The list will be iterated in a first in, first out order, like a queue.

Iteration Behavior

`SplDoublyLinkedList::IT_MODE_DELETE`

Iteration will remove the iterated elements.

`SplDoublyLinkedList::IT_MODE_KEEP`

Iteration will not remove the iterated elements.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

SplDoublyLinkedList::add

SplDoublyLinkedList::add

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

SplDoublyLinkedList::add — Add/insert a new value at the specified index

Description

public function **SplDoublyLinkedList::add**(int `$index`, mixed `$value`): void

Insert the value `value` at the specified `index`, shuffling the previous value at that index (and all subsequent values) up through the list.

Parameters

`index`

The index where the new value is to be inserted.

`value`

The new value for the `index`.

Return Values

No value is returned.

Errors/Exceptions

Throws OutOfRangeException when `index` is out of bounds or when `index` cannot be parsed as an integer.

SplDoublyLinkedList::bottom

SplDoublyLinkedList::bottom

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::bottom — Peeks at the node from the beginning of the doubly linked list

Description

public function **SplDoublyLinkedList::bottom**(): mixed

Parameters

This function has no parameters.

Return Values

The value of the first node.

Errors/Exceptions

Throws RuntimeException when the data-structure is empty.

SplDoublyLinkedList::count

SplDoublyLinkedList::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::count — Counts the number of elements in the doubly linked list

Description

public function **SplDoublyLinkedList::count**(): int

Parameters

This function has no parameters.

Return Values

Returns the number of elements in the doubly linked list.

SplDoublyLinkedList::current

SplDoublyLinkedList::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::current — Return current array entry

Description

public function **SplDoublyLinkedList::current**(): mixed

Get the current doubly linked list node.

Parameters

This function has no parameters.

Return Values

The current node value.

SplDoublyLinkedList::getIteratorMode

SplDoublyLinkedList::getIteratorMode

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::getIteratorMode — Returns the mode of iteration

Description

public function **SplDoublyLinkedList::getIteratorMode**(): int

Parameters

This function has no parameters.

Return Values

Returns the different modes and flags that affect the iteration.

SplDoublyLinkedList::isEmpty

SplDoublyLinkedList::isEmpty

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::isEmpty — Checks whether the doubly linked list is empty

Description

public function **SplDoublyLinkedList::isEmpty**(): bool

Parameters

This function has no parameters.

Return Values

Returns whether the doubly linked list is empty.

SplDoublyLinkedList::key

SplDoublyLinkedList::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::key — Return current node index

Description

public function **SplDoublyLinkedList::key**(): int

This function returns the current node index

Parameters

This function has no parameters.

Return Values

The current node index.

SplDoublyLinkedList::next

SplDoublyLinkedList::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::next — Move to next entry

Description

public function **SplDoublyLinkedList::next**(): void

Move the iterator to the next node.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplDoublyLinkedList::offsetExists

SplDoublyLinkedList::offsetExists

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetExists — Returns whether the requested $index exists

Description

public function **SplDoublyLinkedList::offsetExists**(int `$index`): bool

Parameters

`index`

The index being checked.

Return Values

`true` if the requested `index` exists, otherwise `false`

SplDoublyLinkedList::offsetGet

SplDoublyLinkedList::offsetGet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetGet — Returns the value at the specified $index

Description

public function **SplDoublyLinkedList::offsetGet**(int `$index`): mixed

Parameters

`index`

The index with the value.

Return Values

The value at the specified `index`.

Errors/Exceptions

Throws OutOfRangeException when `index` is out of bounds or when `index` cannot be parsed as an integer.

SplDoublyLinkedList::offsetSet

SplDoublyLinkedList::offsetSet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetSet — Sets the value at the specified $index to $value

Description

public function **SplDoublyLinkedList::offsetSet**(?int `$index`, mixed `$value`): void

Sets the value at the specified `index` to `value`.

Parameters

`index`

The index being set. If `null`, the next value will be added after the last item.

`value`

The new value for the `index`.

Return Values

No value is returned.

Errors/Exceptions

Throws OutOfRangeException when `index` is out of bounds or when `index` cannot be parsed as an integer.

SplDoublyLinkedList::offsetUnset

SplDoublyLinkedList::offsetUnset

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::offsetUnset — Unsets the value at the specified $index

Description

public function **SplDoublyLinkedList::offsetUnset**(int `$index`): void

Unsets the value at the specified index.

Parameters

`index`

The index being unset.

Return Values

No value is returned.

Errors/Exceptions

Throws OutOfRangeException when `index` is out of bounds or when `index` cannot be parsed as an integer.

SplDoublyLinkedList::pop

SplDoublyLinkedList::pop

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::pop — Pops a node from the end of the doubly linked list

Description

public function **SplDoublyLinkedList::pop**(): mixed

Parameters

This function has no parameters.

Return Values

The value of the popped node.

Errors/Exceptions

Throws RuntimeException when the data-structure is empty.

SplDoublyLinkedList::prev

SplDoublyLinkedList::prev

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::prev — Move to previous entry

Description

public function **SplDoublyLinkedList::prev**(): void

Move the iterator to the previous node.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplDoublyLinkedList::push

SplDoublyLinkedList::push

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::push — Pushes an element at the end of the doubly linked list

Description

public function **SplDoublyLinkedList::push**(mixed `$value`): void

Pushes `value` at the end of the doubly linked list.

Parameters

`value`

The value to push.

Return Values

No value is returned.

SplDoublyLinkedList::rewind

SplDoublyLinkedList::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::rewind — Rewind iterator back to the start

Description

public function **SplDoublyLinkedList::rewind**(): void

This rewinds the iterator to the beginning.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplDoublyLinkedList::serialize

SplDoublyLinkedList::serialize

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplDoublyLinkedList::serialize — Serializes the storage

Description

public function **SplDoublyLinkedList::serialize**(): string

Serializes the storage.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The serialized string.

See Also

- SplDoublyLinkedList::unserialize() - Unserializes the storage

SplDoublyLinkedList::setIteratorMode

SplDoublyLinkedList::setIteratorMode

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::setIteratorMode — Sets the mode of iteration

Description

public function **SplDoublyLinkedList::setIteratorMode**(int $mode): int

Parameters

`mode`

There are two orthogonal sets of modes that can be set:

- The direction of the iteration (either one or the other):
  - `SplDoublyLinkedList::IT_MODE_LIFO` (Stack style)
  - `SplDoublyLinkedList::IT_MODE_FIFO` (Queue style)
- The behavior of the iterator (either one or the other):
  - `SplDoublyLinkedList::IT_MODE_DELETE` (Elements are deleted by the iterator)
  - `SplDoublyLinkedList::IT_MODE_KEEP` (Elements are traversed by the iterator)

The default mode is: `SplDoublyLinkedList::IT_MODE_FIFO` | `SplDoublyLinkedList::IT_MODE_KEEP`

> **Warning**
>
> The direction of iteration can not be changed for SplStack and SplQueue classes, it is always `SplDoublyLinkedList::IT_MODE_FIFO`. Trying to modify it will result in a RuntimeException being thrown.

Return Values

Returns the different modes and flags that affect the iteration.

SplDoublyLinkedList::shift

SplDoublyLinkedList::shift

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::shift — Shifts a node from the beginning of the doubly linked list

Description

public function **SplDoublyLinkedList::shift**(): mixed

Parameters

This function has no parameters.

Return Values

The value of the shifted node.

Errors/Exceptions

Throws RuntimeException when the data-structure is empty.

SplDoublyLinkedList::top

SplDoublyLinkedList::top

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::top — Peeks at the node from the end of the doubly linked list

Description

public function **SplDoublyLinkedList::top**(): mixed

Parameters

This function has no parameters.

Return Values

The value of the last node.

Errors/Exceptions

Throws RuntimeException when the data-structure is empty.

SplDoublyLinkedList::unserialize

SplDoublyLinkedList::unserialize

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplDoublyLinkedList::unserialize — Unserializes the storage

Description

public function **SplDoublyLinkedList::unserialize**(string `$data`): void

Unserializes the storage, from SplDoublyLinkedList::serialize().

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

The serialized string.

Return Values

No value is returned.

See Also

- SplDoublyLinkedList::serialize() - Serializes the storage

SplDoublyLinkedList::unshift

SplDoublyLinkedList::unshift

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::unshift — Prepends the doubly linked list with an element

Description

public function **SplDoublyLinkedList::unshift**(mixed `$value`): void

Prepends `value` at the beginning of the doubly linked list.

Parameters

`value`

The value to unshift.

Return Values

No value is returned.

SplDoublyLinkedList::valid

SplDoublyLinkedList::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplDoublyLinkedList::valid — Check whether the doubly linked list contains more nodes

Description

public function **SplDoublyLinkedList::valid**(): bool

Checks if the doubly linked list contains any more nodes.

Parameters

This function has no parameters.

Return Values

Returns `true` if the doubly linked list contains any more nodes, `false` otherwise.

SplStack

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplStack class provides the main functionalities of a stack implemented using a doubly linked list by setting the iterator mode to `SplDoublyLinkedList::IT_MODE_LIFO`.

Class synopsis

class **SplStack** extends SplDoublyLinkedList {
/* Inherited constants */
public const int *SplDoublyLinkedList::IT_MODE_LIFO*;
public const int *SplDoublyLinkedList::IT_MODE_FIFO*;
public const int *SplDoublyLinkedList::IT_MODE_DELETE*;
public const int *SplDoublyLinkedList::IT_MODE_KEEP*;
/* Inherited methods */
public function SplDoublyLinkedList::add(int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::bottom(): mixed
public function SplDoublyLinkedList::count(): int
public function SplDoublyLinkedList::current(): mixed
public function SplDoublyLinkedList::getIteratorMode(): int
public function SplDoublyLinkedList::isEmpty(): bool
public function SplDoublyLinkedList::key(): int
public function SplDoublyLinkedList::next(): void
public function SplDoublyLinkedList::offsetExists(int `$index`): bool
public function SplDoublyLinkedList::offsetGet(int `$index`): mixed
public function SplDoublyLinkedList::offsetSet(?int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::offsetUnset(int `$index`): void
public function SplDoublyLinkedList::pop(): mixed
public function SplDoublyLinkedList::prev(): void
public function SplDoublyLinkedList::push(mixed `$value`): void
public function SplDoublyLinkedList::rewind(): void
public function SplDoublyLinkedList::serialize(): string
public function SplDoublyLinkedList::setIteratorMode(int `$mode`): int
public function SplDoublyLinkedList::shift(): mixed
public function SplDoublyLinkedList::top(): mixed
public function SplDoublyLinkedList::unserialize(string `$data`): void
public function SplDoublyLinkedList::unshift(mixed `$value`): void
public function SplDoublyLinkedList::valid(): bool
}

Examples

Example #1 **SplStack** example

```php
<?php
$q = new SplStack();
$q[] = 1;
$q[] = 2;
$q[] = 3;
foreach ($q as $elem)  {
```

```
 echo $elem."\n";
}
?>
```

The above example will output:

```
3
2
1
```

SplQueue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplQueue class provides the main functionalities of a queue implemented using a doubly linked list by setting the iterator mode to `SplDoublyLinkedList::IT_MODE_FIFO`.

Class synopsis

class **SplQueue** extends SplDoublyLinkedList {
/* Inherited constants */
public const int *SplDoublyLinkedList::IT_MODE_LIFO*;
public const int *SplDoublyLinkedList::IT_MODE_FIFO*;
public const int *SplDoublyLinkedList::IT_MODE_DELETE*;
public const int *SplDoublyLinkedList::IT_MODE_KEEP*;
/* Methods */
public function dequeue(): mixed
public function enqueue(mixed `$value`): void
/* Inherited methods */
public function SplDoublyLinkedList::add(int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::bottom(): mixed
public function SplDoublyLinkedList::count(): int
public function SplDoublyLinkedList::current(): mixed
public function SplDoublyLinkedList::getIteratorMode(): int
public function SplDoublyLinkedList::isEmpty(): bool
public function SplDoublyLinkedList::key(): int
public function SplDoublyLinkedList::next(): void
public function SplDoublyLinkedList::offsetExists(int `$index`): bool
public function SplDoublyLinkedList::offsetGet(int `$index`): mixed
public function SplDoublyLinkedList::offsetSet(?int `$index`, mixed `$value`): void
public function SplDoublyLinkedList::offsetUnset(int `$index`): void
public function SplDoublyLinkedList::pop(): mixed
public function SplDoublyLinkedList::prev(): void
public function SplDoublyLinkedList::push(mixed `$value`): void
public function SplDoublyLinkedList::rewind(): void
public function SplDoublyLinkedList::serialize(): string
public function SplDoublyLinkedList::setIteratorMode(int `$mode`): int
public function SplDoublyLinkedList::shift(): mixed
public function SplDoublyLinkedList::top(): mixed
public function SplDoublyLinkedList::unserialize(string `$data`): void
public function SplDoublyLinkedList::unshift(mixed `$value`): void
public function SplDoublyLinkedList::valid(): bool
}

Examples

Example #1 SplQueue example

```php
<?php
$q = new SplQueue();
$q[] = 1;
$q[] = 2;
$q[] = 3;
foreach ($q as $elem)  {
 echo $elem."\n";
}
?>
```

The above example will output:

```
1
2
3
```

Example #2 Efficiently handling tasks with SplQueue

```php
<?php
$q = new SplQueue();
$q->setIteratorMode(SplQueue::IT_MODE_DELETE);
// ... enqueue some tasks on the queue ...
// process them
foreach ($q as $task) {
    // ... process $task ...
    // add new tasks on the queue
    $q[] = $newTask;
    // ...
}
?>
```

Table of Contents

SplQueue::dequeue

SplQueue::dequeue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplQueue::dequeue — Dequeues a node from the queue

Description

public function **SplQueue::dequeue**(): mixed

Dequeues `value` from the top of the queue.

> **Note** :
>
> **SplQueue::dequeue()** is an alias of SplDoublyLinkedList::shift().

Parameters

This function has no parameters.

Return Values

The value of the dequeued node.

SplQueue::enqueue

SplQueue::enqueue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplQueue::enqueue — Adds an element to the queue

Description

public function **SplQueue::enqueue**(mixed `$value`): void

Enqueues `value` at the end of the queue.

> **Note** :
>
> **SplQueue::enqueue()** is an alias of SplDoublyLinkedList::push().

Parameters

`value`

The value to enqueue.

Return Values

No value is returned.

SplHeap

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplHeap class provides the main functionalities of a Heap.

Class synopsis

abstract class **SplHeap** implements Iterator, Countable {
/* Methods */
protected function compare(mixed `$value1`, mixed `$value2`): int
public function count(): int
public function current(): mixed
public function extract(): mixed
public function insert(mixed `$value`): true
public function isCorrupted(): bool
public function isEmpty(): bool
public function key(): int
public function next(): void
public function recoverFromCorruption(): true
public function rewind(): void
public function top(): mixed
public function valid(): bool
}

Table of Contents

SplHeap::compare

SplHeap::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::compare — Compare elements in order to place them correctly in the heap while sifting up

Description

protected function **SplHeap::compare**(mixed `$value1`, mixed `$value2`): int

Compare `value1` with `value2`.

> **Warning**

> Throwing exceptions in **SplHeap::compare()** can corrupt the Heap and place it in a blocked state. You can unblock it by calling SplHeap::recoverFromCorruption(). However, some elements might not be placed correctly and it may hence break the heap-property.

Parameters

`value1`

The value of the first node being compared.

`value2`

The value of the second node being compared.

Return Values

Result of the comparison, positive integer if `value1` is greater than `value2`, 0 if they are equal, negative integer otherwise.

> **Note** :
>
> Having multiple elements with the same value in a Heap is not recommended. They will end up in an arbitrary relative position.

SplHeap::count

SplHeap::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::count — Counts the number of elements in the heap

Description

public function **SplHeap::count**(): int

Parameters

This function has no parameters.

Return Values

Returns the number of elements in the heap.

SplHeap::current

SplHeap::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::current — Return current node pointed by the iterator

Description

public function **SplHeap::current**(): mixed

Get the current datastructure node.

Parameters

This function has no parameters.

Return Values

The current node value.

SplHeap::extract

SplHeap::extract

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::extract — Extracts a node from top of the heap and sift up

Description

public function **SplHeap::extract**(): mixed

Parameters

This function has no parameters.

Return Values

The value of the extracted node.

Errors/Exceptions

Throws RuntimeException when the data-structure is empty.

SplHeap::insert

SplHeap::insert

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::insert — Inserts an element in the heap by sifting it up

Description

public function **SplHeap::insert**(mixed `$value`): true

Insert `value` in the heap.

Parameters

`value`

The value to insert.

Return Values

Always returns **`true`**.

Changelog

Version	Description
8.4.0	**SplHeap::insert()** now has a tentative return of true.

SplHeap::isCorrupted

SplHeap::isCorrupted

(PHP 7, PHP 8)

SplHeap::isCorrupted — Tells if the heap is in a corrupted state

Description

public function **SplHeap::isCorrupted**(): bool

Parameters

This function has no parameters.

Return Values

Returns **`true`** if the heap is corrupted, **`false`** otherwise.

SplHeap::isEmpty

SplHeap::isEmpty

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::isEmpty — Checks whether the heap is empty

Description

public function **SplHeap::isEmpty**(): bool

Parameters

This function has no parameters.

Return Values

Returns whether the heap is empty.

SplHeap::key

SplHeap::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::key — Return current node index

Description

public function **SplHeap::key**(): int

This function returns the current node index

Parameters

This function has no parameters.

Return Values

The current node index.

SplHeap::next

SplHeap::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::next — Move to the next node

Description

public function **SplHeap::next**(): void

Move to the next node. This will delete the top node of the heap.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplHeap::recoverFromCorruption

SplHeap::recoverFromCorruption

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::recoverFromCorruption — Recover from the corrupted state and allow further actions on the heap

Description

public function **SplHeap::recoverFromCorruption**(): true

Parameters

This function has no parameters.

Return Values

Always returns `true`.

Changelog

Version	Description
8.4.0	**SplHeap::recoverFromCorruption()** now has a tentative return of true.

SplHeap::rewind

SplHeap::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::rewind — Rewind iterator back to the start (no-op)

Description

public function **SplHeap::rewind**(): void

This rewinds the iterator to the beginning. This is a no-op for heaps as the iterator is virtual and in fact never moves from the top of the heap.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplHeap::top

SplHeap::top

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::top — Peeks at the node from the top of the heap

Description

public function **SplHeap::top**(): mixed

Parameters

This function has no parameters.

Return Values

The value of the node on the top.

Errors/Exceptions

Throws RuntimeException when the data-structure is empty.

SplHeap::valid

SplHeap::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplHeap::valid — Check whether the heap contains more nodes

Description

public function **SplHeap::valid**(): bool

Checks if the heap contains any more nodes.

Parameters

This function has no parameters.

Return Values

Returns `true` if the heap contains any more nodes, `false` otherwise.

SplMaxHeap

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplMaxHeap class provides the main functionalities of a heap, keeping the maximum on the top.

Class synopsis

class **SplMaxHeap** extends SplHeap {
/* Methods */
protected function compare(mixed `$value1`, mixed `$value2`): int
/* Inherited methods */
protected function SplHeap::compare(mixed `$value1`, mixed `$value2`): int
public function SplHeap::count(): int
public function SplHeap::current(): mixed
public function SplHeap::extract(): mixed
public function SplHeap::insert(mixed `$value`): true
public function SplHeap::isCorrupted(): bool
public function SplHeap::isEmpty(): bool
public function SplHeap::key(): int
public function SplHeap::next(): void
public function SplHeap::recoverFromCorruption(): true
public function SplHeap::rewind(): void
public function SplHeap::top(): mixed
public function SplHeap::valid(): bool
}

Table of Contents

SplMaxHeap::compare

SplMaxHeap::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplMaxHeap::compare — Compare elements in order to place them correctly in the heap while sifting up

Description

protected function **SplMaxHeap::compare**(mixed `$value1`, mixed `$value2`): int

Compare `value1` with `value2`.

Parameters

`value1`

The value of the first node being compared.

`value2`

The value of the second node being compared.

Return Values

Result of the comparison, positive integer if `value1` is greater than `value2`, 0 if they are equal, negative integer otherwise.

> **Note** :

> Having multiple elements with the same value in a Heap is not recommended. They will end up in an arbitrary relative position.

SplMinHeap

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplMinHeap class provides the main functionalities of a heap, keeping the minimum on the top.

Class synopsis

class **SplMinHeap** extends SplHeap {
/* Methods */
protected function compare(mixed `$value1`, mixed `$value2`): int
/* Inherited methods */
protected function SplHeap::compare(mixed `$value1`, mixed `$value2`): int
public function SplHeap::count(): int
public function SplHeap::current(): mixed
public function SplHeap::extract(): mixed
public function SplHeap::insert(mixed `$value`): true
public function SplHeap::isCorrupted(): bool
public function SplHeap::isEmpty(): bool
public function SplHeap::key(): int
public function SplHeap::next(): void
public function SplHeap::recoverFromCorruption(): true
public function SplHeap::rewind(): void
public function SplHeap::top(): mixed
public function SplHeap::valid(): bool
}

Table of Contents

SplMinHeap::compare

SplMinHeap::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplMinHeap::compare — Compare elements in order to place them correctly in the heap while sifting up

Description

protected function **SplMinHeap::compare**(mixed `$value1`, mixed `$value2`): int

Compare `value1` with `value2`.

Parameters

`value1`

The value of the first node being compared.

`value2`

The value of the second node being compared.

Return Values

Result of the comparison, positive integer if `value1` is lower than `value2`, 0 if they are equal, negative integer otherwise.

> **Note** :
>
> Having multiple elements with the same value in a Heap is not recommended. They will end up in an arbitrary relative position.

SplPriorityQueue

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplPriorityQueue class provides the main functionalities of a prioritized queue, implemented using a max heap.

> **Note** :
>
> The order of elements with identical priority is *undefined*. It may differ from the order in which they have been inserted.

Class synopsis

class **SplPriorityQueue** implements Iterator, Countable {
/* Constants */
public const int *EXTR_BOTH*;
public const int *EXTR_PRIORITY*;
public const int *EXTR_DATA*;
/* Methods */
public function compare(mixed `$priority1`, mixed `$priority2`): int
public function count(): int
public function current(): mixed
public function extract(): mixed
public function getExtractFlags(): int
public function insert(mixed `$value`, mixed `$priority`): true
public function isCorrupted(): bool
public function isEmpty(): bool
public function key(): int
public function next(): void
public function recoverFromCorruption(): true
public function rewind(): void
public function setExtractFlags(int `$flags`): int
public function top(): mixed
public function valid(): bool
}

Predefined Constants

`SplPriorityQueue::EXTR_BOTH`

`SplPriorityQueue::EXTR_PRIORITY`

`SplPriorityQueue::EXTR_DATA`

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

SplPriorityQueue::compare

SplPriorityQueue::compare

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::compare — Compare priorities in order to place elements correctly in the heap while sifting up

Description

public function **SplPriorityQueue::compare**(mixed `$priority1`, mixed `$priority2`): int

Compare `priority1` with `priority2`.

Parameters

`priority1`

The priority of the first node being compared.

`priority2`

The priority of the second node being compared.

Return Values

Result of the comparison, positive integer if `priority1` is greater than `priority2`, 0 if they are equal, negative integer otherwise.

> **Note** :
>
> Multiple elements with the same priority will get dequeued in no particular order.

SplPriorityQueue::count

SplPriorityQueue::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::count — Counts the number of elements in the queue

Description

public function **SplPriorityQueue::count**(): int

Parameters

This function has no parameters.

Return Values

Returns the number of elements in the queue.

SplPriorityQueue::current

SplPriorityQueue::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::current — Return current node pointed by the iterator

Description

public function **SplPriorityQueue::current**(): mixed

Get the current datastructure node.

Parameters

This function has no parameters.

Return Values

The value or priority (or both) of the current node, depending on the extract flag.

SplPriorityQueue::extract

SplPriorityQueue::extract

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::extract — Extracts a node from top of the heap and sift up

Description

public function **SplPriorityQueue::extract**(): mixed

Parameters

This function has no parameters.

Return Values

The value or priority (or both) of the extracted node, depending on the extract flag.

SplPriorityQueue::getExtractFlags

SplPriorityQueue::getExtractFlags

(PHP 7, PHP 8)

SplPriorityQueue::getExtractFlags — Get the flags of extraction

Description

public function **SplPriorityQueue::getExtractFlags**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns the flags of extraction.

SplPriorityQueue::insert

SplPriorityQueue::insert

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::insert — Inserts an element in the queue by sifting it up

Description

public function **SplPriorityQueue::insert**(mixed `$value`, mixed `$priority`): true

Insert `value` with the priority `priority` in the queue.

Parameters

`value`

The value to insert.

`priority`

The associated priority.

Return Values

Always returns `true`.

Changelog

Version	Description
8.4.0	**SplPriorityQueue::insert()** now has a tentative return of true.

SplPriorityQueue::isCorrupted

SplPriorityQueue::isCorrupted

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::isCorrupted — Tells if the priority queue is in a corrupted state

Description

public function **SplPriorityQueue::isCorrupted**(): bool

Parameters

This function has no parameters.

Return Values

Returns `true` if the priority queue is corrupted, `false` otherwise.

SplPriorityQueue::isEmpty

SplPriorityQueue::isEmpty

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::isEmpty — Checks whether the queue is empty

Description

public function **SplPriorityQueue::isEmpty**(): bool

Parameters

This function has no parameters.

Return Values

Returns whether the queue is empty.

SplPriorityQueue::key

SplPriorityQueue::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::key — Return current node index

Description

public function **SplPriorityQueue::key**(): int

This function returns the current node index

Parameters

This function has no parameters.

Return Values

The current node index.

SplPriorityQueue::next

SplPriorityQueue::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::next — Move to the next node

Description

public function **SplPriorityQueue::next**(): void

Extracts the top node from the queue.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplPriorityQueue::recoverFromCorruption

SplPriorityQueue::recoverFromCorruption

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::recoverFromCorruption — Recover from the corrupted state and allow further actions on the queue

Description

public function **SplPriorityQueue::recoverFromCorruption**(): true

Parameters

This function has no parameters.

Return Values

Always returns `true`.

Changelog

Version	Description
8.4.0	**SplPriorityQueue::recoverFromCorruption()** now has a tentative return of true.

SplPriorityQueue::rewind

SplPriorityQueue::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::rewind — Rewind iterator back to the start (no-op)

Description

public function **SplPriorityQueue::rewind**(): void

This rewinds the iterator to the beginning. This is a no-op for heaps as the iterator is virtual and in fact never moves from the top of the heap.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplPriorityQueue::setExtractFlags

SplPriorityQueue::setExtractFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::setExtractFlags — Sets the mode of extraction

Description

public function **SplPriorityQueue::setExtractFlags**(int `$flags`): int

Parameters

`flags`

Defines what is extracted by SplPriorityQueue::current(), SplPriorityQueue::top() and SplPriorityQueue::extract().

- `SplPriorityQueue::EXTR_DATA` (0x00000001): Extract the data
- `SplPriorityQueue::EXTR_PRIORITY` (0x00000002): Extract the priority

- `SplPriorityQueue::EXTR_BOTH` (0x00000003): Extract an array containing both

The default mode is `SplPriorityQueue::EXTR_DATA`.

Return Values

Returns the flags of extraction.

SplPriorityQueue::top

SplPriorityQueue::top

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::top — Peeks at the node from the top of the queue

Description

public function **SplPriorityQueue::top**(): mixed

Parameters

This function has no parameters.

Return Values

The value or priority (or both) of the top node, depending on the extract flag.

SplPriorityQueue::valid

SplPriorityQueue::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplPriorityQueue::valid — Check whether the queue contains more nodes

Description

public function **SplPriorityQueue::valid**(): bool

Checks if the queue contains any more nodes.

Parameters

This function has no parameters.

Return Values

Returns `true` if the queue contains any more nodes, `false` otherwise.

SplFixedArray

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The SplFixedArray class provides the main functionalities of array. The main difference between a SplFixedArray and a normal PHP array is that the SplFixedArray must be resized manually and allows only integers within the range as indexes. The advantage is that it uses less memory than a standard array.

Class synopsis

class **SplFixedArray** implements IteratorAggregate, ArrayAccess, Countable, JsonSerializable {
/* Methods */
public function __construct(int `$size` = 0)
public function count(): int
public function current(): mixed
public static function fromArray(array `$array`, bool `$preserveKeys` = `true`): SplFixedArray
public function getIterator(): Iterator
public function getSize(): int
public function jsonSerialize(): array

```
public function key(): int
public function next(): void
public function offsetExists(int $index): bool
public function offsetGet(int $index): mixed
public function offsetSet(int $index, mixed $value): void
public function offsetUnset(int $index): void
public function rewind(): void
public function __serialize(): array
public function setSize(int $size): true
public function toArray(): array
public function __unserialize(array $data): void
public function valid(): bool
#[\Deprecated(
    since: '8.4',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function __wakeup(): void
}
```

Changelog

Version	Description
8.4.0	Out of bounds accesses in **SplFixedArray** now throw exceptions of type OutOfBoundsException instead of RuntimeException. Because OutOfBoundsException is a child class of RuntimeException no behavioural changes are exhibited when attempting to catch those exceptions.
8.2.0	The SplFixedArray::__serialize() and SplFixedArray::__unserialize() magic methods have been added to **SplFixedArray**.
8.1.0	Using a non-integer key on a **SplFixedArray** now throws a TypeError instead of a RuntimeException.
8.1.0	**SplFixedArray** implements JsonSerializable now.
8.0.0	**SplFixedArray** implements IteratorAggregate now. Previously, Iterator was implemented instead.

Examples

Example #1 **SplFixedArray** usage example

```php
<?php
// Initialize the array with a fixed length
$array = new SplFixedArray(5);

$array[1] = 2;
$array[4] = "foo";

var_dump($array[0]); // NULL
var_dump($array[1]); // int(2)

var_dump($array["4"]); // string(3) "foo"

// Increase the size of the array to 10
$array->setSize(10);

$array[9] = "asdf";

// Shrink the array to a size of 2
$array->setSize(2);

// The following two blocks throw a RuntimeException: Index invalid or out of range
try {
    var_dump($array[-1]);
} catch(RuntimeException $e) {
    echo "RuntimeException: ".$e->getMessage()."\n";
}

try {
    var_dump($array[5]);
} catch(RuntimeException $e) {
    echo "RuntimeException: ".$e->getMessage()."\n";
}
```

```php
// As of PHP 8.1.0, using a non-integer key throws a TypeError.
// Prior to PHP 8.1.0, a RuntimeException was thrown instead.
try {
    var_dump($array["non-numeric"]);
} catch(TypeError $e) {
    echo "TypeError: ".$e->getMessage()."\n";
}
?>
```

The above example will output:

```
NULL
int(2)
string(3) "foo"
RuntimeException: Index invalid or out of range
RuntimeException: Index invalid or out of range
TypeError: Illegal offset type
```

> **Note** :
>
> Prior to PHP 8.1.0, accessing a **SplFixedArray** with a non-integer key threw a RuntimeException instead of a TypeError.

Table of Contents

SplFixedArray::__construct

SplFixedArray::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::__construct — Constructs a new fixed array

Description

public function **SplFixedArray::__construct**(int `$size` = 0)

Initializes a fixed array with a number of `null` values equal to `size`.

Parameters

`size`

The size of the fixed array. This expects a number between `0` and `PHP_INT_MAX`.

Errors/Exceptions

Throws a ValueError when `size` is a negative integer.

Changelog

Version	Description
8.0.0	Now throws a ValueError if `size` is a negative; previously it threw a InvalidArgumentException.

Examples

Example #1 SplFixedArray::__construct() example

```php
<?php
$array = new SplFixedArray(5);

$array[1] = 2;
$array[4] = "foo";
```

```php
foreach($array as $v) {
  var_dump($v);
}
?>
```

The above example will output:

```
NULL
int(2)
NULL
NULL
string(3) "foo"
```

SplFixedArray::count

SplFixedArray::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::count — Returns the size of the array

Description

public function **SplFixedArray::count**(): int

Returns the size of the array.

Parameters

This function has no parameters.

Return Values

Returns the size of the array.

Examples

Example #1 SplFixedArray::count() example

```php
<?php
$array = new SplFixedArray(5);
echo $array->count() . "\n";
echo count($array) . "\n";
?>
```

The above example will output:

```
5
5
```

Notes

> **Note** :
>
> This method is functionally equivalent to SplFixedArray::getSize().

> **Note** :
>
> The count of elements is always equal to the set size because all values are initially initialized with `null`.

See Also

- SplFixedArray::getSize() - Gets the size of the array

SplFixedArray::current

SplFixedArray::current

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::current — Return current array entry

Description

public function **SplFixedArray::current**(): mixed

Get the current array element.

Parameters

This function has no parameters.

Return Values

The current element value.

Errors/Exceptions

Throws RuntimeException when the internal array pointer points to an invalid index or is out of bounds.

SplFixedArray::fromArray

SplFixedArray::fromArray

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::fromArray — Import a PHP array in a SplFixedArray instance

Description

public static function **SplFixedArray::fromArray**(array `$array`, bool `$preserveKeys = true`): SplFixedArray

Import the PHP array `array` in a new SplFixedArray instance

Parameters

`array`

The array to import.

`preserveKeys`

Try to save the numeric indexes used in the original array.

Return Values

Returns an instance of SplFixedArray containing the array content.

Examples

Example #1 SplFixedArray::fromArray() example

```php
<?php
$fa = SplFixedArray::fromArray(array(1 => 1, 0 => 2, 3 => 3));

var_dump($fa);
```

```php
$fa = SplFixedArray::fromArray(array(1 => 1, 0 => 2, 3 => 3), false);

var_dump($fa);
?>
```

The above example will output:

```
object(SplFixedArray)#1 (4) {
  [0]=>
  int(2)
  [1]=>
  int(1)
  [2]=>
  NULL
  [3]=>
  int(3)
}
object(SplFixedArray)#2 (3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

SplFixedArray::getIterator

SplFixedArray::getIterator

(PHP 8)

SplFixedArray::getIterator — Retrieve the iterator to go through the array

Description

public function **SplFixedArray::getIterator**(): Iterator

Returns the iterator to go through the array.

Parameters

This function has no parameters.

Return Values

An instance of an object implementing Iterator to go through the array.

SplFixedArray::getSize

SplFixedArray::getSize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::getSize — Gets the size of the array

Description

public function **SplFixedArray::getSize**(): int

Gets the size of the array.

Parameters

This function has no parameters.

Return Values

Returns the size of the array, as an int.

Examples

Example #1 SplFixedArray::getSize() example

```php
<?php
$array = new SplFixedArray(5);
echo $array->getSize()."\n";
$array->setSize(10);
echo $array->getSize()."\n";
?>
```

The above example will output:

```
5
10
```

Notes

> **Note** :
>
> This method is functionally equivalent to SplFixedArray::count()

See Also

- SplFixedArray::count() - Returns the size of the array

SplFixedArray::jsonSerialize

SplFixedArray::jsonSerialize

(PHP 8 >= 8.1.0)

SplFixedArray::jsonSerialize — Returns a representation that can be converted to JSON

Description

public function **SplFixedArray::jsonSerialize**(): array

Serializes the array to a value that can be serialized natively by json_encode().

Parameters

This function has no parameters.

Return Values

Returns array data which can be serialized by json_encode(), which is a value of any type other than a resource.

SplFixedArray::key

SplFixedArray::key

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::key — Return current array index

Description

public function **SplFixedArray::key**(): int

Returns the current array index.

Parameters

This function has no parameters.

Return Values

The current array index.

SplFixedArray::next

SplFixedArray::next

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::next — Move to next entry

Description

public function **SplFixedArray::next**(): void

Move the iterator to the next array entry.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplFixedArray::offsetExists

SplFixedArray::offsetExists

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetExists — Returns whether the requested index exists

Description

public function **SplFixedArray::offsetExists**(int `$index`): bool

Checks whether the requested index `index` exists.

Parameters

`index`

The index being checked.

Return Values

`true` if the requested `index` exists, otherwise `false`

SplFixedArray::offsetGet

SplFixedArray::offsetGet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetGet — Returns the value at the specified index

Description

public function **SplFixedArray::offsetGet**(int `$index`): mixed

Returns the value at the index `index`.

Parameters

`index`

The index with the value.

Return Values

The value at the specified `index`.

Errors/Exceptions

Throws RuntimeException when `index` is outside the defined size of the array or when `index` cannot be parsed as an integer.

SplFixedArray::offsetSet

SplFixedArray::offsetSet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetSet — Sets a new value at a specified index

Description

public function **SplFixedArray::offsetSet**(int `$index`, mixed `$value`): void

Sets the value at the specified `index` to `value`.

Parameters

`index`

The index being set.

`value`

The new value for the `index`.

Return Values

No value is returned.

Errors/Exceptions

Throws RuntimeException when `index` is outside the defined size of the array or when `index` cannot be parsed as an integer.

SplFixedArray::offsetUnset

SplFixedArray::offsetUnset

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::offsetUnset — Unsets the value at the specified $index

Description

public function **SplFixedArray::offsetUnset**(int `$index`): void

Unsets the value at the specified index.

Parameters

`index`

The index being unset.

Return Values

No value is returned.

Errors/Exceptions

Throws RuntimeException when `index` is outside the defined size of the array or when `index` cannot be parsed as an integer.

SplFixedArray::rewind

SplFixedArray::rewind

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::rewind — Rewind iterator back to the start

Description

public function **SplFixedArray::rewind**(): void

Rewinds the iterator to the beginning.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplFixedArray::__serialize

SplFixedArray::__serialize

(PHP 8 >= 8.2.0)

SplFixedArray::__serialize — Serializes the SplFixedArray object

Description

public function **SplFixedArray::__serialize**(): array

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

SplFixedArray::setSize

SplFixedArray::setSize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::setSize — Change the size of an array

Description

public function **SplFixedArray::setSize**(int `$size`): true

Change the size of an array to the new size of `size`. If `size` is less than the current array size, any values after the new size will be discarded. If `size` is greater than the current array size, the array will be padded with `null` values.

Parameters

`size`

The new array size. This should be a value between `0` and `PHP_INT_MAX`.

Return Values

Always returns `true`.

Errors/Exceptions

Throws ValueError when `size` is less than zero.

Changelog

Version	Description
8.4.0	**SplFixedArray::setSize()** now has a tentative return of true.

Examples

Example #1 SplFixedArray::setSize() example

```
<?php
   $array = new SplFixedArray(5);
   echo $array->getSize()."\n";
   $array->setSize(10);
   echo $array->getSize()."\n";
?>
```

The above example will output:

```
5
10
```

SplFixedArray::toArray

SplFixedArray::toArray

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplFixedArray::toArray — Returns a PHP array from the fixed array

Description

public function **SplFixedArray::toArray**(): array

Returns a PHP array from the fixed array.

Parameters

This function has no parameters.

Return Values

Returns a PHP array, similar to the fixed array.

Examples

Example #1 SplFixedArray::toArray() example

```
<?php
$fa = new SplFixedArray(3);
$fa[0] = 0;
$fa[2] = 2;
var_dump($fa->toArray());
?>
```

The above example will output:

```
array(3) {
  [0]=>
  int(0)
  [1]=>
```

```
  NULL
  [2]=>
  int(2)
}
```

SplFixedArray::__unserialize

SplFixedArray::__unserialize

(PHP 8 >= 8.2.0)

SplFixedArray::__unserialize — Deserializes the `data` parameter into an SplFixedArray object

Description

public function **SplFixedArray::__unserialize**(array `$data`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

The value being deserialized.

Return Values

No value is returned.

SplFixedArray::valid

SplFixedArray::valid

(PHP 5 >= 5.3.0, PHP 7)

SplFixedArray::valid — Check whether the array contains more elements

Description

public function **SplFixedArray::valid**(): bool

Checks if the array contains any more elements.

Parameters

This function has no parameters.

Return Values

Returns `true` if the array contains any more elements, `false` otherwise.

SplFixedArray::__wakeup

SplFixedArray::__wakeup

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

SplFixedArray::__wakeup — Reinitialises the array after being unserialised

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.4.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.4',
    message: 'this method is obsolete, as serialization hooks are provided by __unserialize() and __serialize()',
)]
public function SplFixedArray::__wakeup(): void
```

Reinitialises the array after being unserialised.

Parameters

This function has no parameters.

Return Values

No value is returned.

Changelog

Version	Description
8.4.0	This method is now deprecated, use SplFixedArray::__unserialize() instead.

ArrayObject

(PHP 5, PHP 7, PHP 8)

Introduction

This class allows objects to work as arrays.

> **Note** :
>
> Wrapping objects with this class is fundamentally flawed, and therefore its usage with objects is discouraged.

Class synopsis

```
class ArrayObject implements IteratorAggregate, ArrayAccess, Serializable, Countable {
/* Constants */
public const int STD_PROP_LIST;
public const int ARRAY_AS_PROPS;
/* Methods */
public function __construct(array|object $array = [], int $flags = 0, string $iteratorClass = ArrayIterator::class)
public function append(mixed $value): void
public function asort(int $flags = SORT_REGULAR): true
public function count(): int
public function exchangeArray(array|object $array): array
public function getArrayCopy(): array
public function getFlags(): int
public function getIterator(): Iterator
public function getIteratorClass(): string
public function ksort(int $flags = SORT_REGULAR): true
public function natcasesort(): true
public function natsort(): true
public function offsetExists(mixed $key): bool
public function offsetGet(mixed $key): mixed
public function offsetSet(mixed $key, mixed $value): void
public function offsetUnset(mixed $key): void
public function serialize(): string
public function setFlags(int $flags): void
public function setIteratorClass(string $iteratorClass): void
public function uasort(callable $callback): true
public function uksort(callable $callback): true
public function unserialize(string $data): void
}
```

Predefined Constants

ArrayObject Flags

`ArrayObject::STD_PROP_LIST`

Properties of the object have their normal functionality when accessed as list (var_dump(), `foreach`, etc.).

`ArrayObject::ARRAY_AS_PROPS`

Entries can be accessed as properties (read and write). The **ArrayObject** class uses its own logic to access properties, thus no warning or error is raised when trying to read or write dynamic properties.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

ArrayObject::append

ArrayObject::append

(PHP 5, PHP 7, PHP 8)

ArrayObject::append — Appends the value

Description

public function **ArrayObject::append**(mixed `$value`): void

Appends a new value as the last element.

> **Note** :
>
> This method cannot be called when the ArrayObject was constructed from an object. Use ArrayObject::offsetSet() instead.

Parameters

`value`

The value being appended.

Return Values

No value is returned.

Examples

Example #1 ArrayObject::append() example

```php
<?php
$arrayobj = new ArrayObject(array('first','second','third'));
$arrayobj->append('fourth');
$arrayobj->append(array('five', 'six'));
var_dump($arrayobj);
?>
```

The above example will output:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(5) {
    [0]=>
    string(5) "first"
```

```
    [1]=>
    string(6) "second"
    [2]=>
    string(5) "third"
    [3]=>
    string(6) "fourth"
    [4]=>
    array(2) {
      [0]=>
      string(4) "five"
      [1]=>
      string(3) "six"
    }
  }
}
```

See Also

- ArrayObject::offsetSet() - Sets the value at the specified index to newval

ArrayObject::asort

ArrayObject::asort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::asort — Sort the entries by value

Description

public function **ArrayObject::asort**(int `$flags` = `SORT_REGULAR`): true

Sorts the entries in ascending order, such that its keys maintain their correlation with the values they are associated with.

This is used mainly when sorting associative arrays where the actual element order is significant.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- `SORT_REGULAR` - compare items normally; the details are described in the comparison operators section
- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings
- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- `SORT_NATURAL` - compare items as strings using "natural ordering" like natsort()
- `SORT_FLAG_CASE` - can be combined (bitwise OR) with `SORT_STRING` or `SORT_NATURAL` to sort strings case-insensitively

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 ArrayObject::asort() example

```
<?php
$fruits = array("d" => "lemon", "a" => "orange", "b" => "banana", "c" => "apple");
$fruitArrayObject = new ArrayObject($fruits);
$fruitArrayObject->asort();

foreach ($fruitArrayObject as $key => $val) {
    echo "$key = $val\n";
}
?>
```

The above example will output:

```
c = apple
b = banana
d = lemon
a = orange
```

The fruits have been sorted in alphabetical order, and the key associated with each entry has been maintained.

See Also

- ArrayObject::ksort() - Sort the entries by key
- ArrayObject::natsort() - Sort entries using a "natural order" algorithm
- ArrayObject::natcasesort() - Sort an array using a case insensitive "natural order" algorithm
- ArrayObject::uasort() - Sort the entries with a user-defined comparison function and maintain key association
- ArrayObject::uksort() - Sort the entries by keys using a user-defined comparison function
- asort() - Sort an array in ascending order and maintain index association

ArrayObject::__construct

ArrayObject::__construct

(PHP 5, PHP 7, PHP 8)

ArrayObject::__construct — Construct a new array object

Description

public function **ArrayObject::__construct**(array|object `$array` = [], int `$flags` = 0, string `$iteratorClass` = **`ArrayIterator::class`**)

This constructs a new array object.

> **Warning**
>
> As of PHP 8.5.0, passing an enum as the `array` parameter throws an InvalidArgumentException. Modifying the `$name` or `$value` properties of an enum would break engine assumptions.

Parameters

`array`

The `array` parameter accepts an array or an object. Passing an object is deprecated as of PHP 8.5.0.

`flags`

Flags to control the behaviour of the ArrayObject object. See ArrayObject::setFlags().

`iteratorClass`

Specify the class that will be used for iteration of the ArrayObject object. The class must be a subtype of the ArrayIterator class.

Errors/Exceptions

As of PHP 8.5.0, throws an InvalidArgumentException if `array` is an enum.

Changelog

Version	Description
8.5.0	Passing an enum as `array` now throws an InvalidArgumentException.
8.5.0	Passing an object as `array` is deprecated.

Examples

Example #1 ArrayObject::__construct() example

```php
<?php

$array = [
    '1' => 'one',
    '2' => 'two',
    '3' => 'three'
];

$arrayobject = new ArrayObject($array);

var_dump($arrayobject);

?>
```

The above example will output:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(3) {
    [1]=>
    string(3) "one"
    [2]=>
    string(3) "two"
    [3]=>
    string(5) "three"
  }
}
```

See Also

- ArrayObject::setFlags() - Sets the behavior flags
- ArrayObject::getArrayCopy() - Creates a copy of the ArrayObject
- ArrayIterator::__construct() - Construct an ArrayIterator

ArrayObject::count

ArrayObject::count

(PHP 5, PHP 7, PHP 8)

ArrayObject::count — Get the number of public properties in the ArrayObject

Description

public function **ArrayObject::count**(): int

Get the number of public properties in the ArrayObject.

Parameters

This function has no parameters.

Return Values

The number of public properties in the ArrayObject.

> **Note** :
>
> When the ArrayObject is constructed from an array all properties are public.

Examples

Example #1 ArrayObject::count() example

```
<?php
class Example {
    public $public = 'prop:public';
    private $prv   = 'prop:private';
    protected $prt = 'prop:protected';
}

$arrayobj = new ArrayObject(new Example());
var_dump($arrayobj->count());

$arrayobj = new ArrayObject(array('first','second','third'));
var_dump($arrayobj->count());
?>
```

The above example will output:

```
int(1)
int(3)
```

ArrayObject::exchangeArray

ArrayObject::exchangeArray

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::exchangeArray — Exchange the array for another one

Description

public function **ArrayObject::exchangeArray**(array|object `$array`): array

Exchange the current array with another array or object.

Parameters

`array`

The new array or object to exchange with the current array.

Return Values

Returns the old array.

Examples

Example #1 ArrayObject::exchangeArray() example

```php
<?php
// Array of available fruits
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);
// Array of locations in Europe
$locations = array('Amsterdam', 'Paris', 'London');

$fruitsArrayObject = new ArrayObject($fruits);

// Now exchange fruits for locations
$old = $fruitsArrayObject->exchangeArray($locations);
var_dump($old);
var_dump($fruitsArrayObject);

?>
```

The above example will output:

```
array(4) {
  ["lemons"]=>
  int(1)
  ["oranges"]=>
  int(4)
  ["bananas"]=>
  int(5)
  ["apples"]=>
  int(10)
}
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(3) {
    [0]=>
    string(9) "Amsterdam"
    [1]=>
    string(5) "Paris"
    [2]=>
    string(6) "London"
  }
}
```

ArrayObject::getArrayCopy

ArrayObject::getArrayCopy

(PHP 5, PHP 7, PHP 8)

ArrayObject::getArrayCopy — Creates a copy of the ArrayObject

Description

public function **ArrayObject::getArrayCopy**(): array

Exports the ArrayObject to an array.

Parameters

This function has no parameters.

Return Values

Returns a copy of the array. When the ArrayObject refers to an object, an array of the properties of that object will be returned.

Examples

Example #1 ArrayObject::getArrayCopy() example

```php
<?php
// Array of available fruits
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);
$fruitsArrayObject['pears'] = 4;

// create a copy of the array
$copy = $fruitsArrayObject->getArrayCopy();
var_dump($copy);

?>
```

The above example will output:

```
array(5) {
  ["lemons"]=>
  int(1)
  ["oranges"]=>
  int(4)
  ["bananas"]=>
  int(5)
  ["apples"]=>
  int(10)
  ["pears"]=>
  int(4)
}
```

ArrayObject::getFlags

ArrayObject::getFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::getFlags — Gets the behavior flags

Description

public function **ArrayObject::getFlags**(): int

Gets the behavior flags of the ArrayObject. See the ArrayObject::setFlags method for a list of the available flags.

Parameters

This function has no parameters.

Return Values

Returns the behavior flags of the ArrayObject.

Examples

Example #1 ArrayObject::getFlags() example

```php
<?php
// Array of available fruits
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);

// Get the current flags
```

```php
$flags = $fruitsArrayObject->getFlags();
var_dump($flags);

// Set new flags
$fruitsArrayObject->setFlags(ArrayObject::ARRAY_AS_PROPS);

// Get the new flags
$flags = $fruitsArrayObject->getFlags();
var_dump($flags);
?>
```

The above example will output:

```
int(0)
int(2)
```

See Also

- ArrayObject::setFlags() - Sets the behavior flags

ArrayObject::getIterator

ArrayObject::getIterator

(PHP 5, PHP 7, PHP 8)

ArrayObject::getIterator — Create a new iterator from an ArrayObject instance

Description

public function **ArrayObject::getIterator**(): Iterator

Create a new Iterator (default is ArrayIterator) from an ArrayObject instance.

Parameters

This function has no parameters.

Return Values

An iterator from an ArrayObject.

Examples

Example #1 ArrayObject::getIterator() example

```php
<?php

$array = [
    '1' => 'one',
    '2' => 'two',
    '3' => 'three',
];

$arrayobject = new ArrayObject($array);

$iterator = $arrayobject->getIterator();

while ($iterator->valid()) {
    echo $iterator->key() . ' => ' . $iterator->current() . "\n";

    $iterator->next();
}

?>
```

The above example will output:

```
1 => one
2 => two
3 => three
```

ArrayObject::getIteratorClass

ArrayObject::getIteratorClass

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::getIteratorClass — Gets the iterator classname for the ArrayObject

Description

public function **ArrayObject::getIteratorClass**(): string

Gets the class name of the array iterator that is used by ArrayObject::getIterator().

Parameters

This function has no parameters.

Return Values

Returns the iterator class name that is used to iterate over this object.

Examples

Example #1 ArrayObject::getIteratorClass() example

```php
<?php
// Custom ArrayIterator (inherits from ArrayIterator)
class MyArrayIterator extends ArrayIterator {
    // custom implementation
}

// Array of available fruits
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);

// Get the current class name
$className = $fruitsArrayObject->getIteratorClass();
var_dump($className);

// Set new classname
$fruitsArrayObject->setIteratorClass('MyArrayIterator');

// Get the new iterator classname
$className = $fruitsArrayObject->getIteratorClass();
var_dump($className);
?>
```

The above example will output:

```
string(13) "ArrayIterator"
string(15) "MyArrayIterator"
```

See Also

- The ArrayObject::setIteratorClass method

ArrayObject::ksort

ArrayObject::ksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::ksort — Sort the entries by key

Description

public function **ArrayObject::ksort**(int `$flags` = `SORT_REGULAR`): true

Sorts the entries by key, maintaining key to entry correlations. This is useful mainly for associative arrays.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- `SORT_REGULAR` - compare items normally; the details are described in the comparison operators section
- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings
- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- `SORT_NATURAL` - compare items as strings using "natural ordering" like natsort()
- `SORT_FLAG_CASE` - can be combined (bitwise OR) with `SORT_STRING` or `SORT_NATURAL` to sort strings case-insensitively

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 ArrayObject::ksort() example

```
<?php
$fruits = array("d" => "lemon", "a" => "orange", "b" => "banana", "c" => "apple");
$fruitArrayObject = new ArrayObject($fruits);
$fruitArrayObject->ksort();

foreach ($fruitArrayObject as $key => $val) {
    echo "$key = $val\n";
}
 ?>
```

The above example will output:

```
a = orange
b = banana
c = apple
d = lemon
```

See Also

- ArrayObject::asort() - Sort the entries by value
- ArrayObject::natsort() - Sort entries using a "natural order" algorithm
- ArrayObject::natcasesort() - Sort an array using a case insensitive "natural order" algorithm
- ArrayObject::uasort() - Sort the entries with a user-defined comparison function and maintain key association
- ArrayObject::uksort() - Sort the entries by keys using a user-defined comparison function
- ksort() - Sort an array by key in ascending order

ArrayObject::natcasesort

ArrayObject::natcasesort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::natcasesort — Sort an array using a case insensitive "natural order" algorithm

Description

public function **ArrayObject::natcasesort**(): true

This method is a case insensitive version of ArrayObject::natsort.

This method implements a sort algorithm that orders alphanumeric strings in the way a human being would while maintaining key/value associations. This is described as a "natural ordering".

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

This function has no parameters.

Return Values

No value is returned.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 ArrayObject::natcasesort() example

```php
<?php
$array = array('IMG0.png', 'img12.png', 'img10.png', 'img2.png', 'img1.png', 'IMG3.png');

$arr1 = new ArrayObject($array);
$arr2 = clone $arr1;

$arr1->asort();
echo "Standard sorting\n";
```

```php
var_dump($arr1);

$arr2->natcasesort();
echo "\nNatural order sorting (case-insensitive)\n";
var_dump($arr2);
?>
```

The above example will output:

```
Standard sorting
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(6) {
    [0]=>
    string(8) "IMG0.png"
    [5]=>
    string(8) "IMG3.png"
    [4]=>
    string(8) "img1.png"
    [2]=>
    string(9) "img10.png"
    [1]=>
    string(9) "img12.png"
    [3]=>
    string(8) "img2.png"
  }
}

Natural order sorting (case-insensitive)
object(ArrayObject)#2 (1) {
  ["storage":"ArrayObject":private]=>
  array(6) {
    [0]=>
    string(8) "IMG0.png"
    [4]=>
    string(8) "img1.png"
    [3]=>
    string(8) "img2.png"
    [5]=>
    string(8) "IMG3.png"
    [2]=>
    string(9) "img10.png"
    [1]=>
    string(9) "img12.png"
  }
}
```

For more information see: Martin Pool's » Natural Order String Comparison page.

See Also

- ArrayObject::asort() - Sort the entries by value
- ArrayObject::ksort() - Sort the entries by key
- ArrayObject::natsort() - Sort entries using a "natural order" algorithm
- ArrayObject::uasort() - Sort the entries with a user-defined comparison function and maintain key association
- ArrayObject::uksort() - Sort the entries by keys using a user-defined comparison function
- natcasesort() - Sort an array using a case insensitive "natural order" algorithm

ArrayObject::natsort

ArrayObject::natsort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::natsort — Sort entries using a "natural order" algorithm

Description

public function **ArrayObject::natsort**(): true

This method implements a sort algorithm that orders alphanumeric strings in the way a human being would while maintaining key/value associations. This is described as a "natural ordering". An example of the difference between this algorithm and the regular computer string sorting algorithms (used in ArrayObject::asort) method can be seen in the example below.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

This function has no parameters.

Return Values

No value is returned.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 ArrayObject::natsort() example

```php
<?php
$array = array("img12.png", "img10.png", "img2.png", "img1.png");

$arr1 = new ArrayObject($array);
$arr2 = clone $arr1;

$arr1->asort();
echo "Standard sorting\n";
var_dump($arr1);

$arr2->natsort();
echo "\nNatural order sorting\n";
var_dump($arr2);
?>
```

The above example will output:

```
Standard sorting
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    [3]=>
    string(8) "img1.png"
    [1]=>
    string(9) "img10.png"
    [0]=>
    string(9) "img12.png"
    [2]=>
    string(8) "img2.png"
  }
}

Natural order sorting
object(ArrayObject)#2 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
```

```
    [3]=>
    string(8) "img1.png"
    [2]=>
    string(8) "img2.png"
    [1]=>
    string(9) "img10.png"
    [0]=>
    string(9) "img12.png"
  }
}
```

For more information see: Martin Pool's » Natural Order String Comparison page.

See Also

- ArrayObject::asort() - Sort the entries by value
- ArrayObject::ksort() - Sort the entries by key
- ArrayObject::natcasesort() - Sort an array using a case insensitive "natural order" algorithm
- ArrayObject::uasort() - Sort the entries with a user-defined comparison function and maintain key association
- ArrayObject::uksort() - Sort the entries by keys using a user-defined comparison function
- natsort() - Sort an array using a "natural order" algorithm

ArrayObject::offsetExists

ArrayObject::offsetExists

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetExists — Returns whether the requested index exists

Description

public function **ArrayObject::offsetExists**(mixed $key): bool

Parameters

key

The index being checked.

Return Values

`true` if the requested index exists, otherwise `false`

Examples

Example #1 ArrayObject::offsetExists() example

```php
<?php
$arrayobj = new ArrayObject(array('zero', 'one', 'example'=>'e.g.'));
var_dump($arrayobj->offsetExists(1));
var_dump($arrayobj->offsetExists('example'));
var_dump($arrayobj->offsetExists('notfound'));
?>
```

The above example will output:

```
bool(true)
bool(true)
bool(false)
```

ArrayObject::offsetGet

ArrayObject::offsetGet

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetGet — Returns the value at the specified index

Description

public function **ArrayObject::offsetGet**(mixed $key): mixed

Parameters

key

The index with the value.

Return Values

The value at the specified index or `null`.

Errors/Exceptions

Produces an `E_NOTICE` error message when the specified index does not exist.

Examples

Example #1 ArrayObject::offsetGet() example

```
<?php
$arrayobj = new ArrayObject(array('zero', 7, 'example'=>'e.g.'));
var_dump($arrayobj->offsetGet(1));
var_dump($arrayobj->offsetGet('example'));
var_dump($arrayobj->offsetExists('notfound'));
?>
```

The above example will output:

```
int(7)
string(4) "e.g."
bool(false)
```

ArrayObject::offsetSet

ArrayObject::offsetSet

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetSet — Sets the value at the specified index to newval

Description

public function **ArrayObject::offsetSet**(mixed $key, mixed $value): void

Sets the value at the specified index to newval.

Parameters

key

The index being set.

value

The new value for the key.

Return Values

No value is returned.

Examples

Example #1 ArrayObject::offsetSet() example

```php
<?php
class Example {
    public $property = 'prop:public';
}
$arrayobj = new ArrayObject(new Example());
$arrayobj->offsetSet(4, 'four');
$arrayobj->offsetSet('group', array('g1', 'g2'));
var_dump($arrayobj);

$arrayobj = new ArrayObject(array('zero','one'));
$arrayobj->offsetSet(null, 'last');
var_dump($arrayobj);
?>
```

The above example will output:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  object(Example)#2 (3) {
    ["property"]=>
    string(11) "prop:public"
    ["4"]=>
    string(4) "four"
    ["group"]=>
    array(2) {
      [0]=>
      string(2) "g1"
      [1]=>
      string(2) "g2"
    }
  }
}
object(ArrayObject)#3 (1) {
  ["storage":"ArrayObject":private]=>
  array(3) {
    [0]=>
    string(4) "zero"
    [1]=>
    string(3) "one"
    [2]=>
    string(4) "last"
  }
}
```

See Also

- ArrayObject::append() - Appends the value

ArrayObject::offsetUnset

ArrayObject::offsetUnset

(PHP 5, PHP 7, PHP 8)

ArrayObject::offsetUnset — Unsets the value at the specified index

Description

public function **ArrayObject::offsetUnset**(mixed $key): void

Unsets the value at the specified index.

Parameters

key

The index being unset.

Return Values

No value is returned.

Examples

Example #1 ArrayObject::offsetUnset() example

```
<?php
$arrayobj = new ArrayObject(array(0=>'zero',2=>'two'));
$arrayobj->offsetUnset(2);
var_dump($arrayobj);
?>
```

The above example will output:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(1) {
    [0]=>
    string(4) "zero"
  }
}
```

ArrayObject::serialize

ArrayObject::serialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayObject::serialize — Serialize an ArrayObject

Description

public function **ArrayObject::serialize**(): string

Serializes an ArrayObject.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The serialized representation of the ArrayObject.

Examples

Example #1 ArrayObject::serialize() example

```php
<?php
$o = new ArrayObject();

$s1 = serialize($o);
$s2 = $o->serialize();

var_dump($s1);
var_dump($s2);
?>
```

The above example will output:

```
string(45) "C:11:"ArrayObject":21:{x:i:0;a:0:{};m:a:0:{}}"
string(21) "x:i:0;a:0:{};m:a:0:{}"
```

See Also

- ArrayObject::unserialize() - Unserialize an ArrayObject
- serialize() - Generates a storable representation of a value
- Serializing Objects

ArrayObject::setFlags

ArrayObject::setFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::setFlags — Sets the behavior flags

Description

public function **ArrayObject::setFlags**(int `$flags`): void

Set the flags that change the behavior of the ArrayObject.

Parameters

`flags`

The new ArrayObject behavior. It takes on either a bitmask, or named constants. Using named constants is strongly encouraged to ensure compatibility for future versions.

The available behavior flags are listed below. The actual meanings of these flags are described in the predefined constants.

ArrayObject behavior flags

value	constant
1	ArrayObject::STD_PROP_LIST
2	ArrayObject::ARRAY_AS_PROPS

Return Values

No value is returned.

Examples

Example #1 ArrayObject::setFlags() example

```php
<?php
// Array of available fruits
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);
```

```php
// Try to use array key as property
var_dump($fruitsArrayObject->lemons);
// Set the flag so that the array keys can be used as properties of the ArrayObject
$fruitsArrayObject->setFlags(ArrayObject::ARRAY_AS_PROPS);
// Try it again
var_dump($fruitsArrayObject->lemons);
?>
```

The above example will output something similar to:

```
Warning: Undefined property: ArrayObject::$lemons in ...
NULL
int(1)
```

ArrayObject::setIteratorClass

ArrayObject::setIteratorClass

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayObject::setIteratorClass — Sets the iterator classname for the ArrayObject

Description

public function **ArrayObject::setIteratorClass**(string `$iteratorClass`): void

Sets the classname of the array iterator that is used by ArrayObject::getIterator().

Parameters

`iteratorClass`

The classname of the array iterator to use when iterating over this object.

Return Values

No value is returned.

Examples

Example #1 ArrayObject::setIteratorClass() example

```php
<?php
// Custom ArrayIterator (inherits from ArrayIterator)
class MyArrayIterator extends ArrayIterator {
    // custom implementation
}

// Array of available fruits
$fruits = array("lemons" => 1, "oranges" => 4, "bananas" => 5, "apples" => 10);

$fruitsArrayObject = new ArrayObject($fruits);

// Set the iterator classname to the newly
$fruitsArrayObject->setIteratorClass('MyArrayIterator');
var_dump($fruitsArrayObject->getIterator());

?>
```

The above example will output:

```
object(MyArrayIterator)#2 (1) {
  ["storage":"ArrayIterator":private]=>
```

```
  object(ArrayObject)#1 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["lemons"]=>
      int(1)
      ["oranges"]=>
      int(4)
      ["bananas"]=>
      int(5)
      ["apples"]=>
      int(10)
    }
  }
}
```

ArrayObject::uasort

ArrayObject::uasort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::uasort — Sort the entries with a user-defined comparison function and maintain key association

Description

public function **ArrayObject::uasort**(callable `$callback`): true

This function sorts the entries such that keys maintain their correlation with the entry that they are associated with, using a user-defined comparison function.

This is used mainly when sorting associative arrays where the actual element order is significant.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`callback`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 ArrayObject::uasort() example

```php
<?php
// Comparison function
function cmp($a, $b) {
    if ($a == $b) {
        return 0;
    }
    return ($a < $b) ? -1 : 1;
}

// Array to be sorted
$array = array('a' => 4, 'b' => 8, 'c' => -1, 'd' => -9, 'e' => 2, 'f' => 5, 'g' => 3, 'h' => -4);
$arrayObject = new ArrayObject($array);
var_dump($arrayObject);

// Sort and print the resulting array
$arrayObject->uasort('cmp');
var_dump($arrayObject);
?>
```

The above example will output:

```
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(8) {
    ["a"]=>
    int(4)
    ["b"]=>
    int(8)
    ["c"]=>
    int(-1)
    ["d"]=>
    int(-9)
    ["e"]=>
    int(2)
    ["f"]=>
    int(5)
    ["g"]=>
    int(3)
    ["h"]=>
    int(-4)
  }
}
object(ArrayObject)#1 (1) {
  ["storage":"ArrayObject":private]=>
  array(8) {
    ["d"]=>
    int(-9)
    ["h"]=>
    int(-4)
    ["c"]=>
    int(-1)
    ["e"]=>
    int(2)
    ["g"]=>
    int(3)
    ["a"]=>
    int(4)
    ["f"]=>
    int(5)
    ["b"]=>
    int(8)
  }
}
```

See Also

- ArrayObject::asort() - Sort the entries by value
- ArrayObject::ksort() - Sort the entries by key
- ArrayObject::natsort() - Sort entries using a "natural order" algorithm
- ArrayObject::natcasesort() - Sort an array using a case insensitive "natural order" algorithm
- ArrayObject::uksort() - Sort the entries by keys using a user-defined comparison function
- uasort() - Sort an array with a user-defined comparison function and maintain index association

ArrayObject::uksort

ArrayObject::uksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayObject::uksort — Sort the entries by keys using a user-defined comparison function

Description

public function **ArrayObject::uksort**(callable `$callback`): true

This function sorts the keys of the entries using a user-supplied comparison function. The key to entry correlations will be maintained.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`callback`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

Examples

Example #1 ArrayObject::uksort() example

```php
<?php
function cmp($a, $b) {
    $a = preg_replace('@^(a|an|the) @', '', $a);
    $b = preg_replace('@^(a|an|the) @', '', $b);
    return strcasecmp($a, $b);
}

$array = array("John" => 1, "the Earth" => 2, "an apple" => 3, "a banana" => 4);
$arrayObject = new ArrayObject($array);
$arrayObject->uksort('cmp');

foreach ($arrayObject as $key => $value) {
    echo "$key: $value\n";
```

```
}
?>
```

The above example will output:

```
an apple: 3
a banana: 4
the Earth: 2
John: 1
```

See Also

- ArrayObject::asort() - Sort the entries by value
- ArrayObject::ksort() - Sort the entries by key
- ArrayObject::natsort() - Sort entries using a "natural order" algorithm
- ArrayObject::natcasesort() - Sort an array using a case insensitive "natural order" algorithm
- ArrayObject::uasort() - Sort the entries with a user-defined comparison function and maintain key association
- uksort() - Sort an array by keys using a user-defined comparison function

ArrayObject::unserialize

ArrayObject::unserialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayObject::unserialize — Unserialize an ArrayObject

Description

public function **ArrayObject::unserialize**(string `$data`): void

Unserializes a serialized ArrayObject.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

The serialized ArrayObject.

Return Values

No value is returned.

See Also

- ArrayIterator::serialize() - Serialize
- unserialize() - Creates a PHP value from a stored representation
- Serializing Objects

SplObjectStorage

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The SplObjectStorage class provides a map from objects to data or, by ignoring data, an object set. This dual purpose can be useful in many cases involving the need to uniquely identify objects.

Class synopsis

```
class SplObjectStorage implements Countable, SeekableIterator, Serializable, ArrayAccess {
/* Methods */
public function addAll(SplObjectStorage $storage): int
#[\Deprecated(
    since: '8.5',
    message: "use method SplObjectStorage::offsetSet() instead",
)]
public function attach(object $object, mixed $info = null): void
#[\Deprecated(
    since: '8.5',
    message: "use method SplObjectStorage::offsetExists() instead",
)]
public function contains(object $object): bool
public function count(int $mode = COUNT_NORMAL): int
public function current(): object
#[\Deprecated(
    since: '8.5',
    message: "use method SplObjectStorage::offsetUnset() instead",
)]
public function detach(object $object): void
public function getHash(object $object): string
public function getInfo(): mixed
public function key(): int
public function next(): void
public function offsetExists(object $object): bool
public function offsetGet(object $object): mixed
public function offsetSet(object $object, mixed $info = null): void
public function offsetUnset(object $object): void
public function removeAll(SplObjectStorage $storage): int
public function removeAllExcept(SplObjectStorage $storage): int
public function rewind(): void
public function seek(int $offset): void
public function serialize(): string
public function setInfo(mixed $info): void
public function unserialize(string $data): void
public function valid(): bool
}
```

Examples

Example #1 **SplObjectStorage** as a set

```php
<?php
// As an object set
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;
$o3 = new stdClass;

$s->attach($o1);
$s->attach($o2);

var_dump($s->contains($o1));
var_dump($s->contains($o2));
var_dump($s->contains($o3));

$s->detach($o2);

var_dump($s->contains($o1));
var_dump($s->contains($o2));
var_dump($s->contains($o3));
?>
```

The above example will output:

```
bool(true)
bool(true)
bool(false)
bool(true)
bool(false)
bool(false)
```

Example #2 SplObjectStorage as a map

```php
<?php
// As a map from objects to data
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;
$o3 = new stdClass;

$s[$o1] = "data for object 1";
$s[$o2] = array(1,2,3);

if (isset($s[$o2])) {
    var_dump($s[$o2]);
}
?>
```

The above example will output:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

Changelog

Version	Description
8.4.0	Implement SeekableIterator, previously only Iterator was implemented.

Table of Contents

SplObjectStorage::addAll

SplObjectStorage::addAll

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::addAll — Adds all objects from another storage

Description

public function **SplObjectStorage::addAll**(SplObjectStorage `$storage`): int

Adds all objects-data pairs from a different storage in the current storage.

Parameters

`storage`

The storage you want to import.

Return Values

The number of objects in the storage.

Examples

Example #1 SplObjectStorage::addAll() example

```php
<?php
$o = new stdClass;
$a = new SplObjectStorage();
$a[$o] = "hello";

$b = new SplObjectStorage();
$b->addAll($a);
echo $b[$o]."\n";
?>
```

The above example will output something similar to:

```
hello
```

See Also

- SplObjectStorage::removeAll() - Removes objects contained in another storage from the current storage

SplObjectStorage::attach

SplObjectStorage::attach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::attach — Adds an object in the storage

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.5',
    message: "use method SplObjectStorage::offsetSet() instead",
)]
public function SplObjectStorage::attach(object $object, mixed $info = null): void
```

Adds an object inside the storage, and optionally associate it to some data.

This method is an alias of SplObjectStorage::offsetSet().

Parameters

`object`

The object to add.

`info`

The data to associate with the object.

Return Values

No value is returned.

Changelog

Version	Description
8.5.0	This method has been deprecated in favor of SplObjectStorage::offsetSet().

Examples

Example #1 SplObjectStorage::attach() example

```php
<?php
$o1 = new stdClass;
$o2 = new stdClass;
$s = new SplObjectStorage();
$s->attach($o1); // similar to $s[$o1] = NULL;
$s->attach($o2, "hello"); // similar to $s[$o2] = "hello";

var_dump($s[$o1]);
var_dump($s[$o2]);

?>
```

The above example will output something similar to:

```
NULL
string(5) "hello"
```

See Also

- SplObjectStorage::detach() - Removes an object from the storage
- SplObjectStorage::offsetSet() - Associates data to an object in the storage

SplObjectStorage::contains

SplObjectStorage::contains

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::contains — Checks if the storage contains a specific object

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.5',
    message: "use method SplObjectStorage::offsetExists() instead",
)]
public function SplObjectStorage::contains(object $object): bool
```

Checks if the storage contains the object provided.

Parameters

`object`

The object to look for.

Return Values

Returns `true` if the object is in the storage, `false` otherwise.

Changelog

Version	Description
8.5.0	This method has been deprecated in favor of SplObjectStorage::offsetExists().

Examples

Example #1 SplObjectStorage::contains() example

```
<?php
$o1 = new stdClass;
$o2 = new stdClass;

$s = new SplObjectStorage();

$s[$o1] = "hello";
var_dump($s->contains($o1));
var_dump($s->contains($o2));
?>
```

The above example will output something similar to:

```
bool(true)
bool(false)
```

See Also

- SplObjectStorage::offsetExists() - Checks whether an object exists in the storage

SplObjectStorage::count

SplObjectStorage::count

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::count — Returns the number of objects in the storage

Description

public function **SplObjectStorage::count**(int `$mode` = `COUNT_NORMAL`): int

Counts the number of objects in the storage.

Parameters

`mode`

If the optional `mode` parameter is set to `COUNT_RECURSIVE` (or 1), **SplObjectStorage::count()** will recursively count the storage.

Return Values

The number of objects in the storage.

Examples

Example #1 SplObjectStorage::count() example

```
<?php
$s = new SplObjectStorage();
$o1 = new stdClass;
$o2 = new stdClass;
```

```php
$s->attach($o1);
$s->attach($o2);
$s->attach($o1);
var_dump($s->count());
var_dump(count($s));
?>
```

The above example will output something similar to:

```
int(2)
int(2)
```

See Also

- SplObjectStorage::attach() - Adds an object in the storage
- SplObjectStorage::detach() - Removes an object from the storage

SplObjectStorage::current

SplObjectStorage::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::current — Returns the current storage entry

Description

public function **SplObjectStorage::current**(): object

Returns the current storage entry.

Parameters

This function has no parameters.

Return Values

The object at the current iterator position.

Changelog

Version	Description
8.1.0	**SplObjectStorage::current()** now throws an Error exception if the current position is invalid. Previously, `false` was returned instead.

Examples

Example #1 SplObjectStorage::current() example

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar to current($s)
    $data   = $s->getInfo();
```

```
    var_dump($object);
    var_dump($data);
    $s->next();
}
?>
```

The above example will output something similar to:

```
object(stdClass)#2 (0) {
}
string(2) "d1"
object(stdClass)#3 (0) {
}
string(2) "d2"
```

See Also

- SplObjectStorage::rewind() - Rewind the iterator to the first storage element
- SplObjectStorage::key() - Returns the index at which the iterator currently is
- SplObjectStorage::next() - Move to the next entry
- SplObjectStorage::valid() - Returns if the current iterator entry is valid
- SplObjectStorage::getInfo() - Returns the data associated with the current iterator entry

SplObjectStorage::detach

SplObjectStorage::detach

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::detach — Removes an object from the storage

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 8.5.0. Relying on this function is highly discouraged.

Description

```
#[\Deprecated(
    since: '8.5',
    message: "use method SplObjectStorage::offsetUnset() instead",
)]
public function SplObjectStorage::detach(object $object): void
```

Removes the object from the storage.

Parameters

`object`

The object to remove.

Return Values

No value is returned.

Changelog

Version	Description
8.5.0	This method has been deprecated in favor of SplObjectStorage::offsetUnset().

Examples

Example #1 SplObjectStorage::detach() example

```php
<?php
$o = new stdClass;
$s = new SplObjectStorage();
$s->attach($o);
var_dump(count($s));
$s->detach($o);
var_dump(count($s));
?>
```

The above example will output something similar to:

```
int(1)
int(0)
```

See Also

- SplObjectStorage::attach() - Adds an object in the storage
- SplObjectStorage::removeAll() - Removes objects contained in another storage from the current storage

SplObjectStorage::getHash

SplObjectStorage::getHash

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplObjectStorage::getHash — Calculate a unique identifier for the contained objects

Description

public function **SplObjectStorage::getHash**(object `$object`): string

This method calculates an identifier for the objects added to an SplObjectStorage object.

The implementation in SplObjectStorage returns the same value as spl_object_hash().

The storage object will never contain more than one object with the same identifier. As such, it can be used to implement a set (a collection of unique values) where the quality of an object being unique is determined by the value returned by this function being unique.

Parameters

`object`

The object whose identifier is to be calculated.

Return Values

A string with the calculated identifier.

Errors/Exceptions

A RuntimeException is thrown when the returned value is not a string.

Examples

Example #1 SplObjectStorage::getHash() example

```php
<?php
class OneSpecimenPerClassStorage extends SplObjectStorage {
    public function getHash($o) {
        return get_class($o);
    }
}
class A {}
```

```php
$s = new OneSpecimenPerClassStorage;
$o1 = new stdClass;
$o2 = new stdClass;
$o3 = new A;

$s[$o1] = 1;
//$o2 is considered equal to $o1 so the value is replaced
$s[$o2] = 2;
$s[$o3] = 3;

//these are considered equal to the objects before
//so they can be used to access the values stored under them
$p1 = new stdClass;
$p2 = new A;
echo $s[$p1], "\n";
echo $s[$p2], "\n";
?>
```

The above example will output something similar to:

```
2
3
```

See Also

- spl_object_hash() - Return hash id for given object

SplObjectStorage::getInfo

SplObjectStorage::getInfo

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::getInfo — Returns the data associated with the current iterator entry

Description

public function **SplObjectStorage::getInfo**(): mixed

Returns the data, or info, associated with the object pointed by the current iterator position.

Parameters

This function has no parameters.

Return Values

The data associated with the current iterator position.

Examples

Example #1 SplObjectStorage::getInfo() example

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
```

```
    $index  = $s->key();
    $object = $s->current(); // similar to current($s)
    $data   = $s->getInfo();

    var_dump($object);
    var_dump($data);
    $s->next();
}
?>
```

The above example will output something similar to:

```
object(stdClass)#2 (0) {
}
string(2) "d1"
object(stdClass)#3 (0) {
}
string(2) "d2"
```

See Also

- SplObjectStorage::current() - Returns the current storage entry
- SplObjectStorage::rewind() - Rewind the iterator to the first storage element
- SplObjectStorage::key() - Returns the index at which the iterator currently is
- SplObjectStorage::next() - Move to the next entry
- SplObjectStorage::valid() - Returns if the current iterator entry is valid
- SplObjectStorage::setInfo() - Sets the data associated with the current iterator entry

SplObjectStorage::key

SplObjectStorage::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::key — Returns the index at which the iterator currently is

Description

public function **SplObjectStorage::key**(): int

Returns the index at which the iterator currently is.

Parameters

This function has no parameters.

Return Values

The index corresponding to the position of the iterator.

Examples

Example #1 SplObjectStorage::key() example

```
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
```

```
    $index  = $s->key();
    $object = $s->current(); // similar to current($s)

    var_dump($index);
    var_dump($object);
    $s->next();
}
?>
```

The above example will output something similar to:

```
int(0)
object(stdClass)#2 (0) {
}
int(1)
object(stdClass)#3 (0) {
}
```

See Also

- SplObjectStorage::rewind() - Rewind the iterator to the first storage element
- SplObjectStorage::current() - Returns the current storage entry
- SplObjectStorage::next() - Move to the next entry
- SplObjectStorage::valid() - Returns if the current iterator entry is valid

SplObjectStorage::next

SplObjectStorage::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::next — Move to the next entry

Description

public function **SplObjectStorage::next**(): void

Moves the iterator to the next object in the storage.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 SplObjectStorage::next() example

```
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar to current($s)
```

```
    var_dump($index);
    var_dump($object);
    $s->next();
}
?>
```

The above example will output something similar to:

```
int(0)
object(stdClass)#2 (0) {
}
int(1)
object(stdClass)#3 (0) {
}
```

See Also

- SPLObjectStorage::rewind() - Rewind the iterator to the first storage element

SplObjectStorage::offsetExists

SplObjectStorage::offsetExists

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetExists — Checks whether an object exists in the storage

Description

public function **SplObjectStorage::offsetExists**(object `$object`): bool

Checks whether an object exists in the storage.

> **Note** :
>
> **SplObjectStorage::offsetExists()** is an alias of SplObjectStorage::contains().

Parameters

`object`

The object to look for.

Return Values

Returns `true` if the object exists in the storage, and `false` otherwise.

Examples

Example #1 SplObjectStorage::offsetExists() example

```
<?php
$s = new SplObjectStorage;
$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1);

var_dump($s->offsetExists($o1)); // Similar to isset($s[$o1])
var_dump($s->offsetExists($o2)); // Similar to isset($s[$o2])
?>
```

The above example will output something similar to:

```
bool(true)
bool(false)
```

See Also

- SplObjectStorage::offsetSet() - Associates data to an object in the storage
- SplObjectStorage::offsetGet() - Returns the data associated with an object
- SplObjectStorage::offsetUnset() - Removes an object from the storage

SplObjectStorage::offsetGet

SplObjectStorage::offsetGet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetGet — Returns the data associated with an object

Description

public function **SplObjectStorage::offsetGet**(object `$object`): mixed

Returns the data associated with an object in the storage.

Parameters

`object`

The object to look for.

Return Values

The data previously associated with the object in the storage.

Errors/Exceptions

Throws UnexpectedValueException when `object` could not be found.

Examples

Example #1 SplObjectStorage::offsetGet() example

```php
<?php
$s = new SplObjectStorage;

$o1 = new stdClass;
$o2 = new stdClass;

$s[$o1] = "hello";
$s->attach($o2);


var_dump($s->offsetGet($o1)); // Similar to $s[$o1]
var_dump($s->offsetGet($o2)); // Similar to $s[$o2]
?>
```

The above example will output something similar to:

```
string(5) "hello"
NULL
```

See Also

- SplObjectStorage::offsetSet() - Associates data to an object in the storage
- SplObjectStorage::offsetExists() - Checks whether an object exists in the storage
- SplObjectStorage::offsetUnset() - Removes an object from the storage

SplObjectStorage::offsetSet

SplObjectStorage::offsetSet

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetSet — Associates data to an object in the storage

Description

public function **SplObjectStorage::offsetSet**(object `$object`, mixed `$info` = `null`): void

Associate data to an object in the storage.

> **Note** :
>
> **SplObjectStorage::offsetSet()** is an alias of SplObjectStorage::attach().

Parameters

`object`

The object to associate data with.

`info`

The data to associate with the object.

Return Values

No value is returned.

Examples

Example #1 SplObjectStorage::offsetSet() example

```php
<?php
$s = new SplObjectStorage;

$o1 = new stdClass;

$s->offsetSet($o1, "hello"); // Similar to $s[$o1] = "hello";

var_dump($s[$o1]);
?>
```

The above example will output something similar to:

```
string(5) "hello"
```

See Also

- SplObjectStorage::attach() - Adds an object in the storage
- SplObjectStorage::offsetGet() - Returns the data associated with an object
- SplObjectStorage::offsetExists() - Checks whether an object exists in the storage
- SplObjectStorage::offsetUnset() - Removes an object from the storage

SplObjectStorage::offsetUnset

SplObjectStorage::offsetUnset

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::offsetUnset — Removes an object from the storage

Description

public function **SplObjectStorage::offsetUnset**(object `$object`): void

Removes an object from the storage.

> **Note** :
>
> **SplObjectStorage::offsetUnset()** is an alias of SplObjectStorage::detach().

Parameters

`object`

The object to remove.

Return Values

No value is returned.

Examples

Example #1 SplObjectStorage::offsetUnset() example

```php
<?php
$o = new stdClass;
$s = new SplObjectStorage();
$s->attach($o);
var_dump(count($s));
$s->offsetUnset($o); // Similar to unset($s[$o])
var_dump(count($s));
?>
```

The above example will output something similar to:

```
int(1)
int(0)
```

See Also

- SplObjectStorage::offsetGet() - Returns the data associated with an object
- SplObjectStorage::offsetSet() - Associates data to an object in the storage
- SplObjectStorage::offsetExists() - Checks whether an object exists in the storage

SplObjectStorage::removeAll

SplObjectStorage::removeAll

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::removeAll — Removes objects contained in another storage from the current storage

Description

public function **SplObjectStorage::removeAll**(SplObjectStorage `$storage`): int

Removes objects contained in another storage from the current storage.

Parameters

`storage`

The storage containing the elements to remove.

Return Values

Returns the number of remaining objects.

Examples

Example #1 **SplObjectStorage::removeAll()** example

```
<?php
$o1 = new stdClass;
$o2 = new stdClass;
$a = new SplObjectStorage();
$a[$o1] = "foo";

$b = new SplObjectStorage();
$b[$o1] = "bar";
$b[$o2] = "gee";

var_dump(count($b));
$b->removeAll($a);
var_dump(count($b));
?>
```

The above example will output something similar to:

```
int(2)
int(1)
```

See Also

- SplObjectStorage::addAll() - Adds all objects from another storage

SplObjectStorage::removeAllExcept

SplObjectStorage::removeAllExcept

(PHP 5 >= 5.3.6, PHP 7, PHP 8)

SplObjectStorage::removeAllExcept — Removes all objects except for those contained in another storage from the current storage

Description

public function **SplObjectStorage::removeAllExcept**(SplObjectStorage `$storage`): int

Removes all objects except for those contained in another storage from the current storage.

Parameters

`storage`

The storage containing the elements to retain in the current storage.

Return Values

Returns the number of remaining objects.

Examples

Example #1 SplObjectStorage::removeAllExcept() example

```php
<?php
$a = (object) 'a';
$b = (object) 'b';
$c = (object) 'c';

$foo = new SplObjectStorage;
$foo->attach($a);
$foo->attach($b);

$bar = new SplObjectStorage;
$bar->attach($b);
$bar->attach($c);

$foo->removeAllExcept($bar);
var_dump($foo->contains($a));
var_dump($foo->contains($b));
?>
```

The above example will output something similar to:

```
bool(false)
bool(true)
```

SplObjectStorage::rewind

SplObjectStorage::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::rewind — Rewind the iterator to the first storage element

Description

public function **SplObjectStorage::rewind**(): void

Rewind the iterator to the first storage element.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 SplObjectStorage::rewind() example

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $index  = $s->key();
    $object = $s->current(); // similar to current($s)
    $data   = $s->getInfo();
```

```
    var_dump($object);
    var_dump($data);
    $s->next();
}
?>
```

The above example will output something similar to:

```
object(stdClass)#2 (0) {
}
string(2) "d1"
object(stdClass)#3 (0) {
}
string(2) "d2"
```

See Also

- SplObjectStorage::next() - Move to the next entry

SplObjectStorage::seek

SplObjectStorage::seek

(PHP 8 >= 8.4.0)

SplObjectStorage::seek — Seeks iterator to a position

Description

public function **SplObjectStorage::seek**(int `$offset`): void

Seeks to a given position in the iterator.

Parameters

`offset`

The position to seek to.

Return Values

No value is returned.

Errors/Exceptions

Throws an OutOfBoundsException if the `offset` is not seekable.

Examples

Example #1 SplObjectStorage::seek() example

Seeks to item position 2 in the iterator.

```
<?php
class Test {
    public function __construct(public string $marker) {}
}

$a = new Test("a");
$b = new Test("b");
$c = new Test("c");

$storage = new SplObjectStorage();
$storage[$a] = "first";
```

```php
$storage[$b] = "second";
$storage[$c] = "third";

$storage->seek(2);
var_dump($storage->key());
var_dump($storage->current());
?>
```

The above example will output:

```
int(2)
object(Test)#3 (1) {
  ["marker"]=>
  string(1) "c"
}
```

See Also

- SeekableIterator

SplObjectStorage::serialize

SplObjectStorage::serialize

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplObjectStorage::serialize — Serializes the storage

Description

public function **SplObjectStorage::serialize**(): string

Returns a string representation of the storage.

Parameters

This function has no parameters.

Return Values

A string representing the storage.

Examples

Example #1 SplObjectStorage::serialize() example

```php
<?php
$s = new SplObjectStorage;
$o = new stdClass;
$s[$o] = "data";

echo $s->serialize()."\n";
?>
```

The above example will output something similar to:

```
x:i:1;O:8:"stdClass":0:{},s:4:"data";;m:a:0:{}
```

See Also

- SplObjectStorage::unserialize() - Unserializes a storage from its string representation

SplObjectStorage::setInfo

SplObjectStorage::setInfo

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

SplObjectStorage::setInfo — Sets the data associated with the current iterator entry

Description

public function **SplObjectStorage::setInfo**(mixed `$info`): void

Associates data, or info, with the object currently pointed to by the iterator.

Parameters

`info`

The data to associate with the current iterator entry.

Return Values

No value is returned.

Examples

Example #1 SplObjectStorage::setInfo() example

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    $s->setInfo("new");
    $s->next();
}
var_dump($s[$o1]);
var_dump($s[$o2]);
?>
```

The above example will output something similar to:

```
string(3) "new"
string(3) "new"
```

See Also

- SplObjectStorage::current() - Returns the current storage entry
- SplObjectStorage::rewind() - Rewind the iterator to the first storage element
- SplObjectStorage::key() - Returns the index at which the iterator currently is
- SplObjectStorage::next() - Move to the next entry
- SplObjectStorage::valid() - Returns if the current iterator entry is valid
- SplObjectStorage::getInfo() - Returns the data associated with the current iterator entry

SplObjectStorage::unserialize

SplObjectStorage::unserialize

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplObjectStorage::unserialize — Unserializes a storage from its string representation

Description

public function **SplObjectStorage::unserialize**(string `$data`): void

Unserializes storage entries and attach them to the current storage.

Parameters

`data`

The serialized representation of a storage.

Return Values

No value is returned.

Examples

Example #1 SplObjectStorage::unserialize() example

```php
<?php
$s1 = new SplObjectStorage;
$s2 = new SplObjectStorage;
$o = new stdClass;
$s1[$o] = "data";

$s2->unserialize($s1->serialize());

var_dump(count($s2));
?>
```

The above example will output something similar to:

```
int(1)
```

See Also

- SplObjectStorage::serialize() - Serializes the storage

SplObjectStorage::valid

SplObjectStorage::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplObjectStorage::valid — Returns if the current iterator entry is valid

Description

public function **SplObjectStorage::valid**(): bool

Returns if the current iterator entry is valid.

Parameters

This function has no parameters.

Return Values

Returns `true` if the iterator entry is valid, `false` otherwise.

Examples

Example #1 SplObjectStorage::valid() example

```php
<?php
$s = new SplObjectStorage();

$o1 = new stdClass;
$o2 = new stdClass;

$s->attach($o1, "d1");
$s->attach($o2, "d2");

$s->rewind();
while($s->valid()) {
    echo $s->key()."\n";
    $s->next();
}
?>
```

The above example will output something similar to:

```
0
1
```

See Also

- SplObjectStorage::current() - Returns the current storage entry
- SplObjectStorage::getInfo() - Returns the data associated with the current iterator entry

Exceptions

Table of Contents

SPL provides a set of standard Exceptions.

SPL Exceptions Class Tree

- LogicException (extends Exception)
    - BadFunctionCallException
        - BadMethodCallException
    - DomainException
    - InvalidArgumentException
    - LengthException
    - OutOfRangeException
- RuntimeException (extends Exception)
    - OutOfBoundsException
    - OverflowException
    - RangeException
    - UnderflowException
    - UnexpectedValueException

See Also

- Predefined Exceptions

BadFunctionCallException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if a callback refers to an undefined function or if some arguments are missing.

Class synopsis

class **BadFunctionCallException** extends LogicException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

BadMethodCallException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if a callback refers to an undefined method or if some arguments are missing.

Class synopsis

class **BadMethodCallException** extends BadFunctionCallException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string

private function Exception::__clone(): void
}

DomainException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if a value does not adhere to a defined valid data domain.

Class synopsis

class **DomainException** extends LogicException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

InvalidArgumentException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if an argument is not of the expected type.

Class synopsis

class **InvalidArgumentException** extends LogicException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void

}

LengthException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if a length is invalid.

Class synopsis

class **LengthException** extends LogicException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

LogicException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception that represents error in the program logic. This kind of exception should lead directly to a fix in your code.

Class synopsis

class **LogicException** extends Exception {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

OutOfBoundsException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if a value is not a valid key. This represents errors that cannot be detected at compile time.

Class synopsis

class **OutOfBoundsException** extends RuntimeException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

OutOfRangeException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown when an illegal index was requested. This represents errors that should be detected at compile time.

Class synopsis

class **OutOfRangeException** extends LogicException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

OverflowException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown when adding an element to a full container.

Class synopsis

```
class OverflowException extends RuntimeException {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

RangeException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown to indicate range errors during program execution. Normally this means there was an arithmetic error other than under/overflow. This is the runtime version of DomainException.

Class synopsis

```
class RangeException extends RuntimeException {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

RuntimeException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if an error which can only be found on runtime occurs.

Class synopsis

```
class RuntimeException extends Exception {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

UnderflowException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown when performing an invalid operation on an empty container, such as removing an element.

Class synopsis

```
class UnderflowException extends RuntimeException {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

UnexpectedValueException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Exception thrown if a value does not match with a set of values. Typically this happens when a function calls another function and expects the return value to be of a certain type or value not including arithmetic or buffer related errors.

Class synopsis

class **UnexpectedValueException** extends RuntimeException {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Iterators

Table of Contents

SPL provides a set of iterators to traverse over objects.

SPL Iterators Class Tree

- ArrayIterator
    - RecursiveArrayIterator
- EmptyIterator
- IteratorIterator
    - AppendIterator
    - CachingIterator
        - RecursiveCachingIterator
    - FilterIterator
        - CallbackFilterIterator
            - RecursiveCallbackFilterIterator
        - RecursiveFilterIterator
            - ParentIterator
        - RegexIterator
            - RecursiveRegexIterator
    - InfiniteIterator
    - LimitIterator

- NoRewindIterator
- MultipleIterator
- RecursiveIteratorIterator
  - RecursiveTreeIterator
- DirectoryIterator (extends SplFileInfo)
  - FilesystemIterator
    - GlobIterator
    - RecursiveDirectoryIterator

AppendIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

An Iterator that iterates over several iterators one after the other.

Class synopsis

class **AppendIterator** extends IteratorIterator {
/* Methods */
public function __construct()
public function append(Iterator `$iterator`): void
public function current(): mixed
public function getArrayIterator(): ArrayIterator
public function getIteratorIndex(): ?int
public function key(): scalar
public function next(): void
public function rewind(): void
public function valid(): bool
/* Inherited methods */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

AppendIterator::append

AppendIterator::append

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::append — Appends an iterator

Description

public function **AppendIterator::append**(Iterator `$iterator`): void

Appends an iterator.

Parameters

`iterator`

The iterator to append.

Return Values

No value is returned.

Examples

Example #1 AppendIterator::append() example

```php
<?php
$array_a = new ArrayIterator(array('a', 'b', 'c'));
$array_b = new ArrayIterator(array('d', 'e', 'f'));

$iterator = new AppendIterator;
$iterator->append($array_a);
$iterator->append($array_b);

foreach ($iterator as $current) {
    echo $current;
}
?>
```

The above example will output:

```
abcdef
```

See Also

- AppendIterator::__construct() - Constructs an AppendIterator

AppendIterator::__construct

AppendIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::__construct — Constructs an AppendIterator

Description

public function **AppendIterator::__construct**()

Constructs an AppendIterator.

Parameters

This function has no parameters.

Examples

Example #1 Iterating AppendIterator with foreach

```php
<?php
$pizzas   = new ArrayIterator(array('Margarita', 'Siciliana', 'Hawaii'));
$toppings = new ArrayIterator(array('Cheese', 'Anchovies', 'Olives', 'Pineapple', 'Ham'));

$appendIterator = new AppendIterator;
$appendIterator->append($pizzas);
$appendIterator->append($toppings);

foreach ($appendIterator as $key => $item) {
    echo $key . ' => ' . $item . PHP_EOL;
}
?>
```

The above example will output:

```
0 => Margarita
1 => Siciliana
2 => Hawaii
0 => Cheese
1 => Anchovies
2 => Olives
3 => Pineapple
4 => Ham
```

Example #2 Iterating AppendIterator with the AppendIterator API

```php
<?php
$pizzas   = new ArrayIterator(array('Margarita', 'Siciliana', 'Hawaii'));
$toppings = new ArrayIterator(array('Cheese', 'Anchovies', 'Olives', 'Pineapple', 'Ham'));

$appendIterator = new AppendIterator;
$appendIterator->append($pizzas);
$appendIterator->append($toppings);

while ($appendIterator->valid()) {
    printf(
        '%s => %s => %s%s',
        $appendIterator->getIteratorIndex(),
        $appendIterator->key(),
        $appendIterator->current(),
        PHP_EOL
    );
    $appendIterator->next();
}
?>
```

The above example will output:

```
0 => 0 => Margarita
0 => 1 => Siciliana
0 => 2 => Hawaii
1 => 0 => Cheese
1 => 1 => Anchovies
1 => 2 => Olives
1 => 3 => Pineapple
1 => 4 => Ham
```

Notes

> **Caution**
>
> When using iterator_to_array() to copy the values of the AppendIterator into an array, you have to set the optional `use_key` argument to **`false`**. When `use_key` is not **`false`** any keys reoccurring in inner iterators will get overwritten in the returned array. There is no way to preserve the original keys.

See Also

- AppendIterator::append() - Appends an iterator

AppendIterator::current

AppendIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::current — Gets the current value

Description

public function **AppendIterator::current**(): mixed

Gets the current value.

Parameters

This function has no parameters.

Return Values

The current value if it is valid or `null` otherwise.

See Also

- Iterator::current() - Return the current element
- AppendIterator::key() - Gets the current key
- AppendIterator::valid() - Checks validity of the current element
- AppendIterator::next() - Moves to the next element
- AppendIterator::rewind() - Rewinds the Iterator

AppendIterator::getArrayIterator

AppendIterator::getArrayIterator

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

AppendIterator::getArrayIterator — Gets the ArrayIterator

Description

public function **AppendIterator::getArrayIterator**(): ArrayIterator

This method gets the ArrayIterator that is used to store the iterators added with AppendIterator::append().

Parameters

This function has no parameters.

Return Values

Returns an ArrayIterator containing the appended iterators.

See Also

- **AppendIterator::getInnerIterator()**

AppendIterator::getIteratorIndex

AppendIterator::getIteratorIndex

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

AppendIterator::getIteratorIndex — Gets an index of iterators

Description

public function **AppendIterator::getIteratorIndex**(): ?int

Gets the index of the current inner iterator.

Parameters

This function has no parameters.

Return Values

Returns the zero-based, integer index of the current inner iterator if it exists, or `null` otherwise.

Examples

Example #1 AppendIterator.getIteratorIndex() basic example

```php
<?php
$array_a = new ArrayIterator(array('a' => 'aardwolf', 'b' => 'bear', 'c' => 'capybara'));
$array_b = new ArrayIterator(array('apple', 'orange', 'lemon'));

$iterator = new AppendIterator;
$iterator->append($array_a);
$iterator->append($array_b);

foreach ($iterator as $key => $current) {
    echo $iterator->getIteratorIndex() . '  ' . $key . ' ' . $current . PHP_EOL;
}
?>
```

The above example will output:

```
0  a aardwolf
0  b bear
0  c capybara
1  0 apple
1  1 orange
1  2 lemon
```

See Also

- **AppendIterator::getInnerIterator()**
- AppendIterator::getArrayIterator() - Gets the ArrayIterator

AppendIterator::key

AppendIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::key — Gets the current key

Description

public function **AppendIterator::key**(): scalar

Get the current key.

Parameters

This function has no parameters.

Return Values

The current key if it is valid or `null` otherwise.

Examples

Example #1 AppendIterator::key() basic example

```php
<?php
$array_a = new ArrayIterator(array('a' => 'aardwolf', 'b' => 'bear', 'c' => 'capybara'));
$array_b = new ArrayIterator(array('apple', 'orange', 'lemon'));

$iterator = new AppendIterator;
$iterator->append($array_a);
```

```php
$iterator->append($array_b);

// Manual iteration
$iterator->rewind();
while ($iterator->valid()) {
    echo $iterator->key() . ' ' . $iterator->current() . PHP_EOL;
    $iterator->next();
}

echo PHP_EOL;

// With foreach
foreach ($iterator as $key => $current) {
    echo $key . ' ' . $current . PHP_EOL;
}
?>
```

The above example will output:

```
a aardwolf
b bear
c capybara
0 apple
1 orange
2 lemon

a aardwolf
b bear
c capybara
0 apple
1 orange
2 lemon
```

See Also

- Iterator::key() - Return the key of the current element
- AppendIterator::current() - Gets the current value
- AppendIterator::valid() - Checks validity of the current element
- AppendIterator::next() - Moves to the next element
- AppendIterator::rewind() - Rewinds the Iterator

AppendIterator::next

AppendIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::next — Moves to the next element

Description

public function **AppendIterator::next**(): void

Moves to the next element. If this means to another Iterator then it rewinds that Iterator.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- Iterator::next() - Move forward to next element
- AppendIterator::current() - Gets the current value
- AppendIterator::key() - Gets the current key
- AppendIterator::valid() - Checks validity of the current element
- AppendIterator::rewind() - Rewinds the Iterator

AppendIterator::rewind

AppendIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::rewind — Rewinds the Iterator

Description

public function **AppendIterator::rewind**(): void

Rewind to the first element of the first inner Iterator.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- Iterator::rewind() - Rewind the Iterator to the first element
- AppendIterator::current() - Gets the current value
- AppendIterator::key() - Gets the current key
- AppendIterator::next() - Moves to the next element
- AppendIterator::valid() - Checks validity of the current element

AppendIterator::valid

AppendIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

AppendIterator::valid — Checks validity of the current element

Description

public function **AppendIterator::valid**(): bool

Checks validity of the current element.

Parameters

This function has no parameters.

Return Values

Returns `true` if the current iteration is valid, `false` otherwise.

See Also

- AppendIterator::current() - Gets the current value
- AppendIterator::key() - Gets the current key
- AppendIterator::next() - Moves to the next element
- AppendIterator::rewind() - Rewinds the Iterator

ArrayIterator

(PHP 5, PHP 7, PHP 8)

Introduction

Allows the removal of elements, and the modification of keys or values while iterating over arrays or objects.

To iterate the same array more than once, it is recommended to instantiate ArrayObject and use the **ArrayIterator** instance either implicitly created when using `foreach` to iterate over the array stored internally, or create one by calling the ArrayObject::getIterator() method manually.

Class synopsis

```
class ArrayIterator implements SeekableIterator, ArrayAccess, Serializable, Countable {
/* Constants */
public const int STD_PROP_LIST;
public const int ARRAY_AS_PROPS;
/* Methods */
public function __construct(array|object $array = [], int $flags = 0)
public function append(mixed $value): void
public function asort(int $flags = SORT_REGULAR): true
public function count(): int
public function current(): mixed
public function getArrayCopy(): array
public function getFlags(): int
public function key(): string|int|null
public function ksort(int $flags = SORT_REGULAR): true
public function natcasesort(): true
public function natsort(): true
public function next(): void
public function offsetExists(mixed $key): bool
public function offsetGet(mixed $key): mixed
public function offsetSet(mixed $key, mixed $value): void
public function offsetUnset(mixed $key): void
public function rewind(): void
public function seek(int $offset): void
public function serialize(): string
public function setFlags(int $flags): void
public function uasort(callable $callback): true
public function uksort(callable $callback): true
public function unserialize(string $data): void
public function valid(): bool
}
```

Predefined Constants

ArrayIterator Flags

`ArrayIterator::STD_PROP_LIST`

Properties of the object have their normal functionality when accessed as list (var_dump(), `foreach`, etc.).

`ArrayIterator::ARRAY_AS_PROPS`

Entries can be accessed as properties (read and write).

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

ArrayIterator::append

ArrayIterator::append

(PHP 5, PHP 7, PHP 8)

ArrayIterator::append — Append an element

Description

public function **ArrayIterator::append**(mixed `$value`): void

Appends value as the last element.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`value`

The value to append.

Return Values

No value is returned.

Notes

> **Note** :
>
> This method cannot be called when the ArrayIterator refers to an object.

See Also

- ArrayIterator::next() - Move to next entry

ArrayIterator::asort

ArrayIterator::asort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::asort — Sort entries by values

Description

public function **ArrayIterator::asort**(int `$flags` = `SORT_REGULAR`): true

Sorts entries by their values.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- `SORT_REGULAR` - compare items normally; the details are described in the comparison operators section
- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings
- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- `SORT_NATURAL` - compare items as strings using "natural ordering" like natsort()

- `SORT_FLAG_CASE` - can be combined (bitwise OR) with `SORT_STRING` or `SORT_NATURAL` to sort strings case-insensitively

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

See Also

- ArrayIterator::ksort() - Sort entries by keys
- ArrayIterator::natcasesort() - Sort entries naturally, case insensitive
- ArrayIterator::natsort() - Sort entries naturally
- ArrayIterator::uasort() - Sort with a user-defined comparison function and maintain index association
- ArrayIterator::uksort() - Sort by keys using a user-defined comparison function
- asort() - Sort an array in ascending order and maintain index association

ArrayIterator::__construct

ArrayIterator::__construct

(PHP 5, PHP 7, PHP 8)

ArrayIterator::__construct — Construct an ArrayIterator

Description

public function **ArrayIterator::__construct**(array|object `$array` = [], int `$flags` = 0)

Constructs an ArrayIterator object.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`array`

The array or object to be iterated on. Passing an object is deprecated as of PHP 8.5.0.

`flags`

Flags to control the behaviour of the ArrayIterator object. See ArrayIterator::setFlags().

Errors/Exceptions

As of PHP 8.5.0, throws an InvalidArgumentException if `array` is an enum.

Changelog

Version	Description
8.5.0	Passing an enum as `array` now throws an InvalidArgumentException.
8.5.0	Passing an object as `array` is deprecated.

See Also

- ArrayIterator::getArrayCopy() - Get array copy
- ArrayIterator::setFlags() - Set behaviour flags
- ArrayObject::__construct() - Construct a new array object

ArrayIterator::count

ArrayIterator::count

(PHP 5, PHP 7, PHP 8)

ArrayIterator::count — Count elements

Description

public function **ArrayIterator::count**(): int

Gets the number of elements in the array, or the number of public properties in the object.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The number of elements or public properties in the associated array or object, respectively.

See Also

- ArrayIterator::getFlags() - Get behavior flags

ArrayIterator::current

ArrayIterator::current

(PHP 5, PHP 7, PHP 8)

ArrayIterator::current — Return current array entry

Description

public function **ArrayIterator::current**(): mixed

Get the current array entry.

Parameters

This function has no parameters.

Return Values

The current array entry.

Examples

Example #1 ArrayIterator::current() example

```php
<?php
$array = array('1' => 'one',
               '2' => 'two',
               '3' => 'three');

$arrayobject = new ArrayObject($array);

for($iterator = $arrayobject->getIterator();
    $iterator->valid();
    $iterator->next()) {

    echo $iterator->key() . ' => ' . $iterator->current() . "\n";
}
?>
```

The above example will output:

```
```

```
1 => one
2 => two
3 => three
```

ArrayIterator::getArrayCopy

ArrayIterator::getArrayCopy

(PHP 5, PHP 7, PHP 8)

ArrayIterator::getArrayCopy — Get array copy

Description

public function **ArrayIterator::getArrayCopy**(): array

Get a copy of an array.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A copy of the array, or array of public properties if ArrayIterator refers to an object.

See Also

- ArrayIterator::valid() - Check whether array contains more entries

ArrayIterator::getFlags

ArrayIterator::getFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayIterator::getFlags — Get behavior flags

Description

public function **ArrayIterator::getFlags**(): int

Gets the behavior flags of the ArrayIterator. See the ArrayIterator::setFlags method for a list of the available flags.

Parameters

This function has no parameters.

Return Values

Returns the behavior flags of the ArrayIterator.

See Also

- ArrayIterator::setFlags() - Set behaviour flags
- ArrayIterator::valid() - Check whether array contains more entries

ArrayIterator::key

ArrayIterator::key

(PHP 5, PHP 7, PHP 8)

ArrayIterator::key — Return current array key

Description

public function **ArrayIterator::key**(): string|int|null

This function returns the current array key

Parameters

This function has no parameters.

Return Values

The current array key.

Examples

Example #1 ArrayIterator::key() example

```
<?php
$array = array('key' => 'value');

$arrayobject = new ArrayObject($array);
$iterator = $arrayobject->getIterator();

echo $iterator->key(); //key
?>
```

ArrayIterator::ksort

ArrayIterator::ksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::ksort — Sort entries by keys

Description

public function **ArrayIterator::ksort**(int `$flags` = `SORT_REGULAR`): true

Sorts entries by their keys.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`flags`

The optional second parameter `flags` may be used to modify the sorting behavior using these values:

Sorting type flags:

- `SORT_REGULAR` - compare items normally; the details are described in the comparison operators section
- `SORT_NUMERIC` - compare items numerically
- `SORT_STRING` - compare items as strings
- `SORT_LOCALE_STRING` - compare items as strings, based on the current locale. It uses the locale, which can be changed using setlocale()
- `SORT_NATURAL` - compare items as strings using "natural ordering" like natsort()
- `SORT_FLAG_CASE` - can be combined (bitwise OR) with `SORT_STRING` or `SORT_NATURAL` to sort strings case-insensitively

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

See Also

- ArrayIterator::asort() - Sort entries by values
- ArrayIterator::natcasesort() - Sort entries naturally, case insensitive
- ArrayIterator::natsort() - Sort entries naturally
- ArrayIterator::uasort() - Sort with a user-defined comparison function and maintain index association
- ArrayIterator::uksort() - Sort by keys using a user-defined comparison function
- ksort() - Sort an array by key in ascending order

ArrayIterator::natcasesort

ArrayIterator::natcasesort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::natcasesort — Sort entries naturally, case insensitive

Description

public function **ArrayIterator::natcasesort**(): true

Sort the entries by values using a case insensitive "natural order" algorithm.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

This function has no parameters.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

See Also

- ArrayIterator::asort() - Sort entries by values
- ArrayIterator::ksort() - Sort entries by keys
- ArrayIterator::natsort() - Sort entries naturally
- ArrayIterator::uasort() - Sort with a user-defined comparison function and maintain index association
- ArrayIterator::uksort() - Sort by keys using a user-defined comparison function
- natcasesort() - Sort an array using a case insensitive "natural order" algorithm

ArrayIterator::natsort

ArrayIterator::natsort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::natsort — Sort entries naturally

Description

public function **ArrayIterator::natsort**(): true

Sort the entries by values using a "natural order" algorithm.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

This function has no parameters.

Return Values

Always returns `true`.

Changelog

Version	Description
8.2.0	The return type is `true` now; previously, it was bool.

See Also

- ArrayIterator::asort() - Sort entries by values
- ArrayIterator::ksort() - Sort entries by keys
- ArrayIterator::natcasesort() - Sort entries naturally, case insensitive
- ArrayIterator::uasort() - Sort with a user-defined comparison function and maintain index association
- ArrayIterator::uksort() - Sort by keys using a user-defined comparison function
- natsort() - Sort an array using a "natural order" algorithm

ArrayIterator::next

ArrayIterator::next

(PHP 5, PHP 7, PHP 8)

ArrayIterator::next — Move to next entry

Description

public function **ArrayIterator::next**(): void

Moves the iterator to the next entry.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 ArrayIterator::next() example

```php
<?php
$arrayobject = new ArrayObject();

$arrayobject[] = 'zero';
$arrayobject[] = 'one';

$iterator = $arrayobject->getIterator();

while($iterator->valid()) {
    echo $iterator->key() . ' => ' . $iterator->current() . "\n";
```

```
    $iterator->next();
}
?>
```

The above example will output:

```
0 => zero
1 => one
```

ArrayIterator::offsetExists

ArrayIterator::offsetExists

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetExists — Check if offset exists

Description

public function **ArrayIterator::offsetExists**(mixed $key): bool

Checks if the offset exists.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

key

The offset being checked.

Return Values

`true` if the offset exists, otherwise `false`

See Also

- ArrayIterator::valid() - Check whether array contains more entries

ArrayIterator::offsetGet

ArrayIterator::offsetGet

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetGet — Get value for an offset

Description

public function **ArrayIterator::offsetGet**(mixed $key): mixed

Gets the value from the provided offset.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

key

The offset to get the value from.

Return Values

The value at offset `key`.

See Also

- ArrayIterator::offsetSet() - Set value for an offset
- ArrayIterator::offsetUnset() - Unset value for an offset

ArrayIterator::offsetSet

ArrayIterator::offsetSet

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetSet — Set value for an offset

Description

public function **ArrayIterator::offsetSet**(mixed `$key`, mixed `$value`): void

Sets a value for a given offset.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`key`

The index to set for.

`value`

The new value to store at the index.

Return Values

No value is returned.

See Also

- ArrayIterator::offsetGet() - Get value for an offset
- ArrayIterator::offsetUnset() - Unset value for an offset

ArrayIterator::offsetUnset

ArrayIterator::offsetUnset

(PHP 5, PHP 7, PHP 8)

ArrayIterator::offsetUnset — Unset value for an offset

Description

public function **ArrayIterator::offsetUnset**(mixed `$key`): void

Unsets a value for an offset.

If iteration is in progress, and **ArrayIterator::offsetUnset()** is used to unset the current index of iteration, the iteration position will be advanced to the next index. Since the iteration position is also advanced at the end of a `foreach` loop body, use of **ArrayIterator::offsetUnset()** inside a `foreach` loop may result in indices being skipped.

Parameters

`key`

The offset to unset.

Return Values

No value is returned.

See Also

- ArrayIterator::offsetGet() - Get value for an offset
- ArrayIterator::offsetSet() - Set value for an offset

ArrayIterator::rewind

ArrayIterator::rewind

(PHP 5, PHP 7, PHP 8)

ArrayIterator::rewind — Rewind array back to the start

Description

public function **ArrayIterator::rewind**(): void

This rewinds the iterator to the beginning.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 ArrayIterator::rewind() example

```
<?php
$arrayobject = new ArrayObject();

$arrayobject[] = 'zero';
$arrayobject[] = 'one';
$arrayobject[] = 'two';

$iterator = $arrayobject->getIterator();

$iterator->next();
echo $iterator->key(); //1

$iterator->rewind(); //rewinding to the beginning
echo $iterator->key(); //0
?>
```

ArrayIterator::seek

ArrayIterator::seek

(PHP 5, PHP 7, PHP 8)

ArrayIterator::seek — Seeks to a position

Description

public function **ArrayIterator::seek**(int `$offset`): void

Seeks to a given position in the iterator.

Parameters

`offset`

The position to seek to.

Return Values

No value is returned.

Errors/Exceptions

Throws an OutOfBoundsException if the `offset` is not seekable.

ArrayIterator::serialize

ArrayIterator::serialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayIterator::serialize — Serialize

Description

public function **ArrayIterator::serialize**(): string

Serialize.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The serialized ArrayIterator.

See Also

- ArrayIterator::unserialize() - Unserialize

ArrayIterator::setFlags

ArrayIterator::setFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ArrayIterator::setFlags — Set behaviour flags

Description

public function **ArrayIterator::setFlags**(int `$flags`): void

Set the flags that change the behavior of the ArrayIterator.

Parameters

`flags`

The new ArrayIterator behavior. It takes on either a bitmask, or named constants. Using named constants is strongly encouraged to ensure compatibility for future versions.

The available behavior flags are listed below. The actual meanings of these flags are described in the predefined constants.

ArrayIterator behavior flags

value	constant
1	ArrayIterator::STD_PROP_LIST
2	ArrayIterator::ARRAY_AS_PROPS

Return Values

No value is returned.

See Also

- ArrayIterator::getFlags() - Get behavior flags

ArrayIterator::uasort

ArrayIterator::uasort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::uasort — Sort with a user-defined comparison function and maintain index association

Description

public function **ArrayIterator::uasort**(callable `$callback`): true

This method sorts the elements such that indices maintain their correlation with the values they are associated with, using a user-defined comparison function.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`callback`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Always returns **`true`**.

Changelog

Version	Description
8.2.0	The return type is **`true`** now; previously, it was bool.

See Also

- ArrayIterator::asort() - Sort entries by values
- ArrayIterator::ksort() - Sort entries by keys
- ArrayIterator::natcasesort() - Sort entries naturally, case insensitive
- ArrayIterator::natsort() - Sort entries naturally
- ArrayIterator::uksort() - Sort by keys using a user-defined comparison function
- uasort() - Sort an array with a user-defined comparison function and maintain index association

ArrayIterator::uksort

ArrayIterator::uksort

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

ArrayIterator::uksort — Sort by keys using a user-defined comparison function

Description

public function **ArrayIterator::uksort**(callable `$callback`): true

This method sorts the elements by keys using a user-supplied comparison function.

> **Note** :
>
> If two members compare as equal, they retain their original order. Prior to PHP 8.0.0, their relative order in the sorted array was undefined.

Parameters

`callback`

The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

function callback(mixed `$a`, mixed `$b`): int

> **Caution**
>
> Returning *non-integer* values from the comparison function, such as float, will result in an internal cast to int of the callback's return value. So values such as `0.99` and `0.1` will both be cast to an integer value of `0`, which will compare such values as equal.

Return Values

Always returns **`true`**.

Changelog

Version	Description
8.2.0	The return type is **`true`** now; previously, it was bool.

See Also

- ArrayIterator::asort() - Sort entries by values
- ArrayIterator::ksort() - Sort entries by keys
- ArrayIterator::natcasesort() - Sort entries naturally, case insensitive
- ArrayIterator::natsort() - Sort entries naturally
- ArrayIterator::uasort() - Sort with a user-defined comparison function and maintain index association
- uksort() - Sort an array by keys using a user-defined comparison function

ArrayIterator::unserialize

ArrayIterator::unserialize

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

ArrayIterator::unserialize — Unserialize

Description

public function **ArrayIterator::unserialize**(string `$data`): void

Unserialize.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`data`

The serialized ArrayIterator object to be unserialized.

Return Values

No value is returned.

See Also

- ArrayIterator::serialize() - Serialize

ArrayIterator::valid

ArrayIterator::valid

(PHP 5, PHP 7, PHP 8)

ArrayIterator::valid — Check whether array contains more entries

Description

public function **ArrayIterator::valid**(): bool

Checks if the array contains any more entries.

Parameters

This function has no parameters.

Return Values

Returns `true` if the iterator is valid, otherwise `false`

Examples

Example #1 ArrayIterator::valid() example

```php
<?php
$array = array('1' => 'one');

$arrayobject = new ArrayObject($array);
$iterator = $arrayobject->getIterator();

var_dump($iterator->valid()); //bool(true)

$iterator->next(); // advance to the next item

//bool(false) because there is only one array element
var_dump($iterator->valid());
?>
```

CachingIterator

(PHP 5, PHP 7, PHP 8)

Introduction

This object supports cached iteration over another iterator.

Class synopsis

class **CachingIterator** extends IteratorIterator implements ArrayAccess, Countable, Stringable {
/* Constants */
public const int *CALL_TOSTRING*;
public const int *CATCH_GET_CHILD*;
public const int *TOSTRING_USE_KEY*;
public const int *TOSTRING_USE_CURRENT*;
public const int *TOSTRING_USE_INNER*;
public const int *FULL_CACHE*;
/* Methods */
public function __construct(Iterator `$iterator`, int `$flags` = `CachingIterator::CALL_TOSTRING`)
public function count(): int
public function current(): mixed
public function getCache(): array
public function getFlags(): int
public function hasNext(): bool

public function key(): scalar
public function next(): void
public function offsetExists(string $key): bool
public function offsetGet(string $key): mixed
public function offsetSet(string $key, mixed $value): void
public function offsetUnset(string $key): void
public function rewind(): void
public function setFlags(int $flags): void
public function __toString(): string
public function valid(): bool
/* Inherited methods */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Predefined Constants

`CachingIterator::CALL_TOSTRING`

Convert every element to string.

`CachingIterator::CATCH_GET_CHILD`

Don't throw exception in accessing children.

`CachingIterator::TOSTRING_USE_KEY`

Use key for conversion to string.

`CachingIterator::TOSTRING_USE_CURRENT`

Use current for conversion to string.

`CachingIterator::TOSTRING_USE_INNER`

Use inner for conversion to string.

`CachingIterator::FULL_CACHE`

Cache all read data.

Changelog

Version	Description
8.0.0	**CachingIterator** implements Stringable now.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

CachingIterator::__construct

CachingIterator::__construct

(PHP 5, PHP 7, PHP 8)

CachingIterator::__construct — Construct a new CachingIterator object for the iterator

Description

public function **CachingIterator::__construct**(Iterator $iterator, int $flags = **`CachingIterator::CALL_TOSTRING`**)

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

Iterator to cache

`flags`

Bitmask of flags.

CachingIterator::count

CachingIterator::count

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

CachingIterator::count — The number of elements in the iterator

Description

public function **CachingIterator::count**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

May return the number of elements in the iterator.

Parameters

This function has no parameters.

Return Values

The count of the elements iterated over.

CachingIterator::current

CachingIterator::current

(PHP 5, PHP 7, PHP 8)

CachingIterator::current — Return the current element

Description

public function **CachingIterator::current**(): mixed

> **Warning**
>
> This function is currently not documented; only its argument list is available.

May return the current element in the iteration.

Parameters

This function has no parameters.

Return Values

Mixed

See Also

- Iterator::current() - Return the current element

CachingIterator::getCache

CachingIterator::getCache

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::getCache — Retrieve the contents of the cache

Description

public function **CachingIterator::getCache**(): array

Retrieve the contents of the cache.

> **Note** :
>
> The `CachingIterator::FULL_CACHE` flag must be being used.

Parameters

This function has no parameters.

Return Values

An array containing the cache items.

Errors/Exceptions

Throws a BadMethodCallException when the `CachingIterator::FULL_CACHE` flag is not being used.

Examples

Example #1 CachingIterator::getCache() example

```php
<?php
$iterator = new ArrayIterator(array(1, 2, 3));
$cache    = new CachingIterator($iterator, CachingIterator::FULL_CACHE);

$cache->next();
$cache->next();
var_dump($cache->getCache());

$cache->next();
var_dump($cache->getCache());
?>
```

The above example will output:

```
array(2) {
  [0]=>
  int(1)
  [1]=>
  int(2)
}
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

CachingIterator::getFlags

CachingIterator::getFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::getFlags — Get flags used

Description

public function **CachingIterator::getFlags**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Get the bitmask of the flags used for this CachingIterator instance.

Parameters

This function has no parameters.

Return Values

Description...

CachingIterator::hasNext

CachingIterator::hasNext

(PHP 5, PHP 7, PHP 8)

CachingIterator::hasNext — Check whether the inner iterator has a valid next element

Description

public function **CachingIterator::hasNext**(): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

CachingIterator::key

CachingIterator::key

(PHP 5, PHP 7, PHP 8)

CachingIterator::key — Return the key for the current element

Description

public function **CachingIterator::key**(): scalar

> **Warning**
>
> This function is currently not documented; only its argument list is available.

This method may return a key for the current element.

Parameters

This function has no parameters.

CachingIterator::next

CachingIterator::next

(PHP 5, PHP 7, PHP 8)

CachingIterator::next — Move the iterator forward

Description

public function **CachingIterator::next**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Move the iterator forward.

Parameters

This function has no parameters.

Return Values

No value is returned.

CachingIterator::offsetExists

CachingIterator::offsetExists

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetExists — The offsetExists purpose

Description

public function **CachingIterator::offsetExists**(string $key): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

key

The index being checked.

Return Values

Returns `true` if an entry referenced by the offset exists, `false` otherwise.

CachingIterator::offsetGet

CachingIterator::offsetGet

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetGet — The offsetGet purpose

Description

public function **CachingIterator::offsetGet**(string $key): mixed

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`key`

Description...

Return Values

Description...

CachingIterator::offsetSet

CachingIterator::offsetSet

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetSet — The offsetSet purpose

Description

public function **CachingIterator::offsetSet**(string `$key`, mixed `$value`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`key`

The index of the element to be set.

`value`

The new value for the `key`.

Return Values

No value is returned.

CachingIterator::offsetUnset

CachingIterator::offsetUnset

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::offsetUnset — The offsetUnset purpose

Description

public function **CachingIterator::offsetUnset**(string `$key`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`key`

The index of the element to be unset.

Return Values

No value is returned.

CachingIterator::rewind

CachingIterator::rewind

(PHP 5, PHP 7, PHP 8)

CachingIterator::rewind — Rewind the iterator

Description

public function **CachingIterator::rewind**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Rewind the iterator.

Parameters

This function has no parameters.

Return Values

No value is returned.

CachingIterator::setFlags

CachingIterator::setFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

CachingIterator::setFlags — The setFlags purpose

Description

public function **CachingIterator::setFlags**(int `$flags`): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Set the flags for the CachingIterator object.

Parameters

`flags`

Bitmask of the flags to set.

Return Values

No value is returned.

CachingIterator::__toString

CachingIterator::__toString

(PHP 5, PHP 7, PHP 8)

CachingIterator::__toString — Return the string representation of the current element

Description

public function **CachingIterator::__toString**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Get the string representation of the current element.

Parameters

This function has no parameters.

Return Values

The string representation of the current element.

CachingIterator::valid

CachingIterator::valid

(PHP 5, PHP 7, PHP 8)

CachingIterator::valid — Check whether the current element is valid

Description

public function **CachingIterator::valid**(): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Check whether the current element is valid.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

CallbackFilterIterator

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introduction

Class synopsis

class **CallbackFilterIterator** extends FilterIterator {
/* Methods */
public function __construct(Iterator `$iterator`, callable `$callback`)
public function accept(): bool
/* Inherited methods */
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Examples

The callback should accept up to three arguments: the current item, the current key and the iterator, respectively.

Example #1 Available callback arguments

```
<?php

/**
```

```
 * Callback for CallbackFilterIterator
 *
 * @param $current   Current item's value
 * @param $key       Current item's key
 * @param $iterator  Iterator being filtered
 * @return boolean   TRUE to accept the current item, FALSE otherwise
 */
function my_callback($current, $key, $iterator) {
    // Your filtering code here
}

?>
```

Any callable may be used; such as a string containing a function name, an array for a method, or an anonymous function.

Example #2 Callback basic examples

```
<?php

$dir = new FilesystemIterator(__DIR__);

// Filter large files ( > 100MB)
function is_large_file($current) {
    return $current->isFile() && $current->getSize() > 104857600;
}
$large_files = new CallbackFilterIterator($dir, 'is_large_file');

// Filter directories
$files = new CallbackFilterIterator($dir, function ($current, $key, $iterator) {
    return $current->isDir() && ! $iterator->isDot();
});

?>
```

Table of Contents

CallbackFilterIterator::accept

CallbackFilterIterator::accept

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

CallbackFilterIterator::accept — Calls the callback with the current value, the current key and the inner iterator as arguments

Description

public function **CallbackFilterIterator::accept**(): bool

This method calls the callback with the current value, current key and the inner iterator.

The callback is expected to return `true` if the current item is to be accepted, or `false` otherwise.

Parameters

This function has no parameters.

Return Values

Returns `true` to accept the current item, or `false` otherwise.

See Also

- CallbackFilterIterator Examples
- CallbackFilterIterator::__construct() - Create a filtered iterator from another iterator

CallbackFilterIterator::__construct

CallbackFilterIterator::__construct

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

CallbackFilterIterator::__construct — Create a filtered iterator from another iterator

Description

public function **CallbackFilterIterator::__construct**(Iterator `$iterator`, callable `$callback`)

Creates a filtered iterator using the `callback` to determine which items are accepted or rejected.

Parameters

`iterator`

The iterator to be filtered.

`callback`

The callback, which should return **`true`** to accept the current item or **`false`** otherwise. See Examples.

May be any valid callable value.

See Also

- CallbackFilterIterator Examples
- CallbackFilterIterator::accept() - Calls the callback with the current value, the current key and the inner iterator as arguments

DirectoryIterator

(PHP 5, PHP 7, PHP 8)

Introduction

The DirectoryIterator class provides a simple interface for viewing the contents of filesystem directories.

Class synopsis

class **DirectoryIterator** extends SplFileInfo implements SeekableIterator {
/* Methods */
public function __construct(string `$directory`)
public function current(): mixed
public function getBasename(string `$suffix` = ""): string
public function getExtension(): string
public function getFilename(): string
public function isDot(): bool
public function key(): mixed
public function next(): void
public function rewind(): void
public function seek(int `$offset`): void
public function __toString(): string
public function valid(): bool
/* Inherited methods */
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string `$suffix` = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string `$class` = **`null`**): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string `$class` = **`null`**): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false

public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Table of Contents

DirectoryIterator::__construct

DirectoryIterator::__construct

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::__construct — Constructs a new directory iterator from a path

Description

public function **DirectoryIterator::__construct**(string $directory)

Constructs a new directory iterator from a path.

Parameters

`directory`

The path of the directory to traverse.

Errors/Exceptions

Throws an UnexpectedValueException if the `directory` does not exist.

Throws a ValueError if the `directory` is an empty string.

Changelog

Version	Description
8.0.0	Now throws a ValueError if `directory` is an empty string; previously it threw a RuntimeException.

Examples

Example #1 A DirectoryIterator::__construct() example

This example will list the contents of the directory containing the script.

```
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    if (!$fileinfo->isDot()) {
        var_dump($fileinfo->getFilename());
    }
}
?>
```

See Also

- SplFileInfo
- Iterator

DirectoryIterator::current

DirectoryIterator::current

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::current — Return the current DirectoryIterator item

Description

public function **DirectoryIterator::current**(): mixed

Get the current DirectoryIterator item.

Parameters

This function has no parameters.

Return Values

The current DirectoryIterator item.

Examples

Example #1 A DirectoryIterator::current() example

This example will list the contents of the directory containing the script.

```php
<?php
$iterator = new DirectoryIterator(__DIR__);
while($iterator->valid()) {
    $file = $iterator->current();
    echo $iterator->key() . " => " . $file->getFilename() . "\n";
    $iterator->next();
}
?>
```

The above example will output something similar to:

```
0 => .
1 => ..
2 => apple.jpg
3 => banana.jpg
4 => index.php
5 => pear.jpg
```

See Also

- DirectoryIterator::key() - Return the key for the current DirectoryIterator item
- DirectoryIterator::next() - Move forward to next DirectoryIterator item
- DirectoryIterator::rewind() - Rewind the DirectoryIterator back to the start
- DirectoryIterator::valid() - Check whether current DirectoryIterator position is a valid file
- Iterator::current() - Return the current element

DirectoryIterator::getBasename

DirectoryIterator::getBasename

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

DirectoryIterator::getBasename — Get base name of current DirectoryIterator item

Description

public function **DirectoryIterator::getBasename**(string `$suffix` = ""): string

Get the base name of the current DirectoryIterator item.

Parameters

`suffix`

If the base name ends in `suffix`, this will be cut.

Return Values

The base name of the current DirectoryIterator item.

Examples

Example #1 A DirectoryIterator::getBasename() example

This example will list the full base name and the base name with suffix `.jpg` removed for the files in the directory containing the script.

```
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    if ($fileinfo->isFile()) {
        echo $fileinfo->getBasename() . "\n";
        echo $fileinfo->getBasename('.jpg') . "\n";
    }
}
?>
```

The above example will output something similar to:

```
apple.jpg
apple
banana.jpg
banana
index.php
index.php
pear.jpg
pear
```

See Also

- DirectoryIterator::getFilename() - Return file name of current DirectoryIterator item
- **DirectoryIterator::getPath()**
- **DirectoryIterator::getPathname()**
- basename() - Returns trailing name component of path
- pathinfo() - Returns information about a file path

DirectoryIterator::getExtension

DirectoryIterator::getExtension

(PHP 5 >= 5.3.6, PHP 7, PHP 8)

DirectoryIterator::getExtension — Gets the file extension

Description

public function **DirectoryIterator::getExtension**(): string

Retrieves the file extension.

Parameters

This function has no parameters.

Return Values

Returns a string containing the file extension, or an empty string if the file has no extension.

Examples

Example #1 DirectoryIterator::getExtension() example

```php
<?php

$directory = new DirectoryIterator(__DIR__);
foreach ($directory as $fileinfo) {
    if ($fileinfo->isFile()) {
        echo $fileinfo->getExtension() . "\n";
    }
}

?>
```

The above example will output something similar to:

```
php
txt
jpg
gz
```

Notes

> **Note** :
>
> Another way of getting the extension is to use the pathinfo() function.
>
> ```php
> <?php
> $extension = pathinfo($fileinfo->getFilename(), PATHINFO_EXTENSION);
> ?>
> ```

See Also

- DirectoryIterator::getFilename() - Return file name of current DirectoryIterator item
- DirectoryIterator::getBasename() - Get base name of current DirectoryIterator item
- pathinfo() - Returns information about a file path

DirectoryIterator::getFilename

DirectoryIterator::getFilename

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::getFilename — Return file name of current DirectoryIterator item

Description

public function **DirectoryIterator::getFilename**(): string

Get the file name of the current DirectoryIterator item.

Parameters

This function has no parameters.

Return Values

Returns the file name of the current DirectoryIterator item.

Examples

Example #1 A **DirectoryIterator::getFilename()** example

This example will list the contents of the directory containing the script.

```php
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    echo $fileinfo->getFilename() . "\n";
}
?>
```

The above example will output something similar to:

```
.
..
apple.jpg
banana.jpg
index.php
pear.jpg
```

See Also

- DirectoryIterator::getBasename() - Get base name of current DirectoryIterator item
- **DirectoryIterator::getPath()**
- **DirectoryIterator::getPathname()**
- pathinfo() - Returns information about a file path

DirectoryIterator::isDot

DirectoryIterator::isDot

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::isDot — Determine if current DirectoryIterator item is '.' or '..'

Description

public function **DirectoryIterator::isDot**(): bool

Determines if the current DirectoryIterator item is a directory and either `.` or `..`

Parameters

This function has no parameters.

Return Values

`true` if the entry is `.` or `..`, otherwise `false`

Examples

Example #1 A **DirectoryIterator::isDot()** example

This example will list all files, omitting the `.` and `..` entries.

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));
foreach ($iterator as $fileinfo) {
    if (!$fileinfo->isDot()) {
        echo $fileinfo->getFilename() . "\n";
    }
}
?>
```

The above example will output something similar to:

```
apple.jpg
banana.jpg
example.php
pears.jpg
```

See Also

- **DirectoryIterator::getType()**
- **DirectoryIterator::isDir()**
- **DirectoryIterator::isFile()**
- **DirectoryIterator::isLink()**

DirectoryIterator::key

DirectoryIterator::key

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::key — Return the key for the current DirectoryIterator item

Description

public function **DirectoryIterator::key**(): mixed

Get the key for the current DirectoryIterator item.

Parameters

This function has no parameters.

Return Values

The key for the current DirectoryIterator item as an int.

Changelog

Version	Description
8.1.0	When the iterator is uninitialized, an Error is thrown now. Previously, the method returned `false`.

Examples

Example #1 A **DirectoryIterator::key()** example

```php
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    if (!$fileinfo->isDot()) {
        echo $fileinfo->key() . " => " . $fileinfo->getFilename() . "\n";
    }
}
?>
```

The above example will output something similar to:

```
0 => apple.jpg
1 => banana.jpg
2 => index.php
3 => pear.jpg
```

See Also

- DirectoryIterator::current() - Return the current DirectoryIterator item
- DirectoryIterator::next() - Move forward to next DirectoryIterator item
- DirectoryIterator::rewind() - Rewind the DirectoryIterator back to the start
- DirectoryIterator::valid() - Check whether current DirectoryIterator position is a valid file
- Iterator::key() - Return the key of the current element

DirectoryIterator::next

DirectoryIterator::next

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::next — Move forward to next DirectoryIterator item

Description

public function **DirectoryIterator::next**(): void

Move forward to the next DirectoryIterator item.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 DirectoryIterator::next() example

List the contents of a directory using a while loop.

```
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));
while($iterator->valid()) {
    echo $iterator->getFilename() . "\n";
    $iterator->next();
}
?>
```

The above example will output something similar to:

```
.
..
apple.jpg
banana.jpg
index.php
pear.jpg
```

See Also

- DirectoryIterator::current() - Return the current DirectoryIterator item
- DirectoryIterator::key() - Return the key for the current DirectoryIterator item
- DirectoryIterator::rewind() - Rewind the DirectoryIterator back to the start
- DirectoryIterator::valid() - Check whether current DirectoryIterator position is a valid file
- Iterator::next() - Move forward to next element

DirectoryIterator::rewind

DirectoryIterator::rewind

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::rewind — Rewind the DirectoryIterator back to the start

Description

public function **DirectoryIterator::rewind**(): void

Rewind the DirectoryIterator back to the start.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 DirectoryIterator::rewind() example

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));

$iterator->next();
echo $iterator->key(); //1

$iterator->rewind(); //rewinding to the beginning
echo $iterator->key(); //0
?>
```

See Also

- DirectoryIterator::current() - Return the current DirectoryIterator item
- DirectoryIterator::key() - Return the key for the current DirectoryIterator item
- DirectoryIterator::next() - Move forward to next DirectoryIterator item
- DirectoryIterator::valid() - Check whether current DirectoryIterator position is a valid file
- Iterator::rewind() - Rewind the Iterator to the first element

DirectoryIterator::seek

DirectoryIterator::seek

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

DirectoryIterator::seek — Seek to a DirectoryIterator item

Description

public function **DirectoryIterator::seek**(int `$offset`): void

Seek to a given position in the DirectoryIterator.

Parameters

`offset`

The zero-based numeric position to seek to.

Return Values

No value is returned.

Examples

Example #1 DirectoryIterator::seek() example

Seek to the fourth item in the directory containing the script. The first two are usually `.` and `..`

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));
$iterator->seek(3);
if ($iterator->valid()) {
    echo $iterator->getFilename();
} else {
    echo 'No file at position 3';
}
?>
```

See Also

- DirectoryIterator::rewind() - Rewind the DirectoryIterator back to the start
- DirectoryIterator::next() - Move forward to next DirectoryIterator item
- SeekableIterator::seek() - Seeks to a position

DirectoryIterator::__toString

DirectoryIterator::__toString

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::__toString — Get file name as a string

Description

public function **DirectoryIterator::__toString**(): string

Get the file name of the current DirectoryIterator item.

Parameters

This function has no parameters.

Return Values

Returns the file name of the current DirectoryIterator item.

Examples

Example #1 A DirectoryIterator::__toString() example

This example will list the contents of the directory containing the script.

```php
<?php
$dir = new DirectoryIterator(dirname(__FILE__));
foreach ($dir as $fileinfo) {
    echo $fileinfo;
}
?>
```

The above example will output something similar to:

```
.
..
apple.jpg
banana.jpg
index.php
pear.jpg
```

See Also

- DirectoryIterator::getFilename() - Return file name of current DirectoryIterator item
- The __toString() magic method

DirectoryIterator::valid

DirectoryIterator::valid

(PHP 5, PHP 7, PHP 8)

DirectoryIterator::valid — Check whether current DirectoryIterator position is a valid file

Description

public function **DirectoryIterator::valid**(): bool

Check whether current DirectoryIterator position is a valid file.

Parameters

This function has no parameters.

Return Values

Returns `true` if the position is valid, otherwise `false`

Examples

Example #1 A DirectoryIterator::valid() example

```php
<?php
$iterator = new DirectoryIterator(dirname(__FILE__));

// Loop to end of iterator
while($iterator->valid()) {
    $iterator->next();
}

$iterator->valid(); // FALSE
$iterator->rewind();
$iterator->valid(); // TRUE

?>
```

See Also

- DirectoryIterator::current() - Return the current DirectoryIterator item
- DirectoryIterator::key() - Return the key for the current DirectoryIterator item
- DirectoryIterator::next() - Move forward to next DirectoryIterator item
- DirectoryIterator::rewind() - Rewind the DirectoryIterator back to the start
- Iterator::valid() - Checks if current position is valid

EmptyIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The EmptyIterator class for an empty iterator.

Class synopsis

class **EmptyIterator** implements Iterator {
/* Methods */
public function current(): never
public function key(): never
public function next(): void
public function rewind(): void
public function valid(): false
}

Table of Contents

EmptyIterator::current

EmptyIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::current — The current() method

Description

public function **EmptyIterator::current**(): never

This function must not be called. It throws an exception upon access.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Errors/Exceptions

Throws an Exception if called.

Return Values

No value is returned.

EmptyIterator::key

EmptyIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::key — The key() method

Description

public function **EmptyIterator::key**(): never

This function must not be called. It throws an exception upon access.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Errors/Exceptions

Throws an Exception if called.

Return Values

No value is returned.

EmptyIterator::next

EmptyIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::next — The next() method

Description

public function **EmptyIterator::next**(): void

No operation, nothing to do.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

EmptyIterator::rewind

EmptyIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::rewind — The rewind() method

Description

public function **EmptyIterator::rewind**(): void

No operation, nothing to do.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

EmptyIterator::valid

EmptyIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

EmptyIterator::valid — Checks whether the current element is valid

Description

public function **EmptyIterator::valid**(): false

Checks whether the current element is valid.

> **Note** :
>
> The EmptyIterator is always empty and will never have a valid value. The return value is always `false`.

Parameters

This function has no parameters.

Return Values

`false`

FilesystemIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

The Filesystem iterator

Class synopsis

class **FilesystemIterator** extends DirectoryIterator {
/* Constants */
public const int *CURRENT_MODE_MASK*;
public const int *CURRENT_AS_PATHNAME*;
public const int *CURRENT_AS_FILEINFO*;
public const int *CURRENT_AS_SELF*;
public const int *KEY_MODE_MASK*;
public const int *KEY_AS_PATHNAME*;
public const int *FOLLOW_SYMLINKS*;
public const int *KEY_AS_FILENAME*;
public const int *NEW_CURRENT_AND_KEY*;
public const int *OTHER_MODE_MASK*;
public const int *SKIP_DOTS*;
public const int *UNIX_PATHS*;
/* Methods */
public function __construct(string `$directory`, int `$flags` = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO | FilesystemIterator::SKIP_DOTS)
public function current(): string|SplFileInfo|FilesystemIterator
public function getFlags(): int
public function key(): string
public function next(): void
public function rewind(): void
public function setFlags(int `$flags`): void
/* Inherited methods */
public function DirectoryIterator::current(): mixed
public function DirectoryIterator::getBasename(string `$suffix` = ""): string
public function DirectoryIterator::getExtension(): string
public function DirectoryIterator::getFilename(): string
public function DirectoryIterator::isDot(): bool
public function DirectoryIterator::key(): mixed
public function DirectoryIterator::next(): void
public function DirectoryIterator::rewind(): void
public function DirectoryIterator::seek(int `$offset`): void
public function DirectoryIterator::__toString(): string
public function DirectoryIterator::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string `$suffix` = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string `$class` = **`null`**): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string `$class` = **`null`**): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false

public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Predefined Constants

`FilesystemIterator::CURRENT_AS_PATHNAME`

Makes FilesystemIterator::current() return the pathname.

`FilesystemIterator::CURRENT_AS_FILEINFO`

Makes FilesystemIterator::current() return an SplFileInfo instance.

`FilesystemIterator::CURRENT_AS_SELF`

Makes FilesystemIterator::current() return $this (the FilesystemIterator).

`FilesystemIterator::CURRENT_MODE_MASK`

Masks FilesystemIterator::current()

`FilesystemIterator::KEY_AS_PATHNAME`

Makes FilesystemIterator::key() return the pathname.

`FilesystemIterator::KEY_AS_FILENAME`

Makes FilesystemIterator::key() return the filename.

`FilesystemIterator::FOLLOW_SYMLINKS`

Makes RecursiveDirectoryIterator::hasChildren() follow symlinks.

`FilesystemIterator::KEY_MODE_MASK`

Masks FilesystemIterator::key()

`FilesystemIterator::NEW_CURRENT_AND_KEY`

Same as `FilesystemIterator::KEY_AS_FILENAME | FilesystemIterator::CURRENT_AS_FILEINFO`.

`FilesystemIterator::OTHER_MODE_MASK`

Mask used for FilesystemIterator::getFlags() and FilesystemIterator::setFlags().

`FilesystemIterator::SKIP_DOTS`

Skips dot files (`.` and `..`).

`FilesystemIterator::UNIX_PATHS`

Makes paths use Unix-style forward slash irrespective of system default. Note that the `path` that is passed to the constructor is not modified.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

FilesystemIterator::__construct

FilesystemIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::__construct — Constructs a new filesystem iterator

Description

public function **FilesystemIterator::__construct**(string `$directory`, int `$flags` = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO | FilesystemIterator::SKIP_DOTS)

Constructs a new filesystem iterator from the `directory`.

Parameters

`directory`

The path of the filesystem item to be iterated over.

`flags`

Flags may be provided which will affect the behavior of some methods. A list of the flags can found under FilesystemIterator predefined constants. They can also be set later with FilesystemIterator::setFlags()

Errors/Exceptions

Throws an UnexpectedValueException if the `directory` does not exist.

Throws a ValueError if the `directory` is an empty string.

Changelog

Version	Description
8.2.0	Prior to PHP 8.2.0, `FilesystemIterator::SKIP_DOTS` was always set and could not be removed.
8.0.0	Now throws a ValueError if `directory` is an empty string; previously it threw a RuntimeException.

Examples

Example #1 FilesystemIterator::__construct() example

```php
<?php
$it = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::CURRENT_AS_FILEINFO);
foreach ($it as $fileinfo) {
    echo $fileinfo->getFilename() . "\n";
}
?>
```

Output of the above example in PHP 8.2 is similar to:

```
.
..
apples.jpg
banana.jpg
example.php
```

Output of the above example prior to PHP 8.2.0 is similar to:

```
apples.jpg
banana.jpg
example.php
```

See Also

- FilesystemIterator::setFlags() - Sets handling flags
- DirectoryIterator::__construct() - Constructs a new directory iterator from a path

FilesystemIterator::current

FilesystemIterator::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::current — The current file

Description

public function **FilesystemIterator::current**(): string|SplFileInfo|FilesystemIterator

Get file information of the current element.

Parameters

This function has no parameters.

Return Values

The filename, file information, or $this depending on the set flags. See the FilesystemIterator constants.

Examples

Example #1 FilesystemIterator::current() example

This example will list the contents of the directory containing the script.

```php
<?php
$iterator = new FilesystemIterator(__DIR__, FilesystemIterator::CURRENT_AS_PATHNAME);
foreach ($iterator as $fileinfo) {
    echo $iterator->current() . "\n";
}
?>
```

Output of the above example in PHP 8.2 is similar to:

```
/www/examples/.
/www/examples/..
/www/examples/apple.jpg
/www/examples/banana.jpg
/www/examples/example.php
```

See Also

- FilesystemIterator constants
- DirectoryIterator::current() - Return the current DirectoryIterator item
- DirectoryIterator::getFileName() - Return file name of current DirectoryIterator item

FilesystemIterator::getFlags

FilesystemIterator::getFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::getFlags — Get the handling flags

Description

public function **FilesystemIterator::getFlags**(): int

Gets the handling flags, as set in FilesystemIterator::__construct() or FilesystemIterator::setFlags().

Parameters

This function has no parameters.

Return Values

The integer value of the set flags.

See Also

- FilesystemIterator::__construct() - Constructs a new filesystem iterator
- FilesystemIterator::setFlags() - Sets handling flags

FilesystemIterator::key

FilesystemIterator::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::key — Retrieve the key for the current file

Description

public function **FilesystemIterator::key**(): string

Parameters

This function has no parameters.

Return Values

Returns the pathname or filename depending on the set flags. See the FilesystemIterator constants.

Examples

Example #1 FilesystemIterator::key() example

This example will list the contents of the directory containing the script.

```php
<?php
$iterator = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::KEY_AS_FILENAME);
foreach ($iterator as $fileinfo) {
    echo $iterator->key() . "\n";
}
?>
```

Output of the above example in PHP 8.2 is similar to:

```
.
..
apple.jpg
banana.jpg
example.php
```

See Also

- FilesystemIterator constants
- DirectoryIterator::key() - Return the key for the current DirectoryIterator item
- DirectoryIterator::getFilename() - Return file name of current DirectoryIterator item
- **DirectoryIterator::getPathname()**

FilesystemIterator::next

FilesystemIterator::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::next — Move to the next file

Description

public function **FilesystemIterator::next**(): void

Move to the next file.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 FilesystemIterator::next() example

List the contents of a directory using a while loop.

```
<?php
$iterator = new FilesystemIterator(dirname(__FILE__));
while($iterator->valid()) {
    echo $iterator->getFilename() . "\n";
    $iterator->next();
}
?>
```

The above example will output something similar to:

```
apple.jpg
banana.jpg
example.php
```

See Also

- DirectoryIterator::next() - Move forward to next DirectoryIterator item

FilesystemIterator::rewind

FilesystemIterator::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::rewind — Rewinds back to the beginning

Description

public function **FilesystemIterator::rewind**(): void

Rewinds the directory back to the start.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 FilesystemIterator::rewind() example

```php
<?php
$iterator = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::KEY_AS_FILENAME);

echo $iterator->key() . "\n";

$iterator->next();
echo $iterator->key() . "\n";

$iterator->rewind();
echo $iterator->key() . "\n";
?>
```

The above example will output something similar to:

```
apple.jpg
banana.jpg
apple.jpg
```

See Also

- DirectoryIterator::rewind() - Rewind the DirectoryIterator back to the start

FilesystemIterator::setFlags

FilesystemIterator::setFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

FilesystemIterator::setFlags — Sets handling flags

Description

public function **FilesystemIterator::setFlags**(int `$flags`): void

Sets handling flags.

Parameters

`flags`

The handling flags to set. See the FilesystemIterator constants.

Return Values

No value is returned.

Examples

Example #1 FilesystemIterator::key() example

This example demonstrates the difference between the FilesystemIterator::KEY_AS_PATHNAME and FilesystemIterator::KEY_AS_FILENAME flags.

```php
<?php
$iterator = new FilesystemIterator(dirname(__FILE__), FilesystemIterator::KEY_AS_PATHNAME);
echo "Key as Pathname:\n";
foreach ($iterator as $key => $fileinfo) {
    echo $key . "\n";
}

$iterator->setFlags(FilesystemIterator::KEY_AS_FILENAME);
echo "\nKey as Filename:\n";
foreach ($iterator as $key => $fileinfo) {
    echo $key . "\n";
}
?>
```

Output of the above example in PHP 8.2 is similar to:

```
Key as Pathname:
/www/examples/.
/www/examples/..
/www/examples/apple.jpg
/www/examples/banana.jpg
/www/examples/example.php

Key as Filename:
.
..
apple.jpg
banana.jpg
example.php
```

See Also

- FilesystemIterator::__construct() - Constructs a new filesystem iterator
- FilesystemIterator::getFlags() - Get the handling flags

FilterIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

This abstract iterator filters out unwanted values. This class should be extended to implement custom iterator filters. The FilterIterator::accept() must be implemented in the subclass.

Class synopsis

abstract class **FilterIterator** extends IteratorIterator {
/* Methods */
public function __construct(Iterator `$iterator`)
public function accept(): bool
public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
/* Inherited methods */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed

public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

FilterIterator::accept

FilterIterator::accept

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::accept — Check whether the current element of the iterator is acceptable

Description

public function **FilterIterator::accept**(): bool

Returns whether the current element of the iterator is acceptable through this filter.

Parameters

This function has no parameters.

Return Values

`true` if the current element is acceptable, otherwise `false`.

Examples

Example #1 FilterIterator::accept() example

```php
<?php
// This iterator filters all values with less than 10 characters
class LengthFilterIterator extends FilterIterator {

    public function accept() {
        // Only accept strings with a length of 10 and greater
        return strlen(parent::current()) >= 10;
    }

}

$arrayIterator = new ArrayIterator(array('test1', 'more than 10 characters'));
$lengthFilter = new LengthFilterIterator($arrayIterator);

foreach ($lengthFilter as $value) {
    echo $value . "\n";
}
?>
```

The above example will output:

```
more than 10 characters
```

FilterIterator::__construct

FilterIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::__construct — Construct a filterIterator

Description

public function **FilterIterator::__construct**(Iterator `$iterator`)

Constructs a new FilterIterator, which consists of a passed in `iterator` with filters applied to it.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

The iterator that is being filtered.

See Also

- LimitIterator::__construct() - Construct a LimitIterator

FilterIterator::current

FilterIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::current — Get the current element value

Description

public function **FilterIterator::current**(): mixed

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Get the current element value.

Parameters

This function has no parameters.

Return Values

The current element value.

See Also

- FilterIterator::key() - Get the current key
- FilterIterator::next() - Move the iterator forward

FilterIterator::key

FilterIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::key — Get the current key

Description

public function **FilterIterator::key**(): mixed

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Get the current key.

Parameters

This function has no parameters.

Return Values

The current key.

See Also

- FilterIterator::next() - Move the iterator forward
- FilterIterator::current() - Get the current element value

FilterIterator::next

FilterIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::next — Move the iterator forward

Description

public function **FilterIterator::next**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Move the iterator forward.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- FilterIterator::current() - Get the current element value
- FilterIterator::key() - Get the current key

FilterIterator::rewind

FilterIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::rewind — Rewind the iterator

Description

public function **FilterIterator::rewind**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Rewind the iterator.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- FilterIterator::current() - Get the current element value
- FilterIterator::key() - Get the current key
- FilterIterator::next() - Move the iterator forward

FilterIterator::valid

FilterIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

FilterIterator::valid — Check whether the current element is valid

Description

public function **FilterIterator::valid**(): bool

Checks whether the current element is valid.

> **Note** :
>
> The standard implementation of this function will initially return `false` until the inner iterator is advanced to the first accepted element.

Parameters

This function has no parameters.

Return Values

`true` if the current element is valid, otherwise `false`

GlobIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

Iterates through a file system in a similar fashion to glob().

Class synopsis

class **GlobIterator** extends FilesystemIterator implements Countable {
/* Inherited constants */
public const int *FilesystemIterator::CURRENT_MODE_MASK*;
public const int *FilesystemIterator::CURRENT_AS_PATHNAME*;
public const int *FilesystemIterator::CURRENT_AS_FILEINFO*;
public const int *FilesystemIterator::CURRENT_AS_SELF*;
public const int *FilesystemIterator::KEY_MODE_MASK*;
public const int *FilesystemIterator::KEY_AS_PATHNAME*;
public const int *FilesystemIterator::FOLLOW_SYMLINKS*;
public const int *FilesystemIterator::KEY_AS_FILENAME*;
public const int *FilesystemIterator::NEW_CURRENT_AND_KEY*;
public const int *FilesystemIterator::OTHER_MODE_MASK*;
public const int *FilesystemIterator::SKIP_DOTS*;
public const int *FilesystemIterator::UNIX_PATHS*;
/* Methods */
public function __construct(string `$pattern`, int `$flags` = FilesystemIterator::KEY_AS_PATHNAME |
FilesystemIterator::CURRENT_AS_FILEINFO)
public function count(): int
/* Inherited methods */
public function FilesystemIterator::current(): string|SplFileInfo|FilesystemIterator
public function FilesystemIterator::getFlags(): int
public function FilesystemIterator::key(): string
public function FilesystemIterator::next(): void
public function FilesystemIterator::rewind(): void
public function FilesystemIterator::setFlags(int `$flags`): void
public function DirectoryIterator::current(): mixed

public function DirectoryIterator::getBasename(string $suffix = ""): string
public function DirectoryIterator::getExtension(): string
public function DirectoryIterator::getFilename(): string
public function DirectoryIterator::isDot(): bool
public function DirectoryIterator::key(): mixed
public function DirectoryIterator::next(): void
public function DirectoryIterator::rewind(): void
public function DirectoryIterator::seek(int $offset): void
public function DirectoryIterator::__toString(): string
public function DirectoryIterator::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Table of Contents

GlobIterator::__construct

GlobIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

GlobIterator::__construct — Construct a directory using glob

Description

public function **GlobIterator::__construct**(string $pattern, int $flags = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO)

Constructs a new directory iterator from a glob expression.

Parameters

`pattern`

A glob() pattern.

`flags`

Option flags, the flags may be a bitmask of the FilesystemIterator constants.

Errors/Exceptions

Throws an UnexpectedValueException if the `directory` does not exist.

Throws a ValueError if the `directory` is an empty string.

Changelog

Version	Description
8.0.0	Now throws a ValueError if `directory` is an empty string; previously it threw a RuntimeException.

Examples

Example #1 GlobIterator example

```php
<?php
$iterator = new GlobIterator('*.dll', FilesystemIterator::KEY_AS_FILENAME);

if (!$iterator->count()) {
    echo 'No matches';
} else {
    $n = 0;

    printf("Matched %d item(s)\r\n", $iterator->count());

    foreach ($iterator as $item) {
        printf("[%d] %s\r\n", ++$n, $iterator->key());
    }
}
?>
```

The above example will output something similar to:

```
Matched 2 item(s)
[1] php5ts.dll
[2] php_gd2.dll
```

See Also

- DirectoryIterator::__construct() - Constructs a new directory iterator from a path
- GlobIterator::count() - Get the number of directories and files
- glob() - Find pathnames matching a pattern

GlobIterator::count

GlobIterator::count

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

GlobIterator::count — Get the number of directories and files

Description

public function **GlobIterator::count**(): int

Gets the number of directories and files found by the glob expression.

Parameters

This function has no parameters.

Return Values

The number of returned directories and files, as an int.

Examples

Example #1 GlobIterator::count() example

```php
<?php
$iterator = new GlobIterator('*.xml');

printf("Matched %d item(s)\r\n", $iterator->count());
?>
```

The above example will output something similar to:

```
Matched 8 item(s)
```

See Also

- GlobIterator::__construct() - Construct a directory using glob
- count() - Counts all elements in an array or in a Countable object
- glob() - Find pathnames matching a pattern

InfiniteIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The **InfiniteIterator** allows one to infinitely iterate over an iterator without having to manually rewind the iterator upon reaching its end.

Class synopsis

class **InfiniteIterator** extends IteratorIterator {
/* Methods */
public function __construct(Iterator `$iterator`)
public function next(): void
/* Inherited methods */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

InfiniteIterator::__construct

InfiniteIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

InfiniteIterator::__construct — Constructs an InfiniteIterator

Description

public function **InfiniteIterator::__construct**(Iterator `$iterator`)

Constructs an InfiniteIterator from an Iterator.

Parameters

`iterator`

The iterator to infinitely iterate over.

Examples

Example #1 InfiniteIterator::__construct() example

```php
<?php
$arrayit  = new ArrayIterator(array('cat','dog'));
$infinite = new InfiniteIterator($arrayit);
$limit    = new LimitIterator($infinite, 0, 7);
foreach($limit as $value)
{
    echo "$value\n";
}
?>
```

The above example will output:

```
cat
dog
cat
dog
cat
dog
cat
```

See Also

- InfiniteIterator::next() - Moves the inner Iterator forward or rewinds it

InfiniteIterator::next

InfiniteIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

InfiniteIterator::next — Moves the inner Iterator forward or rewinds it

Description

public function **InfiniteIterator::next**(): void

Moves the inner Iterator forward to its next element if there is one, otherwise rewinds the inner Iterator back to the beginning.

> **Note** :
>
> Even an InfiniteIterator stops if its inner Iterator is empty.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- InfiniteIterator::__construct() - Constructs an InfiniteIterator

IteratorIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

This iterator wrapper allows the conversion of anything that is Traversable into an Iterator. It is important to understand that most classes that do not implement Iterators have reasons as most likely they do not allow the full Iterator feature set. If so, techniques should be provided to prevent misuse, otherwise expect exceptions or fatal errors.

Class synopsis

class **IteratorIterator** implements OuterIterator {
/* Methods */
public function __construct(Traversable `$iterator`, ?string `$class` = `null`)
public function current(): mixed
public function getInnerIterator(): ?Iterator
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
}

Notes

> **Note** :
>
> This class permits access to methods of the inner iterator via the __call magic method.

Table of Contents

IteratorIterator::__construct

IteratorIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::__construct — Create an iterator from anything that is traversable

Description

public function **IteratorIterator::__construct**(Traversable `$iterator`, ?string `$class` = `null`)

Creates an iterator from anything that is traversable.

Parameters

`iterator`

The traversable iterator.

`class`

The class name to use for the inner iterator. It allows to specify a different iterator class to wrap the provided iterator. By default, it will use the `IteratorIterator` class itself.

See Also

- Traversable

IteratorIterator::current

IteratorIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::current — Get the current value

Description

public function **IteratorIterator::current**(): mixed

Get the value of the current element.

Parameters

This function has no parameters.

Return Values

The value of the current element.

See Also

- IteratorIterator::key() - Get the key of the current element

IteratorIterator::getInnerIterator

IteratorIterator::getInnerIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::getInnerIterator — Get the inner iterator

Description

public function **IteratorIterator::getInnerIterator**(): ?Iterator

Get the inner iterator.

Parameters

This function has no parameters.

Return Values

The inner iterator as passed to IteratorIterator::__construct(), or `null` when there is no inner iterator.

See Also

- Iterator
- OuterIterator

IteratorIterator::key

IteratorIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::key — Get the key of the current element

Description

public function **IteratorIterator::key**(): mixed

Get the key of the current element.

Parameters

This function has no parameters.

Return Values

The key of the current element.

See Also

- IteratorIterator::current() - Get the current value

IteratorIterator::next

IteratorIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::next — Forward to the next element

Description

public function **IteratorIterator::next**(): void

Forward to the next element.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- IteratorIterator::rewind() - Rewind to the first element
- IteratorIterator::valid() - Checks if the current element is valid

IteratorIterator::rewind

IteratorIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::rewind — Rewind to the first element

Description

public function **IteratorIterator::rewind**(): void

Rewinds to the first element.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- IteratorIterator::next() - Forward to the next element
- IteratorIterator::valid() - Checks if the current element is valid

IteratorIterator::valid

IteratorIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

IteratorIterator::valid — Checks if the current element is valid

Description

public function **IteratorIterator::valid**(): bool

Checks whether the current element is valid.

Parameters

This function has no parameters.

Return Values

Returns `true` if the current element is valid, otherwise `false`

See Also

- iterator_count() - Count the elements in an iterator
- IteratorIterator::current() - Get the current value

LimitIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The **LimitIterator** class allows iteration over a limited subset of items in an Iterator.

Class synopsis

class **LimitIterator** extends IteratorIterator {
/* Methods */
public function __construct(Iterator `$iterator`, int `$offset` = 0, int `$limit` = -1)
public function current(): mixed
public function getPosition(): int
public function key(): mixed
public function next(): void
public function rewind(): void
public function seek(int `$offset`): int
public function valid(): bool
/* Inherited methods */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Examples

Example #1 LimitIterator usage example

```php
<?php

// Create an iterator to be limited
$fruits = new ArrayIterator(array(
    'apple',
    'banana',
    'cherry',
    'damson',
    'elderberry'
));

// Loop over first three fruits only
foreach (new LimitIterator($fruits, 0, 3) as $fruit) {
    var_dump($fruit);
}

echo "\n";

// Loop from third fruit until the end
// Note: offset starts from zero for apple
foreach (new LimitIterator($fruits, 2) as $fruit) {
    var_dump($fruit);
}

?>
```

The above example will output:

```
string(5) "apple"
string(6) "banana"
string(6) "cherry"

string(6) "cherry"
```

```
string(6) "damson"
string(10) "elderberry"
```

Table of Contents

LimitIterator::__construct

LimitIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::__construct — Construct a LimitIterator

Description

public function **LimitIterator::__construct**(Iterator `$iterator`, int `$offset` = 0, int `$limit` = -1)

Constructs a new LimitIterator from an `iterator` with a given starting `offset` and maximum `limit`.

Parameters

`iterator`

The Iterator to limit.

`offset`

Optional offset of the limit.

`limit`

Optional count of the limit.

Errors/Exceptions

Throws a ValueError if the `offset` is less than `0` or the `limit` is less than `-1`.

Changelog

Version	Description
8.0.0	Now throws a ValueError if `offset` is less than `0`; previously it threw a RuntimeException.
8.0.0	Now throws a ValueError if `limit` is less than `-1`; previously it threw a RuntimeException.

Examples

Example #1 LimitIterator::__construct() example

```php
<?php
$ait = new ArrayIterator(array('a', 'b', 'c', 'd', 'e'));
$lit = new LimitIterator($ait, 1, 3);
foreach ($lit as $value) {
    echo $value . "\n";
}
?>
```

The above example will output:

```
b
c
d
```

See Also

- LimitIterator examples

LimitIterator::current

LimitIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::current — Get current element

Description

public function **LimitIterator::current**(): mixed

Gets the current element of the inner Iterator.

Parameters

This function has no parameters.

Return Values

Returns the current element or `null` if there is none.

See Also

- LimitIterator::key() - Get current key
- LimitIterator::next() - Move the iterator forward
- LimitIterator::rewind() - Rewind the iterator to the specified starting offset
- LimitIterator::seek() - Seek to the given position
- LimitIterator::valid() - Check whether the current element is valid

LimitIterator::getPosition

LimitIterator::getPosition

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::getPosition — Return the current position

Description

public function **LimitIterator::getPosition**(): int

Gets the current zero-based position of the inner Iterator.

Parameters

This function has no parameters.

Return Values

The current position.

Examples

Example #1 LimitIterator::getPosition() example

```php
<?php
$fruits = array(
    'a' => 'apple',
    'b' => 'banana',
    'c' => 'cherry',
    'd' => 'damson',
    'e' => 'elderberry'
);
$array_it = new ArrayIterator($fruits);
$limit_it = new LimitIterator($array_it, 2, 3);
foreach ($limit_it as $item) {
```

```
    echo $limit_it->getPosition() . ' ' . $item . "\n";
}
?>
```

The above example will output:

```
2 cherry
3 damson
4 elderberry
```

See Also

- FilterIterator::key() - Get the current key

LimitIterator::key

LimitIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::key — Get current key

Description

public function **LimitIterator::key**(): mixed

Gets the key for the current item in the inner Iterator.

Parameters

This function has no parameters.

Return Values

Returns the key for the current item.

See Also

- LimitIterator::getPosition() - Return the current position
- LimitIterator::current() - Get current element
- LimitIterator::next() - Move the iterator forward
- LimitIterator::rewind() - Rewind the iterator to the specified starting offset
- LimitIterator::seek() - Seek to the given position
- LimitIterator::valid() - Check whether the current element is valid

LimitIterator::next

LimitIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::next — Move the iterator forward

Description

public function **LimitIterator::next**(): void

Moves the iterator forward.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- LimitIterator::current() - Get current element
- LimitIterator::key() - Get current key
- LimitIterator::rewind() - Rewind the iterator to the specified starting offset
- LimitIterator::seek() - Seek to the given position
- LimitIterator::valid() - Check whether the current element is valid

LimitIterator::rewind

LimitIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::rewind — Rewind the iterator to the specified starting offset

Description

public function **LimitIterator::rewind**(): void

Rewinds the iterator to the starting offset specified in LimitIterator::__construct().

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- LimitIterator::current() - Get current element
- LimitIterator::key() - Get current key
- LimitIterator::next() - Move the iterator forward
- LimitIterator::seek() - Seek to the given position
- LimitIterator::valid() - Check whether the current element is valid

LimitIterator::seek

LimitIterator::seek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::seek — Seek to the given position

Description

public function **LimitIterator::seek**(int `$offset`): int

Moves the iterator to the offset specified by `offset`.

Parameters

`offset`

The position to seek to.

Return Values

Returns the offset position after seeking.

Errors/Exceptions

Throws an OutOfBoundsException if the position is outside of the limits specified in LimitIterator::__construct().

See Also

- LimitIterator::current() - Get current element

- LimitIterator::key() - Get current key
- LimitIterator::rewind() - Rewind the iterator to the specified starting offset
- LimitIterator::next() - Move the iterator forward
- LimitIterator::valid() - Check whether the current element is valid

LimitIterator::valid

LimitIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

LimitIterator::valid — Check whether the current element is valid

Description

public function **LimitIterator::valid**(): bool

Checks whether the current element is valid.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

See Also

- LimitIterator::current() - Get current element
- LimitIterator::key() - Get current key
- LimitIterator::rewind() - Rewind the iterator to the specified starting offset
- LimitIterator::next() - Move the iterator forward
- LimitIterator::seek() - Seek to the given position

MultipleIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

An Iterator that sequentially iterates over all attached iterators

Class synopsis

class **MultipleIterator** implements Iterator {
/* Constants */
public const int *MIT_NEED_ANY*;
public const int *MIT_NEED_ALL*;
public const int *MIT_KEYS_NUMERIC*;
public const int *MIT_KEYS_ASSOC*;
/* Methods */
public function __construct(int `$flags` = MultipleIterator::MIT_NEED_ALL | MultipleIterator::MIT_KEYS_NUMERIC)
public function attachIterator(Iterator `$iterator`, string|int|null `$info` = `null`): void
public function containsIterator(Iterator `$iterator`): bool
public function countIterators(): int
public function current(): array
public function detachIterator(Iterator `$iterator`): void
public function getFlags(): int
public function key(): array
public function next(): void
public function rewind(): void
public function setFlags(int `$flags`): void
public function valid(): bool
}

Predefined Constants

`MultipleIterator::MIT_NEED_ANY`

Do not require all sub iterators to be valid in iteration.

`MultipleIterator::MIT_NEED_ALL`

Require all sub iterators to be valid in iteration.

`MultipleIterator::MIT_KEYS_NUMERIC`

Keys are created from the sub iterators position.

`MultipleIterator::MIT_KEYS_ASSOC`

Keys are created from sub iterators associated information.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

MultipleIterator::attachIterator

MultipleIterator::attachIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::attachIterator — Attaches iterator information

Description

public function **MultipleIterator::attachIterator**(Iterator `$iterator`, string|int|null `$info` = `null`): void

Attaches iterator information.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

The new iterator to attach.

`info`

The associative information for the Iterator, which must be an int, a string, or `null`.

Return Values

Description...

Errors/Exceptions

An **IllegalValueException** if the `iterator` parameter is invalid, or if `info` is already associated information.

See Also

- MultipleIterator::__construct() - Constructs a new MultipleIterator

MultipleIterator::__construct

MultipleIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::__construct — Constructs a new MultipleIterator

Description

public function **MultipleIterator::__construct**(int `$flags` = MultipleIterator::MIT_NEED_ALL | MultipleIterator::MIT_KEYS_NUMERIC)

Construct a new MultipleIterator.

Parameters

`flags`

The flags to set, according to the Flag Constants.

- `MultipleIterator::MIT_NEED_ALL` or `MultipleIterator::MIT_NEED_ANY`
- `MultipleIterator::MIT_KEYS_NUMERIC` or `MultipleIterator::MIT_KEYS_ASSOC`

Defaults to `MultipleIterator::MIT_NEED_ALL|MultipleIterator::MIT_KEYS_NUMERIC`.

Examples

Example #1 Iterating a MultipleIterator

```php
<?php
$people = new ArrayIterator(array('John', 'Jane', 'Jack', 'Judy'));
$roles  = new ArrayIterator(array('Developer', 'Scrum Master', 'Project Owner'));

$team = new MultipleIterator($flags);
$team->attachIterator($people, 'person');
$team->attachIterator($roles, 'role');

foreach ($team as $member) {
    print_r($member);
}
?>
```

Output with `$flags = MIT_NEED_ALL|MIT_KEYS_NUMERIC`

```
Array
(
    [0] => John
    [1] => Developer
)
Array
(
    [0] => Jane
    [1] => Scrum Master
)
Array
(
    [0] => Jack
    [1] => Project Owner
)
```

Output with `$flags = MIT_NEED_ANY|MIT_KEYS_NUMERIC`

```
Array
(
    [0] => John
    [1] => Developer
)
Array
(
    [0] => Jane
    [1] => Scrum Master
```

```
)
Array
(
    [0] => Jack
    [1] => Project Owner
)
Array
(
    [0] => Judy
    [1] =>
)
```

Output with `$flags = MIT_NEED_ALL|MIT_KEYS_ASSOC`

```
Array
(
    [person] => John
    [role] => Developer
)
Array
(
    [person] => Jane
    [role] => Scrum Master
)
Array
(
    [person] => Jack
    [role] => Project Owner
)
```

Output with `$flags = MIT_NEED_ANY|MIT_KEYS_ASSOC`

```
Array
(
    [person] => John
    [role] => Developer
)
Array
(
    [person] => Jane
    [role] => Scrum Master
)
Array
(
    [person] => Jack
    [role] => Project Owner
)
Array
(
    [person] => Judy
    [role] =>
)
```

See Also

- Flag Constants

- MultipleIterator::valid() - Checks the validity of sub iterators

MultipleIterator::containsIterator

MultipleIterator::containsIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::containsIterator — Checks if an iterator is attached

Description

public function **MultipleIterator::containsIterator**(Iterator `$iterator`): bool

Checks if an iterator is attached or not.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

The iterator to check.

Return Values

Returns `true` on success or `false` on failure.

See Also

- MultipleIterator::valid() - Checks the validity of sub iterators

MultipleIterator::countIterators

MultipleIterator::countIterators

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::countIterators — Gets the number of attached iterator instances

Description

public function **MultipleIterator::countIterators**(): int

Gets the number of attached iterator instances.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The number of attached iterator instances (as an int).

See Also

- MultipleIterator::containsIterator() - Checks if an iterator is attached

MultipleIterator::current

MultipleIterator::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::current — Gets the registered iterator instances

Description

public function **MultipleIterator::current**(): array

Get the registered iterator instances current() result.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

An array containing the current values of each attached iterator.

Errors/Exceptions

A RuntimeException if the iterator is invalid (as of PHP 8.1.0), or mode `MIT_NEED_ALL` is set and at least one attached iterator is not valid. Or an **IllegalValueException** if a key is `null` and `MIT_KEYS_ASSOC` is set.

Changelog

Version	Description
8.1.0	A RuntimeException is now thrown if **MultipleIterator::current()** is called on an invalid iterator. Previously, `false` was returned.

See Also

- MultipleIterator::valid() - Checks the validity of sub iterators

MultipleIterator::detachIterator

MultipleIterator::detachIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::detachIterator — Detaches an iterator

Description

public function **MultipleIterator::detachIterator**(Iterator `$iterator`): void

Detaches an iterator.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

The iterator to detach.

Return Values

No value is returned.

See Also

- MultipleIterator::attachIterator() - Attaches iterator information

MultipleIterator::getFlags

MultipleIterator::getFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::getFlags — Gets the flag information

Description

public function **MultipleIterator::getFlags**(): int

Gets information about the flags.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Information about the flags, as an int.

See Also

- Flag Constants
- MultipleIterator::__construct() - Constructs a new MultipleIterator
- MultipleIterator::setFlags() - Sets flags

MultipleIterator::key

MultipleIterator::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::key — Gets the registered iterator instances

Description

public function **MultipleIterator::key**(): array

Get the registered iterator instances key() result.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

An array of all registered iterator instances.

Errors/Exceptions

A RuntimeException if the iterator is invalid (as of PHP 8.1.0), or mode `MIT_NEED_ALL` is set, and at least one attached iterator is not valid.

Calling this method from foreach triggers warning "Illegal type returned".

Changelog

Version	Description
8.1.0	A RuntimeException is now thrown if **MultipleIterator::key()** is called on an invalid iterator. Previously, `false` was returned.

See Also

- MultipleIterator::current() - Gets the registered iterator instances

MultipleIterator::next

MultipleIterator::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::next — Moves all attached iterator instances forward

Description

public function **MultipleIterator::next**(): void

Moves all attached iterator instances forward.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- MultipleIterator::rewind() - Rewinds all attached iterator instances

MultipleIterator::rewind

MultipleIterator::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::rewind — Rewinds all attached iterator instances

Description

public function **MultipleIterator::rewind**(): void

Rewinds all attached iterator instances.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- MultipleIterator::next() - Moves all attached iterator instances forward

MultipleIterator::setFlags

MultipleIterator::setFlags

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::setFlags — Sets flags

Description

public function **MultipleIterator::setFlags**(int `$flags`): void

Sets flags.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`flags`

The flags to set, according to the Flag Constants

Return Values

No value is returned.

See Also

- Flag Constants
- MultipleIterator::__construct() - Constructs a new MultipleIterator
- MultipleIterator::getFlags() - Gets the flag information

MultipleIterator::valid

MultipleIterator::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

MultipleIterator::valid — Checks the validity of sub iterators

Description

public function **MultipleIterator::valid**(): bool

Checks the validity of sub iterators.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns `true` if one or all sub iterators are valid depending on flags, otherwise `false`

See Also

- MultipleIterator::__construct() - Constructs a new MultipleIterator

NoRewindIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

This iterator ignores rewind operations. This allows processing an iterator in multiple partial foreach loops.

Class synopsis

class **NoRewindIterator** extends IteratorIterator {
/* Methods */
public function __construct(Iterator `$iterator`)

public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
/* Inherited methods */
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

NoRewindIterator::__construct

NoRewindIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::__construct — Construct a NoRewindIterator

Description

public function **NoRewindIterator::__construct**(Iterator `$iterator`)

Constructs a NoRewindIterator.

Parameters

`iterator`

The iterator being used.

Examples

Example #1 NoRewindIterator::__construct() example

The second loop does not output because the iterator is only used once, as it does not rewind.

```
<?php
$fruit = array('apple', 'banana', 'cranberry');

$arr = new ArrayObject($fruit);
$it  = new NoRewindIterator($arr->getIterator());

echo "Fruit A:\n";
foreach( $it as $item ) {
    echo $item . "\n";
}

echo "Fruit B:\n";
foreach( $it as $item ) {
    echo $item . "\n";
}
?>
```

The above example will output something similar to:

```
Fruit A:
apple
banana
cranberry
Fruit B:
```

See Also

- NoRewindIterator::valid() - Validates the iterator

NoRewindIterator::current

NoRewindIterator::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::current — Get the current value

Description

public function **NoRewindIterator::current**(): mixed

Gets the current value.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The current value.

See Also

- NoRewindIterator::key() - Get the current key

NoRewindIterator::key

NoRewindIterator::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::key — Get the current key

Description

public function **NoRewindIterator::key**(): mixed

Gets the current key.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The current key.

See Also

- NoRewindIterator::next() - Forward to the next element

NoRewindIterator::next

NoRewindIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::next — Forward to the next element

Description

public function **NoRewindIterator::next**(): void

Forwards to the next element.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

See Also

- NoRewindIterator::rewind() - Prevents the rewind operation on the inner iterator

NoRewindIterator::rewind

NoRewindIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::rewind — Prevents the rewind operation on the inner iterator

Description

public function **NoRewindIterator::rewind**(): void

Prevents the rewind operation on the inner iterator.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 NoRewindIterator::rewind() example

This example demonstrates that calling rewind on a NoRewindIterator object has no effect.

```php
<?php
$fruits = array("lemon", "orange", "apple", "pear");

$noRewindIterator = new NoRewindIterator(new ArrayIterator($fruits));

echo $noRewindIterator->current() . "\n";
$noRewindIterator->next();
// now rewind the iterator (nothing should happen)
$noRewindIterator->rewind();
echo $noRewindIterator->current() . "\n";
?>
```

The above example will output:

```
lemon
orange
```

NoRewindIterator::valid

NoRewindIterator::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

NoRewindIterator::valid — Validates the iterator

Description

public function **NoRewindIterator::valid**(): bool

Checks whether the iterator is valid.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

See Also

- **NoRewindIterator::getInnerIterator()**

ParentIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

This extended FilterIterator allows a recursive iteration using RecursiveIteratorIterator that only shows those elements which have children.

Class synopsis

class **ParentIterator** extends RecursiveFilterIterator {
/* Methods */
public function __construct(RecursiveIterator `$iterator`)
public function accept(): bool
public function getChildren(): ParentIterator
public function hasChildren(): bool
public function next(): void
public function rewind(): void
/* Inherited methods */
public function RecursiveFilterIterator::getChildren(): ?RecursiveFilterIterator
public function RecursiveFilterIterator::hasChildren(): bool
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator

public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

ParentIterator::accept

ParentIterator::accept

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::accept — Determines acceptability

Description

public function **ParentIterator::accept**(): bool

Determines if the current element has children.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if the current element is acceptable, otherwise `false`.

See Also

- ParentIterator::hasChildren() - Check whether the inner iterator's current element has children
- FilterIterator::accept() - Check whether the current element of the iterator is acceptable

ParentIterator::__construct

ParentIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::__construct — Constructs a ParentIterator

Description

public function **ParentIterator::__construct**(RecursiveIterator `$iterator`)

Constructs a ParentIterator on an iterator.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

The iterator being constructed upon.

ParentIterator::getChildren

ParentIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::getChildren — Return the inner iterator's children contained in a ParentIterator

Description

public function **ParentIterator::getChildren**(): ParentIterator

Get the inner iterator's children contained in a ParentIterator.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

An object.

ParentIterator::hasChildren

ParentIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::hasChildren — Check whether the inner iterator's current element has children

Description

public function **ParentIterator::hasChildren**(): bool

Check whether the inner iterator's current element has children.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

ParentIterator::next

ParentIterator::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::next — Move the iterator forward

Description

public function **ParentIterator::next**(): void

Moves the iterator forward.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

ParentIterator::rewind

ParentIterator::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ParentIterator::rewind — Rewind the iterator

Description

public function **ParentIterator::rewind**(): void

Rewinds the iterator.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveArrayIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

This iterator allows for unsetting and modifying values and keys while iterating over arrays and objects, in the same way as the ArrayIterator. Additionally, it is possible to iterate over the current iterator entry.

Class synopsis

class **RecursiveArrayIterator** extends ArrayIterator implements RecursiveIterator {
/* Inherited constants */
public const int *ArrayIterator::STD_PROP_LIST*;
public const int *ArrayIterator::ARRAY_AS_PROPS*;
/* Constants */
public const int *CHILD_ARRAYS_ONLY*;
/* Methods */
public function getChildren(): ?RecursiveArrayIterator
public function hasChildren(): bool
/* Inherited methods */
public function ArrayIterator::__construct(array|object `$array` = [], int `$flags` = 0)
public function ArrayIterator::append(mixed `$value`): void
public function ArrayIterator::asort(int `$flags` = **`SORT_REGULAR`**): true
public function ArrayIterator::count(): int
public function ArrayIterator::current(): mixed
public function ArrayIterator::getArrayCopy(): array
public function ArrayIterator::getFlags(): int
public function ArrayIterator::key(): string|int|null
public function ArrayIterator::ksort(int `$flags` = **`SORT_REGULAR`**): true
public function ArrayIterator::natcasesort(): true
public function ArrayIterator::natsort(): true
public function ArrayIterator::next(): void
public function ArrayIterator::offsetExists(mixed `$key`): bool
public function ArrayIterator::offsetGet(mixed `$key`): mixed
public function ArrayIterator::offsetSet(mixed `$key`, mixed `$value`): void
public function ArrayIterator::offsetUnset(mixed `$key`): void
public function ArrayIterator::rewind(): void

public function ArrayIterator::seek(int $offset): void
public function ArrayIterator::serialize(): string
public function ArrayIterator::setFlags(int $flags): void
public function ArrayIterator::uasort(callable $callback): true
public function ArrayIterator::uksort(callable $callback): true
public function ArrayIterator::unserialize(string $data): void
public function ArrayIterator::valid(): bool
}

Predefined Constants

RecursiveArrayIterator Flags

`RecursiveArrayIterator::CHILD_ARRAYS_ONLY`

Treat only arrays (not objects) as having children for recursive iteration.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

RecursiveArrayIterator::getChildren

RecursiveArrayIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveArrayIterator::getChildren — Returns an iterator for the current entry if it is an array or an object

Description

public function **RecursiveArrayIterator::getChildren**(): ?RecursiveArrayIterator

Returns an iterator for the current iterator entry.

Parameters

This function has no parameters.

Return Values

An iterator for the current entry, if it is an array or object; or `null` on failure.

Errors/Exceptions

An InvalidArgumentException will be thrown if the current entry does not contain an array or an object.

Examples

Example #1 RecursiveArrayIterator::getChildren() example

```php
<?php
$fruits = array("a" => "lemon", "b" => "orange", array("a" => "apple", "p" => "pear"));

$iterator = new RecursiveArrayIterator($fruits);

while ($iterator->valid()) {

    if ($iterator->hasChildren()) {
        // print all children
        foreach ($iterator->getChildren() as $key => $value) {
            echo $key . ' : ' . $value . "\n";
        }
    } else {
        echo "No children.\n";
    }
```

```
    $iterator->next();
}
?>
```

The above example will output:

```
No children.
No children.
a : apple
p : pear
```

See Also

- RecursiveArrayIterator::hasChildren() - Returns whether current entry is an array or an object

RecursiveArrayIterator::hasChildren

RecursiveArrayIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveArrayIterator::hasChildren — Returns whether current entry is an array or an object

Description

public function **RecursiveArrayIterator::hasChildren**(): bool

Returns whether current entry is an array or an object for which an iterator can be obtained via RecursiveArrayIterator::getChildren().

Parameters

This function has no parameters.

Return Values

Returns `true` if the current entry is an array or an object, otherwise `false` is returned.

Examples

Example #1 RecursiveArrayIterator::hasChildren() example

```php
<?php
$fruits = array("a" => "lemon", "b" => "orange", array("a" => "apple", "p" => "pear"));

$iterator = new RecursiveArrayIterator($fruits);

while ($iterator->valid()) {

    // Check if there are children
    if ($iterator->hasChildren()) {
        // print all children
        foreach ($iterator->getChildren() as $key => $value) {
            echo $key . ' : ' . $value . "\n";
        }
    } else {
        echo "No children.\n";
    }

    $iterator->next();
}
?>
```

The above example will output:

```
No children.
No children.
a : apple
p : pear
```

See Also

- RecursiveArrayIterator::getChildren() - Returns an iterator for the current entry if it is an array or an object

RecursiveCachingIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

...

Class synopsis

class **RecursiveCachingIterator** extends CachingIterator implements RecursiveIterator {
/* Inherited constants */
public const int *CachingIterator::CALL_TOSTRING*;
public const int *CachingIterator::CATCH_GET_CHILD*;
public const int *CachingIterator::TOSTRING_USE_KEY*;
public const int *CachingIterator::TOSTRING_USE_CURRENT*;
public const int *CachingIterator::TOSTRING_USE_INNER*;
public const int *CachingIterator::FULL_CACHE*;
/* Methods */
public function __construct(Iterator `$iterator`, int `$flags` = **`RecursiveCachingIterator::CALL_TOSTRING`**)
public function getChildren(): ?RecursiveCachingIterator
public function hasChildren(): bool
/* Inherited methods */
public function CachingIterator::count(): int
public function CachingIterator::current(): mixed
public function CachingIterator::getCache(): array
public function CachingIterator::getFlags(): int
public function CachingIterator::hasNext(): bool
public function CachingIterator::key(): scalar
public function CachingIterator::next(): void
public function CachingIterator::offsetExists(string `$key`): bool
public function CachingIterator::offsetGet(string `$key`): mixed
public function CachingIterator::offsetSet(string `$key`, mixed `$value`): void
public function CachingIterator::offsetUnset(string `$key`): void
public function CachingIterator::rewind(): void
public function CachingIterator::setFlags(int `$flags`): void
public function CachingIterator::__toString(): string
public function CachingIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

RecursiveCachingIterator::__construct

RecursiveCachingIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveCachingIterator::__construct — Construct

Description

public function **RecursiveCachingIterator::__construct**(Iterator `$iterator`, int `$flags` = **`RecursiveCachingIterator::CALL_TOSTRING`**)

Constructs a new RecursiveCachingIterator, which consists of a passed in `iterator`.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

The iterator being used.

`flags`

The flags. Use **`CALL_TOSTRING`** to call **RecursiveCachingIterator::__toString()** for every element (the default), and/or **`CATCH_GET_CHILD`** to catch exceptions when trying to get children.

See Also

- CachingIterator::__construct() - Construct a new CachingIterator object for the iterator

RecursiveCachingIterator::getChildren

RecursiveCachingIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveCachingIterator::getChildren — Return the inner iterator's children as a RecursiveCachingIterator

Description

public function **RecursiveCachingIterator::getChildren**(): ?RecursiveCachingIterator

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The inner iterator's children, as a RecursiveCachingIterator; or **`null`** if there is no children.

RecursiveCachingIterator::hasChildren

RecursiveCachingIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveCachingIterator::hasChildren — Check whether the current element of the inner iterator has children

Description

public function **RecursiveCachingIterator::hasChildren**(): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if the inner iterator has children, otherwise `false`

RecursiveCallbackFilterIterator

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Introduction

Class synopsis

class **RecursiveCallbackFilterIterator** extends CallbackFilterIterator implements RecursiveIterator {
/* Methods */
public function __construct(RecursiveIterator `$iterator`, callable `$callback`)
public function getChildren(): RecursiveCallbackFilterIterator
public function hasChildren(): bool
/* Inherited methods */
public function CallbackFilterIterator::accept(): bool
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Examples

The callback should accept up to three arguments: the current item, the current key and the iterator, respectively.

Example #1 Available callback arguments

```php
<?php

/**
 * Callback for RecursiveCallbackFilterIterator
 *
 * @param $current   Current item's value
 * @param $key       Current item's key
 * @param $iterator  Iterator being filtered
 * @return boolean   TRUE to accept the current item, FALSE otherwise
 */
function my_callback($current, $key, $iterator) {
    // Your filtering code here
}

?>
```

Filtering a recursive iterator generally involves two conditions. The first is that, to allow recursion, the callback function should return `true` if the current iterator item has children. The second is the normal filter condition, such as a file size or extension check as in the example below.

Example #2 Recursive callback basic example

```php
<?php

$dir = new RecursiveDirectoryIterator(__DIR__);

// Filter large files ( > 100MB)
$files = new RecursiveCallbackFilterIterator($dir, function ($current, $key, $iterator) {
    // Allow recursion
```

```
    if ($iterator->hasChildren()) {
        return TRUE;
    }
    // Check for large file
    if ($current->isFile() && $current->getSize() > 104857600) {
        return TRUE;
    }
    return FALSE;
});

foreach (new RecursiveIteratorIterator($files) as $file) {
    echo $file->getPathname() . PHP_EOL;
}

?>
```

Table of Contents

RecursiveCallbackFilterIterator::__construct

RecursiveCallbackFilterIterator::__construct

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RecursiveCallbackFilterIterator::__construct — Create a RecursiveCallbackFilterIterator from a RecursiveIterator

Description

public function **RecursiveCallbackFilterIterator::__construct**(RecursiveIterator `$iterator`, callable `$callback`)

Creates a filtered iterator from a RecursiveIterator using the `callback` to determine which items are accepted or rejected.

Parameters

`iterator`

The recursive iterator to be filtered.

`callback`

The callback, which should return **`true`** to accept the current item or **`false`** otherwise. See Examples.

May be any valid callable value.

See Also

- RecursiveCallbackFilterIterator Examples
- RecursiveCallbackFilterIterator::getChildren() - Return the inner iterator's children contained in a RecursiveCallbackFilterIterator
- RecursiveCallbackFilterIterator::hasChildren() - Check whether the inner iterator's current element has children

RecursiveCallbackFilterIterator::getChildren

RecursiveCallbackFilterIterator::getChildren

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RecursiveCallbackFilterIterator::getChildren — Return the inner iterator's children contained in a RecursiveCallbackFilterIterator

Description

public function **RecursiveCallbackFilterIterator::getChildren**(): RecursiveCallbackFilterIterator

Fetches the filtered children of the inner iterator.

RecursiveCallbackFilterIterator::hasChildren() should be used to determine if there are children to be fetched.

Parameters

This function has no parameters.

Return Values

Returns a RecursiveCallbackFilterIterator containing the children.

See Also

- RecursiveCallbackFilterIterator Examples
- RecursiveCallbackFilterIterator::__construct() - Create a RecursiveCallbackFilterIterator from a RecursiveIterator
- RecursiveCallbackFilterIterator::hasChildren() - Check whether the inner iterator's current element has children

RecursiveCallbackFilterIterator::hasChildren

RecursiveCallbackFilterIterator::hasChildren

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RecursiveCallbackFilterIterator::hasChildren — Check whether the inner iterator's current element has children

Description

public function **RecursiveCallbackFilterIterator::hasChildren**(): bool

Returns `true` if the current element has children, `false` otherwise.

Parameters

This function has no parameters.

Return Values

Returns `true` if the current element has children, `false` otherwise.

Examples

Example #1 RecursiveCallbackFilterIterator::hasChildren() basic usage

```php
<?php

$dir = new RecursiveDirectoryIterator(__DIR__);

// Recursively iterate over XML files
$files = new RecursiveCallbackFilterIterator($dir, function ($current, $key, $iterator) {
    // Allow recursion into directories
    if ($iterator->hasChildren()) {
        return TRUE;
    }
    // Check for XML file
    if (!strcasecmp($current->getExtension(), 'xml')) {
        return TRUE;
    }
    return FALSE;
});

?>
```

See Also

- RecursiveCallbackFilterIterator Examples
- RecursiveCallbackFilterIterator::__construct() - Create a RecursiveCallbackFilterIterator from a RecursiveIterator
- RecursiveCallbackFilterIterator::getChildren() - Return the inner iterator's children contained in a RecursiveCallbackFilterIterator

RecursiveDirectoryIterator

(PHP 5, PHP 7, PHP 8)

Introduction

The **RecursiveDirectoryIterator** provides an interface for iterating recursively over filesystem directories.

Class synopsis

class **RecursiveDirectoryIterator** extends FilesystemIterator implements RecursiveIterator {
/* Inherited constants */

```
public const int FilesystemIterator::CURRENT_MODE_MASK;
public const int FilesystemIterator::CURRENT_AS_PATHNAME;
public const int FilesystemIterator::CURRENT_AS_FILEINFO;
public const int FilesystemIterator::CURRENT_AS_SELF;
public const int FilesystemIterator::KEY_MODE_MASK;
public const int FilesystemIterator::KEY_AS_PATHNAME;
public const int FilesystemIterator::FOLLOW_SYMLINKS;
public const int FilesystemIterator::KEY_AS_FILENAME;
public const int FilesystemIterator::NEW_CURRENT_AND_KEY;
public const int FilesystemIterator::OTHER_MODE_MASK;
public const int FilesystemIterator::SKIP_DOTS;
public const int FilesystemIterator::UNIX_PATHS;
/* Methods */
public function __construct(string $directory, int $flags = FilesystemIterator::KEY_AS_PATHNAME |
FilesystemIterator::CURRENT_AS_FILEINFO)
public function getChildren(): RecursiveDirectoryIterator
public function getSubPath(): string
public function getSubPathname(): string
public function hasChildren(bool $allowLinks = false): bool
public function key(): string
public function next(): void
public function rewind(): void
/* Inherited methods */
public function FilesystemIterator::current(): string|SplFileInfo|FilesystemIterator
public function FilesystemIterator::getFlags(): int
public function FilesystemIterator::key(): string
public function FilesystemIterator::next(): void
public function FilesystemIterator::rewind(): void
public function FilesystemIterator::setFlags(int $flags): void
public function DirectoryIterator::current(): mixed
public function DirectoryIterator::getBasename(string $suffix = ""): string
public function DirectoryIterator::getExtension(): string
public function DirectoryIterator::getFilename(): string
public function DirectoryIterator::isDot(): bool
public function DirectoryIterator::key(): mixed
public function DirectoryIterator::next(): void
public function DirectoryIterator::rewind(): void
public function DirectoryIterator::seek(int $offset): void
public function DirectoryIterator::__toString(): string
public function DirectoryIterator::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
```

public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Table of Contents

RecursiveDirectoryIterator::__construct

RecursiveDirectoryIterator::__construct

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

RecursiveDirectoryIterator::__construct — Constructs a RecursiveDirectoryIterator

Description

public function **RecursiveDirectoryIterator::__construct**(string $directory, int $flags = FilesystemIterator::KEY_AS_PATHNAME | FilesystemIterator::CURRENT_AS_FILEINFO)

Constructs a **RecursiveDirectoryIterator()** for the provided `directory`.

Parameters

`directory`

The path of the directory to be iterated over.

`flags`

Flags may be provided which will affect the behavior of some methods. A list of the flags can found under FilesystemIterator predefined constants. They can also be set later with FilesystemIterator::setFlags().

Errors/Exceptions

Throws an UnexpectedValueException if the `directory` does not exist.

Throws a ValueError if the `directory` is an empty string.

Changelog

Version	Description
8.0.0	Now throws a ValueError if `directory` is an empty string; previously it threw a RuntimeException.

Examples

Example #1 RecursiveDirectoryIterator example

```php
<?php

$directory = '/tmp';

$it = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($directory));

$it->rewind();
while($it->valid()) {

    if (!$it->isDot()) {
        echo 'SubPathName: ' . $it->getSubPathName() . "\n";
        echo 'SubPath:     ' . $it->getSubPath() . "\n";
        echo 'Key:         ' . $it->key() . "\n\n";
    }

    $it->next();
}
```

```
?>
```

The above example will output something similar to:

```
SubPathName: fruit/apple.xml
SubPath:     fruit
Key:         /tmp/fruit/apple.xml

SubPathName: stuff.xml
SubPath:
Key:         /tmp/stuff.xml

SubPathName: veggies/carrot.xml
SubPath:     veggies
Key:         /tmp/veggies/carrot.xml
```

See Also

- FilesystemIterator::__construct() - Constructs a new filesystem iterator
- RecursiveIteratorIterator::__construct() - Construct a RecursiveIteratorIterator
- FilesystemIterator predefined constants

RecursiveDirectoryIterator::getChildren

RecursiveDirectoryIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveDirectoryIterator::getChildren — Returns an iterator for the current entry if it is a directory

Description

public function **RecursiveDirectoryIterator::getChildren**(): RecursiveDirectoryIterator

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The filename, file information, or $this depending on the set flags. See the FilesystemIterator constants.

RecursiveDirectoryIterator::getSubPath

RecursiveDirectoryIterator::getSubPath

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveDirectoryIterator::getSubPath — Get sub path

Description

public function **RecursiveDirectoryIterator::getSubPath**(): string

Returns the sub path relative to the directory given in the constructor.

Parameters

This function has no parameters.

Return Values

The sub path.

Examples

Example #1 getSubPath() example

```
$directory = '/tmp';

      $it = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($directory));

      foreach ($it as $file) {
          echo 'SubPathName: ' . $it->getSubPathName() . "\n";
          echo 'SubPath:     ' . $it->getSubPath() . "\n\n";
      }
```

The above example will output something similar to:

```
SubPathName: fruit/apple.xml
     SubPath:     fruit

     SubPathName: stuff.xml
     SubPath:

     SubPathName: veggies/carrot.xml
     SubPath:     veggies
```

See Also

- RecursiveDirectoryIterator::getSubPathName() - Get sub path and name
- RecursiveDirectoryIterator::key() - Return path and filename of current dir entry

RecursiveDirectoryIterator::getSubPathname

RecursiveDirectoryIterator::getSubPathname

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveDirectoryIterator::getSubPathname — Get sub path and name

Description

public function **RecursiveDirectoryIterator::getSubPathname**(): string

Gets the sub path and filename.

Parameters

This function has no parameters.

Return Values

The sub path (sub directory) and filename.

Examples

Example #1 getSubPathname() example

```
$directory = '/tmp';

      $it = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($directory));

      foreach ($it as $file) {
          echo 'SubPathName: ' . $it->getSubPathname() . "\n";
```

```
        echo 'SubPath:     ' . $it->getSubPath() . "\n\n";
    }
```

The above example will output something similar to:

```
SubPathName: fruit/apple.xml
    SubPath:     fruit

    SubPathName: stuff.xml
    SubPath:

    SubPathName: veggies/carrot.xml
    SubPath:     veggies
```

See Also

- RecursiveDirectoryIterator::getSubPath() - Get sub path
- RecursiveDirectoryIterator::key() - Return path and filename of current dir entry

RecursiveDirectoryIterator::hasChildren

RecursiveDirectoryIterator::hasChildren

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::hasChildren — Returns whether current entry is a directory and not '.' or '..'

Description

public function **RecursiveDirectoryIterator::hasChildren**(bool `$allowLinks = false`): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`allowLinks`

Return Values

Returns whether the current entry is a directory, but not '.' or '..'

RecursiveDirectoryIterator::key

RecursiveDirectoryIterator::key

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::key — Return path and filename of current dir entry

Description

public function **RecursiveDirectoryIterator::key**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The path and filename of the current dir entry.

RecursiveDirectoryIterator::next

RecursiveDirectoryIterator::next

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::next — Move to next entry

Description

public function **RecursiveDirectoryIterator::next**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveDirectoryIterator::rewind

RecursiveDirectoryIterator::rewind

(PHP 5, PHP 7, PHP 8)

RecursiveDirectoryIterator::rewind — Rewind dir back to the start

Description

public function **RecursiveDirectoryIterator::rewind**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveFilterIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

This abstract iterator filters out unwanted values for a RecursiveIterator. This class should be extended to implement custom filters. The **RecursiveFilterIterator::accept()** must be implemented in the subclass.

Class synopsis

abstract class **RecursiveFilterIterator** extends FilterIterator implements RecursiveIterator {
/* Methods */
public function __construct(RecursiveIterator `$iterator`)
public function getChildren(): ?RecursiveFilterIterator
public function hasChildren(): bool
/* Inherited methods */
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed

public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

RecursiveFilterIterator::__construct

RecursiveFilterIterator::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveFilterIterator::__construct — Create a RecursiveFilterIterator from a RecursiveIterator

Description

public function **RecursiveFilterIterator::__construct**(RecursiveIterator `$iterator`)

Create a RecursiveFilterIterator from a RecursiveIterator.

Parameters

`iterator`

The RecursiveIterator to be filtered.

Examples

Example #1 Basic RecursiveFilterIterator() example

```php
<?php
class TestsOnlyFilter extends RecursiveFilterIterator {
    public function accept() {
        // Accept the current item if we can recurse into it
        // or it is a value starting with "test"
        return $this->hasChildren() || (strpos($this->current(), "test") !== FALSE);
    }
}

$array    = array("test1", array("taste2", "test3", "test4"), "test5");
$iterator = new RecursiveArrayIterator($array);
$filter   = new TestsOnlyFilter($iterator);

foreach(new RecursiveIteratorIterator($filter) as $key => $value)
{
    echo $value . "\n";
}
?>
```

The above example will output something similar to:

```
test1
test3
test4
test5
```

Example #2 RecursiveFilterIterator() example

```php
<?php
class StartsWithFilter extends RecursiveFilterIterator {

    protected $word;

    public function __construct(RecursiveIterator $rit, $word) {
        $this->word = $word;
        parent::__construct($rit);
    }

    public function accept() {
        return $this->hasChildren() OR strpos($this->current(), $this->word) === 0;
    }

    public function getChildren() {
        return new self($this->getInnerIterator()->getChildren(), $this->word);
    }
}

$array    = array("test1", array("taste2", "test3", "test4"), "test5");
$iterator = new RecursiveArrayIterator($array);
$filter   = new StartsWithFilter($iterator, "test");

foreach(new RecursiveIteratorIterator($filter) as $key => $value)
{
    echo $value . "\n";
}
?>
```

The above example will output something similar to:

```
test1
test3
test4
test5
```

See Also

- RecursiveFilterIterator::getChildren() - Return the inner iterator's children contained in a RecursiveFilterIterator
- RecursiveFilterIterator::hasChildren() - Check whether the inner iterator's current element has children
- FilterIterator::accept() - Check whether the current element of the iterator is acceptable

RecursiveFilterIterator::getChildren

RecursiveFilterIterator::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveFilterIterator::getChildren — Return the inner iterator's children contained in a RecursiveFilterIterator

Description

public function **RecursiveFilterIterator::getChildren**(): ?RecursiveFilterIterator

Return the inner iterator's children contained in a RecursiveFilterIterator.

Parameters

This function has no parameters.

Return Values

Returns a RecursiveFilterIterator containing the inner iterator's children.

See Also

- RecursiveFilterIterator::hasChildren() - Check whether the inner iterator's current element has children
- RecursiveIterator::getChildren() - Returns an iterator for the current entry

RecursiveFilterIterator::hasChildren

RecursiveFilterIterator::hasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveFilterIterator::hasChildren — Check whether the inner iterator's current element has children

Description

public function **RecursiveFilterIterator::hasChildren**(): bool

Check whether the inner iterator's current element has children.

Parameters

This function has no parameters.

Return Values

`true` if the inner iterator has children, otherwise `false`

See Also

- RecursiveFilterIterator::getChildren() - Return the inner iterator's children contained in a RecursiveFilterIterator
- RecursiveIterator::hasChildren() - Returns if an iterator can be created for the current entry

RecursiveIteratorIterator

(PHP 5, PHP 7, PHP 8)

Introduction

Can be used to iterate through recursive iterators.

Class synopsis

class **RecursiveIteratorIterator** implements OuterIterator {
/* Constants */
public const int *LEAVES_ONLY*;
public const int *SELF_FIRST*;
public const int *CHILD_FIRST*;
public const int *CATCH_GET_CHILD*;
/* Methods */
public function __construct(Traversable `$iterator`, int `$mode` = `RecursiveIteratorIterator::LEAVES_ONLY`, int `$flags` = 0)
public function beginChildren(): void
public function beginIteration(): void
public function callGetChildren(): ?RecursiveIterator
public function callHasChildren(): bool
public function current(): mixed
public function endChildren(): void
public function endIteration(): void
public function getDepth(): int
public function getInnerIterator(): RecursiveIterator
public function getMaxDepth(): int|false
public function getSubIterator(?int `$level` = `null`): ?RecursiveIterator
public function key(): mixed
public function next(): void
public function nextElement(): void
public function rewind(): void
public function setMaxDepth(int `$maxDepth` = -1): void
public function valid(): bool
}

Predefined Constants

`RecursiveIteratorIterator::LEAVES_ONLY`

`RecursiveIteratorIterator::SELF_FIRST`

`RecursiveIteratorIterator::CHILD_FIRST`

`RecursiveIteratorIterator::CATCH_GET_CHILD`

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

RecursiveIteratorIterator::beginChildren

RecursiveIteratorIterator::beginChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::beginChildren — Begin children

Description

public function **RecursiveIteratorIterator::beginChildren**(): void

Is called after calling **RecursiveIteratorIterator::getChildren()**, and its associated RecursiveIteratorIterator::rewind().

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveIteratorIterator::beginIteration

RecursiveIteratorIterator::beginIteration

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::beginIteration — Begin Iteration

Description

public function **RecursiveIteratorIterator::beginIteration**(): void

Called when iteration begins (after the first RecursiveIteratorIterator::rewind() call.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveIteratorIterator::callGetChildren

RecursiveIteratorIterator::callGetChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::callGetChildren — Get children

Description

public function **RecursiveIteratorIterator::callGetChildren**(): ?RecursiveIterator

Get children of the current element.

Parameters

This function has no parameters.

Return Values

A **RecursiveIterator()** on success, or `null` on failure.

RecursiveIteratorIterator::callHasChildren

RecursiveIteratorIterator::callHasChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::callHasChildren — Has children

Description

public function **RecursiveIteratorIterator::callHasChildren**(): bool

Called for each element to test whether it has children.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns whether the element has children.

RecursiveIteratorIterator::__construct

RecursiveIteratorIterator::__construct

(PHP 5 >= 5.1.3, PHP 7, PHP 8)

RecursiveIteratorIterator::__construct — Construct a RecursiveIteratorIterator

Description

public function **RecursiveIteratorIterator::__construct**(Traversable `$iterator`, int `$mode = RecursiveIteratorIterator::LEAVES_ONLY`, int `$flags = 0`)

Creates a RecursiveIteratorIterator from a RecursiveIterator.

Parameters

`iterator`

The iterator being constructed from. Either a RecursiveIterator or IteratorAggregate.

`mode`

Optional mode. Possible values are

- `RecursiveIteratorIterator::LEAVES_ONLY` - The default. Lists only leaves in iteration.
- `RecursiveIteratorIterator::SELF_FIRST` - Lists leaves and parents in iteration with parents coming first.
- `RecursiveIteratorIterator::CHILD_FIRST` - Lists leaves and parents in iteration with leaves coming first.

`flags`

Optional flag. Possible values are `RecursiveIteratorIterator::CATCH_GET_CHILD` which will then ignore exceptions thrown in calls to **RecursiveIteratorIterator::getChildren()**.

Examples

Example #1 Iterating a RecursiveIteratorIterator

```php
<?php
$array = array(
    array(
        array(
            array(
                'leaf-0-0-0-0',
                'leaf-0-0-0-1'
            ),
            'leaf-0-0-0'
        ),
        array(
            array(
                'leaf-0-1-0-0',
                'leaf-0-1-0-1'
            ),
            'leaf-0-1-0'
        ),
        'leaf-0-0'
    )
);

$iterator = new RecursiveIteratorIterator(
    new RecursiveArrayIterator($array),
    $mode
);
foreach ($iterator as $key => $leaf) {
    echo "$key => $leaf", PHP_EOL;
}
?>
```

Output with `$mode = RecursiveIteratorIterator::LEAVES_ONLY`

```
0 => leaf-0-0-0-0
1 => leaf-0-0-0-1
0 => leaf-0-0-0
0 => leaf-0-1-0-0
1 => leaf-0-1-0-1
0 => leaf-0-1-0
0 => leaf-0-0
```

Output with `$mode = RecursiveIteratorIterator::SELF_FIRST`

```
0 => Array
0 => Array
0 => Array
0 => leaf-0-0-0-0
1 => leaf-0-0-0-1
1 => leaf-0-0-0
1 => Array
0 => Array
```

```
0 => leaf-0-1-0-0
1 => leaf-0-1-0-1
1 => leaf-0-1-0
2 => leaf-0-0
```

Output with `$mode = RecursiveIteratorIterator::CHILD_FIRST`

```
0 => leaf-0-0-0-0
1 => leaf-0-0-0-1
0 => Array
1 => leaf-0-0-0
0 => Array
0 => leaf-0-1-0-0
1 => leaf-0-1-0-1
0 => Array
1 => leaf-0-1-0
1 => Array
2 => leaf-0-0
0 => Array
```

RecursiveIteratorIterator::current

RecursiveIteratorIterator::current

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::current — Access the current element value

Description

public function **RecursiveIteratorIterator::current**(): mixed

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The current elements value.

RecursiveIteratorIterator::endChildren

RecursiveIteratorIterator::endChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::endChildren — End children

Description

public function **RecursiveIteratorIterator::endChildren**(): void

Called when end recursing one level.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveIteratorIterator::endIteration

RecursiveIteratorIterator::endIteration

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::endIteration — End Iteration

Description

public function **RecursiveIteratorIterator::endIteration**(): void

Called when the iteration ends (when RecursiveIteratorIterator::valid() first returns `false`.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveIteratorIterator::getDepth

RecursiveIteratorIterator::getDepth

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::getDepth — Get the current depth of the recursive iteration

Description

public function **RecursiveIteratorIterator::getDepth**(): int

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The current depth of the recursive iteration.

RecursiveIteratorIterator::getInnerIterator

RecursiveIteratorIterator::getInnerIterator

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::getInnerIterator — Get inner iterator

Description

public function **RecursiveIteratorIterator::getInnerIterator**(): RecursiveIterator

Gets the current active sub iterator.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The current active sub iterator.

RecursiveIteratorIterator::getMaxDepth

RecursiveIteratorIterator::getMaxDepth

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::getMaxDepth — Get max depth

Description

public function **RecursiveIteratorIterator::getMaxDepth**(): int|false

Gets the maximum allowable depth.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The maximum accepted depth, or `false` if any depth is allowed.

See Also

- RecursiveIteratorIterator::setMaxDepth() - Set max depth

RecursiveIteratorIterator::getSubIterator

RecursiveIteratorIterator::getSubIterator

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::getSubIterator — The current active sub iterator

Description

public function **RecursiveIteratorIterator::getSubIterator**(?int `$level = null`): ?RecursiveIterator

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`level`

Return Values

The current active sub iterator on success; `null` on failure.

Changelog

Version	Description
8.0.0	`level` is now nullable.

RecursiveIteratorIterator::key

RecursiveIteratorIterator::key

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::key — Access the current key

Description

public function **RecursiveIteratorIterator::key**(): mixed

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

The current key.

RecursiveIteratorIterator::next

RecursiveIteratorIterator::next

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::next — Move forward to the next element

Description

public function **RecursiveIteratorIterator::next**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveIteratorIterator::nextElement

RecursiveIteratorIterator::nextElement

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::nextElement — Next element

Description

public function **RecursiveIteratorIterator::nextElement**(): void

Called when the next element is available.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveIteratorIterator::rewind

RecursiveIteratorIterator::rewind

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::rewind — Rewind the iterator to the first element of the top level inner iterator

Description

public function **RecursiveIteratorIterator::rewind**(): void

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveIteratorIterator::setMaxDepth

RecursiveIteratorIterator::setMaxDepth

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

RecursiveIteratorIterator::setMaxDepth — Set max depth

Description

public function **RecursiveIteratorIterator::setMaxDepth**(int `$maxDepth = -1`): void

Set the maximum allowed depth.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`maxDepth`

The maximum allowed depth. `-1` is used for any depth.

Return Values

No value is returned.

Errors/Exceptions

Emits an Exception if `maxDepth` is less than `-1`.

RecursiveIteratorIterator::valid

RecursiveIteratorIterator::valid

(PHP 5, PHP 7, PHP 8)

RecursiveIteratorIterator::valid — Check whether the current position is valid

Description

public function **RecursiveIteratorIterator::valid**(): bool

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if the current position is valid, otherwise `false`

RecursiveRegexIterator

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introduction

This recursive iterator can filter another recursive iterator via a regular expression.

Class synopsis

```
class RecursiveRegexIterator extends RegexIterator implements RecursiveIterator {
/* Inherited constants */
public const int RegexIterator::USE_KEY;
public const int RegexIterator::INVERT_MATCH;
public const int RegexIterator::MATCH;
public const int RegexIterator::GET_MATCH;
public const int RegexIterator::ALL_MATCHES;
public const int RegexIterator::SPLIT;
public const int RegexIterator::REPLACE;
/* Inherited properties */
public ?string $replacement = null;
/* Methods */
public function __construct(
    RecursiveIterator $iterator,
    string $pattern,
    int $mode = RecursiveRegexIterator::MATCH,
    int $flags = 0,
    int $pregFlags = 0
)
public function getChildren(): RecursiveRegexIterator
public function hasChildren(): bool
/* Inherited methods */
public function RegexIterator::accept(): bool
public function RegexIterator::getFlags(): int
public function RegexIterator::getMode(): int
public function RegexIterator::getPregFlags(): int
public function RegexIterator::getRegex(): string
public function RegexIterator::setFlags(int $flags): void
public function RegexIterator::setMode(int $mode): void
public function RegexIterator::setPregFlags(int $pregFlags): void
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
```

public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}

Table of Contents

RecursiveRegexIterator::__construct

RecursiveRegexIterator::__construct

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RecursiveRegexIterator::__construct — Creates a new RecursiveRegexIterator

Description

public function **RecursiveRegexIterator::__construct**(
  RecursiveIterator `$iterator`,
  string `$pattern`,
  int `$mode` = **`RecursiveRegexIterator::MATCH`**,
  int `$flags` = 0,
  int `$pregFlags` = 0
)

Creates a new regular expression iterator.

Parameters

`iterator`

The recursive iterator to apply this regex filter to.

`pattern`

The regular expression to match.

`mode`

Operation mode, see RegexIterator::setMode() for a list of modes.

`flags`

Special flags, see RegexIterator::setFlags() for a list of available flags.

`pregFlags`

The regular expression flags. These flags depend on the operation mode parameter:

RegexIterator preg_flags

operation mode	available flags
RecursiveRegexIterator::ALL_MATCHES	See preg_match_all().
RecursiveRegexIterator::GET_MATCH	See preg_match().
RecursiveRegexIterator::MATCH	See preg_match().
RecursiveRegexIterator::REPLACE	none.
RecursiveRegexIterator::SPLIT	See preg_split().

Examples

Example #1 RecursiveRegexIterator::__construct() example

Creates a new RegexIterator that filters all strings that start with 'test'.

```
<?php
$rArrayIterator = new RecursiveArrayIterator(array('test1', array('tet3', 'test4', 'test5')));
```

```php
$rRegexIterator = new RecursiveRegexIterator($rArrayIterator, '/^test/',
    RecursiveRegexIterator::ALL_MATCHES);

foreach ($rRegexIterator as $key1 => $value1) {

    if ($rRegexIterator->hasChildren()) {

        // print all children
        echo "Children: ";
        foreach ($rRegexIterator->getChildren() as $key => $value) {
            echo $value . " ";
        }
        echo "\n";
    } else {
        echo "No children\n";
    }

}
?>
```

The above example will output something similar to:

```
No children
Children: test4 test5
```

See Also

- preg_match() - Perform a regular expression match
- preg_match_all() - Perform a global regular expression match
- preg_replace() - Perform a regular expression search and replace
- preg_split() - Split string by a regular expression

RecursiveRegexIterator::getChildren

RecursiveRegexIterator::getChildren

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RecursiveRegexIterator::getChildren — Returns an iterator for the current entry

Description

public function **RecursiveRegexIterator::getChildren**(): RecursiveRegexIterator

Returns an iterator for the current iterator entry.

Parameters

This function has no parameters.

Return Values

An iterator for the current entry, if it can be iterated over by the inner iterator.

Errors/Exceptions

An InvalidArgumentException will be thrown if the current entry does not contain a value that can be iterated over by the inner iterator.

Examples

Example #1 RecursiveRegexIterator::getChildren() example

```php
<?php
$rArrayIterator = new RecursiveArrayIterator(array('test1', array('tet3', 'test4', 'test5')));
$rRegexIterator = new RecursiveRegexIterator($rArrayIterator, '/^test/',
```

```
    RecursiveRegexIterator::ALL_MATCHES);

foreach ($rRegexIterator as $key1 => $value1) {

    if ($rRegexIterator->hasChildren()) {

        // print all children
        echo "Children: ";
        foreach ($rRegexIterator->getChildren() as $key => $value) {
            echo $value . " ";
        }
        echo "\n";
    } else {
        echo "No children\n";
    }

}
?>
```

The above example will output:

```
No children
Children: test4 test5
```

See Also

- RecursiveRegexIterator::hasChildren() - Returns whether an iterator can be obtained for the current entry

RecursiveRegexIterator::hasChildren

RecursiveRegexIterator::hasChildren

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RecursiveRegexIterator::hasChildren — Returns whether an iterator can be obtained for the current entry

Description

public function **RecursiveRegexIterator::hasChildren**(): bool

Returns whether an iterator can be obtained for the current entry. This iterator can be obtained via RecursiveRegexIterator::getChildren().

Parameters

This function has no parameters.

Return Values

Returns `true` if an iterator can be obtained for the current entry, otherwise returns `false`.

Examples

Example #1 RecursiveRegexIterator::hasChildren() example

```
<?php
$rArrayIterator = new RecursiveArrayIterator(array('test1', array('tet3', 'test4', 'test5')));
$rRegexIterator = new RecursiveRegexIterator($rArrayIterator, '/^test/',
    RecursiveRegexIterator::ALL_MATCHES);

foreach ($rRegexIterator as $value) {
    var_dump($rRegexIterator->hasChildren());
}
?>
```

The above example will output:

```
bool(false)
bool(true)
```

See Also

- RecursiveRegexIterator::getChildren() - Returns an iterator for the current entry

RecursiveTreeIterator

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

Allows iterating over a RecursiveIterator to generate an ASCII graphic tree.

Class synopsis

```
class RecursiveTreeIterator extends RecursiveIteratorIterator {
/* Inherited constants */
public const int RecursiveIteratorIterator::LEAVES_ONLY;
public const int RecursiveIteratorIterator::SELF_FIRST;
public const int RecursiveIteratorIterator::CHILD_FIRST;
public const int RecursiveIteratorIterator::CATCH_GET_CHILD;
/* Constants */
public const int BYPASS_CURRENT;
public const int BYPASS_KEY;
public const int PREFIX_LEFT;
public const int PREFIX_MID_HAS_NEXT = 1;
public const int PREFIX_MID_LAST = 2;
public const int PREFIX_END_HAS_NEXT = 3;
public const int PREFIX_END_LAST = 4;
public const int PREFIX_RIGHT = 5;
/* Methods */
public function __construct(
    RecursiveIterator|IteratorAggregate $iterator,
    int $flags = RecursiveTreeIterator::BYPASS_KEY,
    int $cachingIteratorFlags = CachingIterator::CATCH_GET_CHILD,
    int $mode = RecursiveTreeIterator::SELF_FIRST
)
public function beginChildren(): void
public function beginIteration(): RecursiveIterator
public function callGetChildren(): RecursiveIterator
public function callHasChildren(): bool
public function current(): mixed
public function endChildren(): void
public function endIteration(): void
public function getEntry(): string
public function getPostfix(): string
public function getPrefix(): string
public function key(): mixed
public function next(): void
public function nextElement(): void
public function rewind(): void
public function setPostfix(string $postfix): void
public function setPrefixPart(int $part, string $value): void
public function valid(): bool
/* Inherited methods */
public function RecursiveIteratorIterator::beginChildren(): void
public function RecursiveIteratorIterator::beginIteration(): void
```

public function RecursiveIteratorIterator::callGetChildren(): ?RecursiveIterator
public function RecursiveIteratorIterator::callHasChildren(): bool
public function RecursiveIteratorIterator::current(): mixed
public function RecursiveIteratorIterator::endChildren(): void
public function RecursiveIteratorIterator::endIteration(): void
public function RecursiveIteratorIterator::getDepth(): int
public function RecursiveIteratorIterator::getInnerIterator(): RecursiveIterator
public function RecursiveIteratorIterator::getMaxDepth(): int|false
public function RecursiveIteratorIterator::getSubIterator(?int `$level = null`): ?RecursiveIterator
public function RecursiveIteratorIterator::key(): mixed
public function RecursiveIteratorIterator::next(): void
public function RecursiveIteratorIterator::nextElement(): void
public function RecursiveIteratorIterator::rewind(): void
public function RecursiveIteratorIterator::setMaxDepth(int `$maxDepth = -1`): void
public function RecursiveIteratorIterator::valid(): bool
}

Predefined Constants

`RecursiveTreeIterator::BYPASS_CURRENT`

`RecursiveTreeIterator::BYPASS_KEY`

`RecursiveTreeIterator::PREFIX_LEFT`

`RecursiveTreeIterator::PREFIX_MID_HAS_NEXT`

`RecursiveTreeIterator::PREFIX_MID_LAST`

`RecursiveTreeIterator::PREFIX_END_HAS_NEXT`

`RecursiveTreeIterator::PREFIX_END_LAST`

`RecursiveTreeIterator::PREFIX_RIGHT`

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

RecursiveTreeIterator::beginChildren

RecursiveTreeIterator::beginChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::beginChildren — Begin children

Description

public function **RecursiveTreeIterator::beginChildren**(): void

Called when recursing one level down.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveTreeIterator::beginIteration

RecursiveTreeIterator::beginIteration

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::beginIteration — Begin iteration

Description

public function **RecursiveTreeIterator::beginIteration**(): RecursiveIterator

Called when iteration begins (after the first RecursiveTreeIterator::rewind() call).

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A RecursiveIterator.

RecursiveTreeIterator::callGetChildren

RecursiveTreeIterator::callGetChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::callGetChildren — Get children

Description

public function **RecursiveTreeIterator::callGetChildren**(): RecursiveIterator

Gets children of the current element.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

A RecursiveIterator.

RecursiveTreeIterator::callHasChildren

RecursiveTreeIterator::callHasChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::callHasChildren — Has children

Description

public function **RecursiveTreeIterator::callHasChildren**(): bool

Called for each element to test whether it has children.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if there are children, otherwise `false`

RecursiveTreeIterator::__construct

RecursiveTreeIterator::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::__construct — Construct a RecursiveTreeIterator

Description

public function **RecursiveTreeIterator::__construct**(
  RecursiveIterator|IteratorAggregate `$iterator`,
  int `$flags` = **`RecursiveTreeIterator::BYPASS_KEY`**,
  int `$cachingIteratorFlags` = **`CachingIterator::CATCH_GET_CHILD`**,
  int `$mode` = **`RecursiveTreeIterator::SELF_FIRST`**
)

Constructs a new RecursiveTreeIterator from the supplied recursive iterator.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`iterator`

The RecursiveIterator or IteratorAggregate to iterate over.

`flags`

Flags may be provided which will affect the behavior of some methods. A list of the flags can found under RecursiveTreeIterator predefined constants.

`caching_it_flags`

Flags to affect the behavior of the RecursiveCachingIterator used internally.

`mode`

Flags to affect the behavior of the RecursiveIteratorIterator used internally.

RecursiveTreeIterator::current

RecursiveTreeIterator::current

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::current — Get current element

Description

public function **RecursiveTreeIterator::current**(): mixed

Gets the current element prefixed and postfixed.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns the current element prefixed and postfixed.

RecursiveTreeIterator::endChildren

RecursiveTreeIterator::endChildren

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::endChildren — End children

Description

public function **RecursiveTreeIterator::endChildren**(): void

Called when end recursing one level.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveTreeIterator::endIteration

RecursiveTreeIterator::endIteration

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::endIteration — End iteration

Description

public function **RecursiveTreeIterator::endIteration**(): void

Called when the iteration ends (when RecursiveTreeIterator::valid() first returns `false`)

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveTreeIterator::getEntry

RecursiveTreeIterator::getEntry

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::getEntry — Get current entry

Description

public function **RecursiveTreeIterator::getEntry**(): string

Gets the part of the tree built for the current element.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns the part of the tree built for the current element.

RecursiveTreeIterator::getPostfix

RecursiveTreeIterator::getPostfix

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::getPostfix — Get the postfix

Description

public function **RecursiveTreeIterator::getPostfix**(): string

Gets the string to place after the current element.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns the string to place after the current element.

RecursiveTreeIterator::getPrefix

RecursiveTreeIterator::getPrefix

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::getPrefix — Get the prefix

Description

public function **RecursiveTreeIterator::getPrefix**(): string

Gets the string to place in front of current element

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns the string to place in front of current element

RecursiveTreeIterator::key

RecursiveTreeIterator::key

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::key — Get the key of the current element

Description

public function **RecursiveTreeIterator::key**(): mixed

Gets the current key prefixed and postfixed.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

Returns the current key prefixed and postfixed.

RecursiveTreeIterator::next

RecursiveTreeIterator::next

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::next — Move to next element

Description

public function **RecursiveTreeIterator::next**(): void

Moves forward to the next element.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveTreeIterator::nextElement

RecursiveTreeIterator::nextElement

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::nextElement — Next element

Description

public function **RecursiveTreeIterator::nextElement**(): void

Called when the next element is available.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveTreeIterator::rewind

RecursiveTreeIterator::rewind

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::rewind — Rewind iterator

Description

public function **RecursiveTreeIterator::rewind**(): void

Rewinds the iterator to the first element of the top level inner iterator.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

No value is returned.

RecursiveTreeIterator::setPostfix

RecursiveTreeIterator::setPostfix

(PHP 5 >= 5.5.3, PHP 7, PHP 8)

RecursiveTreeIterator::setPostfix — Set postfix

Description

public function **RecursiveTreeIterator::setPostfix**(string `$postfix`): void

Sets postfix as used in RecursiveTreeIterator::getPostfix().

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`postfix`

Return Values

No value is returned.

RecursiveTreeIterator::setPrefixPart

RecursiveTreeIterator::setPrefixPart

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::setPrefixPart — Set a part of the prefix

Description

public function **RecursiveTreeIterator::setPrefixPart**(int `$part`, string `$value`): void

Sets a part of the prefix used in the graphic tree.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

`part`

One of the RecursiveTreeIterator::PREFIX_* constants.

`value`

The value to assign to the part of the prefix specified in `part`.

Return Values

No value is returned.

RecursiveTreeIterator::valid

RecursiveTreeIterator::valid

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

RecursiveTreeIterator::valid — Check validity

Description

public function **RecursiveTreeIterator::valid**(): bool

Check whether the current position is valid.

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

`true` if the current position is valid, otherwise `false`

RegexIterator

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

Introduction

This iterator can be used to filter another iterator based on a regular expression.

Class synopsis

```
class RegexIterator extends FilterIterator {
/* Constants */
public const int USE_KEY;
public const int INVERT_MATCH;
public const int MATCH;
public const int GET_MATCH;
public const int ALL_MATCHES;
public const int SPLIT;
public const int REPLACE;
/* Properties */
public ?string $replacement = null;
/* Methods */
public function __construct(
    Iterator $iterator,
```

```
    string $pattern,
    int $mode = RegexIterator::MATCH,
    int $flags = 0,
    int $pregFlags = 0
)
public function accept(): bool
public function getFlags(): int
public function getMode(): int
public function getPregFlags(): int
public function getRegex(): string
public function setFlags(int $flags): void
public function setMode(int $mode): void
public function setPregFlags(int $pregFlags): void
/* Inherited methods */
public function FilterIterator::accept(): bool
public function FilterIterator::current(): mixed
public function FilterIterator::key(): mixed
public function FilterIterator::next(): void
public function FilterIterator::rewind(): void
public function FilterIterator::valid(): bool
public function IteratorIterator::current(): mixed
public function IteratorIterator::getInnerIterator(): ?Iterator
public function IteratorIterator::key(): mixed
public function IteratorIterator::next(): void
public function IteratorIterator::rewind(): void
public function IteratorIterator::valid(): bool
}
```

Predefined Constants

RegexIterator operation modes

`RegexIterator::ALL_MATCHES`

Return all matches for the current entry (see preg_match_all()).

`RegexIterator::GET_MATCH`

Return the first match for the current entry (see preg_match()).

`RegexIterator::MATCH`

Only execute match (filter) for the current entry (see preg_match()).

`RegexIterator::REPLACE`

Replace the current entry (see preg_replace(); Not fully implemented yet)

`RegexIterator::SPLIT`

Returns the split values for the current entry (see preg_split()).

RegexIterator Flags

`RegexIterator::USE_KEY`

Special flag: Match the entry key instead of the entry value.

`RegexIterator::INVERT_MATCH`

Inverts the return value of RegexIterator::accept().

Properties

replacement

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

RegexIterator::accept

RegexIterator::accept

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::accept — Get accept status

Description

public function **RegexIterator::accept**(): bool

Matches `(string)` **RegexIterator::current()** (or **RegexIterator::key()** if the RegexIterator::USE_KEY flag is set) against the regular expression.

Parameters

This function has no parameters.

Return Values

`true` if a match, `false` otherwise.

Examples

Example #1 RegexIterator::accept() example

This example shows that only items matching the regular expression are accepted.

```
<?php
$names = new ArrayIterator(array('Ann', 'Bob', 'Charlie', 'David'));
$filter = new RegexIterator($names, '/^[B-D]/');
foreach ($filter as $name) {
    echo $name . PHP_EOL;
}
?>
```

The above example will output:

```
Bob
Charlie
David
```

See Also

- RegexIterator constants
- RegexIterator::setFlags() - Sets the flags

RegexIterator::__construct

RegexIterator::__construct

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::__construct — Create a new RegexIterator

Description

public function **RegexIterator::__construct**(
  Iterator $iterator,
  string $pattern,
  int $mode = **RegexIterator::MATCH**,
  int $flags = 0,
  int $pregFlags = 0
)

Create a new RegexIterator which filters an Iterator using a regular expression.

Parameters

`iterator`

The iterator to apply this regex filter to.

`pattern`

The regular expression to match.

`mode`

Operation mode, see RegexIterator::setMode() for a list of modes.

`flags`

Special flags, see RegexIterator::setFlags() for a list of available flags.

`pregFlags`

The regular expression flags. These flags depend on the operation mode parameter:

RegexIterator preg_flags

operation mode	available flags
RegexIterator::ALL_MATCHES	See preg_match_all().
RegexIterator::GET_MATCH	See preg_match().
RegexIterator::MATCH	See preg_match().
RegexIterator::REPLACE	none.
RegexIterator::SPLIT	See preg_split().

Errors/Exceptions

Throws an InvalidArgumentException if the `pattern` argument is invalid.

Examples

Example #1 RegexIterator::__construct() example

Creates a new RegexIterator that filters all strings that start with 'test'.

```
<?php
$arrayIterator = new ArrayIterator(array('test 1', 'another test', 'test 123'));
$regexIterator = new RegexIterator($arrayIterator, '/^test/');

foreach ($regexIterator as $value) {
    echo $value . "\n";
}
?>
```

The above example will output something similar to:

```
test 1
test 123
```

See Also

- preg_match() - Perform a regular expression match
- preg_match_all() - Perform a global regular expression match
- preg_replace() - Perform a regular expression search and replace
- preg_split() - Split string by a regular expression

RegexIterator::getFlags

RegexIterator::getFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::getFlags — Get flags

Description

public function **RegexIterator::getFlags**(): int

Returns the flags, see RegexIterator::setFlags() for a list of available flags.

Parameters

This function has no parameters.

Return Values

Returns the set flags.

Examples

Example #1 RegexIterator::getFlags() example

```php
<?php

$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^test/');
$regexIterator->setFlags(RegexIterator::USE_KEY);

if ($regexIterator->getFlags() & RegexIterator::USE_KEY) {
    echo 'Filtering based on the array keys.';
} else {
    echo 'Filtering based on the array values.';
}
?>
```

The above example will output:

```
Filtering based on the array keys.
```

See Also

- RegexIterator::setFlags() - Sets the flags

RegexIterator::getMode

RegexIterator::getMode

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::getMode — Returns operation mode

Description

public function **RegexIterator::getMode**(): int

Returns the operation mode, see RegexIterator::setMode() for the list of operation modes.

Parameters

This function has no parameters.

Return Values

Returns the operation mode.

Examples

Example #1 RegexIterator::getMode() example

```php
<?php

$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^[a-z]+/', RegexIterator::GET_MATCH);

$mode = $regexIterator->getMode();
if ($mode & RegexIterator::GET_MATCH) {
    echo 'Getting the match for each item.';
} elseif ($mode & RegexIterator::ALL_MATCHES) {
    echo 'Getting all matches for each item.';
} elseif ($mode & RegexIterator::MATCH) {
    echo 'Getting each item if it matches.';
} elseif ($mode & RegexIterator::SPLIT) {
    echo 'Getting split pieces of each.';
}
?>
```

The above example will output:

```
Getting the match for each item.
```

See Also

- RegexIterator::setMode() - Sets the operation mode

RegexIterator::getPregFlags

RegexIterator::getPregFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::getPregFlags — Returns the regular expression flags

Description

public function **RegexIterator::getPregFlags**(): int

Returns the regular expression flags, see RegexIterator::__construct() for the list of flags.

Parameters

This function has no parameters.

Return Values

Returns a bitmask of the regular expression flags.

Examples

Example #1 RegexIterator::getPregFlags() example

```php
<?php

$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/\s/', RegexIterator::SPLIT);
$regexIterator->setPregFlags(PREG_SPLIT_NO_EMPTY | PREG_SPLIT_OFFSET_CAPTURE);

if ($regexIterator->getPregFlags() & PREG_SPLIT_NO_EMPTY) {
    echo 'Ignoring empty pieces';
} else {
    echo 'Not ignoring empty pieces';
}

?>
```

The above example will output:

```
Ignoring empty pieces
```

See Also

- RegexIterator::setPregFlags() - Sets the regular expression flags

RegexIterator::getRegex

RegexIterator::getRegex

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

RegexIterator::getRegex — Returns current regular expression

Description

public function **RegexIterator::getRegex**(): string

> **Warning**
>
> This function is currently not documented; only its argument list is available.

Parameters

This function has no parameters.

Return Values

RegexIterator::setFlags

RegexIterator::setFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::setFlags — Sets the flags

Description

public function **RegexIterator::setFlags**(int `$flags`): void

Sets the flags.

Parameters

`flags`

The flags to set, a bitmask of class constants.

The available flags are listed below. The actual meanings of these flags are described in the predefined constants.

RegexIterator flags

value	constant
1	RegexIterator::USE_KEY

Return Values

No value is returned.

Examples

Example #1 RegexIterator::setFlags() example

Creates a new RegexIterator that filters all entries whose key starts with '`test`'.

```
<?php
$test = array ('str1' => 'test 1', 'teststr2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^test/');
$regexIterator->setFlags(RegexIterator::USE_KEY);

foreach ($regexIterator as $key => $value) {
    echo $key . ' => ' . $value . "\n";
}
?>
```

The above example will output:

```
teststr2 => another test
```

See Also

- RegexIterator::getFlags() - Get flags

RegexIterator::setMode

RegexIterator::setMode

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::setMode — Sets the operation mode

Description

public function **RegexIterator::setMode**(int `$mode`): void

Sets the operation mode.

Parameters

`mode`

The operation mode.

The available modes are listed below. The actual meanings of these modes are described in the predefined constants.

RegexIterator modes

value	constant
0	RegexIterator::MATCH
1	RegexIterator::GET_MATCH
2	RegexIterator::ALL_MATCHES
3	RegexIterator::SPLIT
4	RegexIterator::REPLACE

Return Values

No value is returned.

Examples

Example #1 RegexIterator::setMode() example

```php
<?php
$test = array ('str1' => 'test 1', 'test str2' => 'another test', 'str3' => 'test 123');

$arrayIterator = new ArrayIterator($test);
// Filter everything that starts with 'test ' followed by one or more numbers.
$regexIterator = new RegexIterator($arrayIterator, '/^test (\d+)/');
// Operation mode: Replace actual value with the matches
$regexIterator->setMode(RegexIterator::GET_MATCH);

foreach ($regexIterator as $key => $value) {
    // print out the matched number(s)
    echo $key . ' => ' . $value[1] . PHP_EOL;
}
?>
```

The above example will output something similar to:

```
str1 => 1
str3 => 123
```

See Also

- RegexIterator::getMode() - Returns operation mode

RegexIterator::setPregFlags

RegexIterator::setPregFlags

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

RegexIterator::setPregFlags — Sets the regular expression flags

Description

public function **RegexIterator::setPregFlags**(int `$pregFlags`): void

Sets the regular expression flags.

Parameters

`pregFlags`

The regular expression flags. See RegexIterator::__construct() for an overview of available flags.

Return Values

No value is returned.

Examples

Example #1 RegexIterator::setPregFlags() example

Creates a new RegexIterator that filters all entries with where the array key starts with 'test'.

```
<?php
$test = array ('test 1', 'another test', 'test 123');

$arrayIterator = new ArrayIterator($test);
$regexIterator = new RegexIterator($arrayIterator, '/^test/', RegexIterator::GET_MATCH);

$regexIterator->setPregFlags(PREG_OFFSET_CAPTURE);

foreach ($regexIterator as $key => $value) {
    var_dump($value);
}
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(2) {
    [0]=>
    string(4) "test"
    [1]=>
    int(0)
  }
}
array(1) {
  [0]=>
  array(2) {
    [0]=>
    string(4) "test"
    [1]=>
    int(0)
  }
}
```

See Also

- RegexIterator::getPregFlags() - Returns the regular expression flags

File Handling

Table of Contents

SPL provides a number of classes to work with files.

SplFileInfo

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

Introduction

The SplFileInfo class offers a high-level object-oriented interface to information for an individual file.

Class synopsis

```
class SplFileInfo implements Stringable {
/* Methods */
public function __construct(string $filename)
public function getATime(): int|false
public function getBasename(string $suffix = ""): string
public function getCTime(): int|false
public function getExtension(): string
public function getFileInfo(?string $class = null): SplFileInfo
public function getFilename(): string
public function getGroup(): int|false
public function getInode(): int|false
public function getLinkTarget(): string|false
public function getMTime(): int|false
public function getOwner(): int|false
public function getPath(): string
public function getPathInfo(?string $class = null): ?SplFileInfo
public function getPathname(): string
public function getPerms(): int|false
public function getRealPath(): string|false
public function getSize(): int|false
public function getType(): string|false
public function isDir(): bool
public function isExecutable(): bool
public function isFile(): bool
public function isLink(): bool
public function isReadable(): bool
public function isWritable(): bool
public function openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function setFileClass(string $class = SplFileObject::class): void
public function setInfoClass(string $class = SplFileInfo::class): void
public function __toString(): string
}
```

Changelog

Version	Description
8.0.0	**SplFileInfo** implements Stringable now.

Table of Contents

SplFileInfo::__construct

SplFileInfo::__construct

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::__construct — Construct a new SplFileInfo object

Description

public function **SplFileInfo::__construct**(string $filename)

Creates a new SplFileInfo object for the file_name specified. The file does not need to exist, or be readable.

Parameters

`filename`

Path to the file.

Examples

Example #1 SplFileInfo::__construct() example

```php
<?php
$info = new SplFileInfo('example.php');
```

```php
if ($info->isFile()) {
    echo $info->getRealPath();
}
?>
```

SplFileInfo::getATime

SplFileInfo::getATime

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getATime — Gets last access time of the file

Description

public function **SplFileInfo::getATime**(): int|false

Gets the last access time for the file.

Parameters

This function has no parameters.

Return Values

Returns the time the file was last accessed on success, or `false` on failure.

Errors/Exceptions

Throws RuntimeException on error.

Examples

Example #1 SplFileInfo::getATime() example

```php
<?php
$info = new SplFileInfo('example.jpg');
echo 'Last accessed at ' . date('g:i a', $info->getATime());
?>
```

The above example will output something similar to:

```
Last accessed at 1:49 pm
```

See Also

- fileatime() - Gets last access time of file
- SplFileInfo::getCTime() - Gets the inode change time
- SplFileInfo::getMTime() - Gets the last modified time

SplFileInfo::getBasename

SplFileInfo::getBasename

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplFileInfo::getBasename — Gets the base name of the file

Description

public function **SplFileInfo::getBasename**(string `$suffix` = ""): string

This method returns the base name of the file, directory, or link without path info.

> **Caution**

> **SplFileInfo::getBasename()** is locale aware, so for it to see the correct basename with multibyte character paths, the matching locale must be set using the setlocale() function.

Parameters

`suffix`

Optional suffix to omit from the base name returned.

Return Values

Returns the base name without path information.

Examples

Example #1 **SplFileInfo::getBasename()** example

```php
<?php
$info = new SplFileInfo('file.txt');
var_dump($info->getBasename());

$info = new SplFileInfo('/path/to/file.txt');
var_dump($info->getBasename());

$info = new SplFileInfo('/path/to/file.txt');
var_dump($info->getBasename('.txt'));
?>
```

The above example will output something similar to:

```
string(8) "file.txt"
string(8) "file.txt"
string(4) "file"
```

See Also

- SplFileInfo::getFilename() - Gets the filename

SplFileInfo::getCTime

SplFileInfo::getCTime

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getCTime — Gets the inode change time

Description

public function **SplFileInfo::getCTime**(): int|false

Returns the inode change time for the file. The time returned is a Unix timestamp.

Parameters

This function has no parameters.

Return Values

The last change time, in a Unix timestamp on success, or `false` on failure.

Errors/Exceptions

Throws RuntimeException on error.

Examples

Example #1 SplFileInfo::getCTime() example

```php
<?php
$info = new SplFileInfo('example.jpg');
echo 'Last changed at ' . date('g:i a', $info->getCTime());
?>
```

The above example will output something similar to:

```
Last changed at 1:49 pm
```

See Also

- filectime() - Gets inode change time of file
- SplFileInfo::getATime() - Gets last access time of the file
- SplFileInfo::getMTime() - Gets the last modified time

SplFileInfo::getExtension

SplFileInfo::getExtension

(PHP 5 >= 5.3.6, PHP 7, PHP 8)

SplFileInfo::getExtension — Gets the file extension

Description

public function **SplFileInfo::getExtension**(): string

Retrieves the file extension.

Parameters

This function has no parameters.

Return Values

Returns a string containing the file extension, or an empty string if the file has no extension.

Examples

Example #1 SplFileInfo::getExtension() example

```php
<?php

$info = new SplFileInfo('foo.txt');
var_dump($info->getExtension());

$info = new SplFileInfo('photo.jpg');
var_dump($info->getExtension());

$info = new SplFileInfo('something.tar.gz');
var_dump($info->getExtension());

?>
```

The above example will output:

```
string(3) "txt"
string(3) "jpg"
```

```
string(2) "gz"
```

Notes

> **Note** :
>
> Another way of getting the extension is to use the pathinfo() function.
>
> ```
> <?php
> $extension = pathinfo($info->getFilename(), PATHINFO_EXTENSION);
> ?>
> ```

See Also

- SplFileInfo::getFilename() - Gets the filename
- SplFileInfo::getBasename() - Gets the base name of the file
- pathinfo() - Returns information about a file path

SplFileInfo::getFileInfo

SplFileInfo::getFileInfo

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getFileInfo — Gets an SplFileInfo object for the file

Description

public function **SplFileInfo::getFileInfo**(?string `$class = null`): SplFileInfo

This method gets an SplFileInfo object for the referenced file.

Parameters

`class`

Name of an SplFileInfo derived class to use.

Return Values

An SplFileInfo object created for the file.

Changelog

Version	Description
8.0.0	`class` is now nullable.

See Also

- SplFileInfo::setInfoClass() - Sets the class used with SplFileInfo::getFileInfo and SplFileInfo::getPathInfo

SplFileInfo::getFilename

SplFileInfo::getFilename

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getFilename — Gets the filename

Description

public function **SplFileInfo::getFilename**(): string

Gets the filename without any path information.

Parameters

This function has no parameters.

Return Values

The filename.

Examples

Example #1 SplFileInfo::getFilename() example

```php
<?php
$info = new SplFileInfo('foo.txt');
var_dump($info->getFilename());

$info = new SplFileInfo('/path/to/foo.txt');
var_dump($info->getFilename());

$info = new SplFileInfo('http://www.php.net/');
var_dump($info->getFilename());

$info = new SplFileInfo('http://www.php.net/svn.php');
var_dump($info->getFilename());
?>
```

The above example will output something similar to:

```
string(7) "foo.txt"
string(7) "foo.txt"
string(11) "www.php.net"
string(7) "svn.php"
```

See Also

- SplFileInfo::getBasename() - Gets the base name of the file

SplFileInfo::getGroup

SplFileInfo::getGroup

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getGroup — Gets the file group

Description

public function **SplFileInfo::getGroup**(): int|false

Gets the file group. The group ID is returned in numerical format.

Parameters

This function has no parameters.

Return Values

The group id in numerical format on success, or `false` on failure.

Errors/Exceptions

Throws RuntimeException on error.

Examples

Example #1 SplFileInfo::getGroup() example

```php
<?php
$info = new SplFileInfo('example.jpg');
echo $info->getFilename() . ' belongs to group id ' . $info->getGroup() . "\n";
print_r(posix_getgrgid($info->getGroup()));
?>
```

The above example will output something similar to:

```
example.jpg belongs to group id 42
Array
(
    [name] => toons
    [passwd] => x
    [members] => Array
        (
            [0] => tom
            [1] => jerry
        )
    [gid] => 42
)
```

See Also

- filegroup() - Gets file group
- posix_getgrgid() - Return info about a group by group id

SplFileInfo::getInode

SplFileInfo::getInode

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getInode — Gets the inode for the file

Description

public function **SplFileInfo::getInode**(): int|false

Gets the inode number for the filesystem object.

Parameters

This function has no parameters.

Return Values

Returns the inode number for the filesystem object on success, or `false` on failure.

Errors/Exceptions

Throws RuntimeException on error.

See Also

- fileinode() - Gets file inode

SplFileInfo::getLinkTarget

SplFileInfo::getLinkTarget

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplFileInfo::getLinkTarget — Gets the target of a link

Description

public function **SplFileInfo::getLinkTarget**(): string|false

Gets the target of a filesystem link.

> **Note** :
>
> The target may not be the real path on the filesystem. Use SplFileInfo::getRealPath() to determine the true path on the filesystem.

Parameters

This function has no parameters.

Return Values

Returns the target of the filesystem link on success, or `false` on failure.

Errors/Exceptions

Throws RuntimeException on error.

Examples

Example #1 SplFileInfo::getLinkTarget() example

```
<?php
$info = new SplFileInfo('/Users/bbieber/workspace');
if ($info->isLink()) {
    var_dump($info->getLinkTarget());
    var_dump($info->getRealPath());
}
?>
```

The above example will output something similar to:

```
string(19) "Documents/workspace"
string(34) "/Users/bbieber/Documents/workspace"
```

See Also

- SplFileInfo::isLink() - Tells if the file is a link
- SplFileInfo::getRealPath() - Gets absolute path to file

SplFileInfo::getMTime

SplFileInfo::getMTime

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getMTime — Gets the last modified time

Description

public function **SplFileInfo::getMTime**(): int|false

Returns the time when the contents of the file were changed. The time returned is a Unix timestamp.

Parameters

This function has no parameters.

Return Values

Returns the last modified time for the file, in a Unix timestamp on success, or `false` on failure.

Examples

Example #1 SplFileInfo::getMTime() example

```php
<?php
$info = new SplFileInfo('example.jpg');
echo 'Last modified at ' . date('g:i a', $info->getMTime());
?>
```

The above example will output something similar to:

```
Last modified at 1:49 pm
```

See Also

- filemtime() - Gets file modification time
- SplFileInfo::getATime() - Gets last access time of the file
- SplFileInfo::getCTime() - Gets the inode change time

SplFileInfo::getOwner

SplFileInfo::getOwner

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getOwner — Gets the owner of the file

Description

public function **SplFileInfo::getOwner**(): int|false

Gets the file owner. The owner ID is returned in numerical format.

Parameters

This function has no parameters.

Return Values

The owner id in numerical format on success, or `false` on failure.

Errors/Exceptions

Throws RuntimeException on error.

Examples

Example #1 SplFileInfo::getOwner() example

```php
<?php
$info = new SplFileInfo('example.jpg');
echo $info->getFilename() . ' belongs to owner id ' . $info->getOwner() . "\n";
print_r(posix_getpwuid($info->getOwner()));
?>
```

The above example will output something similar to:

```
example.jpg belongs to user id 501
Array
(
    [name] => tom
    [passwd] => x
```

```
    [uid] => 501
    [gid] => 42
    [gecos] => Tom Cat
    [dir] => /home/tom
    [shell] => /bin/bash
)
```

See Also

- posix_getpwuid() - Return info about a user by user id
- SplFileInfo::getGroup() - Gets the file group

SplFileInfo::getPath

SplFileInfo::getPath

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPath — Gets the path without filename

Description

public function **SplFileInfo::getPath**(): string

Returns the path to the file, omitting the filename and any trailing slash.

Parameters

This function has no parameters.

Return Values

Returns the path to the file.

Examples

Example #1 SplFileInfo::getPath() example

```php
<?php
$info = new SplFileInfo('/usr/bin/php');
var_dump($info->getPath());


$info = new SplFileInfo('/usr/');
var_dump($info->getPath());?>
```

The above example will output something similar to:

```
string(8) "/usr/bin"
string(4) "/usr"
```

See Also

- SplFileInfo::getRealPath() - Gets absolute path to file
- SplFileInfo::getFilename() - Gets the filename
- SplFileInfo::getPathInfo() - Gets an SplFileInfo object for the path

SplFileInfo::getPathInfo

SplFileInfo::getPathInfo

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPathInfo — Gets an SplFileInfo object for the path

Description

public function **SplFileInfo::getPathInfo**(?string `$class = null`): ?SplFileInfo

Gets an SplFileInfo object for the parent of the current file.

Parameters

`class`

Name of an SplFileInfo derived class to use, or itself if `null`.

Return Values

Returns an SplFileInfo object for the parent path of the file on success, or `null` on failure.

Changelog

Version	Description
8.0.0	`class` is now nullable.

Examples

Example #1 SplFileInfo::getPathInfo() example

```php
<?php
$info = new SplFileInfo('/usr/bin/php');
$parent_info = $info->getPathInfo();
var_dump($parent_info->getRealPath());
?>
```

The above example will output something similar to:

```
string(8) "/usr/bin"
```

See Also

- SplFileInfo::setInfoClass() - Sets the class used with SplFileInfo::getFileInfo and SplFileInfo::getPathInfo

SplFileInfo::getPathname

SplFileInfo::getPathname

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPathname — Gets the path to the file

Description

public function **SplFileInfo::getPathname**(): string

Returns the path to the file.

Parameters

This function has no parameters.

Return Values

The path to the file.

Examples

Example #1 SplFileInfo::getPathname() example

```php
<?php
$info = new SplFileInfo('/usr/bin/php');
var_dump($info->getPathname());
?>
```

The above example will output something similar to:

```
string(12) "/usr/bin/php"
```

See Also

- SplFileInfo::getRealPath() - Gets absolute path to file

SplFileInfo::getPerms

SplFileInfo::getPerms

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getPerms — Gets file permissions

Description

public function **SplFileInfo::getPerms**(): int|false

Gets the file permissions for the file.

Parameters

This function has no parameters.

Return Values

Returns the file permissions on success, or `false` on failure.

Examples

Example #1 SplFileInfo::getPerms() example

```php
<?php
$info = new SplFileInfo('/tmp');
echo substr(sprintf('%o', $info->getPerms()), -4);

$info = new SplFileInfo(__FILE__);
echo substr(sprintf('%o', $info->getPerms()), -4);
?>
```

The above example will output something similar to:

```
1777
0644
```

SplFileInfo::getRealPath

SplFileInfo::getRealPath

(PHP 5 >= 5.2.2, PHP 7, PHP 8)

SplFileInfo::getRealPath — Gets absolute path to file

Description

public function **SplFileInfo::getRealPath**(): string|false

This method expands all symbolic links, resolves relative references and returns the real path to the file.

Parameters

This function has no parameters.

Return Values

Returns the path to the file, or `false` if the file does not exist.

Examples

Example #1 SplFileInfo::getRealPath() example

```
<?php
$info = new SplFileInfo('/..//./../../'.__FILE__);
var_dump($info->getRealPath());

$info = new SplFileInfo('/tmp');
var_dump($info->getRealPath());

$info = new SplFileInfo('/I/Do/Not/Exist');
var_dump($info->getRealPath());

$info = new SplFileInfo('php://output');
var_dump($info->getRealPath());

$info = new SplFileInfo("");
var_dump($info->getRealPath());
?>
```

The above example will output something similar to:

```
string(28) "/private/tmp/phptempfile.php"
string(12) "/private/tmp"
bool(false)
bool(false)
string(12) "/private/tmp"
```

See Also

- SplFileInfo::isLink() - Tells if the file is a link
- realpath() - Returns canonicalized absolute pathname

SplFileInfo::getSize

SplFileInfo::getSize

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getSize — Gets file size

Description

public function **SplFileInfo::getSize**(): int|false

Returns the filesize in bytes for the file referenced.

Parameters

This function has no parameters.

Return Values

The filesize in bytes on success, or `false` on failure.

Errors/Exceptions

A RuntimeException will be thrown if the file does not exist or an error occurs.

Examples

Example #1 SplFileInfo::getSize() example

```
<?php
$info = new SplFileInfo('example.jpg');
echo $info->getFilename() . " " . $info->getSize();
?>
```

The above example will output something similar to:

```
example.jpg 15385
```

See Also

- filesize() - Gets file size

SplFileInfo::getType

SplFileInfo::getType

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::getType — Gets file type

Description

public function **SplFileInfo::getType**(): string|false

Returns the type of the file referenced.

Parameters

This function has no parameters.

Return Values

A string representing the type of the entry. May be one of `file`, `link`, `dir`, `block`, `fifo`, `char`, `socket`, or `unknown`, or `false` on failure.

Errors/Exceptions

Throws a RuntimeException on error.

Examples

Example #1 SplFileInfo::getType() example

```
<?php

$info = new SplFileInfo(__FILE__);
echo $info->getType().PHP_EOL;
```

```php
$info = new SplFileInfo(dirname(__FILE__));
echo $info->getType();

?>
```

The above example will output something similar to:

```
file
dir
```

SplFileInfo::isDir

SplFileInfo::isDir

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isDir — Tells if the file is a directory

Description

public function **SplFileInfo::isDir**(): bool

This method can be used to determine if the file is a directory.

Parameters

This function has no parameters.

Return Values

Returns `true` if a directory, `false` otherwise.

Examples

Example #1 SplFileInfo::isDir() example

```php
<?php
$d = new SplFileInfo(dirname(__FILE__));
var_dump($d->isDir());

$d = new SplFileInfo(__FILE__);
var_dump($d->isDir());
?>
```

The above example will output something similar to:

```
bool(true)
bool(false)
```

SplFileInfo::isExecutable

SplFileInfo::isExecutable

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isExecutable — Tells if the file is executable

Description

public function **SplFileInfo::isExecutable**(): bool

Checks if the file is executable.

Parameters

This function has no parameters.

Return Values

Returns `true` if executable, `false` otherwise.

Examples

Example #1 SplFileInfo::isExecutable() example

```
<?php
$info = new SplFileInfo('/usr/bin/php');
var_dump($info->isExecutable());

$info = new SplFileInfo('/usr/bin');
var_dump($info->isExecutable());

$info = new SplFileInfo('foo');
var_dump($info->isExecutable());
?>
```

The above example will output something similar to:

```
bool(true)
bool(true)
bool(false)
```

SplFileInfo::isFile

SplFileInfo::isFile

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isFile — Tells if the object references a regular file

Description

public function **SplFileInfo::isFile**(): bool

Checks if the file referenced by this SplFileInfo object exists and is a regular file.

Parameters

This function has no parameters.

Return Values

Returns `true` if the file exists and is a regular file (not a link), `false` otherwise.

Examples

Example #1 SplFileInfo::isFile() example

```
<?php
$info = new SplFileInfo(__FILE__);
var_dump($info->isFile());

$info = new SplFileInfo(dirname(__FILE__));
```

```
var_dump($info->isFile());
?>
```

The above example will output something similar to:

```
bool(true)
bool(false)
```

SplFileInfo::isLink

SplFileInfo::isLink

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isLink — Tells if the file is a link

Description

public function **SplFileInfo::isLink**(): bool

Use this method to check if the file referenced by the SplFileInfo object is a link.

Parameters

This function has no parameters.

Return Values

Returns `true` if the file is a link, `false` otherwise.

Examples

Example #1 SplFileInfo::isLink() example

```
<?php
$info = new SplFileInfo('/path/to/symlink');
if ($info->isLink()) {
    echo 'The real path is '.$info->getRealPath();
}
?>
```

See Also

- SplFileInfo::getRealPath() - Gets absolute path to file

SplFileInfo::isReadable

SplFileInfo::isReadable

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isReadable — Tells if file is readable

Description

public function **SplFileInfo::isReadable**(): bool

Check if the file is readable.

Parameters

This function has no parameters.

Return Values

Returns `true` if readable, `false` otherwise.

Examples

Example #1 SplFileInfo::isReadable() example

```
<?php
$info = new SplFileInfo('readable.jpg');
if ($info->isReadable()) {
    echo $info->getFilename() . ' is readable';
}
?>
```

The above example will output something similar to:

```
readable.jpg is readable
```

SplFileInfo::isWritable

SplFileInfo::isWritable

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::isWritable — Tells if the entry is writable

Description

public function **SplFileInfo::isWritable**(): bool

Checks if the current entry is writable.

Parameters

This function has no parameters.

Return Values

Returns `true` if writable, `false` otherwise;

Examples

Example #1 SplFileInfo::isWritable() example

```
<?php
$info = new SplFileInfo('locked.jpg');
if (!$info->isWritable()) {
    echo $info->getFilename() . ' is not writable';
}
?>
```

The above example will output something similar to:

```
locked.jpg is not writable
```

SplFileInfo::openFile

SplFileInfo::openFile

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::openFile — Gets an SplFileObject object for the file

Description

public function **SplFileInfo::openFile**(string `$mode = "r"`, bool `$useIncludePath = false`, ?resource `$context = null`): SplFileObject

Creates an SplFileObject object of the file. This is useful because SplFileObject contains additional methods for manipulating the file whereas SplFileInfo is only useful for gaining information, like whether the file is writable.

Parameters

`mode`

The mode for opening the file. See the fopen() documentation for descriptions of possible modes. The default is read only.

`useIncludePath`

When set to `true`, the filename is also searched for within the include_path

`context`

Refer to the context section of the manual for a description of `contexts`.

Return Values

The opened file as an SplFileObject object.

Errors/Exceptions

A RuntimeException if the file cannot be opened (e.g. insufficient access rights).

Changelog

Version	Description
8.0.0	`context` is now nullable.

Examples

Example #1 SplFileInfo::openFile() example

```php
<?php
$fileinfo = new SplFileInfo('/tmp/foo.txt');

if ($fileinfo->isWritable()) {

    $fileobj = $fileinfo->openFile('a');

    $fileobj->fwrite("appended this sample text");
}
?>
```

See Also

- SplFileObject
- stream_context_create() - Creates a stream context
- fopen() - Opens file or URL

SplFileInfo::setFileClass

SplFileInfo::setFileClass

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::setFileClass — Sets the class used with SplFileInfo::openFile()

Description

public function **SplFileInfo::setFileClass**(string `$class = SplFileObject::class`): void

Use this method to set a custom class which will be used when SplFileInfo::openFile() is called. The class name passed to this method must be SplFileObject or a class derived from SplFileObject.

Parameters

`class`

The class name to use when SplFileInfo::openFile() is called.

Return Values

No value is returned.

Examples

Example #1 SplFileInfo::setFileClass() example

```php
<?php
// Create a class extending SplFileObject
class MyFoo extends SplFileObject {}

$info = new SplFileInfo(__FILE__);
// Set the class to use
$info->setFileClass('MyFoo');
var_dump($info->openFile());
?>
```

The above example will output something similar to:

```
object(MyFoo)#2 (0) { }
```

See Also

- SplFileInfo::openFile() - Gets an SplFileObject object for the file

SplFileInfo::setInfoClass

SplFileInfo::setInfoClass

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::setInfoClass — Sets the class used with SplFileInfo::getFileInfo() and SplFileInfo::getPathInfo()

Description

public function **SplFileInfo::setInfoClass**(string `$class = SplFileInfo::class`): void

Use this method to set a custom class which will be used when SplFileInfo::getFileInfo() and SplFileInfo::getPathInfo() are called. The class name passed to this method must be SplFileInfo or a class derived from SplFileInfo.

Parameters

`class`

The class name to use when SplFileInfo::getFileInfo() and SplFileInfo::getPathInfo() are called.

Return Values

No value is returned.

Examples

Example #1 SplFileInfo::setFileClass() example

```php
<?php
// Define a class which extends SplFileInfo
class MyFoo extends SplFileInfo {}

$info = new SplFileInfo('foo');
// Set the class name to use
$info->setInfoClass('MyFoo');
var_dump($info->getFileInfo());
?>
```

The above example will output something similar to:

```
object(MyFoo)#2 (0) { }
```

See Also

- SplFileInfo::getFileInfo() - Gets an SplFileInfo object for the file

SplFileInfo::__toString

SplFileInfo::__toString

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileInfo::__toString — Returns the path to the file as a string

Description

public function **SplFileInfo::__toString**(): string

This method will return the file name of the referenced file.

Parameters

This function has no parameters.

Return Values

Returns the path to the file.

Examples

Example #1 SplFileInfo::__toString() example

```php
<?php
$info = new SplFileInfo('foo');
var_dump($info->__toString());
echo $info.PHP_EOL;

$info = new SplFileInfo('/usr/bin/php');
var_dump($info->__toString());
echo $info.PHP_EOL;
?>
```

The above example will output something similar to:

```
string(3) "foo"
foo
string(12) "/usr/bin/php"
/usr/bin/php
```

SplFileObject

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

The SplFileObject class offers an object-oriented interface for a file.

Class synopsis

```
class SplFileObject extends SplFileInfo implements RecursiveIterator, SeekableIterator {
/* Constants */
public const int DROP_NEW_LINE;
public const int READ_AHEAD;
public const int SKIP_EMPTY;
public const int READ_CSV;
/* Methods */
public function __construct(
    string $filename,
    string $mode = "r",
    bool $useIncludePath = false,
    ?resource $context = null
)
public function current(): string|array|false
public function eof(): bool
public function fflush(): bool
public function fgetc(): string|false
public function fgetcsv(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): array|false
public function fgets(): string
public function fgetss(string $allowable_tags = ?): string
public function flock(int $operation, int &$wouldBlock = null): bool
public function fpassthru(): int
public function fputcsv(
    array $fields,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\",
    string $eol = "\n"
): int|false
public function fread(int $length): string|false
public function fscanf(string $format, mixed &...$vars): array|int|null
public function fseek(int $offset, int $whence = SEEK_SET): int
public function fstat(): array
public function ftell(): int|false
public function ftruncate(int $size): bool
public function fwrite(string $data, ?int $length = null): int|false
public function getChildren(): null
public function getCsvControl(): array
public function getFlags(): int
public function getMaxLineLen(): int
public function hasChildren(): false
public function key(): int
public function next(): void
public function rewind(): void
public function seek(int $line): void
public function setCsvControl(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): void
public function setFlags(int $flags): void
public function setMaxLineLen(int $maxLength): void
public function __toString(): string
public function valid(): bool
/* Inherited methods */
```

public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Predefined Constants

SplFileObject::DROP_NEW_LINE

Drop newlines at the end of a line.

SplFileObject::READ_AHEAD

Read on rewind/next.

SplFileObject::SKIP_EMPTY

Skips empty lines in the file. This requires the **READ_AHEAD** flag be enabled, to work as expected.

SplFileObject::READ_CSV

Read lines as CSV rows.

Changelog

Version	Description
8.4.0	The class constants are now typed.

Table of Contents

SplFileObject::__construct

SplFileObject::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::__construct — Construct a new file object

Description

public function **SplFileObject::__construct**(
  string $filename,

```
    string $mode = "r",
    bool $useIncludePath = false,
    ?resource $context = null
)
```

Construct a new file object.

Parameters

`filename`

The file to read.

> **Tip**
>
> A URL can be used as a filename with this function if the fopen wrappers have been enabled. See fopen() for more details on how to specify the filename. See the Supported Protocols and Wrappers for links to information about what abilities the various wrappers have, notes on their usage, and information on any predefined variables they may provide.

`mode`

The mode in which to open the file. See fopen() for a list of allowed modes.

`useIncludePath`

Whether to search in the include_path for `filename`.

`context`

A valid context resource created with stream_context_create().

Errors/Exceptions

Throws a RuntimeException if the `filename` cannot be opened.

Throws a LogicException if the `filename` is a directory.

Examples

Example #1 SplFileObject::__construct() example

This example opens the current file and iterates over its contents line by line.

```php
<?php
$file = new SplFileObject(__FILE__);
foreach ($file as $line_num => $line) {
    echo "Line $line_num is $line";
}
?>
```

The above example will output something similar to:

```
Line 0 is <?php
Line 1 is $file = new SplFileObject(__FILE__);
Line 2 is foreach ($file as $line_num => $line) {
Line 3 is     echo "Line $line_num is $line";
Line 4 is }
Line 5 is ?>
```

See Also

- SplFileInfo::openFile() - Gets an SplFileObject object for the file
- fopen() - Opens file or URL

SplFileObject::current

SplFileObject::current

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::current — Retrieve current line of file

Description

public function **SplFileObject::current**(): string|array|false

Retrieves the current line of the file.

Parameters

This function has no parameters.

Return Values

Retrieves the current line of the file. If the `SplFileObject::READ_CSV` flag is set, this method returns an array containing the current line parsed as CSV data. If the end of the file is reached, `false` is returned.

Examples

Example #1 SplFileObject::current() example

```php
<?php
$file = new SplFileObject(__FILE__);
foreach ($file as $k => $line) {
   echo ($file->key() + 1) . ': ' . $file->current();
}
?>
```

The above example will output something similar to:

```
1: <?php
2: $file = new SplFileObject(__FILE__);
3: foreach ($file as $line) {
4:     echo ($file->key() + 1) . ': ' . $file->current();
5: }
6: ?>
```

See Also

- SplFileObject::key() - Get line number
- SplFileObject::seek() - Seek to specified line
- SplFileObject::next() - Read next line
- SplFileObject::rewind() - Rewind the file to the first line
- SplFileObject::valid() - Not at EOF

SplFileObject::eof

SplFileObject::eof

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::eof — Reached end of file

Description

public function **SplFileObject::eof**(): bool

Determine whether the end of file has been reached

Parameters

This function has no parameters.

Return Values

Returns `true` if file is at EOF, `false` otherwise.

Examples

Example #1 SplFileObject::eof() example

```
<?php
$file = new SplFileObject("fruits.txt");
while ( ! $file->eof()) {
    echo $file->fgets();
}
?>
```

The above example will output something similar to:

```
apple
banana
cherry
date
elderberry
```

See Also

- SplFileObject::valid() - Not at EOF
- feof() - Tests for end-of-file on a file pointer

SplFileObject::fflush

SplFileObject::fflush

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fflush — Flushes the output to the file

Description

public function **SplFileObject::fflush**(): bool

Forces a write of all buffered output to the file.

Parameters

This function has no parameters.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 SplFileObject::fflush() example

```
<?php
$file = new SplFileObject('misc.txt', 'r+');
$file->rewind();
$file->fwrite("Foo");
$file->fflush();
```

```
$file->ftruncate($file->ftell());
?>
```

See Also

- SplFileObject::fwrite() - Write to file

SplFileObject::fgetc

SplFileObject::fgetc

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fgetc — Gets character from file

Description

public function **SplFileObject::fgetc**(): string|false

Gets a character from the file.

Parameters

This function has no parameters.

Return Values

Returns a string containing a single character read from the file or `false` on EOF.

> **Warning**
>
> This function may return Boolean `false`, but may also return a non-Boolean value which evaluates to `false`. Please read the section on Booleans for more information. Use the === operator for testing the return value of this function.

Examples

Example #1 SplFileObject::fgetc() example

```
<?php
$file = new SplFileObject('file.txt');
while (false !== ($char = $file->fgetc())) {
    echo "$char\n";
}
?>
```

See Also

- SplFileObject::fgets() - Gets line from file

SplFileObject::fgetcsv

SplFileObject::fgetcsv

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fgetcsv — Gets line from file and parse as CSV fields

Description

public function **SplFileObject::fgetcsv**(string `$separator = ","`, string `$enclosure = "\""`, string `$escape = "\\"`): array|false

Gets a line from the file which is in CSV format and returns an array containing the fields read.

> **Note** :
>
> The locale settings are taken into account by this function. For example, data encoded in certain one-byte encodings may be parsed incorrectly if `LC_CTYPE` is `en_US.UTF-8`.

Parameters

`separator`

The field delimiter (one single-byte character only). By default `,` or the value set by a prior call to SplFileObject::setCsvControl().

`enclosure`

The field enclosure character (one single-byte character only). By default `"` or the value set by a prior call to SplFileObject::setCsvControl().

`escape`

The escape character (at most one single-byte character). By default `\` or the value set by a prior call to SplFileObject::setCsvControl(). An empty string (`""`) disables the proprietary escape mechanism.

> **Warning**
>
> In the input stream, the `enclosure` character can always be escaped by doubling it inside a quoted string, resulting in a single `enclosure` character in the parsed result. The `escape` character works differently: If a sequence of `escape` and `enclosure` characters appear in the input, both characters will be present in the parsed result. So for the default parameters, a CSV line like `"a""b","c\"d"` will have the fields parsed as `a"b` and `c\"d`, respectively.

> **Warning**
>
> As of PHP 8.4.0, depending on the default value of `escape` is deprecated. It needs to be provided explicitly either positionally or by the use of Named Arguments, or by a call to SplFileObject::setCsvControl().

> **Warning**
>
> When `escape` is set to anything other than an empty string (`""`) it can result in CSV that is not compliant with » RFC 4180 or unable to survive a roundtrip through the PHP CSV functions. The default for `escape` is `"\\"` so it is recommended to set it to the empty string explicitly. The default value will change in a future version of PHP, no earlier than PHP 9.0.

Return Values

Returns an indexed array containing the fields read, or **`false`** on error.

> **Note** :
>
> A blank line in a CSV file will be returned as an array comprising a single **`null`** field unless using **`SplFileObject::SKIP_EMPTY | SplFileObject::DROP_NEW_LINE`**, in which case empty lines are skipped.

Errors/Exceptions

Throws a ValueError if `separator` or `enclosure` is not one byte long.

Throws a ValueError if `escape` is not one byte long or the empty string.

Changelog

Version	Description
8.4.0	Relying on the default value of `escape` is now deprecated.
7.4.0	The `escape` parameter now also accepts an empty string to disable the proprietary escape mechanism.

Examples

Example #1 SplFileObject::fgetcsv() example

```php
<?php
$file = new SplFileObject("data.csv");
while (!$file->eof()) {
    var_dump($file->fgetcsv());
```

```
}
?>
```

Example #2 `SplFileObject::READ_CSV` example

```php
<?php
$file = new SplFileObject("animals.csv");
$file->setFlags(SplFileObject::READ_CSV);
foreach ($file as $row) {
    list($animal, $class, $legs) = $row;
    printf("A %s is a %s with %d legs\n", $animal, $class, $legs);
}
?>
```

Contents of animals.csv

```
crocodile,reptile,4
dolphin,mammal,0
duck,bird,2
koala,mammal,4
salmon,fish,0
```

The above example will output something similar to:

```
A crocodile is a reptile with 4 legs
A dolphin is a mammal with 0 legs
A duck is a bird with 2 legs
A koala is a mammal with 4 legs
A salmon is a fish with 0 legs
```

See Also

- SplFileObject::fputcsv() - Write a field array as a CSV line
- SplFileObject::setCsvControl() - Set the delimiter, enclosure and escape character for CSV
- SplFileObject::getCsvControl() - Get the delimiter, enclosure and escape character for CSV
- SplFileObject::setFlags() - Sets flags for the SplFileObject
- **`SplFileObject::READ_CSV`**
- SplFileObject::current() - Retrieve current line of file
- fputcsv() - Format line as CSV and write to file pointer
- fgetcsv() - Gets line from file pointer and parse for CSV fields
- str_getcsv() - Parse a CSV string into an array

SplFileObject::fgets

SplFileObject::fgets

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fgets — Gets line from file

Description

public function **SplFileObject::fgets**(): string

Gets a line from the file.

Parameters

This function has no parameters.

Return Values

Returns a string containing the next line from the file.

Errors/Exceptions

Throws a RuntimeException if the file cannot be read.

Examples

Example #1 SplFileObject::fgets() example

This example simply outputs the contents of `file.txt` line-by-line.

```
<?php
$file = new SplFileObject("file.txt");
while (!$file->eof()) {
    echo $file->fgets();
}
?>
```

See Also

- fgets() - Gets line from file pointer
- SplFileObject::fgetss() - Gets line from file and strip HTML tags
- SplFileObject::fgetc() - Gets character from file
- SplFileObject::current() - Retrieve current line of file

SplFileObject::fgetss

SplFileObject::fgetss

(PHP 5 >= 5.1.0, PHP 7)

SplFileObject::fgetss — Gets line from file and strip HTML tags

> **Warning**
>
> This function has been *DEPRECATED* as of PHP 7.3.0, and *REMOVED* as of PHP 8.0.0. Relying on this function is highly discouraged.

Description

public function **SplFileObject::fgetss**(string `$allowable_tags = ?`): string

Identical to SplFileObject::fgets(), except that **SplFileObject::fgetss()** attempts to strip any HTML and PHP tags from the text it reads. The function retains the parsing state from call to call, and as such is not equivalent to calling strip_tags() on the return value of SplFileObject::fgets().

Parameters

`allowable_tags`

Optional parameter to specify tags which should not be stripped.

Return Values

Returns a string containing the next line of the file with HTML and PHP code stripped, or `false` on error.

Examples

Example #1 SplFileObject::fgetss() example

```
<?php
$str = <<<EOD
<html><body>
 <p>Welcome! Today is the <?php echo(date('jS')); ?> of <?= date('F'); ?>.</p>
</body></html>
Text outside of the HTML block.
EOD;
file_put_contents("sample.php", $str);

$file = new SplFileObject("sample.php");
while (!$file->eof()) {
```

```
    echo $file->fgetss();
}
?>
```

The above example will output something similar to:

```
Welcome! Today is the  of .

Text outside of the HTML block.
```

See Also

- fgetss() - Gets line from file pointer and strip HTML tags
- SplFileObject::fgets() - Gets line from file
- SplFileObject::fgetc() - Gets character from file
- SplFileObject::current() - Retrieve current line of file
- The string.strip_tags filter

SplFileObject::flock

SplFileObject::flock

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::flock — Portable file locking

Description

public function **SplFileObject::flock**(int `$operation`, int `&$wouldBlock = null`): bool

Locks or unlocks the file in the same portable way as flock().

Parameters

`operation`

`operation` is one of the following:

- `LOCK_SH` to acquire a shared lock (reader).
- `LOCK_EX` to acquire an exclusive lock (writer).
- `LOCK_UN` to release a lock (shared or exclusive).

It is also possible to add `LOCK_NB` as a bitmask to one of the above operations, if flock() should not block during the locking attempt.

`wouldBlock`

Set to `true` if the lock would block (EWOULDBLOCK errno condition).

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 SplFileObject::flock() example

```
<?php
$file = new SplFileObject("/tmp/lock.txt", "w");
if ($file->flock(LOCK_EX)) { // do an exclusive lock
    $file->ftruncate(0);     // truncate file
    $file->fwrite("Write something here\n");
    $file->flock(LOCK_UN);   // release the lock
} else {
```

```
    echo "Couldn't get the lock!";
}
?>
```

See Also

- flock() - Portable advisory file locking

SplFileObject::fpassthru

SplFileObject::fpassthru

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fpassthru — Output all remaining data on a file pointer

Description

public function **SplFileObject::fpassthru**(): int

Reads to EOF on the given file pointer from the current position and writes the results to the output buffer.

You may need to call SplFileObject::rewind() to reset the file pointer to the beginning of the file if you have already written data to the file.

Parameters

This function has no parameters.

Return Values

Returns the number of characters read from `handle` and passed through to the output.

Examples

Example #1 SplFileObject::fpassthru() example

```
<?php

// Open the file in binary mode
$file = new SplFileObject("./img/ok.png", "rb");

// Send the right headers
header("Content-Type: image/png");
header("Content-Length: " . $file->getSize());

// Dump the picture and end script
$file->fpassthru();
exit;

?>
```

See Also

- fpassthru() - Output all remaining data on a file pointer

SplFileObject::fputcsv

SplFileObject::fputcsv

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

SplFileObject::fputcsv — Write a field array as a CSV line

Description

public function **SplFileObject::fputcsv**(
  array `$fields`,
  string `$separator = ","`,
  string `$enclosure = "\""`,
  string `$escape = "\\"`,

```
    string $eol = "\n"
): int|false
```

Writes the `fields` array to the file as a CSV line.

Parameters

`fields`

An array of values.

`separator`

The field delimiter (one single-byte character only). By default `,` or the value set by a prior call to SplFileObject::setCsvControl().

`enclosure`

The field enclosure character (one single-byte character only). By default `"` or the value set by a prior call to SplFileObject::setCsvControl().

`escape`

The escape character (at most one single-byte character). By default `\` or the value set by a prior call to SplFileObject::setCsvControl(). An empty string (`""`) disables the proprietary escape mechanism.

> **Warning**
>
> In the input stream, the `enclosure` character can always be escaped by doubling it inside a quoted string, resulting in a single `enclosure` character in the parsed result. The `escape` character works differently: If a sequence of `escape` and `enclosure` characters appear in the input, both characters will be present in the parsed result. So for the default parameters, a CSV line like `"a""b","c\"d"` will have the fields parsed as `a"b` and `c\"d`, respectively.

> **Warning**
>
> As of PHP 8.4.0, depending on the default value of `escape` is deprecated. It needs to be provided explicitly either positionally or by the use of Named Arguments, or by a call to SplFileObject::setCsvControl().

`eol`

The optional `eol` parameter sets a custom End of Line sequence.

> **Warning**
>
> When `escape` is set to anything other than an empty string (`""`) it can result in CSV that is not compliant with » RFC 4180 or unable to survive a roundtrip through the PHP CSV functions. The default for `escape` is `"\\"` so it is recommended to set it to the empty string explicitly. The default value will change in a future version of PHP, no earlier than PHP 9.0.

> **Note** :
>
> If an `enclosure` character is contained in a field, it will be escaped by doubling it, unless it is immediately preceded by an `escape`.

Return Values

Returns the length of the written string or `false` on failure.

Errors/Exceptions

Throws a ValueError if `separator` or `enclosure` is not one byte long.

Throws a ValueError if `escape` is not one byte long or the empty string.

Changelog

Version	Description
8.4.0	Relying on the default value of `escape` is now deprecated.

Version	Description
8.1.0	The optional `eol` parameter has been added.
7.4.0	The `escape` parameter now also accepts an empty string to disable the proprietary escape mechanism.

Examples

Example #1 SplFileObject::fputcsv() example

```php
<?php

$list = array (
    array('aaa', 'bbb', 'ccc', 'dddd'),
    array('123', '456', '789'),
    array('"aaa"', '"bbb"')
);

$file = new SplFileObject('file.csv', 'w');

foreach ($list as $fields) {
    $file->fputcsv($fields);
}

?>
```

The above example will write the following to `file.csv`:

```
aaa,bbb,ccc,dddd
123,456,789
"""aaa""","""bbb"""
```

See Also

- SplFileObject::fgetcsv() - Gets line from file and parse as CSV fields
- SplFileObject::setCsvControl() - Set the delimiter, enclosure and escape character for CSV
- SplFileObject::getCsvControl() - Get the delimiter, enclosure and escape character for CSV
- fputcsv() - Format line as CSV and write to file pointer
- fgetcsv() - Gets line from file pointer and parse for CSV fields
- str_getcsv() - Parse a CSV string into an array

SplFileObject::fread

SplFileObject::fread

(PHP 5 >= 5.5.11, PHP 7, PHP 8)

SplFileObject::fread — Read from file

Description

public function **SplFileObject::fread**(int `$length`): string|false

Reads the given number of bytes from the file.

Parameters

`length`

The number of bytes to read.

Return Values

Returns the string read from the file or `false` on failure.

Examples

Example #1 SplFileObject::fread() example

```php
<?php
// Get contents of a file into a string
$filename = "/usr/local/something.txt";
$file = new SplFileObject($filename, "r");
$contents = $file->fread($file->getSize());
?>
```

Notes

> **Note** :
>
> Note that **SplFileObject::fread()** reads from the current position of the file pointer. Use SplFileObject::ftell() to find the current position of the pointer and SplFileObject::rewind() (or SplFileObject::fseek()) to rewind the pointer position.

See Also

- fread() - Binary-safe file read

SplFileObject::fscanf

SplFileObject::fscanf

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fscanf — Parses input from file according to a format

Description

public function **SplFileObject::fscanf**(string `$format`, mixed `&...$vars`): array|int|null

Reads a line from the file and interprets it according to the specified `format`.

Any whitespace in the `format` string matches any whitespace in the line from the file. This means that even a tab (`\t`) in the format string can match a single space character in the input stream.

Parameters

`format`

The interpreted format for `string`, which is described in the documentation for sprintf() with following differences:

- Function is not locale-aware.
- `F`, `g`, `G` and `b` are not supported.
- `D` stands for decimal number.
- `i` stands for integer with base detection.
- `n` stands for number of characters processed so far.
- `s` stops reading at any whitespace character.
- `*` instead of `argnum$` suppresses the assignment of this conversion specification.

`vars`

The optional assigned values.

Return Values

If only one parameter is passed to this method, the values parsed will be returned as an array. Otherwise, if optional parameters are passed, the method will return the number of assigned values. The optional parameters must be passed by reference.

If there are more substrings expected in the `format` than there are available in the line read from the file, `null` will be returned.

When optional parameters are used and the end of the line read from the file is reached before any value has been parsed, `-1` is returned.

Examples

Example #1 SplFileObject::fscanf() example

```php
<?php
$file = new SplFileObject("misc.txt");
while ($userinfo = $file->fscanf("%s %s %s")) {
   list ($name, $profession, $countrycode) = $userinfo;
   // Do something with $name $profession $countrycode
}
?>
```

Contents of users.txt

```
javier   argonaut    pe
hiroshi  sculptor    jp
robert   slacker     us
luigi    florist     it
```

See Also

- fscanf() - Parses input from a file according to a format
- sscanf() - Parses input from a string according to a format
- printf() - Output a formatted string
- sprintf() - Return a formatted string

SplFileObject::fseek

SplFileObject::fseek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fseek — Seek to a position

Description

public function **SplFileObject::fseek**(int `$offset`, int `$whence` = **`SEEK_SET`**): int

Seek to a position in the file measured in bytes from the beginning of the file, obtained by adding `offset` to the position specified by `whence`.

Parameters

`offset`

The offset. A negative value can be used to move backwards through the file which is useful when SEEK_END is used as the `whence` value.

`whence`

`whence` values are:

- **`SEEK_SET`** - Set position equal to `offset` bytes.
- **`SEEK_CUR`** - Set position to current location plus `offset`.
- **`SEEK_END`** - Set position to end-of-file plus `offset`.

If `whence` is not specified, it is assumed to be **`SEEK_SET`**.

Return Values

Returns 0 if the seek was successful, -1 otherwise. Note that seeking past EOF is not considered an error.

Examples

Example #1 SplFileObject::fseek() example

```php
<?php
$file = new SplFileObject("somefile.txt");

// Read first line
$data = $file->fgets();

// Move back to the beginning of the file
// Same as $file->rewind();
```

```
$file->fseek(0);
?>
```

See Also

- fseek() - Seeks on a file pointer

SplFileObject::fstat

SplFileObject::fstat

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fstat — Gets information about the file

Description

public function **SplFileObject::fstat**(): array

Gathers the statistics of the file. Behaves identically to fstat().

Parameters

This function has no parameters.

Return Values

Returns an array with the statistics of the file; the format of the array is described in detail on the stat() manual page.

Examples

Example #1 SplFileObject::fstat() example

```
<?php
$file = new SplFileObject("/etc/passwd");
$stat = $file->fstat();

// Print only the associative part
print_r(array_slice($stat, 13));

?>
```

The above example will output something similar to:

```
Array
(
    [dev] => 771
    [ino] => 488704
    [mode] => 33188
    [nlink] => 1
    [uid] => 0
    [gid] => 0
    [rdev] => 0
    [size] => 1114
    [atime] => 1061067181
    [mtime] => 1056136526
    [ctime] => 1056136526
    [blksize] => 4096
    [blocks] => 8
)
```

See Also

- fstat() - Gets information about a file using an open file pointer
- stat() - Gives information about a file

SplFileObject::ftell

SplFileObject::ftell

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::ftell — Return current file position

Description

public function **SplFileObject::ftell**(): int|false

Returns the position of the file pointer which represents the current offset in the file stream.

Parameters

This function has no parameters.

Return Values

Returns the position of the file pointer as an integer, or `false` on error.

Examples

Example #1 SplFileObject::ftell() example

```php
<?php
$file = new SplFileObject("/etc/passwd");

// Read first line
$data = $file->fgets();

// Where are we?
echo $file->ftell();
?>
```

See Also

- ftell() - Returns the current position of the file read/write pointer

SplFileObject::ftruncate

SplFileObject::ftruncate

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::ftruncate — Truncates the file to a given length

Description

public function **SplFileObject::ftruncate**(int `$size`): bool

Truncates the file to `size` bytes.

Parameters

`size`

The size to truncate to.

> **Note** :
>
> If `size` is larger than the file it is extended with null bytes.
>
> If `size` is smaller than the file, the extra data will be lost.

Return Values

Returns `true` on success or `false` on failure.

Examples

Example #1 SplFileObject::ftruncate() example

```php
<?php
// Create file containing "Hello World!"
$file = new SplFileObject("/tmp/ftruncate", "w+");
$file->fwrite("Hello World!");

// Truncate to 5 bytes
$file->ftruncate(5);

// Rewind and read data
$file->rewind();
echo $file->fgets();
?>
```

The above example will output something similar to:

```
Hello
```

See Also

- ftruncate() - Truncates a file to a given length

SplFileObject::fwrite

SplFileObject::fwrite

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::fwrite — Write to file

Description

public function **SplFileObject::fwrite**(string `$data`, ?int `$length = null`): int|false

Writes the contents of `data` to the file

Parameters

`data`

The string to be written to the file.

`length`

If the `length` argument is int, writing will stop after `length` bytes have been written or the end of `data` is reached, whichever comes first.

Return Values

Returns the number of bytes written, or `false` on error.

Changelog

Version	Description
8.5.0	`length` is now nullable.
7.4.0	The function now returns `false` instead of zero on failure.

Examples

Example #1 SplFileObject::fwrite() example

```php
<?php
$file = new SplFileObject("fwrite.txt", "w");
$written = $file->fwrite("12345");
echo "Wrote $written bytes to file";
?>
```

The above example will output something similar to:

```
Wrote 5 bytes to file
```

See Also

- fwrite() - Binary-safe file write

SplFileObject::getChildren

SplFileObject::getChildren

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::getChildren — No purpose

Description

public function **SplFileObject::getChildren**(): null

An SplFileObject does not have children so this method returns `null`.

Parameters

This function has no parameters.

Return Values

Returns `null`.

See Also

- RecursiveIterator::getChildren() - Returns an iterator for the current entry

SplFileObject::getCsvControl

SplFileObject::getCsvControl

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

SplFileObject::getCsvControl — Get the delimiter, enclosure and escape character for CSV

Description

public function **SplFileObject::getCsvControl**(): array

Gets the delimiter, enclosure and escape character used for parsing CSV fields.

Parameters

This function has no parameters.

Return Values

Returns an indexed array containing the delimiter, enclosure and escape character.

Changelog

Version	Description
7.4.0	The escape character can now be an empty string.
7.0.10	Added the escape character to the returned array.

Examples

Example #1 **SplFileObject::getCsvControl()** example

```
<?php
$file = new SplFileObject("data.txt");
print_r($file->getCsvControl());
?>
```

The above example will output something similar to:

```
Array
(
    [0] => ,
    [1] => "
    [2] => \
)
```

See Also

- SplFileObject::setCsvControl() - Set the delimiter, enclosure and escape character for CSV
- SplFileObject::fgetcsv() - Gets line from file and parse as CSV fields
- SplFileObject::fputcsv() - Write a field array as a CSV line
- fputcsv() - Format line as CSV and write to file pointer
- fgetcsv() - Gets line from file pointer and parse for CSV fields
- str_getcsv() - Parse a CSV string into an array

SplFileObject::getCurrentLine

SplFileObject::getCurrentLine

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileObject::getCurrentLine — Alias of SplFileObject::fgets()

Description

This method is an alias of: SplFileObject::fgets().

SplFileObject::getFlags

SplFileObject::getFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::getFlags — Gets flags for the SplFileObject

Description

public function **SplFileObject::getFlags**(): int

Gets the flags set for an instance of SplFileObject as an int.

Parameters

This function has no parameters.

Return Values

Returns an int representing the flags.

Examples

Example #1 SplFileObject::getFlags() example

```php
<?php
$file = new SplFileObject(__FILE__, "r");

if ($file->getFlags() & SplFileObject::SKIP_EMPTY) {
    echo "Skipping empty lines\n";
} else {
    echo "Not skipping empty lines\n";
}

$file->setFlags(SplFileObject::SKIP_EMPTY);

if ($file->getFlags() & SplFileObject::SKIP_EMPTY) {
    echo "Skipping empty lines\n";
} else {
    echo "Not skipping empty lines\n";
}
?>
```

The above example will output something similar to:

```
Not skipping empty lines
Skipping empty lines
```

See Also

- SplFileObject::setFlags() - Sets flags for the SplFileObject

SplFileObject::getMaxLineLen

SplFileObject::getMaxLineLen

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::getMaxLineLen — Get maximum line length

Description

public function **SplFileObject::getMaxLineLen**(): int

Gets the maximum line length as set by SplFileObject::setMaxLineLen().

Parameters

This function has no parameters.

Return Values

Returns the maximum line length if one has been set with SplFileObject::setMaxLineLen(), default is `0`.

Examples

Example #1 SplFileObject::getMaxLineLen() example

```php
<?php
$file = new SplFileObject("file.txt");
var_dump($file->getMaxLineLen());

$file->setMaxLineLen(20);
```

```
var_dump($file->getMaxLineLen());
?>
```

The above example will output something similar to:

```
int(0)
int(20)
```

See Also

- SplFileObject::setMaxLineLen() - Set maximum line length

SplFileObject::hasChildren

SplFileObject::hasChildren

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplFileObject::hasChildren — SplFileObject does not have children

Description

public function **SplFileObject::hasChildren**(): false

An SplFileObject does not have children so this method always return `false`.

Parameters

This function has no parameters.

Return Values

Returns `false`

See Also

- RecursiveIterator::hasChildren() - Returns if an iterator can be created for the current entry

SplFileObject::key

SplFileObject::key

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::key — Get line number

Description

public function **SplFileObject::key**(): int

Gets the current line number.

> **Note** :
>
> This number may not reflect the actual line number in the file if SplFileObject::setMaxLineLen() is used to read fixed lengths of the file.

Parameters

This function has no parameters.

Return Values

Returns the current line number.

Examples

Example #1 SplFileObject::key() example

```php
<?php
$file = new SplFileObject("lipsum.txt");
foreach ($file as $line) {
    echo $file->key() . ". " . $line;
}
?>
```

The above example will output something similar to:

```
0. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
1. Duis nec sapien felis, ac sodales nisl.
2. Lorem ipsum dolor sit amet, consectetur adipiscing elit.
```

Example #2 SplFileObject::key() example with SplFileObject::setMaxLineLen()

```php
<?php
$file = new SplFileObject("lipsum.txt");
$file->setMaxLineLen(20);
foreach ($file as $line) {
    echo $file->key() . ". " . $line . "\n";
}
?>
```

The above example will output something similar to:

```
0. Lorem ipsum dolor s
1. it amet, consectetu
2. r adipiscing elit.
3.

4. Duis nec sapien fel
5. is, ac sodales nisl
6. .

7. Lorem ipsum dolor s
8. it amet, consectetu
9. r adipiscing elit.
```

See Also

- SplFileObject::current() - Retrieve current line of file
- SplFileObject::seek() - Seek to specified line
- SplFileObject::next() - Read next line
- SplFileObject::rewind() - Rewind the file to the first line
- SplFileObject::valid() - Not at EOF

SplFileObject::next

SplFileObject::next

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::next — Read next line

Description

public function **SplFileObject::next**(): void

Moves ahead to the next line in the file.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 SplFileObject::next() example

```php
<?php
// Read through file line by line
$file = new SplFileObject("misc.txt");
while (!$file->eof()) {
    echo $file->current();
    $file->next();
}
?>
```

See Also

- SplFileObject::current() - Retrieve current line of file
- SplFileObject::key() - Get line number
- SplFileObject::seek() - Seek to specified line
- SplFileObject::rewind() - Rewind the file to the first line
- SplFileObject::valid() - Not at EOF

SplFileObject::rewind

SplFileObject::rewind

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::rewind — Rewind the file to the first line

Description

public function **SplFileObject::rewind**(): void

Rewinds the file back to the first line.

Parameters

This function has no parameters.

Return Values

No value is returned.

Errors/Exceptions

Throws a RuntimeException if cannot be rewound.

Examples

Example #1 SplFileObject::rewind() example

```php
<?php
$file = new SplFileObject("misc.txt");

// Loop over whole file
foreach ($file as $line) { }
```

```php
// Rewind to first line
$file->rewind();

// Output first line
echo $file->current();
?>
```

See Also

- SplFileObject::current() - Retrieve current line of file
- SplFileObject::key() - Get line number
- SplFileObject::seek() - Seek to specified line
- SplFileObject::next() - Read next line
- SplFileObject::valid() - Not at EOF

SplFileObject::seek

SplFileObject::seek

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::seek — Seek to specified line

Description

public function **SplFileObject::seek**(int `$line`): void

Seek to specified line in the file.

Parameters

`line`

The zero-based line number to seek to.

Return Values

No value is returned.

Errors/Exceptions

Throws a LogicException if the `line` is negative.

Examples

Example #1 SplFileObject::seek() example

This example outputs the third line of the script which is found at position 2.

```php
<?php
$file = new SplFileObject(__FILE__);
$file->seek(2);
echo $file->current();
?>
```

The above example will output something similar to:

```
$file->seek(2);
```

See Also

- SplFileObject::current() - Retrieve current line of file
- SplFileObject::key() - Get line number
- SplFileObject::next() - Read next line

- SplFileObject::rewind() - Rewind the file to the first line
- SplFileObject::valid() - Not at EOF

SplFileObject::setCsvControl

SplFileObject::setCsvControl

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

SplFileObject::setCsvControl — Set the delimiter, enclosure and escape character for CSV

Description

public function **SplFileObject::setCsvControl**(string `$separator = ","`, string `$enclosure = "\""`, string `$escape = "\\"`): void

Sets the delimiter, enclosure and escape character for parsing CSV fields.

Parameters

`separator`

The `separator` parameter sets the field separator. It must be a single byte character.

`enclosure`

The `enclosure` parameter sets the field enclosure character. It must be a single byte character.

`escape`

The `escape` parameter sets the escape character. It must be a single byte character or the empty string. The empty string (`""`) disables the proprietary escape mechanism.

> **Warning**
>
> In the input stream, the `enclosure` character can always be escaped by doubling it inside a quoted string, resulting in a single `enclosure` character in the parsed result. The `escape` character works differently: If a sequence of `escape` and `enclosure` characters appear in the input, both characters will be present in the parsed result. So for the default parameters, a CSV line like `"a""b","c\"d"` will have the fields parsed as `a"b` and `c\"d`, respectively.

> **Warning**
>
> As of PHP 8.4.0, depending on the default value of `escape` is deprecated. It needs to be provided explicitly either positionally or by the use of named arguments.

> **Warning**
>
> When `escape` is set to anything other than an empty string (`""`) it can result in CSV that is not compliant with » RFC 4180 or unable to survive a roundtrip through the PHP CSV functions. The default for `escape` is `"\\"` so it is recommended to set it to the empty string explicitly. The default value will change in a future version of PHP, no earlier than PHP 9.0.

Return Values

No value is returned.

Errors/Exceptions

Throws a ValueError if `separator` or `enclosure` is not one byte long.

Throws a ValueError if `escape` is not one byte long or the empty string.

Changelog

Version	Description
8.4.0	Relying on the default value of `escape` is now deprecated.
7.4.0	The `escape` parameter now also accepts an empty string to disable the proprietary escape mechanism.

Examples

Example #1 SplFileObject::setCsvControl() example

```php
<?php
$file = new SplFileObject("data.csv");
$file->setFlags(SplFileObject::READ_CSV);
$file->setCsvControl('|');
foreach ($file as $row) {
    list ($fruit, $quantity) = $row;
    // Do something with values
}
?>
```

Contents of data.csv

```
<?php
apples|20
bananas|14
cherries|87
?>
```

See Also

- SplFileObject::getCsvControl() - Get the delimiter, enclosure and escape character for CSV
- SplFileObject::fgetcsv() - Gets line from file and parse as CSV fields
- SplFileObject::fputcsv() - Write a field array as a CSV line
- fputcsv() - Format line as CSV and write to file pointer
- fgetcsv() - Gets line from file pointer and parse for CSV fields
- str_getcsv() - Parse a CSV string into an array

SplFileObject::setFlags

SplFileObject::setFlags

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::setFlags — Sets flags for the SplFileObject

Description

public function **SplFileObject::setFlags**(int `$flags`): void

Sets the flags to be used by the SplFileObject.

Parameters

`flags`

Bit mask of the flags to set. See SplFileObject constants for the available flags.

Return Values

No value is returned.

Examples

Example #1 SplFileObject::setFlags() example

```php
<?php
$file = new SplFileObject("data.csv");
$file->setFlags(SplFileObject::READ_CSV);
foreach ($file as $fields) {
    var_dump($fields);
}
?>
```

See Also

- SplFileObject::getFlags() - Gets flags for the SplFileObject

SplFileObject::setMaxLineLen

SplFileObject::setMaxLineLen

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::setMaxLineLen — Set maximum line length

Description

public function **SplFileObject::setMaxLineLen**(int `$maxLength`): void

Sets the maximum length of a line to be read.

Parameters

`maxLength`

The maximum length of a line.

Return Values

No value is returned.

Errors/Exceptions

Throws DomainException when `maxLength` is less than zero.

Examples

Example #1 SplFileObject::setMaxLineLen() example

```php
<?php
$file = new SplFileObject("lipsum.txt");
$file->setMaxLineLen(20);
foreach ($file as $line) {
    echo $line . "\n";
}
?>
```

Contents of lipsum.txt

```
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Duis nec sapien felis, ac sodales nisl.
Nulla vitae magna vitae purus aliquet consequat.
```

The above example will output something similar to:

```
Lorem ipsum dolor s
it amet, consectetu
r adipiscing elit.

Duis nec sapien fel
is, ac sodales nisl
.

Nulla vitae magna v
itae purus aliquet
consequat.
```

See Also

- SplFileObject::getMaxLineLen() - Get maximum line length

SplFileObject::__toString

SplFileObject::__toString

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::__toString — Returns the current line as a string

Description

public function **SplFileObject::__toString**(): string

This method will return the current line as a string.

Parameters

This function has no parameters.

Return Values

Returns the current line as a string.

Changelog

Version	Description
8.1.14, 8.2.1	Changed from an alias of SplFileObject::fgets() to an implementation of SplFileObject::current() which returns a CSV string when the `SplFileObject::READ_CSV` flag is set.
7.2.19, 7.3.6	Changed from an alias of SplFileObject::current() to an alias of SplFileObject::fgets().

SplFileObject::valid

SplFileObject::valid

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

SplFileObject::valid — Not at EOF

Description

public function **SplFileObject::valid**(): bool

Check whether EOF has been reached.

Parameters

This function has no parameters.

Return Values

Returns `true` if not reached EOF, `false` otherwise.

Examples

Example #1 SplFileObject::valid() example

```php
<?php
// Loop over a file, line by line
$file = new SplFileObject("file.txt");
while ($file->valid()) {
    echo $file->fgets();
}
?>
```

See Also

- SplFileObject::current() - Retrieve current line of file
- SplFileObject::key() - Get line number
- SplFileObject::seek() - Seek to specified line

- SplFileObject::next() - Read next line
- SplFileObject::rewind() - Rewind the file to the first line

SplTempFileObject

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

Introduction

The SplTempFileObject class offers an object-oriented interface for a temporary file.

Class synopsis

```
class SplTempFileObject extends SplFileObject {
/* Inherited constants */
public const int SplFileObject::DROP_NEW_LINE;
public const int SplFileObject::READ_AHEAD;
public const int SplFileObject::SKIP_EMPTY;
public const int SplFileObject::READ_CSV;
/* Methods */
public function __construct(int $maxMemory = 2 * 1024 * 1024)
/* Inherited methods */
public function SplFileObject::current(): string|array|false
public function SplFileObject::eof(): bool
public function SplFileObject::fflush(): bool
public function SplFileObject::fgetc(): string|false
public function SplFileObject::fgetcsv(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): array|false
public function SplFileObject::fgets(): string
public function SplFileObject::fgetss(string $allowable_tags = ?): string
public function SplFileObject::flock(int $operation, int &$wouldBlock = null): bool
public function SplFileObject::fpassthru(): int
public function SplFileObject::fputcsv(
    array $fields,
    string $separator = ",",
    string $enclosure = "\"",
    string $escape = "\\",
    string $eol = "\n"
): int|false
public function SplFileObject::fread(int $length): string|false
public function SplFileObject::fscanf(string $format, mixed &...$vars): array|int|null
public function SplFileObject::fseek(int $offset, int $whence = SEEK_SET): int
public function SplFileObject::fstat(): array
public function SplFileObject::ftell(): int|false
public function SplFileObject::ftruncate(int $size): bool
public function SplFileObject::fwrite(string $data, ?int $length = null): int|false
public function SplFileObject::getChildren(): null
public function SplFileObject::getCsvControl(): array
public function SplFileObject::getFlags(): int
public function SplFileObject::getMaxLineLen(): int
public function SplFileObject::hasChildren(): false
public function SplFileObject::key(): int
public function SplFileObject::next(): void
public function SplFileObject::rewind(): void
public function SplFileObject::seek(int $line): void
public function SplFileObject::setCsvControl(string $separator = ",", string $enclosure = "\"", string $escape = "\\"): void
public function SplFileObject::setFlags(int $flags): void
public function SplFileObject::setMaxLineLen(int $maxLength): void
public function SplFileObject::__toString(): string
public function SplFileObject::valid(): bool
public function SplFileInfo::getATime(): int|false
public function SplFileInfo::getBasename(string $suffix = ""): string
public function SplFileInfo::getCTime(): int|false
public function SplFileInfo::getExtension(): string
```

public function SplFileInfo::getFileInfo(?string $class = null): SplFileInfo
public function SplFileInfo::getFilename(): string
public function SplFileInfo::getGroup(): int|false
public function SplFileInfo::getInode(): int|false
public function SplFileInfo::getLinkTarget(): string|false
public function SplFileInfo::getMTime(): int|false
public function SplFileInfo::getOwner(): int|false
public function SplFileInfo::getPath(): string
public function SplFileInfo::getPathInfo(?string $class = null): ?SplFileInfo
public function SplFileInfo::getPathname(): string
public function SplFileInfo::getPerms(): int|false
public function SplFileInfo::getRealPath(): string|false
public function SplFileInfo::getSize(): int|false
public function SplFileInfo::getType(): string|false
public function SplFileInfo::isDir(): bool
public function SplFileInfo::isExecutable(): bool
public function SplFileInfo::isFile(): bool
public function SplFileInfo::isLink(): bool
public function SplFileInfo::isReadable(): bool
public function SplFileInfo::isWritable(): bool
public function SplFileInfo::openFile(string $mode = "r", bool $useIncludePath = false, ?resource $context = null): SplFileObject
public function SplFileInfo::setFileClass(string $class = SplFileObject::class): void
public function SplFileInfo::setInfoClass(string $class = SplFileInfo::class): void
public function SplFileInfo::__toString(): string
}

Table of Contents

SplTempFileObject::__construct

SplTempFileObject::__construct

(PHP 5 >= 5.1.2, PHP 7, PHP 8)

SplTempFileObject::__construct — Construct a new temporary file object

Description

public function **SplTempFileObject::__construct**(int $maxMemory = 2 * 1024 * 1024)

Construct a new temporary file object.

Parameters

`maxMemory`

> The maximum amount of memory (in bytes, default is 2 MB) for the temporary file to use. If the temporary file exceeds this size, it will be moved to a file in the system's temp directory.
>
> If `maxMemory` is negative, only memory will be used. If `maxMemory` is zero, no memory will be used.

Errors/Exceptions

Throws a RuntimeException if an error occurs.

Examples

Example #1 SplTempFileObject() example

This example writes a temporary file in memory which can be written to and read from.

```php
<?php
$temp = new SplTempFileObject();
$temp->fwrite("This is the first line\n");
$temp->fwrite("And this is the second.\n");
echo "Written " . $temp->ftell() . " bytes to temporary file.\n\n";

// Rewind and read what was written
```

```php
$temp->rewind();
foreach ($temp as $line) {
    echo $line;
}
?>
```

The above example will output something similar to:

```
Written 47 bytes to temporary file.

This is the first line
And this is the second.
```

See Also

- SplFileObject
- PHP input/output streams (for `php://temp` and `php://memory`)

SPL Functions

Table of Contents

class_implements

class_implements

(PHP 5, PHP 7, PHP 8)

class_implements — Return the interfaces which are implemented by the given class or interface

Description

function **class_implements**(object|string `$object_or_class`, bool `$autoload` = **`true`**): array|false

This function returns an array with the names of the interfaces that the given `object_or_class` and its parents implement.

Parameters

`object_or_class`

An object (class instance) or a string (class or interface name).

`autoload`

Whether to autoload if not already loaded.

Return Values

An array on success, or **`false`** when the given class doesn't exist.

Examples

Example #1 class_implements() example

```php
<?php

interface foo { }
class bar implements foo {}

print_r(class_implements(new bar));

// you may also specify the parameter as a string
print_r(class_implements('bar'));
```

```php
spl_autoload_register();

// use autoloading to load the 'not_loaded' class
print_r(class_implements('not_loaded', true));

?>
```

The above example will output something similar to:

```
Array
(
    [foo] => foo
)
Array
(
    [foo] => foo
)
Array
(
    [interface_of_not_loaded] => interface_of_not_loaded
)
```

Notes

> **Note** :
>
> To check that an object implements an interface, `instanceof` or the is_a() function should be used instead.

See Also

- class_parents() - Return the parent classes of the given class
- get_declared_interfaces() - Returns an array of all declared interfaces
- is_a() - Checks whether the object is of a given type or subtype
- `instanceof`

class_parents

class_parents

(PHP 5, PHP 7, PHP 8)

class_parents — Return the parent classes of the given class

Description

function **class_parents**(object|string `$object_or_class`, bool `$autoload = true`): array|false

This function returns an array with the name of the parent classes of the given `object_or_class`.

Parameters

`object_or_class`

An object (class instance) or a string (class name).

`autoload`

Whether to autoload if not already loaded.

Return Values

An array on success, or `false` when the given class doesn't exist.

Examples

Example #1 class_parents() example

```php
<?php

class foo { }
class bar extends foo {}

print_r(class_parents(new bar));

// you may also specify the parameter as a string
print_r(class_parents('bar'));

spl_autoload_register();

// use autoloading to load the 'not_loaded' class
print_r(class_parents('not_loaded', true));

?>
```

The above example will output something similar to:

```
Array
(
    [foo] => foo
)
Array
(
    [foo] => foo
)
Array
(
    [parent_of_not_loaded] => parent_of_not_loaded
)
```

Notes

> **Note** :
>
> To check that an object extends a class, `instanceof` or the is_a() function should be used instead.

See Also

- class_implements() - Return the interfaces which are implemented by the given class or interface
- is_a() - Checks whether the object is of a given type or subtype
- `instanceof`

class_uses

class_uses

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

class_uses — Return the traits used by the given class

Description

function **class_uses**(object|string `$object_or_class`, bool `$autoload` = **`true`**): array|false

This function returns an array with the names of the traits that the given `object_or_class` uses. This does however not include any traits used by a parent class.

Parameters

`object_or_class`

An object (class instance) or a string (class name).

`autoload`

Whether to autoload if not already loaded.

Return Values

An array on success, or `false` when the given class doesn't exist.

Examples

Example #1 class_uses() example

```php
<?php

trait foo { }
class bar {
  use foo;
}

print_r(class_uses(new bar));

print_r(class_uses('bar'));

spl_autoload_register();

// use autoloading to load the 'not_loaded' class
print_r(class_uses('not_loaded', true));

?>
```

The above example will output something similar to:

```
Array
(
    [foo] => foo
)
Array
(
    [foo] => foo
)
Array
(
    [trait_of_not_loaded] => trait_of_not_loaded
)
```

See Also

- class_parents() - Return the parent classes of the given class
- get_declared_traits() - Returns an array of all declared traits

iterator_apply

iterator_apply

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

iterator_apply — Call a function for every element in an iterator

Description

function **iterator_apply**(Traversable `$iterator`, callable `$callback`, ?array `$args` = `null`): int

Calls a function for every element in an iterator.

Parameters

`iterator`

The iterator object to iterate over.

`callback`

The callback function to call on every element. This function only receives the given `args`, so it is nullary by default. If `count($args) === 3`, for instance, the callback function is ternary.

> **Note** :
>
> The function must return `true` in order to continue iterating over the `iterator`.

`args`

An array of arguments; each element of `args` is passed to the callback `callback` as separate argument.

Return Values

Returns the iteration count.

Examples

Example #1 iterator_apply() example

```php
<?php
function print_caps(Iterator $iterator) {
    echo strtoupper($iterator->current()) . "\n";
    return TRUE;
}

$it = new ArrayIterator(array("Apples", "Bananas", "Cherries"));
iterator_apply($it, "print_caps", array($it));
?>
```

The above example will output:

```
APPLES
BANANAS
CHERRIES
```

See Also

- array_walk() - Apply a user supplied function to every member of an array

iterator_count

iterator_count

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

iterator_count — Count the elements in an iterator

Description

function **iterator_count**(Traversable|array `$iterator`): int

Count the elements in an iterator. **iterator_count()** is not guaranteed to retain the current position of the `iterator`.

Parameters

`iterator`

The iterator being counted.

Return Values

The number of elements in `iterator`.

Changelog

Version	Description
8.2.0	The type of `iterator` has been widened from Traversable to Traversable|array.

Examples

Example #1 iterator_count() example

```php
<?php
$iterator = new ArrayIterator(array('recipe'=>'pancakes', 'egg', 'milk', 'flour'));
var_dump(iterator_count($iterator));
?>
```

The above example will output:

```
int(4)
```

Example #2 iterator_count() modifies position

```php
<?php
$iterator = new ArrayIterator(['one', 'two', 'three']);
var_dump($iterator->current());
var_dump(iterator_count($iterator));
var_dump($iterator->current());
?>
```

The above example will output:

```
string(3) "one"
int(3)
NULL
```

Example #3 iterator_count() in `foreach` loops

```php
<?php
$iterator = new ArrayIterator(['one', 'two', 'three']);
foreach ($iterator as $key => $value) {
    echo "$key: $value (", iterator_count($iterator), ")\n";
}?>
```

The above example will output:

```
0: one (3)
```

iterator_to_array

iterator_to_array

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

iterator_to_array — Copy the iterator into an array

Description

function **iterator_to_array**(Traversable|array `$iterator`, bool `$preserve_keys = true`): array

Copy the elements of an iterator into an array.

Parameters

`iterator`

The iterator being copied.

`preserve_keys`

Whether to use the iterator element keys as index.

If a key is an array or object, a warning will be generated. `null` keys will be converted to an empty string, float keys will be truncated to their int counterpart, resource keys will generate a warning and be converted to their resource ID, and bool keys will be converted to integers.

> **Note** :
>
> If this parameter is not set or set to `true`, duplicate keys will be overwritten. The last value with a given key will be in the returned array. Set this parameter to `false` to get all the values in any case.

Return Values

An array containing the elements of the `iterator`.

Changelog

Version	Description
8.2.0	The type of `iterator` has been widened from Traversable to Traversable|array.

Examples

Example #1 iterator_to_array() example

```php
<?php
$iterator = new ArrayIterator(array('recipe'=>'pancakes', 'egg', 'milk', 'flour'));
var_dump(iterator_to_array($iterator, true));
var_dump(iterator_to_array($iterator, false));
?>
```

The above example will output:

```
array(4) {
  ["recipe"]=>
  string(8) "pancakes"
  [0]=>
  string(3) "egg"
  [1]=>
```

```
  string(4) "milk"
  [2]=>
  string(5) "flour"
}
array(4) {
  [0]=>
  string(8) "pancakes"
  [1]=>
  string(3) "egg"
  [2]=>
  string(4) "milk"
  [3]=>
  string(5) "flour"
}
```

spl_autoload

spl_autoload

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload — Default implementation for __autoload()

Description

function **spl_autoload**(string `$class`, ?string `$file_extensions` = **`null`**): void

This function is intended to be used as a default implementation for __autoload(). If nothing else is specified and spl_autoload_register() is called without any parameters then **spl_autoload()** will be used for any later call to __autoload().

Parameters

`class`

> The name of the class being instantiated. When calling the function, the name of the class with the namespace is passed to the parameter. The `class` will not contain the leading backslash of a fully-qualified identifier.

`file_extensions`

> By default it checks all include_paths to contain filenames built up by the lowercase class name appended by the filename extensions `.inc` and `.php`.

Return Values

No value is returned.

Errors/Exceptions

Throws LogicException when the class is not found and there are no other autoloaders registered.

Changelog

Version	Description
8.0.0	`file_extensions` is now nullable.

spl_autoload_call

spl_autoload_call

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_call — Try all registered __autoload() functions to load the requested class

Description

function **spl_autoload_call**(string `$class`): void

This function can be used to manually search for a class, interface, trait, or enumeration using the registered __autoload functions.

Parameters

`class`

The class name being searched.

Return Values

No value is returned.

spl_autoload_extensions

spl_autoload_extensions

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_extensions — Register and return default file extensions for spl_autoload

Description

function **spl_autoload_extensions**(?string `$file_extensions = null`): string

This function can modify and check the file extensions that the built in __autoload() fallback function spl_autoload() will be using.

> **Note** :
>
> There should not be a space between the defined file extensions.

Parameters

`file_extensions`

If `null`, it simply returns the current list of extensions each separated by comma. To modify the list of file extensions, simply invoke the functions with the new list of file extensions to use in a single string with each extensions separated by comma.

Return Values

A comma delimited list of default file extensions for spl_autoload().

Changelog

Version	Description
8.0.0	`file_extensions` is now nullable.

Examples

Example #1 spl_autoload_extensions() example

```php
<?php
spl_autoload_extensions(".php,.inc");
?>
```

spl_autoload_functions

spl_autoload_functions

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_functions — Return all registered __autoload() functions

Description

function **spl_autoload_functions**(): array

Get all registered __autoload() functions.

Parameters

This function has no parameters.

Return Values

An array of all registered __autoload functions. If no function is registered, or the autoload queue is not activated, then the return value will be an empty array.

Changelog

Version	Description
8.0.0	Return value was updated to always be an array; previously this function returned `false` if the autoload queue wasn't activated.

spl_autoload_register

spl_autoload_register

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_register — Register given function as __autoload() implementation

Description

function **spl_autoload_register**(?callable `$callback = null`, bool `$throw = true`, bool `$prepend = false`): bool

Register a function with the spl provided __autoload queue. If the queue is not yet activated it will be activated.

If your code has an existing __autoload() function then this function must be explicitly registered on the __autoload queue. This is because **spl_autoload_register()** will effectively replace the engine cache for the __autoload() function by either spl_autoload() or spl_autoload_call().

If there must be multiple autoload functions, **spl_autoload_register()** allows for this. It effectively creates a queue of autoload functions, and runs through each of them in the order they are defined. By contrast, __autoload() may only be defined once.

Parameters

`callback`

The autoload function being registered. If `null`, then the default implementation of spl_autoload() will be registered.

function callback(string `$class`): void

The `class` will not contain the leading backslash of a fully-qualified identifier.

`throw`

This parameter specifies whether **spl_autoload_register()** should throw exceptions when the `callback` cannot be registered.

> **Warning**
>
> This parameter is ignored as of PHP 8.0.0, and a notice will be emitted if set to `false`. **spl_autoload_register()** will now always throw a TypeError on invalid arguments.

`prepend`

If `true`, **spl_autoload_register()** will prepend the autoloader on the autoload queue instead of appending it.

Return Values

Returns `true` on success or `false` on failure.

Changelog

Version	Description
8.0.0	`callback` is now nullable.

Examples

Example #1 spl_autoload_register() as a replacement for an __autoload() function

```php
<?php

// function __autoload($class) {
//     include 'classes/' . $class . '.class.php';
// }

function my_autoloader($class) {
    include 'classes/' . $class . '.class.php';
}

spl_autoload_register('my_autoloader');

// Or, using an anonymous function
spl_autoload_register(function ($class) {
    include 'classes/' . $class . '.class.php';
});

?>
```

Example #2 spl_autoload_register() example where the class is not loaded

```php
<?php

namespace Foobar;

class Foo {
    static public function test($class) {
        print '[['. $class .']]';
    }
}

spl_autoload_register(__NAMESPACE__ .'\Foo::test');

new InexistentClass;

?>
```

The above example will output something similar to:

```
[[Foobar\InexistentClass]]
Fatal error: Class 'Foobar\InexistentClass' not found in ...
```

Example #3 The identifier will be provided without the leading backslash

```php
<?php

spl_autoload_register(static function ($class) {
    var_dump($class);
});

class_exists('RelativeName');
class_exists('RelativeName\\WithNamespace');
class_exists('\\AbsoluteName');
class_exists('\\AbsoluteName\\WithNamespace');

?>
```

The above example will output:

```
string(12) "RelativeName"
string(26) "RelativeName\WithNamespace"
string(12) "AbsoluteName"
string(26) "AbsoluteName\WithNamespace"
```

See Also

- __autoload() - Attempt to load undefined class

spl_autoload_unregister

spl_autoload_unregister

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

spl_autoload_unregister — Unregister given function as __autoload() implementation

Description

function **spl_autoload_unregister**(callable `$callback`): bool

Removes a function from the autoload queue. If the queue is activated and empty after removing the given function then it will be deactivated.

When this function results in the queue being deactivated, any __autoload function that previously existed will not be reactivated.

Parameters

`callback`

The autoload function being unregistered.

Return Values

Returns **`true`** on success or **`false`** on failure.

Changelog

Version	Description
8.5.0	Passing the spl_autoload_call() function as a callback argument to unregister all autoloaders has been deprecated. Instead iterate over the return value of spl_autoload_functions() and call **spl_autoload_unregister()** on each value.

spl_classes

spl_classes

(PHP 5, PHP 7, PHP 8)

spl_classes — Return available SPL classes

Description

function **spl_classes**(): array

This function returns an array with the current available SPL classes.

Parameters

This function has no parameters.

Return Values

Returns an array containing the currently available SPL classes.

Examples

Example #1 spl_classes() example

```php
<?php

print_r(spl_classes());

?>
```

The above example will output something similar to:

```
Array
(
    [ArrayObject] => ArrayObject
    [ArrayIterator] => ArrayIterator
    [CachingIterator] => CachingIterator
    [RecursiveCachingIterator] => RecursiveCachingIterator
    [DirectoryIterator] => DirectoryIterator
    [FilterIterator] => FilterIterator
    [LimitIterator] => LimitIterator
    [ParentIterator] => ParentIterator
    [RecursiveDirectoryIterator] => RecursiveDirectoryIterator
    [RecursiveIterator] => RecursiveIterator
    [RecursiveIteratorIterator] => RecursiveIteratorIterator
    [SeekableIterator] => SeekableIterator
    [SimpleXMLIterator] => SimpleXMLIterator
)
```

spl_object_hash

spl_object_hash

(PHP 5 >= 5.2.0, PHP 7, PHP 8)

spl_object_hash — Return hash id for given object

Description

function **spl_object_hash**(object `$object`): string

This function returns a unique identifier for the object. This id can be used as a hash key for storing objects, or for identifying an object, as long as the object is not destroyed. Once the object is destroyed, its hash may be reused for other objects. This behavior is similar to spl_object_id().

Parameters

`object`

Any object.

Return Values

A string that is unique for each currently existing object and is always the same for each object.

Examples

Example #1 A spl_object_hash() example

```php
<?php
$id = spl_object_hash($object);
$storage[$id] = $object;
?>
```

Notes

> **Note** :

> When an object is destroyed, its hash may be reused for other objects.

> **Note** :
>
> Object hashes should be compared for identity with `===` and `!==`, because the returned hash could be a numeric string. For example: `0000000000000e60000000000000000`.

See Also

- spl_object_id() - Return the integer object handle for given object

spl_object_id

spl_object_id

(PHP 7 >= 7.2.0, PHP 8)

spl_object_id — Return the integer object handle for given object

Description

function **spl_object_id**(object `$object`): int

This function returns a unique identifier for the object. The object id is unique for the lifetime of the object. Once the object is destroyed, its id may be reused for other objects. This behavior is similar to spl_object_hash().

Parameters

`object`

Any object.

Return Values

An integer identifier that is unique for each currently existing object and is always the same for each object.

Examples

Example #1 A spl_object_id() example

```php
<?php
$id = spl_object_id($object);
$storage[$id] = $object;
?>
```

Notes

> **Note** :
>
> When an object is destroyed, its id may be reused for other objects.